# The pdp64 Architecture Manual

*A 64-bit extension of the PDP-11*

Digital Equity Collective's Pragmatic Digital Processor, 64 bit

David Walther

Copyright © 2026 David Walther. Released under the BSD 2-Clause licence (the file `COPYING` of the repository).
The source of this book is https://github.com/moebiusV/pdp64 (`docs/manual/`); the instruction dictionary is generated from the machine-readable specification. Typeset with Typst in EB Garamond and DejaVu Sans Mono.

# CONTENTS

**About this manual** ..... **ii**
**Part I — The architecture** ..... **1**
Chapter 1 — Introduction ..... 2
Chapter 2 — The bus ..... 12
Chapter 3 — Graphics: the terminal and its devices ..... 30
Chapter 4 — Boot, the BIOS, the disk label and the GPT ..... 48
Chapter 5 — The device driver model ..... 63
Chapter 6 — The kernel: modes, exceptions and memory management ..... 77
Chapter 7 — Multiple processors ..... 97
Chapter 8 — Vector work, AI and array languages: the device route ..... 112
Chapter 9 — The instruction set ..... 120
Chapter 10 — Conclusion: from FPGA to silicon ..... 481
**Part II — The instruction reference tables** ..... **486**
The instruction reference tables ..... 487
Assembler aliases and old spellings ..... 488
The architectural constants ..... 489
The mnemonics, alphabetically ..... 492
The bit layouts of every opcode ..... 499
**Part III — Appendices** ..... **579**
Appendix A — Improvements beyond the PDP-11 ..... 580
Appendix B — Decisions pending ..... 583
Appendix C — Glossary ..... 584
Appendix D — The ABI, version 1 ..... 588
Appendix E — The assembly language and the object format ..... 590
Appendix F — The evidence behind the manual ..... 592
Appendix G — The evolution of the design ..... 595
Appendix H — Proposed ..... 604
Appendix I — Deferred ..... 637
Appendix J — Open items ..... 640

# About this manual

**Each section is a few numbered aphorisms followed by a commentary.** The aphorisms say what is true of the machine, in plain short statements, and read on their own as the main text. The commentary under them gives, as they apply: the *Precedent* (what earlier machines did), the *Trade-offs* (what the choice costs), the *Improvements* (how the pdp64 departs from the precedent), the *Benefits*, the *Modern comparables* (what current architectures do), and, for an instruction that is not obvious, its *Use*. Every decision also carries an *Impact* line saying what it means for an efficient **kernel**, for efficient **compilation** and for an efficient **hardware layout**: the three things a processor designer asks of a choice. An instruction as plain as `MOV` or `ADD` needs no statement of use. The reference entries follow the layout of the classic CPU manuals, in which every instruction has its encoding diagram, its operation and its effects.

**Each aphorism may carry a status.** *(D)* means decided by the author. An aphorism with no tag states the architecture as it is to be. The chapters hold only what is decided; where a decision's details are not yet worked out, the aphorism ends with *Details pending* and the number of the open item that discusses them, or with *Details to be determined*. What is only proposed is in appendix H, what has been put off in appendix I, and each question the architecture has not answered in appendix J; a chapter mentions an open item in a few words and points to its entry there.

**Numbers.** Where a figure is measured, the text says what was measured; where it is an estimate, it says so. Claims about other machines are checked against the fact-check record listed in appendix F.

**The PDF.** The same text as a book, `book/pdp64.pdf`, built by buildabook and Typst from the sources of this directory (`tools/mkbook.sh`; see `book/README.md`).

**Evidence documents.** All of them are listed in appendix F: the full instruction set text (`pdp64-isa.md`), the record of decisions (`errata.md`) and the open decisions with five options each (`decisions-pending.md`).

# Part I — The architecture

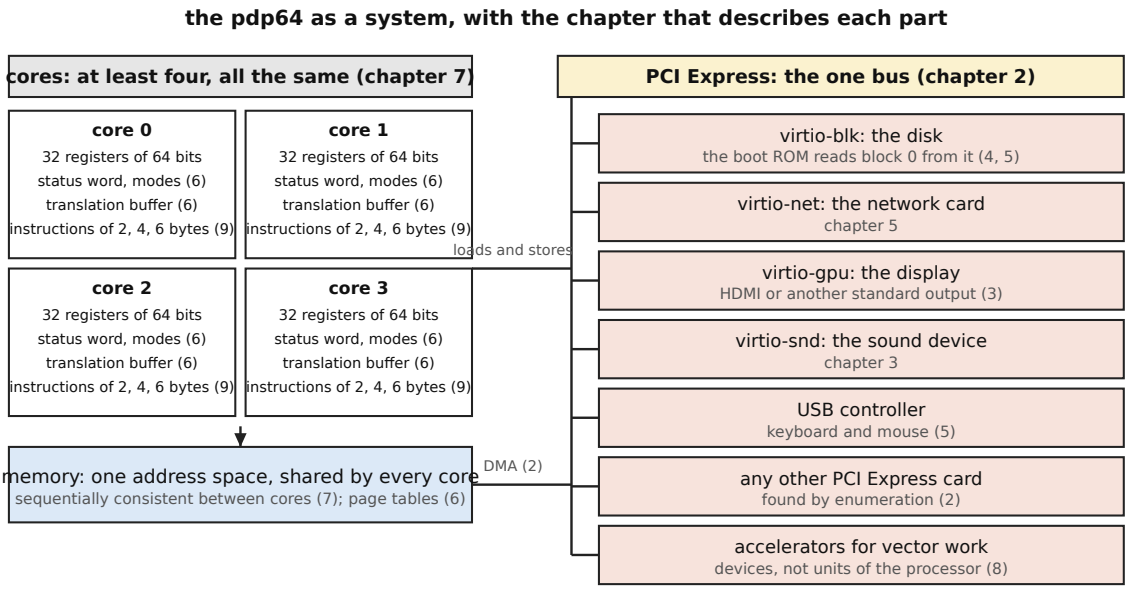
# Chapter 1 — Introduction

**The pdp64 is a 64-bit computer architecture in the PDP-11 line: one CPU, with thirty-two 64-bit registers and instructions of two, four or six bytes, that reaches everything outside itself over PCI Express, where the standard disk, network card, display and sound device are virtio devices (one standard interface, with one driver routine for every kind of device).** It keeps what made the PDP-11 pleasant to program: devices that are memory locations, a stack pointer that is an ordinary register, autoincrement addressing, a small set of regular rules. It changes what fifty years of machines and compilers showed to be a mistake or a missing piece. The manual is written for a first-year computer-science student or an experienced PDP-11 assembly programmer, and assumes that neither knows anything that happened in computer architecture after the 1970s (errata 103).

This chapter has three jobs. It says what the machine is and how to read the manual (1.1, 1.2). It puts the road from the PDP-11 to the pdp64 in one table, with the author's own account of where the design began (1.3). And it states the principles the design follows and the measurements that back them (1.4 to 1.8). After 1.3 the chapters can be read in any order: each starts from what a PDP-11 programmer knows of its subject.

## 1.1 What the pdp64 is

If you have programmed a PDP-11 you know a machine with eight 16-bit registers (R6 the stack pointer, R7 the program counter), a 64 KB address, devices at addresses near the top of the Unibus address space, and one processor. The pdp64 is that machine grown up in each respect, and the figure shows what surrounds the processor.




A device is a few memory locations that ordinary loads and stores reach (2.3): the PDP-11 had no input or output instructions either.
Every standard device is a virtio device or a USB controller (5.3); the driver finds it by identifier, not by a switch setting (2.4).
Starting: a ROM reads block 0 and jumps to it (4); the kernel then runs the machine (6); the instruction set is chapter 9.

A box is a part and a line joins two parts that talk; this is the architecture as it is to be decided (section 1.2).

**1.1.1** The pdp64 is a little-endian machine with thirty-two 64-bit general registers, `R0` to `R31`; `R14` is the stack pointer and the program counter is not a register. An instruction is two, four or six bytes long (ten for the one instruction that carries a 64-bit constant), and its first 16 bits fix what it is and how long it is (chapter 9). *(D)*

**1.1.2** A program uses virtual addresses of 39, 48 or 57 bits, according to a page table of three, four or five levels, and the number is discoverable at run time; a physical address is 40 to 64 bits wide (6.6). The PDP-11 had 16 bits of virtual address and 18 or 22 of physical. *(D)*

**1.1.3** There is one CPU architecture and one manual. Two earlier designs were built and measured against it and are history (1.8). *(D)*

**1.1.4** The machine has several cores from the start: at least four, with sixteen or more as the ceiling that no decision may preclude; all cores see one memory, which is sequentially consistent (chapter 7). *(D)*

**1.1.5** The processor has no input or output instructions: a device is a few memory locations that ordinary loads and stores reach, as on the PDP-11. The bus is PCI Express, so an ordinary PCI Express card works on it unchanged; the standard devices are virtio devices (the disk is virtio-blk) and USB controllers, and the kernel finds them by enumeration, not by switch settings (chapters 2 and 5). *(D)*

*Commentary.* **Chain.** The PDP-11 had 16-bit words and no step beyond. The VAX-11 widened it to 32 bits (1.4). The author's choice, on 2026-08-30, was to skip the 32-bit step: "let's skip 32 bit version and go straight to 64bit" (R29) and "basically upgrade pdp-11 straight to 64 bit" (R30). **Trade-off.** Eight 16-bit registers become thirty-two 64-bit ones, so a register number needs five bits instead of three, and the instruction encoding changes with it (1.8). **Impact.** *Kernel:* the status word, vectors and modes stay recognisable (chapter 6). *Compiler:* 64-bit integers and pointers are one register each. *Hardware:* one register file, one ALU width.

## 1.2 Who the manual is for, and how to read it

You know the PDP-11, or you are learning computing from the start. Either way the manual assumes you have met nothing invented after the 1970s: not the PCI bus, the tree of page tables, the out-of-order core, the GPU or virtio.

**1.2.1** The manual is written for two readers: a first-year computer-science student and an experienced PDP-11 assembly programmer. Neither is assumed to know anything of computer architecture that came after the 1970s, so every later idea is explained where it first appears. *(D)* (errata 103)

**1.2.2** A term is defined where it first appears, as the technical word followed by a plain restatement: a prefix code (a set of bit patterns none of which begins another, so the first bits of an instruction tell the decoder what it is). In the figures a box is a piece of memory or a register and an arrow is an address stored in it, as a textbook draws a linked list. Code blocks are pseudocode (C-like, with `#` starting a comment) unless the text says they are PDP-11 or pdp64 assembly.

**1.2.3** The PDP-11 material is quoted from the Unix Version 6 and 7 and 2.11BSD sources and from the PDP-11/70 handbook; the pdp64 listings are illustrations written for the manual and are not part of the architecture. A fact about another machine that could not be checked against a source on hand is marked *unverified*.

**1.2.4** A chapter that has a history starts from the PDP-11 way, names the machines that came between with the one thing each added, and ends at the pdp64 way with the reason; these are the paragraphs headed **Chain**. The commentary under a decision also gives the **Trade-off** it makes and its **Impact** on an efficient kernel, on efficient compilation and on an efficient hardware layout (errata 58).

**1.2.5** A numbered statement such as 9.3.1 is a rule of the architecture. *(D)* means the author decided it; a statement with no tag describes the architecture as it is to be. *(E)* marks an estimate, not a measurement, and *(measured)* a figure that was measured, with its source.

**1.2.6** The body of the manual holds decided material only. Where a decision's details are not yet worked out, the statement ends with *Details pending* and the number of the open item, or with *Details to be determined*. A question the architecture must still answer is named in a few words, as *Open:…*, with a pointer to its entry in appendix J. What has only been proposed is in appendix H, what has been put off in appendix I, and each open question, with its options and what each costs, in appendix J.

The chapters, and what a PDP-11 programmer brings to each:

| Chapter | What it covers | The PDP-11 starting point |
|---|---|---|
| 2 The bus | PCI Express, virtio, interrupts that are writes, the IOMMU | the Unibus and its registers |
| 3 Graphics | the terminal, HDMI, audio, windows | the teletype, then the Blit and the Commodore 64 |
| 4 Boot | the first block, the second stage, the other cores | the front panel and the boot ROM |
| 5 Drivers | the virtio transport, class drivers, interrupts and DMA | `rk.c` and the `bdevsw` table |

| Chapter | What it covers | The PDP-11 starting point |
|---|---|---|
| 6 The kernel | modes, exceptions, page tables, system calls | the V7 kernel: `trap()`, the KT11, `mfpi` |
| 7 Multiple processors | memory model, atomics, per-core state, translation coherence | one processor and `spl()` |
| 8 Vector work | a standard vector and accelerator engine on the bus, a standard part like the FPU and the MMU, for neural networks and array languages | no vector unit; the FP11 for floating point |
| 9 The instruction set | formats, modes, every instruction | the PDP-11 instruction set |
| 10 Conclusion | the finish line, the security aims, what remains | |

The instruction reference tables, with a chart of every instruction's bit layout, sit beside chapter 9. The appendices are: A, improvements the author approved and has yet to specify; B, pointers to the proposals behind earlier decisions; C, the glossary; D, the application binary interface (how compiled code, the linker and the operating system agree); E, the assembly language and the object format; F, the evidence documents; G, how the design grew, with dates; H, proposed; I, deferred; J, open items.

## 1.3 The road from the PDP-11

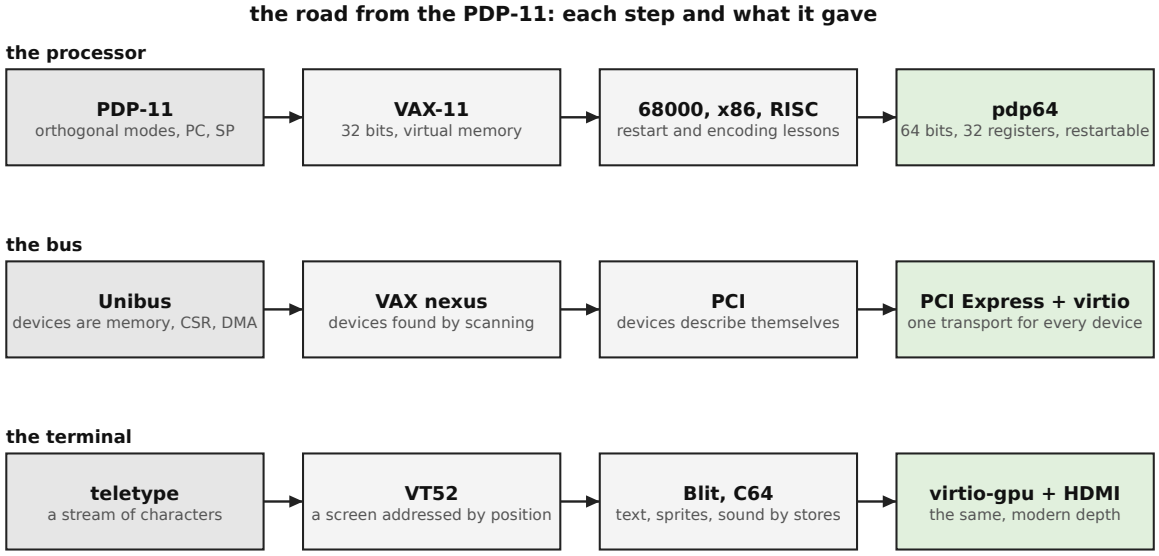
The PDP-11 programmer's first question is which of what he knows still holds. The table answers it row by row; the commentary after it says where each row's history is told.

**1.3.1** The PDP-11 and the VAX-11 were pleasant, even fun, to program in assembly language; afterward computer design went in many directions and that pleasure was largely lost. The pdp64 carries it forward with the lessons of the last fifty years: which instructions are needed to get things done and which are not, what a processor is used for and which instructions it needs the most, to have "less possibility for error, fewer footguns, fewer surprises" and "the most bang for buck" (R197, 2026-10-22).

**1.3.2** The design began as a PDP-11 brought up to date by a little, and grew by decisions that each answered a question. The author's account (2026-11-10, appendix G): "we started off as a pdp16 modernized a tiny bit, then got more and more ambitious", and "the initial idea was, could a pdp16 run effectively today with a blit terminal and off-loaded 8-register nic", with "the blit being a mashup with the c64 for audio" (R393). In the author's usage a pdp16 is a 16-bit machine of the PDP-11 line (*inference*: R280, "pdp64 is an extension of pdp16"); the Blit is the Bell Labs bitmap terminal (3.3); an off-loaded network card is one that carries out the network protocols itself, so that the processor only hands it data.

**1.3.3** The first design brief (2026-08-29) was "use pdp-11 as the base, fold in the good parts of c64" (R20), "taking the best isa/architecture and improving it with the good stuff of the other one" (R23). The steps that followed are in appendix G; the ones that shaped this chapter are the 64-bit step (1.1), the little-endian rule ("none of this middle endian stuff", R22), the bus (PCI Express, errata 80; virtio across the board, errata 95), several cores (errata 92) and the priorities of 1.6. The "off-loaded 8-register nic" of the first idea was replaced by virtio devices (errata 95).

The figure gives the road in three tracks, the processor, the bus and the terminal, each from the PDP-11 to the pdp64 through the steps between; the table that follows gives the same road row by row.




Chapters: processor 9 and 6; bus 2 and 5; terminal 3. Steps between are named in the chapters; the author's dated record is appendix G.

| What | The PDP-11 | The pdp64 | Why | Where |
|---|---|---|---|---|
| Registers | eight visible at a time, of 16 bits; R6 stack pointer, R7 program counter (11/70 handbook 1.4) | thirty-two of 64 bits; `R14` stack pointer; the program counter is not a register and `R7` is ordinary (*D*) | 64-bit data; on the programs measured, 32 registers cut the frame loads and stores executed from 65.2 to 24.3 million against 16 (1.8.3); a write to a register never jumps | 9.3 |
| Byte order | little-endian words; a 32-bit long is two words, high word first (*inference*, V7 `t3.c`) | little-endian throughout, no middle-endian long (R22) | one rule for every width | 9.4 |
| Instruction length | 2, 4 or 6 bytes: one word, plus one for each operand that has an index, an immediate or an address (handbook 3.2, 3.5) | 2, 4 or 6 bytes; the first 16 bits alone fix the length (*D*) | a decoder that must read one operand after another to know where the next instruction starts is slow | 9.1, 9.4 |
| Operands | two, each any addressing mode, so memory to memory | at most one memory operand; the other is a register (*D*) | only one operand can fault, so restart is simple; 5-bit registers and 12-bit displacements fit in four bytes | 9.2, 6.4 |
| Addressing | eight modes with a deferred (indirect) form of each; autoincrement and autodecrement; index with a following word | autoincrement and autodecrement kept, with a stride; base plus displacement, scaled index, PC-relative, direct page, immediate, absolute; deferred forms removed (*D*) | a deferred mode is a second memory reference inside one operand | 9.5 |
| Flags | N, Z, V, C written by most instructions | the same four; moves, loads, stores and compare-and-branch write none; no instruction writes part of N, Z, V (*D*) | with the PDP-11 rule 68.2% of compiled instructions wrote a flag and 1.5% read one; flags are what blocks a fast implementation (1.6.4) | 9.3.4 |
| Branches | conditional branch with an 8-bit word offset, reach 256 bytes | 14 conditions; the field grows with the length: about 64 to 512 bytes, 64 KiB, 4 GiB; compare-and-branch; subtract-one-and-branch both ways | long jumps needed a trampoline | 9.10 |
| Calls | `JSR Rn` keeps the return address in a register of the caller's choosing | `CALL` pushes it on the stack: no link register; `PUSHM` and `POPM` save a set of registers (*D*) | every unwinder and debugger expects the return address on the stack | 9.10, 9.18 |

| What | The PDP-11 | The pdp64 | Why | Where |
|---|---|---|---|---|
| Devices | registers at fixed addresses set by switches on the card; vectors on the data lines | still memory locations, no input or output instructions; found by enumeration; one virtio interface | one driver routine serves every device | 2, 5 |
| Interrupts and traps | a vector pair (new PC, new PS) in low memory; the old PS and PC are pushed on the kernel stack | vectors and a three-bit priority kept; entry writes the old state into registers and cannot fault | the push could fault; the PDP-11 needed a journal to undo it | 6.3, 6.4 |
| Memory management | the KT11: eight segments of up to 8 KB for each mode, base and length; swapping whole images | page tables of three, four or five levels; a translation buffer with address-space numbers | pages are handed out singly and need no contiguous memory | 6.6 to 6.8 |
| Modes | kernel, supervisor, user; a stack pointer for each | kept (*D*) | the PDP-11 arrangement was already right | 6.2 |
| Floating point | the FP11 with mode bits for double precision, long integers and truncation | IEEE 754 half, single and double precision in the general registers, with a fused multiply-add; no mode bits (*D*) | one result format, no hidden state | 9.12 |
| Processors | one; the V7 kernel guards itself by raising the priority (`spl`) | several cores, sequentially consistent memory, atomics (*D*) | a second core must be easy to add | 7 |
| Boot | a ROM reads one block from the disk and jumps to it | the same (*D*) | it is small and every stage after it is ordinary software | 4 |
| Terminal | a teletype, then video terminals with escape codes | text and graphics set by simple stores, in the spirit of the Blit and the Commodore 64 | the author based the design on both machines (R20, R291, R393) | 3 |

**1.3.4** The PDP-11 is a source, not an obligation: what earned its place is kept, what did not is dropped, and what fifty years showed to be missing is added.

*Commentary.* **Chain.** Each row has its history in the chapter named: the instruction set in 1.4 and 1.5, the bus in section 2.1 (the Unibus, the Q-bus, the VAX interconnect, PCI, PCI Express, virtio), the terminal in 3.1 to 3.5 (the teletype, the VT52, the Blit, the Commodore 64, VGA), boot in 4.1 and 4.2, drivers in 5.1 and 5.2, the kernel in 6.1. The machines the author himself named first, in the order of his messages (appendix G.2), are the Blit, the VAX instruction set and the DEC Alpha, the Unibus and Q-bus against PCI (all 2026-08-23); the Commodore 64 with its SID sound chip, USB and NVMe, V7 Unix as the software base, the 6502 family and IEEE 754 floating point (2026-08-29); x86 and UTF-8's length-first coding (2026-08-30). **Trade-off.** The table lists what changed, and a change has a price: a load and a store where the PDP-11 had one memory-to-memory `mov`, a longer encoding for a deferred access, no compact `JSR Rn` for coroutines. 9.18 lists the idioms restored, lost and replaced. **Impact.** *Kernel:* chapters 4 to 7 follow the V7 kernel's order. *Compiler:* a register-to-register core with one memory operand, as a RISC compiler expects, and the PDP-11's addressing for the loops that want it. *Hardware:* a decoder that finds the length from a prefix and no microcode.

## 1.4 What made the PDP-10, the PDP-11 and the VAX-11 good to program

What you know: the PDP-11 loop. This is V7's start-up code clearing the user structure (`usr/sys/conf/mch.s`; the comments are added here):

```
        mov     $_u,r0          / r0 points at the user structure
1:      clr     (r0)+           / store zero where r0 points, then add 2 to r0
        cmp     r0,sp           / has r0 reached the stack pointer?
        blo     1b              / no (unsigned lower): go round again
```

`clr (r0)+` is one two-byte instruction that stores and steps the pointer: in C, `*p++ = 0`. The pdp64 keeps the instruction, and the whole loop is the same three lines (an illustration; `r0` already points at the start):

```
1:      clr     (r0)+           ; store 8 bytes of zero, then add 8 to r0 (a stride of 0 means the access
size)
        cmp     r0, sp          ; has r0 reached the stack pointer?
        blo     1b              ; no (unsigned lower): go round again
```

Three differences. The register is 64 bits and the store is 8 bytes, so each trip clears four times as much. `clr` is four bytes, because a memory operand with a post-increment needs a 12-bit stride field and a 5-bit register field (the leaf `SOF.PI12` of the grammar); `cmp` and `blo` are two bytes each here, because `r0` and `sp` are among the hot registers that a one-parcel form can name

(9.4.6), so the loop is eight bytes where the PDP-11's is six. And a compare-and-branch, `cblo r0, sp, 1b`, can take the place of the last two (four bytes, no flags written).

**1.4.1** The PDP-10 gave a programmer one word per instruction that said everything: the operation, the register, the index register, the indirect bit and the address. *(unverified)*

**1.4.2** The PDP-10's registers were also memory: the sixteen accumulators answered to the addresses 0 to 15. *(unverified)*

**1.4.3** The PDP-10 had half-word and byte operations, stack and skip instructions, and a procedure call that was one instruction (`PUSHJ`). *(unverified)*

**1.4.4** The PDP-11 made every addressing mode available with every register, so that the language had few exceptions to remember (11/70 handbook, chapter 3: the general registers can be used as accumulators, pointers, stepping pointers and index registers).

**1.4.5** The PDP-11's program counter and stack pointer were ordinary registers (R7 and R6), and its autoincrement and autodecrement modes made `*p++` and `*--p` single instructions.

**1.4.6** The PDP-11 had no input or output instructions: a device was a few memory locations, and a driver was an ordinary program.

**1.4.7** The VAX-11 widened the PDP-11 to 32 bits and added three-operand arithmetic, a complete virtual-memory system, and instructions for the jobs programs actually do: procedure calls (`CALLS`), queues (`INSQUE`), character strings (`MOVC3`), array indexing (`INDEX`) and polynomial evaluation (`POLY`). (VAX architecture handbook, EK-VAXAR-RM-001)

*Commentary.* **Chain.** The PDP-10's simplicity of format and its habit of making the machine's own state addressable (1.4.1, 1.4.2) are the origin of the idea that the machine should hold together in the head; the PDP-10 handbooks are not on this machine, so 1.4.1 to 1.4.3 are general knowledge and unverified. The PDP-11's regularity (1.4.4) is what mattered most to assembly programmers and later to compiler writers: an instruction that takes a register takes an address, an immediate or a stack slot with the same syntax. Its memory-mapped devices (1.4.6) are the reason the pdp64's devices are still memory locations in the one address space (chapters 2 and 5). The VAX-11's richness (1.4.7) bought programmability and cost hidden state in its longest instructions: a page fault in the middle of a character move could not be resumed without a bit in the status word, the "first part done" bit (`PSL_FPD`, 0x08000000, in 4.3BSD `psl.h`), which tells the microcode that the instruction is half done. The Motorola 68000 took the PDP-11's regularity and widened it, and could not restart an instruction after a bus error, so it could not support demand-paged virtual memory (memory fetched from disk when first touched) until its successor, the 68010, patched the fault model (both *unverified* here; the project's instruction-set text, section 10, records them). **Trade-offs.** Each of these qualities had a price that a later machine paid: the PDP-11's regularity a decoder that must read both operand fields to find an instruction's length (1.8), the VAX's richness hidden state in its longest instructions, the 68000's the lost restart. **Improvements.** The pdp64 keeps what made programming pleasant and removes what made implementation hard: no hidden instruction state, so every instruction can be restarted (9.2.6); at most one memory operand (9.2.2); a small fixed maximum instruction length (9.2.3); and no mode bits that change what unrelated instructions mean unless the benefit outweighs the cost (1.6.3). **Modern comparables.** x86-64 kept memory operands and a variable-length encoding. AArch64 has one length, 4 bytes (4.000 bytes an instruction on the 840 files that clang compiled, 1.7), and RISC-V has a 4-byte base with an optional 2-byte compressed form (2.93 bytes an instruction at `-Os` with it). The pdp64 sits between: closer to x86 in the form of its instructions and closer to RISC-V in the discipline of its encoding (`encoding-work/arch-comparison.md`, section 5).

## 1.5 More minimal than its ancestors, where hindsight allows

The PDP-11 was orthogonal (any operation with any addressing mode on any register), and the VAX carried that to completeness: every operation in every data type, with up to six operands, each of any mode.

**1.5.1** In several ways the pdp64 is more minimal than the PDP-11 and the VAX-11: it gives up theoretical completeness where fifty years of use showed the completeness was never exercised, and spends what that saves on power that was (the author, 2026-10-23, R272: "in hindsight, in some ways, we are even more minimal than the vax and pdp11; hindsight let us know which theoretical completeness we could trade for more power").

**1.5.2** Not every instruction works with every addressing mode on every operand: at most one operand of an instruction is a memory reference. *(D)*

**1.5.3** The deferred addressing modes are gone; an indirection is a `JMPI`, a `CALLI` or an ordinary load. *(D)*

**1.5.4** The ALU has one width, there is no arithmetic in the narrow sizes, and there is no conversion opcode group: conversions are moves. *(D)*

**1.5.5** There is one rotate direction, and no slot exists only to complete a table.

**1.5.6** The VAX's packed decimal, string editing and `CALLS` and `CALLG` procedure-call instructions are absent. *(D)*

**1.5.7** There are no transcendental instructions, no separate floating-point register file and no rounding-mode bits. *(D)*

**1.5.8** What the saving buys is power that programs use: a bit unit (population count, count of leading and trailing zeros, byte reversal, bitfields, a carry-less multiply), block operations that restart from their registers, a fused multiply-add with the IEEE exception flags, the full set of atomics, UTF-8 instructions, and a user-memory guard. *(D)*

*Commentary.* **Chain.** The completeness of the PDP-11 and the VAX made assembly programming regular and a compiler's job simple, and it was designed when memory was dear and compilers weak. It was never free. It bought a decoder that must parse specifier after specifier before the next instruction's address is known, and instructions that could fault in the middle with unbounded state to save (1.4). What hindsight lets the pdp64 do is separate the two benefits: keep the regularity that programmers and compilers use (every register is general, every mode means the same wherever it is allowed) and drop the completeness that nobody exercised (a memory-to-memory arithmetic instruction, a deferred mode on every operand, a sized version of every operation). The RISC machines of the 1980s went further and made every instruction register to register except loads and stores; x86 kept one memory operand and has survived on it; the pdp64 takes the middle position. **Trade-offs.** The price is a few more instructions in rare cases, such as a load and a store where the VAX had a memory-to-memory move (`MCPY` covers the block case). Measurements that bear on what was dropped: a signed multiply and an unsigned multiply have the same low 64 bits, so a second multiply would duplicate the first, while the instruction that returns the high half is the one multi-precision arithmetic needs; a carry-less multiply is the primitive beneath every CRC, so no CRC instruction exists; in the first compiler's output on 827 files the multiply is 2,204 of about 370,000 instructions (0.6%) and the four divide instructions together 235 (0.06%) (`codegen-analysis.md`; the rejected features are in `pdp64-isa.md`, section 11). **Improvements.** Each piece of dropped completeness was weighed against what it freed: the one memory operand leaves an undo buffer of one entry and a fault matrix whose third row, an operand write, can never fault (6.4), and it pays for thirty-two registers and a 12-bit displacement in four bytes; dropping the sized ALU removes byte-lane multiplexing and partial-register writes altogether; dropping the transcendentals leaves the multiply-add datapath, which accurate library mathematics is built from. **Impact.** *Kernel:* no instruction can leave half its work done, so the fault handler needs no journal. *Compiler:* instruction selection is register allocation with one memory operand, as on a RISC. *Hardware:* a small decoder, small exception machinery, and the area that completeness would have used goes to the bit unit, the block-move sequencer and the UTF-8 decoder, together under 7% of the core's estimated 92,000 transistors (10.2).

## 1.6 The principles

You know the PDP-11's one rule, that any instruction takes any operand in any mode. The pdp64 has several rules, ranked, and each can be broken for a reason.

**1.6.1** (errata 83, R323) The design follows these priorities, in this order: **security and safety**, meaning errors that can never arise, then errors that are easy to detect, then errors that are cheap to recover from (restart and recovery go together, and an instruction does what its name says and nothing else); **opcode choice**, so that fewer instructions executed means fewer memory fetches; **opcode encoding**, dense, with rare instructions in longer encodings; **speed and power**, served by a low gate count; a **good compiler target**; and **pleasant in assembler**. *(D)*

**1.6.2** All the rules are soft rules: a rule gives way when breaking it is worth more than it costs, with the cost written down (R242: "all the rules are soft rules"). *(D)*

**1.6.3** Between two designs that both work, prefer by this order: the one with no strange side effects (an instruction does what its name says and nothing else); the one that does not push an existing instruction into a longer encoding. Beside the ranking, at every level, do not multiply entities without benefit. Mode bits are decided case by case: by default a mode is visible state, and a bit in a register that is saved anyway costs nothing extra (errata 83).

**1.6.4** The architecture is designed to be efficient with in-order and out-of-order implementations (R297: "designed to be efficient with in-order and out-of-order execution implementations"). An in-order core runs instructions one after another in the order written. An out-of-order core runs whichever are ready, and makes the result look as if they had run in order. The first implementations are in order; nothing is specified in a way that is safe only because the core does not speculate or reorder, and nothing is specified that an out-of-order core could implement only at a large cost (R295, R220). *(D)*

**1.6.5** An instruction is judged by its weight in the loops where it is used, not by how often it appears: a rarely used instruction that sits inside a critical, tight hot loop can earn its place (R296). The measure is profile-weighted (executions multiplied by the instructions and bytes it saves), not a static count. *(D)*

*Commentary.* **Chain.** The PDP-11's rule was regularity (1.4) and the VAX's completeness (1.5); the handbooks on this machine state no ranking of goals (*inference*: none was found in them). Most architectures since have stated none either (*unverified*); the nearest in spirit is RISC-V's published rule of a small base and optional extensions (*unverified*). **Trade-offs.** The priorities agree on almost everything, because a small, regular machine is also compact in code and cheap in gates; the stated order decides which wins when they conflict. Error prevention, detection and recovery is the principle that most often changes a decision. It means counting an option's cost not only in gates and bytes but in the code a kernel must write to handle its failures: a feature that adds a fault, an error return or a half-finished state costs recovery code in every system that uses it, and a feature that makes an operation repeatable from the start costs none. **Improvements.** The soft-rules principle covers the two strongest defaults, "no mode bits" and "natural alignment of quad accesses"; breaking either is weighed like any other choice. Four bits of the design were accepted on those terms: the translation-enable bit that is set once at boot, the user-memory guard in the status word, the data-independent-timing bit `DIT`, and the sticky floating-point flags, which never change what an instruction does. The priority list is written with the soft-rule test and the hot-loop weight as standing constraints, not ranks (the priority lists, `docs/priority-lists.md`). **Impact.** *Kernel:* a restartable instruction needs no recovery code. *Compiler:* weight by use steers the compiler's register use and the fused pairs (9.18). *Hardware:* the out-of-order constraint is why the flags are written as in 9.3.4 and why `CBCC` writes none.

## 1.7 Does the instruction set matter?

You may have heard that a good implementation hides the instruction set. The author wrote this section for a reader who holds that view (R275).

**1.7.1** The instruction set is not interchangeable with any other: it fixes what every implementation must preserve, and so what a processor costs to build, to verify, to program and to secure.

**1.7.2** The common view, that a good implementation hides the instruction set, is true of one thing: the peak speed of a very large core, where a great deal of hardware can absorb almost any encoding.

**1.7.3** It is not true of the cost of a small core, the size of the code, the amount of software a system needs, or the number of ways a system can fail; those follow the instruction set, and they can be measured.

**1.7.4** Each of the rows below is a measurement or an estimate, and each is explained in the chapter named beside it.

| What differs | The figure | Where |
|---|---|---|
| The logic of a complete implementation, same company and era | the PDP-11's T-11, a full integer core without memory management or floating point, about 12,000 transistors; the J-11, the full PDP-11/70 with both, about 120,000; the MicroVAX 78032, a subset of the VAX with no floating point, about 125,000 (all *unverified*) | 10.2 |
| What it takes to find where the next instruction starts | pdp64: a prefix of 3 to 16 bits of the first parcel; PDP-11: both operand specifiers; VAX: every specifier in order; x86: prefixes, opcode, ModR/M and SIB in sequence (*unverified*) | 9.1, 9.4 |
| Whether flags block a faster implementation | with the PDP-11 rule 68.2% of compiled instructions wrote a flag and 1.5% read one (229,389 and 4,999 of 336,408); with the pdp64's rules 16.7% write one, at a measured cost of 0.39% in code size | 9.3.4, 1.6.4 |
| Size of compiled code | 964,158 bytes of instructions on the 841-file corpus; clang `-Os` is 1.21 times that for x86-64, 1.34 for AArch64, 1.22 for RV64GC and 0.91 for Thumb-2; counting the pdp64's constant pools too, x86-64 and RV64GC are 0.97 and 0.98 times | 9.2.13, F.4 |
| Lines of kernel code to start a second core (Linux) | x86: 1,528 plus 1,258 of real-mode code; arm64: 124 (PSCI) or 127 (spin table); RISC-V: 108 or 58 | 4.7 |
| Machine-dependent code in an OpenBSD port | riscv64 35,600 lines; arm64 61,613; amd64 79,167 | 10.5 |
| Errata, workaround and quirk lines per 1,000 code lines in Linux's architecture directories | x86 0.8; arm64 0.7; arm 0.5; powerpc 0.4; mips 0.3; m68k 0.1; riscv 0.1; s390 0.0 | appendix F |
| Hardware-erratum options in Linux's configuration | arm64 64; arm 32; riscv 14 | appendix F |

The code-size row compares the pdp64's portable C compiler with clang for the others, on the files each compiled (840 or 841), so it measures compilers as well as instruction sets (`encoding-work/arch-comparison.md`, section 5). The flag figures are for the first compiler's output on 827 files (`pdp64-ooo-readiness.md`).

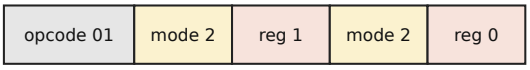
*Commentary.* **Chain.** The view that instruction sets are equivalent has real support. Published comparisons of x86 and ARM implementations (the 2013 study usually cited is "Power Struggles: Revisiting the RISC vs. CISC Debate on Contemporary ARM and x86 Architectures", a citation not verified) found that at the same performance level the instruction set explains little of the difference in power or speed, and the very large cores of both families have converged. The pdp64 does not dispute that. **Trade-offs.** The claim is about steady-state performance of comparable large cores, and the table is about other costs. The Linux figures are measurements of real software, not controlled experiments: part of the difference between architectures is age and deployment (x86 is the oldest and most widely deployed), and the start-up and line-count figures are counts of what is written, not run times. They show that the instruction set goes with these costs, and the chapters show the mechanism in each case. **Impact.** *Kernel:* fewer special cases to write and test. *Compiler:* fewer rules for the code generator. *Hardware:* a smaller decoder and a smaller verification effort, because what an instruction may leave behind is fixed by the architecture (precise exceptions, ordering, flag behaviour, length, restart). RISC-V's own case for a clean base and x86's survival by microarchitecture are the two sides of the argument.

## 1.8 How the encoding was chosen

You know the PDP-11 instruction word: four bits of operation and two six-bit operand fields, each a three-bit mode and a three-bit register. The pdp64's encoding starts from a different idea, shown here.

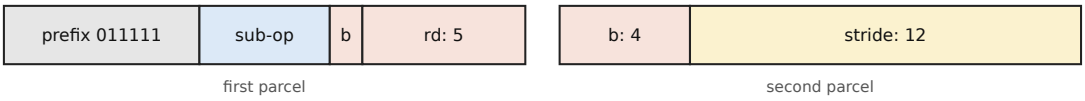
**one instruction word of the PDP-11, and the same kind of instruction on the pdp64**

**PDP-11: mov (r1)+,(r0)+ = 012120 octal, 16 bits, memory to memory**




source: mode 2 is autoincrement, register 1; destination: mode 2, register 0; an extra word follows only for the modes 6 and 7, and for modes 2 and 3 on register 7 (immediate, absolute); so the length, 2, 4 or 6 bytes, depends on both six-bit fields.

**pdp64: movq (r1)+, r0 = 2 parcels of 16 bits (4 bytes), one memory operand**




prefix: a bit pattern that no other prefix begins; it alone says which instruction this is and that it is two parcels long (9.1).
sub-op: which of the sixteen operations of this form (here movq); b: the five bits of the base register, one in the first parcel and four in the second;
rd: the destination register, five bits because there are thirty-two; stride: how far the base register moves afterwards, 0 meaning the size of the acce

The PDP-11 word is read from the handbook (11/70, section 3.2); the pdp64 fields are the leaf LDF.PI12 of the grammar (dictionary/by-encoding.md).

The assembler spells the pdp64 instruction as shown in the heading of this figure (appendix E).

How a decoder finds where the next instruction starts, for each:

```
# PDP-11: the length depends on both operand fields (handbook 3.3.4 and 3.5)
length(word):
    n = 2
    for each operand field f of the word:        # none, one or two of them
        if f.mode is 6 or 7: n += 2              # index: a word follows
        if f.mode is 2 or 3 and f.reg is 7: n += 2   # immediate or absolute: a word follows
    return n

# pdp64: the length depends on the first 16 bits only
length(parcel):
    leaf = the one leaf whose prefix (3 to 16 bits) begins parcel
    return 2 * leaf.parcels                      # 1, 2 or 3 parcels (5 for the 64-bit literal move)
```

**1.8.1** The pdp64 has thirty-two registers and an instruction encoded by a prefix code over the first parcel (a parcel is 16 bits; a prefix code is a set of bit patterns none of which begins another, so the first bits of an instruction tell the decoder what it is and how long), with 5-bit register fields in the two-parcel forms. *(D)*

**1.8.2** Two earlier iterations were built and measured against it and were superseded: pdp64/r20 (the original: sixteen registers, a first parcel of `len`, opcode, `reg` and `low`) and pdp64/16 (sixteen registers, a prefix code). They are no longer CPUs

and have no manuals. They shared with the pdp64 the roles of the registers, the memory model, the exception mechanism, the page tables, the bus and the devices, the calling convention and the assembly language, and differed in how an instruction is encoded and in the size of the register file. The record of the comparison is in `encoding-work/` and the last commit that carries their manuals is the git tag `pre-single-cpu`. *(D)*

**1.8.3** The encoding was chosen for performance, by testing the three on the same programs. Over six programs (lz4, zlib, Lua and three BearSSL kernels) compiled for each and run on the simulator, the pdp64 executes 572,801,956 instructions against 609,755,229 for pdp64/16 and 755,816,022 for pdp64/r20 (the original compiler for the latter); its frame loads and stores fall to 24,276,195 against 65,203,703 and 68,936,548; and on the in-order timing model (an ideal memory, no branch predictor, issue width 1 or 2) its cycle counts are 22.5 to 28.1% below pdp64/r20's across fetch windows of 2 to 16 bytes and 6.9 to 8.5% below pdp64/16's with a window of 4 bytes or more (the model is a check of the pipeline it states, not hardware). Its code is larger than pdp64/16's: 3.18% in instruction bytes over the 841-file corpus and 2.92% in bytes fetched over the six programs. *(measured; the record is `encoding-work/integration.md`, section 4)*

*Commentary.* **Chain.** PDP-11: one length rule that reads both operand fields (shown above). VAX: every operand specifier in order, with no bound on the length. x86: prefixes, opcode, ModR/M and SIB read one after another (*unverified*). The pdp64: a prefix over the first parcel, which is the length-first idea of UTF-8 that the author cited on 2026-08-30 (R29: "utf8 showed the way encoding can be done"; that the prefix code descends from it is an *inference*). **Trade-offs.** The prefix code makes the decoder read up to 16 bits to find the length, and it costs a little code size against the 16-register prefix code; the measurements above are what it buys. **Impact.** *Kernel:* the system architecture does not depend on the encoding. *Compiler:* one back end. *Hardware:* one decoder and one register file.

# Chapter 2 — The bus

**The processor reaches the world through one bus, PCI Express (backward compatible, so an ordinary card works on it unchanged) with convenience and security circuitry added; every standard device is a virtio device (the disk is virtio-blk) or a USB controller, and any other PCI Express device may be attached, all found by PCI Express enumeration; every transfer of data is a descriptor in a queue in memory that the device carries out and marks done.**

The chapter starts from the Unibus of the PDP-11 and explains each later idea by contrast with it: the PCI Express link, the configuration space, base-address registers, bus-mastering DMA, message-signalled interrupts, virtio and the IOMMU. Between the PDP-11 and the pdp64 it names the intermediate steps that changed the idea (the Q-bus, the VAX interconnect, PCI, PCI Express, virtio), each with the one thing it contributed; section 2.1 gives the route in a table, and each commentary repeats the part of it that concerns its topic. The numbered statements are the rules; the rest is explanation. A fact about PCI Express or virtio that could not be checked on this machine is marked *unverified* and listed in section 2.12; the facts about the Q-bus were checked against DEC handbooks (section 2.12). How a driver talks to a device, the virtio transport and the default devices are in chapter 5; interrupts and modes are chapter 6; the other cores are chapter 7.

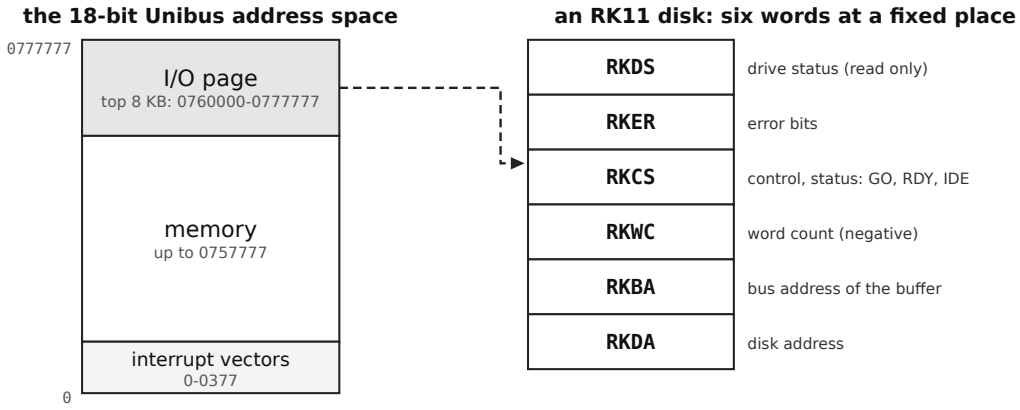
A term is defined where it first appears, as the technical word followed by a plain restatement. The boxes-and-arrows figures are the kind a textbook draws for a linked list: a box is a piece of memory, an arrow is an address stored in it. Code blocks are pseudocode (C-like; `mmio_read16(address)` and the like are ordinary loads and stores to a device register, section 2.3); the PDP-11 assembly is quoted from a 2.11BSD driver or written in its style.

## 2.1 What you already know: the Unibus

A PDP-11 reaches everything outside the processor over its **bus** (a set of shared wires that every part of the machine hangs on), the Unibus. Four ideas of it matter here, because each later idea in this chapter keeps or replaces one of them.

**Registers at fixed addresses.** The Unibus has 18 address lines, so 256 KB of **address space** (the set of numbers that name locations). The top 8 KB of it is the **I/O page** (the region where devices, not memory, answer). A device is a handful of 16-bit **registers** (small storage cells in the device that the processor can read or write) at addresses that were set by switches on the card. Nothing tells the processor what is plugged in. 2.11BSD's standalone boot code has the RK11 at `0177400` written into it (`pdpstand/rk.c`), and the kernel's autoconfiguration has a probe routine per device that pokes the controller (`rkprobe` in `autoconfig/rkauto.c` writes a drive reset with interrupt enable and returns `ACP_IFINTR`, a flag meaning "present if it interrupts"). A driver is written against a C structure laid over those addresses; this is the whole of 2.11BSD's declaration for the RK05 disk controller (`rkreg.h`):

```
struct rkdevice {
        short   rkds;   /* drive status        */
        short   rker;   /* error bits          */
        short   rkcs;   /* control and status  */
        short   rkwc;   /* word count          */
        caddr_t rkba;   /* bus address         */
        short   rkda;   /* disk address        */
};
```




a driver holds one pointer, the address of RKDS; each register is a field
(the wiring, the vector and the CSR address, was set by switches on the card)

**The CSR convention.** Most devices have a **CSR** (control and status register: one word whose bits both command the device and report on it). The RK11's has GO in bit 0 (do it now), READY in bit 7 (the device is idle and may be given work), IDE in bit 6 (interrupt on done enable) and ERROR in bit 15 (`rkreg.h`: `RKCS_GO` 01, `RKCS_RDY` 0200, `RKCS_IDE` 0100, `RKCS_ERR` 0100000). The console serial line has the same shape: its status register is at `0177560` in `pdp/cons.c`, with a **DBR** (data buffer register: the word the data passes through) beside it, `dlrbuf`; the line printer has `lpcs` and `lpdb` (`pdpuba/lp.c`).

**DMA.** To move a block, the processor does not copy words: it writes the memory address into a register (`rkba`), a negative count into another (`rkwc`), the disk address into a third and then GO. The controller becomes a **bus master** (a part that is allowed to start transfers itself, not only answer them) and **DMA** (direct memory access: moving data between a device and memory without the processor's help) does the copying, requesting the bus by **NPR** (non-processor request: the Unibus line a device raises to be given the bus for a data transfer at once, at a priority above every interrupt). The 18-bit bus address was too narrow once memory grew, and a **map register** (a table entry that translates a device's address to a memory address) appears in 2.11BSD as the UNIBUS map: `struct ubmap` (two 16-bit words per register) at `0170200`, allocated by `mapalloc` (`h/uba.h`, `pdp/machdep.c`). It is the ancestor of the IOMMU of section 2.11.

**Interrupts.** A device that wants attention asserts one of four **BR** lines (bus request, levels 4 to 7: the priority at which it wishes to interrupt; `br4` to `br7` in 2.11BSD's `pdp/scb.s`); when the processor's priority is lower, it grants the bus, and the device places its own **interrupt vector** (the address of a pair of words, new program counter and new status word, that the processor loads) on the data lines. The vector too was set by switches on the card; the RK11's is `0220` at level 5 (`DEVTRAP(220, rkintr, br5)` in `pdp/scb.s`). The interrupt routine returns with `RTI` (return from interrupt). A driver that does not want interrupts **polls** (reads the status register again and again until the bit it wants changes).

Here is the RK11 driver in the two styles, both real or in the style of the real code. The interrupt-driven start, from 2.11BSD `pdpuba/rk.c`:

```
rkaddr->rkda = (dn << 13) | (cn << 4) | sn;     /* disk address     */
rkaddr->rkba = bp->b_un.b_addr;                  /* where in memory  */
rkaddr->rkwc = -(bp->b_bcount >> 1);             /* words, negative  */
com = ((bp->b_xmem & 3) << 4) | RKCS_IDE | RKCS_GO;
com |= (bp->b_flags & B_READ) ? RKCS_RCOM : RKCS_WCOM;
rkaddr->rkcs = com;                              /* go; return; rkintr() runs when done */
```

and the same command polled, in PDP-11 assembly (the registers are at `0177400` plus the structure's offsets: `rkds` 0, `rker` 2, `rkcs` 4, `rkwc` 6, `rkba` 010, `rkda` 012; `RCOM` is `RKCS_RCOM`, 04):

```
        MOV  #<disk address>, @#177412     ; RKDA
        MOV  #<buffer>,       @#177410     ; RKBA
        MOV  #-256.,          @#177406     ; RKWC: 256 words, negative
        MOV  #RCOM+GO,        @#177404     ; RKCS: read, go, no interrupt enable
1$:     TSTB @#177404                      ; bit 7 of the low byte is READY
        BPL  1$                            ; plus means not yet: look again
```

```
        TST  @#177404                   ; bit 15 is ERROR
        BMI  error
```

The processor knows the register layout, loads each register, waits, and knows the meaning of every bit of that one device. A device with different registers needs a new driver. The rest of the chapter is the answer the pdp64 adopts: every device has the same shape.

Gone in the pdp64: the fixed I/O page, the switches, the vectors on the data lines, the bus request levels, and the single shared set of wires (with it the locked read-modify-write DATIP/DATO, section 2.10). Kept: devices are locations in the one address space, reached by ordinary loads and stores with no I/O instructions (2.3.2); the device moves its own data by DMA; the interrupt says only "look at the device"; polling stays allowed (2.6.3).

**The Q-bus, the Unibus's lower-cost sibling.** DEC made a second PDP-11 bus, the **Q-bus**, which gives a driver the same programming model: registers in an I/O page, interrupt vectors, the BR priority levels. DEC's own handbooks call it the **LSI-11 bus**; the word "Qbus" appears in the Micro/PDP-11 Handbook (MH, p. 11, 1983-84) and the 22-bit backplane layout is labelled "Q22 format" there (MH, p. 28). Sources, cited by printed page: BH is DEC's *PDP-11 Bus Handbook* (1979), MH is the *Micro/PDP-11 Handbook* (EB-24944-18, 1983-84), LH is the *LSI-11 Processor Handbook* (1975-76), all scans on bitsavers.org (section 2.12). 2.11BSD's `intro(4)` says its autoconfiguration probes for a device "on either the UNIBUS (or Q-bus)", and 2.11BSD has Q-bus drivers such as `qe` (the DEQNA Ethernet interface). The I/O page is checked: the Unibus reserves "the top 4,096 word locations" (8 KB) for device registers (BH p. 7), and on the Q-bus the master asserts a line, BBS7 ("bank 7 select"), when the address is in "the I/O page (the upper 4K address space)", and a device decodes BBS7 with the thirteen low address bits and ignores the high ones (BH pp. 107-108). Interrupts keep the Unibus model: a device that wins the acknowledge puts its vector on the bus, and the vector is the first of a pair of words, a new PC and a new status word, below location 1000 octal (BH pp. 117-121); there are four request levels, BIRQ4 to BIRQ7, but only the 11/23 class of processor tells them apart, while the 11/03 class "treats all interrupts as level 4" (BH p. 118).

What differs is below the driver. The Q-bus **multiplexes** address and data: "16 multiplexed data/address lines" (LH p. 1-3), BDAL<15:00>, carry the address first and the data after it, 38 lines in all for the processor, memory and devices (BH p. 97), where the Unibus has 16 data lines and 18 address lines of its own (BH p. 21). DEC says why: the LSI-11 bus is "similar in concept to the UNIBUS but with fewer lines for lower cost" (MH p. 19), and "the low-end member of DIGITAL's bus family" (MH p. 247); a buyer is told to prefer the Micro/PDP-11 "if low cost is a key factor" and the Unibus PDP-11 otherwise (MH p. 11). Which machine came first is also checked, and it corrects an earlier guess here: the PDP-11/03, the LSI-11, "used the new LSI-11 bus", and "the PDP-11/23 became the second LSI-11 bus processor" in 1979 (MH p. 19); the LSI-11 is therefore the first Q-bus machine and the 11/23 the second. The 2.10 setup file (`doc/2.10/setup.2.10/a.t`) lists the 11/23, 11/53, 11/73 and 11/83 among the machines that never have Unibus map registers, and its Q22 option ("22-bit Qbus with no 18-bit devices") is "always" for the 11/53, 11/73 and 11/83 and "sometimes" for the 11/23. The cycles have the same shape as on the Unibus but the cycle type is not coded in two control lines (C0, C1, BH p. 22) but is told by the strobes: the master puts the address on BDAL and asserts BSYNC; for a read it then asserts BDIN and the slave answers with data and BRPLY; for a write the master asserts BWTBT with the address, then puts data on the lines and asserts BDOUT, and the slave answers BRPLY (BH pp. 103-109). The named cycles are DATI (read word), DATO (write word), DATOB (write byte), DATIO (read-modify-write word, done by holding BSYNC asserted between the read and the write) and DATIOB (BH pp. 106, 108); the Q-bus has DATIO where the Unibus has the locked DATIP/DATO.

**Q22 and block transfers.** The 16-bit bus carried 18 address bits through BDAL<17:16>, which are "used for extended addressing, memory parity error, and memory parity error enable" (BH p. 106). **Q22** is the extended bus with four more lines, BDAL<21:18>, for 22-bit addresses (4 MB); the 22-bit addressing of the 11/23, the Micro/PDP-11 and the 11/23-PLUS needs it ("the extended LSI-11 Bus is required", MH p. 247), and the I/O page is then recognised by BBS7 with the high nine bits ignored (MH p. 253). A controller built for the 18-bit Unibus, such as the RK11, cannot reach memory above 256 KB there (18 bits; the Computer History Wiki's Q-bus page, a pointer only, says the same); 2.11BSD handles it with the **soft Unibus map**: `sofub_alloc` in `pdp/machdep.c`, "for 18 bit controllers present in a 22-bit q-bus systems", copies the data through a bounce buffer below 256 KB (QMEG) with `copyv` and `sofub_relse` copies it back (chapter 5, 5.1). That is the bounce buffer of section 2.11 on the PDP-11. The later handbook adds **block transfers**, DATBI and DATBO: the address is sent once and the data words follow to consecutive addresses, which "almost doubled" the rate; the master asks for it by re-using BBS7, and a

slave that can take another word says so with BREF (MH pp. 251-253, 263-266). A master that does not use block mode may transfer at most four DATI, four DATO or two DATIO per acquisition of the bus (MH p. 267).

**Arbitration and parity.** How the Q-bus arbitrates is checked: the interrupt acknowledge (BIAKI, BIAKO) and the DMA grant (BDMGI, BDMGO) "are physically unidirectional in a daisy-chain fashion" (BH p. 97). A **daisy chain** (each card passes the grant on unless it wants the bus, so the card nearest the processor wins) is spelt out for both: a device accepts the acknowledge only if it requested an interrupt and holds the highest-priority request, else it passes it on, and a DMA device accepts the grant only if it raised BDMR (BH pp. 104-105, 115-116). A DMA master then asserts BSACK and stays master until it releases it (BH p. 115). The Unibus passes the grant slot to slot too ("passing the grant", BH p. 18), but with five grant lines (BG4 to BG7 and NPG; BH p. 4) where the Q-bus has one acknowledge and one DMA grant, so the Q-bus 11/23 class decides among the four levels by having each device watch the higher request lines before it takes the acknowledge (BH pp. 120-121). Parity is optional and rides on the address lines: memory drives BDAL16 during the read to say that an error occurred and BDAL17 says that parity checking is enabled; the processor traps through location 114 octal when both are asserted (BH pp. 113-114, KDF11-AA); MH p. 247 calls them the "two multiplexed address/parity lines".

**The route from the Unibus to the pdp64.** Each row is one step and what it contributed; the later sections say where the contribution is used.

| step | what it contributed | basis |
|---|---|---|
| PDP-11 Unibus | registers in one address space, CSR, DMA by NPR, vectors and BR levels, map registers | 2.11BSD sources |
| PDP-11 Q-bus | the same programming model on multiplexed address and data lines, 18 shared in place of 34 separate, at lower cost (the LSI-11 of 1975-76 first, the 11/23 second); one interrupt acknowledge and one DMA grant, each passed by daisy chain; DATIO for the locked read-modify-write; later block transfers (DATBI, DATBO); Q22, 22-bit addresses, so that an 18-bit controller needs the soft Unibus map, a bounce buffer | DEC *PDP-11 Bus Handbook* 1979 pp. 7, 21, 97, 104-122; *Micro/PDP-11 Handbook* 1983-84 pp. 11, 19, 247, 253, 263-267; *LSI-11 Processor Handbook* p. 1-3; 2.11BSD `pdp/machdep.c`, `doc/2.10/setup.2.10/a.t`, `intro(4)` |
| VAX-11/780 | devices behind adapters on an interconnect whose slots report their type in a status register, so the kernel scans instead of being told; the Unibus adapter keeps the map registers and adds buffered data paths that a driver purges; a Massbus adapter is a second kind of adapter (NEX_MBA, NEX_UBA0) | 4.3BSD `nexus.h`, `ubareg.h`, `autoconf.c` |
| PCI | every function describes itself in a configuration space, and windows are placed by software through base-address registers | the layout is checked in `pci_regs.h`; the date and history are *unverified* |
| PCI Express | the same software model over point-to-point links and packets; MSI-X interrupts as memory writes; bus mastering as a command bit | `pci_regs.h`, QEMU `docs/pcie.txt` |
| virtio | one transport and one queue structure for every device kind | `virtio_*.h`, QEMU |
| the pdp64 | PCI Express with the pdp64's decided additions: no device-space address bit, an IOMMU, precise faults and one fence rule | the decisions of errata 75, 80, 95, 97, 101, 102, 104 |

## 2.2 PCI Express: links, packets and configuration space

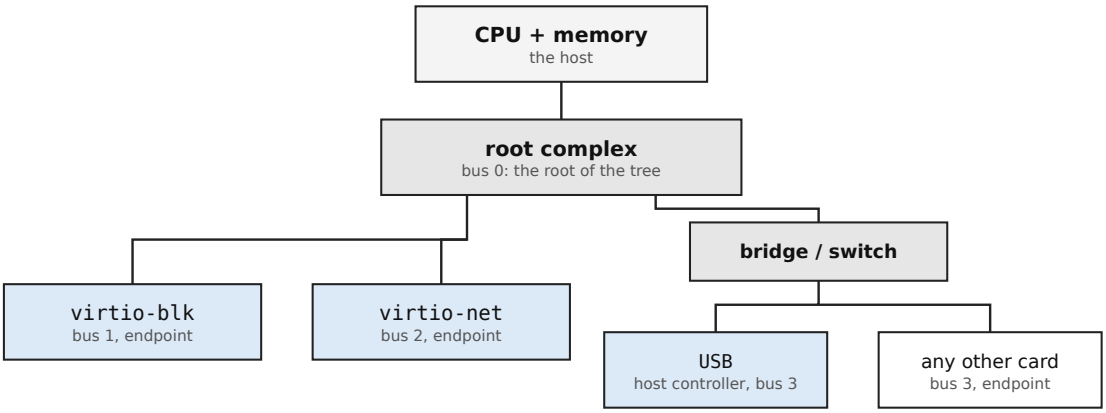
PCI Express (written PCIe) is the bus of PCs and servers (*unverified*: that it has been so since about 2005 is general knowledge, not in any source here). It descends from PCI, the 1990s parallel bus (*unverified* likewise: dates and lineage) whose configuration space and software model it kept. Its ideas, each against the Unibus.

**A link, not a shared bus.** A Unibus is one set of wires that all devices share, so only one transfer happens at a time and every card loads the wires. PCI Express uses **point-to-point links** (a connection between exactly two ends; nothing else is on it). A **lane** is the unit of width: a pair of wires carrying data one way and a second pair the other way, so a lane sends and receives at the same time. A link of more lanes is faster: `pci_regs.h` defines negotiated widths x1, x2, x4 and x8 and has a six-bit width field (x16 and wider are *unverified*). The speed codes of `pci_regs.h` are 2.5, 5, 8, 16, 32 and 64 **GT/s** (gigatransfers a second: signal changes, not yet bytes); each generation is faster than the last. Software sees none of this; it sees addresses.

**A tree.** Because links are point-to-point, a machine is a tree. The root is the **root complex** (the part, in the processor chip or beside it, that connects the processor and memory to the tree and is the PCI Express counterpart of the Unibus's processor-side logic); below it are **root ports** (its links to the outside), **switches** and **bridges** (parts with one link up and several down,

which pass traffic between them; to software a switch is a set of bridges) and at the leaves **endpoints** (the devices). QEMU's `docs/pcie.txt` uses these words: root complex, root port, downstream port, bridge, integrated endpoint.

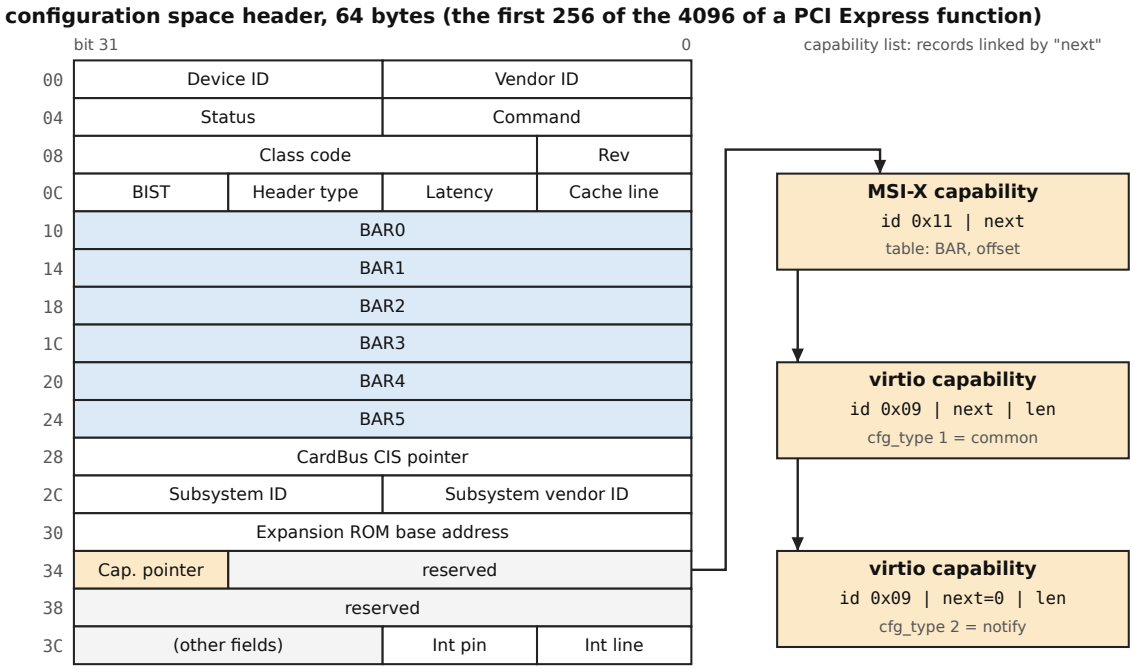
**a PCI Express tree: a bus of point-to-point links, not a shared set of wires**




each line is a link: one or more lanes, each a pair of one-way wire pairs
Bus numbers and the shape are examples.

**Packets.** Where the Unibus signals on dedicated wires, PCI Express sends everything as a **packet** (a small message with a header that says what it is, where it goes and how much data follows). The packets that carry reads and writes are **TLPs** (transaction layer packets: the unit software sees, above the layers that frame bytes and retry on error). The kinds a programmer meets (*unverified*; the specification is not on this machine): a memory write (address and data, with no answer, called **posted**: the sender has handed it over and moves on); a memory read (address and length), answered later by a **completion** (a packet carrying the data back); configuration reads and writes; and messages. A read is a **split transaction** (request and answer are separate packets, and the link is free in between); on the Unibus the bus master held the bus, with `MSYN` asserted, until the slave replied with `SSYN` (*unverified* here). Two consequences: a write has "happened" in the program's mind while it is still on its way, which is why section 2.8 has a `FENCE`; and an interrupt (section 2.6) can be a packet like any other.

**Configuration space: the device describes itself.** The Unibus had nothing like it. Every device, which PCI Express calls a **function** (one logical device; a card may carry several), has a **configuration space** (a block of 4096 bytes of registers, separate from the device's working registers, that says who made the device, what it is and how much address space it wants). The first 256 bytes are the older PCI part, with a 64-byte header; the 4096 bytes in all are the PCI Express part (`PCI_CFG_SPACE_SIZE` 256 and `PCI_CFG_SPACE_EXP_SIZE` 4096). A function is named by three numbers, its **bus**, **device** and **function** numbers (the position in the tree). The processor reaches the configuration space of function (b, d, f) with ordinary loads and stores to one address range: byte `offset` of function (b, d, f) is at `base + (b << 20) + (((d << 3) + f) << 12) + offset` (this is **ECAM**, enhanced configuration access mechanism; the shifts are from QEMU's `pcie_host.h`, `PCIE_MMCFG_BUS_BIT` 20 and `PCIE_MMCFG_DEVFN_BIT` 12). Open: where `base` is in the physical address space (J.2.1).



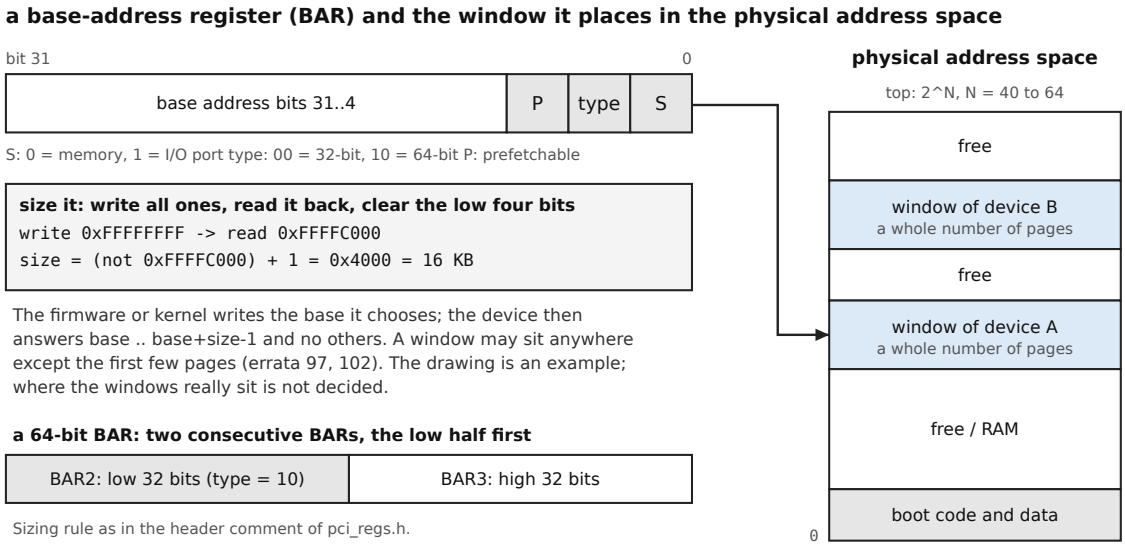

The 64-byte header of an ordinary function (type 0) holds, as the figure shows and `pci_regs.h` defines: the **vendor ID** and **device ID** (two 16-bit numbers that name the maker and the product; 0xFFFF as a vendor ID means that no function answers), the **command** register (bit 1 lets the function answer memory addresses, bit 2 lets it be a bus master, bit 10 turns off its old-style interrupt pin), the **status** register, the **class code** (what kind of device), the **header type** at 0x0E (0 for an ordinary function, 1 for a bridge, with bit 7 set when the card has several functions) and six **base-address registers** at 0x10 to 0x24 (section 2.4). A pointer to the capability list is at 0x34. A bridge's header (type 1) has a different layout after the first 16 bytes; the bus-number bytes at 0x18 to 0x1A are its.

**Capabilities.** The header is fixed; everything optional hangs off a **capability list**: a linked list, inside the configuration space, of small records, each starting with an identifier byte and a byte holding the offset of the next record (0 ends the list). It is a linked list of the kind drawn in section 2.7. The records a pdp64 driver looks for are the MSI-X record (identifier 0x11, section 2.6), the PCI Express record (0x10) and the vendor-specific record (0x09) which virtio uses to say where its structures are (section 2.7).

**What the pdp64 adds.** A PCI Express endpoint looks to software like a PCI device, so one driver model serves both, and has served about thirty years of hardware (*unverified* as history). The author's decision (errata 80) is that the bus is PCI Express, so an ordinary card works on it unchanged, with circuitry added on the processor side for convenience and security (2.3.4).

## 2.3 The address space

**Memory-mapped registers.** On the PDP-11 the registers of a device are **memory-mapped** (they have addresses in the same address space as memory, so the instruction that reads memory reads a register). The pdp64 keeps this and widens it: a function's registers are in **windows** (ranges of the physical address space that the function answers), and a driver reaches them with ordinary loads and stores, with no input or output instructions. What changed is who chooses the addresses. On the Unibus the switches on the card did. In PCI Express the function says how much room it needs, the system assigns an address, and the function answers there (section 2.4). The **base-address register** (BAR: a register in the configuration-space header that holds where a window starts and, by the trick of section 2.4, how big it is) places a window in the address map:




Against the Unibus, three things differ. The fixed I/O page at the top of the address space is gone (errata 97, 102: no address bit says "device"); the address is wider (40 to 64 bits, against 18); and a window may be placed anywhere except the first few pages, where the boot code and boot data are loaded.

**2.3.1** The system bus is PCI Express, so that an ordinary PCI Express device works on it unchanged, with the programming model of the PDP-11 Q-bus (devices as memory locations in the one address space) widened to 64-bit data and a physical address that is not limited to 40 bits, and with convenience and security circuitry added. *(D)*

**2.3.2** Devices live in the ordinary address space; the processor has no input or output instructions. *(D)*

**2.3.3** Every page has a memory type: normal cached, normal uncached, device strongly ordered, or device weakly ordered. *(D)*

**2.3.4** The transactions on the bus are those of PCI Express; the circuitry the pdp64 adds is for convenience (overlays, helper registers) and security (the IOMMU, error and reset rules), and is measured against the list of pain points of `kernel-pain-points.md`. An extension is announced to a card, through a PCI Express extended capability, only if the card must take part in it; a mitigation that lives on the processor side is invisible to cards. *(D)*

Open: the detail of the pdp64's additions to PCI Express (J.2.5).

**2.3.5** No bit of the physical address selects device space: a device window is an ordinary range of physical addresses, found by enumeration (section 2.4), and the physical address is at least 40 bits and at most 64, an implementation choosing its width in that range, so that mappings of petabytes are possible. *(D)* A window can be mapped anywhere in the physical address space except the first few pages, where the boot code and boot data are loaded. *(D)*

Open: how many pages are "the first few" (J.4.1).

Open: how software discovers the implemented physical address width (J.6.1).

**2.3.6** The windows of a device (the regions its base-address registers describe) start on 4 KB page boundaries and are a whole number of pages, so that one device can be mapped to one protection domain. *(D)* Details to be determined.

*Commentary on 2.3.5.* **Chain.** The PDP-11 put its devices in a fixed page at the top of the address space (a driver's pointer to its registers is a constant in that page: `cnaddr = 0177560` in 2.11BSD `pdp/cons.c`). The VAX-11/780 put them in nexus space from `0x20000000`, one 8 KB window (`NEXSIZE` 0x2000) per slot (`NEX780` in 4.3BSD `nexus.h`), and `nexus.h` gives the 11/750, 730 and 8600 different bases, so the VAX kept changing the map from model to model. PCI Express places each function's windows where enumeration assigns them. The pdp64 follows PCI Express. **Trade-off.** A decoding bit makes the first routing decision in the fabric trivial and makes a mapping of the wrong memory type detectable, at the cost of half the physical address space; the author dropped it because mapping petabytes needs those bits. Without it, a device's DMA into another device's window is refused by the IOMMU (errata 85), not by the address. **Impact.** The kernel maps devices where enumeration says and recognises a device mapping by its memory type and window; the root complex decodes the windows

it has assigned. Whether other systems split the map by platform-chosen ranges, and whether x86 has a separate port space, are *unverified* here.

## 2.4 Enumeration

**Finding the devices.** The PDP-11 kernel is told where each device is and probes the address (2.1); the VAX kernel scans the slots of its interconnect and reads each one's type; the pdp64 kernel asks every function. **Enumeration** (walking the whole tree and reading each function's configuration space to find out what is there) goes: for every bus, for every device number 0 to 31 and function number 0 to 7, read the vendor ID; if a function answers, read its header type; if it is a bridge, give the bus behind it a number and walk that bus too; if it is an ordinary function, size and place its windows and record it. The pseudocode (the bus numbers follow the `PCI_PRIMARY_BUS`, `PCI_SECONDARY_BUS` and `PCI_SUBORDINATE_BUS` bytes at offsets 0x18 to 0x1A of a bridge's header):

```
next_bus = 1                                # bus 0 is the root complex's own

scan(bus):
    for dev in 0 .. 31:
        for fn in 0 .. 7:
            vendor = cfg_read16(bus, dev, fn, 0x00)
            if vendor == 0xFFFF:            # nothing here
                if fn == 0: break           # no function 0, no card: skip its other functions
                continue
            htype = cfg_read8(bus, dev, fn, 0x0E)
            if (htype & 0x7F) == 1:         # a bridge: a bus lies behind it
                secondary = next_bus++
                cfg_write8(bus, dev, fn, 0x18, bus)          # primary bus
                cfg_write8(bus, dev, fn, 0x19, secondary)    # secondary bus
                cfg_write8(bus, dev, fn, 0x1A, 255)          # subordinate: provisional
                scan(secondary)                              # the recursion
                cfg_write8(bus, dev, fn, 0x1A, next_bus - 1)  # now the real highest number
            else:
                size_and_place_bars(bus, dev, fn)
                add_to_device_list(bus, dev, fn, vendor, cfg_read16(bus, dev, fn, 0x02))
            if fn == 0 and not (htype & 0x80): break         # single-function card
```

(`cfg_read16` is a load from the ECAM address of section 2.2. A real PCI Express root complex and QEMU answer all ones for an empty position (*unverified* on real hardware); the pdp64's root complex does so by decision (2.4.7). The windows of a bridge must also be programmed so that traffic for the functions behind it passes; that step is omitted here.)

**Sizing a window.** The register must say how big its window is, and it is the same register that holds the address. The bits below the size are not storage: they always read as zero. So the system writes all ones, reads the register back, and the bits that stuck show the size (the header comment of `pci_regs.h` says the same: "writing a value of 0xffffffff to the register, and reading it back. Only 1 bits are decoded"; QEMU builds the answer as `~(size - 1)`). The low four bits of a memory BAR are not part of the address (the listing below handles memory BARs only): bit 0 is 0 for memory and 1 for an I/O port, bits 2:1 give the width (00 is a 32-bit address, 10 a 64-bit one that uses the **next** BAR as its high half), and bit 3 says the memory is **prefetchable** (reading it has no side effects, so it can be read ahead).

```
size_bar(bus, dev, fn, index):                      # returns (size, is_64bit)
    reg = 0x10 + 4 * index
    cmd = cfg_read16(bus, dev, fn, 0x04)
    cfg_write16(bus, dev, fn, 0x04, cmd & ~0x2)     # stop the function answering memory while we play
    old = cfg_read32(bus, dev, fn, reg)
    cfg_write32(bus, dev, fn, reg, 0xFFFFFFFF)
    got = cfg_read32(bus, dev, fn, reg)
    cfg_write32(bus, dev, fn, reg, old)             # put it back
    is64 = ((got >> 1) & 3) == 2
    if is64:                                        # the same dance on the high half
        oldh = cfg_read32(..., reg + 4)
        cfg_write32(..., reg + 4, 0xFFFFFFFF)
        goth = cfg_read32(..., reg + 4)
        cfg_write32(..., reg + 4, oldh)
        mask = (goth << 32) | (got & 0xFFFFFFF0)
```

```
        size = (~mask + 1) & 0xFFFFFFFFFFFFFFFF
    else:
        mask = got & 0xFFFFFFF0
        size = (~mask + 1) & 0xFFFFFFFF
    cfg_write16(bus, dev, fn, 0x04, cmd)            # restore the command register
    return (size, is64)

size_and_place_bars(bus, dev, fn):
    index = 0
    while index < 6:
        (size, is64) = size_bar(bus, dev, fn, index)
        if size != 0:
            base = allocate(size, align = size)     # a window starts on a multiple of its own size
            cfg_write32(bus, dev, fn, 0x10 + 4*index, base & 0xFFFFFFFF)
            if is64: cfg_write32(bus, dev, fn, 0x14 + 4*index, base >> 32)
        index += 2 if is64 else 1
    cmd = cfg_read16(bus, dev, fn, 0x04)
    cfg_write16(bus, dev, fn, 0x04, cmd | 0x2 | 0x4) # answer memory (bit 1); may be a bus master (bit 2)
```

The worked example of the figure: reading back 0xFFFFC000 means the low 14 bits are not storage, so the size is $2^{14}$ = 16 KB. The last line sets bit 2 of the command register; until it is set the function may not start a transfer of its own (section 2.5).

**2.4.1** The processor finds the devices by PCI Express enumeration: it reads the configuration space of each function through the root complex, sizes the base-address registers of the function and assigns its windows in device space. *(D)*

**2.4.2** A function is a virtio device when its configuration space says so (the virtio vendor identifier and a device identifier that carries the virtio device type); the capabilities in its configuration space locate the virtio structures in its windows (chapter 5). One transport driver serves every virtio device. *(D)*

**2.4.3** A function may itself be a bus: a PCI Express bridge presents the functions behind it, and a USB host controller (itself a PCI Express function) enumerates its own tree of hubs and devices, so any bus can be attached to the bus and its devices enumerated, to any depth. The kernel's enumeration is one recursive walk. *(D)*

**2.4.4** The bus alerts the processor when a device enters or leaves, by an interrupt; the kernel then rescans the bus, which is also what it does at boot. *(D)*

Open: the mechanism of the alert, and how it is reported for a device behind a bridge (J.2.2).

**2.4.5** The queued work of a departed device stays in memory untouched, so the driver can read, save, resend or fail it. *(D)* Details to be determined.

**2.4.6** A bus reset produces no removal events, so a reset is not mistaken for the removal of every device, and the identity of every function is defined after the bus has lost power. *(D)*

Open: where the configuration space and the windows of the functions sit in the physical address space (J.2.1).

**2.4.7** A read of a configuration position that never held a function completes normally and returns all ones in every byte, so a vendor ID of `0xFFFF` means that nothing is there; it is not a bus fault (the read of other non-existent addresses stays a precise `BUSFAULT`, 6.4.3). *(D)*

**2.4.8** The root complex records such a miss in a sticky bit of its own status register, cleared by writing a one to it, so that a dead link can be told from an empty slot. *(D)*

Open: where that register sits (J.2.5).

**2.4.9** When a device leaves, an access to its windows is at once a precise bus fault, not a read of all ones: a read of the window faults, a posted write is dropped and counted, and the device's in-flight bus transactions are aborted. *(D)* Details to be determined.

*Commentary on 2.4.9.* The PDP-11 kernel was not told when a device left. On a surprise removal PCI Express returns all ones (*unverified*), so every driver would have to treat all ones as a possible error. The pdp64 gives instead the precise bus fault that the instruction set already defines (6.4.3), so a driver has one recovery path, and a rescan on an event makes hot-plug and boot the same code.

*Commentary.* **Chain.** The PDP-11 had no enumeration: devices at switch-set addresses, found by probes such as `rkprobe`. The VAX scanned its interconnect: `probenexi()` in 4.3BSD `autoconf.c` reads each nexus slot's status register (`badaddr` first, then the type field, `NEX_MEM4`, `NEX_MBA`, `NEX_UBA0` and so on), and `unifind()` probes the devices behind each Unibus adapter. PCI added

configuration space and sizing by writing all ones, so a card can be placed anywhere. PCI Express keeps that. The pdp64 uses standard enumeration, so an ordinary card works unchanged and the kernel needs one address, the root complex's. Amiga AutoConfig (each board announces itself at a fixed address and is told where to live) and NuBus (a fixed share of the address space per slot) are other answers (*unverified* here); assignment on demand supports any layout at the cost of a sizing step at boot, a fixed window per slot needs no assignment and limits the slot count. **Where the pdp64 differs.** An empty configuration position reads all ones, as in PCI, by decision and not as a fault (2.4.7). **Impact.** One recursive loop over the functions of each bus; the root complex is a standard one.

*Commentary on 2.4.3.* **Chain.** The VAX did this: the Unibus hung off the 11/780's interconnect through a Unibus adapter, a nexus device whose registers include the map registers for DMA and whose Unibus memory (the devices' registers) appears in a window of its own (`UMEM780(i)` at `0x20100000 + i * 0x40000`, `vaxuba/ubareg.h`, 4.3BSD). PCI-to-PCI bridges nest buses, a USB host controller enumerates a tree of hubs and devices, and Thunderbolt tunnels PCI Express over a cable, and CXL and the device-tree and ACPI descriptions of nested buses are further examples (all *unverified*). The pdp64 allows any bus to be attached behind a bridge. **Trade-off.** Each level adds a mapping of windows and a hop of latency; a bridge's windows are ordinary device windows (2.3.5). **Impact.** A new bus costs one bridge and no change to the processor, the kernel's walk or the driver model; a bridge is a standard PCI Express bridge.

## 2.5 DMA, coherence and the hand-off

**Bus mastering.** On the Unibus any device could become master by winning an NPR grant. In PCI Express a function is a **bus master** (it may originate memory reads and writes of its own, as packets, instead of only answering) once the processor sets bit 2 of its command register; until then it only answers. A DMA transfer is a function sending memory-write packets (to deliver data) or memory-read packets (to fetch it) to addresses in memory the processor gave it. There is no request line, no grant chain and no priority between devices; the switches of the tree carry the packets. The danger is the one the Unibus map registers hinted at: a device that can write anywhere can write into the kernel (section 2.11).

**The hand-off.** Telling each side that the data is ready is the harder half of DMA, and a queue in memory does it (section 2.7). Statement 2.5.3 gives the five steps in order.

**2.5.1** Between processors the data caches are coherent without any software action, by snooping with three states (modified, shared, invalid), for two to four cores. *(D)*

**2.5.2** Between a processor and a DMA device they are not: buffers are mapped uncached and the hand-off is explicit. *(D)*

**2.5.3** The hand-off is five steps: the processor builds the descriptors and data in uncached memory, executes `FENCE`, writes the device's notification address; the device walks the descriptors, moves the data by DMA, writes the result and its used-ring entry, and raises an interrupt; the processor reads the result. *(D)*

**2.5.4** The write that raises a device's interrupt (a message-signalled write) cannot overtake the data writes that precede it on the same path. *(D)*

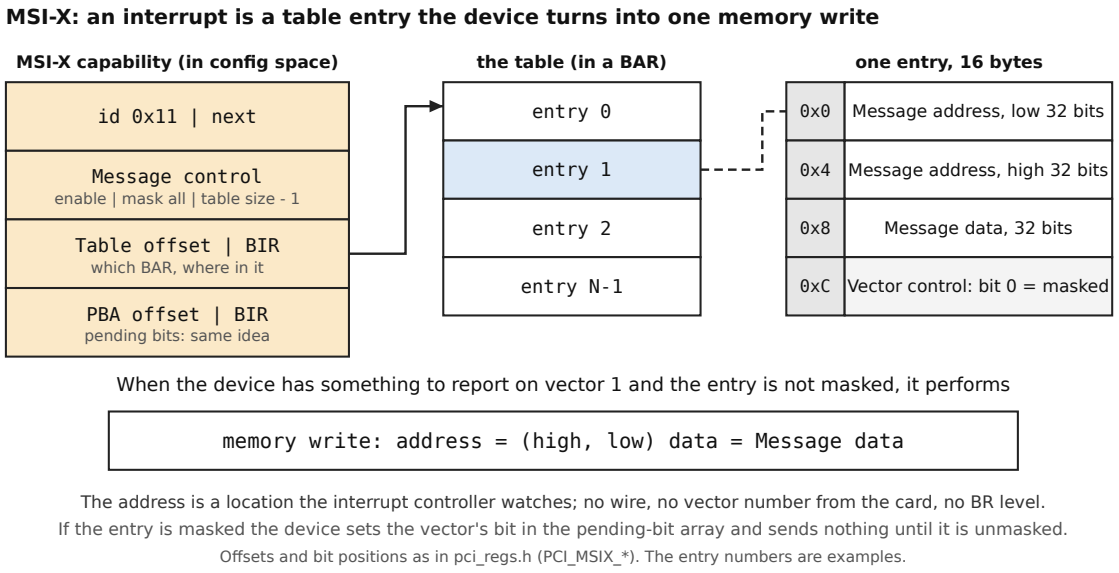
*Commentary.* **Chain.** The RK11 driver of 2.1 contains no cache or ordering step. On the VAX the Unibus adapter had buffered data paths, and a driver had to purge them before reading what a device had written (`UBA_PURGE780` and `UBA_PURGE750` in 4.3BSD `ubareg.h`). That a PDP-11/70's cache watched Unibus writes so that DMA appeared coherent is *unverified* here. Modern x86 DMA is coherent, while ARM and MIPS systems with non-coherent DMA need a kernel layer that places cache maintenance (*unverified*). The pdp64 chooses two mechanisms, each where it is cheap: snooping between cores, which are few, and uncached buffers with an explicit hand-off for DMA, which is large and infrequent. **Trade-off.** The processor reads received data from uncached memory, which is slow. No driver can forget a cache clean or invalidate, because there is none. **Impact.** A snoop port on each data cache for core coherence only; none for DMA.

## 2.6 Interrupts that are writes: MSI and MSI-X

On the Unibus an interrupt was a signal: a BR line went up, the processor granted the bus, and the device put its vector on the data lines. A PCI Express link has no such lines to raise. The first answer, kept for old devices, was four shared **interrupt pins** per slot (`INTA` to `INTD`; the header's **interrupt line** and **interrupt pin** bytes at 0x3C and 0x3D describe them, and bit 10 of the command register turns the function's pin off), which several devices share, so that the handler must ask each in turn whether it was the one. PCI Express emulates the pins with messages for old software (*unverified*).

The answer for new devices is that **an interrupt is a memory write**. A function that wants attention does not raise a signal; it performs a write, of a value the system chose, to an address the system chose, and the part of the machine that

owns that address (the **interrupt controller**: the unit that decides which core runs which handler) takes the arrival of the write as the interrupt. **MSI** (message-signalled interrupts, capability identifier 0x05) gives a function one address and one data value; how it varies the data to give several interrupts is *unverified*. **MSI-X** (capability 0x11, the "extended" form, and the form virtio uses, section 2.7) gives a function a **table** (an array in one of its windows) of entries, up to 2048 of them (the table-size field is 11 bits, `PCI_MSIX_FLAGS_QSIZE` 0x07FF), each with a separate address and data, so that every queue of every device can have an interrupt of its own, aimed at a core of its own.




The capability record (`pci_regs.h`, offsets from the record's start) is: the identifier and next byte; a 16-bit **message control** at 2, whose bit 15 enables MSI-X, bit 14 masks every vector of the function and bits 10:0 hold the table size minus one; a 32-bit word at 4 whose low three bits say which BAR holds the table (the **BIR**, BAR indicator register) and the rest an offset in it; and a 32-bit word at 8 of the same form for the **PBA**, the **pending-bit array** (one bit per vector, set by the function when it wanted to interrupt and could not because the entry was masked). An entry is 16 bytes: the message address, low half at 0 and high half at 4; the message data at 8; and at 12 the **vector control**, whose bit 0 is the **mask** (while it is set the function sends nothing for that vector and records the wish in the PBA).

```
enable_msix(fn, nvec):
    cap  = find_capability(fn, 0x11)
    ctrl = cfg_read16(fn, cap + 2)
    table_bar    = cfg_read32(fn, cap + 4) & 7
    table_offset = cfg_read32(fn, cap + 4) & ~7
    table = bar_base(fn, table_bar) + table_offset
    cfg_write16(fn, cap + 2, ctrl | 0x8000 | 0x4000)     # enable, but mask every vector for now
    for v in 0 .. nvec - 1:
        entry = table + 16 * v
        (addr, data) = interrupt_controller_allocate(core_for(v), handler_for(v))
        mmio_write32(entry + 0,  addr & 0xFFFFFFFF)
        mmio_write32(entry + 4,  addr >> 32)
        mmio_write32(entry + 8,  data)
        mmio_write32(entry + 12, 0)                      # unmask this vector
    cfg_write16(fn, cap + 2, (ctrl | 0x8000) & ~0x4000)  # lift the function-wide mask

# in virtio (section 2.7.4), each queue is told its vector, and the device-wide changes have one too:
write16(msix_config, 0);  write16(queue_msix_vector, 1)    # 0xFFFF means no vector
```

When the device raises vector *v*, it sends a write of `data[v]` to `address[v]`, if the entry is not masked; a handler then learns what happened by reading the used ring (2.7.7) and, for the device as a whole, the interrupt-status structure of the virtio capabilities (type 3, section 2.7.2, which comes next). On the Unibus there were four levels and the RK11's vector was `0220` at level 5, so a handler for a level shared with other devices had to find out which had spoken; here there is a vector for each queue, assigned by software, and no wire.

**2.6.1** A device interrupts by a message-signalled write (MSI-X): the configuration change and each queue have a vector that the driver sets in the common configuration, and the interrupt status structure says why (5.9.1). The standard devices are virtio devices, so this is the interrupt of every one of them, and the interrupt controller must be the target of the writes. *(D)* Details pending (J.6.3).

**2.6.2** The write that raises an interrupt cannot overtake the data writes that precede it on the same path, so a handler that runs because of the interrupt finds the data in memory (2.5.4). *(D)*

**2.6.3** A driver may mask a device's interrupts and poll the used ring instead (2.7.9, 5.9.2). *(D)*

*Commentary.* **Chain.** PDP-11: four BR levels and switch-set vectors, a scarce shared resource (`pdp/scb.s`). VAX: the Unibus adapter latched the vector of each level in a **receive vector register**, one per BR level (`uba_brrvr[4]`, `UBABRRVR_DIV`, `ubareg.h`, 4.3BSD). PCI: four shared interrupt pins per slot, described by the header bytes at 0x3C and 0x3D (`pci_regs.h`). PCI Express adds MSI and MSI-X, an interrupt as a write (`pci_regs.h`, QEMU `hw/pci/msix.c`). virtio gives each queue its own vector. **Trade-off.** A message carries its own vector, so no switch and no assignment table are needed and there can be thousands; the interrupt controller must be an address that many devices write, and a confused device that writes to the wrong address causes an interrupt (the IOMMU of 2.11 can refuse a write to an address that is not allowed). That the x86 local APIC and the ARM GIC's translation service receive such writes is *unverified*. **Impact.** An interrupt routine reads the used ring and the status structure; the table is programmed once at attach.

## 2.7 virtio: one shape for every device

The Unibus gave every device its own registers, and every driver learned them. **virtio** (a specification for devices meant to be imitated in software; the name is from "virtual I/O") gives every device the same few structures in memory, and only the meaning of the requests differs. A **driver** (the program that runs a device) and a **device** (the hardware, or the emulator that pretends to be it) talk by **shared memory** (both sides read and write the same region of memory under a fixed protocol, instead of passing values through registers). The author decided on virtio for every standard device (errata 95, 79, 101): the network card, the graphics device, the audio device and the disk are virtio-net, virtio-gpu, virtio-snd and virtio-blk. One transport driver (2.7.2) and a thin driver per device kind serve the machine. The chain from the RK11: one register set per device (PDP-11); the VAX added a second adapter kind, the Massbus (`NEX_MBA`), beside the Unibus adapter, so its interconnect carried several device interfaces; PCI Express made discovery and addressing uniform and left the device interface to each vendor (NVMe, which errata 75 first chose, is one such interface and may still be attached, errata 101); virtio makes the device interface uniform too. Chapter 5 states the rules; this section teaches the shape.

### 2.7.1 The five ideas, against the RK11

| the RK11 | virtio |
|---|---|
| the kernel is told where the device is | the device is found by enumeration, by vendor and device ID (2.7.2) |
| six registers, each with bits of its own | a standard set of registers of **feature bits** and a **status** byte, the same for every device (2.7.3) |
| one command at a time: load the registers, GO | a **virtqueue**, a queue of commands in memory, so many are outstanding at once (2.7.4) |
| the command is in the registers | the command is a **descriptor chain**, boxes in memory linked like a list (2.7.5) |
| READY bit, or an interrupt, says it is done | the device writes the finished chain into the **used ring** in memory, and may interrupt (2.7.6, 2.6) |

### 2.7.2 Finding a virtio device

A virtio function is recognised by its identifiers: the vendor ID is 0x1AF4 (QEMU's `PCI_VENDOR_ID_REDHAT_QUMRANET`), and its device ID is 0x1040 plus the **device type** (QEMU's `PCI_DEVICE_ID_VIRTIO_10_BASE`). The types are in the Linux header `virtio_ids.h`: network 1, block 2, console 3, GPU 16, sound 25, so virtio-blk is device ID 0x1042, virtio-net 0x1041, virtio-gpu 0x1050 and virtio-snd 0x1059. (These are the "modern" device IDs. An older numbering exists; that it starts at 0x1000 is *unverified*, and it is not used, section 2.7.3.)

The registers are not at fixed places in the window. The function's **vendor-specific capabilities** (capability identifier 0x09 in the list of section 2.2) say where each structure is. Each record is the 16 bytes of `struct virtio_pci_cap`: the identifier, the next-record offset, the record length, a `cfg_type` byte that says which structure it describes, a `bar` byte (which of the six windows), an `id` byte, two bytes of padding, a 32-bit `offset` into that window and a 32-bit `length`. The `cfg_type` values are 1 common configuration, 2 notifications, 3 interrupt status, 4 device-specific configuration, 5 a PCI configuration access window, 8 shared memory, 9 vendor data (`virtio_pci.h`); a driver for the pdp64 needs 1 to 4. The notification record (`struct virtio_pci_notify_cap`) is four bytes longer: it ends with a 32-bit `notify_off_multiplier`.

```
find_virtio_structures(fn):
    cap = cfg_read8(fn, 0x34)                       # head of the capability list
    while cap != 0:
        if cfg_read8(fn, cap + 0) == 0x09:          # vendor-specific: could be virtio
            type   = cfg_read8 (fn, cap + 3)
            bar    = cfg_read8 (fn, cap + 4)
            offset = cfg_read32(fn, cap + 8)
            length = cfg_read32(fn, cap + 12)
            where[type] = bar_base(fn, bar) + offset        # the physical address of the structure
            if type == 2: notify_multiplier = cfg_read32(fn, cap + 16)
        cap = cfg_read8(fn, cap + 1)                # follow "next"
```

### 2.7.3 The handshake: status and features

The **common configuration** structure is a block of registers, the same in every device; chapter 5 (5.4.1) lists them with their offsets, from `device_feature_select` at 0x00 to `queue_device` at 0x30 (the Linux header `virtio_pci.h` spells the last three `queue_desc_lo`, `queue_avail_lo` and `queue_used_lo` with `_hi` halves, and the feature registers `guest_feature_select` and `guest_feature`; the offsets follow from its field order). Three are the handshake.

`device_status` is one byte whose bits the driver sets in order, each meaning "I have got this far": ACKNOWLEDGE (1, "I have seen you"), DRIVER (2, "I know how to drive you"), FEATURES_OK (8, "I have chosen my features"), DRIVER_OK (4, "I am running"); the device may set NEEDS_RESET (0x40) and the driver FAILED (0x80) (`virtio_config.h`). Writing 0 resets the device (*unverified*: the Linux headers do not say). The RK11's GO and READY, spread over a conversation, correspond to these bits.

**Feature bits** let two sides that were not written together agree. The device offers a 64-bit set of **features** (optional abilities, one bit each; for example indirect descriptors, bit 28), read 32 bits at a time through a window selected by `device_feature_select`; the driver writes back the subset it will use, and nothing outside that subset is used. A device can add a feature without breaking an old driver, which never asks for it. One feature is required of every device the pdp64 drives, `VIRTIO_F_VERSION_1` (bit 32: "modern layout"; errata 95 and 5.4.4). Two more matter to the processor: `VIRTIO_F_ACCESS_PLATFORM` (bit 33: device addresses go through the platform's translation, section 2.11) and `VIRTIO_F_ORDER_PLATFORM` (bit 36: the device respects the platform's ordering rules). Open: whether the pdp64 declares them (J.2.4).

```
init_device(fn):
    status = 0;               write_status(0)                   # reset
    write_status(status |= ACKNOWLEDGE)                         # 1
    write_status(status |= DRIVER)                              # 2
    wanted = (1 << 32) | class_driver_features                  # VERSION_1 and what this driver uses
    offered = read_features_64()                                # 2 reads through device_feature_select
    if (offered & (1 << 32)) == 0: fail()
    write_features_64(wanted & offered)                         # 2 writes through the driver's feature-
select window
    write_status(status |= FEATURES_OK)                         # 8
    if (read_status() & FEATURES_OK) == 0: fail()               # the device may refuse the subset
    for q in 0 .. num_queues - 1: setup_queue(q)                # section 2.7.4
    write_status(status |= DRIVER_OK)                           # 4: go
```

### 2.7.4 A virtqueue: three areas of memory

A **virtqueue** (a queue of requests that lives in ordinary memory, shared by the driver and the device) is three arrays, each in memory the driver allocates: the **descriptor table** (the requests), the **available ring** (the driver's list of which requests are new) and the **used ring** (the device's list of which are finished). The size, a power of two, is a number of entries chosen by the driver up to the device's maximum. Sizes: a descriptor is 16 bytes aligned to 16, the available ring is aligned to 2 and the used ring to 4 (the `VRING_*_ALIGN_SIZE` of `virtio_ring.h`); all fields are little-endian (5.3.4). The driver tells the device where the three are by writing their physical addresses into `queue_desc`, `queue_driver` and `queue_device` (offsets 0x20, 0x28 and 0x30) after selecting the queue with `queue_select`, and then sets `queue_enable`.

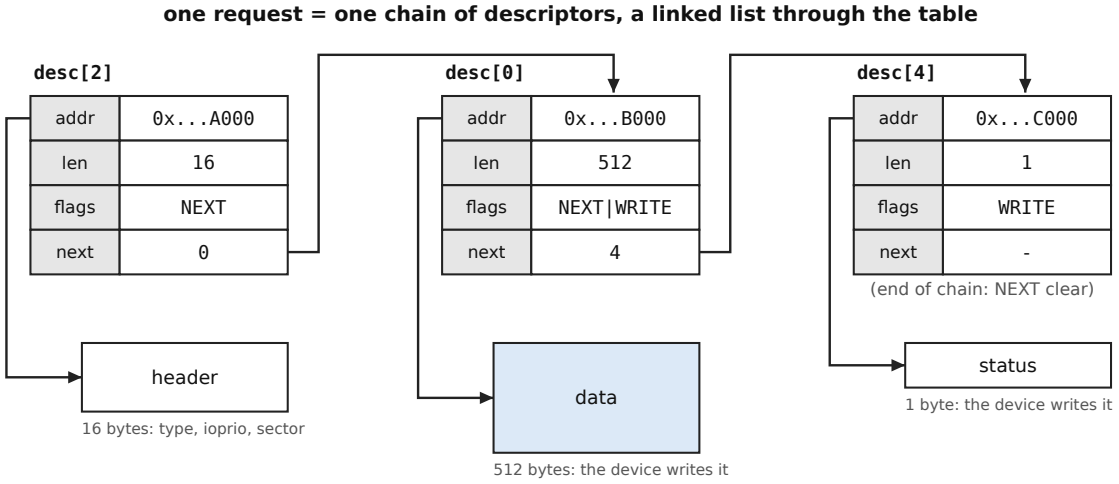
```
setup_queue(q):
    write16(queue_select, q)
    n = min(read16(queue_size), 256)                # the device proposes a size; the driver may lower it
    write16(queue_size, n)
```

```
desc  = alloc_uncached(16 * n, align = 16)       # the descriptor table
avail = alloc_uncached(6 + 2 * n, align = 2)     # flags, idx, ring[n], one optional word
used  = alloc_uncached(6 + 8 * n, align = 4)     # flags, idx, ring[n] of 8-byte elements
zero(desc, avail, used)
write64(queue_desc, desc);  write64(queue_driver, avail);  write64(queue_device, used)
write16(queue_msix_vector, vector_for(q))        # section 2.6
write16(queue_enable, 1)
notify_addr[q] = where[2] + read16(queue_notify_off) * notify_multiplier    # the doorbell of this queue
free_head = 0; link all n descriptors into a free list through their `next` fields
avail_idx = 0; last_used = 0                     # the driver's private counters
```

(The queue memory is in uncached memory on the pdp64, 2.5.2, so no cache instructions appear anywhere. The sizes 6 + 2n and 6 + 8n follow from the structures `struct vring_avail` and `struct vring_used` of `virtio_ring.h`, each two 16-bit fields followed by the ring, and a final optional 16-bit field, section 2.7.9.)

### 2.7.5 The descriptor, and the chain

A **descriptor** is 16 bytes, four fields (`struct vring_desc`): `addr`, a 64-bit physical address of a buffer; `len`, a 32-bit length in bytes; `flags`, 16 bits; and `next`, a 16-bit index of another descriptor. The flags are `NEXT` (1: the request continues in the descriptor `next` names), `WRITE` (2: the device will write into this buffer; absent, the device only reads it) and `INDIRECT` (4: the buffer is itself a table of descriptors). One request is a **chain** (a sequence of descriptors, each pointing to the next by index, ending at one without `NEXT`): in the language of a textbook, a linked list whose nodes are the 16-byte descriptors, whose pointers are table indices and whose payloads are the buffers.

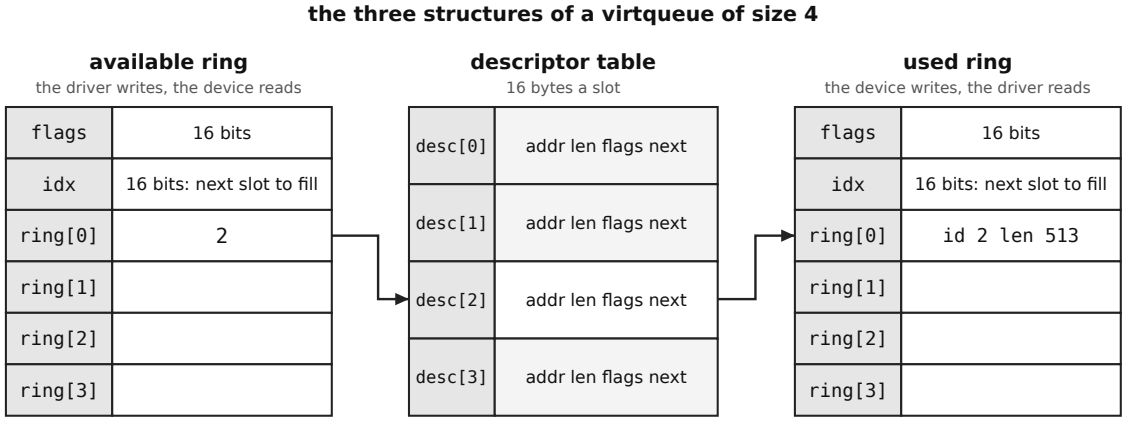



The cards are not in the table in order: the chain 2, 0, 4 is the order of the links, as in any linked list. Indices are examples.

The figure is a virtio-blk read of one sector. The first box is the header the driver filled in (`struct virtio_blk_outhdr`: a 32-bit request type, a 32-bit priority, a 64-bit sector number counted in 512-byte units; type 0 is a read, 1 a write, 4 a flush: `virtio_blk.h`); it has no `WRITE` flag because the device only reads it. The second is the 512-byte buffer for the data, with `WRITE`, because for a read the device fills it. The third is one byte the device writes with the result (0 is OK, 1 an I/O error, 2 unsupported). A chain may use any free slots of the table in any order; the free ones are kept on a list of the driver's own. A buffer that straddles a page is simply two descriptors (5.10.2). Compare the RK11: the whole request was in registers, and could not be more than one; here it is memory the device reads, and the table holds many.

### 2.7.6 The available ring and the used ring

The two rings are arrays of 16-bit and 32-bit numbers with an index each.




The driver offers a chain by writing its head into ring[idx mod size] and then adding 1 to idx.
The device reports it done by writing (head, bytes written) into the used ring and then adding 1 to its idx.
Each side keeps its own private count of how far it has read the other side's ring. Indices are examples.

The **available ring** (`struct vring_avail`) is a 16-bit `flags`, a 16-bit `idx` and `ring[]`, 16-bit entries. The driver puts the index of the **head** (first descriptor) of a chain into `ring[idx mod size]` and then adds 1 to `idx`; `idx` only ever increases and wraps at 65536, so that "how many have been offered in all" is the difference of two readings and no extra state is kept. The **used ring** (`struct vring_used`) is the mirror: `flags`, `idx` and `ring[]` of **used elements** (`struct vring_used_elem`), each a 32-bit `id` (the head of the finished chain) and a 32-bit `len` (the number of bytes the device wrote into the chain's buffers). The device writes the element and then adds 1 to its own `idx`. Each side keeps a private count of how far it has read the other's ring (`last_used` for the driver); each ring has one writer, so neither side waits for the other and no lock is needed: the one place a lock would be needed on the Unibus, two parties writing one location, does not occur (inference from the single-writer rule). This is the **producer-consumer ring** (a fixed circular array with one writer and one reader, each owning one index) that the 4.3BSD DEUNA's transmit and receive rings already were, except that the "own" bit of each DEUNA entry is replaced by the two indices.

### 2.7.7 Posting a buffer and reaping the completions

```
post(q, head):                                      # head: first descriptor of a built chain
    avail.ring[avail_idx mod n] = head              # 1. name the chain
    FENCE                                           # 2. the descriptors and the ring entry are visible...
    avail_idx = avail_idx + 1 (mod 65536)
    avail.idx = avail_idx                           # 3. ...before the device is told there is one more
    FENCE
    if (used.flags & NO_NOTIFY) == 0:               # the device may ask not to be rung (section 2.7.9)
        mmio_write16(notify_addr[q], q)             # 4. the doorbell

reap(q):                                            # called from the interrupt routine, or when polling
    while last_used != used.idx:                    # the device has added finished chains
        FENCE                                       # read the index first, then the element and the data
(5.10.3)
        e = used.ring[last_used mod n]              # e.id = head of chain, e.len = bytes written
        complete(e.id, e.len)                       # the class driver reads status, frees the chain
        free_chain(e.id)
        last_used = last_used + 1 (mod 65536)
```

The doorbell is an ordinary 16-bit write to a notification address: the address is the notification area's base plus `queue_notify_off` times the `notify_off_multiplier` (QEMU decodes the queue from the address, `addr / multiplier`; what value the specification asks the driver to write is *unverified*: the code above writes the queue number, which QEMU ignores). The `FENCE` between step 1 and step 3 is the pdp64's statement 2.8.2; the processor owes nothing else (2.8.1).
A complete read of one sector, from the driver's side, putting the pieces of this section together:

```
blk_read(sector, buf):                              # buf: a physical address of 512 bytes
    h = alloc_hdr();  h.type = 0; h.ioprio = 0; h.sector = sector      # 16 bytes
    s = alloc_status_byte()
```

```
    d0 = get_free();  d1 = get_free();  d2 = get_free()
    desc[d0] = { addr: h,   len: 16,  flags: NEXT,         next: d1 }
    desc[d1] = { addr: buf, len: 512, flags: NEXT | WRITE, next: d2 }
    desc[d2] = { addr: s,   len: 1,   flags: WRITE,        next: 0  }
    post(q, d0)
    wait_for_completion(d0)                          # polled or interrupt-driven
    return status_byte(s) == 0
```

### 2.7.8 The polled driver, two ways

The two loops side by side, the RK11 of section 2.1 and the same job as virtio-blk:

```
RK11 (the PDP-11 way)                       virtio-blk (the virtio way)
------------------------------------------  ---------------------------------------------
write RKDA, RKBA, RKWC                      build 3 descriptors in memory (blk_read above)
write RKCS = READ | GO                      post(q, head)           # one doorbell write
while (RKCS & READY) == 0 : spin            while last_used == used.idx : spin   # reads memory
if RKCS & ERROR : read RKER, reset          status = the byte the device wrote in d2
one command in flight                       up to `size` commands in flight
the registers are this device's             the structures are every device's
```

The polling loop differs only in what it reads: a word of memory, the used index, where the RK11 loop reads a register. The read is an ordinary load as the page's memory type says, not a transaction on the device's link, which is also why a driver can poll the used ring while the device is on the other side of a bridge (inference: the ring is memory the driver owns).

### 2.7.9 Suppressing notifications and interrupts

Doorbells and interrupts cost time on both sides, so each side can ask the other to stay quiet. Both flags are in the rings' `flags` fields and both are **hints** (a request the other side may, for a short time, not honour; the driver must work either way). The device sets `VRING_USED_F_NO_NOTIFY` (bit 0 of `used.flags`) to say "do not ring the doorbell; I am looking at the available ring anyway". The driver sets `VRING_AVAIL_F_NO_INTERRUPT` (bit 0 of `avail.flags`) to say "do not interrupt; I am polling". A finer scheme exists when feature bit 29 (`VIRTIO_RING_F_EVENT_IDX`) is agreed: each side writes into an extra 16-bit word at the end of its ring the **index** at which it wants to be woken, and the other side then rings or interrupts only when it crosses that number (`virtio_ring.h`: the driver "publishes the used index for which it expects an interrupt at the end of the avail ring"; the host "publishes the avail index for which it expects a kick at the end of the used ring"). Open: whether the pdp64 uses this feature (J.2.4). A driver may mask a device's interrupts and poll the used ring (2.6.3).

## 2.8 What the processor owes the bus

**2.8.1** The processor owes the bus exactly three things: a memory type per page, a `FENCE` instruction whose meaning against PCI Express is defined, and atomics that can reach device memory. *(D)*

**2.8.2** A driver can be sure a descriptor is visible before it notifies the device: it builds the descriptor and publishes it in the queue, executes `FENCE`, and writes the queue's notification address (the doorbell). *(D)*

*Commentary.* **Chain.** The RK11 driver of 2.1 contains no ordering step; its writes reach the device in program order over one bus (*unverified* for models with a cache). PCI Express posts writes, and the driver must order them (2.2); multi-core machines add the same need between cores. ARM's `DMB`/`DSB` and device-memory types, and RISC-V's `FENCE` with I/O bits, play the same part (*unverified*). The pdp64's contract has three items: a memory type per page, a `FENCE` whose meaning against PCI Express is defined, and atomics that reach device memory. A virtio device is written against the virtio specification and these three promises alone, without knowing the processor's pipeline. **Impact.** One `FENCE` before the doorbell is the whole ordering rule; `FENCE` is a compiler intrinsic; the memory type is two bits per page-table entry (6.7.1) and the bus honours it.

## 2.9 When an access fails

**2.9.1** A read or instruction fetch of non-existent memory is a precise abort with cause `BUSFAULT`, at the faulting instruction. *(D)*

**2.9.2** A posted write that times out, and an uncorrectable memory error, are machine checks: imprecise and not restartable. A `FENCE` before a probe makes even a posted write precise at a known point. *(D)*

**2.9.3** `PROBER` and `PROBEW` test whether an address can be read or written without faulting. *(D)*

**2.9.4** Resetting the bus is a write to a register of the root complex, not an instruction. *(D)*

Open: where that register sits (J.2.5).

*Commentary.* **Chain.** On the PDP-11 a read of a missing address takes the bus-error trap through vector 4 (`T_BUSFLT`, `pdp/scb.s`), which the kernel could not restart; V7's `nofault` idiom (`m40.s`: set `nofault`, touch the address, clear it) existed to probe for devices without dying. The pdp64 makes the read timeout a precise, restartable abort, and `PROBER` and `PROBEW` do what the idiom did; a driver handles a missing device in a few lines of ordinary fault handling. x86 machine checks and ARM's `SError` correspond to the imprecise class, and PCI returns all ones for a read of a removed device (both *unverified*), which every driver must learn to distrust; the pdp64 returns all ones only for an empty configuration position (2.4.7). **Impact.** A timeout counter in the bridge and a way to make a read abort precise.

## 2.10 Atomics on the bus

**2.10.1** The atomics are defined on a monitor: a load-linked takes a reservation, a write by any initiator clears it, and a store-conditional succeeds only if it is intact; the Unibus's `DATIP`/`DATO` locked pair is not an architectural primitive. *(D)*

**2.10.2** A device that writes memory shared with a core takes part by clearing reservations. *(D)*

*Commentary.* **Chain.** PDP-11: the locked read-modify-write `DATIP`/`DATO` (*unverified* here; `decisions-pending.md` records it). VAX: interlocked instructions (`BBSSI`, `BBCCI`, `ADAWI`, `INSQHI`, checked in `decisions-pending.md` against the VAX-11 Architecture Reference Manual). Both lock; a lock is simple and a bottleneck as cores multiply. RISC-V `LR`/`SC` and AArch64 exclusive loads and stores use a monitor, which maps onto AXI exclusive access, at the cost of forward-progress rules; large systems also have atomics that perform the operation at the memory (all *unverified* here). The pdp64 chose the monitor (errata 45). **Impact.** Lock words and lock-free structures use the atomics of chapter 7; C11 atomics lower to them; the hardware needs a reservation monitor and an exclusive access, not a bus lock.

## 2.11 The IOMMU: a page table for each device

A device that is a bus master (section 2.5) can write wherever its packets say, and a descriptor is only an address the driver gave it. If the driver is wrong, or the device is, or the device has been modified by an attacker, that address can be the kernel's. The chain: on the PDP-11 the map registers of 2.1 (`UBMAP` at `0170200`) existed so that an 18-bit address could reach more memory, and as a side effect limited what a device could reach; the VAX Unibus adapter kept them (`ubareg.h`). The **IOMMU** (input-output memory management unit: a translator between the device's addresses and memory's addresses, with a page table of its own for each device, as the processor's MMU has for each program) makes the limit the purpose. A device is given **device addresses** (also called I/O virtual addresses) in its descriptors; the IOMMU walks that device's table to turn each into a physical address, and refuses the access if the table has no entry or the entry forbids that kind of access. The table is per device (the unit is a **domain**: a set of mappings that one or more devices share), so a network card can reach its buffers and nothing else.

An IOMMU is part of the design (errata 75). The five recommendations of the synthesis in `decisions-pending.md` (F25) are accepted (errata 85): translation is on by default; each device is its own domain; descriptors contain no kernel pointers (a device never sees an address that is useful to an attacker as the address of kernel data); the mappings of the rings, which stay put for the life of the queue, are permanent; and a small buffer is **bounced** (copied into a buffer mapped for the device, so the device cannot reach the page that holds other data). The IOMMU's leaf entry shares bits 0 to 39 with the processor's page-table entry, and bits 40 to 63 are free (errata 85).

Without translation a device addresses all of memory, and a user-space driver can direct its device to write anywhere in memory.

Open: the format of the device table, where the unit sits in the machine, how a fault is reported and virtualization (J.2.3).

With no address bit that says "device" (errata 97), another device's window is an ordinary address, so the IOMMU also refuses a device's DMA into another device's window (2.3.5). In virtio the feature bit `VIRTIO_F_ACCESS_PLATFORM` (33) states that the addresses in a device's descriptors are subject to the platform's translation; (2.7.3).

## 2.12 What was checked, and what was not

The specifications of PCI Express and virtio are not on this machine. What the facts rest on:

- **Linux headers** (`/usr/include/linux/pci_regs.h`, `virtio_pci.h`, `virtio_config.h`, `virtio_ring.h`, `virtio_blk.h`, `virtio_ids.h`): configuration-space header offsets and the command, status and header-type bits; base-address register fields and the all-ones sizing rule; capability-list fields; the MSI-X record, table entry, mask and size fields; link speed and width codes; the virtio

vendor-specific capability layout and `cfg_type` values; the common configuration field order (offsets computed from it); status bits; feature bits 28, 29, 32, 33 and 36; the descriptor, available ring and used ring, their flags and alignments; the virtio-blk header, request types and status values; the device types.

- **QEMU sources** (`<work>/qemu-src`): vendor ID 0x1AF4 and device-ID base 0x1040; the ECAM shifts (`pcie_host.h`); the doorbell decoding (`virtio_pci_notify_write`); MSI-X entry layout and mask handling; the BAR sizing mask; the tree vocabulary of `docs/pcie.txt`.
- **2.11BSD** (`<work>/facetlang/unixtree/2.11BSD/sys`): the RK11 register structure and bits (`pdpuba/rkreg.h`), `rkstart` (`pdpuba/rk.c`), `rkprobe` (`autoconfig/rkauto.c`), the RK11 at `0177400` (`pdpstand/rk.c`), its vector `0220` at level 5 and vector 4 as the bus error (`pdp/scb.s`), `br4` to `br7`, the UNIBUS map and `mapalloc` (`h/uba.h`, `pdp/machdep.c`), the console at `0177560` and its `dlrbuf` (`pdp/cons.c`), `lpcs/lpdb` (`pdpuba/lp.c`), the soft Unibus map `sofub_alloc` and `sofub_relse` for 22-bit Q-bus machines (`pdp/machdep.c`), the `Q22` and `UNIBUS_MAP` options and the model lists (`doc/2.10/setup.2.10/a.t`), and the Q-bus devices named in `intro(4)` and `qe(4)`.
- **DEC handbooks** (bitsavers.org `pdf/dec/pdp11/handbooks/`, scans with a text layer, read in the PDF; cited by printed page): *PDP-11 Bus Handbook* 1979 (`PDP-11_Bus_Handbook_1979.pdf`, part 1 the Unibus, part 2 the LSI-11 bus: Unibus I/O page and 18-bit address p. 7, Unibus lines p. 21, the LSI-11 bus lines p. 97, daisy-chain grants pp. 97, 104-105, 115-116, cycles and addressing pp. 106-108, `BBS7` and the I/O page p. 107-108, parity pp. 113-114, interrupts pp. 117-122); *Micro/PDP-11 Handbook* 1983-84 (EB-24944-18: the term Qbus p. 11, the PDP-11/03 and 11/23 history p. 19, the Q22 backplane label p. 28, the extended bus pp. 247-248, cycles including `DATBI`/`DATBO` pp. 251-253, block mode pp. 263-267); *LSI-11 Processor Handbook* 1975-76 (the multiplexed lines and daisy-chained grants p. 1-3). The Computer History Wiki (gunkies.org) page on the QBUS was read as a pointer only and nothing here rests on it alone.
- **4.3BSD** (`unixtree/4.3BSD/usr/src/sys`): `nexus.h` (`NEX780`, `NEXSIZE`, the nexus types), `autoconf.c` (`probenexi`, `unifind`, `badaddr`), `ubareg.h` (`UMEM780`, `UBA_PURGE780`, `uba_brrvr`), `if_de.c` (the DEUNA ring ownership flags). V7 `m40.s` for `nofault`.
- **Unverified** (general knowledge): `MSYN`/`SSYN` and NPR priority above interrupts (the Unibus side; the Unibus grant passing and 18 address lines and 8 KB I/O page are checked, BH pp. 7, 18, 21); the locked `DATIP`/`DATO` as hardware; lane counts beyond x8; the kinds of TLP, posting and completions; the negotiation of width and speed; the dates and lineage of PCI and PCI Express; that an empty configuration position reads all ones on real hardware; PCI Express emulation of the interrupt pins; the way MSI varies its data; that writing 0 to `device_status` resets; the value the specification asks to be written to a doorbell (QEMU ignores it); that the specification asks for a barrier between publishing the available index and reading the used flags; the older virtio numbering; PCI Express hot-plug signalling; Amiga AutoConfig, NuBus, Thunderbolt; the cache behaviour of PDP-11 models; the architectures named in the commentaries as “modern comparables”.

# Chapter 3 — Graphics: the terminal and its devices

**The pdp64's display is meant to bring back what the Blit terminal and the Commodore 64 had, text and graphics on the screen at once, set by simple stores to memory, at a modern colour depth and screen size; its output is HDMI or another standard graphics output (errata 105), and its standard video device is virtio-gpu (errata 79).** The chapter is written for a reader who knows the PDP-11 and its teletype, or who is learning computing from the start, and who has met nothing invented after the 1970s. It follows the order in which the ideas arrived, because each one answers a complaint about the one before:

| Stage | What it contributed | Section |
|---|---|---|
| PDP-11 teletype (KL11) | a stream of characters, one line at a time | 3.1 |
| VT52, VT100 (the author's "vt55") | a screen the program can address by position, through escape codes | 3.2 |
| Blit (Bell Labs, 1980s) | a bitmap in the address space, the bit-block-transfer, a terminal that runs downloaded programs | 3.3 |
| Commodore 64 (1982) | text, bitmap, sprites and sound together, all set by pokes | 3.4 |
| VGA, SVGA, VESA | memory-mapped text and pixels again, but planes, banks and no sprites | 3.5 |
| GPU | vertices, shaders and command buffers; the text path of every terminal emulator | 3.6, 3.7 |
| pdp64 display | the Blit and the C64 at a modern depth, on a standard output | 3.8 to 3.13 |

After the terminal come the problems that made graphics programming annoying (3.14), tearing (3.15), the C128 (3.16), audio (3.17) and windows (3.18). The PDP-11 had a teletype and nothing else of this chapter; where a later topic has no PDP-11 antecedent the text says so. A fact that could not be checked on this machine is marked *unverified* and listed in 3.19.

*How to read this chapter.* A term is defined where it first appears, as the technical word and a plain restatement. In the boxes-and-arrows figures a box is a piece of memory and an arrow an address or index stored in it. Code blocks are pseudocode (C-like; `poke(address, value)` and `peek(address)` are an ordinary store and load, as in chapter 2) unless marked PDP-11 assembly. The numbered statements are the rules; the history and explanation are unnumbered prose, as in chapter 2.

## 3.1 The teletype: one line at a time

A PDP-11 in 1975 talked to a person through a **teletype** (a keyboard and a printer on one machine, joined to the computer by a serial line: characters go one after another along a single wire pair, each as a short burst of bits). The machine knew nothing about the page. A character went out and the printer put it to the right of the last one; a carriage-return character moved the print head back to the left margin and a line-feed character moved the paper up one line. Nothing could be erased, because the ink was on paper. This is **line-at-a-time output**: the program's only notion of position is "the end of what I have printed".

The hardware was a Unibus device like those of chapter 2: a few registers. The console of 2.11BSD is a KL11/DL11, declared in `sys/pdp/cons.h` as four words (receive status, receive buffer, transmit status, transmit buffer) at address 0177560 (`cnaddr` in `cons.c`); the transmitter-ready bit of the transmit status word is `DLXCSR_TRDY`, 0200 (both checked in the 2.11BSD tree on this machine, where the files are stored gzipped). Printing a line was a loop that waits for ready and stores a byte:

```
print_line(string):
    for each byte c in string:
        while (peek(XCSR) & TRDY) == 0:     # the printer is slow: ten characters a second or so
(unverified)
            wait
        poke(XBUF, c)
    poke(XBUF, CR); poke(XBUF, LF)          # carriage return, line feed
```

and in PDP-11 assembly, with the console's register addresses (transmit status 0177564, transmit buffer 0177566, the third and fourth words of the structure; the loop is in the style of the `rk` driver of chapter 2, not copied from a driver):

```
        MOV  #msg, R0
1$:     MOVB (R0)+, R1        ; next character; R0 now points at the one after
        BEQ  3$               ; a zero byte ends the string
2$:     TSTB @#177564         ; XCSR: bit 7 is READY
        BPL  2$               ; plus: not ready, look again
        MOVB R1, @#177566     ; XBUF: this sends it
        BR   1$
3$:
```

Everything a program can say to such a device is a stream of characters. The Unix **tty** (short for teletype: the device file that a program reads and writes characters from and to) is therefore a stream, and the programs written for it, the shell, the editor `ed` and the compiler's messages, think in lines. The stream has no way to say "put this here", which matters as soon as the printer is replaced by a screen.

## 3.2 The glass teletype and its escape codes

A **video terminal** is a keyboard and a screen on the end of the same serial line. At first it imitated the teletype (the "glass teletype": characters appear at the bottom and the page scrolls up). Then it did what paper could not: move the cursor to any place and change what was there. The program asked for this with **escape sequences** (characters that the terminal obeys instead of displaying, introduced by the escape character, code 27, written ESC), so the stream carries both text and commands.

Two generations matter here. The DEC VT52 is the textbook example of the problem, and the author's chat of 2026-06-18 (`c64stuff.txt`, line 4888, the assistant's words) records its cursor-to-position code: ESC Y, then a row character and a column character, each with 32 added so that the numbers arrive as printable characters. In pseudocode:

```
goto(row, col):
    send(ESC); send('Y'); send(row + 32); send(col + 32)   # the VT52 form, as recorded in the chat
send_string("Hello")                                       # the text appears at the cursor
```

The later DEC VT100 and the ANSI standard that followed it use ESC, an opening square bracket, numbers and a letter: `ESC [ 12 ; 40 H` moves the cursor to row 12, column 40, and `ESC [ 31 m` makes what follows red (these two sequences are general knowledge, *unverified* here). The chat records only that "VT100/ANSI" and the "xterm-256color" subset are what ordinary programs speak (`c64stuff.txt`, lines 218 and 4875).

The author's request names a "vt55" between the teletype and the Blit. No text in the repository mentions a VT55; the records name the VT52 and the VT100, so the chapter describes those. (The DEC model of that number was a graphics terminal, which would fit the author's order; that is general knowledge and *unverified*.)

The escape-code terminal gave a **character-cell display** (a screen that is a grid of cells, 80 columns by 24 rows being the usual size, each holding one character) that a program could address by position. It gave no control over the shape of a character, no drawing, no mix of fonts, and it kept a slow serial line between the program and the screen, speaking escape codes that differed from one make to the next. Unix answered the variety with a database of terminal descriptions (**termcap**, then **terminfo**: a file saying, for each make of terminal, which bytes mean "move the cursor") and a library (**curses**: a program that works out the fewest bytes to send to turn the screen as it was into the screen as wanted). *The UNIX-HATERS Handbook* complains of this (chapter 6, "Original Sin", recorded in `docs/kernel-pain-points.md`, section 14G: "Unfortunately for us, Unix was designed in the days of teletypes"). Each change to a cell cost several bytes of command over a slow line; the Blit answered that.

## 3.3 The Blit: a bitmap, and the blit

The Bell Labs terminal called the **Blit** (Rob Pike, Bart Locanthi and John Reiser; the repository records their papers as *Blit 1983* and *Tradeoffs 1985* in `docs/kernel-pain-points.md`, section 14I) took the opposite course. Instead of obeying commands about a grid of characters it had a **bitmap display** (a screen in which every dot, every pixel, is one or more bits in memory, so a program draws anything by changing bits) and a Motorola 68000 processor at 8 MHz to do the drawing: "256K bytes of memory dual-ported between an 800×1024×1 bit display and a Motorola MC68000 microprocessor". The screen memory was in the processor's address space like any other memory, and a program changed the picture by storing into it.

A bitmap is a picture stored row after row (the figure in 3.5 shows it). The operation that makes it usable is the **bit-block-transfer**, written **bitblt** and said "bit blit": copy a rectangle of bits from one place to another, combining each source bit

with the destination bit by a chosen rule, the **raster operation** (RasterOp: the bit-by-bit rule that gives the result, such as "replace", "OR" or "exclusive-OR"). A rule of two inputs has sixteen possible forms; the Blit authors found that four sufficed (store, OR, clear, XOR), and XOR dominated its use, because drawing a shape twice with it erases it again, so a cursor moves without a saved copy of what was beneath it. With bitblt a program scrolls a window (copy the rectangle up by one line of text), moves a window (copy it), draws a letter (copy the letter's rectangle from a stored font to the screen) and fills an area. In pseudocode in the style of the Blit's library:

```
bitblt(source_bitmap, source_rect, destination_bitmap, destination_point, operation)
bitblt(font, rect_of('A'), screen, (x, y), OR)       # draws an A with its top left at (x, y)
bitblt(screen, window_rect, screen, window_rect + (0, -line_height), STORE)   # scroll one line
```

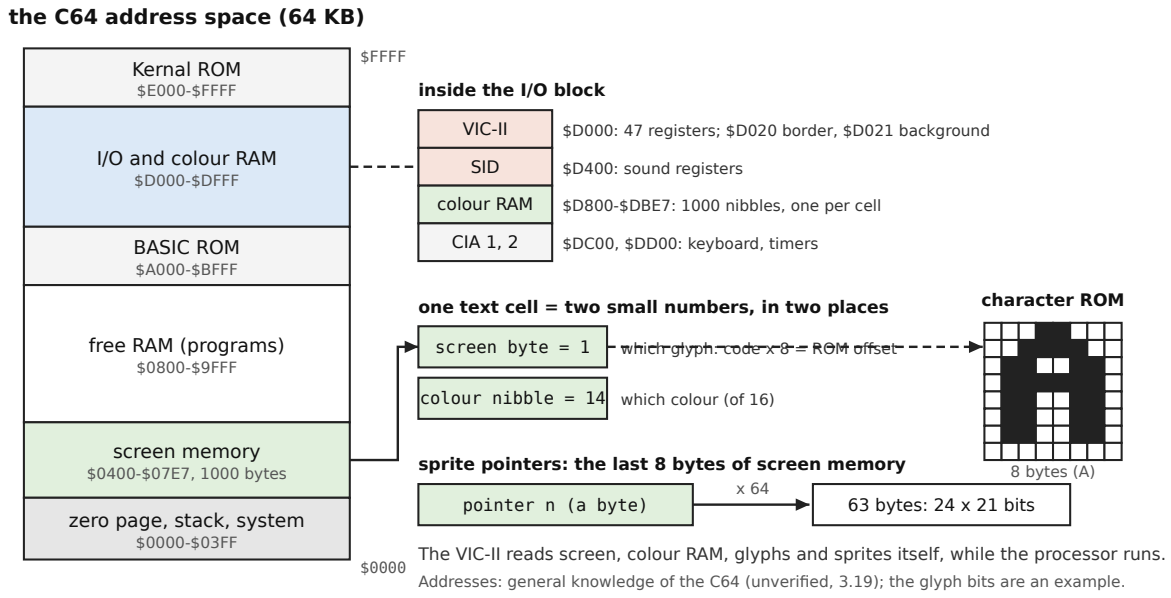
The Blit was a small computer, not a protocol. Its authors give the reason: "To draw graphics in the windows, sending escape sequences is traditional but makes poor use of the processing power of the terminal"; an earlier terminal that interpreted escape sequences failed because "too much state information was maintained by the terminal and guessed at by the Unix program". The host sent it programs over an RS-232 serial line, and the authors name that line as the weak point: "The greatest problem, though, is certainly the low bandwidth." The pdp64 keeps the framebuffer in the address space (errata 66). The authors' advice in 1985 about the blit was that it be "transparent, so it can be bypassed".

On the author's account the Blit inspired rio (Plan 9) and NeWS (Sun's); the sources confirm the line from the Blit's `mux` through 8½ to rio. Section 3.18 says what they were.

The Blit drew its cursor in software, in XOR, redrawing it at 60 Hz (`kernel-pain-points.md`, section 14I); other machines had **sprites** (small pictures that the display hardware overlays on the screen at a position the program gives, without touching the screen memory beneath). The next machine had eight of them.

## 3.4 The Commodore 64: text, graphics, sprites and sound together

The Commodore 64 (1982) was a home computer with a 6510 processor (a 6502 with a few input and output lines added) and two custom chips, the **VIC-II** (video interface chip) and the **SID** (sound interface device); every figure in this section is general knowledge, *unverified* here. It was built for games, and a game needs a scrolling background, characters moving over it, colour, music and noise at once. The chips read their data from memory by themselves, and every control is a memory location that the programmer can **peek** (read a byte) or **poke** (write a byte): the names are the C64 BASIC commands for a load and a store, and the source of the author's phrase "peek poke style".




**Text mode.** The standard screen is 40 columns by 25 rows, 1000 cells. The picture is *two* arrays in memory: screen memory, 1000 bytes, normally at $0400 (the number of the glyph for each cell), and **colour RAM**, 1000 four-bit entries at $D800

(the colour of each cell, one of 16). The shape of each glyph is not in these arrays; it is in the **character ROM** (a read-only memory of 8 bytes a glyph, one byte a row of eight dots, 256 glyphs: 2 KB for one set), which the video chip reads, and which a program can also replace by pointing the chip at glyphs it has made in RAM. To put an A at the top left, in red, the BASIC programmer writes two pokes, and the video chip, which reads screen memory sixty times a second by itself, shows it (screen codes, colour numbers and addresses *unverified*):

```
poke(1024, 1)          # $0400: screen code 1 is the letter A
poke(55296, 2)         # $D800: colour 2 is red
poke(53280, 0)         # $D020: the border colour, black
poke(53281, 6)         # $D021: the background colour, blue
```

This is the "peek poke" model: the screen is memory, a cell is an address, and the chip does the display. There are no escape codes and no stream; a program that pokes address 1024 + 40 * row + column has put a character there, at any time, in any order.

**Bitmap mode.** The same chip could show a bitmap of 320 by 200 dots in memory, with colour attributes per 8 by 8 block, and **multicolour** modes that trade width for colour. These were modes of one chip, and a program could mix them, text in one band of the screen and graphics in another, by changing mode at a chosen line (the raster interrupt below).

**Sprites.** The VIC-II had eight **sprites**: movable objects of 24 by 21 dots, each described by 63 bytes of data in memory, a pointer byte (the last eight bytes of screen memory, $07F8 to $07FF, say which 64-byte block holds each sprite's shape) and a few registers (x position, y position, colour, enable, size, priority; collision registers that say which sprites touched). The chip draws the sprites over the background as the picture is read out; to move a sprite, the program changes two registers, and the picture underneath is never disturbed. A ship, a bullet and an enemy are each a sprite, and a game's movement is a handful of pokes per frame.

**The raster and the raster interrupt.** The VIC-II draws the screen by reading it in order, row by row, while an electron beam sweeps the tube (the **raster**; the whole sweep is a **frame**, 50 or 60 times a second depending on the television standard). A register holds the number of the line being drawn, and another can be set to a line number so that the chip interrupts the processor when the beam reaches it: the **raster interrupt**. Programs used it to change a register mid-screen, such as the background colour or a sprite position, and to make changes while the beam was elsewhere (3.15).

**Sound.** The SID is a three-voice synthesiser with filters, again with registers that the programmer pokes: a voice has a frequency, a waveform (triangle, sawtooth, pulse, noise), and an attack, decay, sustain and release envelope. A note is a handful of pokes. Sound came from the same few chips, controlled the same way: **one machine, one address space, a few chips whose registers are memory, and a program that sets them directly**.

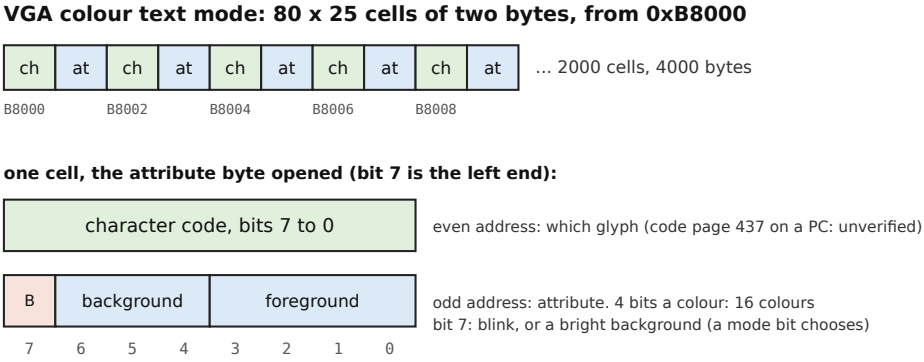
The C64's limits are those of 1982: 16 colours, 40 columns, 8 by 8 cells, 8 sprites, three voices. The author asks for the same ease without them (3.8).

## 3.5 The PC years: VGA, SVGA, VESA

The IBM PC took a different road: its video adapters were separate cards with their own memory and registers, and the software model mixed memory with input/output ports (separate I/O instructions, which neither the PDP-11 nor the C64 had). The **VGA** (Video Graphics Array, IBM, 1987) became the baseline that later adapters imitated. Everything in this section about the PC adapters is general knowledge, *unverified* here.

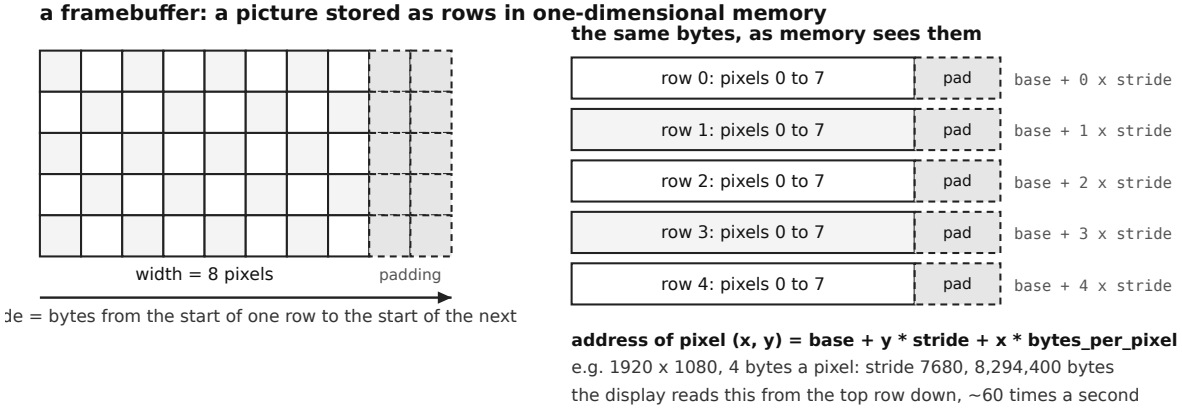
**VGA text mode.** The text mode is the C64's idea with the colour folded into the cell: 80 columns by 25 rows, 2000 cells, in memory at 0xB8000, two bytes a cell: the character code and an **attribute byte** (four bits of foreground, three of background and one bit for blink, or for a bright background). The shapes come from a font in the adapter's own memory, which a program can replace. To put a red A at the top left:

```
poke(0xB8000, 'A')          # the character
poke(0xB8001, 0x04)         # attribute: black background, red foreground
```




**VGA graphics.** The same card had graphics modes, and here the trouble began. Mode 13h (320 by 200, 256 colours, one byte a pixel, at 0xA0000) was the easy one: the framebuffer is a single array and a pixel is the byte at y * 320 + x. A byte names one of 256 palette entries, and the palette (a table of 256 colours that the program loads through I/O ports) is in the card. Modes with 16 colours and higher resolutions were **planar**: the four bits of a pixel's colour were stored in four separate **planes** (four arrays, the same pixel position in each), and a write reached whichever planes the registers selected. That was fast for some operations and a puzzle for the rest. The adapter's window onto its memory was 64 KB at 0xA0000, so any picture larger than 64 KB needed **bank switching** (the card has more memory than the window shows; a register chooses which 64 KB slice appears in the window, and the program must change the register whenever its drawing crosses a slice boundary).

**SVGA and VESA.** Every manufacturer's Super VGA added modes (800 by 600, 1024 by 768, more colours) and, since there was no standard, each used its own registers to switch to them and to select a bank. **VESA** (a standards body of display makers) wrote the **VBE** (VESA BIOS Extensions: calls into the adapter's ROM that list the modes, set one, and switch banks), and a later version let the card present its whole memory as one **linear framebuffer** at a high address, ending the banks. The result is a block of memory, a **width** and **height** in pixels, a number of bits per pixel, and a **stride** (also called pitch: the number of bytes from the start of one row to the start of the next, which can be more than the width times the bytes per pixel, because rows are padded to a convenient boundary).




Padding makes every row start on an aligned address; software must use the stride, not width x bytes.

**What the programmer had to know.** A graphics programmer of the late 1980s and the 1990s carried all of this: the VGA registers, set through ports, that choose which planes a write reaches and how its bits combine with what is already there; the planar layout, with one pixel's colour spread over four arrays; **Mode X** (the games community's name for an unofficial VGA mode of 320 by 240 that used the planar memory for more than one screen of pixels and faster fills); the bank register of each Super VGA chip, which differed from chip to chip, with quirks that the VESA calls hid only in part;

the palette, whose entries could be rewritten to cycle or fade colours without touching the picture; the **retrace**, because a change made at the wrong moment tore the picture; and hand-written assembly **inner loops** for the fill, the copy and the line, because a few cycles a pixel decided whether a game ran. The author's referent is the books of Michael Abrash on PC graphics programming: "still, is ok to mention the problems graphics coders faced, which is why we had 800 page michael abrash books" (errata 105). The titles, dates and page counts are *unverified*; no copy is on this machine and no record in the repository names a title. The depth of those books measures what the hardware pushed onto the programmer.

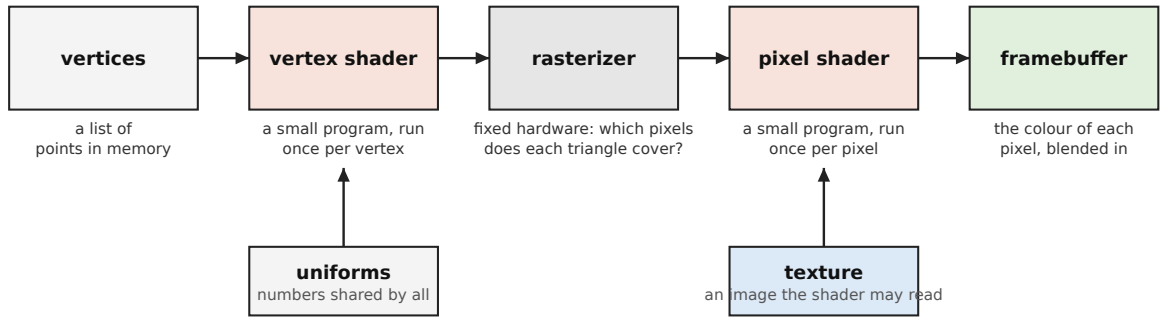
**The uncomfortable split.** The PC now had two worlds. In **text mode** the hardware drew the characters, so text was fast, small (4000 bytes) and easy, and nothing else could be on the screen. In **graphics mode** a program could show anything but drew every dot: even a letter was the program's job (copy its rectangle from a font, the Blit's bitblt in software), and text became slow and ugly. Changing between the two meant switching modes, blanking the screen and losing the conveniences of the other world. The C64 had mixed text, bitmap and sprites in one frame; the PC made the programmer choose. Windowing systems, which need both together, chose graphics and drew everything.

**Why sprites vanished.** The VGA had no sprites, so games of its era drew every moving object into the framebuffer in software, while a second copy of the screen was being prepared (3.15). The processor was by then fast enough to move a screenful of dots in a frame time, the framebuffer was ordinary memory that any program could write, and windowing systems composed overlapping windows in software, which left no use for a hardware plane or sprite. The cost was real: moving a mouse cursor, a ship or a window meant writing and restoring pixels while the display was reading the same pixels (3.15).

## 3.6 The GPU: pipelines, vertices, shaders

In the 1990s a new kind of chip took the drawing from the processor: the **GPU** (graphics processing unit), first for three-dimensional games and now for every screen (general knowledge, *unverified* here, as is the rest of this section). Its design follows from what a three-dimensional scene is: a list of **vertices** (points in space with a position, and perhaps a colour, a direction the surface faces and a place in a picture), joined into **triangles** (three vertices: the one shape that is always flat, so any surface can be built from them), seen by a camera. To draw it, the machine must (a) work out where on the screen each vertex lands, (b) find which pixels each triangle covers (**rasterization**: turning a shape into the set of pixels it covers), and (c) decide each pixel's colour, which is usually where a **texture** (a picture stored in memory, which is looked up at a position given by the vertex, to paint a surface with, say, brick) comes in. The three steps are the **graphics pipeline**.

**the graphics pipeline: data goes in at the left, pixels come out at the right**




Every vertex goes through the same program at the same time on different lanes; so does every pixel.
The programmer writes the two shaders (pink); the rasterizer between them is fixed.

This is the model of general knowledge of current GPUs (unverified here, 3.19); stages such as geometry and tessellation are left out.

The early GPUs did each step in fixed hardware. The change that made the current GPU is that two of the steps became **programmable**: the programmer supplies a **shader** (a small program, in a special language, that the GPU runs; the name is historical, because shaders began as programs that computed the shading of a surface). A **vertex shader** runs once for every vertex and says where it goes; a **pixel shader** (also called a fragment shader) runs once for every pixel a triangle covers and says what colour it is. The rasterizer between them stays fixed, because it is the same arithmetic for every program.

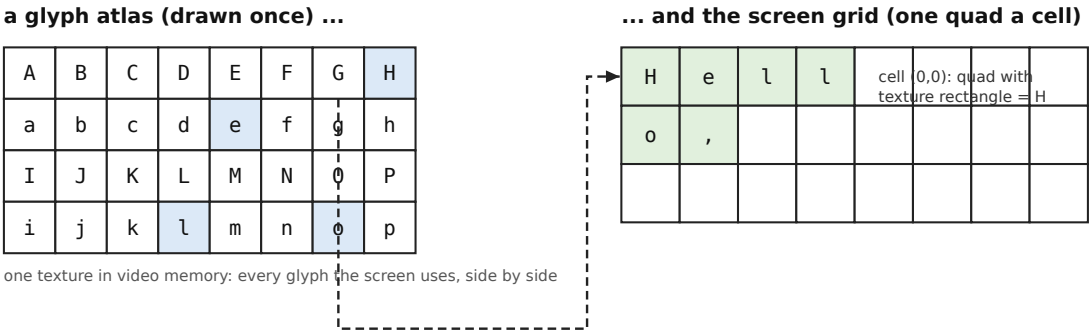
**Why a GPU is fast.** A processor core runs one thread of arbitrary code as fast as it can, with a deep pipeline, large caches and logic to guess and reorder. A GPU is built for *the same small program run on a very large number of independent items*: a million pixels each need the same few dozen instructions. So it has thousands of simple **lanes** (units that each do arithmetic on one item), grouped so that 32 or 64 lanes (the size varies by maker) execute one instruction at the same time on different data (**SIMD**: single instruction, multiple data), and memory of very high **bandwidth** (bytes per second) feeds them. When one group waits for memory the hardware switches to another. Three more things matter in practice. The CPU does not touch pixels: it writes a **command buffer** (a list of commands in memory, such as "bind this texture, use these shaders, draw these 3,000 vertices as triangles") that the GPU reads and carries out by itself, like the descriptor rings of chapter 2. Work is **batched**: one command draws thousands of things, so its cost is paid once. And textures are stored in a layout that keeps neighbouring pixels close together, with the filtering done in hardware.

The GPU is poor at code with many decisions or with items that depend on each other, and the programmer must describe the whole job as a pipeline, write shaders and feed it in large batches. Section 3.14 shows how much graphics boilerplate comes from that model.

## 3.7 Why a terminal emulator uses the GPU, and why a font needs three dimensions

A **terminal emulator** (a program that shows a grid of text in a window of a graphical desktop and acts like the VT-series terminal of 3.2) has a small job: a grid of cells, each a glyph with a foreground and a background colour. Alacritty, kitty and WezTerm draw that grid on the GPU (general knowledge, *unverified* here; the author's chat names kitty). Why would a program that shows only text use machinery built for three-dimensional games?

**How a GPU draws text.** A screen of text is the same glyphs over and over. The program draws each glyph once, with a font rasteriser, into a **glyph atlas** (one texture holding every glyph the screen uses, side by side), then describes the screen as a list with one entry per cell: its position, the part of the atlas to show, and the two colours. The GPU draws each cell as a **quad** (a rectangle of two triangles) textured with that part of the atlas, and the pixel shader mixes foreground and background by the glyph's value at each pixel. The whole screen is **one draw call** (one command that draws a whole list).




**To draw the screen: send the GPU a list with one entry per cell, and run one draw call.**

entry = (cell column, cell row, atlas rectangle of the glyph, foreground, background)
vertex shader: turn the cell position into screen corners; pixel shader: look the glyph up, mix fg and bg
a change of one cell, or a scroll by a row, is a change to the list or to a number, not to 2 million pixels

The atlas is made by a font rasteriser (CPU) and uploaded once; new glyphs are added as they first appear.

The processor never touches a pixel. It changes a cell in the list when the text changes, and scrolling a line is a change of one number (where the list starts) or a shift of the list, not a copy of two million pixels. The glyph is stored as coverage values, so the shader can place it at a fraction of a pixel and blend it: **antialiasing** (smoothing an edge by giving the pixels it crosses an in-between colour) and **subpixel positioning** (placing a glyph more finely than the pixel grid). The GPU does this from its own memory, and the processor stays free to run the program whose output is shown.

**Why a font needs the three-dimensional machinery.** There are two parts to the answer. First, a glyph in a TrueType font is not a bitmap but an **outline** (curves in a coordinate system independent of size), and turning it into pixels is **rasterization** again, the job the GPU's rasterizer does for triangles. A font rasteriser on the processor produces the atlas once; the GPU can also do the curves itself, by tessellating the outline into triangles, by testing in the pixel shader whether each pixel is

inside the curve, or by storing a **signed distance field** (a texture in which each pixel holds its distance to the nearest edge of the glyph, positive outside and negative inside, so a pixel shader draws the glyph sharply at any size from a small texture; credited to a Valve paper of 2007, *unverified*). Each is the 3D pipeline used on a flat shape.

Second, the pipeline is three-dimensional only by convention. A current GPU has **no 2D mode**: the 2D accelerator of the 1990s, with its blits, line draws and rectangle fills, is gone from the hardware, and drivers draw a rectangle as two triangles with a shader (general knowledge, *unverified*). A flat picture is the case of the 3D machinery in which every z value is the same and the camera looks straight on. A flat-rectangle engine would be smaller and would do the terminal's job; it is not built because the GPU has to handle the harder case anyway. That is how the terminal ends up on the 3D chip.

## 3.8 The pdp64 terminal: the Blit's bitmap and the C64's pokes

The chain so far: the PDP-11 sent characters down a wire (3.1); the VT52 and VT100 let the program address a screen by position (3.2); the Blit put the bitmap in the address space and the C64 put text, sprites and sound behind pokes (3.3, 3.4); the PC split text from pixels (3.5); the GPU made drawing a pipeline of shaders (3.6). The pdp64 terminal takes the Blit's and the C64's side of that history.

**3.8.1** The graphics terminal is based on two machines, not one: the **Commodore 64**, for text, graphics, sprites and sound together under simple stores to memory, and the **Blit terminal** (Rob Pike's bit-mapped windowing terminal), for its framebuffer in the address space and the bit-block-transfer operation that moves rectangles of pixels. The pdp64 modernises and extends both (errata 66). *(D)*

## 3.9 The form of the terminal

The author, 2026-11-10: "our virtio-console device is an attempt to bring the nice properties of the blit terminal and c64 for doing simple and easy peek poke style display of text and also doing graphics, but with modern color depths and screen sizes, and with glyph maps we can even have a ttf engine generate glyphs and they will go into their memory slots for use."

**3.9.1** The pdp64's terminal brings the nice properties of the Blit terminal and the C64: a display that a program drives by simple peeks and pokes, for text and for graphics together, at modern colour depths and screen sizes; and a **glyph map** (a table of slots, each a place in memory for the picture of one glyph) that a TrueType engine fills, so that the shapes of the characters are made by software and used by the display through their slots (errata 105). *(D)* Details pending (J.3.2): the form of the device and the name the author gives it, which is not yet the virtio-console of the virtio specification, a serial text device.

Details pending (J.3.4): whether a cell of the text display holds a character code or a glyph slot, and the layout of a slot.

Open: whether the terminal has sprites or a hardware cursor (J.3.5).

Open: how text, pixels and sprites are composed on the display, the author's mashup of a GPU, video memory, the C64 and the Blit (J.3.3).

## 3.10 How the terminal drives HDMI and the other ports

**3.10.1** The display connector is standard kit: HDMI (and DVI and DisplayPort) is a transmitter chip that takes pixels and sync signals in parallel and does the link protocol, the encoding and the electrical signalling itself; on the first board it is the DE10-Nano's ADV7513, set up once over an I2C-style control bus. *(D)*

**3.10.2** The output is HDMI or some other standard graphics output: a different transmitter (DisplayPort, a VGA digital-to-analogue converter, an LVDS panel) takes the same pixel stream, so the pdp64's side does not change (errata 105: "it is agreed that output is hdmi or some other standard graphics output"). *(D)*

**3.10.3** The ports that carry data, SATA for storage and USB for peripherals, follow the same pattern: the protocol is in the chip or controller block, and the processor's side is a slot with the common registers and a ring (chapter 5). *(D)*

*Commentary.* **Chain.** The PDP-11's output was the serial line of 3.1, with no display connector. The MiSTer framework on the DE10-Nano already works this way: the retro cores produce a pixel stream, and the framework's scaler and the HDMI transmitter do the rest, which is why the plan buys HDMI there. **Trade-offs.** The transmitter's control bus needs one-time setup, which a boot routine does with a few register writes; the pixel clock must be generated to the mode, with a phase-locked loop on an FPGA and the same block in silicon. **Improvements.** Replacing the transmitter or the connector changes nothing in the program that draws, as a SATA controller behind its ring can be replaced without changing the disk driver. **Benefits.** No part of the operating system carries an HDMI, SATA or USB protocol stack. **Modern comparables.** Single-

board ARM computers with a display controller and an HDMI transmitter (general knowledge, *unverified*). **Impact.** *Kernel:* hot-plug of a monitor is a bus event if the transmitter reports one. *Compiler:* none. *Hardware:* a pixel generator (the scan-out), a clock and an off-chip transmitter, with no protocol logic in the pdp64's own gates.

## 3.11 The framebuffer, the memory type and the memory budget

**3.11.1** The framebuffer is mapped with the uncached memory type, or the processor's writes sit in the data cache while the video device scans out stale memory; a `FENCE` separates writing a device's registers from setting its go bit. *(D)*

**3.11.2** On the first board the block RAM (about 5.5 megabits on the DE10-Nano) cannot hold a modern frame (a 1920 by 1080 frame at 32 bits is 8.3 MB), so the framebuffer lives in main memory and the block RAM holds the scan-out line buffer of a few kilobytes. *(D)*

**3.11.3** The block RAM budget (caches, translation buffer and the scan-out line buffer) is settled before the video hardware is built. *(D)*

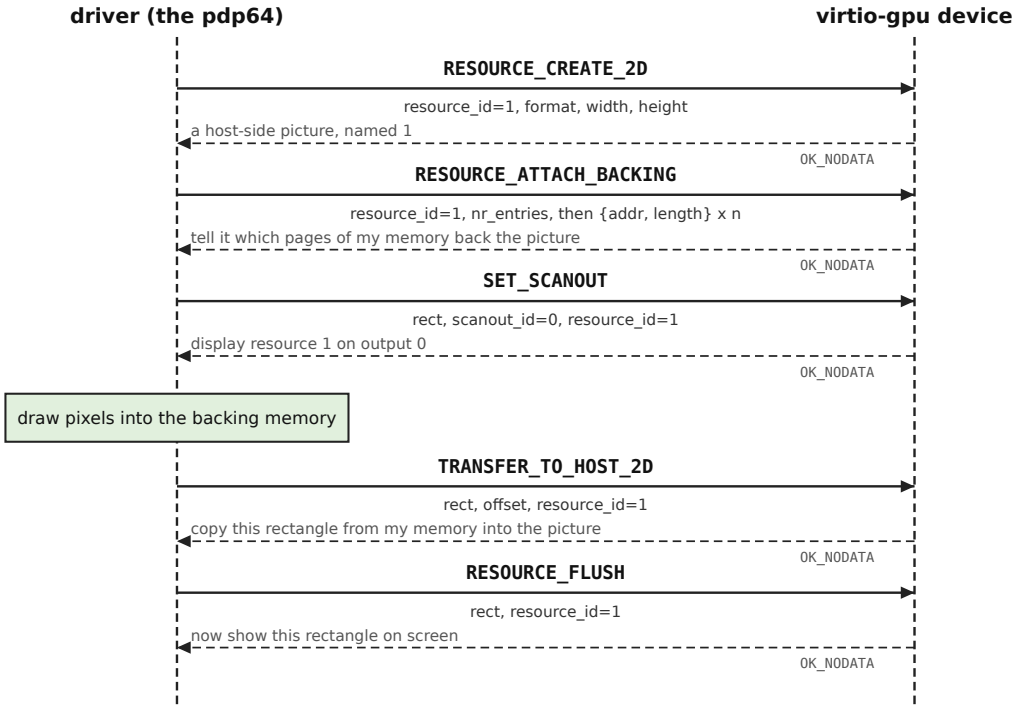
Open: whether the framebuffer has a write-combining memory type, which the four memory types of 2.3.3 do not name (J.3.10).

*Commentary.* **Chain.** The PDP-11 had no framebuffer; whether the 11/70's cache watched Unibus writes so that DMA looked coherent is *unverified* (2.5, commentary). The VAX Unibus adapter's buffered data paths needed purging (2.5), and ARM and MIPS systems with non-coherent DMA place cache maintenance in the kernel (*unverified*). The pdp64 chooses the memory type instead (2.5.2). **Trade-offs.** An uncached framebuffer makes a single pixel write slow and a long run of writes no slower per byte. That punishes pixel-by-pixel drawing from the processor. **Modern comparables.** Write-combining memory types on x86 and ARM serve the same purpose (general knowledge, *unverified*). **Impact.** *Kernel:* the mapping rule is one memory-type value in the page-table entry. *Compiler:* none. *Hardware:* the scan-out reads main memory by bus mastering and competes with the processor for bandwidth.

## 3.12 virtio-gpu: the standard graphics device

The standard display (errata 79: "virtio is fine"; virtio-gpu in its 2D mode) is a virtio device like the disk of chapter 2, so 2.7 applies: it is found by enumeration, its queues are rings of descriptors in memory, and the work is commands in a queue. The commands below are from the Linux header `virtio_gpu.h` and from QEMU's `hw/display/virtio-gpu.c`.

**The vocabulary.** A **resource** is a picture that the device owns, named by a number the driver chooses (`resource_id`), with a pixel format, a width and a height. Its **backing** is the driver's own memory that holds the pixels: a list of `{addr, length}` entries (`struct virtio_gpu_mem_entry`) naming the pages. A **scanout** is one output of the device (up to `VIRTIO_GPU_MAX_SCANOUTS`, 16, in the header); `SET_SCANOUT` says which resource, and which rectangle of it, an output shows. Two commands move and show the pixels: `TRANSFER_TO_HOST_2D` copies a rectangle of the backing memory into the device's picture, and `RESOURCE_FLUSH` tells the device that a rectangle changed and should be shown. The pixel formats are 32 bits with 8 bits a channel (eight in the header): `B8G8R8A8_UNORM`, `B8G8R8X8_UNORM`, `A8R8G8B8_UNORM`, `X8R8G8B8_UNORM`, `R8G8B8A8_UNORM`, `X8B8G8R8_UNORM`, `A8B8G8R8_UNORM`, `R8G8B8X8_UNORM` (values 1 to 4, 67, 68, 121 and 134; the names give the channel order, the *colour order* annoyance of 3.14). Every command starts with a `virtio_gpu_ctrl_hdr`: a type, flags, a 64-bit fence id, a context id and a ring index; the flag `VIRTIO_GPU_FLAG_FENCE` asks that the response carry the fence id once the work is done, which is the device's only completion-ordering tool. The rectangle is `struct virtio_gpu_rect`: x, y, width, height, four 32-bit numbers.




Each line is one command in the control queue, answered by a response; the header of every command is a type, flags, a fence id, a context id.

```
# the whole of a virtio-gpu 2D start-up and one update, as pseudocode over the queue of chapter 2.7
send(RESOURCE_CREATE_2D,     resource_id=1, format=X8R8G8B8, width=1024, height=768)
send(RESOURCE_ATTACH_BACKING, resource_id=1, entries=[{addr=PAGES, length=1024*768*4}])
send(SET_SCANOUT,            scanout_id=0, resource_id=1, rect={0,0,1024,768})

draw_into(PAGES)                                    # the driver writes pixels: ordinary stores
send(TRANSFER_TO_HOST_2D,    resource_id=1, rect={0,0,1024,768}, offset=0)
send(RESOURCE_FLUSH,         resource_id=1, rect={0,0,1024,768})
```

In QEMU's `virtio-gpu.c`, `transfer_to_host_2d` rejects a rectangle that is not inside the resource with `VIRTIO_GPU_RESP_ERR_INVALID_PARAMETER`, then copies it **row by row** from the backing into the resource using the resource's stride, or in one copy if the rectangle spans whole rows. `resource_flush` rejects a rectangle outside the resource the same way, then asks each scanout showing the resource to update the part of its display that the rectangle covers. The transfer is the **upload** and the flush is the **present**, and the program does both after drawing.

**What it does not define.** Neither the header nor the QEMU function mentions the vertical blank, a page flip, a refresh rate or tearing: the flush says "show this rectangle now" and the host decides when its display next reads it. The only timing handle is the fence of the header. QEMU's configuration has a `refresh_rate` field for each output (`include/hw/virtio/virtio-gpu.h`), and the device can report an EDID with the optional feature `VIRTIO_GPU_F_EDID`; the header does not say that either changes how a flush is timed. The standard device therefore gives a program a damage rectangle (the flush rectangle), a cursor (the cursor queue, `UPDATE_CURSOR` and `MOVE_CURSOR`), a fence and display information (`GET_DISPLAY_INFO`: size and enabled flag of each output), and nothing about the vertical blank.

**3.12.1** The standard graphics device is virtio-gpu in 2D mode over the virtio PCI Express transport (errata 79); its commands are the five of the figure, plus the cursor queue and the display-information query. *(D)*

Open: the feature subset of virtio-gpu and of virtio-snd that the pdp64 requires, the hot-plug scope and whether OpenBSD has a virtio-snd driver (J.3.7).

Open: the 48-bit pixel against the 32-bit formats of virtio-gpu (J.3.6).

*Commentary.* **Chain.** The PDP-11 has no antecedent: its devices had registers, not queues (2.1). QEMU offers virtio-gpu as a virtual machine's display (`hw/display`): the guest draws into its own memory and tells the host which rectangle to show. **Trade-offs.** It is the most standard route (one tested driver, the same in QEMU, `sim64` and the FPGA, as errata 79 says) and does everything in software: the processor writes every pixel and the host copies them, which is cheap for a terminal with

little damage and costs a game what 3.5 described. **Benefits.** No special hardware. **Modern comparables.** Linux's `virtio_gpu` driver, which sits on the DRM interface of 3.14. **Impact.** *Kernel:* a driver for the five commands, one queue of 2.7 and a cursor queue. *Compiler:* none. *Hardware:* none beyond the virtio device and the bus.

## 3.13 A standard graphics card

**3.13.1** A PCI Express graphics card may be attached and supported with its own driver, as any PCI Express device may; the standard does not require one (errata 78). *(D)*

**3.13.2** The standard does not require a card because of what one costs: PCI Express configuration space, base-address mapping and message-signalled interrupts, a driver stack for a programming model that for some vendors must be reverse-engineered, the card's firmware and the initialisation a PC's firmware performs, and no hard PCI Express block on the first board (the Cyclone V and the Zynq 7020 are recorded as lacking one, *unverified* here; a Zynq UltraScale+ has a hard root complex, `docs/pdp64-fpga-plan.md`). *(D)*

*Commentary.* **Trade-offs.** A standard card needs a PCI Express root complex, a large block where the chip has none, and a graphics driver ported to the pdp64, which is a larger task than the architecture (`docs/pdp64-fpga-plan.md` says the same and answers no to a discrete GPU). **Modern comparables.** PowerPC and RISC-V workstations take PCI Express graphics cards with open drivers (general knowledge, *unverified*). **Impact.** *Kernel:* a large graphics driver ported. *Compiler:* none. *Hardware:* a PCI Express root complex and a bridge to the bus.

## 3.14 What made graphics programming annoying

Graphics programming has been annoying, cumbersome and full of **boilerplate** (code written the same way in every program before the program can do anything of its own). This section lists the boilerplate that exists and shows three pieces of it. The PDP-11 has no antecedent: its console needed none (3.1). The chain runs from the VGA programmer's register lore (3.5) to the Linux and Vulkan setup lists below.

**3.14.1** What made graphics programming annoying, cumbersome or boilerplate is sugared away by the architecture to almost nothing (the author, 2026-11-10: "basically, anything that made graphics programming annoying or cumbersome or boiler platey, we want to sugar away to almost nothing", errata 105). *(D)*

**3.14.2** The measure that every design in this chapter is held to is the author's: "graphics programming should be as simple and pleasurable as the c64 was." *(D)* (Errata 105.) Details pending (J.3.9): which sugar, for 3.14.1 and 3.14.2; the table below lists the pain points.

The books of Michael Abrash on VGA and PC graphics programming (the author's "800 page michael abrash books"; titles and page counts *unverified*, 3.5) exist because a programmer could not get a fast picture without knowing the VGA's registers, its planar memory, its retrace, the banking of each chip and how to hand-tune the inner loops. Each row of the table below is a piece of that knowledge, or of its successor (the long setup lists of the Linux display interface and of Vulkan), The knowledge was real and was needed.

Three worked examples follow. Each shows the boilerplate as the real systems have it (names are those of the real interfaces, checked in the headers on this machine where marked, otherwise general knowledge, *unverified*).

**Example 1: getting a picture on the screen at all.** On a VGA/VESA PC the program asked the BIOS for the list of modes, picked one, set it, and found the framebuffer:

```
# VESA (VBE), 1990s: about 15 lines in a real program (illustrative, not measured)
info = vbe_get_controller_info()
for m in info.mode_list:
    mi = vbe_get_mode_info(m)
    if mi.width == 1024 and mi.height == 768 and mi.bits_per_pixel == 32 and mi.has_linear_framebuffer:
        mode = m
vbe_set_mode(mode | LINEAR_FRAMEBUFFER_BIT)
fb = map_physical(mi.physical_base, mi.pitch * mi.height)
```

On Linux (DRM/KMS, the kernel's display interface) the same job is longer. The program opens the device, lists **connectors** (the outputs), reads the monitor's modes (**EDID**, the data block in which a monitor describes the modes it supports; general knowledge), chooses one (`struct drm_mode_modeinfo` in `drm_mode.h`: pixel clock, horizontal and vertical totals and sync positions, refresh rate), allocates a **dumb buffer** (a plain pixel buffer), registers it as a framebuffer (`DRM_IOCTL_MODE_ADDFB2`,

with a format code and a pitch), maps it, and sets the mode on a **CRTC** (the unit that does scanout; the name comes from cathode-ray tube controller):

```
# Linux DRM/KMS, one output, no atomic: about 40 lines in a real program (illustrative, not measured)
fd = open("/dev/dri/card0")
res = drmModeGetResources(fd)
conn = first connected connector in res
mode = conn.modes[0]                          # the monitor's preferred mode, by convention first
crtc = a crtc the encoder of conn can use
buf = create_dumb(fd, mode.hdisplay, mode.vdisplay, 32)    # returns handle, pitch, size
fb_id = drmModeAddFB2(fd, w, h, XRGB8888, handle, pitch)
ptr = mmap(fd, map_dumb(buf).offset, size)
drmModeSetCrtc(fd, crtc, fb_id, 0, 0, conn, mode)
```

A program on Vulkan or OpenGL meets another list: create an instance, enumerate physical devices, choose a queue family, create a logical device, a window surface, a swap chain and its images, image views, a render pass, framebuffers, command pools and buffers and synchronisation objects (the names `vkCreateInstance` and `vkCreateSwapchainKHR` are Vulkan's, whose headers are not on this machine, *unverified*; OpenGL's `glGenBuffers`, `glCompileShader` and `glLinkProgram` are in `glext.h` here). A "hello triangle" in Vulkan is commonly several hundred lines (general knowledge, illustrative). In virtio-gpu the equivalent is five commands (3.12): create resource, attach backing, set scanout, transfer, flush, with the drawing between them.

**Example 2: the cache and the flush.** A processor has a **cache** (a small fast memory that keeps recently used data close to the core). Pixels the program has written may still be in the cache and not in memory, and a device that reads memory by DMA then sees the old data. Where the device does not snoop the cache, the program must flush:

```
# a display that does not snoop the cache: about 3 lines per update, in every program (illustrative)
draw(framebuffer)
cache_clean_range(framebuffer, size)       # push the dirty lines to memory
memory_barrier()                           # and make sure the writes are ordered before the notification
poke(DOORBELL, 1)
```

The pdp64 already removes this, and chapter 2 says how: processor caches are coherent with each other, a device and the processor are not, and the answer is a memory type, **uncached**, instead of a flush instruction: "buffers are mapped uncached and the hand-off is explicit" (2.5.2), and "No driver can forget a cache clean or invalidate, because there is none" (2.5, commentary). The framebuffer is mapped uncached too (3.11.1). So on the pdp64 the program does none of this *(D)*.

**Example 3: drawing text.** On a GL or Vulkan system text is the longest boilerplate, because the GPU has no text:

```
# a GPU text path (illustrative; about 150 lines in a real small program, not measured)
font = ft_load("DejaVuSansMono.ttf"); ft_set_size(font, 16)
atlas = new_texture(1024, 1024)
for each character c needed:
    bitmap = ft_render_glyph(font, c)          # the font rasteriser (CPU)
    rect = pack_into_atlas(atlas, bitmap)      # find room: a bin-packing problem
    upload(atlas, rect, bitmap)
    remember(c, rect, advance, bearing)        # metrics: how far to move on, where the glyph sits
for each cell on screen:
    append_quad(vertex_list, x, y, rect_of(char), fg, bg)
upload(vertex_list); bind(atlas, shader); draw(vertex_list)
# plus kerning, ligatures, right-to-left runs, combining marks, fallback fonts, scaling for HiDPI
```

## The pain points, in a table

Each row names something that has annoyed graphics programmers and the size of its boilerplate (line counts are *illustrative* and **not measured**).

| Pain point | Typical boilerplate (illustrative) |
|---|---|
| Mode setting and discovering what the display supports (EDID, modes) | 15 lines (VESA) to 40 (DRM) |
| Pixel formats | one format code to get right, and conversion code when it is not |
| Stride and pitch | one multiply in every pixel address |
| Endianness and colour order (RGB or BGR, alpha first or last) | a swap or a table, in every program |
| Bank switching and mapping the framebuffer | 5 to 20 lines |
| Double buffering and tearing | 10 to 30 lines, plus a frame loop |

| Pain point | Typical boilerplate (illustrative) |
|---|---|
| Synchronising with the display | polling or a vblank interrupt handler |
| Cache coherence between CPU writes and the device | 3 lines per update, forgotten once per project |
| Device initialisation and context creation | 50 to 200 lines (Vulkan), 25 (virtio-gpu) |
| Buffer and memory management (allocate, map, upload, barriers, fences) | 20 to 100 lines |
| Shader compilation and pipeline state | 50 to 300 lines |
| Drawing text (glyph atlases, kerning, layout) | 100 to 300 lines |
| Clipping and windows | 10 to 30 lines per drawing call |
| Sprites and cursors | 20 lines to save, draw and restore |
| Colour spaces and gamma | a table and a conversion, or wrong colours |
| Resolution changes and hot-plug | 20 to 60 lines of event handling |
| Fonts and scaling (HiDPI: screens with several pixels to a logical dot) | a scale factor threaded through every program |
| Input of palettes | 256 writes through ports (VGA) |
| Sound and video sync | a timestamp and a resampler |

The pdp64 has decided two of them: cache coherence is a memory type and no flush exists (2.5.2, 3.11.1), and the machine is little-endian (errata 72).

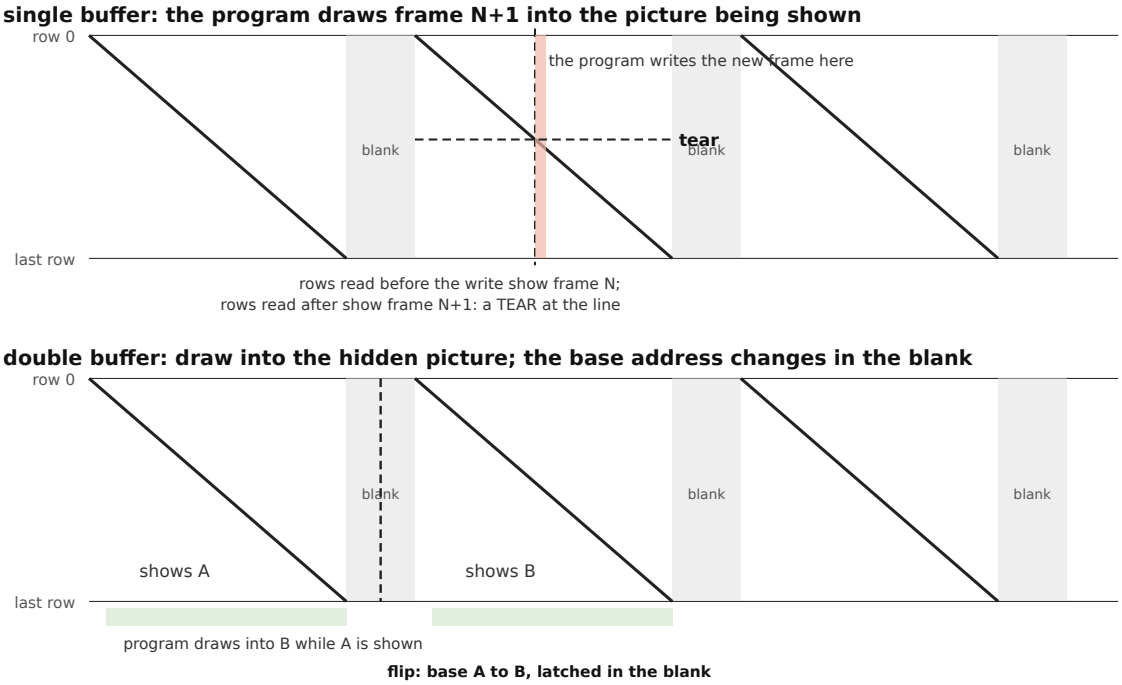
Details pending (J.3.9): the means by which the architecture removes the rest.

## 3.15 Scanout, tearing and double buffering

The chain: the C64's raster register and interrupt (3.4) let a program time its changes to the beam; the VGA's retrace (3.5) asked the same of PC programmers; double buffering and compositors moved the timing into the operating system. The PDP-11's teletype had nothing to tear.

**Scanout.** A display does not hold a picture; it is **refreshed**. The adapter reads the framebuffer from the first row to the last, in order, at a fixed rate, and sends each pixel to the monitor as it is read: this is **scanout**. The rate is the **refresh rate**, 60 times a second on most monitors (the C64's was 50 or 60, by television standard). Between frames, and between rows, there are **blanking intervals** when the monitor is sent no pixels (the beam of a tube had to fly back); the gap between the last row of a frame and the first of the next is the **vertical blank**, which VGA called the vertical **retrace**. The C64's raster register and interrupt (3.4) are the program's view of it: the number of the row being read now, and a way to be told when the beam reaches a chosen row.

**Tearing.** If the program changes the picture while the scanout is reading it, the part of the screen above the row being read shows the old picture and the part below shows the new one. The seam is a **tear**: a horizontal line where a moved object appears cut in two. **Flicker** is the related fault: a program that clears the screen and then draws lets the scanout catch it empty or half drawn. In the figure time runs to the right and the row being read rises from the first row to the last once a frame.




Time runs to the right; a frame is about 16.7 ms at 60 Hz (an example rate). The slanted line is the display reading one row after another.

**Double buffering.** The standard fix is two pictures. The **front buffer** is being shown; the program draws the next frame in the **back buffer**, and when it is complete the two exchange roles while the scanout is in the vertical blank and no row is being read. The exchange is a **swap** or a **page flip**. It can *change the base address* the scanout reads from (a register write; nothing is copied) or *copy* the back buffer into the front buffer (a **blit**, cheap with a blitter and slow without, and the copy itself tears if it outlasts the blank). In peek/poke pseudocode, single buffer first:

```
# 1. single buffer: tears and flickers
loop:
    clear(FRAME)                          # the scanout can see an empty screen here
    draw_scene(FRAME)                     # and half a scene here
```

```
# 2. double buffer, swap by changing the base address (a page flip), done by polling
front = BUF_A; back = BUF_B
poke(SCANOUT_BASE, front)
loop:
    draw_scene(back)                      # nobody is reading `back`
    while peek(RASTER_LINE) != 0:         # wait for the start of the blank: busy-waiting
        wait                              # (or wait for a vertical-blank interrupt)
    poke(SCANOUT_BASE, back)              # the flip: takes effect now
    swap(front, back)                     # draw next into the picture just retired
```

Double buffering has four costs. It **needs twice the memory**: at 1920 by 1080 and 4 bytes a pixel, 8.3 MB twice (3.11.2). It **needs a deadline**: if the next frame takes longer than a frame time the flip misses the blank, and the program either waits for the next one (the rate falls from 60 to 30 and then to 20, the "half frame rate" cost of `kernel-pain-points.md` 0.6 #3) or flips at once and tears. **Triple buffering** (three pictures, so the program can start the next frame while the finished one waits for the blank) removes the wait and adds a frame of memory and of **latency** (the delay between the program's input and what the screen shows). Polling the blank occupies the processor, and the interrupt version needs a driver. Finally, a base-address register must change *in the blank*, which a program that is itself interrupted cannot always manage: the Linux driver for one common graphics chip disables interrupts and busy-waits for the row to pass before it updates registers ("vblank evasion", `kernel-pain-points.md` 0.6 #13, from `intel_pipe_update_start`).

**Is this still a problem with HDMI?** The author asked (2026-11-10): "with modern hdmi is tearing even a concern?" Tearing is not a property of HDMI. HDMI (like DVI and DisplayPort, 3.10) carries a pixel stream at a steady rate and

cannot know whether the picture was complete when it was read. A tear happens one step earlier, in the display controller that reads the framebuffer while a program changes it. On a phone or desktop the **compositor** (the system program that gathers the pictures of all windows and builds the one that is shown) and the driver hide it: they draw the next frame in a spare buffer and flip it at the vertical blank. A program on the pdp64 that writes straight into the framebuffer with no operating system between (a boot program, a game written to the metal, a C64-style program) can still tear, whatever the cable.

Variable refresh (HDMI VRR, adaptive sync) turns a late frame into a short delay, which reduces the need for a blank-time flip but does not remove it: the frame still must not change *while* it is read. A television may hold a whole frame before showing it (latency), and a laptop panel may refresh itself from its own memory when nothing changes (panel self-refresh). Checked on this machine: HDMI is a pixel-stream transmitter (3.10.1), and the DRM header `drm_mode.h` has a page-flip call with a completion event (`DRM_MODE_PAGE_FLIP_EVENT`) and an asynchronous flag that allows a flip with tearing (`DRM_MODE_PAGE_FLIP_ASYNC`). *Unverified* general knowledge: the VRR and adaptive-sync standards and what a sink does with a late frame, television latency, panel self-refresh, and what phone and desktop compositors do.

**3.15.1** The architecture offers conveniences, "nice architectural sugar", that avoid tearing, flicker and the other faults that forced graphics programmers to double-buffer (the author, 2026-11-10, errata 105). *(D)* Details pending (J.3.8): which conveniences, and whether a tear-free flip is worth hardware when the display is variable-refresh.

## 3.16 The C128

**3.16.1** The pdp64 has a **c64 mode**: a C128 built onto the die, with the SID and the C64's other circuits, into whose address space a program is loaded by DMA over PCI Express so that it acts as a terminal, the way the Blit's terminal and the VT-series terminals did (the author, 2026-11-10: "we will have a c64 mode, where a c128 is built onto the die with sid etc, so a bit of code loaded into its address space by pcie dma will let it act as a terminal the way the blit and vt55 did"). *(D)* The C128 is a device on the bus, as 3.16.4 says. Details pending (J.3.11): which core, whether unmodified C64 software runs, how the core and the host talk, how the loaded image is contained.

**3.16.2** C64 mode is the enabling or disabling of the C128 as a PCI Express device: when it is enabled it takes over the graphics and audio output, or sends them to a compositor (the author: "the mode is just enabling/disabling the c128 as a pcie device so it takes over the graphics and audio output... or sends them to a compositor"). *(D)* A **compositor** is the program or hardware that combines the pictures of several windows or sources into the one picture that is scanned out. Details pending (J.3.12): the form of the compositor path.

**3.16.3** Of the paths to a compositor, the C128 draws into a buffer that the host's compositor treats as a surface: "c128 drawing into a buffer makes most sense" (the author, 2026-11-10, errata 106). Details pending (J.3.12): how the compositor learns that the buffer is the core's, who owns the memory, how the core's frame timing meets the compositor's, and the audio form (3.17).

**3.16.4** The Commodore 128 is another device on the PCI Express bus and is not part of the pdp64 standard: its 8502 core, memory management, 80-column video chip and compatibility island are an optional device reached through the IOMMU window (errata 77 and 75). The instruction set, the memory model and the standard devices neither contain nor depend on it, and no instruction, register or PSW bit exists for it. *(D)*

## 3.17 The audio path: samples, mixers and the pdp64

Audio has the same structure as video, with a **mixer** (the audio compositor) where video has a compositor, and this section follows the order of 3.15. The chain: the PDP-11 had no sound hardware in this repository's records, so there is no antecedent there; the C64's SID synthesised sound from registers (3.4); PC sound cards stored samples and read them out; a sound server mixes several programs.

**What a sound card does.** A sound is a pressure wave, stored as a list of **samples** (numbers, each the height of the wave at one instant) taken at a fixed rate, commonly 44,100 or 48,000 a second (both appear in the virtio-snd rate list below), each sample a 16-bit integer, 24 bits or a floating-point number. A **DAC** (digital-to-analogue converter: the chip that turns a number into a voltage) takes one sample on each tick of a clock, and the speaker moves. The device is fed from a **ring buffer** (a block of memory treated as a circle, which the program fills and the device empties, the device reading the next sample at each tick and the program writing ahead of it). This is the scanout of 3.15 again: the display reads the framebuffer at a fixed rate and the DAC reads the ring buffer at a fixed rate. A **buffer underrun** (the device reaches samples the program

has not yet written) is **audio's tearing**, heard as a click or a gap, and as certain when the program is late as a tear is when it draws late.

```
# fill the ring buffer: the program writes ahead of the read pointer, and notices an underrun
fill_ring(ring, next_frame):                # called by an interrupt each period, or in a loop
    while free_space(ring) >= PERIOD:
        samples = generate_or_read(PERIOD)   # this period's samples
        write_to_ring(ring, samples)
    if device_read_pointer_passed_write_pointer(ring):    # underrun
        count_underrun(); write_silence_up_to(...)        # a click is unavoidable; at least not noise
```

**The SID's way.** The C64 stored no samples (3.4). The SID has three **voices**, each an oscillator with a waveform, an envelope and a filter, in hardware: a program says "this frequency, this waveform, this attack and release" and the chip *synthesises* the sound. The sound is never in memory as numbers, so nothing underruns and nothing is mixed in software; the chip mixes its three voices inside itself. This is the difference between the C64's text mode (the chip draws from a description) and a framebuffer (the program draws every dot).

**What a mixer does.** When several programs play sound the card has one ring buffer and one DAC, so something must combine the sounds into one stream: the **mixer**, the audio compositor. Its job, per **period** (a short block of samples, a few milliseconds):

```
mix_period(streams, out):                   # one block of the output; runs once per period
    for each frame i in 0 .. PERIOD-1:
        sum = 0
        for s in streams:
            x = s.next_sample()             # after converting s to the output's rate and format
            sum += x * s.volume             # per-stream volume
        out[i] = clip(sum)                  # or a limiter: a gentle gain reduction instead of a hard cut
    write_to_ring(out)
```

A mixer also has four other jobs. It **converts each stream to a common rate and format**, because one program plays 44.1 kHz music, another 48 kHz speech and a third floating-point effects; this **sample-rate conversion** (resampling: computing the samples of a wave at the new rate) is a filtering job, so a mixer is more than an adder. It **applies per-stream volume** and **clips or limits** the sum, because two loud samples added exceed what the output can hold (**clipping**: a hard cut of the top of the wave, heard as distortion; a **limiter** lowers the gain smoothly instead). It balances **latency against underrun**: a small buffer follows the program quickly and leaves little margin for the program being late; a large one is safe and late. And it handles **clock drift**: the program's timer and the DAC's clock are different crystals, so over minutes one runs ahead of the other. The remedy is to drive the mixer from the *device's* clock (write when the ring buffer has room) and to resample by a fraction of a percent to absorb the difference.

**Who the mixer is in real systems** (general knowledge, *unverified* except where noted): on Linux a **sound server** (PulseAudio and, more recently, PipeWire) or ALSA's software mixing plug-in (`dmix`); on macOS CoreAudio; on Windows WASAPI's shared mode. This machine has the `pulseaudio` program and the ALSA library `libasound`; it has PipeWire's client library but no PipeWire program. QEMU has a mixing engine (`audio/mixeng.c`, `audio-mixeng-be.c`) and backends for ALSA, PulseAudio, PipeWire, CoreAudio, DirectSound, JACK, SDL and others (`qemu-src/audio`): a virtual machine's sound is mixed by the **host**, like any other program's.

**What virtio-snd defines** (checked in `/usr/include/linux/virtio_snd.h`). Four queues: control, event, transmit (`TX`) and receive (`RX`). Streams are **PCM streams** (pulse-code modulation: plain samples) with a direction (output or input), described by `virtio_snd_pcm_info`: a bitmap of formats (from 4-bit ADPCM and 8-bit mu-law through `S16`, `S24`, `S32`, `FLOAT` and `FLOAT64` to DSD and IEC958), a bitmap of rates (5,512 Hz to 384,000 Hz, including 44,100 and 48,000), and a range of channels. The driver sets a stream's parameters (`PCM_SET_PARAMS`: `buffer_bytes`, `period_bytes`, channels, format, rate), prepares, starts and stops it, then sends audio as buffers on the TX queue, each tagged with a `stream_id` and answered with a `virtio_snd_pcm_status` that carries a `status` and the **current device latency** in bytes. The event queue reports `PCM_PERIOD_ELAPSED` and `PCM_XRUN` (an underrun or overrun: the same word as the ring buffer's), and jack events; each stream has feature bits for shared memory, polling, and period and xrun events (`VIRTIO_SND_PCM_F_*`). Controls such as volume are optional (`VIRTIO_SND_F_CTLS`). The header says nothing about **who mixes**: it defines one stream at a time, as one card. Mixing several programs happens on the host side of the device (QEMU's engine above) or, on a machine with no host, in the pdp64's own kernel or library before the stream is sent.

**3.17.1** The standard audio device is virtio-snd (errata 79); the device carries one stream at a time as a PCM stream of the formats and rates the device lists. *(D)*

Open: where the pdp64's mixer lives (the kernel, a library or the device) and who drives it (J.3.13).

## 3.18 Windows: from the Blit to rio and NeWS

**What the sources confirm and what they do not.** The Blit's `mux` was a window system in the terminal; its successor on Plan 9 was **8½**, whose paper says "Primarily for reasons of history and familiarity, the general model and appearance of 8½ are similar to those of mux"; and **rio** replaced 8½ ("Note that 8½ has been replaced by rio (see rio(1))"; rio(1): "Rio manages asynchronous layers of text, or windows, on a raster display"; *layers* is the Blit's word). All of this is recorded in `docs/kernel-pain-points.md` section 14I. That the Blit inspired **NeWS** is the author's statement; no source read for the repository makes the claim, and the glossary says so. From *The UNIX-HATERS Handbook* (section 14G of the same file) the repository records that NeWS was an extensible window server in which "the client can download programs and data structures to draw the whole schematic", and that "Ultimately, NeWS was not economically or politically viable" (the reasons given are about licensing). That NeWS was Sun's, written in PostScript (a page-description language that is also a programming language) and designed by James Gosling is general knowledge, *unverified*.

**What rio is.** rio is the window system of Plan 9 (the research operating system that followed Unix at Bell Labs; Rob Pike wrote 8½ and rio descends from it). Its central idea, in the 8½ paper, is that the window system is a **file server**: "The most important contributor to its small size, though, is its overall design as a file server." A program uses a window by opening files (`/dev/cons` for text, `/dev/mouse`, `/dev/bitblt` or later `/dev/draw` for graphics); each window is a private screen, and the server keeps its bitmap and does the drawing ("no bitmap graphics operations are executed in the client at all"). It is one small server with no **toolkit** (a library of buttons, menus and other widgets that every program of a window system links in; that X has several and rio's base system none is general knowledge, *unverified*).

**What NeWS was.** In a conventional window system a program on the host sends the display server drawing commands one at a time over a slow link, and each round trip costs time (the latency complaint of 3.2). NeWS made the server programmable: the host *downloaded a program* into the display server, which ran it locally for every mouse movement, so dragging a shape never crossed the link. This is the Blit's idea one step on: the Blit downloaded a program into a *terminal*, NeWS into a *window server*. The repository's note in section 14G of `kernel-pain-points.md` says the NeWS argument "is the same shape as the Blit lineage" and records that the text read does not mention the Blit.

The chain for windows: the PDP-11's teletype had none; the Blit's `mux` (terminal-side windows over a serial line), 8½ and rio (windows as files, on a local bus), NeWS (windows as downloaded code).

**3.18.1** The pdp64 brings over what it can of the simplicity of rio and NeWS, the windowing systems that the Blit terminal inspired (the author, 2026-11-10: "we'd like to bring over what we can of that simplicity also", errata 106). *(D)* Details pending (J.3.14): which simplicities, and in what form.

## 3.19 What was checked, and what was not

- **Checked against the Linux headers on this machine:** `virtio_gpu.h` (command names and structures, formats and their values, the fence flag, cursor commands, `VIRTIO_GPU_MAX_SCANOUTS`, feature bits; no vertical-blank or flip command), `virtio_snd.h` (queues, formats, rates, `PCM_SET_PARAMS`, events, features), `/usr/include/drm/drm.h` and `drm_mode.h` (the page-flip and add-framebuffer ioctls and their flags, the mode structure) and `GL/glext.h` (the OpenGL entry points named).
- **Checked against QEMU sources** (`<work>/qemu-src`): `transfer_to_host_2d` and `resource_flush` and what they reject (`hw/display/virtio-gpu.c`); the `refresh_rate` field (`include/hw/virtio/virtio-gpu.h`); the audio backends (`audio/`).
- **Checked against 2.11BSD** (`<work>/facetlang/unixtree/2.11BSD/sys/pdp/cons.h.gz` and `cons.c.gz`): the KL11/DL11 register structure, address 0177560 and `DLXCSR_TRDY`.
- **Checked against the repository's records:** the Blit's statements and numbers (the Blit and Tradeoffs papers as recorded in `docs/kernel-pain-points.md` section 14I; the Handbook quotations in section 14G); the VT52 `ESC Y` code and the "VT100/ANSI" and "xterm-256color" remarks (`c64stuff.txt`, lines 218, 4875, 4888, the assistant's words).
- **Unverified** (general knowledge; no manual of these machines is on this computer): everything about the C64 chips (addresses, sizes, sprite size and count, colours, the SID's voices, the processor) and the IBM PC adapters (VGA text addresses and attributes, mode 13h and the planar modes, the 64 KB window, VBE, the palette, Mode X); the details of VT52 and VT100 sequences beyond the records; **what "vt55" refers to**; the speed of a teletype; which terminal

emulators use the GPU; the Valve signed-distance-field paper; the titles, dates and page counts of Michael Abrash's books; GPU lane group sizes; variable refresh and its names; the Vulkan names; EDID as a data block; the Amiga's blitter; the C128's chips; NeWS as Sun's, in PostScript, by Gosling; the toolkit comparison; the real mixers on other systems; the Cyclone V and Zynq 7020 lacking a hard PCI Express block; and the line counts of boilerplate (illustrative throughout).

# Chapter 4 — Boot, the BIOS, the disk label and the GPT

**The machine starts the way the PDP-11 did: a small ROM in the controller chip loads one block from the boot device and jumps to it, and every later stage is an ordinary program, open source, compiled by the project's own compiler.** This chapter is written for a reader who has keyed a bootstrap in on a PDP-11 front panel, used a boot ROM, or typed `boot` at the `:` prompt of Unix Version 7 or 2.11BSD, or who is learning computing from the start, and who has not met anything invented after the 1970s: no BIOS, no UEFI, no partition table, no device tree, no start-up of a second processor, no firmware update. It follows the author's order (errata 110): the PDP-11 boot (4.1), then the systems that came between, the VAX, the Commodore 64, the PC's BIOS with the MBR, and the GPT with UEFI (4.2), each with the one thing it added; then the pdp64's boot, stage by stage (4.3 to 4.9), with the numbered decisions where the reader meets them; then the five boots and the pdp64 side by side (4.11). The wider review of how other systems boot is `boot.md`. A fact about an older or another machine that could not be checked on this computer is marked *unverified* and listed in 4.12.

*How to read this chapter.* A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures are the kind a textbook draws for a linked list: a box is a piece of memory, a register or a program, an arrow says what is handed on. Code blocks are pseudocode (C-like, `#` starts a comment) unless the text says they are PDP-11 assembly; the PDP-11 code is quoted from the V7M sources on this machine. The decisions are the numbered statements. Where a statement is decided and its details are not yet worked out, the text says `Details pending` and points to appendix J.

## 4.1 What you already know: starting a PDP-11

A processor that has just been powered on has a problem that no program can solve for it: the program that loads programs has to come from somewhere, and the memory holds nothing it can trust. This is the **bootstrap problem** (after the old joke of pulling yourself up by your own bootstraps, shortened to "boot"). A PDP-11 solved it in three ways, each a smaller job for the operator than the one before.

**The front panel.** The console had rows of switches. You set the address switches to a memory address and pressed LOAD ADDRESS, set the data switches to a word and pressed DEPOSIT (which stores the word and steps the address), repeated that for each word of a short **bootstrap loader** (a program of a few dozen words that reads a bigger program from a device), set the start address and pressed START. The V7M `boot(8)` page still says so: "load block 0 of the boot device by fiddling with the console keys as appropriate for your hardware. If you have no appropriate ROM, some programs suitable for manual use are given below". (The switch mechanics are general knowledge, unverified.)

**The boot ROM.** Later machines kept the loader in a **ROM** (read-only memory: storage that holds its contents without power) on the processor board. The V7M tape bootstrap names one such card in a comment (`sas/htboot.s`: "The M9312, which is the only boot for the TS11 ... leaves the CSR address in r1"): the M9312 is a ROM card that holds a boot program per device kind and hands the program the device's register address in `r1` (read; that a console switch or command chose the device is general knowledge, unverified). 2.11BSD's block-0 programs show the other side of the contract: they begin `nop` then `br start`, with the comment "These two lines must be present or DEC boot ROMs will refuse to run boot block!", and start with `mov r0,unit` and `mov r1,csr`, so DEC's ROM evidently checks that block 0 begins with those two instructions (inference from the comment) and passes the unit in `r0` and the controller's address in `r1` (read: `mdec/rkuboot.s`). What the ROM did is the subject of this chapter in one sentence: it read **block 0** (the first 512-byte **block**, the unit a disk is read in, numbered from 0) of the boot device into low memory and jumped to it.

**Block 0, the primary bootstrap.** The program in block 0 is the **primary bootstrap**, small enough to fit in one block. The V7M page says it in one paragraph: "A PDP11 UNIX system is started by a two-stage process. The first is a primary bootstrap which is able to read in relatively small stand-alone programs; the second (called *boot*) is used to read in the system itself. The primary bootstrap must reside in the otherwise unused block zero of the boot device. It can be read in and started by the standard ROM programs, or if necessary by keying in a small startup routine." Here is the head of the primary bootstrap

for the RM02/3 and RP04/5/6 disks, `sys/mdec/hpuboot.s` (285 lines, counted), which shows what a block-0 program has to do for itself:

```
/ UNIX/v7m RM02/3 and RP04/5/6 block 0 bootstrap
/ Can only boot from unit zero.
/ Assumes disk at standard address (0176700).

core = 28.
.. = [core*2048.]-512.          / 28 K words = 56 KB, less 512 bytes: high in memory

start:
        mov     $..,sp          / the stack is just below the program's new home
        clr     r0
        mov     sp,r1
        cmp     pc,r1           / are we running below the intended origin?
        bhis    2f
1:      mov     (r0)+,(r1)+     / yes: copy this program up, out of the way
        cmp     r1,$end
        blo     1b
        jmp     (sp)
2:      clr     (r0)+           / clear core to make things clean
        cmp     r0,sp
        blo     2b
        mov     $drive,*$hpcs2  / initialize the disk hardware and select the drive type
        ...
        mov     $'\r,r0         / print CRLF and '#' as a prompt
        ...                     / read a pathname from the console, then walk the
                                / inodes from the root (inode 2) to find the file
        jsr     pc,*$0          / the loaded program is entered by a call
```

The ROM put this program at address 0. It copies itself to the top of the first 56 KB so that the kernel can be loaded at 0, clears the rest of memory, starts the disk, prints `#`, reads a file name, finds the file by reading the disk's **inodes** (the file system's table entries, one per file) from the root directory down, loads it and enters it. The file it loads is the second stage.

**The second stage, `boot`.** `boot` is an ordinary C program (`sys/sas/boot.c`, 272 lines, counted). The V7M page: "When read into location 0 and executed, *boot* sets up memory management, relocates itself into high memory, and types `Boot` and `:` on the console. Then it reads from the console a device specification followed immediately by a pathname", for example `hp(0,0)unix`: the device, the unit, and the block offset of the file system on it. "*Boot* finds the corresponding file on the given device, loads that file into memory location zero, sets up memory management as required, and calls the program by executing a `trap` instruction." So there were two prompts, `#` from the first stage (type `boot`) and `:` from the second (type `hp(0,0)unix`). There were two stages because block 0 holds 512 bytes of program, enough to find a file by name but not also a menu, a device table and an error message for every disk; and a bigger program with all of that can be an ordinary compiled file, replaced by copying a new one in. The page also provides `/boot.bu`, "a backup copy of the bootstrap", the PDP-11's first recovery measure.

**2.11BSD**, the same kernel grown up, is explicit about the limit and about what the stages hand on. Its `boot(8)` page: "a primary bootstrap (limited to 512 bytes) ... must reside in block zero of the boot device (the disklabel resides in block one) ... `disklabel(8)` is normally used to place a copy of the appropriate bootstrap in block zero of new file systems"; copies are kept in `/mdec`. The header of its `hpuboot.s` says "assembled size must be <= 512; if > 494, the 16-byte a.out header must be removed" and that features "had to be removed to make room". The next stage receives the machine **in registers**: the comment in `pdpstand/srt0.s` reads "the bootstrap passes the cputype through in r0", `r1` holds the controller's register address and `r3` the boot device's major and minor number; `pdpstand/boot.c` declares a `checkword` that is "one's complements of bootopts", and its comment says "If r2 (checkword) was the complement of bootopts, this is an automatic reboot, otherwise do it the hard way": `r2` carries the check word, so that the program can tell an unattended reboot, which passed its options, from a person typing at the prompt (read). That is the PDP-11 form of what the pdp64 does with `r0`, `r1` and the boot-information block (4.4, 4.6).

**`ODT` and the console.** When nothing booted you were not stranded: the processor or the ROM carried a tiny monitor, **ODT** (Octal Debugging Technique, a console program that lets you type an address, see the word there, change it, and start a program at an address). A PDP-11 with a dead disk could still be inspected and a bootstrap keyed in. (2.11BSD's `cons(4)`

page names it: "ODT (console debugger mode) prompt", entered by control-P on an 11/44, whose prompt is `>>>`, and by the front-panel halt switch on an 11/73, whose prompt is `@`; "the operation of the console in this mode varies per-processor" (read). That it can examine, deposit and start is general knowledge, unverified.)

**What a PDP-11 did not have.** It had no partition table on the disk: V7's `hp.c` compiled the disk's regions into the driver (`hp_sizes[8]`, eight pairs of a size and a first cylinder, with comments such as "cyl 44 thru 429"; the V7M version has 24 entries for three drive types), so changing the layout meant recompiling the kernel (4.5). It handed the kernel no description of the machine: the devices and their addresses were compiled in (chapter 6's vector table is that table) and the bootstrap passed three or four registers (4.6). It had one processor, so no other processor to start (4.7); no device that needed a program of its own before it would work; and no firmware, so no BIOS and no UEFI (4.2, 4.9). What the pdp64 keeps is the chain: a ROM that reads block 0 and jumps; a first stage that fits in a block; a second stage that is an ordinary program and finds the kernel by name.

## 4.2 What came between: the VAX, the Commodore 64, the PC with a BIOS, the GPT with UEFI

The author named four systems between the PDP-11 and the pdp64: "pdp11 boot, vax boot, c64 boot, the x86 bios boot with mbr, then finally the gpt/uefi" (errata 110). Each added or removed one thing from the PDP-11 chain, and each is told here as steps and with the one idea it contributed. The marks: **(read)** is read in the sources on this machine, **(repo)** is from the repository's documents, **(unverified)** is general knowledge, the handbooks and specifications not being here.

### The VAX: a console computer starts the machine

*What it added:* a processor whose only job is to start the main one, and a boot block of 15 sectors (the PDP-11's 512-byte block 0 and ROM made larger). The 4.3BSD sources are on this machine; the console and DEC's own boot program are not.

1. A **console processor** (on the VAX-11/780 a separate small computer) powers up and runs a self-test; it also provides the console commands and starts the other processors on multiprocessor models. (repo: `boot.md`; unverified otherwise)
2. On a boot command the console, or a boot ROM on the smaller models, reads the boot block (block 0) of the device into memory and starts it with registers describing the device. (repo; read for the registers below)
3. That first-level program is `raboot.c` in 4.3BSD-Tahoe. Its header says it "loads next 7.5Kbytes from boot sector of file system" to `0x50000` (sectors 1 to 15 of an 8 KB boot area, `BOOTLAST` 15) and "should work with any disk using 750 boot rom". It takes the Unibus I/O page address from `r1` (the adapter number is cut from it), the controller address from `r2`, the unit from `r3`, and the boot flags and partition from `r5`. (read)
4. The second level is `vaxstand/boot.c`: it takes `r10` (the device) and `r11` (the options), prints `Boot` and then the `:` prompt, opens a file by name and enters it. The default name is `hd(6,0,0) /vmunix` (`saio.h`). (read)
5. The kernel runs. In 4.3BSD `rpb.h` holds the "restart parameter block, which is a page in (very) low core which runs after a crash", a record for a dump routine and not a boot hand-off; what DEC's own boot program (VMB, a name from general knowledge) passed is unverified.

The 8 KB boot area of step 3 is where the pdp64's 8 KB boot area comes from (4.5.3).

### The Commodore 64: the ROM is the whole boot

*What it added:* nothing to load. All of this is general knowledge, unverified; nothing about the C64's boot is in the repository beyond the machine's description in chapter 3.

1. The 6510 processor, on reset, reads its start address from the two bytes at `0xFFFC`; they lie in the **KERNAL ROM**, the top 8 KB of memory, so the first code is in ROM and nothing is loaded.
2. The KERNAL's cold-start routine initialises the chips, tests the RAM, sets the vectors and the screen.
3. It looks at `0x8000` for the signature `CBM80`; if a cartridge carries it, the cartridge's start address is called (**cartridge autostart**).
4. Otherwise it enters the **BASIC ROM's** cold start, which prints its banner and `READY.`
5. The user types `LOAD` and `RUN`; the program comes from the datasette or from the 1541 disk drive, a separate machine with its own processor and ROM.

The C64 has no second stage: the ROM is the operating system, and a program on a cartridge or disk replaces it by hand. The pdp64 takes from it one point: the first code is in a ROM that storage cannot damage.

## The PC with a BIOS and an MBR: services in firmware, and a table in block 0

*What it added:* a firmware library the first stage can call, so that 512 bytes can read a disk; and a partition table sharing block 0 with the code. The facts come from the OpenBSD sources on this machine where they can, and are otherwise general knowledge, marked unverified.

**The BIOS.** The IBM PC of 1981 (unverified) kept a ROM called the **BIOS** (Basic Input/Output System). The name has stuck to four jobs that are worth pulling apart: (1) **bring-up and self-test**, initialising the chipset, sizing the memory, checking the basics (the power-on self-test); (2) **the first code**: find a boot device, read its first sector (the PC's word for a block, 512 bytes) to a fixed address and jump to it; (3) **a library of services** called by **software interrupt** (an instruction that behaves like a trap through a vector: the PDP-11's `emt` or `sys` is the same kind of thing), one service each for the disk, the screen, the keyboard and the memory map; (4) **configuration**, a setup screen and a small battery-backed memory for the clock and the settings. The third job is why a 512-byte boot sector can read the rest of a disk: it carries no disk driver, it calls the BIOS's. OpenBSD's PC first stage does exactly that: `biosboot.S` and `mbr.S` call `INT 0x13`, the BIOS disk service (read). The PDP-11's bootstrap carried its own driver (the `hpcs1` and `hpcs2` writes above); the PC's borrowed the firmware's.

**The steps** (the OpenBSD loaders are read; the rest is unverified):

1. The processor starts at the **reset vector**, `0xFFFFFFF0`, at the top of the 4 GB space, where the firmware flash is mapped, in 16-bit real mode. (unverified)
2. The BIOS runs its self-test (POST), initialises the chipset and memory and runs the **option ROMs** of add-in cards. (unverified)
3. It goes through the **boot-device order**, reads the first 512-byte sector, the **MBR**, to `0x7C00`, and checks the signature `0x55AA`; the code in the first 446 bytes runs. (the 446, the table and the signature are read in OpenBSD's `disklabel.h`; the address and the order are unverified)
4. The MBR code finds the **active** partition in the four-entry table and loads its first sector, the **partition boot record** (OpenBSD: `mbr.S`, which "finds the first active partition", then `biosboot`, whose manual page says "roughly 512 bytes of code"). (read)
5. That program, using `INT 13h` for every disk read, loads the next stage, `/boot`, which loads the kernel. (read: `biosboot.S`)
6. The kernel gets the memory map and the machine's description from the loader and the BIOS. (unverified)

**The boot sector and its 446 bytes.** The MBR (master boot record) is one block with three parts: 446 bytes of first-stage program, a **partition table** of four 16-byte entries (a **partition** is a named region of the disk, so that several operating systems can share one disk), then the two-byte signature `0xAA55` (the bytes `0x55`, `0xAA`). OpenBSD's `sys/disklabel.h` has it in C: `dmbr_boot[446]`, four `dos_partition` entries, `dmbr_sign` at byte `0x1fe`; an entry holds a flag byte (`0x80`, active), a type byte, and a start and a size of 32 bits each (read). The code gets 446 bytes because that is what a table and a signature leave in a sector. The 32-bit size caps the scheme at 2 TiB of 512-byte sectors ($2^{32}$ x 512, arithmetic), and four entries is why "extended" partitions with a chain of further tables exist. The OpenBSD chain is BIOS, `mbr.S`, `biosboot`, `/boot`, kernel: four programs before the kernel, against the PDP-11's three (ROM, block 0, `boot`) or V7's own count of two (block 0 and `boot`).

**Real mode.** The BIOS and the first stages run in **real mode**, the 16-bit mode of the 8086 of 1978 (unverified) that addresses one megabyte, the only mode the processor starts in. Getting out of it is the loader's job and one thing the pdp64 does not have to do (4.7).

## The GPT and UEFI: a table that outgrew the sector, and firmware that reads files

*What it added:* a partition table with 64-bit sector numbers, and a firmware that understands a file system and runs a program from it, so there is no boot sector to jump to.

**The GPT.** The four 32-bit entries ran out, and UEFI brought the **GPT** (GUID partition table; a **GUID** is a 128-bit identifier, long enough that two made anywhere will not collide). OpenBSD's `gpt_header` and `gpt_partition` structures (read) show it: the header sits in the second block (`GPTSECTOR 1`) with "EFI PART" in its first eight bytes; it holds a **CRC32** (a 32-bit check value computed over the bytes) of itself, the location of an alternate header (the **backup copy** at the end of the disk, so a damaged table can be rebuilt), and a CRC32 of the entry array; OpenBSD's constants are 128 entries of at least 128 bytes (a type GUID, a unique GUID, a 64-bit first and last block, attribute bits, a 36-character UTF-16 name). All values are little-endian (low byte first, the PDP-11's and the pdp64's byte order). The UEFI specification itself was not opened. Block 0 of a GPT disk is a **protective MBR**: an MBR with one entry of type `0xEE` ("EFI Protective Partition") covering the disk, so that an old tool that reads only the MBR sees a full disk and does not offer to partition it.

**UEFI.** The BIOS's successor is a firmware standard, **UEFI** (Unified Extensible Firmware Interface). The steps (OpenBSD's `efiboot` is read; the rest is unverified):

1. The firmware starts, through phases of its own, and runs its **boot manager**, which reads **boot variables** from non-volatile memory (**NVRAM**) to learn which loader to run. (unverified; the loaders call `GetVariable` and `SetVariable`: read)
2. It reads the GPT, finds the **EFI system partition** (`GPT_UUID_EFI_SYSTEM`, read) and its **FAT** file system (the MS-DOS file system, kept as the common denominator), and opens the loader file by name.
3. The loader is a **PE/COFF** program, the executable format of Windows (portable executable, from the Common Object File Format), where the PDP-11 and the BSDs use `a.out` or ELF. OpenBSD's arm64 loader begins "We need to be a PE32+ file for EFI" (read: `start.S`). The firmware runs it with a table of **boot services** (function pointers: allocate memory, read the memory map, read a file, write to the console, draw on the screen) and a few **runtime services** (clock, reset, variables) that stay after boot.
4. The loader calls the boot services and picks up the **memory map** and the **configuration tables** that describe the machine to the kernel: **ACPI** (a standard of tables, with a byte-code interpreter the kernel must carry) or a **device tree** (a tree of nodes naming each device and its addresses, flattened into a blob). OpenBSD's loaders look for both (`EFI_ACPI_TABLE_GUID`, `FDT_TABLE_GUID`; read).
5. It loads the kernel and calls **exit boot services** with the memory map's key, retrying once because the call refuses a stale key (`efiboot.c`, read); the firmware's boot services are gone after that.
6. The kernel runs with the memory map and the tables. With **Secure Boot**, each image is signed and checked against keys held in the firmware. (unverified)

Beneath this the firmware is a large program with phases of its own. The QEMU tree here carries built images of the open-source implementation EDK II (`pc-bios/edk2-*.fd.bz2`); the x86-64 code image is 3,653,632 bytes uncompressed (measured).

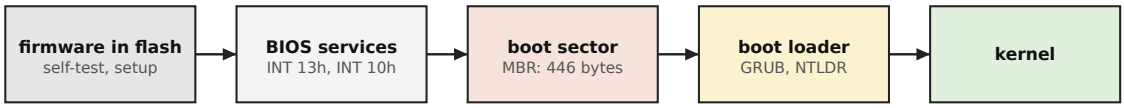
## The chain, and the six jobs side by side

From the PDP-11 to the pdp64 the first code moved from the operator's keys, to a ROM, to a console computer (VAX), to the processor's own ROM with nothing loaded (C64), to a BIOS with services (PC), to a firmware that runs files (UEFI); and the pdp64 returns to the ROM and block 0 of the PDP-11. The jobs, by system:

| Job | PDP-11 | PC with a BIOS | PC with UEFI | pdp64 |
|---|---|---|---|---|
| Bring-up | the processor and its ROM | the BIOS | the firmware | the controller chip |
| First code | ROM or front panel reads block 0 | BIOS reads the MBR | firmware runs a loader file | the boot ROM reads block 0 |
| Services for the loader | none: the bootstrap has its own driver | software interrupts | boot services | the loader has its own drivers |
| Finding the kernel | a name typed at : | partition boot record, `/boot` | the loader on a FAT partition | the second stage finds the kernel by name |
| Telling the kernel about the machine | registers | registers and the BIOS's memory map (unverified) | the memory map and ACPI or a device tree | the boot-information block |
| Starting other processors | none | INIT and START-UP signals | the same | the controller chip starts them (details pending, J.4.6) |

**three ways to start a machine: the PC with a BIOS, the PC with UEFI, the pdp64**

**PC with a BIOS (since 1981)**



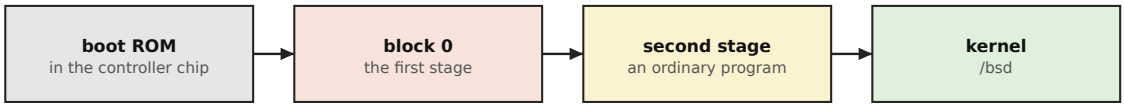

the loader calls the BIOS for every disk read, in 16-bit real mode, and has no drivers of its own

**PC with UEFI (since about 2005)**




no boot sector: the firmware parses the GPT and the FAT file system, runs the loader file, which calls the services and then gives them up

**the pdp64**




every stage after the ROM is an ordinary program built by the project's compiler

The PDP-11 chain was the pdp64's: ROM or front panel, block 0, boot, unix (4.1).
Both PC chains are general knowledge, unverified (4.12), except where the OpenBSD loaders show it: the amd64 tree on this machine has both, biosboot (862 lines) with boot (189), and efiboot (4,593 lines), measured (4.12).
Years are general knowledge, unverified. Real UEFI firmware has more phases than drawn, and keeps some code resident that the operating system cannot see.

## 4.3 What takes the place of a BIOS

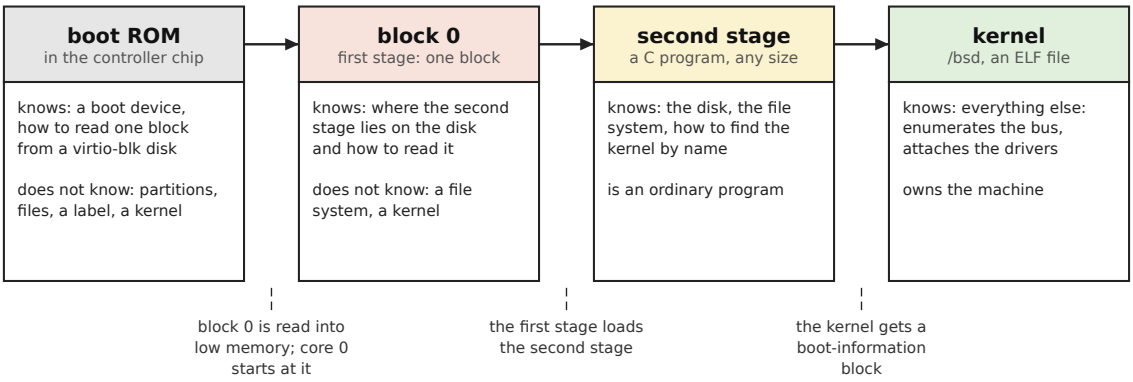
**4.3.1** The pdp64's "BIOS" is the controller chip's boot ROM and its monitor. *(D)*

**4.3.2** The boot software is open source and part of the specification: the ROM and monitor, the first-block program, the second-stage loader and the boot tools. *(D)*

**4.3.3** Everything in the project, the boot software included, is under the BSD 2-clause licence. *(D)*

**Why there are stages.** The chain is a ROM, a first block, a second stage and a kernel, and each exists because the one before it is too small to do the next one's job. The ROM sits in the controller chip, where every byte costs silicon: it reads one block from the boot device and does nothing else. The first block is 512 bytes, enough to find and read the next program. Only the second stage, an ordinary program loaded from disk, can be as large as it needs to be to understand a file system and find a kernel in it. The PDP-11 had the same chain (the `#` and `:` prompts of 4.1).

**the boot stages: each is small, knows one more thing than the one before, and hands on a little**




Stage sizes are the reason for the stages: the ROM and the first block are the smallest code that can be made to find the next, and only the second stage is big enough to read a file system.

The kernel is OpenBSD (errata 41 and 47).

**Why open source, and why it is in the specification.** The author's words (errata 47, 2026-10-22): "we'll have an open source boot sector boot loader as part of the spec; that is part of the spec that the boot loader/bios is open source." The firmware that ships on a PC is typically a vendor's build whose source the owner does not have (unverified), runs before anything the owner can inspect, and stays resident afterwards (4.2). In the pdp64 every stage after the controller's own sequencer is a published source file built by the project's compiler, and the ROM is in the same position, so no closed program sits at the root of the chain. The licence is that of the whole project (errata 48: "bsd license for everything"; the 2-clause variant is the assistant's choice of the common default, noted there). The UEFI specification itself is published (4.9), so what is closed on a PC is the shipped firmware and not the specification.

*Commentary.* **Chain.** The PDP-11's ROM and `ODT` did jobs 1, 2 and (as a monitor) 4 of the BIOS list; the PC added job 3, the services, so that a 512-byte sector could read a disk; UEFI turned job 2 into a loader file and job 3 into boot and runtime services. Other architectures fill the same roles under other names: Open Firmware on SPARC and PowerPC (OpenBSD's `sparc64/stand/ofwboot` is a loader for it, and QEMU carries `openbios-sparc64` and `openbios-ppc` images: read), U-Boot on ARM and PowerPC boards (QEMU carries `u-boot` images: read; its use on ARM is unverified), OpenSBI on RISC-V (QEMU carries `opensbi-riscv64-generic-fw_dynamic.bin`, and OpenBSD's riscv64 kernel calls it through `sbi_*` functions: read), the console of the Alpha (OpenBSD's `alpha/include/rpb.h` takes its restart block from DEC's system programmer's manual, 4.6; the name SRM is unverified) and the console processor of the VAX (4.2). x86 firmware also keeps resident system-management code that the operating system cannot see (unverified); arm64 and RISC-V servers need trusted firmware and usually UEFI beside it (unverified), though OpenBSD's arm64 and riscv64 ports do boot through `efiboot` (read). **Impact.** *Compiler:* builds the ROM, loader and tools.

## 4.4 The reset state and the first block

As on the PDP-11, a ROM reads block 0 of the boot device into low memory and starts core 0 there. The ROM is in the **controller chip** (the chip that holds the boot ROM, `boot.md`), which brings up the memory and holds every other core in **reset** (a state in which a core executes nothing and keeps no state). On the PDP-11 the equivalent was the processor's own power-up and the ROM or the front panel. The author summarised the PDP-11 way (2026-10-22) as "load 512 byte block 0 then jmp 0", "not precisely true, but directionally true"; here the "jmp 0" becomes a jump to the address block 0 was loaded to, because the first page holds the vectors.

**4.4.1** At reset the ROM in the controller chip reads block 0 of the boot device into low memory and starts core 0 at it, as the PDP-11's ROM did. *(D)*

Details pending: the address block 0 is loaded to and how many pages "the first few" are (J.4.1); how the ROM finds the boot device and maps its windows before any software has assigned them (J.4.3); which boot media the ROM tries, and in what order (J.4.7); the state core 0 starts in (J.4.9).

The ROM's job as pseudocode (an illustration of 4.4.1; the register state and the checks are pending):

```
rom_entry():                                # core 0; every other core is held in reset
    for dev in boot_order:                  # the order is pending (J.4.7)
        if not bring_up(dev): continue      # enumerate the bus, find the device, initialise it (4.8)
        if not read_block(dev, 0, LOAD):    # LOAD: the address is pending (J.4.1)
            continue
        info = fill_boot_info(dev)          # the hand-off of 4.6
        set up core 0 (mode, stack, registers: pending, J.4.9)
        core0.pc = LOAD                     # the first instruction of block 0
        return                              # core 0 now runs the block; the ROM is done
    monitor()                               # nothing booted: the monitor (4.10)
```

**The first few pages of memory.** The **vector table** (the table of handlers that an exception or an interrupt jumps through, chapter 6) is **one whole page**, 512 entries of 8 bytes, at address 0, because the system register `VBR` (vector base register) is zero at reset (6.12). The PDP-11's was a fixed table of a few hundred bytes at 0 to 0377, and its boot programs lived elsewhere. Block 0 is therefore loaded above the table, so that the first stage runs without overwriting the vectors. The author's rule of errata 102 is that no device window is mapped in **the first few pages**, where "the boot stuff is loaded" (2.3.5).

*Commentary.* **Chain.** PDP-11: ROM or keys read block 0 and jump, the boot device in registers. VAX: a console or a boot ROM does the same with the boot block and registers `r1` to `r5` (4.2). PC: the BIOS reads one 512-byte sector to a fixed address (unverified: the address); the IBM mainframes' IPL record and the Alpha console's boot block are the same idea (unverified); ARM boards load a first-stage loader from the boot medium (unverified). UEFI: no block 0, a file run from a FAT partition. pdp64: the PDP-11 step again.

## 4.5 The disk: a GPT around the BSD label

**What a partition is, and why V7 had none on the disk.** A disk is one long run of blocks, and a computer wants several regions of it: a root file system, a swap area, a place for the users' files. A **partition** is such a region, and something has to record where each begins and ends. There are three places for the record, and the chain from the PDP-11 visits each. V7 put it **in the kernel**: the disk driver compiles in a table (`hp_sizes[8]` in V7's `hp.c`), so changing the layout means a new kernel; 4.3BSD's `vaxmba/hp.c` also compiles in its size tables (`rp06_sizes[8]`, `rm03_sizes[8]` and others). The 4.3BSD-Tahoe `disklabel.h` (1988: its header is dated 6/27/88) put it **on the disk**: a **disk label**, a structure in one of the first blocks holding the disk's geometry and a table of its partitions, "in block 0 or 1, possibly offset from the beginning to leave room for a bootstrap", marked by the magic number `0x82564557`. 2.11BSD's block 0 holds the bootstrap and its block 1 the label, the same two places as here. OpenBSD's label is its descendant: the same magic number, **16 partitions** (`MAXPARTITIONS16`, against Tahoe's 8) and sector counts beyond 32 bits (`d_secperunith` holds the high part). The third place is the PC's, a **table in the first sector** (the MBR table of 4.2, then the GPT).

**Why the pdp64 has both a GPT and a label.** The author (errata 50, 2026-10-22, R260): "fine, we will have the gpt wrapper for the disk label." The disk carries a GPT with a protective MBR, so that other operating systems and partitioning tools recognise it, and the operating system's own partition (type `GPT_UUID_OPENBSD`) holds the BSD label, which says where the root file system, swap and the rest are. The label alone would be smaller and unreadable to everything else; the wrapper costs a GPT reader in the second stage and the kernel. The author added (errata 51, "follow the majority as far as boot block offset"): inside the OS partition the label is at **sector 1, offset 0**, the sector that most of OpenBSD's ports use, leaving sector 0 for a boot block.

**Block 0 is the protective MBR.** Because the disk uses the GPT wrapper (errata 50), block 0 is the protective MBR of that wrapper: 446 bytes of boot code, a partition table (one entry, type `0xEE`) and the signature `0xAA55` (the bytes `0x55`, `0xAA`). The first stage is the code in those 446 bytes.

**4.5.1** The disk carries a GUID partition table with a protective MBR; the operating system's own partition holds the BSD disk label, which names the root, swap and other partitions. *(D)*

**4.5.2** Block 0 is the protective MBR, so the first stage lives in its 446 boot-code bytes (errata 50). *(D)*

Details pending: what the first stage carries to find the second stage (J.4.8), and how the first stage, which is one block of code, reads blocks from the disk (J.4.2).

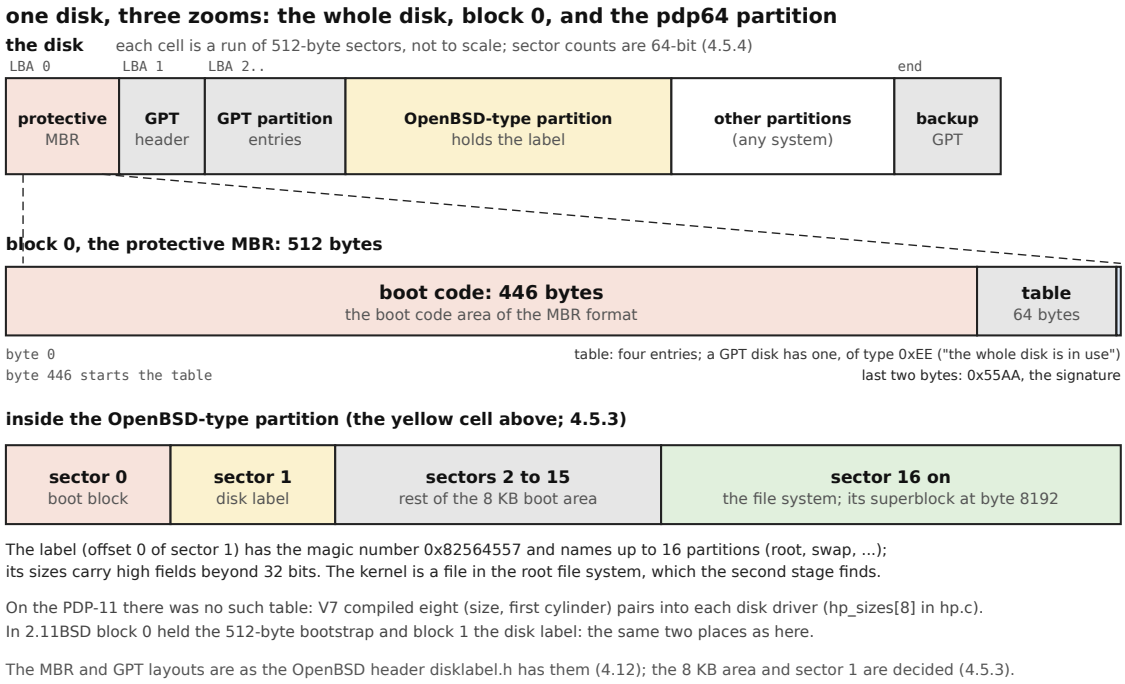
The first stage as pseudocode (an illustration of 4.5.2):

```
first_stage():                                  # at LOAD: 446 bytes of code
    read the second stage from the disk         # where it lies and how it is read: pending (J.4.8, J.4.2)
    jump to the second stage, with the boot device and the boot-information block as the ROM gave them
```

**4.5.3** Inside the BSD partition, the label is at sector 1, offset 0; the boot area is 8192 bytes at the start of the partition; the file system begins after it, its superblock at offset 8192. *(D)*

| Sector (512 bytes) of the BSD partition | Contents |
|---|---|
| 0 | the partition's own boot block |
| 1 | the disk label, at offset 0 |
| 2 to 15 | the rest of the 8 KB boot area |
| 16 on | the file system, superblock at offset 8192 |

**4.5.4** Sector counts are 64-bit throughout; the label has 16 partitions and its sizes carry high fields beyond 32 bits. *(D)*




The label (offset 0 of sector 1) has the magic number 0x82564557 and names up to 16 partitions (root, swap, ...); its sizes carry high fields beyond 32 bits. The kernel is a file in the root file system, which the second stage finds.

On the PDP-11 there was no such table: V7 compiled eight (size, first cylinder) pairs into each disk driver (hp_sizes[8] in hp.c). In 2.11BSD block 0 held the 512-byte bootstrap and block 1 the disk label: the same two places as here.

The MBR and GPT layouts are as the OpenBSD header disklabel.h has them (4.12); the 8 KB area and sector 1 are decided (4.5.3).

The **superblock** (the file system's first record: its size, its block size and where its tables are) sits at byte 8192 of the partition, after the 8 KB boot area that the BSD tradition reserves for the bootstraps (4.3BSD-Tahoe's `disklabel.h` comments the area as "primary (block 0) and secondary (block 1-15) bootstraps", read; errata 51: sector 0 the primary bootstrap, sectors 1 to 15 the label and a secondary bootstrap; the VAX's `raboot` of 4.2 loads sectors 1 to 15).

Details pending: where the second stage lies on the disk, and whether a dedicated boot partition is needed (J.4.8).

*Commentary.* **Chain.** V7: sizes in the driver, nothing on the disk. 4.3BSD-Tahoe: the label on the disk. PC: an MBR table of four 32-bit entries, then the GPT with a protective MBR. pdp64: the GPT outside, the BSD label inside. OpenBSD's tree has seven architecture headers with the label at sector 1 offset 0 (amd64, i386, arm, arm64, riscv64, hppa, powerpc64) and Alpha and SPARC64 at sector 0 (offsets 64 and 128); the port adds one header of a dozen lines. **Trade-offs.** The label alone is the smallest design and unreadable to other systems' tools; the GPT wrapper costs a GPT reader in the second stage and the kernel (OpenBSD's kernel already has one) and gives a disk other systems recognise. **Benefits.** The label code, `disklabel`, `newfs` and `installboot` come with OpenBSD. BSDs on UEFI machines combine the same pieces, a GPT with the label inside an OpenBSD-type partition and a FAT partition for the loader (inference: OpenBSD's `disklabel.h` defines `GPT_UUID_OPENBSD` and `GPT_UUID_EFI_SYSTEM` and its `efiboot` reads the latter); Linux has no label of its own (unverified). **Impact.** *Kernel:* the label code and tools come with OpenBSD. *Compiler, hardware:* none.

## 4.6 The second stage and the hand-off

The second stage does what the PDP-11's `boot` did: it finds the kernel by name, loads it and enters it (4.1). Being an ordinary program loaded from disk, it can be as large as it needs and be replaced by copying a file. It has its own drivers and no firmware to call.

**The hand-off: how the kernel learns about the machine.** The PDP-11 had four registers and a kernel built for the machine. A kernel that is to run on any pdp64 has to be told about the machine it is on. The way to tell it is a **boot-information block**, a structure in memory. The chain: the PDP-11 passed registers, as did the VAX (`r10`, `r11` to `boot`, 4.2); UEFI passes a memory map and **ACPI tables** or a **device tree** through configuration tables (4.2), and an operating system for several architectures carries a parser for each. The VAX's restart parameter block is a crash-dump record, not a hand-off (4.2); the Alpha's hardware restart parameter block, in OpenBSD's `alpha/include/rpb.h` (read), is a larger record of the same kind: the magic "HWRPB", the page size, the primary CPU, and offsets to per-CPU, console-terminal and memory-data tables, taken from DEC's system programmer's manual. The VAX and Alpha blocks are fixed-layout records of the machine; x86 and ARM servers pass ACPI tables, arm64 and RISC-V a device tree (OpenBSD's loaders look for both, 4.2).

**4.6.1** The second stage carries its own drivers, finds the kernel, loads it and enters it (errata 49: drivers are small, so the second stage carries them). *(D)*

**4.6.2** The kernel is told about the machine through a boot-information block; because the bus describes its own devices, the kernel enumerates them and the block does not list them (2.4). *(D)*

Details pending: which fields the boot-information block has, who fills each, and its layout, widths, checksum and version (J.4.4); how the second stage names the kernel and its file format.

The second stage as pseudocode (an illustration of 4.6.1 and 4.6.2; the file system, the file name and the format are pending):

```
second_stage():                              # loaded by the first stage
    dev = open_boot_device()                 # its own driver (4.6.1)
    read the GPT, then the BSD label of the OS partition (4.5)
    find the root file system and the kernel by name in it
    load the kernel into memory
    add to the boot-information block what only the second stage knows (4.6.3)
    jump to the kernel with the boot-information block
```

**4.6.3** The boot-information block carries the video device's base address, size, pitch and pixel format when the boot loader has set it up, and nothing about the device's clock or power state is changed behind the kernel: its pixel clock stays up until the kernel's driver owns the device. *(D)*

Statement 4.6.3 exists so that the loader can set up a picture for the boot messages and the kernel can keep drawing on it until its own driver takes over, without the screen flickering or going blank: the **framebuffer** the loader set up is described in the block, and the device is left as it is. (That the PC learned this the hard way is the assistant's earlier wording and is unverified.)

*Commentary.* **Chain.** PDP-11: a kernel configured at build time, three or four registers. VAX: registers again (`r10`, `r11`), and a restart block. Alpha: the hardware restart parameter block. UEFI-era systems: ACPI tables or a device tree. pdp64: a boot-information block.

## 4.7 Starting the other cores

**The problem the PDP-11 did not have.** A PDP-11 had one processor, so "start the machine" was one thing. A pdp64 may have several **cores** (processors on one chip; chapter 7), and at reset only core 0 runs the boot code. The kernel has to start the others, each with its own stack and its own idea of who it is. The precedents, from the boot review (`boot.md`; counts measured there in Linux `7be4bc52` and re-counted for this chapter): on x86 the boot core sends INIT and START-UP signals through the interrupt controller (Linux's comment: "INIT, INIT, STARTUP sequence"), the START-UP carrying an address below one megabyte; the other core wakes in the 8086's **real mode** (4.2) and a hand-written **trampoline** (a short program that takes a processor from a limited mode to its full one, through two more modes) must walk it into 64-bit mode. Linux's `arch/x86/kernel/smpboot.c` is 1,528 lines and `arch/x86/realmode` another 1,258. arm64 asks firmware through a call, PSCI, in 124 lines (`arch/arm64/kernel/psci.c`), or has the waiting cores **spin** (loop on a word in memory) on a **spin table** until the boot core writes an address there, in 127 (`smp_spin_table.c`); RISC-V asks the firmware to start a **hart** (its word for a

core) in 108 lines (`cpu_ops_sbi.c`) or spins in 58 (`cpu_ops_spinwait.c`). That the VAX console processor started the VAX's other processors is unverified.

**4.7.1** At reset only core 0 runs; at least one other core exists, so a way to start it is always used, and the controller chip provides the way in which the other cores are started. *(D)*

Details pending: how a core other than core 0 is started, and what state it starts in (J.4.6).

## 4.8 Starting the devices

**The INIT bit.** On the Unibus every device had a **control and status register** (CSR) with a bit that told the device "initialise yourself" and a bit that said it was ready: you wrote `INIT`, waited for `READY`, and the device was in a known state (the RK11 has `GO` and `RDY` in its CSR and the RX02 floppy controller a bit named `RX_INIT`; 5.1 shows the driver code for each). All the work happened behind the register, in the device's own logic. The pdp64 keeps the idea and makes it a conversation: under virtio (2.7.3) the driver writes **zero** to the device's `device_status` byte, which resets it, then sets the bits `ACKNOWLEDGE`, `DRIVER`, `FEATURES_OK` and `DRIVER_OK` in order, each meaning "I have got this far", and the device answers by refusing or accepting the features the driver chose. That is the `INIT` and `READY` of the CSR spread over a few steps. Said plainly: `INIT` and `READY` are the PDP-11's way of starting a device, and the virtio `device_status` handshake is their descendant. 5.1 and 5.5 explain the idea and what a driver does with it, and 2.7.3 gives the bits and `init_device(fn)`, the pdp64's device start, so they are not repeated here.

**4.8.1** A device is initialised by a write to a register of the device that tells it to initialise itself, and the initialisation happens behind the bus; a device may also be given values, or a binary firmware image, that it loads. *(D)*

Details pending: how a virtio device is given firmware, and which devices need it (J.4.5).

## 4.9 Why UEFI is not used

**4.9.1** The pdp64 does not use UEFI: its boot is the block-0 route of sections 4.4 and 4.5, its hand-off is the boot-information block, and its devices describe themselves. UEFI firmware is not part of the specification and is not built. *(D)*

**4.9.2** The hand-off and the second stage are kept simple enough that a UEFI shim could be written by someone who wanted one. *(D)*

Both statements are the author's decision of 2026-11-10 (errata 109). The record, in his words (the 2026-10-22 messages are rows R256 and R257 of the decision register; the 2026-11-10 ones are quoted from errata 109 and 110 and are not in that register). On 2026-10-22 he asked, of boot, "look into uefi; is it worth it at all? can't we do what pdp11 used to do, load 512 byte block 0 then jmp 0? (not precisely true, but directionally true)", and asked how UEFI plays into partitions and the GPT; the review (`boot.md`) answered that UEFI was not worth it as the machine's own boot, and the earlier text's proposals for the boot were never a decision. On 2026-11-10 he asked "i thought we did decide to allow uefi? check all the threads where we discussed it" (no decision to allow UEFI was found in any recorded message), then "or we kept the originnal simple pdp11 boot block method?", and, given three options, chose "option 2, block 0 with a shim possible".

**Is UEFI an open specification, and what does an open-source system do with it?** The author asked, after the decision (2026-11-10): "if uefi isn't an open spec how are all the open source os supporting it? it requires binary firmware blobs?" By what could be checked:

- **The specification** is published by the UEFI Forum, the industry body that maintains it, and can be read without charge (general knowledge, unverified: it is not on this machine, and whether implementing it needs a membership, a licence or a compliance test is not known here). The closed part is elsewhere.
- **An open-source implementation exists.** EDK II, the TianoCore project's reference implementation, is under BSD-2-Clause-Patent (read: `<work>/qemu-src/pc-bios/edk2-licenses.txt`, and `/usr/share/doc/ovmf/copyright`, "Upstream-Name: edk2"). Built images are on this machine (QEMU's `pc-bios/edk2-*.fd.bz2` and `/usr/share/OVMF`, including `OVMF_CODE_4M.secboot.fd`), so an emulated PC boots under open firmware. The source is not here (`roms/edk2` is empty), so nothing about its size or contents was read.
- **Open-source operating systems write the loader side.** OpenBSD's `efiboot` (read) runs on top of firmware already on the machine and calls its services; it ships no firmware. NetBSD has its own `efiboot` in `sys/arch/i386/stand/efiboot`, which calls `ExitBootServices` like OpenBSD's (read). Linux takes the other route: the kernel is itself a PE/COFF image that the firmware loads, "the EFI boot stub" (`Documentation/admin-guide/efi-stub.rst`, read), so no separate loader exists; the stub still runs on firmware already present. None of the three ships firmware, and a loader needs no blob.

- **The proprietary blobs are in the firmware shipped on real PCs:** the vendor's builds, and the CPU vendor's initialisation code that brings up memory and the chipset, which many boards include as a binary (unverified, not on this machine). That is a property of how PCs are made, not of the UEFI specification.
- **Secure Boot** adds a chain of signed loaders and keys kept in the firmware's variable store (unverified; the `secboot` image names above are all that was seen).

The author's conclusion (2026-11-10): "I guess leaving the door open to uefe support but having the classic mbr fallback, gpt wrapper is a good choice then." That **confirms option 2** of errata 109 and decides nothing new (errata 110). The two routes share one disk: block 0 of the pdp64 disk already has the protective MBR and GPT layout (errata 50, 4.5), so the classic route (the ROM reads block 0) and a future UEFI route (a firmware, or the shim, reads the GPT and a FAT system partition) read the same disk without repartitioning it.

*Commentary.* **Chain.** PDP-11 and VAX: no firmware services. PC: the BIOS, then UEFI to give one installer a way to boot on any machine offering it. OpenBSD's arm64 and riscv64 ports use UEFI and a device tree, with loaders of 7,195 and 4,513 lines; for amd64 the loaders total 15,440 lines, a BIOS boot block of 862 lines and a second stage of 189 beside the UEFI loader of 4,593 (counted; the total includes `libsa`, 6,319). pdp64: block 0, with the door left open. **Trade-offs.** UEFI firmware would let existing generic loaders run unchanged and offer a standard memory map and a secure-boot chain; it costs a large firmware, adds failure modes (variable storage, drivers, boot-order state), enlarges the attack surface, and still needs a device tree or ACPI to describe the devices. **Benefits.** The pdp64 has one architecture and one platform description, the case UEFI was not built for. **Impact.** *Kernel:* no firmware interface, no memory-map translation in the machine itself. *Hardware:* no firmware stack.

## 4.10 Recovery

**4.10.1** The controller's ROM includes a monitor, a small program that takes over when nothing boots and offers a way in to examine and start the machine. *(D)*

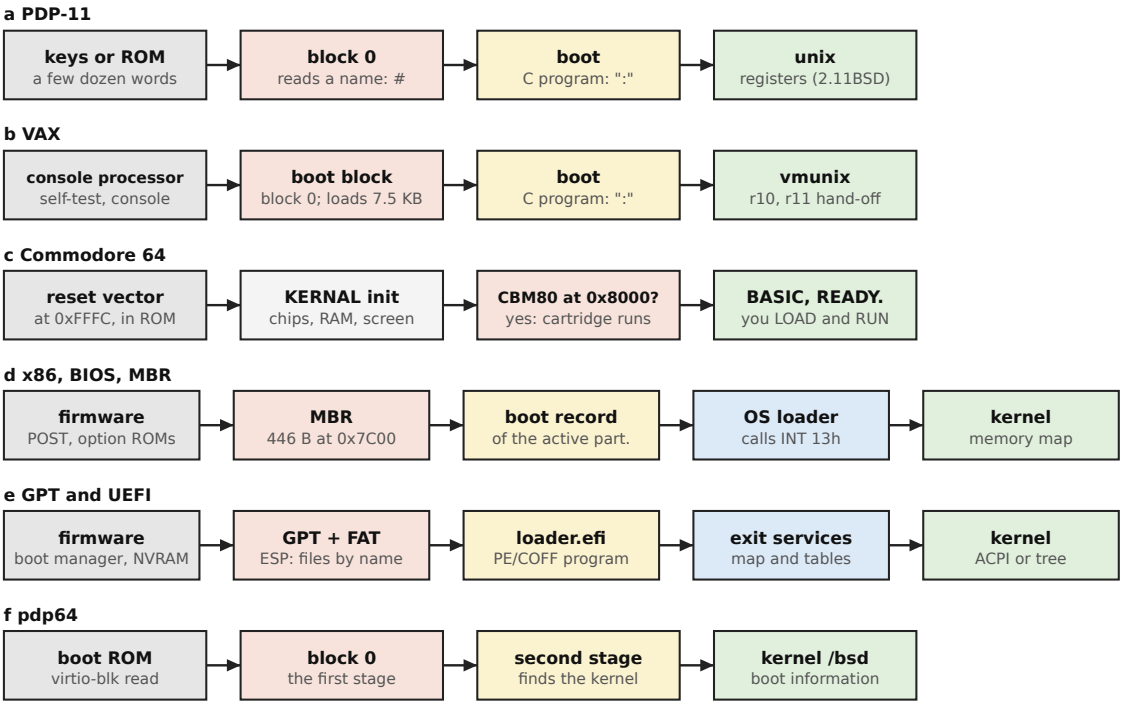
A PDP-11 with a dead disk was not a dead machine: `ODT` let you examine memory and key a bootstrap in (4.1), and V7 shipped `/boot.bu`, a backup of the second stage. The monitor is the pdp64's form of the same idea, and the ROM that reads block 0 (4.4) also holds it. Details pending: its commands, how it is reached, its console and speed, and what is kept in non-volatile memory (J.4.10). As pseudocode (an illustration; the command letters and the prompt are not decided):

```
monitor():                                  # the ROM's console program
    loop:
        print("pdp64> ")
        line = read_line(console)
        case first word of line:
            "x" addr [n]    : print n words at addr                 # examine (the PDP-11's `addr/`)
            "d" addr value  : store value at addr                   # deposit
            "g" addr        : start core 0 at addr
            "b" device      : try to boot from this device
            otherwise       : print("?")
```

## 4.11 Five boots side by side

The five referents in the author's order, and the pdp64 as the sixth, as the chains of 4.1 and 4.2 and 4.3 to 4.9 give them. A dagger marks a cell that rests on general knowledge, unverified; the last column says which of the five the pdp64 resembles.

**five boots and the pdp64: who runs, in what order, and what each stage knows**




Sources: a and b read in V7M, 2.11BSD and 4.3BSD-Tahoe; d and e partly read in OpenBSD; c, and the rest of b, d and e, are general knowledge, unverified (4.12).

The pdp64 (f), in one paragraph: the ROM in the controller chip runs on core 0, reads block 0 through a virtio-blk request and starts core 0 at it (4.4); the first stage in block 0 loads the second stage (4.5); the second stage finds the kernel by name and enters it with a boot-information block (4.6); the kernel enumerates the bus.

| Property | (a) PDP-11 | (b) VAX | (c) C64 | (d) x86 BIOS, MBR | (e) GPT, UEFI | (f) pdp64, and which it resembles |
|---|---|---|---|---|---|---|
| Who runs first | the processor, then the operator's keys or a ROM† | the console processor† | the 6510 from its reset vector† | the processor at `0xFFFFFFF0`† | the same, then the firmware† | the ROM on core 0: like (a) and (b), without the operator |
| Where the first code lives | front panel or boot ROM | console ROM and storage† | the KERNAL ROM† | firmware flash† | firmware flash† | a ROM in the controller chip: like (a) and (c) |
| How big the first code may be | a few dozen words (keys)†, a ROM's size†; block 0 of 512 bytes (read) | the boot block; the 750 loaded 7.5 KB next (read) | the ROM's 8 KB† | 446 bytes in the MBR (read) | the loader file, any size† | the ROM's size; block 0 of 512 bytes: like (a) |
| How it finds the next stage | block 0 reads a name (`boot`) by walking inodes; `boot` reads `hp(0,0)unix` (read) | the boot block loads sectors 1 to 15; `boot` opens a name (read) | it does not: `LOAD` by the user† | the active flag in the table, then `INT 13h` reads (read) | the boot variables and the GPT's type, then a name on FAT† | block 0 loads the second stage, which finds a name: like (b) and (a) |
| What the hand-off carries | registers (2.11BSD: `r0`, `r1`, `r3`, a check word) (read) | `r10`, `r11` to `boot` (read) | nothing: the program owns the machine† | registers and the BIOS's memory map† | the memory map and ACPI or a device tree (read) | the boot-information block: like (e) in being a description of the machine |
| How it is repaired | keys, `ODT`†, the backup `/boot.bu` (read for the last) | the console† | the ROM cannot be damaged by storage; a cartridge† | re-write the MBR from another disk† | the firmware's setup and boot manager† | by writing the disk from another machine: like (d) |

| Property | (a) PDP-11 | (b) VAX | (c) C64 | (d) x86 BIOS, MBR | (e) GPT, UEFI | (f) pdp64, and which it resembles |
|---|---|---|---|---|---|---|
| What is open source | the BSD and V7 stages (the sources here); ROMs† | the BSD stages (read); the console† | the ROMs are Commodore's† | the loaders, in OpenBSD (read); the BIOS is a vendor's† | the specification is published†; EDK II is BSD-2-Clause-Patent (read, 4.9); shipped firmware often not† | everything, by decision (4.3.2) |
| How many programs before the kernel | 3 (ROM or keys, block 0, `boot`) | 3 (console, boot block, `boot`) | 1 (ROM) or 2 with a cartridge† | 4 (BIOS, `mbr.s`, `biosboot`, `/boot`) (read) | 3 or more, with firmware phases† | 3 (ROM, block 0, second stage): like (a) and (b) |
| What recovery exists | `ODT` and the front panel† | the console† | reset, a cartridge† | the BIOS setup, a rescue disk† | the setup, rescue media, a fallback loader name† | the same disk readable by a UEFI route (4.9) |

Property by property, the pdp64's boot is the PDP-11's and the VAX's: a ROM, a first block, a second stage that finds a file by name, a hand-off to the kernel. It takes two things from the PC: the protective MBR and the GPT (d, e), and, in the boot-information block, a description of the machine (e) that the PDP-11 and the VAX did not pass. From the C64 it takes the ROM as the first code, undamaged by what is on storage; the C64 had no second stage. It differs from all five in that every stage is open source.

## 4.12 What was checked, and what was not

The rule of the project is: check, do not guess. The handbooks and specifications are not on this machine. What each fact rests on:

- **Unix sources** (`<work>/facetlang/unixtree`, files are gzipped): V7M `sys/mdec/hpuboot.s` (285 lines), `sys/sas/boot.c` (272 lines), the `boot(8)` page, `sas/htboot.s` (the M9312 comment), `dev/hp.c` (24 entries); 2.11BSD `man4/cons.4` (`ODT`), `mdec/rkuboot.s` (the `nop` and `br` that DEC's ROMs require, `r0` and `r1`), `pdpstand/boot.c` (the `r2` comment); V7 `usr/sys/dev/hp.c` (`hp_sizes[8]`, "cyl 44 thru 429"); 2.11BSD's `boot(8)` page, `sys/mdec/hpuboot.s` (the 512 and 494 bytes), `sys/pdpstand/srt0.s` (`r0`, `r1`, `r3`) and `boot.c` (`checkword`), `sys/h/disklabel.h`; 4.3BSD-Tahoe `h/disklabel.h` (the block 0 or 1 comment, the 8 KB boot-area comment, `0x82564557`, 8 partitions), `mdec/raboot.c` (registers, 7.5 KB, `0x50000`, `BOOTLAST`), `vaxstand/boot.c` (`r10`, `r11`, `Boot`, `:`), `vaxstand/saio.h` (the default name), `vax/rpb.h`; 4.3BSD `vaxmba/hp.c` (size tables); OpenBSD 4.6 `sparc/stand/installboot/installboot.8`.
- **OpenBSD** (`<work>/kernel-src/openbsd`, commit `2114d7b7`): `sys/sys/disklabel.h` (`DOSPARTOFF` 446, the signature at `0x1fe`, type `0xee`, "EFI PART", `gpt_header`, `gpt_partition`, `NGPTPARTITIONS` 128, `GPTMINPARTSIZE` 128, `GPTPARTNAMESIZE` 36, `MAXPARTITIONS16`, `d_secperunith`, both GUIDs); the `arch/*/include/disklabel.h` label sectors; `arch/amd64/stand` (`mbr.S` 521 lines, `biosboot.S` 862 and `biosboot.8` ("roughly 512 bytes of code", the `installboot` patching, "its inode may change"), `boot/` 189, `libsa` 6,319, `efiboot` 4,593, 15,440 in all); `arch/arm64/stand/efiboot` (7,195 lines, `start.S` 176, `self_reloc.c` 124) and riscv64 (4,513); the `ExitBootServices` retry.
- **Other trees:** OpenBSD `share/man/man9/loadfirmware.9`, `arch/alpha/include/rpb.h`, `arch/sparc64/stand/ofwboot`, the `sbi_*` calls in `arch/riscv64`; NetBSD `share/man/man9/firmload.9` and `sys/arch/i386/stand/efiboot`; QEMU `pc-bios` (the `openbios`, `u-boot` and `opensbi` image names); Linux `include/linux/firmware.h` and `Documentation/admin-guide/efi-stub.rst`.
- **Linux** (`<work>/kernel-src/linux`, `7be4bc52`), re-counted: `smpboot.c` 1,528 lines, `arch/x86/realmode` 1,258 (`.c`, `.h`, `.S`), PSCI 124, spin table 127, RISC-V SBI start 108 and spin 58.
- **EDK II:** the licence in `qemu-src/pc-bios/edk2-licenses.txt` and `/usr/share/doc/ovmf/copyright`; the x86-64 code image 3,653,632 bytes uncompressed; the `secboot` image names in `/usr/share/OVMF`; no source in `roms/edk2`.
- **The repository:** the statements of this chapter against `docs/errata.md` (47, 48, 50, 51, 95, 101, 102, 103, 109, 110), `docs/boot.md`, chapters 2, 5 and 6 (2.3.5, 2.7.3, 5.1, 6.12), and the author's messages of 2026-10-22 in `docs/decision-register.md` (R256, R257, R260).
- **Unverified** (general knowledge): the front panel's switches and keys and the bootstrap loader of a few words; that a console switch or command chose the boot device; that `ODT` can examine, deposit and start; the date of the IBM PC and the 8086; the BIOS's four jobs and its other services; real mode and the 1 MB limit; the reset vector `0xFFFFFFF0`, the load address `0x7C00` and the boot-device order; the whole Commodore 64 boot (`0xFFFC`, KERNAL, `CBM80` at `0x8000`, BASIC, datasette, 1541); the VAX console processor, VMB and that the console started other processors; the name SRM for the Alpha console; the IBM mainframe IPL record; ARM first-stage loaders; resident system-management code in x86

firmware; the UEFI boot manager, NVRAM variables and Secure Boot; that the UEFI specification requires PE32+ images and FAT on the system partition; the publication terms of the UEFI specification; that the PC learned the video lesson of 4.6.3 “the hard way”.

- **Decided by the author and recorded in `errata.md`:** everything stated as a decision with a tag *(D)*. The author’s 2026-11-10 words are quoted from errata 109 and 110, which are not in the decision register.

# Chapter 5 — The device driver model

**A driver on the pdp64 is an ordinary program that finds a device by its identifiers, agrees features with it, gives it queues in memory, hands it requests as chains of descriptors and learns from the used ring that they are done, by an interrupt or by polling; because every standard device speaks virtio, one transport routine serves the disk, the network card, the display and the sound device, and each has a thin class driver on top.** Open: the form of the console, the timer and the entropy source (J.5.1).

This chapter is written for a reader who has written or read a driver for V6, V7 or 2.11BSD Unix (a `strategy` routine, a `struct buf`, the CSR of an RK11, an interrupt vector pair), or who is learning computing from the start, and who has not met virtio, PCI Express, MSI-X, a driver that attaches by matching identifiers, or a machine description that is not compiled into the kernel. It runs from what that reader knows to what the pdp64 does: the PDP-11 driver (5.1), the systems between (5.2), what a driver is now (5.3), the interface it programs (5.4), its life (5.5), the disk (5.6), the console, timer and entropy source (5.7), the network, display, sound and USB drivers (5.8), interrupts (5.9), the memory a device reaches (5.10), errors (5.11), firmware (5.12) and where drivers run and how they are written in C (5.13) The bus, the virtio structures and the MSI-X table are chapter 2; this chapter uses them and does not repeat them. A fact about an older or another machine is marked **(read)** when it was read in a source on this computer, **(repo)** when it comes from the repository's documents, and *unverified* when it is general knowledge that could not be checked here; 5.14 lists what was checked.

*How to read this chapter.* A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures are the kind a textbook draws for a linked list: a box is a piece of memory or a program, an arrow says what is handed on. Code blocks are C-like pseudocode (`#` starts a comment, `mmio_read32(address)` and the like are ordinary loads and stores to a device register) unless the text says they are V7 or 2.11BSD C, quoted from the sources on this computer. Every pdp64 listing in this chapter is an illustration. The decisions are the numbered statements.

## 5.1 What you already know: a PDP-11 driver

A **driver** (the part of the kernel that knows how to operate one kind of device, and nothing else) in V7 or 2.11BSD has four parts: a structure laid over the device's registers, entry points listed in a table, an interrupt routine, and a way of being found and started. The disk driver `rk.c` has all four (135 lines in V7, 204 in 2.11BSD, counted), and the console driver is a second example.

**The registers.** V7's `rk.c` begins with `#define RKADDR ((struct device *)0177400)` and declares the structure `struct device { int rkds; int rker; int rkcs; int rkwc; caddr_t rkba; int rkda; }`: six words at a fixed address, which chapter 2 (2.1) explains and draws. The driver reads and writes the fields. In the control and status register (the **CSR**, one word whose bits both command the device and report on it) the bits it needs are GO (01), IDE (interrupt on done enable, 0100), RDY (0200) and ERR (0100000) (2.11BSD `rkreg.h`).

**The device switch.** The kernel does not know drivers by name. Two arrays, `bdevsw` for block devices and `cdevsw` for character devices, hold one row of function pointers per driver, and the **major device number** (one half of a device number; the other half, the **minor number**, selects a unit within the driver) picks the row. In 2.11BSD's `pdp/conf.c` the RK05 is row 6 of `bdevsw` and row 15 of `cdevsw`:

```
/* bdevsw: open, close, strategy, root, size, flags */
/* rk = 6 */    rkopen,  rkclose,  rkstrategy,  nulldev,  rksize,  0,
/* cdevsw: open, close, read, write, ioctl, stop, ttys, select, strategy */
/* rk = 15 */   rkopen, rkclose, rawrw, rawrw, nodev, nulldev, 0, seltrue, rkstrategy,
```

A disk has both rows. Through the block row the kernel reads and writes 512-byte blocks by way of its **buffer cache** (disk blocks kept in memory), and through the character row `rawrw` reaches the same `rkstrategy` for the **raw** interface, which moves data between the disk and a user's memory without the cache.

**The strategy, start and interrupt routines.** A block driver is three routines around a queue. This is V7's `rk.c` with the disk-statistics lines left out:

```
rkstrategy(bp)  register struct buf *bp;
{
        /* give the device an address it can use */
        if(bp->b_flags&B_PHYS)  mapalloc(bp);
        if (bp->b_blkno >= NRKBLK) { bp->b_flags |= B_ERROR; iodone(bp); return; }
        bp->av_forw = (struct buf *)NULL;
        /* keep rkintr out while the queue changes */
        spl5();
        if(rktab.b_actf == NULL) rktab.b_actf = bp; else rktab.b_actl->av_forw = bp;
        rktab.b_actl = bp;
        /* the device is idle: start it */
        if(rktab.b_active == NULL) rkstart();
        spl0();
}
rkstart()
{
        if ((bp = rktab.b_actf) == NULL) return;
        rktab.b_active++;
        bn = bp->b_blkno;  dn = minor(bp->b_dev);  cn = bn/12;  sn = bn%12;
        RKADDR->rkda = (dn<<13) | (cn<<4) | sn;          /* disk address     */
        RKADDR->rkba = bp->b_un.b_addr;                  /* memory address   */
        RKADDR->rkwc = -(bp->b_bcount>>1);               /* words, negative  */
        com = ((bp->b_xmem&3) << 4) | IENABLE | GO;
        if(bp->b_flags & B_READ) com |= RCOM; else com |= WCOM;
        /* go; the interrupt comes when done */
        RKADDR->rkcs = com;
}
rkintr()
{
        if (rktab.b_active == NULL) return;
        bp = rktab.b_actf;  rktab.b_active = NULL;
        if (RKADDR->rkcs < 0) {                          /* the error bit */
                deverror(bp, RKADDR->rker, RKADDR->rkds);
                RKADDR->rkcs = RESET|GO;
                while((RKADDR->rkcs&CTLRDY) == 0) ;
                /* retry up to ten times */
                if (++rktab.b_errcnt <= 10) { rkstart(); return; }
                bp->b_flags |= B_ERROR;
        }
        rktab.b_errcnt = 0;
        rktab.b_actf = bp->av_forw;
        /* wake whoever sleeps on this buffer */
        iodone(bp);
        rkstart();                                       /* the next request */
}
```

The **buffer header** `struct buf` is the kernel's record of one transfer: the device (`b_dev`), the block number (`b_blkno`), the byte count (`b_bcount`), the memory address (`b_un.b_addr`, with `b_xmem` for the bits above 16), flags such as `B_READ` and `B_ERROR`, and a link (`av_forw`) that chains it onto the driver's queue (`buf.h`). The strategy routine is called by the file system or by `physio`, one buffer at a time. Because the controller does one transfer at a time, the queue of waiting requests lives in the kernel's memory, and `rkstart` feeds the hardware from it. `spl5()` is one of the **spl** functions (set priority level: they write the priority bits of the processor status word); while the priority is 5, an interrupt of level 5 or lower waits, so `rkintr` cannot run between the test of the queue and its change. 2.11BSD writes `s = splbio(); ... splx(s)`, which restores the previous level and does not drop to 0, and `splx` writes the low byte of the status word at `0177776` (`rk.c`, `machparam.h`, read).

**The interrupt vector.** A device that finishes asserts a **bus request** line, BR4 to BR7 (the RK11 uses BR5); when the processor's priority is lower, it takes the interrupt, and the device supplies a **vector**, a number set by switches on the card that is the address of two words in low memory: a new program counter and a new status word. The processor pushes the old pair on the stack and loads the new one (6.3 gives the pdp64's version). 2.11BSD's `pdp/scb.s` fills the pair for the RK11

with `DEVTRAP(220, rkintr, br5)`: at address 0220, the address of `rkintr` and priority 5. The routine ends with `RTI`, return from interrupt.

**INIT and READY.** A device is put in a known state by a command and a wait. `rkintr` above writes `RESET|GO` to the CSR and spins until `CTLRDY` (the RDY bit) comes back; `rkprobe` writes `DRESET|GO`; the RX02 floppy controller has a bit named `RX_INIT`, "initialize" (`rxreg.h`, read). All the work happens behind the register, in the device's own logic: the driver writes one bit and waits for another. 5.5 shows what it becomes.

**Character devices and the clist.** A terminal is read and written a byte at a time, and the kernel shares the work with the **line discipline** (the kernel code that edits the input line and echoes it, so that no driver does). 2.11BSD's console driver for the KL11 serial line (`pdp/cons.c`, 196 lines) is five small routines: the receive interrupt `cnrint` reads the data buffer register and passes the byte up with `(*linesw[tp->t_line].l_rint)(c, tp)`; `cnstart` takes the next byte from the terminal's output queue with `getc(&tp->t_outq)`, writes it to the transmit buffer when the transmitter is ready (`DLXCSR_TRDY`) and marks the line busy; the transmit interrupt `cnxint` clears the busy mark and calls the line discipline's `l_start`, which calls `cnstart` again. The queues are **clists** (character lists: chains of small blocks of `CBSIZE` bytes taken from one free list; `getc` returns -1 when the list is empty). The console is at `0177560` and interrupts at vectors 060 and 064, both BR4; the line clock KW11-L is at vector 0100, BR6, and runs `hardclock` (`scb.s`).

**Being found.** In V7 nothing finds a device: `conf.c` and the vector table `l.s` are written by hand. 2.11BSD added `autoconfig(8)`, a user program that "needs the ucall() system call and special kernel to work" (`autoconfig/main.c`): it reads `/etc/dtab`, calls the driver's **probe** routine for each line (`rkprobe` writes the CSR to make the controller raise an interrupt and returns `ACP_IFINTR`, "exists if the interrupt arrives"), patches the interrupt vector, and calls the driver's **attach** routine (`rkattach(addr, unit)` stores the controller's address in `RKADDR`).

**Moving data: `physio` and map registers.** A raw read or write goes through `physio`, which builds a buffer header that points into the user's memory: it turns the user's virtual address into a physical one from the user's segment registers, locks the process in memory (`SLOCK`), calls the strategy routine and sleeps until `B_DONE` (`vm_swp.c`, read). It translates only the first address and passes the whole length, so it assumes that the buffer is one contiguous run of physical memory. The Unibus carries 18 address bits, so a device cannot name all of a larger memory; the **map registers** (table entries that translate a device's address to a memory address) at `0170200` do (`h/uba.h`), and `mapalloc(bp)` of `rkstrategy` allocates registers for a raw transfer and rewrites the buffer's address into one the device can use (`pdp/machdep.c`, read).

**The Q-bus.** The PDP-11's second bus, the Q-bus, carries the same programming model: registers in an I/O page, vectors, BR levels (that the Q-bus served the lower-cost PDP-11s such as the 11/23 is *unverified* here; no source on this computer describes it). What reaches a driver is the address width. 2.11BSD supports "22-bit q-bus systems" (`MAXCLICK_22`, "22 bit QBUS", `pdp/machdep2.c`), where memory is larger than an 18-bit controller can name. A controller built for the 18-bit Unibus, such as the RK11, then cannot reach a buffer above 256 KB, and `rkstrategy` calls `sofub_alloc(bp)`: a **soft Unibus map** that copies the data through a bounce buffer below 256 KB with `copyv`, and `sofub_relse` copies back (`pdp/machdep.c`, `rk.c`, read). `rl.c` notes that third-party 18-bit RL controllers work this way on a Q-bus and that the DEC RLV11 "does not properly monitor the I/O page signal", so it cannot (read). The drivers, the registers and the vectors were the same on both buses; the DMA address was the part that differed (5.10).

**What the PDP-11 driver did not have.** The driver is told what is there; the device is not asked. A device does one command at a time. The register set is the device's own, so a new kind of device is a new set of bits to learn. The interrupt vector and its level are set by hand. No device needed a program loaded into it before it worked (4.1). The pdp64's drivers change each of these, and 5.3 starts from this list.

## 5.2 What came between: the VAX, PCI, virtio and OpenBSD

The author's own statements about drivers name few referents. He said, of the eight-register devices, that drivers "should be trivially small" (errata 49, since withdrawn with the eight-register model by errata 95); he asked for "a standard audio/video interface so there is no bullshit with supporting a bunch of device drivers, although as a pcie bus any device MAY be attached and supported" (errata 78); and he chose virtio: "yeah, virtio is fine" (errata 79), "forget the native 8 register devices. gone. virtio across the board" (errata 95), "disk is virtio-blk" (errata 101). OpenBSD is the first operating system (errata 41). On 2026-11-09 he also wrote that "we will support all the virtio devices in silicon (the ones it makes sense to support, that is)" and asked what "syntactic sugar" (not new opcodes) could make devices easier to use "like the 12 register

+ dma thing" (R359). No other system is named in his messages on drivers (the decision log was searched). The VAX, PCI and the intermediate OpenBSD interfaces below are the assistant's steps, from the sources read.

**5.2.1** The pdp64 supports in silicon the virtio devices that it makes sense to support, and it standardises on the latest buses, USB, NVMe and PCI Express. *(D)* Syntactic sugar (notation, and not new opcodes) makes a device easier to use than the registers-plus-DMA devices of the PDP-11 style. *(D)* Details pending (J.5.6): which virtio devices count, and what the sugar is.

| step | what it contributed to drivers | basis |
|---|---|---|
| PDP-11, V6 to 2.11BSD | register structure, device switch, strategy/start/interrupt, `spl`, `INIT`/`READY`, probe and attach (2.11BSD), map registers | read (5.1) |
| PDP-11 on the Q-bus | the same drivers and registers; 22-bit memory, so an 18-bit controller needs a bounce buffer (the soft Unibus map) | read (5.1); which models *unverified* |
| VAX, 4.3BSD | `config(8)` turns a text file into the device tables (`ioconf.c`); a driver is one table of probe, slave, attach and start routines (`struct uba_driver`); the kernel scans the adapter and calls the probe where something answers; `ubalLoc` hands out map registers; `ubareset` calls every driver's reset routine | read |
| PCI | a function names itself by vendor and device number, so a driver's probe tests numbers and pokes no address; windows are placed by software | structure read in OpenBSD; history *unverified* |
| PCI Express, MSI-X | an interrupt is a memory write, so each queue can have its own vector; a bus-mastering device reads and writes memory by itself | chapter 2 (2.5, 2.6) |
| virtio | one transport and one queue structure for every class of device; a class driver is only the request format; features are agreed at start | `virtio_*.h`, QEMU, OpenBSD `vioblk.c` (read) |
| OpenBSD, the pdp64's first system | a softc and a `cfattach` of `match` and `attach`; DMA through `bus_dma` maps with explicit syncs; the interrupt priority survives as `IPL_BIO`; `bdevsw` and its strategy routine survive; a virtio disk appears as a SCSI adapter under `sd(4)` | read (5.13) |
| pdp64 | virtio across the board; found by enumeration; MSI-X; uncached buffers and no cache syncs; an IOMMU | errata 79, 85, 95, 101 |

The VAX's `config(8)` and `uba_driver` are in `etc/config/mkioconf.c` and `sys/vaxuba/ubavar.h` of the 4.3BSD tree; the kernel's scan (`unifind` in `sys/vax/autoconf.c`) calls `(*udp->ud_probe)` where `badaddr` finds a device. `ubareset` (`vaxuba/uba.c`) "generate[s] a reset on uba number uban" and then calls `(*cdp->d_reset)(uban)` for every row of the character device table, so that each driver re-issues what was pending.

## 5.3 What a driver is on the pdp64

The pdp64 changes the four parts of 5.1 in turn. The register structure is a PCI Express function whose windows hold the virtio structures instead of the device's own registers (statements 5.3.2 to 5.3.5, and 5.4). The device switch row is the OpenBSD driver's own interface (5.13). The vector and the level are a message that the device writes (5.9). The probe and attach are an identifier match and a handshake (5.5). Figure 5.1 draws the two side by side.

**5.3.1** Every standard device is a virtio device or a USB controller; the disk, network, graphics and audio devices are virtio-blk, virtio-net, virtio-gpu (2D) and virtio-snd, over the virtio PCI Express transport, so that one transport driver and a thin class driver per device serve the system. *(D)* Any other PCI Express device, an NVMe controller for example, may be attached with its own driver; it is not a standard device. *(D)*

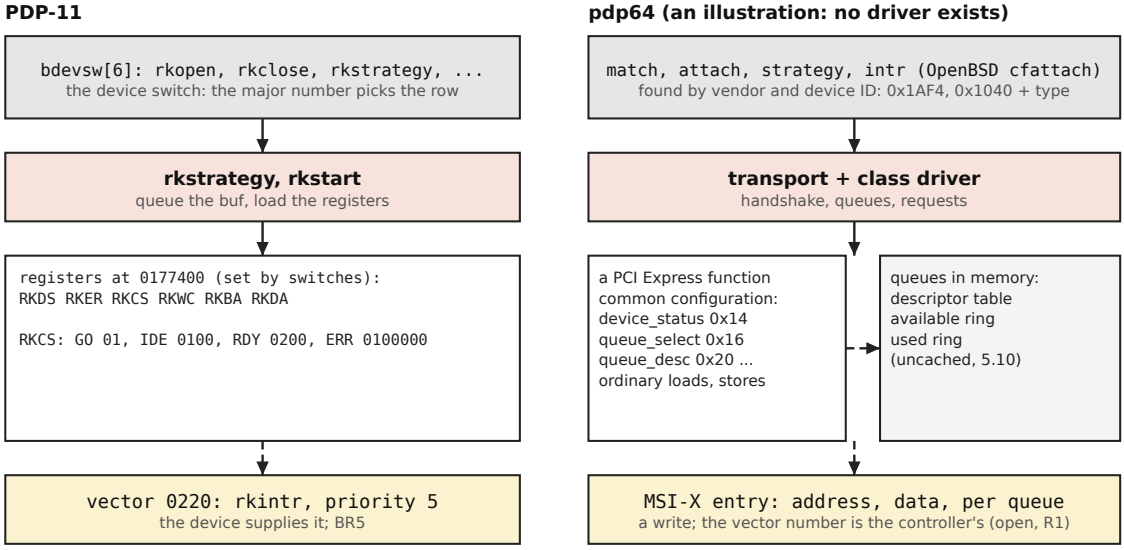
**5.3.2** A device is a PCI Express function whose windows (the regions its base-address registers describe) a driver maps with the device memory type, strongly ordered. *(D)*

**5.3.3** A driver reaches a register with an ordinary load or store; there are no input or output instructions and no port space. *(D)*

**5.3.4** Registers are naturally aligned and accessed at their own width; the machine is little-endian throughout and the virtio structures are little-endian, so a driver has no byte-order or byte-lane cases. *(D)*

**5.3.5** Accesses to a device page happen in program order; one `FENCE` separates the last write to memory the device will read from the write that notifies the device. *(D)*

**the structure of a driver: the PDP-11 (V7, 2.11BSD) beside the pdp64**




What a driver was told on the PDP-11 (the address, the vector, the level) it asks or reads on the pdp64; what it knew (the register bits) it negotiates.
Sources: V7 rk.c, 2.11BSD rkreg.h, pdp/conf.c, pdp/scb.s (read); virtio_pci.h, OpenBSD vioblk.c (read); chapter 2.

*Commentary.* **Chain.** PDP-11: a structure laid over fixed addresses (`RKADDR` at `0177400`), so that the register is a field and the driver needs no accessor. VAX: the same structure at an address the kernel found by scanning the adapter, plus map registers and data paths to manage. PCI and PCI Express: the registers are in windows that software placed, so the address is a value the kernel hands the driver; Linux and BSD wrap every access in an accessor (`readl`, `bus_space_read_8`) to hide port-or-memory differences and byte order (*unverified* for the names; OpenBSD's `virtio_read_device_config_4` and `_8` are read in `vioblk.c`). pdp64: ordinary loads and stores through a pointer to a structure, because the machine has no port space and one byte order, so the accessor is a load or a store. The assistant's reasoning from this: the register's address is a virtual address the kernel maps, so the driver is position-independent, which `RKADDR` at `0177400` was not. **Trade-offs.** Taking virtio rather than a design of the pdp64's own gives up the freedom to shape the registers and gains every driver and every emulated device that already exists. **Impact.** *Kernel:* one transport driver and a thin driver per class; no accessor layer. *Compiler:* register access is a `volatile` load or store of the register's width through a pointer to a structure (5.13). *Hardware:* the devices are standard PCI Express functions.

## 5.4 The interface a driver programs: structures, queues and features

Chapter 2 explains the structures (2.7.2 to 2.7.9) and gives pseudocode for the handshake (`init_device`), the queue set-up (`setup_queue`), posting and reaping (`post`, `reap`). This section keeps the decisions as the driver writer needs them.

**5.4.1** The driver finds a virtio device's structures through the capabilities in its configuration space: the common configuration, the notification area, the interrupt status and the device-specific configuration, each at an offset in one of the device's windows. *(D)* The common configuration has these fields, as the virtio specification lays them out (the offsets are those of `struct virtio_pci_common_cfg` in the Linux header `virtio_pci.h`, read; the header spells the driver's feature fields `guest_feature_select` and `guest_feature`, the interrupt-vector field of the device `msix_config`, and each queue address as a `_lo` and a `_hi` word):

| Offset | Field | Meaning |
|---|---|---|
| `0x00` | `device_feature_select` | which 32 bits of the device's features `device_feature` shows |
| `0x04` | `device_feature` | the device's feature bits (read-only) |
| `0x08` | `driver_feature_select` | which 32 bits of the driver's features `driver_feature` sets |
| `0x0C` | `driver_feature` | the features the driver accepts |
| `0x10` | `msix_config` | the interrupt vector for a change of configuration |
| `0x12` | `num_queues` | how many queues the device has (read-only) |
| `0x14` | `device_status` | the handshake bits; writing zero resets the device |
| `0x15` | `config_generation` | changes when the device-specific configuration changes |
| `0x16` | `queue_select` | which queue the next fields describe |

| Offset | Field | Meaning |
|---|---|---|
| 0x18 | queue_size | the number of entries of the selected queue |
| 0x1A | queue_msix_vector | the interrupt vector of the selected queue |
| 0x1C | queue_enable | the driver sets it to 1 to start the queue |
| 0x1E | queue_notify_off | where, in the notification area, the driver notifies this queue |
| 0x20 | queue_desc | physical address of the descriptor table |
| 0x28 | queue_driver | physical address of the available ring |
| 0x30 | queue_device | physical address of the used ring |

**5.4.2** A transfer is a chain of descriptors in a queue (a virtqueue) in host memory. A descriptor is 16 bytes: a 64-bit buffer address, a 32-bit length, 16 bits of flags (NEXT: the chain continues; WRITE: the device writes the buffer; INDIRECT: the buffer is a table of descriptors) and a 16-bit index of the next descriptor. *(D)*

**5.4.3** The driver puts the head of a chain in the available ring and advances its index, executes FENCE, and writes the queue's notification address; the device carries the chain out and writes the head and the byte count in the used ring, advances the used index, and may interrupt. Progress is therefore in memory, not in the device. *(D)*

**5.4.4** The driver and device agree on features before any queue runs: the driver reads the device's features, writes the subset it accepts, and the device confirms it; the version-1 feature is required. *(D)* Details pending (J.5.2): the feature subset that the pdp64 accepts for each device, and the platform-ordering feature that the pdp64 declares.

**5.4.5** A device that carries traffic in two directions (a network card) has one queue for each direction in the one device; a console-like device is the same pattern. *(D)*

*Commentary.* **Chain.** PDP-11: the command is the register values; one is in flight, and the completion is the interrupt or the RDY bit. VAX and PCI: the same, with the DEUNA Ethernet controller of 4.3BSD (four registers, `dereg.h`) as the exception, whose transmit and receive **rings** (circular arrays of entries in memory, each with an ownership bit) let the driver queue many frames; its `if_de.c` is where the ring appears in the sources read. virtio: the ring becomes the standard, with the done mark in a second ring (the used ring) and not in the descriptor, and the control registers shrink to a handshake. pdp64: the standard as it is, on a PCI Express function. **Modern comparables** (*unverified*, general knowledge): NVMe's submission and completion queues, the descriptor-done bit of Intel Ethernet controllers and the command lists of SATA controllers are all rings with a done mark, so a virtqueue is the common shape and not a novelty. **Trade-offs.** A queue per direction keeps each ring in order and simple; the used ring costs one write per chain. The pdp64 does not control the standard, and a feature or a field it dislikes stays. **Impact.** *Kernel:* one transport driver and a thin driver per class. *Compiler:* the descriptor is a 16-byte structure with natural alignment (no padding is needed: `8 + 4 + 2 + 2` bytes), and register offsets are named constants. *Hardware:* the interface is the specification's, so a standard device works.

## 5.5 The life of a driver: probe, attach, run, fail, detach

The five steps are the PDP-11's, in the order autoconfiguration calls them, with the bus doing the finding.

**5.5.1** *Probe.* The kernel enumerates the bus (section 2.4), reads each function's identifiers, and matches the virtio device type (or the USB class) against the drivers it has. *(D)*

**5.5.2** *Attach.* The driver resets the device (zero to `device_status`), sets the acknowledge and driver bits, negotiates features, sets the features-ok bit and reads it back, sets up each queue (its size and the three addresses, then `queue_enable`), and sets the driver-ok bit. *(D)*

**5.5.3** *Run.* The driver puts the queue in uncached memory, fills descriptors, publishes the chain in the available ring, executes FENCE, and notifies the queue; a chain is complete when it appears in the used ring, which the driver sees by polling or by the interrupt. *(D)*

**5.5.4** *Fail.* A device that sets its needs-reset bit, or that stops answering, is reset and its queues reprogrammed. *(D)* Details pending (J.5.4): what happens to the chains still queued.

**5.5.5** *Detach.* A departure event, or a bus fault from an absent device, tells the driver to stop. The queued work of the departed device is left in memory (2.4.5), where the driver can read, save, resend or fail it. *(D)* Details pending (J.5.4): which of these the driver does, and in what cases.

**INIT and READY, and the status handshake.** The PDP-11's start of a device (5.1) is one command and one wait: write INIT (or RESET|GO), spin until READY. The virtio `device_status` byte is that idea spread over steps: writing zero resets the device (the INIT), and the driver then sets ACKNOWLEDGE, DRIVER, FEATURES_OK and DRIVER_OK, each meaning "I have got this far", reading FEATURES_OK back

to learn whether the device accepts the features chosen (the READY answer, now with a refusal possible; 2.7.3). The assistant's reasoning: the step that is new is the negotiation, which the one-bit form could not carry because the PDP-11 device had nothing to negotiate. The handshake follows from "virtio across the board" (errata 95).

Probe and attach for a virtio-blk driver, written over the helpers of chapter 2 (`find_virtio_structures`, `init_device`, `setup_queue`); the names are the assistant's, an illustration:

```
# 5.5.1: called for every function the bus walk found
vioblk_match(fn):
    # 0x1040 + device type 2 (2.7.2)
    return fn.vendor == 0x1AF4 and fn.device == 0x1042

vioblk_attach(fn):                              # 5.5.2
    # 2.7.2: where the common configuration, notify, status, device config are
    find_virtio_structures(fn)
    # 2.7.3: reset, ACKNOWLEDGE, DRIVER, features, FEATURES_OK, queue 0, DRIVER_OK
    init_device(fn)
    # virtio_blk_config.capacity, in 512-byte sectors (virtio_blk.h)
    capacity = mmio_read64(device_config + 0)
    # the block driver's row of bdevsw, 5.13
    register_disk(fn, capacity, vioblk_strategy)
```

OpenBSD's `vioblk_match` is the same test, on the device number alone (`va->va_devid == PCI_PRODUCT_VIRTIO_BLOCK`, with the vendor tested one level down in `virtio_pci_match`), and its `vioblk_attach` asks for the features `RO`, `NOTIFY_ON_EMPTY`, `SIZE_MAX`, `SEG_MAX` and `FLUSH`, reads the capacity, allocates the queue with `virtio_alloc_vq` and registers its completion routine in `vq_done` (read, `vioblk.c`). What differs from the PDP-11's `rkattach`: the driver is not told an address, there is a negotiation, and every step can fail with a defined status.

*Commentary.* **Chain.** PDP-11: V7 has no probe; 2.11BSD's `rkprobe` pokes an address and `rkattach` stores it. VAX: `ud_probe` and `ud_attach` in a table, called by the kernel's scan. PCI: the identifier match replaces the poke, and BSD's autoconfiguration calls `match` then `attach`; OpenBSD's `cfattach` for `vioblk` is `{ sizeof(softc), vioblk_match, vioblk_attach, NULL }`, so it has **no detach routine** (read). Linux's driver model has `probe` and `remove` (*unverified*). PCI returns all ones for a read of a removed device (*unverified*), which, as the assistant reasons, is why a pdp64 driver that finds all ones cannot tell a departed device from a register that holds them, and why the pdp64 faults instead (2.4.9). pdp64: probe by identifier, attach by the status handshake. The probe reads the bus and not a configuration file, so a new device needs no line in `dtab` or `ioconf.c`. **Trade-offs.** The five steps are the same for every device, so the work that differs is the personality: what the requests mean. **Impact.** *Kernel:* the steps are the same for every driver, so only the requests differ. *Compiler:* none. *Hardware:* none beyond the device's own.

## 5.6 The block path: virtio-blk

**5.6.1** Storage is virtio-blk: a virtqueue of requests, each a header (read, write or flush, and the sector), the data buffers and a status byte, the device marking the request done in the used ring. *(D)* An NVMe controller, which keeps a submission queue and a completion queue in host memory with a doorbell register for each (the shape of the virtqueue with a different command encoding), may be attached as any PCI Express device may; it is not the standard disk. *(D)*

**5.6.2** The **disk** is virtio-blk (the author's decision of 2026-11-10, errata 101) with read, write and flush at 512-byte sectors. *(D)*

**5.6.3** The boot ROM reads block 0 of the disk by the block-0 route (4.4, errata 109) and does so through virtio-blk, with the polled routine of the table below (errata 101: "the boot ROM and the drivers of the manual are written for it"). *(D)* Details to be determined: the request that the ROM issues, as far as it differs from the polled routine below.

The request is the three-descriptor chain of chapter 2 (2.7.5, with its figure): a header the device reads (type 0 read, 1 write, 4 flush, the priority and the sector), the data buffer (device-writable for a read) and one status byte the device writes (0 OK, 1 I/O error, 2 unsupported; `virtio_blk.h`, read). Figure 5.2 follows one request through the driver, the rings and the device.

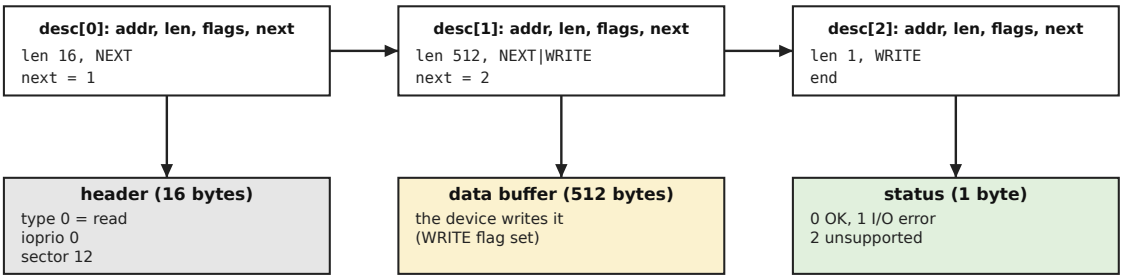
**the life of one block request (a read of one sector) on the pdp64**

**the two rings**




1 the driver fills the header and the three descriptors
2 it writes the head (0) into the available ring, executes FENCE, then adds 1 to its idx
3 it writes the queue's notification address (the doorbell; FENCE ordered)
4 the device reads the chain, moves the sector into the buffer, writes the status byte
5 it writes { id, len } in the used ring, then adds 1 to the used idx
6 an interrupt (or the driver polling used.idx) lets the driver read the status, free the chain, iodone(bp)

**the descriptor table (three descriptors chained by next) and the buffers they point to**




Sizes and flags as in chapter 2 (2.7.5 to 2.7.7) and virtio_blk.h; the numbers 12 and 513 are an example. The data is written before the used entry before the used idx

The strategy, start and interrupt routines of 5.1 become the following, a block driver written as an illustration over the `post` and `reap` of chapter 2 (2.7.7). The queue of waiting requests is still the kernel's, but the hardware now takes up to `queue_size` chains at once, so `start` fills the ring until it is full, not until the controller is busy:

```
# called one buf at a time, as rkstrategy is
vioblk_strategy(bp):
    s = splbio()                                # IPL_BIO in OpenBSD (5.9)
    append bp to queue
    vioblk_start()
    splx(s)

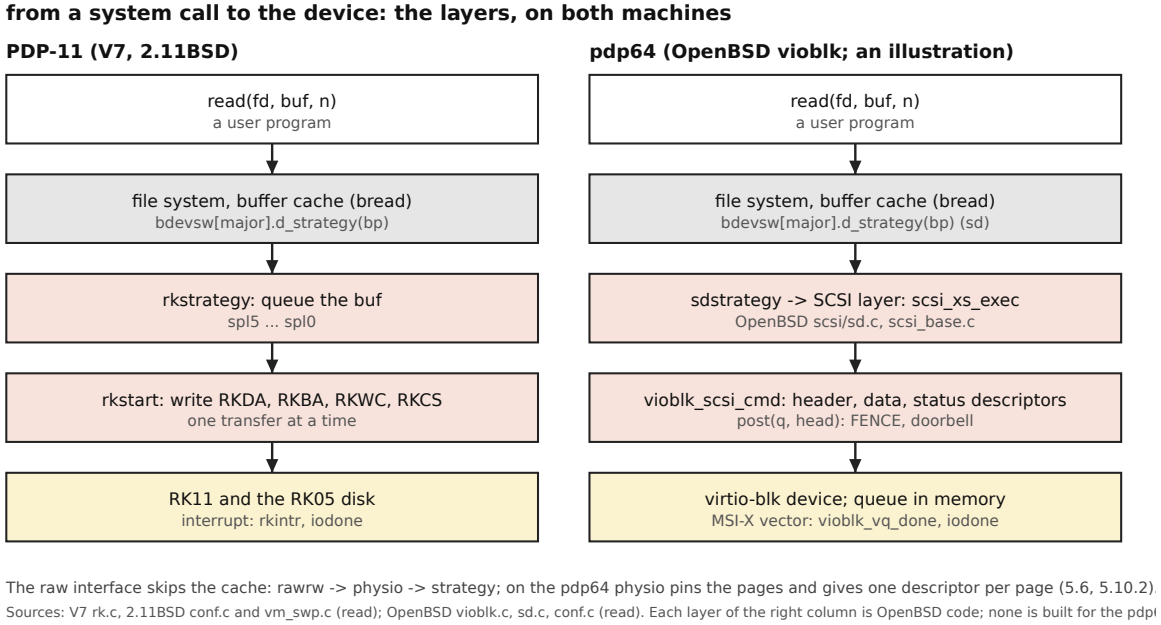
vioblk_start():
    while queue not empty and free_descriptors() >= 3:
        bp = take head of queue
        h = alloc_hdr();  h.type = (bp.flags & B_READ) ? 0 : 1
        h.ioprio = 0;  h.sector = bp.blkno
        s = alloc_status_byte()
        d0 = get_free(); d1 = get_free(); d2 = get_free()
        desc[d0] = { addr: h,         len: 16,         flags: NEXT,         next: d1 }
        wr = (bp.flags & B_READ) ? WRITE : 0       # the device writes the buffer on a read
        desc[d1] = { addr: bp.addr, len: bp.bcount, flags: NEXT | wr, next: d2 }
        desc[d2] = { addr: s,         len: 1,          flags: WRITE,        next: 0 }
        inflight[d0] = bp
        post(q, d0)                             # FENCE, then the doorbell (5.3.5)

vioblk_intr():                                  # the queue's vector, at IPL_BIO
    # reap(q), with the buf completed as rkintr completes it
    while last_used != used.idx:
        e = used.ring[last_used mod n];  bp = inflight[e.id]
        if status_byte(e.id) != 0: bp.flags |= B_ERROR
        free_chain(e.id);  last_used += 1
        iodone(bp)                              # wake the sleeper, as in 5.1
    vioblk_start()                              # slots are free again: send more
```

The routine that polls, which the boot ROM and a crash dump use because they run with no interrupt set up, differs only in who waits (chapter 2, 2.7.8, gives the polled loops side by side):

| | RK11, interrupt style (5.1) | virtio-blk, interrupt style (above) | virtio-blk, polled (the boot ROM) |
|---|---|---|---|
| strategy | queue the buf; start if idle | queue the buf; fill the ring | `blk_read` (2.7.7): set `NO_INTERRUPT` in `avail.flags` |
| start | four register writes, GO | `post(q, d0)`: one doorbell write | `post(q, d0)` |
| completion | `rkintr`: ERR? retry up to 10 times; `iodone`; start the next | `vioblk_intr`: status byte, `B_ERROR`, `iodone`, start more | spin while `last_used == used.idx`; read the status byte of `d2` |
| in flight | one transfer | up to `queue_size` transfers | up to `queue_size` transfers |

The raw interface changes with paging. `physio` of 5.1 translated one address and passed the whole length, because a PDP-11 process's memory was contiguous; with a page-table memory (chapter 6) a user's buffer is a list of pages scattered in physical memory, so the driver describes it with one descriptor per page (statement 5.10.2) and the pages must stay in place until the status byte is written. Figure 5.3 draws the layers a request crosses on the two machines.




The raw interface skips the cache: rawrw -> physio -> strategy; on the pdp64 physio pins the pages and gives one descriptor per page (5.6, 5.10.2).
Sources: V7 rk.c, 2.11BSD conf.c and vm_swp.c (read); OpenBSD vioblk.c, sd.c, conf.c (read). Each layer of the right column is OpenBSD code; none is built for the pdp

*Commentary.* **Chain.** PDP-11: one command at a time, a software queue per drive, one interrupt per transfer. VAX: the Unibus RK611 driver of 4.3BSD (`vaxuba/rk.c`) is a `uba_driver` whose start routine `rkdgo` loads the registers after `ubalioc` has found map registers. PCI: the same registers, now with a controller that queues commands itself (SCSI, ATA, NVMe: the shape of the virtqueue, errata 61 and 75). virtio-blk: three descriptors and a status byte. OpenBSD: `vioblk.c` (747 lines, counted) is not a block driver with a strategy routine of its own but a SCSI adapter (`vioblk_scsi_cmd`) under the `sd(4)` disk driver, whose `sdstrategy` (`scsi/sd.c`) keeps the `bdevsw` row (read). pdp64: the same driver, with `bus_dmamap_sync` calls that do nothing (5.10). **Trade-offs.** A request with many commands outstanding replaces the one-at-a-time loop and its retry-ten-times error handling by a status byte and a reset (5.11). The cost is memory for the queue and the ordering rule between data and the used index (5.10.3). **Impact.** *Kernel:* a disk driver is a skeleton shared by all devices; OpenBSD's `vioblk.c` is the model. *Compiler:* none. *Hardware:* the device writes the data before the used entry before the used index.

## 5.7 The console, the timer and the entropy source

**5.7.1** A core in secure mode takes no periodic timer tick, and the main cores are tickless (errata 86; the author's words, R323). *(D)* Details pending (J.5.1): the form of the timer that serves them.

**5.7.2** The entropy source is one pool shared by all cores, with no per-process pool, no version register and no updatable firmware; it assists the operating system's own generator, which absorbs into the pool and squeezes from it, and a read with a known and wanted effect is not a strange side effect (errata 85, random numbers). *(D)* Details pending (J.5.1): whether a physical source stands behind the pool, and how the pool is reached.

Open: the form of the console (J.5.1).

## 5.8 The network, graphics, sound and USB drivers

Each class driver is a few request formats over the transport of 5.4. What the driver does is the part that differs.

**5.8.1** The **network card** is virtio-net: a receive queue the host fills with buffers for the device and a transmit queue of frames to send; the device completes each receive buffer with the frame length. *(D)*

**5.8.2** The **graphics device** is virtio-gpu in its 2D mode (create a surface, point it at memory, flush a rectangle) and the **audio device** is virtio-snd (streams in queues in shared memory). *(D)*

**virtio-net** (device type 1). The driver keeps the receive queue full: it posts empty buffers with `WRITE`, and the device fills one with a frame and reports its length in the used ring; to send, the driver posts the frame and the device reads it. Each frame starts with a 10-byte `struct virtio_net_hdr` (flags, GSO type, header length, GSO size, checksum start and offset; `virtio_net.h`, read), which tells the device which checksum to fill in and where. Queue 0 receives, queue 1 transmits, and a control queue exists with the feature `CTRL_VQ`; the MAC address is in the device configuration with the feature `MAC` (`virtio_net.h`, QEMU `virtio-net.c`, read). OpenBSD's `if_vio.c` is 2,184 lines (counted), the largest virtio driver of the tree. The PDP-11's counterpart is the DEUNA of 4.3BSD or the network driver of a Unix with a network (V7 has none).

**virtio-gpu** (type 16). Two queues: the control queue, 256 entries in QEMU, carries the commands of chapter 3 (3.12: create a resource, attach its backing memory, set the scanout, transfer a rectangle, flush it), and the cursor queue, 16 entries, carries cursor updates. The driver sends a command and waits for the response in the same chain. OpenBSD's `viogpu.c` (727 lines, read) has the functions `viogpu_create_2d`, `viogpu_attach_backing`, `viogpu_set_scanout`, `viogpu_transfer_to_host_2d` and `viogpu_flush_resource`, the five commands of the figure of 3.12. The PDP-11 had no counterpart; the nearest is the display register pair of a VT11 or the bitmap of a Blit, which chapter 3 tells.

**virtio-snd** (type 25). Four queues in the order control, event, transmit, receive (`virtio_snd.h`, read; QEMU adds them in that order); the control queue sets up the streams, and the transmit and receive queues carry the samples of an output or an input stream. Chapter 3 (3.17) says what a stream is and what a mixer does with it. **OpenBSD's tree on this computer has no virtio-snd driver:** `sys/dev/pv` holds `if_vio.c`, `vioblk.c`, `viocon.c`, `viogpu.c`, `viomb.c`, `viornd.c`, `vioscsi.c` and `virtio.c`, and no sound file, so the question of errata 79 ("whether OpenBSD has a virtio-snd driver (if not, one is written)") is answered for that tree: one is written. The PDP-11 had no audio device.

**5.8.3** USB is a standard device: a **USB host controller** is a PCI Express function that attaches like any other (5.3.1), is driven by the operating system's USB stack and enumerates the devices attached to it (2.4.3). *(D)* The author's uses of USB are the keyboard, the mouse and a hard drive ("modern usb keyboard, mouse, and nvme hard drive", R20). Details pending (J.5.7): the controller interface and the class drivers.

*Commentary.* **Chain.** PDP-11: one driver per controller, each with its own registers (`rk.c`, `cons.c`, `dz.c`, 579 lines for the multiplexer). VAX: the same, with the adapter handling the memory mapping. PCI: one driver per chip family, with firmware and quirk tables (`kernel-pain-points.md`, repo). virtio: one transport and a thin driver per class, so that a card of any vendor that speaks the standard works with the drivers already written. pdp64: that, for network, graphics and audio; USB controllers as PCI Express functions. **Trade-offs.** One transport means one driver to write, at the cost of feature negotiation and a queue format that the pdp64 does not control. **Impact.** *Kernel:* one transport driver and a thin driver per class; a virtio-snd driver to write. *Compiler:* none. *Hardware:* standard PCI Express functions; the same virtio devices are the aim in QEMU, `sim64` and the FPGA.

## 5.9 Interrupts as a driver sees them

**5.9.1** A device interrupts by a message-signalled write (MSI-X): the configuration change and each queue have a vector that the driver sets in the common configuration, and the interrupt status structure says why. *(D)* Details pending (J.6.3).

**5.9.2** A driver may mask a device's interrupts and poll the used ring instead. *(D)*

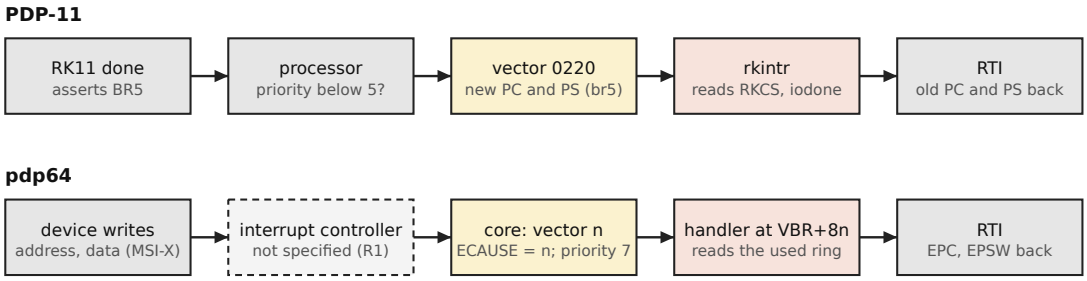
**5.9.3** A changeable interrupt target is changed with its vector masked and then read back, or is a single atomic register write. *(D)*

**5.9.4** The write that raises an interrupt cannot overtake the data writes that precede it (section 2.5.4). *(D)*

What changes for the driver is where the vector number and the level come from. On the PDP-11 the programmer chose them by switches and by the vector table: the RK11's pair at 0220, level 5. On the pdp64 the driver asks the interrupt controller for an address and a data word (`interrupt_controller_allocate` in 2.6), writes them into the device's MSI-X entry for the queue, tells the queue its entry (`queue_msix_vector`), and installs its handler in the vector table at `VBR + 8 × vector` (6.3.3). One

queue, one vector, one core: the handler of `vioblk_intr` runs when the queue's vector arrives, and reads the used ring to learn what finished, where `rkintr` read the CSR. Figure 5.4 draws the two paths.

**the interrupt path from the device to the handler**




PDP-11: a wire and a level shared by every device on it; the vector was set by switches on the card; the handler asks the CSR what happened.
pdp64: a memory write that cannot pass the data written before it (5.9.4); one vector for each queue; the handler asks the used ring.
The dashed box is open: which unit receives the write, how a core is chosen and how the vector number is made are decision R1 (appendix B, 2.6.3)

Sources: 2.11BSD pdp/scb.s (DEVTRAP(220, rkintr, br5)); chapter 6 (6.3.1 to 6.3.6) for the entry; chapter 2 (2.6) for MSI-X.
The priority of 7 on entry is the PDP-11's own habit (errata 40); the handler lowers it with SPL.

An entry puts the processor at priority 7 with the interrupted program counter and status word in registers (6.3.1 and 6.3.2); the handler saves them with `PUSHX`, lowers the priority with `SPL` to its own level, does its work, restores the pair with `POPX` and returns with `RTI` (6.3.4). The level of a device is the level the interrupt controller gives its vector; the kernel's `splbio` is the status-word priority at which the block code runs. OpenBSD keeps the idea: `vioblk_attach` sets `vsc->sc_ipl = IPL_BIO` (read), and `virtio_pci.c` can give a device one MSI-X vector for configuration changes and one for each queue (`IRQ_MSIX_PER_VQ`), or one shared by all queues (`IRQ_MSIX_SHARED`), up to `MAX_MSIX_VECS`, 16 (read).

Details pending (J.6.3): the unit that receives the write, how a core is chosen, how the vector number is made and the level assigned.

*Commentary.* **Chain.** PDP-11: four bus-request lines and switch-set vectors; a level shared by several devices, so a handler may have to ask which spoke; `spl` raises the priority to keep a handler out (5.1). VAX: 31 priority levels and a **system control block**, a table of vectors in memory (the repository's `decisions-pending.md`, checked there against a VAX architecture manual; not on this computer); `spl` becomes `splbio`, `spltty`, `splclock`. PCI: four shared pins per slot, and a handler that asks each device whether it was the one. PCI Express: MSI-X, an interrupt as a write (2.6). virtio: a vector per queue. pdp64: MSI-X for the standard devices, vectors in the same table as exceptions, priority 7 on entry (errata 40, "a throwback to pdp-11 design"). **Trade-offs.** A message carries its own vector, so no switch and no assignment table are needed and there can be thousands; the controller is an address that many devices write, and a confused device that writes the wrong address causes an interrupt, which the IOMMU can refuse (2.11). **Impact.** *Kernel:* an interrupt routine reads the used ring and the status structure; a controller driver. *Compiler:* none. *Hardware:* an MSI-X target in the interrupt controller, and the eight-level priority encoder.

## 5.10 DMA and the buffer's memory

**5.10.1** Buffers and queues are in memory mapped uncached, and the driver does no cache clean or invalidate. *(D)*

**5.10.2** A buffer that crosses a page is described by several descriptors chained with the `NEXT` flag; the driver does not make buffers contiguous. *(D)*

**5.10.3** The device writes the data, then the used-ring entry, then advances the used index; the driver reads the used index and then the data. *(D)*

Three questions arise for a driver: which addresses the device sees, whether the processor's cache agrees with memory, and in what order the writes land. The PDP-11 answered the first with map registers, which the driver loaded (`mapalloc`, 5.1), and on a Q-bus machine with a bounce buffer for controllers that could not reach all of memory (`sofub_alloc`, 5.1); the second hardly arose (that the 11/70's cache watched Unibus writes, so that DMA data appeared coherent, is *unverified*, as in 2.5); and the third by one bus that delivered writes in order. The pdp64 answers them in this way:

- **Addresses.** The IOMMU of chapter 2 (2.11) gives each device its own address space: the descriptor holds a **device address** (an address in the device's own mapped space, which the IOMMU turns into a physical address or refuses); the driver asks the kernel to map a buffer for the device and puts the returned address in the descriptor, and `VIRTIO_F_ACCESS_PLATFORM` (feature 33) says so to the device. The five recommendations of errata 85 (translation on by default, a domain per device, no kernel pointers in descriptors, permanent ring mappings, small buffers bounced) shape what that mapping call does. Details pending (J.5.2): which feature bits the pdp64 declares.
- **Coherence.** Buffers and rings are mapped uncached (memory type 6.7.4), so that the processor's cache never holds a stale copy of what a device wrote and the driver never cleans a line. OpenBSD's `vioblk.c` calls `bus_dmamap_sync` with `PREWRITE`, `PREREAD`, `POSTREAD` and `POSTWRITE` around each request (read); on the pdp64 those calls have nothing to do (5.13.1). The cost is that the processor reads received data from uncached memory, which is slow.
- **Order.** The processor's `FENCE` before the doorbell, the device's data-before-used-entry-before-used-index, and the interrupt that cannot pass the data (5.9.4) are the whole contract; chapter 2 (2.5, 2.8) states them.

*Commentary.* **Chain.** PDP-11: 18 address bits, so map registers (`UBMAP` at `0170200`) translate; `physio` assumed a contiguous buffer. VAX: `ubalonce` for map registers and **buffered data paths** that a driver purged before reading what a device had written (`UBA_PURGE780`, `ubareg.h`, read). PCI and PCI Express: a bus master uses physical addresses, and Linux and BSD wrap the work in a DMA-mapping layer (`bus_dma` in OpenBSD, read) that places cache maintenance on machines that need it and copies into a **bounce buffer** (a buffer in memory the device can reach; 2.11BSD's soft Unibus map is the same idea on the PDP-11) when a device cannot reach the data (`swiotlb` in Linux, *unverified*). That x86 DMA is coherent while ARM and MIPS DMA often is not is *unverified*. pdp64: uncached memory for coherence, descriptors per page for scattered buffers, an IOMMU for the addresses. **Trade-offs.** Uncached buffers make the processor's reads slow and make the driver unable to forget a cache operation; a device that is not coherent would call for cache-maintenance instructions. **Impact.** *Kernel:* no cache clean or invalidate in any driver; a mapping call that the IOMMU of 2.11 serves. *Compiler:* none. *Hardware:* DMA goes around the processor's data cache, so the cache needs no snoop port for devices.

## 5.11 Errors, departure and recovery

**5.11.1** A request that fails completes with the status of its device class (for a virtio-blk request, a status byte in the last descriptor of its chain); a device that is broken sets its needs-reset bit in `device_status`. *(D)*

**5.11.2** A device that leaves the bus leaves its queued work in memory, "so it can be read, saved, resent" (R262; 2.4.5), and the driver finds it there. *(D)* Details pending (J.5.4): what the driver does with the chains still queued when a device is reset or leaves the bus.

On the PDP-11 an error was the ERR bit of the CSR, the error word `RKER`, and the driver's choice: `rkintr` reset the controller and retried up to ten times before it set `B_ERROR`. A virtio-blk request reports a status of 0, 1 (I/O error) or 2 (unsupported) in its last byte, and no retry happens in the device. A driver that finds the device stuck resets it. OpenBSD's `vioblk_reset` (read) stops the device with `virtio_reset`, runs `virtio_check_vq` to complete what finished, and fails every other request with a driver error.

*Commentary.* **Chain.** PDP-11: errors in device registers, a retry loop in the driver, no device that can leave. VAX: `ubareset` resets the adapter and calls every driver's reset routine so that each re-issues its own pending work (read). PCI Express: an error completion and a link-down; hot-plug exists (2.4). virtio: a status byte per request and a needs-reset bit. pdp64: the precise bus fault for a departed device (2.4.9, 2.9.1) and not all ones (an empty configuration position is the one read that returns all ones, 2.4.7). **Trade-offs.** The cost is a write to the used ring per chain and the ordering rule between data and the used index. **Impact.** *Kernel:* a recovery routine for a failed, reset or departed device. *Compiler:* none. *Hardware:* the device writes data before the used entry before the used index.

## 5.12 Firmware for a device

A device that contains a small processor may need a program, **firmware**, loaded into it before it works. A PDP-11 peripheral needed none (4.1). On a PC the driver reads the blob from a file at attach time: OpenBSD's `loadfirmware(name, &buf, &len)` reads `/etc/firmware/<name>` (`dev/firmload.c`, read), and 19 files under `sys/dev/pci` call it (counted); Linux's `request_firmware` is *unverified*. The `INIT` and `READY` of the PDP-11 take the form of the `device_status` handshake under virtio (5.5), and that handshake has no firmware half: **no virtio header on this computer mentions firmware** (a search of all 29 `virtio_*.h` files and of QEMU's

hw/virtio), and each device class defines its own request set, so a firmware download would be a request of a class that none defines. A device the boot depends on (the disk, and the console if it is a device) needs none.

Open: how a virtio device is handed firmware (J.5.3).

*Commentary.* **Chain.** PDP-11: none. PC: a file per device, a loader in the kernel (loadfirmware(9) in OpenBSD, firmload(9) in NetBSD) and a separate package of files. pdp64: errata 85 sets one precedent, an entropy pool with "no updatable firmware". **Impact.** *Kernel:* a loader that runs after the root file system is up, and a rule that no boot device needs it. *Compiler:* none. *Hardware:* a device-side image check.

## 5.13 Where drivers run, and how they are written

**5.13.1** In OpenBSD the register accessors become loads and stores, the DMA mapping interface returns the address the device uses, with no cache-synchronisation work to do, and the autoconfiguration framework attaches a driver to each function the bus reports.

**In OpenBSD** (read in the source tree at <work>/facetlang/pdp11/openbsd-src, version 8.0). A driver is a struct cfattach (the size of its per-device structure, the **softc**, a match and an attach function, and an optional detach) and a struct cfdriver. The virtio drivers attach below the transport virtio.c (1,075 lines) and virtio_pci.c (1,246 lines), which do the handshake of 5.5, the queues, and the MSI-X set-up; a class driver calls virtio_negotiate_features, virtio_alloc_vq and virtio_enqueue and sets vq_done. The device switch of 5.1 is still there, in the machine's conf.c: bdevsw[] rows are built by bdev_disk_init from open, close, strategy, ioctl, dump, size and flags. A virtio-over-memory-mapped-registers transport, virtio_mmio.c, exists for machines with a device tree; the pdp64's transport is the PCI Express one.

**In C.** A V7 driver is written in the PDP-11's C, where int is 16 bits and the register structure's fields are int and short. On the pdp64 int is 32 bits and long and pointers 64 (LP64, appendix D, D.1), so a register or a descriptor field is declared by its width (uint16_t, uint32_t, uint64_t); natural alignment gives the 16-byte descriptor, the 10-byte virtio_net_hdr and the offsets of the virtio structures with no packed attribute. The headers that define those structures are the Linux virtio_*.h files, each of which says that it "is BSD licensed so anyone can use the definitions to implement compatible drivers" (read in virtio_rng.h; the same line is in all 29), which fits the project's BSD 2-clause licence (errata 48). A register is read and written through a volatile pointer. Details pending (J.5.5): how C code executes the FENCE of 5.3.5.

*Commentary.* **Chain.** PDP-11: a driver is one C file and a line in conf.c, in the language of the day with 16-bit int. VAX: the same, with config(8) writing the tables. OpenBSD: softc, cfattach, bus_dma, a framework of its own. pdp64: OpenBSD's framework, with the DMA calls reduced to the identity and the accessors to loads and stores. **Impact.** *Kernel:* the accessors reduce to loads and stores, the DMA interface to the addresses the device uses, and attach to the functions the bus reports. *Compiler:* fixed-width types, volatile, and a way to execute FENCE from C. *Hardware:* none beyond the device's own.

## 5.14 What was checked, and what was not

The virtio and PCI Express specifications are not on this computer. What the chapter rests on:

- **2.11BSD** (unixtree/2.11BSD/sys): pdpuba/rk.c, rkreg.h, ubavar.h, pdp/conf.c (the bdevsw and cdevsw rows), h/conf.h, h/buf.h, h/uba.h, pdp/scb.s (the vectors and levels of the RK11, the console and the clock), pdp/cons.c, sys/tty_subr.c, h/clist.h, sys/vm_swp.c (physio, rawrw), pdp/machdep.c (mapalloc), sys/ufs_bio.c (biodone), autoconfig/ (main.c, do_config.c, attach.c, rkauto.c), pdp/machparam.h (splx). **V7** usr/sys/dev/rk.c.
- **4.3BSD** (unixtree/4.3BSD/usr/src): sys/vaxuba/ubavar.h, uba.c (ubareset), rk.c, sys/vax/autoconf.c (unifind), etc/config/mkioconf.c, sys/vaxif/if_dereg.h.
- **Linux headers** in /usr/include/linux: virtio_pci.h (the common-configuration offsets), virtio_config.h, virtio_ring.h, virtio_ids.h, virtio_blk.h, virtio_net.h, virtio_console.h, virtio_snd.h, virtio_gpu.h, virtio_rng.h, virtio_rtc.h, and a search of all virtio_*.h for "firmware" and "BSD licensed".
- **QEMU** (<work>/qemu-src/hw): the queues created by virtio-rng.c, virtio-serial-bus.c (including the emergency write in set_config), virtio-blk.c, virtio-net.c, virtio-gpu-base.c, virtio-snd.c.
- **OpenBSD 8.0** (facetlang/pdp11/openbsd-src/sys): dev/pv/vioblk.c, viornd.c, viocon.c, viogpu.c, if_vio.c, virtio.c, dev/pci/virtio_pci.c, dev/firmload.c, scsi/sd.c, arch/amd64/amd64/conf.c; line counts from wc. The older OpenBSD tree at kernel-src/openbsd has no sys/dev/pv.
- **The repository:** docs/errata.md, docs/smp-decisions.md, docs/decisions-pending.md and docs/decision-register.md (the author's messages).

- **Unverified** (general knowledge): the PDP-11 caches and the contiguity of a process's memory beyond what `physio` assumes; the PCI and virtio history; that `request_firmware` is Linux's; Linux's `probe` and `remove`, `swiotlb`, `VFIO`, `UIO` and DPDK; that x86 DMA is coherent and some ARM and MIPS DMA is not.

# Chapter 6 — The kernel: modes, exceptions and memory management

**An operating system on the pdp64 sees a status word with a mode and a priority, an exception mechanism that writes only registers, and a page table of three, four or five levels whose accessed and dirty bits the kernel maintains in software.** The reader is taken to be a first-year computer-science student or a PDP-11 programmer who has read the Unix Version 6 or 7 kernel, and to know nothing that appeared after the 1970s. The chapter starts from that kernel (the status word, the vectors, `trap()`, the KT11 memory-management unit with its eight page registers per mode, `mfpi` and `mtpi`, swapping). Each later idea follows with the PDP-11 way first, the intermediate machines that changed it, and the pdp64 way with the reason. Pseudocode walks a system call, a page fault, a timer interrupt and a context switch. The design is System v1; the comparisons with other architectures are in `pdp64-system-architecture.md` (section 15) and `pdp64-system-design.md`. A fact about an older or another machine that could not be checked on this computer is marked *unverified* and listed in section 6.14.

*How to read this chapter.* A term is defined where it first appears, as the technical word followed by a plain restatement. In the pictures a box is a piece of memory or a register and an arrow is an address stored in it. Code blocks are pseudocode (C-like, `#` starts a comment) unless the text says they are PDP-11 or pdp64 assembly. The PDP-11 code is quoted from the V7 and 2.11BSD kernels on this machine (section 6.14). The pdp64 code is **an illustration written for this chapter**, not part of the architecture (the first operating system is OpenBSD, 6.12.2). The decisions are the numbered statements.

## 6.1 What you already know: the kernel of a PDP-11

Take Unix Version 7 (or 2.11BSD, the same kernel grown up) on a PDP-11/45, 70 or 73. Six things about it carry the rest of the chapter.

**The status word and the modes.** The **processor status word** (PS, also called PSW: one 16-bit register holding the flags, the interrupt priority and the mode of the running program) sits at address 0177776. Bits 15-14 are the **current mode** (the privilege the program runs with: kernel, supervisor or user), bits 13-12 the **previous mode** (the mode running before the last trap), bits 7-5 the **priority** (the level at or below which interrupts are held back), bit 4 the **trace** bit (take a trap after every instruction) and bits 3-0 the flags N, Z, V and C. The 2.11BSD header `psl.h` says the same in C: `PSL_CURMOD` 0xc000, `PSL_PRVMOD` 0x3000, `PSL_IPL` 0xe0, `PSL_T` 0x10, the condition codes in 0xf. Each mode has its **own stack pointer** (R6 names a different register in each mode), so a trap arriving in user mode finds a kernel stack waiting. User mode may not execute `halt`, `reset`, `wait` or change the priority (*unverified*).

**Vectors in low memory.** A **trap** (the machine transferring control to the kernel because something happened) goes through a **vector**: two words at a fixed low address, the new program counter and the new PS. The hardware pushes the old PS and PC on the new mode's stack and loads the pair. This is the start of the V7 table, `conf/l.s` (the octal addresses are added; the file's first two words are a branch and a 4, the vectors follow):

```
        trap;  br7+0.           / 004  bus error
        trap;  br7+1.           / 010  illegal instruction
        trap;  br7+2.           / 014  bpt-trace trap
        trap;  br7+3.           / 020  iot trap
        trap;  br7+4.           / 024  power fail
        trap;  br7+5.           / 030  emulator trap
        start; br7+6.           / 034  system call (overlaid by 'trap')
        . = ZERO+60
        klin;  br4              / 060  console keyboard
        klou;  br4              / 064  console printer
        . = ZERO+100
        kwlp;  br6              / 100  line clock
```

The first word of each pair is the handler's address. The second is the new PS: `br7` is priority 7 (nothing may interrupt) plus a small number that the handler passes to `trap()` to say which trap it was. The device vectors carry the device's priority, 4 for the console and 6 for the clock. A handler ends with `rti` (return from interrupt: pop the PC and PS) or `rtt` (the same, but a pending trace trap is not taken at the first instruction).

**`trap()`.** Assembly code at the vector (`call` in the V7 file `mch_i.s`) saves the registers and calls one C function, `trap(dev, sp, r1, nps, r0, pc, ps)`; its arguments are, in the words of the V7 comment, "the words saved on the system stack by the hardware and software during the trap processing". A system call is one of these traps: the program executes `sys n` and `trap()` fetches that instruction back from user memory and uses its low six bits as an index (`callp = &sysent[fuiword((caddr_t)(a-1))&077]`).

**The memory-management unit.** A PDP-11 program uses 16-bit addresses (64 KB) while physical memory is wider: 18 bits (256 KB) on the Unibus machines, 22 bits on later ones (*unverified*, section 6.14). The **KT11** (the memory-management unit: the hardware between processor and memory that turns the program's addresses into memory addresses, a step called **translation**) has for each mode **eight pairs** of **page registers**: a **PAR** (page address register: where the segment starts in physical memory, in units of 64 bytes) and a **PDR** (page descriptor register: how long it is, up to 8 KB, and whether it may be read or written). The top three bits of a program's address pick a pair; the other thirteen are the offset in that 8 KB piece. 2.11BSD names the user's registers `UISA` and `UISD` (0177640 and 0177600) and the kernel's `KISA0` (0172340) and following. With **separate instruction and data space** (code and data each get their own 64 KB) there are sixteen pairs. `estabur()` fills a process's saved pairs from the sizes of its text, data and stack and fails with `ENOMEM` if they need more than eight segments; `sureg()` copies the saved pairs into the hardware registers when the process is switched in. Each pair describes **base and length** (a start and a size, the oldest form of memory protection), so a program's text, data and stack are each one **contiguous** run of memory, handed out by a first-fit allocator (`malloc.c`: "the core map unit is 64 bytes").

**Reaching the other space.** The kernel's address space is separate from the user's, so a kernel pointer to user memory means something else. `mfpi` and `mtpi` (move from and to the **previous** instruction space; `mfpd` and `mtpd` for data space) take the address from the kernel's registers but use the previous mode's page registers. `fubyte` and `subyte` are built from them, with a global named `nofault` that makes an error in the access return -1 instead of panicking.

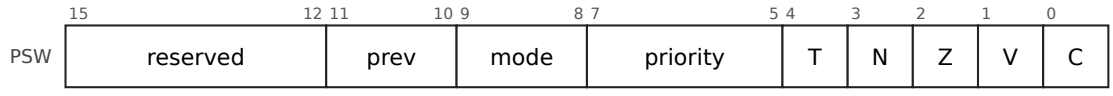
**Swapping.** A process image was one block of memory. When memory ran short, `sched()` called `swapout()`, which wrote the whole image to the **swap device**, and `swapin()` read it back whole. No part of a program could be in memory without the rest.

**What carries over.** The pdp64 keeps three modes with a recognisable status word, a stack pointer per mode, vectors, `RTI`, a priority the kernel raises and lowers, a previous mode and an idle instruction. Eight things are new, each met where it first matters: entry stores state in registers instead of pushing it (6.3); exceptions are precise (6.4); control registers are reached by number (6.5); the eight page registers become a tree of page tables that the hardware walks (6.6, 6.7); a translation buffer caches the walk and must be kept honest (6.8); its entries carry an address-space number so a context switch need not empty it (6.8); the kernel half of the address space has its own table (6.9); and `R7` is an ordinary register (errata 100): the PC is not a register at all and the stack pointer is `R14`.

## 6.2 Modes and the status word

The three modes keep their PDP-11 values: kernel 0, supervisor 1, user 3. Loading the reserved value 2 into the status word is an illegal instruction. The fields moved down to make room for one new bit:

| | PDP-11 (`psl.h`) | pdp64 |
|---|---|---|
| flags (four bits) | 3-0 | 3-0 |
| trace | 4 | 4 |
| priority | 7-5 | 7-5 |
| current mode | 15-14 | 9-8 |
| previous mode | 13-12 | 11-10 |
| user-memory guard | none | 12 |




The **user-memory guard** (a status bit that says whether the kernel may touch user memory; section 6.10) is the new bit. The **previous mode** has the same job as on the PDP-11: after a trap from user mode it says where the kernel came from,

and the instructions that act for the previous mode (`PROBER` and `PROBEW`, 6.10) use it. A **privileged instruction** (one that only the kernel may execute, because it could break the protection between programs) executed outside kernel mode **aborts** (is cancelled before it changes anything, 6.4) with the privileged-instruction vector, as `halt` and `reset` did on the PDP-11.

**6.2.1** There are three modes, with the PDP-11's encoding: kernel 0, supervisor 1, user 3. The supervisor mode exists in the status word and in the banked registers, and no code uses it. *(D)*

**6.2.2** The status word holds the flags in bits 0 to 3, the trace bit in bit 4, the interrupt priority in bits 5 to 7, the current mode in bits 8 and 9, the previous mode in bits 10 and 11, and the user-access guard in bit 12; the other bits are zero. *(D)*

**6.2.3** `MTPS` in user mode writes only the flags; in kernel mode it writes every writable bit. `SPL #n` is kernel-only. *(D)*

**6.2.4** The stack pointer `R14` and the direct-page register `DP` each have one copy per mode; the mode selects which one an instruction names. *(D)*

**6.2.5** These instructions abort with the privileged-instruction vector outside kernel mode: `RTI`, `RTT`, `HALT`, `SPL`, `MTASID`, `MFASID`, `TLBI`, `TLBIA`, `TLBSYNC`, `MTDMW`, `MFSR`, `MTSR`, `PROBER`, `PROBEW`, `WFE`, `ISYNC`, `STUA`, `CLUA`, `PUSHX`, `POPX`. *(D)*

**6.2.6** These are not privileged: `MTDP` and `MFDP` (in user mode, the thread pointer), `ICINV` and `ICINVA` (a JIT needs them), `FENCE`, `SEV`, `HINT`, `TRAP`, `BPT`, `NOP`, and `MTPS` and `MFPS` with the restriction above. *(D)*

`DP` is new to a PDP-11 reader. It is the **direct-page register** (a base register that makes one region addressable with a short displacement, descended from the 6502's zero page). The kernel's copy points to the structure of the core it is running on (the **per-CPU pointer**: which core am I, and where is its private data); a user program's copy points to its thread's private data, so a thread-local variable is one `MOVQ d(DP), Rd`.

*Lineage.* **PDP-11:** three modes, a stack pointer per mode, priority and mode fields in the status word; V7 used kernel and user, and 2.11BSD runs its network code in supervisor mode (`trap()` tests `SUPVMODE(ps)`, checked). **Between:** x86 has four rings of which two are used; AArch64 has exception levels 0 to 3; RISC-V has user, supervisor and machine modes (all *unverified*). Each later design kept a privilege ladder and a banked stack pointer. **pdp64:** the PDP-11's three modes and its stack-pointer banking, so entry from user mode needs no stack switch code, plus a banked `DP`. The third mode costs a third copy of two registers and no logic the kernel uses.

*Impact. Kernel:* a trap from user mode needs no stack switch, and the banked `DP` is a per-CPU pointer in kernel mode and a thread pointer in user mode. *Hardware:* three copies of `R14` and `DP`, selected by the mode, and one mode comparison at the decode of a privileged instruction. `PROBER` and `PROBEW` do the work of the `nofault` idiom of V6 and V7 (6.4).

Details pending: what, if anything, will use supervisor mode (J.6.9).

## 6.3 Exceptions, interrupts and traps: how the machine enters the kernel

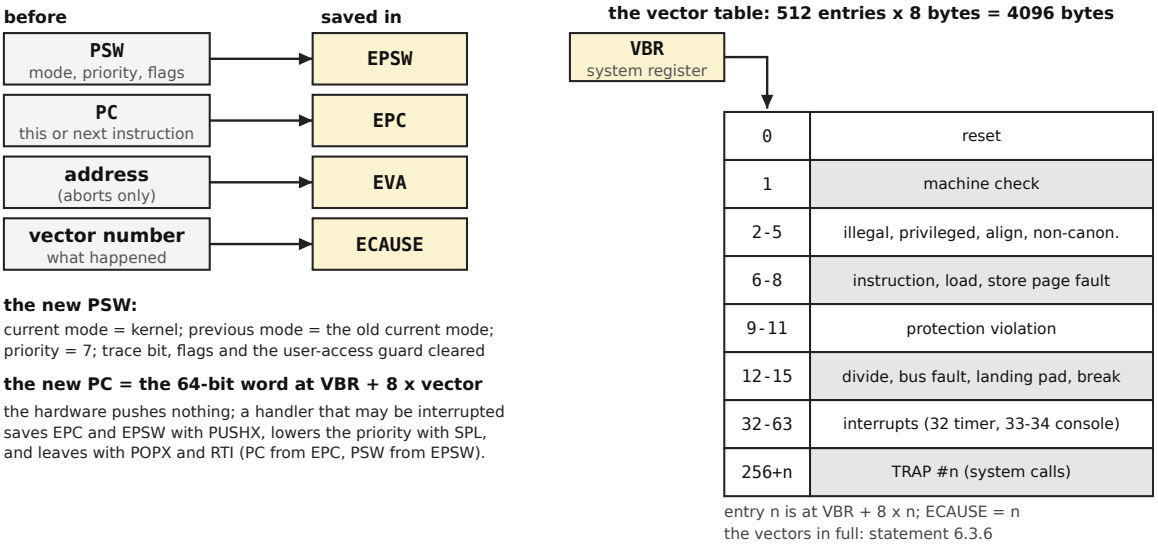
The PDP-11 used one word, "trap", for three different events, and the pdp64 names them apart. An **exception** is the machine stopping an instruction because it cannot carry it out (a page that is not in memory, an illegal instruction, a division by zero); it belongs to the instruction. An **interrupt** is a device asking for attention; it belongs to no instruction and is taken between two of them. A **trap**, in the narrow sense used here, is a deliberate entry by an instruction that succeeds (`TRAP #n`, the system call). All three enter the kernel the same way.

**The PDP-11's way.** The hardware read the new PC and PS from the vector, pushed the old pair on the kernel stack and ran the handler. This is **entry through memory**: two words are written to the stack before one handler instruction runs.

**The pdp64's way.** Entry writes only registers and then reads one word. Four **system registers** (registers outside the general ones, reached with `MFSR` and `MTSR`, 6.5) receive the facts: `EPC` (exception program counter: where to resume), `EPSW` (the old status word), `EVA` (exception virtual address: the address that caused a fault) and `ECAUSE` (what happened: the vector number). The new status word is made, and the new program counter is read from the **vector table**: an array of 512 entries of 8 bytes, 4096 bytes in all, whose base is in the register `VBR` (vector base register). `kind` below is one of the families of 6.4:

```
enter(vector, kind):
    EPC    = (kind is abort or progress) ? address of this instruction
                                         : address of the next instruction
    EPSW   = PSW
    EVA    = (kind is abort) ? the faulting address : 0
    ECAUSE = vector
    PSW.previous_mode = PSW.current_mode
    PSW.current_mode  = kernel
```

```
    PSW.priority      = 7
    PSW.trace = 0 ; PSW.flags = 0 ; PSW.guard = 0        # the guard: section 6.10
    PC = load64_physical(VBR + 8 * vector)               # a bus error here is a machine check
```




Three differences from the PDP-11 matter to a kernel writer. Entry **cannot fault**: writing registers cannot meet a missing page, as a push on a kernel stack could. Entry **leaves the saving to the handler**: the hardware pushes nothing, so a handler that may be interrupted first saves EPC and EPSW (one instruction, PUSHX), and one that cannot be interrupted need not save them. Entry is **the same for every cause**: V7's trap vectors say `br7` and its device vectors `br4` and `br6`, each vector choosing its own priority, while the pdp64 always enters at priority 7 and the handler lowers it. A second exception before the handler has saved EPC and EPSW would overwrite them. The first instructions of every handler, and the memory they touch, therefore must not fault, which is why the kernel's text and stacks are in pages every address space shares or in a direct-mapping window (6.9). (That consequence is this chapter's reading; the architecture document states a related rule, that the entry code and kernel stack are pinned until the save is done.)

**The table.** ECAUSE receives the vector number, so one handler can serve several vectors and tell them apart. Vectors 0 to 31 are for the processor, 32 to 63 for interrupts, 64 to 255 are reserved and 256 to 511 are one per TRAP number. In V7 every `sys n` shared the single vector at 034 and the number was fetched from the instruction.

**6.3.1** Entry to an exception or an interrupt writes only registers: EPC receives the address of the faulting instruction (for an abort or a progress exception) or of the next instruction (for a trap or an interrupt), EPSW the old status word, EVA the faulting address (zero except for aborts), ECAUSE the vector number. *(D)*

**6.3.2** The new status word has the current mode kernel, the previous mode the old current mode, priority 7, and the trace bit and the flags cleared. *(D)*

**6.3.3** The new program counter is the 64-bit word at the physical address `VBR + 8 × vector`; a bus error on that fetch is a machine check that halts. *(D)*

**6.3.4** The hardware pushes nothing. A handler that can be interrupted saves EPC and EPSW with PUSHX, lowers the priority with SPL, and before returning restores them with POPX and executes RTI. *(D)*

**6.3.5** RTI loads the program counter from EPC and the status word from EPSW; RTT does the same and sets the trace bit after the return. *(D)*

**6.3.6** The vector table has 512 entries and ECAUSE equals the vector number. *(D)*

| Vectors | Meaning |
|---|---|
| 0 | reset |
| 1 | machine check |
| 2, 3, 4, 5 | illegal instruction, privileged instruction, misaligned access, non-canonical address |

| Vectors | Meaning |
|---|---|
| 6, 7, 8 | instruction, load and store page fault |
| 9, 10, 11 | protection violation on instruction, load and store |
| 12, 13 | divide by zero, bus fault |
| 14, 15 | landing-pad violation, breakpoint |
| 16 | check (`TRAPCC`; `ADDV`, `SUBV` and `MULV` on overflow) |
| 17 to 31 | reserved |
| 32 to 63 | interrupts (in the virt machine, 32 timer, 33 console input, 34 console output) |
| 64 to 255 | reserved (device interrupts are message-signalled, section 5.9) |
| 256 + n | `TRAP #n` and `TRAPL #n`, n from 0 to 255 |

**6.3.7** There are eight interrupt priority levels; the processor takes an interrupt between instructions when its level is greater than the priority in the status word; the timer is at level 6 and the console at level 4. *(D)*

The two levels are the PDP-11's: V7 vectors the line clock at priority 6 and the console at 4 (`l.s`, checked above). A **message-signalled interrupt** is a device raising an interrupt by writing a word to a memory address, so that no interrupt wire is needed (section 2.6). Vectors 64 to 255 are reserved for it.

Details pending: how such a write becomes a vector number, and which core receives it (J.6.3).

*Lineage.* **PDP-11:** push PC and PS on the kernel stack, load a new pair from the vector; trap vectors conventionally held priority 7 (V7 `l.s`: every trap vector is `br7+code`). **Between:** the VAX pushed the program counter and the processor status longword, and raised the priority to the interrupt's level for an interrupt but not for an exception (*unverified*); x86 pushes five words on the stack (*unverified*); AArch64 and RISC-V latch the return address and status in registers (*unverified*); each of the three masks interrupts on entry (*unverified*). **pdp64:** the register form, so entry cannot fault and costs no memory write; priority 7 is fixed on entry so that no interrupt can overwrite the saved pair before the handler stores it. The kernel survey estimates a system-call entry at four or five instructions on paper, against 45 to 70 on the RISC kernels it examined; it is a paper estimate, not a measurement. Handlers written in C need a short assembler stub to save and restore registers.

*Impact. Kernel:* priority 7 on entry removes the code that would otherwise guard the saved state, and `PUSHX` and `POPX` make the save and restore one instruction each. *Hardware:* entry writes registers only, so it cannot fault, and the vector fetch is one 64-bit read.

## 6.4 Faults and restart: what "precise" means

When an instruction cannot finish (its page is absent, say), the machine must stop it, tell the kernel and, once the kernel has removed the cause, **restart** it (run it again from the start). Whether that works depends on what the instruction did before it stopped.

**On the PDP-11** it did not always work. `cmp -(sp),-(sp)` changes a register (`sp` goes down by 2, twice) while it works out its operands, and may then fault on the second operand; rerun as it stands, it leaves `sp` 4 too low. The hardware therefore kept a small journal of the register changes (the register `MMR1`, on the machines that had one), and the kernel's `backup()` read it and undid them before the restart. The 2.11BSD `trap()` says that on the 11/40, which had no such register, "the routine backup/mch.s may fail", and then so may the automatic growth of the user stack, which relies on restarting. Floating-point errors were worse: V7 and 2.11BSD both say "the floating exception is an imprecise trap", meaning that by the time it was taken the processor had run on, and the saved PC no longer named the faulting instruction.

**Between.** The VAX saved resumable microstate and a status-word bit for instructions interrupted half way; the 68000 had no restart path for a bus error in its first form (both *unverified*, repository text). The next step was to make exceptions precise by design and let the compiler and hardware avoid the cases that were not.

**On the pdp64** every exception that belongs to an instruction is **precise**. A **precise exception** (one taken so that the machine looks as if it ran one instruction at a time, in order: every earlier instruction has completed, and the faulting instruction and every later one have changed nothing) lets the kernel treat a page fault as a subroutine call the program never noticed. It matters here because the pdp64 is meant to be built both as a simple **in-order core** (one that runs instructions one after the other, as the PDP-11 did) and as an **out-of-order core** (one that looks ahead, starts later instructions whose inputs are ready before earlier ones have finished, and holds the results in a buffer until the earlier instructions are known not to have faulted; errata 68). In the second kind the faulting instruction is one of dozens in flight, some already half done; the machine must discard their results and report the fault as if none had started. The architecture makes this possible without a

journal. An instruction has at most one memory operand, and the register update of an auto-increment or auto-decrement mode is held until the instruction **retires** (is certain to complete) and thrown away if it aborts (6.4.2; `docs/pdp64-isa.md` section 7.2 has the argument and 7.3 the table of every reference that can fault). The author's remark on this (2026-11-09): "pdp64 specifically designs the instructions for minimal uops, so we have minimal restart/recovery code needed, no?".

**Four families.** The kernel sees separately the cases that the PDP-11 mixed together:

| Family | The instruction | `EPC` on entry | Restartable |
|---|---|---|---|
| **trap** | completes | the next instruction | not applicable |
| **abort** | annulled, as if it had not begun | the faulting instruction | yes, by `RTI` |
| **progress** | partly done, the progress is in registers | the faulting instruction | yes, by `RTI` |
| **machine check** | indeterminate | best effort | no |

A **progress** exception belongs to the block instructions (`MCPY` and the others of the `MBLK` group, section 9.7). A copy of a megabyte may take 256 page faults; the instruction keeps its position in its own registers, so `RTI` re-executes it and it carries on from where it stopped. A **machine check** descends from the PDP-11's bus error at vector 004: something happened that the machine cannot repair or locate exactly (an uncorrectable memory error, a posted write that timed out), and the kernel can only stop the process or the machine. A **posted write** (a store the processor has handed on and no longer waits for) is the reason: when the bus reports the failure the instruction retired long ago.

**6.4.1** Faults fall in four categories: traps (the instruction completes), aborts (it is annulled and restarted), progress (a block operation stopped part-way, restarted from its registers) and machine checks (not restartable). *(D)*

**6.4.2** The register update of an auto-increment or auto-decrement mode is held until the instruction retires and discarded by an abort, so a handler never undoes anything. *(D)*

**6.4.3** A read of non-existent memory is a precise abort with cause `BUSFAULT`; a posted write that times out and an uncorrectable memory error are machine checks. *(D)*

**6.4.4** A deferred load (`MOVQ.D` and the six other `.D` words) does not fault at the load: a faulting load leaves a poison mark on its destination register, and the fault is taken at the first instruction that writes the flags from a poisoned source. The kernel decides the fault of a deferred load, and the kernel dispatcher, not the hardware, tests the argument registers for poison. *(D)* Details to be determined: the form of the poison bit, and the combination `.UD` of the two qualifiers.

Three consequences. The undo buffer is a single entry, and row 3 of the fault matrix (an operand write) can never fault, so no journal or undo protocol exists. One small page-fault handler serves first touch, first write and demand paging (6.11), and there is no `backup()` and no `MMR1`. The `nofault` idiom of V6 and V7 `trap.c` (set the global, do the probe, `longjmp` out of the handler instead of panicking) remains the pattern for probing hardware, though `PROBER` and `PROBEW` cover address validation. A read of a configuration position that holds no device is not a fault at all (it returns all ones, 2.4.7), so a bus scan does not take a `BUSFAULT` for every empty slot. The pdp64 makes floating-point exceptions flags that never trap, so they need no imprecise case; whether current architectures keep exceptions precise for page faults and leave some floating-point and bus errors imprecise is *unverified*.

*Impact.* The price of precision is in the implementation and not in the kernel: an out-of-order core needs the buffer that holds its results until retirement. *Kernel:* one small page-fault handler for first touch, first write and demand paging; no `backup()` and no `MMR1`. *Compiler:* no scheduling restriction arises from faults, because the register update of an addressing mode is held until retirement. *Hardware:* a single-entry undo buffer, a result buffer for the out-of-order implementation and a retirement stage that decides.

## 6.5 System registers: `MFSR` and `MTSR`

The PDP-11 reached its control registers by address. The KT11's page registers, the status word and the other control and status registers sit in the top 8 KB of the address space, the I/O page (`KISA6` at 0172354, `UISD` at 0177600, the PS at 0177776), and the kernel used ordinary `mov` instructions on them. The pdp64 cannot, because its address space is translated (6.6): a load or store to a control register would itself be translated, and an instruction that changes the translation would change the mapping of the access that reads it. Control registers are **system registers**, numbered 0 to 255, read and written by `MFSR Rd, name` (move from system register) and `MTSR name, Rs` (move to system register). Both are privileged (6.2.5).

**6.5.1** The system registers are reached by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

| Number | Register | Meaning |
|---|---|---|
| 0 | PTBR | physical address of the level-2 page table |
| 1 | EPC | exception program counter |
| 2 | EVA | exception virtual address |
| 3 | ECAUSE | exception cause (the vector number) |
| 4, 5 | DMW0, DMW1 | direct-mapping windows (stored; no effect on translation) |
| 6, 7 | USP, SSP | the user and supervisor stack pointers |
| 8 | EPSW | exception status word |
| 9 | VBR | vector base register |
| 10 | MMUCTL | memory-management control (bit 0 enables translation) |
| 11 | ASID | address-space identifier |
| 12 | CPUID | the core number (read-only) |
| 13 and up | | reserved |

EVA and ECAUSE are read only, as is CPUID; the rest read and write. PTBR is called a level-2 table for the three-level form; it holds the top table of whichever number of levels the machine has (6.6). USP is the user's stack pointer, needed because in kernel mode R14 is the kernel's; it replaces the PDP-11 idiom `mfpd sp`, and the kernel reads and writes it to build a signal frame, copy a process or switch context. SSP is the supervisor's and nothing uses it.

**6.5.2** A write to PTBR, ASID or MMUCTL, or an ICINV, is guaranteed to take effect only after the next ISYNC. *(D)*

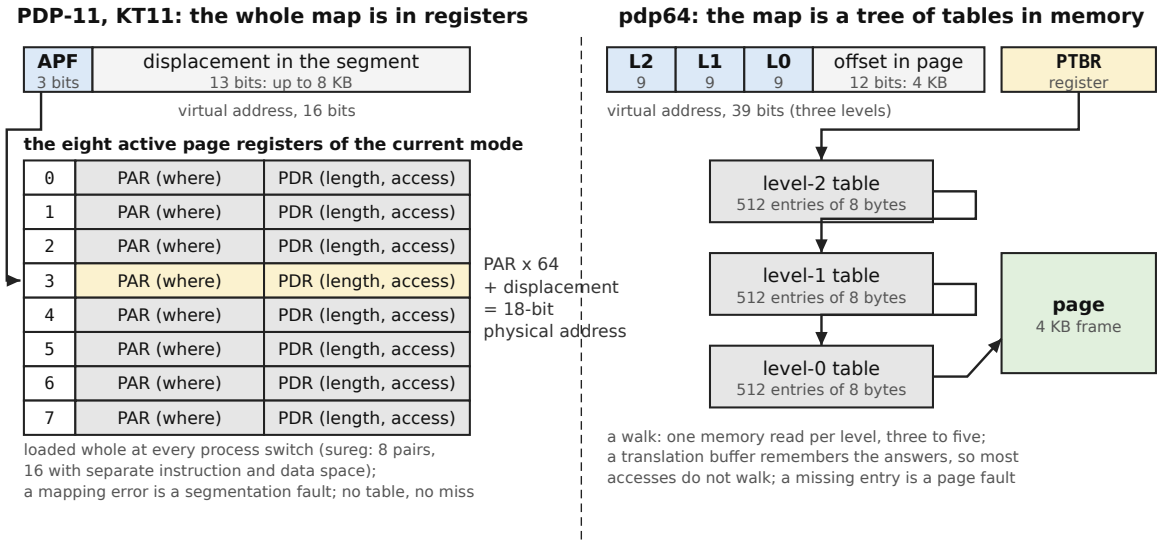
ISYNC (instruction synchronisation) makes this safe on a machine with more than one instruction in progress. A pipelined or out-of-order core has already fetched and started the instructions after the MTSR under the old translation; ISYNC waits for every earlier instruction to finish, discards everything fetched after it and fetches again under the new rules. The PDP-11 needed no such instruction, because an instruction was fetched after the previous one had finished.

*Lineage.* **PDP-11:** control registers at fixed high addresses, reached with ordinary moves. **Between:** the VAX had MTPR and MFPR with an internal-register number; AArch64 has MRS and MSR, RISC-V CSRR and CSRW, PowerPC MFSPR and MTSPR. **pdp64:** the VAX's numbered form. The pair costs two opcodes; a separate instruction per register would spend opcode space with every register. A new register is a number and not an instruction, and the 243 unused numbers cost nothing until used. The kernel survey found privileged-register access to be 48% of all the compiler-invisible instructions in four kernels; the compiler reaches them through an intrinsic with a constant number. *Hardware:* a control-register file addressed by an 8-bit number, with a read-only decode for the identification registers.

## 6.6 From segments to pages: virtual memory

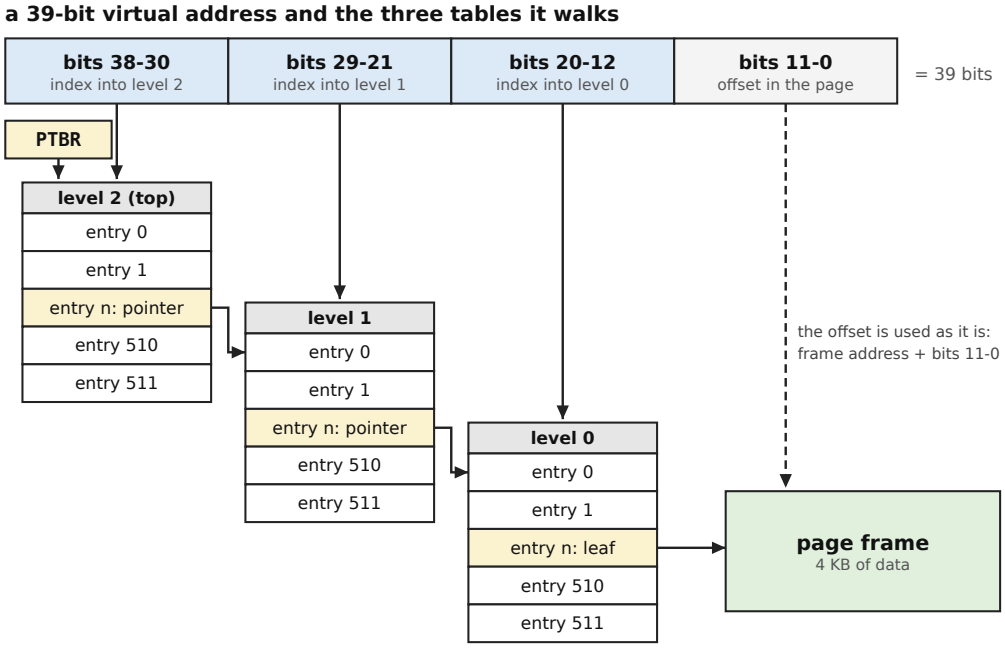
**Two kinds of address.** A **virtual address** is the number a program uses; a **physical address** is the number the memory chips answer to. The PDP-11 had both, 16 bits against 18 or 22, and the KT11 translated one into the other. A pdp64 program has 64-bit pointers of which only some bits count (below), and the physical address is 40 to 64 bits wide (errata 102): 40 bits is one terabyte, and the maximum, $2^{64}$, is sixteen exabytes.

**What the KT11 cost.** The figure puts the PDP-11 scheme (left) beside the pdp64's (right). On the left, the three top bits of a 16-bit address pick one of eight page registers, which says where an 8 KB piece starts and how long it is. Nothing is looked up, because the whole map of the running program is in the eight pairs. Three costs follow. A process had at most 64 KB (eight pieces of 8 KB; 128 KB with separate instruction and data space), and `estabur()` returns ENOMEM for a program that needs a ninth segment. Each piece was one **contiguous** block of physical memory, so memory fragmented, and the allocator either found a hole big enough or the kernel swapped something out to make one. And with only eight registers there was no way to leave a hole in the middle of a program unmapped, to share half a segment, to start a program before all of it was in memory, or to learn which part had been used.




**Between: pages.** The VAX had the remedy that most machines since have copied (*unverified*, repository text): cut both address spaces into equal pieces and let any virtual piece sit in any physical piece. The virtual piece is a **page** and the physical piece a **frame** (the same size, here 4 KB). A virtual address is a **page number** and an **offset** within the page; translation replaces the page number by a frame number and keeps the offset. The map from page numbers to frame numbers is a **page table**: an array indexed by page number, each element (a **page-table entry**, PTE, 6.7) saying where the frame is, who may touch it and whether it is present. With pages a program's memory need not be contiguous; a page can be absent (fetched from disk when touched, which is **demand paging**) or shared by two programs; and the kernel can tell which pages are in use.

**The tree.** One flat table is too large. A 39-bit virtual address with 4 KB pages has $2^{27}$ pages (134 million); at 8 bytes an entry the table is 1 GB for every process, nearly all of it empty. Instead the table is a **tree**. Each table is exactly one page, 4096/8 = 512 entries, so nine bits of index, and some of its entries point to tables of the next level rather than to frames. The address is cut into 9-bit fields, one per level, over a 12-bit offset: with **three levels** it has 9 + 9 + 9 + 12 = 39 bits. Translation is a **walk**:




Each table is 512 entries x 8 bytes = 4 KB. PTBR holds the physical address of the top table.
With four levels bits 47-39 index a level-3 table above these; with five, bits 56-48 index a level-4 table above that.
A leaf found at level 1 maps 2 MB and ends the walk there; a leaf at level 2 maps 1 GB.

```
translate(va, access, mode):                        # three, four or five levels
    if va is not canonical:  fault NONCANONICAL     # bits above the top one must equal it
    table = PTBR                                    # physical address of the top table
    for level = LEVELS-1 down to 0:
        index = (va >> (12 + 9*level)) & 511
        pte   = load64_physical(table + 8*index)    # one memory read per level
        if not pte.V or pte has a reserved bit set: fault PAGE_FAULT
        if pte.R or pte.W or pte.X:                 # a leaf: the walk ends here
            break
        if level == 0:       fault PAGE_FAULT       # a pointer where nothing is left to point to
        table = pte.PPN << 12                       # follow the pointer to the next table
    check the leaf (6.7): permissions for this mode, accessed and dirty (6.8)
    size  = 4 KB << (9*level)                       # level 0: 4 KB; level 1: 2 MB; level 2: 1 GB
    return (pte.PPN << 12) | (va & (size - 1))
```

The pseudocode renders statements 6.6.1, 6.7.1 and 6.7.2; a leaf not aligned to its size is a page fault. The cost is one memory read per level, three to five. A program that touches a few megabytes has a handful of tables, and the **sparse** address space (most of the 512 GB unused) costs nothing, because an entry that is not valid has no table under it. A leaf found above level 0 maps a larger piece, 2 MB at level 1 and 1 GB at level 2: a **large page** (many times the basic size, mapped by one entry; used for the kernel's own mapping and for big buffers). Its frame address must be aligned to its size.

**Three, four or five.** The author decided (errata 96) that the walk may have three levels (39 virtual bits: what System v1 specifies), four (48 bits) or five (57 bits), with the number discoverable at run time. Each level adds nine bits and one more memory read per miss. His own referent was the Loongson five-level table ("what other parts of the design are effected if we have 5 level tlb like loongson arch?"). A **canonical** address has all bits above the top virtual bit equal to that bit, so a 39-bit machine accepts a 64-bit pointer only if bits 63 to 38 are all the same. This rules out storing a tag in the unused high bits, which would stop working on a machine with more levels.

**6.6.1** Virtual addresses are translated by three, four or five levels of nine bits over 4 KB pages, which gives virtual addresses of 39, 48 or 57 bits. With three levels, address bits 38 to 30 index level 2, 29 to 21 level 1 and 20 to 12 level 0; with four, bits 47 to 39 index level 3 above them; with five, bits 56 to 48 index level 4 above that; PTBR holds the physical address of the table of the top level. *(D)*

The number of levels is readable by software at runtime, so a kernel adapts to the implementation without a build option (errata 96); so is the physical address width. Details pending: the register or field that reports them (J.6.1).

**6.6.2** Virtual addresses are 39, 48 or 57 bits, translated by three, four or five levels of nine bits over 4 KB pages (6.6.1) to a physical address of 40 to 64 bits (section 2.3.5). *(D)*

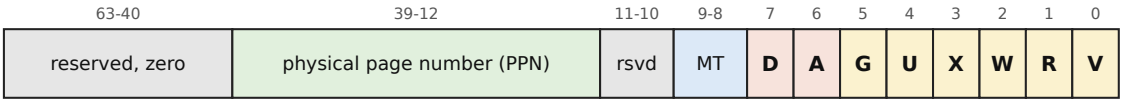
*Lineage.* **PDP-11:** eight base-and-length segments per mode, no tables. **Between:** the VAX had a linear page table per region with a hardware modify bit and no reference bit (*unverified*, repository text); RISC-V's Sv39, Sv48 and Sv57 are the three-, four- and five-level forms with nine bits per level (`satp` modes 8, 9 and 10, checked in the QEMU source); x86-64 uses four levels, and five on newer parts through `LA57`; LoongArch describes the width of each level in a register (`PWCL`, checked). **pdp64:** the Sv39 geometry, with the level count left to the implementation. A leaf at any level makes large pages free for the kernel's own mapping, and the walker only reads, so it is a small state machine. Position-independent code uses PC-relative addressing, and pointers must not carry tags in the high bits.

*Impact.* Thirty-nine bits is 512 GB, what three levels index with nine bits each; each further level adds nine bits (48, then 57) at the price of one more read on every miss, and the kernel chooses what the implementation offers. *Kernel:* the same walk code serves three, four and five levels, with the level count read at run time. *Hardware:* one read per level, three to five.

## 6.7 The page-table entry

Each entry is one 64-bit word. The diagram shows every bit.

**a page-table entry: 64 bits, the physical page number in place**




| | |
|---|---|
| **V** | valid: the entry means something; clear is a page fault |
| **R W X** | read, write, execute; all three clear means "pointer to the next table"; W without R is reserved |
| **U** | user: user mode may touch it (kernel mode may read and write a user page, never execute one) |
| **G** | global: the entry ignores the address-space number |
| **A D** | accessed, dirty: set by the kernel, never by the hardware; a clear bit is a page fault |
| **MT** | memory type: 0 normal cached, 1 normal uncached, 2 device strongly ordered, 3 device weakly ordered |
| **rsvd** | bits 10-11 and 40-63 must be zero; a set bit is a page fault (for a 40-bit physical address) |

Bit positions are those of statement 6.7.1; bits 0 to 7 are in the same places as in RISC-V.

An entry has two shapes. A **leaf** maps: at least one of `R`, `W`, `X` is set, and the physical page number is the frame the page lives in. A **pointer** says that the next table is here: `V` is set, `R`, `W` and `X` are all clear, and the page number is the next table's. When **valid** `V` is zero nothing is mapped, and the other 63 bits are the kernel's to use, for example for the place on the swap device where the page now lives.

**Permissions and modes.** `R`, `W` and `X` allow reads, writes and instruction fetches. `U` says user mode may use the page, and user mode needs it. Kernel mode may read and write a user page, which is what lets it copy to and from user memory directly (6.10), but may never **execute** one, so a kernel tricked into jumping to user memory takes a fault. `W` without `R` is not a valid combination and is a fault. `G` (global) marks a translation the hardware keeps whatever the address space (6.8). Any undefined bit must be zero in a valid entry, and a set reserved bit is a page fault, which lets the format grow and catches a wild value written by the kernel.

**Memory types.** Each page says how the memory system may treat accesses to it, in the field `MT`. **Normal cached** (0) is ordinary memory, which the **caches** (small fast memories near the processor holding copies of recently used memory) may keep. **Normal uncached** (1) must not be cached, as for a frame buffer that the display hardware reads and the processor writes. The two device types are for device registers: **device strongly ordered** (2) sends every access to the device in exactly the order written, one at a time and never combined, and **device weakly ordered** (3) lets the machine combine or reorder accesses, for device memory where only the final result matters. The architecture text gives the four names and numbers; the glosses of the last two are the usual meanings and are *unverified*. The PDP-11 had no memory type: everything at the top of the address space was a device. The pdp64 has **no address bit that says "device"** (errata 97). The memory type is

what a driver sets when it maps a device, and a device window can be anywhere in the physical address space except the first few pages (errata 102, section 2.3.5).

**6.7.1** A page-table entry is 64 bits: `V` (valid) in bit 0, `R`, `W`, `X`, `U` and `G` (global) in bits 1 to 5, `A` (accessed) and `D` (dirty) in bits 6 and 7, the memory type `MT` in bits 8 and 9, and, for a 40-bit physical address, the physical page number in bits 12 to 39, in place; all other bits are zero, and a set reserved bit is a page fault. *(D)*

The physical address is 40 to 64 bits (6.6.2), so the entry has a form for a physical address wider than 40 bits. Details pending: that format (J.6.2).

**6.7.2** A non-leaf entry has `V` set and `R`, `W` and `X` clear and points to the next table; a leaf at level 2 maps 1 GB, at level 1 maps 2 MB and at level 0 maps 4 KB, and where the levels exist a leaf at level 3 maps 512 GB and at level 4 maps 256 TB. *(D)*

**6.7.3** A user-mode access needs `U`; kernel mode may read and write user pages but may not execute them; `W` without `R` is reserved. *(D)*

**6.7.4** The memory type of a page is one of normal cached, normal uncached, device strongly ordered and device weakly ordered. *(D)*

**6.7.5** A page-table entry carries the physical page number in place and read, write, execute and user permission bits; accessed and dirty bits are managed by software; one memory type per page selects cached, uncached, strongly ordered or weakly ordered. *(D)*

*Lineage.* **PDP-11:** a PDR with length and access bits, no per-page type. **Between:** the VAX entry held a hardware modify bit and a four-bit protection field (*unverified*); RISC-V's low eight bits are `V R W X U G A D` in the same places as here (checked in `cpu_bits.h`); ARM's memory types and LoongArch's cache attributes are the source of the four types, reduced to what a machine of this size can tell apart (`docs/pdp64-isa.md` section 8.2). **pdp64:** one entry shape at every level keeps the walker simple. A memory type per page, not per access, means the code that writes a frame buffer needs no cooperation to be uncached, and a driver maps a device register uncached and strongly ordered without a special instruction.

*Impact.* One entry shape at every level makes the walker simple, and a leaf that is not aligned to its size is a fault, which costs a check and saves the walker from reasoning about partial frames. The physical page number is in place, so no shifting. *Kernel:* the entry is built and read with shifts and masks that are the same for every level, and leaves at any level let the kernel map itself with 1 GB pages. *Hardware:* a translation-buffer entry holds a page number as wide as the physical address, 28 bits at 40.

## 6.8 Accessed, dirty, the translation buffer and address-space numbers

**Accessed and dirty.** A kernel that runs programs larger than memory must choose pages to throw out. It needs two facts: whether a page has been used lately (**accessed**, `A`) and whether it has been written since it was brought in (**dirty**, `D`: a dirty page must be written to disk before its frame is reused, a clean one can simply be dropped). On the VAX the hardware set a modify bit and the kernel emulated the reference bit (*unverified*, repository text). Many later machines, x86-64 and AArch64 among them (*unverified*), have the **walker** (the part of the memory-management hardware that reads the tables) set both bits itself by writing the page-table entry. The pdp64's walker **never writes a page table**. An access to a page with `A` clear, or a write to a page with `D` clear, takes the page fault of its kind; the kernel sets the bit and runs the instruction again. These are **software-managed** accessed and dirty bits. The cost is one extra fault the first time each page is touched and one the first time it is written; precise exceptions (6.4) make that fault cheap, and the walker stays a small read-only state machine.

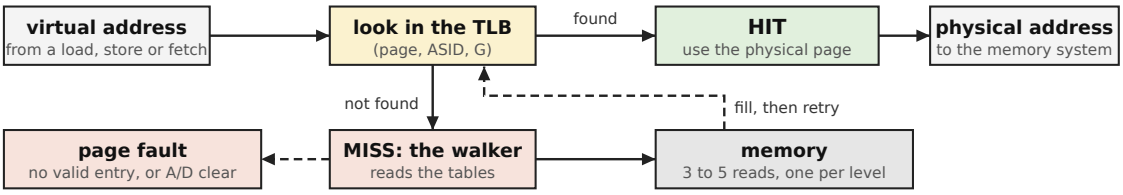
**6.8.1** The accessed and dirty bits are managed by software: an access to a page with `A` clear, or a write to a page with `D` clear, raises the page fault of its kind and the kernel sets the bit; the hardware walker never writes a page table. *(D)*

**The translation buffer.** The walk costs a memory read per level, three to five reads for every load, store and instruction fetch, which is unaffordable. Every machine with a walk therefore has a **translation buffer** (TLB, translation lookaside buffer: a small, very fast memory in the processor that keeps recent translations, so that a repeated access to the same page needs no walk). It is a **cache** (a small copy of a larger store, kept for speed) of translations. The PDP-11 had none: its eight page registers were the translation, loaded whole at each process switch, and there was no table to be slow.

**the translation buffer (TLB): a cache of finished translations**

| ASID | G | virtual page | physical page | R W X U | A D MT |
|---|---|---|---|---|---|
| 7 | 0 | 0x00000412 | 0x0003a1 | R W . U | A D 0 |
| 7 | 0 | 0x00000413 | 0x0007c2 | R . X U | A . 0 |
| - | 1 | 0xFFFFFC001 | 0x000100 | R W X . | A D 0 |
| 12 | 0 | 0x00000412 | 0x0009ff | R W . U | A D 0 |

the same virtual page 0x412 appears twice, for address spaces 7 and 12: that is what the ASID is for;
the global entry (G) answers for every address space and has no ASID.




TLBI rN drops the entry for one page (this ASID and the global ones); TLBIA drops all; both reach every core; TLBSYNC waits until all have finished.
The size and organisation are the implementation's choice (section 6.8).

An access looks in the buffer first, by virtual page number and address-space number (below). On a match (a **hit**) the physical page and permissions come out and the access proceeds with no memory read for the translation. Otherwise (a **miss**) the walker reads the tables; if the walk ends in a valid leaf with the right permissions and the right `A` and `D` it **fills** the buffer and the access is retried, and if not the access takes the page fault of 6.3. A miss costs one read per level; a program that touches no more pages than the buffer holds pays only for its first touches. The size and organisation are the implementation's choice (a **set-associative** buffer lets an entry live in any of a few places chosen by the low bits of the page number, which is cheaper than one that can put an entry anywhere). A buffer entry holds a page number as wide as the physical address, 28 bits at 40.

**Keeping it honest.** The buffer holds copies. When the kernel changes a page-table entry it must see that no stale copy remains, or a page it has unmapped or made read-only would still be reachable. An **invalidate** (removing entries from the buffer; "flush" in the older word) is `TLBI rN`, which drops the translation of the page containing the address in `rN` for the current address space and the global entries, or `TLBIA`, which drops everything. Each core has its own buffer, so the invalidate must reach all of them. On the pdp64 both instructions are **broadcast** (sent to every core by the hardware) and `TLBSYNC` waits until every core has finished, so the kernel knows nobody holds the old translation. Other architectures leave this to software, which sends an inter-processor interrupt to every other core and waits for the answers (a **shootdown**); the pain-points report (`docs/kernel-pain-points.md`) counts the IPI deadlock cases among the costs. Unmapping a page is then three lines, as chapter 7 says (7.12.2):

```
unmap_page(space, va):
    pte_of(space, va) = 0          # change the entry in memory first
    TLBI va                        # drop the old translation, on every core
    TLBSYNC                        # wait until every core has done so
    # now the frame may be reused
```

**6.8.2** Translation is off at reset (`MMUCTL` is zero and every address is physical) and is enabled by bit 0 of `MMUCTL`; the `ASID` selects non-global entries. *(D)*

**6.8.3** `TLBI rN` invalidates the translation of the page containing the address in `rN` for the current address space and the global entries; `TLBIA` invalidates all; both broadcast to every core and `TLBSYNC` waits for the broadcast to finish. *(D)*

**Address-space numbers.** A process switch changes the page tables. If the buffer cannot tell which process an entry belongs to, the old process's entries would answer for the new one, so the whole buffer must be emptied at every switch and the new process starts with all misses. On the PDP-11 the corresponding cost was `sureg()` writing the new process's eight (or sixteen) pairs into the page registers: sixteen (or thirty-two) register writes with nothing to flush. For tables walked from memory the cost is worse, because the emptied buffer refills one miss at a time. The remedy is to **tag** every buffer entry with the number of the address space it came from, the **address-space identifier** (ASID, 16 bits on the pdp64, set with `MTASID`). An entry matches only if its tag equals the current ASID or it is **global** (`G` set, so it answers in every address space, as the kernel's own pages do). A switch is then a write to `PTBR` and to `ASID` and an `ISYNC` (6.5.2): the old process's entries stay in the buffer,

harmless, and answer again if that process runs soon. The whole switch is pictured in 6.11. Open: whether a write to PTBR, ASID or MMUCTL flushes the buffer, as the entry of MFSR in chapter 9 says, which would defeat the purpose of the tags (J.6.8).

There are 65,536 numbers. When they run out the kernel starts again: it allocates in order, and when the counter wraps it invalidates everything (TLBIA) and starts a new **generation**, so that a process holding a number from before is given a fresh one when it next runs. The policy is the kernel's. The architecture has no invalidate by ASID.

*Lineage.* **PDP-11:** no buffer, no tags; `sureg()` rewrote the registers at every switch. **Between:** the VAX emulated reference bits in software under 4.3BSD (*unverified*); MIPS refills its translation buffer in software (*unverified*); x86 added a process-context tag late (the PCIDE bit of CR4, checked in the QEMU source; its width is *unverified*); RISC-V's `satp` carries a 16-bit address-space number (SATP64_ASID, checked) and software flushes other cores' buffers; AArch64 has a broadcast invalidate (*unverified*). The ISA text takes the ASID and the G bit from the Commodore 128's memory-management unit, which had four sets of map registers selected by one store and a common region that no map set could hide (`docs/pdp64-isa.md` section 8.1). **pdp64:** a 16-bit tag and a global bit; broadcast invalidates, so the kernel has no shootdown code; a walker that only reads. A core with no speculation has no page-table side channels, which is one reason for the small walker.

*Impact.* Software-managed bits cost one extra fault on the first touch of each page and one on the first write. A walker that never writes cannot damage a table. *Kernel:* the handler sets the accessed and dirty bits; TLBI broadcasts, so there is no shootdown code; a context switch writes two registers. *Hardware:* the walker only reads; each buffer entry carries an ASID tag and a global bit, compared on lookup.

## 6.9 Splitting the space and the layout of an address space

**One space or two.** On the PDP-11 the kernel had its own address space, with its own eight page registers, and the user could not see it. A kernel on a later machine usually sits in the upper part of every process's own address space, so it can reach the process's memory directly, and the permission bits keep the user out. That arrangement has a hole. A processor that **speculates** (runs instructions ahead before it knows they are allowed, discarding the results if they were not) could on some designs read kernel memory before the permission check caught up, and leave a trace in the caches that a user program could measure. This is the flaw class made public in January 2018 as Meltdown (`docs/hardware-vulnerabilities.md`, section 3.2: CVE-2017-5754, a permission check performed late with data forwarded early). The repository gives it as the reason for the split below: `docs/kernel-pain-points.md`, row 1, "Kernel mapped in every user address space; after Meltdown the page tables had to be split". The pdp64's answer is that **the user does not walk the kernel's tables at all**.

**6.9.1** The translation is split by the top bit of the virtual address (bit 38 with three levels, 47 with four, 56 with five): clear selects the lower half, translated from the table at PTBR (one per address space), and set selects the upper half, translated from the table at the system register KTBR (one for the whole system, the kernel's). In user mode the upper half is never walked: an access to it takes a translation fault without consulting any page table, so no kernel translation is present for a speculating core to read around a permission check. In kernel mode both halves are walked, so the copy routines of 6.10 need no change of root. Entry to an exception changes the mode in hardware and so changes the halves walked; no entry trampoline is mapped in user space. Translation-buffer entries carry the half they came from, and a mode change flushes nothing. *(D)*

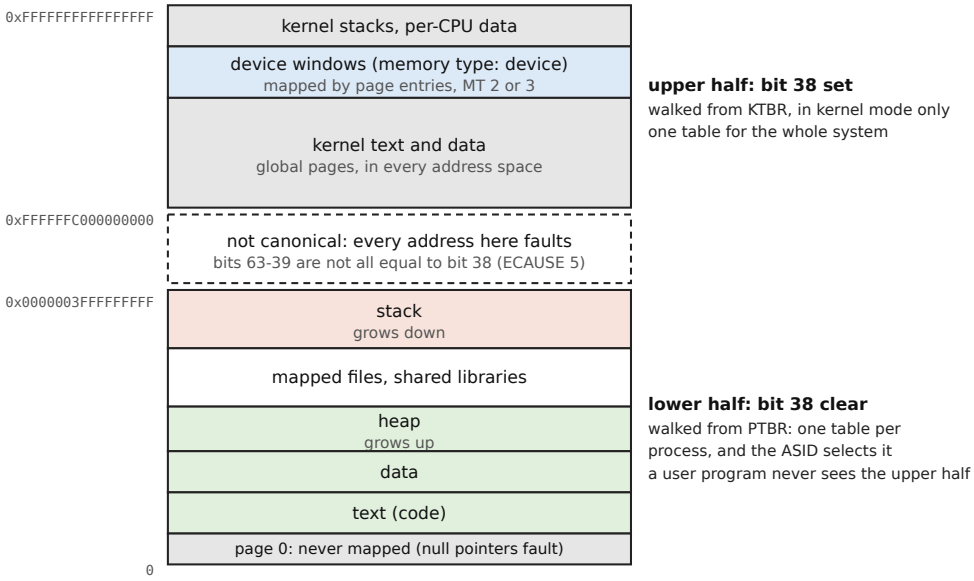
The author accepted this split (errata 71). Details pending: the system-register number of KTBR and the layout of the top-level tables of the two roots (J.6.4).

**6.9.2** The system registers DMW0 and DMW1 are stored and have no effect on translation; MMUCTL bit 0 enables translation and is zero at reset, so every address is physical until the kernel turns it on. The direct-mapping windows exist; their format is deferred until a kernel needs a window (errata 44). *(D)*

The kernel reaches physical memory through the page tables, with leaves of 1 GB where it can (6.7.2).

**The layout of a process's address space.** On a three-level machine a process sees the picture below. The two halves are the architecture's; what goes where in each half is the kernel's to choose, and the figure is one example.

**the 39-bit virtual space of three levels: one example arrangement**




The halves are decided; what goes in them is not specified. Each half is 2^38 bytes = 256 GB with three levels, 128 TB with four, 64 PB with five. The order and the sizes of the regions are the kernel's choice; nothing here is tested.

The **user half** holds the program's text, data, heap and stack, in pages with `U` set, reached through the process's own tree (`PTBR`, selected by the `ASID`); address zero is left unmapped so that a null pointer faults. The **kernel half** holds the kernel's text and data (global pages: one copy of the entries answers in every address space), the per-CPU structures (the kernel's `DP` points at them), the kernel stacks and the mappings of **devices**. A device's registers are mapped by page entries with the device memory type (6.7) at the physical address that PCI Express enumeration found for them (chapter 2); no address bit and no fixed page says "this is a device". Between the two halves lie the addresses that are not canonical, and an access to one takes the non-canonical-address fault (vector 5).

*Lineage.* **PDP-11:** separate kernel and user spaces, crossed with `mfpi` and `mtpi`. **Between:** the VAX divided its address space into a process half and a system half (*unverified*: the VAX manual is not on this machine, though the 4.3BSD sources are); x86 kernels shared the user's tables until KAISER and page-table isolation were merged in January 2018 (`docs/kernel-pain-points.md`, row 1), and have since paid a table switch on every entry (a `cr3` write, `docs/hardware-vulnerabilities.md`, section 3.2); AArch64 has two translation base registers, `TTBR0` and `TTBR1` (*unverified*, not in the QEMU source checked here). **pdp64:** the PDP-11's separation recovered in the VAX's shape, by one half-selecting bit and a second root register. An entry trampoline (a page of kernel code that some architectures map into user space so that an exception has somewhere to land) is not needed. The cost is a second root register, a second walk to cache and a half bit in each buffer entry.

*Impact.* The split removes the kernel from the translation that a user program's speculation can reach. *Kernel:* the kernel maps itself once, global, with large pages where it can. *Hardware:* a second root register and a half bit in each translation-buffer entry.

## 6.10 The user-memory guard and system calls

**Reaching the user's memory.** On the PDP-11 the kernel's pointers and the user's pointers lived in different spaces, so reaching user memory took `mfpi` and `mtpi` (6.1), and a user pointer used by mistake in kernel code failed on its own because the same number meant something else. On the pdp64 kernel and user pages share one address space, so the two kinds of pointer are the same kind of number. The kernel can copy from a user buffer with an ordinary load, and a **buggy kernel** (one that follows a user-supplied pointer without checking) can do it by mistake, which is how many attacks work. The guard is a status-word bit, `UA`: with it clear, a kernel-mode data access to a user page faults. The kernel sets it (`STUA`) only around the copy and clears it (`CLUA`) after, and entry to every exception clears it.

**6.10.1** With the guard bit `UA` clear, a kernel-mode data access to a user page faults; `STUA` sets it and `CLUA` clears it; exception entry clears it and `EPSW` keeps the old value. *(D)*

**System calls.** A system call is `TRAP #n`, or `TRAPL #n` for the long form. It enters through vector `256 + n`, the next instruction becomes `EPC`, and the call number and arguments travel in registers named by the ABI. In V7, `sys n` entered one vector, `trap()` fetched the number back from the user's instruction with `fuiword`, and the arguments were read from the user's stack by `fuword`.

Details pending: which registers carry the system-call number, the arguments and the result (J.6.5).

**6.10.2** `TRAP #n` and `TRAPL #n` (n up to 255, two bytes each: one word with an 8-bit vector number) enter the kernel through vector 256 + n; the system-call number and arguments travel in registers by the ABI. *(D)*

**6.10.3** A copy routine opens the guard, copies, and closes it; every error path out of the copy must close it, and entry clearing it bounds the damage if one does not. *(D)*

**6.10.4** A move with the qualifier `.U` (`MOVQ.U`, `MOVL.U` and the others, seven words) accesses its memory operand with user privilege, so that a copy-in or copy-out routine needs no window in the status word. A fault on that operand is attributed to the instruction and recovered through a **fixup table** (a table of recovery addresses, one entry for each instruction that may fault): the handler looks up the faulting `EPC` in the table, and the copy returns an error. Bit 15 of `ECAUSE` marks the fault. `UA`, `STUA` and `CLUA` stay until the OpenBSD port confirms them, and the atomics take the user form once the operand placement of `ATOM` is fixed. *(D)* Details pending (J.6.12).

A copy routine, for the case that a user address is bad:

```
copyin(user_src, kernel_dst, n):
    if PROBER(user_src) == 0 or PROBER(user_src + n - 1) == 0:   # readable in the previous mode?
        return EFAULT                     # PROBER tests, and takes no fault
    STUA                                  # open the guard
    memcpy(kernel_dst, user_src, n)       # may still fault, if the file was truncated under us
    CLUA                                  # close it, on every path out
    return 0
```

*Lineage.* **PDP-11:** explicit `MFPI` and `MTPI`, so accidental access was impossible. **Between:** the VAX read user addresses directly after a `PROBER` check (*unverified*); x86 added `SMAP` with `STAC` and `CLAC`, AArch64 `PAN` and RISC-V `SUM` (`SUM` checked in the QEMU source; the others *unverified*). **pdp64:** the guard is a mode bit because it lives in the status word, which is saved anyway. A stray kernel pointer becomes a clean fault, at the cost of an open and a close around every user copy, and every error path must close it.

*Impact. Kernel:* every user copy opens and closes the guard; every error path must close it, and entry clearing it bounds the damage of one that does not. The user copy routines are `copyin`, `copyout` and `copyinstr`. *Compiler:* the copy routines are written in assembler or inline assembler; a system-call stub is one `TRAP`. *Hardware:* one status-word bit and one check on a kernel-mode data access to a user page.

## 6.11 The kernel at work: four walks in pseudocode

Everything above is machinery; this section runs it. The pseudocode is **an illustration written for this chapter**, in the style of a V7 kernel, from the statements of this chapter. It is not part of the architecture. Names such as `find_region`, `alloc_frame` and `tick` are what any Unix kernel has and are not specified by the architecture. The mnemonics (`PUSHX`, `POPX`, `SPL`, `MFSR`, `MTSR`, `TLBI`, `TLBSYNC`, `MTASID`, `ISYNC`, `WFE`, `RTI`) are real.

**A system call.** The program puts the call number and arguments in the registers the ABI names and executes `TRAP #0` (or whichever number the system uses). The processor does `enter(256 + n, trap)` of 6.3, which makes `EPC` the instruction after the `TRAP`:

```
vector 256+n  ->  syscall_entry:        # kernel mode, priority 7, R14 is the kernel stack
    PUSHX                               # save EPC and EPSW: one instruction
    save the registers the C code will clobber on the kernel stack
    SPL #0                              # let interrupts in: a system call may take a while
    number = the register the ABI names
    result = sysent[number].handler(args from the registers)     # copyin/copyout use STUA, CLUA
    put result and error in the registers the ABI names
    restore the registers
    SPL #7
    POPX                                # EPC and EPSW back
    RTI                                 # PC = EPC (the instruction after TRAP); PSW = EPSW: user mode again
```

**A page fault.** A load, store or instruction fetch whose translation is not valid, or whose `A` or `D` bit is clear, aborts with vector 7, 8 or 6 (6.3.6). `EVA` holds the address, `EPC` the faulting instruction, `ECAUSE` the kind. The handler below covers the page faults; vectors 9 to 11 (protection violations) are not handled in it.

```
page_fault:                             # vectors 6, 7, 8 (instruction, load, store)
    PUSHX
    save the registers the C code will clobber
    va    = MFSR EVA                    # read both before lowering the priority
    cause = MFSR ECAUSE
    SPL #0                              # we may sleep, reading the page from disk
    region = find_region(current_process, va)
    if region is none:                  # a wild pointer
        send SIGSEGV; goto leave        # (leave delivers the signal on the way out)
    if cause == STORE_FAULT and not region.writable:
        send SIGSEGV; goto leave
    pte = find_pte(current_process.space, va)        # the kernel walks the tree in software, too
    if pte is invalid:                               # the first touch
        frame = alloc_frame()
        fill frame from the file or with zeroes      # may sleep
        pte = make_pte(frame, region.permissions, A = 1, D = (cause == STORE_FAULT))
    else:                                            # valid, but A or D was clear
        pte.A = 1
        if cause == STORE_FAULT: pte.D = 1
    store pte
    TLBI va ; TLBSYNC                                # no stale copy of the old entry on any core
leave:
    SPL #7
    restore the registers
    POPX
    RTI                                              # EPC is the faulting instruction: it runs again
```

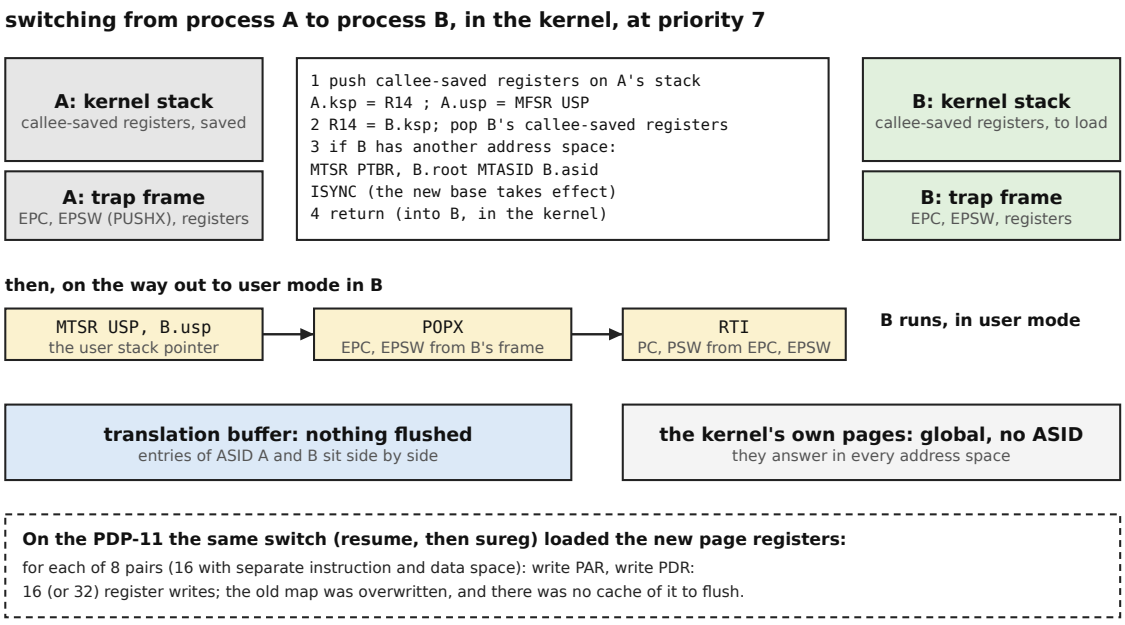
The same handler serves first touch, first write and paging in; only the bits it sets differ. The faulting instruction restarts because the abort was precise (6.4).

**A timer interrupt.** The timer is vector 32 at level 6 (6.3.6, 6.3.7). This is the pdp64's `hardclock`:

```
timer_interrupt:                        # vector 32; entered at priority 7
    PUSHX
    save the registers the C code will clobber
    SPL #6                              # at the device's own level: only a higher one may interrupt
    tick()                              # advance time; charge the running process; run due timeouts
    if --current.time_slice <= 0: need_resched = 1
    SPL #7
    restore the registers
    if need_resched and the interrupted mode (EPSW) was user:
        call switch()                   # V7: "runrun"; see below
    POPX
    RTI
```

The V7 `call` routine does the same test (previous mode user, `_runrun` set) and makes an artificial trap number 12, "give up cpu", before it returns (`mch_i.s`, checked above).

**A context switch.** The kernel has decided to run another process. The user program's registers are already in the trap frame on its own kernel stack, so the switch deals only with the kernel's state and with the user stack pointer, which lives in a banked register that entry did not save. The picture shows what is touched; the V7 equivalents are `swtch()`, `resume()` and `sureg()`.




```
switch_to(old, new):                    # kernel mode, priority 7
    push the callee-saved registers (appendix D) on old's kernel stack
    old.ksp = R14                       # R14 in kernel mode is the kernel stack pointer
    old.usp = MFSR USP                  # the user's stack pointer is not in the trap frame
    R14     = new.ksp                   # one ordinary move: the stack changes here
    pop the callee-saved registers from new's kernel stack
    if new.space != old.space:
        MTSR PTBR, new.space.root       # the new top-level table
        MTASID new.space.asid           # the new address-space number
        ISYNC                           # both take effect now (6.5.2)
    return                              # to wherever new was when it called switch_to
                                        # (the return address was on the stack just loaded)

return_to_user(p):
    MTSR USP, p.usp                     # the user's own stack pointer; R14 is the kernel's
    restore the user registers from p's trap frame
    POPX                                # EPC and EPSW from p's trap frame
    RTI                                 # user mode, user PC
```

No entry in the translation buffer is touched (provided that a write to PTBR and ASID flushes nothing, 6.8): the entries of the process that stopped stay, tagged with its ASID, and are used again if it runs soon, and the kernel's own entries are global and serve both.

Details pending: how the kernel saves and restores a thread's **user** DP (the thread pointer, a banked register of user mode) at a thread switch, since the system-register table (6.5.1) has no number for it and MTDP and MFDP name only the current mode's copy (J.6.7).

**The idle loop.** With no process runnable the kernel waits for an interrupt. The PDP-11 lowered the priority so that an interrupt could be taken and executed `wait`; the V7 `idle` routine clears the priority bits and waits, and 2.11BSD's does SPLLOW and `wait` (both checked). The pdp64's WFE (wait for event) is the idle instruction (7.10.3). WFE ends its sleep for an interrupt that is pending even though the priority masks it, so there is no gap between testing for work and sleeping. The pseudocode is an illustration, written for this chapter:

```
idle():
    loop:
        SPL #7                          # mask everything while we look
        if run_queue is not empty:  SPL #0 ; return
        WFE                             # sleep until an event or an interrupt, masked or not
        SPL #0                          # now take the interrupt that woke us
```

Details pending: the rest of the wake rule (J.7.1).

## 6.12 Reset and the first kernel

**What the machine contains at power-on.** There is no kernel yet. A boot ROM (chapter 4) reads one block into low memory and starts core 0. The **vector table** is 512 × 8 = 4096 bytes, exactly one page; the kernel places it by writing `VBR` and executing `ISYNC`. The PDP-11 also had its vectors at address 0, but as a fixed table of a few hundred bytes (V7's `l.s` goes past octal 0224), and its boot ROMs lived elsewhere. The first few pages of physical memory hold the boot code and the boot data, with the vector table; by errata 102 no device window is mapped there. Details pending: how many pages that is, and where block 0 goes (J.4.1).

**6.12.1** At reset core 0 starts in kernel mode at priority 7 (the entry of the reset vector, 6.3.2) with translation off, the user-memory guard clear and `VBR` zero. To turn translation on the kernel sets `VBR`, builds page tables, writes `PTBR`, sets `MMUCTL` bit 0 and executes `ISYNC`. *(D)*

**6.12.2** The first operating system is OpenBSD, with Unix/32V as the fallback if OpenBSD proves too hard. *(D)*

The first kernel in outline (illustration):

```
kernel_entry:                           # core 0, translation off
    R14 is already a stack
    MTSR VBR, kernel_vector_table       # the table the kernel built or linked
    ISYNC
    read the boot-information block : memory ranges, core count, root complex
    build the kernel's page tables (the upper half, global pages; section 6.9)
    MTSR PTBR, user_root_for_process_0  # the lower half; ASID 0
    MTSR MMUCTL, 1                      # bit 0: translation on
    ISYNC                               # from here every address is virtual (see below)
    enumerate the bus (section 2.4); start the other cores; start the clock; run init
```

Details pending: what covers the instruction fetch after the write that turns translation on (J.6.6).

*Lineage.* **PDP-11:** the memory manager was turned on by writing a bit in its status register (V7 `mch_i.s` does `mov $1,SSR0`, checked), and the VAX did the same with a jump to a mapped address (*unverified*); each needed a transition in the boot code. **Between:** RISC-V's `satp` has a bare mode and AArch64 has `SCTLR.M` (*unverified*). **pdp64:** translation off at reset, one enable bit in `MMUCTL`, one `ISYNC`. The first kernel code is position-independent and uses no virtual address until translation is on.

*Impact. Kernel:* boot code runs at physical addresses until the kernel is ready, and the transition is one write and one `ISYNC`; what covers the next fetch is open (J.6.6). *Hardware:* translation-enable is one bit, a machine-wide mode bit accepted for that reason.

## 6.13 System instructions, exceptions and interrupts

This section collects, in the vocabulary of the kernel, the statements of 6.2 to 6.10 that an implementer needs side by side. The entries of the instructions are in chapter 9.

**6.13.1** There are three modes (kernel, supervisor, user); supervisor is defined and unused. System instructions executed outside kernel mode abort with the privileged-instruction vector. *(D)*

**6.13.2** `TRAP #n` and `TRAPL #n` (vectors 0 to 255, two bytes each) enter the kernel; `RTI` returns, and `RTT` returns and sets the trace bit after. *(D)*

**6.13.3** An exception or interrupt writes only registers: `EPC` receives the address of the faulting instruction (or the next one, for a trap or interrupt), `EPSW` the old status word, `EVA` the faulting address and `ECAUSE` the vector number; the new program counter is the 64-bit word at `VBR + 8 × vector`. *(D)*

**6.13.4** Entry sets the priority to 7 for every exception and every interrupt; the handler lowers it. *(D)*

**6.13.5** There are eight interrupt priority levels; `SPL #n` sets the priority without touching the flags. *(D)*

**6.13.6** `MFSR` and `MTSR` read and write the system registers by number; `TLBI`, `TLBIA` and `TLBSYNC` invalidate translations (broadcast to other cores); `MTASID` and `MFASID` set the address-space tag; `ICINV` and `ICINVA` invalidate the instruction cache; `PROBER` and `PROBEW` test an address; `MFCPUID` identifies the processor. *(D)*

**6.13.7** `UA` in the status word guards user memory: with it clear, a kernel-mode data access to a user page faults; `STUA` sets it and `CLUA` clears it; entry clears it. *(D)*

**6.13.8** `TRAPCC.cc` (fourteen words, one for each condition; one parcel, no operand) enters the kernel when its condition holds; it is a trap and not an abort, so the instruction completes and `EPC` is the next instruction. It takes the check vector, 16 (6.3). `ADDV`, `SUBV` and `MULV` take the same vector on signed overflow, in the two-register form, as an abort that writes nothing. *(D)*

Details pending: the number of the vector and the value of `EVA` (J.6.10).

The modes and exceptions are 6.2 to 6.4 and the multiprocessor instructions are chapter 7. `PROBER` and `PROBEW` replace the `nofault` idiom for address checks, and `ISYNC` follows a write to the page-table base or the vector base. The compiler never emits these instructions except through intrinsics. This is the kernel's whole instruction vocabulary: `MFSR`, `MTSR`, `TLBI`, `RTI`, `SPL`, `ISYNC` and the probes. *Hardware:* the system group takes its sub-opcode from the register field, and the privilege check is one comparison on the current mode.

**The instruction entries for this section are in chapter 9, section 9.14:** each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

| Entry | Mnemonics |
|---|---|
| No operation and landing pad | `NOP`, `LPAD` |
| Return from interrupt or exception | `RTI`, `RTT` |
| Return from subroutine | `RET` |
| Halt the processor | `HALT` |
| Breakpoint | `BPT` |
| Memory fence | `FENCE` |
| Wait for an event | `WFE` |
| Send an event | `SEV` |
| Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache | `TLBIA`, `TLBSYNC`, `ICINVA` |
| Push and pop the exception state | `PUSHX`, `POPX` |
| Instruction synchronisation | `ISYNC` |
| Hint | `HINT` |
| Move from and to a system register | `MFSR`, `MTSR` |
| Move to and from the processor status word | `MTPS`, `MFPS` |
| Set the interrupt priority level | `SPL` |
| System call and trap | `TRAP`, `TRAPL` |
| Invalidate a translation | `TLBI` |
| Move to and from the address-space identifier | `MTASID`, `MFASID` |
| Move to and from the direct pointer | `MTDP`, `MFDP` |
| Move to a direct-mapping window | `MTDMW` |
| Read the processor identifier | `MFCPUID` |
| Invalidate the instruction cache for an address | `ICINV` |
| Probe an address for read or write access | `PROBER`, `PROBEW` |
| Trap on condition | `TRAPCC.NE`, `TRAPCC.EQ`, `TRAPCC.PL`, `TRAPCC.MI`, `TRAPCC.VC`, `TRAPCC.VS`, `TRAPCC.CC`, `TRAPCC.CS`, `TRAPCC.GE`, `TRAPCC.LT`, `TRAPCC.GT`, `TRAPCC.LE`, `TRAPCC.HI`, `TRAPCC.LOS` |

## 6.14 What was checked, and what was not

The rule of the project is to check and not to guess. The handbooks for the PDP-11 and the VAX are not on this machine. What was available, and what each fact rests on:

- **Checked against the Unix sources on this machine** (`<work>/facetlang/unixtree`, files gzipped: the V7M kernel under `V7M/sys` and 2.11BSD under `2.11BSD/sys`): the V7 vector table (`conf/l.s`: the pairs `trap; br7+n`, the console at 060 and 064 at `br4`, the clock at 0100 at `br6`; the file has no `. = ZERO+4` line, the vectors follow a branch and the word 4 at 0 and 2); the processor-status address 0177776 (`PS` in 2.11BSD `mch_iopage.h`) and the status fields in `pdp/psl.h` (`PSL_CURMOD`, `PSL_PRVMOD`, `PSL_IPL`, `PSL_T`, the condition codes); the KT11 register addresses in `pdp/seg.h` (`UISD` 0177600, `UISA` 0177640, `KISA0` 0172340, `KISA6` 0172354); `trap(dev, sp, r1, nps, r0, pc, ps)` and its comment; the system-call dispatch `sysent[fuiword((caddr_t)(a-1))&077]` (V7 `sys/trap.c`); `estabur()` and `sureg()` (eight page-register pairs per mode, sixteen with separate instruction and data space, `ENOMEM` when the segments exceed eight or the image exceeds memory, the segment of at most 128 blocks of 64 bytes); the first-fit core allocator (`malloc.c`: "The core map unit is 64 bytes", "Algorithm is first-fit"); `mfpi`, `mtpi` and the `nofault` global in `mch_i.s`; the `wait` of the V7 and 2.11BSD idle routines; the artificial trap 12 "give up cpu" of the interrupt exit when `_runrun` is set, and the write of 1 to `SSR0` (`mch_i.s`); the comments that the floating exception is an imprecise trap and that `backup()`

may fail on the 11/40 (2.11BSD `pdp/trap.c`); supervisor mode used by the 2.11BSD network (`SUPVMODE`); `sched()`, `swapout()` and `swapin()` by name; the comment about "18 bit controllers present in" 22-bit Q-bus systems (2.11BSD `machdep.c`).

- **Checked against the QEMU sources** (`<work>/qemu-src`): RISC-V's `satp` modes 8, 9 and 10 (Sv39, Sv48, Sv57), its 16-bit address-space field, its entry bits `V R W X U G A D` in bits 0 to 7 and its page number at bit 10, its `SUM` bit; x86's `CPUID` leaf 0x80000008 and the `PCIDE` bit of `CR4`; AArch64's `ID_AA64MMFR0` field `PARange`; LoongArch's `CPUCFG1` fields and `PWCL` register.
- **Checked against the repository:** the statements of this chapter against `docs/pdp64-isa.md` sections 7 and 8, `docs/pdp64-system-architecture.md`, `docs/errata.md` (43, 60, 65, 71, 96, 97, 100, 102, 103), `docs/kernel-pain-points.md` and `docs/hardware-vulnerabilities.md`. The vector numbers and system-register numbers are those of the generated constants (`qemu/target-pdp64/gen/pdp64-consts.h`), which come from the grammars.
- **The repository's sizing, not measured:** the kernel survey's estimate of four or five instructions for a system-call entry, against 45 to 70 on RISC kernels (`docs/kernel-pain-points.md`).
- **Unverified** (general knowledge, the handbooks not being on this machine): the names **APF**, and the layouts of the **PAR** and **PDR** beyond what the 2.11BSD code shows (the length field and the access and extension bits); that the KT11 gave 18-bit physical addresses on the original models and 22 bits on later ones; that the I/O page is the top 8 KB of the physical space; the bit positions of the status word apart from those 2.11BSD's header gives; `MMR1` as the register that recorded the auto-increment changes (the repository's text says so, and the 2.11BSD `backup()` routine is its user); the VAX's regions, modify bit and missing reference bit, its entry sequence, its process and system halves, its `MTPR`, its protection field and its `PROBER` use; that the 68000 had no restart path; that MIPS refills its buffer in software; the width of x86's process-context tag and the five words x86 pushes; AArch64's two translation base registers, `PAN`, hardware-managed accessed and dirty bits and broadcast invalidate; x86 rings, AArch64 exception levels and RISC-V modes; the usual meanings of "strongly ordered" and "weakly ordered"; which instructions the PDP-11 forbade in user mode; that current architectures keep exceptions precise for page faults.
- **Decided by the author and recorded in `errata.md`:** everything stated as a decision with a tag *(D)*; the illustrations of 6.11 and 6.12 are not decisions.

# Chapter 7 — Multiple processors

**A second core costs the pdp64 seventeen operations and no ordering code: memory is sequentially consistent, so a program never needs a barrier between two ordinary accesses; ten atomic instructions make a lock one instruction; the per-CPU pointer is a register that already exists; and one broadcast invalidates the other cores' translation buffers.** The reader is taken to be a first-year computer-science student or a PDP-11 programmer who has read the Unix Version 6 or 7 kernel, and to know nothing of multiprocessing, because the PDP-11 of that kernel had one processor. The chapter starts from how that kernel keeps its data safe on one processor (7.1), follows the machines and kernels that came between (7.2) and then takes the problems in the order a second core raises them: an instruction that cannot be interrupted (7.4), what another core sees and when (7.5), copies of memory in caches (7.6), data and translations that belong to one core (7.8 and 7.9), waiting and signalling (7.10) and cores that differ (7.11). The instruction-set text is `pdp64-isa.md`, section 17; the comparisons with other architectures are in `pdp64-system-architecture.md` and the SMP study `smp-decisions.md`. The start of the other cores at power-on is chapter 4 (4.7), translation buffers are chapter 6 (6.8), and the bus side of atomics is chapter 2 (2.10).

*How to read this chapter.* A term is defined where it first appears, as the technical word followed by a plain restatement. In the pictures a box is a piece of memory, a register or a core and an arrow is an address or a message. Code blocks are pseudocode (C-like, `#` starts a comment) unless the text says they are PDP-11 or VAX assembly; the PDP-11 and VAX code is quoted from the V7, 4.3BSD and NetBSD sources on this machine (7.14). The pdp64 code is **an illustration written for this chapter**, not part of the architecture: the placement of the operands of an atomic instruction is open (7.4.6), so atomics are written as function calls. The decisions are the numbered statements. A fact marked *unverified* is general knowledge that could not be checked on this machine.

## 7.1 What you already know: a kernel for one processor

Take Unix Version 7 on a PDP-11/45 or 70. It has one **processor** (the part that executes instructions; a modern chip holds several, each called a **core**). Even so, two things can disturb a piece of kernel code in the middle: an interrupt handler (a device asks for attention and the processor runs the handler between two instructions) and another process (the kernel switches to it). V7 defends against each with a method that depends on there being one processor.

**Against an interrupt: the priority.** The processor status word holds a priority, 0 to 7, and an interrupt is taken only if its level is higher (chapter 6, 6.3). Code that shares data with an interrupt handler raises the priority, works, and restores it. This is V7's `brelse()` (`dev/bio.c`) putting a buffer back on the free list, which the disk interrupt does too:

```
    s = spl6();                     # priority 6: the disk and the clock are held back
    ... link bp into bfreelist ...
    splx(s);                        # back to the old priority
```

`sleep()` and `wakeup()` in `slp.c` begin the same way, `s = spl6()`. This protects against the interrupt and against nothing else: it says nothing to a second processor, which does not look at this processor's priority.

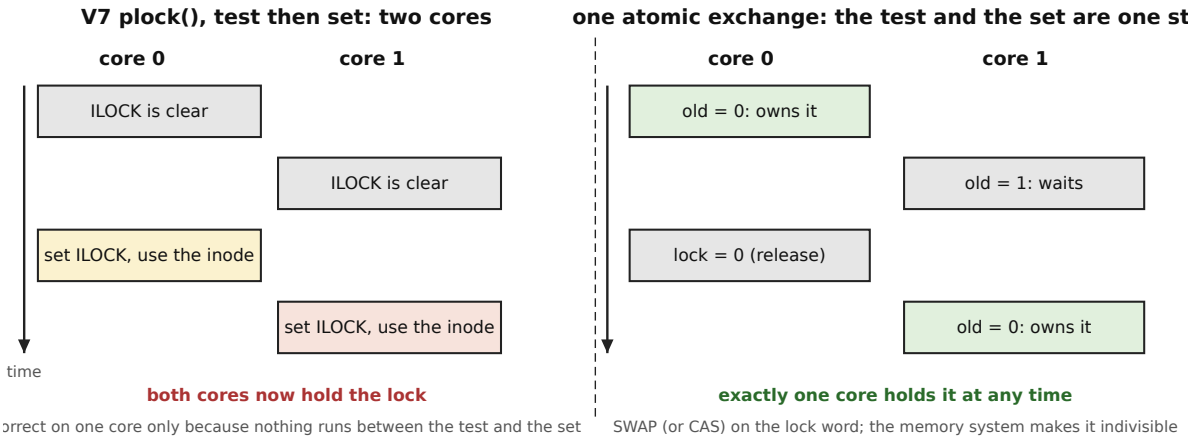
**Against another process: the kernel is not preempted.** `trap()` ends with `if (runrun) qswtch();` (`sys/trap.c`): the clock only sets a flag, and the processor is taken from a process when it returns to user mode or goes to sleep. A process in the kernel therefore runs until it chooses to stop. V7's lock on an inode relies on that. This is `plock()` from `pipe.c` (the loop in `iget()` is the same):

```
plock(ip):
    while (ip->i_flag & ILOCK):     # test: is it locked?
        ip->i_flag |= IWANT
        sleep(ip, PINOD)            # wait; the unlocker calls wakeup
    ip->i_flag |= ILOCK             # then set
```

Test, then set, with no protection between: another process cannot run between the test and the set, so this is correct. It is correct only because no other processor exists (figure below, left).

**An instruction that cannot be split.** `inc`, `bis` and `bic` with a memory operand read the word, change it and write it back. On the Unibus the processor keeps the bus between the read and the write (the cycles `DATIP` and `DATO`; *unverified*, the handbooks are not on this machine), so a device doing its own transfer cannot slip between them. This is the PDP-11's only atomic operation, and it is only atomic against devices.

**Waiting for work.** When no process can run, `swtch()` calls `idle()` in a loop, and `idle()` is a few instructions (V7's `conf/mch_i.s`: `mov PS,-(sp)`, `bic $HIPRI,PS`, `wait`, then `mov (sp)+,PS` and `rts pc`; the variant for machines with separate instruction and data space, `mch_id.s`, writes `spl 0` for the second; 2.11BSD's `idle` in `pdp/mch_xxx.s` uses the macro `SPLLOW`, which is `spl 0` or `clrb PS`, and also loads `r0` for the display lights). `wait` stops the processor until an interrupt comes. There is a gap: an interrupt that arrives after the priority is lowered and before `wait` runs is handled, makes a process runnable, and returns, and the processor then waits with work on the run queue. The clock (`HZ` is 60 in V7's `param.h`) ends the wait within a sixtieth of a second, so a lost wake-up costs a tick and nothing worse.




**The PDP-11 with several processors.** DEC built multiprocessor PDP-11s, the 11/74 among them (*unverified*: the handbook for it is not on this machine, and the repository's notes name it from memory). None of the Unix sources on this machine, V6, V7, 2.9BSD or 2.11BSD, contains code for a second processor: a search for `tstset` and `wrtlck`, the names the notes give for the 11/74's special instructions, found nothing, and the one mention of an 11/74 in these trees is a line of the 2.9BSD setup document that tells its owner to configure a uniprocessor CPU type, 73. The question a second processor poses was therefore answered in no kernel that the reader has read.

**What carries over, and what is new.** The pdp64 keeps the priority and `SPL`, a stack pointer for each mode and an instruction that waits (`WFE`, 7.10). Six things are new, each met where it first matters: an indivisible read-modify-write that works against another core (7.4); a rule for the order in which another core sees one core's accesses (7.5); caches that must agree (7.6); per-core data that the kernel must find (7.8); translation buffers in every core, which one core's page-table change must reach (7.9); and cores that wait for each other, signal each other and are not all alike (7.10 and 7.11).

## 7.2 What came between: the VAX, the BSDs and the PC

**The VAX added the instructions and left the kernel alone.** The VAX-11 architecture has interlocked bit and queue instructions: `BBSSI` and `BBCCI` (branch on a bit, setting or clearing it in the same indivisible step), `ADAWI` (add an aligned word, indivisibly) and the queue instructions `INSQHI` and `REMQHI` (insert at or remove from the head of a queue, indivisibly). These are checked against the VAX-11 Architecture Reference Manual, revision 6.1 (`docs/codegen-data/verification/claims.csv`, claim C028); the 1977 handbook has only the uninterlocked `INSQUE` and `REMQUE`. DEC sold VAX multiprocessors: the 11/782 (*unverified*: two 11/780 processors, only one of which handles input and output), and the 8300, 8800 and 6000 families (all *unverified*). The NetBSD tree on this machine supports several processors on the KA820 (the CPU of the VAX 8200, `vax/ka820.c`), on a KA88 and on a KA6400, through one machine-independent file, `vax/multicpu.c` ("CPU-type independent code to spin up other VAX CPU's"). Its inter-processor interrupt is a message bit set in the target's word with `bbssi`, then a write to a processor register (`PR_IPIR`, "KA820 Interprocessor register", `mtpr.h`), and cleared by the receiver with `bbcci`; one of the messages is `IPI_TBIA`, "invalidate all translations", which `pmap.c` sends to every processor where it invalidates the translation

buffer. A **VAXcluster** is a different thing: several complete computers on a fast bus sharing disks, with no shared memory (*unverified*). Multiprocessing means one memory shared by several processors, and it is the subject of this chapter.

**4.3BSD on the VAX did not use them.** `vax/locore.s` of 4.3BSD takes its exclusion from the interrupt priority register (`mtpr $0x18,$IPL`) and uses `bbss` and `bbcc` (the uninterlocked forms) on the run-queue summary; a search of the `vax` directory finds no `BBSSI`. Its idle loop spins: `idle: mtpr $0,$IPL; tstl _whichqs; bneq sw1; brb idle`, a loop that rechecks the run queues forever at priority 0 (the V7 gap does not arise, since the loop retests every pass). One comment shows what its authors expected: "Masterpaddr is the p->p_addr of the running process on the master processor. When a multiprocessor system, the slave processors will have an array of slavepaddr's". The design is **master and slave**: one processor runs all kernel code and the others run user programs (*unverified* as a history of Unix ports; the comment is the evidence here).

**The kernels grew locks.** Three stages follow, each a bigger change to the kernel than the last (*unverified* as a history, but each stage is visible in a tree on this machine). A **giant lock** is one lock held whenever any processor is in the kernel. Linux 2.6.33's `lib/kernel_lock.c` still begins "This is the traditional BKL - big kernel lock. Largely relegated to obsolescence", and OpenBSD's current `kern_lock.c` has `kernel_lock`, a ticket lock with a spin-out counter. **Fine-grained locks** protect each data structure separately, which lets processors run kernel code at once: FreeBSD 5.3's `sys/mutex.h` carries a 1997 Berkeley Software Design copyright, the BSD/OS mutexes that FreeBSD's SMP rework took over (*unverified* as the lineage). The Linux 0.96c source (a 1992 release, *unverified*) contains no multiprocessor code (a search for `smp`, `spinlock` and `multiprocessor` found none); by 2.6.33 the x86 spin lock is a **ticket lock** (a lock that hands out numbers, like a delicatessen), taken with `lock; xaddw`. The pain-points report records a measured cost of the next design step: an OpenBSD developer reports that replacing its ticket lock by an Anderson lock (each waiter spins on its own memory word: *unverified* as a description) raised an `fstat` loop from 2.3 to 5.9 million operations a second, 155% more (`docs/kernel-pain-points.md`, citing a December 2025 message to the openbsd-tech list).

**The PC added the APIC and kept the work in software.** x86 machines described their processors to the operating system with the Intel MultiProcessor Specification (versions 1.1 and 1.4, named in the header of Linux's `arch/x86/kernel/mpparse.c`) and the **APIC** (advanced programmable interrupt controller: a controller in each core, the local APIC, plus one or more that take device interrupts, the I/O APICs). A core interrupts another by writing to its local APIC; Linux 2.6.33 reserves vectors for it (`irq_vectors.h`: `RESCHEDULE_VECTOR` 0xfd, `CALL_FUNCTION_VECTOR` 0xfc, and eight vectors 0xf0 to 0xf7 for translation shootdowns). The other cores are started by the INIT and START-UP signals (chapter 4, 4.7). The PC's contribution to the ordering problem was a hardware one: x86 allowed itself a store buffer (7.5) and gave programs the `lock` prefix, a one-word atomic read-modify-write, and the `PAUSE` and `MONITOR`/`MWAIT` instructions for waiting (`arch/x86/include/asm/mwait.h`, read).

**Then the memories went weak.** ARM, POWER and RISC-V (*unverified* as a list) let a core reorder its memory accesses as seen by others unless the program inserts a barrier. The DEC Alpha was the extreme: the Linux kernel's `Documentation/memory-barriers.txt` calls it "one of the most relaxed CPUs there is", with a split data cache that updates two related lines at separate times, and writes the Linux rules for the weakest case ("the Alpha defines the Linux kernel's memory model"). That one file is 3,016 lines (counted). 7.5 returns to this.

## 7.3 The multiprocessor instructions at a glance

Before the details, the whole list. The instructions that exist for several processors fall into two sets, ten atomics and seven others, and the umbrella is conceptual: they are not one group in the encoding. The count is of **folded operations**: variants of one operation count once. The plain and the `_DB` form of an atomic are one operation, so are the signed and the unsigned maximum, and so are `TLBI` with its "all" form `TLBIA` and `ICINV` with `ICINVA`. The seventeen are the author's umbrella of 2026-10-21 (R163: "how many folded opcodes in system and smp? i thought atomics and smp would be in the same umbrella"), counted that day over the whole instruction set under this rule (93 operations in all, of which 17 are multiprocessor and 12 other system operations); the table gives the words of the grammar that implement each, the group and the section.

The counts of that day, side by side:

| Set | Operations | Words in the grammar |
|---|---|---|
| atomics | 10 | 22; they are 12 operations when only the `_DB` forms are folded, before the signed and unsigned maximum and minimum are folded |
| other multiprocessor operations | 7 | 9 |
| **multiprocessor, in all** | **17** | 31 |

| Set | Operations | Words in the grammar |
|---|---|---|
| system operations that are not multiprocessor | 12 | no-op (NOP); return from interrupt (RTI, RTT); halt (HALT); breakpoint (BPT); landing pad (LPAD); processor-status access (MTPS, MFPS); set priority (SPL); trap (TRAP, TRAPL); address-space-identifier access (MTASID, MFASID); direct-page access (MTDP, MFDP); direct-map-window write (MTDMW); address probe (PROBER, PROBEW) |

| # | Operation | Words in the grammar | Group | Section |
|---|---|---|---|---|
| 1 | compare-and-swap | CAS, CASDB | atomic (ATOM) | 7.4 |
| 2 | swap | SWAP, SWAPDB | atomic | 7.4 |
| 3 | fetch-and-add | XADD, XADDDB | atomic | 7.4 |
| 4 | fetch-and-and | XAND, XANDDB | atomic | 7.4 |
| 5 | fetch-and-or | XOR_, XOR_DB | atomic | 7.4 |
| 6 | fetch-and-exclusive-or | XXOR, XXORDB | atomic | 7.4 |
| 7 | fetch-and-maximum, signed and unsigned | XMAX, XMAXDB, XMAXU, XMAXUDB | atomic | 7.4 |
| 8 | fetch-and-minimum, signed and unsigned | XMIN, XMINDB, XMINU, XMINUDB | atomic | 7.4 |
| 9 | load-linked | LL | atomic | 7.4 |
| 10 | store-conditional | SC | atomic | 7.4 |
| 11 | device-access fence | FENCE | system (SYS) | 7.5.7 |
| 12 | wait for event | WFE (assembler alias WAIT) | system | 7.10 |
| 13 | send event | SEV | system | 7.10 |
| 14 | invalidate translations, broadcast | TLBI, TLBIA | system | 7.9 |
| 15 | wait for the broadcast to finish | TLBSYNC | system | 7.9 |
| 16 | invalidate the instruction cache | ICINV, ICINVA | system | 7.9 |
| 17 | read the core number | MFCPUID | system | 7.8.1 |

The ten atomics are 22 words (twenty read-modify-write words, a plain and a _DB form each, and LL and SC, which have no _DB form); counting the signed and the unsigned maximum and minimum apart they would be twelve. The seven others are nine words, ten with the alias. Six words were added the day after the count (2026-10-22) and are not in it: MFSR and MTSR, the system-register moves (errata 32), PUSHX and POPX (errata 37), and ISYNC and HINT (errata 39), which the discussion of 7.9.3 and 7.10.1 uses. ISYNC and HINT belong with the twelve system operations that are not multiprocessor, which then number fourteen: like the landing pad or the trap, no second core needs them, and whether HINT #1, the spin-wait pause, belongs under the umbrella is a question of classification and not of count. A read of the system register CPUID (MFSR, 6.5) also gives the core number and is not counted: MFSR is the general register move.

**7.3.1** The instructions that exist for multiple processors fall under one umbrella of seventeen operations: ten atomics and seven others (the table above, folded operations). *(D)*

**7.3.2** Every read-modify-write atomic (every atomic except LL and SC) has a variant, _DB, that folds a barrier into the instruction, so that a lock acquire or release is one instruction. Under sequential consistency the barrier is a convenience (errata 45). *(D)*

By inference (the text says no more), what the barrier still orders under sequential consistency is the accesses that FENCE orders, those to devices (7.5.7); for ordinary memory the model already does it.

**7.3.3** The atomics live in the atomic leaves and the other operations in the system leaves; the umbrella is conceptual, not an encoding. *(D)*

**7.3.4** Supporting these are the banked direct-page register, the address-space tag, the global-page bit and the banked stack pointer (section 7.8). *(D)*

*Commentary.* **Chain.** PDP-11: read-modify-write instructions that the bus keeps whole, and nothing else. VAX: interlocked bit, word and queue instructions, added to a machine that BSD still ran with `spl`. The RISC architectures that followed: load-linked and store-conditional (MIPS, Alpha, POWER, ARM, RISC-V; `arch/arm64/include/asm/atomic_ll_sc.h` uses `ldxr` and `stxr`, `arch/riscv/include/asm/cmpxchg.h` uses `lr.w` and `sc.w`, both read). x86: the `lock` prefix on a read-modify-write instruction (`LOCK_PREFIX "addl"`, read). Later ARM and RISC-V added single-instruction forms, `cas` (`atomic_lse.h`) and `amoswap` (`cmpxchg.h`), both read. pdp64: all three designs, a compare-and-swap, a fetch-and-operate family and a load-linked pair. **Trade-offs.** The set is the union of the three designs, which the repository's survey counts as more atomic operations than any architecture it surveyed (repo, `docs/decisions-pending.md`), in exchange for nothing a kernel writer has to emulate; each fetch-and-operate is a couple of hundred lookup tables and saves a compare-and-swap retry loop (repo, `docs/pdp64-isa.md` section 15). **Improvements.** The _DB forms were taken from LoongArch: a lock acquire should be one instruction, with no separate

fence. **Benefits.** The seven non-atomic operations are the minimum that a kernel cannot synthesise: which core am I, wait and signal, invalidate everywhere, and make the instruction fetch see what the data side wrote (and, a seventh, the device fence). **Impact.** *Kernel:* each of the seventeen operations is one instruction in the kernel's lock, idle, shootdown and per-CPU code. *Compiler:* C11 atomics and the kernel's intrinsics map to them directly. *Hardware:* the atomics live in one escape group and the other seven in the system group, so the decoder has two places to look.

## 7.4 Atomic operations

**The problem.** The figure of 7.1 shows two cores both finding `ILOCK` clear and both setting it. The test and the set must be **one step that no other core can interrupt**: an **atomic** operation (from the Greek for indivisible; here, an operation of which every other core sees either all or none). The hardware offers four shapes of atomic, and the pdp64 has all four. In the pseudocode each line of an atomic's body happens as one step, and `a` is the address of an aligned word:

```
SWAP(a, v):      old = *a ; *a = v ; return old                        # test-and-set when v is 1
CAS(a, e, n):    old = *a ; if old == e: *a = n ; return old            # compare-and-swap
XADD(a, v):      old = *a ; *a = old + v ; return old                  # one of the fetch-and-operate
family
LL(a):           take a reservation on a's granule ; return *a         # load-linked
SC(a, v):        if the reservation is intact: *a = v ; return success   # store-conditional
                 else: write nothing ; return failure
```

`XAND`, `XOR_`, `XXOR`, `XMAX`, `XMIN` and the unsigned `XMAXU` and `XMINU` have the shape of `XADD` with another operation. `CAS` and `XADD` are the same idea as `SWAP`, extended: `CAS` writes only if the word still has the value the program expected, which is how a lock-free list changes a pointer (if two cores race, one `CAS` fails and the loser retries). The weakness of `CAS` is the **ABA problem**: it compares values, so a word that changed from A to B and back to A looks untouched, though the structure behind it may have changed. `LL` and `SC` do not have it, because `SC` fails if anything wrote the word in between, even if it wrote the same value. `CAS` operates on 4 or 8 bytes, and an 8-byte `CAS` may span two adjacent 4-byte fields (9.13.2).

**The reservation.** `LL` does not lock anything. It marks a **granule** (a naturally aligned piece of memory, at least 8 bytes, of a size the implementation chooses) as reserved by this core; a write to the granule by any other core, by a device, or by an atomic clears the mark, and `SC` writes only if the mark is still there. This is called a **monitor**, in the word of the bus designs it maps onto, as against the PDP-11's bus lock, which stops everyone else's access for the duration of the instruction pair. A lock is simple and a bottleneck as cores multiply; a monitor lets other cores go on with other addresses. A monitor needs a **progress rule**: with two cores each doing `LL`, then writing, then `SC`, both reservations could be cleared forever. The pdp64 takes the rule that RISC-V gives (checked in the RISC-V manual in an earlier review: `docs/decisions-pending.md`): a constrained sequence, short and with no other memory access or exception between the `LL` and the `SC`, must eventually succeed if no other initiator keeps writing the granule.

**7.4.1** `LL` takes a reservation on a naturally aligned granule of memory, of a size the implementation chooses and not smaller than 8 bytes; a write to the granule by any initiator, core or device, clears it; `SC` succeeds exactly when the reservation is intact and otherwise writes nothing. *(D)*

**7.4.2** A constrained `LL`/`SC` sequence (short, with no other memory access and no exception between the two) must eventually succeed if no other initiator keeps writing the granule. *(D)*

**7.4.3** The read-modify-write atomics (`CAS`, `SWAP`, `XADD` and the others) are single-copy atomic with respect to every initiator, cores and devices alike; the memory system may build them from an exclusive access with hardware retry. *(D)*

**7.4.4** A device that writes memory shared with a core takes part by clearing reservations. *(D)*

**7.4.5** Open: what an atomic does on uncached memory and on device memory (J.7.4).

**7.4.6** Open: where the registers and the size of the atomic words sit in the instruction (J.7.3); the granule, and how software learns its size, are not fixed either (J.7.6).

**What a kernel does with them.** The lock of 7.1 becomes one `CAS` with the barrier built in; the release is an ordinary store, because under sequential consistency (7.5) nothing more is needed. A **ticket lock**, the lock V7's successors converged on, is one `XADD` to take a number and a plain store to call the next. A counter is one `XADD`; a lock-free structure uses `CAS`; the rest use `LL` and `SC`. Illustration, not architecture:

```
lock_acquire(l):                          # l holds 0 (free) or 1 (held)
    while CAS_DB(l, 0, 1) != 0:           # one instruction, with the barrier folded in
        HINT #1                           # tell the core this is a spin (7.10)
lock_release(l):
    *l = 0                                # an ordinary store

ticket_acquire(t):                        # t has two counters: next, serving
    mine = XADD_DB(&t->next, 1)           # take a number
    while t->serving != mine: HINT #1
ticket_release(t):
    t->serving = t->serving + 1           # serve the next number

atomic_or(a, bits):                       # any operation that is not in the fetch-and-operate set
    loop:
        old = LL(a)
        if SC(a, old | bits) fails: goto loop     # nothing between LL and SC but register work
```

*Commentary.* **Chain.** PDP-11: one processor, the lock of 7.1 and the `spl` calls; atomic only against devices. VAX: `BBSSI`, `ADAWI` and `INSQHI` in the architecture and `spl` in 4.3BSD; in the NetBSD VAX port `bbssi` takes a spin lock and sets an inter-processor message bit (`vax/lock_stubs.S`, `multicpu.c`, read). x86: `lock cmpxchg` and `lock xadd`; Linux 2.6.33 builds its ticket lock on `lock; xaddw` (`arch/x86/include/asm/spinlock.h`, read). ARM and RISC-V: exclusive pairs and, later, single-instruction atomics. pdp64: a monitor, so that the bus and the memory controller need no lock cycle (2.10), and the whole set so that the compiler's C11 atomics map one to one. **Trade-offs.** A monitor needs forward-progress rules and, under contention, can livelock if the rules are not specified; the guarantee for a constrained sequence is the model. The pain-points report records that the kernels' simple lock does not scale (7.2: the Anderson lock) and that Linux moved from test-and-set through ticket locks to queued locks; the instruction set provides the primitives and the lock design is the kernel's. **Benefits.** A lock needs one instruction; lock-free queues need `CAS`. **Use.** `CAS` for lock-free structures and lock words; `XADD` for counters and ticket numbers; `LL`/`SC` for everything else. **Modern comparables.** RISC-V `LR`/`SC` with its constrained-loop guarantee, AArch64 load-exclusive and `CAS`, x86 `LOCK CMPXCHG`. **Impact.** *Kernel:* a lock is one `_DB` instruction. *Compiler:* a `LL`/`SC` loop must be emitted in the constrained form, with no other memory access between the two; C11 atomics map one to one, and under sequential consistency no ordering fence is emitted. *Hardware:* a reservation granule tag in the cache controller and an exclusive-access path to memory; the read-modify-write atomics use an exclusive access with hardware retry.

**The instruction entries for this section are in chapter 9, section 9.13:** each entry there is complete (syntax, encoding diagram, operation, flags, exceptions, status); the commentary above gives the reasons and the use.

| Entry | Mnemonics |
|---|---|
| Compare and swap | `CAS`, `CASDB` |
| Atomic exchange | `SWAP`, `SWAPDB` |
| Atomic fetch and add | `XADD`, `XADDDB` |
| Atomic fetch and and, or, xor | `XAND`, `XANDDB`, `XOR`, `XOR_DB`, `XXOR`, `XXORDB` |
| Atomic fetch and minimum or maximum | `XMAX`, `XMAXDB`, `XMIN`, `XMINDB`, `XMAXU`, `XMAXUDB`, `XMINU`, `XMINUDB` |
| Load-linked and store-conditional | `LL`, `SC` |

## 7.5 Memory ordering: from sequential consistency to the weak models

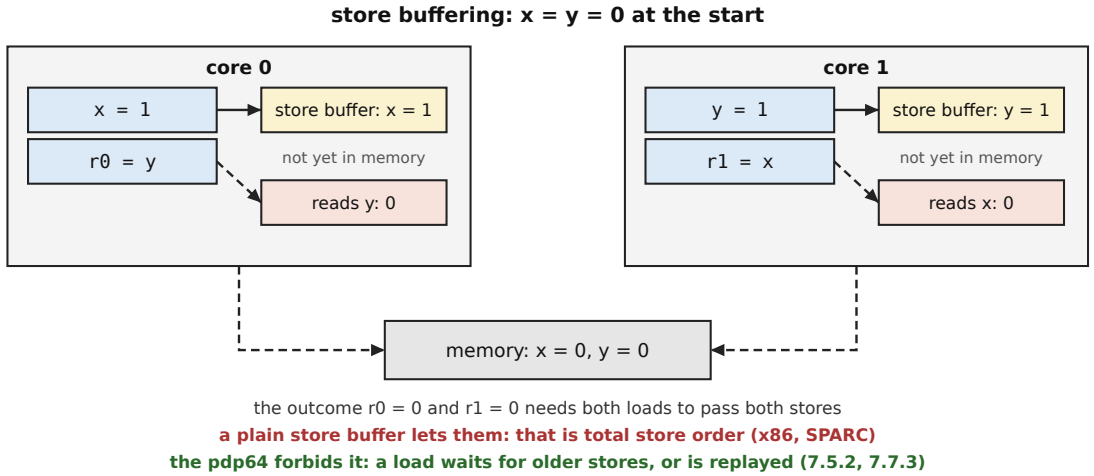
**What the question is.** Two cores share memory. Core 0 executes two stores, core 1 two loads. In what order may core 1 see the stores? On a PDP-11 the question had one answer, program order, because one processor did one access after another. With several cores each core has a **store buffer** (a small queue between the core and its cache that holds stores that have been executed and not yet delivered, so that the core need not wait for the slow memory), and a core may also execute loads early. A **memory model** is the architecture's promise about which values a load may then return. **Sequential consistency** (SC; Lamport, 1979, *unverified*) is the strongest and simplest promise: the result is as if the accesses of all the cores were executed one at a time, in one order, with each core's accesses in that core's program order. Weaker models allow results that no such order explains, in return for letting the hardware reorder.

**Three litmus tests.** A **litmus test** is a tiny two-core program with one interesting outcome that a model either forbids or allows. Memory starts at zero; `r0` and `r1` are registers.

```
SB (store buffering)        MP (message passing)        LB (load buffering)
core 0:  x = 1              core 0:  data = 1           core 0:  r0 = y
         r0 = y                      flag = 1                    x = 1
core 1:  y = 1              core 1:  r0 = flag          core 1:  r1 = x
         r1 = x                      r1 = data                   y = 1
the outcome r0 = 0, r1 = 0   the outcome r0 = 1, r1 = 0   the outcome r0 = 1, r1 = 1
```

| Model | SB | MP | LB | Machines |
|---|---|---|---|---|
| Sequential consistency | forbidden | forbidden | forbidden | the pdp64 |
| Total store order (TSO) | allowed | forbidden | forbidden | x86, SPARC |
| Weak | allowed | allowed | allowed | ARM, POWER, RISC-V, Alpha |

(*Unverified*: the table gives the standard results of the published litmus-test literature; the Linux documentation on this machine describes the weak case in `memory-barriers.txt`, and the Linux tree keeps real tests in `Documentation/litmus-tests`.) MP is the one every program meets: core 1 waits for the flag and then reads the data. Under SC and TSO it works with no help; under a weak model it needs a barrier on both cores. Linux's `locking/DCL-broken.litmus` is **double-checked locking** (test a flag without the lock, take the lock only if it is clear): its header says "Result: Sometimes", and the file's own comment says "more than just locking is required". The kernel's answer on weak machines is a barrier at each such place; the ordering vocabulary of the current Linux tree matches 5,596 lines of code in 855 files and 743 lines of documentation (repo, counted in `docs/kernel-pain-points.md`).




**The pdp64's choice.** SC is the one model that costs software nothing. A program written against a weak model runs on a strong machine and not the reverse, so the choice is close to irreversible and is made once. It costs hardware: a core that reorders, speculates and buffers is faster than one that does not, and holds SC only by checking afterwards. The first pdp64 cores are in order and do not speculate, so they have nothing to buy with a weaker promise (7.7).

**7.5.1** Memory is sequentially consistent: the result is as if all cores' accesses happened in one order that respects each program's order. *(D)* The one exception is the accesses inside a block operation (7.5.8).

**7.5.2** A store buffer is allowed, provided loads snoop it, stores drain in program order, and a load does not complete ahead of an older store to another address that is still in the buffer (or is replayed if it did, 7.7.3). Without the last condition the buffer gives the SB outcome of the figure, which is total store order and not sequential consistency. *(D)*

**7.5.3** Quad accesses are naturally aligned, so every aligned access is atomic within its own size and cannot be torn across cores. An explicit unaligned form (`MOVUB` to `MOVUQ`, 9.6.5) is two accesses in a defined order, each atomic and the pair not. *(D)*

**7.5.4** The first implementations are in order and do not speculate, so no speculative side channel exists in them to be mitigated; an out-of-order core must keep that property (7.7). *(D)*

**7.5.5** The ABI says nothing about memory ordering, because with sequential consistency there is nothing to say. *(D)*

**7.5.6** Memory is sequentially consistent per byte: a load returns, for each byte, the value written by the last store to that byte in the single order, and an aligned access of N bytes is single-copy atomic. A load that overlaps a buffered store of a

different size either waits for the buffer to drain or merges the buffer and memory in one step; it never returns a mixture that no order of the stores could produce. *(D)*

**7.5.7** `FENCE` orders device accesses. *(D)*

**7.5.8** The accesses inside one block operation (`MCPY`, `MMOV`, `MSET`, `MCMP`, `MSCN`, `MSCNN` and `MVAL`, 9.11.3) are unordered with respect to one another: only the final contents of memory and the progress held in the registers are defined (errata 39, item 7). *(D)* Details pending (J.7.7).

**7.5.9** Open: what another core may see of a block operation that is still running, how large an access inside it may be torn, what a program must execute to publish its result, and what happens to the other rules of this chapter when it is interrupted or faults (J.7.7).

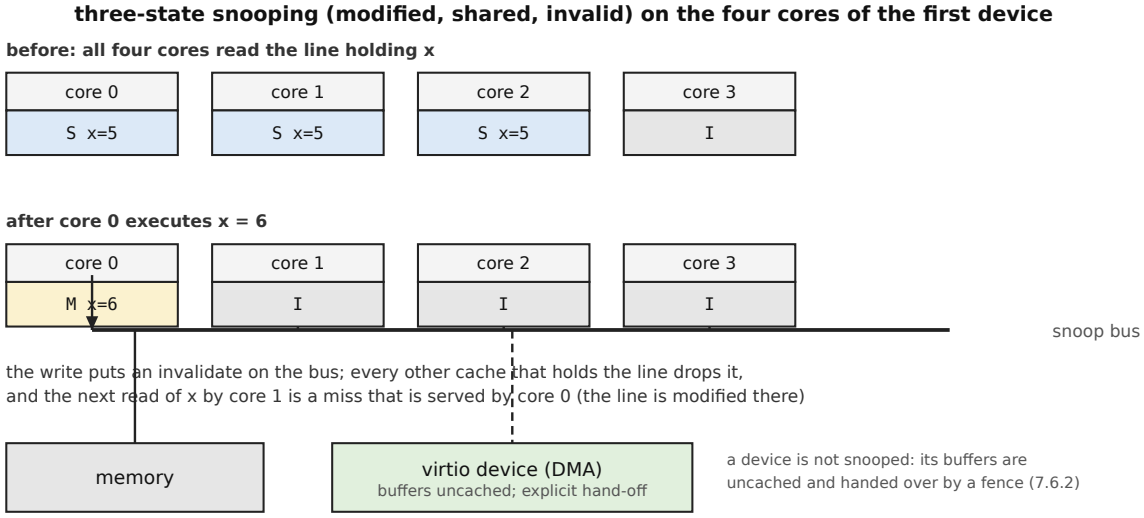
Under 7.5.8 a block operation as a whole is not atomic (inference: it may stop between steps, 9.11.4) and its accesses do not follow the single order of 7.5.1, which is why the review wrote the exception down: an implementation that moves 64 bytes a step, as 9.11.5 allows, can show another core an order that no byte-at-a-time copy produces.

*Commentary.* **Chain.** PDP-11 and VAX: one processor in the model that mattered, so the question did not arise; devices were ordered by the bus. Alpha (1992): the weakest model ever shipped, with the Linux rule that even a load that depends on another load's result may need a barrier (`memory-barriers.txt`, "AND THEN THERE'S THE ALPHA", read). x86: total store order, a store buffer and nothing else. ARM, POWER and RISC-V: weak, with formal models that took years to write (*unverified*), and C11 and C++11, whose model exists to hide the differences between them (*unverified*). pdp64: SC, by the rule that a load never passes an older store to another address. **Trade-offs.** SC is the model with no cost to software and the highest cost to a large speculative core; the pdp64 pays it with a small in-order core and with the load replay of 7.7.3 if it ever builds a large one. The cost of a store buffer under SC is that a store followed soon by a load of another address makes the load wait; a store followed by arithmetic does not stall. **Benefits.** None of the barrier code that fills other kernels: the kernel survey found barrier vocabulary in 5,596 lines of Linux code and none of it arises here. **Modern comparables.** x86 and SPARC are total store order; ARM, POWER and RISC-V are weak. **Impact.** *Kernel:* none of the 5,596 lines of ordering code found in Linux, nor the barrier documentation, is needed. *Compiler:* the optimiser may move non-volatile accesses as C allows, and no fence is inserted for ordering. *Hardware:* in order with a snooped store buffer and loads that wait for older stores to other addresses; the expensive part of SC appears only on a speculative core.

## 7.6 Cache coherence

**What you know, and what the problem is.** The PDP-11/70 had a **cache** (a small fast memory that holds copies of recently used words of the big slow one) in front of main memory (*unverified*; the handbooks are not on this machine). With one processor and a device that writes memory by DMA, the only stale copy is a cache word that a device has overwritten, and the cache controller can watch the bus for that. With several cores, each core has its own cache, and a write by core 0 leaves a stale copy of the word in core 1's. **Cache coherence** is the rule that makes the copies agree: for any one word, all cores see the writes to it in one order. Coherence concerns one word; the memory model of 7.5 concerns the order across many words, and a machine needs both.

**Snooping.** In a **snooping** protocol every cache watches a shared bus. Each cache keeps a state for each line (a **line** is the unit of copying, 64 bytes on most current machines, *unverified*; the pdp64's line size is open, J.7.2): **modified** (this cache has the only copy and has changed it), **shared** (several caches hold it unchanged) or **invalid** (no usable copy). A write to a shared line first puts an invalidate on the bus, and every other cache drops its copy (the figure). A read of a line that another cache holds modified is served from that cache. This is the protocol of the first device, with three states, and it needs a snoop port on each data cache, about 500 lookup tables per port and a small state machine (repo, `docs/pdp64-isa.md` section 17.3: a paper estimate). **MESI** adds a fourth state, **exclusive** (the only copy, unchanged), so that a write to a line nobody else holds needs no bus transaction. A **directory** replaces the bus watching by a table that records which caches hold each line, and sends messages to those only.




**Devices are the other case.** A DMA device does not take part in snooping: the queue memory and the buffers of a virtio device are uncached (2.5.2) and the hand-off between the core and the device is explicit, a FENCE and a doorbell write. The reasons are that snooping the DMA path costs hardware and that DMA transfers are large, infrequent and have an explicit hand-off.

**7.6.1** Between processors, coherence is transparent to software, by snooping with three states (modified, shared, invalid), for the four cores of the first device. *(D)*

**7.6.2** A DMA device is the opposite case: the buffers are uncached and the hand-off is explicit, because snooping the DMA path is expensive and DMA transfers are large, infrequent and have an explicit hand-off. *(D)*

**7.6.3** The design supports at least four cores; sixteen or more is the preferred ceiling that no decision may preclude; the initial device has four cores, which it is hoped will fit the FPGA (errata 92). *(D)*

**7.6.4** Open: the coherence protocol for more than four cores (J.7.2).

*Commentary.* **Chain.** PDP-11: one processor and a cache that DMA writes invalidate (*unverified*). Bus-snooping multiprocessors (Sequent, Sun Enterprise): snooping to some 16 to 64 processors, where every cache sees every transaction (*unverified*). Directory machines (Stanford DASH, SGI Origin): messages to the sharers only (*unverified*). x86 and ARM: filters or directories inside a chip and between sockets (*unverified*). pdp64: snooping with three states for four cores; the design does not preclude sixteen, and the protocol that serves them is open (J.7.2). **Trade-offs.** Software-managed coherence is cheaper in hardware and is where concurrency bugs live, since every missing flush is a race that reproduces once a week. Three states cost less than MESI, whose benefit at four cores is small (repo, ISA section 17.3). **Benefits.** The instruction set owes nothing to the coherence protocol, which keeps the two separate: the architecture promises "coherent and sequentially consistent" and says nothing of the protocol, so more cores are an implementation and not a change. **Impact.** *Kernel:* no coherence code, in either direction between cores. *Compiler:* none. *Hardware:* a snoop port on each data cache and a three-state machine; DMA buffers stay uncached.

## 7.7 What a simple model does not give for free, and the road to out of order

**7.7.1** A simple memory model does not make an atomic implementation simple by itself: cache coherence, exclusive ownership, invalidation, ordering and the interaction with interrupts still have to be designed and tested. *(D)*

**7.7.2** Out-of-order execution is not ruled out: the first implementations are in order, and the architecture is kept ready for a large out-of-order design. *(D)*

**7.7.3** Sequential consistency on a speculative core needs loads that are replayed when another core intervenes: the core executes a load early and, before the load retires, checks whether another core has written its line since, and if so executes it again. The readiness review calls this awkward but defined, and names the flag rules, the reservation rules and the ISYNC and HINT instructions as what a speculative core needs. *(D)*

The tests are part of the work. A declaration of sequential consistency is easy to violate by accident, in a store buffer that forwards wrongly or a cache that reorders its fills, so the standard litmus suites (the `litmus7` and `herd7` families) are to run against two cores before any lock-based software is trusted (repo, `docs/pdp64-fpga-plan.md` item 7; open as part of J.9.11).

*Commentary.* **Chain.** One instruction set often spans small in-order and large out-of-order cores: the x86 Pentium and P6, Alpha's 21064 and 21264, ARM's small and large cores, RISC-V's Rocket and BOOM (all *unverified*). The out-of-order designs that kept SC did it with speculative loads and a check (MIPS R10000, *unverified*), the technique 7.7.3 names. **Trade-offs.** What the pdp64 gains by refusing speculation is a smaller design with fewer error cases and none of the speculative side channels; what it gives up is the performance of an aggressive core, which an FPGA or a small chip could not build anyway. The readiness document (`pdp64-ooo-readiness.md`) and the hardware-vulnerabilities report give the rules a future core would follow. **Impact.** *Kernel:* none. *Compiler:* none. *Hardware:* a speculative core needs loads that replay when another core intervenes, and the flag and reservation rules keep that path open.

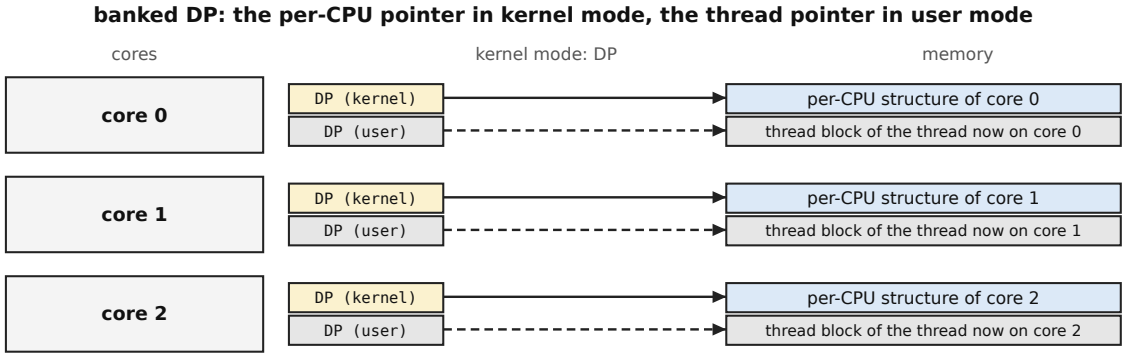
## 7.8 Per-CPU state and cheap context

**What you know.** V7 reaches the data of the running process as the global `u`: the memory manager maps the process's user structure at one fixed kernel address (`_u = 140000` in `conf/mch_i.s`, octal), and `runrun`, `curpri` and the run queue are ordinary globals, because there is one processor. With two cores, `u` must mean a different structure on each. Every SMP kernel therefore needs a **per-CPU pointer**: from any code, find the structure that belongs to this core. The difficulty is that on entry from user mode the kernel has no free register and no stack yet, so something must be reachable without either.

**Each architecture's answer.** x86 swaps the `GS` base register with a saved kernel value on entry (`swapgs`, 65 uses in the four kernels of the survey); AArch64 holds the pointer in `tpidr_el1` and the current task in `sp_el0`; RISC-V holds it in the supervisor scratch register `sscratch` and exchanges it with the thread pointer in one `csrrw`, and OpenBSD's riscv64 port keeps `curcpu()` in a register, `register struct cpu_info *__curcpu asm ("tp")` (`cpu.h`, read), with `cpu_number()` as `curcpu()->ci_cpuid`; MIPS reserves two registers, `k0` and `k1`, for the kernel in the ABI; the VAX and the 68000 push everything and need no scratch (the numbers are from `docs/kernel-asm-survey.md`; the OpenBSD and Linux lines were read).

**The pdp64's answer.** `DP`, the direct-page register of chapter 6 (6.2), has a copy for each mode. In kernel mode it is the per-CPU pointer: a field of this core's structure is one `MOVQ d(DP), Rd`, a six-byte load with a 28-bit displacement (9.3.5). In user mode the same register is the thread pointer, so thread-local data costs the same, and the two copies do not interfere. The kernel sets its copy with `MTDP` when it starts the core. A store through `d(DP)` is not in the grammar yet (J.9.5); a store takes the pointer in a register.

```
cpu_number():   MOVQ ci_cpuid(DP), Rd       # a field of this core's structure; MFCPUID is read once, at
start
curcpu():       MFDP Rd                     # kernel mode: DP is the address of this core's struct cpu_info
```




a field of this core's structure: MOVQ d(DP), Rd, a six-byte load; no search, no swapgs, no reserved register
an interrupt that arrives in user mode finds this core's kernel R14 and DP already in place (7.8.4, 6.2)
MFCPUID gives the core number, which no program can compute (7.8.1)

**7.8.1** `MFCPUID` returns the number of the core that executes it. It cannot be synthesised and per-CPU data needs it. *(D)*

**7.8.2** `DP`, the direct-page register, has a copy per mode; in kernel mode it is the per-CPU pointer and a field of the per-CPU structure is one six-byte instruction, `MOVQ d(DP), Rd`. *(D)*

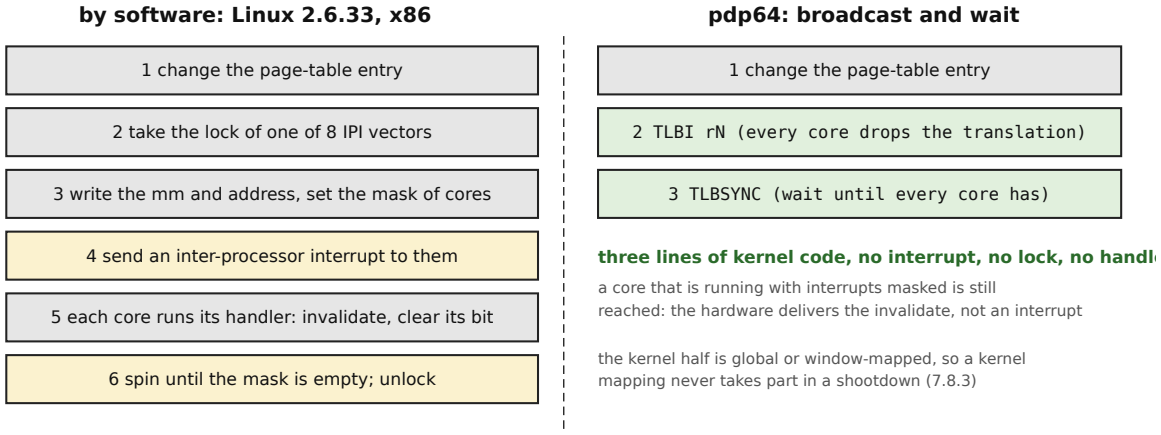
**7.8.3** The address-space tag makes a context switch constant-time, and global page-table entries and the direct-mapping windows keep the kernel's own mappings out of the invalidation path. *(D)*

**7.8.4** The stack pointer is banked by mode, so an interrupt arriving in user mode on any core finds that core's kernel stack without a search. *(D)*

*Commentary.* **Chain.** PDP-11: globals and one fixed `u`; nothing to find. VAX: the same, with the process pointer in a processor register (`PR_SSP`, as `lock_stubs.S` of the NetBSD VAX port reads it to get `curlwp`, read). x86, ARM, RISC-V, MIPS: a hand-built per-CPU base, a special register or a reserved one, and a rule about who may touch it when. pdp64: `DP`. **Improvements.** `DP` serves code density, and is also what both an SMP kernel and a threaded C runtime need most, in the mode where each needs it. **Benefits.** No `swapgs`, no reserved scratch register, no code spent finding the CPU; the context switch of chapter 6 (6.11) writes two registers. **Impact.** *Kernel:* per-CPU data is one six-byte `MOVQ d(DP), Rd`. *Compiler:* the compiler addresses per-CPU variables relative to `DP`. *Hardware:* three copies of `DP` and of the stack pointer, one per mode.

## 7.9 Translation and instruction-cache coherence

**Translation buffers.** A core keeps a **translation buffer** (TLB: a small memory that holds recent virtual-to-physical translations, chapter 6, 6.8), one for each core. When the kernel changes or removes a page-table entry, the old translation may be in any core's buffer. On the PDP-11 there was none: the page registers *were* the translation. On a multiprocessor the other cores must be told. The older method is the **shootdown**: the core that changed the entry sends an inter-processor interrupt to the others, each runs a handler that invalidates its entry, and the sender waits for all of them. Linux 2.6.33 on x86 does exactly that (`arch/x86/mm/tlb.c`, read): eight interrupt vectors, one lock for each, a mask of the cores to reach, an interrupt to them, and a loop `while (!cpumask_empty(...)) cpu_relax();` until the last has cleared its bit. FreeBSD 5.3 sends `IPI_INVLTLB` and NetBSD's VAX port sends `IPI_TBIA` (both read). ARM made the invalidate a broadcast instruction (`tlbi vae1is`, the inner-shareable form, in `arch/arm64/include/asm/tlbflush.h`, read); RISC-V's `sfence.vma` acts on the executing core (*unverified* as architecture), and Linux either sends an interrupt or asks the firmware for a remote fence (`arch/riscv/mm/tlbflush.c`, read); x86 got a broadcast instruction late, `INVLPGB`, which the current Linux uses (`arch/x86/mm/tlb.c`, read; its vendor and date are *unverified*).




**The pdp64's way.** `TLBI` and `TLBIA` (chapter 6, 6.8) are broadcast by the hardware, and `TLBSYNC` waits until every core has finished, so the kernel knows no core holds the old translation. A broadcast that cannot be waited for is no use, which is why the completion instruction exists. The kernel's own mappings are global or window-mapped (6.9) and are never part of a shootdown.

**Instruction caches.** The data cache is coherent between cores (7.6). The **instruction cache** (the cache that the instruction fetch reads) is not coherent with the data cache. A program that writes instructions as data (a loader, a **JIT** compiler that generates code as it runs, a trampoline, a Lisp runtime compiling a function) must make the instruction fetch see them. x86 keeps its instruction cache coherent (*unverified*). ARM requires cache maintenance and an `isb` (`arch/arm64/mm/cache.S`, read) and RISC-V a `fence.i`, which across cores Linux does by interrupt or firmware call (`arch/riscv/mm/cacheflush.c`, read). The pdp64 follows ARM: coherent instruction caches would cost a snoop port for a case that almost never happens.

**7.9.1** `TLBI` and `TLBIA` broadcast to all cores; `TLBSYNC` waits until the broadcast has completed everywhere. *(D)*

**7.9.2** Instruction caches are not coherent with data caches; `ICINV` invalidates a line and `ICINVA` all of it, and neither is privileged, because a JIT needs them. *(D)*

**7.9.3** A write to the page-table base, the address-space tag, the translation-enable bit or the vector base takes effect only after the next `ISYNC`; `ISYNC` makes earlier system-register writes and earlier `ICINV` take effect for what follows. *(D)*

*Commentary.* **Chain.** PDP-11: no buffer, no shootdown. VAX: a software interrupt to the other processors (NetBSD's `IPI_TBIA`). x86: an interrupt and a software handshake; AMD's broadcast instruction late. ARM: a broadcast instruction and a barrier. RISC-V: software, by interrupt or firmware. pdp64: a broadcast instruction and a completion wait. **Trade-offs.** A broadcast is simpler in hardware and far faster for two to four cores; it needs the completion barrier. Making instruction caches coherent would cost a snoop port. **Improvements.** The kernel's own mappings never take part in a shootdown. **Benefits.** The pain-points report finds that remote translation-buffer shootdown, a source of deadlocks in the BSDs and of much code in Linux, does not arise (repo, `docs/kernel-pain-points.md`). **Impact.** *Kernel:* a shootdown is `TLBI` then `TLBSYNC` (the listing is in 6.8); a module or JIT write is `ICINV` then `ISYNC`. *Compiler:* a JIT or a trampoline generator must call `ICINV` after writing code. *Hardware:* a broadcast on the interconnect and a counter for completion; the instruction cache has an invalidate port but no snoop port.

## 7.10 Waiting, signalling and interrupting another core

**The PDP-11's wait and its gap.** 7.1 showed `idle()`: lower the priority, `wait`, with the gap in which a wake-up can be lost and the clock tick that repairs it. A core that takes no clock tick has no repair, and a core that spins instead of waiting, as 4.3BSD's VAX idle does, uses memory bandwidth that other cores need. On an FPGA, where a spinning core competes for the memory the others use, that is the reason to prefer a real wait.

**The precedents.** x86's `PAUSE` tells a spinning core to back off, and `MONITOR`/`MWAIT` waits on a cache line (`arch/x86/include/asm/mwait.h`, `vdso/processor.h`, read). ARM's `WFE` (wait for event) stops the core until another core executes `SEV` (send event), an interrupt arrives, or the monitor of a load-exclusive is cleared; Linux's `__cmpwait` is `sevl; wfe; ldxr; eor; cbnz; wfe`, a loop that sleeps until a store by another core changes the word (`arch/arm64/include/asm/cmpxchg.h`, read; the architectural wake conditions are *unverified*). RISC-V's `wfi` ends when an interrupt is pending and enabled in its own register; OpenBSD's riscv64 idle is `intr_enable(); wfi`, with a comment that asks whether a barrier is needed (`XXX Data Sync Barrier?`, read), and the same gap as V7's.

**The pdp64's way.** `WFE` is both the multiprocessor wait and the uniprocessor idle (`WAIT` is its assembler alias, 7.10.3). The gap closes by one rule, which the author decided (2026-10-22, "Yes, wake on pending"): `WFE` ends its sleep for an interrupt that is pending even if the priority masks it. The idle loop is then correct with no tick (this is the loop of 6.11):

```
idle():
    loop:
        SPL #7                      # mask everything while looking
        if run_queue is not empty:  SPL #0 ; return
        WFE                         # an interrupt that arrives after the test ends the sleep at once
        SPL #0                      # now take the interrupt that woke us
```

**Interrupting another core.** A core wakes another by `SEV`, or interrupts it: an **inter-processor interrupt** (IPI), an interrupt raised by one core on another. It reschedules a core that is running a low-priority process, stops a core, or runs a function on it; a core in secure mode is stopped and preempted by one (7.11). x86 writes to the other core's local APIC (`CALL_FUNCTION_VECTOR`, read), the NetBSD VAX port writes a processor register (`PR_IPIR`, read), and OpenBSD's riscv64 port needs two kinds of IPI, `IPI_NOP` and `IPI_DDB` (`intr.h`, read).

**7.10.1** `WFE` sleeps until another core executes `SEV` or an interrupt arrives, and `HINT #1` is the spin-wait pause; `HINT #n` has no other architectural effect (2 is a speculation barrier, the rest are reserved). *(D)*

**7.10.2** `WFE` ends its sleep when an interrupt is pending, even if the priority masks it, so that an idle loop is `SPL #7`, check for work, `WFE`, `SPL #0`, with no gap in which a wake-up can be lost. *(D)* Details pending (J.7.1).

**7.10.3** On one core `WFE` is the idle instruction, and `WAIT` is its assembler alias. *(D)*

**7.10.4** One core can interrupt another: an inter-processor interrupt. A core in secure mode is stopped and preempted by one (7.11.3). *(D)* Details pending (J.6.3).

**7.10.5** Open: how a device interrupt and an inter-processor interrupt reach a core (J.6.3).

*Commentary.* **Chain.** PDP-11: `wait` at priority 0 in a loop that retests the run queue, a lost wake-up costing one tick. VAX: an idle loop that spins; in the NetBSD port a message bit and a processor-register write. x86: `PAUSE`, `MWAIT` and an APIC write.

ARM: `WFE`/`SEV`. RISC-V: `wfi`. pdp64: `WFE` with the wake on pending. **Trade-offs.** On an FPGA a spinning core consumes memory bandwidth the other cores want, which is the reason to prefer a real wait; a rule that wakes for a masked interrupt costs the core one extra instruction after the wake. **Benefits.** An idle core costs nothing and wakes on a signal. **Modern comparables.** ARM `WFE` and `SEV`, RISC-V `WFI`. **Impact.** *Kernel:* an idle core waits on `WFE` and costs no bandwidth. *Compiler:* spin-wait loops use `HINT #1`. *Hardware:* `WFE` stalls or gates the core's clock until an event or an interrupt.

## 7.11 Cores that differ: classes and the secure mode

**What you know.** Every PDP-11 processor in a machine was alike, but some machines had a second, unlike processor: the VAX's console processor, which started the machine and ran the front panel, was a second computer (*unverified*). The Blit terminal had a processor of its own (Pike, Locanthi and Reiser, 1985; chapter 3). The CDC 6600's peripheral processors and the IBM channel processors were cores given to input and output (*unverified*).

**Classes.** The pdp64 allows cores that are not alike, on one condition: software must be able to tell which is which, and a core may differ in speed, never in what a program can observe. A read-only **identification register** in each core gives its class and capacity, and a **stop-and-drain** register lets a kernel take a core offline: the core finishes its outstanding bus transactions and stops. Cores of an in-order and an out-of-order design may share a die (the author, 2026-11-04, "its ok to have in order and out of order cores on the same die as long as we can tell which is which"); the memory model and the meaning of every instruction are the same on every core.

**The secure mode.** A **timing channel** is a leak of secret data through how long something takes: if a division stops early when the quotient is small, how long it ran tells an observer something about the operands. Cryptographic code that must not leak a key needs instructions whose time does not depend on the data. At least one core can run in **secure mode**, in which the hardware gives that guarantee. The author approved the design of such a core ("i like your design. approved.", errata 86; the document is `crypto-secure-core.md`) and the rule that no tick or timing interrupt is introduced where an instruction can be made constant-time without one. The same instruction set runs in secure mode, with promises about time, memory and isolation added. The status-word bit `DIT` (9.3.7) is the part of the promise that an ordinary core can keep: while it is set, instruction latency, the clock and any data-dependent prefetcher hold no data-dependent timing. Secure mode adds what a bit cannot give: a private memory window of fixed latency, a locked translation buffer, no speculation and a hardware clear between domains (7.11.4 and 7.11.5).

Whether every core has the mode or one core only was decided on 2026-11-10 (errata 132): "at least one core can be in 'secure' mode; if it doesn't take too many gates, let that be a mode for all the cores, but if it slows things down too much then return to a single secure core." The rule is in 7.11.3; the thresholds and the measurement are open (J.7.8).

```
core in secure mode, one domain at a time:
    run the process                     # every instruction takes a time set by the opcode and length only
    on an inter-processor interrupt from another core (7.10.4):
        stop or preempt                 # no periodic tick exists; the inter-processor interrupt is what
stops it
        handler touches no secure state; its duration does not depend on secure state
    switching to another domain:
        hardware clears the private window ; the kernel clears the registers
```

The dedicated cores of the author's recommendation of 2026-11-04 (errata 76: one core each for graphics, network and audio) are no longer part of the decided design: the author wrote on 2026-11-10 that the idea shrinks. What remains of them is open (J.7.5; the earlier statements are in Appendix H, 7.11).

**7.11.1** Each core has a read-only identification register giving its class and capacity, defined from the first system although every core is alike, and a stop-and-drain register that a kernel writes to take a core offline: the core finishes its outstanding bus transactions and stops. *(D)*

**7.11.2** In-order and out-of-order cores may share a die if software can tell which is which, which the class register of 7.11.1 reports; the memory model and the semantics of every instruction are identical on every core, and a core class may differ in speed and never in what a program can observe. *(D)*

**7.11.3** At least one core can run in secure mode. While a core runs in that mode every timing channel is removed (every instruction and every cache fetch takes a time independent of its data), it has private cache lines and takes no periodic timer tick, and it is stopped and preempted by an inter-processor interrupt. A core that has the mode has constant-time AES and SHA instructions (9.9.7), or whichever opcodes would speed up and enable AES and SHA. If the mode costs few gates it

is a mode of every core; if it slows a core too much the machine has one secure core only. *(D)* Details to be determined: the instruction words (J.9.12) and which ordinary instructions are constant-time (J.9.2). Details pending (J.7.8): what counts as few gates and as too much slowdown, how each is measured, and the threshold, which the author sets.

**7.11.4** The memory of a core in secure mode has no hit or miss: it is a private window of memory with one fixed access time, mapped by a direct-mapping window, and a load or store outside the window aborts; its translation buffer is locked, so a miss aborts and no page walk occurs; in secure mode it has no demand paging, no simultaneous multithreading, no speculation and no data-dependent prefetch, and its clock is held fixed while it runs a secure domain. *(D)* Details to be determined: the size of the window, the latencies and the field numbers of the class register.

**7.11.5** Before another domain runs on a core in secure mode the hardware clears the window and the kernel clears the registers; transfers between the window and main memory take a time that depends only on their length, which the interconnect secures with a reserved slot. *(D)* Details to be determined: the slot period and the clear engine.

*Commentary.* **Chain.** PDP-11: processors alike; the Unibus devices and later the VAX console processor carried the unlike work. VAX: a console processor that starts the machine (*unverified*). ARM and RISC-V: big and little cores of one instruction set. x86: the same, from the Pentium and P6 pair on (*unverified*). Security: Arm's `DIT` bit asks the core to avoid data-dependent timing optimisations (repo, `docs/hardware-vulnerabilities.md`), x86 has AES instructions and OpenBSD uses them (`aesni.c`, read), RISC-V's scalar cryptography extensions `Zkne` and `Zknh` appear in Linux's hwprobe list (read). pdp64: a class register and a secure mode whose guarantee is in hardware, on every core or on one, by the rule of 7.11.3. Arm's TrustZone and Intel's SGX are the precedents for a mode of every core (Linux has an SGX driver, `arch/x86/kernel/cpu/sgx`, read; the rest is *unverified*), and Apple's Secure Enclave and the AMD platform security processor for a separate core (*unverified*; J.7.8). **Trade-offs.** A core in secure mode is a crypto engine rather than a general core: a process with a 100 MB secret-indexed working set does not fit, which is acceptable because the use is keys and cipher state, kilobytes. A mode of every core costs gates and perhaps clock speed in every core; one secure core costs a core that general work cannot use while a secure domain runs, which on a machine of four cores is a quarter of it. **Impact.** *Kernel:* one affinity rule where only some cores have the mode, a domain-switch routine, and interrupt handlers for a core in secure mode that obey the rules of 7.11.5. *Compiler:* secure code is written with intrinsics. *Hardware:* a class register, a stop-and-drain register, and for secure mode a fixed-latency memory, a clear engine and a bus slot.

## 7.12 A kernel's view

The idioms of 7.3 to 7.11, in the order a kernel meets them. Illustration, not architecture:

```
lock       :  CAS_DB(l, 0, 1) until it returns 0                    # 7.4
unmap page :  pte = 0 ; TLBI va ; TLBSYNC                           # 7.9, listing in 6.8
new code   :  write the instructions ; ICINV line ; ISYNC           # 7.9
idle       :  SPL #7 ; test ; WFE ; SPL #0                          # 7.10
wake a core:  SEV, or an inter-processor interrupt                  # 7.10
my data    :  MOVQ d(DP), Rd                                        # 7.8
```

**7.12.1** The kernel keeps its per-CPU structure at the address in `DP` and takes a lock with one `_DB` atomic. *(D)*

**7.12.2** It unmaps a page by changing the entry, issuing `TLBI`, and waiting with `TLBSYNC`. *(D)*

**7.12.3** It makes new instructions visible by writing them, issuing `ICINV` and executing `ISYNC`. *(D)*

**7.12.4** It idles a core with `WFE` and wakes it with `SEV`. *(D)*

*Commentary.* Each item is one instruction, and each is a place where other architectures need a trampoline, a handshake or a fence. The port's machine-dependent SMP work is estimated, from the line counts of OpenBSD's riscv64 port, at 8,000 to 9,000 lines (repo, `docs/smp-decisions.md` section 6); that is the size of the task, not a measurement of a pdp64 port.

## 7.13 What was avoided

**7.13.1** The architecture has no weak ordering, no speculative execution, no hardware transactional memory, no software-managed coherence between processors, no coherent instruction caches, no inter-processor-interrupt shootdown as the main mechanism, and no memory-model-dependent ABI. *(D)*

*Commentary.* **Trade-offs.** Intel's transactional memory (a way to run a group of loads and stores as one atomic step, rolling back if another core interferes) has a record of errata and side channels in the hardware-vulnerabilities report (repo: the TAA entry), for a feature few programs used; `CAS` and `LL`/`SC` suffice. Each avoided item is a tax on every concurrent program,

or on every implementation, in exchange for performance the small in-order core cannot use. **Impact.** *Kernel:* each avoided feature is code a kernel does not have to write: no barriers, no software coherence, no shootdown interrupts. *Compiler:* each is analysis the compiler does not have to do: no memory-model-dependent code generation. *Hardware:* each is a block not built: no speculation, no transactional memory, no coherent instruction cache.

## 7.14 What was checked, and what was not

- **Read in the Unix sources on this machine** (`facetlang/unixtree`, files gzipped; V7M under `V7M/sys`): `plock()` of `pipe.c` and the loop of `iget()` (`ILOCK`, `IWANT`); `sleep()` and `wakeup()` with `spl6()`; `brelse()` with `spl6()` and `splx()`; the end of `trap()` (`if (runrun) qswtch();`); `swtch()` and its call of `idle()`; `idle()` in `mch_i.s` (`bic $HIPRI,PS`, `wait`) and 2.11BSD's (`spl 0`, `wait`); `_u = 140000`; `HZ` 60. 4.3BSD: `vax/locore.s` (the `idle` loop, `bbss` and `bbcc`, `insque` and `remque`, `masterpaddr`), `vax/mtpr.h` (`IPL`, `SIRR`, `TBIA`), and the absence of `BBSSI` in `vax/`. Linux 0.96c (no `smp`, `spinlock` or `multiprocessor`); Linux 2.6.33.2: `lib/kernel_lock.c`, `arch/x86/mm/tlb.c` (the shootdown), `irq_vectors.h`, `spinlock.h` (`lock; xaddw`), `mpparse.c` (the MP specification), `smpboot.c`; FreeBSD 5.3 `mutex.h` and `i386/mp_machdep.c` (`IPI_INVLTLB`). 2.9BSD `usr/doc/setup`: the line on the 11/74.
- **Read in the kernel trees on this machine** (`kernel-src`): NetBSD `sys/arch/vax` (`multicpu.c`, `lock_stubs.S`, `include/lock.h`, `ka820.c`, `ka88.c`, `ka6400.c`, `pmap.c`, `mtpr.h`); Linux `arch/arm64` (`atomic_ll_sc.h`, `atomic_lse.h`, `cmpxchg.h`, `barrier.h`, `tlbflush.h`, `percpu.h`, `mm/cache.S`), `arch/arm64/include/uapi/asm/ptrace.h` (`PSR_DIT_BIT`), `arch/riscv` (`cmpxchg.h`, `mm/tlbflush.c`, `mm/cacheflush.c`, `csr.h`), `arch/x86` (`cpu/sgx`, `atomic.h`, `mwait.h`, `vdso/processor.h`, `irq_vectors.h`, `percpu.h`, `mm/tlb.c`), `Documentation/memory-barriers.txt` (3,016 lines, the Alpha section) and `Documentation/litmus-tests` (`DCL-broken.litmus`); OpenBSD `arch/riscv64` (`cpu.h`: `tp`, `MAXCPUS`; `machdep.c`: `cpu_idle_cycle`; `intr.h`: `IPI_NOP`, `IPI_DDB`) and `kern/kern_lock.c` (`kernel_lock`).
- **Read in the repository:** the statements of this chapter against `docs/pdp64-isa.md` section 17 and 4.11, `docs/errata.md` entries 39, 45, 76, 85, 86 and 92, `docs/crypto-secure-core.md` section 1, `docs/decisions-pending.md` (the VAX claim C028 in `docs/codegen-data/verification/claims.csv`), `docs/kernel-pain-points.md`, `docs/kernel-asm-survey.md`, `docs/smp-decisions.md`, `docs/pdp64-fpga-plan.md`; the grammar `spec/pdp64-mnemonics.hbnf` for the names of the atomic and system words, from which the table of 7.3 is counted (22 atomic words, 9 system words and the alias `WAIT`); `docs/pdp64-ooo-readiness.md` section 6.7 for the block operations (7.5.8).
- **Counted, not re-counted here:** the 5,596 lines in 855 files of ordering vocabulary in Linux and 743 lines of documentation (`docs/kernel-pain-points.md`); the 155% of the Anderson lock (its source, a mailing-list message, was not read here).
- **Unverified** (general knowledge; the handbooks and papers are not on this machine): that the 11/74 was a multiprocessor and the names of its instructions; the Unibus cycles `DATIP` and `DATO` as one locked pair; the cache of the 11/70; the VAX-11/782, 8300, 8800 and 6000 as multiprocessors and the VAXcluster as shared-disk; the history of master and slave and of the BSD/OS mutexes; Lamport's definition and date; the litmus results of the table in 7.5; the memory models of ARM, POWER and RISC-V; Alpha's year; the history of snooping and directory machines; the big and small cores named in 7.7 and 7.11; TrustZone, the Secure Enclave and the AMD platform security processor as precedents for 7.11 (J.7.8); the Blit and the CDC 6600 as cores for input and output; the architectural conditions that end a `WFE` and a `wfi`; the date of `INVLPGB`; the estimate of 500 lookup tables per snoop port, which is the ISA text's own paper estimate (the earlier text added about 600 per core with no source, and that figure is dropped).
- **Decided by the author and recorded in `errata.md`:** everything stated as a decision with a tag *(D)*; the pseudocode of this chapter is an illustration and not a decision.

# Chapter 8 — Vector work, AI and array languages: the device route

**The pdp64 has a vector and accelerator engine as a standard part of the architecture, as the FPU, the MMU and the IOMMU are: every system has one, software can rely on it, and the architecture specifies its descriptor and its operations. The engine is not in the processor. It is a device on the bus that uses the same RAM as the processor, moves data by DMA and is told what to do by descriptors (records in memory) that the processor writes; the processor itself has no vector register file and no SIMD instruction set, and gains only the memory ordering and the `fence` instruction that every device already needs.** Open: the operations and number formats of the engine (J.8.1), its descriptor (J.8.2), packed-lane operations in the integer registers (J.8.3), which engine is built first (J.8.4) and operations on regions of memory in the instruction set (J.8.5).

This chapter is written for a reader who has programmed a PDP-11 with the FP11 floating-point unit (the `movf` and `addf` instructions of V7 assembly), or who is learning computing from the start, and who has not met a vector register, a SIMD instruction or a graphics processor. It runs from what that reader knows to what the pdp64 does: the PDP-11 and its floating-point unit (8.1), what a vector unit and a SIMD unit are and the machines that had them (8.2), the decision (8.3), the engine as a standard part, the accelerator route and what "GPU" means in this manual (8.4), inference, array languages and the bit instructions (8.5), the number formats (8.6) and what was checked (8.7). Graphics has its own chapter (chapter 3), since its devices are the first a user sees; the descriptor rings of chapter 5 are the transport this chapter assumes.

*How to read this chapter.* A term is defined where it first appears, as the technical word followed by a plain restatement. Boxes-and-arrows pictures draw a box for a piece of memory or a program and an arrow for what is handed on. Code blocks are pseudocode (`#` starts a comment), or PDP-11 assembly in the syntax of the V7 assembler when the text says so. Every listing for the pdp64 is an illustration, and so is every record layout; the decisions are the numbered statements.

## 8.1 What you already know: the PDP-11 and its floating-point unit

The PDP-11 had one way to do arithmetic on real numbers: the **FP11**, a floating-point unit (a second processor-like block, an option that a model could lack) with six **accumulators** (floating-point registers) named `AC0` to `AC5`. Each holds a 64-bit value; a mode instruction, `setd` for 64-bit double and `setf` for 32-bit single, says how the instructions that follow read them. V7 Unix tests for the unit at boot (`mch.s` executes `setd` with a trap catcher installed, and sets a flag, `fpp`, if the instruction did not trap) and keeps the state of a process in its `struct user` (`user.h`):

```
struct {
    int     u_fpsr;          /* FP status register */
    double  u_fpregs[6];     /* FP registers: AC0..AC5 */
} u_fps;                     /* 2 + 6 * 8 = 50 bytes */
```

The kernel saves it with `savfp` and restores it with `restfp`, two assembly routines in `mch.s` (`stfps`, `setd`, six `movf` to memory; the reverse for restore; the code moves `AC4` and `AC5` through `AC0` before storing them, which suggests, by inference, that this store form reaches only `AC0` to `AC3`). `swtch()` in `slp.c` calls `savfp` for the process it is leaving, unless `u_fpsaved` says that has been done; `trap()` clears `u_fpsaved` on entry and calls `restfp` on the way out only if it was set. So a system call that never switches processes costs no floating-point save at all, and a switch costs fifty bytes.

A loop that adds two arrays of 64 doubles on this machine (written for this chapter, V7 assembler syntax, after `setd`):

```
        mov     $64., r0          / count
loop:   movf    (r1)+, fr0        / fr0 = a[i]; r1 steps by 8
        addf    (r2)+, fr0        / fr0 = fr0 + b[i]
        movf    fr0, (r3)+        / c[i] = fr0
        sob     r0, loop          / count down, repeat
```

Each pass loads, adds, stores and decrements: four instructions per number, and the instruction fetches take as long as the arithmetic. The FP11 is a **scalar** unit (it works on one number at a time), and the PDP-11 had no vector unit and no SIMD instructions. Programs that needed more than the FP11 gave went to an **attached array processor** (a separate box, with its own memory and its own program store, that the host fills with data and starts): the Floating Point Systems AP-120B (1976) listed PDP-11 hosts in its brochure *(unverified here: the repository's notes record the brochure, which is not on this computer)*. The host talked to it through a library of calls and a set of device registers. That is the oldest precedent for this chapter's route.

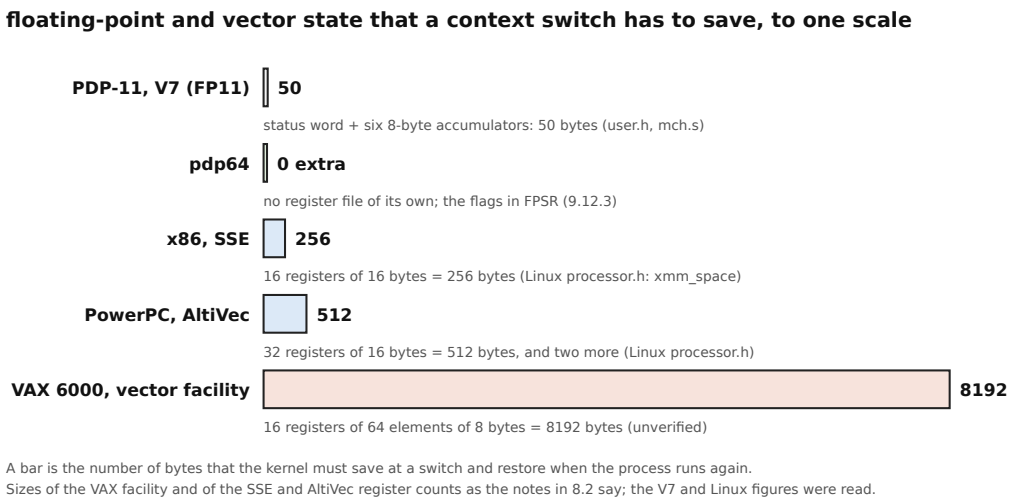
## 8.2 What came between: vector machines, SIMD and the graphics processor

Three ideas followed, and each answered a limit of the loop above.

**The vector machine.** The loop above spends its time on instruction fetch and loop control. A **vector machine** (a processor whose registers each hold a whole array of numbers, and whose instructions act on all of them) replaces the loop with four instructions:

```
vl = 64                  # the vector length register: how many elements to use
v0 = vload(a)            # one instruction loads 64 numbers
v1 = vload(b)
v2 = vadd(v0, v1)        # one instruction adds 64 pairs, one pair after another in a pipeline
vstore(c, v2)
```

The **Cray-1** (1976) had eight vector registers of 64 elements of 64 bits *(unverified)*. DEC's answer was the **vector facility** of the VAX 6000 series (1990): sixteen vector registers of 64 elements each, a vector-length register and a mask register, an option in a separate processor board *(unverified here: the repository's notes cite the VAX 6000 Vector Processor Programmer's Guide, not on this computer)*. A longer array is cut into pieces of 64 (**strip mining**: a loop that handles one register's worth at a time and a final short piece with a shorter length). The price is state: sixteen registers of 64 elements of 8 bytes is 8192 bytes that the operating system must save at a context switch, 164 times the 50 bytes of the FP11, drawn to one scale below.

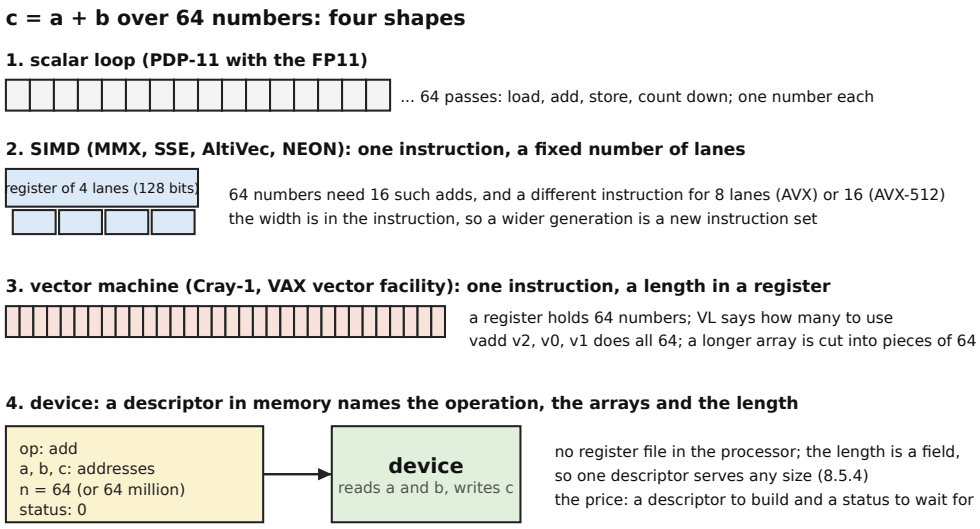



**SIMD, in the 1990s. SIMD** (single instruction, multiple data: one instruction applied to several numbers packed side by side in one register, each position a **lane**) took the idea down to the size of an ordinary register. Intel's MMX (1997) reused the floating-point registers as packed integers, SSE (1999) added sixteen-byte registers, and AltiVec on the PowerPC (1999) had 32 of them *(dates unverified)*. The Linux 2.6.33 headers on this machine show the cost: an x86 process carries 16 SSE registers of 16 bytes, `xmm_space[64]` of 32-bit words (`processor.h`), and a PowerPC process carries `vr[32]` of 16 bytes, a status register and a save mask. ARM's NEON registers are the floating-point file seen as 128-bit registers *(unverified)*; the Linux `vfp_hard_struct` holds 32 registers of 64 bits. A SIMD instruction does not take the vector length from a register; the width is in the instruction. When x86 went from 128-bit to 256-bit (AVX) and 512-bit (AVX-512) registers, each width brought new instructions, new state and new kernel code, and a binary built for one could not use the next width *(unverified for the dates and the widths)*. ARM's SVE and the RISC-V vector extension went back to the vector-machine idea, with a length that the hardware chooses and the program asks for *(unverified)*.

The state has a second cost, recorded in this repository's survey of kernel pain points: x86 kernels delayed saving and restoring the floating-point registers until a process used them again (**lazy** switching), and in 2018 that was found to leak the previous process's registers; Linux went back to saving eagerly (`docs/hardware-vulnerabilities.md`, the commit it cites calls the lazy design "rather messy"). On the PowerPC the Linux source says in a comment that on a multiprocessor "we don't do lazy VMX switching because it just gets too horrendously complex" (`arch/powerpc/kernel/vector.S`). V7's own trick, saving only when a switch happens, was an early form of the same laziness.

**The graphics processor.** A **GPU** (graphics processing unit) is a second processor on a board of its own, with many small arithmetic units that run one program on many data at once *(unverified)*. It does not execute instructions from the host's instruction stream. The driver writes **commands** (small packets: draw this, run that program on these arrays) into a **ring buffer** in memory (a circular array that the producer fills and the consumer drains), and tells the chip how far the ring has been filled. The Linux 2.6.33 Radeon driver (`radeon_ring.c`: `radeon_ring_lock`, `radeon_ring_write`) shows the pattern. This is the attached array processor of 8.1 grown up, with the difference that it reads and writes the host's RAM by DMA instead of having its own program store and memory to load.

**The chain.** The PDP-11 had a scalar floating-point unit and a library call to an attached box. The vector machines (the Cray-1, the VAX vector facility) put the array in registers of the processor and paid in state. SIMD put a fixed few lanes in the registers and paid in instruction sets that grew with each width. The GPU and the neural-network engine returned to the attached box and put it on the memory bus. The pdp64 takes the last of these (8.3, 8.4).




The descriptor drawn is an illustration; its format is open (8.4.7).

Machines 1 and 2 as described in 8.1 and 8.2: the V7 sources and the Linux headers were read, the rest is unverified.

## 8.3 The decision: no vector or SIMD unit in the processor

The author's instruction (register row R277): "just before the isa section, add a chapter about sdim/avx2 vector stuff, i agree we should go the mmap device arm style route for that. so it is about graphics and ai/llm stuff... also mention apl/k as making vector languages easier to support and more efficient." The decision it records (errata 59) is stated here and in the statements below.

**8.3.1** The processor has no vector register file and no SIMD instruction set of the SSE, AVX2 or NEON kind. *(D)*

**8.3.2** The reasons recorded with the decision: a vector register file is a second wide register file, which the instruction set deliberately avoids (floating point shares the general registers, 9.12.1), and it brings wide loads and stores entangled with the alignment rules of the memory system. *(D)*

**8.3.3** A fixed-width design (128-bit NEON, 256-bit AVX2) bakes its width into every instruction, so a wider generation needs new instructions and old binaries cannot use the new width. *(D)*

**8.3.4** Open: whether the integer registers get packed-lane operations, a handful of instructions that treat a 64-bit register as eight bytes or four halves (J.8.3).

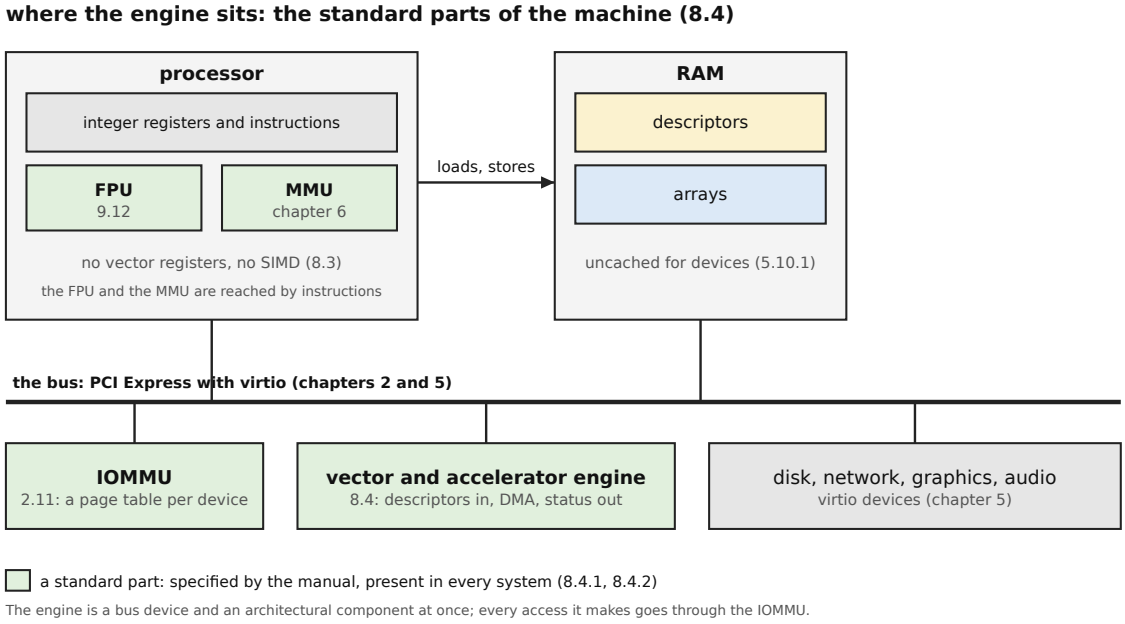
**8.3.5** Open: whether the instruction set gets operations on regions of memory, with no vector register named (J.8.5).

*Commentary.* **Chain.** PDP-11: a scalar floating-point unit with 50 bytes of state, and attached array processors for heavy work (8.1). VAX: a vector facility with 8192 bytes of state (8.2). x86, PowerPC, ARM: fixed-width SIMD in the processor, with state of 256 bytes to 512 bytes and a width that changed every few years; kernels that carried lazy-save code and then removed it. pdp64: none of this, because the floating-point formats use the general registers (9.12.1), the only floating-point state beyond them is the flag register FPSR (9.12.3), and the wide work leaves the processor. **Trade-offs.** A device needs a setup (a descriptor) and a completion (a status), so small arrays are faster on the processor and large ones on the device; a SIMD register file has no setup and a cost in state, opcodes and width that grows with every generation. **Improvements.** The decision removes the questions that a vector register file raises: whether a context switch saves the vector state, whether the kernel may use the registers, how wide they are, and what the calling convention does with them (the lazy-switching leak above does not arise for a state that does not exist). **Modern comparables.** ARM-based systems attach their neural-network engines (Ethos) and graphics engines (Mali) as memory-mapped devices *(unverified)*; Apple's matrix coprocessor is reached through a library and Google's TPU is a PCI Express card *(unverified)*. **Impact.** *Kernel:* no vector state to save at a context switch and no rule about the kernel using vector registers. *Compiler:* no auto-vectoriser (a compiler pass that turns a loop into SIMD instructions) and no vector types in the ABI. *Hardware:* no second register file, no wide load and store path, no opcodes spent; the processor's size does not grow with the workload.

## 8.4 The engine: a standard part of the architecture, and the device route

What the processor does not do, a device does, and the author has said that the device is part of the architecture. His words, 2026-11-10 (errata 131): "even if it doesn't involve specific opcodes 'in the cpu' still it must be built in as a standard part of the architecture, just as the fpu mmu and iommu are."

The PDP-11 treated its floating-point unit as an option: a model could lack the FP11, and V7 tested for it at boot (8.1). The pdp64 has no such options among the units the author names. The floating-point instructions are in the instruction set (9.12), the MMU is the page-table machinery of chapter 6, and the IOMMU is the translator between a device's addresses and memory's (2.11). The vector and accelerator engine joins them. It is reached in a different way, because the decision of errata 59 stands: the processor has no vector registers and no SIMD instructions, so the engine is not another group of instructions. The FPU is reached by instructions, the MMU by system registers and by the translation of every access, and the IOMMU by a register block on the bus; the engine is reached by descriptors. What makes it a standard part is that the manual specifies its interface the way it specifies FADD.




**8.4.1** The vector and accelerator engine is a standard part of the architecture, as the FPU, the MMU and the IOMMU are. *(D)* Details pending (J.8.1 to J.8.4): its operations and number formats, its descriptor, and which engine is built first.

**8.4.2** The word "standard" has five consequences: every pdp64 system has the engine, including the smallest; software may rely on its presence; the kernel and a program can find out that it is present and what it implements; the architecture specifies its descriptor and its operation set, so that a program written for one engine runs on any other; and it is not an optional PCI Express card. *(D)* These five are the reading of the author's comparison with the FPU, the MMU and the IOMMU (inference). How software finds the engine is pending (J.8.1).

**8.4.3** The engine is a device on the bus and an architectural component at once: the processor reaches it as it reaches every device (a memory-mapped interface, descriptors, DMA, FENCE), and the manual defines what it does. *(D)*

The definition behind the device route is the author's, 2026-10-20 (register row R136, thread 533 message 51): "when i say gpu i just mean a helper/accelerator that uses the same ram, does dma, etc, and can do the same or similar helps that a gpu would. and the simd/avax stuff."

**8.4.4** Wide work is described to the engine by descriptors in memory: an operation, the addresses of the arrays and their length. *(D)*

**8.4.5** Such a device is a helper or accelerator that uses the same RAM as the processor and does DMA; this is what "GPU" means in this manual. *(D)*

**8.4.6** The processor owes such a device what it owes every device: buffers in uncached memory (5.10.1) and a FENCE; the instruction set is unchanged. *(D)*

**8.4.7** Open: how the descriptor of the engine relates to the virtio descriptor of chapter 5 (J.8.2).

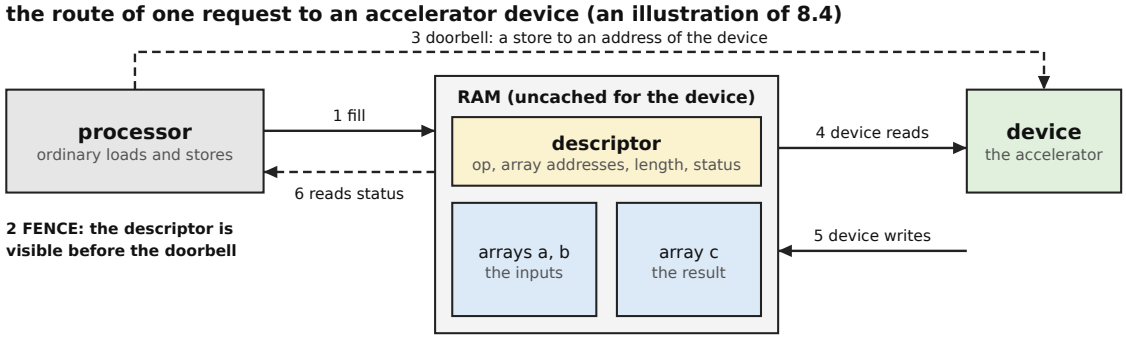
**8.4.8** Open: which engine is built first (J.8.4).

What this looks like to a driver (a pseudocode illustration: the field names and the layout are invented here, and the real record is the open item of 8.4.7):

```
struct vwork {                      # one request to an accelerator
    u32 op;                         # ADD, MUL, DOT, SUM, ...
    u32 format;                     # how to read the elements: binary64, int32, ...
    u64 a, b, c;                    # addresses of the two inputs and the result
    u64 n;                          # the length, in elements
    u32 status;                     # the device writes this when it is done
}

work.op = ADD;  work.a = &x;  work.b = &y;  work.c = &z;  work.n = 1000000
work.status = 0
```

```
fence()                              # the record is visible in memory ...
mmio_write64(doorbell, &work)        # ... before the device is told to look
while (work.status == 0) ;           # or wait for the interrupt (5.9)
```




PDP-11: the same shape as the RK11 with its registers replaced by a record in memory (5.1): the program fills it, starts the device, and learns by the RDY bit or an interrupt.
pdp64: the record is in memory and can chain; the device reads it and the arrays by DMA (direct memory access: the device moves data without the processor) and writes the result and a status.
The processor owes the device two things: memory the device sees in order, and FENCE (8.4.6).
The instruction set is unchanged. Whether the record is a virtio descriptor, or one that points at it, is open (8.4.7).

Sources: V7 rk.c (read), chapter 5 (5.4, 5.10), chapter 2 (2.5).

This is the shape of the RK11 driver of 8.1 and chapter 5, with the registers (RKDA, RKBA, RKWC) replaced by a record in memory, and it is the shape of a virtio request: the device reads descriptors from memory, reads or writes the buffers they name, writes a status and (optionally) interrupts. A device that speaks virtio takes the same transport routine as a disk (5.4), so what differs between a disk and an adder of arrays is what the operation field means. Whether the engine's own work record is a virtio descriptor, or something a virtio descriptor points to, is the open item of 8.4.7.

A device that does DMA writes memory without the processor's knowledge, which is why the processor owes it ordering: the descriptor must be visible before the doorbell (FENCE), and the buffers are in memory that is not cached in the processor's way (5.10.1), as for every device. A device in a user program's hands is confined by the IOMMU (2.11): the descriptor holds device addresses, and the IOMMU refuses any address that the device's table does not map, so a user-space driver can direct its engine only at its own buffers. (Without translation, which is not the default, the device would write anywhere, as 2.11 says.)

*Commentary.* **Chain.** PDP-11: heavy numerical work went to an attached array processor through library calls (8.1); the interface was a few device registers. VAX: the same for most customers, with the vector facility for the 6000 series. GPUs and neural-network engines: a ring of commands in memory and a doorbell, with data by DMA (8.2). pdp64: the same, with the ring and the transport shared with the standard devices of chapter 5, and with the engine specified by the manual and present in every system. **Reconciliation with the earlier decisions.** Errata 59 chose the memory-mapped device route and no vector register file in the processor; that stands, and the engine of 8.4.1 is that device made standard. Errata 78 and the author's remark of 2026-10-23 (R278, "if we can plug a regular card into the bus, why not") allow any PCI Express card to be attached and supported; that stands too, and such a card is an addition to the machine and not the engine, which every system has. Errata 95 made every device a virtio device, which is why the descriptor of the engine is measured against the virtio descriptor (J.8.2). The FP11 and the VAX vector facility were options, and each SIMD generation of x86 and ARM was a new part of the processor; the nearest comparison for the engine is the IOMMU, which is a bus unit and not an instruction group and is standard all the same. **Trade-offs.** The slow part is the hand-off: the buffers are uncached, so reading many small results back is slow, and a result that the next operation needs should stay where the device left it. **Impact.** *Kernel:* a device is one more slot, mapped like any other; the driver is the driver skeleton of chapter 5. *Compiler:* none; a library or a language runtime calls the device at the level of an operation on an array. *Hardware:* one ring engine shared with every device.

## 8.5 Inference, array languages and the bit instructions

Three workloads use the route, and one set of instructions stays in the processor.

**Inference.** A neural network's **inference** (running a trained model to produce an answer, as opposed to training it) is mostly multiplications and additions over large arrays of weights, which is the work vector machines were built for. The author, 2026-10-20 (R139): "just optimize inference whether in cpu or gpu whatever makes most sense."

**8.5.1** Inference is the workload that the machine is optimised for; whether a given part of it runs in the processor or on an accelerator is whichever makes most sense. *(D)*

**The bit instructions.** The weights of some small models are one bit each (the sign of the weight), and the multiply of a weight and an activation of one bit is an exclusive-nor followed by a count of the bits set. `POPCNT` (population count: the number of one bits in a register) and `XNORPOP` (population count of the exclusive-nor of two registers) do that for 64 weights at a time, and serve bitsets and Hamming distance (the number of positions at which two bit strings differ) as well. They are bit instructions of the ordinary integer unit (9.9), not vector instructions: they use the general registers and add no state.

**8.5.2** The processor has `POPCNT` and `XNORPOP` (9.9.1, 9.9.3) for the binary operations that small models and bitsets use. *(D)*

These instructions do not serve every model with binary weights: a language model whose weights are one bit but whose activations are 16-bit floating-point numbers needs a conditional add, not a count, and its inner loop is limited by memory bandwidth, which is what an accelerator is for (the note on `XNORPOP` in 9.9).

**Array languages.** In **APL**, created by Kenneth Iverson, and its descendants J and K (the language of Arthur Whitney's kdb+ database), a program is written as operations on whole arrays: add two arrays, multiply them, compare, reduce (sum), scan (running sum), index, select where a condition holds. An interpreter spends its time in a few tight loops over contiguous data. There is no loop for a compiler to find and turn into vector instructions: the programmer has said "add these two arrays", so the parallelism is in the source. DEC offered APL for the PDP-11 *(unverified)*; the Cray machines were built for the same shape of loop *(unverified)*. That is why the author asked for the mention: a device that takes a descriptor with an operation, the addresses of the arrays and a length is the natural implementation of the primitives, and a small interpreter with a library of primitives that map onto descriptors is "easier to support and more efficient" than a conventional compiler aimed at SIMD (his words, R277).

**8.5.3** The primitives of an array language map one to one onto descriptors: one descriptor per primitive. *(D)*

**8.5.4** There is no vector length in a register: the length is a field of the descriptor, so one implementation serves any array size. *(D)*

**8.5.5** Open: the operations of the engine, and its number formats (J.8.1).

*Commentary.* **Chain.** PDP-11: scalar loops in assembly, and APL interpreters in software *(unverified)*. Cray and VAX vector machines: the loop of 8.2, run by a vector register. x86 with AVX: K and kdb+ interpreters use SIMD instructions heavily *(unverified)*. pdp64: the interpreter pays its overhead once per primitive and not once per element, and the device streams the data at the speed of memory. **Trade-offs.** A device has a fixed cost for each descriptor, so an interpreter keeps short arrays on the processor. **Impact.** *Kernel:* a runtime that drives a device from user space needs the device's page mapped into its address space (confined by the IOMMU, 8.4). *Compiler:* none; the interpreter lowers primitives to descriptors, and a conventional compiler never sees a vector.

## 8.6 Number formats

The FP11 had two formats, single and double, chosen by a mode bit. The pdp64's formats are chosen by the instruction (9.12.5), and there are three.

**8.6.1** The processor handles `binary64`, `binary32` and `binary16` (the IEEE 754 formats of 64, 32 and 16 bits) as scalar floating point. *(D)*

**8.6.2** Open: the number formats of the engine, and how a program learns them (J.8.1).

*Commentary.* **Chain.** PDP-11: single and double, in DEC's own formats and not IEEE's *(unverified)*. VAX: F, D, G and H formats (9.12). pdp64: the IEEE formats, three of them in the processor. **Trade-offs.** A format fixed in the processor's instruction set cannot change when the field's formats change; machine-learning models have used several (bfloat16, 8-bit integers, 8-bit floating point) since 2017 *(unverified)*, and an engine whose formats are fields of its descriptor and entries of a feature list can follow the field where an instruction set cannot (J.8.1).

## 8.7 What was checked, and what was not

- **V7** (`unixtree/V7/usr/sys`): `h/user.h` (`u_fps`, `u_fpsaved`), `conf/mch.s` (`savfp`, `restfp`, the `fpp` test), `sys/slp.c` (`swtch`), `sys/trap.c`, `sys/sig.c`, `sys/sys1.c`; the loop in 8.1 is written for this chapter and was not assembled.
- **Linux 2.6.33.2** (`unixtree/Linux-2.6.33.2`): `arch/x86/include/asm/processor.h` (`xmm_space`, `st_space`), `arch/powerpc/include/asm/processor.h` and `kernel/vector.S`, `arch/arm/include/asm/fpstate.h`, `drivers/gpu/drm/radeon/radeon_ring.c`.
- **The repository:** `docs/errata.md` (59), `docs/decision-register.md` (R136, R139, R277), `docs/hardware-vulnerabilities.md` (lazy floating-point restore), `docs/decisions-pending.md` (the notes on the VAX 6000 and the AP-120B).
- **Sources read for the open items of the engine** (appendix J, J.8.1 to J.8.5): the Linux 7.3 sources (`Documentation/accel/`, `Documentation/arch/x86/xstate.rst`, `Documentation/arch/riscv/vector.rst`, `Documentation/arch/arm64/sve.rst`), the Linux user-space headers (`linux/virtio_ring.h`, `virtio_blk.h`, `virtio_gpu.h`, `virtio_net.h`, `virtio_ids.h`, `io_uring.h`, `idxd.h`), QEMU 11.1.2 (`target/riscv/insn32.decode`, `target/arm/tcg/sve.decode` and `sme.decode`, `target/hppa/insns.decode`, `target/alpha/translate.c`, `target/sparc/vis_helper.c`, `hw/arm/smmuv3-internal.h`, `hw/net/e1000_regs.h`, `include/block/nvme.h`) and the LLVM target descriptions (`llvm/lib/Target/X86`, `AArch64`, `RISCV`, `Hexagon`, `ARM`, `llvm/include/llvm/IR/IntrinsicsNVVM.td`, `mlir/include/mlir/Dialect/Tosa`) and 4.3BSD `sys/vaxif/if_dereg.h`. The counts of instructions were made from these files and are stated with their units in J.8.5.
- **Unverified** (general knowledge, or cited by the repository's notes from documents not on this computer): the AP-120B brochure; the Cray-1 and VAX vector-facility register counts; the dates and widths of MMX, SSE, AltiVec, AVX, SVE and RISC-V V; ARM Ethos, Apple's matrix coprocessor and the TPU; DEC's APL for the PDP-11; the graphics processor's design; the use of SIMD by K and kdb+; the date of the machine-learning formats; and every machine, format and number marked unverified in J.8.1 to J.8.5.

# Chapter 9 — The instruction set

**The instruction set is built around one idea: the first parcel of an instruction says both what the instruction is and how long it is, so the commonest instructions are two bytes and nothing is longer than six, one literal excepted.** The chapter covers how the opcode space is divided into prefix-coded leaves, the properties and principles that hold throughout, the registers, the format and the addressing modes, each group of instructions, faults, restart and memory management, position-independent code, what is rejected, the idioms of the PDP-11 and VAX that are restored, lost or replaced and the idioms of the compiler, the comparison with the PDP-11 and VAX, what is taken from LoongArch, and what is open. Every instruction has its entry (syntax, bit-layout chart, operation, flags, exceptions) in the section of its group below; the atomic instructions are in section 9.13 and the system instructions in section 9.14; the reference tables hold the aliases, the constants, the lists of mnemonics and the chart of every leaf. The grammars the tools are generated from are described in `../../spec/README.md`; every number here comes from them or from the measurements cited.

## 9.1 How the opcode space is divided

**9.1.1** Every instruction begins with a 16-bit parcel (the unit of instruction encoding), and the first parcel alone fixes what the instruction is and how long it is: its top 3 to 16 bits are a prefix, and the prefix selects a leaf. *(D)*

**9.1.2** A leaf is a group of mnemonics that share one form. A form lists the operand shapes (how each operand is written in assembly, and how many bits each part of it has); the leaf fixes where every field lies. After the prefix comes the sub-opcode, which selects one mnemonic of the leaf, and then the pieces of the operands: register fields from bit 0 of the first parcel upward, the last register operand lowest, displacements and immediates after them, the widest field last. No bit is padding. The encoding has 207 forms in 268 leaves.

**9.1.3** The prefixes form a prefix code (a set of bit strings none of which begins another, so that the first parcel matches at most one leaf and the match needs no look-ahead). The lengths were chosen by an optimiser that was given the compiled corpus, so the shortest prefixes belong to the forms the compiler emits most. *(D)*

**9.1.4** A leaf is one, two or three parcels long, which is two, four or six bytes; 36, 166 and 65 of the leaves are of those lengths. One leaf, `MOVQL`, is five parcels (ten bytes) because it carries a 64-bit literal. *(D)*

**9.1.5** 3,278 of the 65,536 patterns of a first parcel (5.0%) match no prefix and are illegal instructions; so is a sub-opcode that no mnemonic uses and a hot-register index beyond the table (9.4.6), so that a wild jump fails early and nothing is undefined.

**9.1.6** The assembly language and the mnemonics of the base specification are kept. The leaves add 54 words to them: the four fused pairs (`MOVCALL`, `POPRET`, `POPMRET`, `MOV2`), `MASKEQZ` and `MASKNEZ`, `BSEL`, `MOVQL`, the checked arithmetic `ADDV`, `SUBV` and `MULV`, the fourteen `TRAPCC` words, the deferred-fault and user-qualified moves (`.D` and `.U`, fourteen words), and the seven AES and eight SHA words. Each entry says what is still to be worked out.

*Commentary.* **Precedent.** The PDP-11 partitioned its opcodes by the addressing modes of the first word; the VAX by operand specifiers parsed in sequence; x86 by prefixes, opcode, ModR/M and SIB read one after another. The prefix code is the device of a Huffman code (the commonest symbols get the shortest strings) applied to the first parcel, with the length of the instruction as a by-product of the match. **Trade-offs.** The decoder reads up to 16 bits to find the length. In the timing model the decoder pays one cycle of length-decode latency on every taken branch and refill, and fetch-wait is 7.2% of the cycles at four bytes per cycle (1.7% for the first iteration's encoding, whose length was two bits, 1.8). What the prefix buys is density: on the 841-file corpus the new compiler's output is 11.04% smaller than the old compiler's output for the first encoding; the encoding alone, measured on the old compiler's code, was 6.48% smaller for an earlier prefix-coded iteration (1.8) and 9.66% smaller with the four fused pairs, and most of the saving in instructions executed is the compiler (appendix F, F.4). **Improvements.** Every bit is a prefix, sub-opcode or operand bit, so an unused code is an illegal instruction and not a silent no-op; the fields of every leaf come from the grammar, so no table is written twice. **Benefits.** The common operations, a register move, a load from the stack, a branch on a register against zero, cost two bytes with no look at a second parcel,

and the rare ones are as long as they need to be. **Modern comparables.** Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms with the length in a few bits; x86 finds the length by parsing; the prefix code here is the general case of the first and avoids the second. **Impact.** *Kernel:* nothing in the kernel depends on the opcode map; a debugger and the fault handler's step-over read the same generated table as the assembler. *Compiler:* the cost model charges 2, 4 or 6 bytes (10 for `movql`) and knows which operands fit a one-parcel form, the hot registers above all (9.4.6). *Hardware:* the length is a function of up to 16 bits of the first parcel, a lookup in the fetch path; a front end that decodes several instructions a cycle needs such a lookup at each parcel position, a cost that the timing model does not measure beyond the one cycle above.

## 9.2 Properties and principles

**9.2.1** The length of an instruction is a function of its first parcel alone: the prefix selects the leaf, and the leaf has 1, 2 or 3 parcels, for 2, 4 or 6 bytes. *(D)*

**9.2.2** At most one operand of an instruction may be a memory reference; the other, where present, is a register. The block operations are the one admitted exception. *(D)*

**9.2.3** No instruction is longer than six bytes, with one exception: `movql` carries a 64-bit literal and is ten bytes, the literal not being counted as part of the instruction. *(D)*

**9.2.4** A register is always written in full: narrow loads sign- or zero-extend, and there is no partial-register write. *(D)*

**9.2.5** Anything that changes what an instruction means is, by default, visible state set by an instruction that names it, not hidden state; whether a mode bit is worth its cost is decided case by case (errata 83). Compatibility is by conversion instructions where that is cheap. *(D)*

**9.2.6** Every instruction can be restarted after a fault, and exceptions are precise; the machine has no hidden instruction state. *(D)*

**9.2.7** Floating point is done in the general registers; there is no separate floating-point register file. *(D)*

**9.2.8** Memory is sequentially consistent between processors. *(D)*

**9.2.9** The ALU has one width, 64 bits; sizes appear only on loads, stores and conversions. *(D)*

**9.2.10** Addresses are canonical: 39, 48 or 57 bits of virtual address, according to the three, four or five levels of the page table (6.6.1), and a physical address of 40 to 64 bits, 40 in the present implementation (2.3.5). *(D)*

**9.2.11** Instructions are aligned to two bytes; quad data accesses are naturally aligned, and the explicit unaligned forms exist for the cases that need them. *(D)*

**9.2.12** An instruction does what its name says and nothing else; no flag is set that its entry does not list. *(D)*

**9.2.13** Measured on the 841-file corpus of the compiler (instruction bytes, `as64 --stats`), the output of the new compiler is 964,158 bytes in 296,070 instructions (3.26 bytes an instruction), 11.04% fewer bytes than the old compiler's 1,083,870 for the first encoding (1.8); over six large programs it executes 24.21% fewer instructions and 28.94% fewer bytes than the same programs on the first encoding, an average of 2.934 bytes an executed instruction against 3.129. The fused pairs account for 19,786 instructions of the corpus. Linker relaxation (9.5.5) takes a further 3.76% off the text of the corpus. The tables and their provenance are in appendix F.

*Commentary.* **Precedent.** 9.2.2 is x86's ModR/M idea made architectural; it is also what lets the exception machinery be small (the PDP-11 recorded at most two register side effects per instruction because it had two specifiers; with one memory operand there is at most one). 9.2.6 exists because the Motorola 68000, which was very nearly this machine, could not restart an instruction after a bus error and so could not do demand paging until its successor retrofitted a fault model. 9.2.4 is the rule x86-64 adopted for 32-bit writes and failed to apply to the 8- and 16-bit forms. 9.2.7 is the arrangement RISC-V standardised as Zfinx. **Trade-offs.** 9.2.9 means narrow arithmetic is done wide and converted, which costs an extension instruction after some narrow operations. 9.2.8 is the cheapest model on a small in-order core and the most expensive on a large speculative one. 9.2.10 lets the page table have three levels, as RISC-V's Sv39 does, or four or five as x86-64, AArch64 and Sv48 and Sv57 do, at a longer walk for each level. **Improvements.** The rules are soft defaults: each says what it protects and what breaking it costs. **Benefits.** Together they make the machine small, restartable and easy for a compiler; the measured code size is in chapter 1. **Modern comparables.** AArch64 and RISC-V have fixed-length register-to-register encodings and weak memory models; x86-64 has memory operands and total store order; the pdp64 takes memory operands, strong ordering and a bounded fault surface. **Impact.** *Kernel:* Restartable instructions and precise exceptions mean a page-fault handler repairs the mapping and returns, with no instruction-specific cases, and sequential

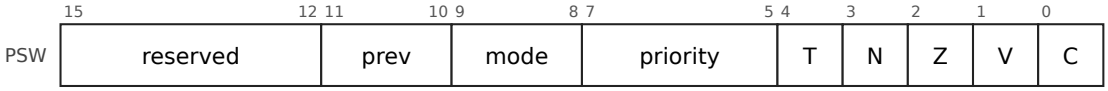
consistency means no barrier code. *Compiler:* One memory operand keeps instruction selection and register allocation simple; a register written in full removes partial-register merges; the single ALU width costs an extension instruction after some narrow arithmetic. *Hardware:* One address generator, at most one register side effect, a single-entry undo buffer, no byte-lane multiplexing and a length that is a lookup on one parcel are what keep the die small.

## 9.3 State: registers and flags

**9.3.1** There are thirty-two 64-bit general registers, `R0` to `R31`. `R14` is the stack pointer `SP`, with three copies (kernel, supervisor, user) chosen by the current mode; `R13` is the frame pointer by convention only; the others carry no meaning to the hardware. *(D)*

**9.3.2** The program counter is not a general register, and `R7` is an ordinary general register, not a view of the program counter; the program counter is read with `LEA 0(PC), Rn`. *(D)*

**9.3.3** The processor status word is 16 bits. *(D)*




(Bit positions as in System v1: `C` 0, `V` 1, `Z` 2, `N` 3, `T` 4, priority 5-7, current mode 8-9, previous mode 10-11, `UA` 12.)

**9.3.4** The flags form two groups, `C` and `{N, Z, V}`, and no instruction writes part of `{N, Z, V}` or writes it conditionally. Moves, loads, stores and the compare-and-branch instructions write no flags; shifts write the same flags for every count; rotates and the funnel shift write none; the by-one left shift sets `V` to `N` xor `C`. *(D)*

**9.3.5** `DP`, the direct-page base, has three copies by mode. It makes `d(DP)` an address form that needs no base register (a 28-bit displacement in six bytes; the grammar has loads through `d(DP)` and no store); in kernel mode it is the per-CPU pointer, and in user mode the thread pointer. *(D)*

**9.3.6** Other special registers: `FPSR` (five sticky floating-point flags), `ASID`, `PTBR`, `DMW0` and `DMW1`, `EPC`, `EPSW`, `EVA`, `ECAUSE`, `VBR`, `MMUCTL`; the system registers are read and written by `MFSR` and `MTSR` with an 8-bit register number. *(D)*

**9.3.7** A bit `DIT` in the status word promises data-independent timing: while it is set no data-dependent timing effect of any kind remains, neither in instruction latency (divide, `MSCN`, `MCMP` and floating point included), nor in frequency or power management (a core that scales frequency holds it steady while the bit is set, or says in a feature register that it cannot), nor from any data-dependent prefetcher or value predictor (a core may have one, but it is off while the bit is set). User code may set it for its own use. *(D)* Open: the position of the bit in the status word, and which instructions take constant time while it is clear (J.9.2).

**9.3.8** A register-save mask (`PUSHM`, `POPM`, `POPMRET`) names the registers by bit: bit n is `Rn`, and the mask is 32 bits. The forms carry the mask in 8 bits (two bytes), 16 or 24 bits (four bytes) and 32 bits (six bytes); the fused `POPMRET` has an 8-bit field scaled by 256, which names `R8` to `R15` and no other register. A mask containing `R14` is reserved. Masks are written in binary with the hexadecimal in brackets: the mask of `R8` to `R11` is `0000 1111 0000 0000` (0x0F00), and the 8-bit field of `POPMRET` that names `R8` to `R15` is `1111 1111` (0xFF, which is 0xFF00 as a mask).

**9.3.9** `R16` to `R31` are ordinary registers to the hardware. `R24`, `R25` and `R26` are among the sixteen hot registers of the four-bit fields (9.4.6). *(D)*

*Commentary.* **Precedent.** The PDP-11's PC and SP were ordinary registers and its flags were set by nearly every instruction. The VAX put the PC in R15. **Trade-offs.** Keeping the PC out of the register file means a jump to a computed address uses `JMP`, not a write to a register, so that writing `SP` has no control-flow consequence and the hazard logic treats it like any other destination. Condition codes are kept (rather than a flagless design) because compare-and-branch and flag-setting arithmetic make for dense code; the flag rules make them cheap for an out-of-order implementation, because the compiled corpus shows that 68% of instructions write a flag and only 1.5% read one. **Improvements.** The status word carries a user-memory guard bit (9.14) and the per-mode `DP` serves two jobs, the per-CPU pointer in the kernel and the thread pointer in user mode. **Benefits.** An exception handler finds a kernel stack and a per-CPU pointer without first searching for them; a threaded program's thread-local variable is one instruction. **Modern comparables.** x86-64 keeps `FS` and `GS` base registers for the same two jobs; AArch64 has `TPIDR_EL0` and `TPIDR_EL1`; RISC-V uses `tp` and the supervisor scratch register. **Impact.** *Kernel:* Banked stack and direct-page registers let entry find a kernel stack and a per-CPU pointer with no code, and the status word

is saved by one `PUSHX`. *Compiler:* The flag rules decide when a compare may fuse with its branch (1,893 pairs stay separate because a second branch reads the flags) and what the compiler may move between a compare and a branch. *Hardware:* Two flag groups need one or two rename targets, not one per bit; the register file is 32 by 64 bits; the banked copies are a three-way mux.

## 9.4 The instruction format

**9.4.1** An instruction is one, two or three parcels (two, four or six bytes; `MOVQL` is five); the first parcel holds the prefix, the sub-opcode and the operand pieces that fit, and the later parcels hold the rest. The bit layout of every leaf is a chart in the reference tables. *(D)*

**9.4.2** Instructions are parcel-aligned and little-endian; an instruction may cross any boundary of 8 bytes, a cache line or a page. *(D)*

**9.4.3** Most instructions are two or four bytes. Register-to-register work and the whole conversion set are one parcel; so is a memory operand through a hot base register with a displacement of up to three bits, a load or store from the stack pointer at a small offset, and a constant of up to seven bits. A base register with a 12-bit displacement, a post-increment or pre-decrement with a 12-bit stride, a scaled index with a 6- or 8-bit displacement, a pc-relative address within ±32 KiB and an immediate of up to 18 bits are two parcels. Displacements to ±128 MiB, absolute addresses, 32-bit immediates and branches to ±4 GiB are three. *(D)*

**9.4.4** Where an addressing mode names no base register (immediate, absolute, PC-relative), no field is spent on one: the later parcels are all value, so those modes reach 16 bits at four bytes and 32 bits at six. *(D)*

**9.4.5** A constant wider than 32 bits is either written in the instruction, `MOVQL #imm64, Rd`, which is ten bytes with the literal in the four later parcels, or loaded from a literal pool with `MOVQ =K, Rd`, written with an explicit `=`; nothing is pooled silently, and a `MOVQ #imm` whose constant fits no `MOVQ` form is assembled as `MOVQL`. *(D)*

**9.4.6** In a one-parcel form a 3-bit register field is not a register number but an index into a table of the eight hottest registers, `R0 R1 R8 R2 SP R3 R9 R10`. A 4-bit field indexes the sixteen hottest, `R0 R1 R8 R2 SP R3 R9 R10 R11 R4 R15 R5 R24 R25 R26 R6`. The tables are part of the ABI, and the compiler's register allocator prefers the hot registers. *(D)*

*Commentary.* **Precedent.** The PDP-11 had the length in the first word, but as a function of both of its six-bit specifiers; Thumb-2 and RISC-V's compressed instructions mix 16- and 32-bit forms with the length in a few bits. Here the length and the identity of the instruction are one match on the prefix. **Trade-offs.** Byte-granular encoding, as in x86, buys perhaps a tenth of the code size and costs a full alignment rotator and cross-line splice logic in the fetch path on a 64-bit datapath; this CPU refuses it. A hot-register index buys a one-parcel form and costs a rule for the compiler: a value that lives in one of the eight (for the 4-bit fields, sixteen) hottest registers is cheaper to use, which the allocator learns, and the ABI cannot move the tables without changing the encoding. Forcing an instruction to stay inside an eight-byte window would simplify fetch and would cost 5% to 10% of code size in padding. **Improvements.** `MOVQL` takes a 64-bit constant in one instruction instead of a literal-pool load, at ten bytes against six, the literal not counting against the three-parcel rule (errata 85). **Benefits.** The measured average is 2.7 to 3.3 bytes an instruction (9.2.13). **Modern comparables.** AArch64 and RISC-V have fixed or two-length encodings; x86 has byte-granular ones; this CPU has parcel-granular ones with three common lengths. **Impact.** *Kernel:* a one-parcel form is cheaper in kernel text with no source change. *Compiler:* the assembler picks the shortest form, branch and call relaxation is a length choice, and the cost model knows 2, 4 and 6 bytes and which registers are hot. *Hardware:* the fetch unit has a parcel buffer and a length lookup on the first parcel; no instruction crosses more than two parcel-aligned windows of eight bytes except the ten-byte `MOVQL`.

## 9.5 Addressing modes

**9.5.1** The addressing mode is not a field: it is the leaf, whose form lists the shape of each operand. The assembly language has the eight modes below, and the assembler gives each operand the shortest leaf that holds it. *(D)*

| Syntax | Effect | One parcel | Two parcels | Three parcels |
|---|---|---|---|---|
| `d(Rn)` | base plus displacement | a hot base register with a displacement of 1 to 3 bits; `d(SP)` with a displacement of 4 to 7 bits | 8-, 10- or 12-bit displacement (scaled by the size of the access in most forms) | 28-bit signed displacement, ±128 MiB |

| Syntax | Effect | One parcel | Two parcels | Three parcels |
|---|---|---|---|---|
| `d(Rn)(Rx*s)` | scaled index, `s` in {1, 2, 4, 8} | none | 6- or 8-bit displacement, or none with a 5-bit base and index | 22-bit displacement |
| `d(PC)` | PC-relative; the base is the first parcel of the instruction | none | 12-, 14- or 16-bit byte displacement: ±2 KiB, ±8 KiB, ±32 KiB | 32-bit displacement, ±2 GiB |
| `d(DP)` | direct page, loads only | none | none | 28-bit displacement |
| `#imm` | immediate, sign-extended | 3 to 7 bits | up to 18 bits | 32 bits (`MOVQL`: 64 bits, five parcels) |
| `@abs` | absolute, sign-extended | none | none | 32-bit address |
| `(Rn)+d` | post-increment by a stride `d` (0: the operand size) | none | 12-bit stride | none |
| `-(Rn)` or `(Rn)-d` | pre-decrement by a stride (0: the operand size) | none | 12-bit stride | none |

**9.5.2** The deferred modes of the PDP-11 are removed; indirection is done with `JMPI`, `CALLI` and a load. *(D)*

**9.5.3** At most one register side effect occurs per instruction, and it is committed at retirement; a fault discards it. *(D)*

Open: the displacement of the shortest scaled-index form, and a store through `d(DP)` (J.9.5).

**9.5.4** A pc-relative operand is measured in bytes from the first parcel of the instruction; the field counts bytes for a memory operand and parcels for a branch or call. A branch has a field of 6 to 9 bits in one parcel (±64 to ±512 bytes), 16 bits in two (±64 KiB) and 32 bits in three (±4 GiB); a `CALL` has 18 bits in two parcels (±256 KiB) and 32 in three; a `LEA` or a load through `d(PC)` reaches ±2 KiB to ±32 KiB in two parcels and ±2 GiB in three. The dictionary lists the width and reach of every operand shape.

**9.5.5** The assembler cannot know the distance to a symbol in another object, so it writes such a `CALL` or `LEA` of a data symbol in its long form (six bytes) with a relocation; the linker lays out the sections, gives each such instruction the shortest form that reaches (a 4-byte `CALL` within ±256 KiB, a 4-byte `LEA` or pc-relative load within ±32 KiB, a 4-byte `JMP` or `MOVCALL` within ±64 KiB) and moves everything behind it. It grows each instruction from its shortest form, so the lengths only grow and the iteration ends. Relaxation is on by default; `as64 --no-relax` and `ld64 --no-relax` give the unrelaxed bytes. *(D)*

*Commentary on 9.5.1, 9.5.4 and 9.5.5.* **Precedent.** The PDP-11 chose the mode in each six-bit specifier; x86 chooses it in ModR/M and SIB; here the leaf chooses it, and the cost of a mode is the length of the leaf that has it. **Trade-offs.** The scaled index is the thin case: its shortest form has a 6-bit displacement, and `d(DP)` has only the six-byte load form, so a thread-local access through `DP` costs six bytes. Relaxation turns the assembler's guess into the linker's measurement: it saved 3.76% of the text of the corpus (an upper bound for the reach of the 4-byte call, since the corpus stub places symbols right behind each file) and 1.65% of the linked text of the six large programs; external calls, 6 bytes each, are the bulk. **Improvements.** The stride of the post-increment and pre-decrement is the same field in every leaf, so an array of 16-byte records is one instruction per element; the scaled index takes a conversion of a narrower index in the compiler (`bs_ec`'s inner loop, 31% fewer instructions). **Benefits.** An object file from `as64` is correct without relaxation and smaller with it, and a program linked either way produces the same output on the six large programs. **Modern comparables.** RISC-V's linker relaxes `call` and `lui`/`addi` pairs for the same reason; AArch64 needs linker veneers for far branches. **Impact.** *Kernel:* the kernel is relaxed like any other program, and text near the reach limits is smaller by the 4-byte forms. *Compiler:* it emits the long form for an external symbol and leaves the choice to the linker; it prefers scaled index and strided autoincrement where its loops allow (appendix F, F.4). *Hardware:* nothing: the address adder takes a field and a base in every leaf.

*Commentary.* **Precedent.** The PDP-11 had eight modes for each of eight registers, with a deferred flavour of each, and it made `*p++` and `*--p` single instructions. **Trade-offs.** Dropping the deferred modes removes the case where a fault occurs after a register has been updated at an address that itself came from a load; keeping the strided autoincrement costs nothing, because the undo information was already a delta (the PDP-11's `MMR1` recorded an amount for the same reason). **Improvements.** The increment is a stride, so walking an array of 16-byte records is one instruction; the PC-relative modes count from the instruction itself, so a branch needs no adjustment and `LEA` reads the PC. **Benefits.** Address generation is one adder pass, never load-then-add, which is the difference between one cycle and a long path on an FPGA. **Modern comparables.** AArch64 has pre- and post-indexed forms with an immediate; x86 has scaled index with a base; RISC-V has base plus immediate only. **Impact.** *Kernel:* A strided autoincrement walks a table of records, and `d(DP)` reaches the per-CPU structure in one instruction. *Compiler:* The modes match C idioms (`*p++`, `a[i]`) one-to-one, and dropping the deferred

modes removes the cases the selector would otherwise have to consider. *Hardware:* Address generation is one adder pass, never load-then-add, and the PC-relative base is the first parcel, so no adjustment is needed.

## 9.6 Moves, conversions and address formation

**The instructions.** Every instruction has an entry in the section of its group, laid out as in a processor reference manual: its mnemonics, its syntax, its encoding (the chart of its layout and the list of all its layouts, each with a chart of its own), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains the matter it touches (*Cf.*). Every instruction, the atomic and the system instructions included, has its official entry below (sections 9.13 and 9.14 for those two groups); the chapters on the kernel and on several processors give the reasons, and an entry may point back to them for a fuller explanation. In the pseudocode, ← is assignment, = and ≠ compare, × and ÷ multiply and divide, and `M[a]` is memory at address `a`. Numbers that are encodings (prefixes, sub-opcodes, field values) are written in binary. Masks are binary too, with a subscript 2 to mark the base (`Rn & 111₂` keeps the low three bits). The layouts are generated from the grammars of the encodings, `spec/pdp64-enc.hbnf` and the grammar of the CPU, so a chart here cannot disagree with the assembler, the linker or the simulator. The aliases, the constants, the lists of the mnemonics and the bit layouts of every opcode are in the reference tables.

**The stories.** An instruction that is not common (the PDP-11 core and the operations every architecture has carry none) is followed by *Why it is here*: the machine the idea came from and the reason it was chosen, with the measurements that bear on it. Where no reason is recorded, the paragraph says so. The documents behind the paragraphs are listed in appendix F.

**Notation.** `Rd` is the destination register, `Rs` the first source, `Rt` the second, `Ru` the third; the assembler writes the destination last, and the operands of a chart are numbered `op1`, `op2`, ... in the order of the assembly syntax. `rm` is a general operand: a register, or a memory operand, an immediate, a pc-relative or an absolute address, by the addressing forms that the layouts of the instruction give (section 9.5). `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. `N Z V C` are the condition flags. In the charts the bits of a parcel are numbered from 15 at the left, a shaded box is a fixed value, and a white box is a field that varies.

**Registers.** The pdp64 has 32 registers, `R0` to `R31`. A register field of an instruction is the full number of bits (5), or fewer where the field names one of the hot registers; the entries that name `R0` to `R15` apply to `R16` to `R31` too; the hot-register tables are part of the ABI: the 3-bit fields hold, in order, `R0 R1 R8 R2 SP R3 R9 R10`; the 4-bit fields hold, in order, `R0 R1 R8 R2 SP R3 R9 R10 R11 R4 R15 R5 R24 R25 R26 R6`. `PUSHM` and `POPM` take a 32-bit mask (errata 87).

*Cf. section 6.7 for the memory types that decide whether an access is cached, and section 6.4 for what a faulting load leaves behind.*

**9.6.1** `MOVQ`, `MOVL`, `MOVW` and `MOVB` load and store 64, 32, 16 and 8 bits; the loads sign-extend. `MOVZL`, `MOVZW` and `MOVZB` load with zero extension.

**9.6.2** With a register operand, the same opcodes are the conversion set: `MOVB R1, R2` sign-extends the low byte, `MOVZW R1, R2` zero-extends the low word.

**9.6.3** `LEA` forms an effective address; `LEA 0(PC), Rn` reads the program counter.

**9.6.4** Moves, loads and stores write no flags. *(D)*

**9.6.5** The explicit unaligned forms `MOVUB`, `MOVUW`, `MOVUL` and `MOVUQ` exist in both directions. *(D)*

**9.6.6** The one-parcel forms abbreviate the common cases: loads and stores through a hot base register or the stack pointer at small offsets, `LEA` from `SP`, and `MOVQ #imm, rd` with an immediate of up to seven bits. *(D)*

**9.6.7** A load may be qualified `.D` (deferred fault: a load that can be hoisted above its guard, because a fault that the kernel finds invalid poisons the register instead of stopping the program, and the fault is taken at the instruction that writes the flags from a poisoned source), and a load or store may be qualified `.U` (the memory operand is accessed with user privilege, so that the user-memory guard of 6.10 is not opened and closed around a copy). *(D)* Details pending (J.9.12).

*Commentary.* **Precedent.** The PDP-11 had `MOV` and `MOVB` with a byte sign-extended into a register; the VAX had a sized `MOV` for every width. **Trade-offs.** Keeping the sized loads costs four extra opcodes against a quad-only design, but without `MOVL` a struct field load becomes two `MOVW`s, a shift and an or, and ELF, timestamps and most on-disk and wire structures are full of 32- and 16-bit fields. **Improvements.** There is no separate `CVT` group: the conversions are the moves with a register source, so they cost nothing. **Benefits.** No partial-register state anywhere. **Modern comparables.** AArch64 and RISC-V have the same sized loads with sign and zero extension; x86-64 keeps 8- and 16-bit register writes that preserve the upper

bits, which makes narrow writes depend on the old value. **Use.** `LEA 0(PC)` is how a position-independent program finds itself; the unaligned forms are for packed on-disk and network structures. **Impact.** *Kernel:* `LEA 0(PC)` lets position-independent kernel code find itself; the unaligned forms read packed on-disk and network structures. *Compiler:* Sized loads avoid the shift, mask and or of a struct-field read, and the conversions are moves, so no separate conversion selection exists. *Hardware:* No partial-register write anywhere; the conversions reuse the load-extend path.

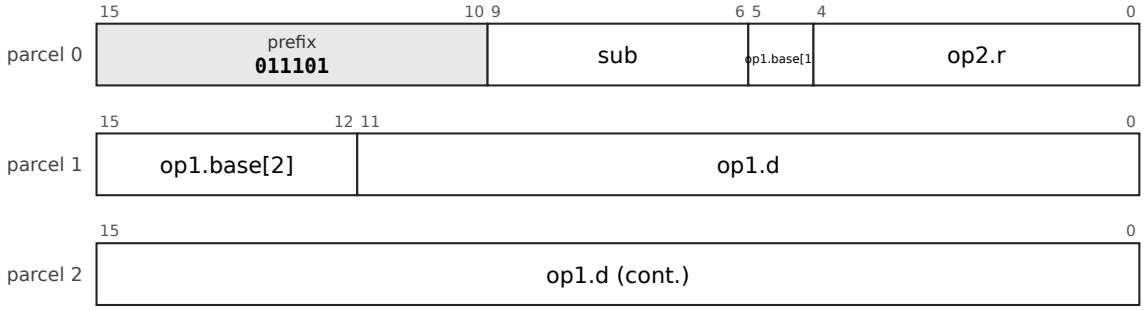
## MOV: Move, load and store (64, 32, 16 and 8 bits)

**Mnemonics:** `MOVQ`, `MOVL`, `MOVW`, `MOVB`

### Syntax

```
MOVQ rm, Rd        ; load
MOVQ Rs, rm        ; store
MOVL, MOVW, MOVB likewise
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `STF-B28-1`, `STF-X22-1`, `STF-PI12-1`, `STF-PD12-1`, `STF-PC32-1`, `STF-A32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `RR5-1`, `STBu6-a-1`, `LDX0-a-1`, `LDPC12H-a-1`, `LDI14H-a-1`, `LDPC14H-a-1`, `STBs8H-a-1`, `LDBu8H-a-1`, `LDI16H-a-1`, `STX0-a-1`, `IMMh43-a-1`, `LDI12-a-1`, `LDPC16H-a-1`, `STBu10H-a-1`, `LDBu10H-a-1`, `LDHHH1-a-1`, `STHHH1-a-1`, `LDBs6-a-1`, `LDSPh4-a-1`, `IMMh45-a-1`, `STSPh4-a-1`, `LDBu12H-a-1`, `STBu12H-a-1`, `LDHHH3-a-1`, `LDPC12-a-1`, `IMMh47-a-1`, `LDBu8-a-1`, `STBs6-a-1`, `LDX6H-a-1`, `STX6H-a-1`, `LDBs10-a-1`, `STBs8-a-1`, `LDFI-I32-2`, `STBu8H-b-1`, `LDBu6-b-1`, `STX0-b-1`, `STBs6-b-1`, `LDX0-b-1`, `STBu10H-b-1`, `LDBu8H-b-1`, `STHHH1-b-1`, `STBu12H-b-1`, `LDPC12H-b-1`, `STBs8H-b-1`, `LDS12H-b-1`, `STX4H-b-1`, `LDBu12H-b-1`, `LDBs6-b-1`, `STBu6-b-1`, `LDI12-d-2`.

### Operation

```
size = 8 (MOVQ), 4 (MOVL), 2 (MOVW) or 1 (MOVB) bytes
load :  Rd ← sext(M[EA(rm)], size)          ; len = 00: Rd ← sext(Rs[8×size-1:0])
store:  M[EA(rm)] ← Rs[8×size-1:0]          ; len = 00: Rd ← sext(Rs[8×size-1:0])
```

**Flags** None written.

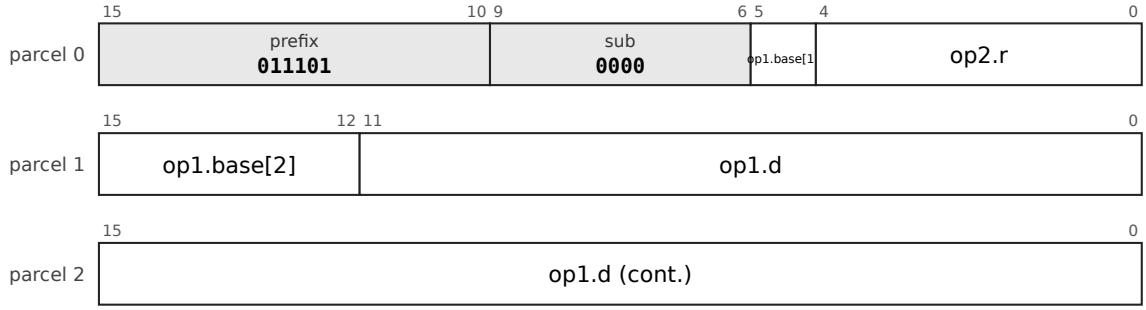
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

A register is always written in full: a narrow load sign-extends to 64 bits and there is no partial-register write anywhere. The register-to-register forms of the sized moves are the conversion set: `MOVB R1, R2` sign-extends the low byte of `R1` into `R2`.

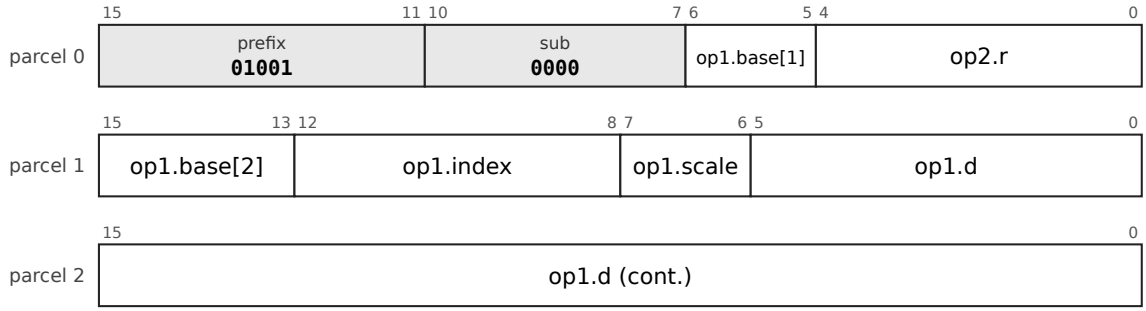
**Cf.** 6.7 for the memory type that decides whether the access is cached; 6.4 for what a faulting access leaves behind.

## *MOVQ*

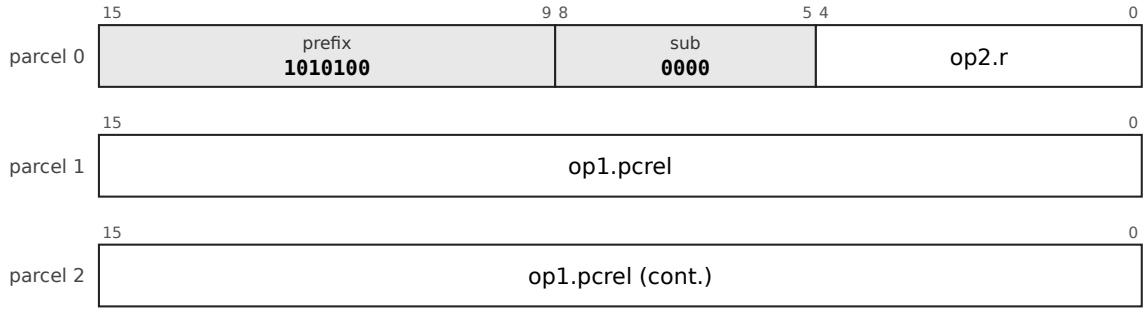
**MOVQ: LDF.B28, prefix 011101, sub-opcode 0000; operands b5d28s-x1, reg5; 6 bytes**



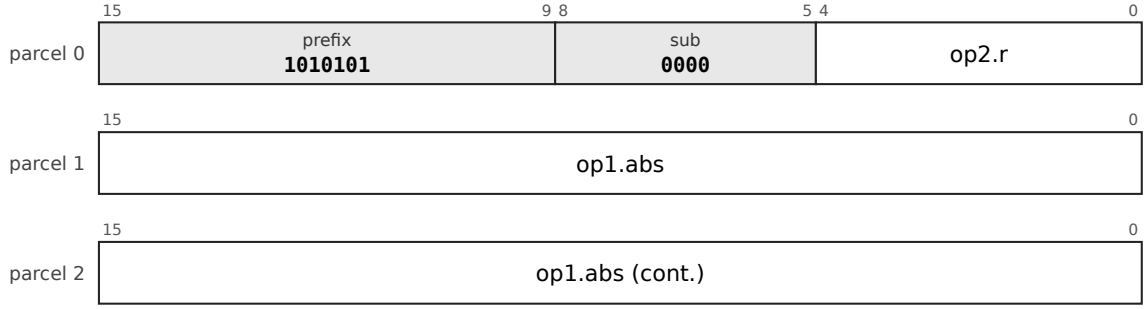

**MOVQ: LDF.X22, prefix 01001, sub-opcode 0000; operands x5-5d22s, reg5; 6 bytes**



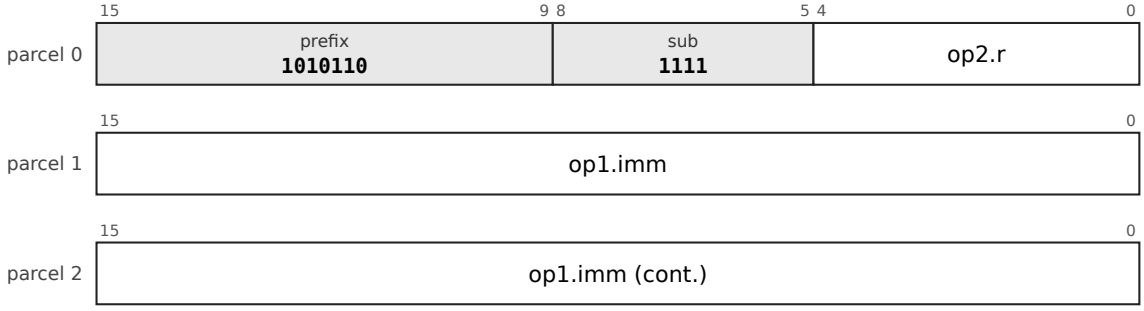

**MOVQ: LDF.PC32, prefix 1010100, sub-opcode 0000; operands pc32-u1, reg5; 6 bytes**



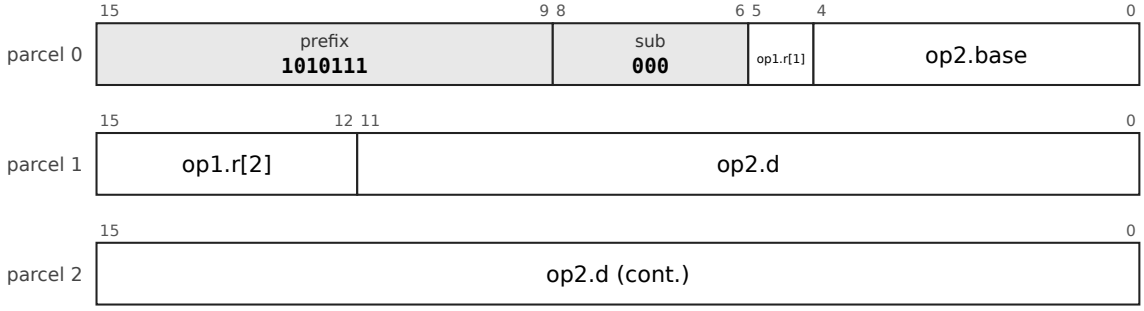

**MOVQ: LDF.A32, prefix 1010101, sub-opcode 0000; operands abs32, reg5; 6 bytes**



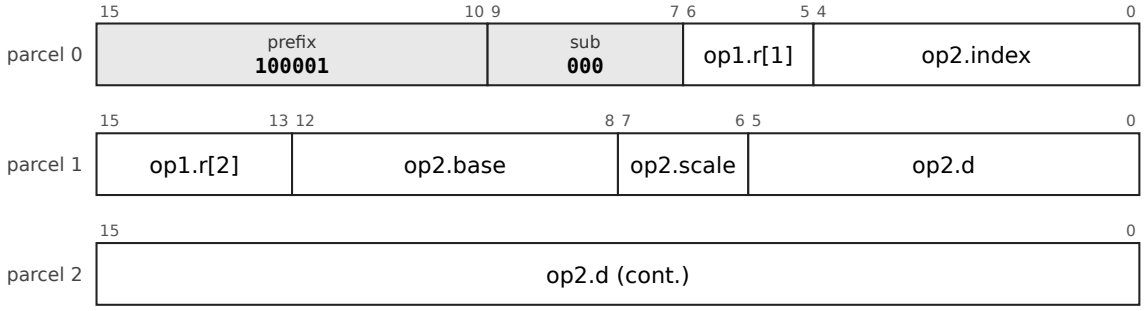
**MOVQ: LDFI.I32, prefix 1010110, sub-opcode 1111; operands imm32s, reg5; 6 bytes**



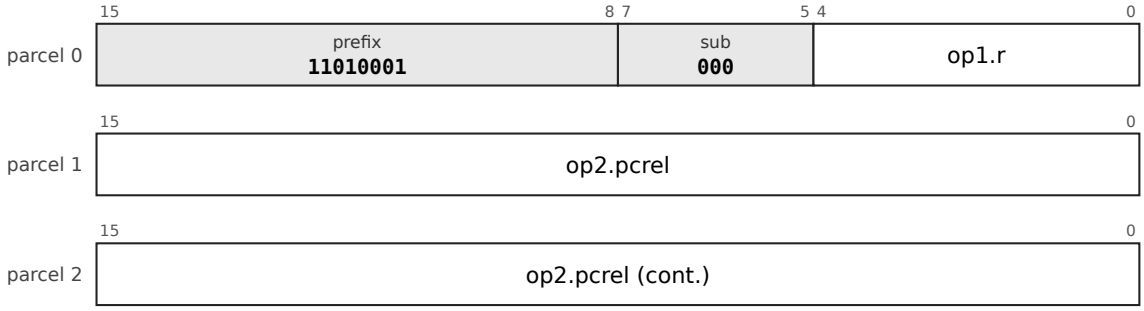

**MOVQ: STF.B28, prefix 1010111, sub-opcode 000; operands reg5, b5d28s-x1; 6 bytes**



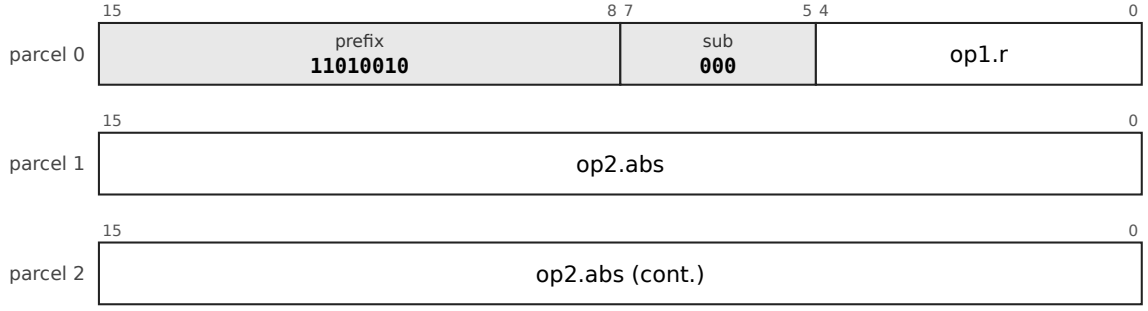

**MOVQ: STF.X22, prefix 100001, sub-opcode 000; operands reg5, x5-5d22s; 6 bytes**



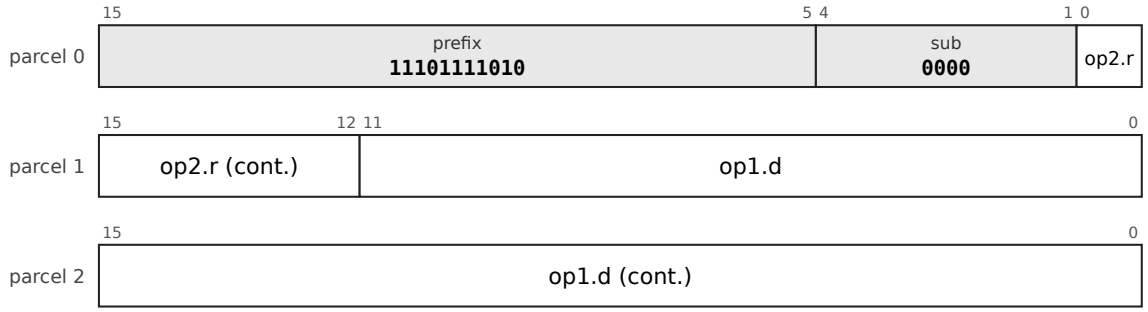

**MOVQ: STF.PC32, prefix 11010001, sub-opcode 000; operands reg5, pc32-u1; 6 bytes**



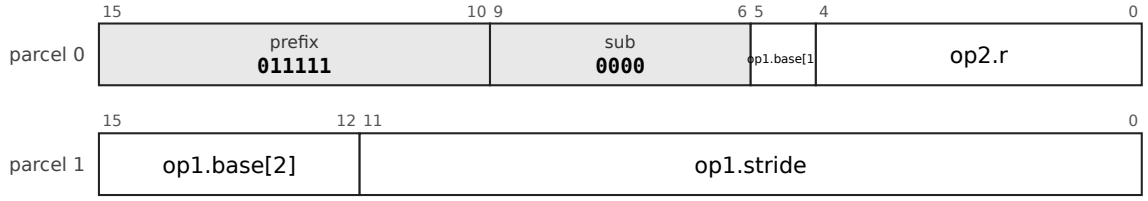
**MOVQ: STF.A32, prefix 11010010, sub-opcode 000; operands reg5, abs32; 6 bytes**



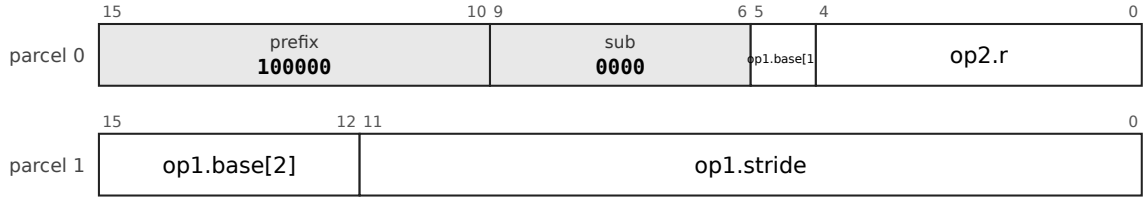

**MOVQ: DPF.D28, prefix 11101111010, sub-opcode 0000; operands dpd28s-x1, reg5; 6 bytes**



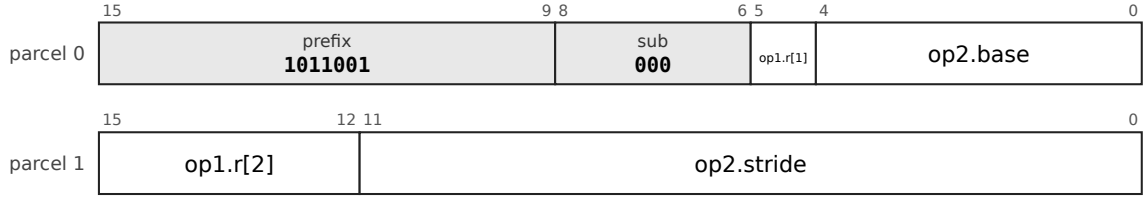

**MOVQ: LDF.PI12, prefix 011111, sub-opcode 0000; operands pi5-s12, reg5; 4 bytes**



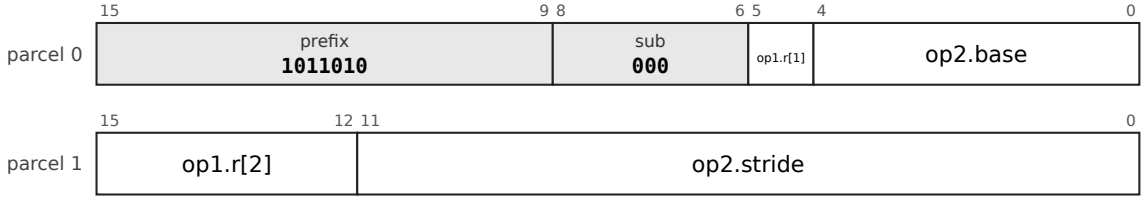

**MOVQ: LDF.PD12, prefix 100000, sub-opcode 0000; operands pd5-s12, reg5; 4 bytes**



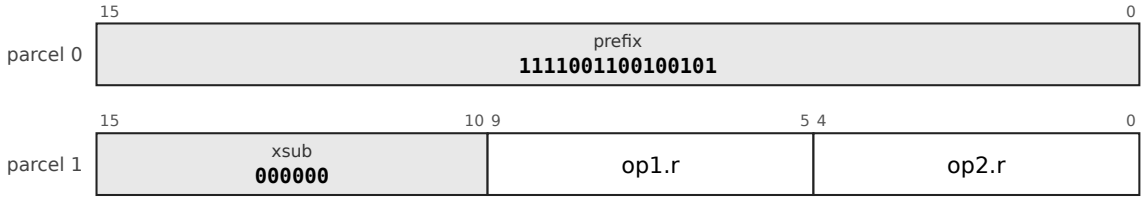

**MOVQ: STF.PI12, prefix 1011001, sub-opcode 000; operands reg5, pi5-s12; 4 bytes**



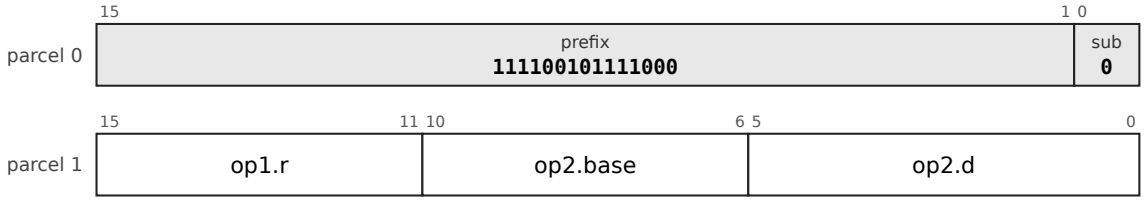
**MOVQ: STF.PD12, prefix 1011010, sub-opcode 000; operands reg5, pd5-s12; 4 bytes**



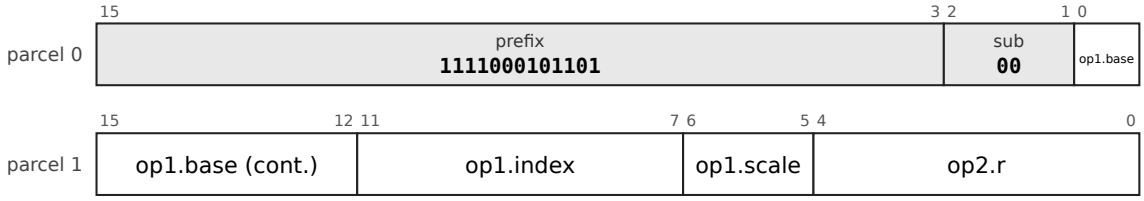

**MOVQ: RRF5, prefix 1111001100100101, sub-opcode 000000; operands reg5, reg5; 4 bytes**



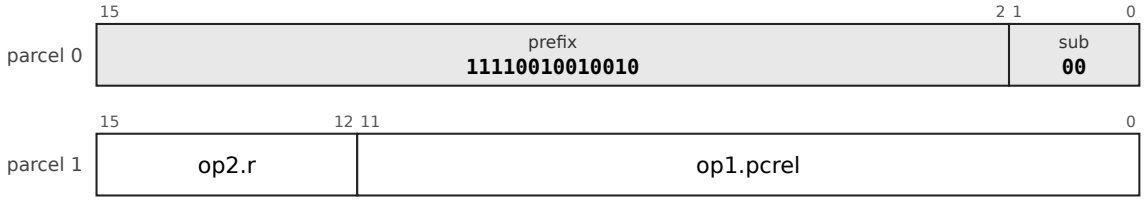

**MOVQ: STBu6/a, prefix 111100101111000, sub-opcode 0; operands reg5, b5d6u-xsz; 4 bytes**



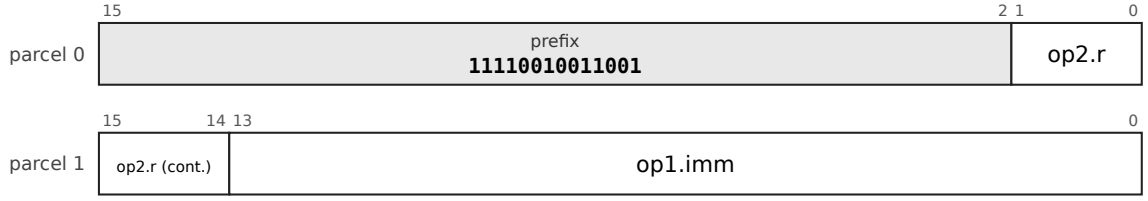

**MOVQ: LDX0/a, prefix 1111000101101, sub-opcode 00; operands x5-5d0s, reg5; 4 bytes**



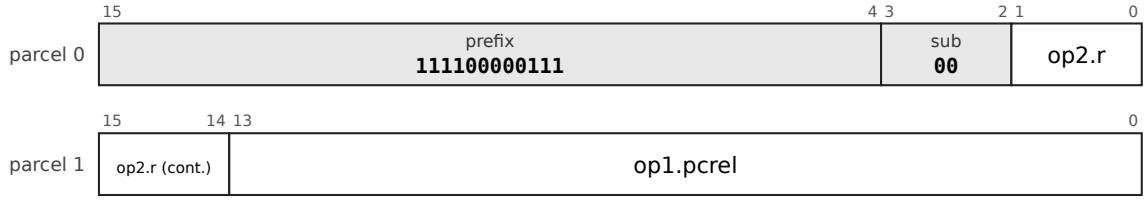

**MOVQ: LDPC12H/a, prefix 11110010010010, sub-opcode 00; operands pc12-u1, reg4; 4 bytes**



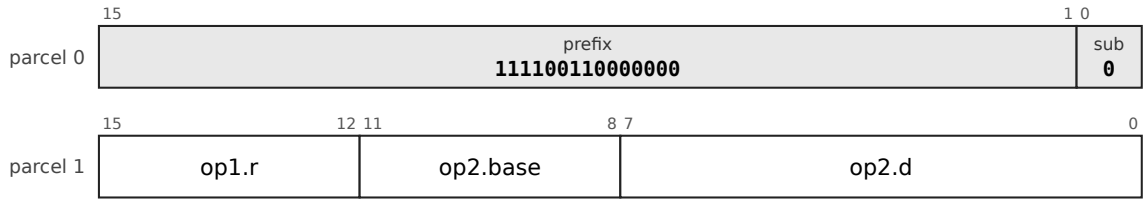
**MOVQ: LDI14H/a, prefix 11110010011001; operands imm14s, reg4; 4 bytes**



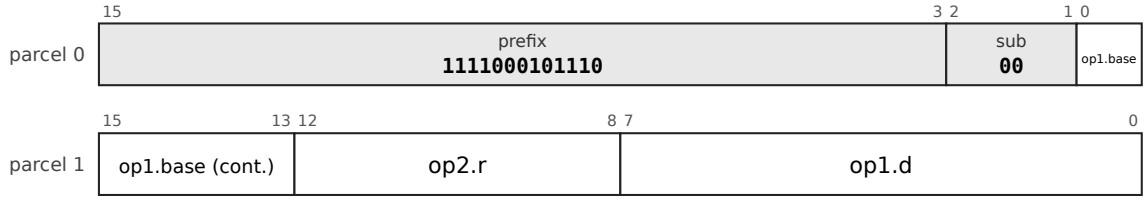

**MOVQ: LDPC14H/a, prefix 111100000111, sub-opcode 00; operands pc14-u1, reg4; 4 bytes**



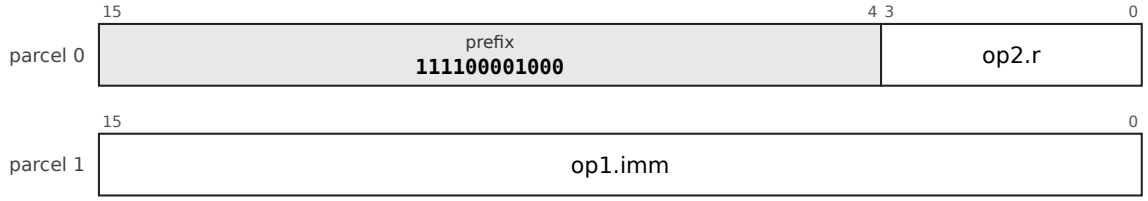

**MOVQ: STBs8H/a, prefix 111100110000000, sub-opcode 0; operands reg4, b4d8s-xsz; 4 bytes**



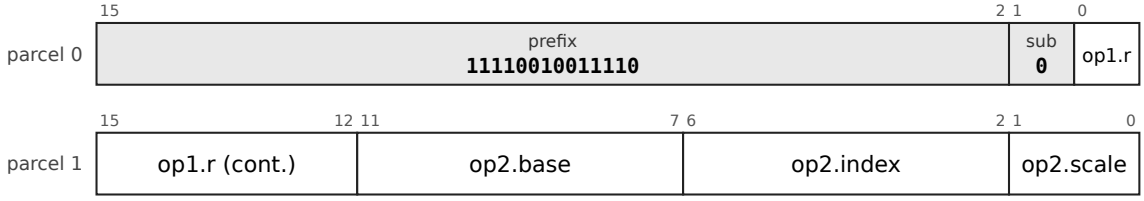

**MOVQ: LDBu8H/a, prefix 1111000101110, sub-opcode 00; operands b4d8u-xsz, reg5; 4 bytes**



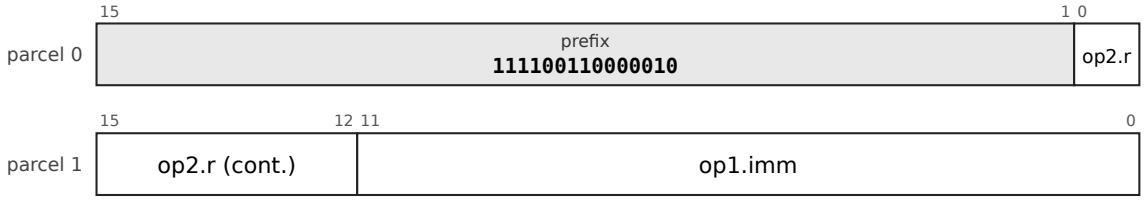

**MOVQ: LDI16H/a, prefix 111100001000; operands imm16s, reg4; 4 bytes**



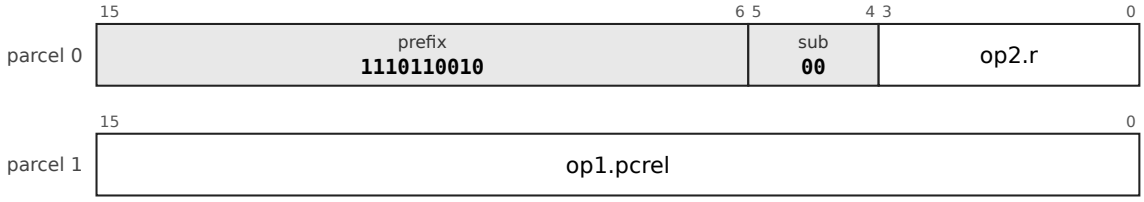
**MOVQ: STX0/a, prefix 11110010011110, sub-opcode 0; operands reg5, x5-5d0s; 4 bytes**



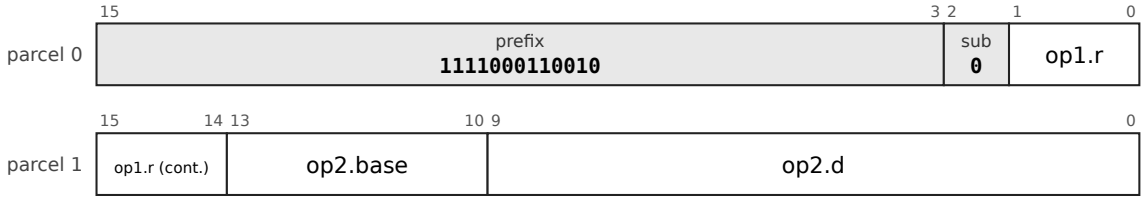

**MOVQ: LDI12/a, prefix 111100110000010; operands imm12s, reg5; 4 bytes**



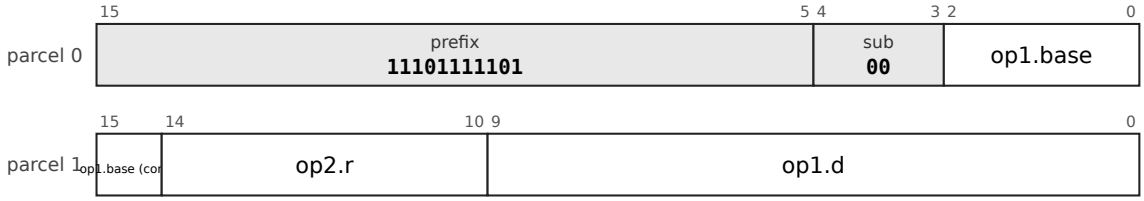

**MOVQ: LDPC16H/a, prefix 1110110010, sub-opcode 00; operands pc16-u1, reg4; 4 bytes**



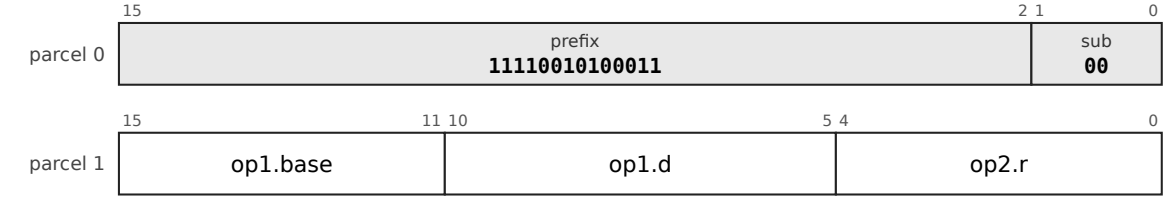

**MOVQ: STBu10H/a, prefix 1111000110010, sub-opcode 0; operands reg4, b4d10u-xsz; 4 byte**



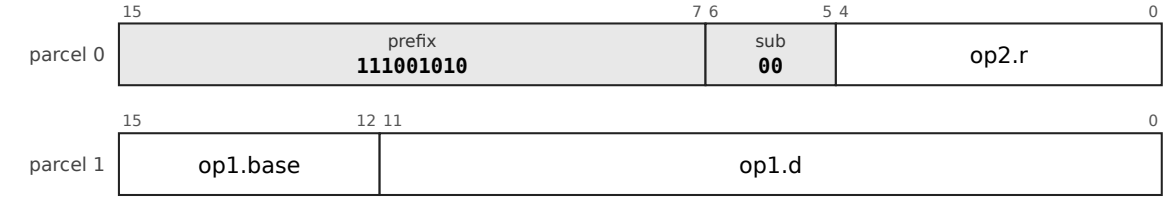

**MOVQ: LDBu10H/a, prefix 11101111101, sub-opcode 00; operands b4d10u-xsz, reg5; 4 bytes**



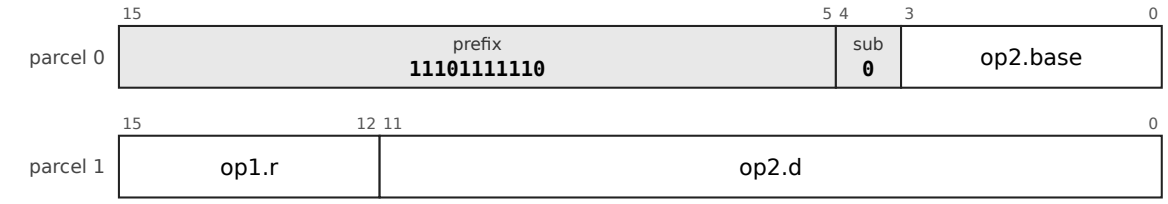
**MOVQ: LDBs6/a, prefix 11110010100011, sub-opcode 00; operands b5d6s-xsz, reg5; 4 bytes**



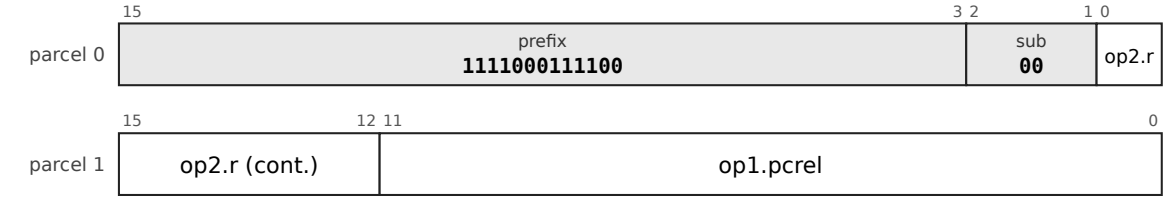

**MOVQ: LDBu12H/a, prefix 111001010, sub-opcode 00; operands b4d12u-xsz, reg5; 4 bytes**



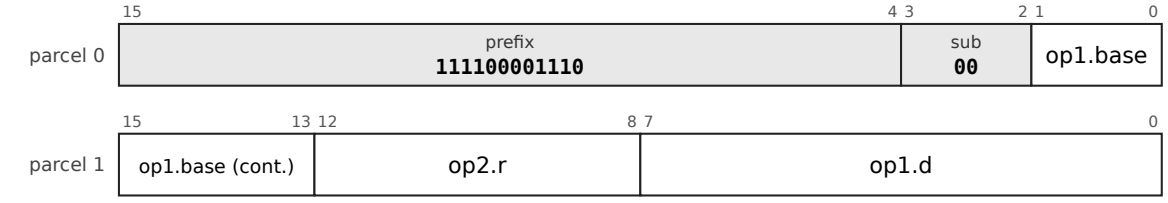

**MOVQ: STBu12H/a, prefix 11101111110, sub-opcode 0; operands reg4, b4d12u-xsz; 4 bytes**



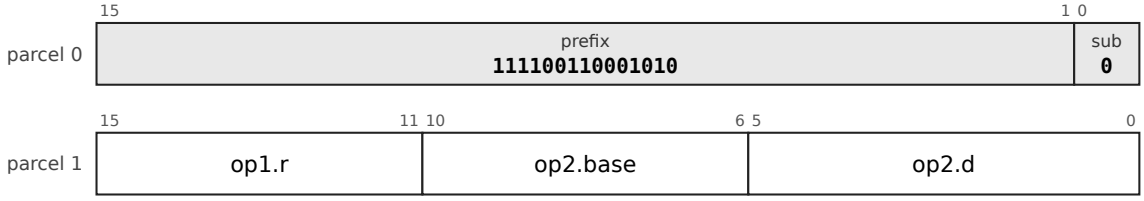

**MOVQ: LDPC12/a, prefix 1111000111100, sub-opcode 00; operands pc12-u1, reg5; 4 bytes**



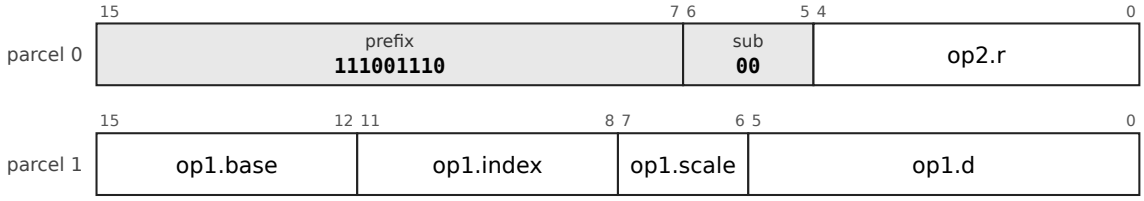

**MOVQ: LDBu8/a, prefix 111100001110, sub-opcode 00; operands b5d8u-xsz, reg5; 4 bytes**



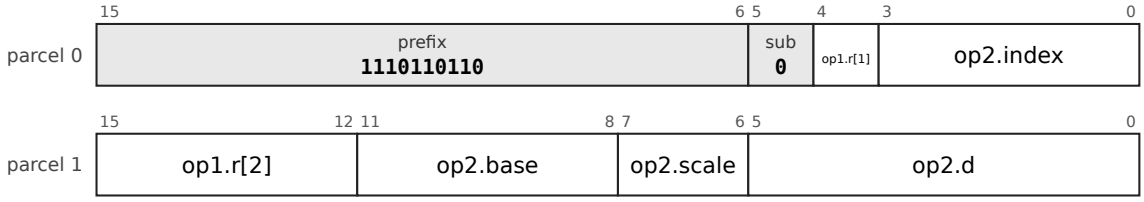
**MOVQ: STBs6/a, prefix 111100110001010, sub-opcode 0; operands reg5, b5d6s-xsz; 4 bytes**



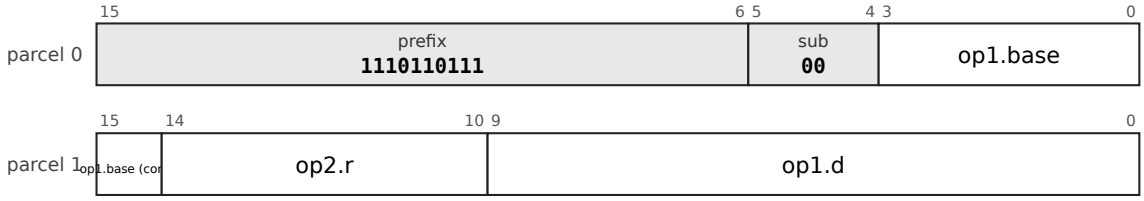

**MOVQ: LDX6H/a, prefix 111001110, sub-opcode 00; operands x4-4d6s, reg5; 4 bytes**



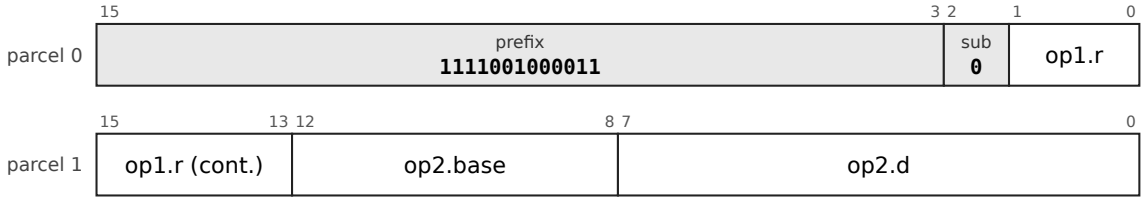

**MOVQ: STX6H/a, prefix 1110110110, sub-opcode 0; operands reg5, x4-4d6s; 4 bytes**



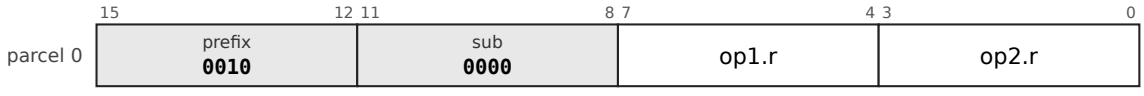

**MOVQ: LDBs10/a, prefix 1110110111, sub-opcode 00; operands b5d10s-xsz, reg5; 4 bytes**



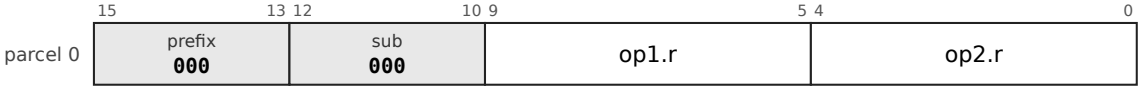

**MOVQ: STBs8/a, prefix 1111001000011, sub-opcode 0; operands reg5, b5d8s-xsz; 4 bytes**




**MOVQ: RRF, prefix 0010, sub-opcode 0000; operands reg4, reg4; 2 bytes**



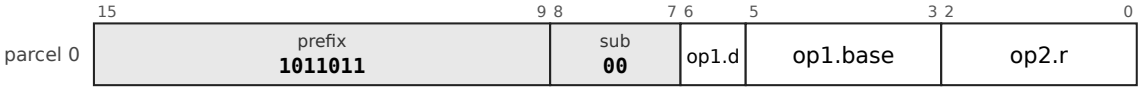
**MOVQ: RR5, prefix 000, sub-opcode 000; operands reg5, reg5; 2 bytes**



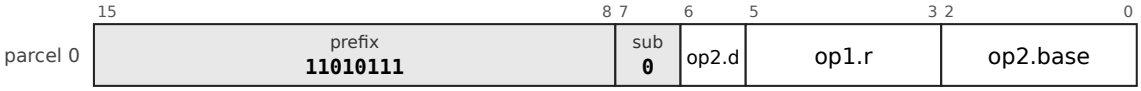

**MOVQ: IMMh43/a, prefix 111000101; operands imm3s, reg4; 2 bytes**



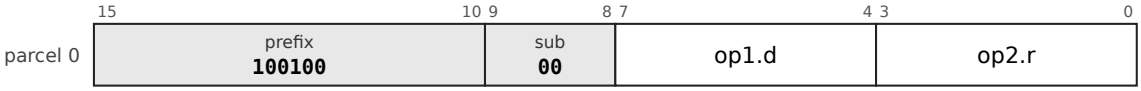

**MOVQ: LDHHH1/a, prefix 1011011, sub-opcode 00; operands b3d1u-xsz, reg3; 2 bytes**



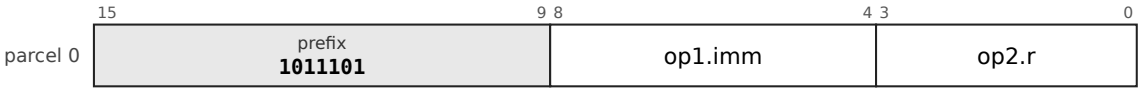

**MOVQ: STHHH1/a, prefix 11010111, sub-opcode 0; operands reg3, b3d1u-xsz; 2 bytes**



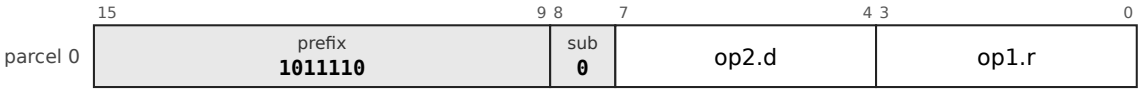

**MOVQ: LDSPh4/a, prefix 100100, sub-opcode 00; operands spd4u-xsz, reg4; 2 bytes**



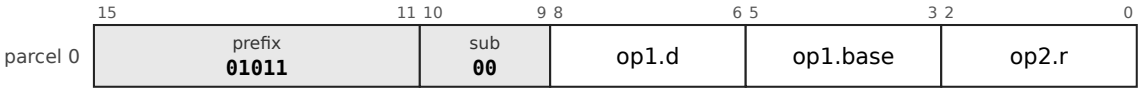

**MOVQ: IMMh45/a, prefix 1011101; operands imm5s, reg4; 2 bytes**



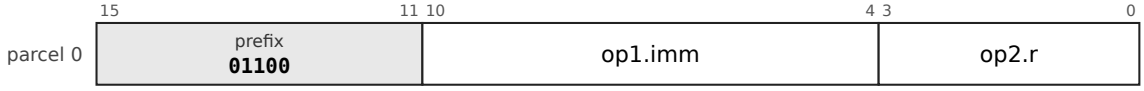

**MOVQ: STSPh4/a, prefix 1011110, sub-opcode 0; operands reg4, spd4u-xsz; 2 bytes**




**MOVQ: LDHHH3/a, prefix 01011, sub-opcode 00; operands b3d3u-xsz, reg3; 2 bytes**



**MOVQ: IMMh47/a, prefix 01100; operands imm7s, reg4; 2 bytes**

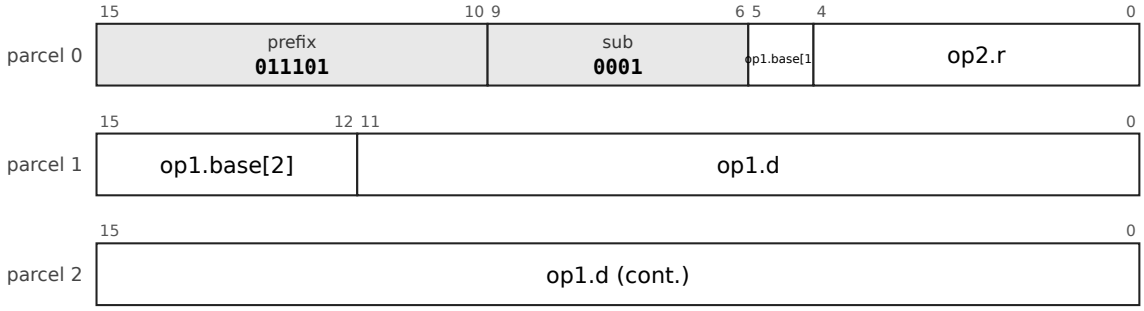



```
MOVQ rm, Rd
MOVQ Rs, rm
```

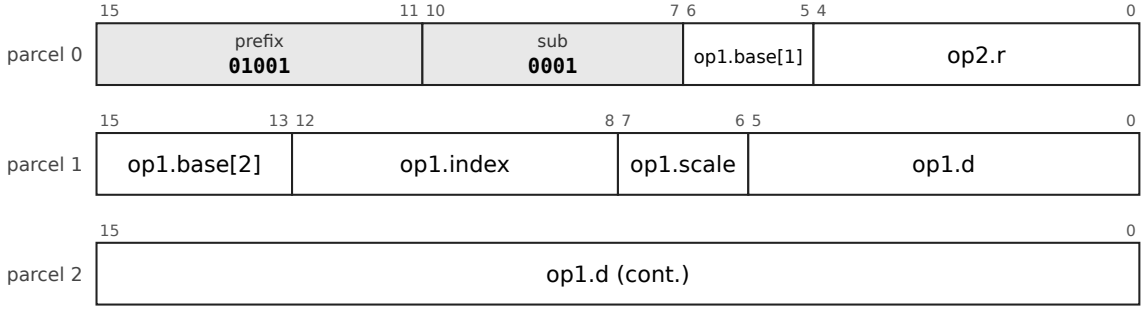
Operation, flags and exceptions as for the entry MOV above; 48 layout forms, each a chart of its own (above).

### *MOVL*

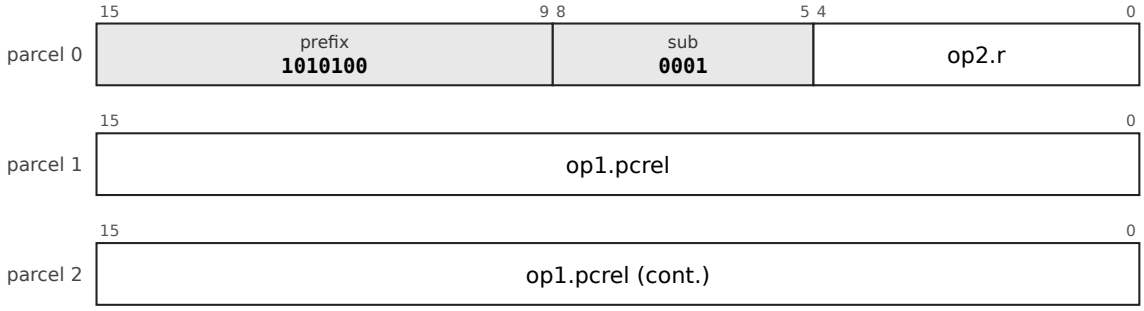
**MOVL: LDF.B28, prefix 011101, sub-opcode 0001; operands b5d28s-x1, reg5; 6 bytes**



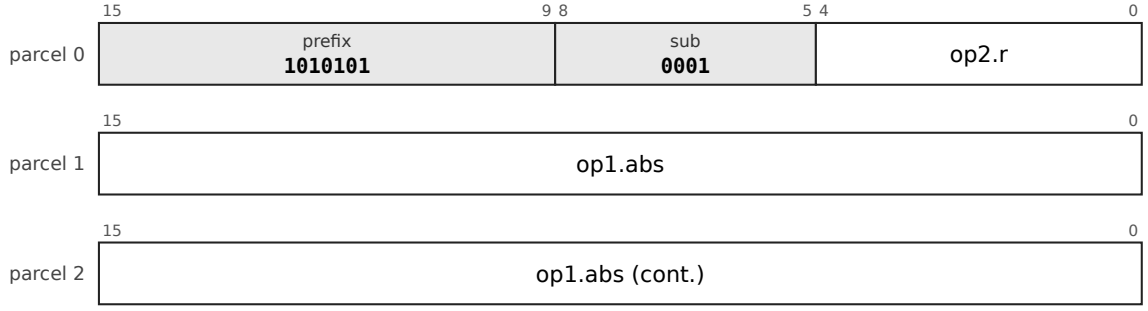

**MOVL: LDF.X22, prefix 01001, sub-opcode 0001; operands x5-5d22s, reg5; 6 bytes**



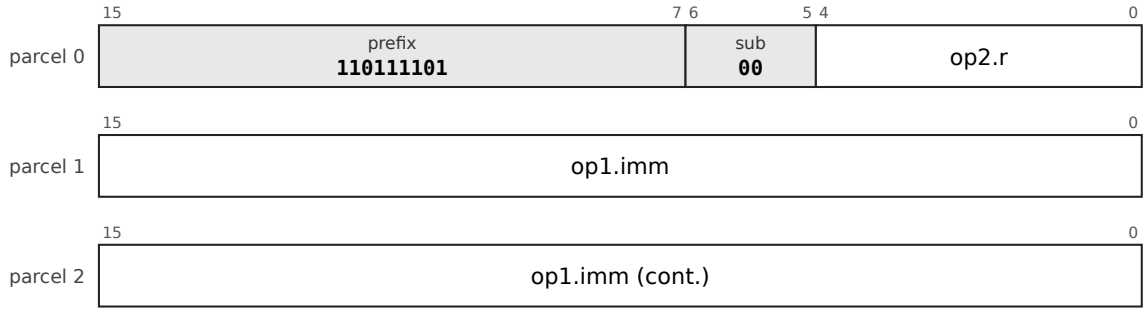

**MOVL: LDF.PC32, prefix 1010100, sub-opcode 0001; operands pc32-u1, reg5; 6 bytes**



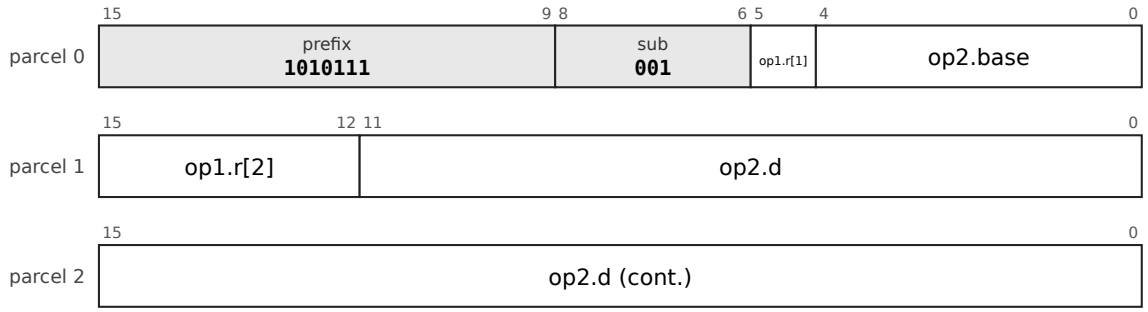
**MOVL: LDF.A32, prefix 1010101, sub-opcode 0001; operands abs32, reg5; 6 bytes**



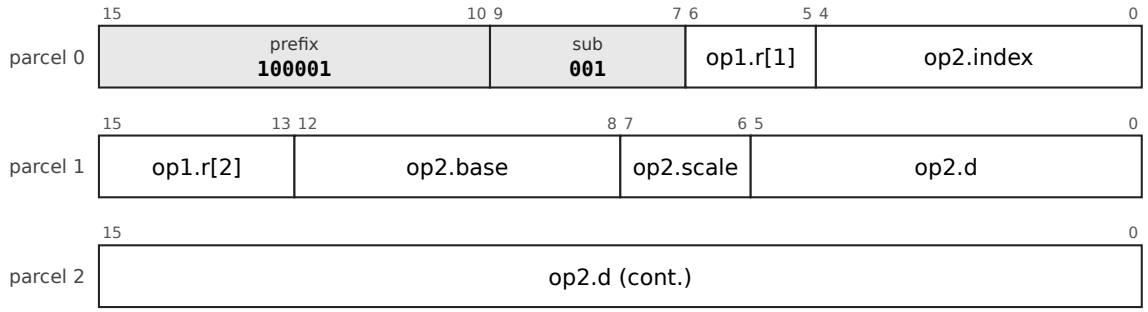

**MOVL: LDFI.I32, prefix 110111101, sub-opcode 00; operands imm32s, reg5; 6 bytes**



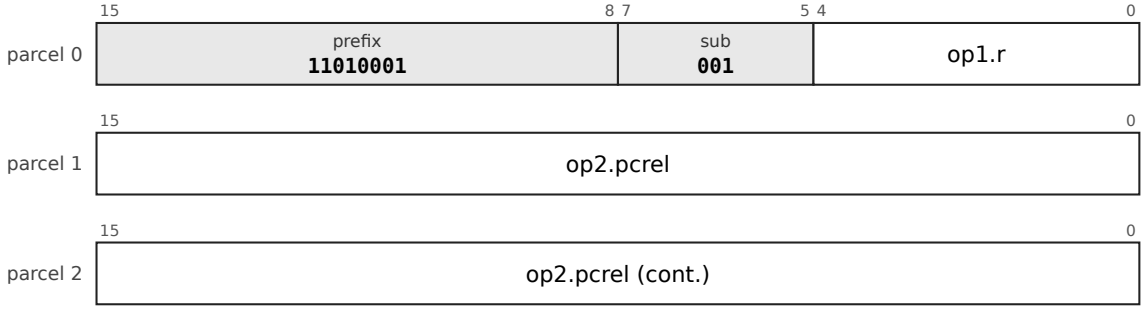

**MOVL: STF.B28, prefix 1010111, sub-opcode 001; operands reg5, b5d28s-x1; 6 bytes**



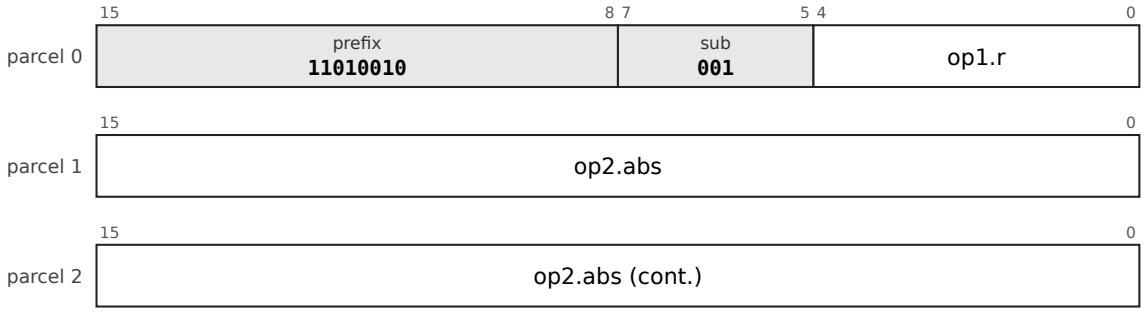

**MOVL: STF.X22, prefix 100001, sub-opcode 001; operands reg5, x5-5d22s; 6 bytes**



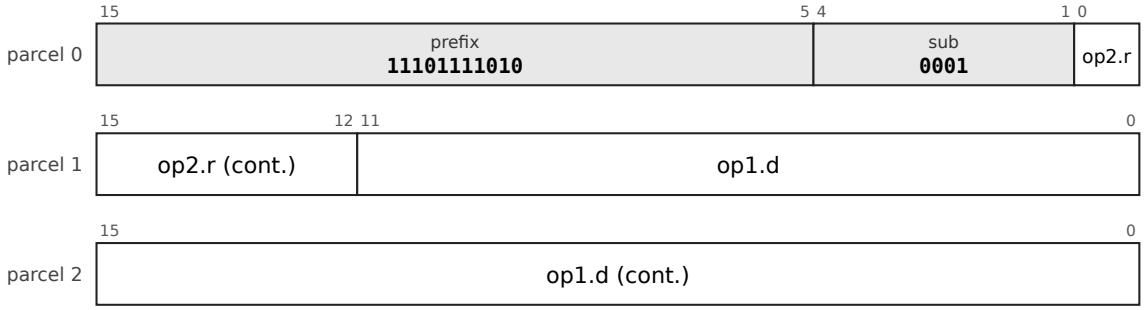
**MOVL: STF.PC32, prefix 11010001, sub-opcode 001; operands reg5, pc32-u1; 6 bytes**



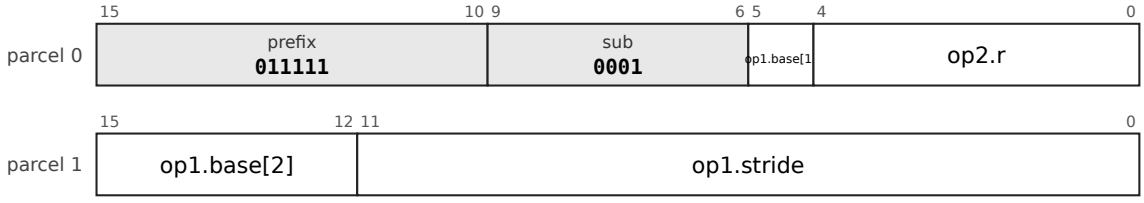

**MOVL: STF.A32, prefix 11010010, sub-opcode 001; operands reg5, abs32; 6 bytes**



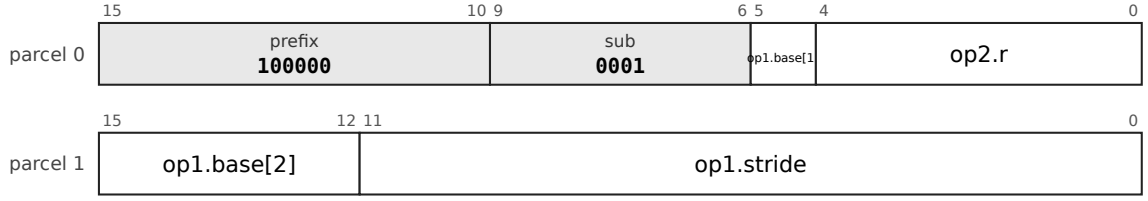

**MOVL: DPF.D28, prefix 11101111010, sub-opcode 0001; operands dpd28s-x1, reg5; 6 bytes**



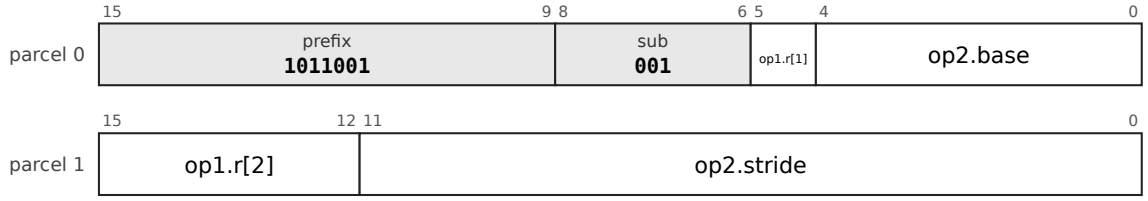

**MOVL: LDF.PI12, prefix 011111, sub-opcode 0001; operands pi5-s12, reg5; 4 bytes**



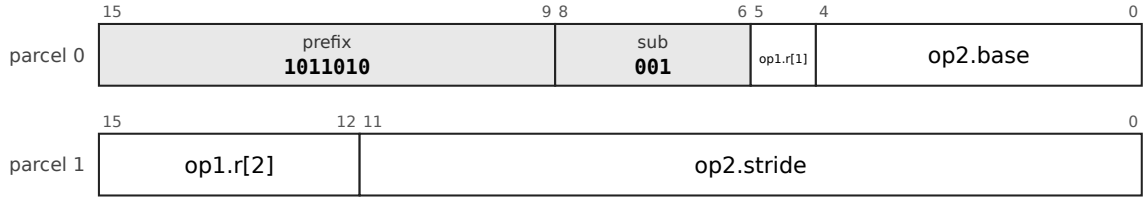
**MOVL: LDF.PD12, prefix 100000, sub-opcode 0001; operands pd5-s12, reg5; 4 bytes**



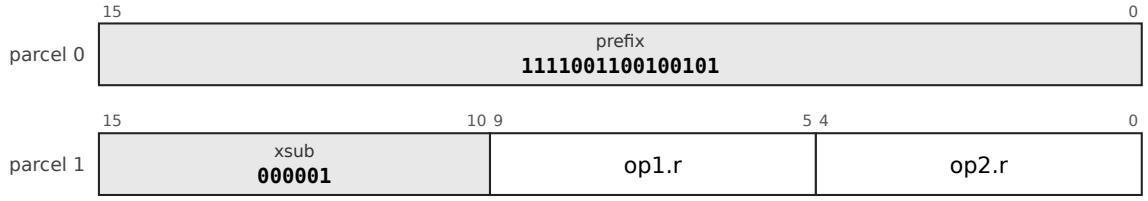

**MOVL: STF.PI12, prefix 1011001, sub-opcode 001; operands reg5, pi5-s12; 4 bytes**



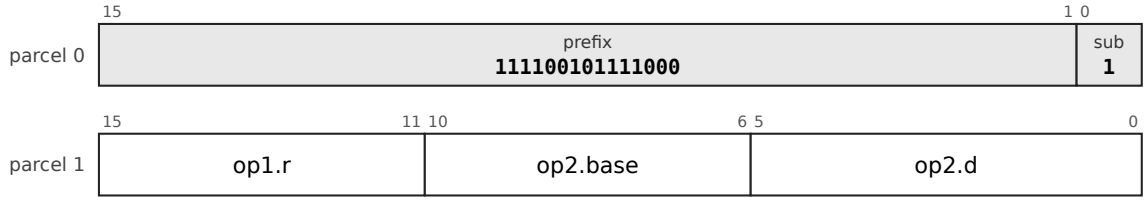

**MOVL: STF.PD12, prefix 1011010, sub-opcode 001; operands reg5, pd5-s12; 4 bytes**



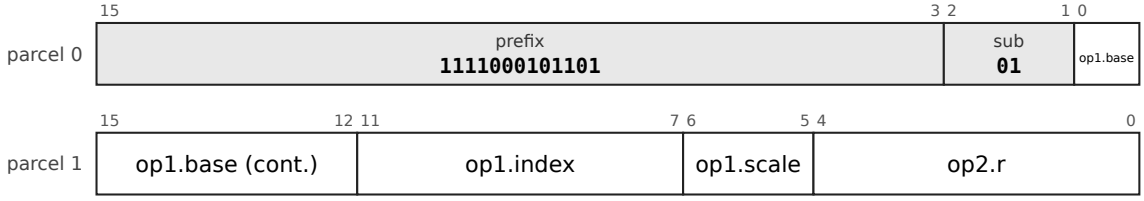

**MOVL: RRF5, prefix 1111001100100101, sub-opcode 000001; operands reg5, reg5; 4 bytes**



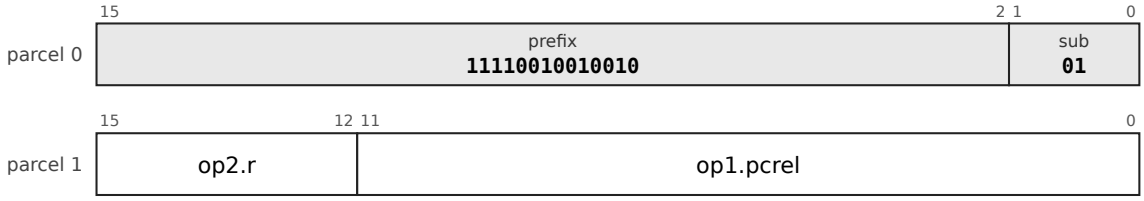

**MOVL: STBu6/a, prefix 111100101111000, sub-opcode 1; operands reg5, b5d6u-xsz; 4 bytes**



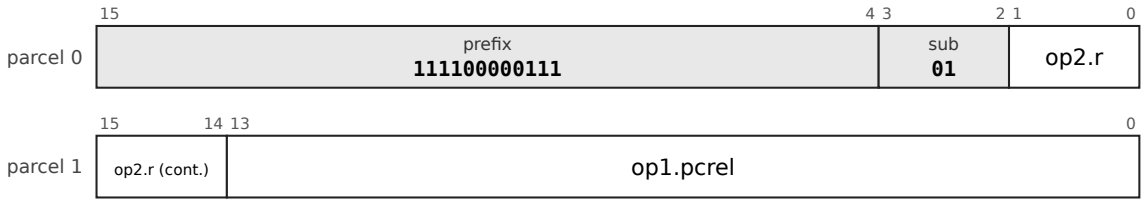
**MOVL: LDX0/a, prefix 1111000101101, sub-opcode 01; operands x5-5d0s, reg5; 4 bytes**



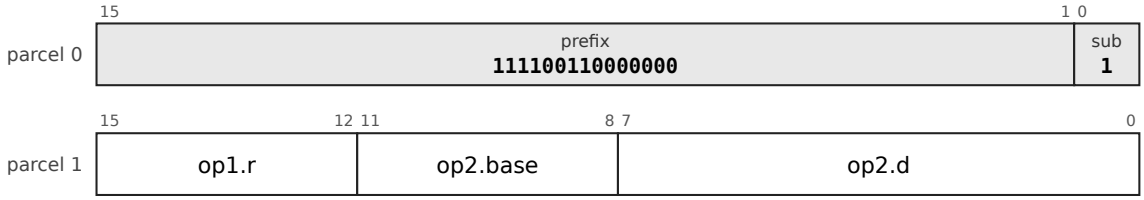

**MOVL: LDPC12H/a, prefix 11110010010010, sub-opcode 01; operands pc12-u1, reg4; 4 bytes**



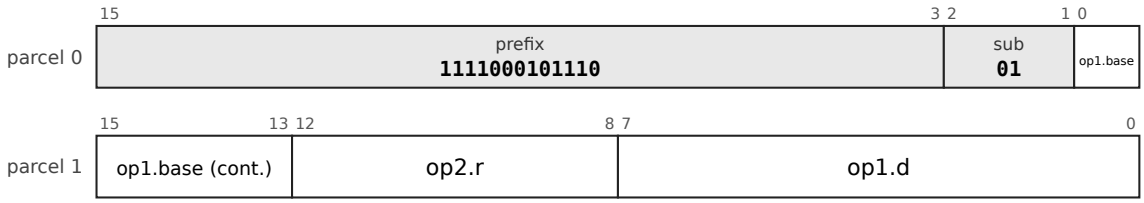

**MOVL: LDPC14H/a, prefix 111100000111, sub-opcode 01; operands pc14-u1, reg4; 4 bytes**



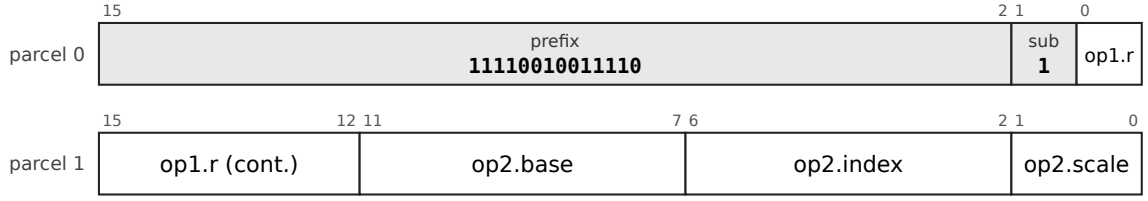

**MOVL: STBs8H/a, prefix 111100110000000, sub-opcode 1; operands reg4, b4d8s-xsz; 4 bytes**



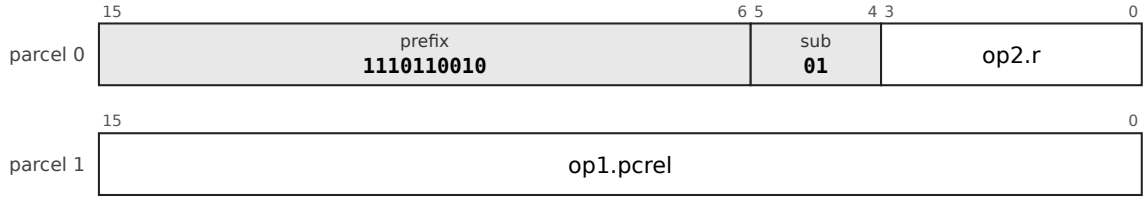

**MOVL: LDBu8H/a, prefix 1111000101110, sub-opcode 01; operands b4d8u-xsz, reg5; 4 bytes**



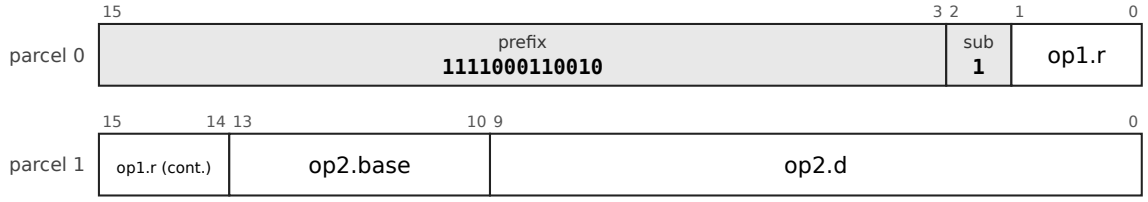
**MOVL: STX0/a, prefix 11110010011110, sub-opcode 1; operands reg5, x5-5d0s; 4 bytes**



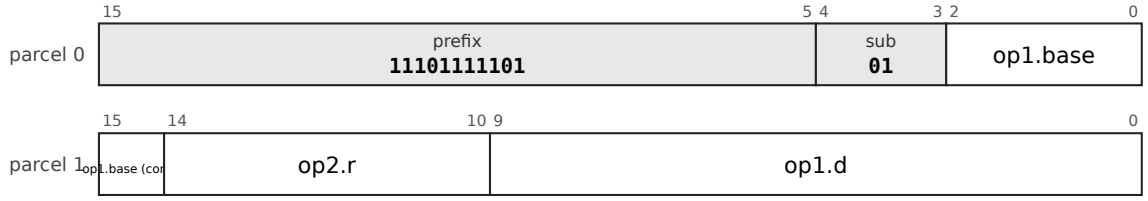

**MOVL: LDPC16H/a, prefix 1110110010, sub-opcode 01; operands pc16-u1, reg4; 4 bytes**



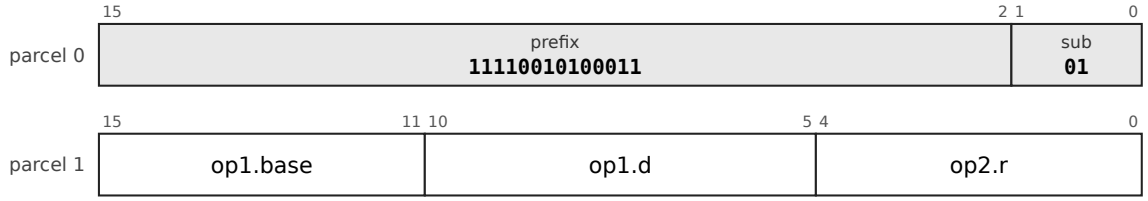

**MOVL: STBu10H/a, prefix 1111000110010, sub-opcode 1; operands reg4, b4d10u-xsz; 4 bytes**



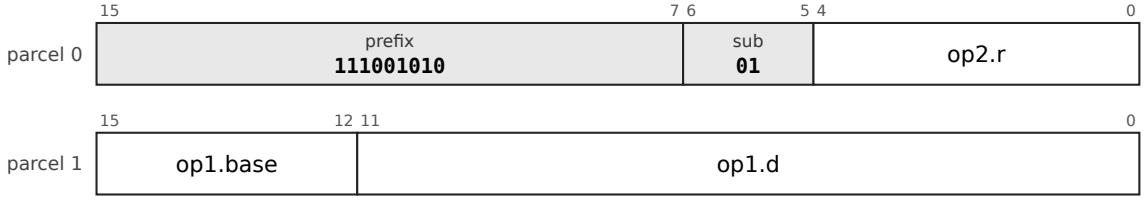

**MOVL: LDBu10H/a, prefix 11101111101, sub-opcode 01; operands b4d10u-xsz, reg5; 4 bytes**



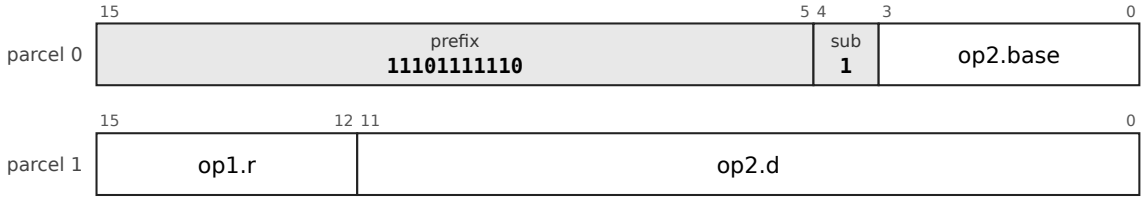

**MOVL: LDBs6/a, prefix 11110010100011, sub-opcode 01; operands b5d6s-xsz, reg5; 4 bytes**



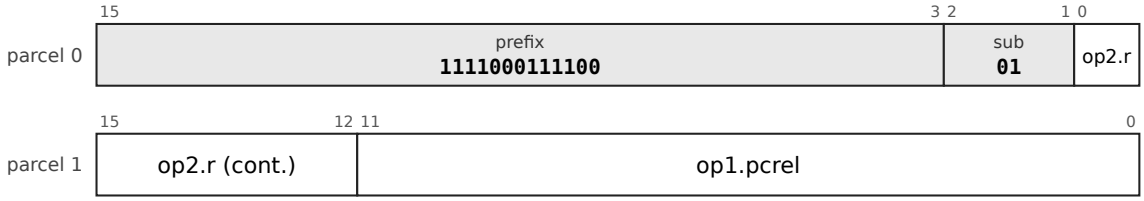
**MOVL: LDBu12H/a, prefix 111001010, sub-opcode 01; operands b4d12u-xsz, reg5; 4 bytes**



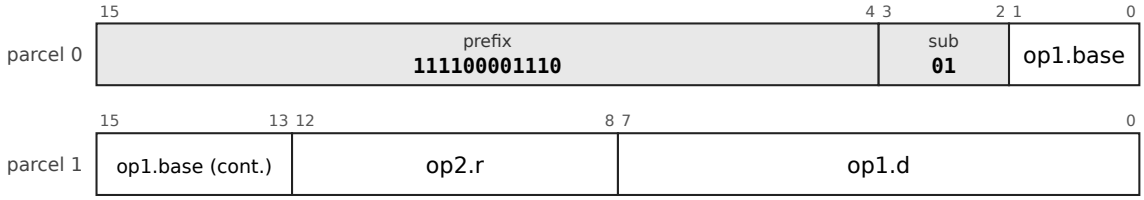

**MOVL: STBu12H/a, prefix 11101111110, sub-opcode 1; operands reg4, b4d12u-xsz; 4 bytes**



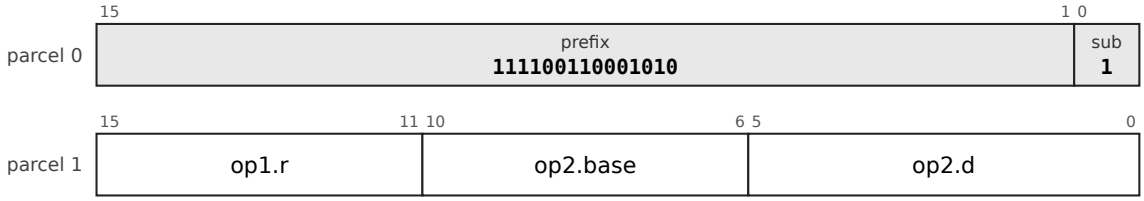

**MOVL: LDPC12/a, prefix 1111000111100, sub-opcode 01; operands pc12-u1, reg5; 4 bytes**



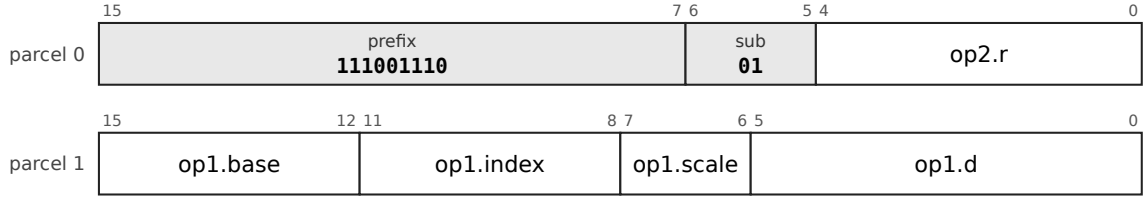

**MOVL: LDBu8/a, prefix 111100001110, sub-opcode 01; operands b5d8u-xsz, reg5; 4 bytes**



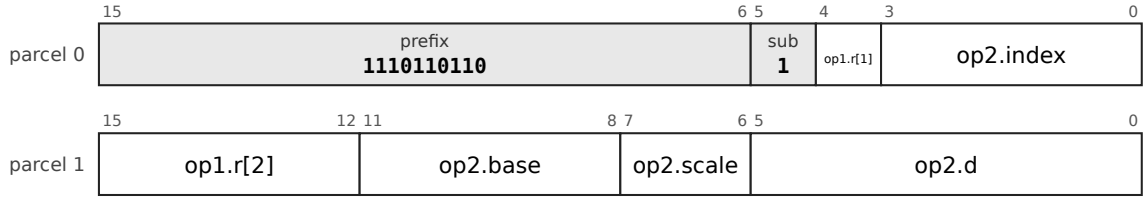

**MOVL: STBs6/a, prefix 111100110001010, sub-opcode 1; operands reg5, b5d6s-xsz; 4 bytes**



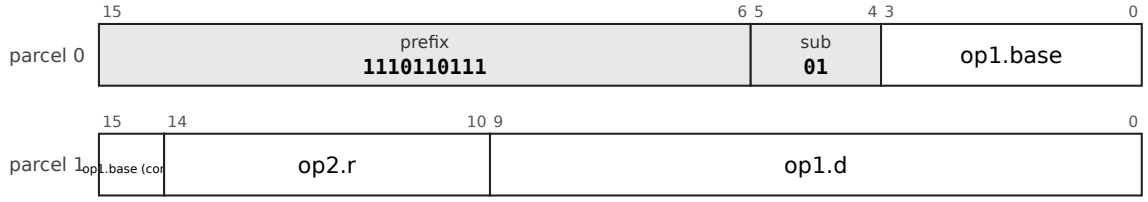
**MOVL: LDX6H/a, prefix 111001110, sub-opcode 01; operands x4-4d6s, reg5; 4 bytes**



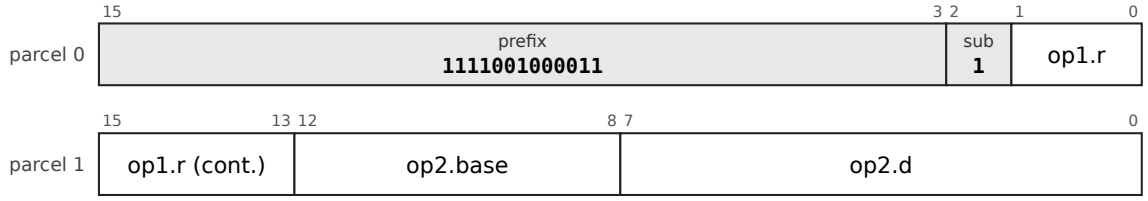

**MOVL: STX6H/a, prefix 1110110110, sub-opcode 1; operands reg5, x4-4d6s; 4 bytes**



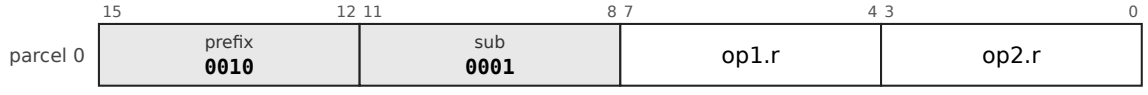

**MOVL: LDBs10/a, prefix 1110110111, sub-opcode 01; operands b5d10s-xsz, reg5; 4 bytes**



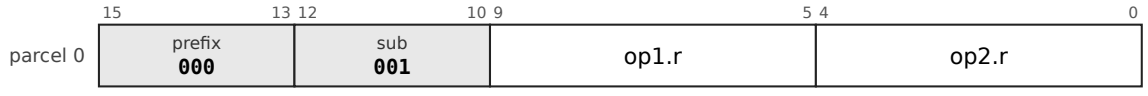

**MOVL: STBs8/a, prefix 1111001000011, sub-opcode 1; operands reg5, b5d8s-xsz; 4 bytes**



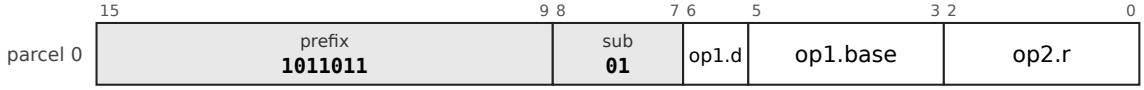

**MOVL: RRF, prefix 0010, sub-opcode 0001; operands reg4, reg4; 2 bytes**



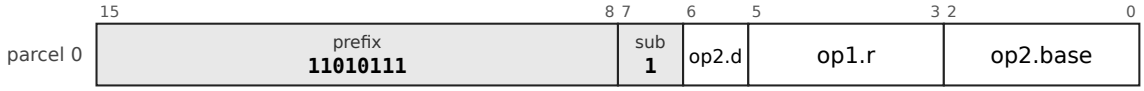

**MOVL: RR5, prefix 000, sub-opcode 001; operands reg5, reg5; 2 bytes**



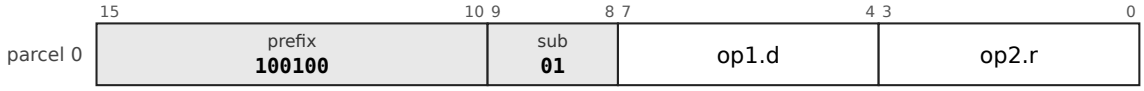
**MOVL: LDHHH1/a, prefix 1011011, sub-opcode 01; operands b3d1u-xsz, reg3; 2 bytes**



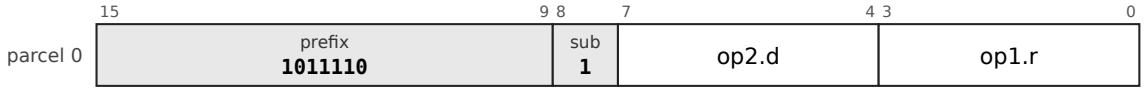

**MOVL: STHHH1/a, prefix 11010111, sub-opcode 1; operands reg3, b3d1u-xsz; 2 bytes**



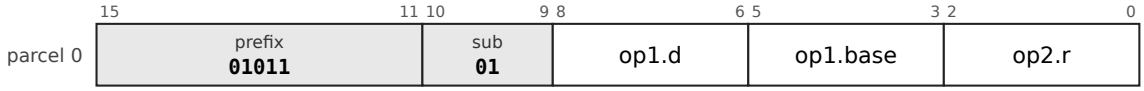

**MOVL: LDSPh4/a, prefix 100100, sub-opcode 01; operands spd4u-xsz, reg4; 2 bytes**




**MOVL: STSPh4/a, prefix 1011110, sub-opcode 1; operands reg4, spd4u-xsz; 2 bytes**




**MOVL: LDHHH3/a, prefix 01011, sub-opcode 01; operands b3d3u-xsz, reg3; 2 bytes**

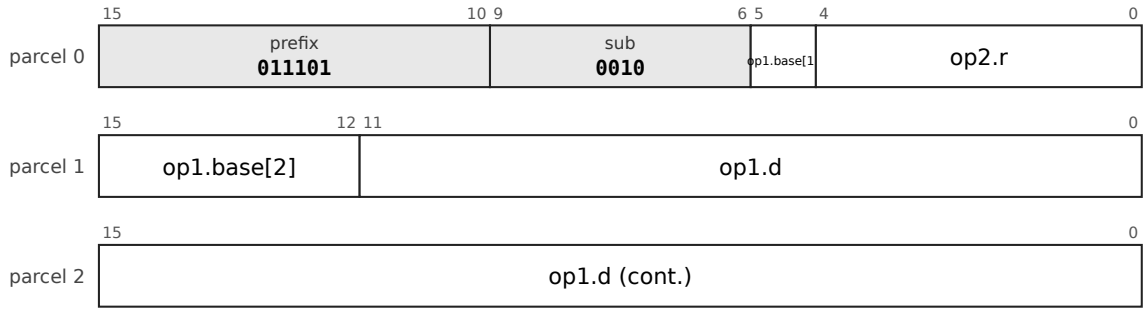



```
MOVL, MOVW, MOVB likewise
```

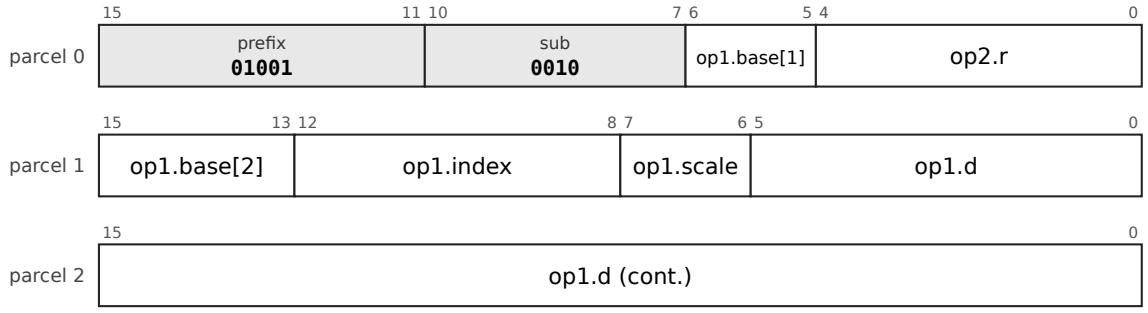
Operation, flags and exceptions as for the entry MOV above; 42 layout forms, each a chart of its own (above).

## *MOVW*

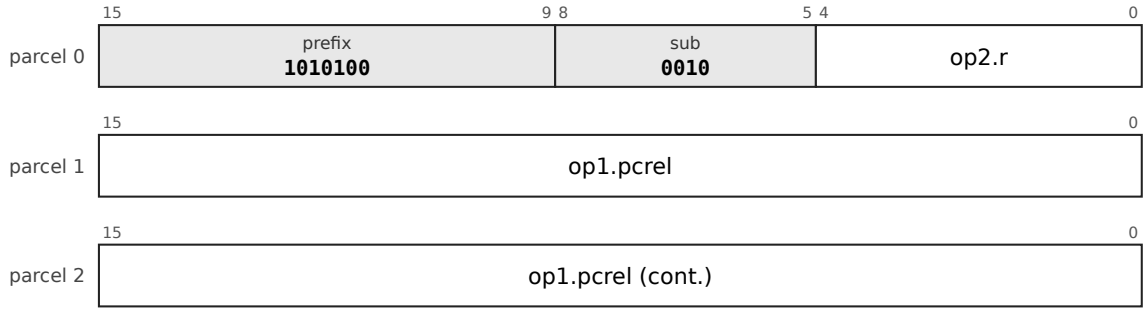
**MOVW: LDF.B28, prefix 011101, sub-opcode 0010; operands b5d28s-x1, reg5; 6 bytes**



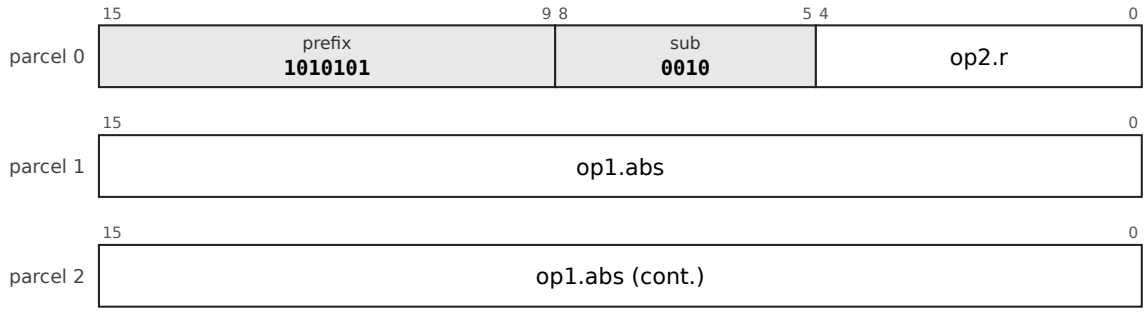

**MOVW: LDF.X22, prefix 01001, sub-opcode 0010; operands x5-5d22s, reg5; 6 bytes**



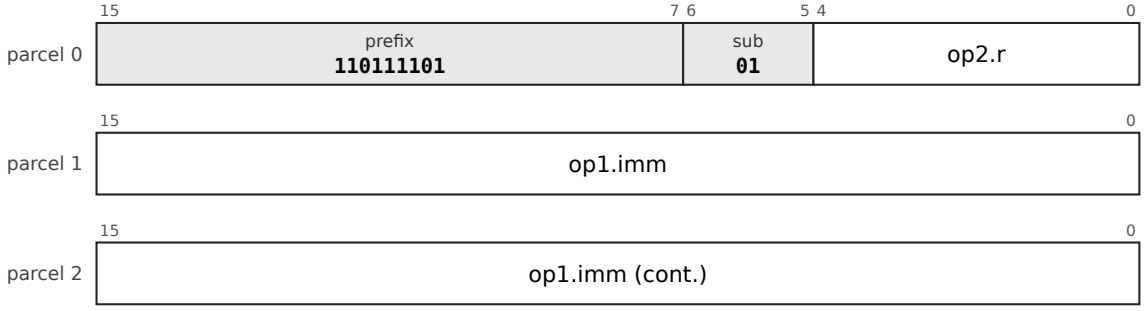

**MOVW: LDF.PC32, prefix 1010100, sub-opcode 0010; operands pc32-u1, reg5; 6 bytes**



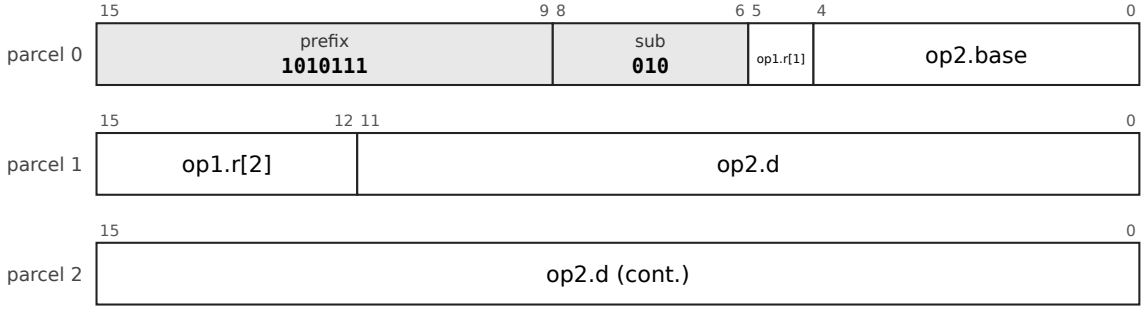

**MOVW: LDF.A32, prefix 1010101, sub-opcode 0010; operands abs32, reg5; 6 bytes**



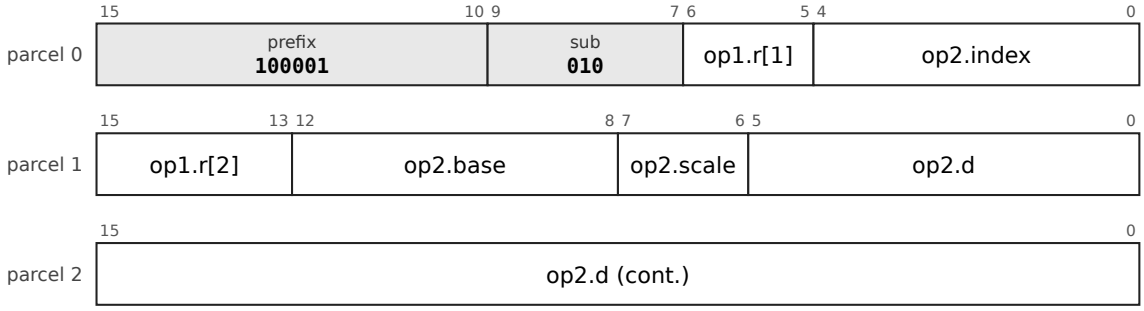
**MOVW: LDFI.I32, prefix 110111101, sub-opcode 01; operands imm32s, reg5; 6 bytes**



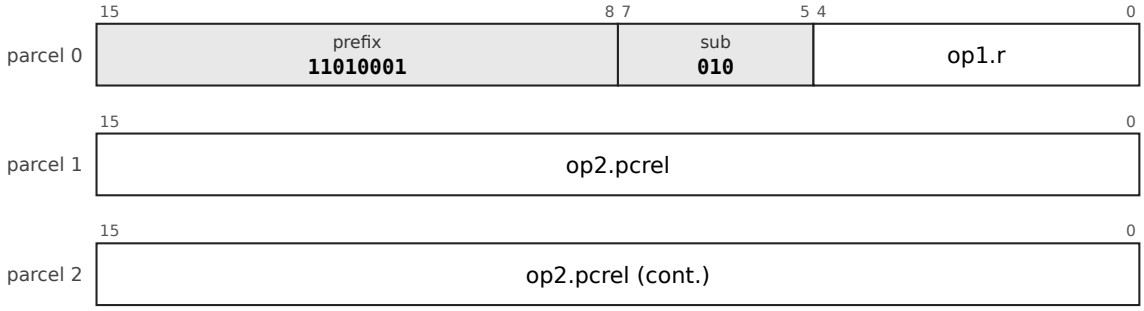

**MOVW: STF.B28, prefix 1010111, sub-opcode 010; operands reg5, b5d28s-x1; 6 bytes**



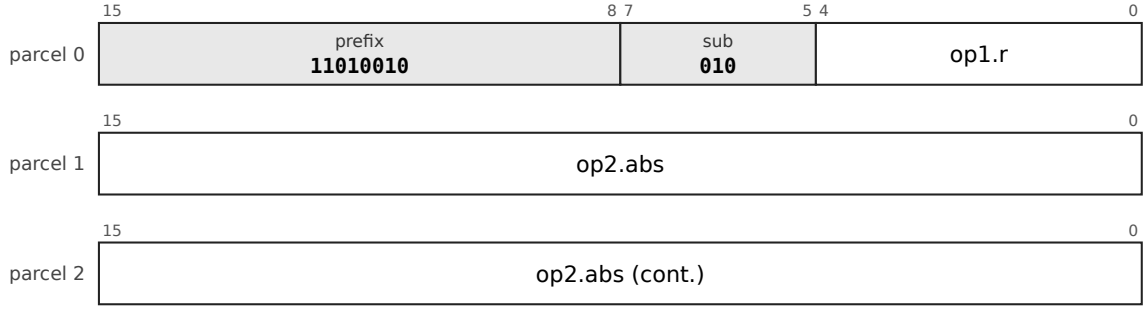

**MOVW: STF.X22, prefix 100001, sub-opcode 010; operands reg5, x5-5d22s; 6 bytes**



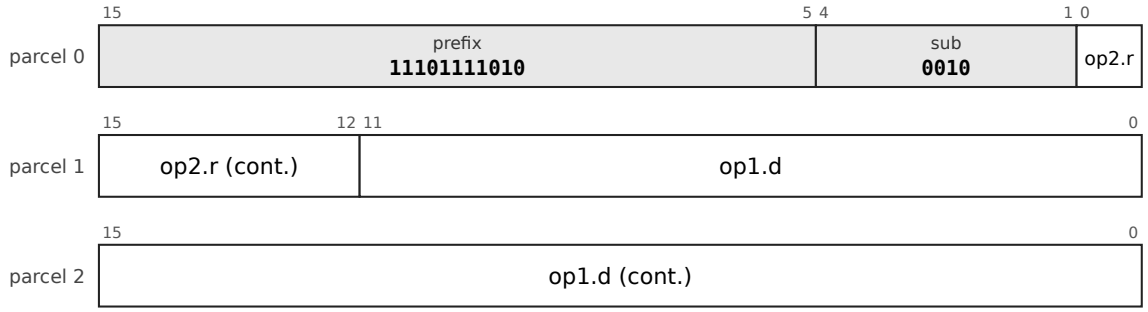

**MOVW: STF.PC32, prefix 11010001, sub-opcode 010; operands reg5, pc32-u1; 6 bytes**



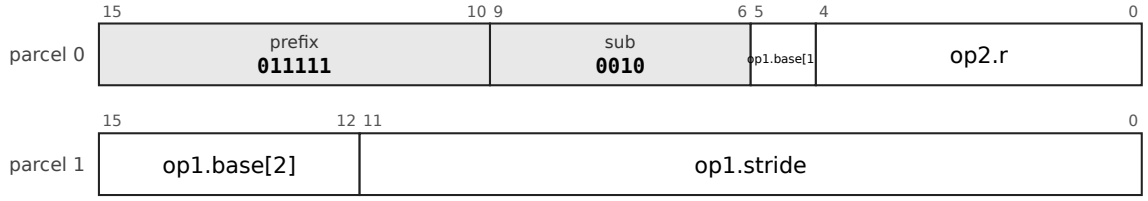
**MOVW: STF.A32, prefix 11010010, sub-opcode 010; operands reg5, abs32; 6 bytes**



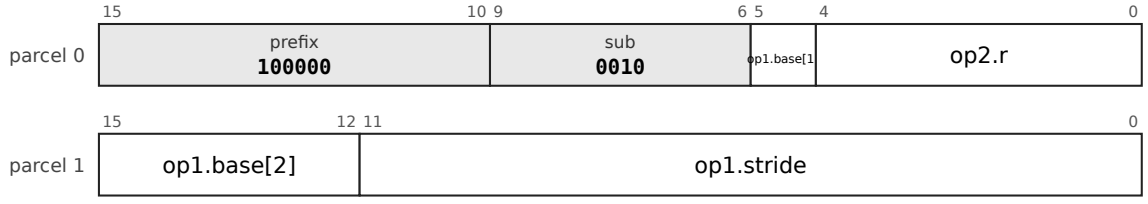

**MOVW: DPF.D28, prefix 11101111010, sub-opcode 0010; operands dpd28s-x1, reg5; 6 bytes**



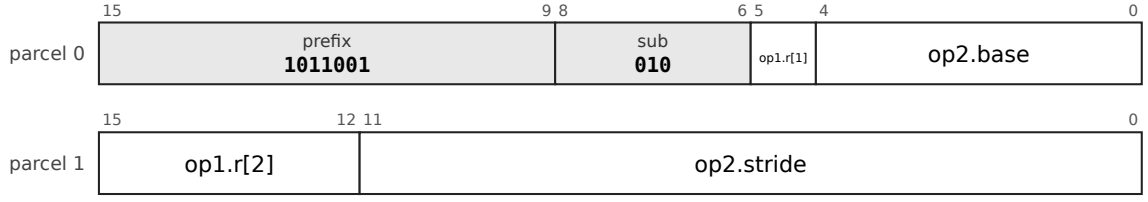

**MOVW: LDF.PI12, prefix 011111, sub-opcode 0010; operands pi5-s12, reg5; 4 bytes**



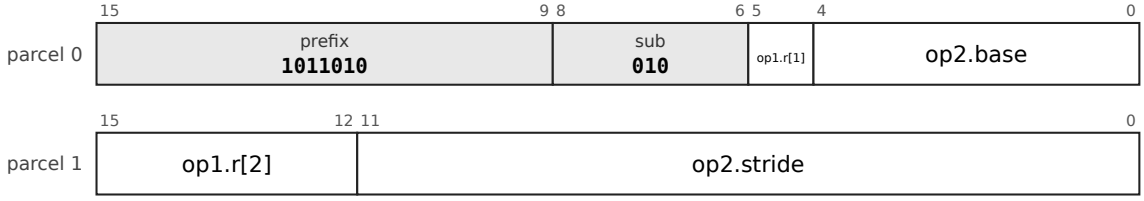

**MOVW: LDF.PD12, prefix 100000, sub-opcode 0010; operands pd5-s12, reg5; 4 bytes**



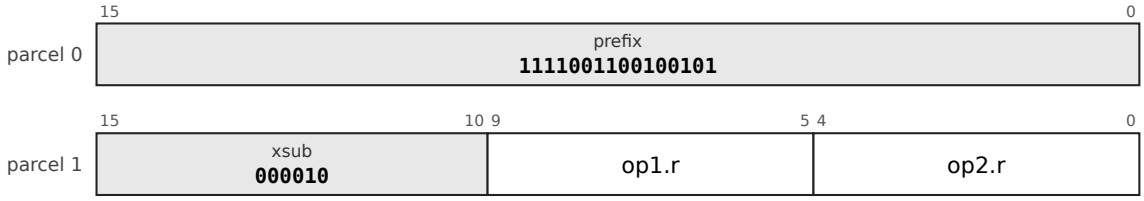

**MOVW: STF.PI12, prefix 1011001, sub-opcode 010; operands reg5, pi5-s12; 4 bytes**



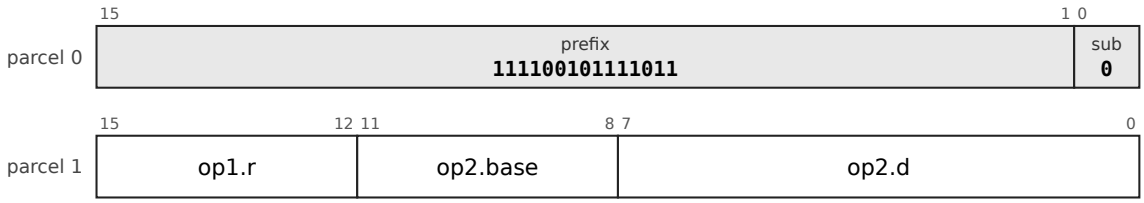
**MOVW: STF.PD12, prefix 1011010, sub-opcode 010; operands reg5, pd5-s12; 4 bytes**



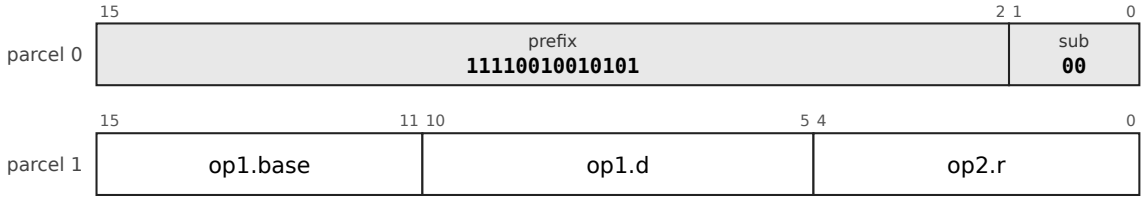

**MOVW: RRF5, prefix 1111001100100101, sub-opcode 000010; operands reg5, reg5; 4 bytes**



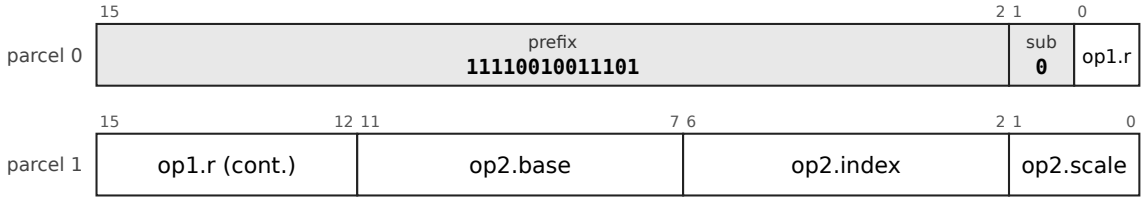

**MOVW: STBu8H/b, prefix 111100101111011, sub-opcode 0; operands reg4, b4d8u-xsz; 4 byte**



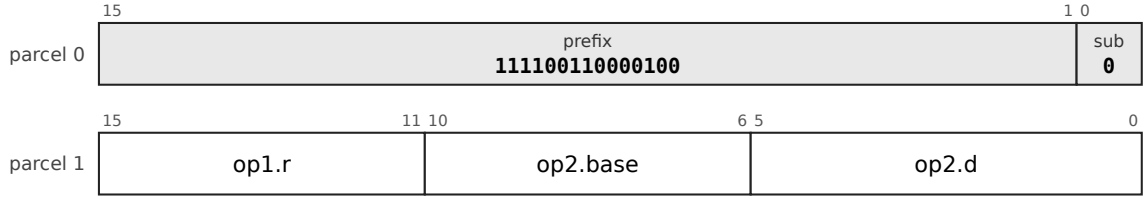

**MOVW: LDBu6/b, prefix 11110010010101, sub-opcode 00; operands b5d6u-xsz, reg5; 4 bytes**



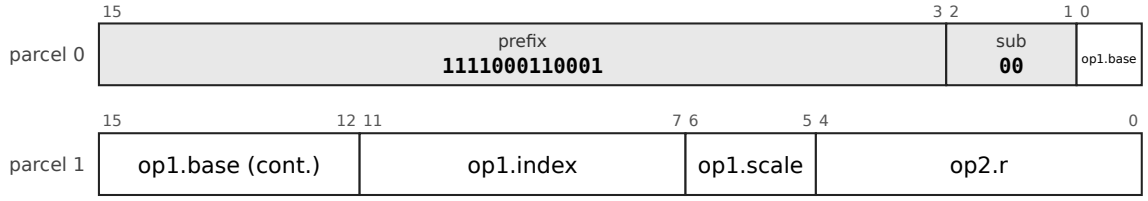

**MOVW: STX0/b, prefix 11110010011101, sub-opcode 0; operands reg5, x5-5d0s; 4 bytes**



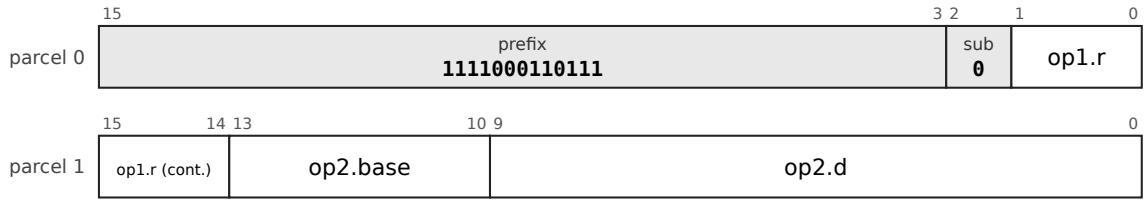
**MOVW: STBs6/b, prefix 111100110000100, sub-opcode 0; operands reg5, b5d6s-xsz; 4 bytes**



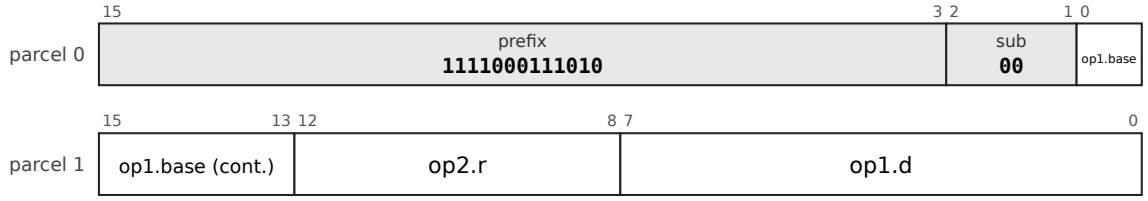

**MOVW: LDX0/b, prefix 1111000110001, sub-opcode 00; operands x5-5d0s, reg5; 4 bytes**



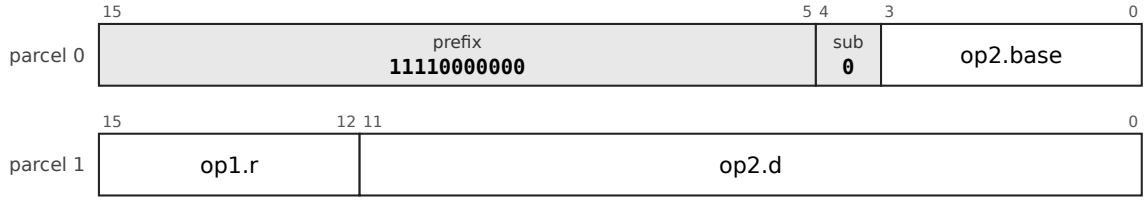

**MOVW: STBu10H/b, prefix 1111000110111, sub-opcode 0; operands reg4, b4d10u-xsz; 4 byte**



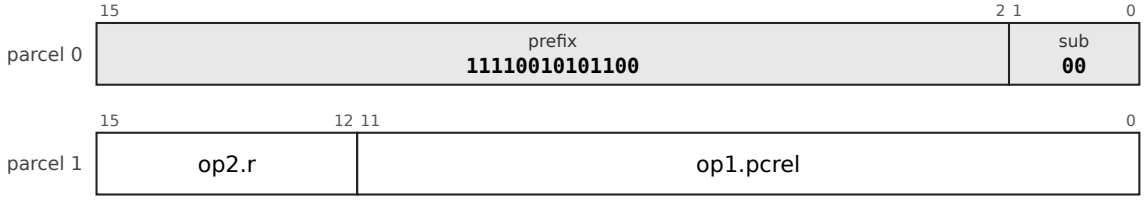

**MOVW: LDBu8H/b, prefix 1111000111010, sub-opcode 00; operands b4d8u-xsz, reg5; 4 bytes**



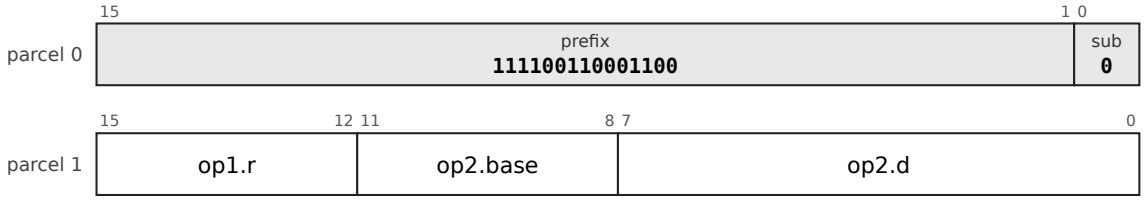

**MOVW: STBu12H/b, prefix 11110000000, sub-opcode 0; operands reg4, b4d12u-xsz; 4 bytes**



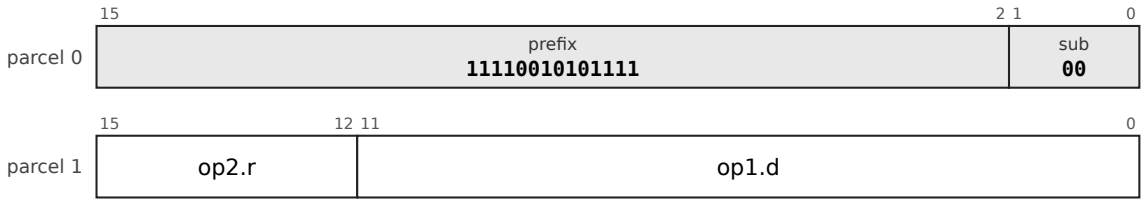
**MOVW: LDPC12H/b, prefix 11110010101100, sub-opcode 00; operands pc12-u1, reg4; 4 bytes**



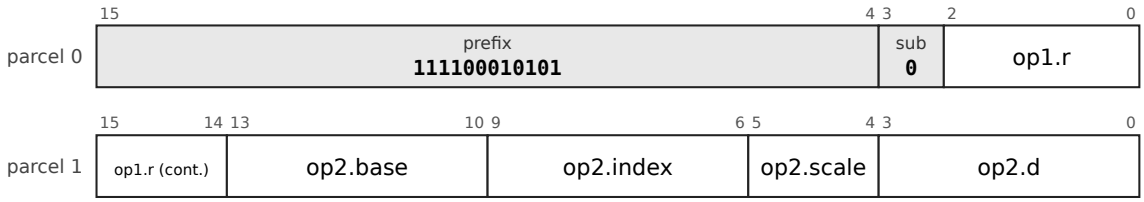

**MOVW: STBs8H/b, prefix 111100110001100, sub-opcode 0; operands reg4, b4d8s-xsz; 4 byte**



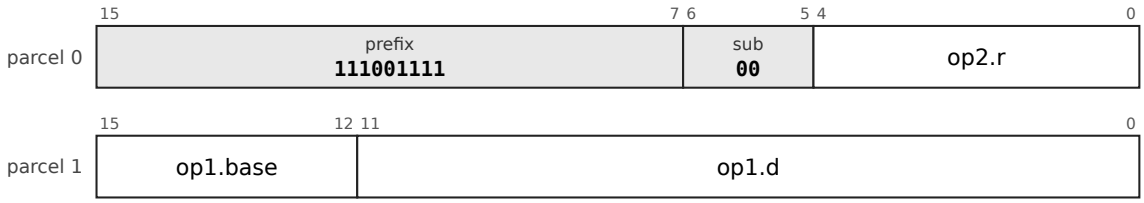

**MOVW: LDS12H/b, prefix 11110010101111, sub-opcode 00; operands spd12u-xsz, reg4; 4 byt**



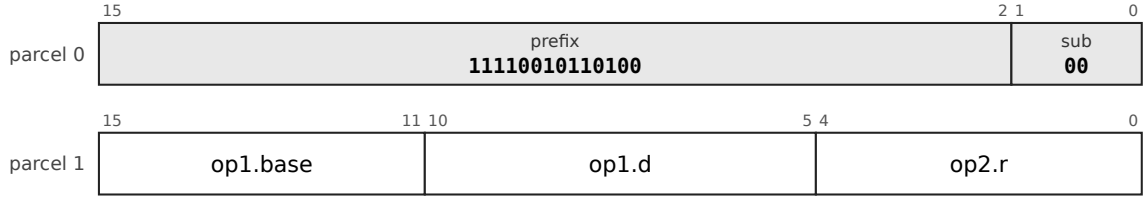

**MOVW: STX4H/b, prefix 111100010101, sub-opcode 0; operands reg5, x4-4d4s; 4 bytes**



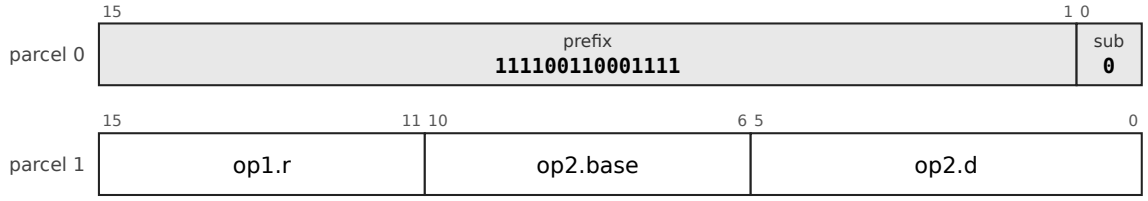

**MOVW: LDBu12H/b, prefix 111001111, sub-opcode 00; operands b4d12u-xsz, reg5; 4 bytes**



**MOVW: LDBs6/b, prefix 11110010110100, sub-opcode 00; operands b5d6s-xsz, reg5; 4 bytes**



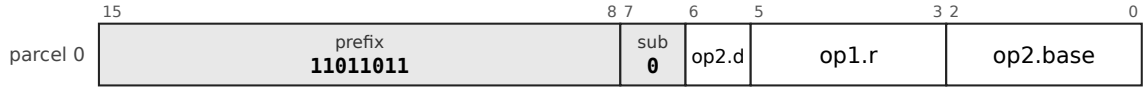

**MOVW: STBu6/b, prefix 111100110001111, sub-opcode 0; operands reg5, b5d6u-xsz; 4 bytes**




**MOVW: RRF, prefix 0010, sub-opcode 0010; operands reg4, reg4; 2 bytes**



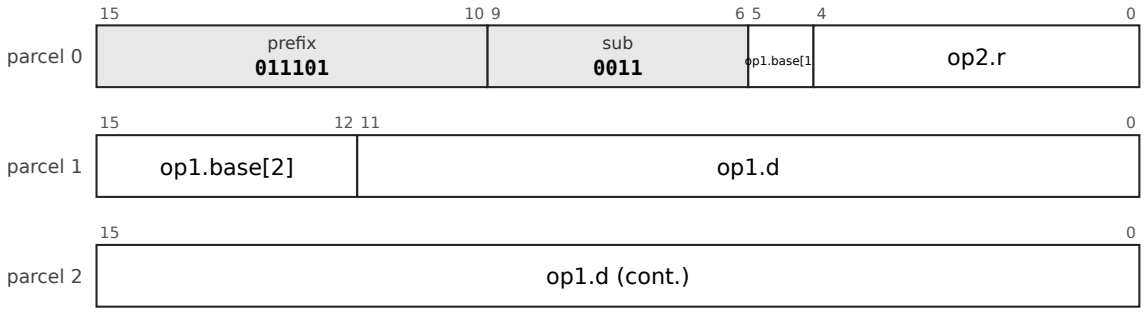

**MOVW: STHHH1/b, prefix 11011011, sub-opcode 0; operands reg3, b3d1u-xsz; 2 bytes**



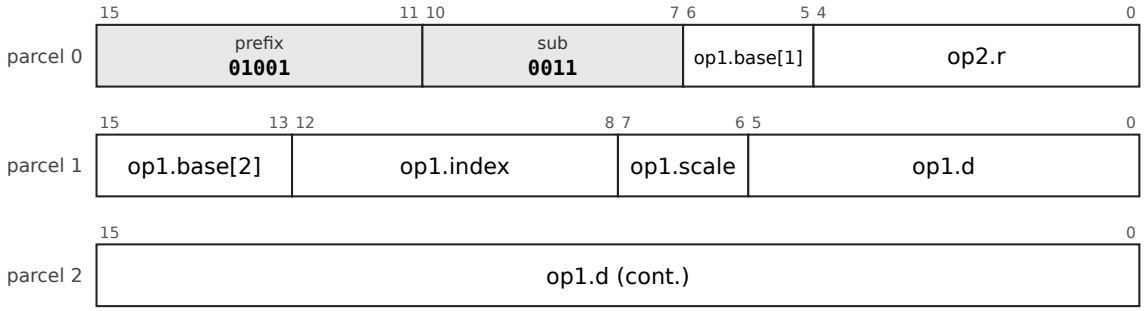

Operation, flags and exceptions as for the entry MOV above; 32 layout forms, each a chart of its own (above).

## *MOVB*

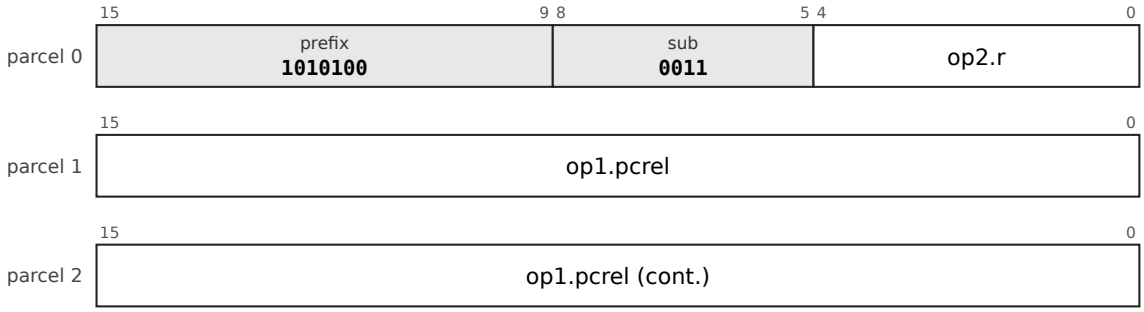
**MOVB: LDF.B28, prefix 011101, sub-opcode 0011; operands b5d28s-x1, reg5; 6 bytes**



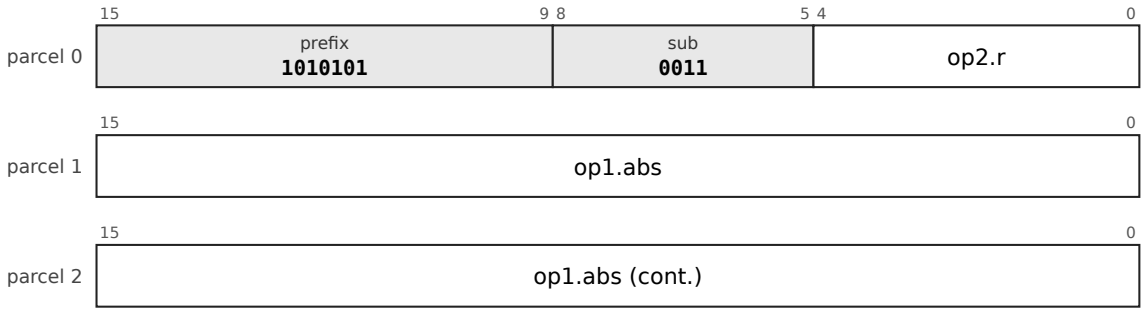

**MOVB: LDF.X22, prefix 01001, sub-opcode 0011; operands x5-5d22s, reg5; 6 bytes**



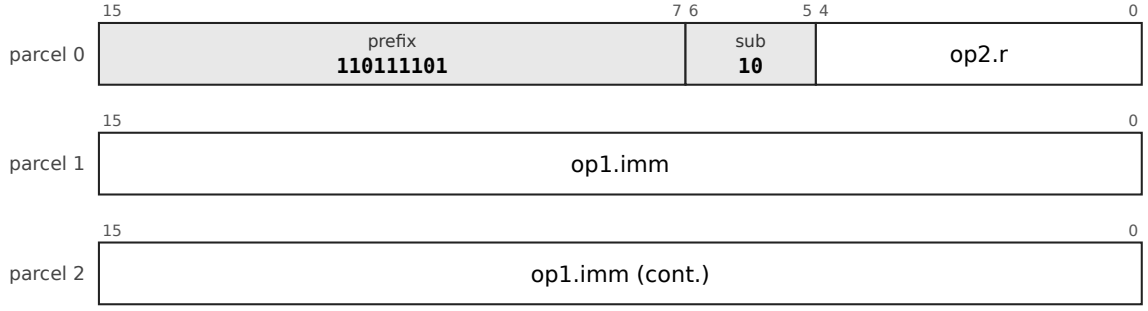

**MOVB: LDF.PC32, prefix 1010100, sub-opcode 0011; operands pc32-u1, reg5; 6 bytes**



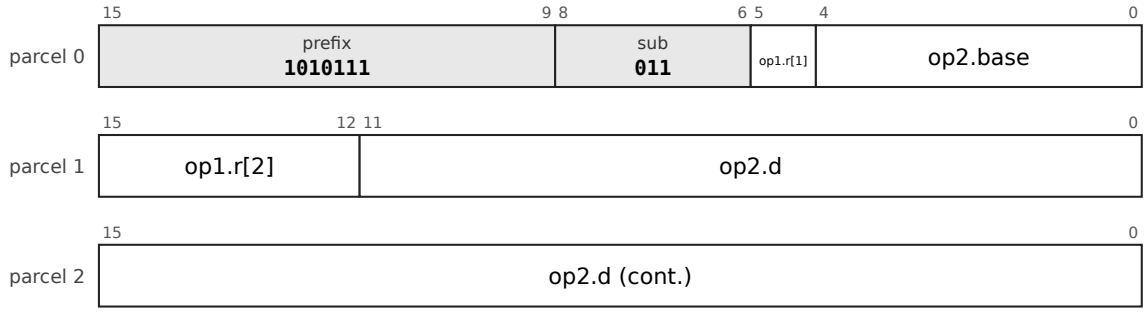

**MOVB: LDF.A32, prefix 1010101, sub-opcode 0011; operands abs32, reg5; 6 bytes**



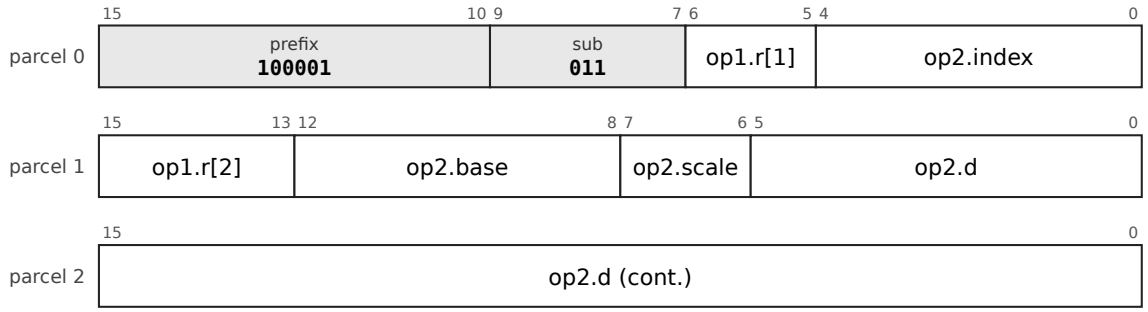
**MOVB: LDFI.I32, prefix 110111101, sub-opcode 10; operands imm32s, reg5; 6 bytes**



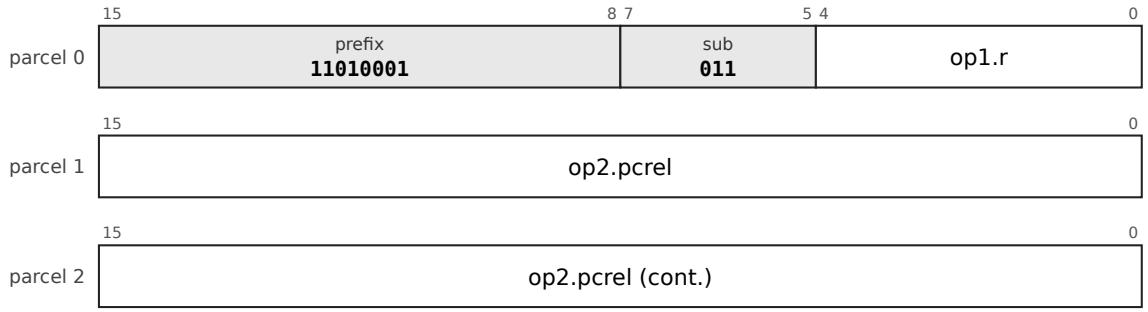

**MOVB: STF.B28, prefix 1010111, sub-opcode 011; operands reg5, b5d28s-x1; 6 bytes**



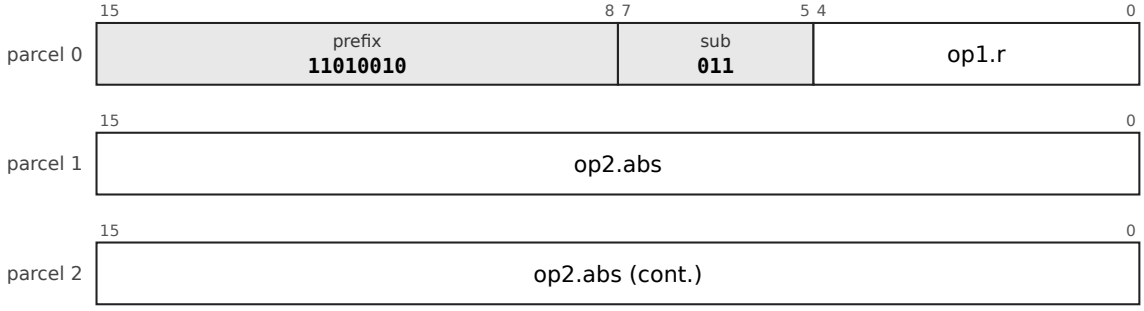

**MOVB: STF.X22, prefix 100001, sub-opcode 011; operands reg5, x5-5d22s; 6 bytes**



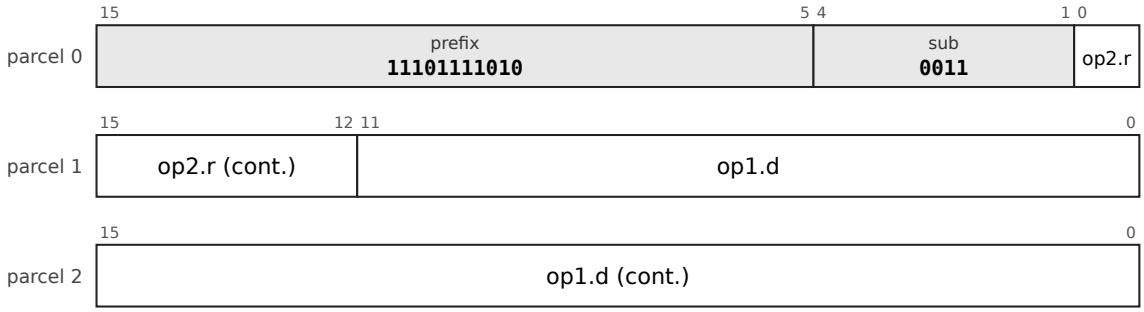

**MOVB: STF.PC32, prefix 11010001, sub-opcode 011; operands reg5, pc32-u1; 6 bytes**



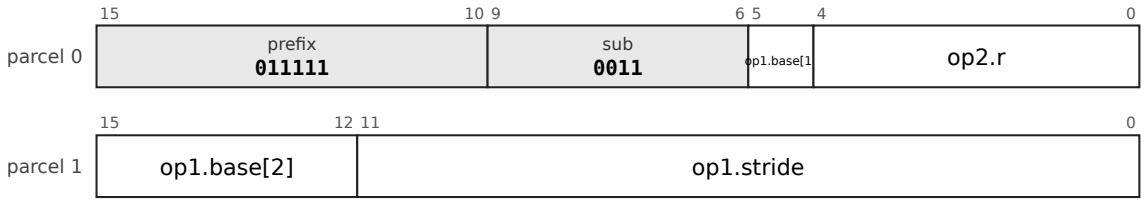
**MOVB: STF.A32, prefix 11010010, sub-opcode 011; operands reg5, abs32; 6 bytes**



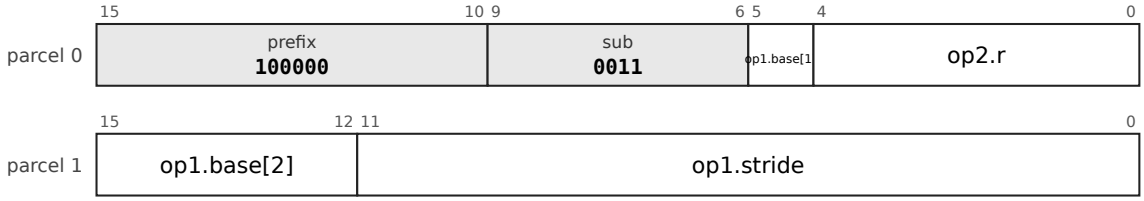

**MOVB: DPF.D28, prefix 11101111010, sub-opcode 0011; operands dpd28s-x1, reg5; 6 bytes**



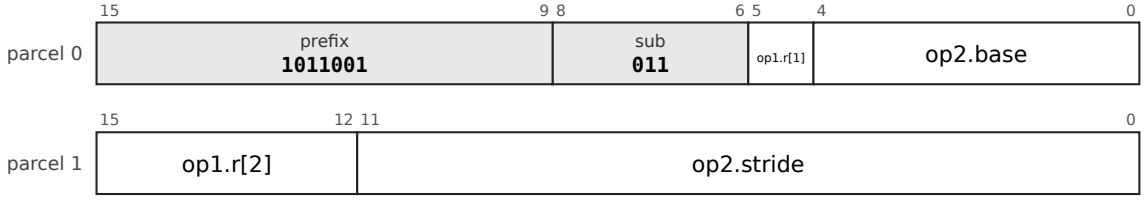

**MOVB: LDF.PI12, prefix 011111, sub-opcode 0011; operands pi5-s12, reg5; 4 bytes**



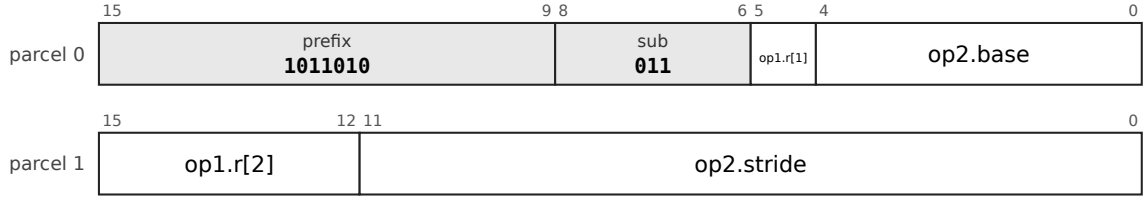

**MOVB: LDF.PD12, prefix 100000, sub-opcode 0011; operands pd5-s12, reg5; 4 bytes**



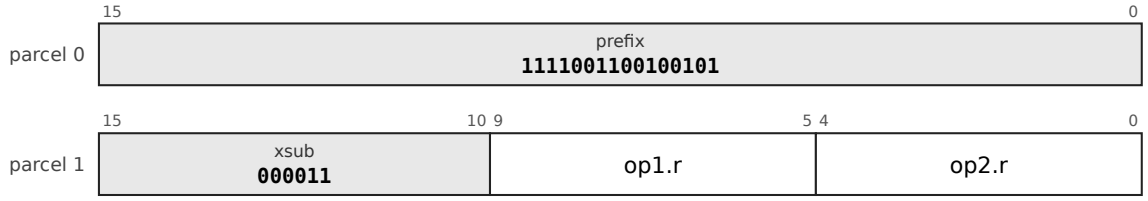

**MOVB: STF.PI12, prefix 1011001, sub-opcode 011; operands reg5, pi5-s12; 4 bytes**



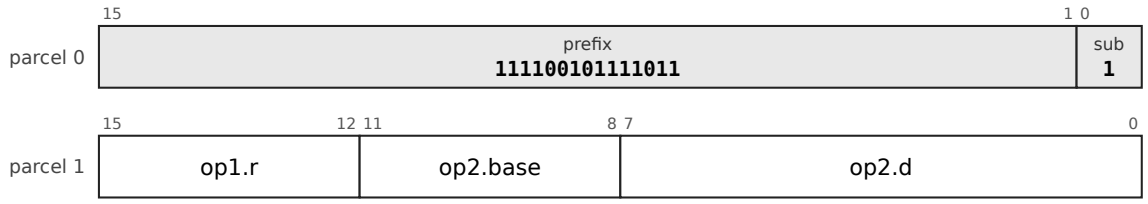
**MOVB: STF.PD12, prefix 1011010, sub-opcode 011; operands reg5, pd5-s12; 4 bytes**



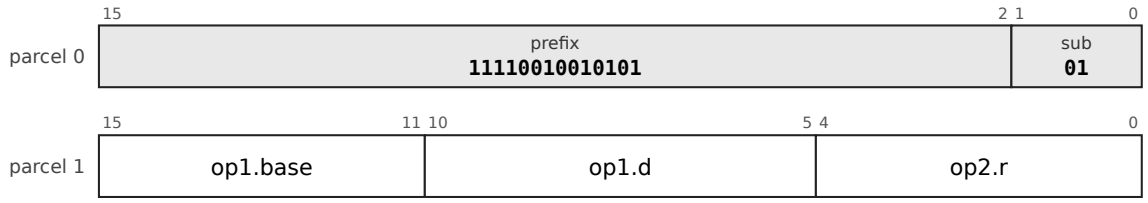

**MOVB: RRF5, prefix 1111001100100101, sub-opcode 000011; operands reg5, reg5; 4 bytes**



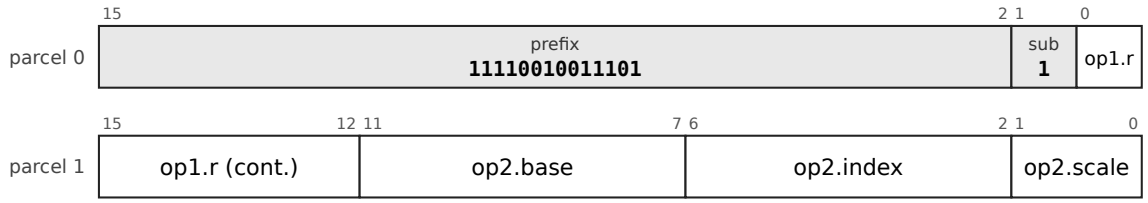

**MOVB: STBu8H/b, prefix 111100101111011, sub-opcode 1; operands reg4, b4d8u-xsz; 4 byte**



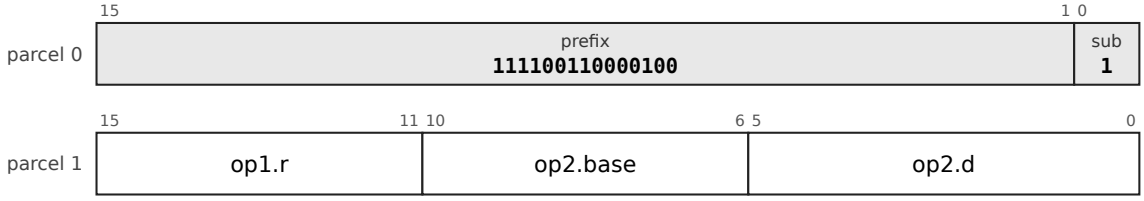

**MOVB: LDBu6/b, prefix 11110010010101, sub-opcode 01; operands b5d6u-xsz, reg5; 4 bytes**



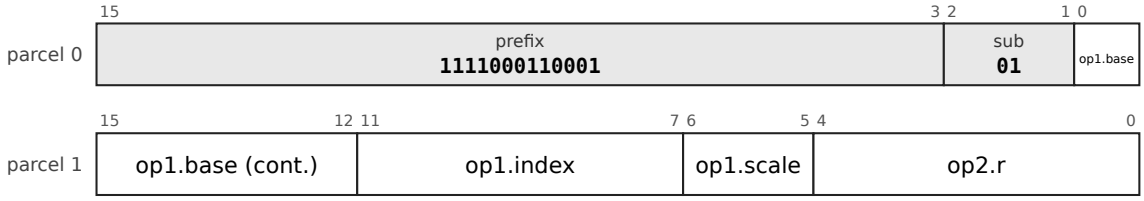

**MOVB: STX0/b, prefix 11110010011101, sub-opcode 1; operands reg5, x5-5d0s; 4 bytes**



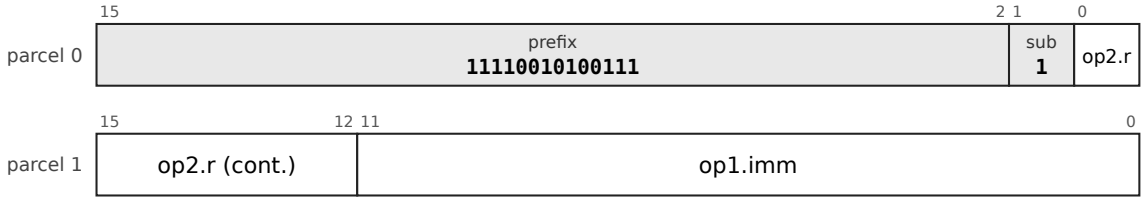
**MOVB: STBs6/b, prefix 111100110000100, sub-opcode 1; operands reg5, b5d6s-xsz; 4 bytes**



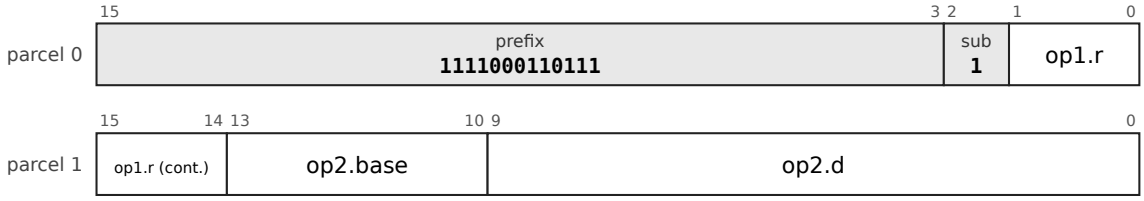

**MOVB: LDX0/b, prefix 1111000110001, sub-opcode 01; operands x5-5d0s, reg5; 4 bytes**



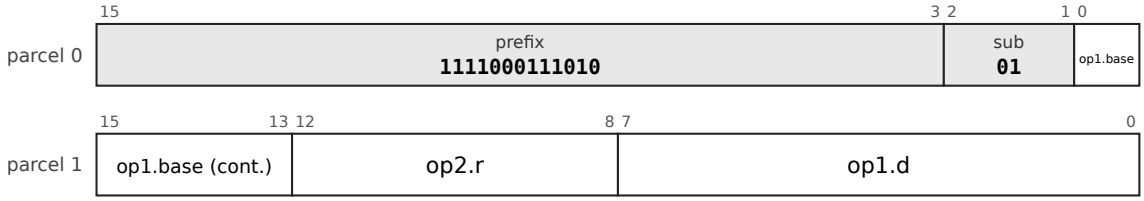

**MOVB: LDI12/d, prefix 11110010100111, sub-opcode 1; operands imm12s, reg5; 4 bytes**



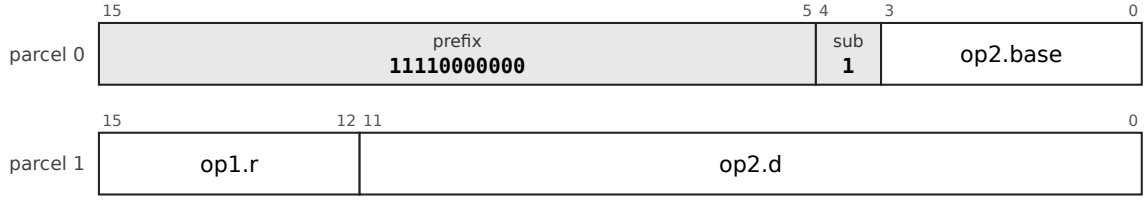

**MOVB: STBu10H/b, prefix 1111000110111, sub-opcode 1; operands reg4, b4d10u-xsz; 4 byte**



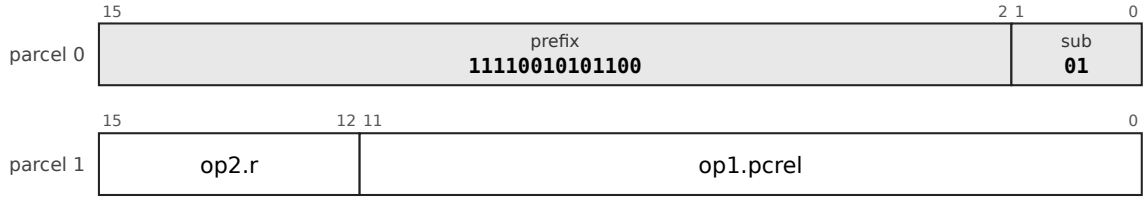

**MOVB: LDBu8H/b, prefix 1111000111010, sub-opcode 01; operands b4d8u-xsz, reg5; 4 bytes**



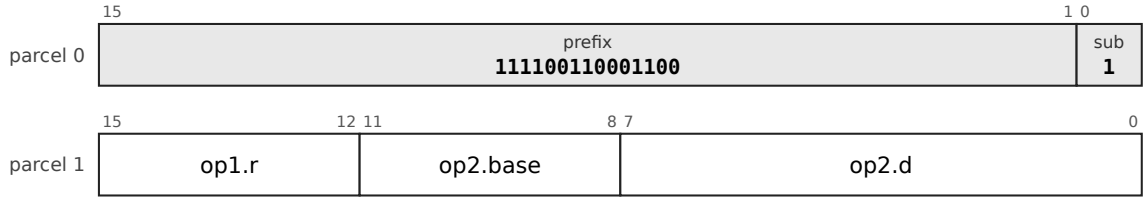
**MOVB: STBu12H/b, prefix 11110000000, sub-opcode 1; operands reg4, b4d12u-xsz; 4 bytes**



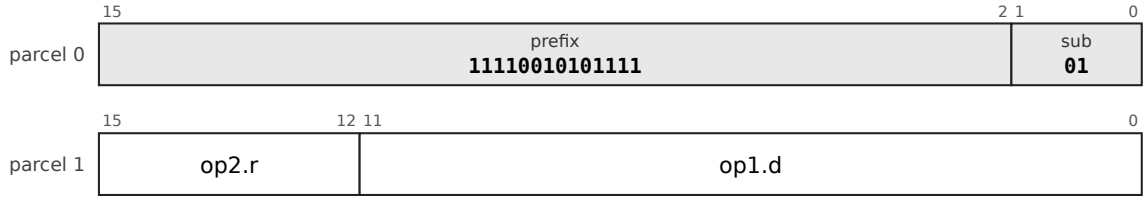

**MOVB: LDPC12H/b, prefix 11110010101100, sub-opcode 01; operands pc12-u1, reg4; 4 bytes**



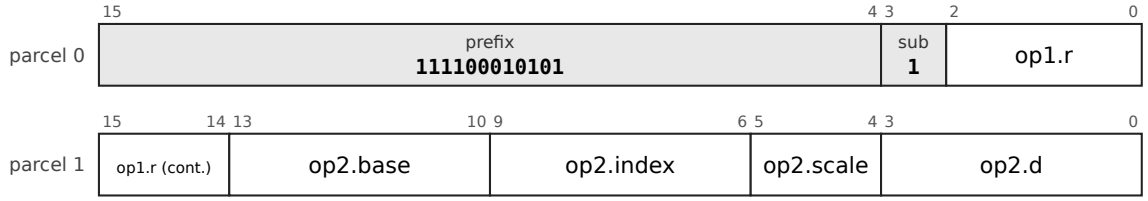

**MOVB: STBs8H/b, prefix 111100110001100, sub-opcode 1; operands reg4, b4d8s-xsz; 4 bytes**



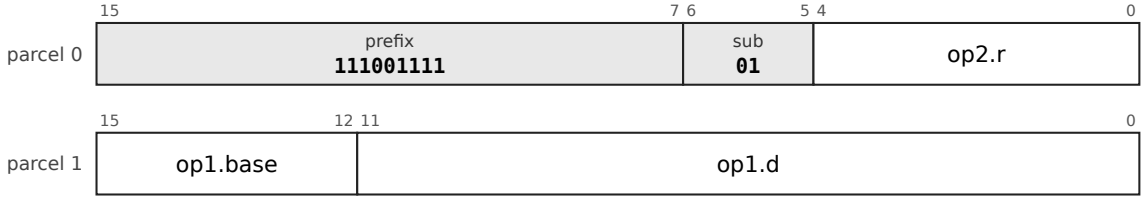

**MOVB: LDS12H/b, prefix 11110010101111, sub-opcode 01; operands spd12u-xsz, reg4; 4 byt**



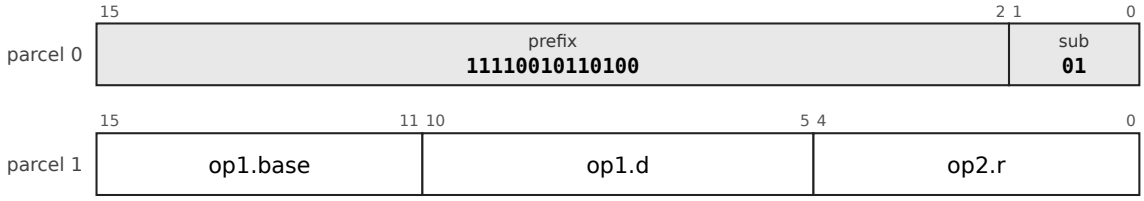

**MOVB: STX4H/b, prefix 111100010101, sub-opcode 1; operands reg5, x4-4d4s; 4 bytes**



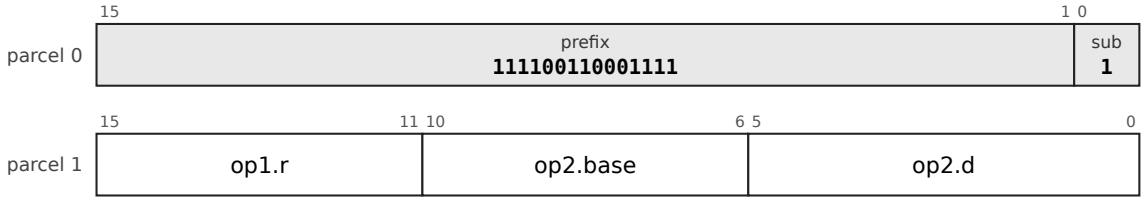
**MOVB: LDBu12H/b, prefix 111001111, sub-opcode 01; operands b4d12u-xsz, reg5; 4 bytes**



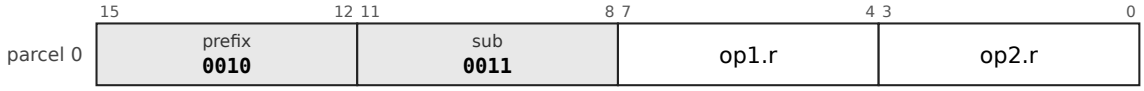

**MOVB: LDBs6/b, prefix 11110010110100, sub-opcode 01; operands b5d6s-xsz, reg5; 4 bytes**



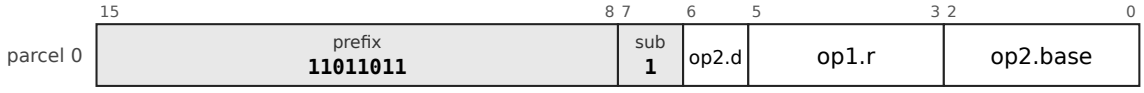

**MOVB: STBu6/b, prefix 111100110001111, sub-opcode 1; operands reg5, b5d6u-xsz; 4 bytes**




**MOVB: RRF, prefix 0010, sub-opcode 0011; operands reg4, reg4; 2 bytes**




**MOVB: STHHH1/b, prefix 11011011, sub-opcode 1; operands reg3, b3d1u-xsz; 2 bytes**




Operation, flags and exceptions as for the entry MOV above; 33 layout forms, each a chart of its own (above).

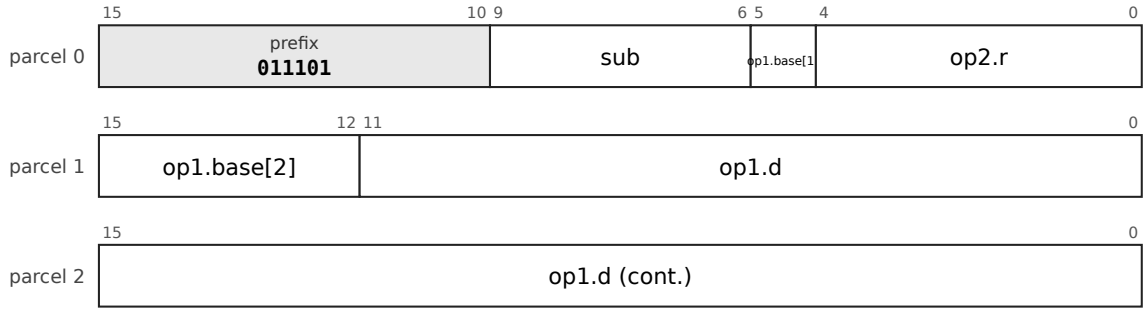
## MOVZ: Move zero-extended (32, 16 and 8 bits)

**Mnemonics:** `MOVZL`, `MOVZW`, `MOVZB`

### Syntax

```
MOVZL rm, Rd
MOVZW rm, Rd
MOVZB rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-2`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `LDBu6-b-1`, `LDX0-b-1`, `LDBu8H-b-1`, `LDPC12H-b-1`, `LDS12H-b-1`, `LDBu12H-b-1`, `LDBs6-b-1`, `LDFI-I32-3`, `LDX0-a-1`, `LDPC12H-a-1`, `LDPC14H-a-1`, `LDBu8H-a-1`, `LDPC16H-a-1`, `LDBu10H-a-1`, `LDHHH1-a-1`, `LDBs6-a-1`, `LDSPh4-a-1`, `LDBu12H-a-1`, `LDHHH3-a-1`, `LDPC12-a-1`, `LDBu8-a-1`, `LDX6H-a-1`, `LDBs10-a-1`.

### Operation

```
Rd ← zext(M[EA(rm)], size)      ; len = 00: Rd ← zext(Rs[8×size-1:0])
```

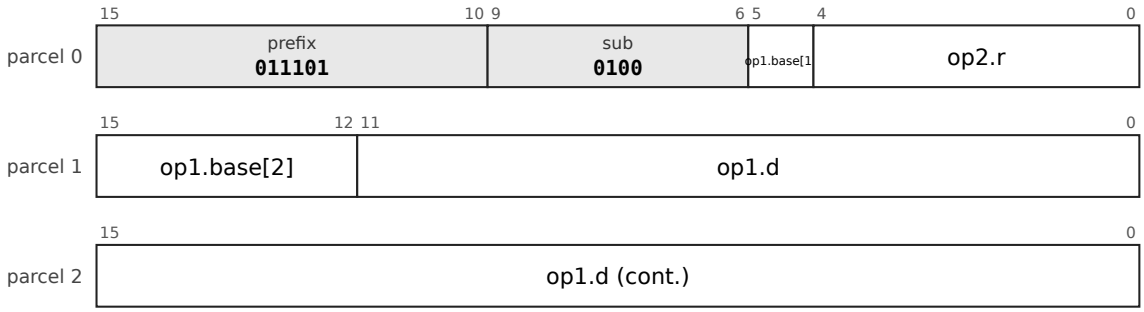
**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

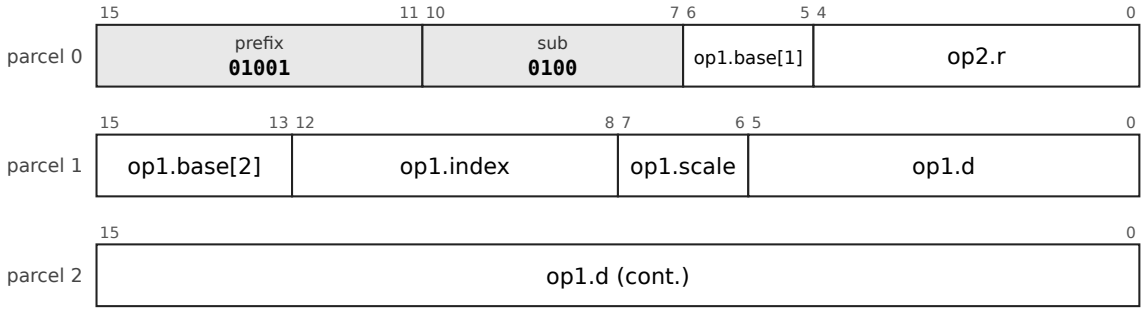
There is no zero-extending 64-bit move (it would be `MOVQ`). The register-to-register forms are the zero-extending half of the conversion set.

## *MOVZL*

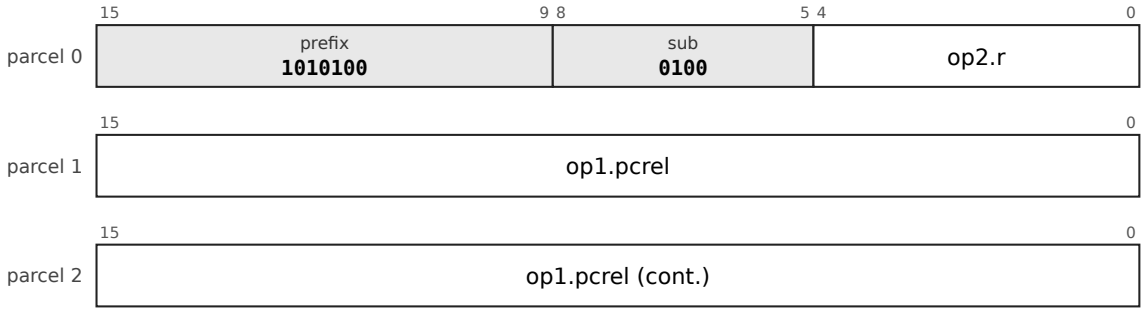
**MOVZL: LDF.B28, prefix 011101, sub-opcode 0100; operands b5d28s-x1, reg5; 6 bytes**



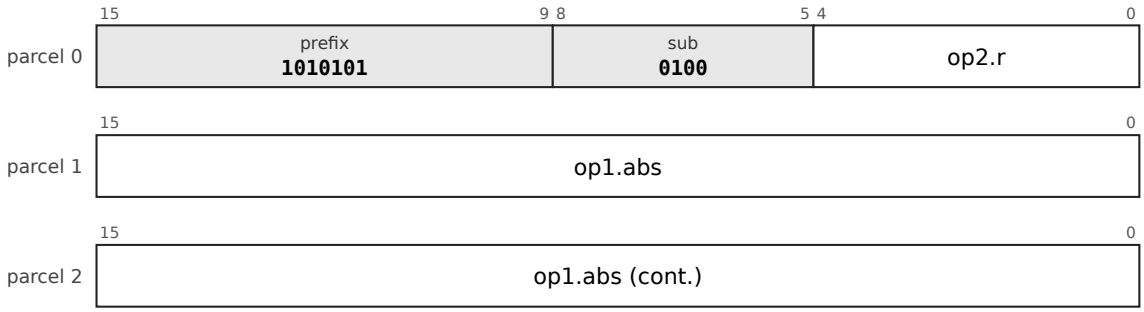

**MOVZL: LDF.X22, prefix 01001, sub-opcode 0100; operands x5-5d22s, reg5; 6 bytes**



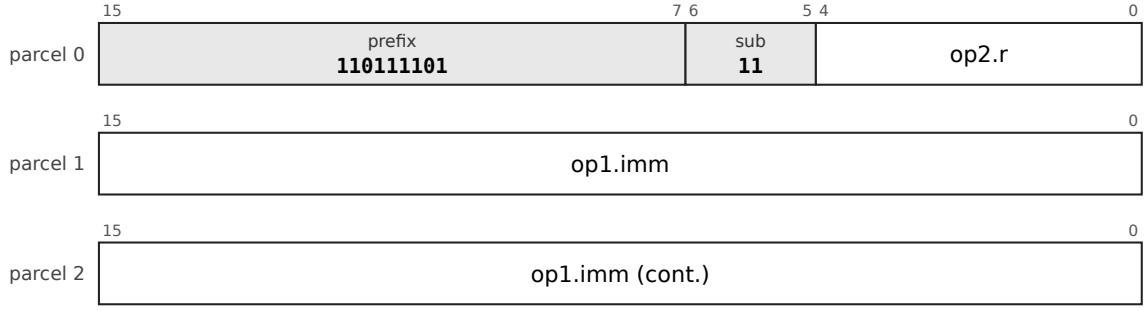

**MOVZL: LDF.PC32, prefix 1010100, sub-opcode 0100; operands pc32-u1, reg5; 6 bytes**



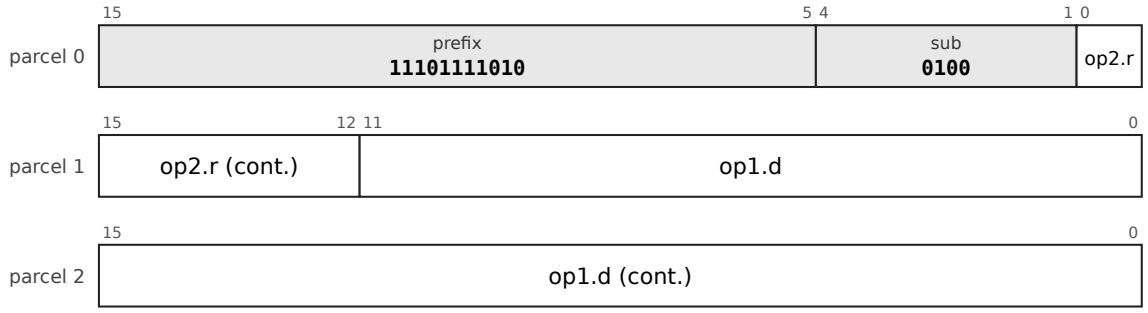

**MOVZL: LDF.A32, prefix 1010101, sub-opcode 0100; operands abs32, reg5; 6 bytes**



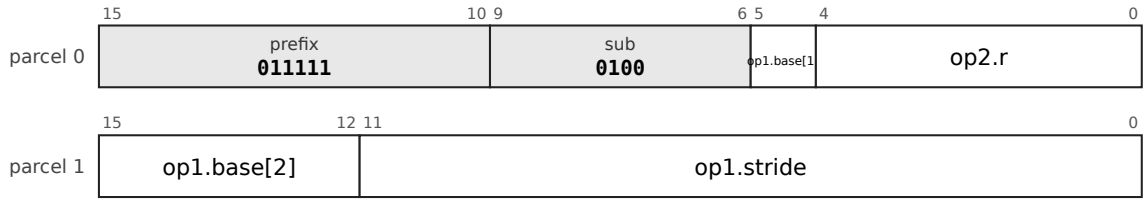
**MOVZL: LDFI.I32, prefix 110111101, sub-opcode 11; operands imm32s, reg5; 6 bytes**



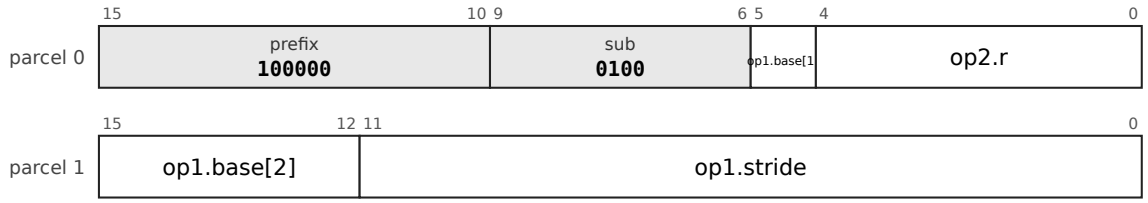

**MOVZL: DPF.D28, prefix 11101111010, sub-opcode 0100; operands dpd28s-x1, reg5; 6 bytes**



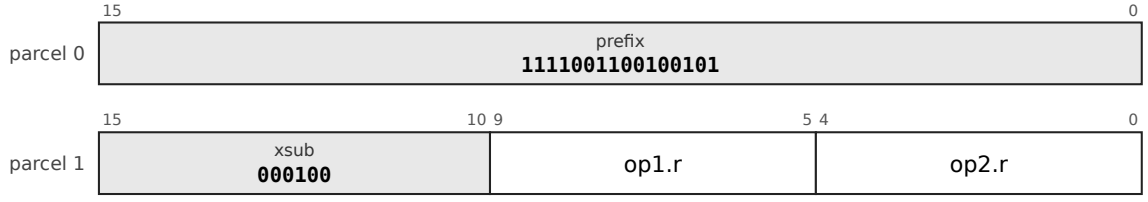

**MOVZL: LDF.PI12, prefix 011111, sub-opcode 0100; operands pi5-s12, reg5; 4 bytes**



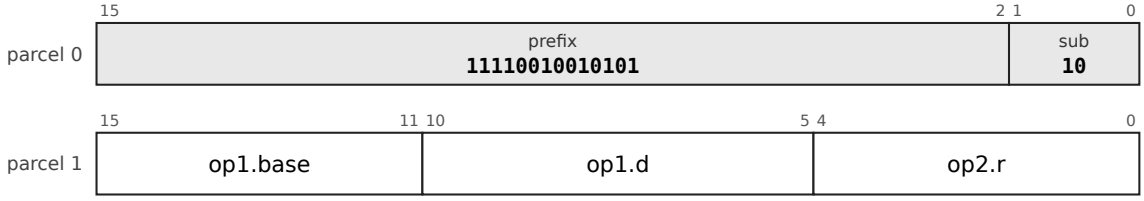

**MOVZL: LDF.PD12, prefix 100000, sub-opcode 0100; operands pd5-s12, reg5; 4 bytes**



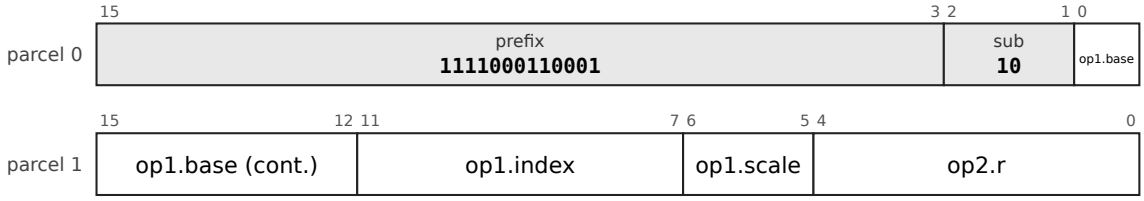

**MOVZL: RRF5, prefix 1111001100100101, sub-opcode 000100; operands reg5, reg5; 4 bytes**



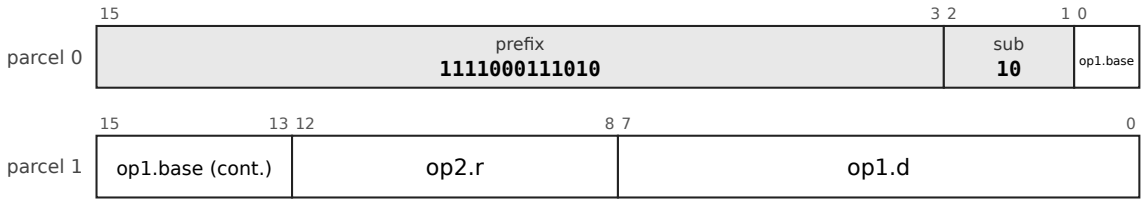
**MOVZL: LDBu6/b, prefix 11110010010101, sub-opcode 10; operands b5d6u-xsz, reg5; 4 bytes**



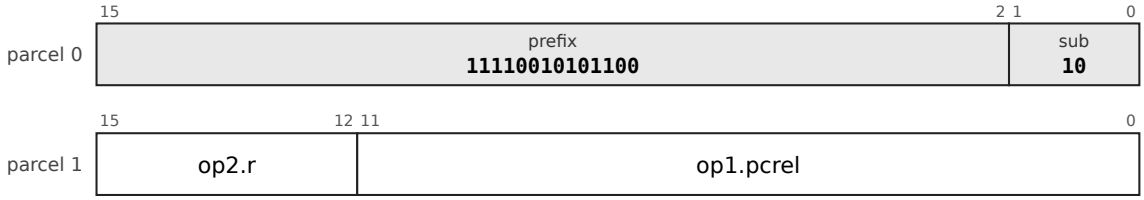

**MOVZL: LDX0/b, prefix 1111000110001, sub-opcode 10; operands x5-5d0s, reg5; 4 bytes**



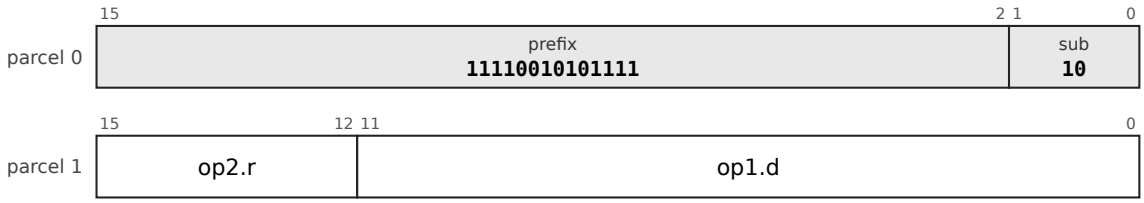

**MOVZL: LDBu8H/b, prefix 1111000111010, sub-opcode 10; operands b4d8u-xsz, reg5; 4 bytes**



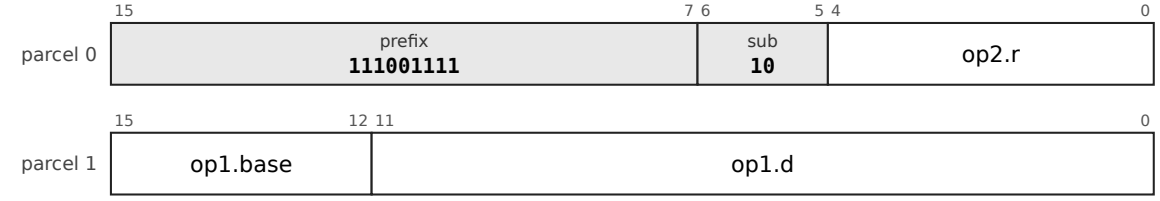

**MOVZL: LDPC12H/b, prefix 11110010101100, sub-opcode 10; operands pc12-u1, reg4; 4 byte**



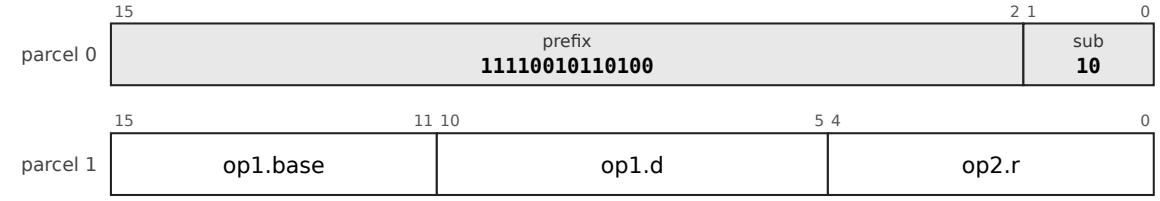

**MOVZL: LDS12H/b, prefix 11110010101111, sub-opcode 10; operands spd12u-xsz, reg4; 4 by**



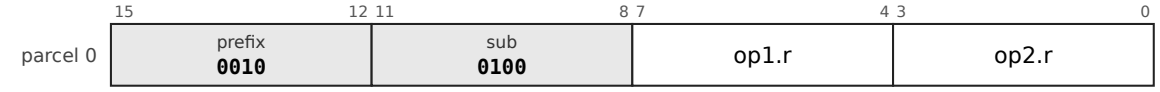
**MOVZL: LDBu12H/b, prefix 111001111, sub-opcode 10; operands b4d12u-xsz, reg5; 4 bytes**




**MOVZL: LDBs6/b, prefix 11110010110100, sub-opcode 10; operands b5d6s-xsz, reg5; 4 bytes**




**MOVZL: RRF, prefix 0010, sub-opcode 0100; operands reg4, reg4; 2 bytes**

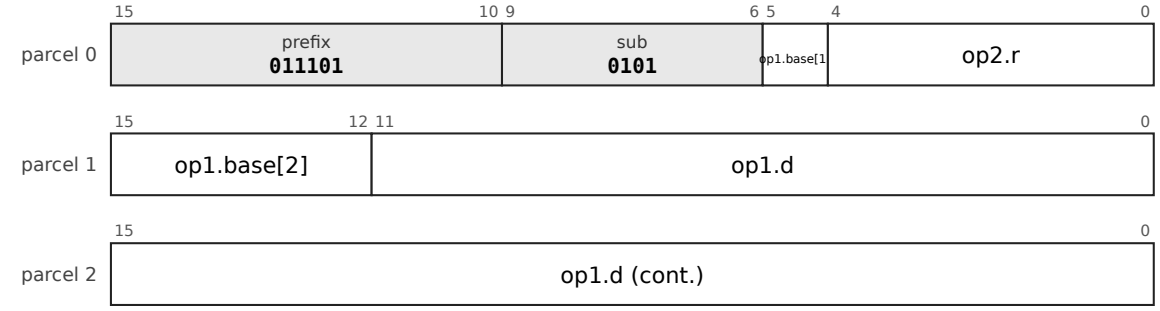



```
MOVZL rm, Rd
```

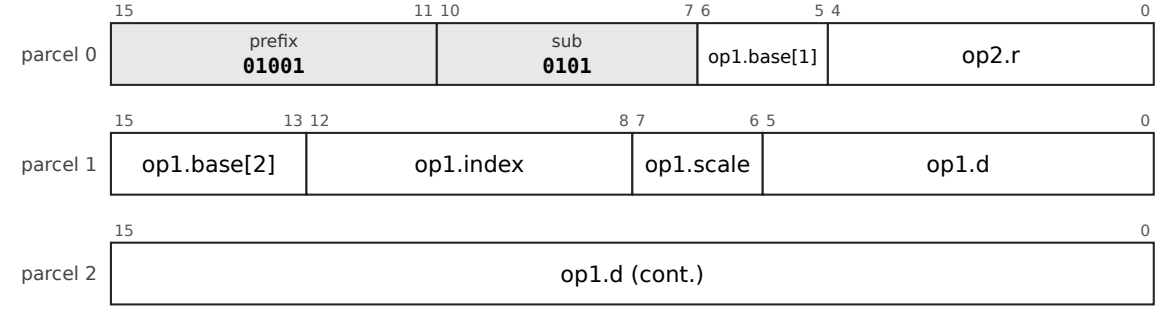
Operation, flags and exceptions as for the entry MOVZ above; 17 layout forms, each a chart of its own (above).

### *MOVZW*

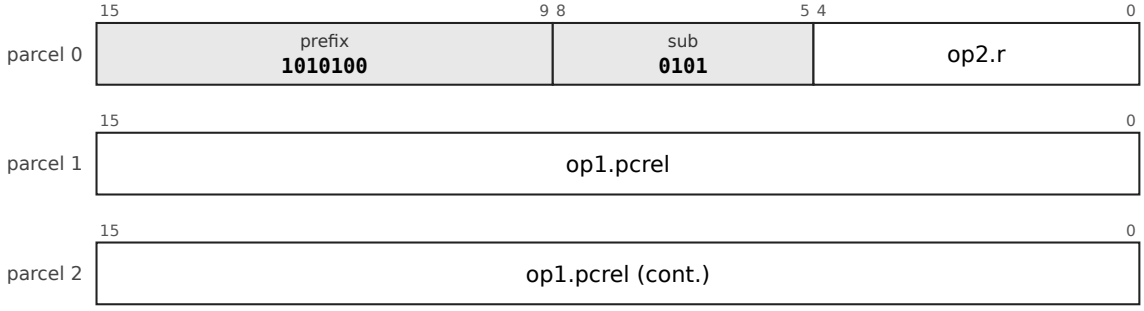
**MOVZW: LDF.B28, prefix 011101, sub-opcode 0101; operands b5d28s-x1, reg5; 6 bytes**



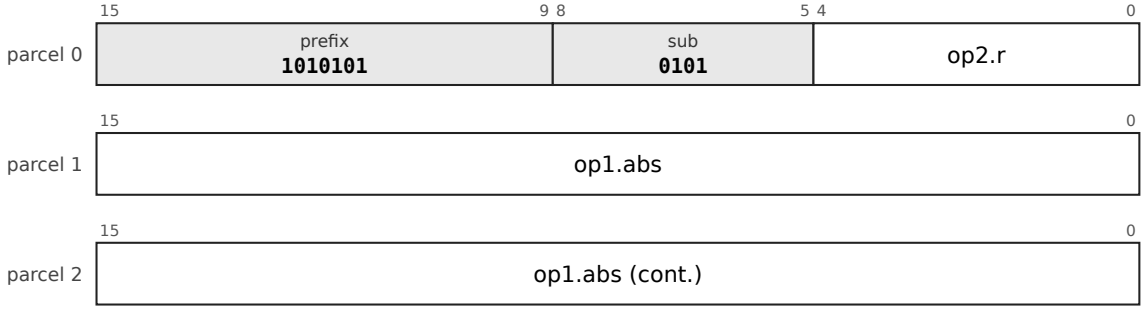

**MOVZW: LDF.X22, prefix 01001, sub-opcode 0101; operands x5-5d22s, reg5; 6 bytes**



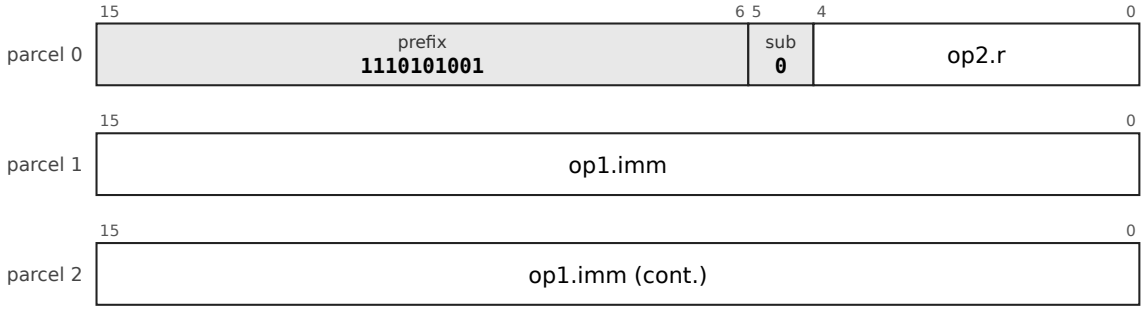
**MOVZW: LDF.PC32, prefix 1010100, sub-opcode 0101; operands pc32-u1, reg5; 6 bytes**



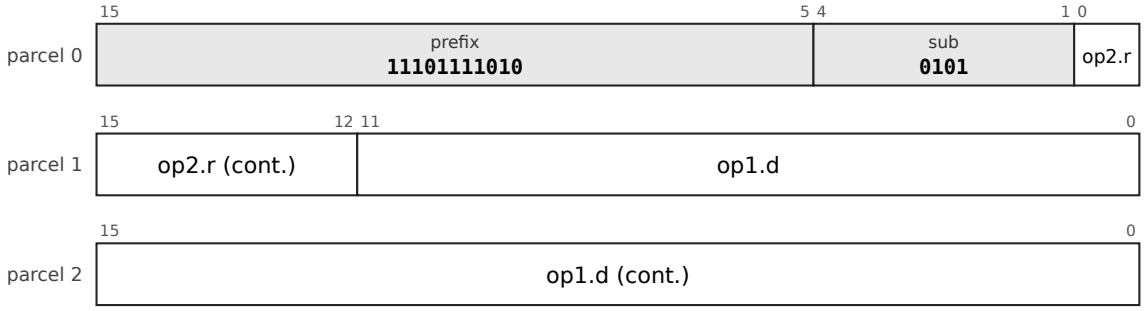

**MOVZW: LDF.A32, prefix 1010101, sub-opcode 0101; operands abs32, reg5; 6 bytes**



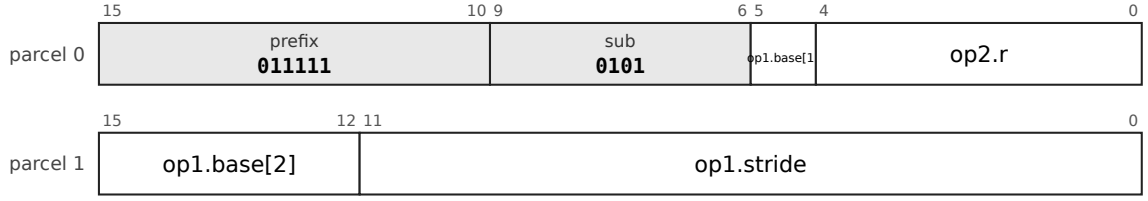

**MOVZW: LDFI.I32, prefix 1110101001, sub-opcode 0; operands imm32s, reg5; 6 bytes**



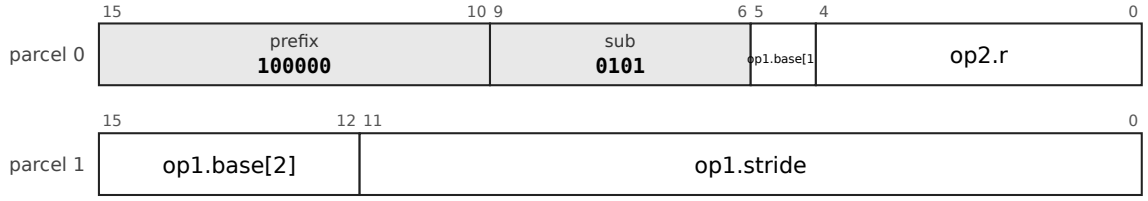

**MOVZW: DPF.D28, prefix 11101111010, sub-opcode 0101; operands dpd28s-x1, reg5; 6 bytes**



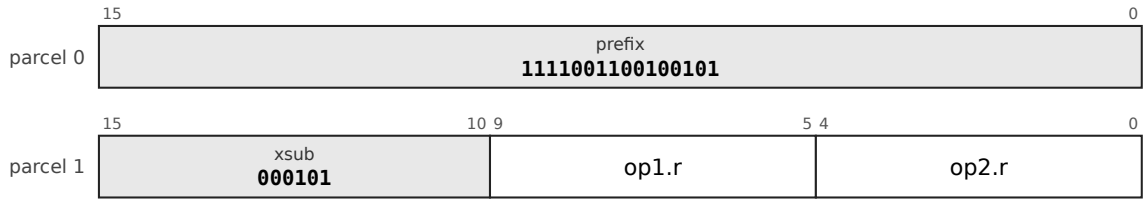
**MOVZW: LDF.PI12, prefix 011111, sub-opcode 0101; operands pi5-s12, reg5; 4 bytes**



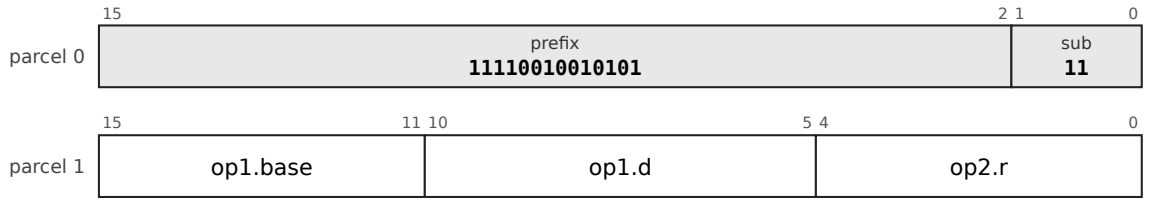

**MOVZW: LDF.PD12, prefix 100000, sub-opcode 0101; operands pd5-s12, reg5; 4 bytes**



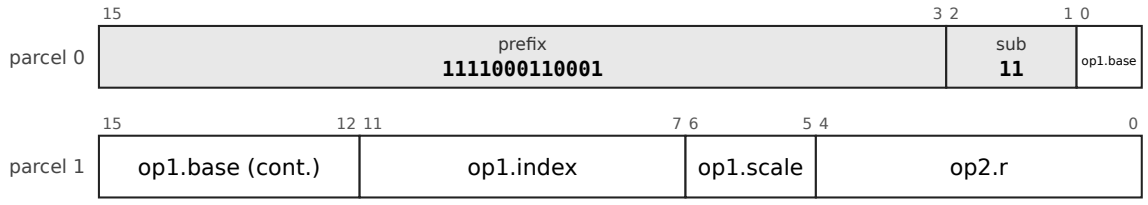

**MOVZW: RRF5, prefix 1111001100100101, sub-opcode 000101; operands reg5, reg5; 4 bytes**



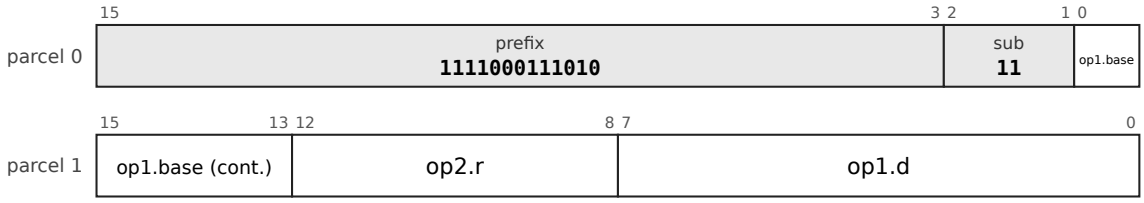

**MOVZW: LDBu6/b, prefix 11110010010101, sub-opcode 11; operands b5d6u-xsz, reg5; 4 byte**



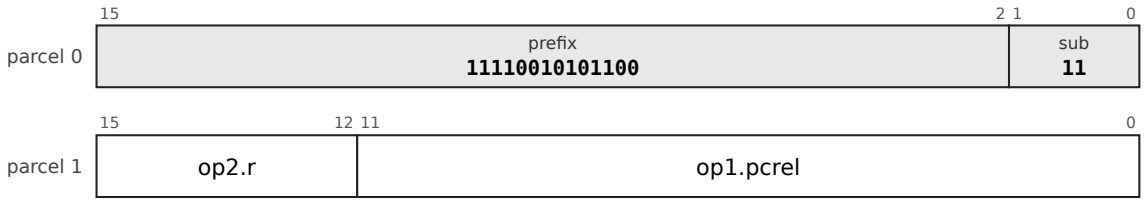

**MOVZW: LDX0/b, prefix 1111000110001, sub-opcode 11; operands x5-5d0s, reg5; 4 bytes**



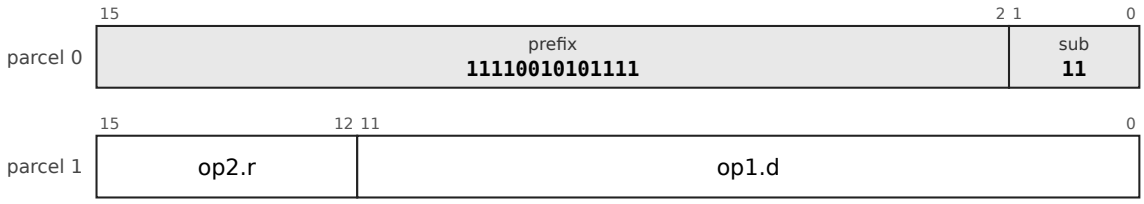
**MOVZW: LDBu8H/b, prefix 1111000111010, sub-opcode 11; operands b4d8u-xsz, reg5; 4 byte**



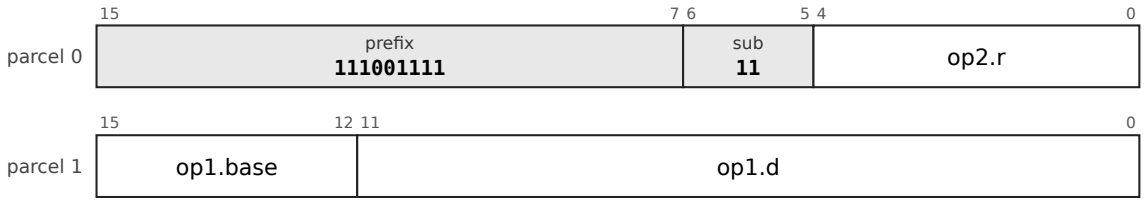

**MOVZW: LDPC12H/b, prefix 11110010101100, sub-opcode 11; operands pc12-u1, reg4; 4 byt**



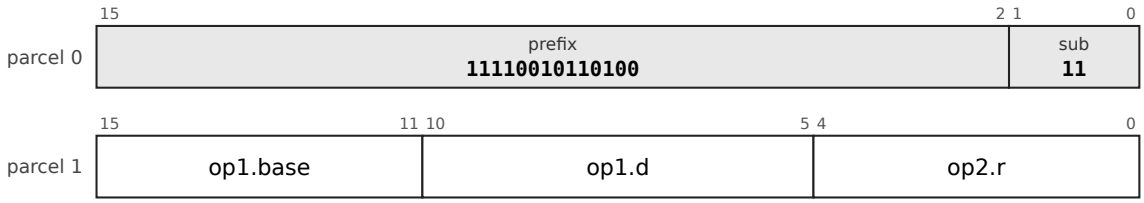

**MOVZW: LDS12H/b, prefix 11110010101111, sub-opcode 11; operands spd12u-xsz, reg4; 4 by**



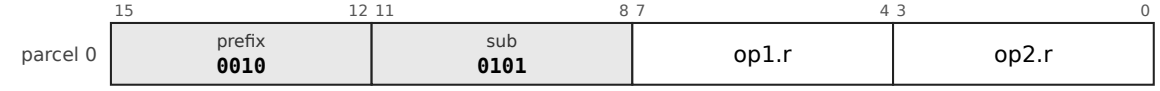

**MOVZW: LDBu12H/b, prefix 111001111, sub-opcode 11; operands b4d12u-xsz, reg5; 4 bytes**




**MOVZW: LDBs6/b, prefix 11110010110100, sub-opcode 11; operands b5d6s-xsz, reg5; 4 byte**



**MOVZW: RRF, prefix 0010, sub-opcode 0101; operands reg4, reg4; 2 bytes**

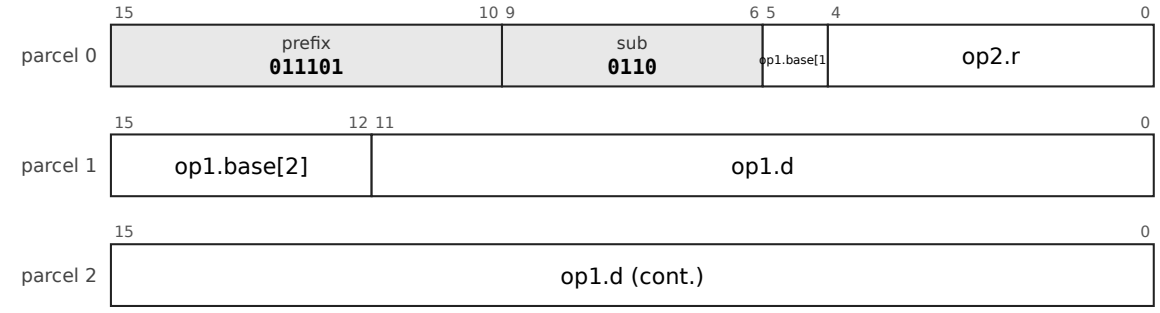



```
MOVZW rm, Rd
```

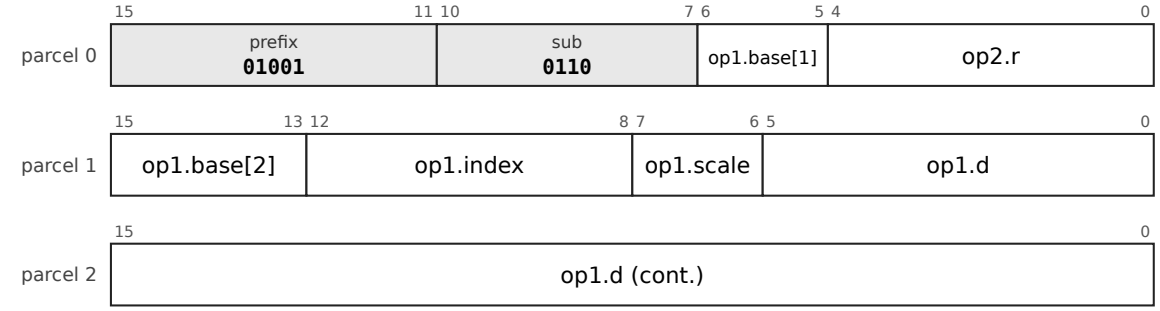
Operation, flags and exceptions as for the entry MOVZ above; 17 layout forms, each a chart of its own (above).

***MOVZB***

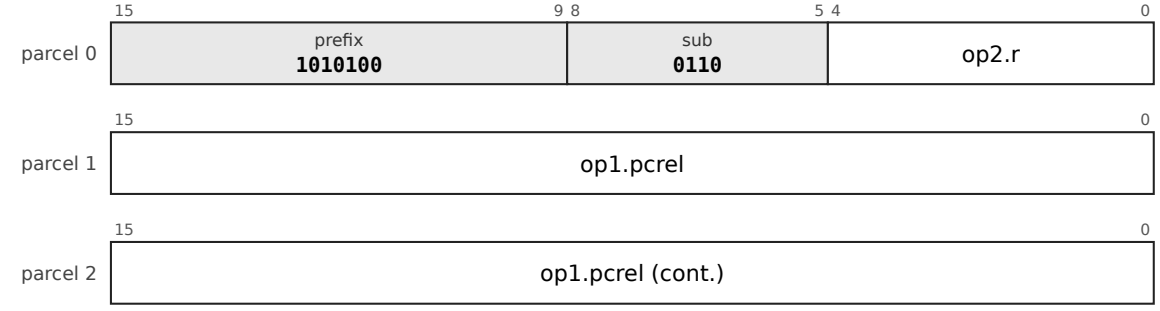
**MOVZB: LDF.B28, prefix 011101, sub-opcode 0110; operands b5d28s-x1, reg5; 6 bytes**



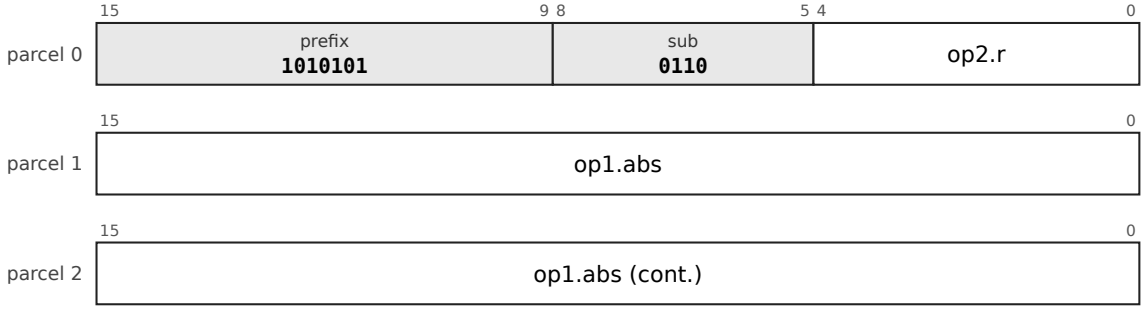

**MOVZB: LDF.X22, prefix 01001, sub-opcode 0110; operands x5-5d22s, reg5; 6 bytes**



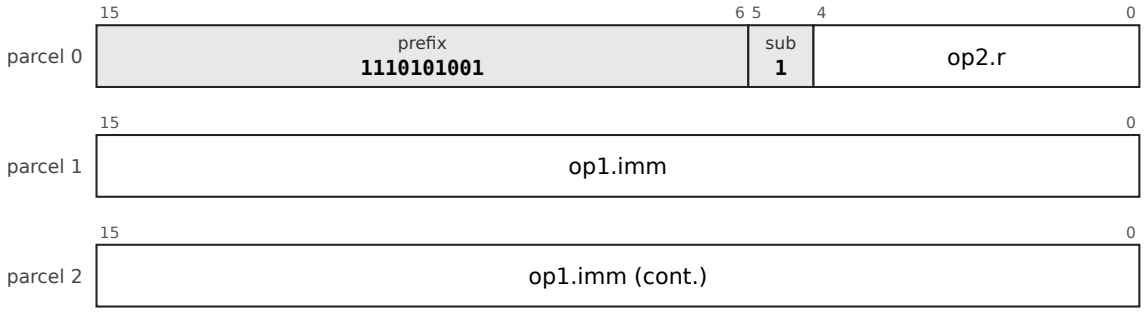

**MOVZB: LDF.PC32, prefix 1010100, sub-opcode 0110; operands pc32-u1, reg5; 6 bytes**



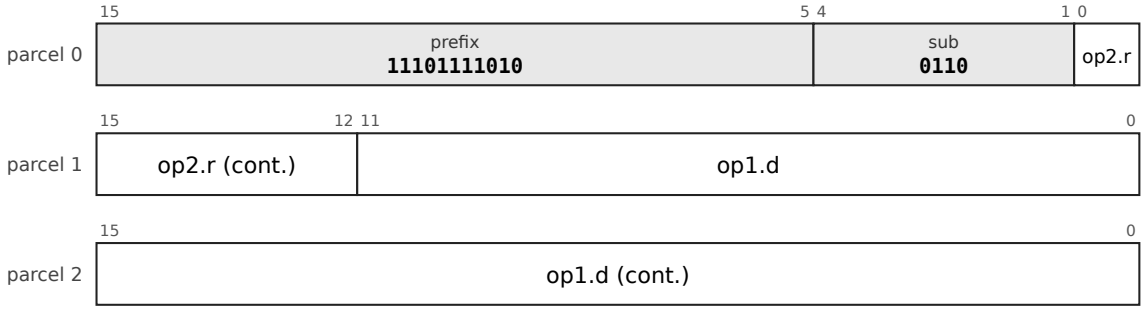
**MOVZB: LDF.A32, prefix 1010101, sub-opcode 0110; operands abs32, reg5; 6 bytes**



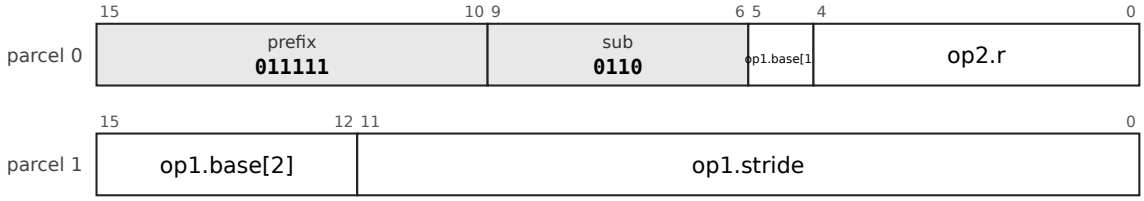

**MOVZB: LDFI.I32, prefix 1110101001, sub-opcode 1; operands imm32s, reg5; 6 bytes**



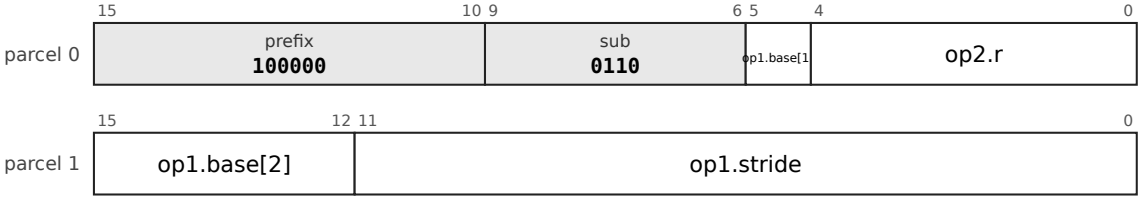

**MOVZB: DPF.D28, prefix 11101111010, sub-opcode 0110; operands dpd28s-x1, reg5; 6 bytes**



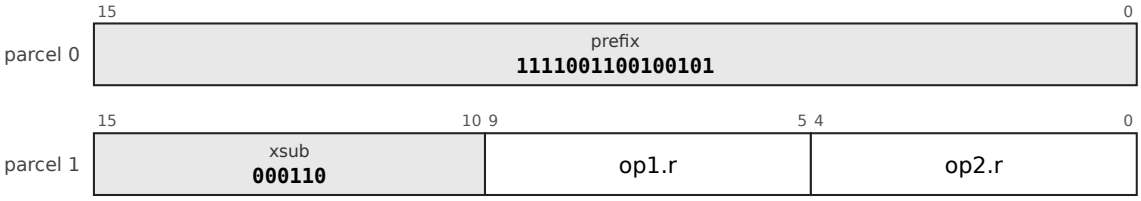

**MOVZB: LDF.PI12, prefix 011111, sub-opcode 0110; operands pi5-s12, reg5; 4 bytes**



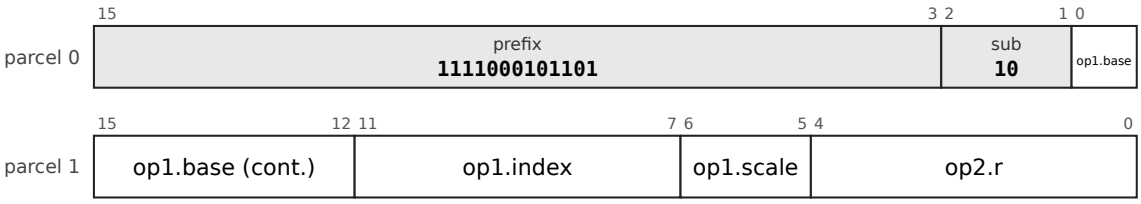
**MOVZB: LDF.PD12, prefix 100000, sub-opcode 0110; operands pd5-s12, reg5; 4 bytes**



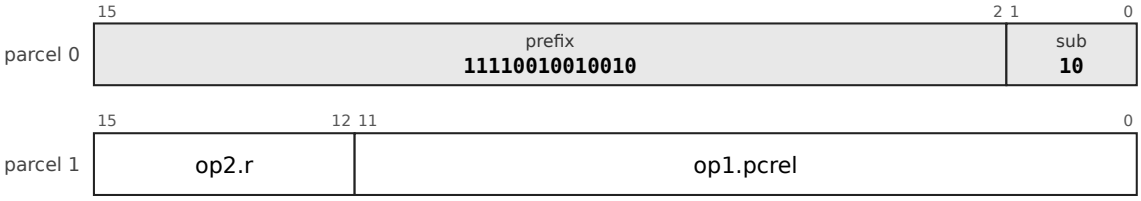

**MOVZB: RRF5, prefix 1111001100100101, sub-opcode 000110; operands reg5, reg5; 4 bytes**



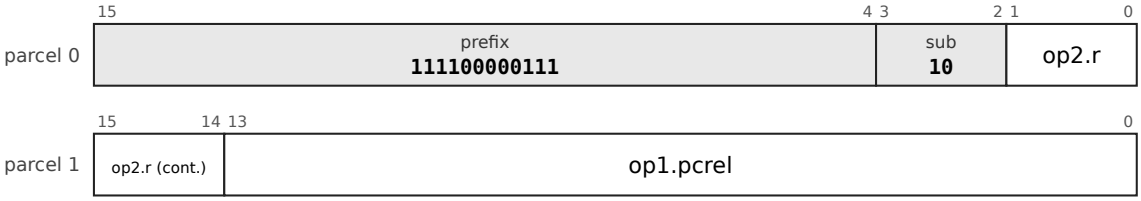

**MOVZB: LDX0/a, prefix 1111000101101, sub-opcode 10; operands x5-5d0s, reg5; 4 bytes**



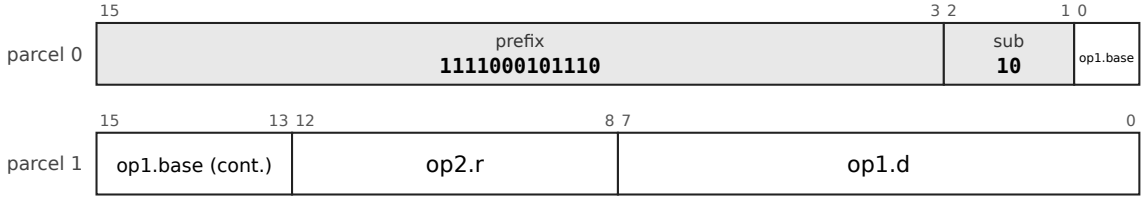

**MOVZB: LDPC12H/a, prefix 11110010010010, sub-opcode 10; operands pc12-u1, reg4; 4 byte**



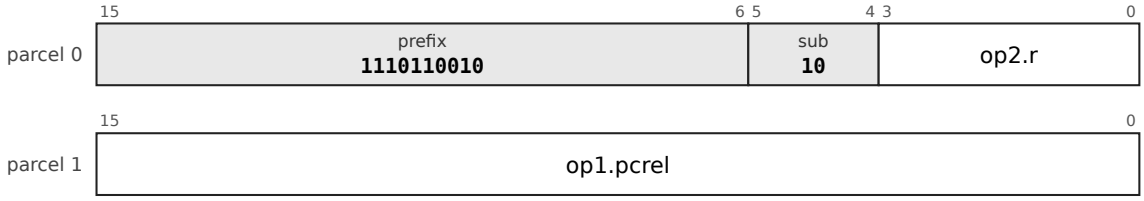

**MOVZB: LDPC14H/a, prefix 111100000111, sub-opcode 10; operands pc14-u1, reg4; 4 bytes**



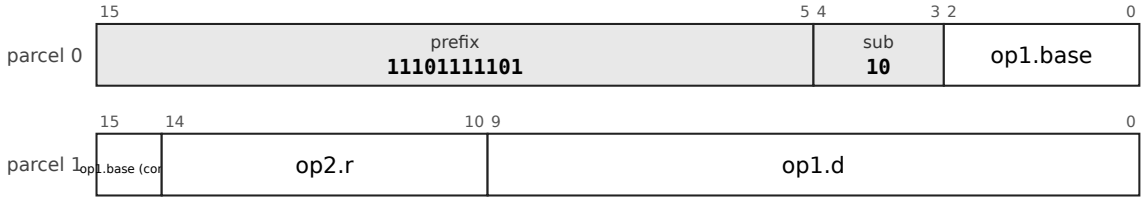
**MOVZB: LDBu8H/a, prefix 1111000101110, sub-opcode 10; operands b4d8u-xsz, reg5; 4 byte**



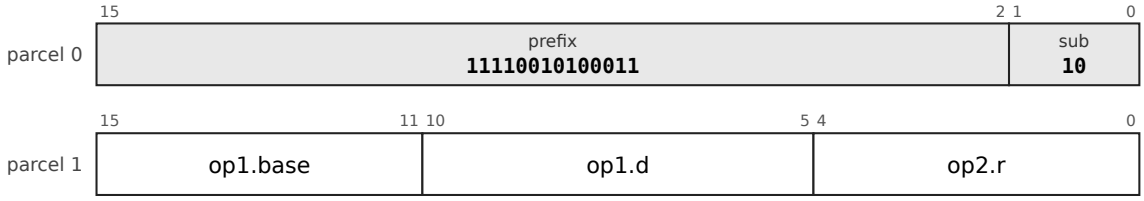

**MOVZB: LDPC16H/a, prefix 1110110010, sub-opcode 10; operands pc16-u1, reg4; 4 bytes**



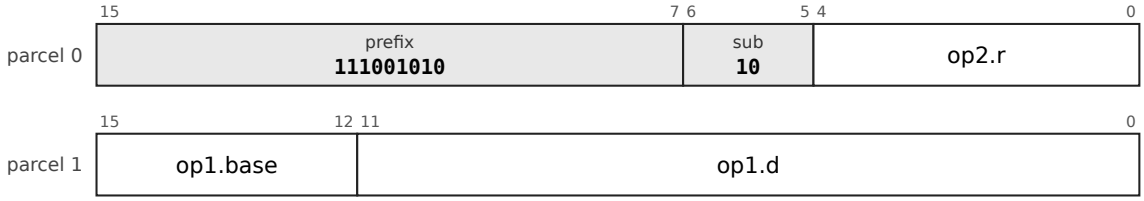

**MOVZB: LDBu10H/a, prefix 11101111101, sub-opcode 10; operands b4d10u-xsz, reg5; 4 byte**



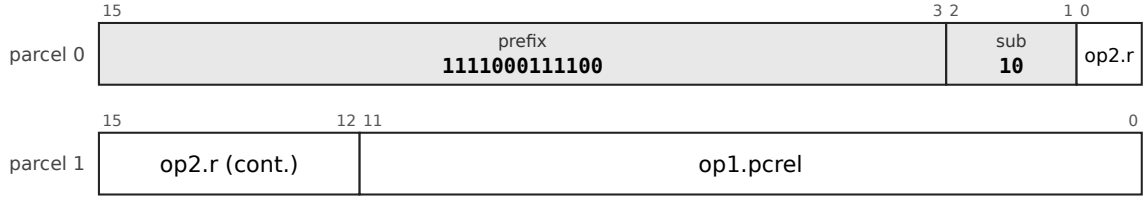

**MOVZB: LDBs6/a, prefix 11110010100011, sub-opcode 10; operands b5d6s-xsz, reg5; 4 bytes**



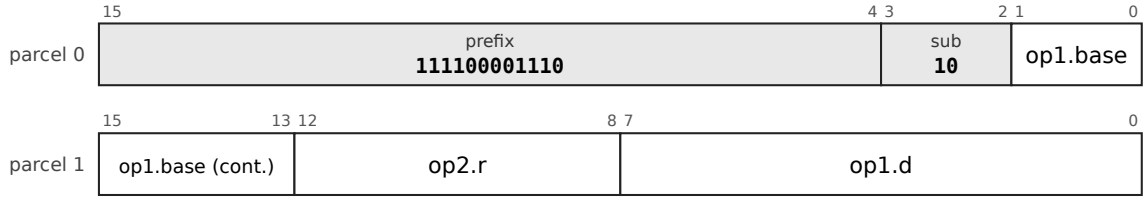

**MOVZB: LDBu12H/a, prefix 111001010, sub-opcode 10; operands b4d12u-xsz, reg5; 4 bytes**



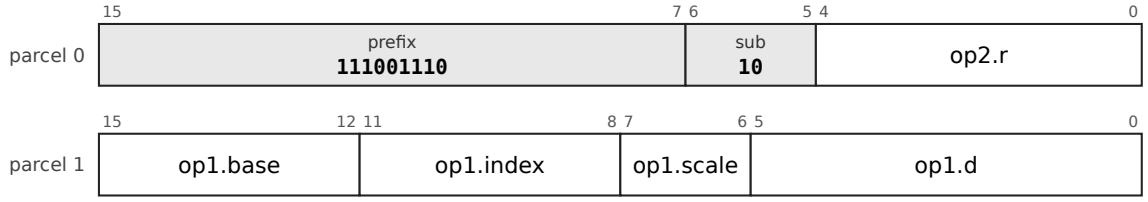
**MOVZB: LDPC12/a, prefix 1111000111100, sub-opcode 10; operands pc12-u1, reg5; 4 bytes**



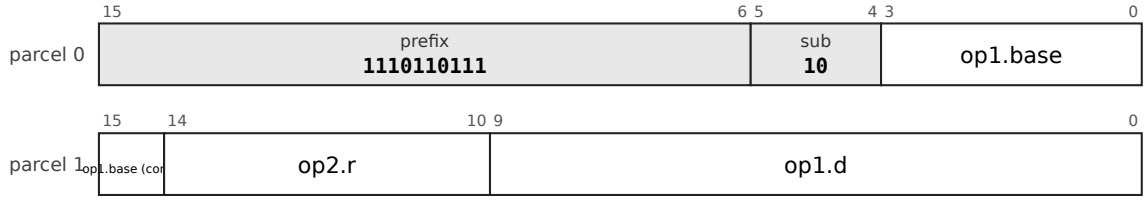

**MOVZB: LDBu8/a, prefix 111100001110, sub-opcode 10; operands b5d8u-xsz, reg5; 4 bytes**



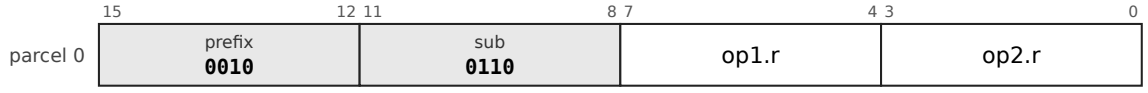

**MOVZB: LDX6H/a, prefix 111001110, sub-opcode 10; operands x4-4d6s, reg5; 4 bytes**



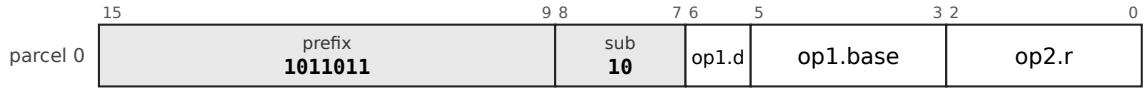

**MOVZB: LDBs10/a, prefix 1110110111, sub-opcode 10; operands b5d10s-xsz, reg5; 4 bytes**



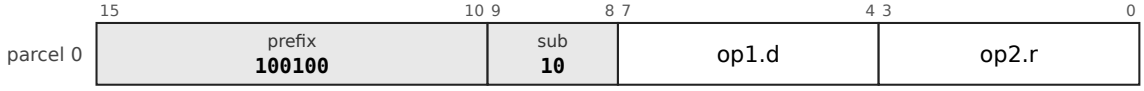

**MOVZB: RRF, prefix 0010, sub-opcode 0110; operands reg4, reg4; 2 bytes**



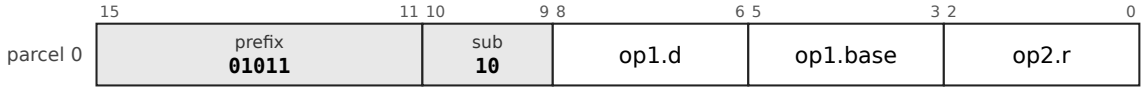

**MOVZB: LDHHH1/a, prefix 1011011, sub-opcode 10; operands b3d1u-xsz, reg3; 2 bytes**



**MOVZB: LDSPh4/a, prefix 100100, sub-opcode 10; operands spd4u-xsz, reg4; 2 bytes**




**MOVZB: LDHHH3/a, prefix 01011, sub-opcode 10; operands b3d3u-xsz, reg3; 2 bytes**

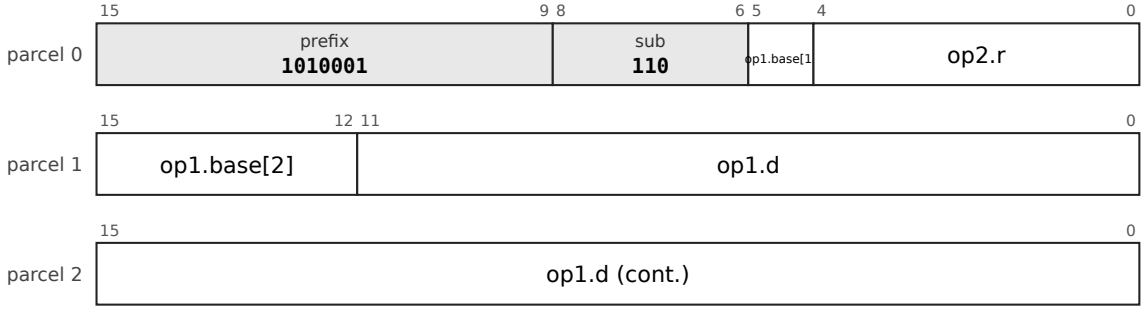



```
MOVZB rm, Rd
```

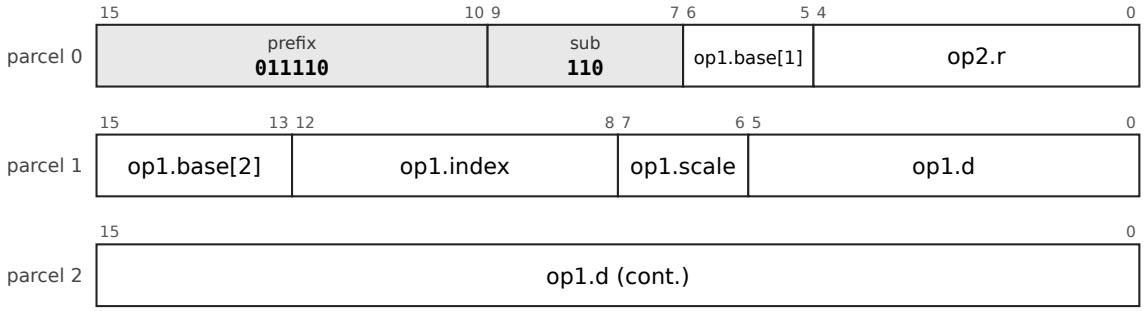
Operation, flags and exceptions as for the entry MOVZ above; 25 layout forms, each a chart of its own (above).

## LEA: Load effective address

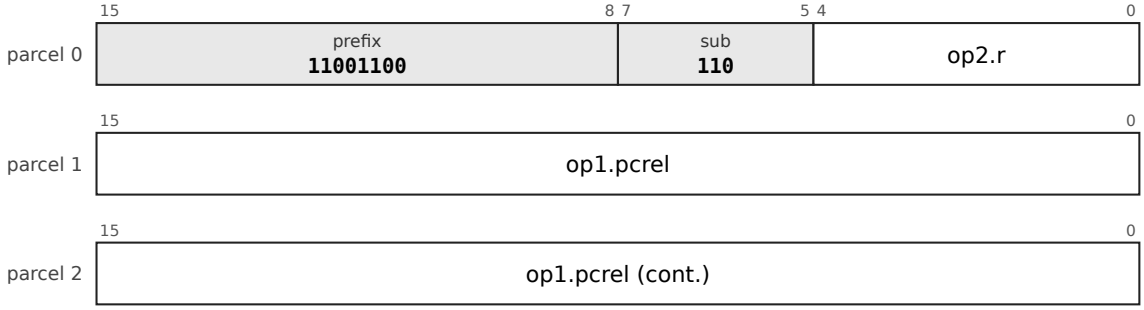
**LEA: LDF.B28, prefix 1010001, sub-opcode 110; operands b5d28s-x1, reg5; 6 bytes**



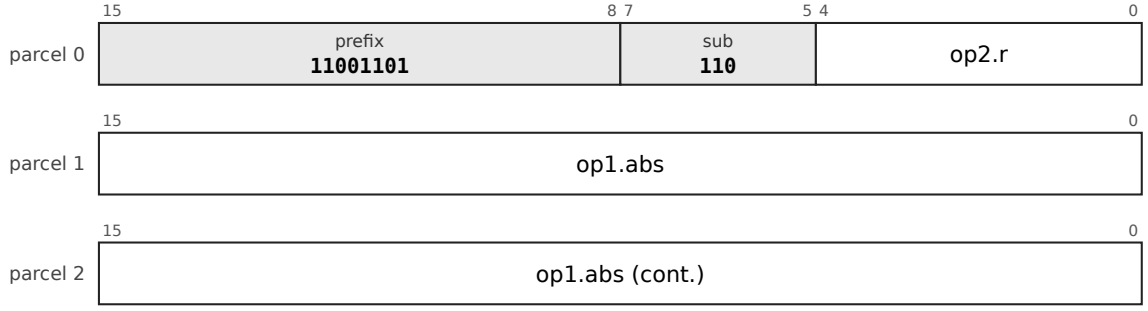

**LEA: LDF.X22, prefix 011110, sub-opcode 110; operands x5-5d22s, reg5; 6 bytes**



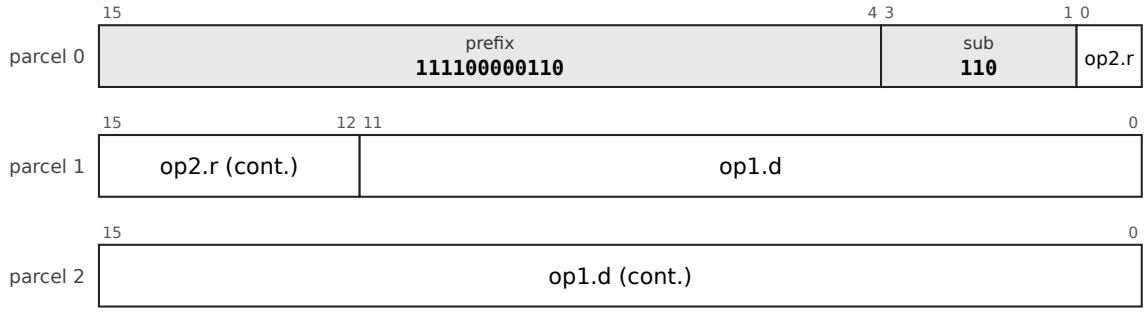

**LEA: LDF.PC32, prefix 11001100, sub-opcode 110; operands pc32-u1, reg5; 6 bytes**



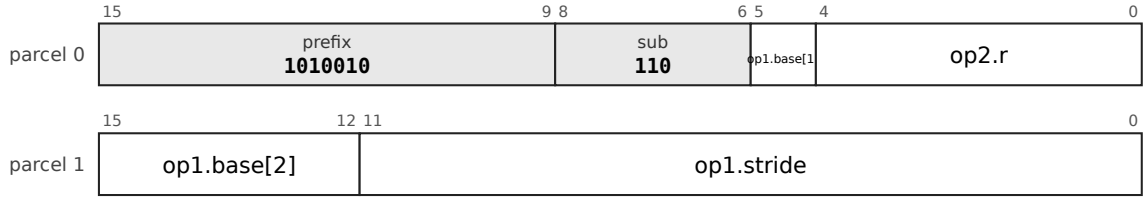
**LEA: LDF.A32, prefix 11001101, sub-opcode 110; operands abs32, reg5; 6 bytes**



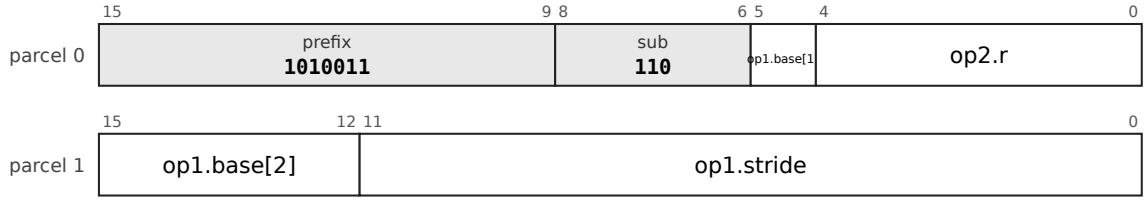

**LEA: DPF.D28, prefix 111100000110, sub-opcode 110; operands dpd28s-x1, reg5; 6 bytes**



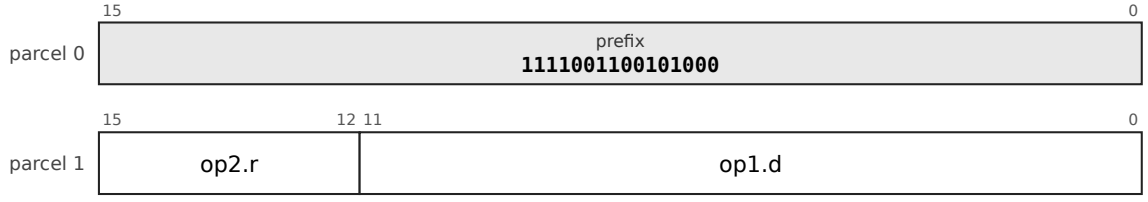

**LEA: LDF.PI12, prefix 1010010, sub-opcode 110; operands pi5-s12, reg5; 4 bytes**



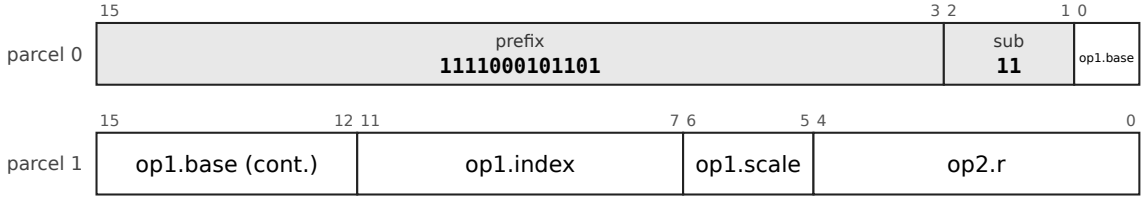

**LEA: LDF.PD12, prefix 1010011, sub-opcode 110; operands pd5-s12, reg5; 4 bytes**



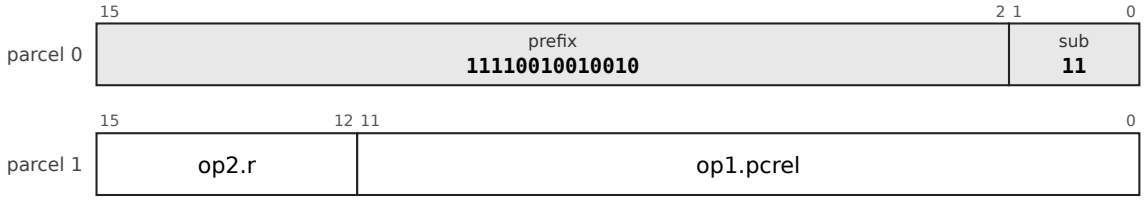

**LEA: LEASx12H/a, prefix 1111001100101000; operands spd12s-x1, reg4; 4 bytes**



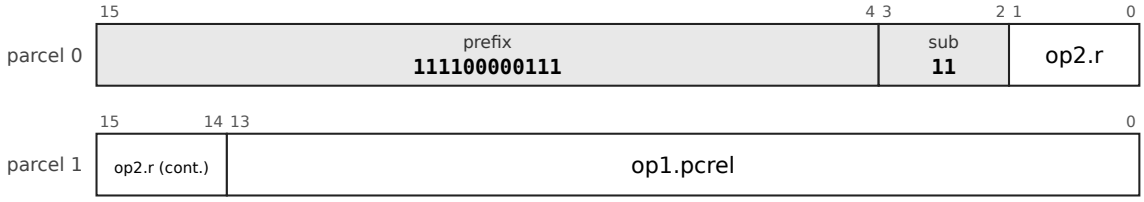
**LEA: LDX0/a, prefix 1111000101101, sub-opcode 11; operands x5-5d0s, reg5; 4 bytes**



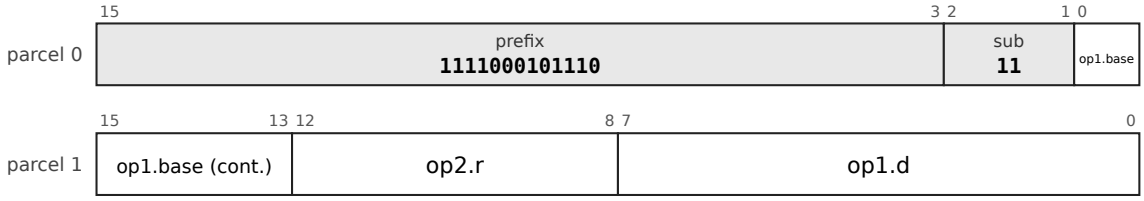

**LEA: LDPC12H/a, prefix 11110010010010, sub-opcode 11; operands pc12-u1, reg4; 4 bytes**



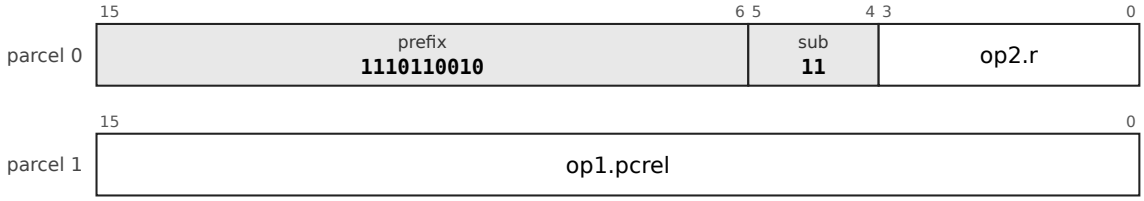

**LEA: LDPC14H/a, prefix 111100000111, sub-opcode 11; operands pc14-u1, reg4; 4 bytes**



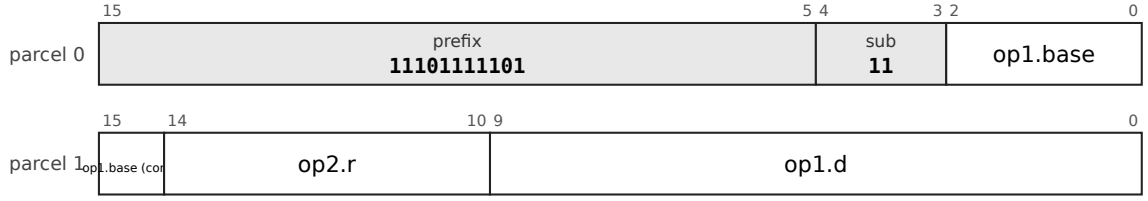

**LEA: LDBu8H/a, prefix 1111000101110, sub-opcode 11; operands b4d8u-xsz, reg5; 4 bytes**



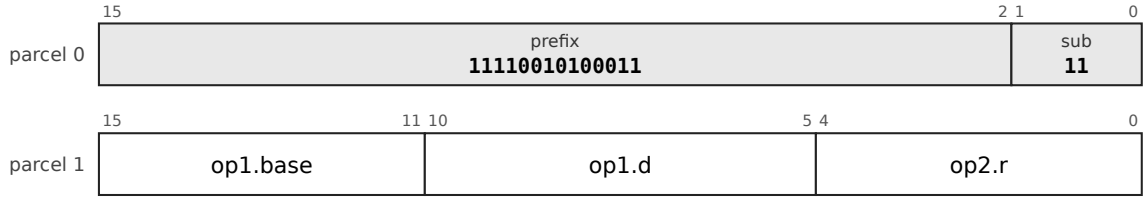

**LEA: LDPC16H/a, prefix 1110110010, sub-opcode 11; operands pc16-u1, reg4; 4 bytes**



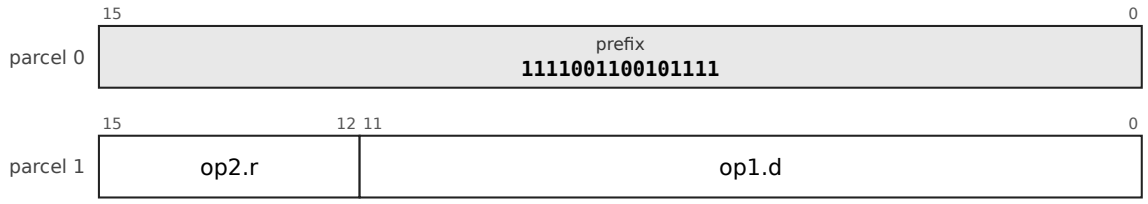
**LEA: LDBu10H/a, prefix 11101111101, sub-opcode 11; operands b4d10u-xsz, reg5; 4 bytes**



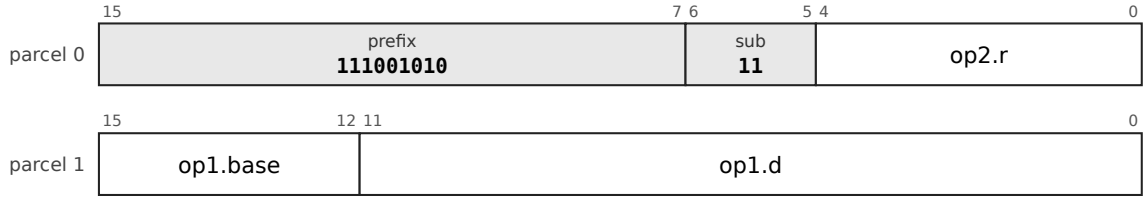

**LEA: LDBs6/a, prefix 11110010100011, sub-opcode 11; operands b5d6s-xsz, reg5; 4 bytes**



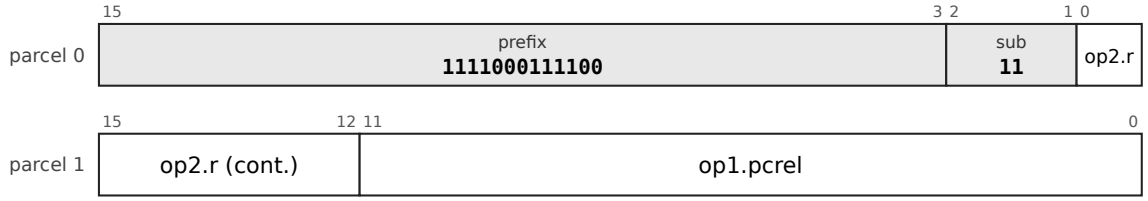

**LEA: LEAS12H/a, prefix 1111001100101111; operands spd12u-x8, reg4; 4 bytes**



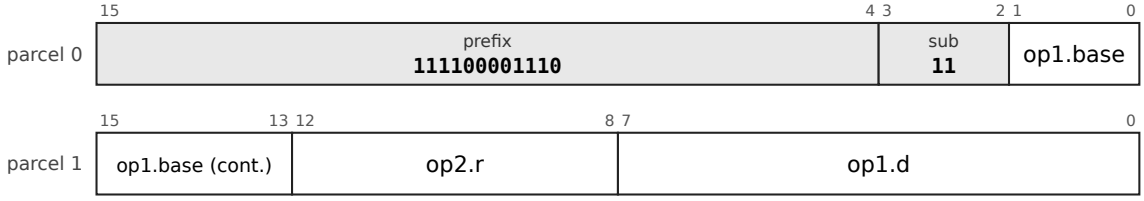

**LEA: LDBu12H/a, prefix 111001010, sub-opcode 11; operands b4d12u-xsz, reg5; 4 bytes**



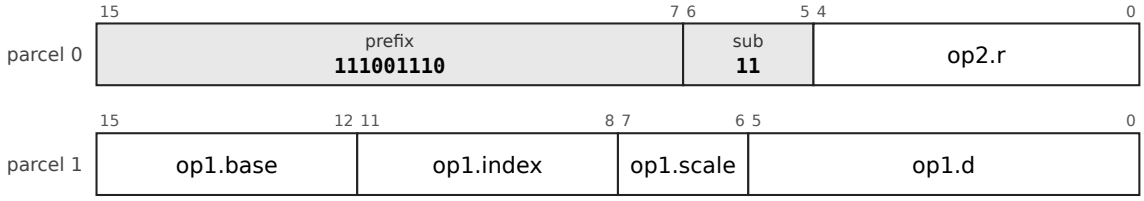

**LEA: LDPC12/a, prefix 1111000111100, sub-opcode 11; operands pc12-u1, reg5; 4 bytes**



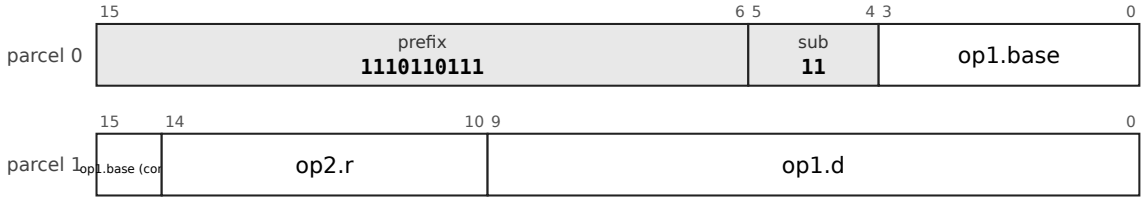
**LEA: LDBu8/a, prefix 111100001110, sub-opcode 11; operands b5d8u-xsz, reg5; 4 bytes**



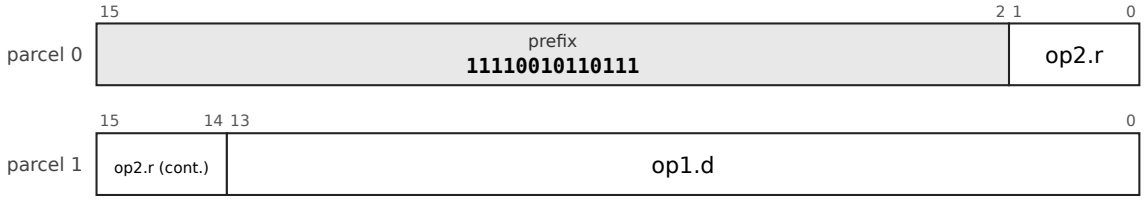

**LEA: LDX6H/a, prefix 111001110, sub-opcode 11; operands x4-4d6s, reg5; 4 bytes**



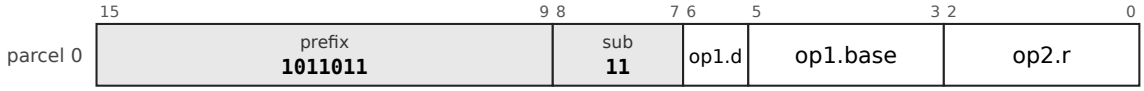

**LEA: LDBs10/a, prefix 1110110111, sub-opcode 11; operands b5d10s-xsz, reg5; 4 bytes**




**LEA: LEAS14H/a, prefix 11110010110111; operands spd14u-x8, reg4; 4 bytes**



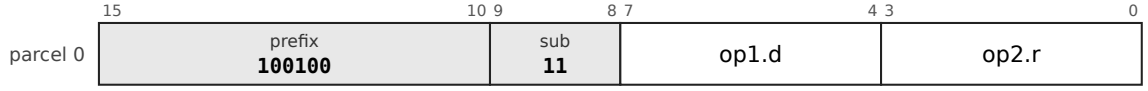

**LEA: LDHHH1/a, prefix 1011011, sub-opcode 11; operands b3d1u-xsz, reg3; 2 bytes**



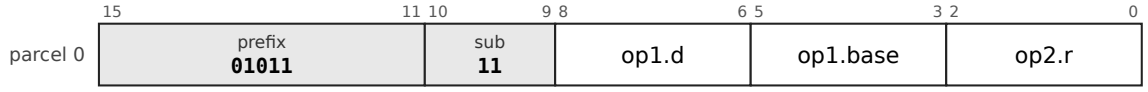

**LEA: LEASPh4/a, prefix 11011000; operands spd4u-x8, reg4; 2 bytes**



**LEA: LDSPh4/a, prefix 100100, sub-opcode 11; operands spd4u-xsz, reg4; 2 bytes**




**LEA: LDHHH3/a, prefix 01011, sub-opcode 11; operands b3d3u-xsz, reg3; 2 bytes**

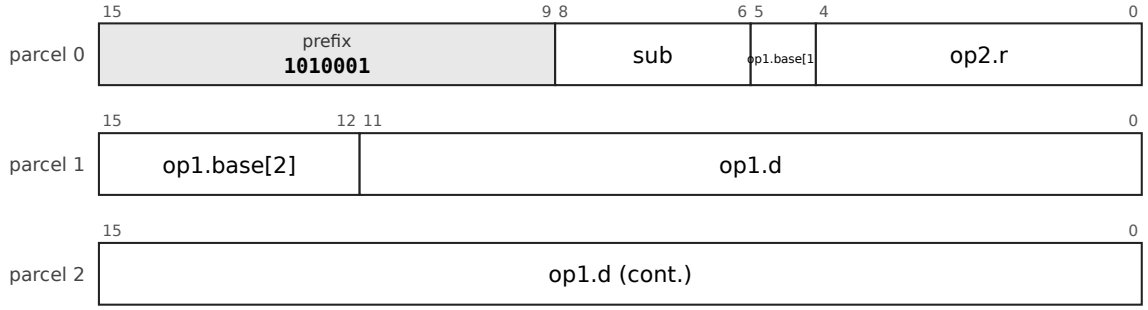



**Mnemonics:** `LEA`

### Syntax

```
LEA d(Rn), Rd
LEA d(PC), Rd
LEA d(Rn)(Rx×s), Rd
```

### Encoding




The chart is the leaf `LDF-B28-2`, LDF.B28, prefix `1010001`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-2`, `LDF-X22-2`, `LDF-PI12-2`, `LDF-PD12-2`, `LDF-PC32-2`, `LDF-A32-2`, `DPF-D28-2`, `LEASx12H-a-1`, `LDX0-a-1`, `LDPC12H-a-1`, `LDPC14H-a-1`, `LDBu8H-a-1`, `LDPC16H-a-1`, `LDBu10H-a-1`, `LDHHH1-a-1`, `LEASPh4-a-1`, `LDBs6-a-1`, `LEAS12H-a-1`, `LDSPh4-a-1`, `LDBu12H-a-1`, `LDHHH3-a-1`, `LDPC12-a-1`, `LDBu8-a-1`, `LDX6H-a-1`, `LDBs10-a-1`, `LEAS14H-a-1`.

### Operation

```
Rd ← EA(rm)          ; no memory access
```

**Flags** None written.

**Exceptions** The operand must name memory (a register or an immediate is not a form). No memory access is made, so the instruction takes no memory fault. A reserved encoding aborts with the illegal-instruction vector.

`LEA 0(PC), Rd` is how the program counter is read: it puts the address of the `LEA` itself in `Rd`. The base of every PC-relative displacement is the address of the first parcel of the instruction.

**Why it is here.** The PDP-11 and the VAX made the program counter a register (R7, R15). Here the program counter is kept out of the register file, because every register write would then be a potential branch, and a read-only view of it (R7 as a view of the PC) is refused, because it would fix for ever *which* PC is meant (ARM32 is stuck with `PC+8`). `LEA 0(PC), Rn` gives the capability without the commitment: the address of the instruction itself, in one instruction. The same instruction forms any address PC-relatively, which is what makes position-independent code free (section 9.16).

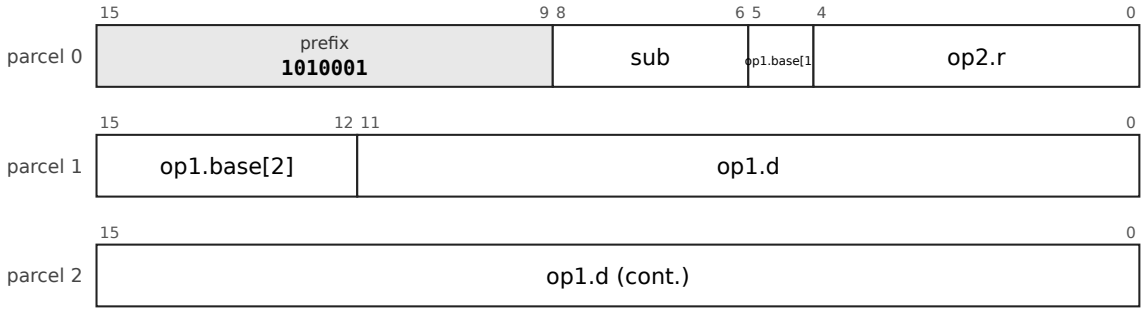
## MOVU: Unaligned load and store

**Mnemonics:** `MOVUB`, `MOVUW`, `MOVUL`, `MOVUQ`

### Syntax

```
MOVUQ rm, Rd        ; load, any alignment
MOVUQ Rs, rm        ; store, any alignment
MOVUW, MOVUL, MOVUB likewise
```

### Encoding




The chart is the leaf `LDF-B28-2`, LDF.B28, prefix `1010001`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-2`, `LDF-X22-2`, `LDF-PI12-2`, `LDF-PD12-2`, `LDF-PC32-2`, `LDF-A32-2`, `STF-B28-1`, `STF-X22-1`, `STF-PI12-1`, `STF-PD12-1`, `STF-PC32-1`, `STF-A32-1`, `DPF-D28-2`, `LDF-B28-3`, `LDF-X22-3`, `LDF-PI12-3`, `LDF-PD12-3`, `LDF-PC32-3`, `LDF-A32-3`, `DPF-D28-3`, `LDF-B28-4`, `LDF-X22-4`, `LDF-PI12-4`, `LDF-PD12-4`, `LDF-PC32-4`, `LDF-A32-4`, `DPF-D28-4`.

### Operation

```
size = 1, 2, 4 or 8 bytes
load :  Rd ← sext(M[EA(rm)], size)           ; the access may cross an alignment boundary
store:  M[EA(rm)] ← Rs[8×size-1:0]
```

**Flags** None written.

**Exceptions** Page and protection faults and bus errors as for any memory operand; **no alignment fault**.

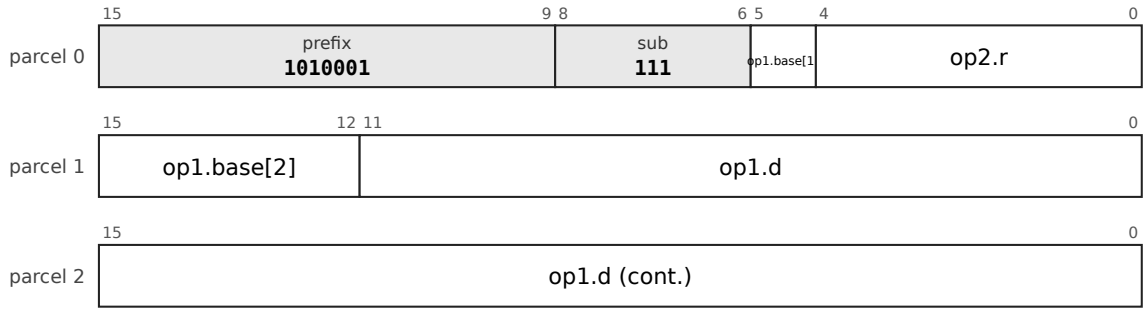
The ordinary moves fault on a misaligned address (the alignment fault is the tag check of section 9.5); these are the explicit forms for data that is not aligned.

**Why it is here.** Data alignment is architectural on this machine (9.2.11): an ordinary quad access to an address that is not a multiple of 8 aborts, and that abort doubles as the tag check for tagged pointers; a load form that ignores low pointer bits is rejected for that reason. Packed on-disk and network structures still need unaligned access, so the architecture has explicit unaligned forms rather than weakening the rule for every load. No further reason is recorded for these four.

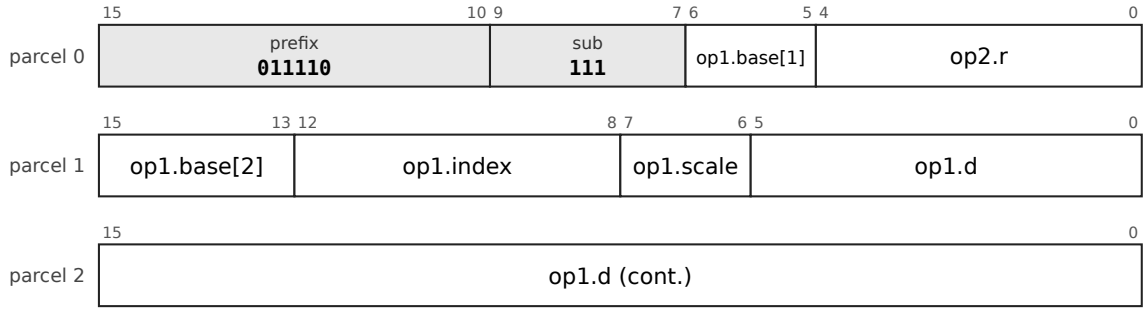
**Cf.** 6.6, 6.4.

### *MOVUB*

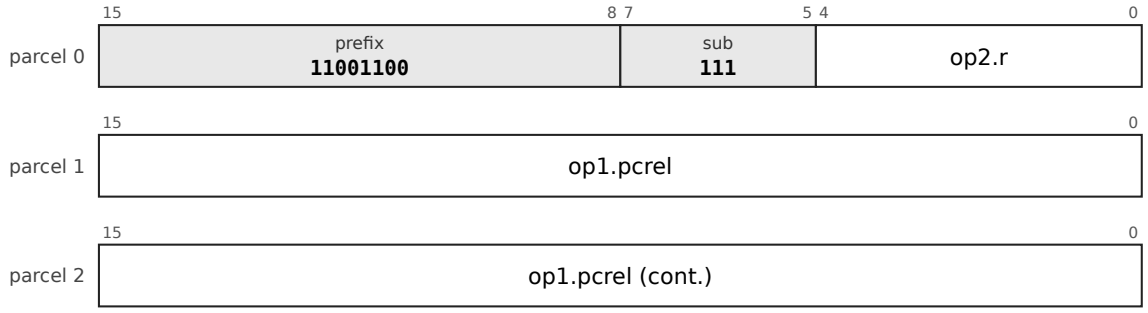
**MOVUB: LDF.B28, prefix 1010001, sub-opcode 111; operands b5d28s-x1, reg5; 6 bytes**



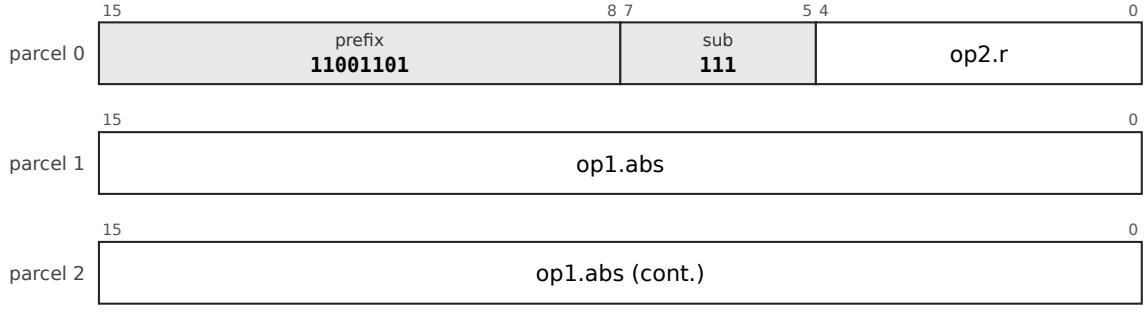

**MOVUB: LDF.X22, prefix 011110, sub-opcode 111; operands x5-5d22s, reg5; 6 bytes**



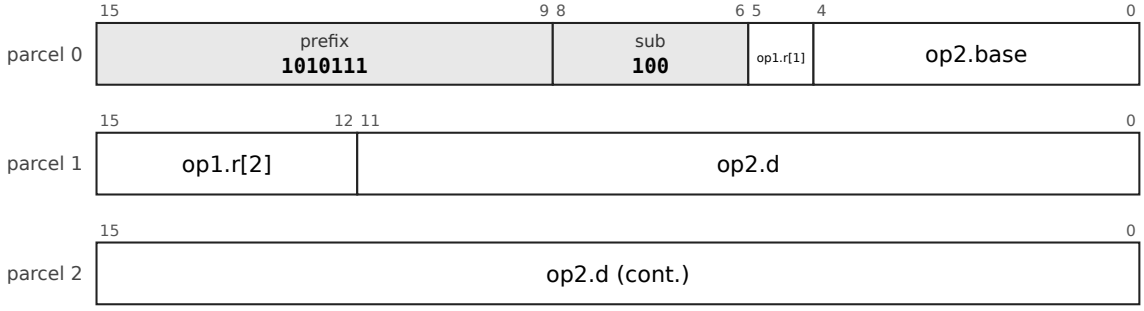

**MOVUB: LDF.PC32, prefix 11001100, sub-opcode 111; operands pc32-u1, reg5; 6 bytes**



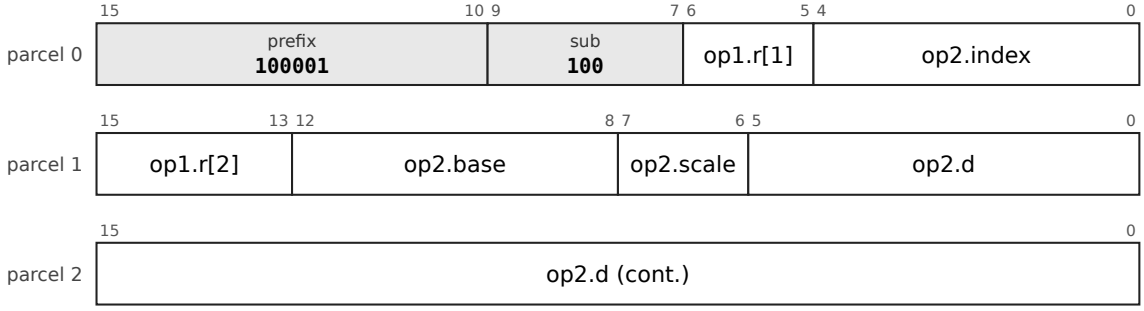

**MOVUB: LDF.A32, prefix 11001101, sub-opcode 111; operands abs32, reg5; 6 bytes**



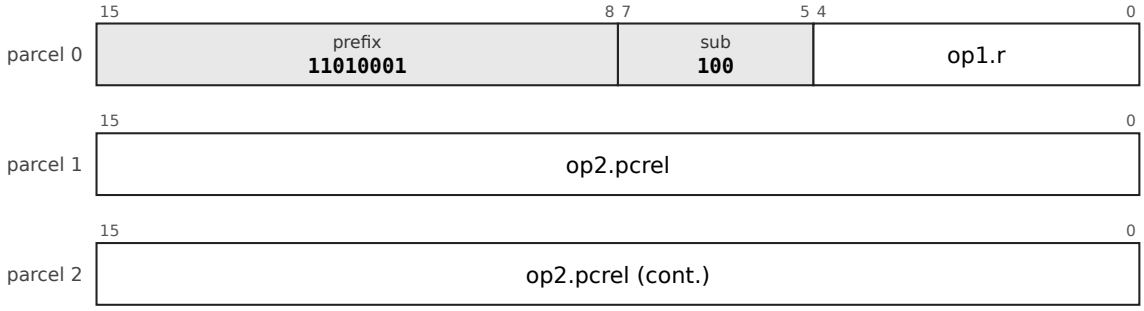
**MOVUB: STF.B28, prefix 1010111, sub-opcode 100; operands reg5, b5d28s-x1; 6 bytes**



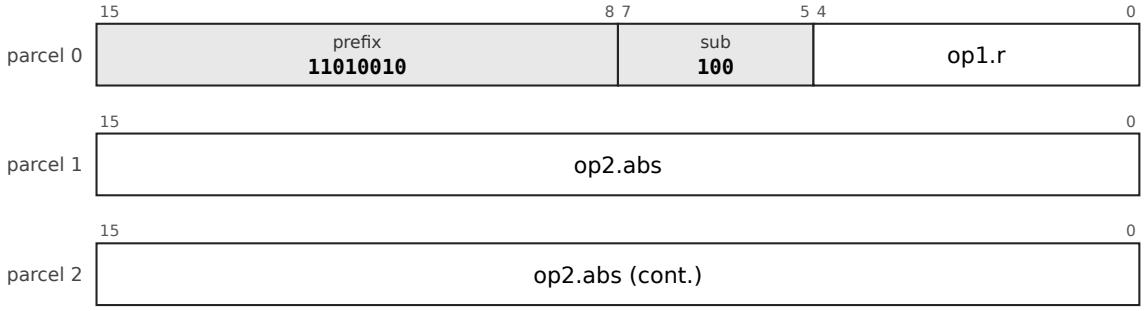

**MOVUB: STF.X22, prefix 100001, sub-opcode 100; operands reg5, x5-5d22s; 6 bytes**



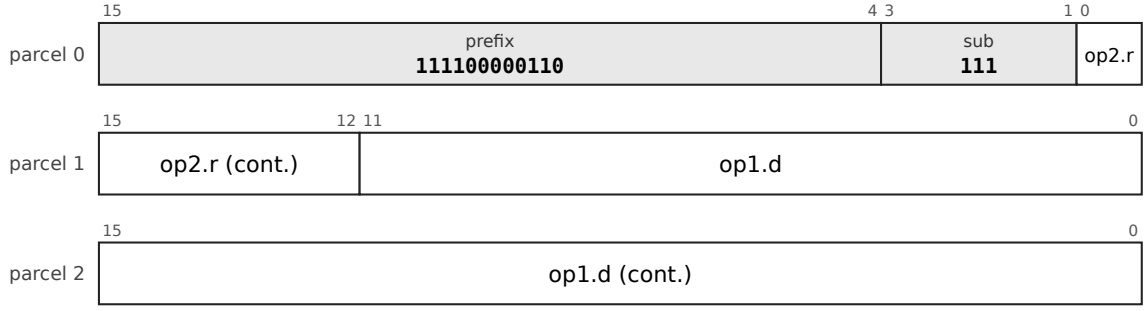

**MOVUB: STF.PC32, prefix 11010001, sub-opcode 100; operands reg5, pc32-u1; 6 bytes**



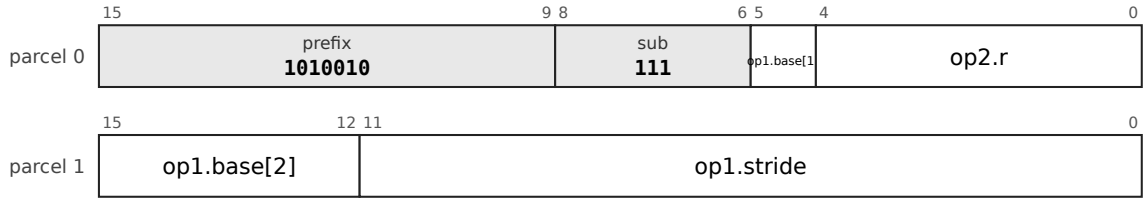

**MOVUB: STF.A32, prefix 11010010, sub-opcode 100; operands reg5, abs32; 6 bytes**



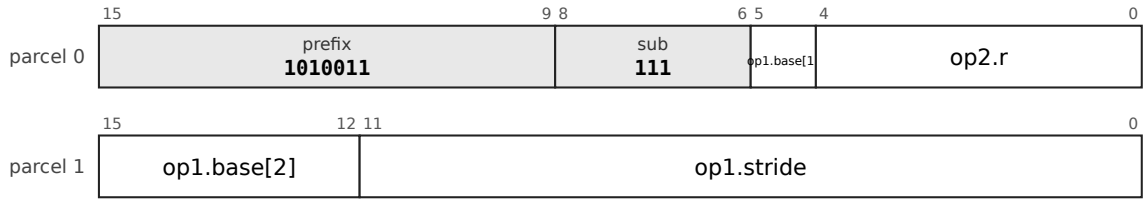
**MOVUB: DPF.D28, prefix 111100000110, sub-opcode 111; operands dpd28s-x1, reg5; 6 bytes**



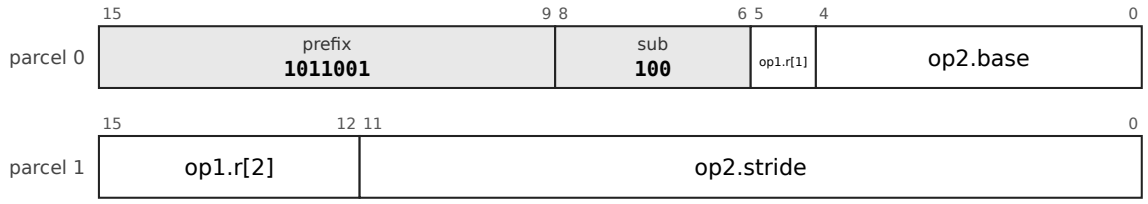

**MOVUB: LDF.PI12, prefix 1010010, sub-opcode 111; operands pi5-s12, reg5; 4 bytes**



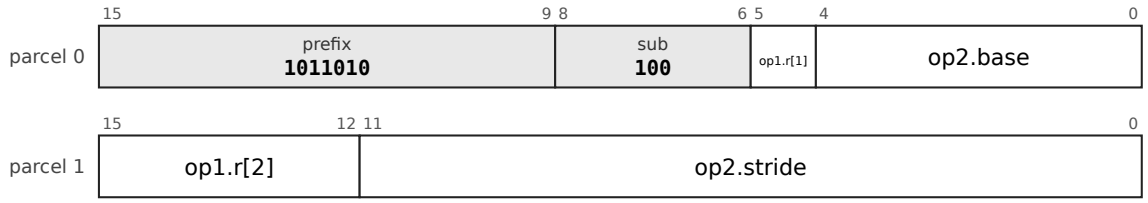

**MOVUB: LDF.PD12, prefix 1010011, sub-opcode 111; operands pd5-s12, reg5; 4 bytes**




**MOVUB: STF.PI12, prefix 1011001, sub-opcode 100; operands reg5, pi5-s12; 4 bytes**



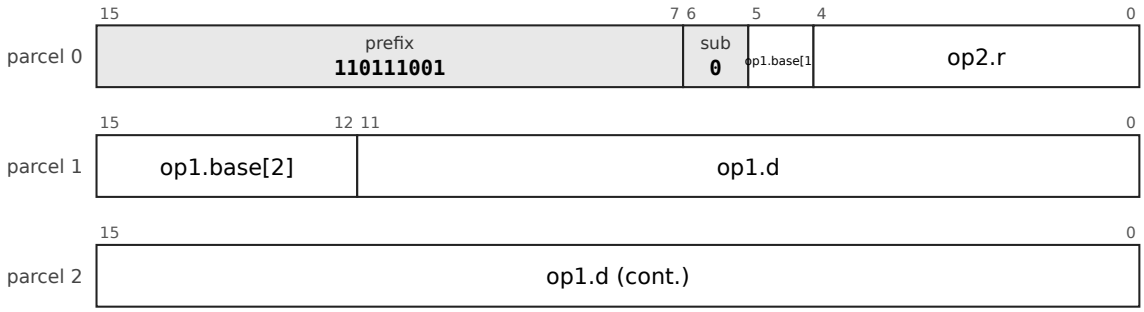

**MOVUB: STF.PD12, prefix 1011010, sub-opcode 100; operands reg5, pd5-s12; 4 bytes**



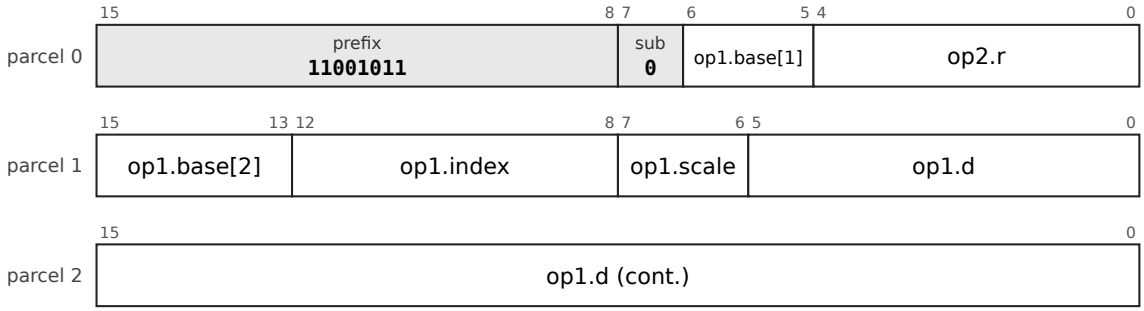

Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

## *MOVUW*

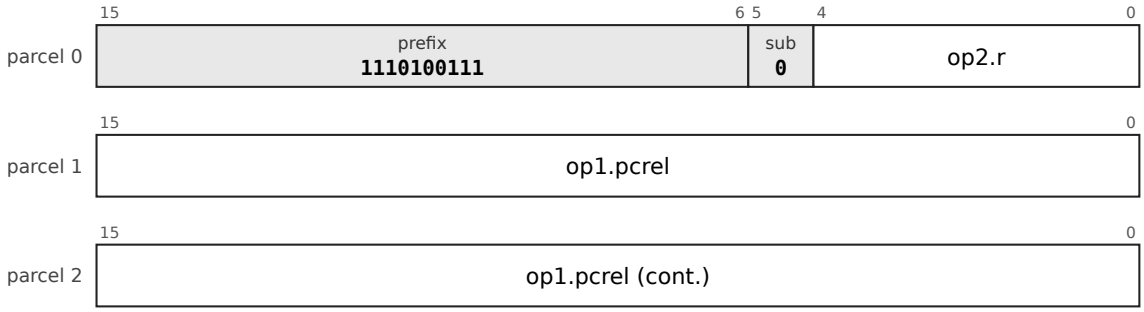
**MOVUW: LDF.B28, prefix 110111001, sub-opcode 0; operands b5d28s-x1, reg5; 6 bytes**



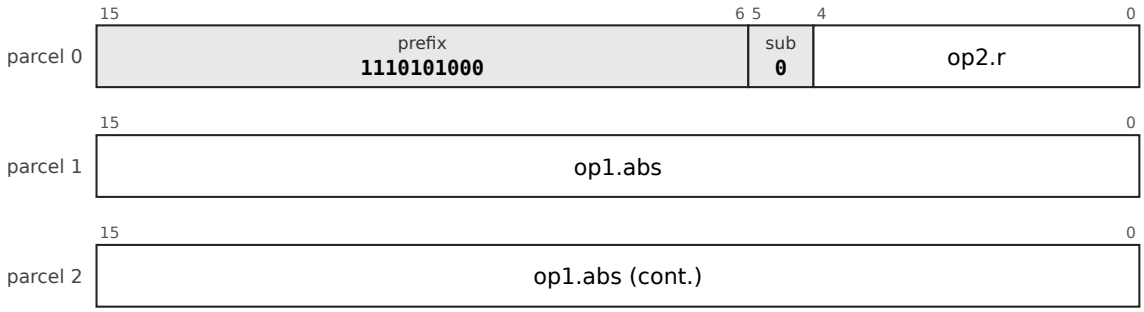

**MOVUW: LDF.X22, prefix 11001011, sub-opcode 0; operands x5-5d22s, reg5; 6 bytes**



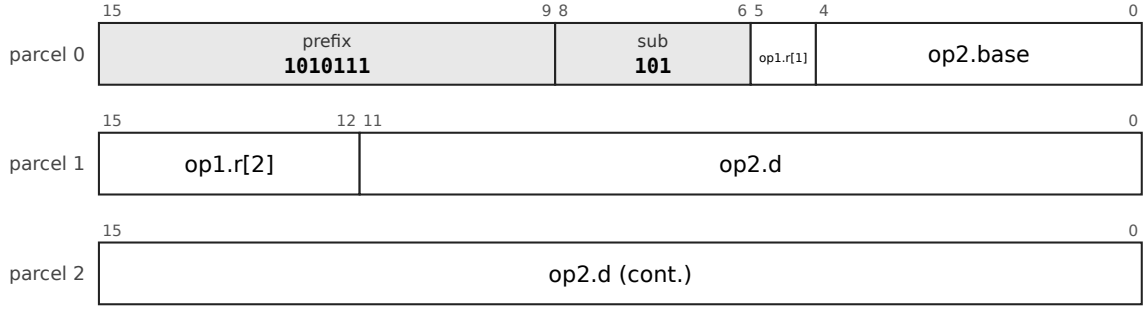

**MOVUW: LDF.PC32, prefix 1110100111, sub-opcode 0; operands pc32-u1, reg5; 6 bytes**



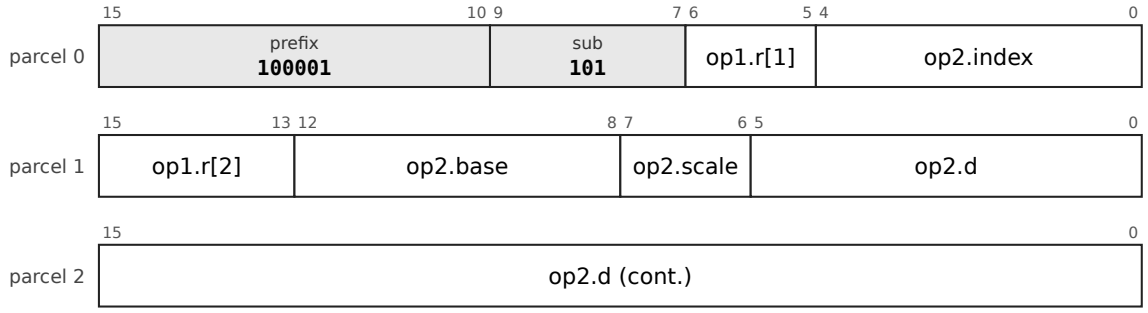

**MOVUW: LDF.A32, prefix 1110101000, sub-opcode 0; operands abs32, reg5; 6 bytes**



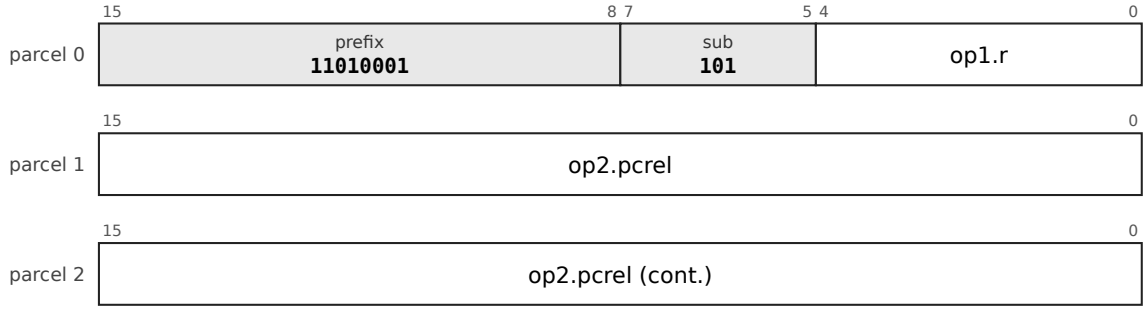
**MOVUW: STF.B28, prefix 1010111, sub-opcode 101; operands reg5, b5d28s-x1; 6 bytes**



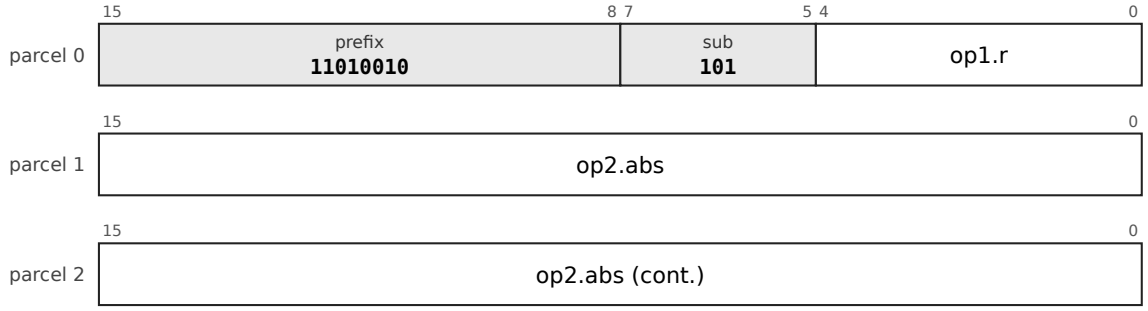

**MOVUW: STF.X22, prefix 100001, sub-opcode 101; operands reg5, x5-5d22s; 6 bytes**



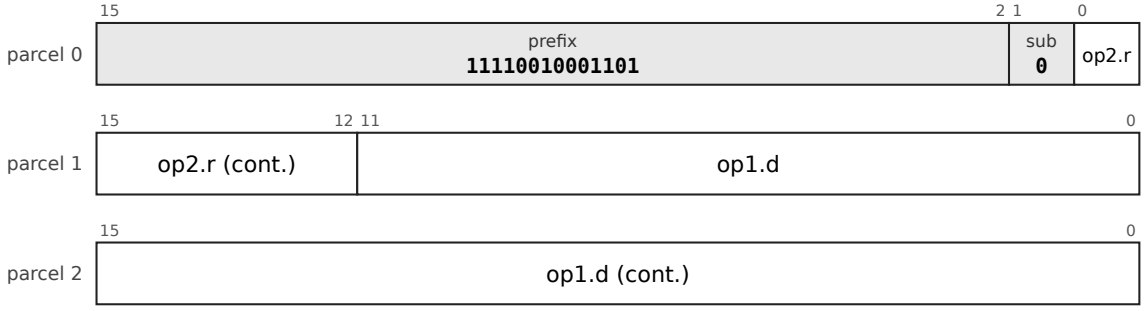

**MOVUW: STF.PC32, prefix 11010001, sub-opcode 101; operands reg5, pc32-u1; 6 bytes**



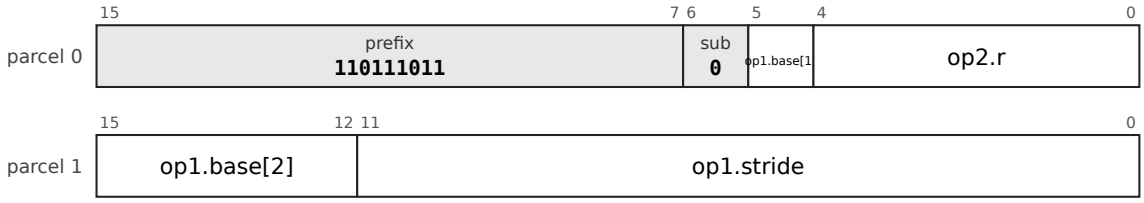

**MOVUW: STF.A32, prefix 11010010, sub-opcode 101; operands reg5, abs32; 6 bytes**



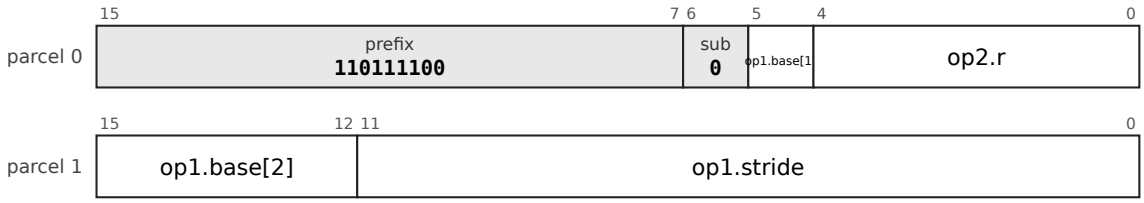
**MOVUW: DPF.D28, prefix 11110010001101, sub-opcode 0; operands dpd28s-x1, reg5; 6 bytes**



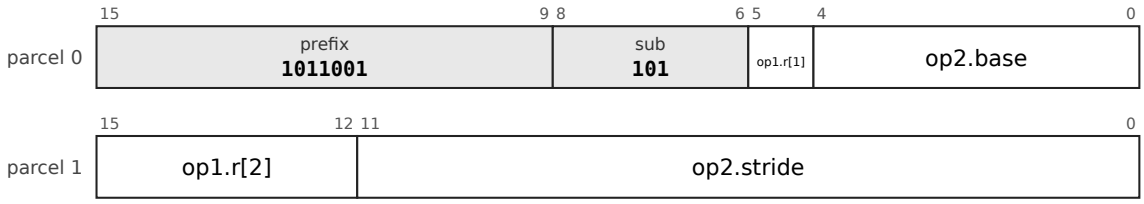

**MOVUW: LDF.PI12, prefix 110111011, sub-opcode 0; operands pi5-s12, reg5; 4 bytes**



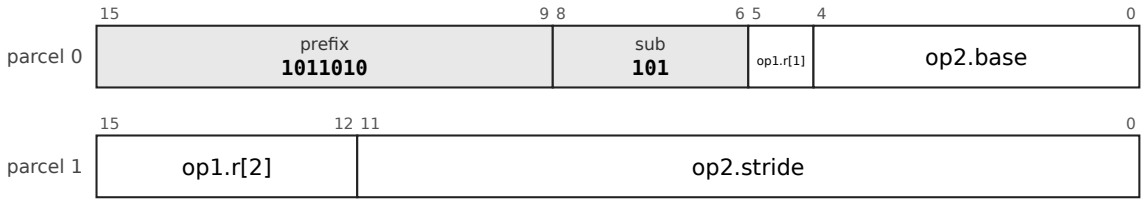

**MOVUW: LDF.PD12, prefix 110111100, sub-opcode 0; operands pd5-s12, reg5; 4 bytes**




**MOVUW: STF.PI12, prefix 1011001, sub-opcode 101; operands reg5, pi5-s12; 4 bytes**




**MOVUW: STF.PD12, prefix 1011010, sub-opcode 101; operands reg5, pd5-s12; 4 bytes**

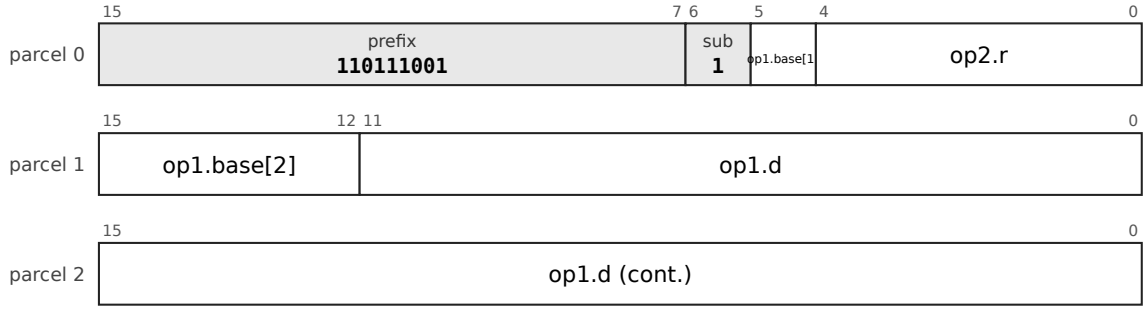



```
MOVUW, MOVUL, MOVUB likewise
```

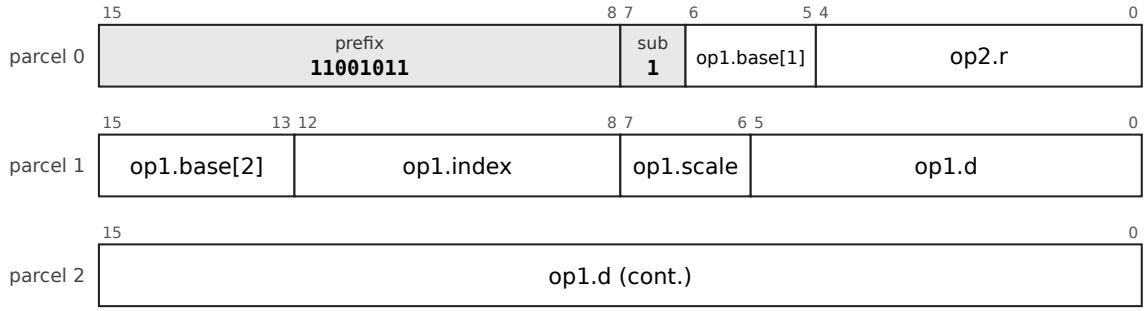
Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

## *MOVUL*

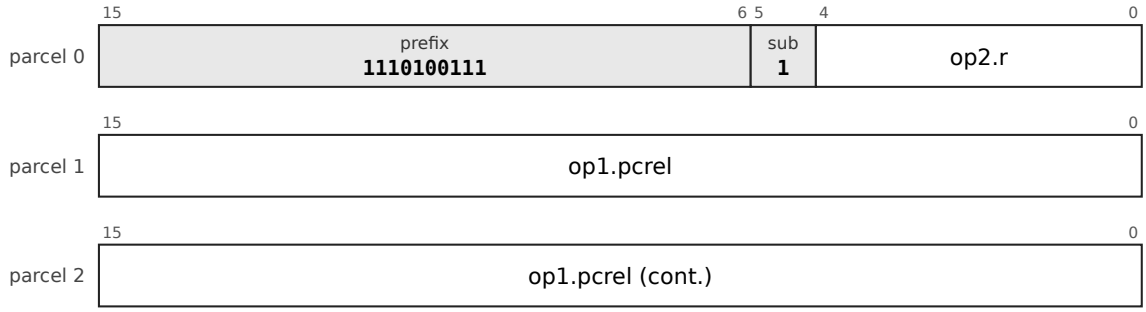
**MOVUL: LDF.B28, prefix 110111001, sub-opcode 1; operands b5d28s-x1, reg5; 6 bytes**



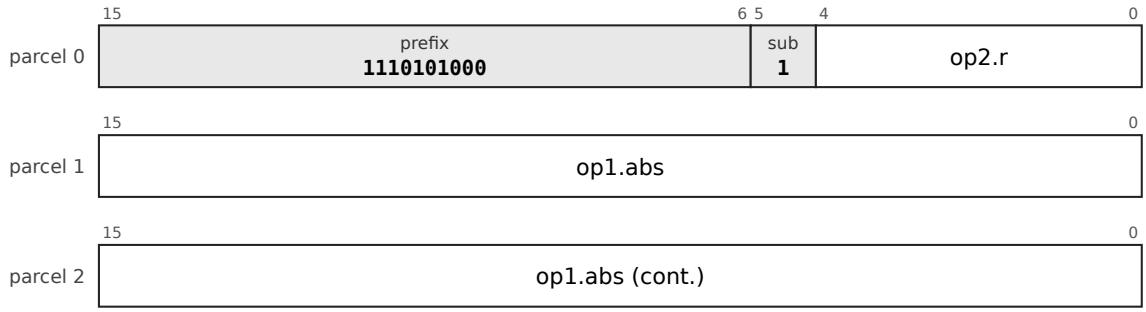

**MOVUL: LDF.X22, prefix 11001011, sub-opcode 1; operands x5-5d22s, reg5; 6 bytes**



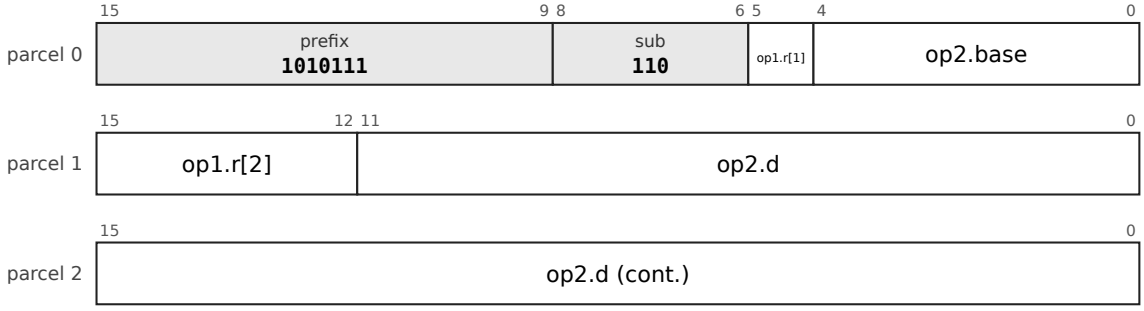

**MOVUL: LDF.PC32, prefix 1110100111, sub-opcode 1; operands pc32-u1, reg5; 6 bytes**



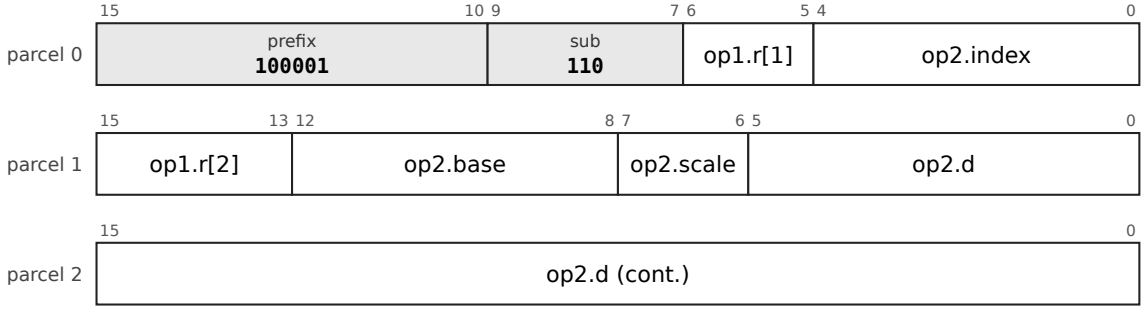

**MOVUL: LDF.A32, prefix 1110101000, sub-opcode 1; operands abs32, reg5; 6 bytes**



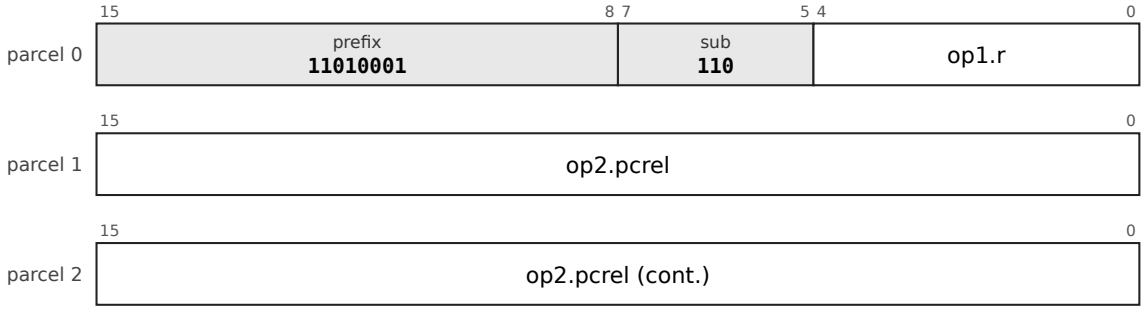
**MOVUL: STF.B28, prefix 1010111, sub-opcode 110; operands reg5, b5d28s-x1; 6 bytes**



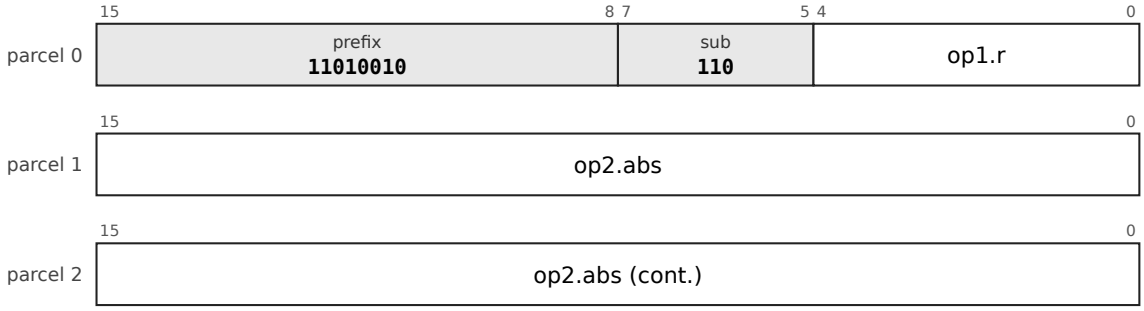

**MOVUL: STF.X22, prefix 100001, sub-opcode 110; operands reg5, x5-5d22s; 6 bytes**



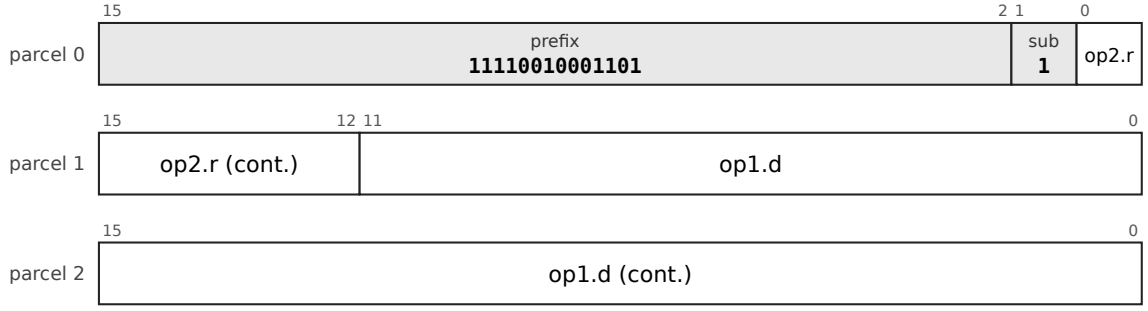

**MOVUL: STF.PC32, prefix 11010001, sub-opcode 110; operands reg5, pc32-u1; 6 bytes**



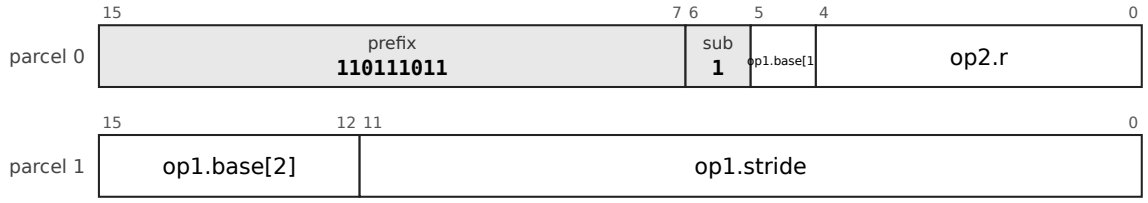

**MOVUL: STF.A32, prefix 11010010, sub-opcode 110; operands reg5, abs32; 6 bytes**



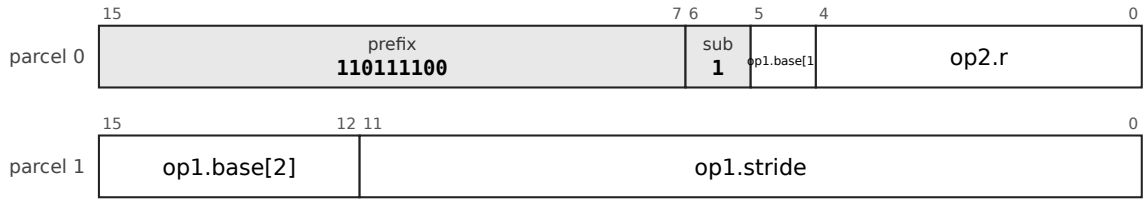
**MOVUL: DPF.D28, prefix 11110010001101, sub-opcode 1; operands dpd28s-x1, reg5; 6 bytes**



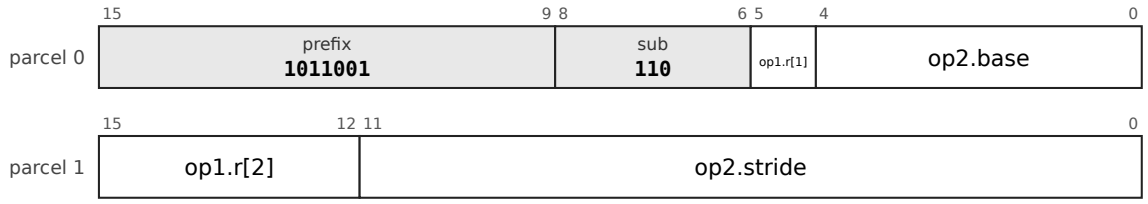

**MOVUL: LDF.PI12, prefix 110111011, sub-opcode 1; operands pi5-s12, reg5; 4 bytes**



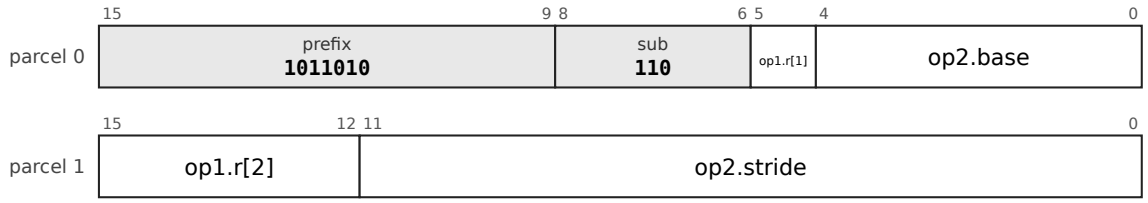

**MOVUL: LDF.PD12, prefix 110111100, sub-opcode 1; operands pd5-s12, reg5; 4 bytes**




**MOVUL: STF.PI12, prefix 1011001, sub-opcode 110; operands reg5, pi5-s12; 4 bytes**



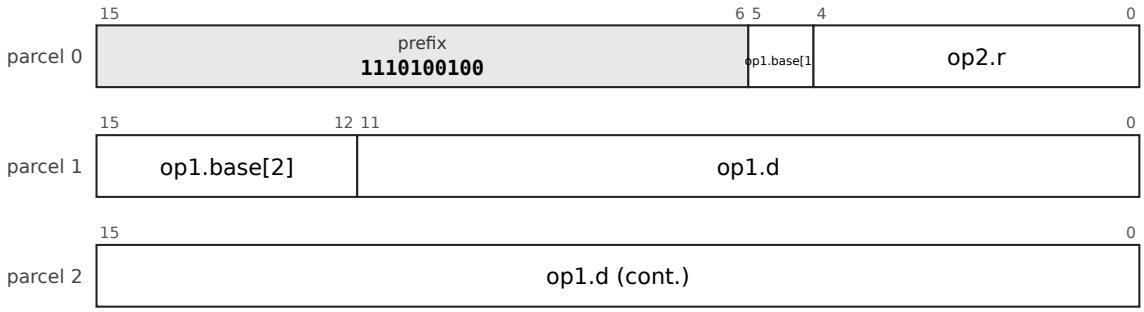

**MOVUL: STF.PD12, prefix 1011010, sub-opcode 110; operands reg5, pd5-s12; 4 bytes**



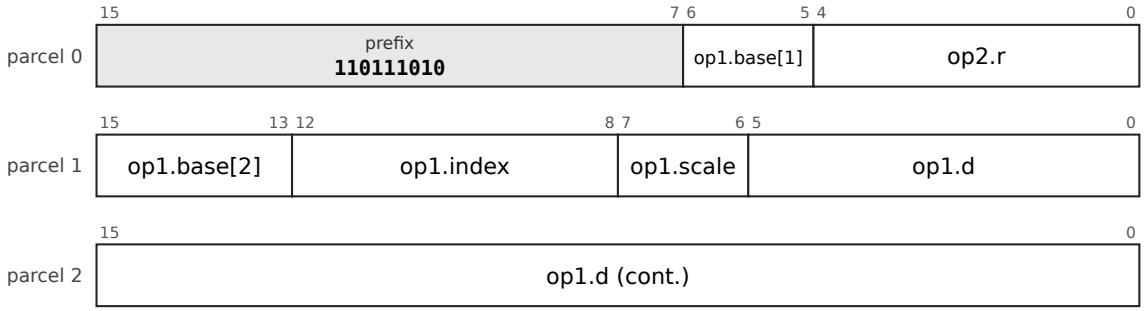

Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

## *MOVUQ*

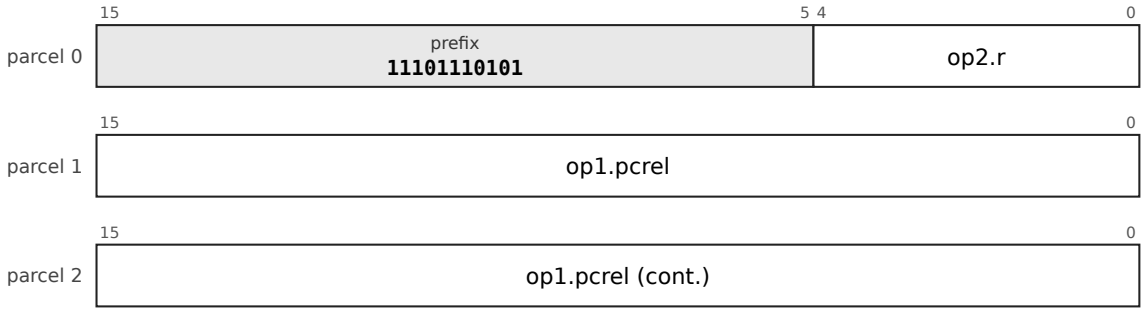
**MOVUQ: LDF.B28, prefix 1110100100; operands b5d28s-x1, reg5; 6 bytes**



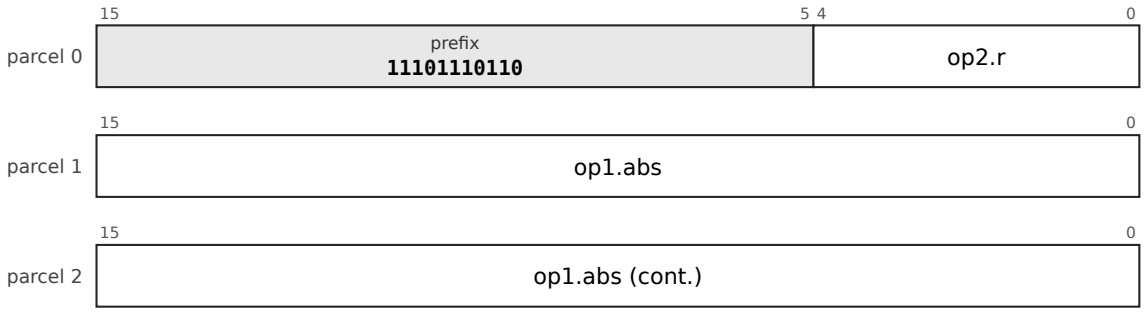

**MOVUQ: LDF.X22, prefix 110111010; operands x5-5d22s, reg5; 6 bytes**



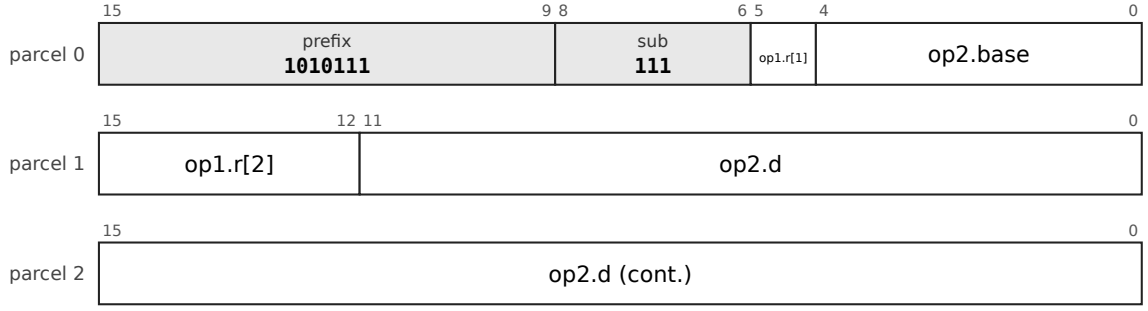

**MOVUQ: LDF.PC32, prefix 11101110101; operands pc32-u1, reg5; 6 bytes**



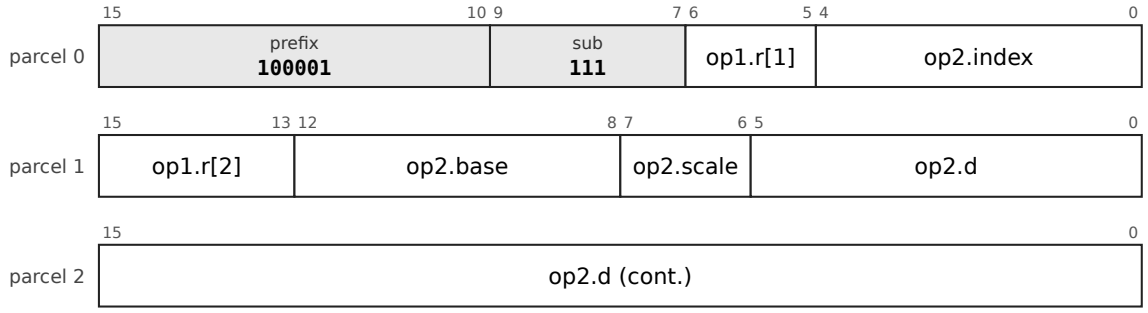

**MOVUQ: LDF.A32, prefix 11101110110; operands abs32, reg5; 6 bytes**



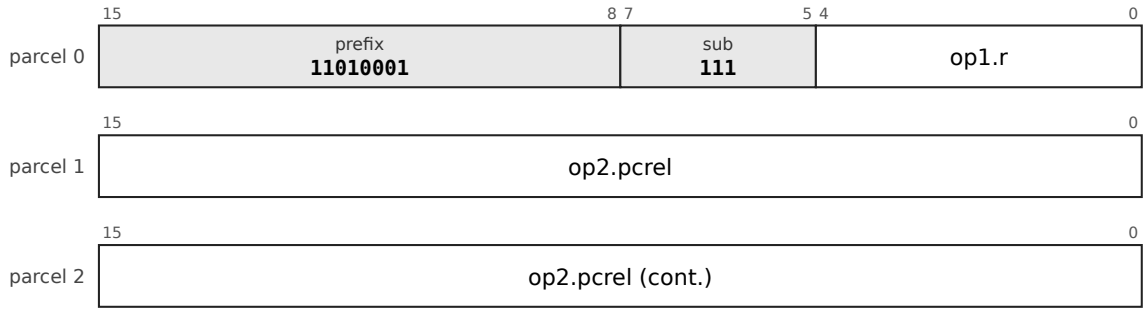
**MOVUQ: STF.B28, prefix 1010111, sub-opcode 111; operands reg5, b5d28s-x1; 6 bytes**



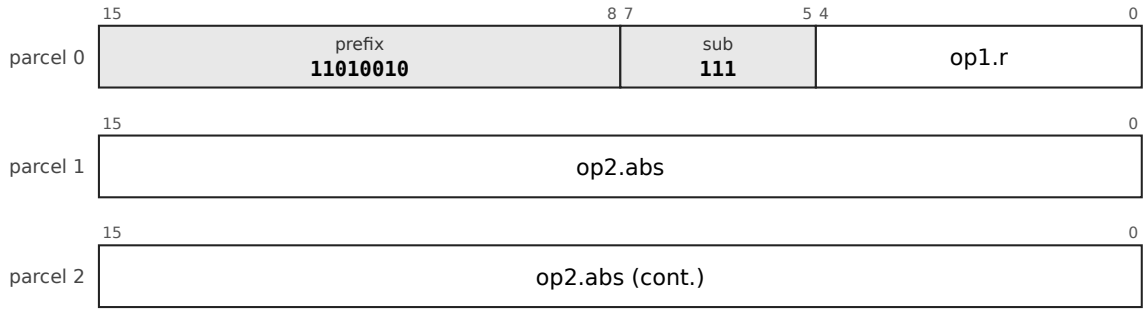

**MOVUQ: STF.X22, prefix 100001, sub-opcode 111; operands reg5, x5-5d22s; 6 bytes**



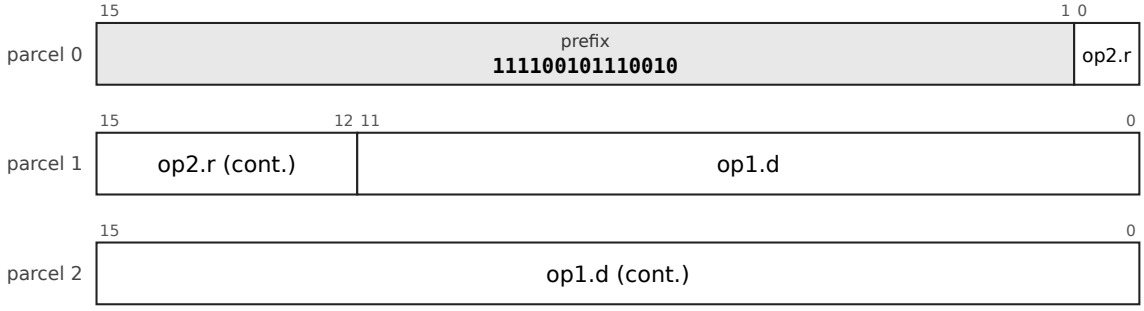

**MOVUQ: STF.PC32, prefix 11010001, sub-opcode 111; operands reg5, pc32-u1; 6 bytes**



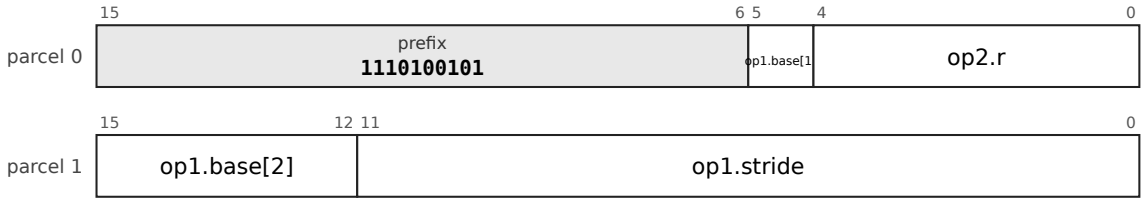

**MOVUQ: STF.A32, prefix 11010010, sub-opcode 111; operands reg5, abs32; 6 bytes**



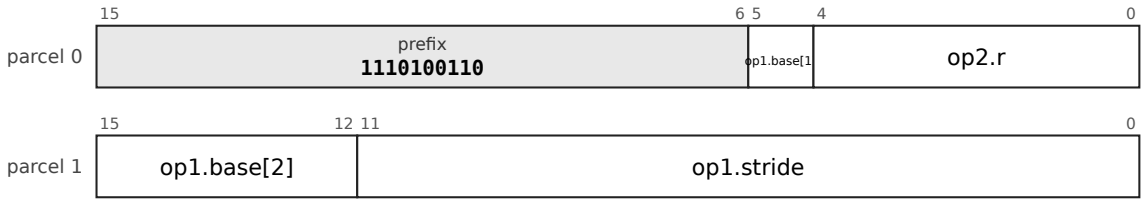
**MOVUQ: DPF.D28, prefix 111100101110010; operands dpd28s-x1, reg5; 6 bytes**



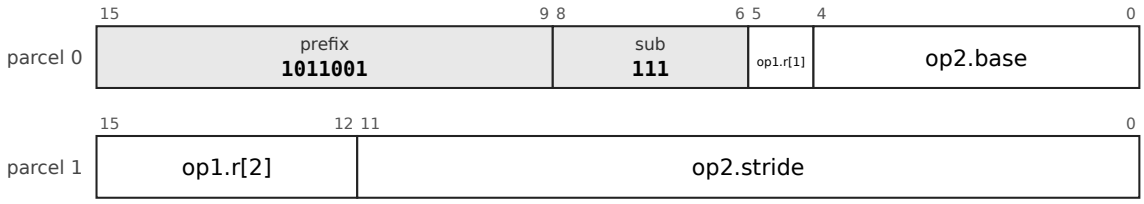

**MOVUQ: LDF.PI12, prefix 1110100101; operands pi5-s12, reg5; 4 bytes**



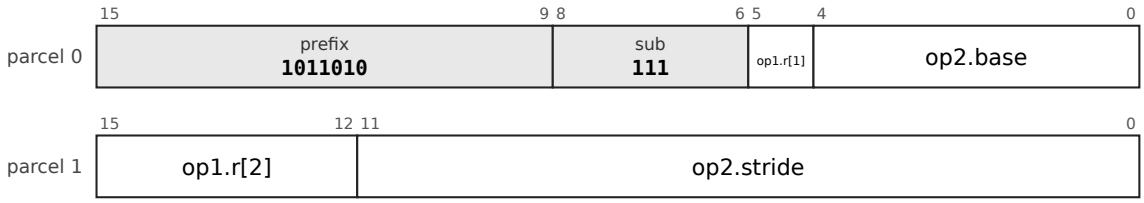

**MOVUQ: LDF.PD12, prefix 1110100110; operands pd5-s12, reg5; 4 bytes**




**MOVUQ: STF.PI12, prefix 1011001, sub-opcode 111; operands reg5, pi5-s12; 4 bytes**




**MOVUQ: STF.PD12, prefix 1011010, sub-opcode 111; operands reg5, pd5-s12; 4 bytes**

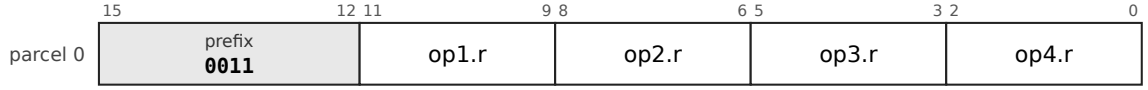



```
MOVUQ rm, Rd
MOVUQ Rs, rm
```

Operation, flags and exceptions as for the entry MOVU above; 13 layout forms, each a chart of its own (above).

## MOV2: Two register moves

**MOV2: FX.MOV2, prefix 0011; operands reg3, reg3, reg3, reg3; 2 bytes**

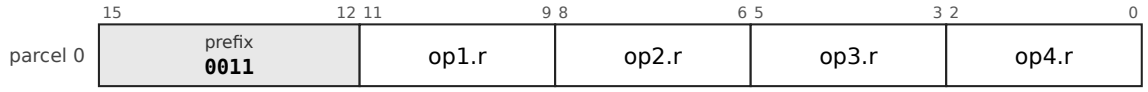



**Mnemonics:** `MOV2`

### Syntax

```
MOV2 Ra, Rb, Rc, Rd
```

### Encoding




The chart is the leaf `FX-MOV2-1`, FX.MOV2, prefix `0011`. `op1` is `reg3`: register, 3 bits (an index into the hot-register table). `op2` is `reg3`: register, 3 bits (an index into the hot-register table). `op3` is `reg3`: register, 3 bits (an index into the hot-register table). `op4` is `reg3`: register, 3 bits (an index into the hot-register table).

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `FX-MOV2-1`.

### Operation

```
Rb ← Ra
Rd ← Rc          ; in this order
```

**Flags** None written.

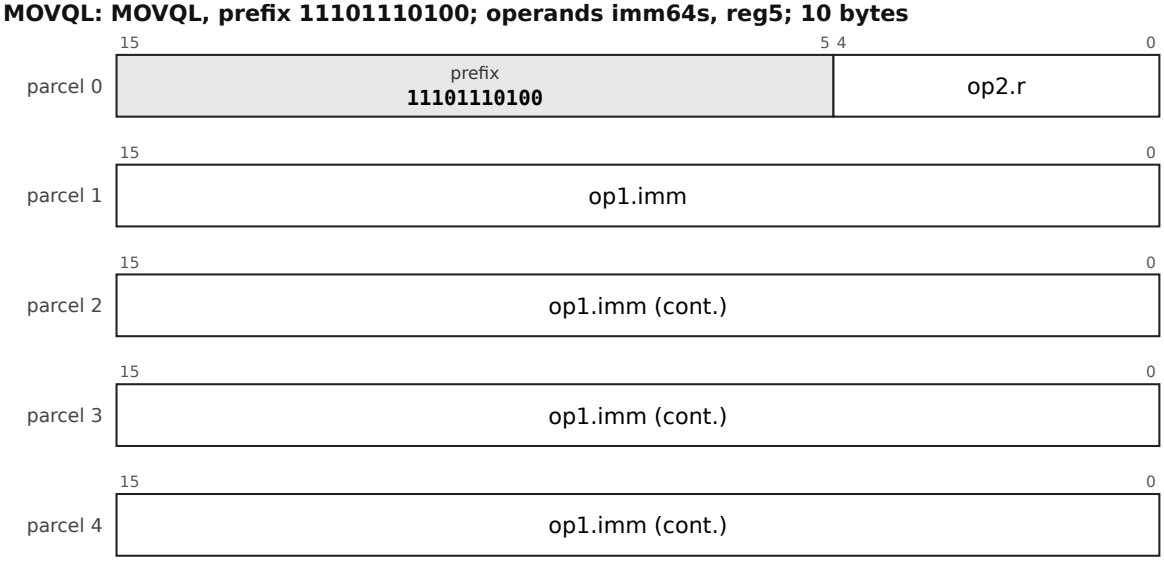
**Exceptions** None.

*(D)* `MOVQ Ra, Rb` followed by `MOVQ Rc, Rd`, fused (errata 87). The four operands are 3-bit fields: only the eight hot registers (`R0 R1 R8 R2 SP R3 R9 R10`, the ABI table of the encoding) can be named, and the instruction takes one sixteenth of the first-parcel space, which is why the grammar gives it a prefix of four bits. If `Rb` is `Rc` the second move reads the new value, as two instructions would.

**Why it is here.** The author decided the four fused pairs in errata 87 (`movcall`, `popmret`, `popret`, `mov2`) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of `docs/encoding-proposal.md` section 8 (static size of the training corpus, in the encoding): `MOVQ Ra, Rb` followed by `MOVQ Rc, Rd` occurs 6,742 times and saves 0.50%, the smallest gain of the four and the dearest in code space: four 3-bit register fields use one sixteenth of the first-parcel space and can name only the eight hot registers. The encoding study lists both as costs, and one instruction that writes two registers (the rename width of an out-of-order core).

**Cf.** Appendix D (the hot-register tables).

## MOVQL: Move a 64-bit literal

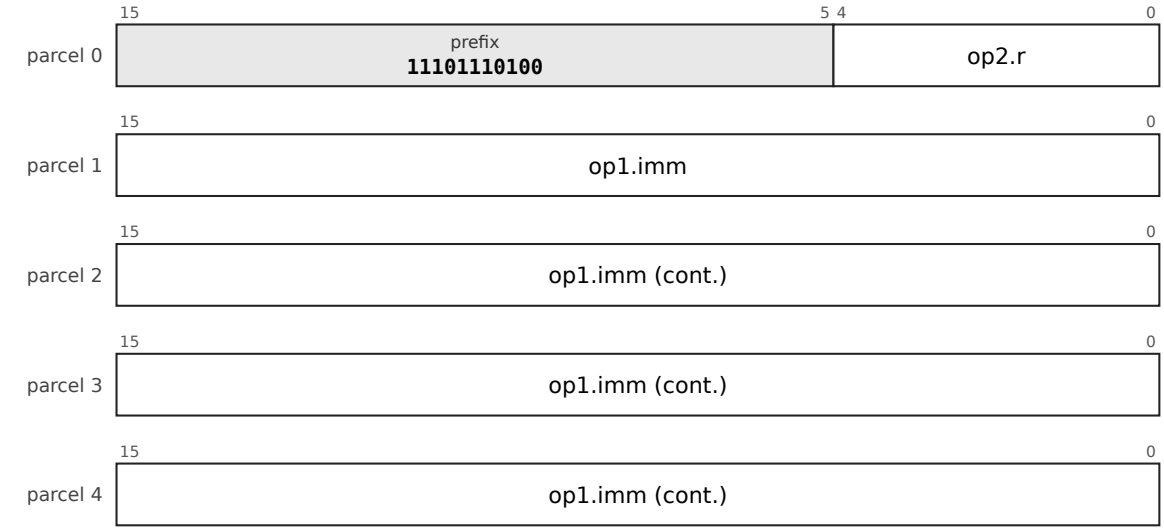



**Mnemonics:** `MOVQL`

### Syntax

```
MOVQL #imm64, Rd
```

### Encoding




The chart is the leaf `MOVQL-1`, MOVQL, prefix `11101110100`. `op1` is `imm64s`: signed 64-bit immediate, -9223372036854775808 to 9223372036854775807. `op2` is `reg5`: register, 5 bits.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `MOVQL-1`.

### Operation

```
Rd ← imm64
```

**Flags** None written.

**Exceptions** None.

*(D)* Errata 85: `MOVQL #imm64, Rd` is accepted, and breaking the three-parcel rule for it is acceptable because a 64-bit literal is not counted as part of the instruction. The instruction is five parcels, ten bytes: the literal is inside the instruction, so the length is still known from parcel 0 and the fetch has no exception for a trailing literal (`docs/encoding-proposal.md` section 3). The assembler uses it for a `MOVQ #imm, Rd` that fits no `MOVQ` form (errata 87, assumption 6).

**Why it is here.** The author accepted it in the second round of security answers (errata 85, proposal P18). A 64-bit constant otherwise comes from the literal pool; the literal inside the instruction is one instruction, and it is not counted against the three-parcel rule because it is data, not instruction. The encoding work found no use of a trailing literal worth its exception to the fetch rule, so the literal is part of a five-parcel instruction (`docs/encoding-proposal.md` section 3).

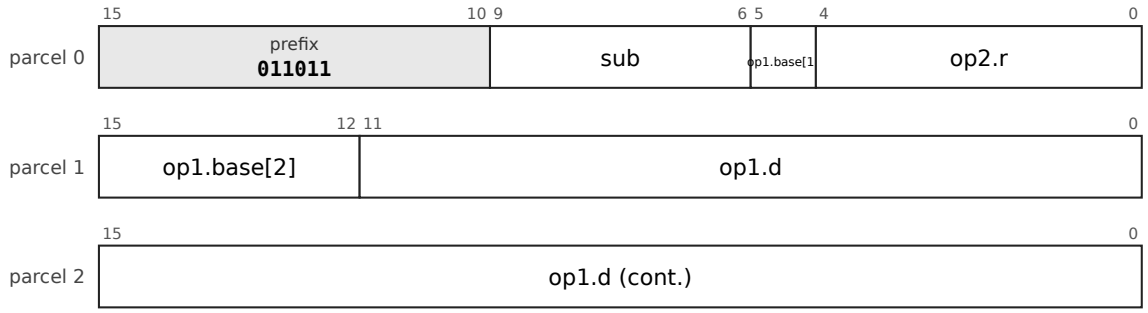
## MOVD: Deferred-fault load

**Mnemonics:** `MOVQ.D`, `MOVL.D`, `MOVW.D`, `MOVB.D`, `MOVZL.D`, `MOVZW.D`, `MOVZB.D`

### Syntax

```
MOVQ.D d(Rn), Rd
MOVL.D ..., MOVW.D ..., MOVB.D ...
MOVZL.D ..., MOVZW.D ..., MOVZB.D ...
```

### Encoding




The chart is the leaf `LDU-B28-1`, LDU.B28, prefix `011011`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDU-B28-1`, `LDU-B12-1`, `LDU-X6-1`.

### Operation

```
Rd ← M[EA]                    ; as the load without .D, when the access succeeds
when the access faults: the fault is precise as always; the kernel's handler repairs the cause and restarts,
    or, if the access is invalid, marks Rd as poisoned in the saved state and goes on at the next
instruction
```

**Flags** None written.

**Exceptions** The ordinary faults of a load (page fault, protection fault, alignment fault, bus error), precise; what the handler does with them is the difference.

*(D)* The deferred-fault load is item 10 of `docs/study-proposals.md` (the poison bit), whose three leanings the author accepted (errata 86): the fault is taken at the instruction that writes the flags from a poisoned source; the kernel decides a deferred load's fault; the kernel dispatcher, not the hardware, tests the argument registers for poison. Base and indexed operands only (a mode that needs a base register needs `LEA` first). Details pending (J.9.12).

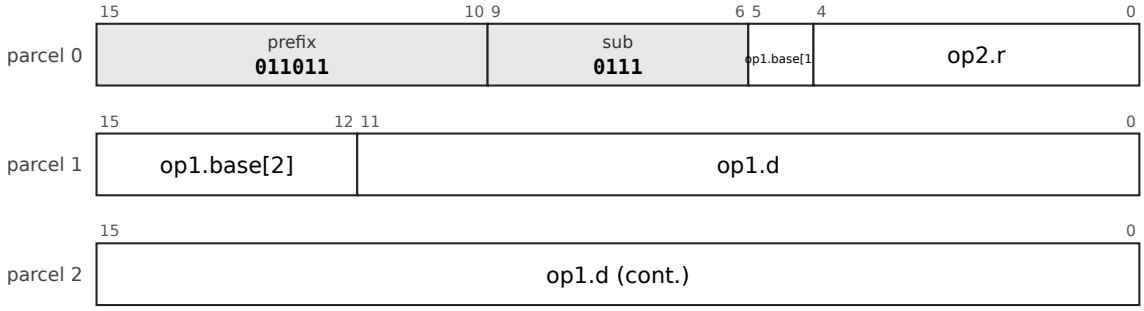
**Why it is here.** The poison bit of item 10 of the study (`docs/study-proposals.md`) wants a load that can be hoisted above its guard: it either gives a value or a poisoned value, and a use of a poisoned value is the fault the guard would have prevented.

That removes the recovery code of if-conversion and software pipelining, gives a deterministic Spectre version 1 form (a deferred, bounds-checked address computation yields poison, not a branch the predictor can train), and makes a chain of checks one fault. The author accepted the three leanings of the proposal (errata 86).

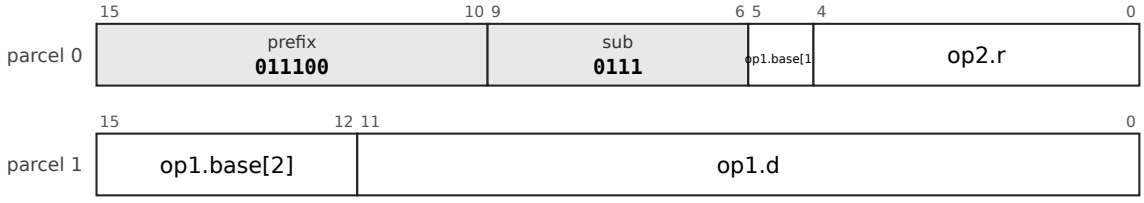
**Cf.** 6.4, docs/study-proposals.md item 10.

### *MOVQ.D*

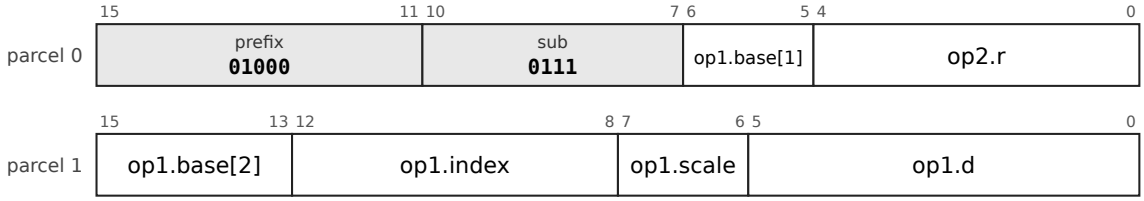
**MOVQ.D: LDU.B28, prefix 011011, sub-opcode 0111; operands b5d28s-x1, reg5; 6 bytes**




**MOVQ.D: LDU.B12, prefix 011100, sub-opcode 0111; operands b5d12s-x1, reg5; 4 bytes**




**MOVQ.D: LDU.X6, prefix 01000, sub-opcode 0111; operands x5-5d6s, reg5; 4 bytes**

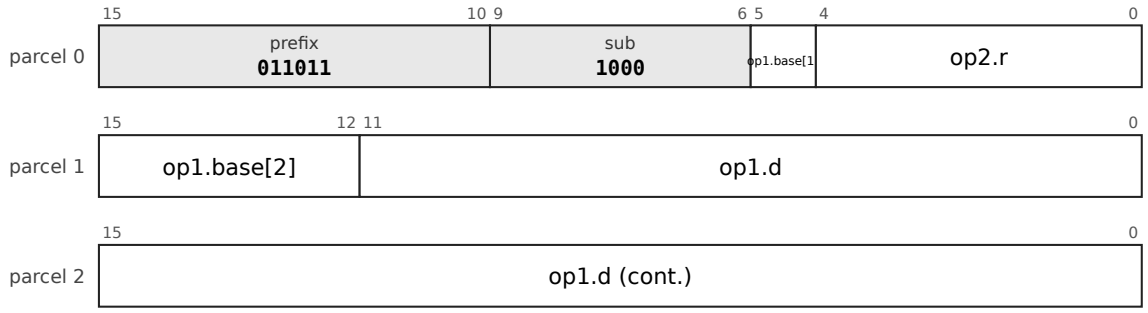



```
MOVQ.D d(Rn), Rd
```

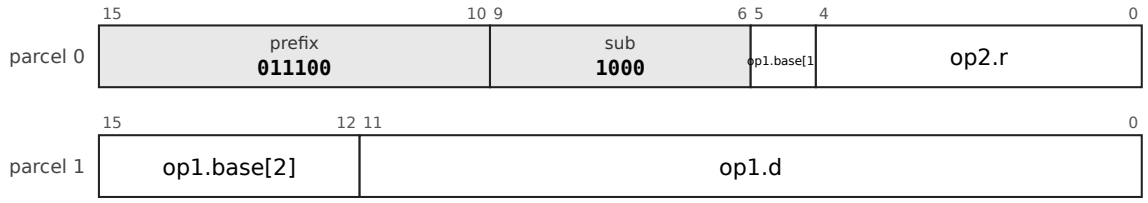
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVL.D*

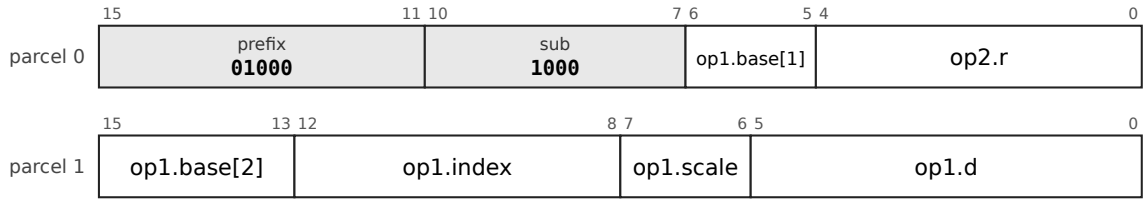
**MOVL.D: LDU.B28, prefix 011011, sub-opcode 1000; operands b5d28s-x1, reg5; 6 bytes**




**MOVL.D: LDU.B12, prefix 011100, sub-opcode 1000; operands b5d12s-x1, reg5; 4 bytes**




**MOVL.D: LDU.X6, prefix 01000, sub-opcode 1000; operands x5-5d6s, reg5; 4 bytes**

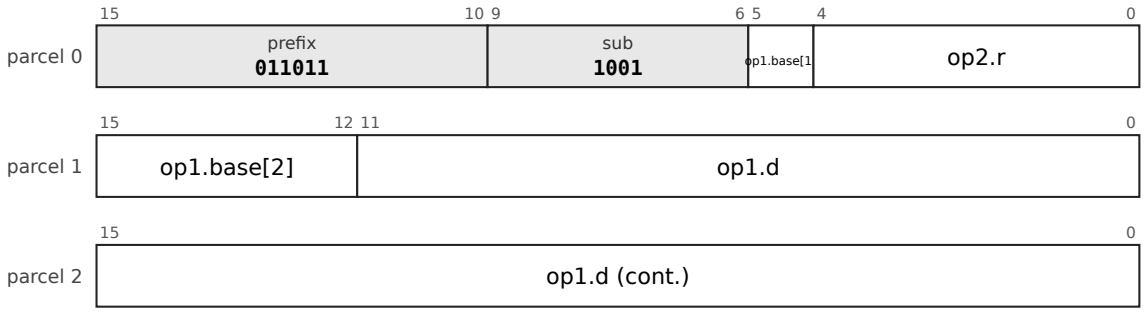



```
MOVL.D ..., MOVW.D ..., MOVB.D ...
```

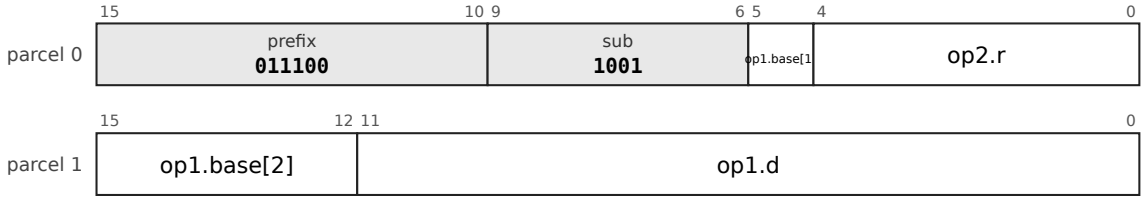
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVW.D*

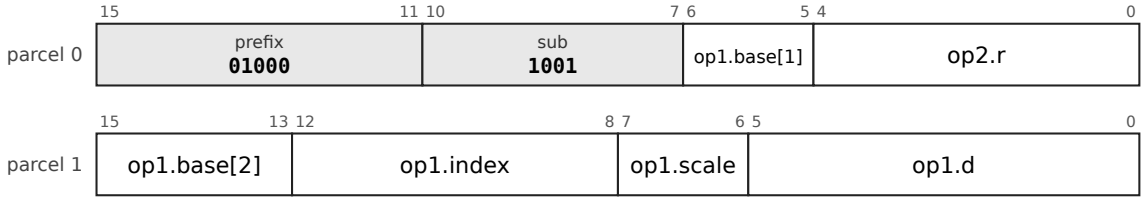
**MOVW.D: LDU.B28, prefix 011011, sub-opcode 1001; operands b5d28s-x1, reg5; 6 bytes**




**MOVW.D: LDU.B12, prefix 011100, sub-opcode 1001; operands b5d12s-x1, reg5; 4 bytes**



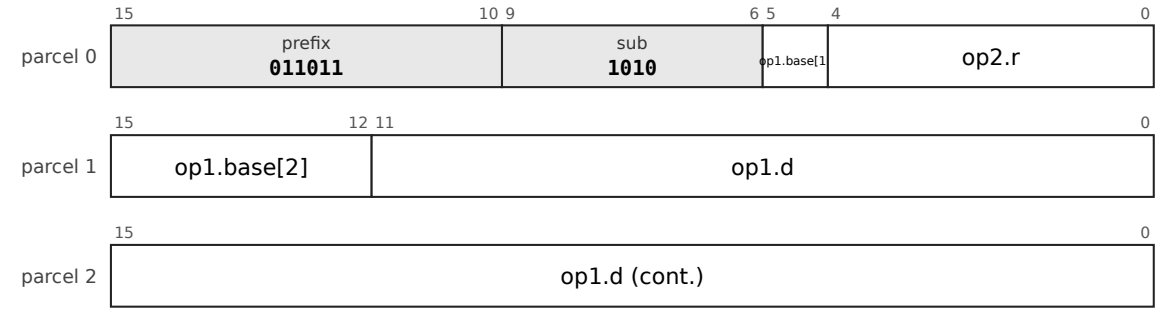

**MOVW.D: LDU.X6, prefix 01000, sub-opcode 1001; operands x5-5d6s, reg5; 4 bytes**



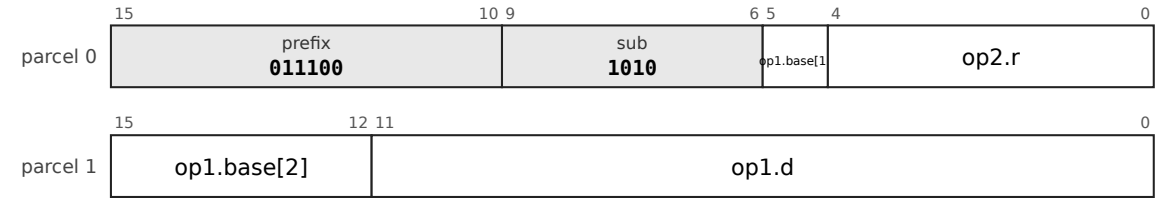

Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVB.D*

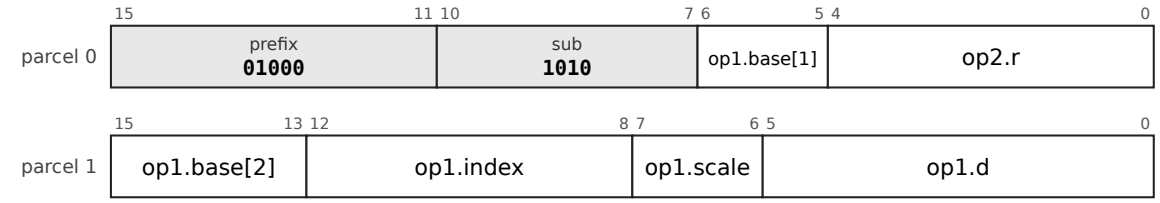
**MOVB.D: LDU.B28, prefix 011011, sub-opcode 1010; operands b5d28s-x1, reg5; 6 bytes**




**MOVB.D: LDU.B12, prefix 011100, sub-opcode 1010; operands b5d12s-x1, reg5; 4 bytes**



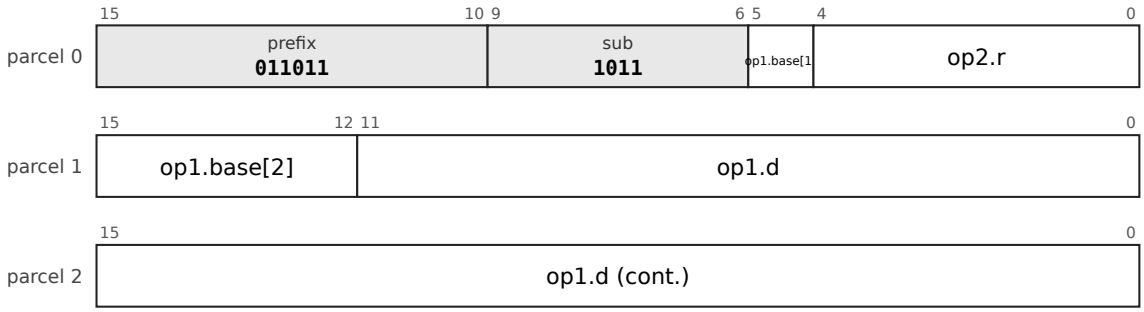

**MOVB.D: LDU.X6, prefix 01000, sub-opcode 1010; operands x5-5d6s, reg5; 4 bytes**



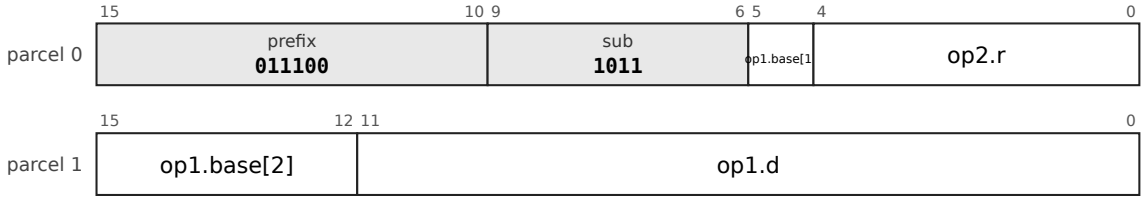

Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVZL.D*

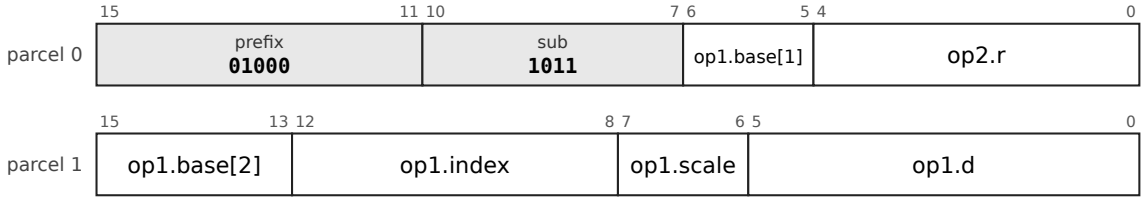
**MOVZL.D: LDU.B28, prefix 011011, sub-opcode 1011; operands b5d28s-x1, reg5; 6 bytes**




**MOVZL.D: LDU.B12, prefix 011100, sub-opcode 1011; operands b5d12s-x1, reg5; 4 bytes**




**MOVZL.D: LDU.X6, prefix 01000, sub-opcode 1011; operands x5-5d6s, reg5; 4 bytes**

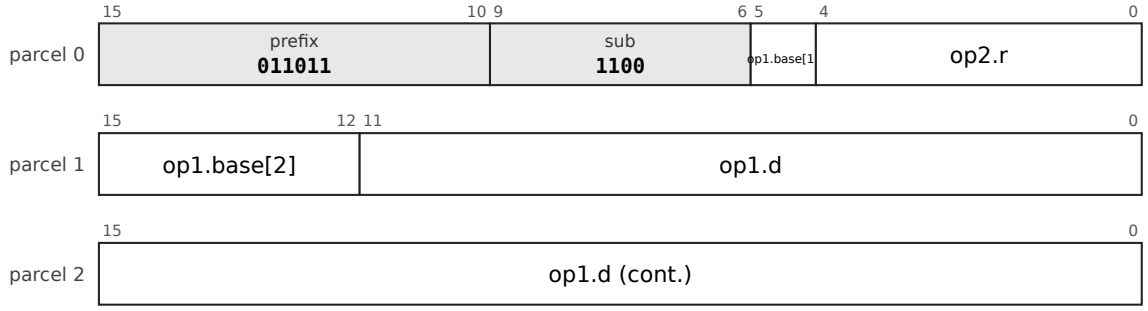



```
MOVZL.D ..., MOVZW.D ..., MOVZB.D ...
```

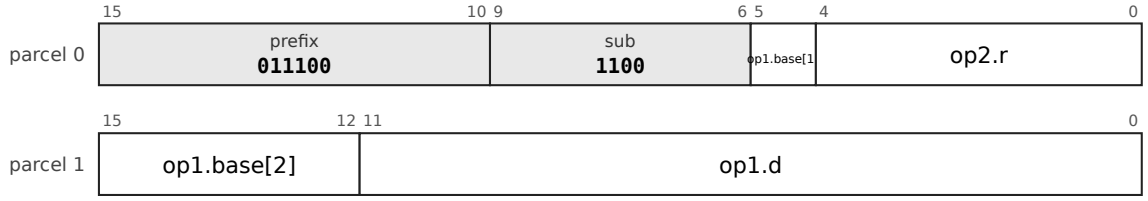
Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVZW.D*

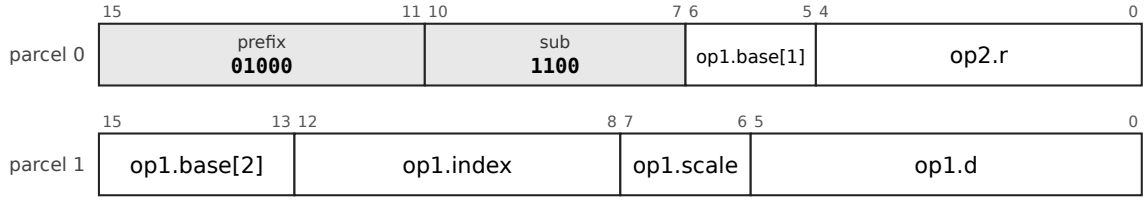
**MOVZW.D: LDU.B28, prefix 011011, sub-opcode 1100; operands b5d28s-x1, reg5; 6 bytes**




**MOVZW.D: LDU.B12, prefix 011100, sub-opcode 1100; operands b5d12s-x1, reg5; 4 bytes**



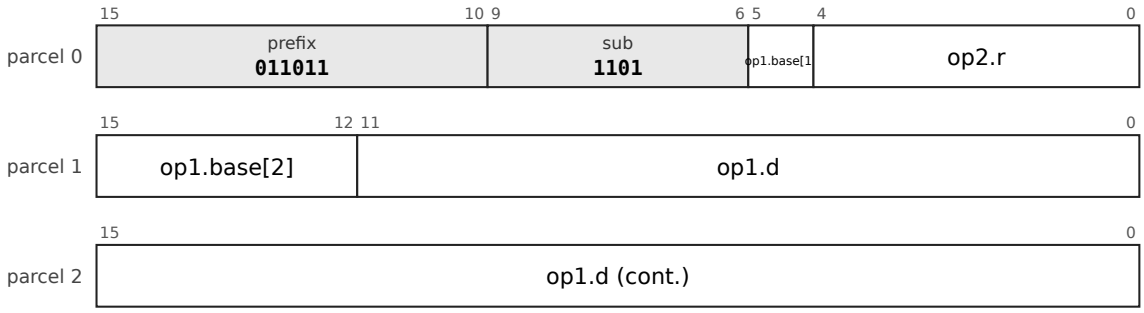

**MOVZW.D: LDU.X6, prefix 01000, sub-opcode 1100; operands x5-5d6s, reg5; 4 bytes**



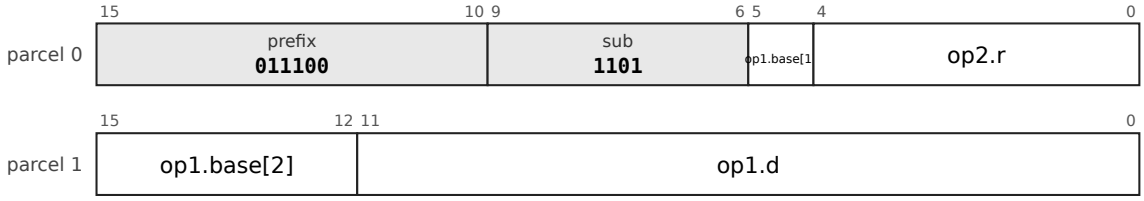

Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

### *MOVZB.D*

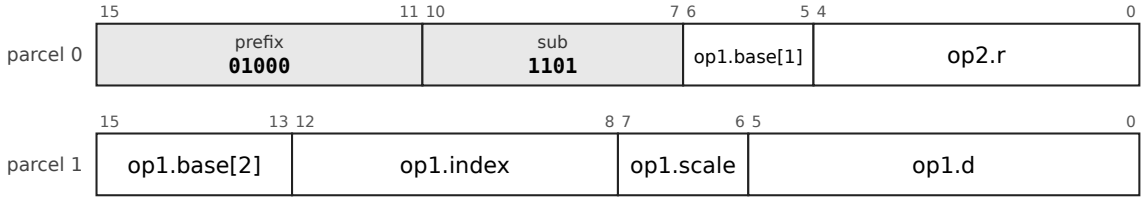
**MOVZB.D: LDU.B28, prefix 011011, sub-opcode 1101; operands b5d28s-x1, reg5; 6 bytes**




**MOVZB.D: LDU.B12, prefix 011100, sub-opcode 1101; operands b5d12s-x1, reg5; 4 bytes**




**MOVZB.D: LDU.X6, prefix 01000, sub-opcode 1101; operands x5-5d6s, reg5; 4 bytes**




Operation, flags and exceptions as for the entry MOVD above; 3 layout forms, each a chart of its own (above).

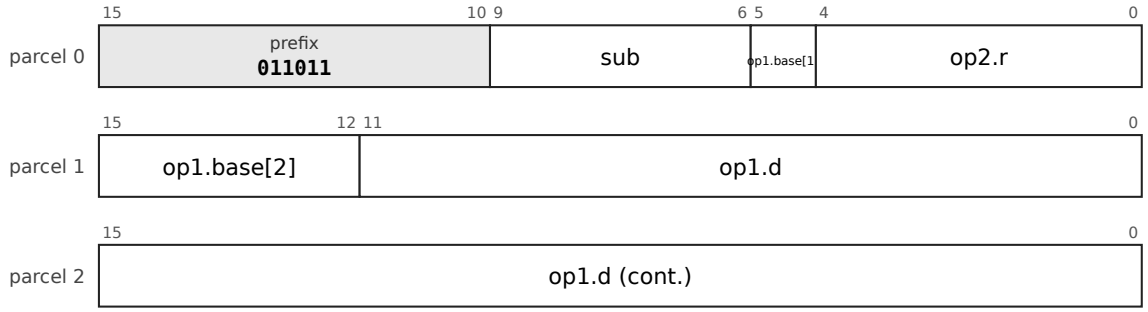
## MOVUSER: User-qualified load and store

**Mnemonics:** `MOVQ.U`, `MOVL.U`, `MOVW.U`, `MOVB.U`, `MOVZL.U`, `MOVZW.U`, `MOVZB.U`

### Syntax

```
MOVQ.U d(Rn), Rd
MOVQ.U Rs, d(Rn)
MOVL.U ..., MOVW.U ..., MOVB.U ...
MOVZL.U ..., MOVZW.U ..., MOVZB.U ...    ; loads only
```

### Encoding




The chart is the leaf `LDU-B28-1`, LDU.B28, prefix `011011`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDU-B28-1`, `STU-B28-1`, `LDU-B12-1`, `STU-B12-1`, `LDU-X6-1`, `STU-X6-1`.

### Operation

```
the access of MOVQ, MOVL, MOVW, MOVB and MOVZ* with the operand checked as a user-mode access:
the page must be present, have U = 1, and permit R (a load) or W (a store); a kernel page takes the
protection abort
whatever PSW.UA says
```

**Flags** None written.

**Exceptions** The faults of the access, as a user-mode access. In user mode the encoding is a privileged-instruction abort.

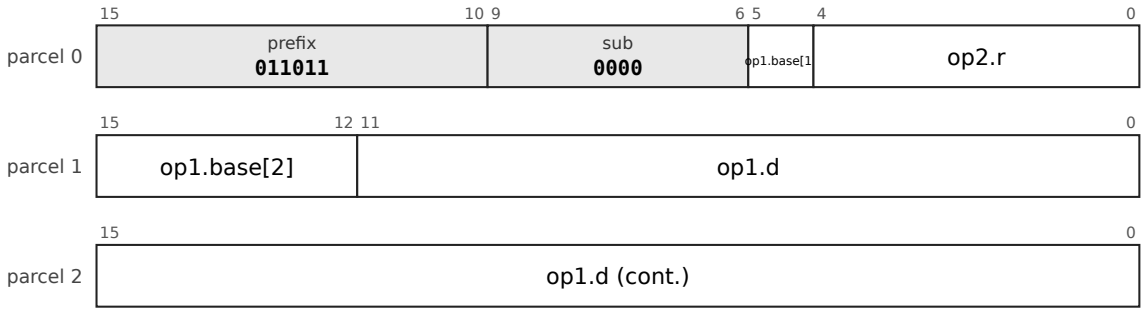
*(D)* Item 18 of `docs/study-proposals.md`: user access is an attribute of one memory operand, so it cannot outlive the instruction, cannot leak across an interrupt, and `RTI` re-executes the instruction with the attribute. The author accepted the five leanings (errata 86): a fixup table in place of `pcb_onfault`, the `.U` spelling, keeping `PSW.UA`, `STUA` and `CLUA` until the OpenBSD port confirms, `ECAUSE` bit 15 to say that the access was user-qualified, and the atomics taking the user form once the operand placement of `ATOM` is fixed. (Not to be confused with `MOVU`, the unaligned move.) Details pending (J.9.12).

**Why it is here.** Copying between the kernel and the user is where kernels pay for the user-memory guard. x86 opens a window with `STAC` and closes it with `CLAC` (35 and 53 sites in the kernel-assembly corpus), and a forgotten `CLAC` on an error path is the bug class; Arm's `LDTR` and `STTR` make the access itself unprivileged (52 sites), but only for plain loads and stores. Item 18 of the study makes user access an attribute of one operand, so that no window exists to leave open and the range check of the copy routines (`addr + len` against the top of the user half) disappears into the hardware: a kernel address faults at the first byte. The author accepted its five leanings (errata 86) and kept `PSW.UA`, `STUA` and `CLUA` until the OpenBSD port confirms.

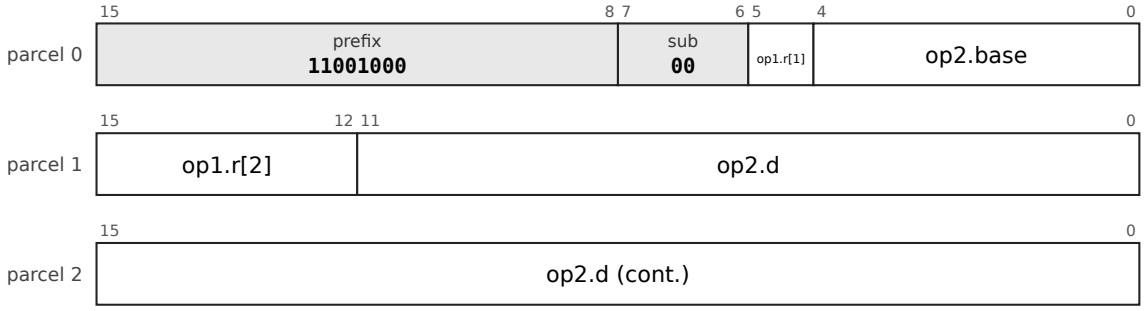
**Cf.** 6.10 (the user-memory guard), docs/study-proposals.md item 18.

## *MOVQ.U*

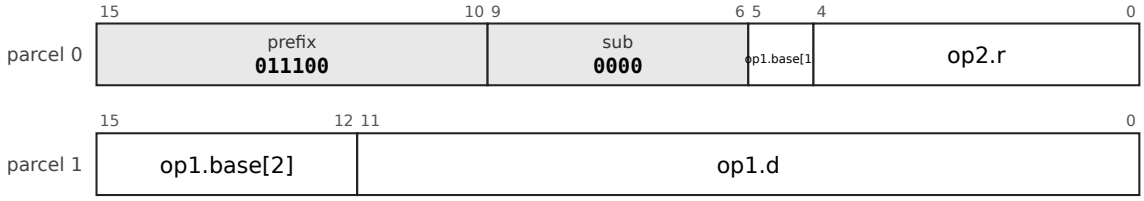
**MOVQ.U: LDU.B28, prefix 011011, sub-opcode 0000; operands b5d28s-x1, reg5; 6 bytes**



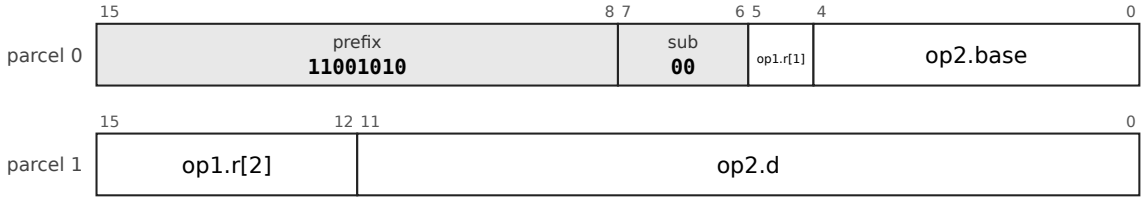

**MOVQ.U: STU.B28, prefix 11001000, sub-opcode 00; operands reg5, b5d28s-x1; 6 bytes**



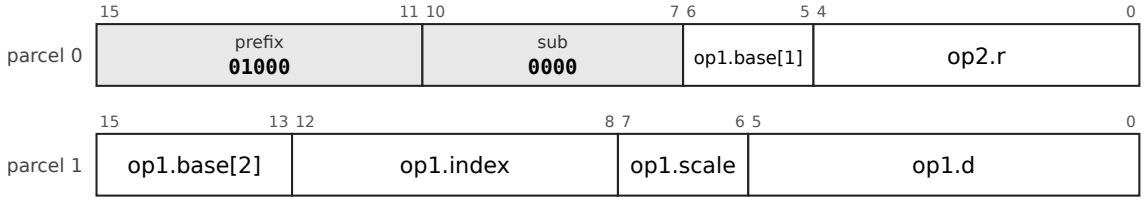

**MOVQ.U: LDU.B12, prefix 011100, sub-opcode 0000; operands b5d12s-x1, reg5; 4 bytes**



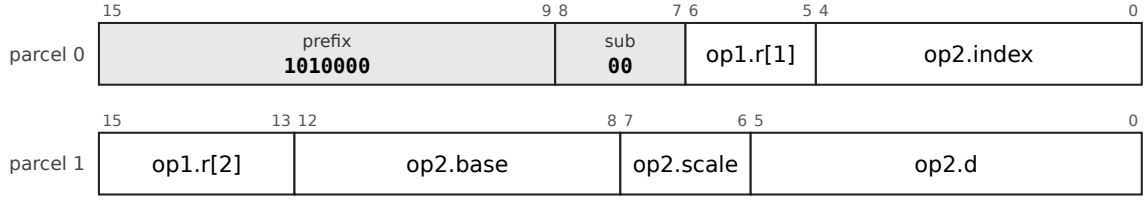

**MOVQ.U: STU.B12, prefix 11001010, sub-opcode 00; operands reg5, b5d12s-x1; 4 bytes**




**MOVQ.U: LDU.X6, prefix 01000, sub-opcode 0000; operands x5-5d6s, reg5; 4 bytes**



**MOVQ.U: STU.X6, prefix 1010000, sub-opcode 00; operands reg5, x5-5d6s; 4 bytes**

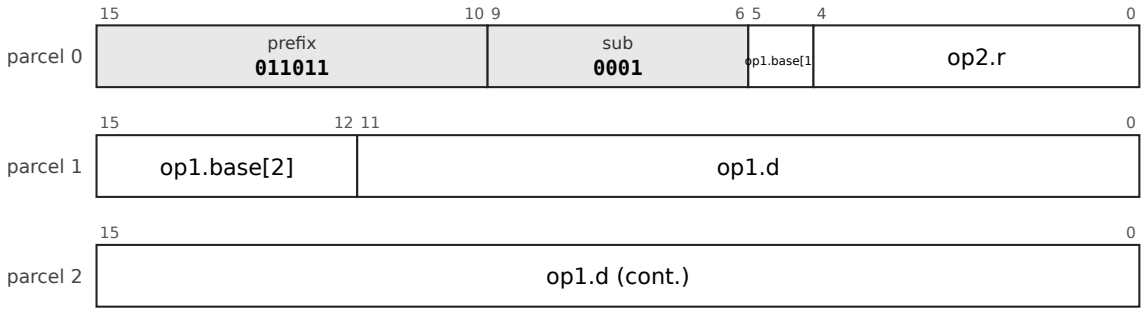



```
MOVQ.U d(Rn), Rd
MOVQ.U Rs, d(Rn)
```

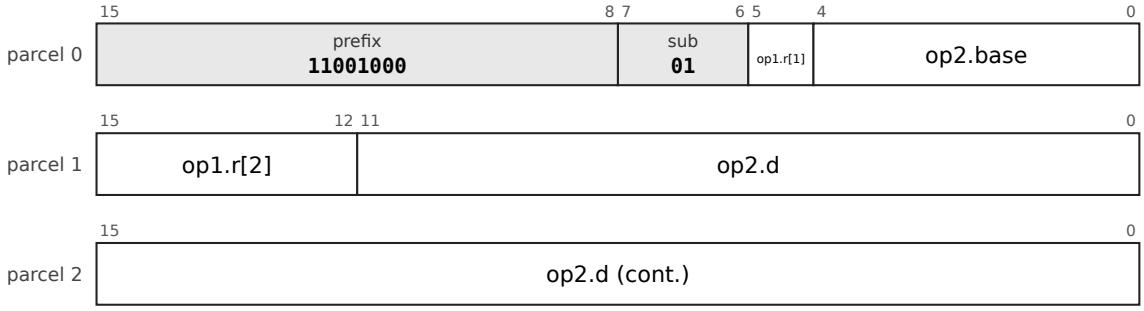
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

## *MOVL.U*

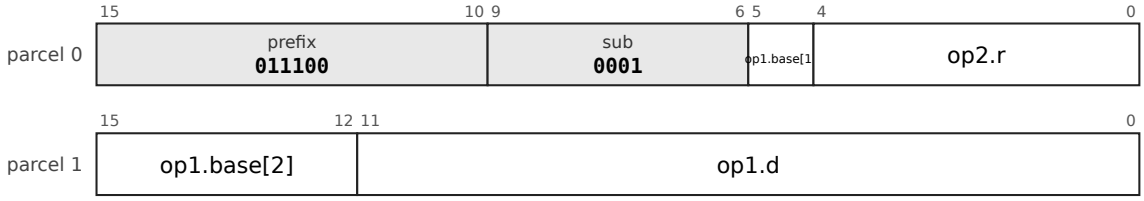
**MOVL.U: LDU.B28, prefix 011011, sub-opcode 0001; operands b5d28s-x1, reg5; 6 bytes**



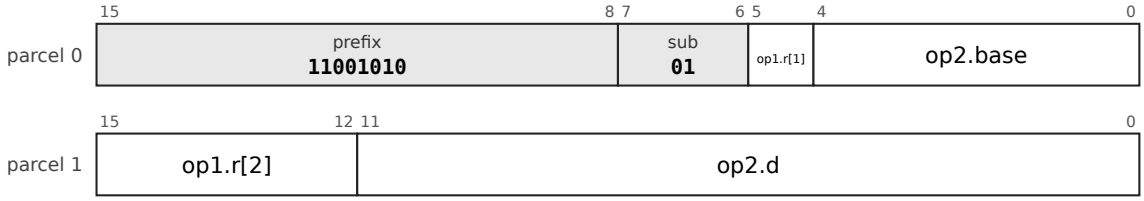

**MOVL.U: STU.B28, prefix 11001000, sub-opcode 01; operands reg5, b5d28s-x1; 6 bytes**



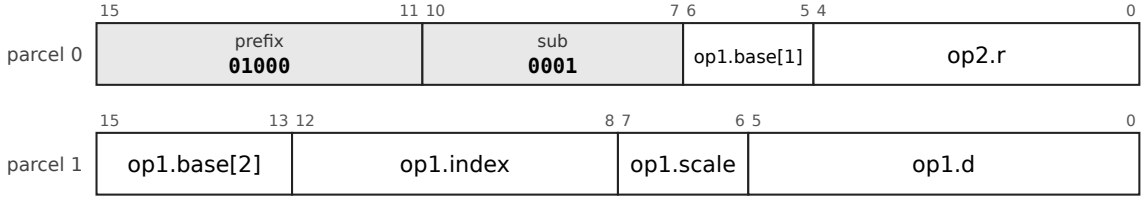

**MOVL.U: LDU.B12, prefix 011100, sub-opcode 0001; operands b5d12s-x1, reg5; 4 bytes**



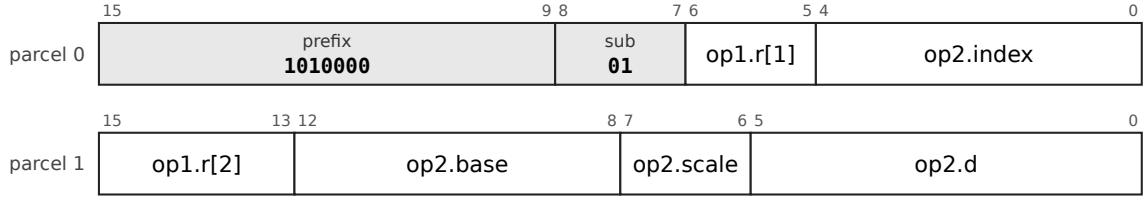

**MOVL.U: STU.B12, prefix 11001010, sub-opcode 01; operands reg5, b5d12s-x1; 4 bytes**




**MOVL.U: LDU.X6, prefix 01000, sub-opcode 0001; operands x5-5d6s, reg5; 4 bytes**



**MOVL.U: STU.X6, prefix 1010000, sub-opcode 01; operands reg5, x5-5d6s; 4 bytes**

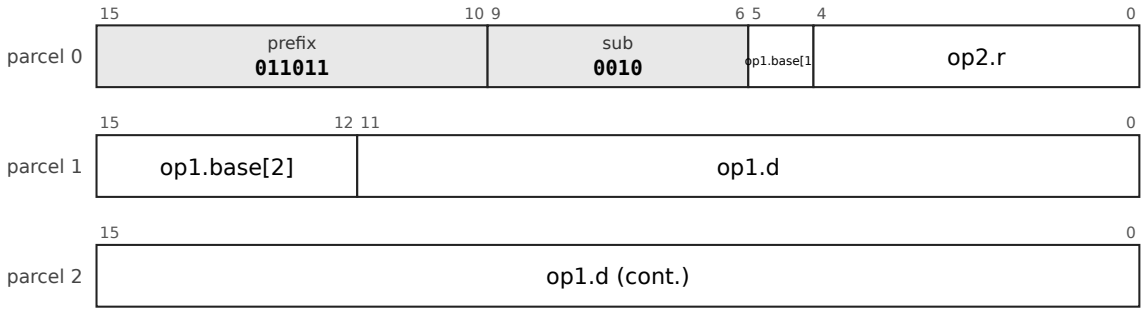



```
MOVL.U ..., MOVW.U ..., MOVB.U ...
```

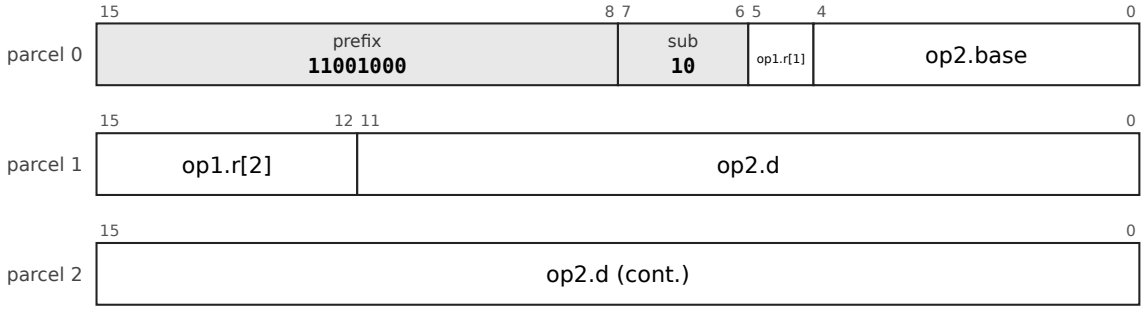
Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

## *MOVW.U*

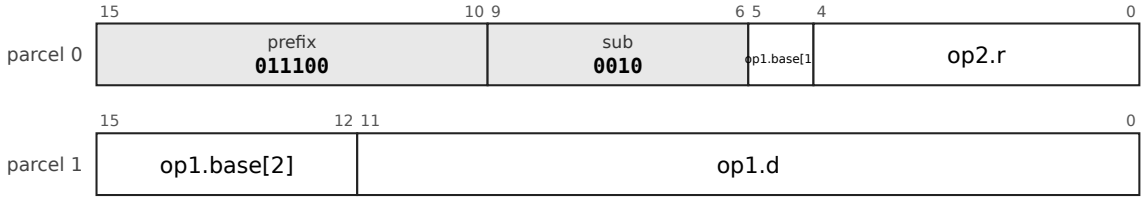
**MOVW.U: LDU.B28, prefix 011011, sub-opcode 0010; operands b5d28s-x1, reg5; 6 bytes**



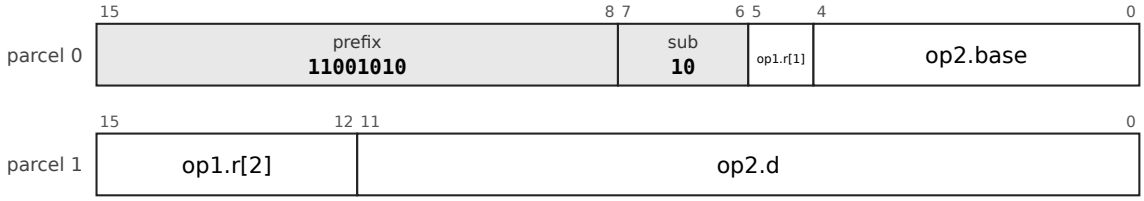

**MOVW.U: STU.B28, prefix 11001000, sub-opcode 10; operands reg5, b5d28s-x1; 6 bytes**



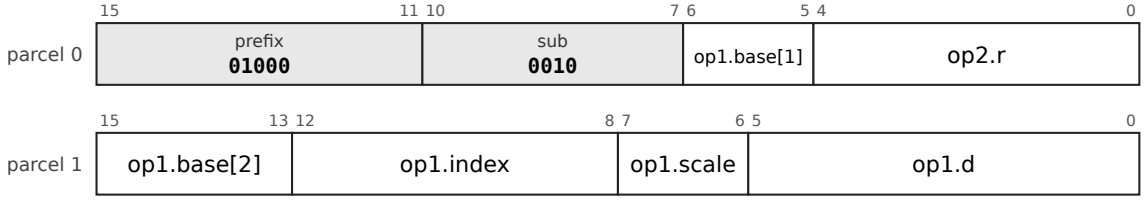

**MOVW.U: LDU.B12, prefix 011100, sub-opcode 0010; operands b5d12s-x1, reg5; 4 bytes**



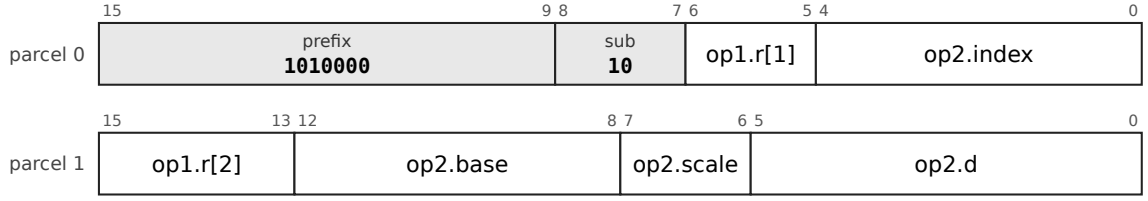

**MOVW.U: STU.B12, prefix 11001010, sub-opcode 10; operands reg5, b5d12s-x1; 4 bytes**




**MOVW.U: LDU.X6, prefix 01000, sub-opcode 0010; operands x5-5d6s, reg5; 4 bytes**



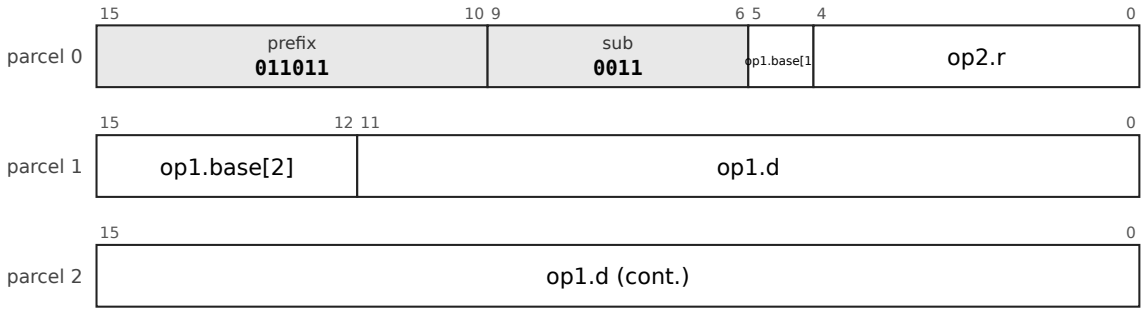
**MOVW.U: STU.X6, prefix 1010000, sub-opcode 10; operands reg5, x5-5d6s; 4 bytes**



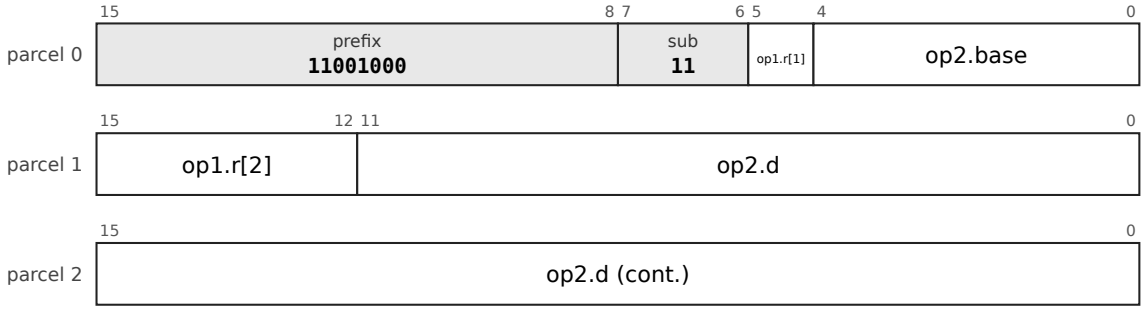

Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

## *MOVB.U*

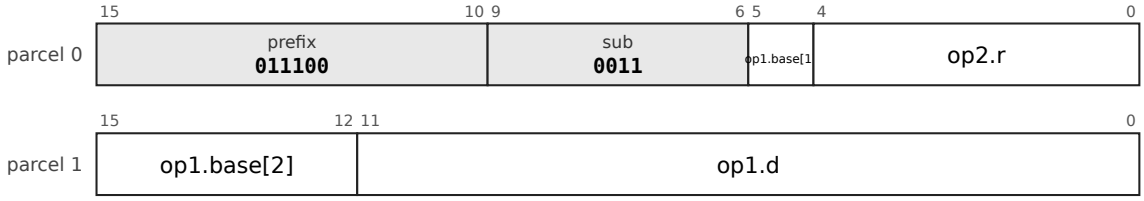
**MOVB.U: LDU.B28, prefix 011011, sub-opcode 0011; operands b5d28s-x1, reg5; 6 bytes**



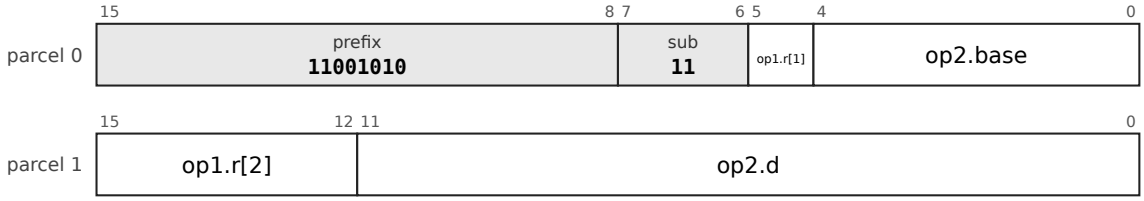

**MOVB.U: STU.B28, prefix 11001000, sub-opcode 11; operands reg5, b5d28s-x1; 6 bytes**



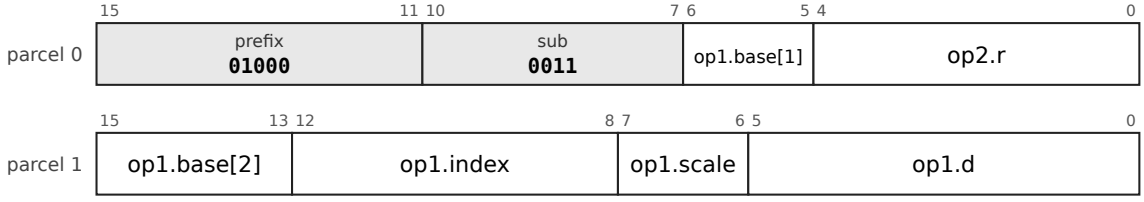

**MOVB.U: LDU.B12, prefix 011100, sub-opcode 0011; operands b5d12s-x1, reg5; 4 bytes**



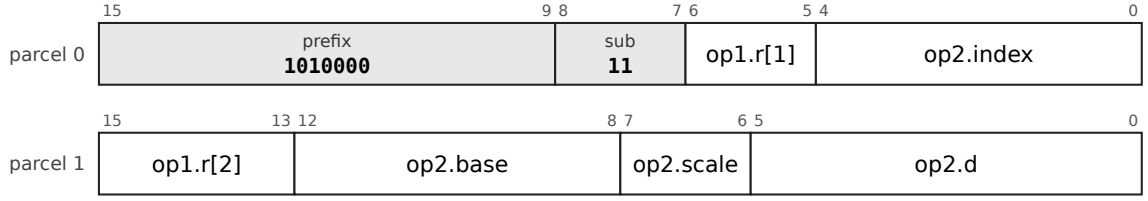

**MOVB.U: STU.B12, prefix 11001010, sub-opcode 11; operands reg5, b5d12s-x1; 4 bytes**




**MOVB.U: LDU.X6, prefix 01000, sub-opcode 0011; operands x5-5d6s, reg5; 4 bytes**



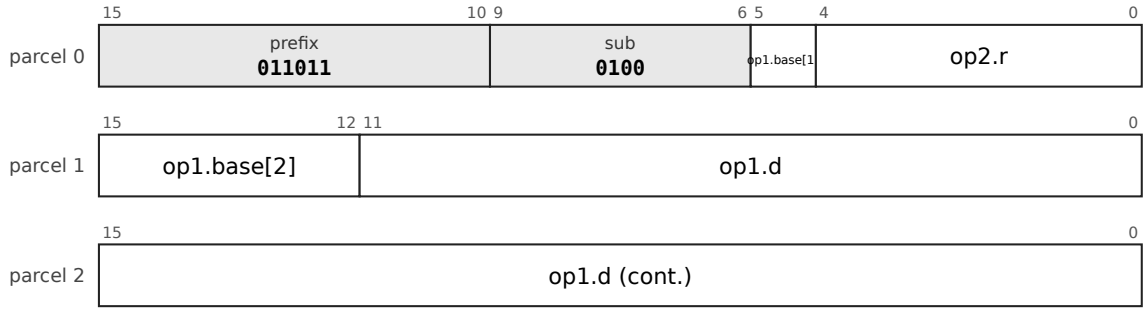
**MOVB.U: STU.X6, prefix 1010000, sub-opcode 11; operands reg5, x5-5d6s; 4 bytes**



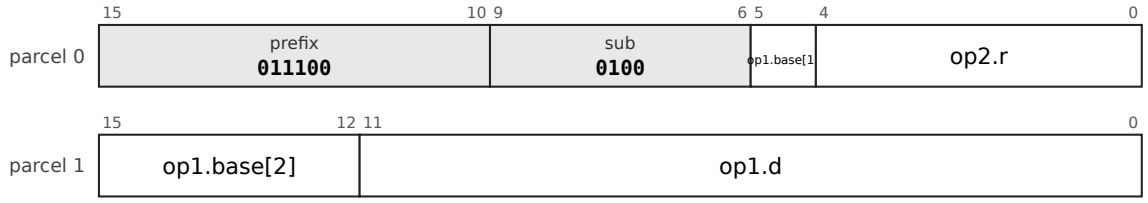

Operation, flags and exceptions as for the entry MOVUSER above; 6 layout forms, each a chart of its own (above).

***MOVZL.U***

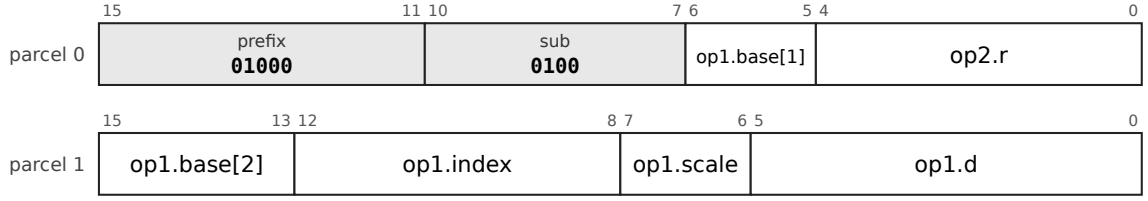
**MOVZL.U: LDU.B28, prefix 011011, sub-opcode 0100; operands b5d28s-x1, reg5; 6 bytes**




**MOVZL.U: LDU.B12, prefix 011100, sub-opcode 0100; operands b5d12s-x1, reg5; 4 bytes**




**MOVZL.U: LDU.X6, prefix 01000, sub-opcode 0100; operands x5-5d6s, reg5; 4 bytes**

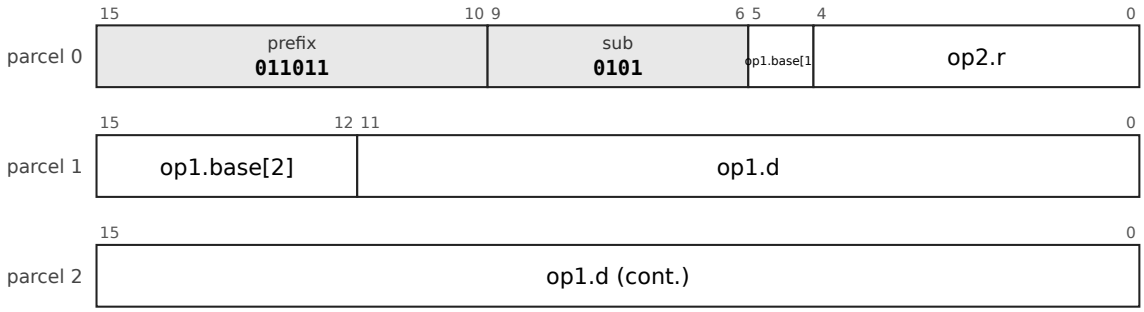



```
MOVZL.U ..., MOVZW.U ..., MOVZB.U ...
```

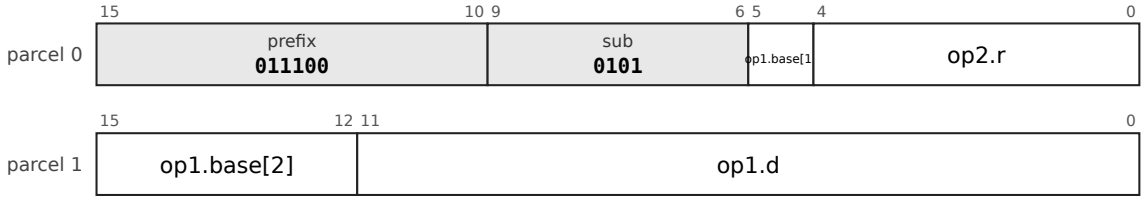
Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

### *MOVZW.U*

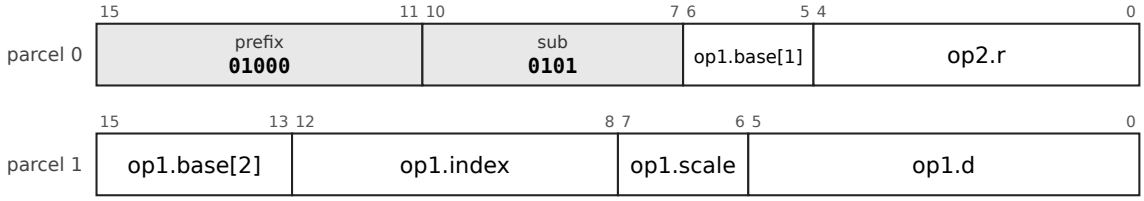
**MOVZW.U: LDU.B28, prefix 011011, sub-opcode 0101; operands b5d28s-x1, reg5; 6 bytes**




**MOVZW.U: LDU.B12, prefix 011100, sub-opcode 0101; operands b5d12s-x1, reg5; 4 bytes**



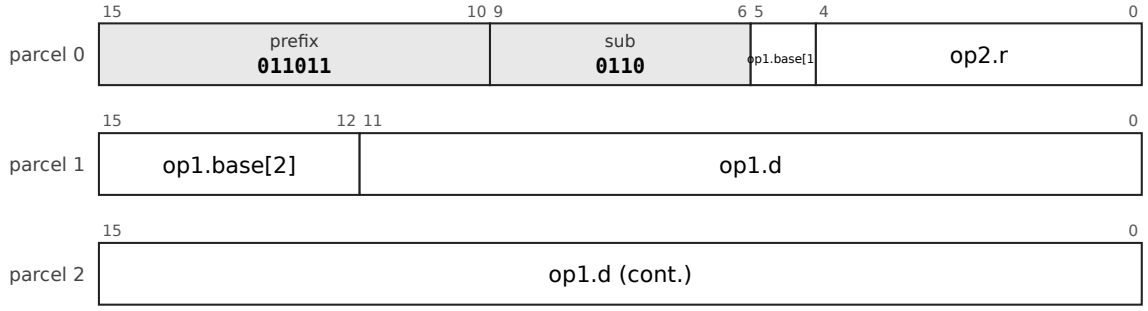

**MOVZW.U: LDU.X6, prefix 01000, sub-opcode 0101; operands x5-5d6s, reg5; 4 bytes**



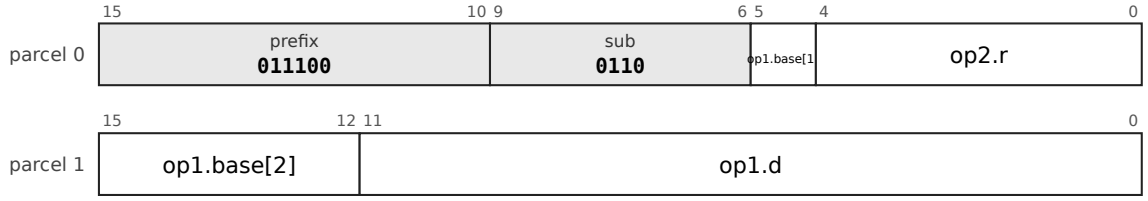

Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

***MOVZB.U***

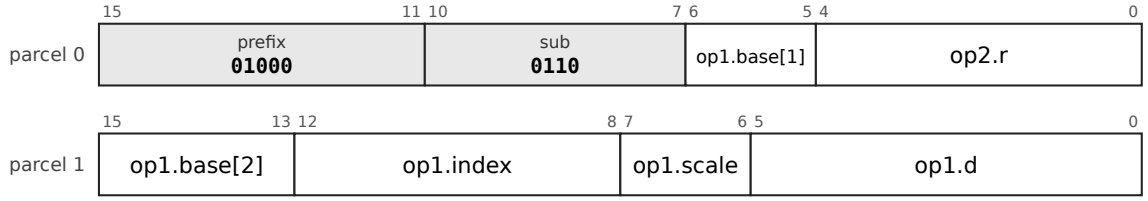
**MOVZB.U: LDU.B28, prefix 011011, sub-opcode 0110; operands b5d28s-x1, reg5; 6 bytes**




**MOVZB.U: LDU.B12, prefix 011100, sub-opcode 0110; operands b5d12s-x1, reg5; 4 bytes**




**MOVZB.U: LDU.X6, prefix 01000, sub-opcode 0110; operands x5-5d6s, reg5; 4 bytes**




Operation, flags and exceptions as for the entry MOVUSER above; 3 layout forms, each a chart of its own (above).

## 9.7 Integer arithmetic

*Cf. section 6.3 for the divide-by-zero vector (`DIV`, `DIVU`, `MOD`, `MODU`).*

**9.7.1** `ADD`, `SUB`, `CMP`, `MUL` and `MULH` are two bytes with register operands; `MULHU` (high half of an unsigned product), `DIV`, `DIVU`, `MOD` and `MODU` are in the arithmetic escape. *(D)*

**9.7.2** `CMP` is written `CMP Rd, rm`, register first, and computes `Rd - rm`. *(D)*

**9.7.3** `ADD` sets `C` on a carry; `SUB` and `CMP` set `C` on a borrow, the PDP-11 sense, so `BHIS` and `BCC` mean "no borrow", unsigned greater or equal. *(D)*

**9.7.4** `ADC` and `SBC` add and subtract the carry; `INC`, `DEC`, `NEG`, `CLR` and `TST` are single-operand forms that work on a register or memory.

**9.7.5** `SET cc, Rd` (write 1 if the condition holds, else 0) is in the arithmetic escape. *(D)*

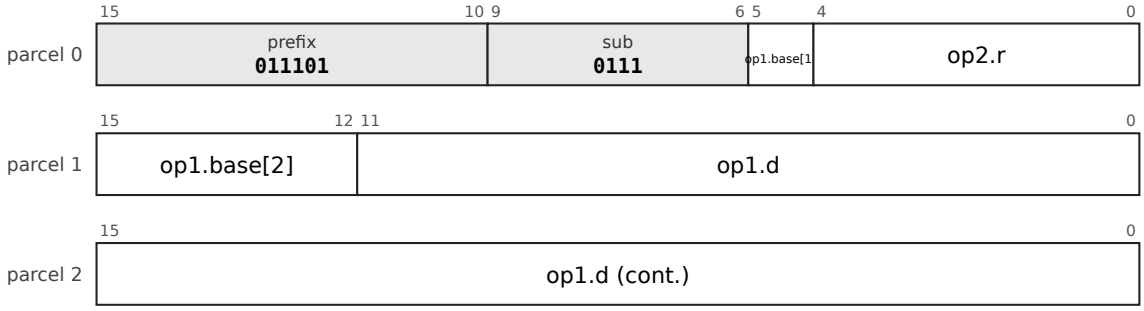
**9.7.6** `ABS`, `MIN`, `MINU`, `MAX` and `MAXU` are assembler aliases, not native instructions: `ABS` is the absolute value, and the others the smaller or the larger of two registers, signed or unsigned. *(D)* Open: their expansion, and with it the flags they leave (J.9.1).

*Commentary.* **Precedent.** `ADD`, `SUB`, `CMP`, `BIT`, `BIS`, `BIC` and the single-operand set are the PDP-11's, unchanged in name and meaning except for the order of `CMP`'s operands. **Trade-offs.** The PDP-11 wrote `CMP a, b` to compute `a - b`; since the destination of an ALU operation here must be a register, a literal `src, dst` reading would force every comparison against an immediate or memory to be reversed, so the register comes first. Divide is in the escape group, at four bytes, because compilers turn division by a constant into `MULH` plus a shift, so a real divide is rare. **Improvements.** The high-half multiply that bignum code needs is present; a second unsigned multiply is the same instruction as the first and is not provided. **Benefits.** `SET` replaces

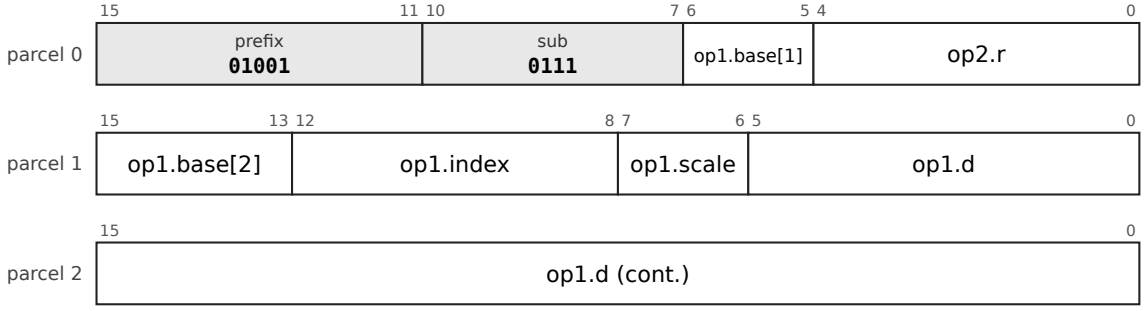
a branch around a move for the very common "store the truth value of a comparison", and absolute value, minimum and maximum (as aliases) remove a compare-and-branch from small loops. **Modern comparables.** RISC-V has `MULH`, `MULHU` and `MULHSU`, and AArch64 `SMULH` and `UMULH`; x86 has a one-operand multiply that produces both halves; AArch64's conditional select and set instructions are the nearest to `SET`. **Use.** `MULH` and `MULHU` are for multi-precision arithmetic and for division by constants; `ADC` and `SBC` carry a multi-word add or subtract; `MIN`, `MAX` and `ABS` (aliases) appear under a tenth of a percent of compiled code but each replaces a branch. **Impact.** *Kernel:* `ADC` and `SBC` carry multiword arithmetic for the kernel's 128-bit counters and for the bignum code in the base system. *Compiler:* `MULH` turns division by a constant into a multiply and a shift; `SET` materialises a comparison without a branch; `CMP` takes the register first, as the compiler orders its operands. *Hardware:* One adder on the carry chain; on an FPGA the multiplier is built from two to four multiplier blocks over four to eight cycles; the divide is a sequencer.

## ADD: Add

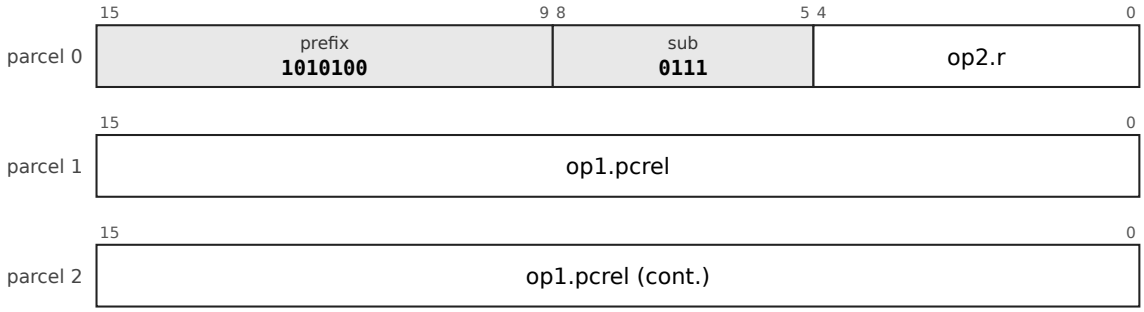
**ADD: LDF.B28, prefix 011101, sub-opcode 0111; operands b5d28s-x1, reg5; 6 bytes**



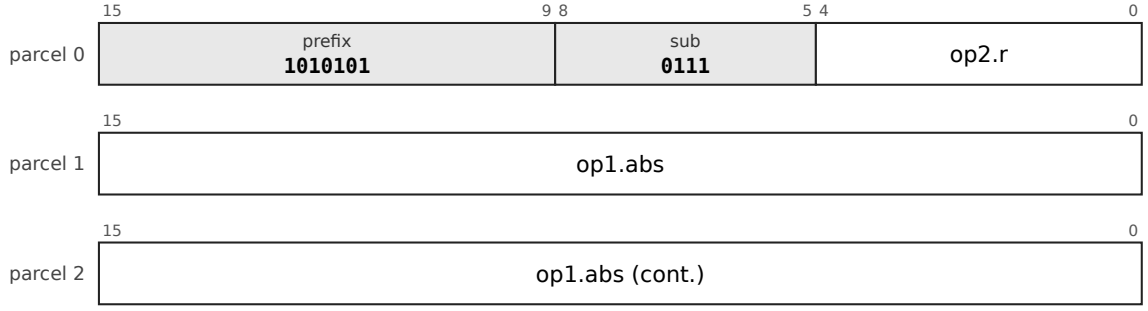

**ADD: LDF.X22, prefix 01001, sub-opcode 0111; operands x5-5d22s, reg5; 6 bytes**



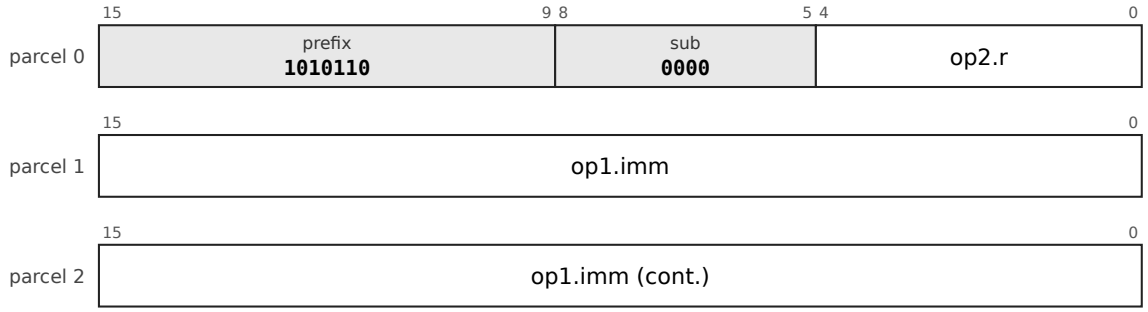

**ADD: LDF.PC32, prefix 1010100, sub-opcode 0111; operands pc32-u1, reg5; 6 bytes**



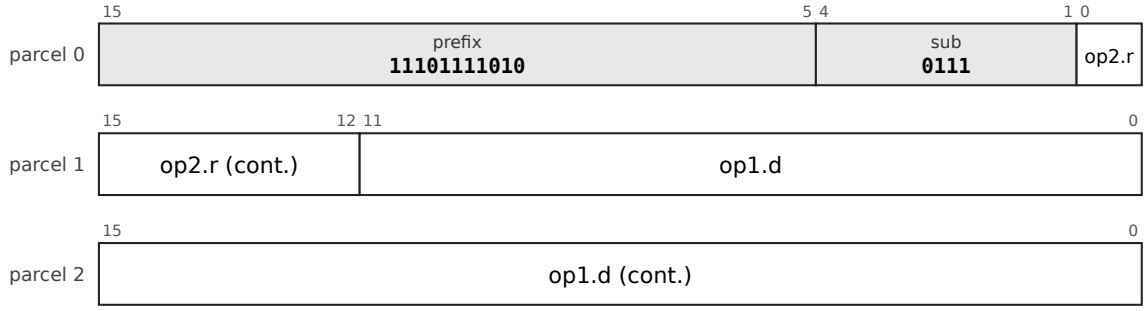
**ADD: LDF.A32, prefix 1010101, sub-opcode 0111; operands abs32, reg5; 6 bytes**



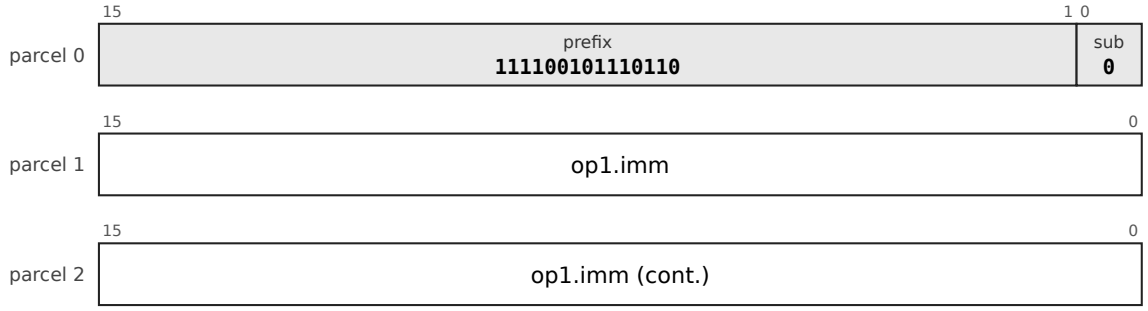

**ADD: LDFI.I32, prefix 1010110, sub-opcode 0000; operands imm32s, reg5; 6 bytes**



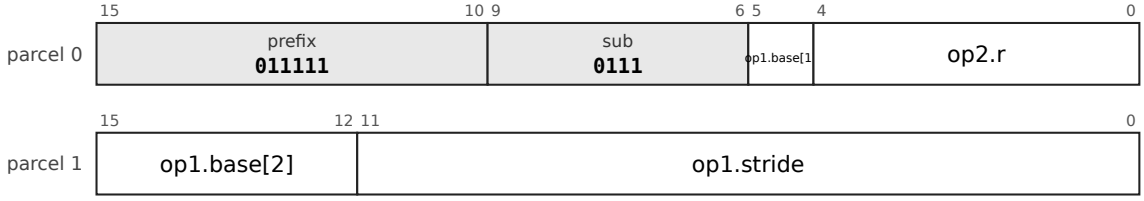

**ADD: DPF.D28, prefix 11101111010, sub-opcode 0111; operands dpd28s-x1, reg5; 6 bytes**



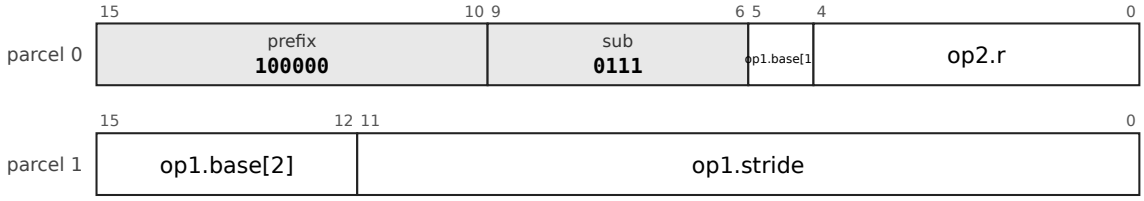

**ADD: SPADJF, prefix 111100101110110, sub-opcode 0; operands imm32s, k-sp; 6 bytes**



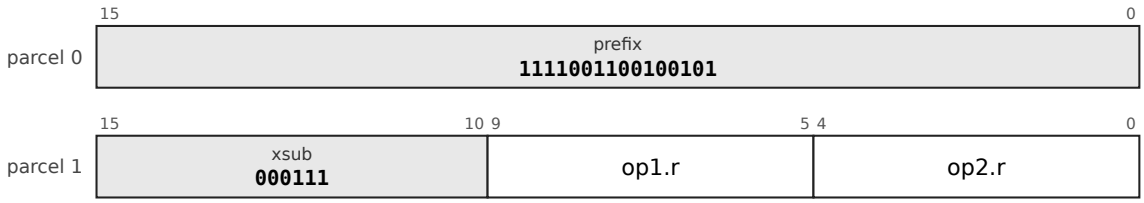
**ADD: LDF.PI12, prefix 011111, sub-opcode 0111; operands pi5-s12, reg5; 4 bytes**



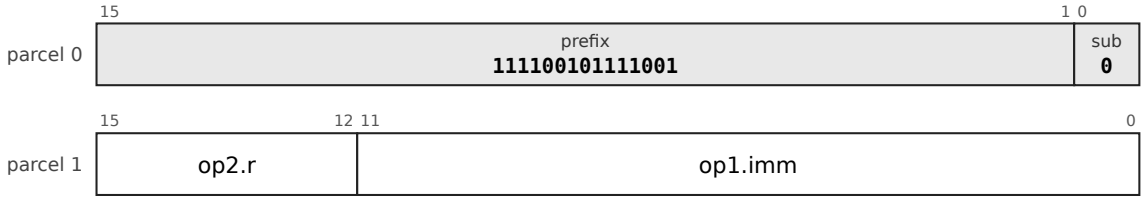

**ADD: LDF.PD12, prefix 100000, sub-opcode 0111; operands pd5-s12, reg5; 4 bytes**



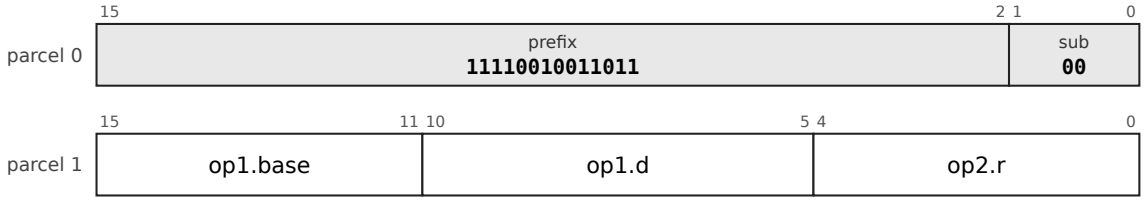

**ADD: RRF5, prefix 1111001100100101, sub-opcode 000111; operands reg5, reg5; 4 bytes**



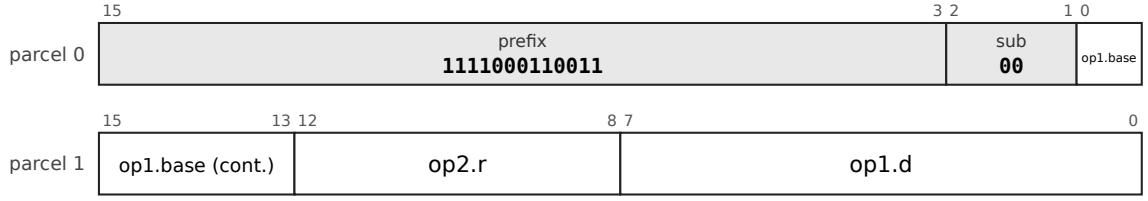

**ADD: LDI12H/b, prefix 111100101111001, sub-opcode 0; operands imm12s, reg4; 4 bytes**



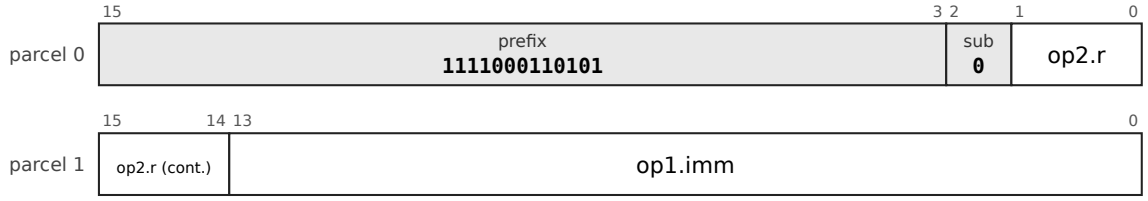

**ADD: LDBu6/c, prefix 11110010011011, sub-opcode 00; operands b5d6u-xsz, reg5; 4 bytes**



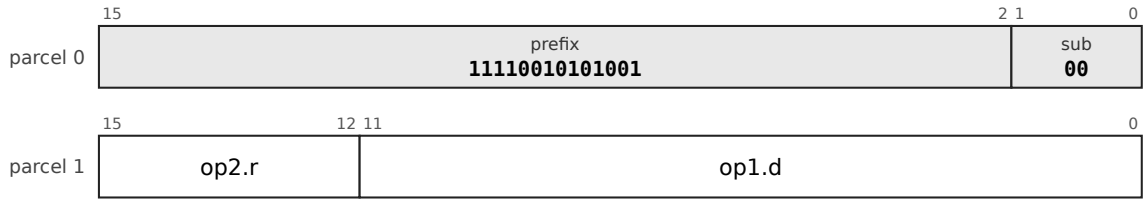
**ADD: LDBu8H/c, prefix 1111000110011, sub-opcode 00; operands b4d8u-xsz, reg5; 4 bytes**



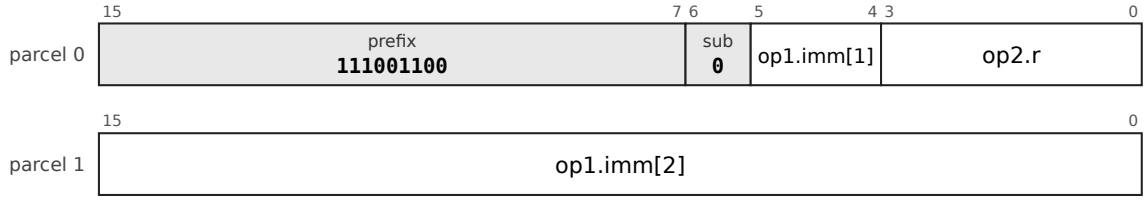

**ADD: LDI14H/b, prefix 1111000110101, sub-opcode 0; operands imm14s, reg4; 4 bytes**



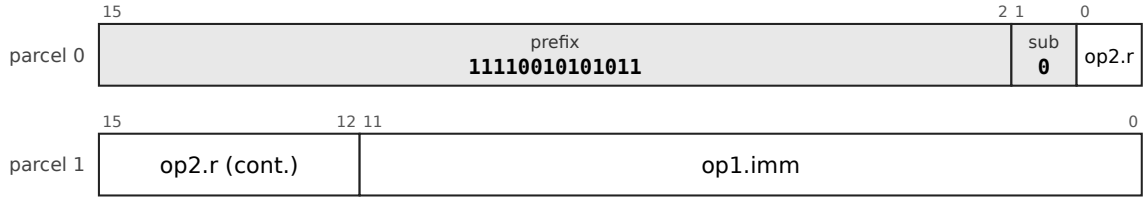

**ADD: LDS12H/c, prefix 11110010101001, sub-opcode 00; operands spd12u-xsz, reg4; 4 bytes**



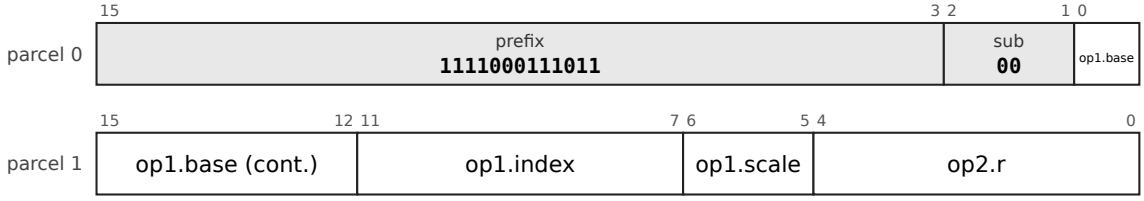

**ADD: LDI18H/b, prefix 111001100, sub-opcode 0; operands imm18s, reg4; 4 bytes**



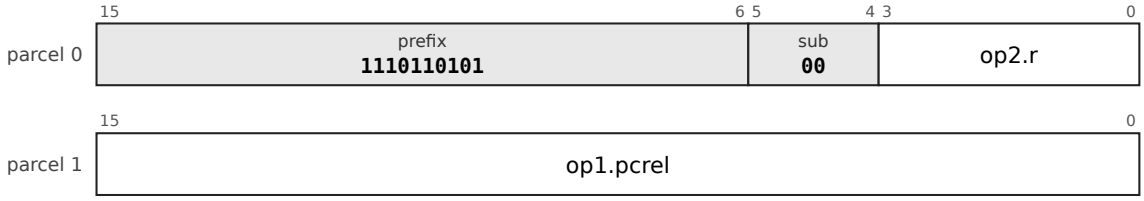

**ADD: LDI12/b, prefix 11110010101011, sub-opcode 0; operands imm12s, reg5; 4 bytes**



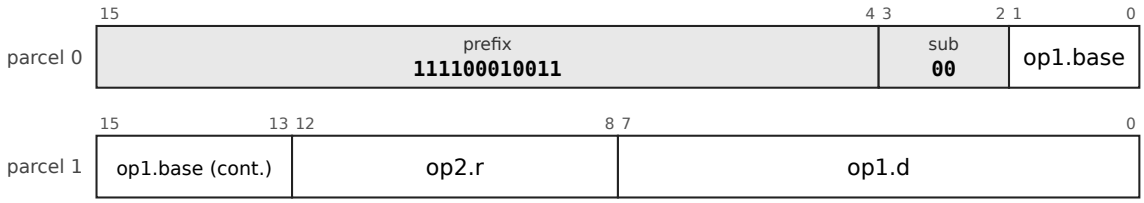
**ADD: LDX0/c, prefix 1111000111011, sub-opcode 00; operands x5-5d0s, reg5; 4 bytes**



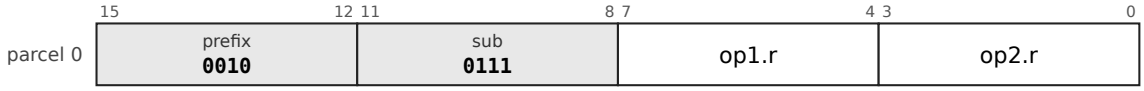

**ADD: LDPC16H/c, prefix 1110110101, sub-opcode 00; operands pc16-u1, reg4; 4 bytes**



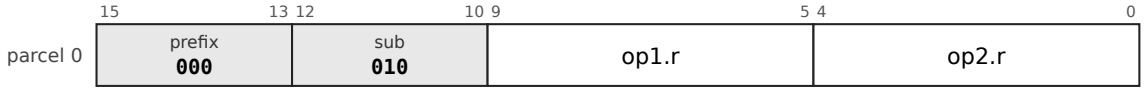

**ADD: LDBu8/c, prefix 111100010011, sub-opcode 00; operands b5d8u-xsz, reg5; 4 bytes**




**ADD: RRF, prefix 0010, sub-opcode 0111; operands reg4, reg4; 2 bytes**



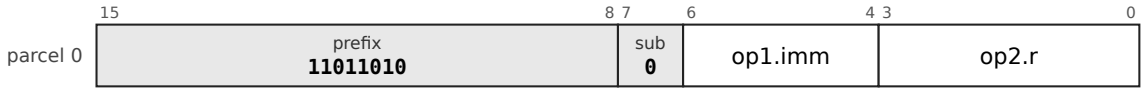

**ADD: RR5, prefix 000, sub-opcode 010; operands reg5, reg5; 2 bytes**




**ADD: SPADJ6/a, prefix 111000111, sub-opcode 0; operands imm6u-x8, k-sp; 2 bytes**




**ADD: IMMh43/b, prefix 11011010, sub-opcode 0; operands imm3s, reg4; 2 bytes**

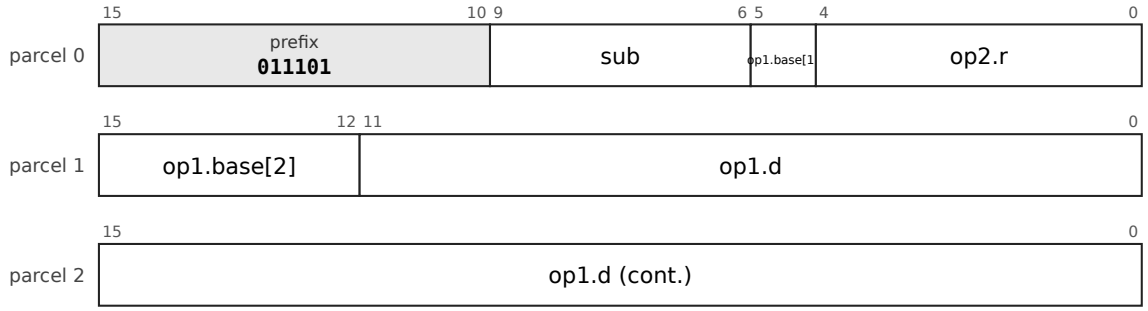



**Mnemonics:** ADD

### Syntax

```
ADD rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `SPADJF-1`, `RR5-1`, `LDI12H-b-1`, `LDBu6-c-1`, `SPADJ6-a-1`, `LDBu8H-c-1`, `LDI14H-b-1`, `LDS12H-c-1`, `IMMh43-b-1`, `LDI18H-b-1`, `LDI12-b-1`, `LDX0-c-1`, `LDPC16H-c-1`, `LDBu8-c-1`.

### Operation

```
Rd ← Rd + rm
```

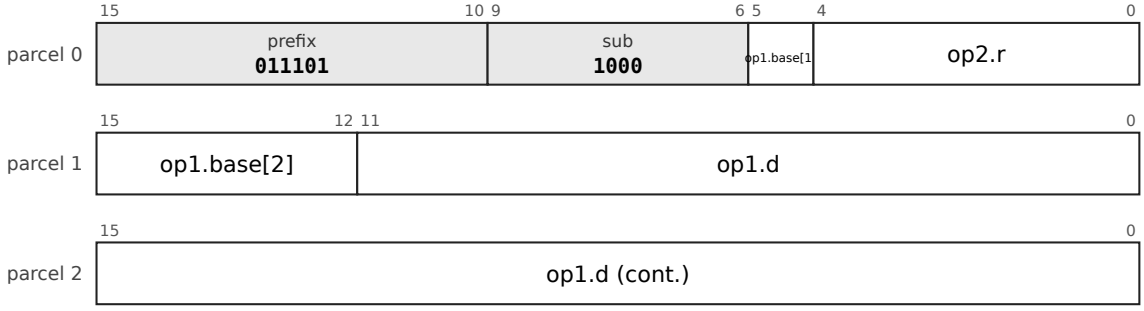
**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

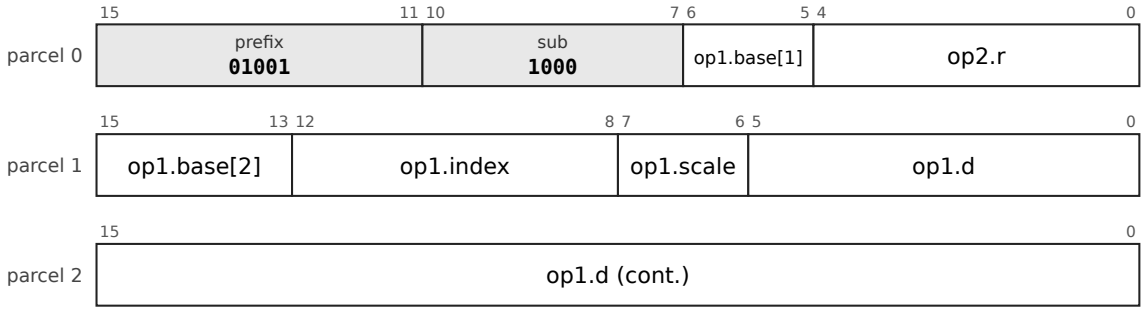
All ALU operations are 64 bits, with no size field.

## SUB: Subtract

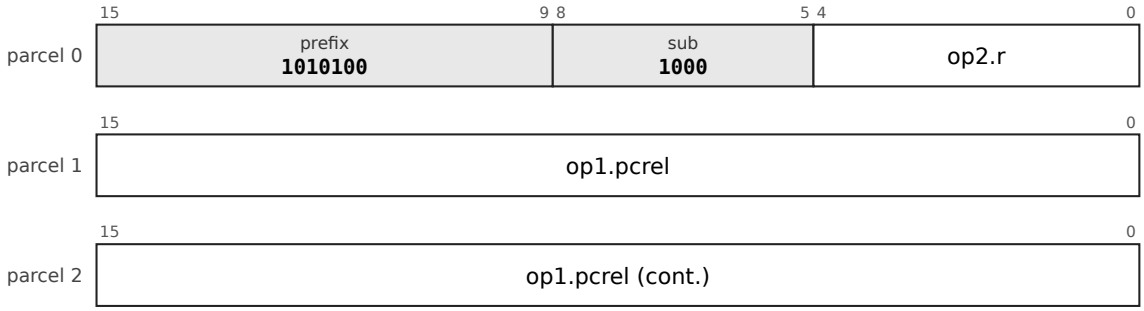
**SUB: LDF.B28, prefix 011101, sub-opcode 1000; operands b5d28s-x1, reg5; 6 bytes**



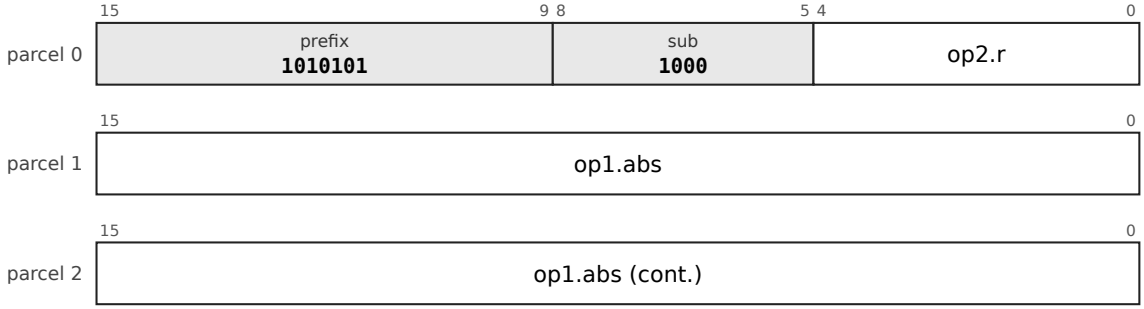

**SUB: LDF.X22, prefix 01001, sub-opcode 1000; operands x5-5d22s, reg5; 6 bytes**



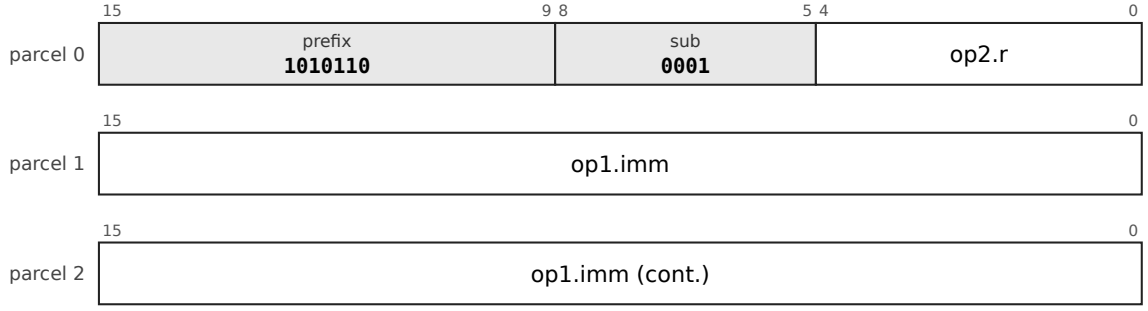

**SUB: LDF.PC32, prefix 1010100, sub-opcode 1000; operands pc32-u1, reg5; 6 bytes**



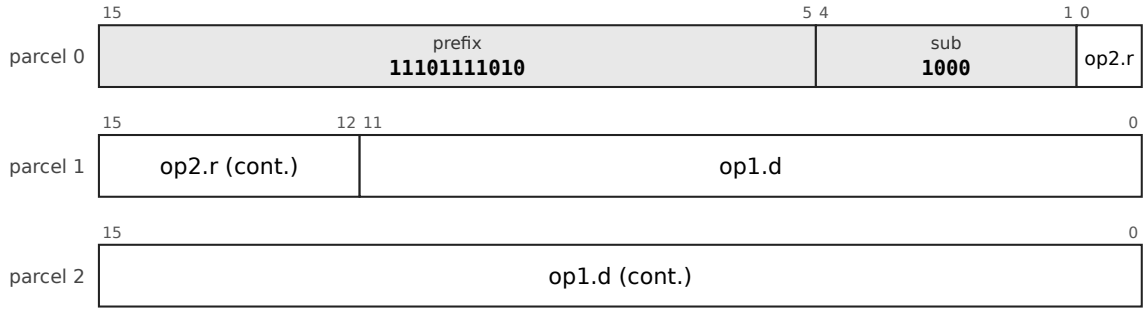

**SUB: LDF.A32, prefix 1010101, sub-opcode 1000; operands abs32, reg5; 6 bytes**



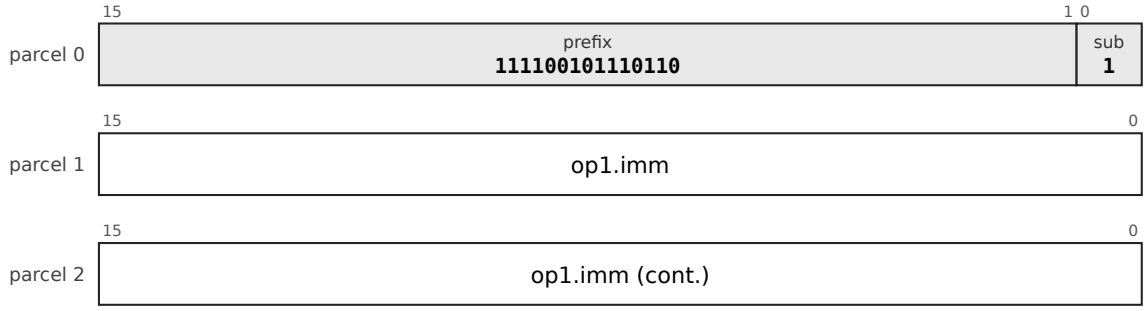
**SUB: LDFI.I32, prefix 1010110, sub-opcode 0001; operands imm32s, reg5; 6 bytes**



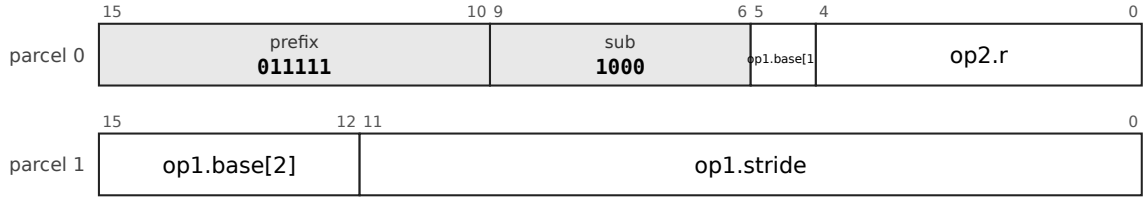

**SUB: DPF.D28, prefix 11101111010, sub-opcode 1000; operands dpd28s-x1, reg5; 6 bytes**



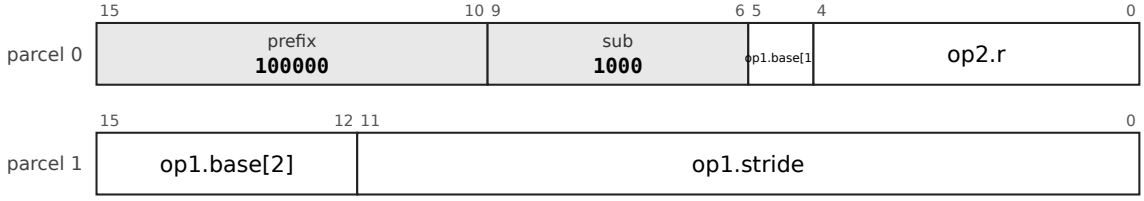

**SUB: SPADJF, prefix 111100101110110, sub-opcode 1; operands imm32s, k-sp; 6 bytes**



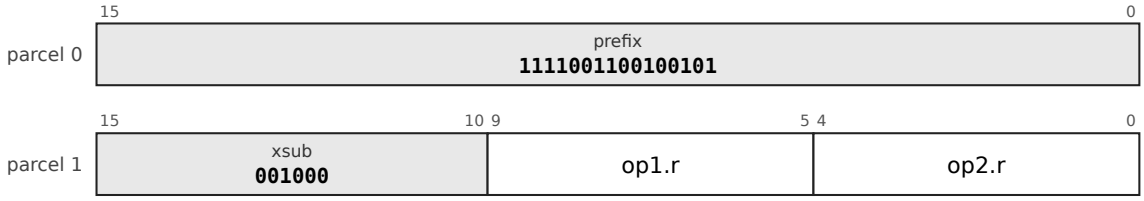

**SUB: LDF.PI12, prefix 011111, sub-opcode 1000; operands pi5-s12, reg5; 4 bytes**



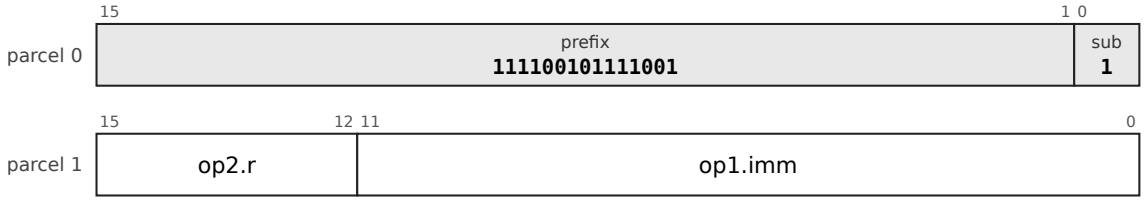
**SUB: LDF.PD12, prefix 100000, sub-opcode 1000; operands pd5-s12, reg5; 4 bytes**



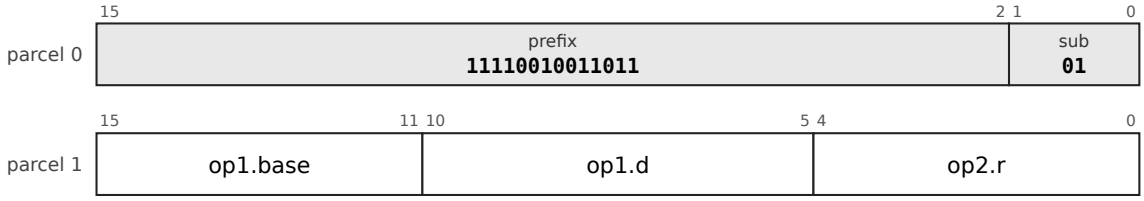

**SUB: RRF5, prefix 1111001100100101, sub-opcode 001000; operands reg5, reg5; 4 bytes**



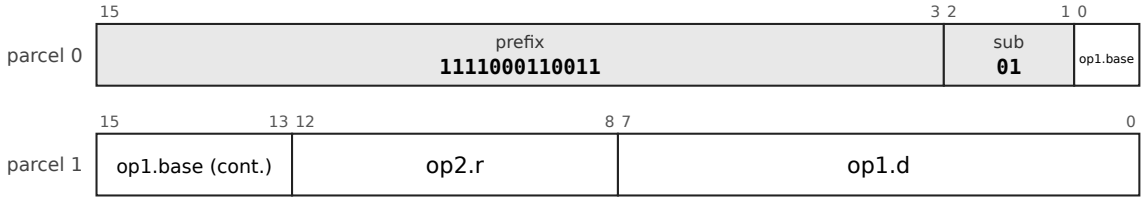

**SUB: LDI12H/b, prefix 111100101111001, sub-opcode 1; operands imm12s, reg4; 4 bytes**



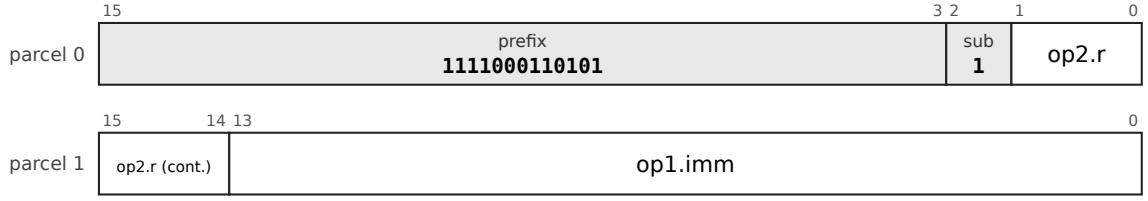

**SUB: LDBu6/c, prefix 11110010011011, sub-opcode 01; operands b5d6u-xsz, reg5; 4 bytes**



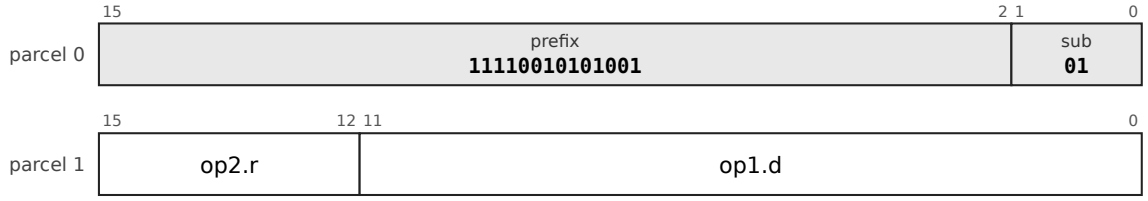

**SUB: LDBu8H/c, prefix 1111000110011, sub-opcode 01; operands b4d8u-xsz, reg5; 4 bytes**



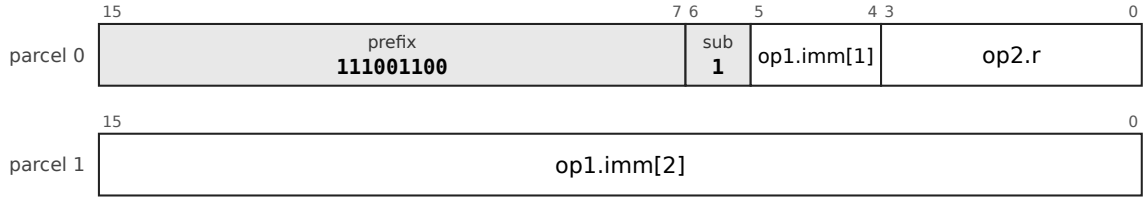
**SUB: LDI14H/b, prefix 1111000110101, sub-opcode 1; operands imm14s, reg4; 4 bytes**



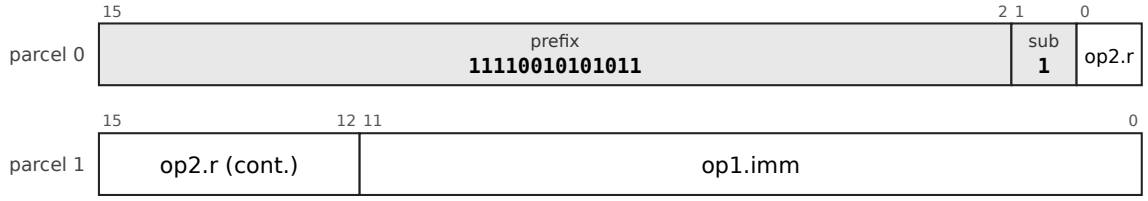

**SUB: LDS12H/c, prefix 11110010101001, sub-opcode 01; operands spd12u-xsz, reg4; 4 bytes**



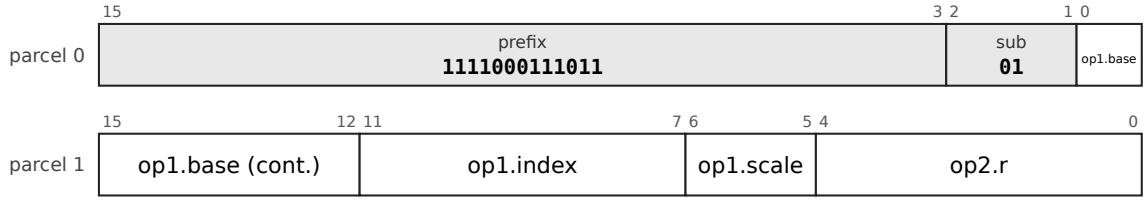

**SUB: LDI18H/b, prefix 111001100, sub-opcode 1; operands imm18s, reg4; 4 bytes**



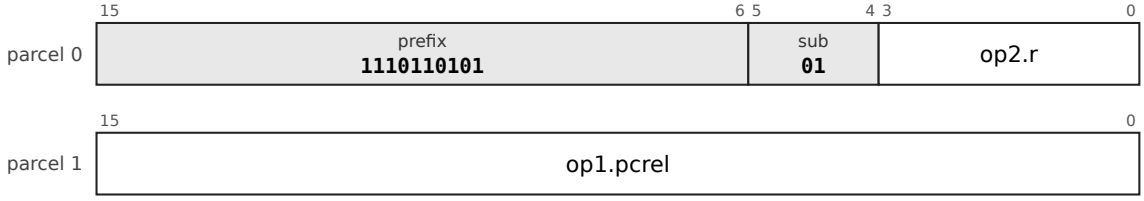

**SUB: LDI12/b, prefix 11110010101011, sub-opcode 1; operands imm12s, reg5; 4 bytes**



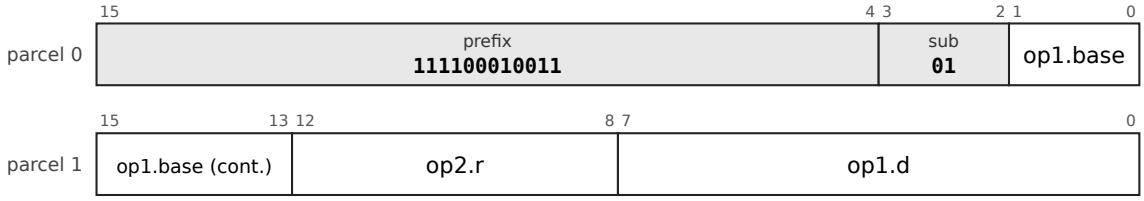

**SUB: LDX0/c, prefix 1111000111011, sub-opcode 01; operands x5-5d0s, reg5; 4 bytes**



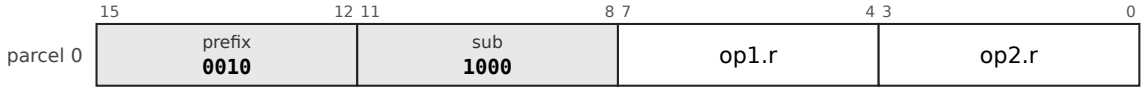
**SUB: LDPC16H/c, prefix 1110110101, sub-opcode 01; operands pc16-u1, reg4; 4 bytes**



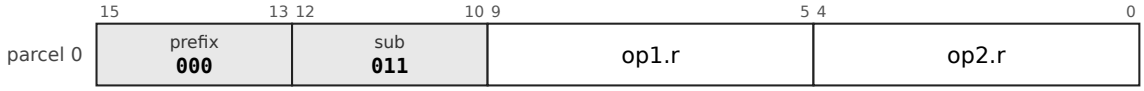

**SUB: LDBu8/c, prefix 111100010011, sub-opcode 01; operands b5d8u-xsz, reg5; 4 bytes**




**SUB: RRF, prefix 0010, sub-opcode 1000; operands reg4, reg4; 2 bytes**



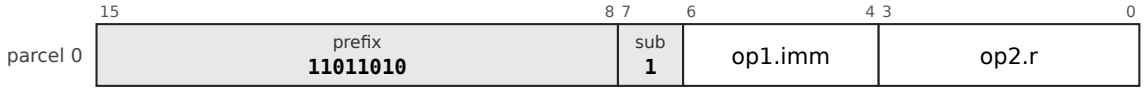

**SUB: RR5, prefix 000, sub-opcode 011; operands reg5, reg5; 2 bytes**




**SUB: SPADJ6/a, prefix 111000111, sub-opcode 1; operands imm6u-x8, k-sp; 2 bytes**




**SUB: IMMh43/b, prefix 11011010, sub-opcode 1; operands imm3s, reg4; 2 bytes**

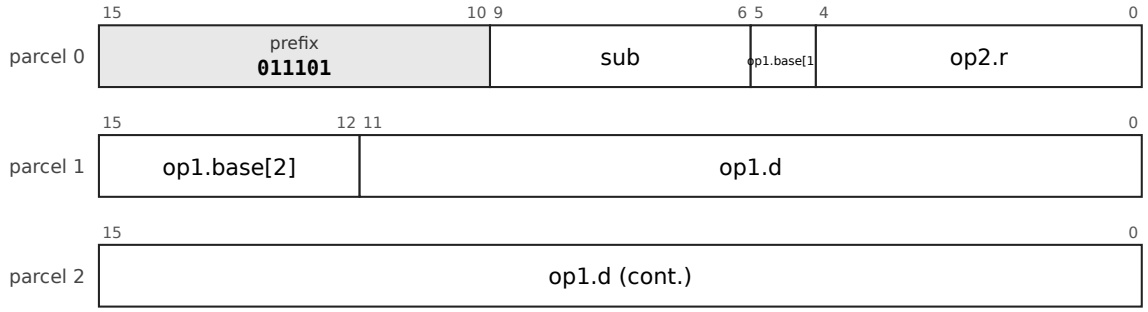



**Mnemonics:** SUB

## Syntax

```
SUB rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `SPADJF-1`, `RR5-1`, `LDI12H-b-1`, `LDBu6-c-1`, `SPADJ6-a-1`, `LDBu8H-c-1`, `LDI14H-b-1`, `LDS12H-c-1`, `IMMh43-b-1`, `LDI18H-b-1`, `LDI12-b-1`, `LDX0-c-1`, `LDPC16H-c-1`, `LDBu8-c-1`.

### Operation

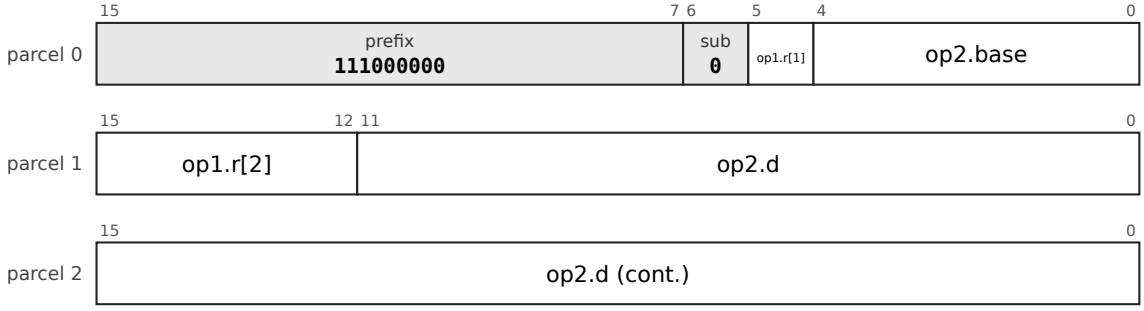
```
Rd ← Rd - rm
```

**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

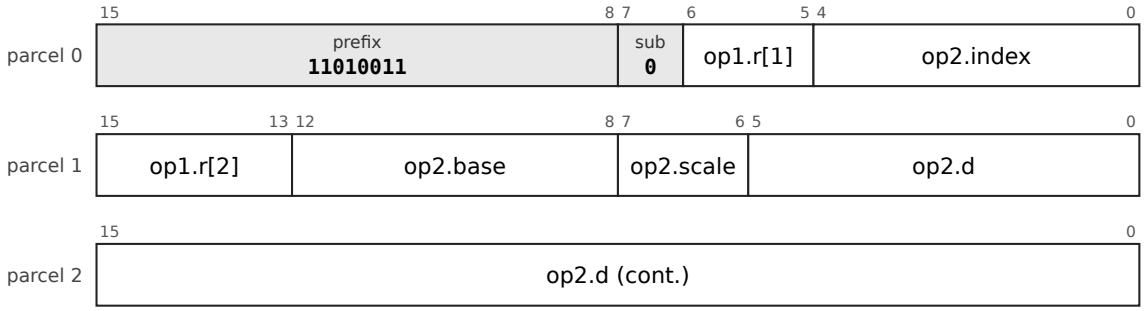
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## CMP: Compare

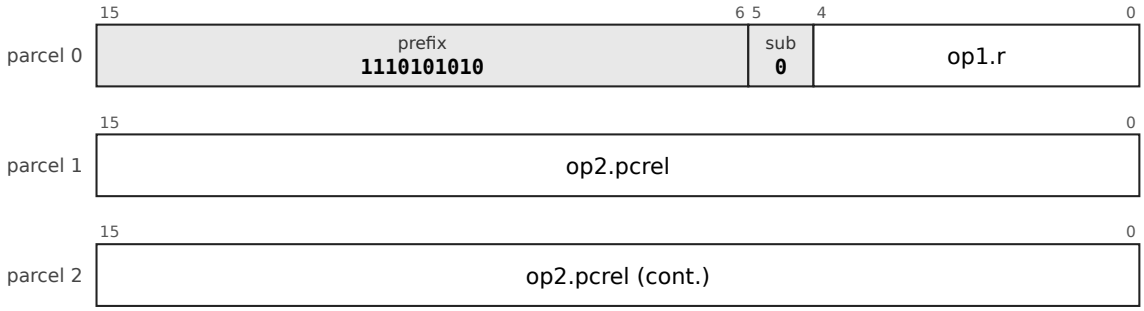
**CMP: CMF.B28, prefix 111000000, sub-opcode 0; operands reg5, b5d28s-x1; 6 bytes**



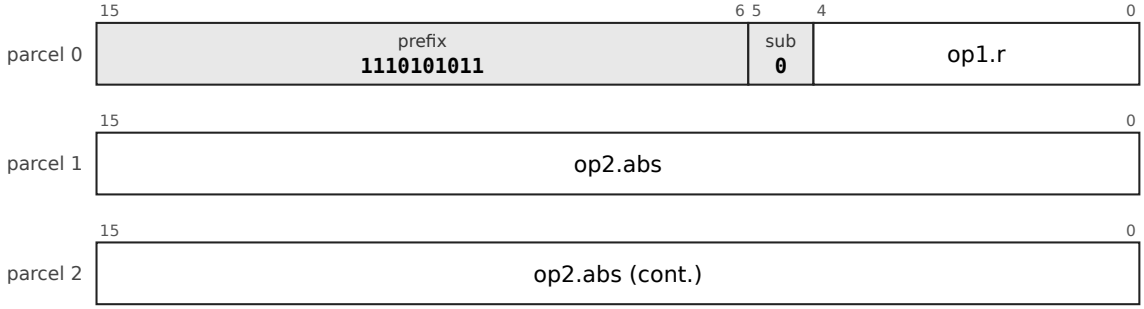

**CMP: CMF.X22, prefix 11010011, sub-opcode 0; operands reg5, x5-5d22s; 6 bytes**



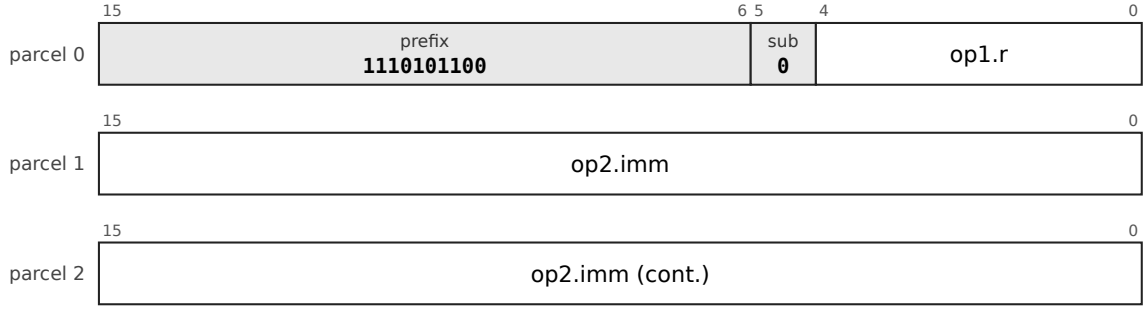

**CMP: CMF.PC32, prefix 1110101010, sub-opcode 0; operands reg5, pc32-u1; 6 bytes**



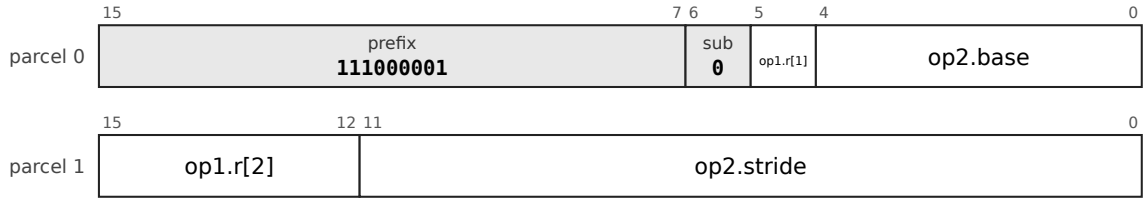

**CMP: CMF.A32, prefix 1110101011, sub-opcode 0; operands reg5, abs32; 6 bytes**



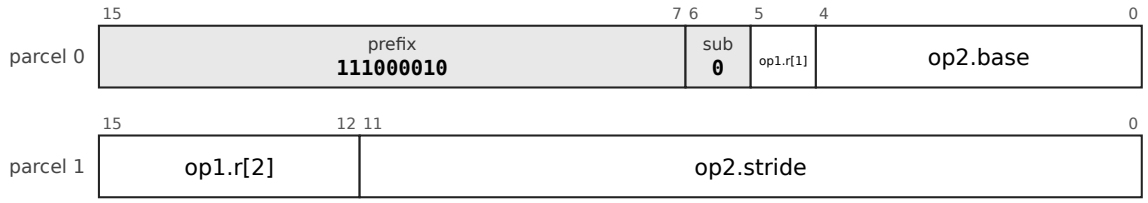
**CMP: CMF.I32, prefix 1110101100, sub-opcode 0; operands reg5, imm32s; 6 bytes**



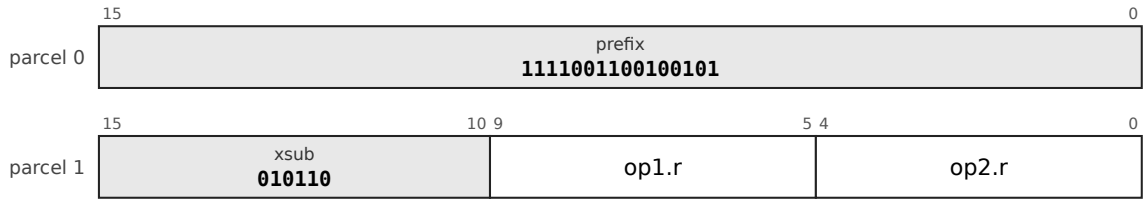

**CMP: CMF.PI12, prefix 111000001, sub-opcode 0; operands reg5, pi5-s12; 4 bytes**



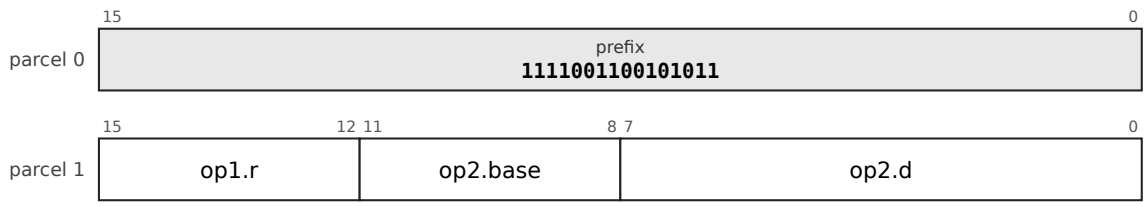

**CMP: CMF.PD12, prefix 111000010, sub-opcode 0; operands reg5, pd5-s12; 4 bytes**



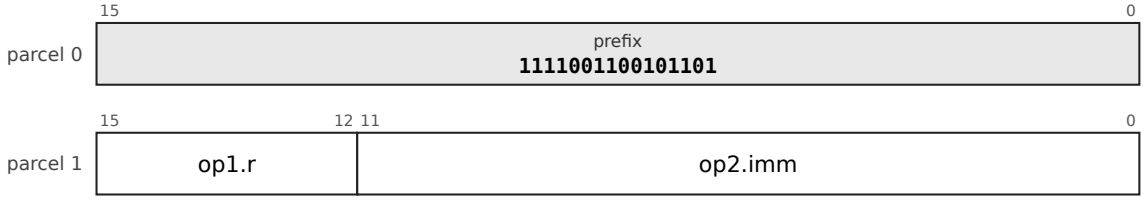

**CMP: RRF5, prefix 1111001100100101, sub-opcode 010110; operands reg5, reg5; 4 bytes**



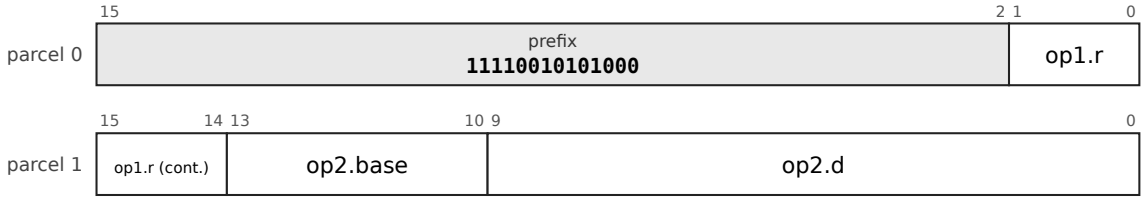

**CMP: CMBu8H/a, prefix 1111001100101011; operands reg4, b4d8u-xsz; 4 bytes**



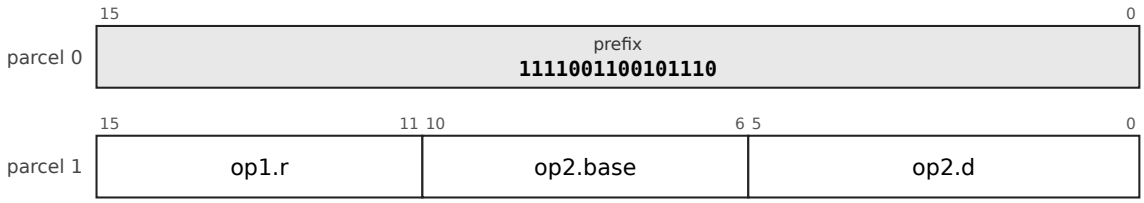
**CMP: CMI12H/a, prefix 1111001100101101; operands reg4, imm12s; 4 bytes**



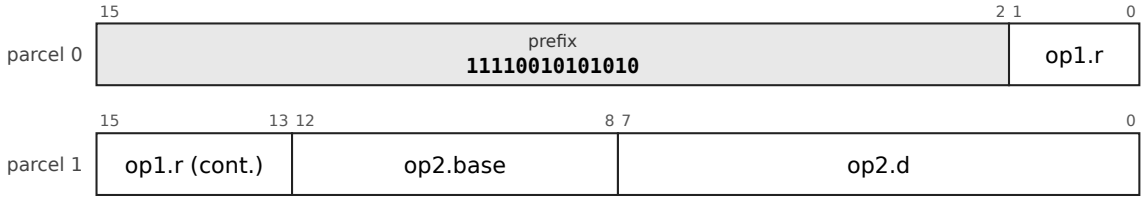

**CMP: CMBu10H/a, prefix 11110010101000; operands reg4, b4d10u-xsz; 4 bytes**



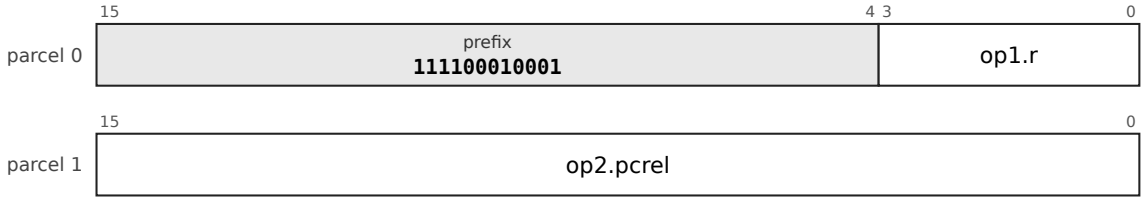

**CMP: CMBu6/a, prefix 1111001100101110; operands reg5, b5d6u-xsz; 4 bytes**



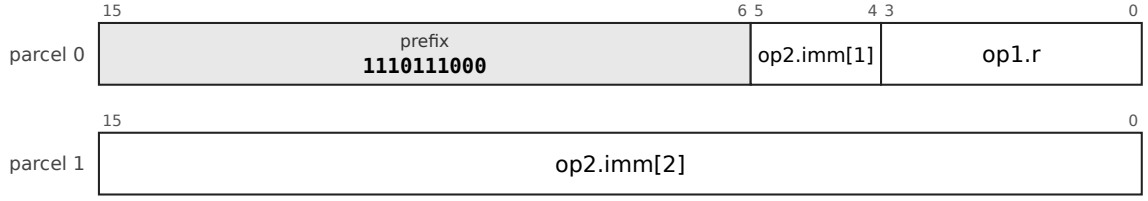

**CMP: CMBu8/a, prefix 11110010101010; operands reg5, b5d8u-xsz; 4 bytes**



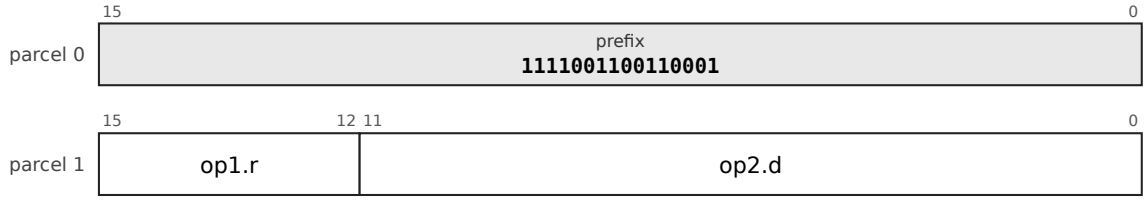

**CMP: CMPC16H/a, prefix 111100010001; operands reg4, pc16-u1; 4 bytes**



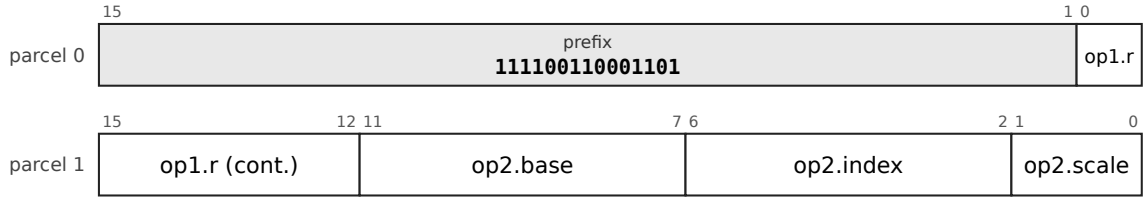
**CMP: CMI18H/a, prefix 1110111000; operands reg4, imm18s; 4 bytes**



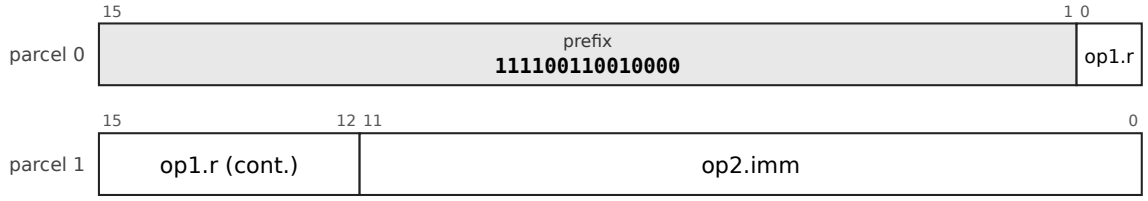

**CMP: CMS12H/a, prefix 1111001100110001; operands reg4, spd12u-xsz; 4 bytes**



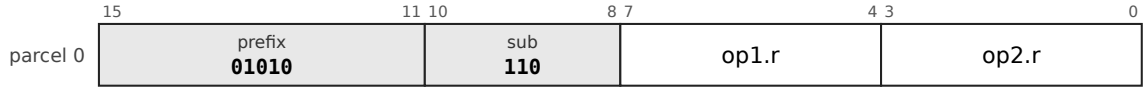

**CMP: CMX0/a, prefix 111100110001101; operands reg5, x5-5d0s; 4 bytes**




**CMP: CMI12/a, prefix 111100110010000; operands reg5, imm12s; 4 bytes**




**CMP: RRF, prefix 01010, sub-opcode 110; operands reg4, reg4; 2 bytes**

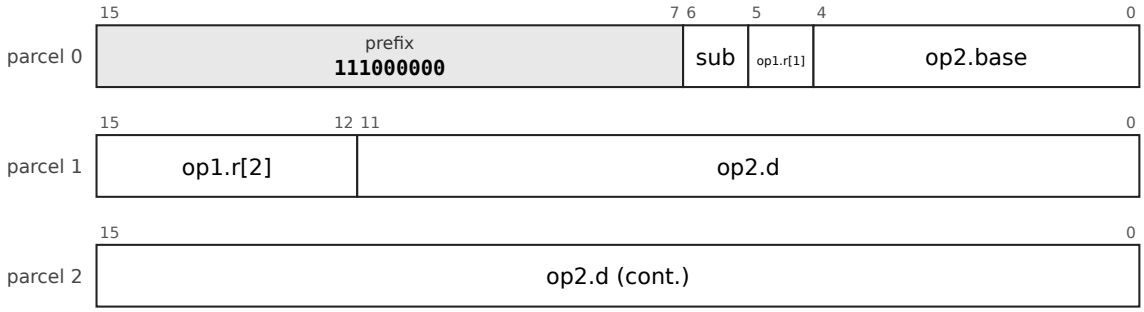



**Mnemonics:** `CMP`

**Syntax**

```
CMP Rd, rm        ; register first
```

### Encoding




The chart is the leaf `CMF-B28-1`, CMF.B28, prefix `111000000`. `op1` is `reg5`: register, 5 bits. `op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `CMF-B28-1`, `CMF-X22-1`, `CMF-PI12-1`, `CMF-PD12-1`, `CMF-PC32-1`, `CMF-A32-1`, `CMF-I32-1`, `RRF-2`, `RRF5-1`, `CMBu8H-a-1`, `CMI12H-a-1`, `CMBu10H-a-1`, `CMBu6-a-1`, `CMBu8-a-1`, `CMPC16H-a-1`, `CMI18H-a-1`, `CMS12H-a-1`, `CMX0-a-1`, `CMI12-a-1`.

### Operation

```
tmp ← Rd - rm         ; the result is discarded
```

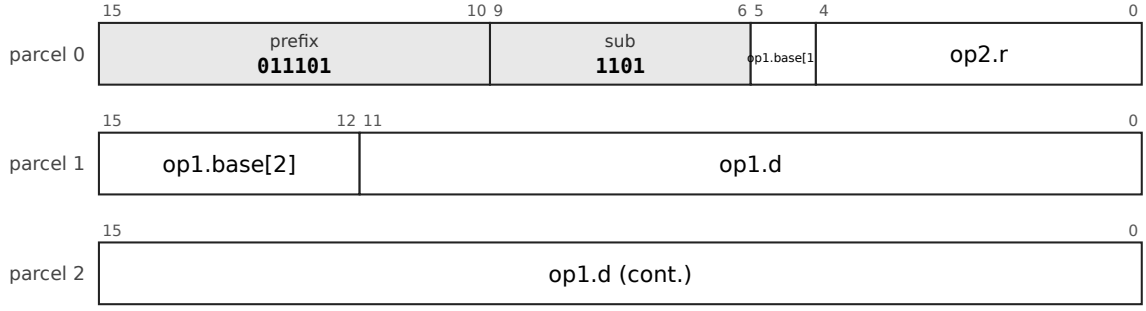
**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

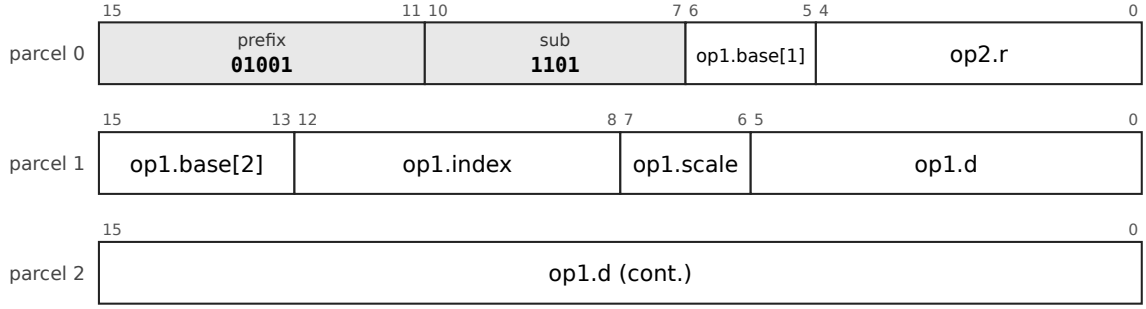
**The operands are in the order register, then the other operand**, the reverse of the PDP-11 `CMP src,dst`: `CMP Rd, rm` computes `Rd - rm`, so a compare of a register against an immediate or memory operand does not reverse its branch condition. After `CMP`, `BHIS`/`BCC` mean unsigned `>=` (no borrow) and `BLO`/`BCS` mean a borrow occurred.

## MUL: Multiply, low 64 bits

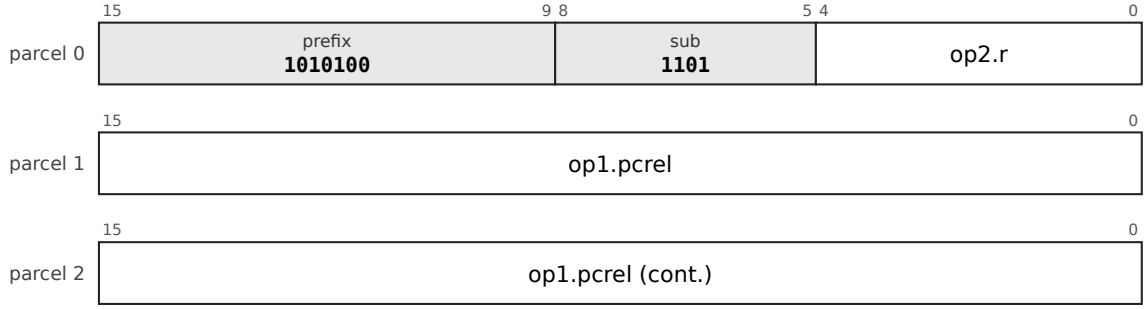
**MUL: LDF.B28, prefix 011101, sub-opcode 1101; operands b5d28s-x1, reg5; 6 bytes**



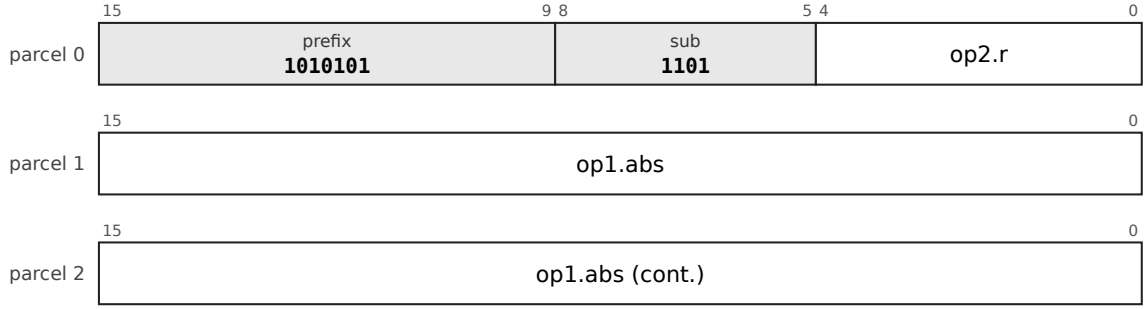

**MUL: LDF.X22, prefix 01001, sub-opcode 1101; operands x5-5d22s, reg5; 6 bytes**



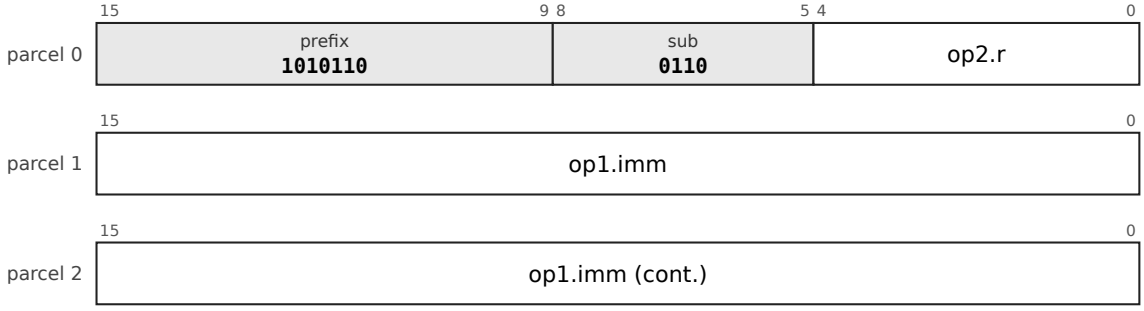

**MUL: LDF.PC32, prefix 1010100, sub-opcode 1101; operands pc32-u1, reg5; 6 bytes**



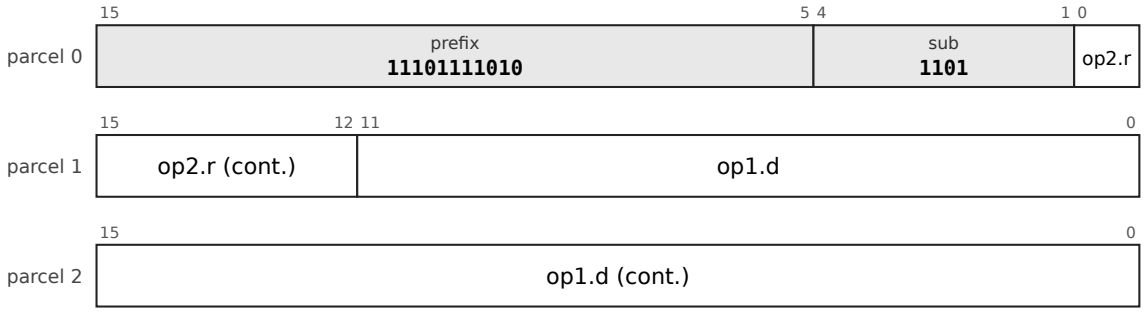

**MUL: LDF.A32, prefix 1010101, sub-opcode 1101; operands abs32, reg5; 6 bytes**



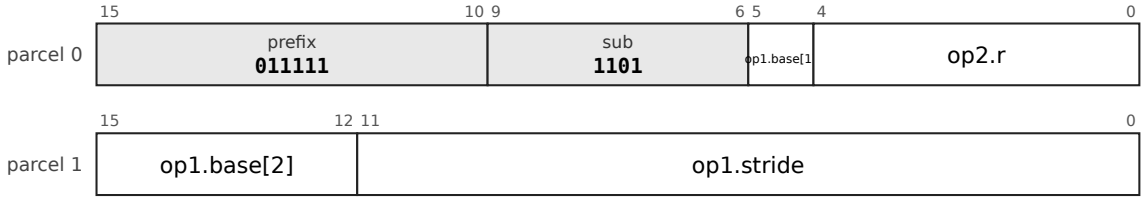
**MUL: LDFI.I32, prefix 1010110, sub-opcode 0110; operands imm32s, reg5; 6 bytes**



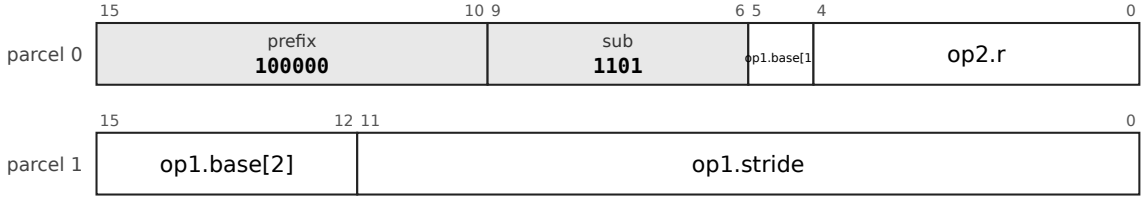

**MUL: DPF.D28, prefix 11101111010, sub-opcode 1101; operands dpd28s-x1, reg5; 6 bytes**



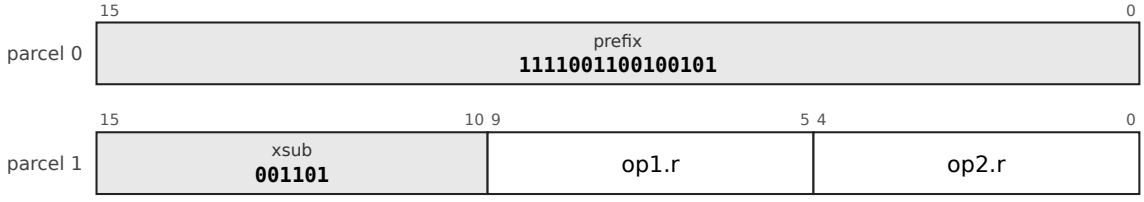

**MUL: LDF.PI12, prefix 011111, sub-opcode 1101; operands pi5-s12, reg5; 4 bytes**



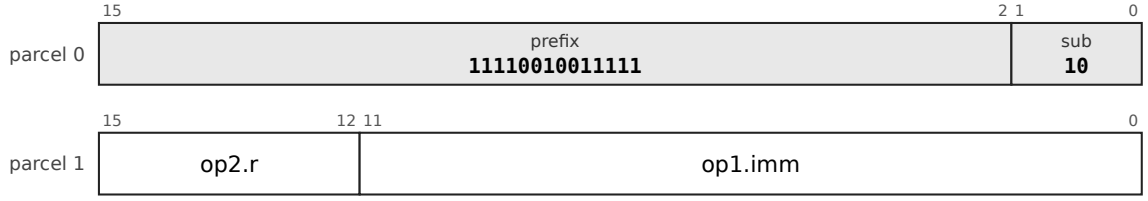

**MUL: LDF.PD12, prefix 100000, sub-opcode 1101; operands pd5-s12, reg5; 4 bytes**



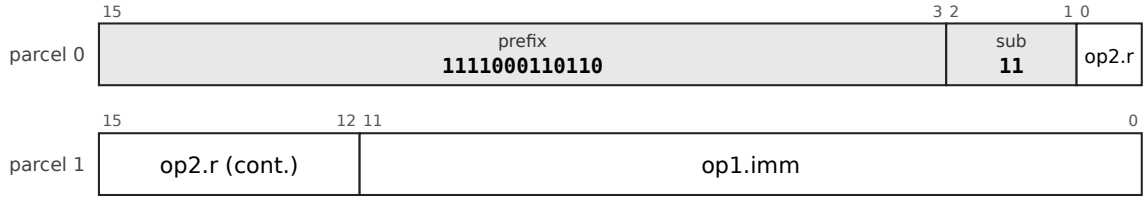

**MUL: RRF5, prefix 1111001100100101, sub-opcode 001101; operands reg5, reg5; 4 bytes**



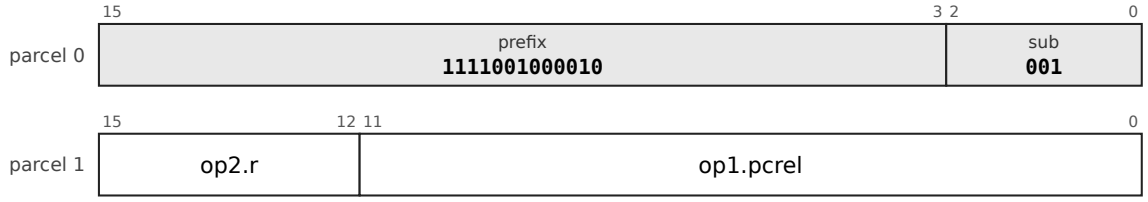
**MUL: LDI12H/c, prefix 11110010011111, sub-opcode 10; operands imm12s, reg4; 4 bytes**



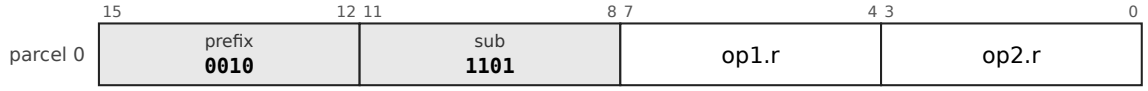

**MUL: LDI12/d, prefix 1111000110110, sub-opcode 11; operands imm12s, reg5; 4 bytes**




**MUL: LDPC12H/d, prefix 1111001000010, sub-opcode 001; operands pc12-u1, reg4; 4 bytes**




**MUL: RRF, prefix 0010, sub-opcode 1101; operands reg4, reg4; 2 bytes**

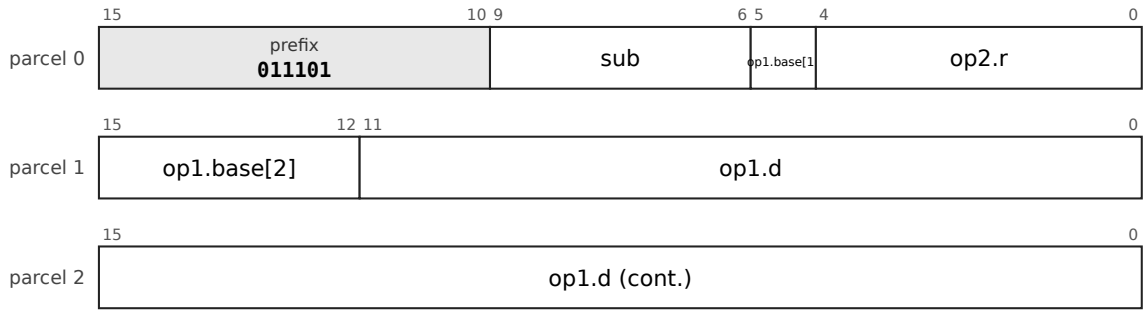



**Mnemonics:** `MUL` (aliases: `MULU`)

### Syntax

```
MUL rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000,

`movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `LDI12H-c-1`, `LDI12-d-1`, `LDPC12H-d-1`.

### Operation

```
Rd ← (Rd × rm)[63:0]         ; sign-agnostic
```

**Flags** `N` and `Z` from the result, `V` set when the signed 128-bit product does not fit in 64 bits, `C` unchanged.

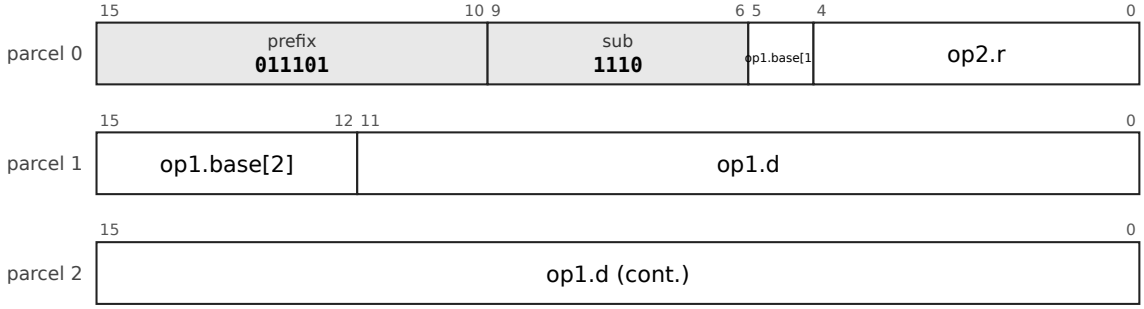
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The low 64 bits of a two's-complement product are the same whether the operands are read as signed or unsigned, so there is one low-half multiply. The mnemonic `MULU` is an alias of `MUL`. The high half is `MULH` (signed) and `MULHU` (unsigned).

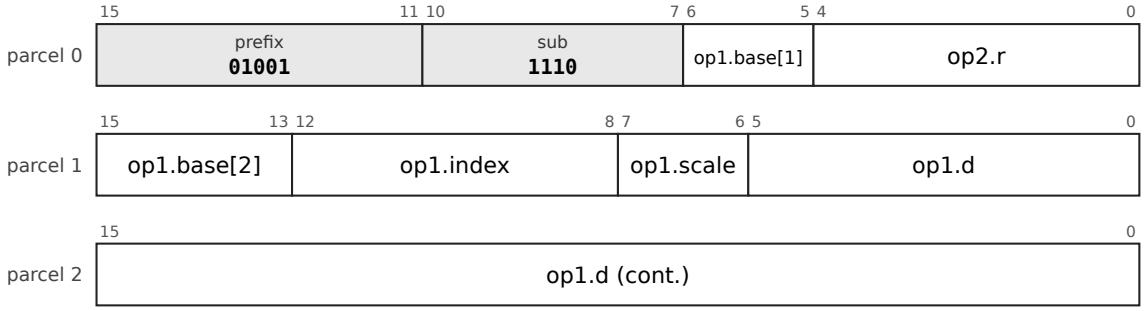
**`MULU`** is an alias of `MUL`: the assembler writes it as that word.

## MULH: Multiply, high 64 bits, signed

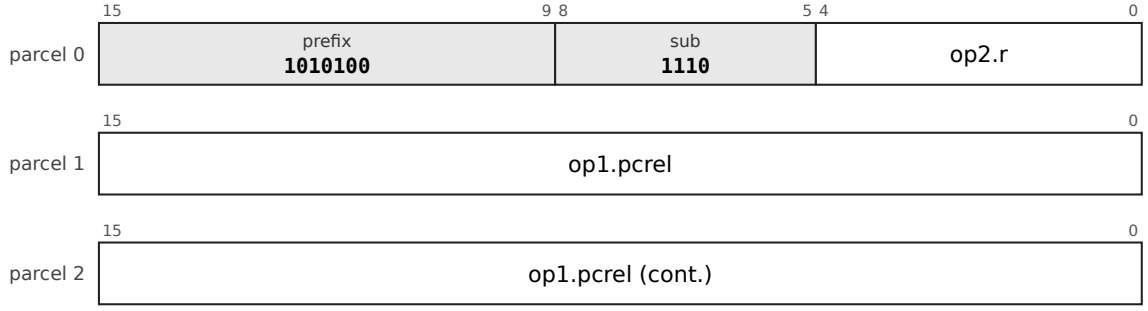
**MULH: LDF.B28, prefix 011101, sub-opcode 1110; operands b5d28s-x1, reg5; 6 bytes**



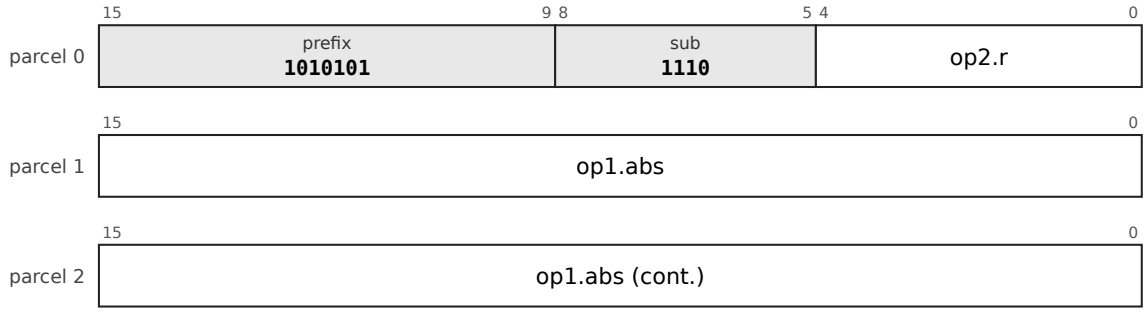

**MULH: LDF.X22, prefix 01001, sub-opcode 1110; operands x5-5d22s, reg5; 6 bytes**



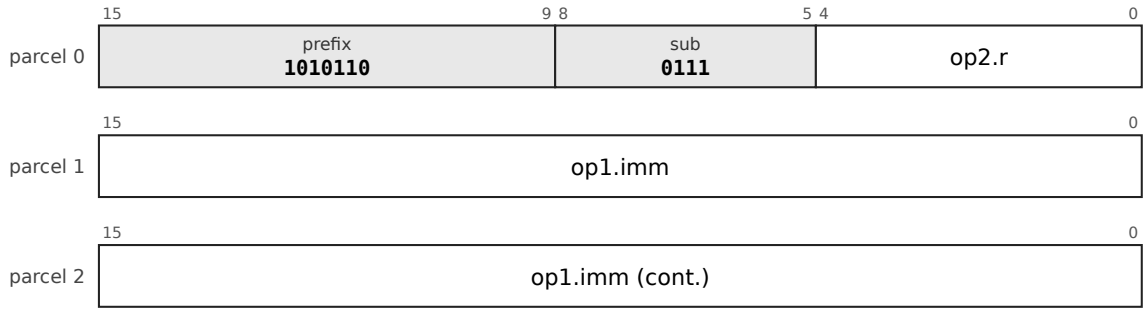
**MULH: LDF.PC32, prefix 1010100, sub-opcode 1110; operands pc32-u1, reg5; 6 bytes**



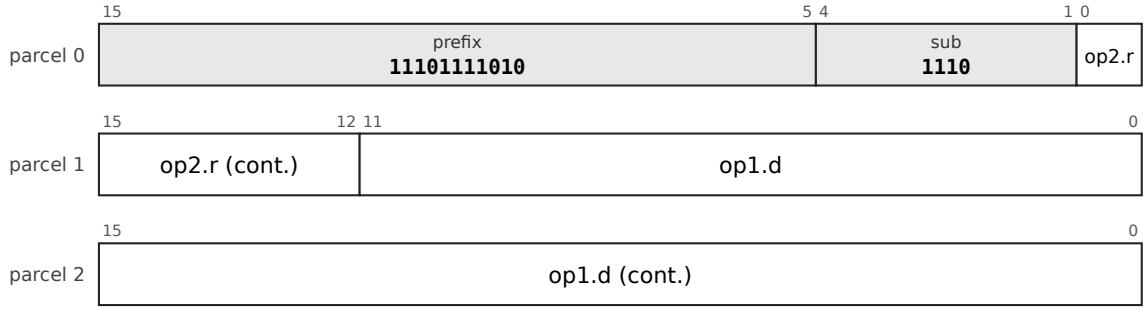

**MULH: LDF.A32, prefix 1010101, sub-opcode 1110; operands abs32, reg5; 6 bytes**



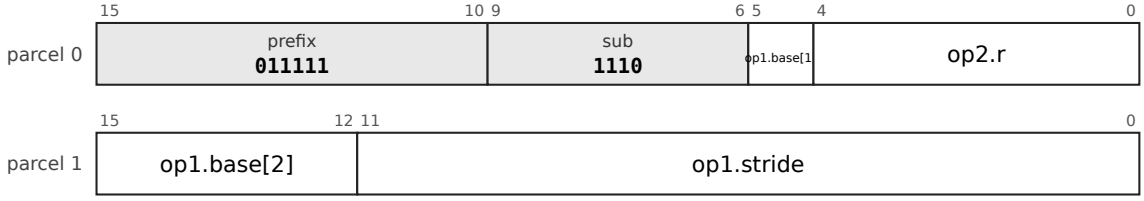

**MULH: LDFI.I32, prefix 1010110, sub-opcode 0111; operands imm32s, reg5; 6 bytes**



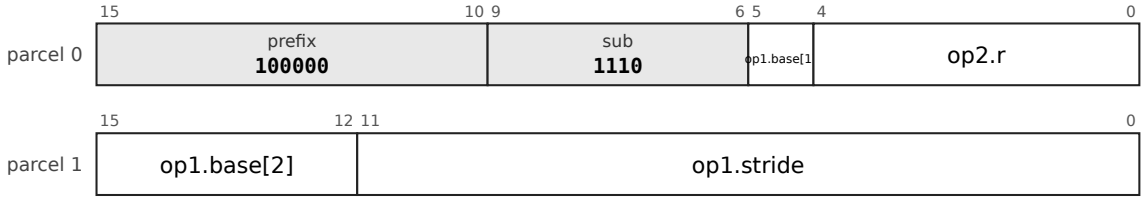

**MULH: DPF.D28, prefix 11101111010, sub-opcode 1110; operands dpd28s-x1, reg5; 6 bytes**



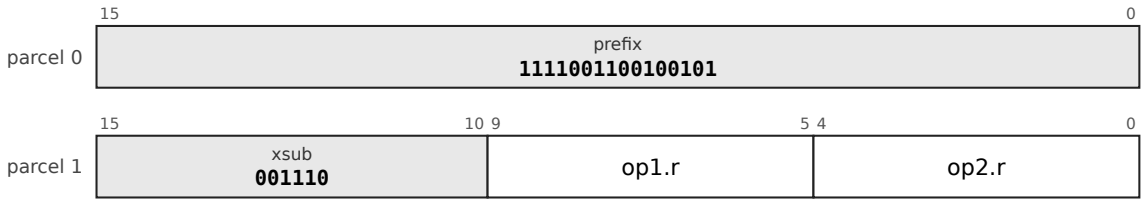
**MULH: LDF.PI12, prefix 011111, sub-opcode 1110; operands pi5-s12, reg5; 4 bytes**



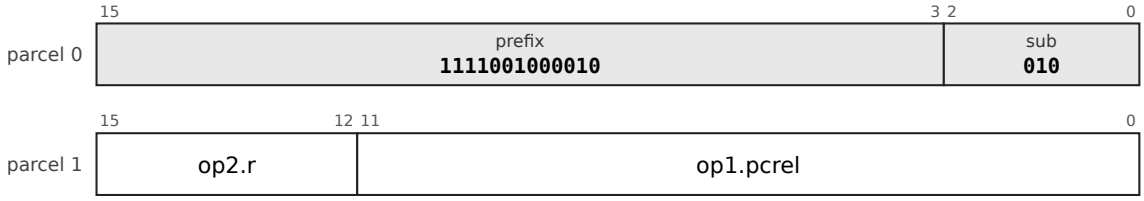

**MULH: LDF.PD12, prefix 100000, sub-opcode 1110; operands pd5-s12, reg5; 4 bytes**



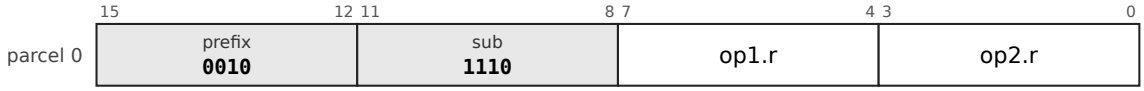

**MULH: RRF5, prefix 1111001100100101, sub-opcode 001110; operands reg5, reg5; 4 bytes**




**MULH: LDPC12H/d, prefix 1111001000010, sub-opcode 010; operands pc12-u1, reg4; 4 bytes**




**MULH: RRF, prefix 0010, sub-opcode 1110; operands reg4, reg4; 2 bytes**

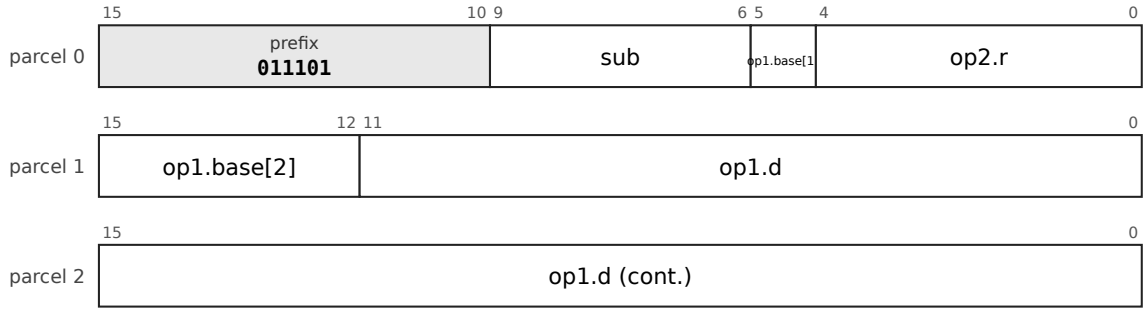



**Mnemonics:** `MULH`

## Syntax

```
MULH rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `LDPC12H-d-1`.

### Operation

```
Rd ← (sext(Rd) × sext(rm))[127:64]
```

**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

With `MULH`, division by a constant is a multiply by a reciprocal and a shift, which is what compilers emit.

**Why it is here.** The low 64 bits of a two's-complement product are identical signed or unsigned, so a second multiply (`MULU`) would duplicate `MUL`; what the architecture needs in that place is the *high* half of the product. Without it a 64 by 64 to 128-bit multiply, every bignum multiply, 128-bit fixed point and divide-by-a-constant by multiplying by the reciprocal are impossible. `MULH` is that instruction. It is also why a divide is rare in compiled code: compilers turn division by a constant into `MULH` and a shift.

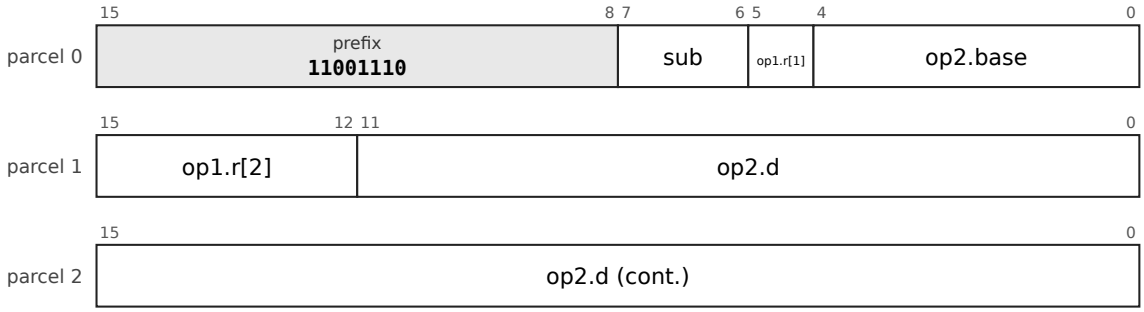
## ADDM: ALU operation with a memory destination

**Mnemonics:** `ADDM`, `SUBM`, `BICM`, `BISM`

### Syntax

```
ADDM Rs, rm
SUBM Rs, rm
BICM Rs, rm
BISM Rs, rm
```

### Encoding




The chart is the leaf `STF-B28-2`, STF.B28, prefix `11001110`. `op1` is `reg5`: register, 5 bits. `op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `STF-B28-2`, `STF-X22-2`, `STF-PI12-2`, `STF-PD12-2`, `STF-PC32-2`, `STF-A32-2`.

### Operation

```
ADDM: M[EA] ← M[EA] + Rs
SUBM: M[EA] ← M[EA] - Rs
BICM: M[EA] ← M[EA] AND NOT Rs
BISM: M[EA] ← M[EA] OR Rs
```

**Flags** As for `ADD`, `SUB`, `BIC` and `BIS`.

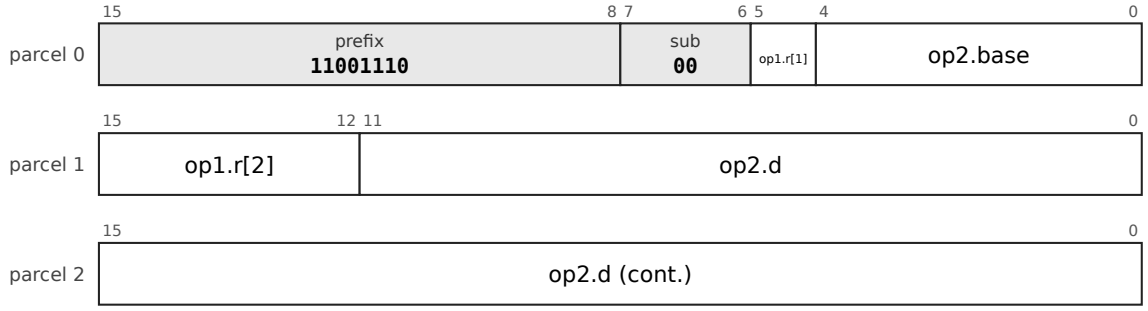
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Write permission is checked during the read access, so the write cannot fault after the read has succeeded.

The forms with a memory destination are in the ordinary memory families of the encoding, with every addressing form (errata 87).

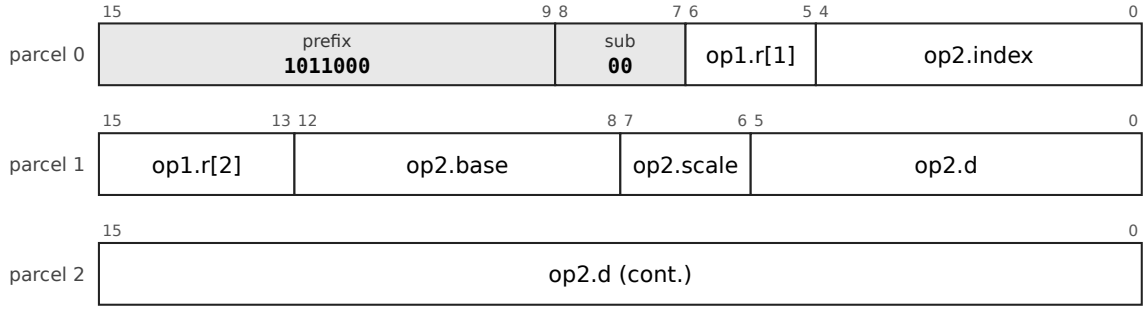
**Why it is here.** These are the read-modify-write forms with a memory destination. They cover the case where the second operand is a register, for example `BISM Rmask, flags(DP)`. The compiled corpus has 265 sites (1.6 KB) for them, measured for the original encoding, which is why they are not worth an opcode of their own there.

## *ADDM*

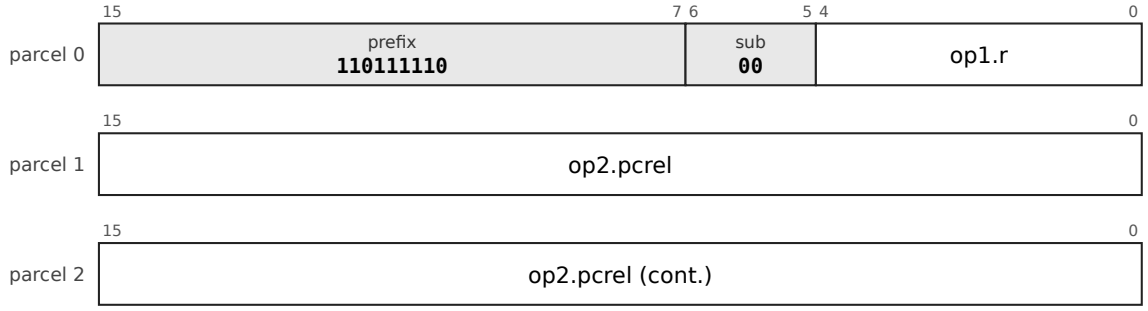
**ADDM: STF.B28, prefix 11001110, sub-opcode 00; operands reg5, b5d28s-x1; 6 bytes**



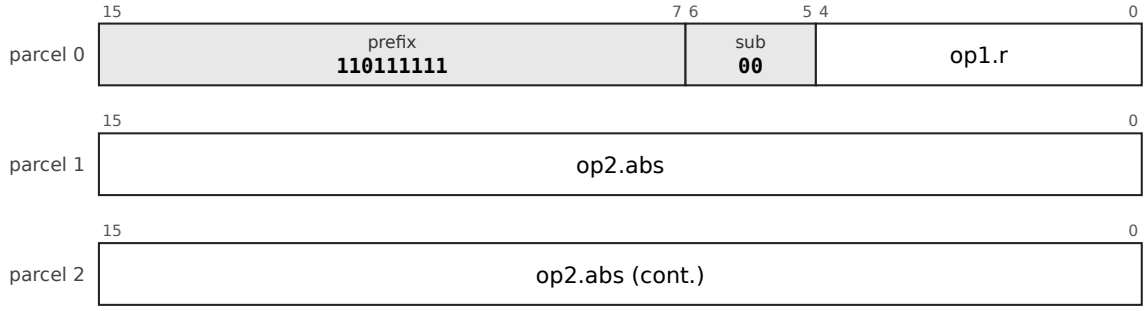

**ADDM: STF.X22, prefix 1011000, sub-opcode 00; operands reg5, x5-5d22s; 6 bytes**



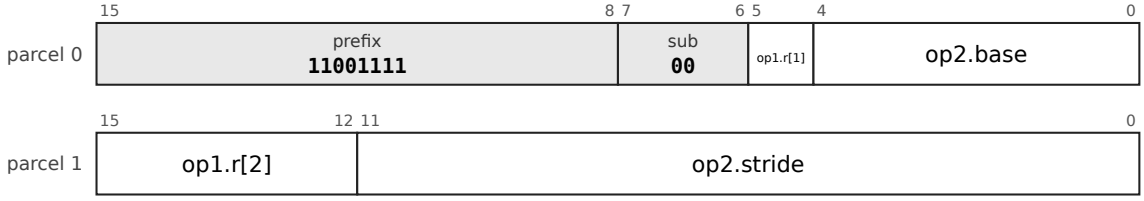

**ADDM: STF.PC32, prefix 110111110, sub-opcode 00; operands reg5, pc32-u1; 6 bytes**



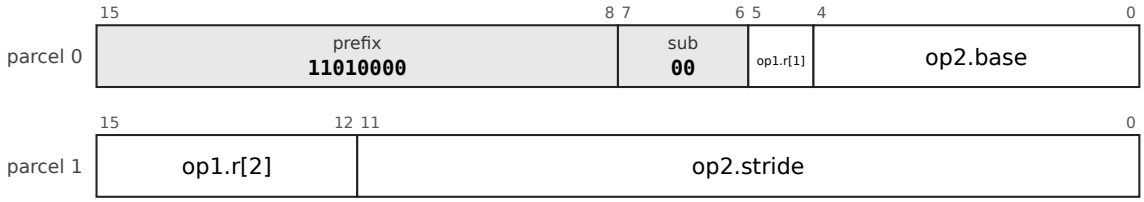

**ADDM: STF.A32, prefix 110111111, sub-opcode 00; operands reg5, abs32; 6 bytes**



**ADDM: STF.PI12, prefix 11001111, sub-opcode 00; operands reg5, pi5-s12; 4 bytes**




**ADDM: STF.PD12, prefix 11010000, sub-opcode 00; operands reg5, pd5-s12; 4 bytes**

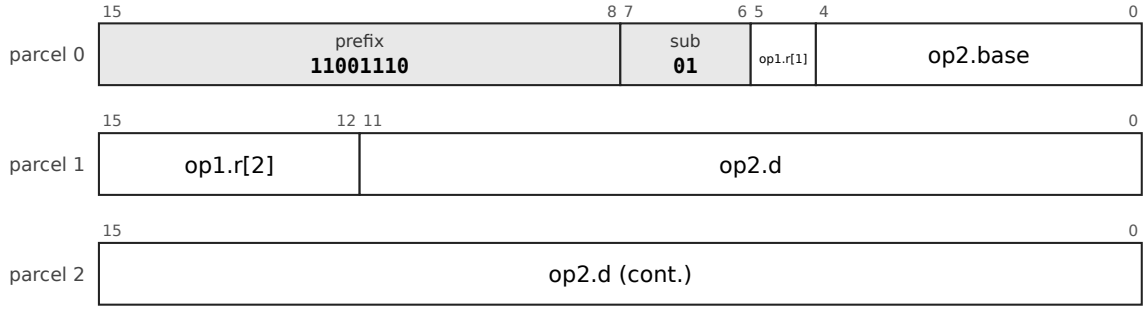



```
ADDM Rs, rm
```

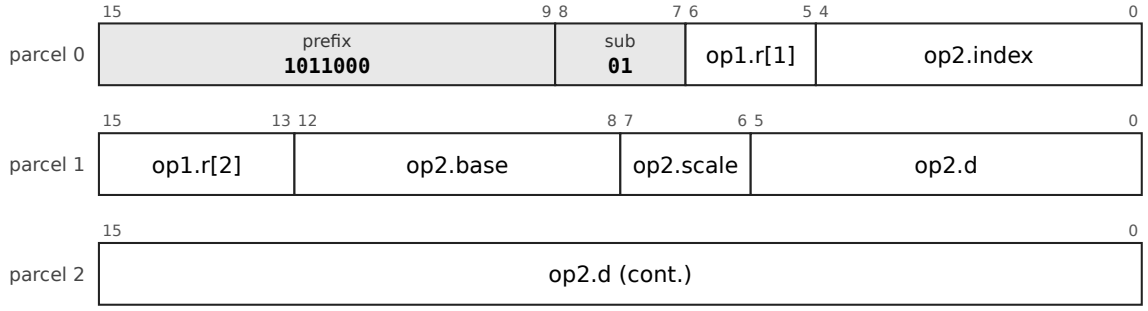
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

### *SUBM*

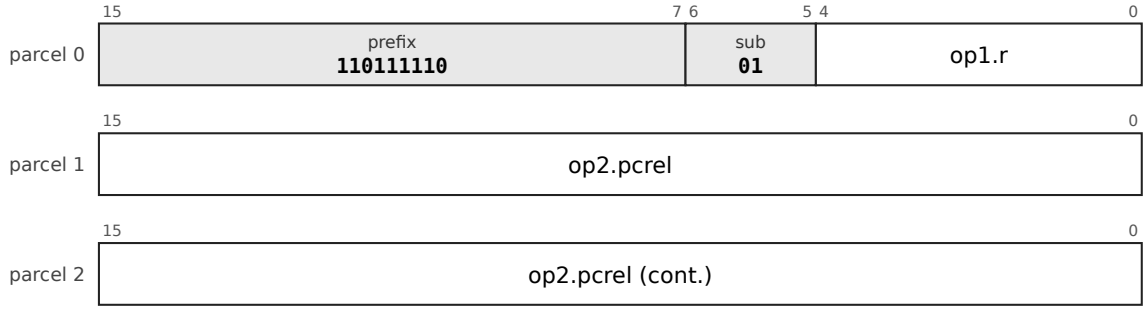
**SUBM: STF.B28, prefix 11001110, sub-opcode 01; operands reg5, b5d28s-x1; 6 bytes**



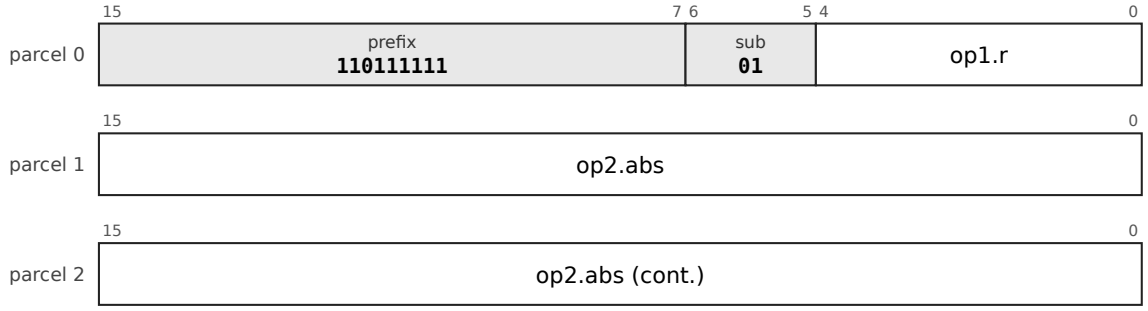

**SUBM: STF.X22, prefix 1011000, sub-opcode 01; operands reg5, x5-5d22s; 6 bytes**



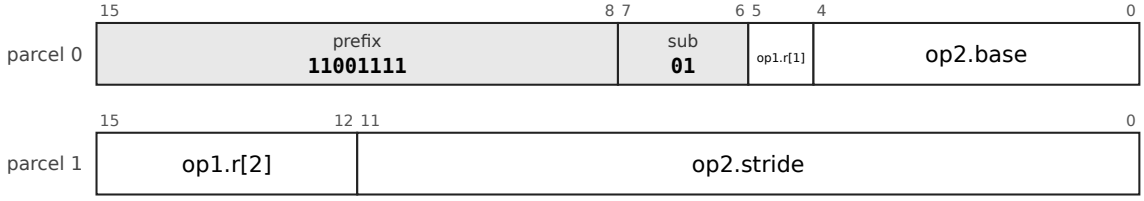

**SUBM: STF.PC32, prefix 110111110, sub-opcode 01; operands reg5, pc32-u1; 6 bytes**



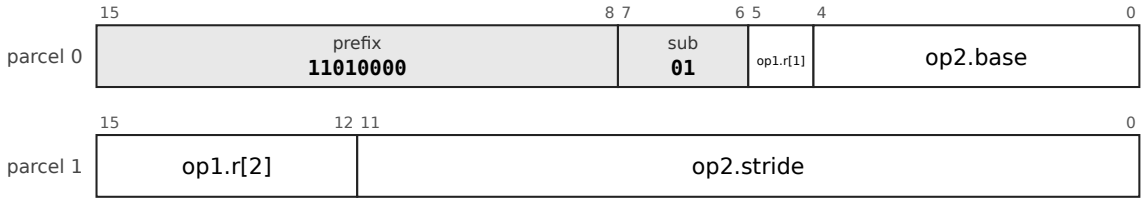

**SUBM: STF.A32, prefix 110111111, sub-opcode 01; operands reg5, abs32; 6 bytes**



**SUBM: STF.PI12, prefix 11001111, sub-opcode 01; operands reg5, pi5-s12; 4 bytes**




**SUBM: STF.PD12, prefix 11010000, sub-opcode 01; operands reg5, pd5-s12; 4 bytes**

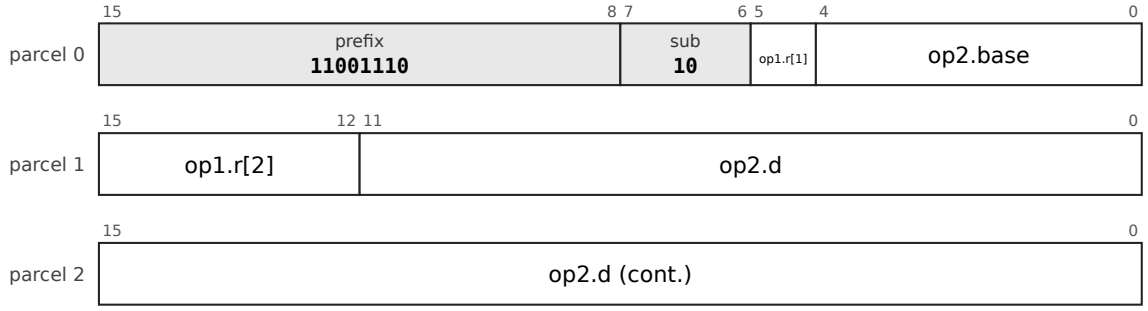



```
SUBM Rs, rm
```

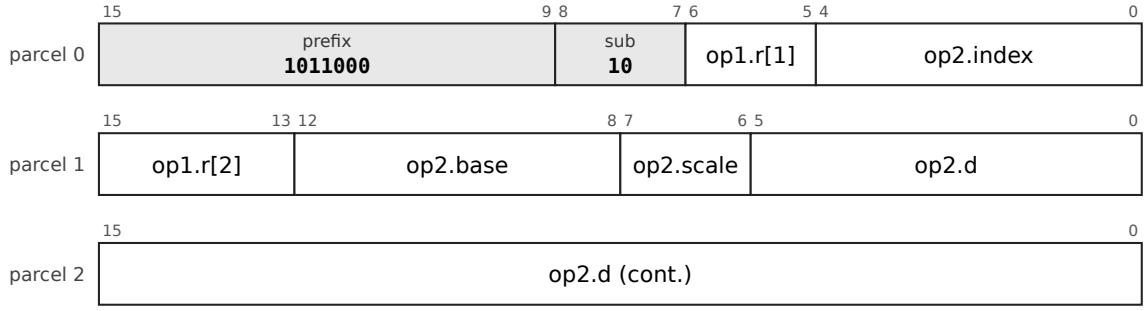
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

## *BICM*

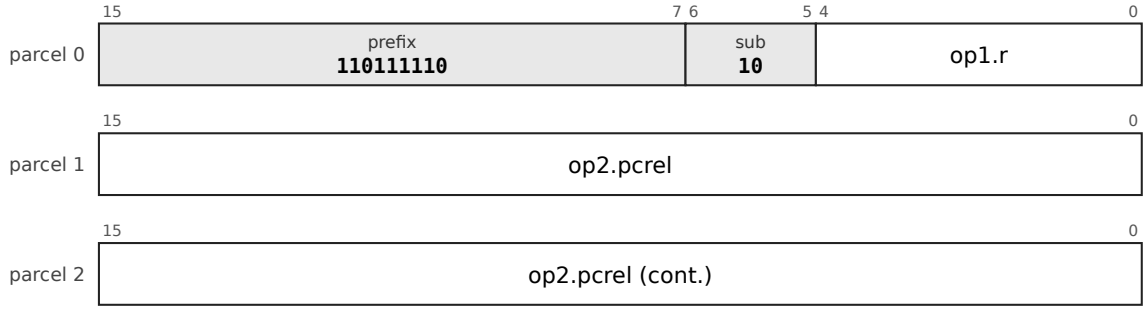
**BICM: STF.B28, prefix 11001110, sub-opcode 10; operands reg5, b5d28s-x1; 6 bytes**



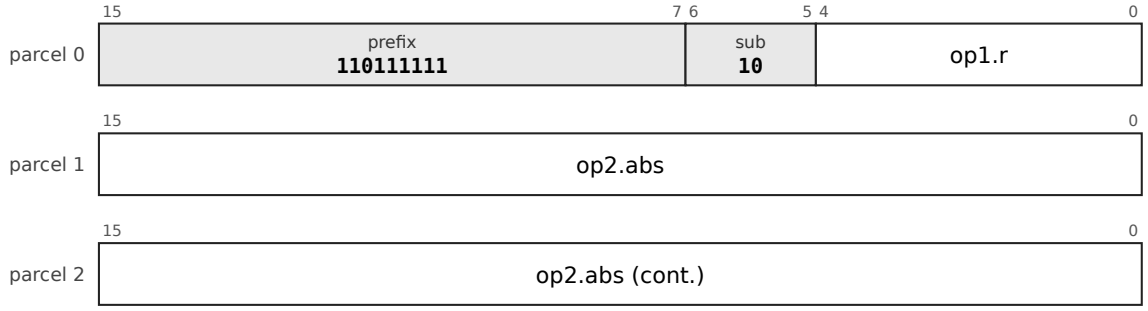

**BICM: STF.X22, prefix 1011000, sub-opcode 10; operands reg5, x5-5d22s; 6 bytes**



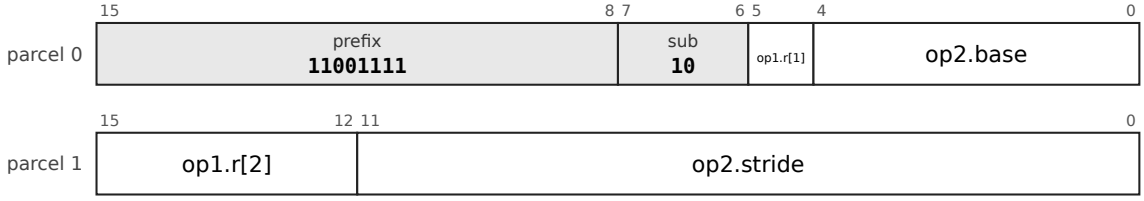

**BICM: STF.PC32, prefix 110111110, sub-opcode 10; operands reg5, pc32-u1; 6 bytes**



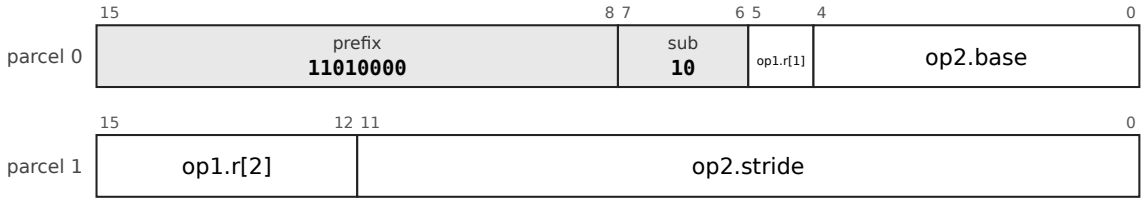

**BICM: STF.A32, prefix 110111111, sub-opcode 10; operands reg5, abs32; 6 bytes**



**BICM: STF.PI12, prefix 11001111, sub-opcode 10; operands reg5, pi5-s12; 4 bytes**




**BICM: STF.PD12, prefix 11010000, sub-opcode 10; operands reg5, pd5-s12; 4 bytes**

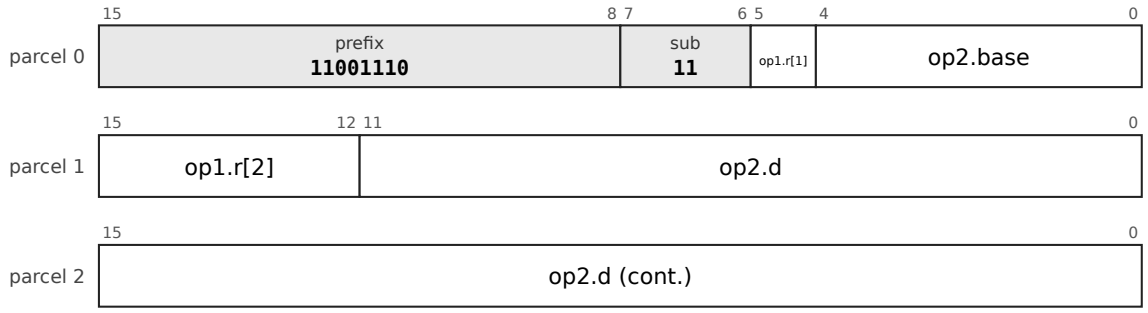



```
BICM Rs, rm
```

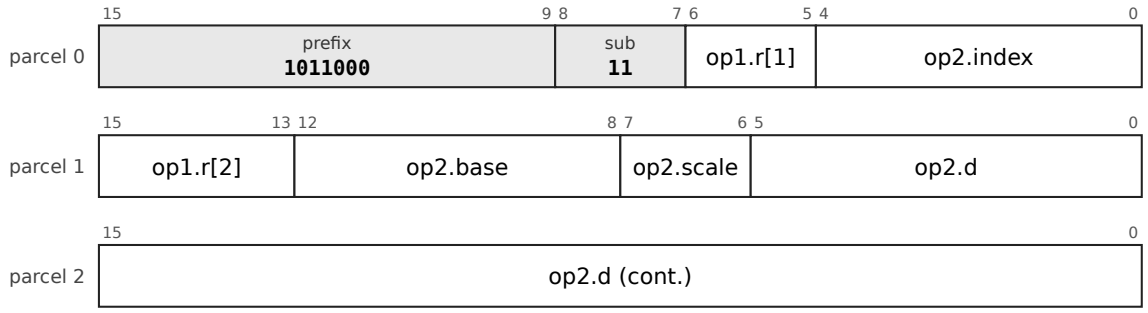
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

### *BISM*

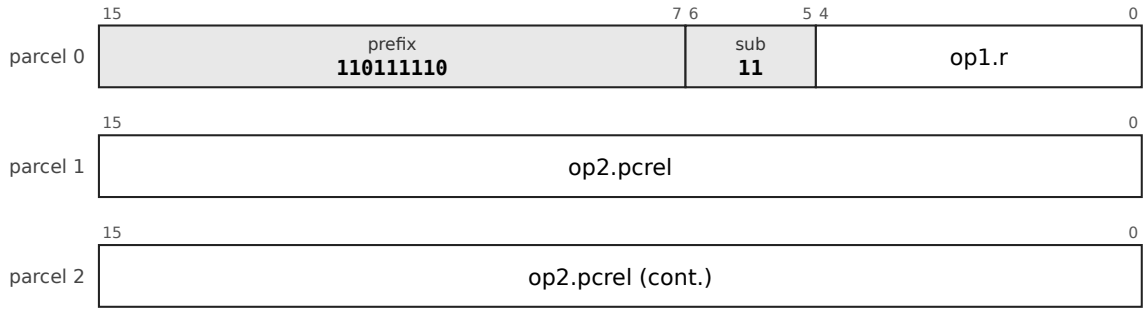
**BISM: STF.B28, prefix 11001110, sub-opcode 11; operands reg5, b5d28s-x1; 6 bytes**



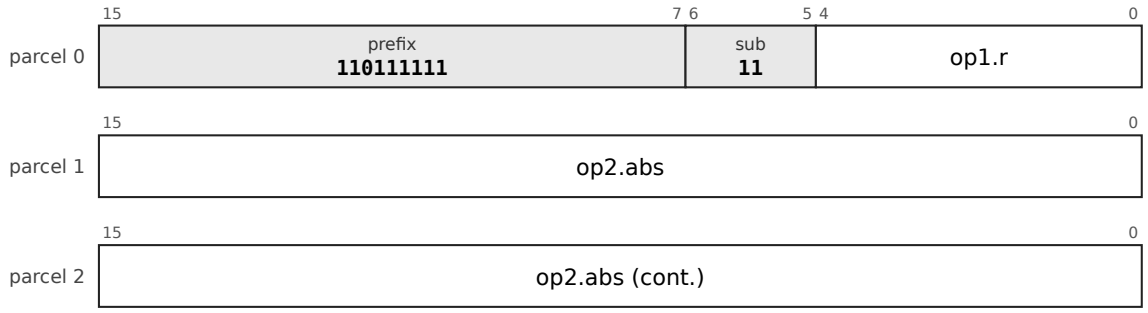

**BISM: STF.X22, prefix 1011000, sub-opcode 11; operands reg5, x5-5d22s; 6 bytes**



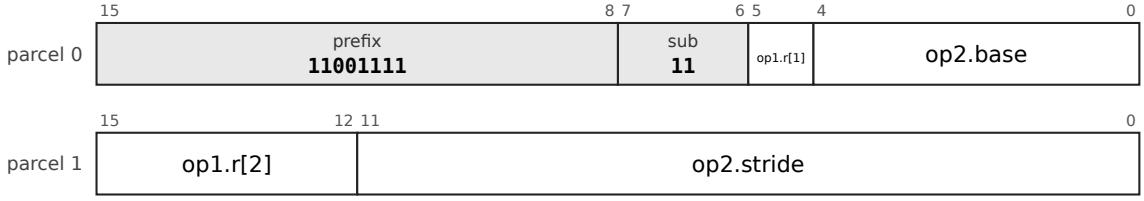

**BISM: STF.PC32, prefix 110111110, sub-opcode 11; operands reg5, pc32-u1; 6 bytes**



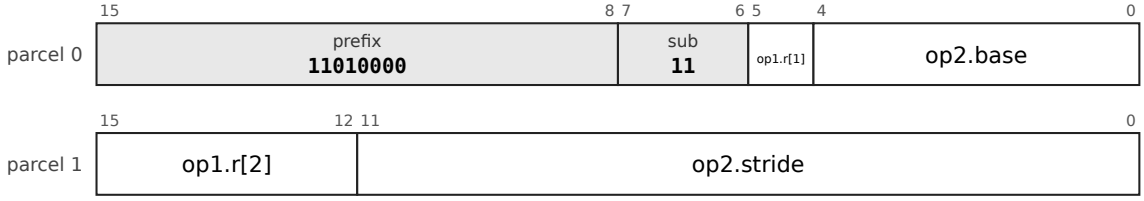

**BISM: STF.A32, prefix 110111111, sub-opcode 11; operands reg5, abs32; 6 bytes**



**BISM: STF.PI12, prefix 11001111, sub-opcode 11; operands reg5, pi5-s12; 4 bytes**




**BISM: STF.PD12, prefix 11010000, sub-opcode 11; operands reg5, pd5-s12; 4 bytes**

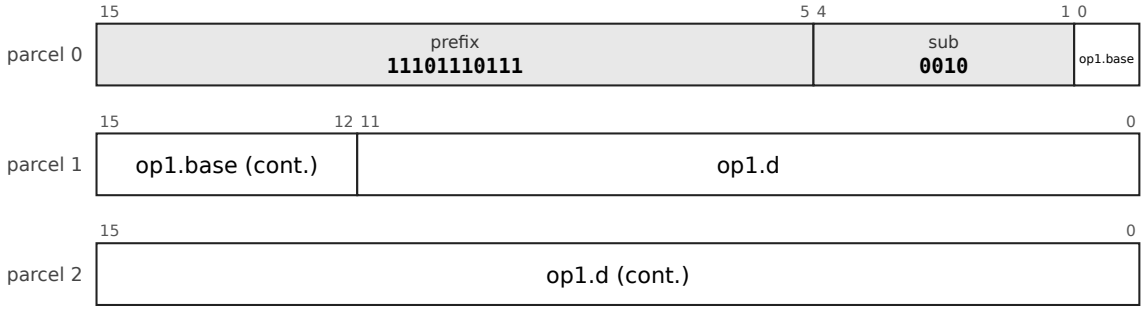



```
BISM Rs, rm
```

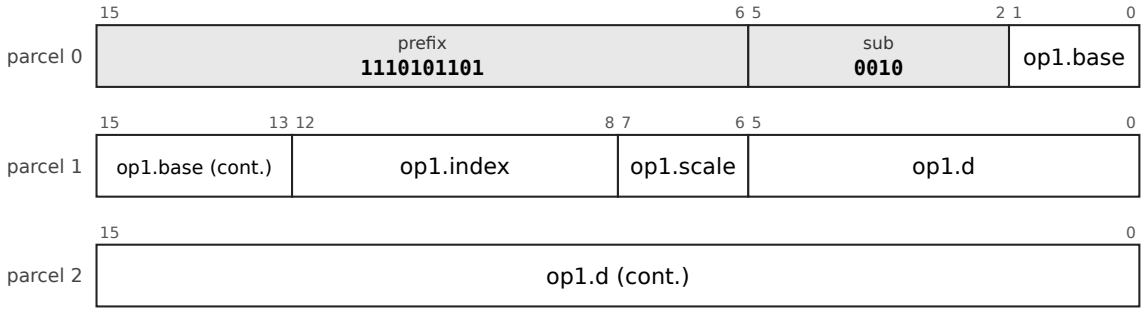
Operation, flags and exceptions as for the entry ADDM above; 6 layout forms, each a chart of its own (above).

## INC: Increment

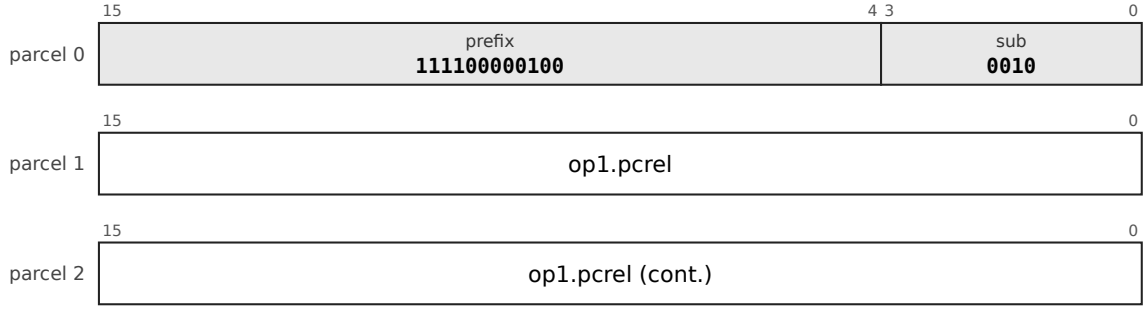
**INC: SOF.B28, prefix 11101110111, sub-opcode 0010; operands b5d28s-x1; 6 bytes**



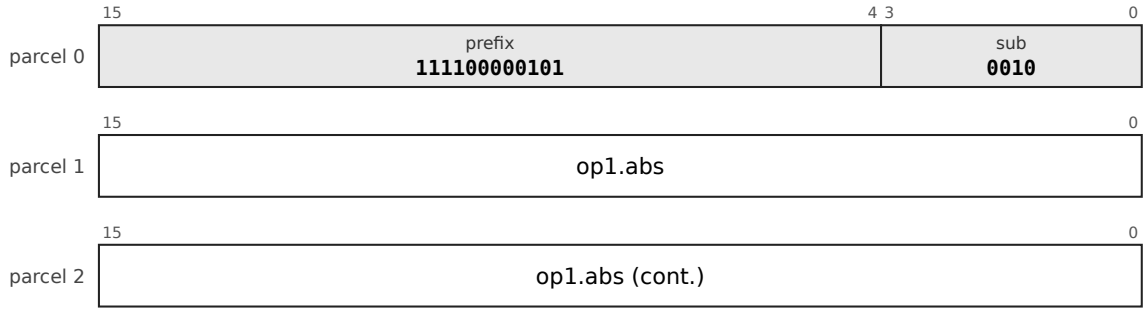

**INC: SOF.X22, prefix 1110101101, sub-opcode 0010; operands x5-5d22s; 6 bytes**



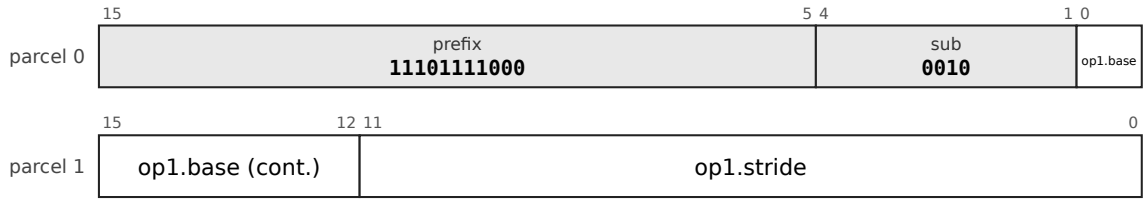
**INC: SOF.PC32, prefix 111100000100, sub-opcode 0010; operands pc32-u1; 6 bytes**



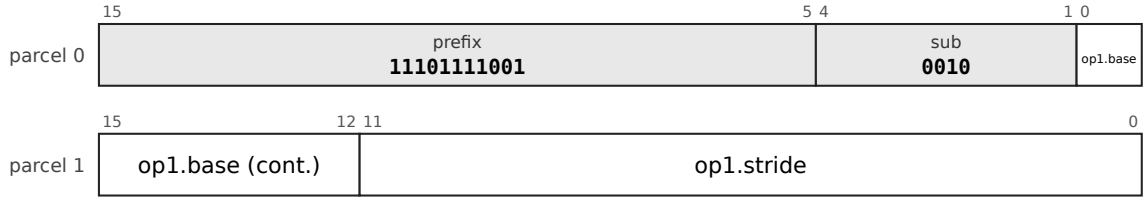

**INC: SOF.A32, prefix 111100000101, sub-opcode 0010; operands abs32; 6 bytes**



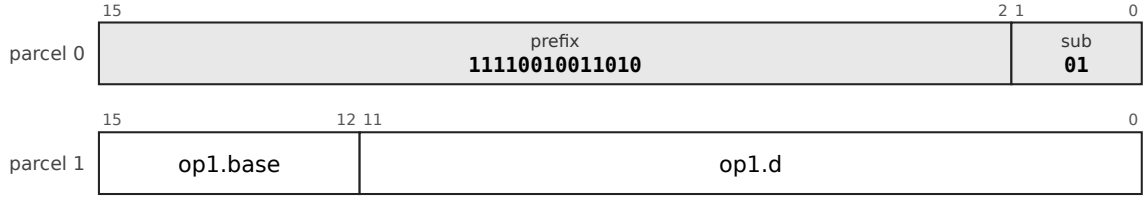

**INC: SOF.PI12, prefix 11101111000, sub-opcode 0010; operands pi5-s12; 4 bytes**



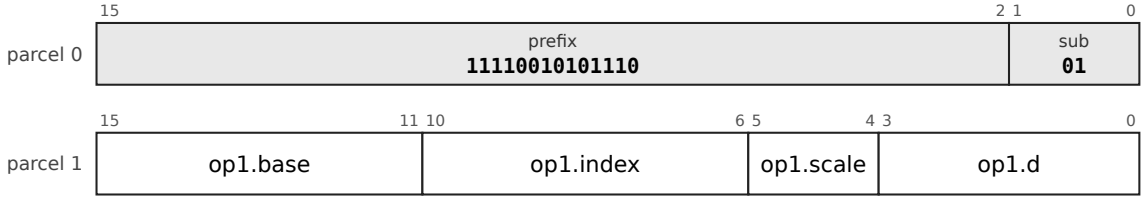

**INC: SOF.PD12, prefix 11101111001, sub-opcode 0010; operands pd5-s12; 4 bytes**



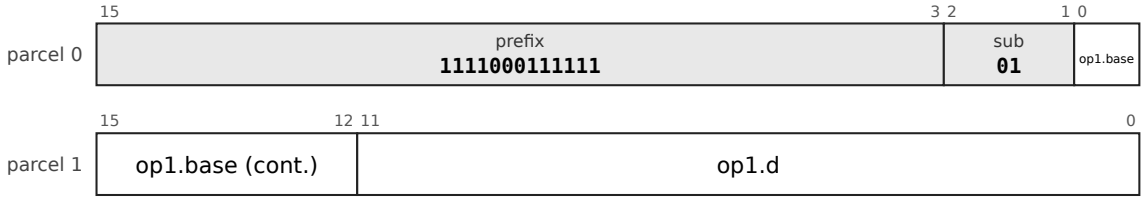

**INC: SOBu12H/a, prefix 11110010011010, sub-opcode 01; operands b4d12u-xsz; 4 bytes**



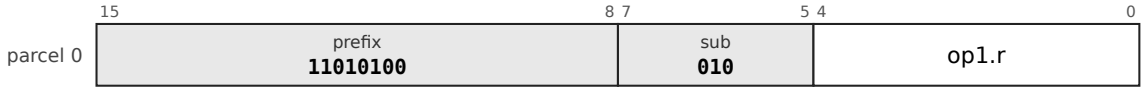
**INC: SOX4/a, prefix 11110010101110, sub-opcode 01; operands x5-5d4s; 4 bytes**




**INC: SOBs12/a, prefix 1111000111111, sub-opcode 01; operands b5d12s-xsz; 4 bytes**




**INC: SOPR, prefix 11010100, sub-opcode 010; operands reg5; 2 bytes**

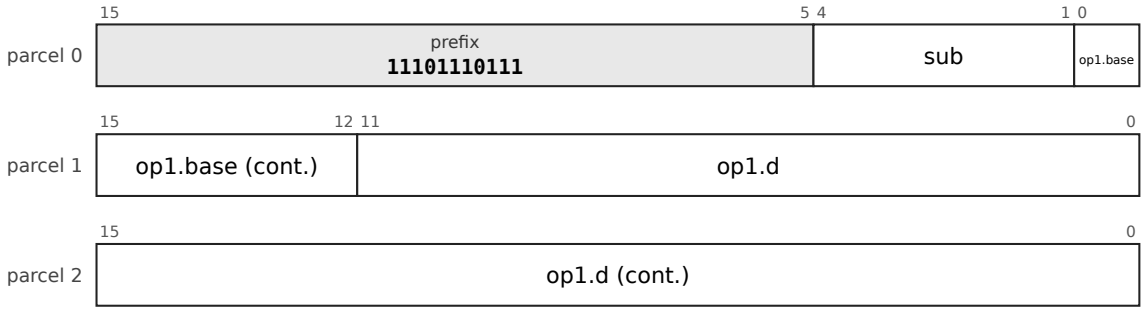



**Mnemonics:** `INC`

### Syntax

```
INC rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`, `SOBu12H-a-1`, `SOX4-a-1`, `SOBs12-a-1`.

### Operation

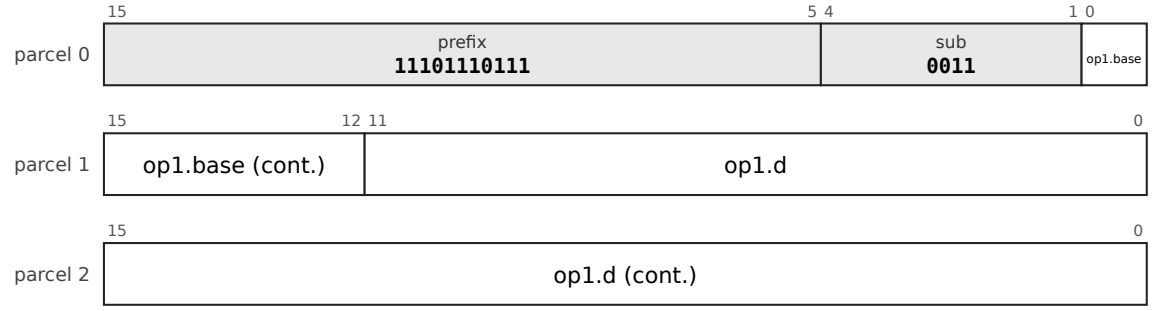
```
rm ← rm + 1
```

**Flags** `N`, `Z` and `V` as for `ADD`; `C` **unchanged**.

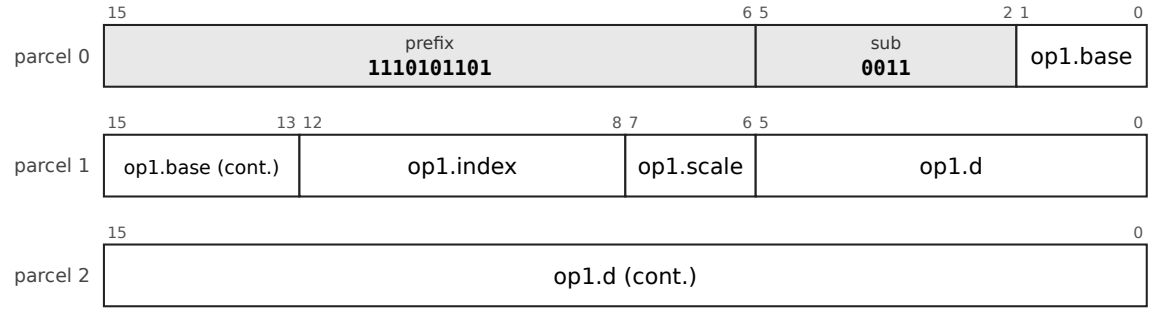
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## DEC: Decrement

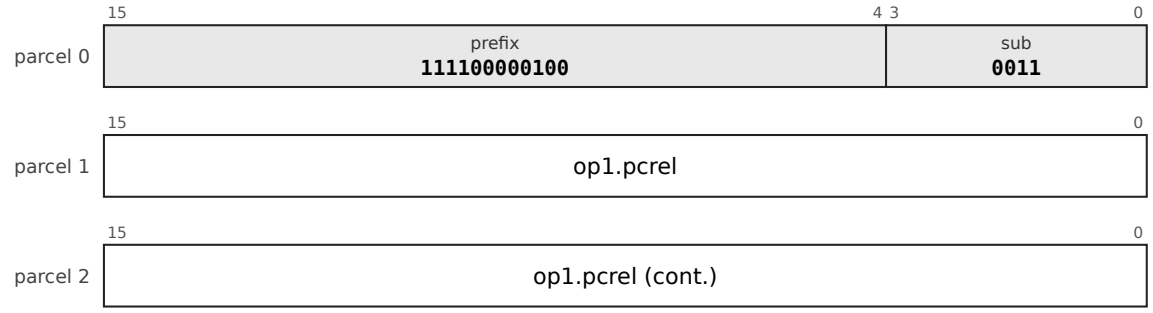
**DEC: SOF.B28, prefix 11101110111, sub-opcode 0011; operands b5d28s-x1; 6 bytes**



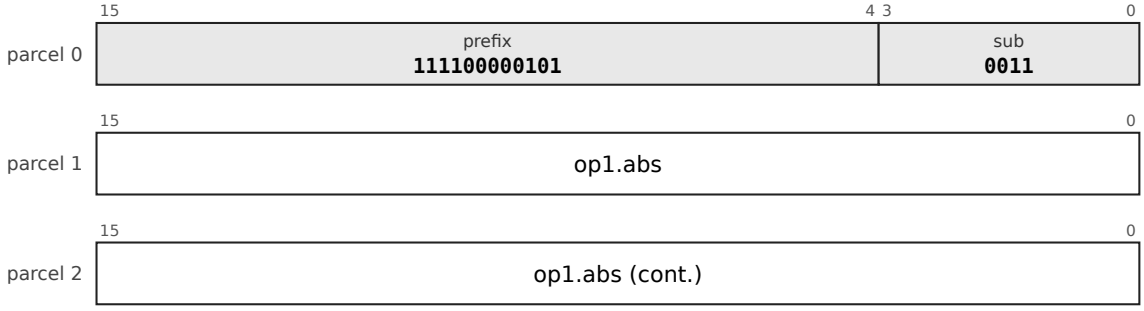

**DEC: SOF.X22, prefix 1110101101, sub-opcode 0011; operands x5-5d22s; 6 bytes**



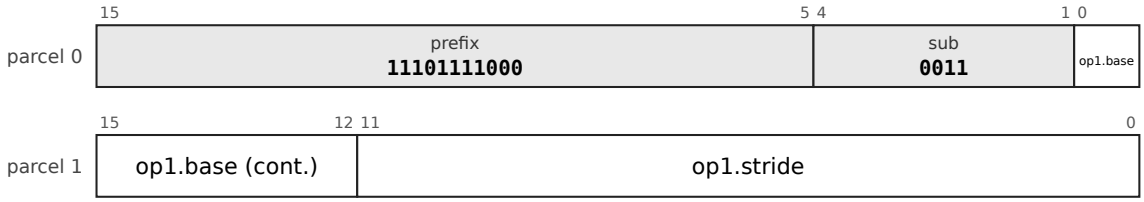

**DEC: SOF.PC32, prefix 111100000100, sub-opcode 0011; operands pc32-u1; 6 bytes**



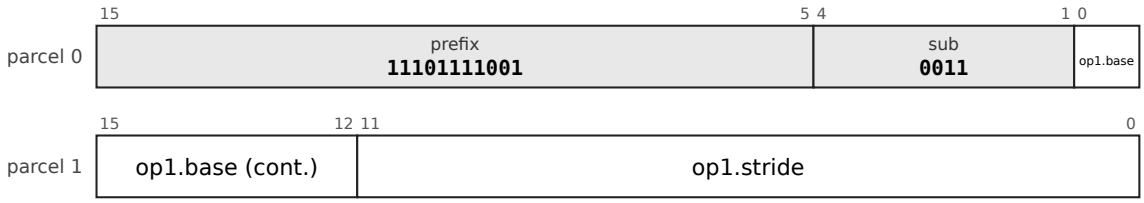
**DEC: SOF.A32, prefix 111100000101, sub-opcode 0011; operands abs32; 6 bytes**



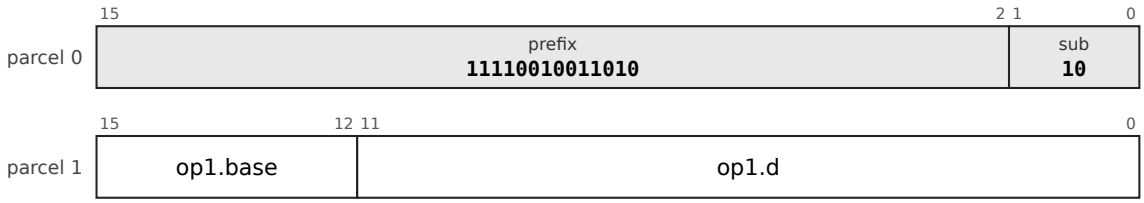

**DEC: SOF.PI12, prefix 11101111000, sub-opcode 0011; operands pi5-s12; 4 bytes**



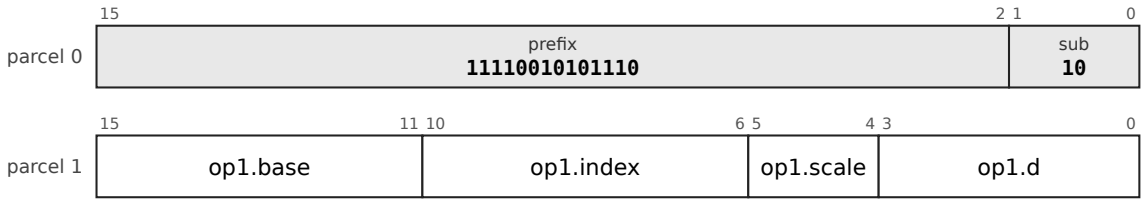

**DEC: SOF.PD12, prefix 11101111001, sub-opcode 0011; operands pd5-s12; 4 bytes**



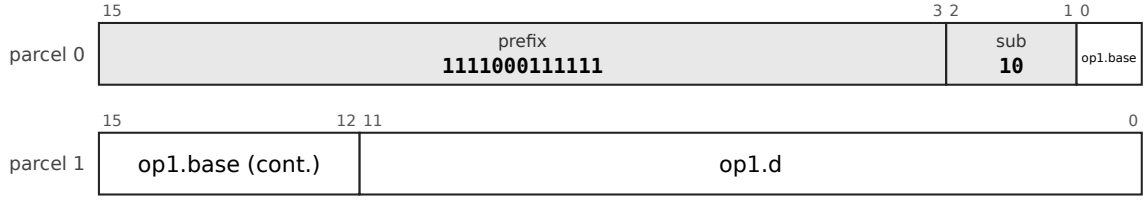

**DEC: SOBu12H/a, prefix 11110010011010, sub-opcode 10; operands b4d12u-xsz; 4 bytes**



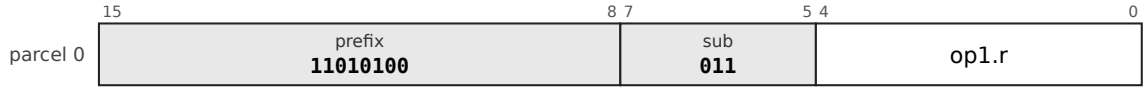

**DEC: SOX4/a, prefix 11110010101110, sub-opcode 10; operands x5-5d4s; 4 bytes**



**DEC: SOBs12/a, prefix 1111000111111, sub-opcode 10; operands b5d12s-xsz; 4 bytes**




**DEC: SOPR, prefix 11010100, sub-opcode 011; operands reg5; 2 bytes**

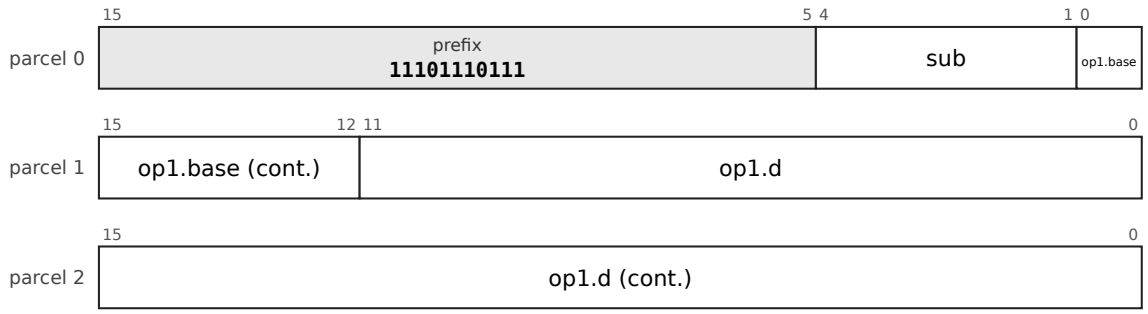



**Mnemonics:** `DEC`

### Syntax

```
DEC rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`, `SOBu12H-a-1`, `SOX4-a-1`, `SOBs12-a-1`.

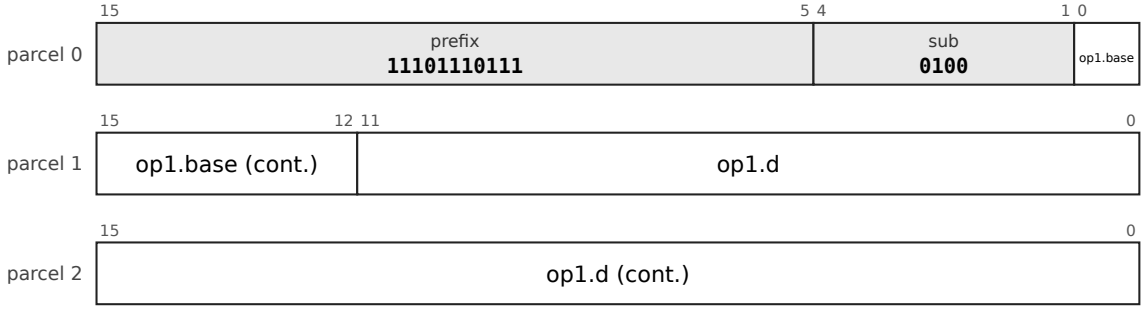
### Operation

```
rm ← rm - 1
```

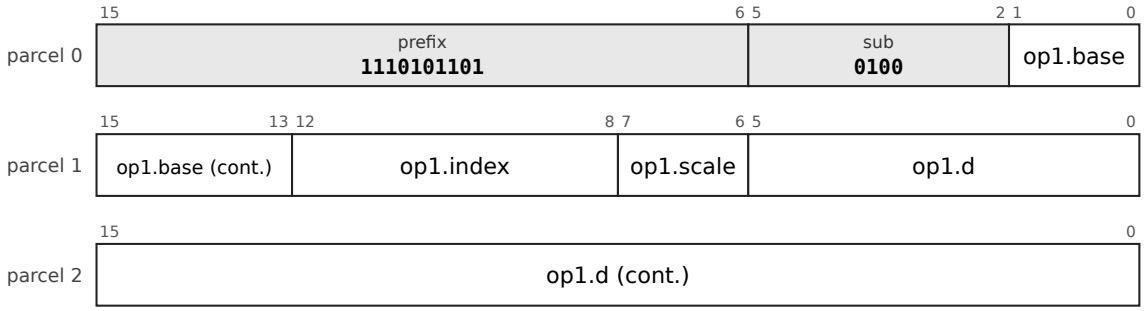
**Flags** `N`, `Z` and `V` as for `SUB`; `C` **unchanged**.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## NEG: Negate

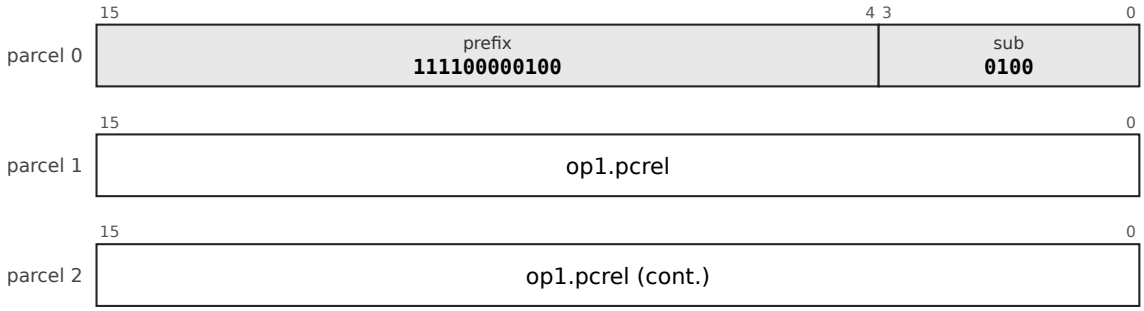
**NEG: SOF.B28, prefix 11101110111, sub-opcode 0100; operands b5d28s-x1; 6 bytes**



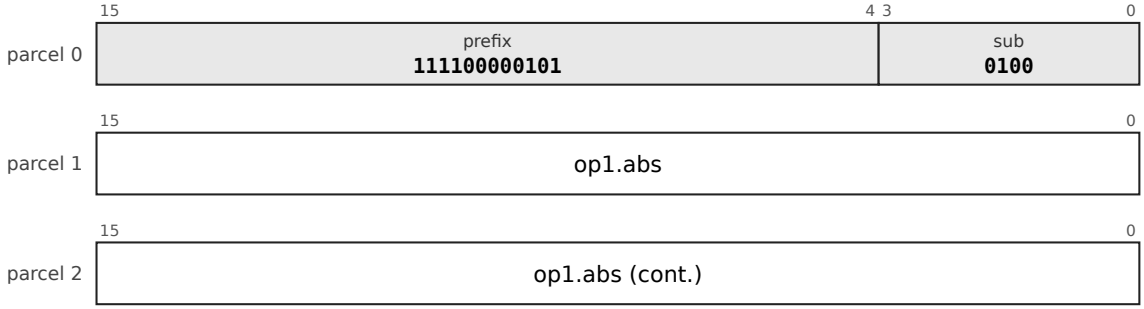

**NEG: SOF.X22, prefix 1110101101, sub-opcode 0100; operands x5-5d22s; 6 bytes**



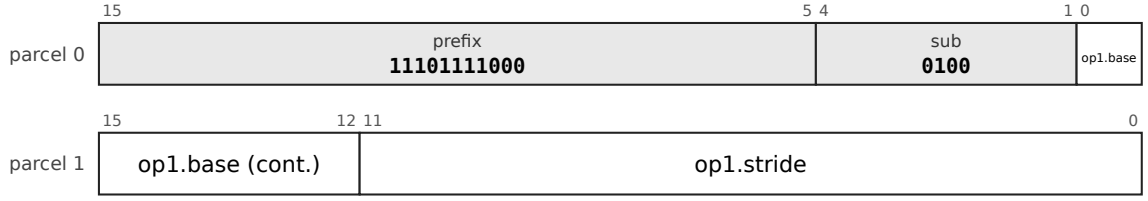

**NEG: SOF.PC32, prefix 111100000100, sub-opcode 0100; operands pc32-u1; 6 bytes**



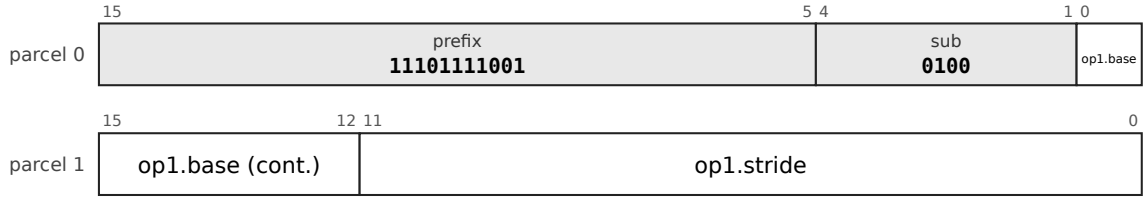

**NEG: SOF.A32, prefix 111100000101, sub-opcode 0100; operands abs32; 6 bytes**



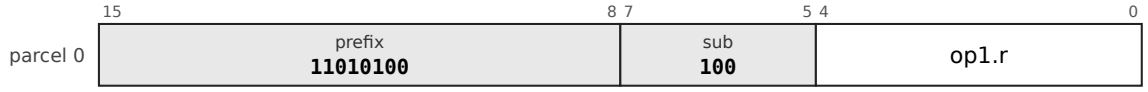
**NEG: SOF.PI12, prefix 11101111000, sub-opcode 0100; operands pi5-s12; 4 bytes**




**NEG: SOF.PD12, prefix 11101111001, sub-opcode 0100; operands pd5-s12; 4 bytes**




**NEG: SOPR, prefix 11010100, sub-opcode 100; operands reg5; 2 bytes**

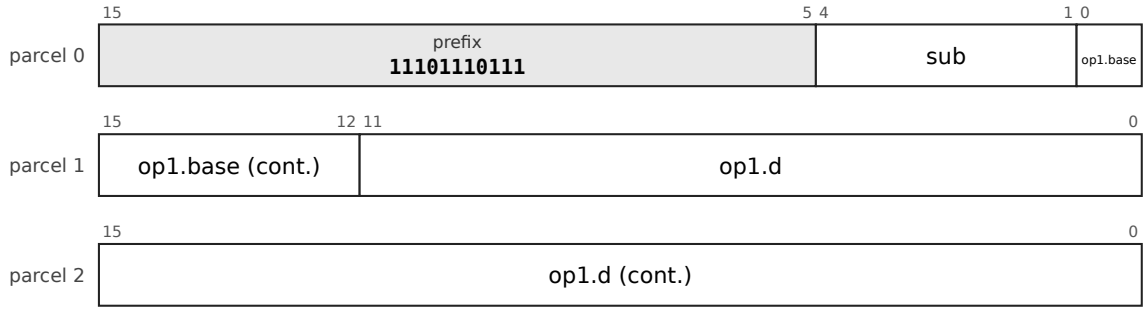



**Mnemonics:** `NEG`

### Syntax

```
NEG rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`.

### Operation

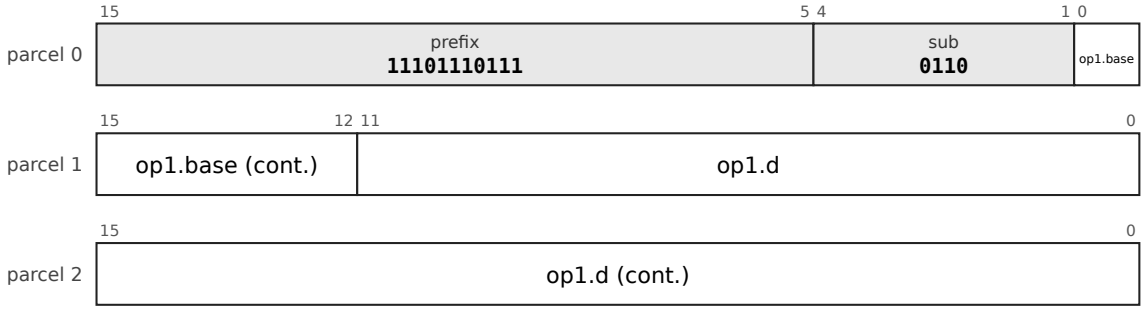
```
rm ← 0 - rm
```

**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger. (`C` = 1 unless the operand was 0.)

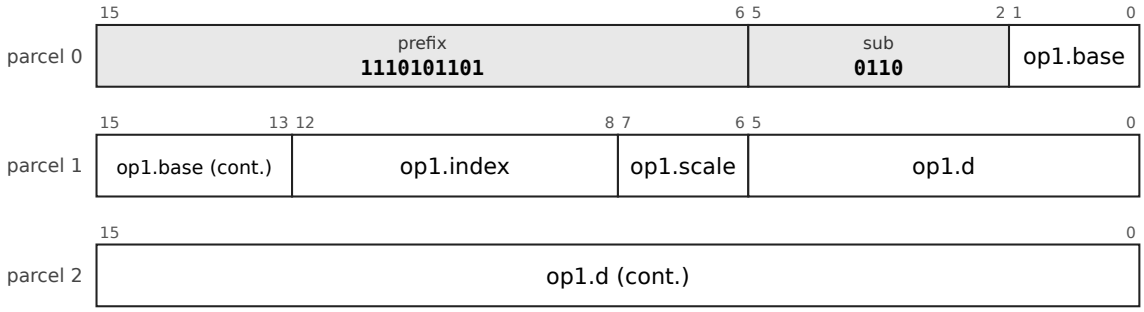
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## ADC: Add carry

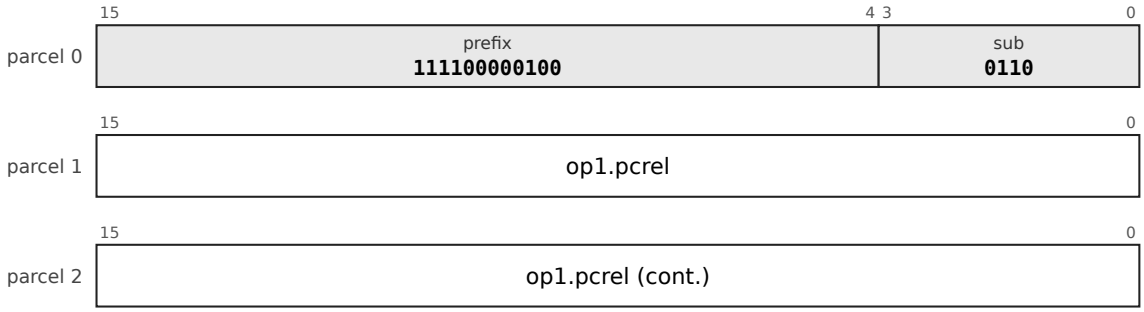
**ADC: SOF.B28, prefix 11101110111, sub-opcode 0110; operands b5d28s-x1; 6 bytes**



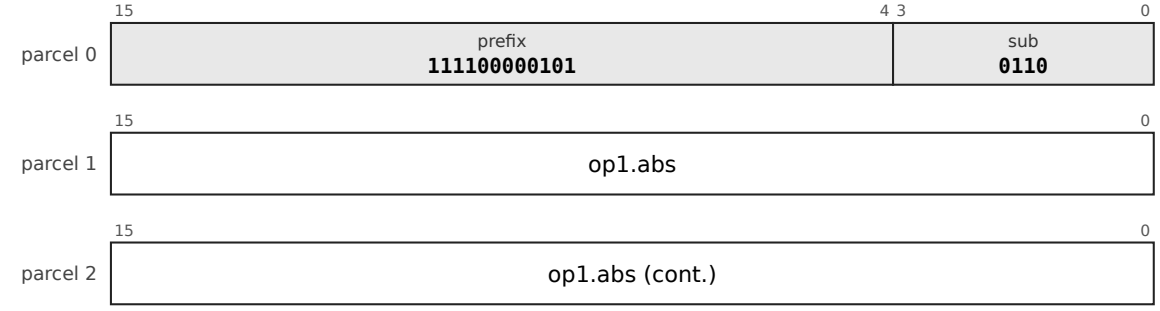

**ADC: SOF.X22, prefix 1110101101, sub-opcode 0110; operands x5-5d22s; 6 bytes**



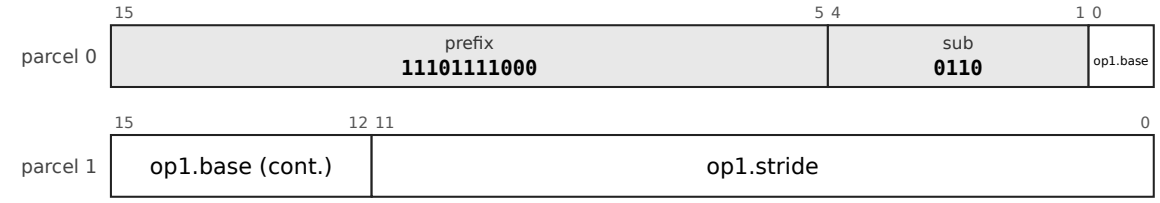

**ADC: SOF.PC32, prefix 111100000100, sub-opcode 0110; operands pc32-u1; 6 bytes**



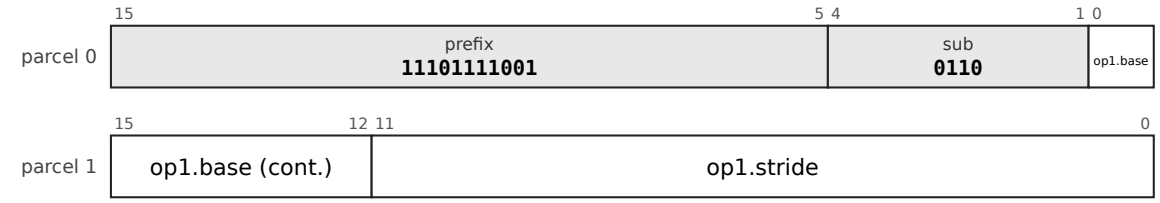
**ADC: SOF.A32, prefix 111100000101, sub-opcode 0110; operands abs32; 6 bytes**



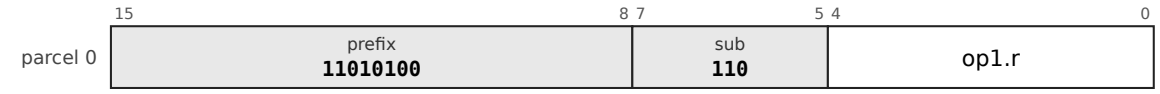

**ADC: SOF.PI12, prefix 11101111000, sub-opcode 0110; operands pi5-s12; 4 bytes**




**ADC: SOF.PD12, prefix 11101111001, sub-opcode 0110; operands pd5-s12; 4 bytes**




**ADC: SOPR, prefix 11010100, sub-opcode 110; operands reg5; 2 bytes**

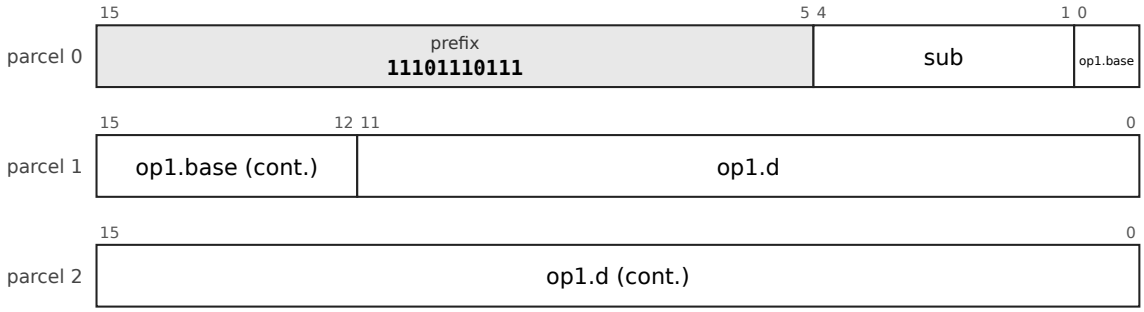



### Mnemonics: ADC

### Syntax

```
ADC rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`.

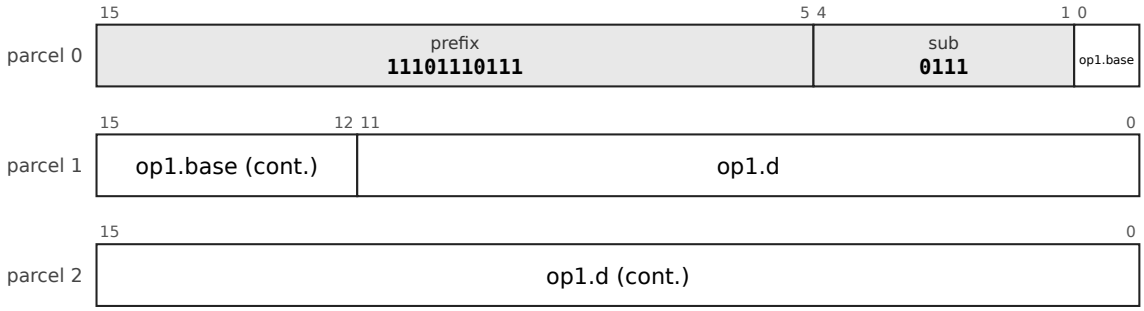
### Operation

```
rm ← rm + C
```

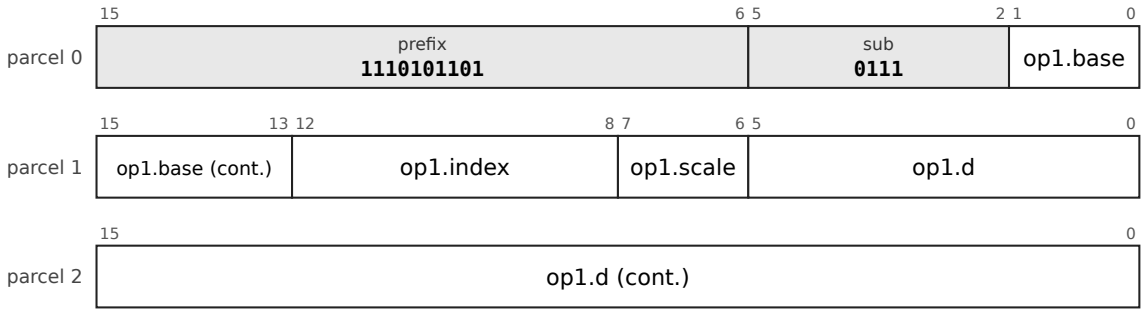
**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a carry out of bit 63.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
With `ADD`, multi-word addition.

## SBC: Subtract carry

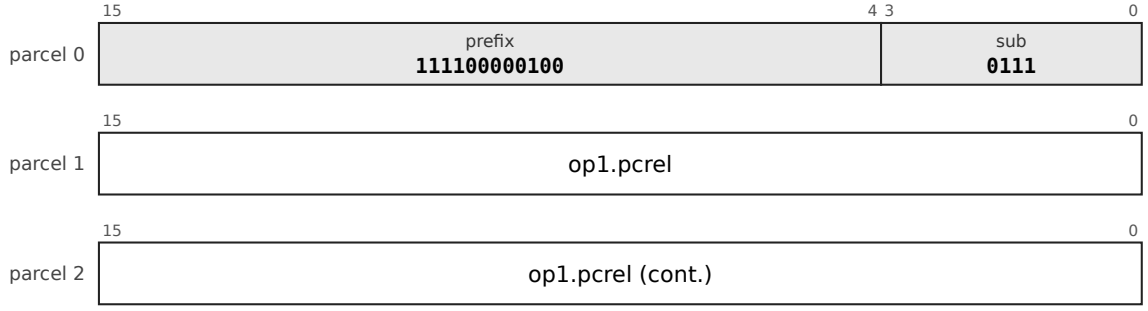
**SBC: SOF.B28, prefix 11101110111, sub-opcode 0111; operands b5d28s-x1; 6 bytes**



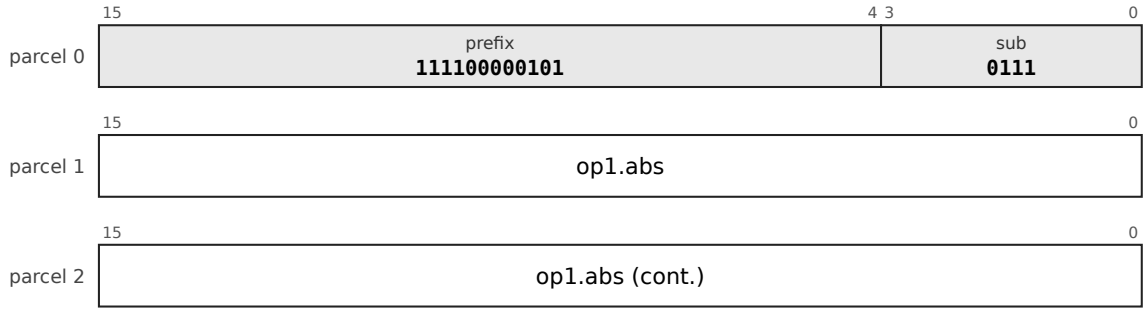

**SBC: SOF.X22, prefix 1110101101, sub-opcode 0111; operands x5-5d22s; 6 bytes**



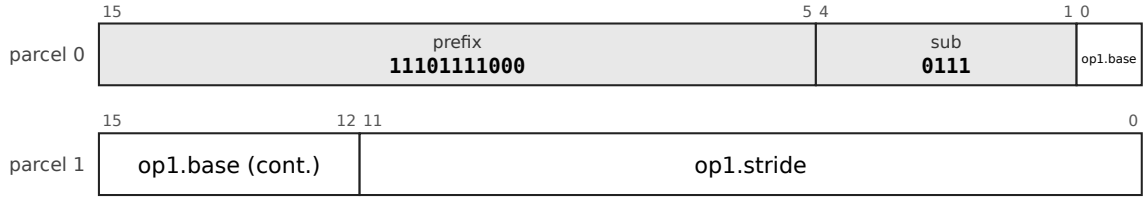
**SBC: SOF.PC32, prefix 111100000100, sub-opcode 0111; operands pc32-u1; 6 bytes**



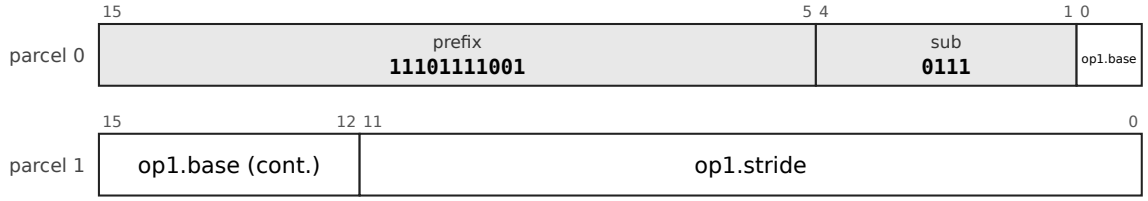

**SBC: SOF.A32, prefix 111100000101, sub-opcode 0111; operands abs32; 6 bytes**



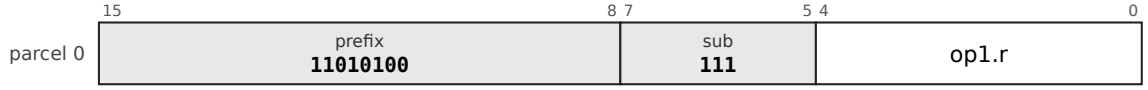

**SBC: SOF.PI12, prefix 11101111000, sub-opcode 0111; operands pi5-s12; 4 bytes**




**SBC: SOF.PD12, prefix 11101111001, sub-opcode 0111; operands pd5-s12; 4 bytes**




**SBC: SOPR, prefix 11010100, sub-opcode 111; operands reg5; 2 bytes**

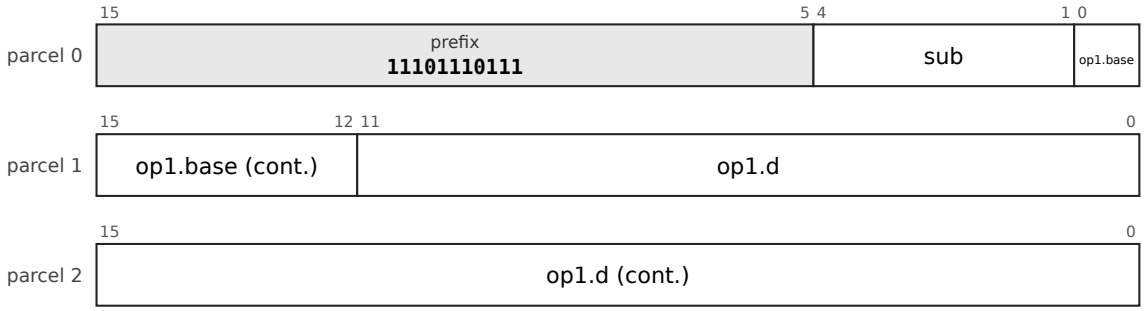



**Mnemonics:** SBC

### Syntax

```
SBC rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`.

### Operation

```
rm ← rm - C
```

**Flags** `N` and `Z` from the result, `V` set on signed overflow, `C` set on a **borrow** (the PDP-11 sense): `C` = 1 when the unsigned operand being subtracted is the larger.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

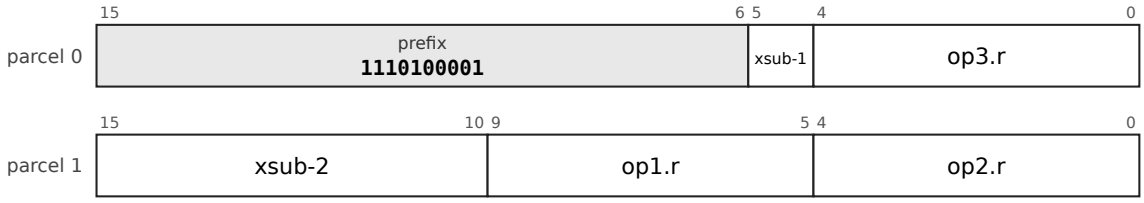
## DIV: Divide and remainder, signed and unsigned

**Mnemonics:** `DIV`, `DIVU`, `MOD`, `MODU`

### Syntax

```
DIV Ra, Rb, Rd
DIVU Ra, Rb, Rd
MOD Ra, Rb, Rd
MODU Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
if Rb = 0 then divide-by-zero abort
DIV : Rd ← Ra ÷ Rb  (signed, rounded toward zero; INT_MIN ÷ -1 = INT_MIN)
DIVU: Rd ← Ra ÷ Rb  (unsigned)
MOD : Rd ← Ra mod Rb (signed, the sign of Ra; any value mod -1 = 0)
MODU: Rd ← Ra mod Rb (unsigned)
```

**Flags** None written.

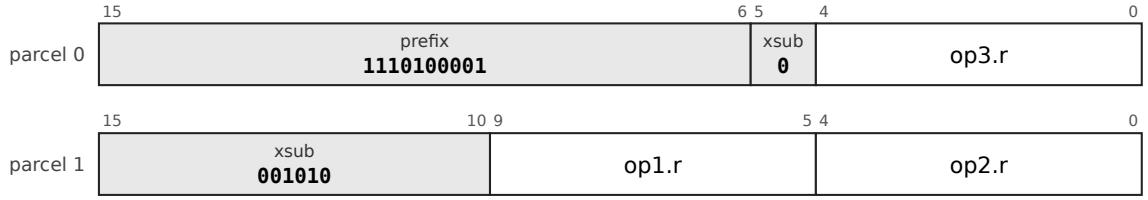
**Exceptions** Division by zero aborts with the divide-by-zero vector (cause `DIVZERO`). There is no overflow trap: `INT_MIN / -1` returns `INT_MIN`.

Division by a constant is a `MULH` and a shift, so a real divide is rare; this is why divide has no two-byte form.

**Cf.** 6.3 (the divide-by-zero vector).

***DIV***

**DIV: R3, prefix 1110100001, sub-opcode 0001010; operands reg5, reg5, reg5; 4 bytes**

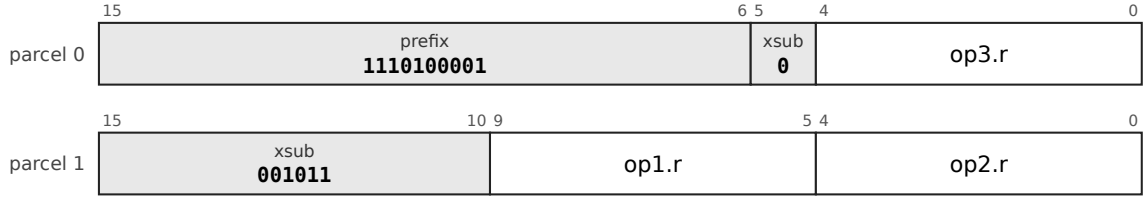



```
DIV Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

***DIVU***

**DIVU: R3, prefix 1110100001, sub-opcode 0001011; operands reg5, reg5, reg5; 4 bytes**

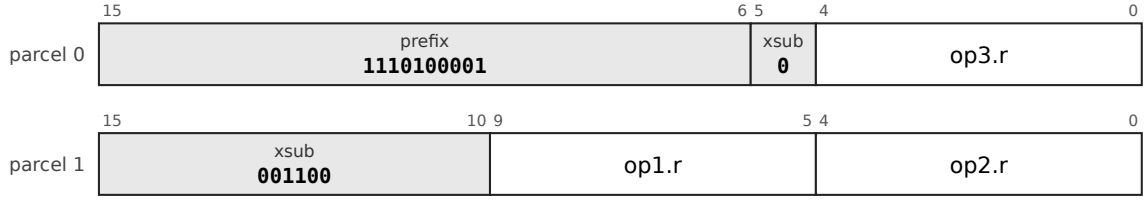



```
DIVU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

***MOD***

**MOD: R3, prefix 1110100001, sub-opcode 0001100; operands reg5, reg5, reg5; 4 bytes**

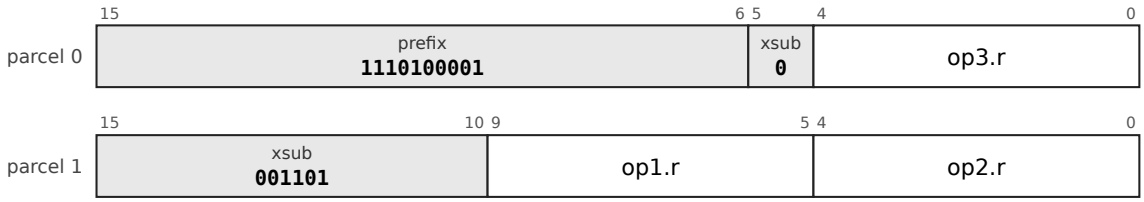



```
MOD Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

### *MODU*

**MODU: R3, prefix 1110100001, sub-opcode 0001101; operands reg5, reg5, reg5; 4 bytes**

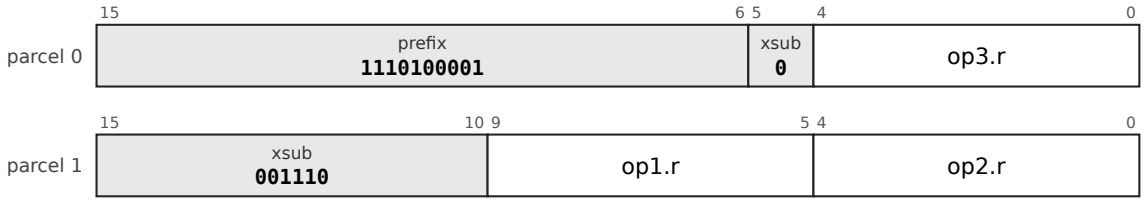



```
MODU Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry DIV above; 1 layout form, each a chart of its own (above).

## MULHU: Multiply, high 64 bits, unsigned

**MULHU: R3, prefix 1110100001, sub-opcode 0001110; operands reg5, reg5, reg5; 4 bytes**

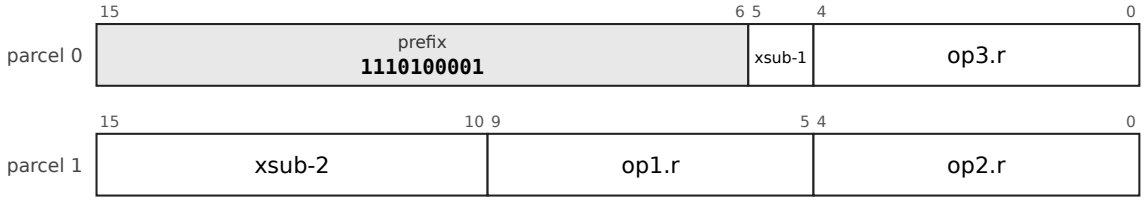



**Mnemonics:** `MULHU`

### Syntax

```
MULHU Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
Rd ← (Ra × Rb)[127:64]    ; unsigned
```

**Flags** None written.

**Exceptions** None.

**Why it is here.** The unsigned partner of `MULH`, for the same reason (the high half of the product). `MULHSU`, signed times unsigned for a bignum's top limb, is reserved because it can be built from `MULH` with one correction.

## ABS: Absolute value (assembler alias)

**Mnemonics:** `ABS`

**Alias** *(D)* `ABS` is an assembler alias, not a native instruction (the author's decision, errata 98). Meaning: `Rd <- |Rs|`. Open: its expansion, and with it the flags it leaves (J.9.1).

**Syntax**

```
ABS Rs, Rd
```

**Encoding** None of its own: the assembler expands an alias into other instructions. Open: the expansion (J.9.1).

**Operation**

```
Rd ← |Rs|
```

**Flags** Open with the expansion (J.9.1).

**Exceptions** None.

**Why it is here.** Integer absolute value is the counterpart of `FABS`. The author decided (2026-11-10) that it is an assembler alias, not a native instruction (errata 98); the compiler's work of recognising `x < 0 ? -x : x` is the real cost, and the expansion is open (J.9.1). The compiled corpus has 18 uses of `abs` and 5 of its unsigned form, under 0.1% of code, so the case is clarity and the idiom more than size.

## MINMAX: Minimum and maximum, signed and unsigned (assembler aliases)

**Mnemonics:** `MIN`, `MINU`, `MAX`, `MAXU`

**Alias** *(D)* `MIN`, `MINU`, `MAX` and `MAXU` are assembler aliases, not native instructions (the author's decision, errata 98). Meaning: the smaller or the larger of `Ra` and `Rb`, signed for `MIN` and `MAX` and unsigned for `MINU` and `MAXU`. Open: their expansion, and with it the flags they leave (J.9.1).

**Syntax**

```
MIN Ra, Rb, Rd
MINU Ra, Rb, Rd
MAX Ra, Rb, Rd
MAXU Ra, Rb, Rd
```

**Encoding** None of its own: the assembler expands an alias into other instructions. Open: the expansion (J.9.1).

**Operation**

```
MIN : Rd ← smaller of Ra, Rb (signed)        MINU: unsigned
MAX : Rd ← larger of Ra, Rb (signed)         MAXU: unsigned
```

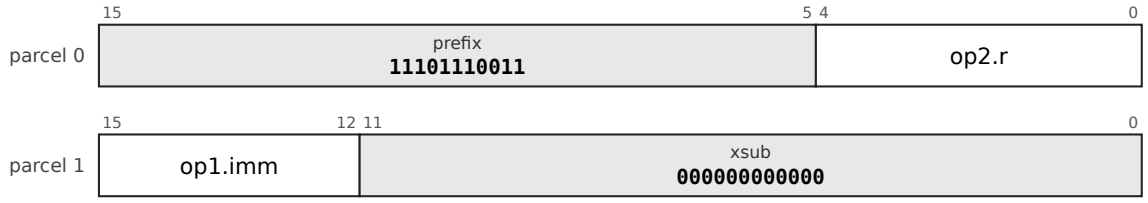
**Flags** Open with the expansion (J.9.1).

**Exceptions** None.

**Why it is here.** The integer counterparts of `FMIN` and `FMAX`, provided with `ABS`; the author decided (2026-11-10) that they are assembler aliases, not native instructions (errata 98). The corpus has 171 uses of `MIN` and 52 of `MAX` across 95 files, under 0.1% of code in all. Their expansion is open (J.9.1).

## SET: Set a register from a condition

**SET: SETF, prefix 11101110011, sub-opcode 000000000000; operands imm4u, reg5; 4 bytes**

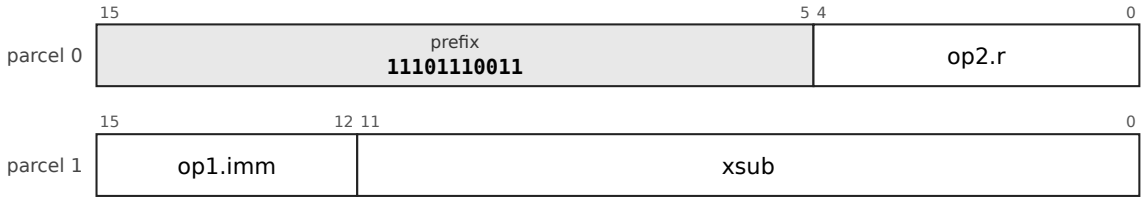



**Mnemonics:** `SET`

### Syntax

```
SET cc, Rd        ; for example SET EQ, R1
```

### Encoding




The chart is the leaf `SETF-1`, SETF, prefix `11101110011`. `op1` is `imm4u`: unsigned 4-bit immediate, 0 to 15. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SETF-1`.

### Operation

```
Rd ← 1 if condition cc holds on the current flags, else 0
```

**Flags** None written.

**Exceptions** None.

The condition is numbered as the low nibble of the branch opcodes (`al` 0, `ne` 1, `eq` 2, ... `los` 14).

**Why it is here.** The first instruction written with a condition word: one instruction, `SET cc, Rd`, rather than one opcode per condition, with the condition numbered like the low nibble of the branch opcodes. It turns a comparison used as a value into 0 or 1 without a branch. gcc `-Os` for amd64 emits 673 `set<cc>` on the corpus, and `SET` would remove about six bytes per site, around 0.4% of code; the compiler's first pass turns such a comparison into branches before its second pass sees it, so the saving is not reached in compiled code.

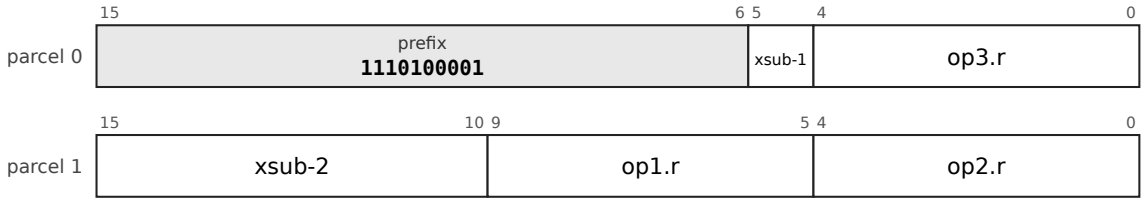
## MASK: Mask by a condition register

**Mnemonics:** `MASKEQZ`, `MASKNEZ`

### Syntax

```
MASKEQZ Ra, Rc, Rd
MASKNEZ Ra, Rc, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

**Operation**

```
MASKEQZ: Rd ← (Rc = 0) ? 0 : Ra
MASKNEZ: Rd ← (Rc ≠ 0) ? 0 : Ra
```

**Flags** None written.

**Exceptions** None.

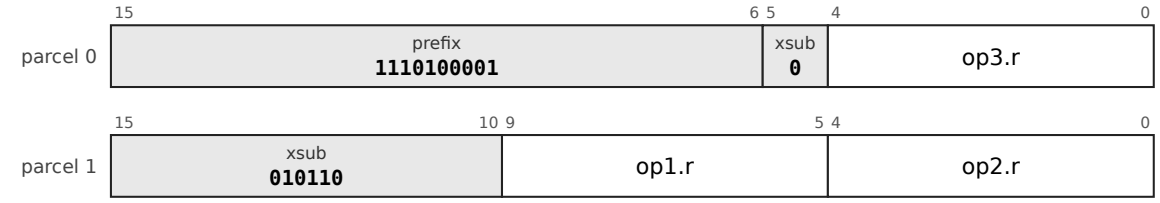
*(D)* Errata 85: the branch-free select is adopted. `Rc` is tested whole, all 64 bits, so any non-zero value is true; no flag is read or written, and the latency is fixed. `Rd` may be `Ra` or `Rc`. A constant-time select `d = c ? a : b` is `MASKEQZ a, c, t`, `MASKNEZ b, c, d`, `BIS t, d`. The semantics are those of LoongArch `maskeqz` and `masknez` and of RISC-V Zicond (the crypto study states this from memory). Open: whether a flag-reading select is added (J.9.1).

**Why it is here.** The author adopted the branch-free select (errata 85) when the crypto study asked for it: constant-time code must choose between two values without a branch, and a flag-reading conditional move would put a secret in a flag. `MASKEQZ` and `MASKNEZ` are the select of LoongArch and of RISC-V Zicond (stated from memory in the study, not checked against a manual); the pair and an `OR` make `c ? a : b` in three instructions, against two for `CMP` and a conditional move, but the conditional move would be used at 783 sites in 284,641 instructions (0.28%), and the case for the pair is the constant-time idiom, where the condition is already a register. The author confirmed the pair and wrote "looks good" on the section (errata 86). About 100 look-up tables, one cycle (the study's estimate).

**Cf.** 9.7 (SET), docs/crypto-secure-core.md section 2.

***MASKEQZ***

**MASKEQZ: R3, prefix 1110100001, sub-opcode 0010110; operands reg5, reg5, reg5; 4 bytes**

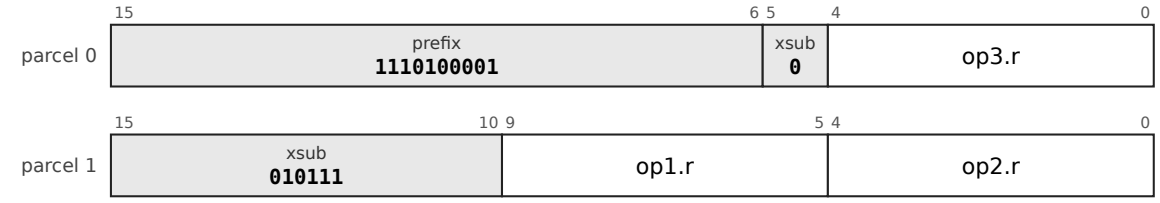



```
MASKEQZ Ra, Rc, Rd
```

Operation, flags and exceptions as for the entry MASK above; 1 layout form, each a chart of its own (above).

***MASKNEZ***

**MASKNEZ: R3, prefix 1110100001, sub-opcode 0010111; operands reg5, reg5, reg5; 4 bytes**




```
MASKNEZ Ra, Rc, Rd
```

Operation, flags and exceptions as for the entry MASK above; 1 layout form, each a chart of its own (above).

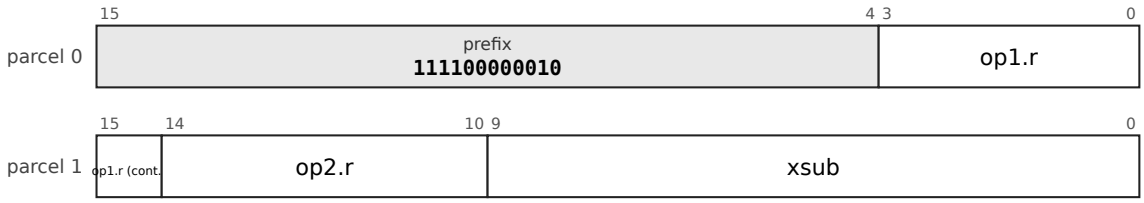
## OVF: Add, subtract and multiply, trapping on signed overflow

**Mnemonics:** `ADDV`, `SUBV`, `MULV`

### Syntax

```
ADDV Rs, Rd
SUBV Rs, Rd
MULV Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

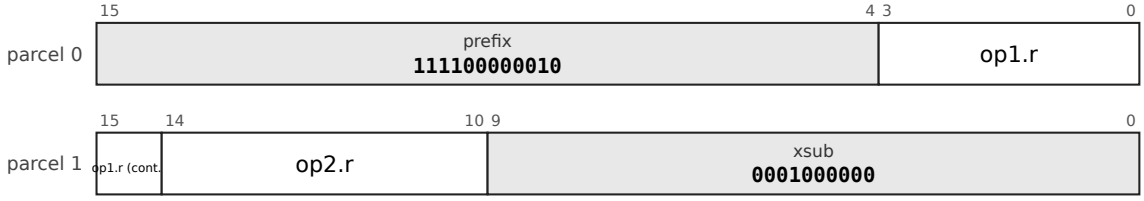
### Operation

```
t ← Rd op Rs                ; op is +, - or ×, on signed 64-bit values
if t does not fit in 64 bits then abort at this instruction (vector CHECK), Rd unwritten
else Rd ← t
```

**Flags** None written.
**Exceptions** Aborts with the CHECK vector on signed overflow, with EPC at the instruction and the destination unwritten; a blind restart faults again unless the handler changed an operand.
*(D)* Errata 85: TRAPCC with the fused ADDV, SUBV and MULV forms. The tables of the grammars hold the two-register form (`Rd <- Rd op Rs`, like ADD, SUB, MUL) and no immediate or memory form. Open: whether the form has two registers or three (J.9.4). Open: the number of the CHECK vector (J.9.4).
**Why it is here.** Checked arithmetic is what clang `-ftrapv` and `__builtin_*_overflow`, Rust's debug overflow checks, Swift's default arithmetic and Ada's `Constraint_Error` need. The author decided TRAPCC with the fused ADDV, SUBV and MULV and no compare-and-trap form (errata 85), the fused forms as the case where one non-destructive instruction beats ADD and TRAPCC. The corpus cannot size the use: it contains no BVS or BVC at all, because the overflow builtins were not implemented in the compiler that produced it, and the study says that sanitiser builds are the target. The record gives the precedent only for the trap, not for the fused forms.
**Cf.** 9.14 (TRAPCC), docs/study-proposals.md item 9.

### *ADDV*

**ADDV: R2, prefix 111100000010, sub-opcode 0001000000; operands reg5, reg5; 4 bytes**

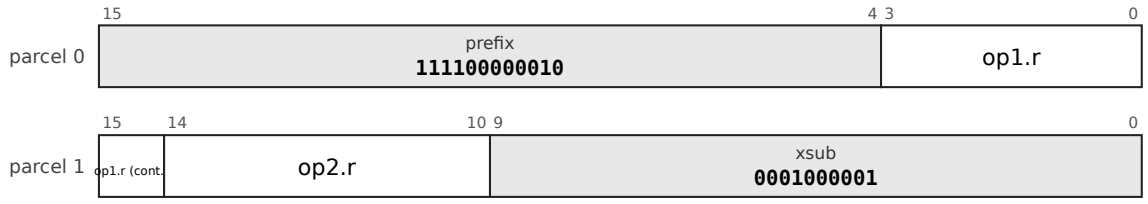



```
ADDV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

***SUBV***

**SUBV: R2, prefix 111100000010, sub-opcode 0001000001; operands reg5, reg5; 4 bytes**

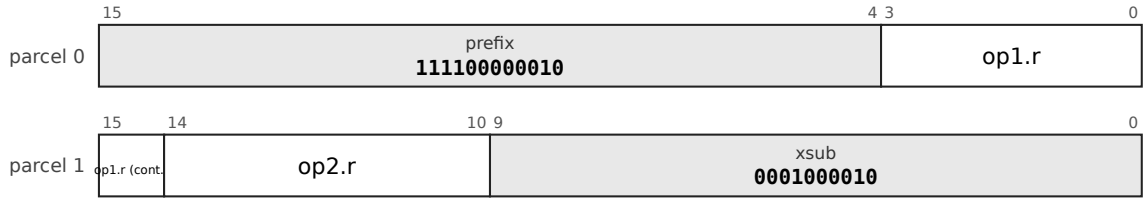



```
SUBV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

***MULV***

**MULV: R2, prefix 111100000010, sub-opcode 0001000010; operands reg5, reg5; 4 bytes**




```
MULV Rs, Rd
```

Operation, flags and exceptions as for the entry OVF above; 1 layout form, each a chart of its own (above).

## 9.8 Logic, shifts and rotates

**9.8.1** `AND`, `BIC` (bit clear), `BIS` (bit set, the or), `XOR` and `BIT` (test) are two-byte ALU operations; `COM` is one's complement.

**9.8.2** `SHL`, `SHR` and `SAR` shift by a count taken from `rm` (a register, an immediate, or memory); `ASL` and `ASR` are the by-one forms in two bytes. *(D)*

**9.8.3** A shift count of 64 or more shifts the value entirely away; `C` receives the last bit that left. *(D)*

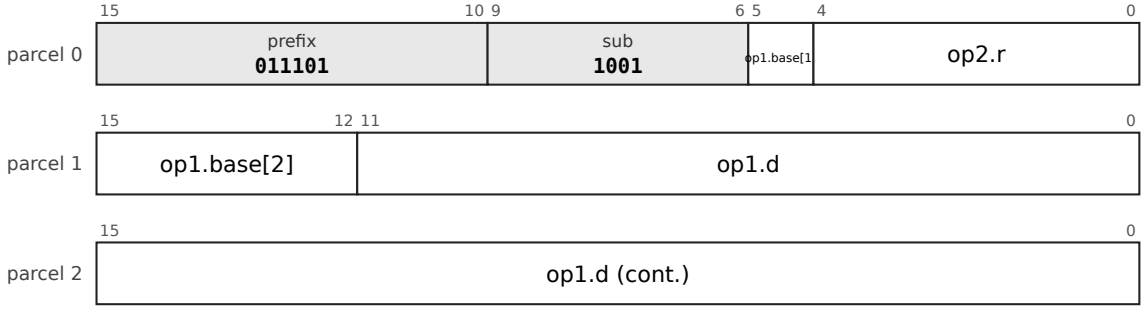
**9.8.4** `ROR` rotates right by a register count and `RORI` by an immediate; `ROL Rd, Rs, #n` assembles to `RORI` with `64 - n`; `RORI32` rotates the low 32 bits and sign-extends. Rotates write no flags. *(D)*

**9.8.5** `DSHR` is a double-width funnel shift. It writes no flags. *(D)*

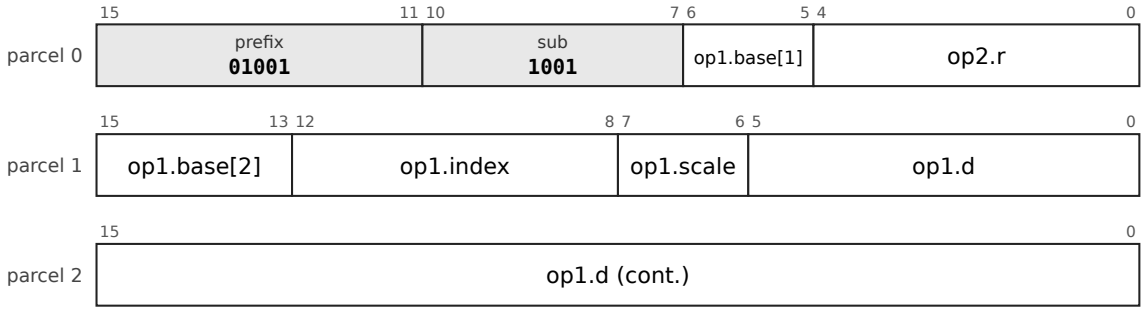
*Commentary.* **Precedent.** The PDP-11 had `ASL`, `ASR`, `ROL` and `ROR` by one through the carry; x86's `SHRD` and ARM's `EXTR` are the funnel shifts. **Trade-offs.** One rotate direction is enough, so there is no by-one `ROL` in a two-byte slot; rotates are not through the carry because a carry chain is fragile when the carry flag is shared. **Improvements.** The shift count comes from `rm`, not from a fixed register like x86's `CL`, which costs a register shuffle in every hand-written loop. **Benefits.** Multi-precision shifting is `DSHR`, two instructions for any distance. **Modern comparables.** RISC-V and AArch64 have rotates by register and by immediate; x86 has `ROL`/`ROR` through the carry as well. **Use.** `RORI` and `RORI32` are for hashes and ciphers, which rotate by constants; `DSHR` shifts a bignum. **Impact.** *Kernel:* Bit tests and mask operations on the status and page-table words are single instructions. *Compiler:* The shift count may be any register, immediate or memory operand, so the compiler never has to move a count into a fixed register; hashes and ciphers rotate by constants. *Hardware:* The barrel shifter (about 450 lookup tables) serves shifts and rotates, and writing no flag for rotates removes a result path.

## BIC: Bit clear

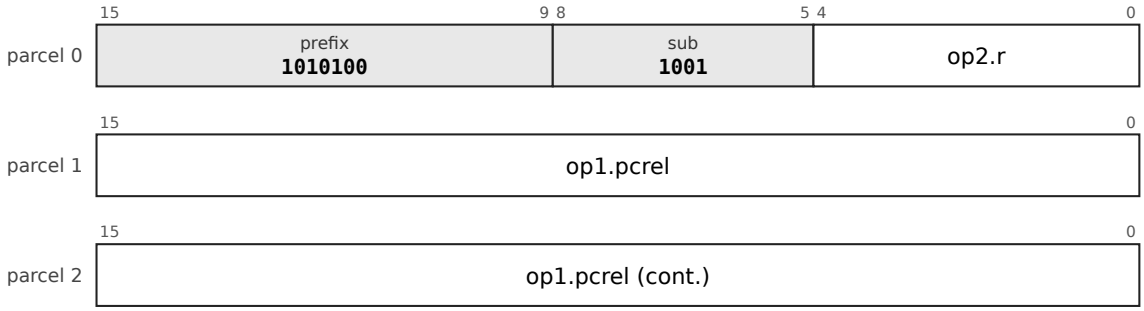
**BIC: LDF.B28, prefix 011101, sub-opcode 1001; operands b5d28s-x1, reg5; 6 bytes**



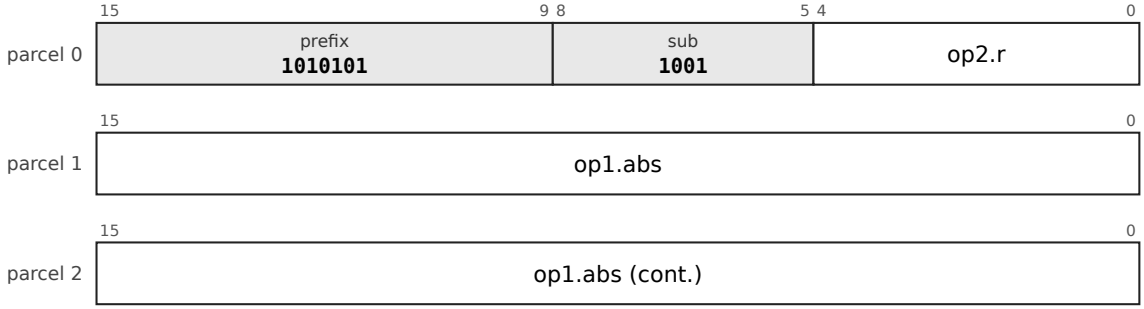

**BIC: LDF.X22, prefix 01001, sub-opcode 1001; operands x5-5d22s, reg5; 6 bytes**



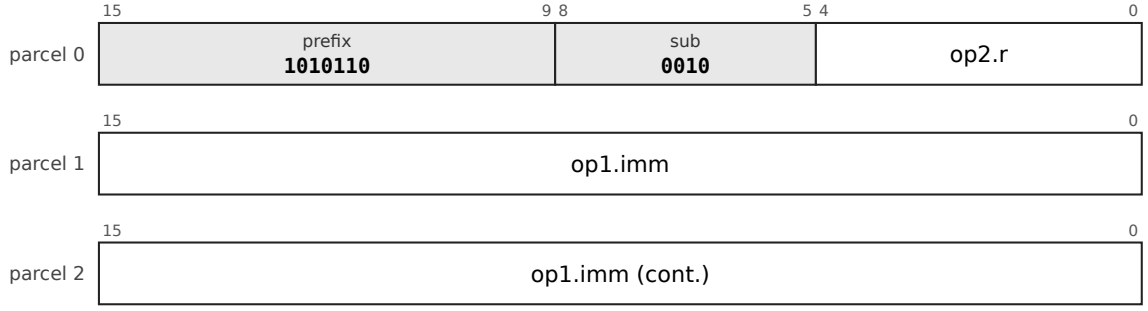

**BIC: LDF.PC32, prefix 1010100, sub-opcode 1001; operands pc32-u1, reg5; 6 bytes**



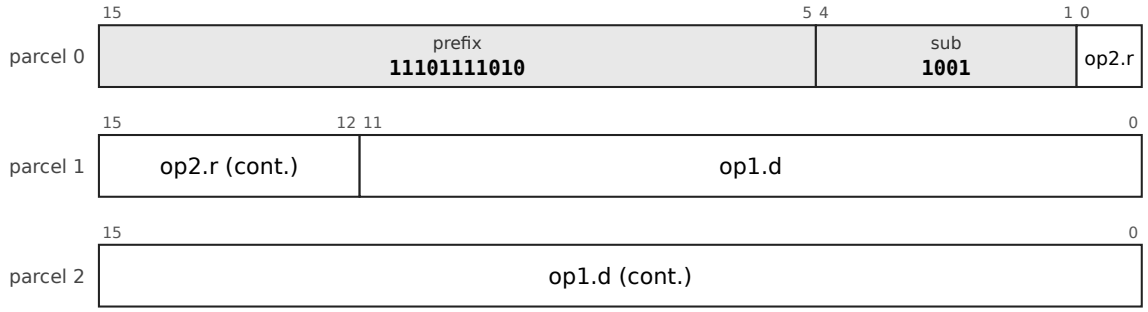

**BIC: LDF.A32, prefix 1010101, sub-opcode 1001; operands abs32, reg5; 6 bytes**



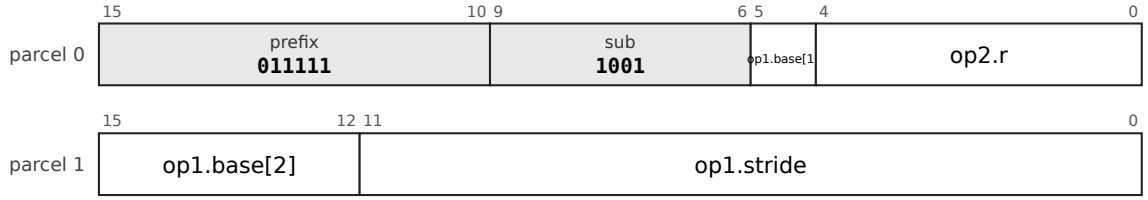
**BIC: LDFI.I32, prefix 1010110, sub-opcode 0010; operands imm32s, reg5; 6 bytes**



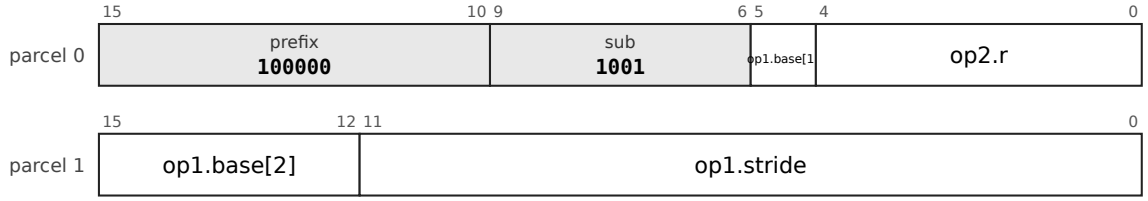

**BIC: DPF.D28, prefix 11101111010, sub-opcode 1001; operands dpd28s-x1, reg5; 6 bytes**



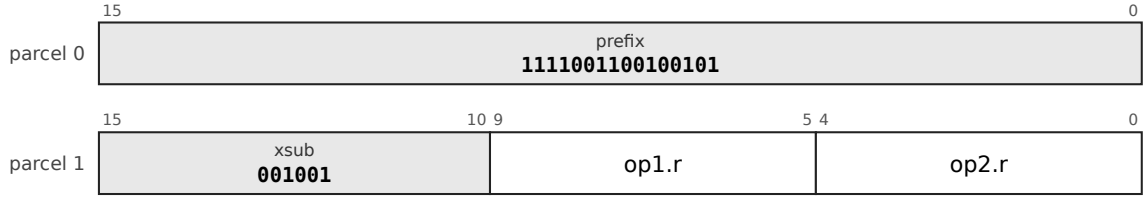

**BIC: LDF.PI12, prefix 011111, sub-opcode 1001; operands pi5-s12, reg5; 4 bytes**



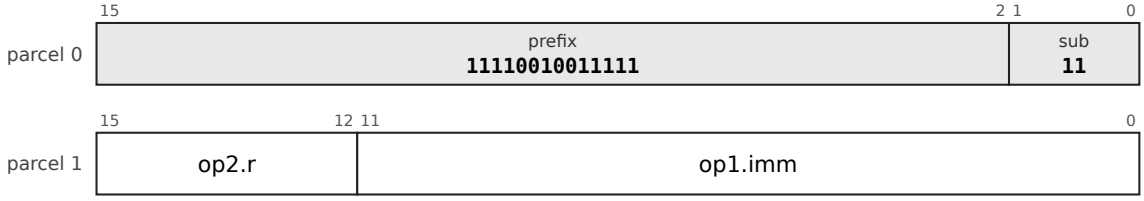

**BIC: LDF.PD12, prefix 100000, sub-opcode 1001; operands pd5-s12, reg5; 4 bytes**



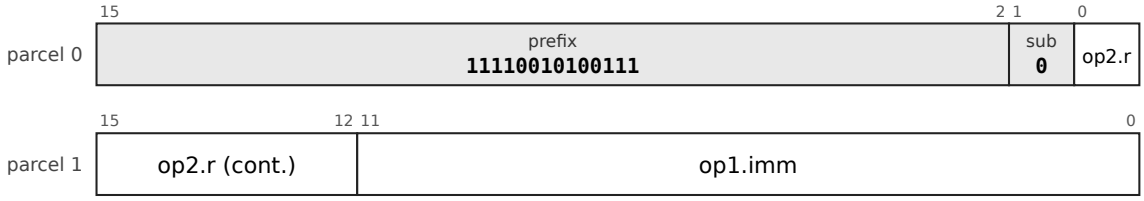

**BIC: RRF5, prefix 1111001100100101, sub-opcode 001001; operands reg5, reg5; 4 bytes**



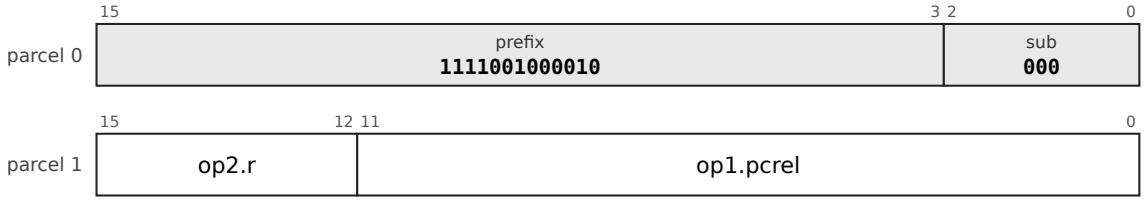
**BIC: LDI12H/c, prefix 11110010011111, sub-opcode 11; operands imm12s, reg4; 4 bytes**



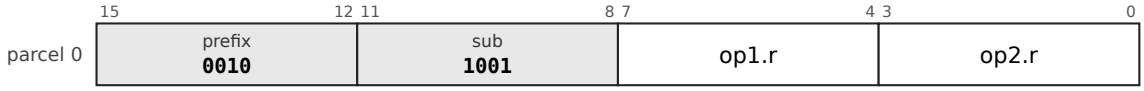

**BIC: LDI12/d, prefix 11110010100111, sub-opcode 0; operands imm12s, reg5; 4 bytes**




**BIC: LDPC12H/d, prefix 1111001000010, sub-opcode 000; operands pc12-u1, reg4; 4 bytes**




**BIC: RRF, prefix 0010, sub-opcode 1001; operands reg4, reg4; 2 bytes**

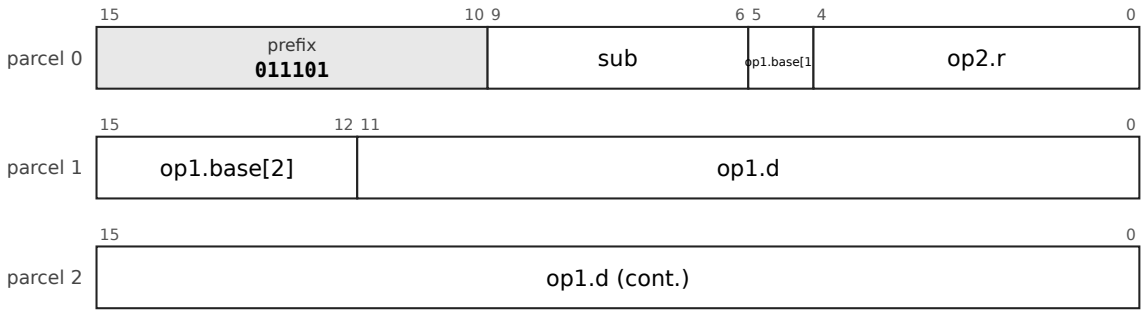



**Mnemonics:** `BIC`

## Syntax

```
BIC rm, Rd
```

## Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000,

`movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `LDI12H-c-1`, `LDI12-d-2`, `LDPC12H-d-1`.

### Operation

```
Rd ← Rd AND NOT rm
```

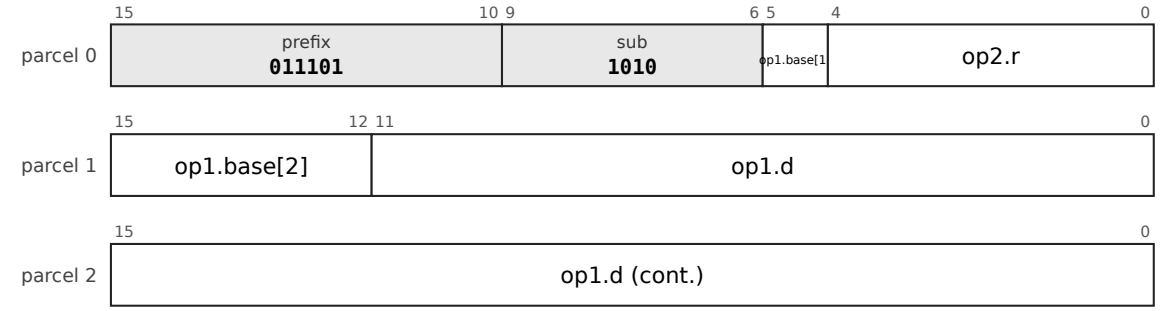
**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

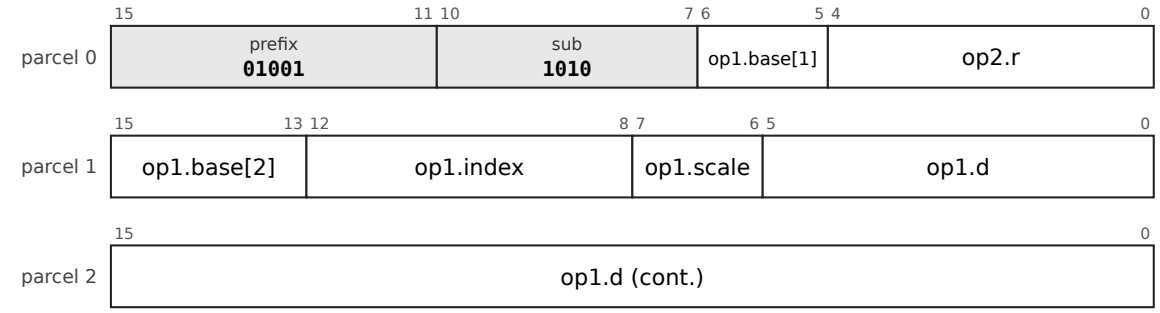
The assembler also accepts `BIC #imm, rm` as `AND #~imm, rm`.

## BIS: Bit set (inclusive or)

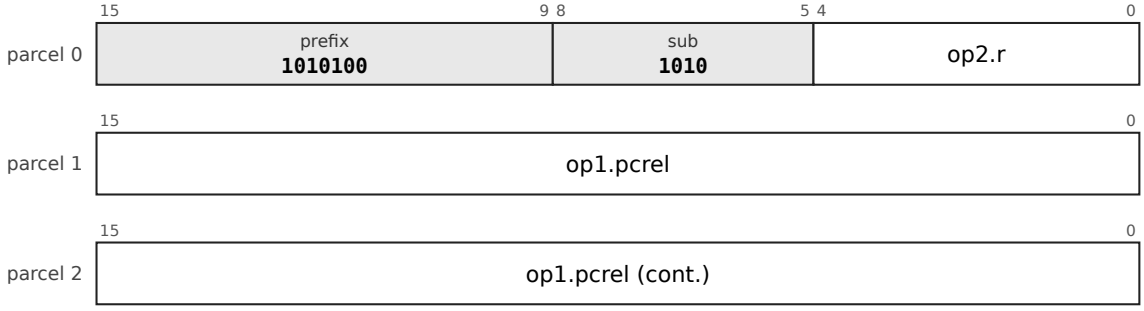
**BIS: LDF.B28, prefix 011101, sub-opcode 1010; operands b5d28s-x1, reg5; 6 bytes**



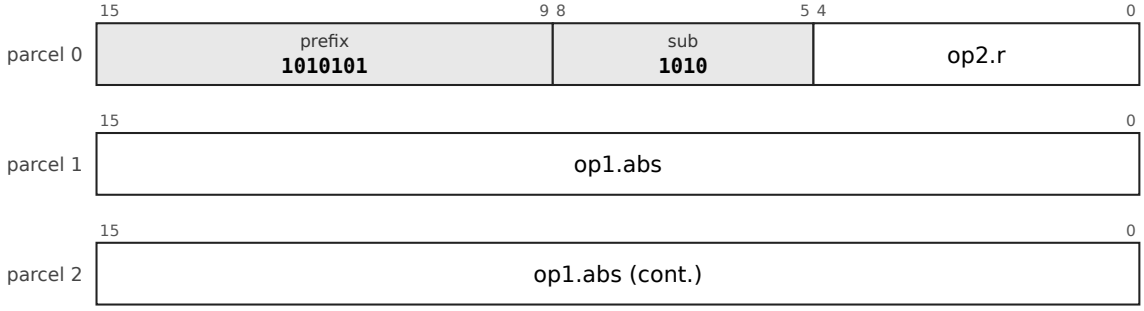

**BIS: LDF.X22, prefix 01001, sub-opcode 1010; operands x5-5d22s, reg5; 6 bytes**



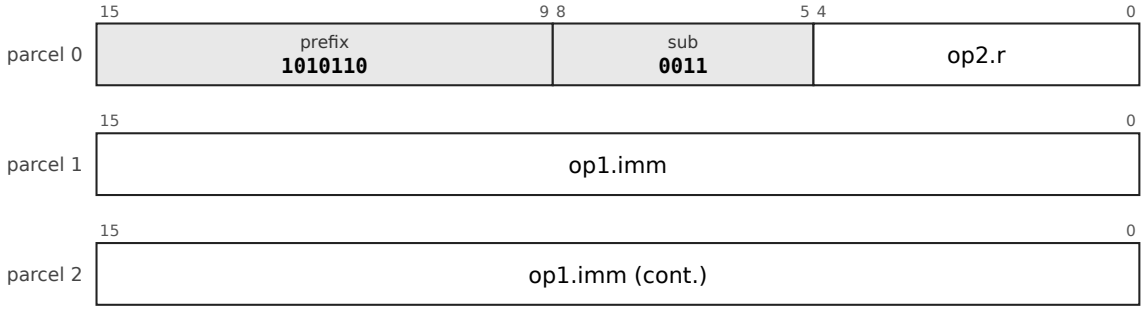
**BIS: LDF.PC32, prefix 1010100, sub-opcode 1010; operands pc32-u1, reg5; 6 bytes**



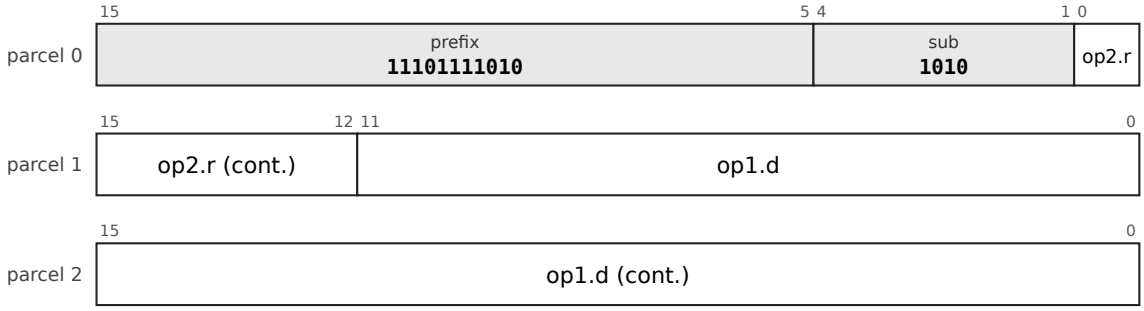

**BIS: LDF.A32, prefix 1010101, sub-opcode 1010; operands abs32, reg5; 6 bytes**



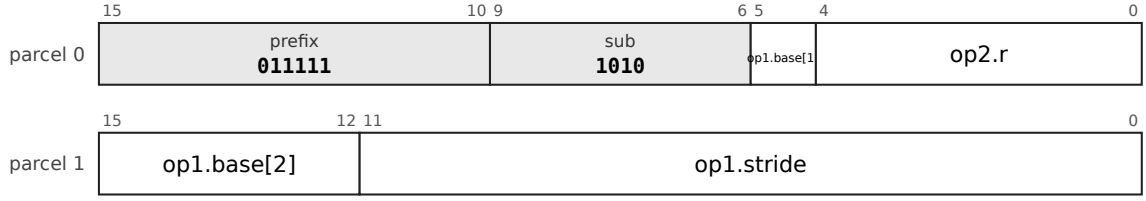

**BIS: LDFI.I32, prefix 1010110, sub-opcode 0011; operands imm32s, reg5; 6 bytes**



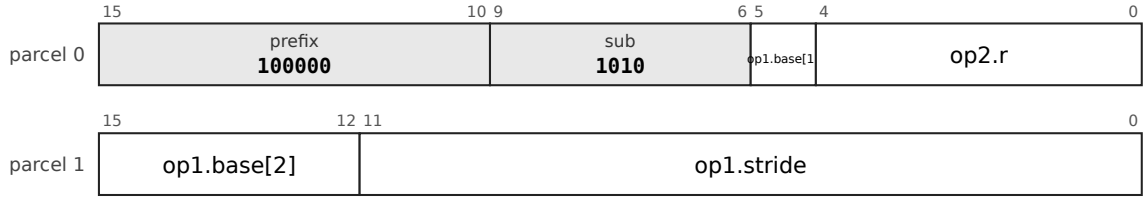

**BIS: DPF.D28, prefix 11101111010, sub-opcode 1010; operands dpd28s-x1, reg5; 6 bytes**



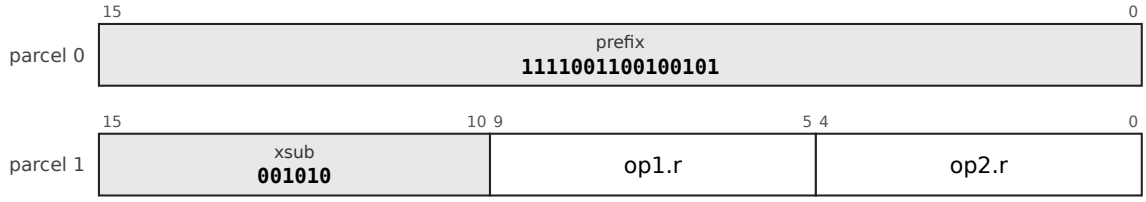
**BIS: LDF.PI12, prefix 011111, sub-opcode 1010; operands pi5-s12, reg5; 4 bytes**



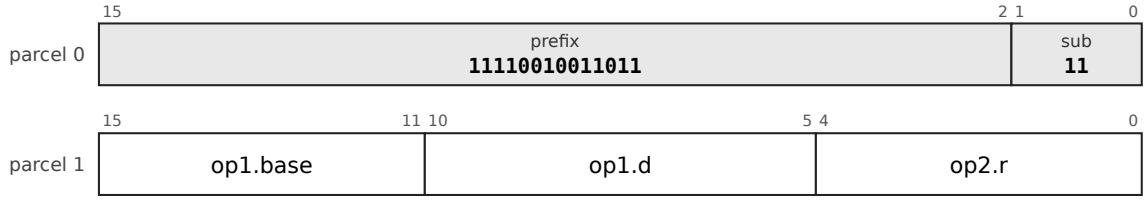

**BIS: LDF.PD12, prefix 100000, sub-opcode 1010; operands pd5-s12, reg5; 4 bytes**



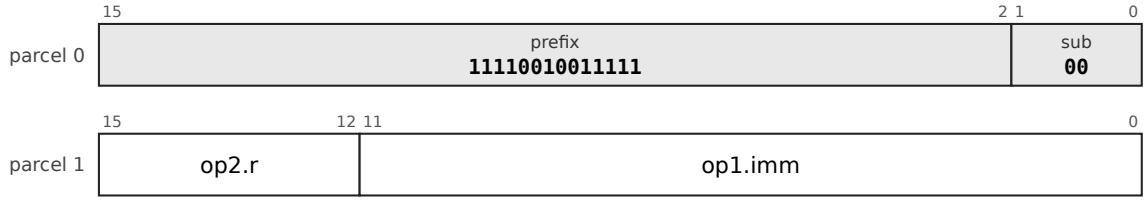

**BIS: RRF5, prefix 1111001100100101, sub-opcode 001010; operands reg5, reg5; 4 bytes**



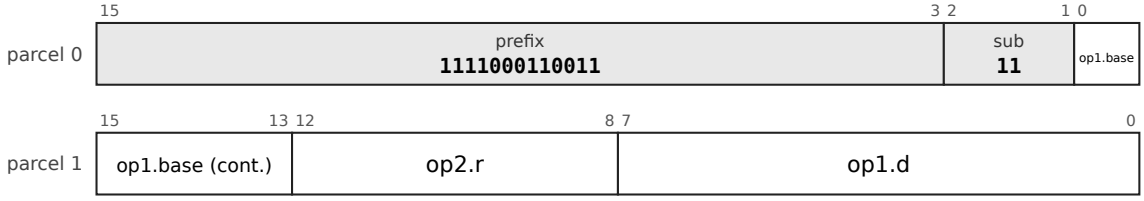

**BIS: LDBu6/c, prefix 11110010011011, sub-opcode 11; operands b5d6u-xsz, reg5; 4 bytes**



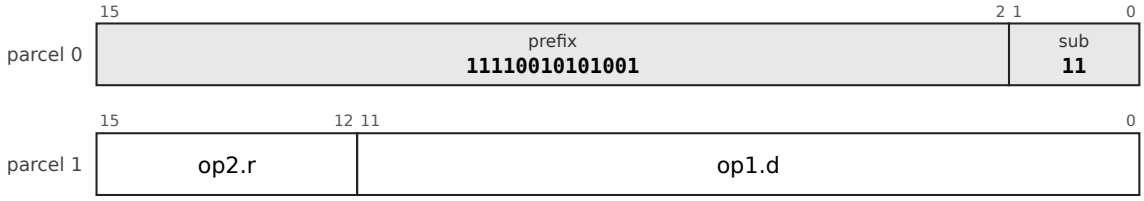

**BIS: LDI12H/c, prefix 11110010011111, sub-opcode 00; operands imm12s, reg4; 4 bytes**



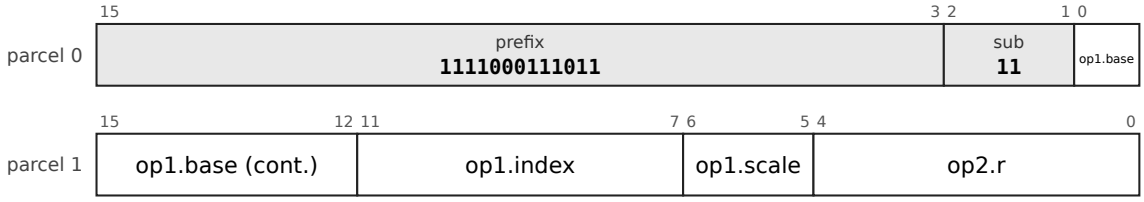
**BIS: LDBu8H/c, prefix 1111000110011, sub-opcode 11; operands b4d8u-xsz, reg5; 4 bytes**



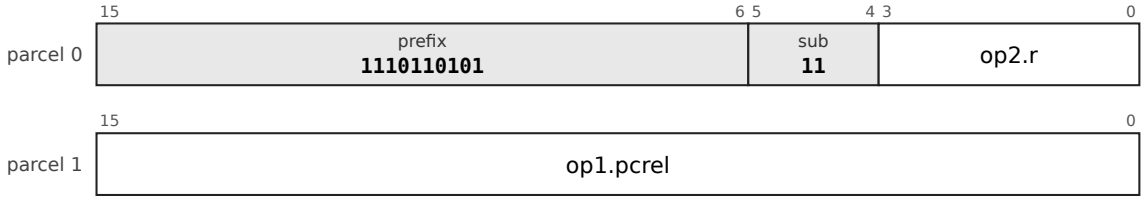

**BIS: LDS12H/c, prefix 11110010101001, sub-opcode 11; operands spd12u-xsz, reg4; 4 bytes**



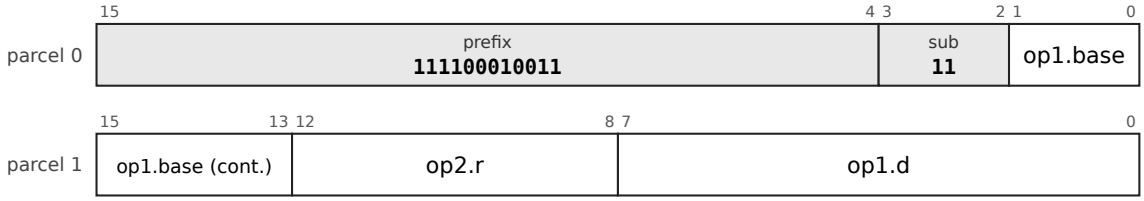

**BIS: LDX0/c, prefix 1111000111011, sub-opcode 11; operands x5-5d0s, reg5; 4 bytes**



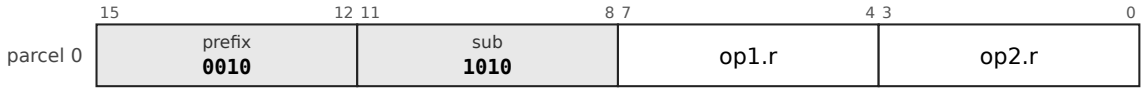

**BIS: LDPC16H/c, prefix 1110110101, sub-opcode 11; operands pc16-u1, reg4; 4 bytes**



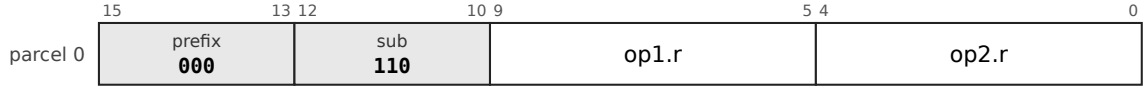

**BIS: LDBu8/c, prefix 111100010011, sub-opcode 11; operands b5d8u-xsz, reg5; 4 bytes**




**BIS: RRF, prefix 0010, sub-opcode 1010; operands reg4, reg4; 2 bytes**



**BIS: RR5, prefix 000, sub-opcode 110; operands reg5, reg5; 2 bytes**

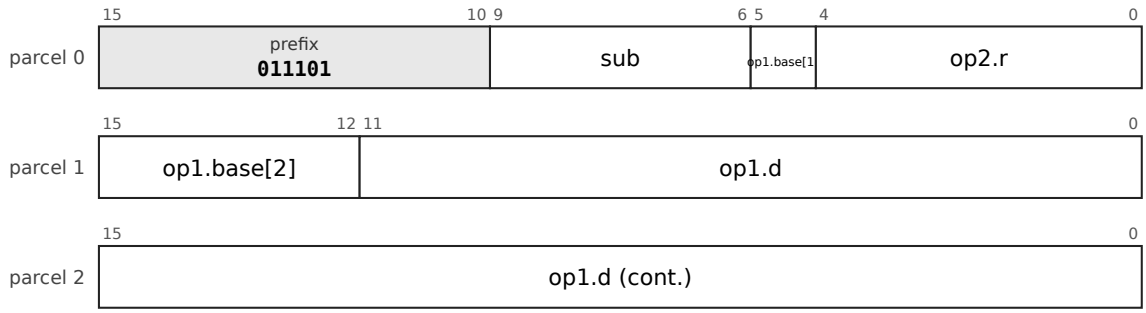



**Mnemonics:** `BIS`

### Syntax

```
BIS rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `RR5-1`, `LDBu6-c-1`, `LDI12H-c-1`, `LDBu8H-c-1`, `LDS12H-c-1`, `LDX0-c-1`, `LDPC16H-c-1`, `LDBu8-c-1`.

### Operation

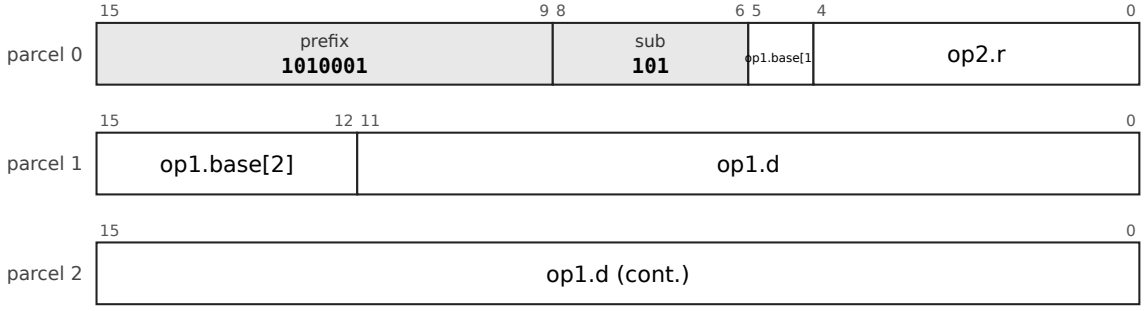
```
Rd ← Rd OR rm
```

**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

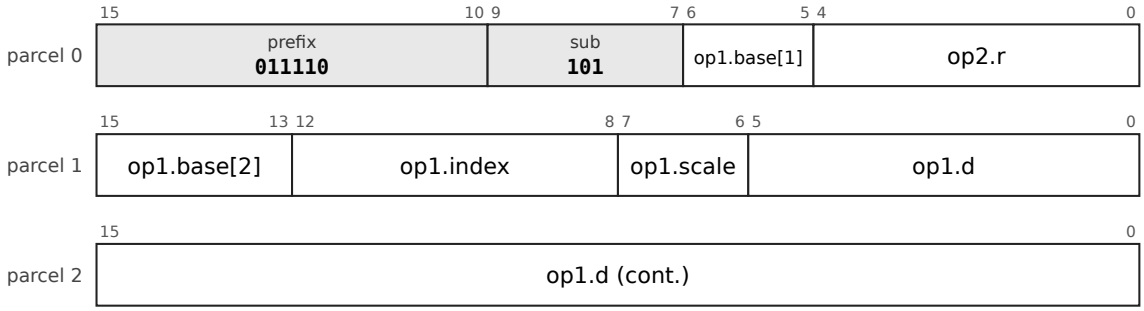
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## BIT: Bit test

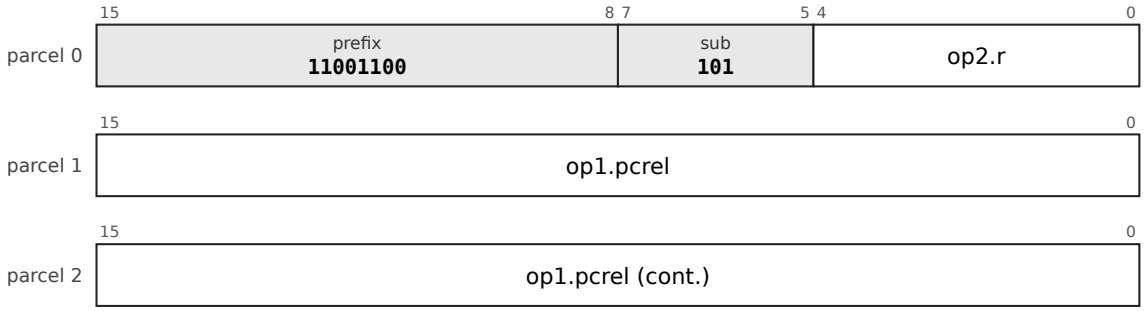
### BIT: LDF.B28, prefix 1010001, sub-opcode 101; operands b5d28s-x1, reg5; 6 bytes



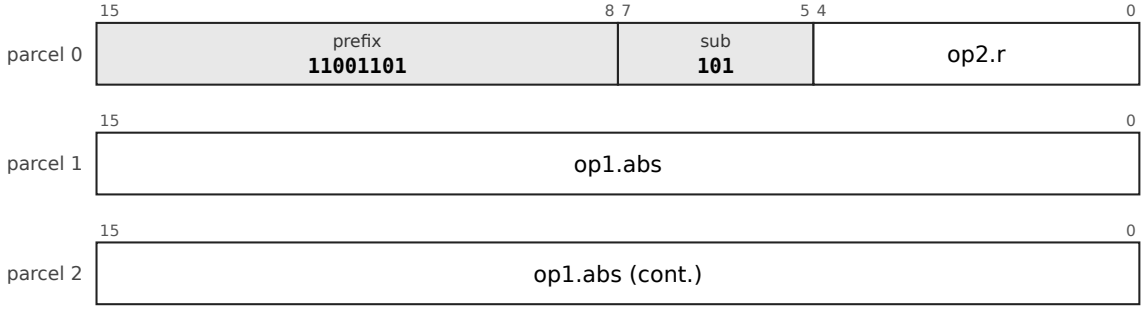

### BIT: LDF.X22, prefix 011110, sub-opcode 101; operands x5-5d22s, reg5; 6 bytes



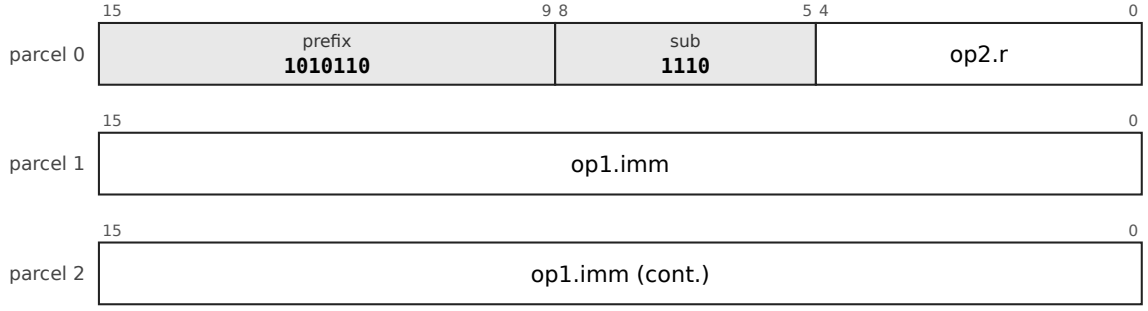

### BIT: LDF.PC32, prefix 11001100, sub-opcode 101; operands pc32-u1, reg5; 6 bytes



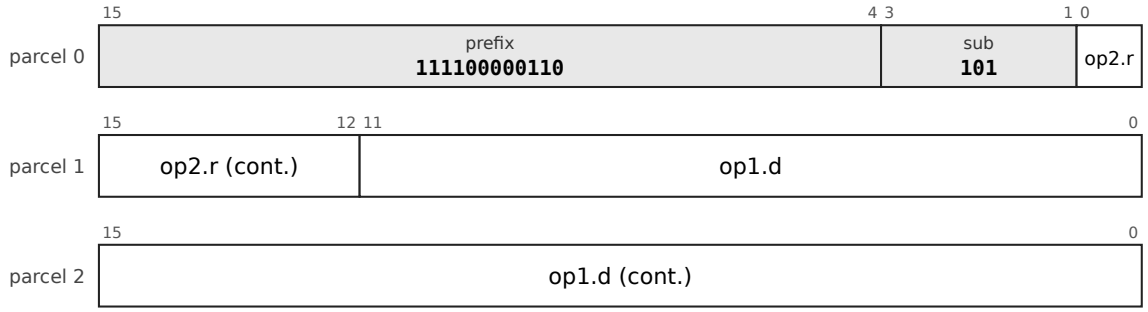

### BIT: LDF.A32, prefix 11001101, sub-opcode 101; operands abs32, reg5; 6 bytes



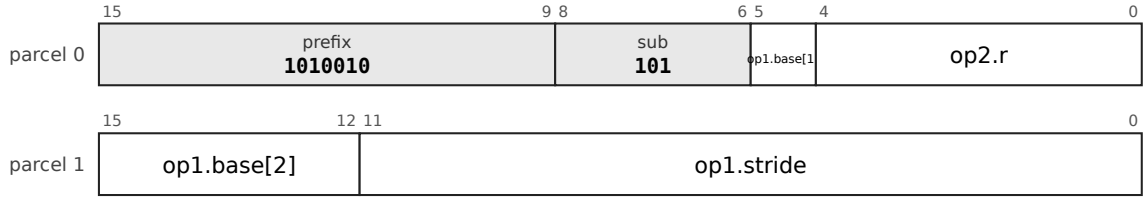
**BIT: LDFI.I32, prefix 1010110, sub-opcode 1110; operands imm32s, reg5; 6 bytes**



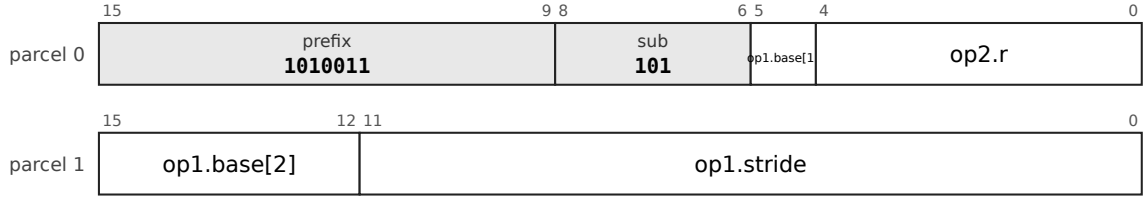

**BIT: DPF.D28, prefix 111100000110, sub-opcode 101; operands dpd28s-x1, reg5; 6 bytes**



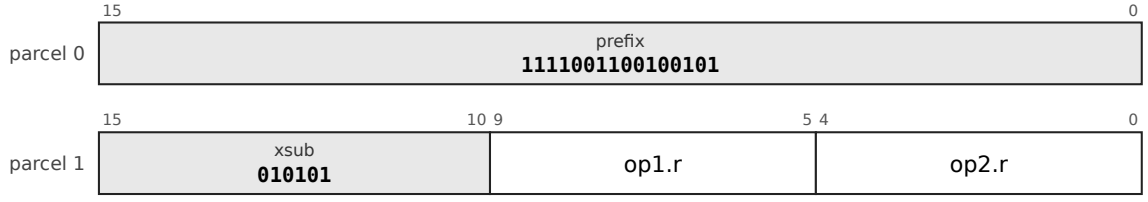

**BIT: LDF.PI12, prefix 1010010, sub-opcode 101; operands pi5-s12, reg5; 4 bytes**



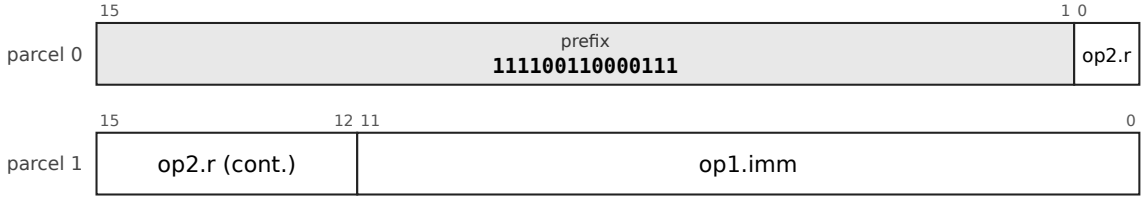

**BIT: LDF.PD12, prefix 1010011, sub-opcode 101; operands pd5-s12, reg5; 4 bytes**



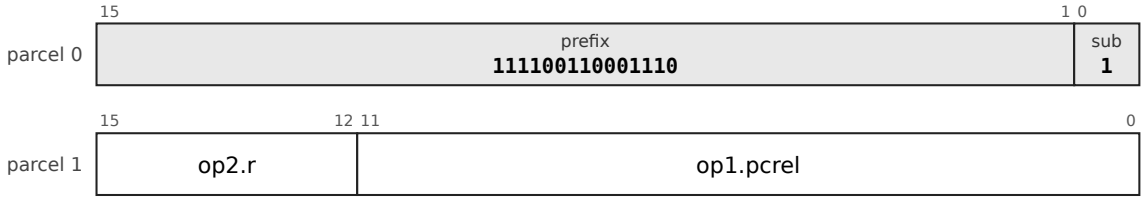

**BIT: RRF5, prefix 1111001100100101, sub-opcode 010101; operands reg5, reg5; 4 bytes**



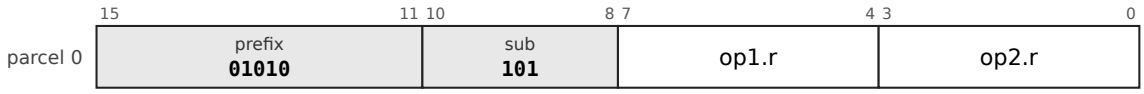
**BIT: LDI12/d, prefix 111100110000111; operands imm12s, reg5; 4 bytes**




**BIT: LDPC12H/d, prefix 111100110001110, sub-opcode 1; operands pc12-u1, reg4; 4 bytes**




**BIT: RRF, prefix 01010, sub-opcode 101; operands reg4, reg4; 2 bytes**

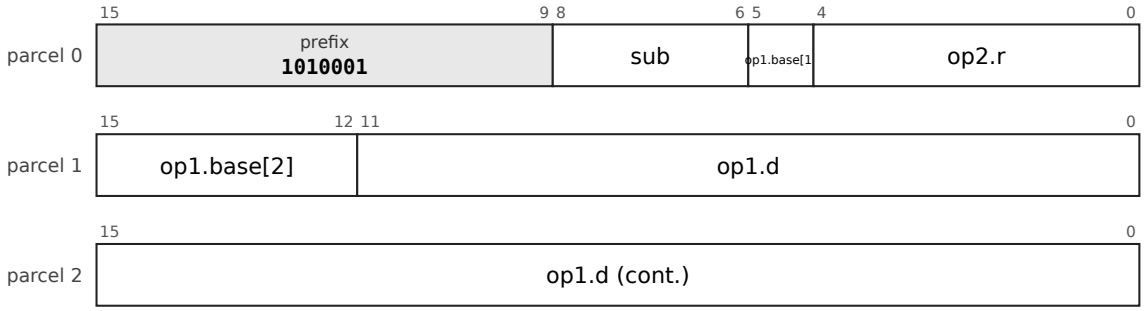



**Mnemonics:** `BIT`

### Syntax

```
BIT rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-2`, LDF.B28, prefix `1010001`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-2`, `LDF-X22-2`, `LDF-PI12-2`, `LDF-PD12-2`, `LDF-PC32-2`, `LDF-A32-2`, `LDFI-I32-1`, `DPF-D28-2`, `RRF-2`, `RRF5-1`, `LDI12-d-3`, `LDPC12H-d-2`.

### Operation

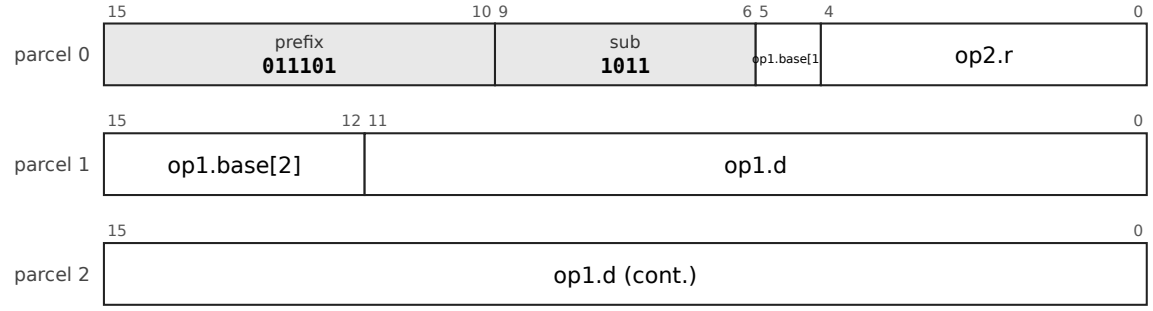
```
tmp ← Rd AND rm         ; the result is discarded
```

**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

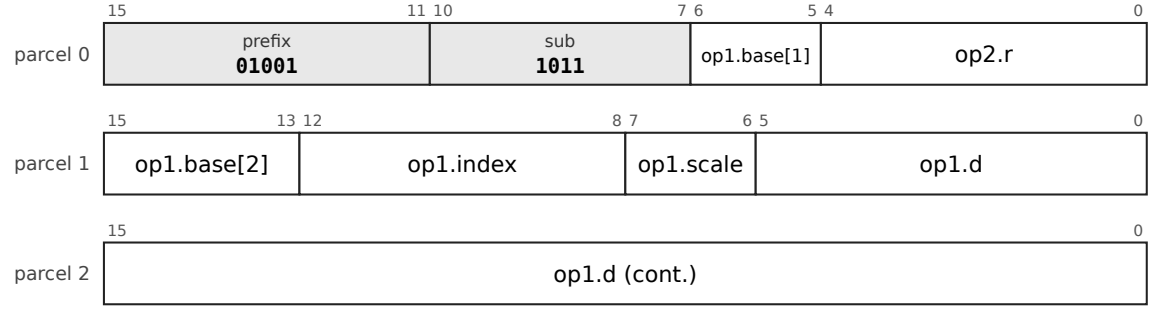
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## XOR: Exclusive or

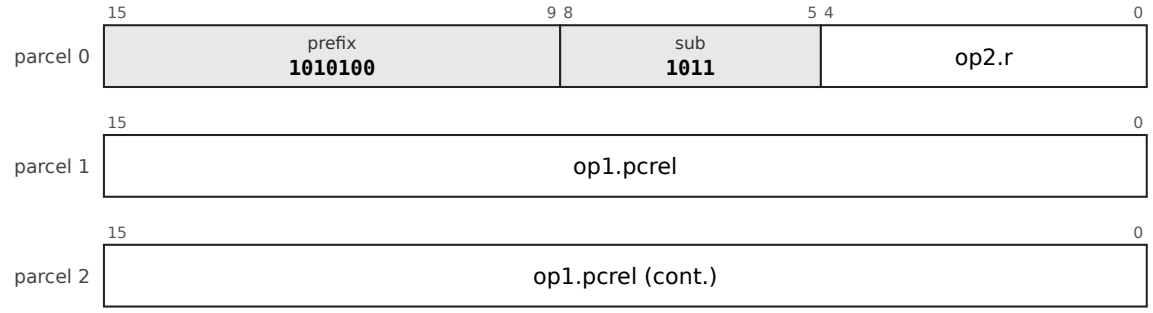
**XOR: LDF.B28, prefix 011101, sub-opcode 1011; operands b5d28s-x1, reg5; 6 bytes**



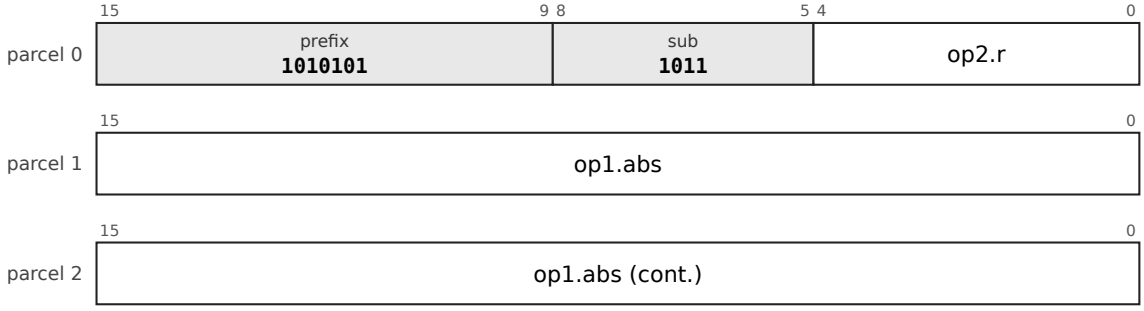

**XOR: LDF.X22, prefix 01001, sub-opcode 1011; operands x5-5d22s, reg5; 6 bytes**



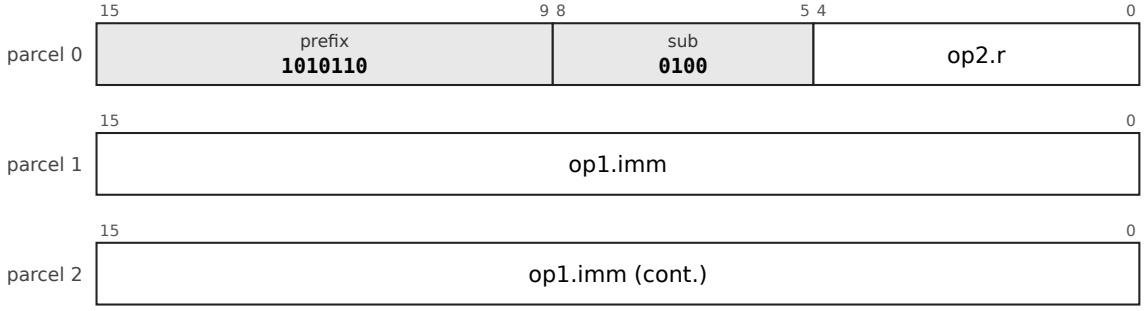

**XOR: LDF.PC32, prefix 1010100, sub-opcode 1011; operands pc32-u1, reg5; 6 bytes**



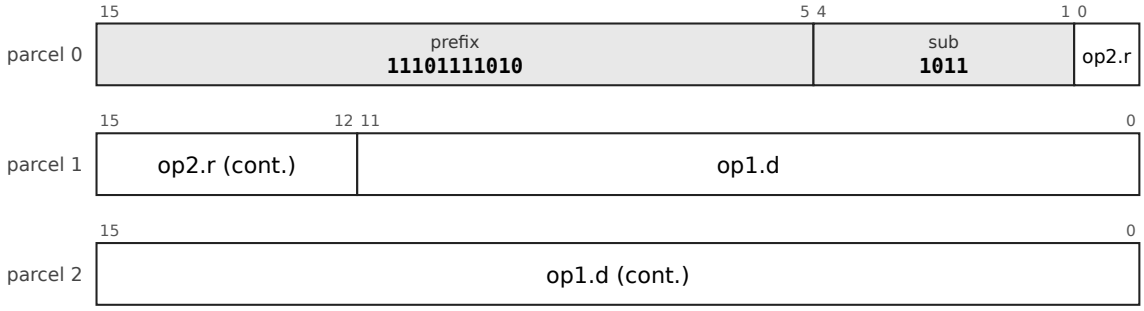
**XOR: LDF.A32, prefix 1010101, sub-opcode 1011; operands abs32, reg5; 6 bytes**



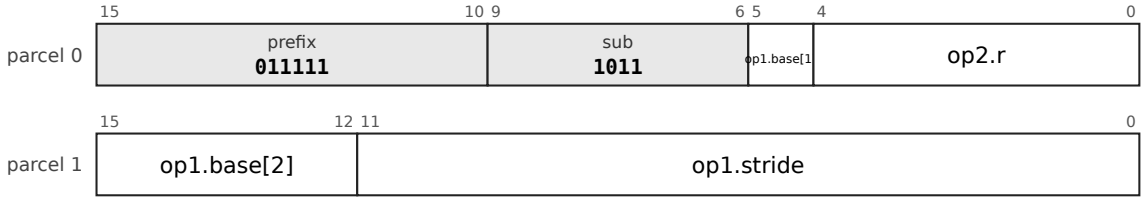

**XOR: LDFI.I32, prefix 1010110, sub-opcode 0100; operands imm32s, reg5; 6 bytes**



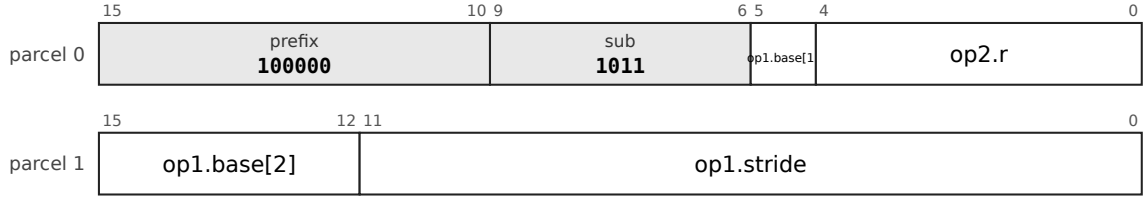

**XOR: DPF.D28, prefix 11101111010, sub-opcode 1011; operands dpd28s-x1, reg5; 6 bytes**



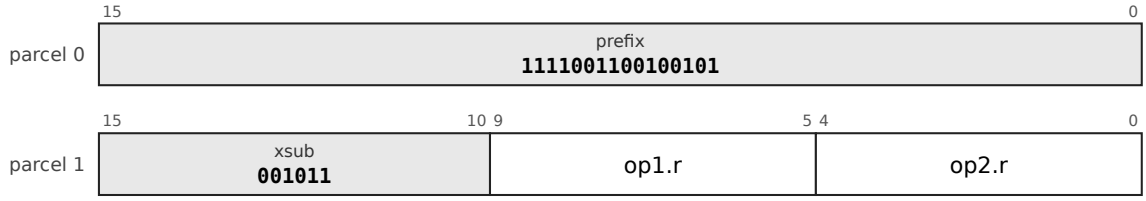

**XOR: LDF.PI12, prefix 011111, sub-opcode 1011; operands pi5-s12, reg5; 4 bytes**



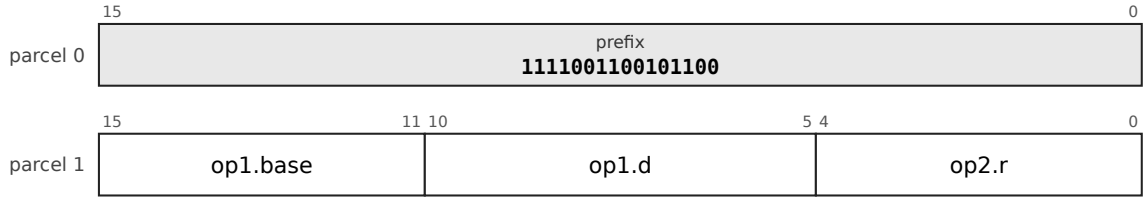
**XOR: LDF.PD12, prefix 100000, sub-opcode 1011; operands pd5-s12, reg5; 4 bytes**



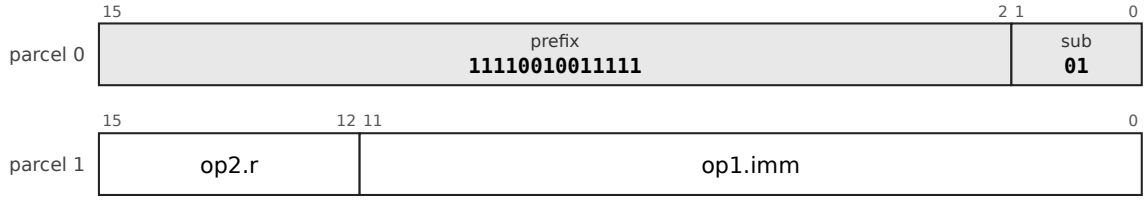

**XOR: RRF5, prefix 1111001100100101, sub-opcode 001011; operands reg5, reg5; 4 bytes**



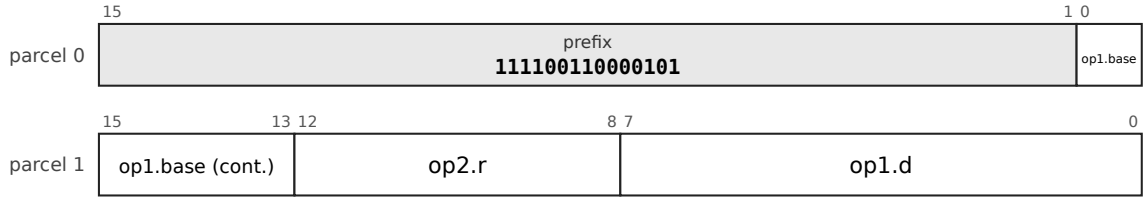

**XOR: LDBu6/c, prefix 1111001100101100; operands b5d6u-xsz, reg5; 4 bytes**



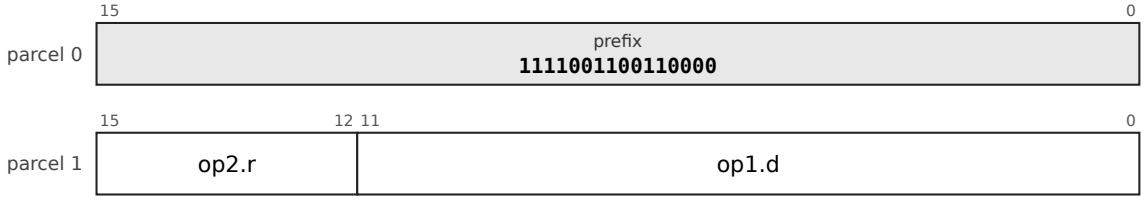

**XOR: LDI12H/c, prefix 11110010011111, sub-opcode 01; operands imm12s, reg4; 4 bytes**



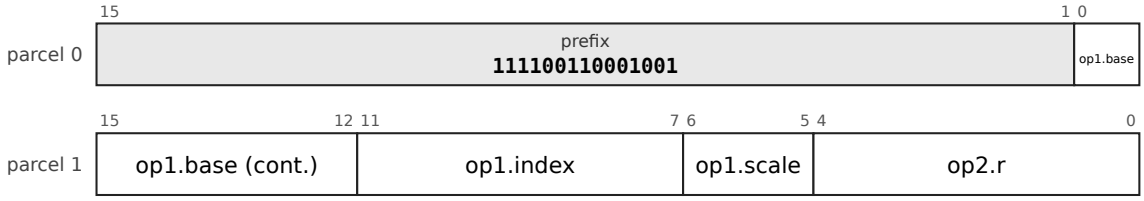

**XOR: LDBu8H/c, prefix 111100110000101; operands b4d8u-xsz, reg5; 4 bytes**



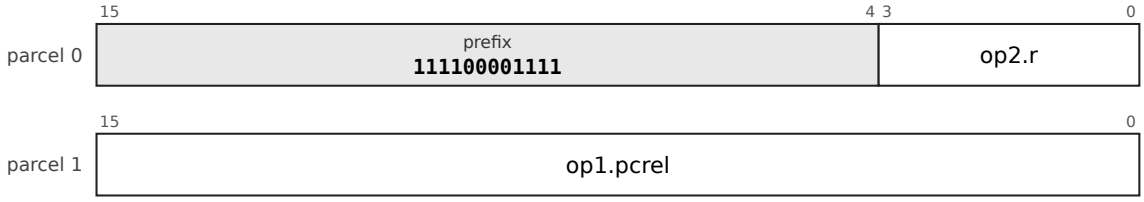
**XOR: LDS12H/c, prefix 1111001100110000; operands spd12u-xsz, reg4; 4 bytes**



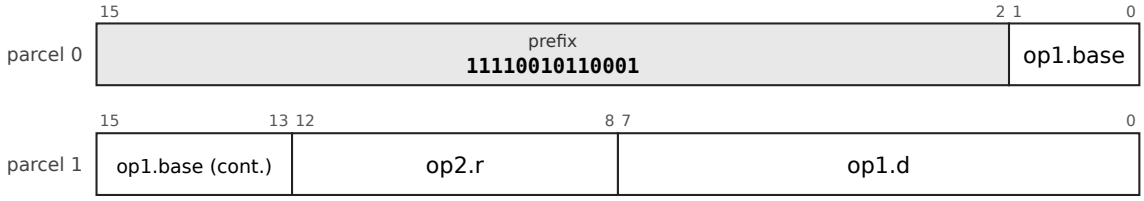

**XOR: LDX0/c, prefix 111100110001001; operands x5-5d0s, reg5; 4 bytes**



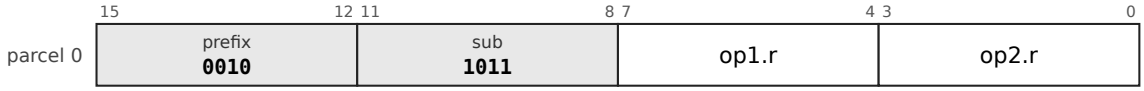

**XOR: LDPC16H/c, prefix 111100001111; operands pc16-u1, reg4; 4 bytes**



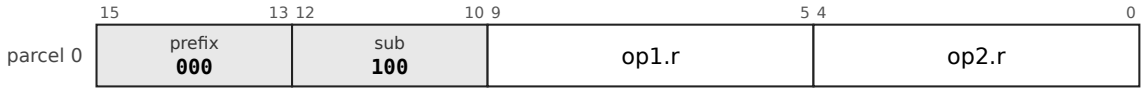

**XOR: LDBu8/c, prefix 11110010110001; operands b5d8u-xsz, reg5; 4 bytes**




**XOR: RRF, prefix 0010, sub-opcode 1011; operands reg4, reg4; 2 bytes**




**XOR: RR5, prefix 000, sub-opcode 100; operands reg5, reg5; 2 bytes**

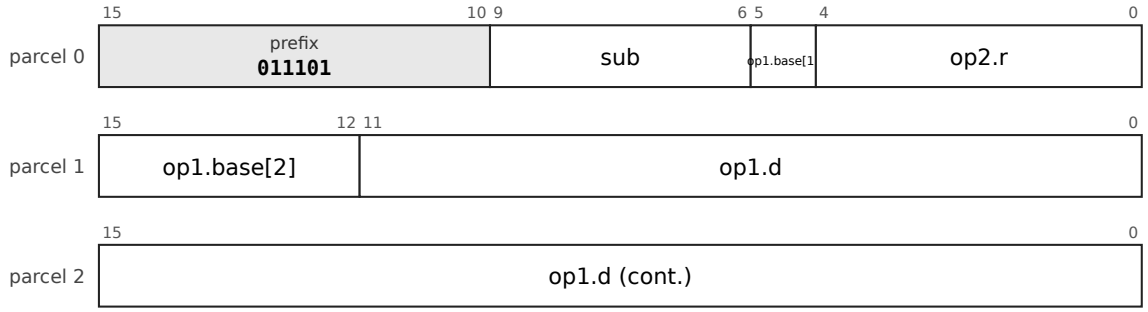



**Mnemonics:** `XOR`

## Syntax

```
XOR rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `RR5-1`, `LDBu6-c-2`, `LDI12H-c-1`, `LDBu8H-c-2`, `LDS12H-c-2`, `LDX0-c-2`, `LDPC16H-c-2`, `LDBu8-c-2`.

### Operation

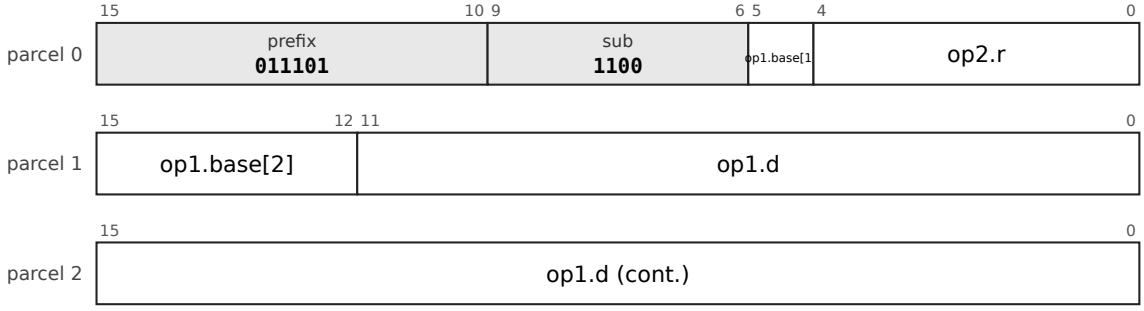
```
Rd ← Rd XOR rm
```

**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

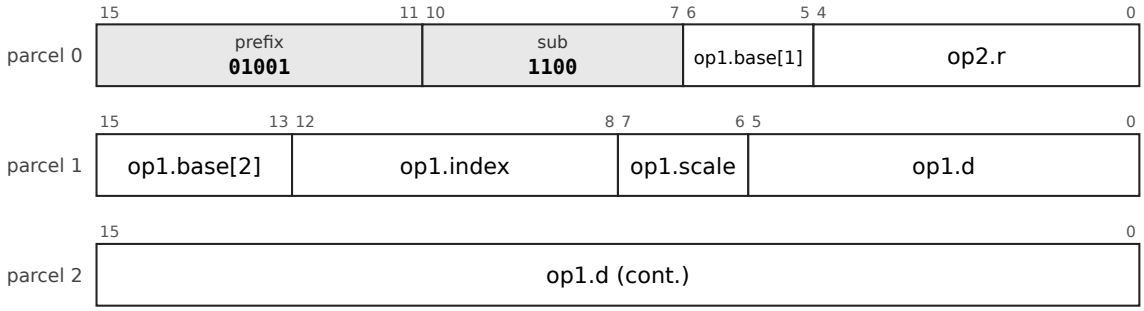
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## AND: And

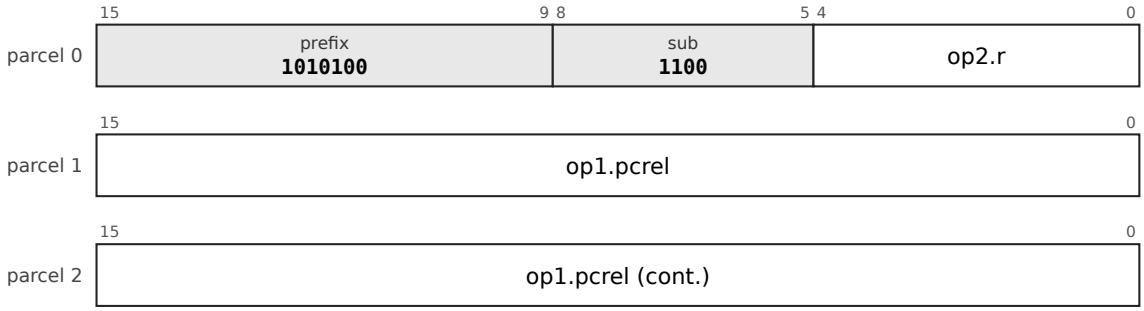
**AND: LDF.B28, prefix 011101, sub-opcode 1100; operands b5d28s-x1, reg5; 6 bytes**



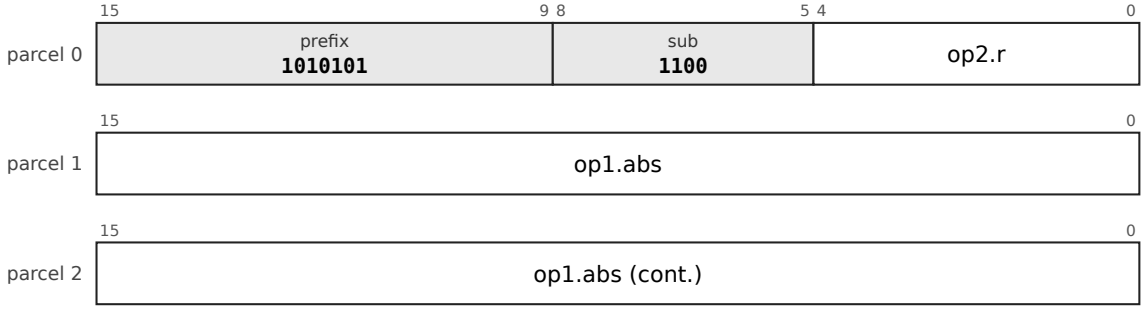

**AND: LDF.X22, prefix 01001, sub-opcode 1100; operands x5-5d22s, reg5; 6 bytes**



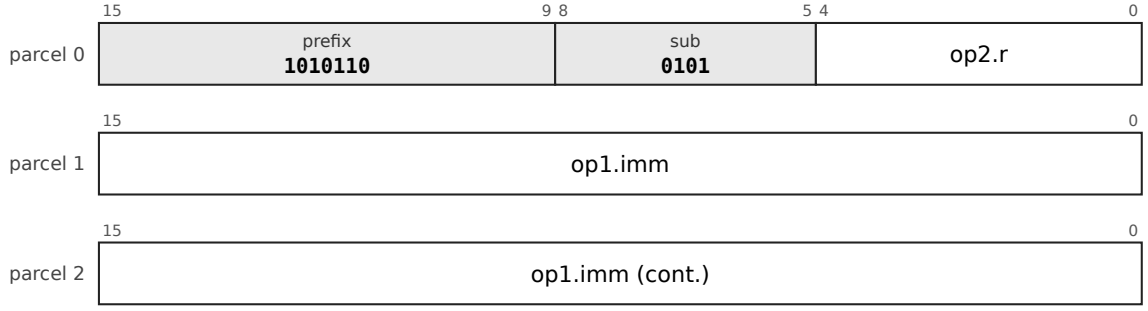

**AND: LDF.PC32, prefix 1010100, sub-opcode 1100; operands pc32-u1, reg5; 6 bytes**



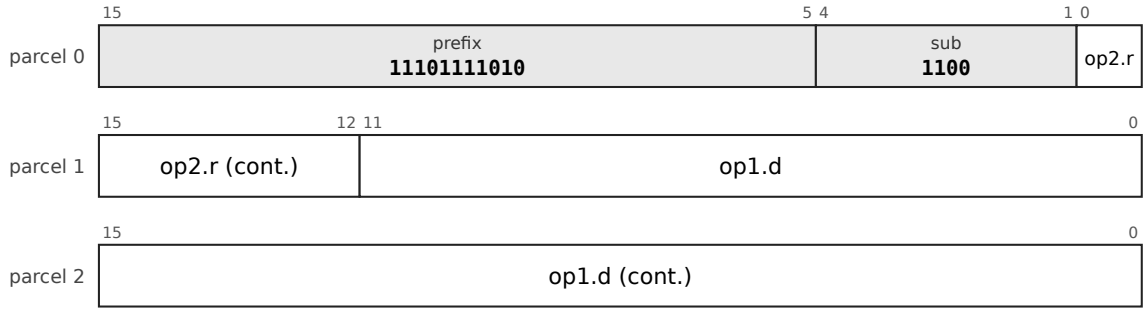

**AND: LDF.A32, prefix 1010101, sub-opcode 1100; operands abs32, reg5; 6 bytes**



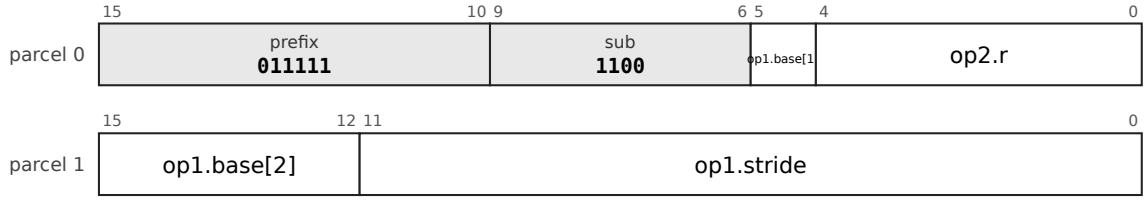
**AND: LDFI.I32, prefix 1010110, sub-opcode 0101; operands imm32s, reg5; 6 bytes**



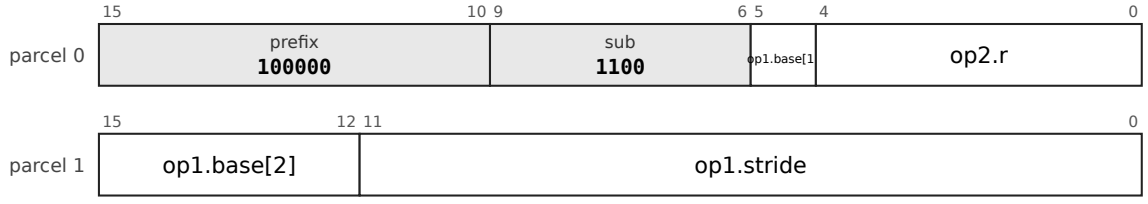

**AND: DPF.D28, prefix 11101111010, sub-opcode 1100; operands dpd28s-x1, reg5; 6 bytes**



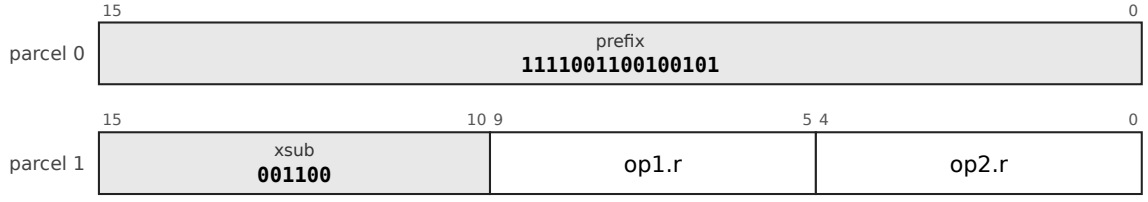

**AND: LDF.PI12, prefix 011111, sub-opcode 1100; operands pi5-s12, reg5; 4 bytes**



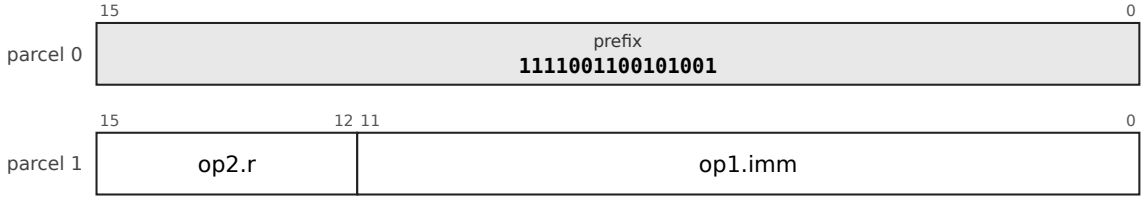

**AND: LDF.PD12, prefix 100000, sub-opcode 1100; operands pd5-s12, reg5; 4 bytes**



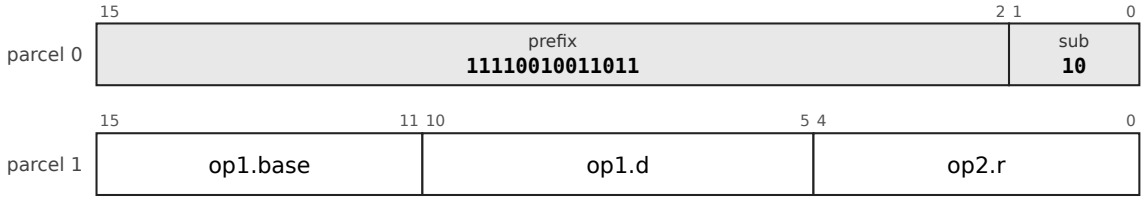

**AND: RRF5, prefix 1111001100100101, sub-opcode 001100; operands reg5, reg5; 4 bytes**



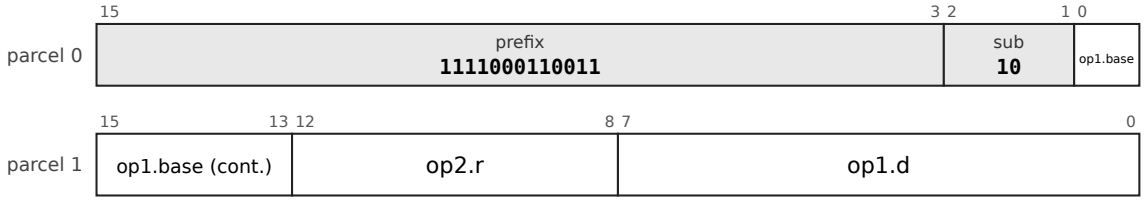
**AND: LDI12H/b, prefix 1111001100101001; operands imm12s, reg4; 4 bytes**



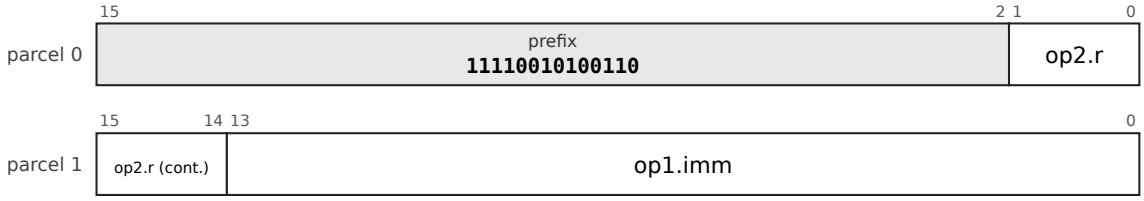

**AND: LDBu6/c, prefix 11110010011011, sub-opcode 10; operands b5d6u-xsz, reg5; 4 bytes**



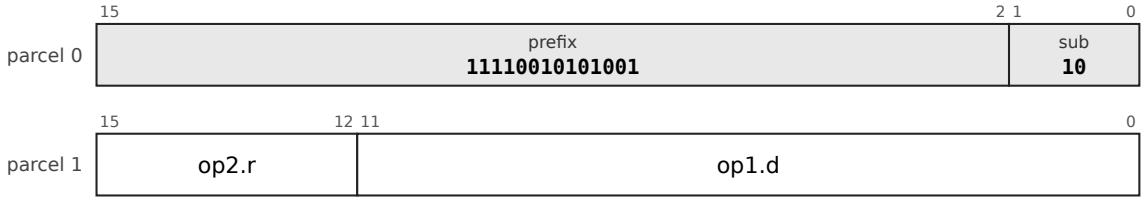

**AND: LDBu8H/c, prefix 1111000110011, sub-opcode 10; operands b4d8u-xsz, reg5; 4 bytes**



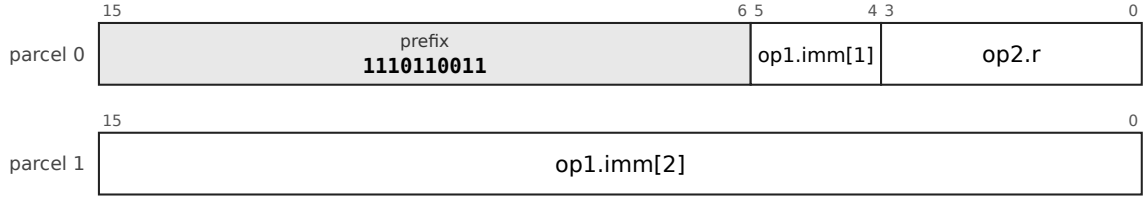

**AND: LDI14H/b, prefix 11110010100110; operands imm14s, reg4; 4 bytes**



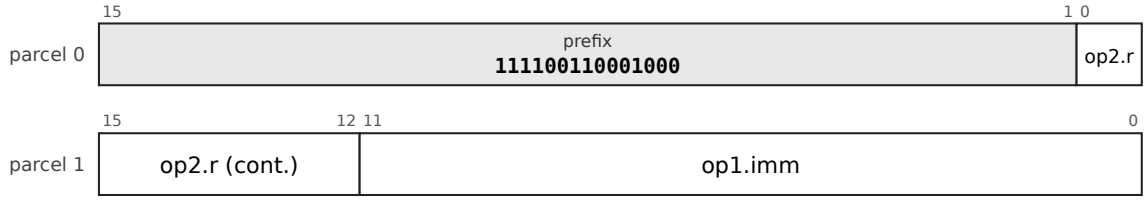

**AND: LDS12H/c, prefix 11110010101001, sub-opcode 10; operands spd12u-xsz, reg4; 4 bytes**



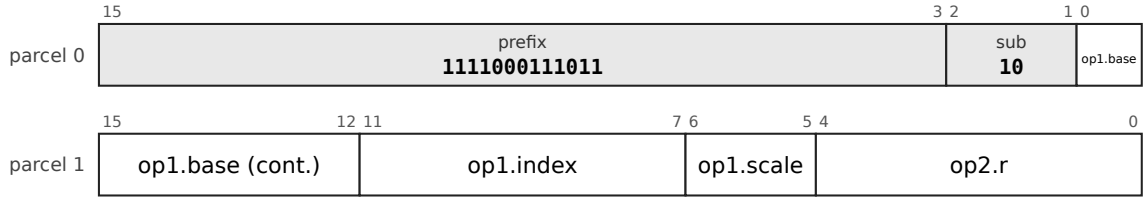
**AND: LDI18H/b, prefix 1110110011; operands imm18s, reg4; 4 bytes**



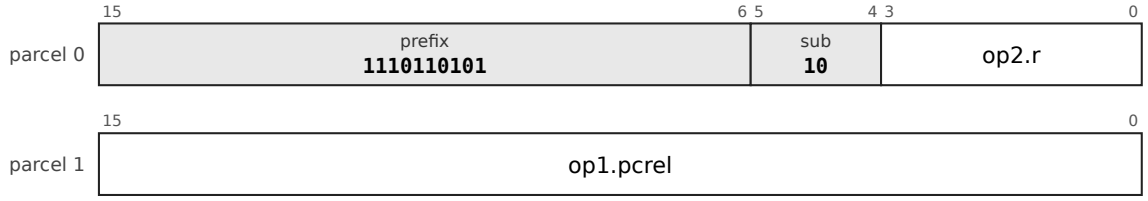

**AND: LDI12/b, prefix 111100110001000; operands imm12s, reg5; 4 bytes**



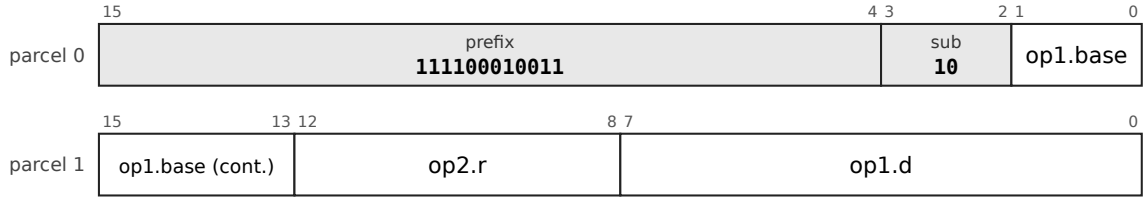

**AND: LDX0/c, prefix 1111000111011, sub-opcode 10; operands x5-5d0s, reg5; 4 bytes**



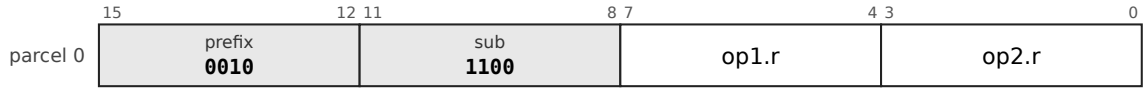

**AND: LDPC16H/c, prefix 1110110101, sub-opcode 10; operands pc16-u1, reg4; 4 bytes**



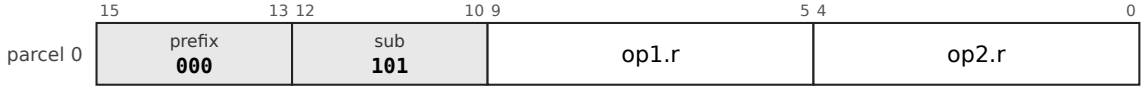

**AND: LDBu8/c, prefix 111100010011, sub-opcode 10; operands b5d8u-xsz, reg5; 4 bytes**




**AND: RRF, prefix 0010, sub-opcode 1100; operands reg4, reg4; 2 bytes**



**AND: RR5, prefix 000, sub-opcode 101; operands reg5, reg5; 2 bytes**




**AND: IMMh43/b, prefix 111001011; operands imm3s, reg4; 2 bytes**

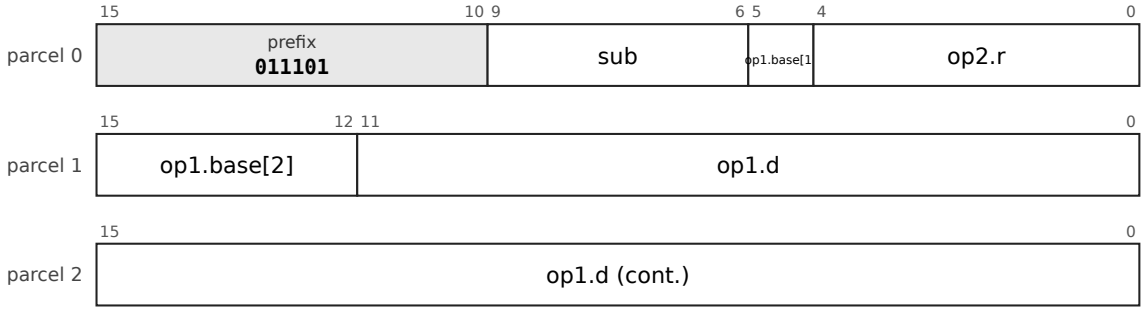



**Mnemonics:** `AND`

### Syntax

```
AND rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `RR5-1`, `LDI12H-b-2`, `LDBu6-c-1`, `LDBu8H-c-1`, `LDI14H-b-2`, `LDS12H-c-1`, `IMMh43-b-2`, `LDI18H-b-2`, `LDI12-b-2`, `LDX0-c-1`, `LDPC16H-c-1`, `LDBu8-c-1`.

### Operation

```
Rd ← Rd AND rm
```

**Flags** `N` and `Z` from the result, `V` cleared, `C` unchanged.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

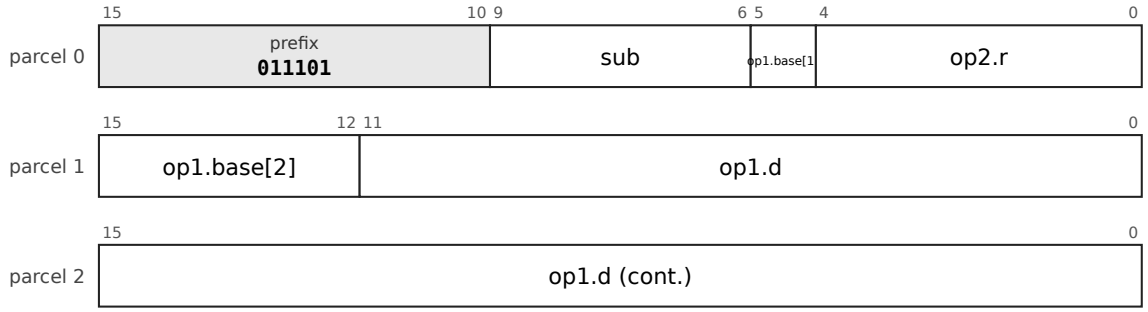
## SHIFT: Shift left, shift right logical, shift right arithmetic

**Mnemonics:** `SHL`, `SHR`, `SAR`

### Syntax

```
SHL rm, Rd
SHR rm, Rd
SAR rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-1`, LDF.B28, prefix `011101`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-1`, `LDF-X22-1`, `LDF-PI12-1`, `LDF-PD12-1`, `LDF-PC32-1`, `LDF-A32-1`, `LDFI-I32-1`, `DPF-D28-1`, `RRF-1`, `RRF5-1`, `RR5-1`, `LDI12-d-1`, `SHIh43-a-1`, `SHIh44-a-1`, `LDPC12H-d-1`, `LDF-B28-2`, `LDF-X22-2`, `LDF-PI12-2`, `LDF-PD12-2`, `LDF-PC32-2`, `LDF-A32-2`, `DPF-D28-2`, `RRF-2`.

### Operation

```
n ← rm                                ; the count: a register, an immediate, or memory
SHL: Rd ← (n ≥ 64) ? 0 : Rd << n
SHR: Rd ← (n ≥ 64) ? 0 : Rd >>u n
SAR: Rd ← (n ≥ 64) ? (Rd < 0 ? -1 : 0) : Rd >>s n
```

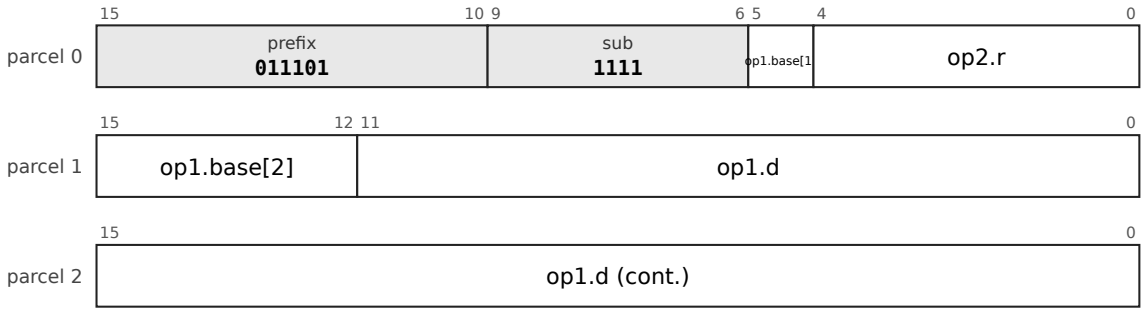
**Flags** `N` and `Z` from the result, `V` cleared, `C` the last bit shifted out (bit 63 for `SHL`, bit 0 for `SHR` and `SAR`). **The same flags are written for every count**: a count of zero gives `N` and `Z` from the unchanged operand, `V` = 0 and `C` = 0.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

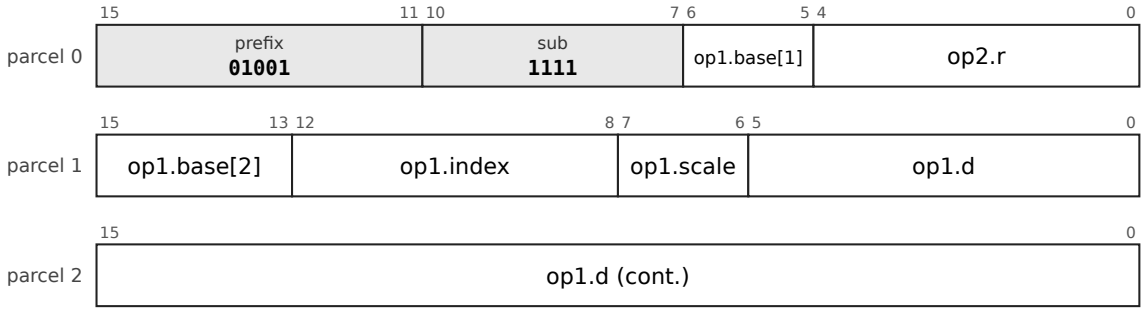
The count is read as an unsigned 64-bit value, so a count of 64 or more shifts the value entirely away. Small immediate counts have one-parcel forms (the layouts of this entry); `ASL` and `ASR` are the by-one forms; rotates are `ROR` and `RORI`.

## *SHL*

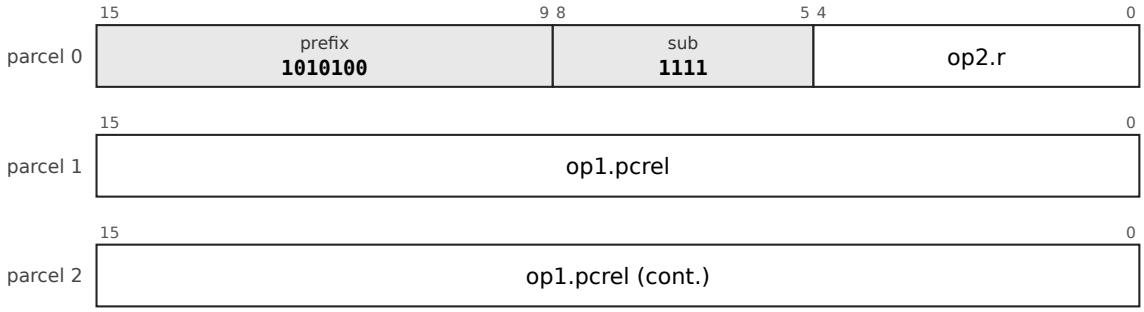
**SHL: LDF.B28, prefix 011101, sub-opcode 1111; operands b5d28s-x1, reg5; 6 bytes**



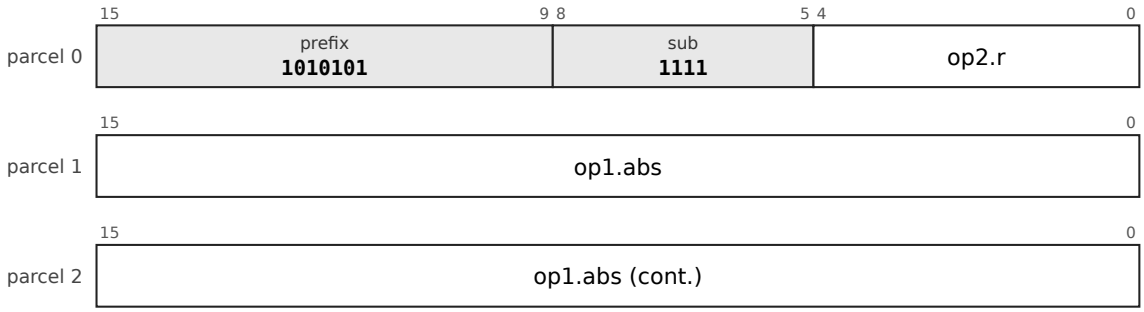

**SHL: LDF.X22, prefix 01001, sub-opcode 1111; operands x5-5d22s, reg5; 6 bytes**



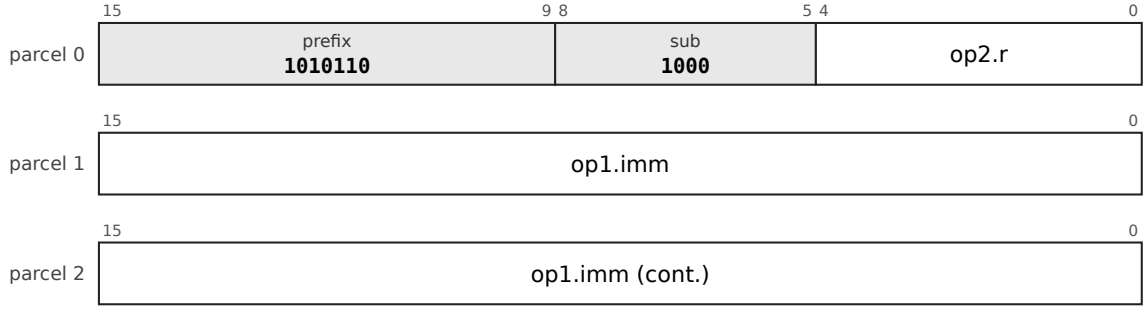

**SHL: LDF.PC32, prefix 1010100, sub-opcode 1111; operands pc32-u1, reg5; 6 bytes**



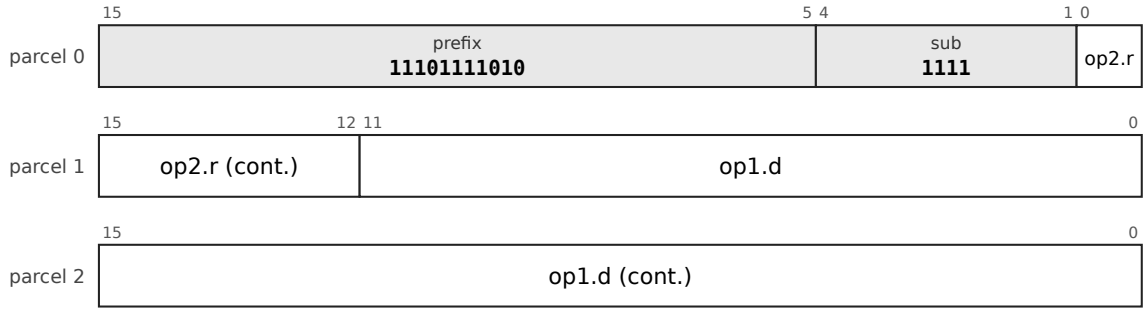

**SHL: LDF.A32, prefix 1010101, sub-opcode 1111; operands abs32, reg5; 6 bytes**



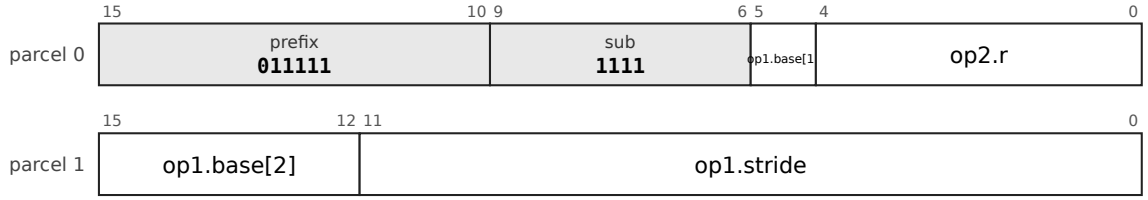
**SHL: LDFI.I32, prefix 1010110, sub-opcode 1000; operands imm32s, reg5; 6 bytes**



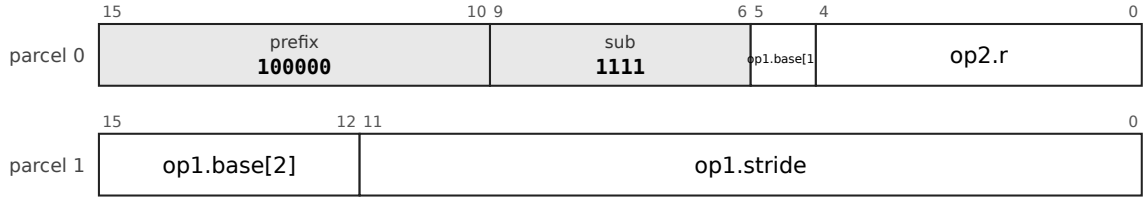

**SHL: DPF.D28, prefix 11101111010, sub-opcode 1111; operands dpd28s-x1, reg5; 6 bytes**



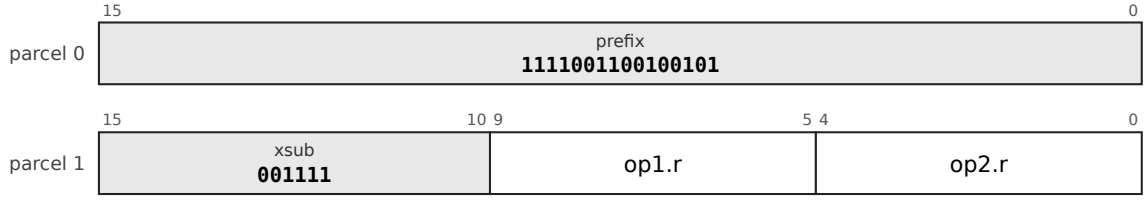

**SHL: LDF.PI12, prefix 011111, sub-opcode 1111; operands pi5-s12, reg5; 4 bytes**



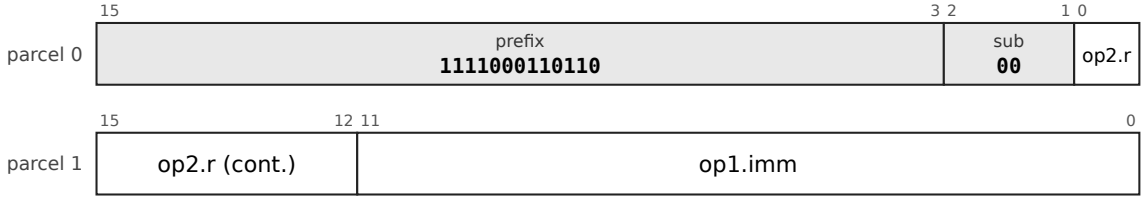

**SHL: LDF.PD12, prefix 100000, sub-opcode 1111; operands pd5-s12, reg5; 4 bytes**



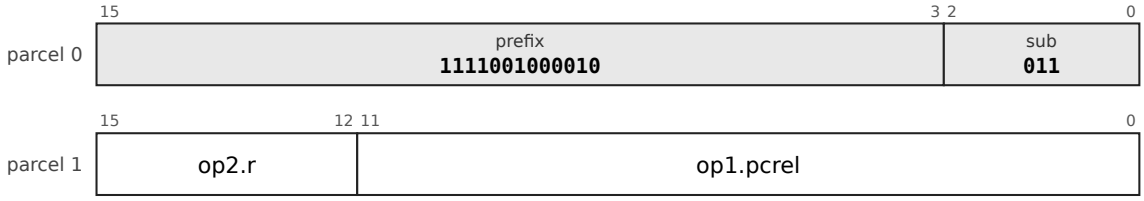

**SHL: RRF5, prefix 1111001100100101, sub-opcode 001111; operands reg5, reg5; 4 bytes**



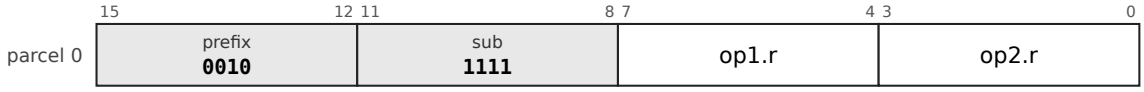
**SHL: LDI12/d, prefix 1111000110110, sub-opcode 00; operands imm12s, reg5; 4 bytes**



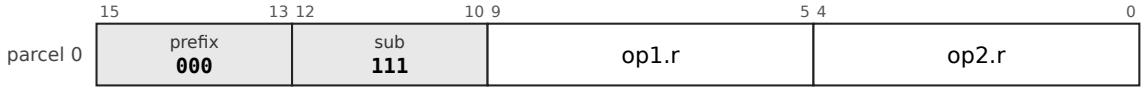

**SHL: LDPC12H/d, prefix 1111001000010, sub-opcode 011; operands pc12-u1, reg4; 4 bytes**



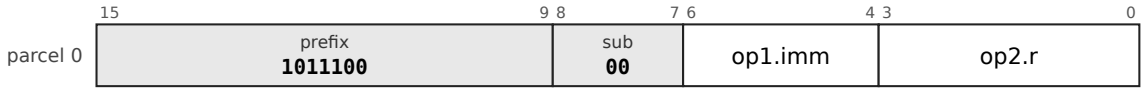

**SHL: RRF, prefix 0010, sub-opcode 1111; operands reg4, reg4; 2 bytes**



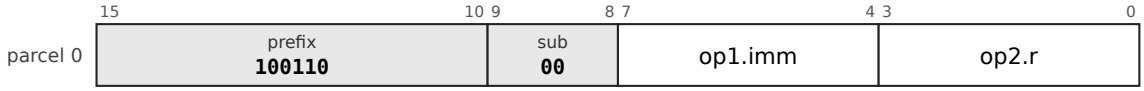

**SHL: RR5, prefix 000, sub-opcode 111; operands reg5, reg5; 2 bytes**




**SHL: SHIh43/a, prefix 1011100, sub-opcode 00; operands imm3u, reg4; 2 bytes**




**SHL: SHIh44/a, prefix 100110, sub-opcode 00; operands imm4u, reg4; 2 bytes**

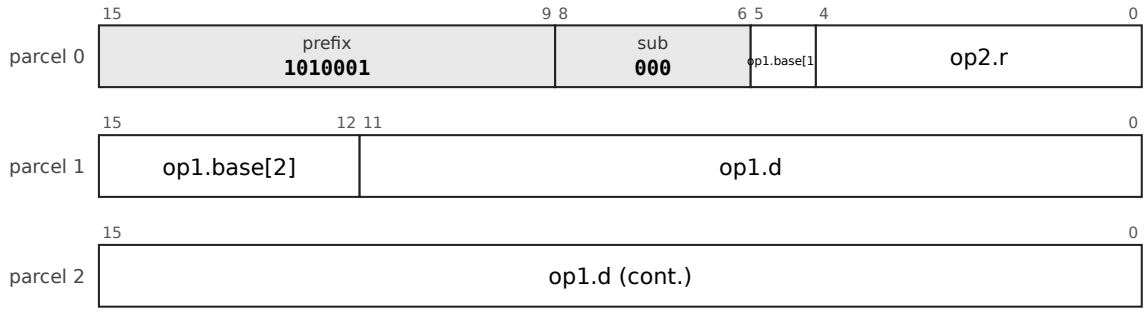



```
SHL rm, Rd
```

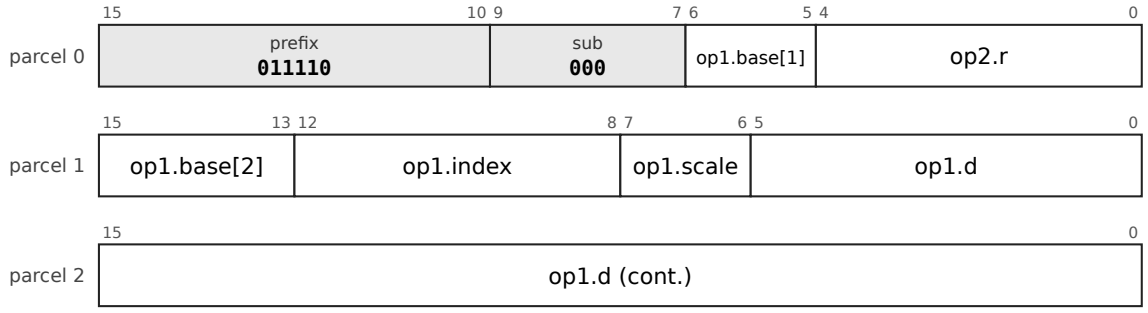
Operation, flags and exceptions as for the entry SHIFT above; 15 layout forms, each a chart of its own (above).

### *SHR*

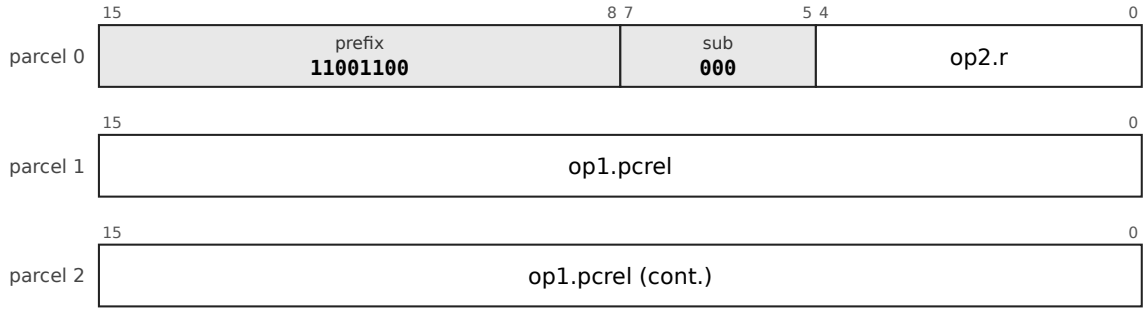
**SHR: LDF.B28, prefix 1010001, sub-opcode 000; operands b5d28s-x1, reg5; 6 bytes**



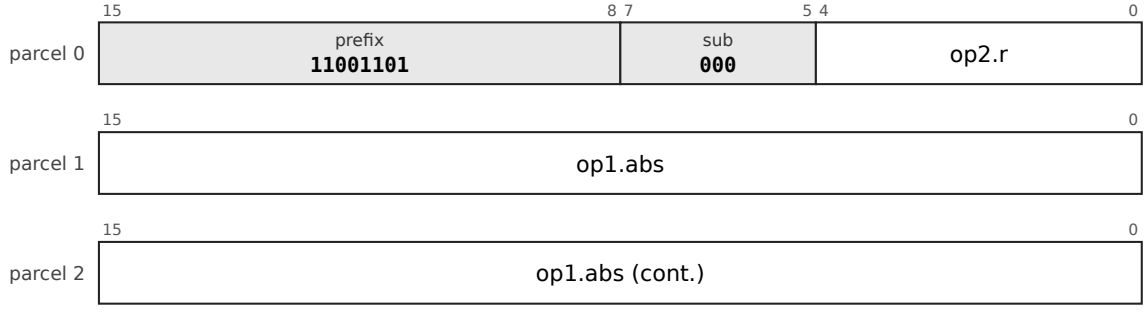

**SHR: LDF.X22, prefix 011110, sub-opcode 000; operands x5-5d22s, reg5; 6 bytes**



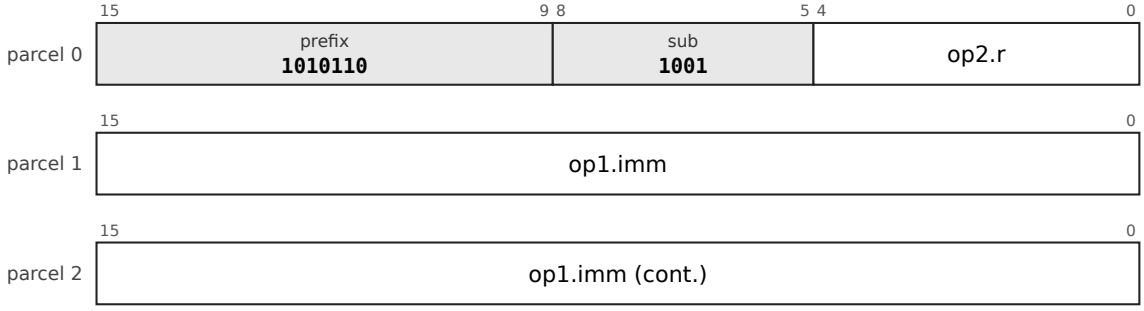

**SHR: LDF.PC32, prefix 11001100, sub-opcode 000; operands pc32-u1, reg5; 6 bytes**



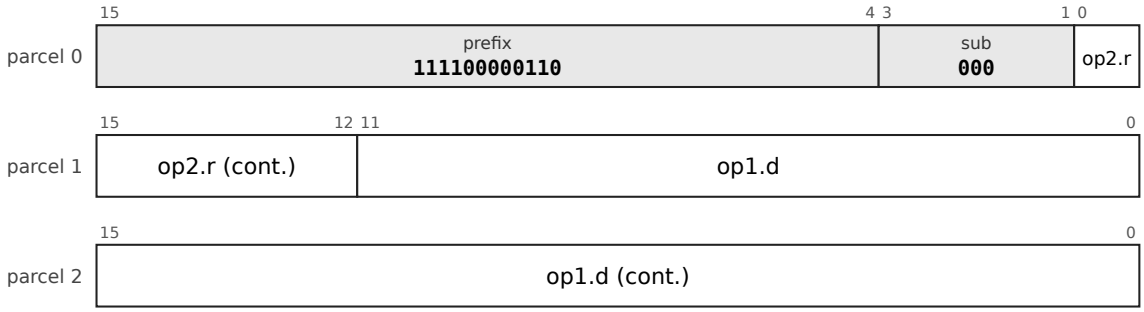

**SHR: LDF.A32, prefix 11001101, sub-opcode 000; operands abs32, reg5; 6 bytes**



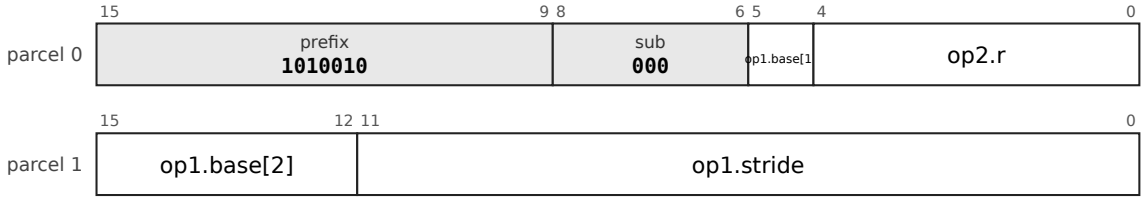
**SHR: LDFI.I32, prefix 1010110, sub-opcode 1001; operands imm32s, reg5; 6 bytes**



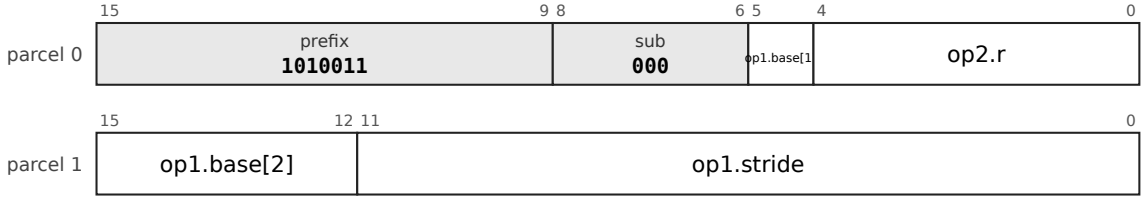

**SHR: DPF.D28, prefix 111100000110, sub-opcode 000; operands dpd28s-x1, reg5; 6 bytes**



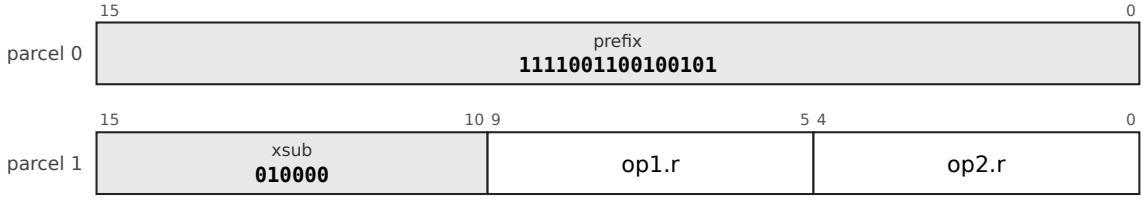

**SHR: LDF.PI12, prefix 1010010, sub-opcode 000; operands pi5-s12, reg5; 4 bytes**



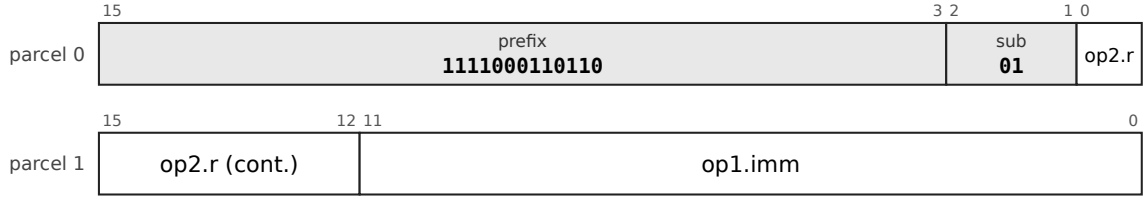

**SHR: LDF.PD12, prefix 1010011, sub-opcode 000; operands pd5-s12, reg5; 4 bytes**



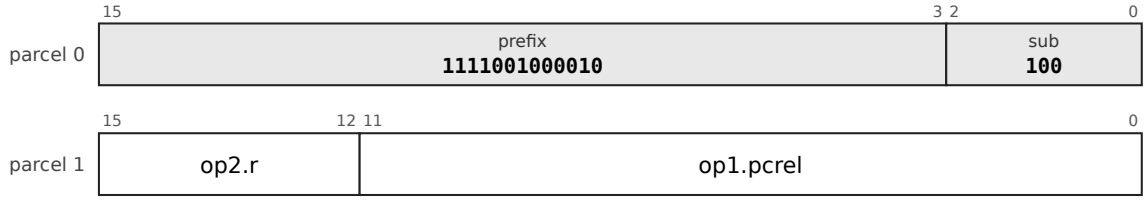

**SHR: RRF5, prefix 1111001100100101, sub-opcode 010000; operands reg5, reg5; 4 bytes**



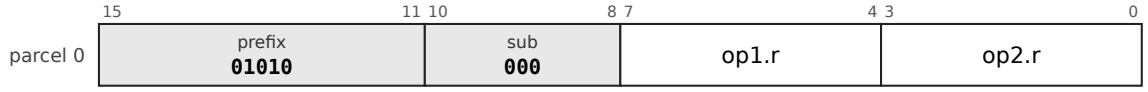
**SHR: LDI12/d, prefix 1111000110110, sub-opcode 01; operands imm12s, reg5; 4 bytes**



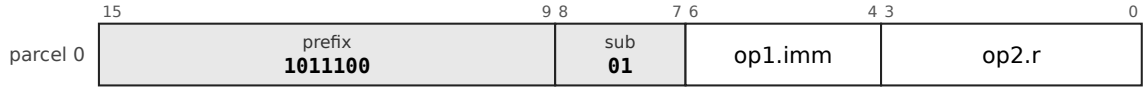

**SHR: LDPC12H/d, prefix 1111001000010, sub-opcode 100; operands pc12-u1, reg4; 4 bytes**



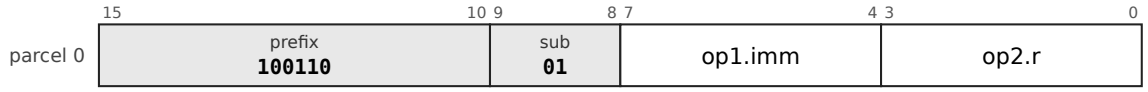

**SHR: RRF, prefix 01010, sub-opcode 000; operands reg4, reg4; 2 bytes**




**SHR: SHlh43/a, prefix 1011100, sub-opcode 01; operands imm3u, reg4; 2 bytes**




**SHR: SHlh44/a, prefix 100110, sub-opcode 01; operands imm4u, reg4; 2 bytes**

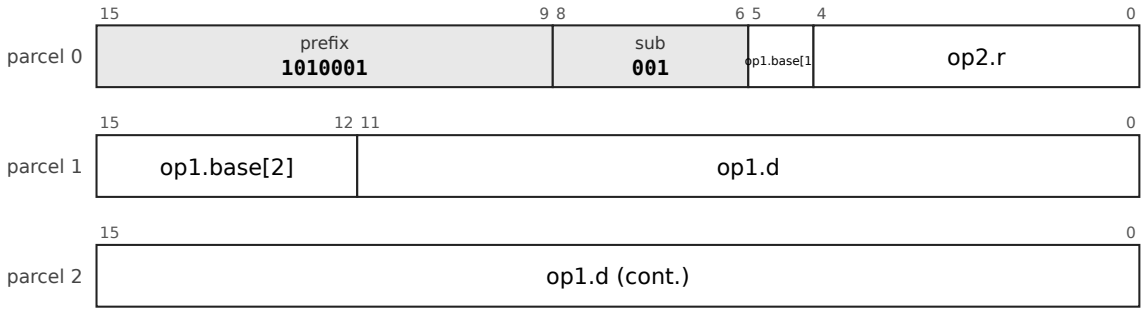



```
SHR rm, Rd
```

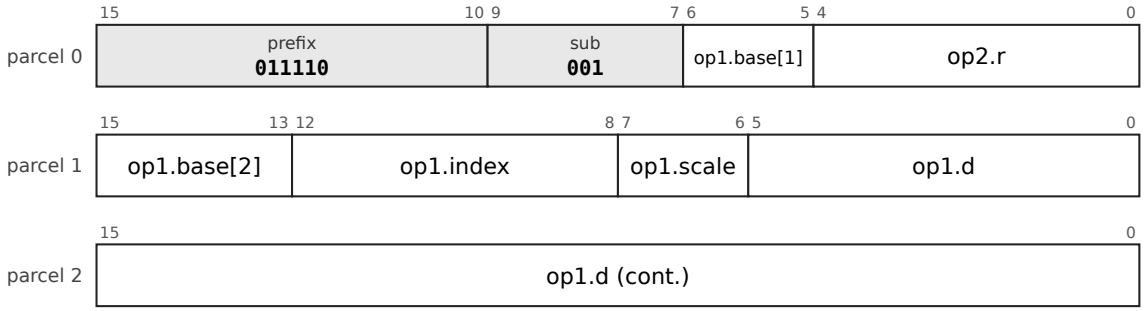
Operation, flags and exceptions as for the entry SHIFT above; 14 layout forms, each a chart of its own (above).

## *SAR*

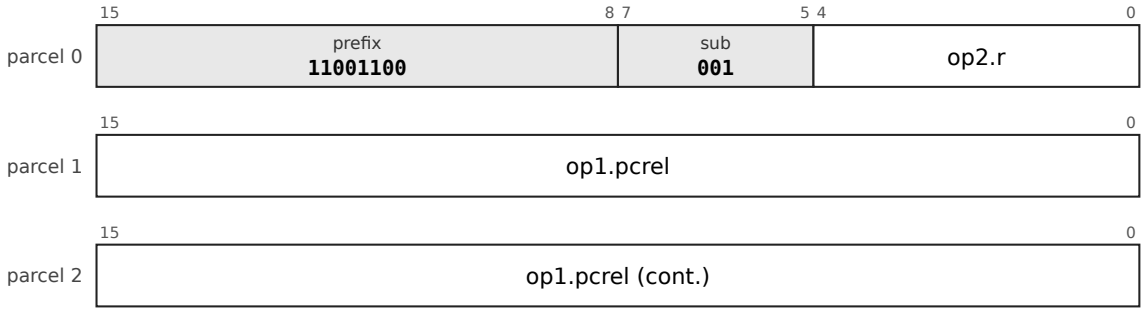
**SAR: LDF.B28, prefix 1010001, sub-opcode 001; operands b5d28s-x1, reg5; 6 bytes**



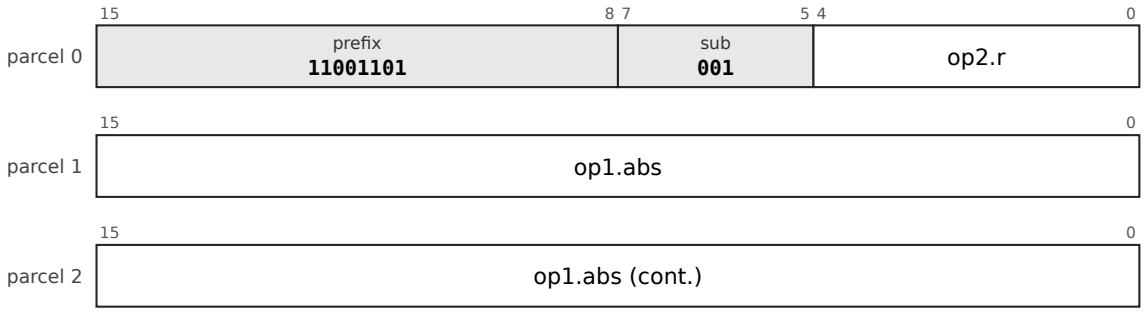

**SAR: LDF.X22, prefix 011110, sub-opcode 001; operands x5-5d22s, reg5; 6 bytes**



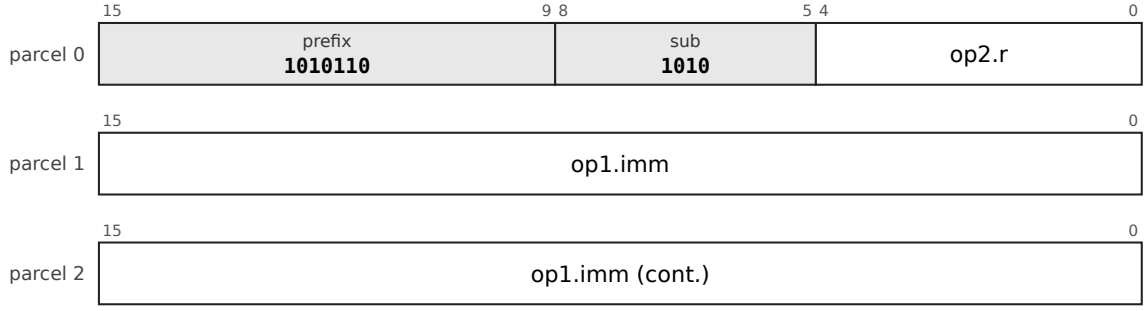

**SAR: LDF.PC32, prefix 11001100, sub-opcode 001; operands pc32-u1, reg5; 6 bytes**



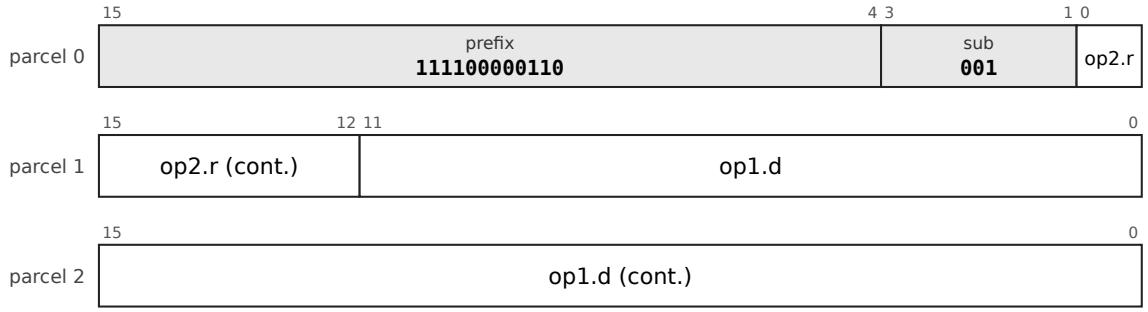

**SAR: LDF.A32, prefix 11001101, sub-opcode 001; operands abs32, reg5; 6 bytes**



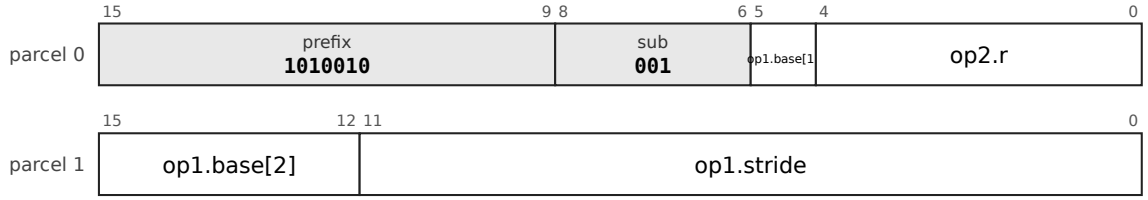
**SAR: LDFI.I32, prefix 1010110, sub-opcode 1010; operands imm32s, reg5; 6 bytes**



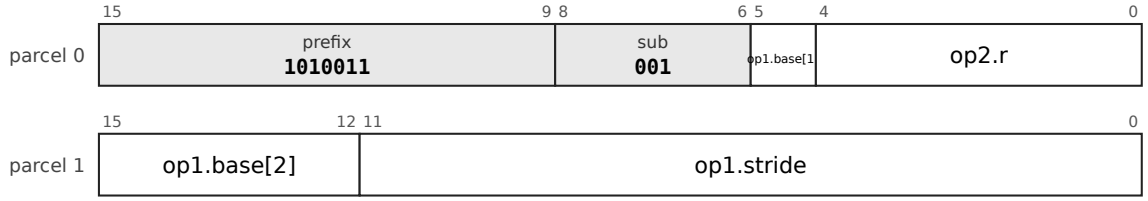

**SAR: DPF.D28, prefix 111100000110, sub-opcode 001; operands dpd28s-x1, reg5; 6 bytes**



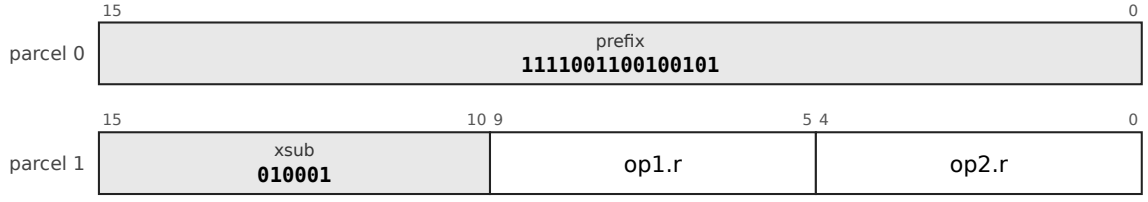

**SAR: LDF.PI12, prefix 1010010, sub-opcode 001; operands pi5-s12, reg5; 4 bytes**



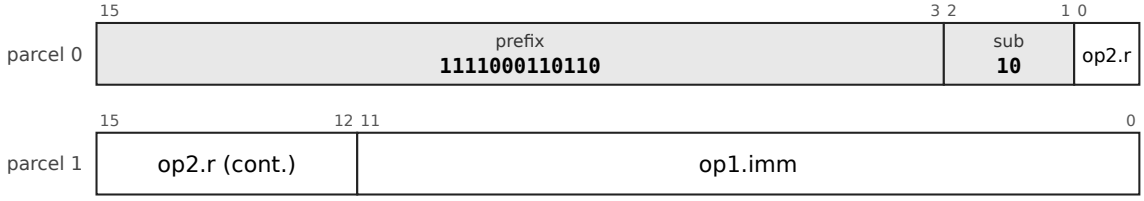

**SAR: LDF.PD12, prefix 1010011, sub-opcode 001; operands pd5-s12, reg5; 4 bytes**



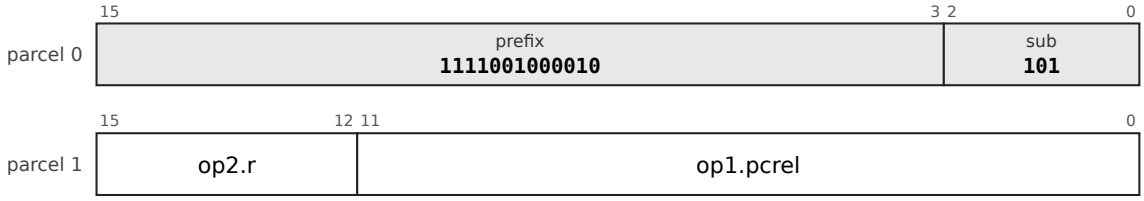

**SAR: RRF5, prefix 1111001100100101, sub-opcode 010001; operands reg5, reg5; 4 bytes**



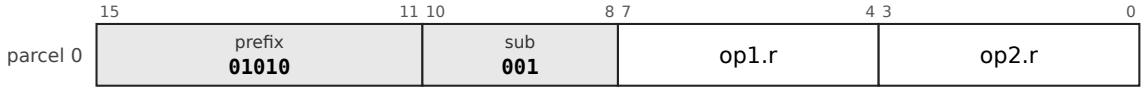
**SAR: LDI12/d, prefix 1111000110110, sub-opcode 10; operands imm12s, reg5; 4 bytes**



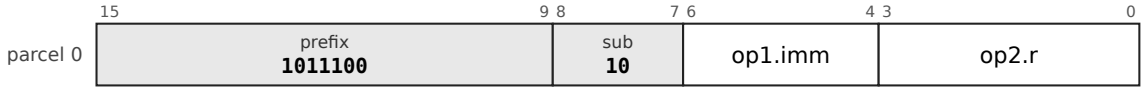

**SAR: LDPC12H/d, prefix 1111001000010, sub-opcode 101; operands pc12-u1, reg4; 4 bytes**



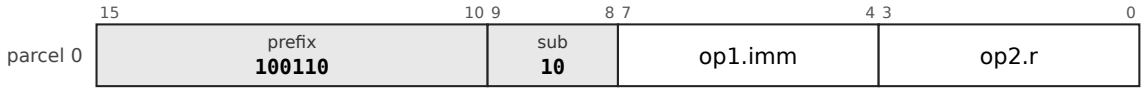

**SAR: RRF, prefix 01010, sub-opcode 001; operands reg4, reg4; 2 bytes**




**SAR: SHIh43/a, prefix 1011100, sub-opcode 10; operands imm3u, reg4; 2 bytes**




**SAR: SHIh44/a, prefix 100110, sub-opcode 10; operands imm4u, reg4; 2 bytes**

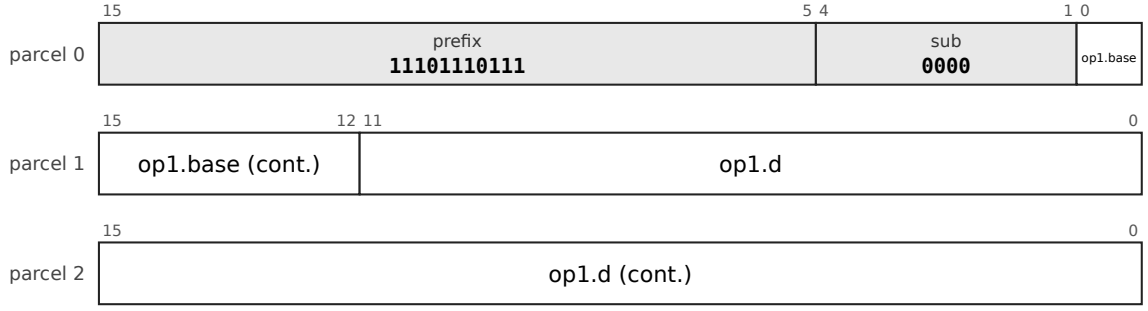



```
SAR rm, Rd
```

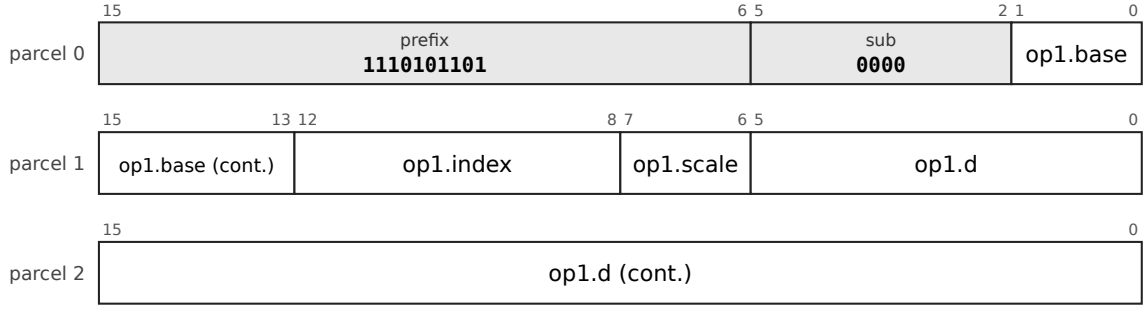
Operation, flags and exceptions as for the entry SHIFT above; 14 layout forms, each a chart of its own (above).

## CLR: Clear

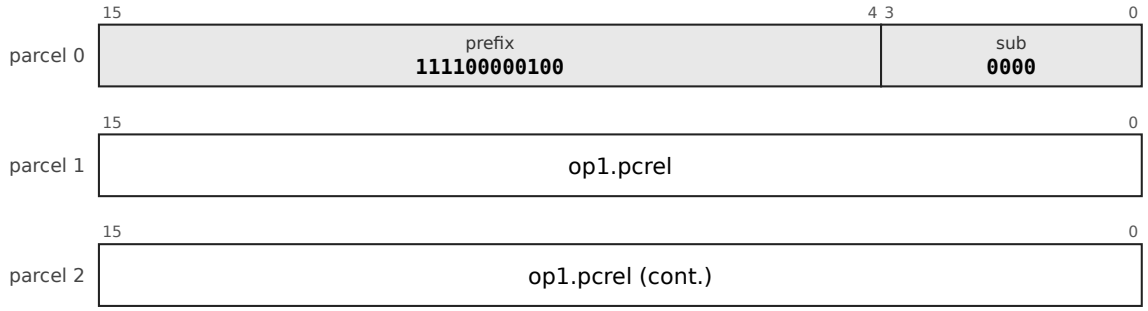
**CLR: SOF.B28, prefix 11101110111, sub-opcode 0000; operands b5d28s-x1; 6 bytes**



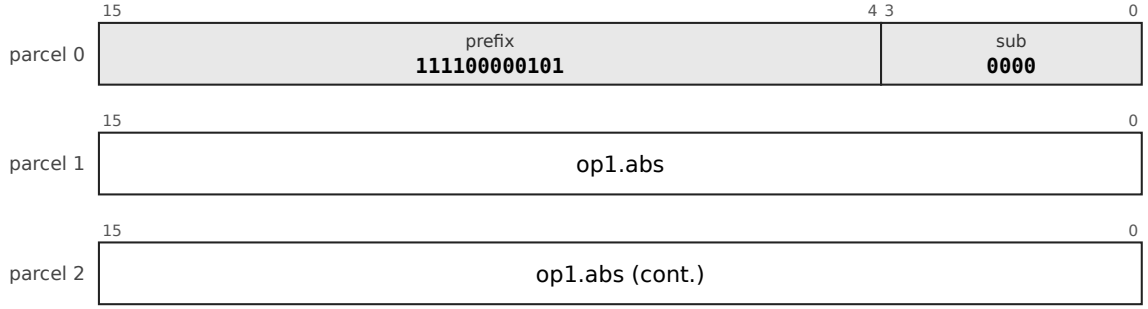

**CLR: SOF.X22, prefix 1110101101, sub-opcode 0000; operands x5-5d22s; 6 bytes**



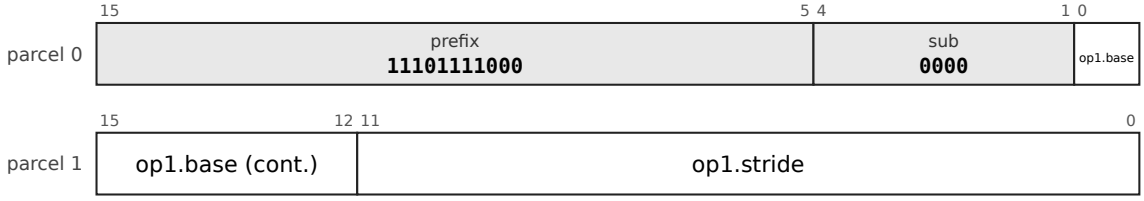

**CLR: SOF.PC32, prefix 111100000100, sub-opcode 0000; operands pc32-u1; 6 bytes**



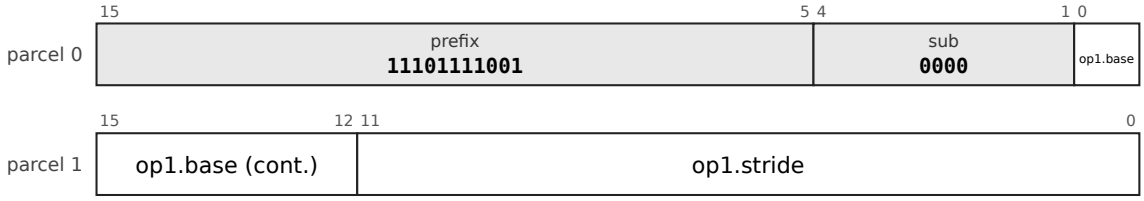

**CLR: SOF.A32, prefix 111100000101, sub-opcode 0000; operands abs32; 6 bytes**



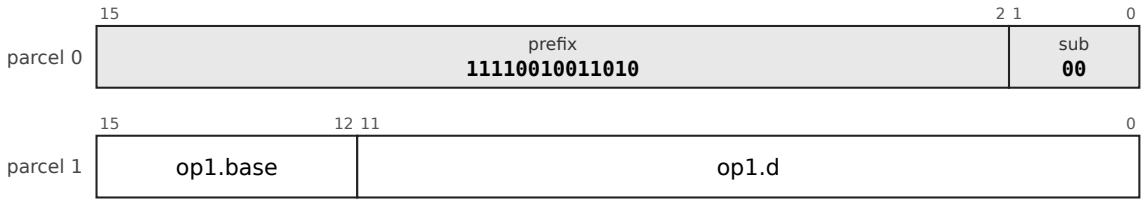
**CLR: SOF.PI12, prefix 11101111000, sub-opcode 0000; operands pi5-s12; 4 bytes**



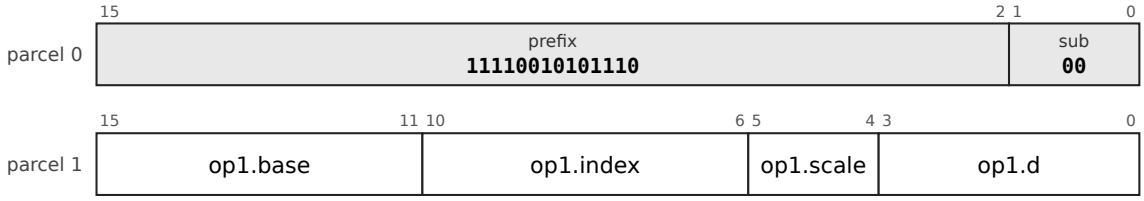

**CLR: SOF.PD12, prefix 11101111001, sub-opcode 0000; operands pd5-s12; 4 bytes**



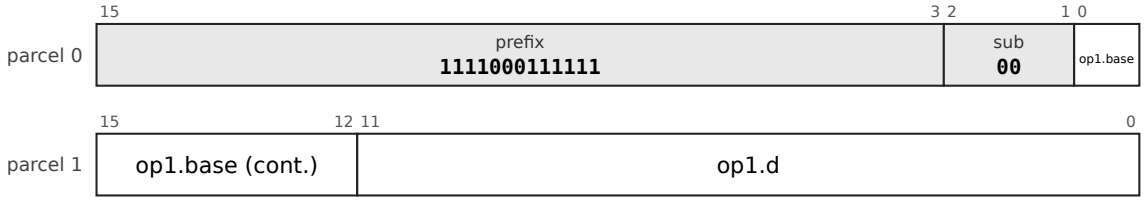

**CLR: SOBu12H/a, prefix 11110010011010, sub-opcode 00; operands b4d12u-xsz; 4 bytes**



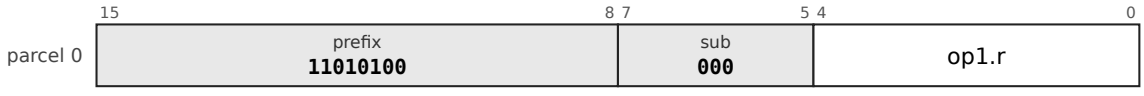

**CLR: SOX4/a, prefix 11110010101110, sub-opcode 00; operands x5-5d4s; 4 bytes**




**CLR: SOBs12/a, prefix 1111000111111, sub-opcode 00; operands b5d12s-xsz; 4 bytes**




**CLR: SOPR, prefix 11010100, sub-opcode 000; operands reg5; 2 bytes**

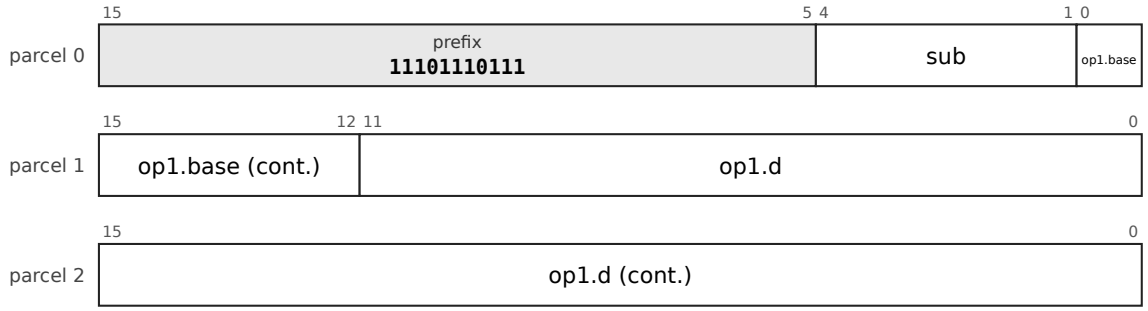



**Mnemonics:** CLR

### Syntax

```
CLR rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`, `SOBu12H-a-1`, `SOX4-a-1`, `SOBs12-a-1`.

### Operation

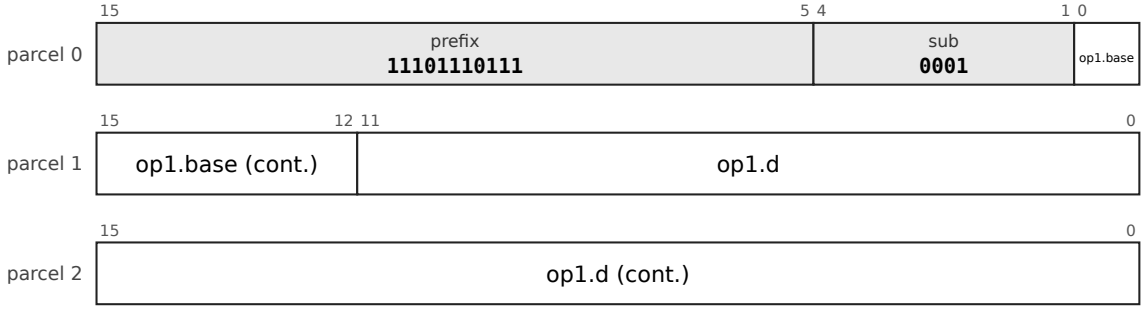
```
rm ← 0
```

**Flags** `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

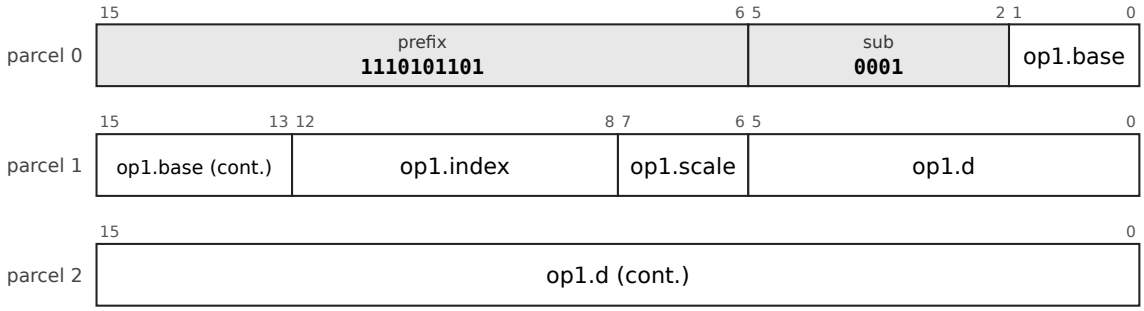
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## COM: Complement

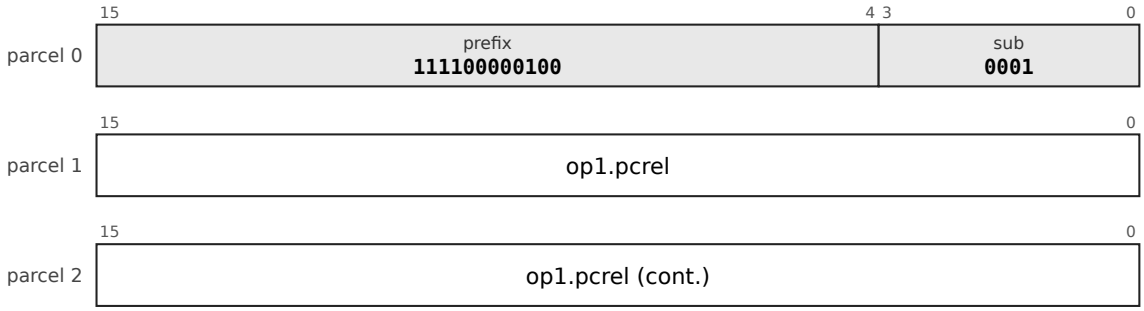
**COM: SOF.B28, prefix 11101110111, sub-opcode 0001; operands b5d28s-x1; 6 bytes**



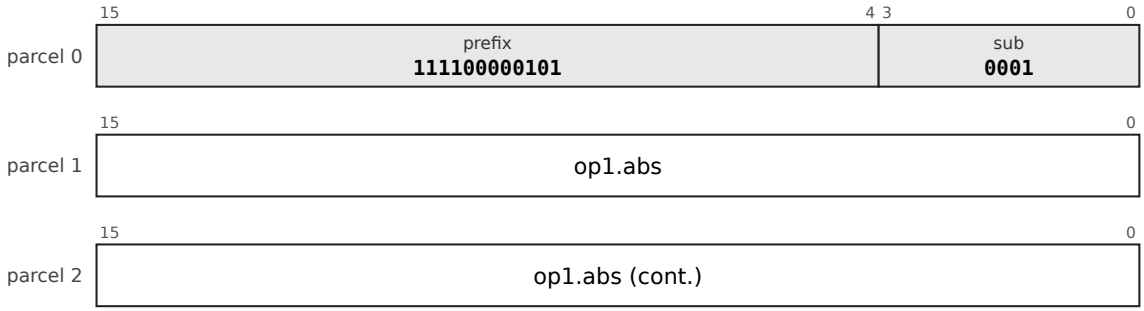

**COM: SOF.X22, prefix 1110101101, sub-opcode 0001; operands x5-5d22s; 6 bytes**



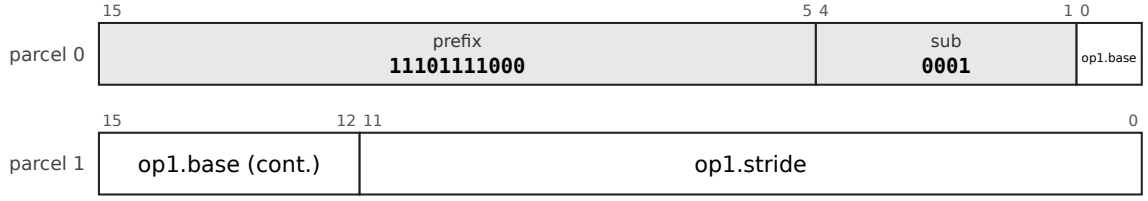

**COM: SOF.PC32, prefix 111100000100, sub-opcode 0001; operands pc32-u1; 6 bytes**



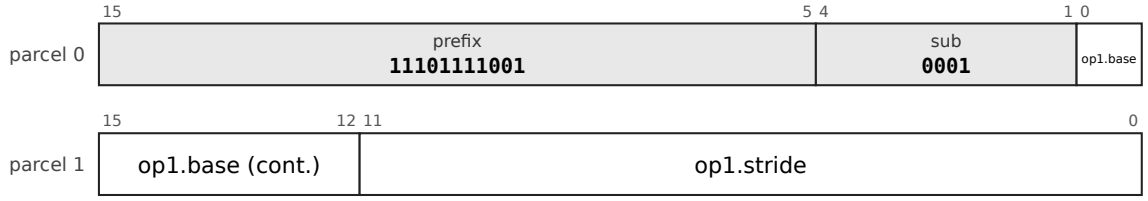

**COM: SOF.A32, prefix 111100000101, sub-opcode 0001; operands abs32; 6 bytes**



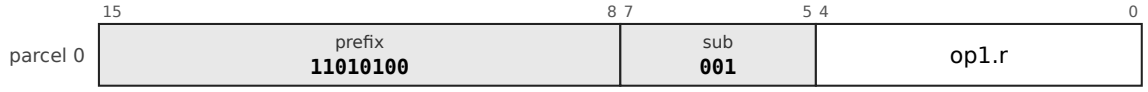
**COM: SOF.PI12, prefix 11101111000, sub-opcode 0001; operands pi5-s12; 4 bytes**




**COM: SOF.PD12, prefix 11101111001, sub-opcode 0001; operands pd5-s12; 4 bytes**




**COM: SOPR, prefix 11010100, sub-opcode 001; operands reg5; 2 bytes**

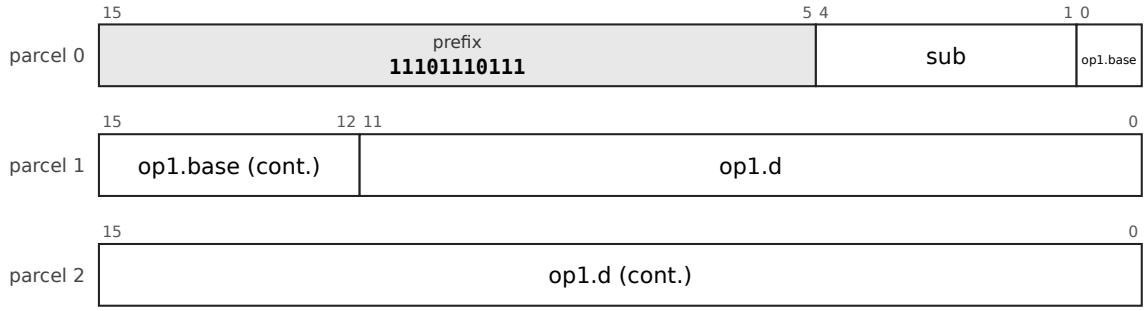



**Mnemonics:** `COM`

### Syntax

```
COM rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`.

### Operation

```
rm ← NOT rm
```

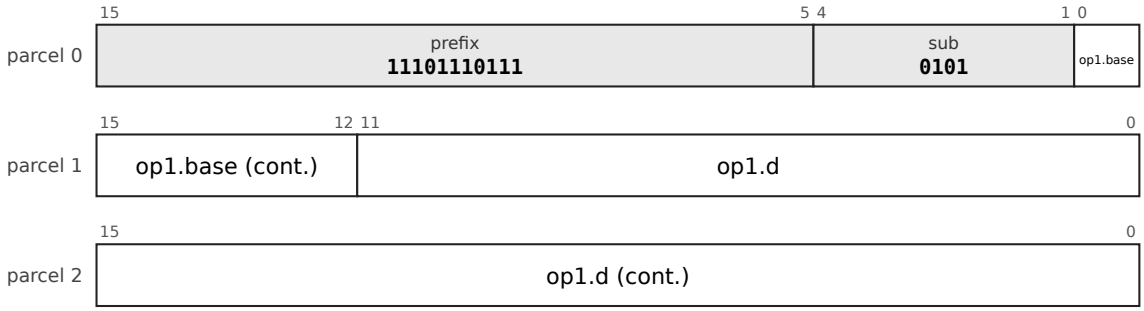
**Flags** `N` and `Z` from the result, `V` = 0, `C` = 1 (the PDP-11 rule).

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

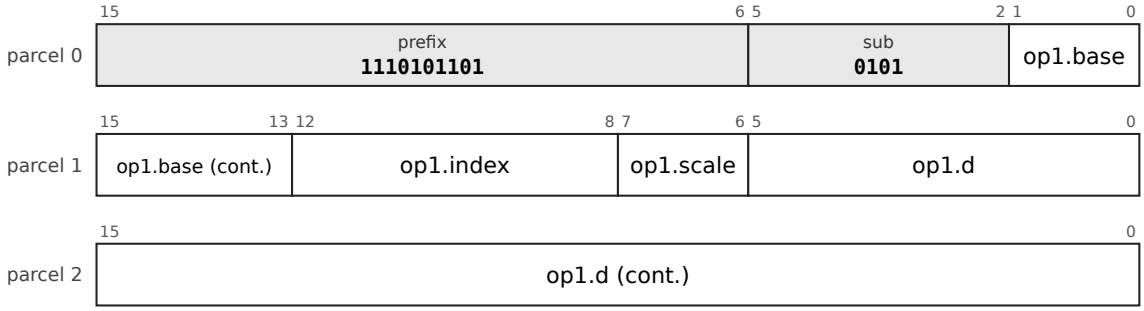
A two-byte instruction for a register operand.

## TST: Test

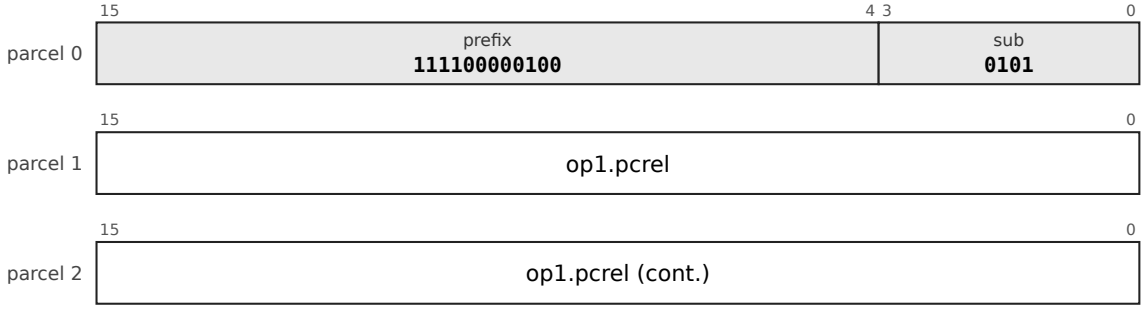
**TST: SOF.B28, prefix 11101110111, sub-opcode 0101; operands b5d28s-x1; 6 bytes**



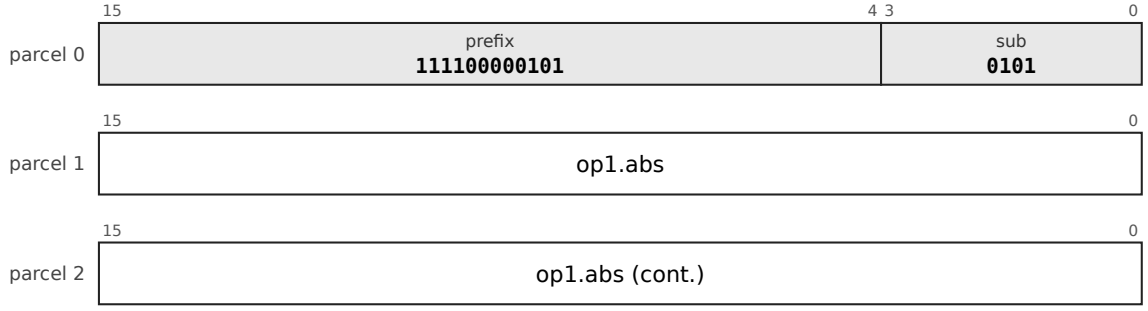

**TST: SOF.X22, prefix 1110101101, sub-opcode 0101; operands x5-5d22s; 6 bytes**



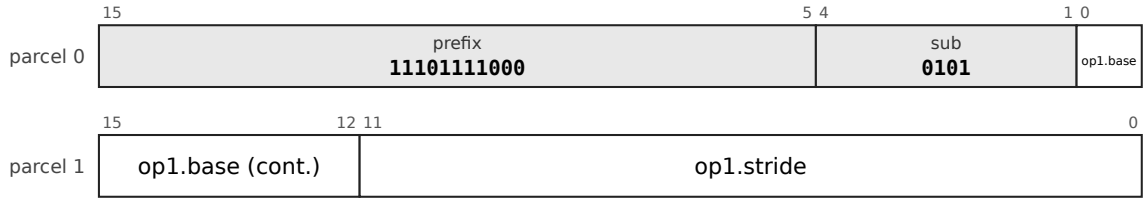

**TST: SOF.PC32, prefix 111100000100, sub-opcode 0101; operands pc32-u1; 6 bytes**



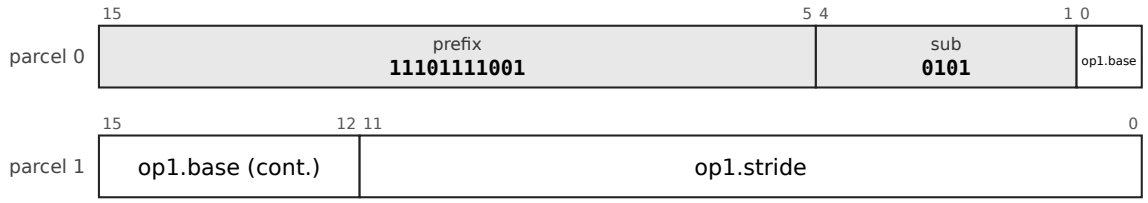
**TST: SOF.A32, prefix 111100000101, sub-opcode 0101; operands abs32; 6 bytes**



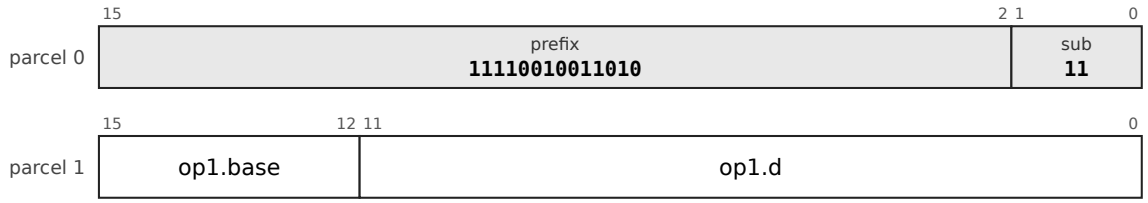

**TST: SOF.PI12, prefix 11101111000, sub-opcode 0101; operands pi5-s12; 4 bytes**



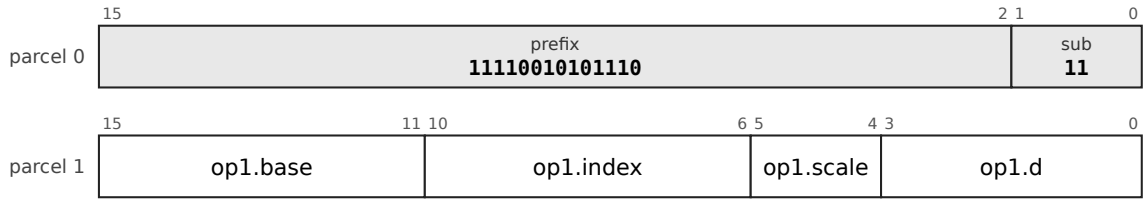

**TST: SOF.PD12, prefix 11101111001, sub-opcode 0101; operands pd5-s12; 4 bytes**



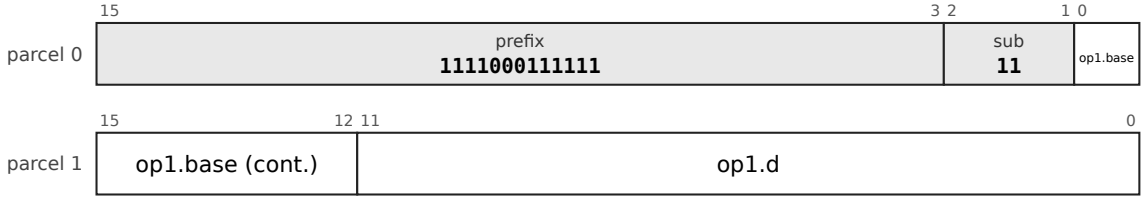

**TST: SOBu12H/a, prefix 11110010011010, sub-opcode 11; operands b4d12u-xsz; 4 bytes**



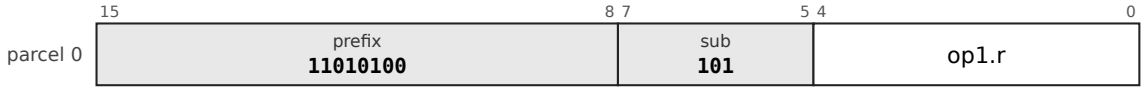

**TST: SOX4/a, prefix 11110010101110, sub-opcode 11; operands x5-5d4s; 4 bytes**



**TST: SOBs12/a, prefix 1111000111111, sub-opcode 11; operands b5d12s-xsz; 4 bytes**




**TST: SOPR, prefix 11010100, sub-opcode 101; operands reg5; 2 bytes**

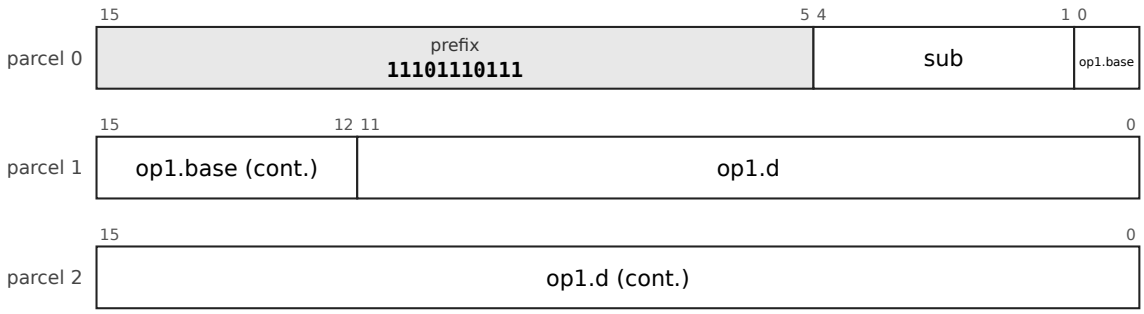



**Mnemonics:** `TST`

### Syntax

```
TST rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-1`, `SOBu12H-a-1`, `SOX4-a-1`, `SOBs12-a-1`.

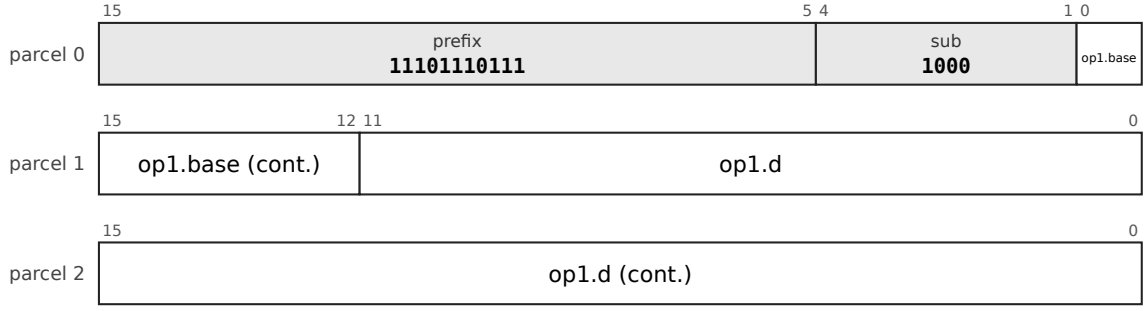
### Operation

```
set flags from rm; nothing is written
```

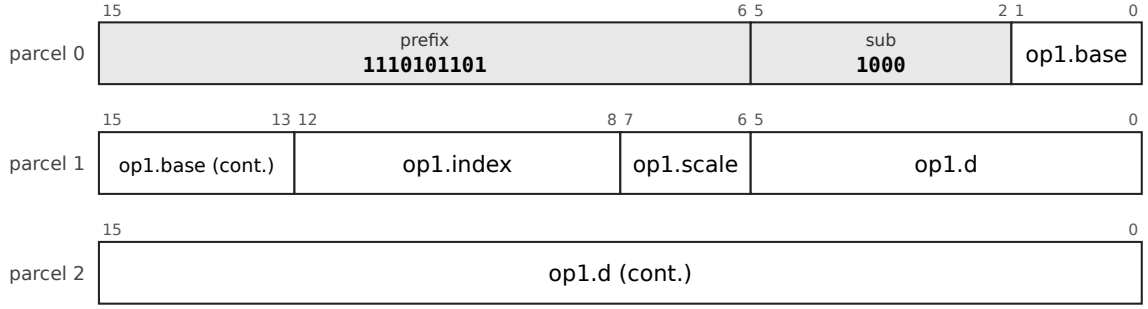
**Flags** `N` and `Z` from the operand, `V` = 0, `C` = 0.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## ASL: Arithmetic shift left by one

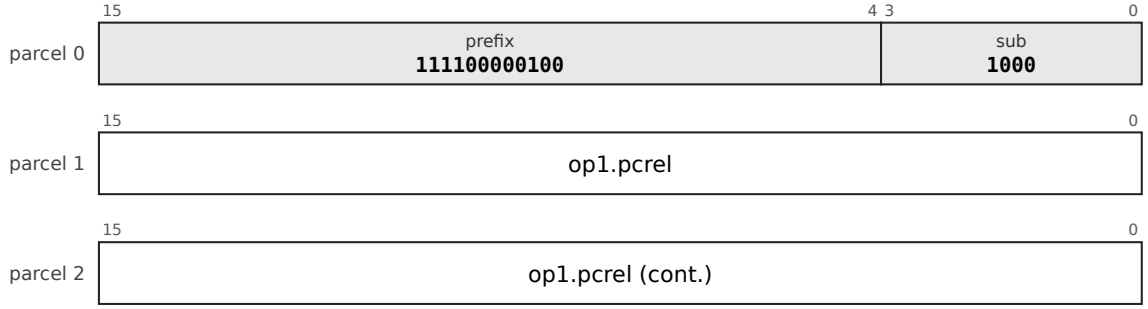
**ASL: SOF.B28, prefix 11101110111, sub-opcode 1000; operands b5d28s-x1; 6 bytes**



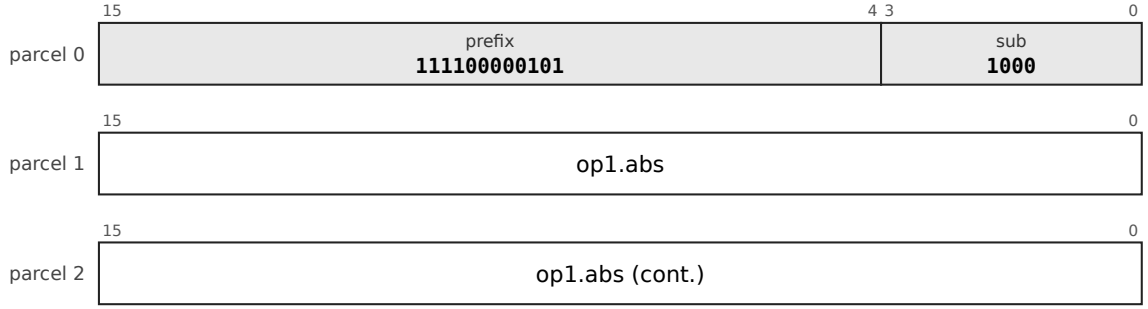

**ASL: SOF.X22, prefix 1110101101, sub-opcode 1000; operands x5-5d22s; 6 bytes**



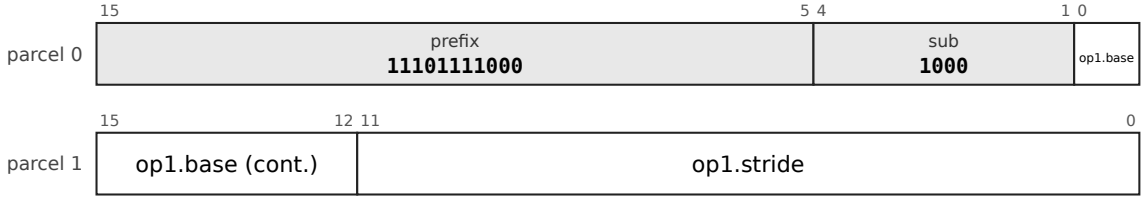

**ASL: SOF.PC32, prefix 111100000100, sub-opcode 1000; operands pc32-u1; 6 bytes**



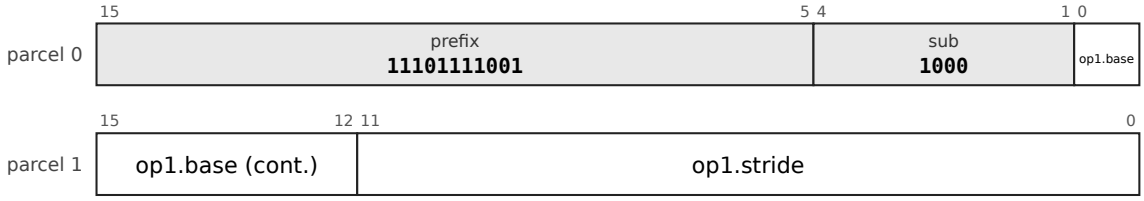

**ASL: SOF.A32, prefix 111100000101, sub-opcode 1000; operands abs32; 6 bytes**



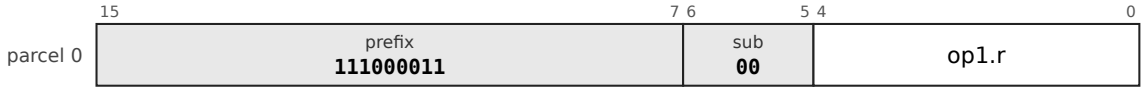
**ASL: SOF.PI12, prefix 11101111000, sub-opcode 1000; operands pi5-s12; 4 bytes**




**ASL: SOF.PD12, prefix 11101111001, sub-opcode 1000; operands pd5-s12; 4 bytes**




**ASL: SOPR, prefix 111000011, sub-opcode 00; operands reg5; 2 bytes**

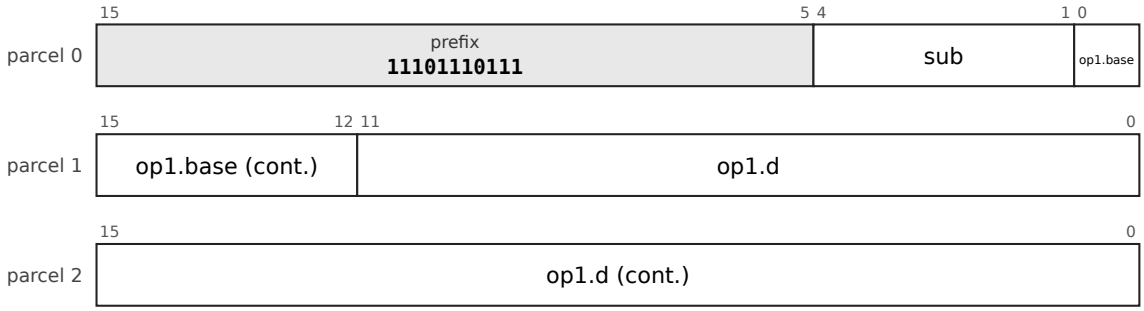



**Mnemonics:** `ASL`

### Syntax

```
ASL rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-2`.

### Operation

```
C ← rm[63] ; rm ← rm << 1
```

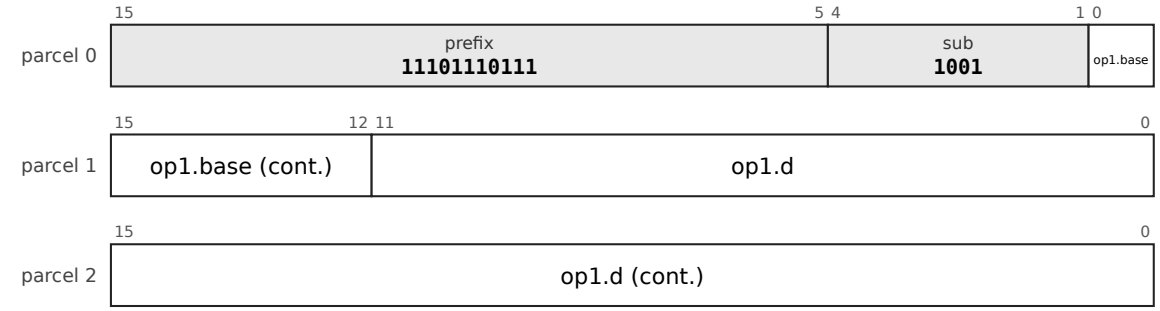
**Flags** `N` and `Z` from the result, `C` the bit shifted out of bit 63, `V` = `N` xor `C` (set if the sign changed: the PDP-11 rule).

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

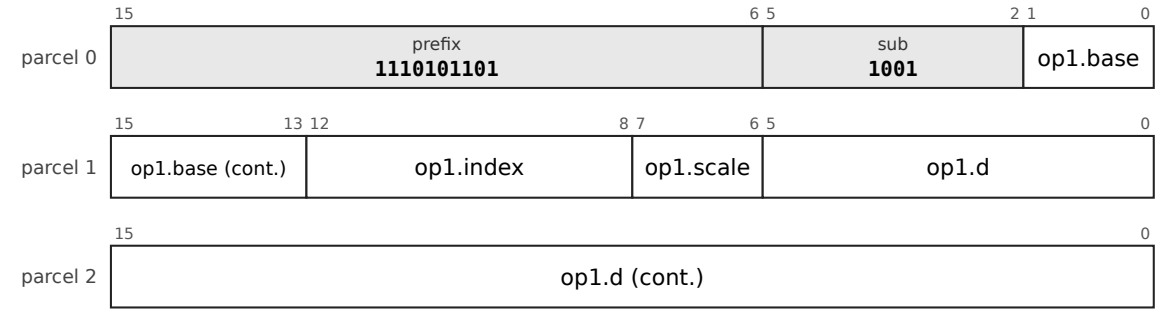
The by-one form of SHL.

## ASR: Arithmetic shift right by one

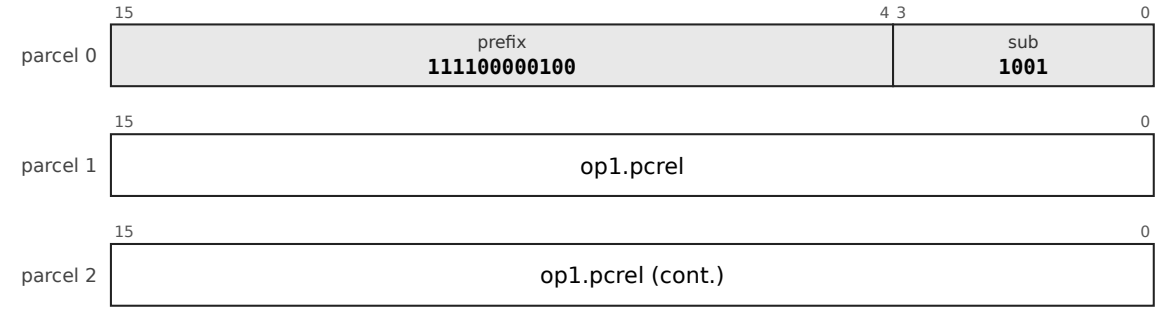
**ASR: SOF.B28, prefix 11101110111, sub-opcode 1001; operands b5d28s-x1; 6 bytes**



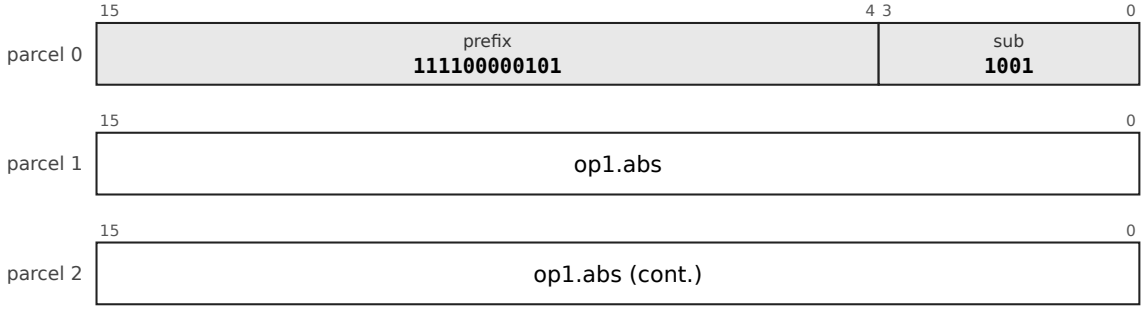

**ASR: SOF.X22, prefix 1110101101, sub-opcode 1001; operands x5-5d22s; 6 bytes**



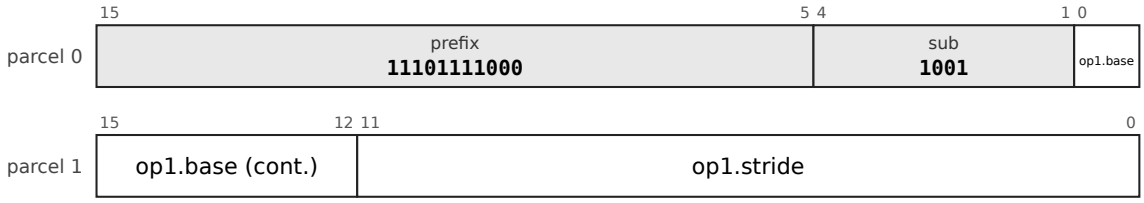

**ASR: SOF.PC32, prefix 111100000100, sub-opcode 1001; operands pc32-u1; 6 bytes**



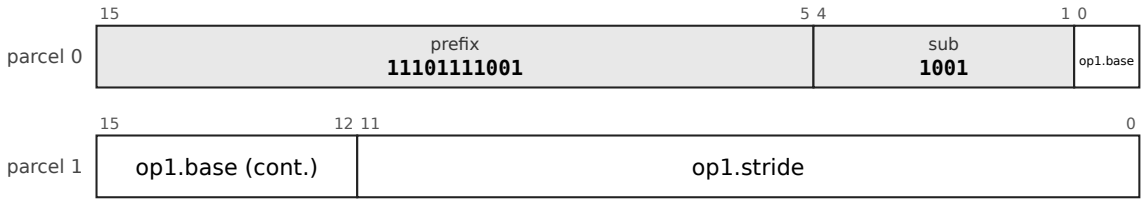
**ASR: SOF.A32, prefix 111100000101, sub-opcode 1001; operands abs32; 6 bytes**



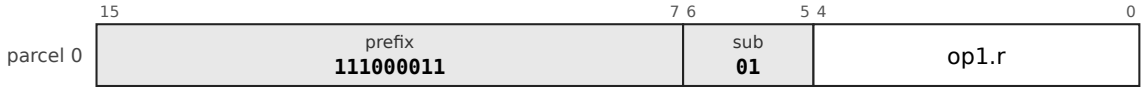

**ASR: SOF.PI12, prefix 11101111000, sub-opcode 1001; operands pi5-s12; 4 bytes**




**ASR: SOF.PD12, prefix 11101111001, sub-opcode 1001; operands pd5-s12; 4 bytes**




**ASR: SOPR, prefix 111000011, sub-opcode 01; operands reg5; 2 bytes**

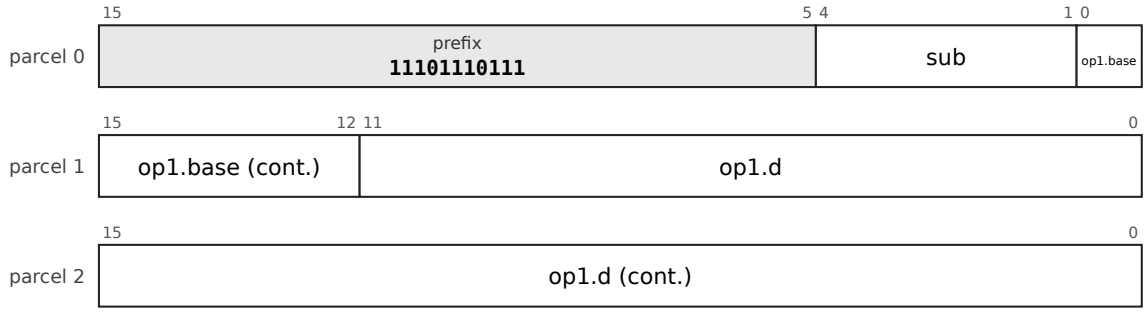



## Mnemonics: ASR

## Syntax

```
ASR rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-2`.

### Operation

```
C ← rm[0] ; rm ← rm >>s 1
```

**Flags** `N` and `Z` from the result, `C` the bit shifted out of bit 0, `V` = 0.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
The by-one form of `SAR`.

## 9.9 Bits, bytes and text

**9.9.1** The bit-and-field escape provides `CLZ`, `CTZ`, `CLO`, `CTO` (count leading and trailing zeros and ones), `POPCNT`, `BSWAP`, `BSWAPW`, `BSWAPL` and `BITREV`.

**9.9.2** `EXTB` and `EXTZB` extract a bitfield at a position and width, sign- or zero-extended, and `INSB` inserts one; the position and the width are 6-bit fields (a width of 64 cannot be written), so these are six bytes. *(D)*

**9.9.3** `XNORPOP` computes the population count of the exclusive-nor of two registers. *(D)*

**9.9.4** `CLMULL` and `CLMULH` are the low and high halves of a carry-less multiply; no polynomial is built into the architecture. *(D)*

**9.9.5** `UTF8LEN`, `UTF8DEC` and `UTF8ENC` find the length of, decode and encode a UTF-8 sequence, and reject overlong forms, surrogates, out-of-range values and bad continuation bytes. *(D)*

**9.9.6** `BSEL` selects bit by bit between two registers under the control of a third: it is `Ch` and `Maj` of SHA-2 in one instruction each, and the constant-time swap of two registers in two. *(D)*

**9.9.7** A core that has the secure mode has instructions for the AES round (`AESES`, `AESESM`, `AESDS`, `AESDSM`), for its key schedule (`AESKS1`, `AESKS2`, `AESIM`) and for the SHA-256 and SHA-512 functions (`SHA256SIG0` and the others); each takes a time independent of its operands. *(D)* Details pending (J.9.12).

*Commentary.* **Precedent.** The PDP-11 had `SWAB`; the VAX had bit-field instructions and a table-driven `CRC`; x86 added `POPCNT`, `LZCNT` and `PCLMULQDQ` late, and committed `CRC32` to one polynomial. **Trade-offs.** A bit unit is one shared structure: `POPCNT`, `XNORPOP` and a garbage collector's mark-bitmap scan all want the same 64-input adder tree, and `CLZ` and its relatives want a priority encoder over the same operand. **Improvements.** A carry-less multiply stands in for a `CRC` instruction: no polynomial is baked in, and the same instruction serves CRC, the Galois-counter-mode authentication used in TLS, and hashing. Every UTF-8 security hole of the past is three comparators, which an instruction cannot omit and software often does. **Benefits.** The bit unit and the UTF-8 decoder together cost less than 5% of the die. **Modern comparables.** RISC-V's bit-manipulation and carry-less-multiply extensions, AArch64's `CLZ`, `RBIT`, `REV` and `PMULL`, and x86's `PCLMULQDQ`. **Use.** `POPCNT` for bitsets and Hamming distance; `XNORPOP` for binary-weight neural networks and bitset intersection; `CLMUL` for CRC, GCM and fingerprinting; `UTF8*`

for text validation in parsers and kernels. **Impact.** *Kernel:* CTZ and CLZ find the first set bit in a run-queue or free-page bitmap; POPCNT counts bits in bitmaps; BSWAP converts network byte order. *Compiler:* The compiler maps __builtin_clz, __builtin_popcount and __builtin_bswap directly, and a carry-less multiply is the primitive for CRC and hashing code. *Hardware:* One bit unit holds a shared 64-input adder tree and a priority encoder, and the UTF-8 comparators are about 300 lookup tables.

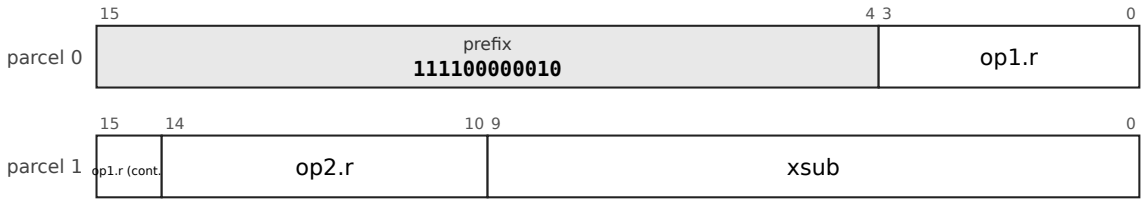
## COUNT: Count leading and trailing zeros and ones

**Mnemonics:** CLZ, CTZ, CLO, CTO

### Syntax

```
CLZ Rs, Rd
CTZ Rs, Rd
CLO Rs, Rd
CTO Rs, Rd
```

### Encoding




The chart is the leaf R2-1, R2, prefix 111100000010. op1 is reg5: register, 5 bits. op2 is reg5: register, 5 bits. xsub selects the mnemonic. **All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): R2-1.

### Operation

```
CLZ: Rd ← number of leading zero bits of Rs        (64 if Rs = 0)
CTZ: Rd ← number of trailing zero bits of Rs       (64 if Rs = 0)
CLO: Rd ← number of leading one bits of Rs         (64 if Rs = all ones)
CTO: Rd ← number of trailing one bits of Rs        (64 if Rs = all ones)
```

**Flags** None written.

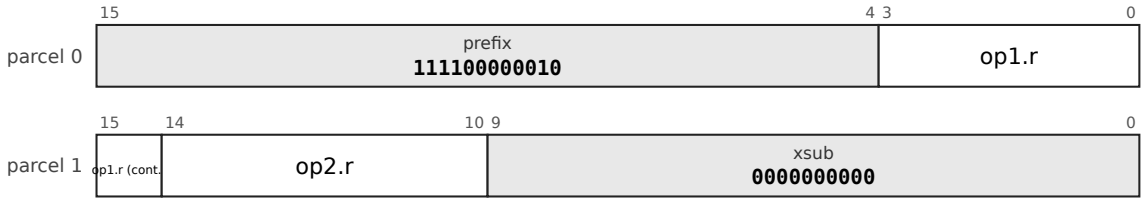
**Exceptions** None.

CTZ is the VAX FFS. Bit manipulation is core on a machine that does filesystem and bignum work, not an extension.

**Why it is here.** Bit manipulation is not an extension on a machine that does filesystem work, bignum arithmetic and the like. The counts are the VAX FFS and FFC idiom (find first set, find first clear), and LoongArch, which has CLO, CLZ, CTO and CTZ in its base, is the model for putting them there. The compiler emits none of them, since C has no operator for them; the corpus shows zero uses.

### *CLZ*

**CLZ: R2, prefix 111100000010, sub-opcode 0000000000; operands reg5, reg5; 4 bytes**

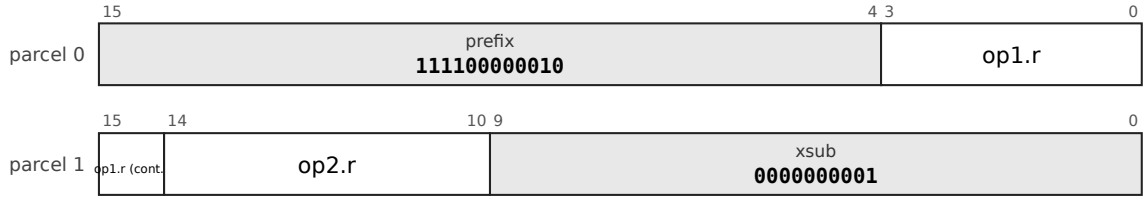



```
CLZ Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

### *CTZ*

**CTZ: R2, prefix 111100000010, sub-opcode 0000000001; operands reg5, reg5; 4 bytes**

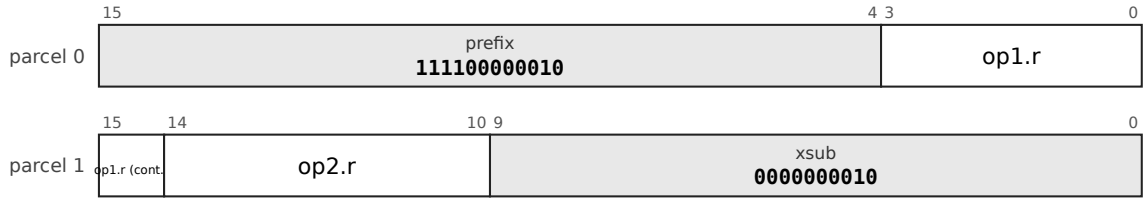



```
CTZ Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

### *CLO*

**CLO: R2, prefix 111100000010, sub-opcode 0000000010; operands reg5, reg5; 4 bytes**

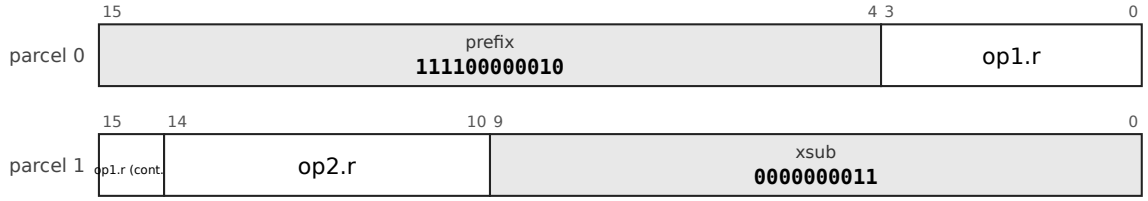



```
CLO Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

### *CTO*

**CTO: R2, prefix 111100000010, sub-opcode 0000000011; operands reg5, reg5; 4 bytes**

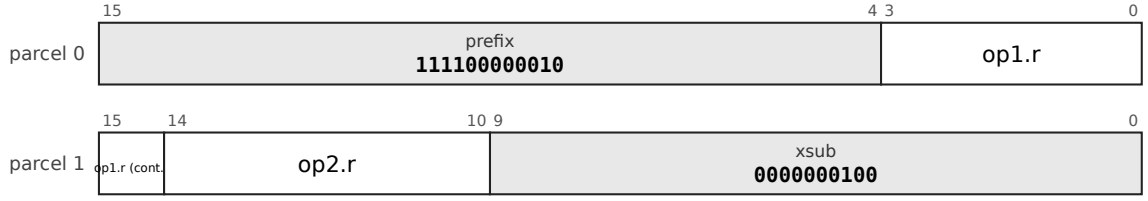



```
CTO Rs, Rd
```

Operation, flags and exceptions as for the entry COUNT above; 1 layout form, each a chart of its own (above).

## POPCNT: Population count

**POPCNT: R2, prefix 111100000010, sub-opcode 0000000100; operands reg5, reg5; 4 bytes**

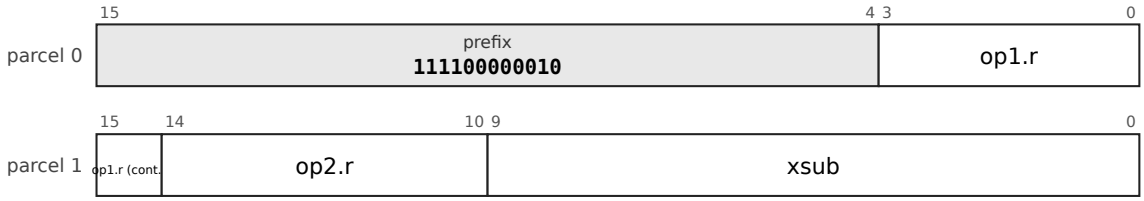



**Mnemonics:** `POPCNT`

### Syntax

```
POPCNT Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
Rd ← number of one bits in Rs
```

**Flags** None written.
**Exceptions** None.
**Why it is here.** One of the counts; it is also the building block of `XNORPOP`. A compiler needs `__builtin_popcount` to emit it, so the corpus shows no use (see `COUNT`).

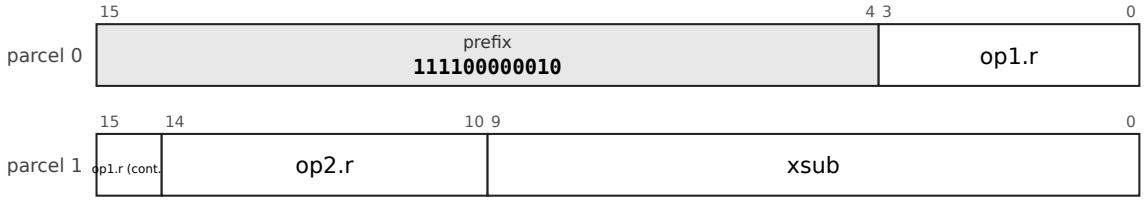
## BSWAP: Byte swap

**Mnemonics:** `BSWAP`, `BSWAPW`, `BSWAPL`

### Syntax

```
BSWAP Rs, Rd
BSWAPW Rs, Rd
BSWAPL Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
BSWAP : Rd ← Rs with all 8 bytes reversed
BSWAPW: Rd ← Rs with the bytes of each 16-bit word reversed
BSWAPL: Rd ← Rs with the bytes of each 32-bit long reversed
```

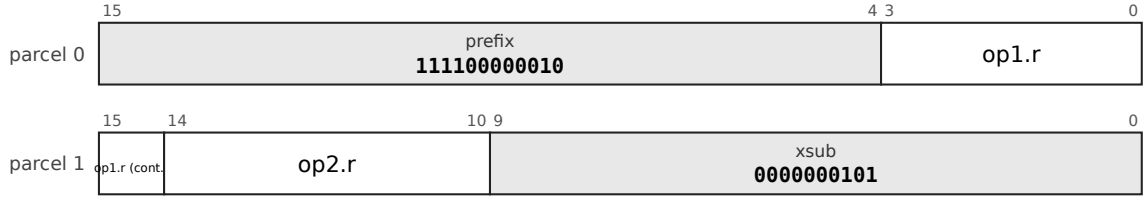
**Flags** None written.
**Exceptions** None.

The generalisation of the PDP-11 SWAB to 64 bits.

**Why it is here.** The PDP-11 SWAB swapped the bytes of a word; BSWAP generalises it to 64 bits, with word and long variants. SWAB is therefore subsumed, not lost (9.18).

### *BSWAP*

**BSWAP: R2, prefix 111100000010, sub-opcode 0000000101; operands reg5, reg5; 4 bytes**

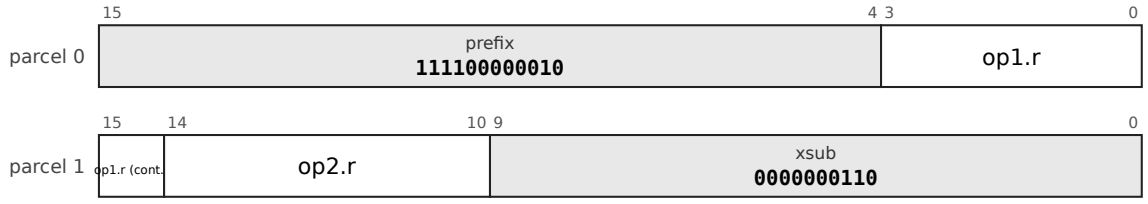



```
BSWAP Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

### *BSWAPW*

**BSWAPW: R2, prefix 111100000010, sub-opcode 0000000110; operands reg5, reg5; 4 bytes**

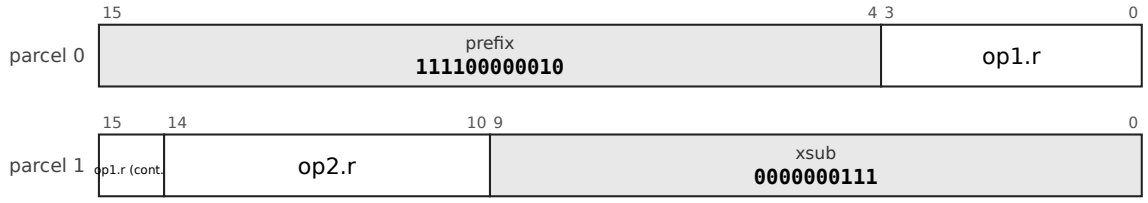



```
BSWAPW Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

### *BSWAPL*

**BSWAPL: R2, prefix 111100000010, sub-opcode 0000000111; operands reg5, reg5; 4 bytes**

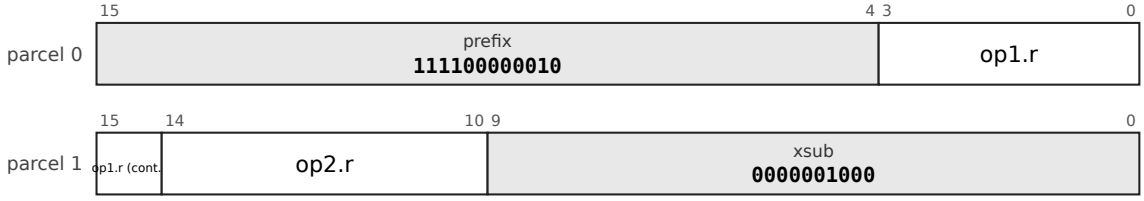



```
BSWAPL Rs, Rd
```

Operation, flags and exceptions as for the entry BSWAP above; 1 layout form, each a chart of its own (above).

## BITREV: Reverse the bits

**BITREV: R2, prefix 111100000010, sub-opcode 0000001000; operands reg5, reg5; 4 bytes**

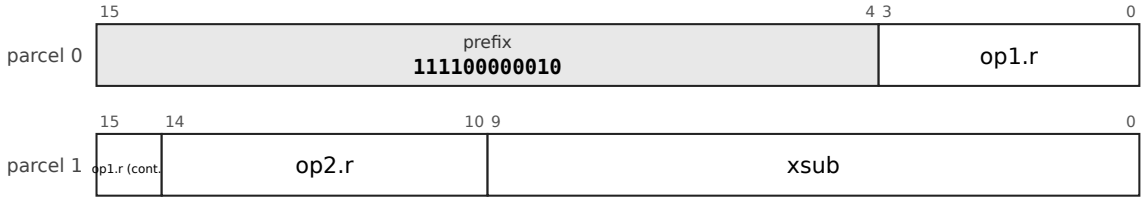



**Mnemonics:** BITREV

### Syntax

```
BITREV Rs, Rd
```

### Encoding




The chart is the leaf R2-1, R2, prefix 111100000010. op1 is reg5: register, 5 bits. op2 is reg5: register, 5 bits. xsub selects the mnemonic. **All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): R2-1.

### Operation

```
Rd ← Rs with bit i moved to bit 63-i
```

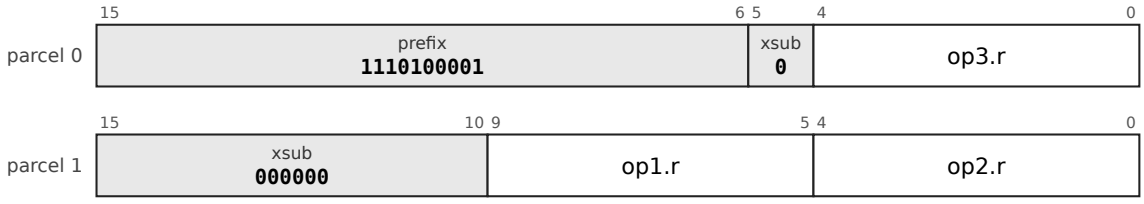
**Flags** None written.

**Exceptions** None.

**Why it is here.** Borrowed with the other LoongArch bit operations (REVB, REVH, BITREV). No workload from the corpus is recorded.

## ROR: Rotate right by a register count

**ROR: R3, prefix 1110100001, sub-opcode 0000000; operands reg5, reg5, reg5; 4 bytes**

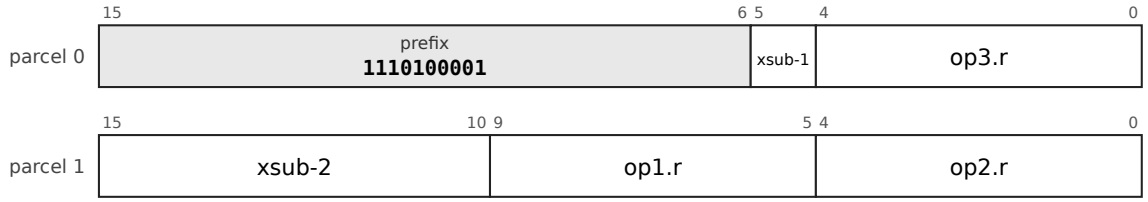



**Mnemonics:** ROR

### Syntax

```
ROR Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
Rd ← Ra rotated right by (Rb mod 64)
```

**Flags** None written (rotates write no flags).

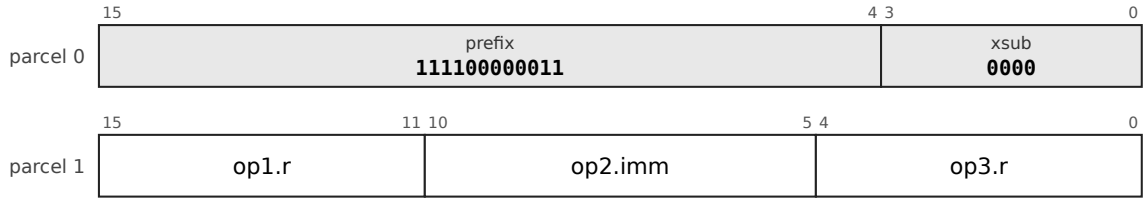
**Exceptions** None.

There is no rotate left instruction: `ROL rs, #n, rd` is an assembler spelling of `RORI rs, #(64-n), rd`. No rotate is through carry; multi-word shifting uses `DSHR`.

**Why it is here.** One rotate direction suffices and `ROR` is the one to keep: `ROL x, #n` is exactly `ROR x, #(64-n)`, and a funnel shifter yields `ROR` directly. No rotate is through carry (see `DSHR`). `ROR` is a bit operation and not an ALU operation: cryptographic code rotates by compile-time constants, so an immediate form is needed either way, and the register-count form has almost no users. Rotates write no flags.

## RORI: Rotate right by an immediate

**RORI: RRI, prefix 111100000011, sub-opcode 0000; operands reg5, imm6u, reg5; 4 bytes**

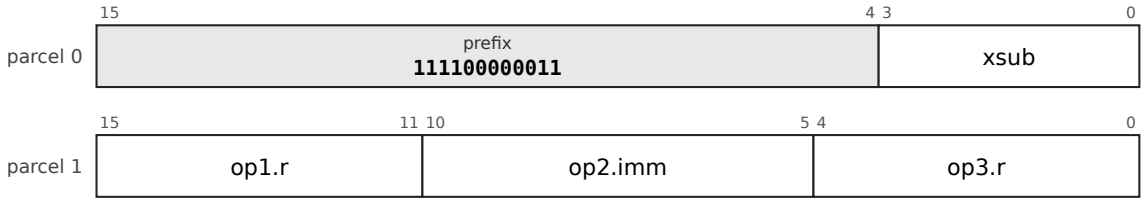



**Mnemonics:** `RORI` (aliases: `ROL`)

### Syntax

```
RORI Rs, #n, Rd
ROL Rs, #n, Rd        ; assembles to RORI Rs, #(64-n), Rd
```

### Encoding




The chart is the leaf `RRI-1`, RRI, prefix `111100000011`. `op1` is `reg5`: register, 5 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `RRI-1`.

### Operation

```
Rd ← Rs rotated right by (n mod 64)
```

**Flags** None written.

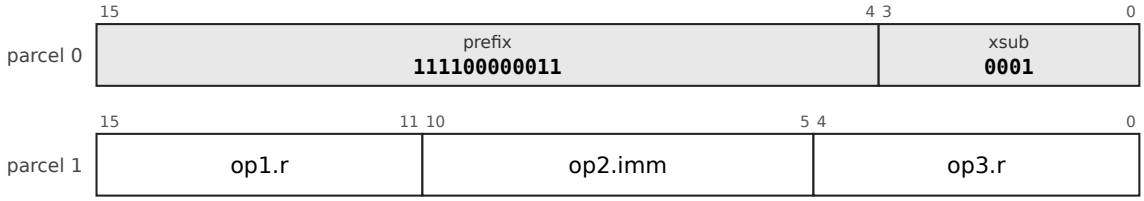
**Exceptions** None.

**Why it is here.** It has its own mnemonic because an immediate count and a register count are different encodings. `ROL Rs, #n, Rd` is accepted by the assembler as `RORI Rs, #(64-n), Rd`: the complement costs nothing at assembly time, so there is no reason to make programmers do it, and a register-count `ROL` is rejected so that one mnemonic stays one instruction.

**`ROL`** is an alias of `RORI`: the assembler writes it as that word.

## RORI32: Rotate right by an immediate, 32 bits

**RORI32: RRI, prefix 111100000011, sub-opcode 0001; operands reg5, imm6u, reg5; 4 bytes**

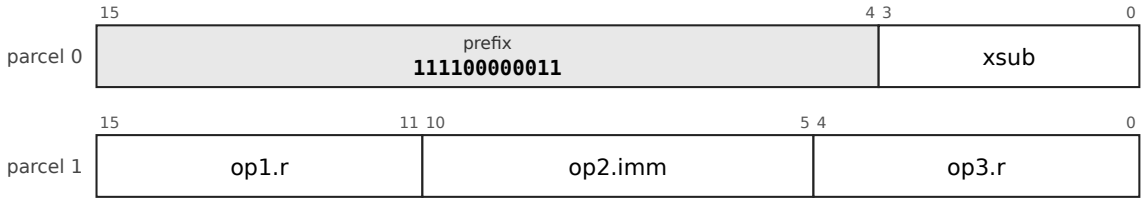



**Mnemonics:** `RORI32`

### Syntax

```
RORI32 Rs, #n, Rd
```

### Encoding




The chart is the leaf `RRI-1`, RRI, prefix `111100000011`. `op1` is `reg5`: register, 5 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `RRI-1`.

### Operation

```
Rd ← sext32(ror32(Rs[31:0], n mod 32))
```

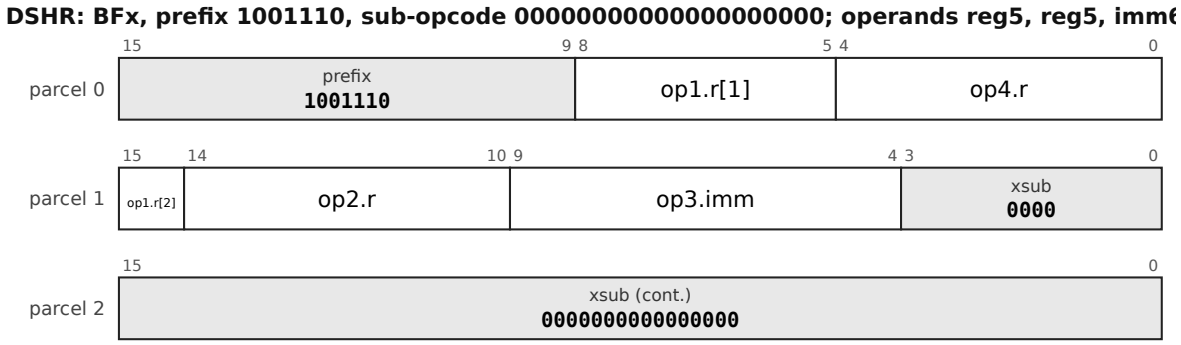
**Flags** None written.

**Exceptions** None.

The 32-bit rotate with the result sign-extended like every 32-bit result; the primitive of most hash functions and ciphers.

**Why it is here.** A 32-bit rotate by a constant, the shape of crypto and hash inner loops, estimated together with CBCC at 22 KB (1.3%) of the compiled image. The result is sign-extended like the rest of the 32-bit operations, so that no instruction is needed after it. It removes 44 MOVL instructions in the corpus (the demand analysis already removes nearly all of them).

## DSHR: Double-width funnel shift right

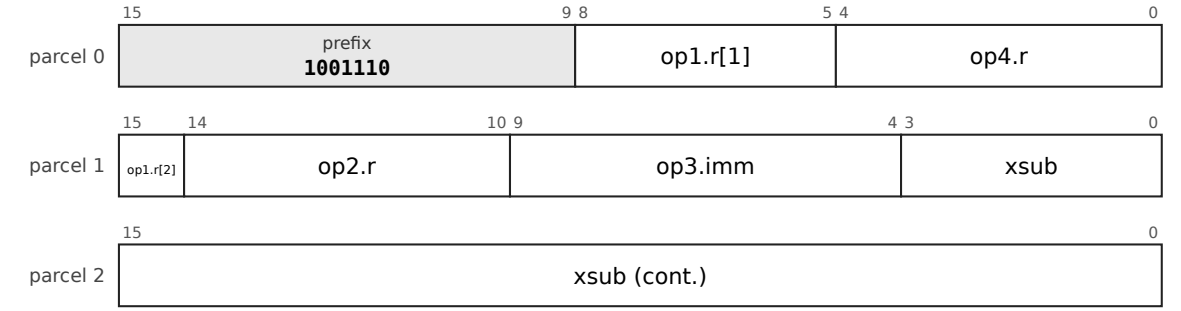



### Mnemonics: DSHR

### Syntax

```
DSHR Ra, Rb, #n, Rd
```

### Encoding




The chart is the leaf BFx-1, BFx, prefix 1001110. op1 is reg5: register, 5 bits. op2 is reg5: register, 5 bits. op3 is imm6u: unsigned 6-bit immediate, 0 to 63. op4 is reg5: register, 5 bits. xsub selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): BFx-1.

### Operation

```
t ← Ra:Rb                   ; 128 bits, Ra the high half
Rd ← (t >> n)[63:0]         ; n = 0..63
```

**Flags** None written.

**Exceptions** None.

x86 SHRD, ARM EXTR. A 128-bit left shift by k is `DSHR hi, hi, lo, #(64-k)` then `SHL #k, lo`.

**Why it is here.** A double-width funnel shift (x86 SHRD, ARM EXTR), the right primitive for multi-word shifting. Two instructions shift a 128-bit value by any distance, where rotate-through-carry takes one bit per instruction, and that is why no rotate is through carry. It also subsumes ROR when both sources are the same register. It leaves every flag alone.

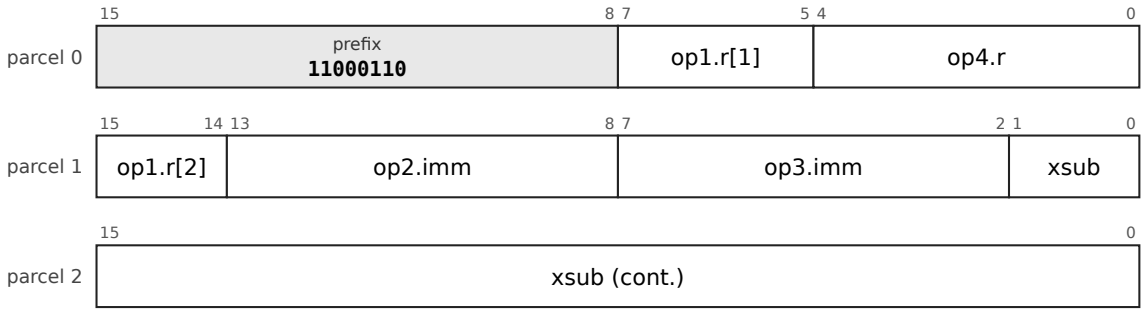
## EXTB: Extract a bit field

**Mnemonics:** EXTB, EXTZB

### Syntax

```
EXTB Rs, #pos, #width, Rd        ; sign-extended
EXTZB Rs, #pos, #width, Rd       ; zero-extended
```

### Encoding




The chart is the leaf BFy-1, BFy, prefix 11000110. op1 is reg5: register, 5 bits. op2 is imm6u: unsigned 6-bit immediate, 0 to 63. op3 is imm6u: unsigned 6-bit immediate, 0 to 63. op4 is reg5: register, 5 bits. xsub selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): BFy-1.

### Operation

```
Rd ← ext((Rs >> pos)[width-1:0])          ; sext for EXTB, zext for EXTZB; width 0 or > 64 means 64
```

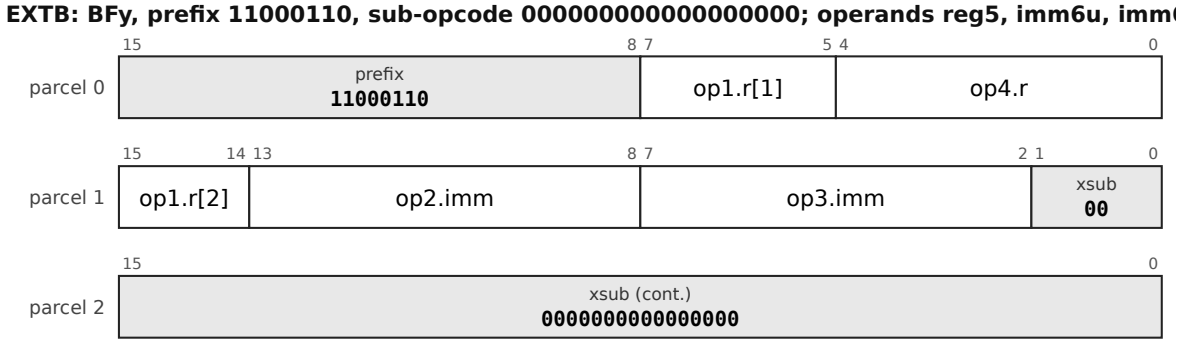
**Flags** None written.

**Exceptions** None.

Position and width are immediate operands; a width of 64 cannot be encoded (errata 87, assumption 8).

**Why it is here.** Bit-field extract and insert are the VAX EXTV, EXTZV and INSV idiom. The compiler does not use them: the shift-pair extraction pattern occurs 151 times in 26 files of the corpus, about 600 bytes at best (0.03%), and there are four true bit-field extracts in all. They are for hand-written protocol and bit-stream code.

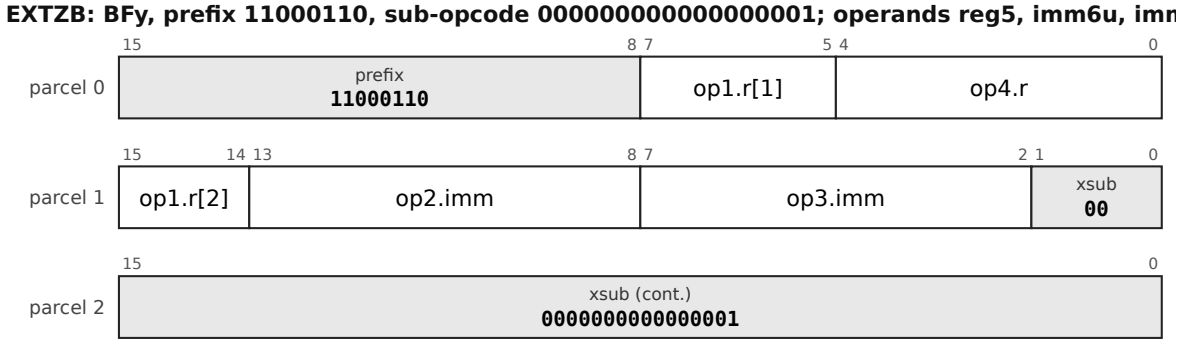
### *EXTB*




```
EXTB Rs, #pos, #width, Rd
```

Operation, flags and exceptions as for the entry EXTB above; 1 layout form, each a chart of its own (above).

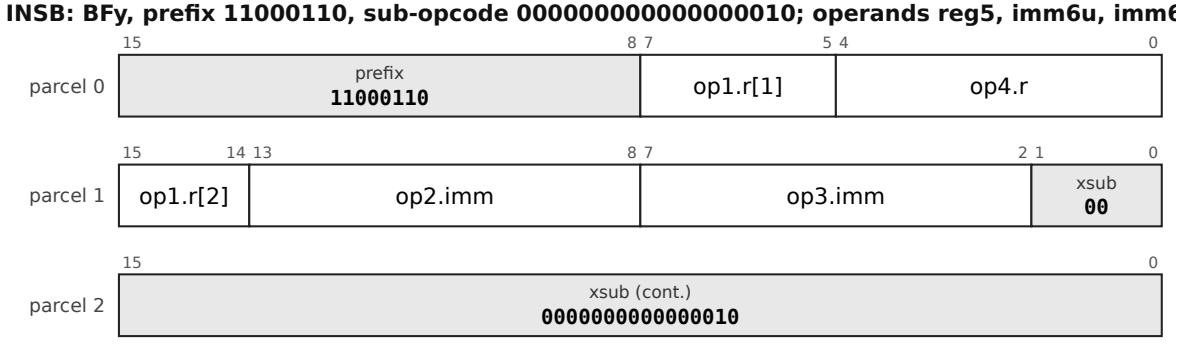
### *EXTZB*




```
EXTZB Rs, #pos, #width, Rd
```

Operation, flags and exceptions as for the entry EXTB above; 1 layout form, each a chart of its own (above).

## INSB: Insert a bit field

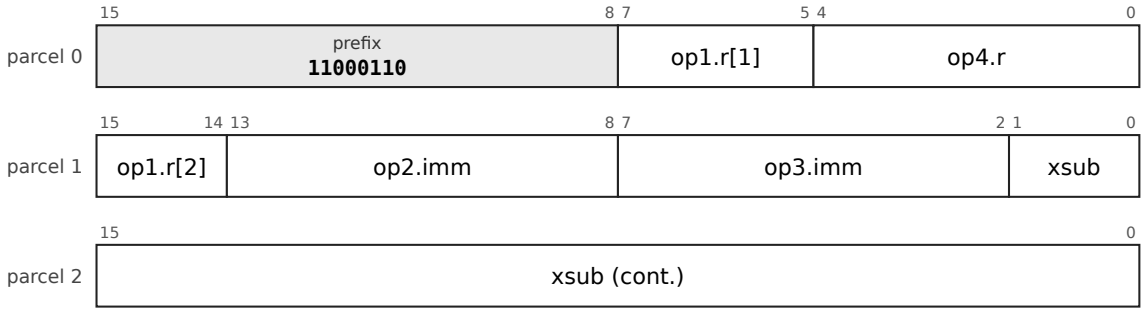



**Mnemonics:** `INSB`

**Syntax**

```
INSB Rs, #pos, #width, Rd
```

### Encoding




The chart is the leaf `BFy-1`, BFy, prefix `11000110`. `op1` is `reg5`: register, 5 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op4` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `BFy-1`.

### Operation

```
mask ← ((1 << width) - 1) << pos
Rd ← (Rd AND NOT mask) OR ((Rs << pos) AND mask)
```

**Flags** None written.

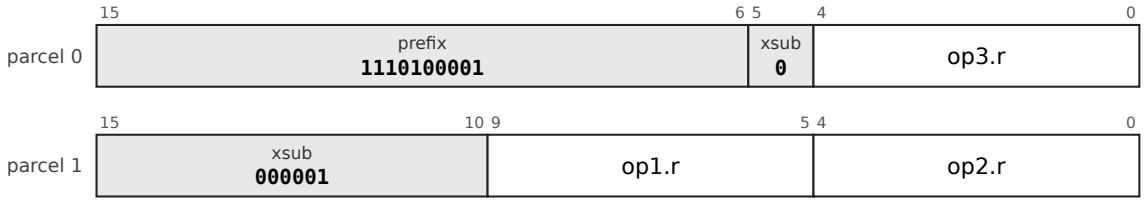
**Exceptions** None.

Six bytes. `Rd` is both source and destination.

**Why it is here.** The VAX `INSV` idiom, with `EXTB`. No use in the compiled corpus is recorded.

## XNORPOP: Population count of the equality of two words

**XNORPOP: R3, prefix 1110100001, sub-opcode 0000001; operands reg5, reg5, reg5; 4 bytes**

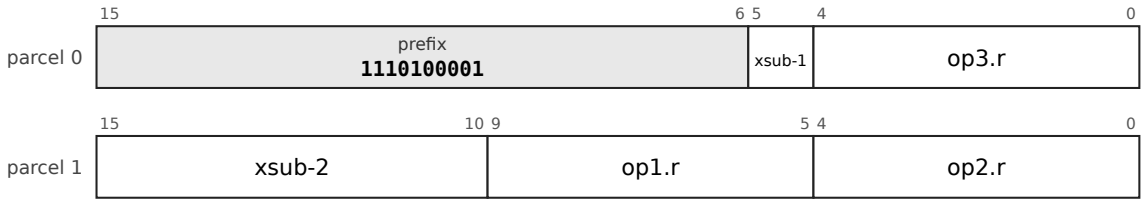



**Mnemonics:** `XNORPOP`

### Syntax

```
XNORPOP Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
Rd ← popcount(NOT (Ra XOR Rb))
```

**Flags** None written.

**Exceptions** None.

For binary-activation networks, Hamming distance and bitset intersection cardinality. It does **not** serve sub-1-bit language models (section 8.5).

**Why it is here.** It serves binary-activation networks, Hamming distance, bitset intersection counts and counting a collector's mark bitmap. It does not serve sub-1-bit language models: their activations (in the LittleBit work) are FP16 and the inner loop is a conditional add, not a popcount. The sub-1-bit kernel is bandwidth-bound and belongs to a bus-mastering accelerator, not an opcode (chapter 8).

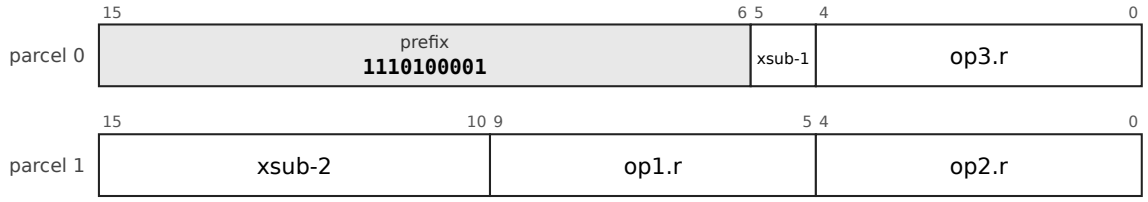
## CLMUL: Carry-less multiply

**Mnemonics:** `CLMULL`, `CLMULH`

### Syntax

```
CLMULL Ra, Rb, Rd
CLMULH Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
p ← carry-less (polynomial over GF(2)) product of Ra and Rb      ; 128 bits
CLMULL: Rd ← p[63:0]
CLMULH: Rd ← p[127:64]
```

**Flags** None written.

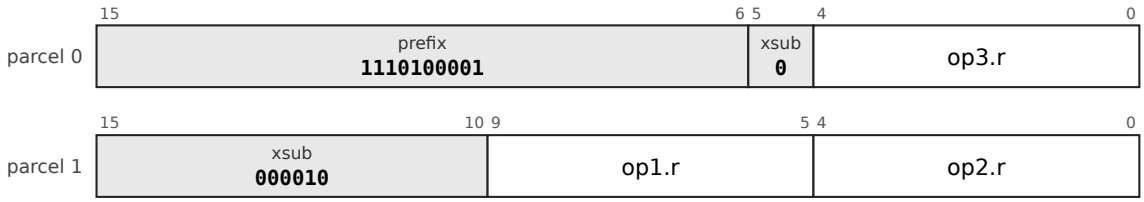
**Exceptions** None.

The primitive under every CRC (with a Barrett reduction), GF(2^k) arithmetic and GCM; no polynomial is committed to the architecture.

**Why it is here.** Carry-less multiply is the primitive beneath every CRC polynomial (a CRC is a carry-less multiply plus a Barrett reduction), GF(2^k) erasure codes and AES-GCM. It is chosen over a `CRC` instruction because x86 and ARM each committed a polynomial to the architecture; there are no `CRC32` or `CRC32C` words.

### *CLMULL*

**CLMULL: R3, prefix 1110100001, sub-opcode 0000010; operands reg5, reg5, reg5; 4 bytes**

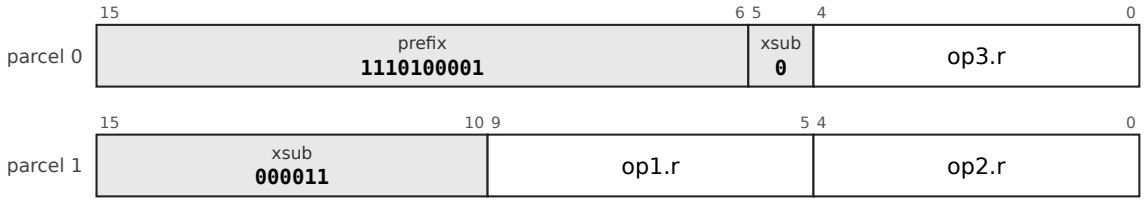



```
CLMULL Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry CLMUL above; 1 layout form, each a chart of its own (above).

### *CLMULH*

**CLMULH: R3, prefix 1110100001, sub-opcode 0000011; operands reg5, reg5, reg5; 4 bytes**

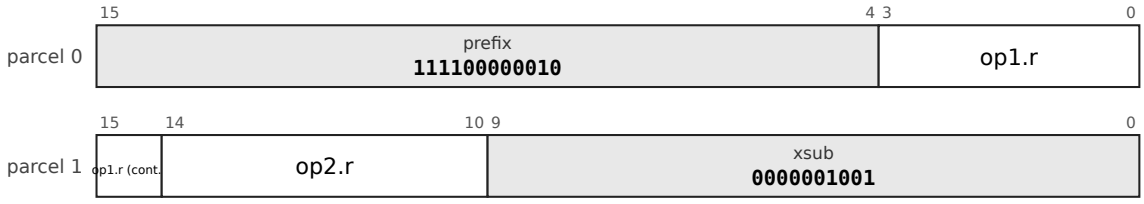



```
CLMULH Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry CLMUL above; 1 layout form, each a chart of its own (above).

## UTF8LEN: Length of a UTF-8 sequence from its leading byte

**UTF8LEN: R2, prefix 111100000010, sub-opcode 0000001001; operands reg5, reg5; 4 bytes**

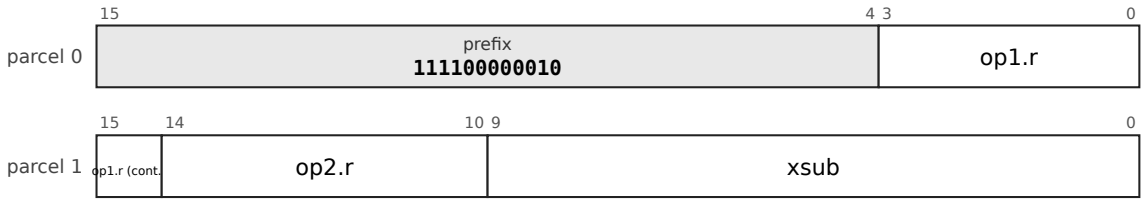



**Mnemonics:** `UTF8LEN`

### Syntax

```
UTF8LEN Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

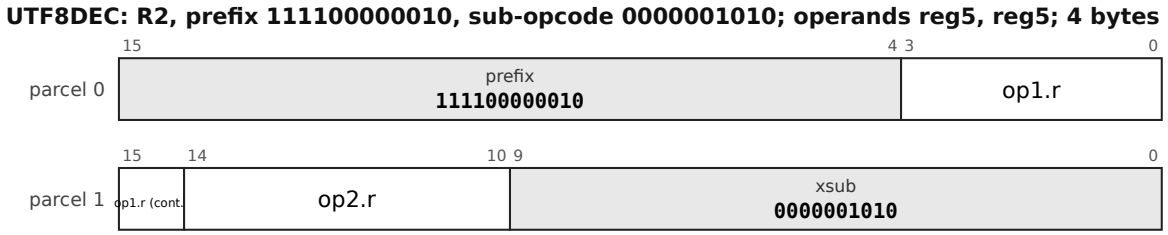
```
Rd ← length (1 to 4) implied by the leading byte Rs[7:0]; 0 if it cannot begin a valid sequence
```

**Flags** None written.

**Exceptions** None.

**Why it is here.** Most of what UTF-8 needs is general bit manipulation, but the validity rules are where the bugs are, so the group has three instructions. `UTF8LEN` is the one that is nearly free: `CLO` on the leading byte, mapped to 1, 2, 3 or 4.

## UTF8DEC: Decode a UTF-8 sequence

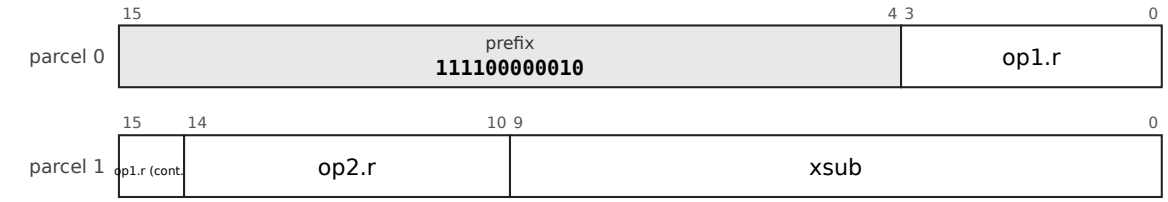



**Mnemonics:** `UTF8DEC`

### Syntax

```
UTF8DEC Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
Rs holds up to 4 bytes, leading byte in bits 7..0
Rd ← the scalar value
V  ← 1 if the sequence is not a valid encoding
```

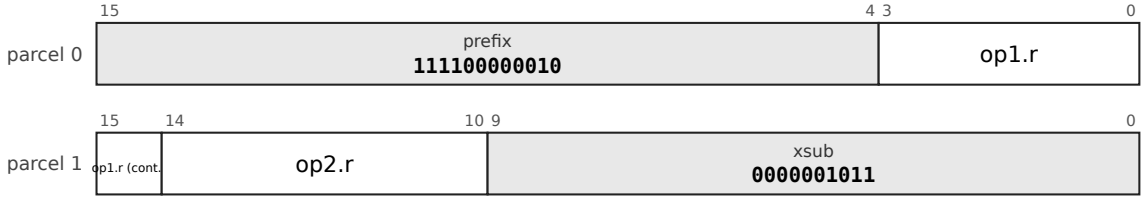
**Flags** `V` set for a bad continuation byte, an overlong form, a surrogate (U+D800 to U+DFFF), a value above U+10FFFF, or a leading byte of `11000000 (0xC0)`, `11000001 (0xC1)` or `11110101 (0xF5)` to `11111111 (0xFF)`; the length consumed is reported in a scratch field of the status word (details in the instruction-set text, section 4.6.1).

**Exceptions** None.

**Why it is here.** Not for speed but for correctness: overlong forms, surrogates, values above U+10FFFF and the bytes that can never begin a sequence are each a historical security hole, each is three comparators in hardware, and software implementations get them wrong or omit them for speed (about 300 lookup tables).

## UTF8ENC: Encode a Unicode scalar value

**UTF8ENC: R2, prefix 111100000010, sub-opcode 0000001011; operands reg5, reg5; 4 bytes**

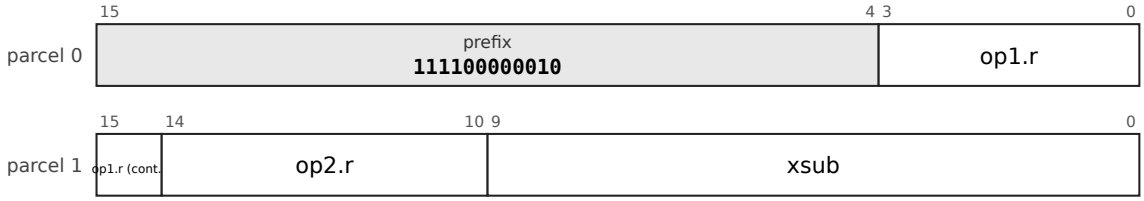



**Mnemonics:** `UTF8ENC`

### Syntax

```
UTF8ENC Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic. **All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
Rd ← the UTF-8 bytes of the scalar value Rs, leading byte in bits 7..0
V  ← 1 if Rs is not a Unicode scalar value
```

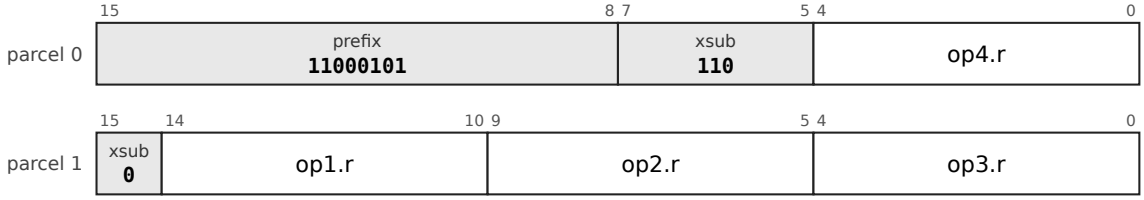
**Flags** V set if Rs is not a Unicode scalar value (a surrogate or above U+10FFFF).

**Exceptions** None.

**Why it is here.** The encoding half of the UTF-8 pair; it refuses surrogates and values above U+10FFFF so that an encoder cannot produce what a decoder must reject.

## BSEL: Bitwise select

**BSEL: R4, prefix 11000101, sub-opcode 1100; operands reg5, reg5, reg5, reg5; 4 bytes**

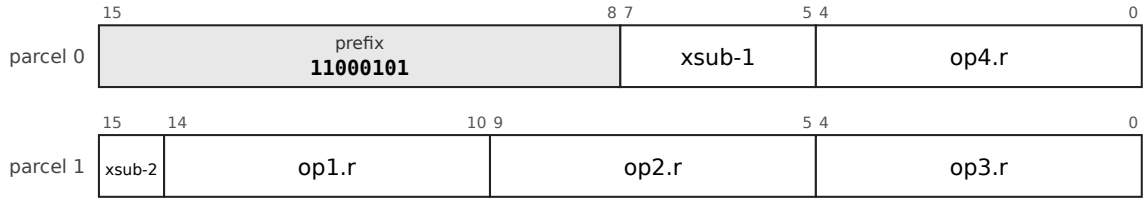



**Mnemonics:** `BSEL`

### Syntax

```
BSEL Rm, Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R4-1`, R4, prefix `11000101`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `op4` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R4-1`.

### Operation

```
Rd ← (Rm AND Ra) OR (NOT Rm AND Rb)       ; bit by bit
```

**Flags** None written.

**Exceptions** None.

*(D)* One of the select instructions of the secure-core design, which the author approved (errata 85 and 86: the select instructions "looks good"); it is one of the stage 1 words of `docs/crypto-secure-core.md` section 3. `Ch(e, f, g)` of SHA-2 is `BSEL e, f, g`; `Maj(a, b, c)` is `BSEL (a XOR b), c, b`; a mask-controlled swap of two registers is two `BSEL`. Three sources and a destination: the four register fields of the three-operand group.

**Why it is here.** A bit select takes three sources and a destination, the four register fields of the three-operand group. Its case in the crypto proposal is SHA-2 (`Ch` and `Maj` are one `BSEL` each) and the constant-time swap: the X25519 ladder swaps ten limbs per step, 50 instructions with the mask pair against about 25 with `BSEL`. It is the select of ARM (`BSL`) and of x86 ternary logic, and it serves MD5 and SHA-1 round functions, blends and bit insertion. It is one of the select instructions of the secure core.

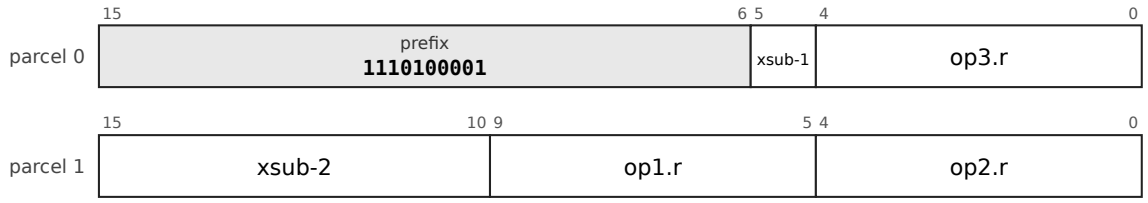
## AESR: AES round on a 128-bit block in two registers

**Mnemonics:** `AESES`, `AESESM`, `AESDS`, `AESDSM`

### Syntax

```
AESES Ra, Rb, Rd
AESESM Ra, Rb, Rd
AESDS Ra, Rb, Rd
AESDSM Ra, Rb, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
the 16 bytes of the block are Rb:Ra (Ra holds bytes 0 to 7, columns 0 and 1; Rb bytes 8 to 15)
AESES  : ShiftRows, then SubBytes
AESESM : ShiftRows, SubBytes, then MixColumns
AESDS  : inverse ShiftRows, then inverse SubBytes
AESDSM : inverse ShiftRows, inverse SubBytes, then inverse MixColumns
Rd ← the 8 bytes of the new state that belong to columns 0 and 1; with Ra and Rb exchanged, columns 2 and 3
```

**Flags** None written.

**Exceptions** None.

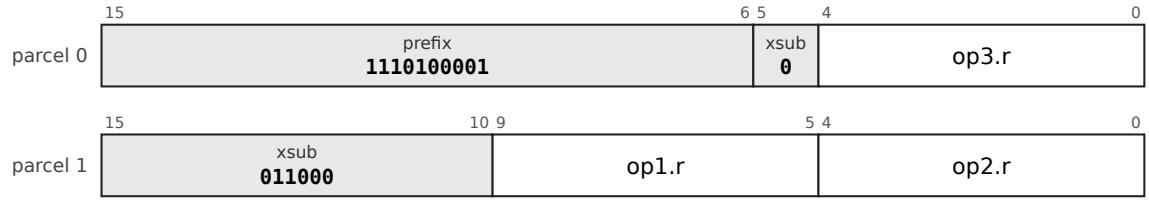
*(D)* The author asked for constant-time AES and SHA instructions "or whichever opcodes would speed up and enable aes and sha" for the secure core (errata 85) and approved its design (errata 86); the words are those of `docs/crypto-secure-core.md` section 3.2. Any two registers hold a block (no even-odd pairing): a round on one block is `AESESM Ra, Rb, T` and `AESESM Rb, Ra, Rb`, with the round key XORed by `XOR` with a memory operand. The latency is fixed and the instruction reads and writes no memory, so it has no cache-timing channel. The definitions are those of FIPS 197. Details pending (J.9.12).

**Why it is here.** Constant-time software AES takes about 1,000 instructions a block (BearSSL's bitsliced code, scaled; the proposal's estimate), and table AES leaks through the cache. The author asked for constant-time AES and SHA instructions "or whichever opcodes would speed up and enable aes and sha" for the secure core (errata 85). The design is RISC-V's scalar cryptography (`aes64es` and its kin): the state is in the general registers, so no vector register file is needed and, unlike the AES drivers of OpenBSD for x86 and ARM, no `fpu_kernel_enter()` bracket either: a kernel AES loop saves nothing and is preemptible with the ordinary register save. A block costs 42 instructions for AES-128, 20 to 25 times fewer than the software sequence (estimates). Precedent for the pair-in-registers shape: RISC-V; the other machines (x86 AES-NI, ARM `AESE`) use vector registers.

**Cf.** docs/crypto-secure-core.md section 3.

### *AESES*

**AESES: R3, prefix 1110100001, sub-opcode 0011000; operands reg5, reg5, reg5; 4 bytes**

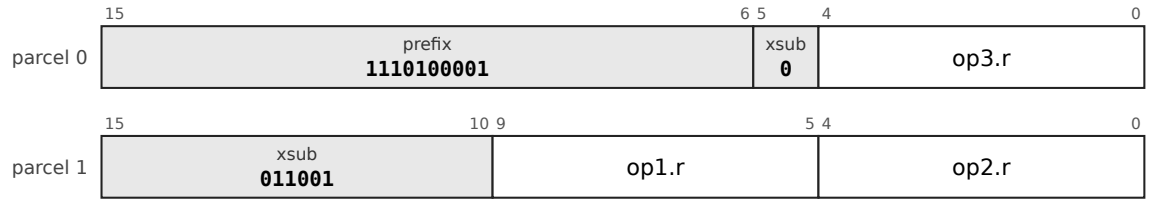



```
AESES Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

### *AESESM*

**AESESM: R3, prefix 1110100001, sub-opcode 0011001; operands reg5, reg5, reg5; 4 bytes**

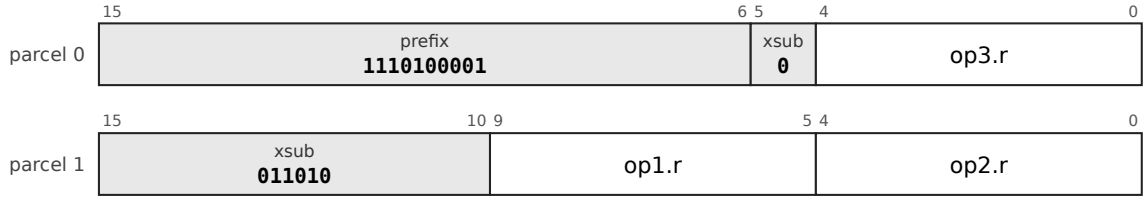



```
AESESM Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

### *AESDS*

**AESDS: R3, prefix 1110100001, sub-opcode 0011010; operands reg5, reg5, reg5; 4 bytes**

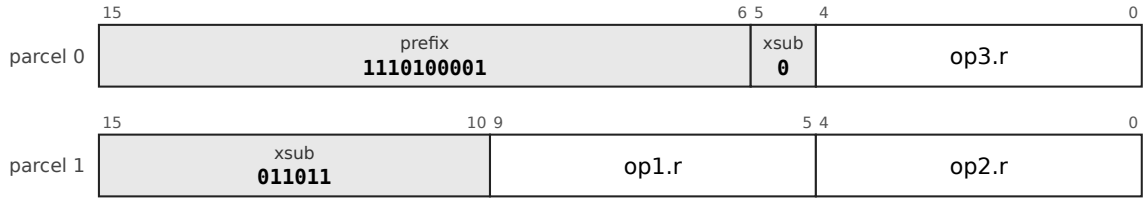



```
AESDS Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

### *AESDSM*

**AESDSM: R3, prefix 1110100001, sub-opcode 0011011; operands reg5, reg5, reg5; 4 bytes**




```
AESDSM Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESR above; 1 layout form, each a chart of its own (above).

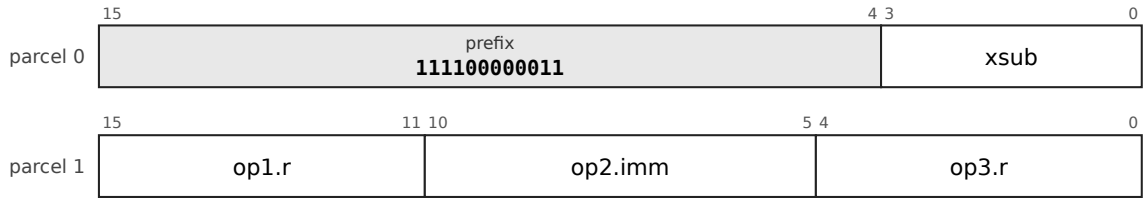
## AESK: AES key schedule steps

**Mnemonics:** `AESKS1`, `AESKS2`, `AESIM`

### Syntax

```
AESKS1 Rs, #n, Rd
AESKS2 Ra, Rb, Rd
AESIM Rs, Rd
```

### Encoding




The chart is the leaf `RRI-1`, RRI, prefix `111100000011`. `op1` is `reg5`: register, 5 bits. `op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `RRI-1`, `R3-1`, `R2-1`.

### Operation

```
AESKS1: Rd ← (SubWord(RotWord(Rs[63:32])) XOR rcon(n)), replicated in both halves   ; n = 10: no round
constant (AES-256)
```

```
AESKS2: Rd[31:0] ← Ra[31:0] XOR Rb[63:32] ; Rd[63:32] ← Ra[63:32] XOR Rd[31:0]
AESIM : Rd ← InvMixColumns of the two columns in Rs         ; for the decryption key schedule
```

**Flags** None written.

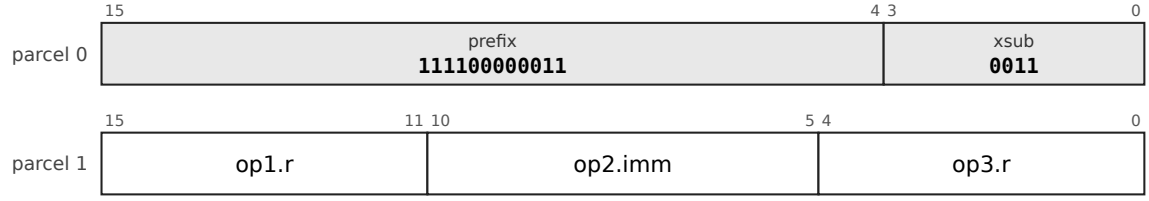
**Exceptions** None.

*(D)* As AESR. The key schedule of AES-128 takes three instructions per round. Details pending (J.9.12).

**Why it is here.** The key schedule and the inverse mix of the round key complete the round instructions: three instructions per round of the AES-128 schedule, about 30 for the whole schedule instead of about 150, once per key; AESIM adds 18 for the decryption schedule. The same precedent as AESR.

### *AESKS1*

**AESKS1: RRI, prefix 111100000011, sub-opcode 0011; operands reg5, imm6u, reg5; 4 bytes**

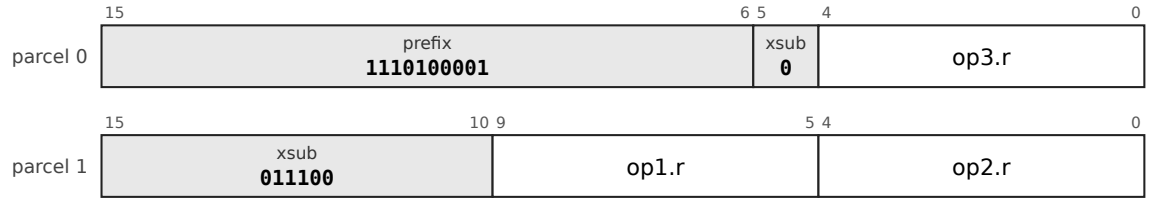



```
AESKS1 Rs, #n, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

### *AESKS2*

**AESKS2: R3, prefix 1110100001, sub-opcode 0011100; operands reg5, reg5, reg5; 4 bytes**

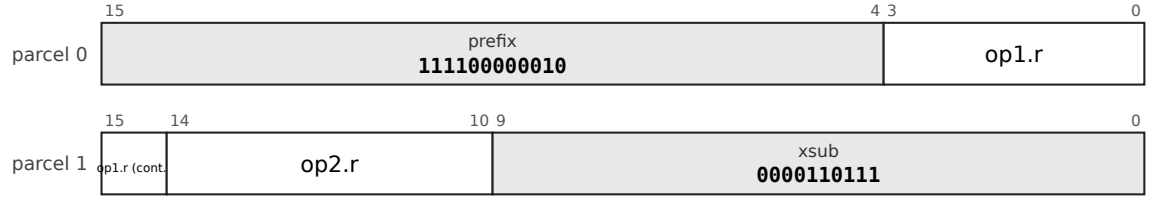



```
AESKS2 Ra, Rb, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

### *AESIM*

**AESIM: R2, prefix 111100000010, sub-opcode 0000110111; operands reg5, reg5; 4 bytes**




```
AESIM Rs, Rd
```

Operation, flags and exceptions as for the entry AESK above; 1 layout form, each a chart of its own (above).

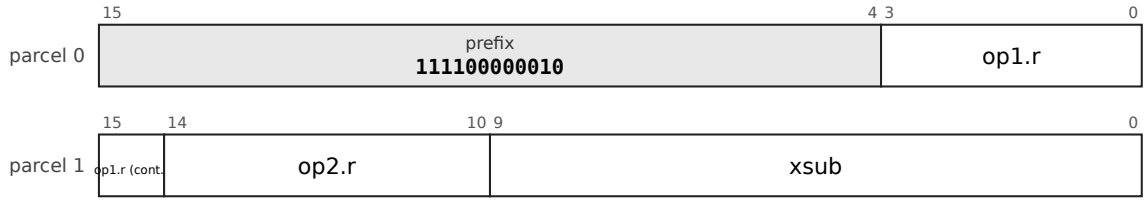
## SHA256: The SHA-256 functions

**Mnemonics:** SHA256SIG0, SHA256SIG1, SHA256SUM0, SHA256SUM1

### Syntax

```
SHA256SIG0 Rs, Rd
SHA256SIG1 Rs, Rd
SHA256SUM0 Rs, Rd
SHA256SUM1 Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic. **All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
on the low 32 bits x of Rs; the result is in the low 32 bits of Rd, the upper bits zero
SIG0: ror(x,7)  XOR ror(x,18) XOR shr(x,3)
SIG1: ror(x,17) XOR ror(x,19) XOR shr(x,10)
SUM0: ror(x,2)  XOR ror(x,13) XOR ror(x,22)
SUM1: ror(x,6)  XOR ror(x,11) XOR ror(x,25)
```

**Flags** None written.

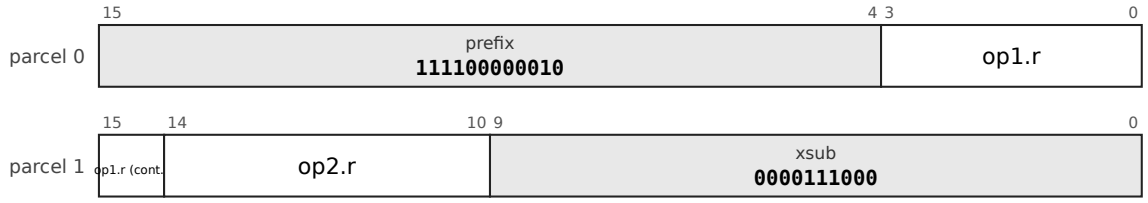
**Exceptions** None.

*(D)* As `AESR`. Rotations and three-input exclusive ors, one cycle in hardware, no data-dependent timing. Details pending: whether the 32-bit result is zero-extended, as the study has it, or sign-extended, as every other 32-bit result of the machine is (`RORI32`) (J.9.12).

**Why it is here.** The SHA-2 rounds are rotations and three-input exclusive ors, which the integer instructions do in several steps; the RISC-V scalar cryptography extension has the eight functions as instructions. The crypto proposal counts the gain honestly: the primitives remove half the work, not nineteen twentieths, because a round still needs the adds and the register traffic (about 1,200 against 2,500 instructions for a 64-byte block with `BSEL`, estimates).

### *SHA256SIG0*

**SHA256SIG0: R2, prefix 111100000010, sub-opcode 0000111000; operands reg5, reg5; 4 byt**

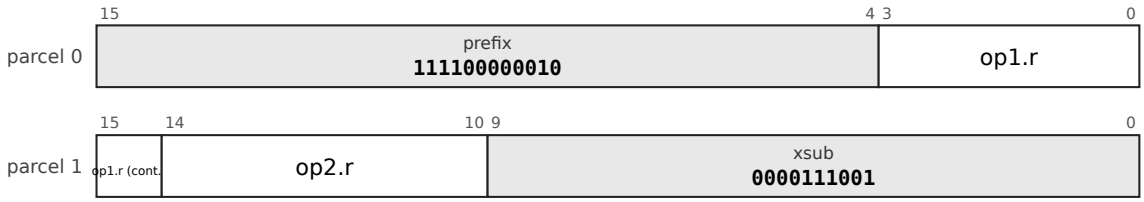



```
SHA256SIG0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

### *SHA256SIG1*

**SHA256SIG1: R2, prefix 111100000010, sub-opcode 0000111001; operands reg5, reg5; 4 byt**

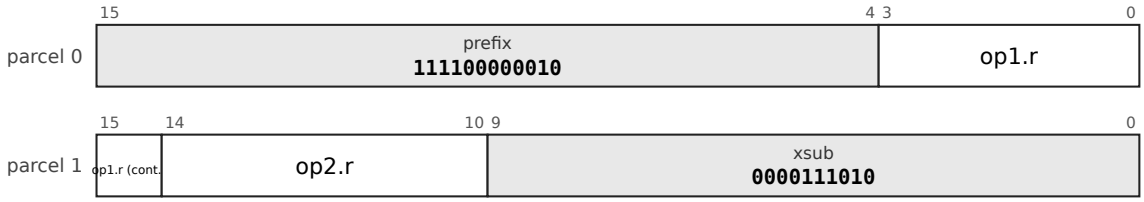



```
SHA256SIG1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

### *SHA256SUM0*

**SHA256SUM0: R2, prefix 111100000010, sub-opcode 0000111010; operands reg5, reg5; 4 by**

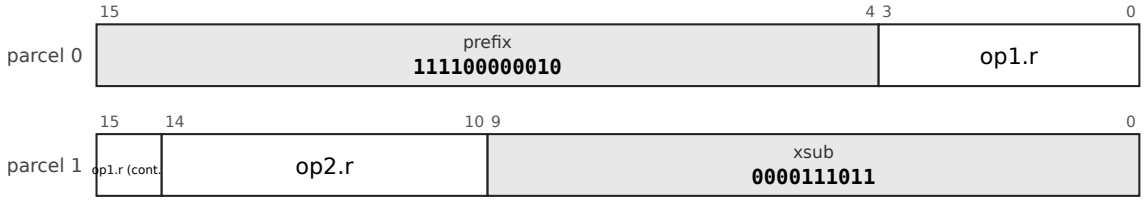



```
SHA256SUM0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

### *SHA256SUM1*

**SHA256SUM1: R2, prefix 111100000010, sub-opcode 0000111011; operands reg5, reg5; 4 by**




```
SHA256SUM1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA256 above; 1 layout form, each a chart of its own (above).

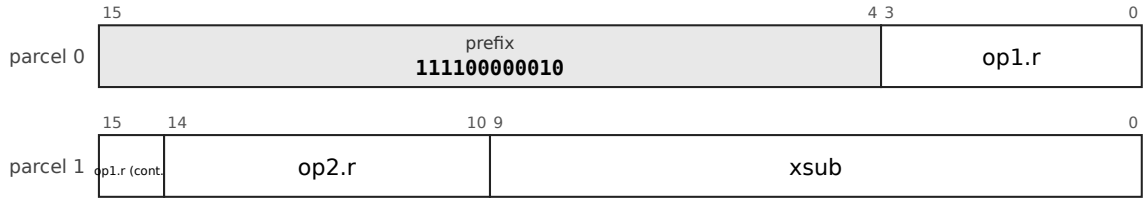
## SHA512: The SHA-512 functions

**Mnemonics:** `SHA512SIG0`, `SHA512SIG1`, `SHA512SUM0`, `SHA512SUM1`

### Syntax

```
SHA512SIG0 Rs, Rd
SHA512SIG1 Rs, Rd
SHA512SUM0 Rs, Rd
SHA512SUM1 Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
on the 64 bits x of Rs
SIG0: ror(x,1)  XOR ror(x,8)  XOR shr(x,7)
SIG1: ror(x,19) XOR ror(x,61) XOR shr(x,6)
SUM0: ror(x,28) XOR ror(x,34) XOR ror(x,39)
SUM1: ror(x,14) XOR ror(x,18) XOR ror(x,41)
```

**Flags** None written.

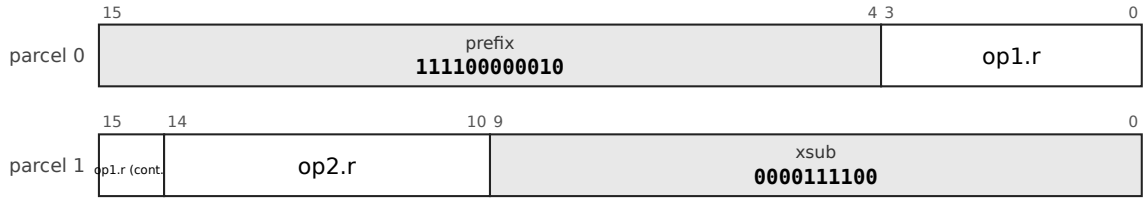
**Exceptions** None.

*(D)* As `SHA256`. The definitions are those of FIPS 180-4.

**Why it is here.** See `SHA256`: the four 64-bit functions of SHA-512, with the same gain of about two (about 1,500 against 3,100 instructions for a 128-byte block, estimates).

## *SHA512SIG0*

**SHA512SIG0: R2, prefix 111100000010, sub-opcode 0000111100; operands reg5, reg5; 4 byt**

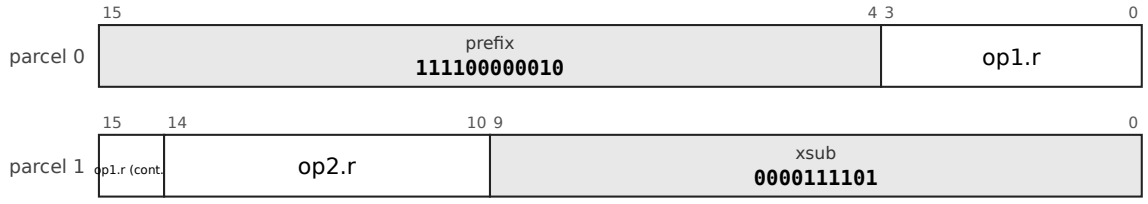



```
SHA512SIG0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

## *SHA512SIG1*

**SHA512SIG1: R2, prefix 111100000010, sub-opcode 0000111101; operands reg5, reg5; 4 byt**

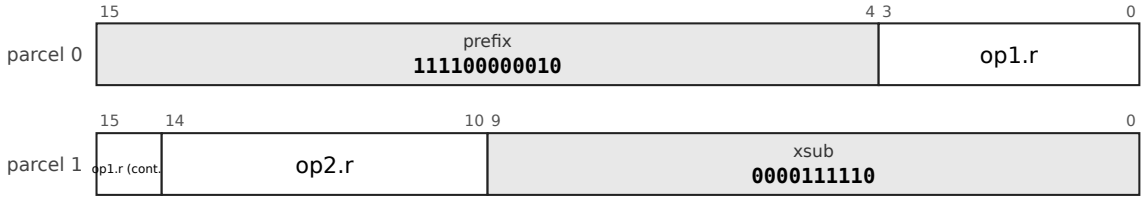



```
SHA512SIG1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

### *SHA512SUM0*

**SHA512SUM0: R2, prefix 111100000010, sub-opcode 0000111110; operands reg5, reg5; 4 by**

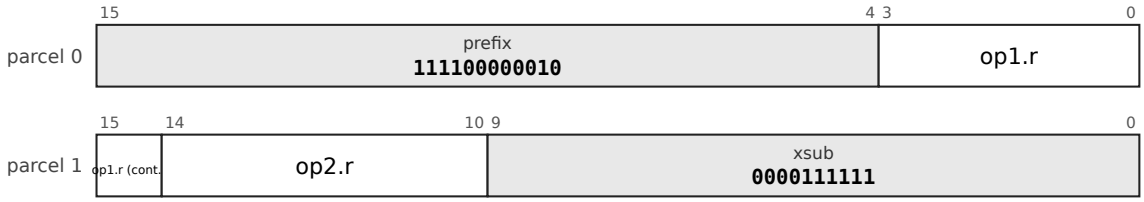



```
SHA512SUM0 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

### *SHA512SUM1*

**SHA512SUM1: R2, prefix 111100000010, sub-opcode 0000111111; operands reg5, reg5; 4 by**




```
SHA512SUM1 Rs, Rd
```

Operation, flags and exceptions as for the entry SHA512 above; 1 layout form, each a chart of its own (above).

## 9.10 Branches and control flow

*Cf. section 6.13 for* TRAP, RTI *and* RTT*, and section 6.3 for how a trap enters the kernel; section 9.16 for* CALLI *and* JMPI *in position-independent code.*

**9.10.1** There is an unconditional branch BR and fourteen conditional branches, two bytes with a field of 6 to 9 bits, four with 16 bits, six with 32 bits, counting two-byte parcels from the first parcel of the branch: ±64 to ±512 bytes, ±64 KiB, ±4 GiB.

**9.10.2** The conditions are ne, eq, pl, mi, vc, vs, cc/hs, cs/lo, ge, lt, gt, le, hi and los.

**9.10.3** SOB Rn, dst subtracts one from Rn and branches if the result is nonzero, forward or backward. It is four bytes, with a 16-bit field of parcels (±64 KiB).

**9.10.4** CBCC compares a register with zero, with an immediate or with a second register and branches on the condition, and writes no flags. It is one parcel for CBEQ and CBNE against zero (a field of 4 to 7 bits: ±16 to ±128 bytes); two parcels for an immediate of 3 to 10 bits or a second register, with a field of 8 to 14 bits (±256 bytes to ±16 KiB); three parcels for a 16-bit immediate with a 12-bit field (±4 KiB), an 8-bit immediate with a 20-bit field (±1 MiB) or two registers with a 24-bit field (±16 MiB). *(D)*

**9.10.5** JMP and CALL take any address mode; RET returns; JMPI and CALLI jump and call through a pointer in memory; JTAG dispatches on the tag in the low three bits of a register; BTAG and BNTAG branch on a tag. *(D)*

**9.10.6** A call pushes the return address on the stack; there is no link register. *(D)*

**9.10.7** Four pairs of instructions are fused into one. MOVCALL Rs, Rd, label is MOVQ Rs, Rd followed by CALL label, in four bytes (a 16-bit field of parcels, ±64 KiB) or six (32 bits); POPRET Rd is POP Rd followed by RET, and POPMRET #mask is POPM #mask followed by RET (the 8-bit mask field, scaled by 256, names R8 to R15), each in two bytes; MOV2 Ra, Rb, Rc, Rd is MOVQ Ra, Rb followed by MOVQ Rc, Rd, in two bytes, with all four registers in the 3-bit hot table. The assembler never fuses: choosing a fused instruction is the compiler's job. A fused instruction is restarted whole after a fault. *(D)*

**9.10.8** A CBCC for which no form reaches (a label in another object, an operand pair that no form holds) is assembled as CMP followed by BCC. This writes the flags, which CBCC does not, and executes two instructions for one; the compiler forms a CBCC

only where the flags are dead after the branch, so the program is unchanged. On the 841-file corpus 13 sites need the pair; in the six large programs 12 sites need it and 79,894 pairs are executed, 0.014% of the instructions.

*Commentary.* **Precedent.** The complete PDP-11 branch set, which the long-branch problem of the PDP-11 (±256 bytes) made awkward, here reaches ±4 GB with no long-branch opcode and no trampoline, because the displacement widens with the length. `SOB` was the PDP-11's, which could only branch backward. **Trade-offs.** A compare-and-branch that writes no flags cannot feed a second branch that reads them, so a three-way compare is compare-then-branch; the compiler fuses only where the flags are dead afterwards, which costs about 0.4% in code size. **Improvements.** The tag branches serve a Lisp or other tagged runtime in one instruction. **Benefits.** Branch relaxation in the assembler is choosing a length. **Modern comparables.** RISC-V and MIPS are compare-and-branch machines with no flags; AArch64 has `CBZ` and `TBZ`; x86 uses flags like the pdp64's branches. **Use.** `JMPI` and `CALLI` are the shared-library call and the virtual call; `JTAG` is the dispatch of a dynamic-language interpreter. **Impact.** *Kernel:* `JMPI`, `CALLI` and `JTAG` are the shared-library call, the virtual call and the dispatch of an interpreter; `SOB` and `CBCC` are the loops of the kernel's inner routines. *Compiler:* Branch relaxation is choosing a length; fusing a compare and a branch is legal only where the flags are dead afterwards, so the compiler needs a flag-liveness analysis. *Hardware:* The next-PC mux takes the load-return path for the indirect forms; the compare-and-branch compare runs in the branch unit beside the ALU; displacements count parcels, so no shifter is needed.

*Commentary on 9.10.4, 9.10.7 and 9.10.8.* **Precedent.** The PDP-11 branched on the flags of the previous instruction only; compare-and-branch machines (MIPS, RISC-V) branch on two registers in one instruction, and the long-branch problem of both was solved by a longer encoding, here by the three lengths of `CBCC`. Macro-op fusion in current x86 and ARM cores joins a compare and a branch in the decoder; here the compiler does the joining and the instruction set names the result. **Trade-offs.** The reach of the shortest `CBCC` forms was set from the corpus: the optimiser trained on the compiler's own output, priced a `CBCC` that no form reaches as the `CMP` and `BCC` that the assembler substitutes, and offered the three-parcel forms; the cost is that one-parcel `CBCC` reaches less than in the earlier prefix-coded iteration (1.8), and lz4 fetches 1.2% more bytes for it (appendix F). The four fused pairs save two bytes a pair: 39,674 bytes (4.16%) in the corpus when the compiler was first given them. **Improvements.** The fused call has a three-parcel form, so a call beyond ±64 KiB is still one instruction; and the assembler's `CMP` plus `BCC` makes every program assemble, so a form that does not reach is a slower program and never an error. **Benefits.** Fusion is a decision of the compiler, so the instruction set stays free of microarchitectural pairing rules, and a core that does not fuse in hardware loses nothing. **Modern comparables.** The fused pairs are the ones the compiled corpus shows most often adjacent: a move before a call, a pop before a return. **Impact.** *Kernel:* `POPRET` and `POPMRET` are the epilogue of every function and `MOVCALL` the call of every wrapper; they are smaller kernel text with no source change. *Compiler:* it must know the flags are dead before forming `CBCC`, fuse after register allocation, and keep a mask of `R24` to `R31` out of `POPMRET` (the field names `R8` to `R15` only). *Hardware:* a fused instruction is decoded as two operations that retire together; a hardware implementation may execute them as one or two.

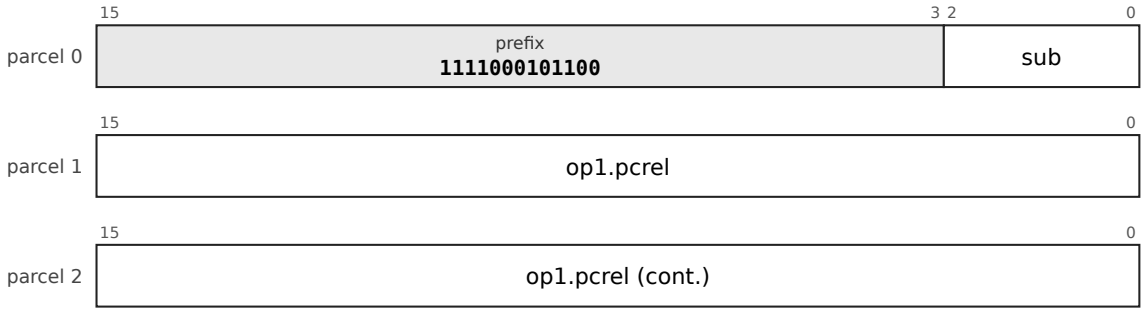
## BCC: Branch on condition

**Mnemonics:** `BR`, `BNE`, `BEQ`, `BPL`, `BMI`, `BVC`, `BVS`, `BCC`, `BCS`, `BGE`, `BLT`, `BGT`, `BLE`, `BHI`, `BLOS` (aliases: `BHIS`, `BLO`)

### Syntax

```
BR target
BNE target
BEQ target ... and the rest
```

### Encoding




The chart is the leaf `BRF-1`, BRF, prefix `1111000101100`. `op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes). `sub` (3 bits) selects the mnemonic: `br` 000, `bne` 001, `beq` 010, `bpl` 011, `bmi` 100, `bvc` 101, `bvs` 110, `bcc` 111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `BRF-1`, `BR2-16-a-1`, `BR1-6-a-1`, `BR1-7-a-1`, `BR1-8-a-1`, `BR1-9-a-1`, `BR2-16-b-1`, `BR1-6-b-1`, `BR2-16-c-1`, `BRF-2`, `BR2-16-c-2`, `BRF-3`, `BRF-4`.

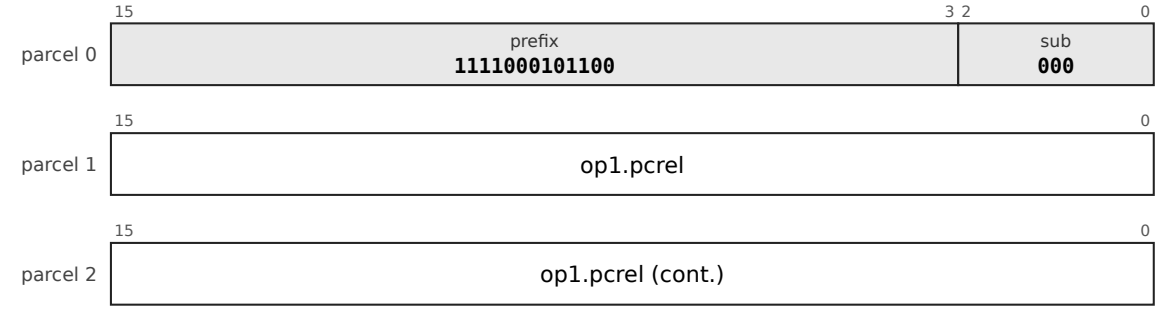
### Operation

```
if cond(c) then PC ← address of this instruction + 2 × disp
```

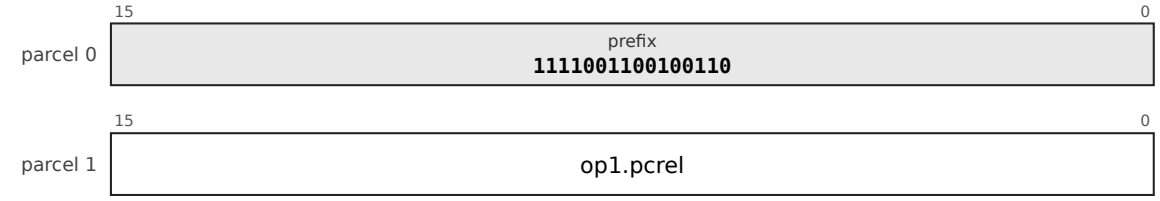
**Flags** None written.
**Exceptions** None; a branch to an unmapped or non-executable address faults on the fetch of the target.
Condition numbers: `al` 0 (`BR`), `ne` 1, `eq` 2, `pl` 3, `mi` 4, `vc` 5, `vs` 6, `cc`/`hs` 7, `cs`/`lo` 8, `ge` 9, `lt` 10, `gt` 11, `le` 12, `hi` 13, `los` 14. Conditions on the flags: `ne` `!Z`; `eq` `Z`; `pl` `!N`; `mi` `N`; `vc` `!V`; `vs` `V`; `cc` `!C`; `cs` `C`; `ge` `N == V`; `lt` `N != V`; `gt` `!Z and N == V`; `le` `Z or N != V`; `hi` `!C and !Z`; `los` `C or Z`. The displacement counts 2-byte parcels; the reach of each form is in its chart.
**Cf.** 6.13 for `TRAP`, `RTI` and `RTT`.

### *BR*

**BR: BRF, prefix 1111000101100, sub-opcode 000; operands pc32-u2; 6 bytes**




**BR: BR2_16/a, prefix 1111001100100110; operands pc16-u2; 4 bytes**



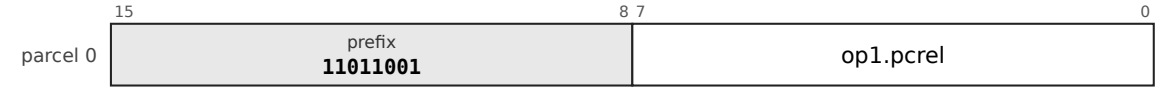

**BR: BR1_6/a, prefix 1110110000; operands pc6-u2; 2 bytes**



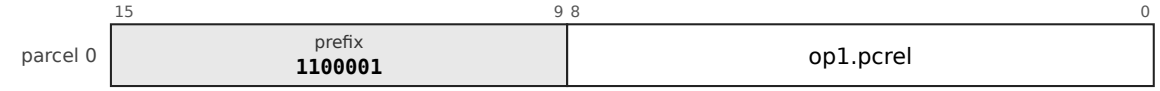

**BR: BR1_7/a, prefix 111001000; operands pc7-u2; 2 bytes**




**BR: BR1_8/a, prefix 11011001; operands pc8-u2; 2 bytes**




**BR: BR1_9/a, prefix 1100001; operands pc9-u2; 2 bytes**

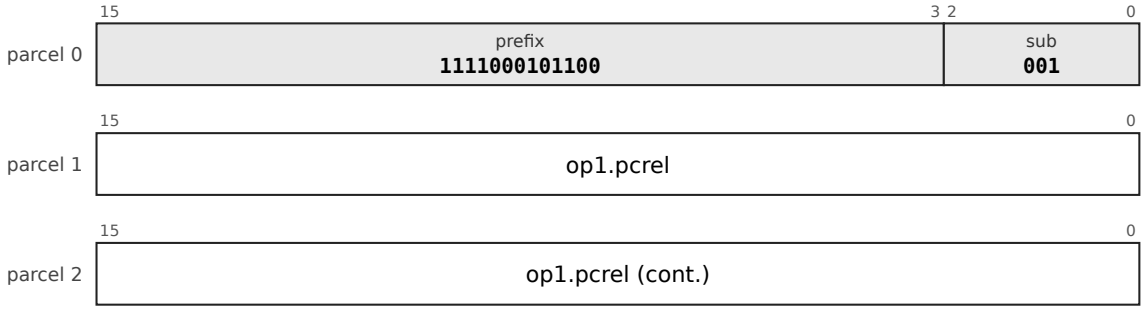



```
BR target
```

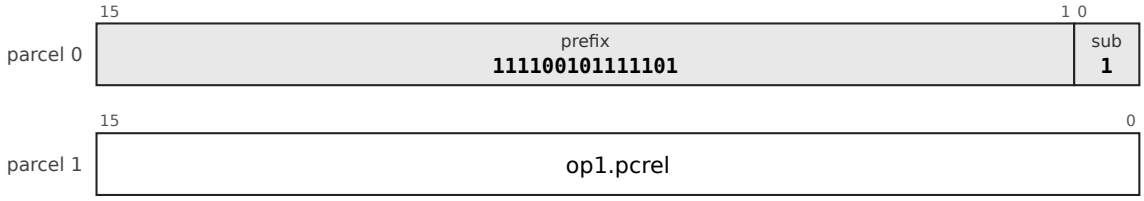
Operation, flags and exceptions as for the entry BCC above; 6 layout forms, each a chart of its own (above).

### *BNE*

**BNE: BRF, prefix 1111000101100, sub-opcode 001; operands pc32-u2; 6 bytes**




**BNE: BR2_16/b, prefix 111100101111101, sub-opcode 1; operands pc16-u2; 4 bytes**




**BNE: BR1_6/b, prefix 111001001, sub-opcode 1; operands pc6-u2; 2 bytes**

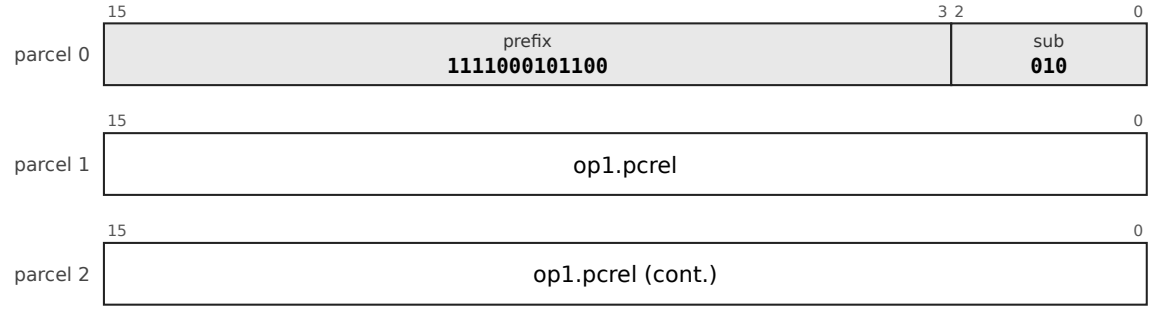



```
BNE target
```

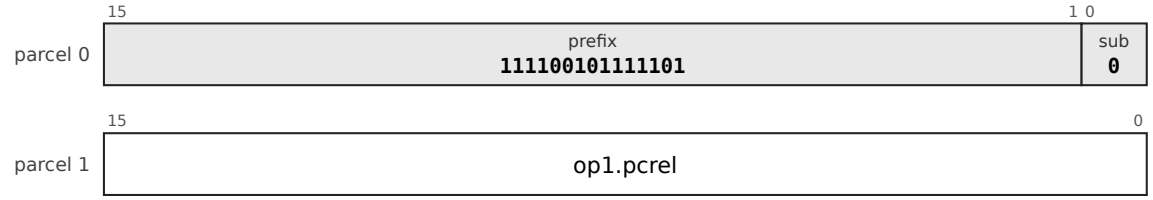
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

### *BEQ*

**BEQ: BRF, prefix 1111000101100, sub-opcode 010; operands pc32-u2; 6 bytes**




**BEQ: BR2_16/b, prefix 111100101111101, sub-opcode 0; operands pc16-u2; 4 bytes**




**BEQ: BR1_6/b, prefix 111001001, sub-opcode 0; operands pc6-u2; 2 bytes**

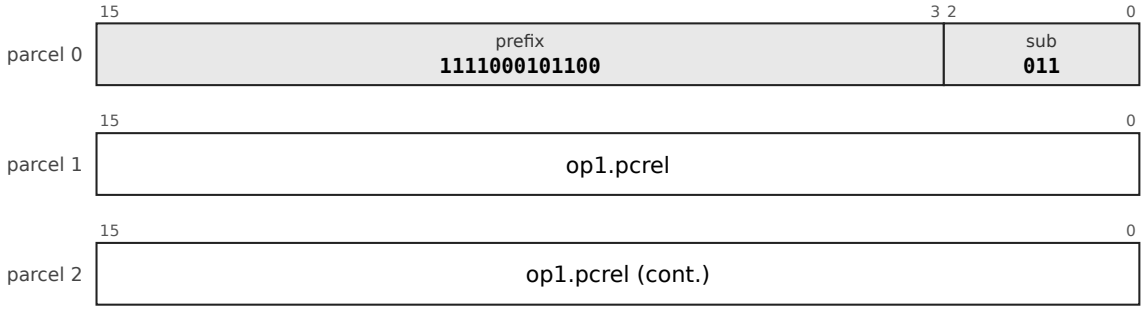



```
BEQ target ... and the rest
```

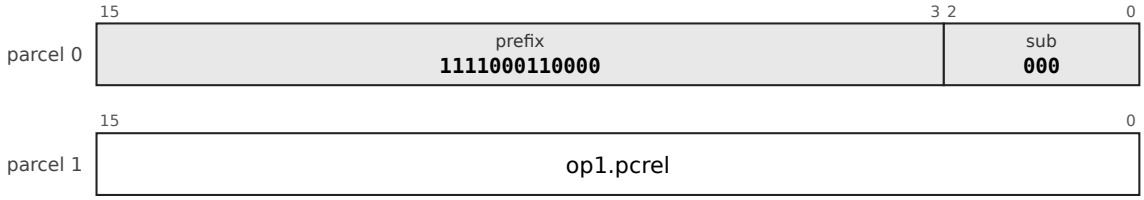
Operation, flags and exceptions as for the entry BCC above; 3 layout forms, each a chart of its own (above).

### *BPL*

**BPL: BRF, prefix 1111000101100, sub-opcode 011; operands pc32-u2; 6 bytes**



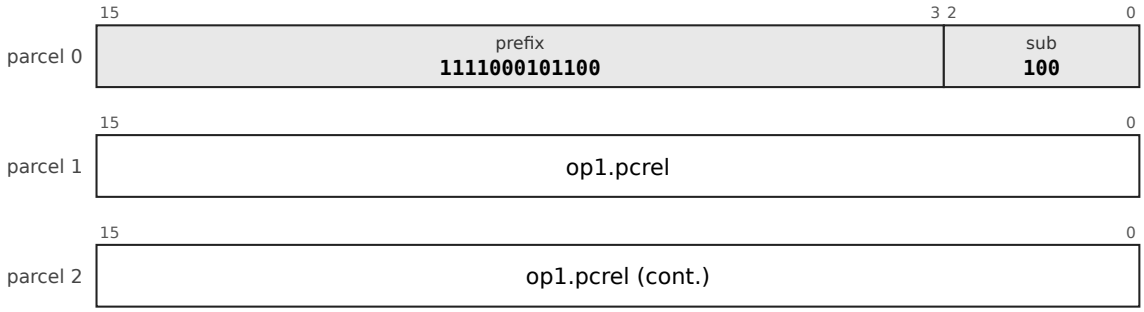

**BPL: BR2_16/c, prefix 1111000110000, sub-opcode 000; operands pc16-u2; 4 bytes**



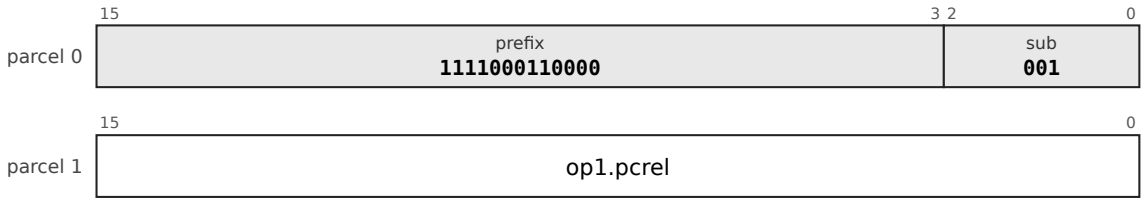

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BMI*

**BMI: BRF, prefix 1111000101100, sub-opcode 100; operands pc32-u2; 6 bytes**



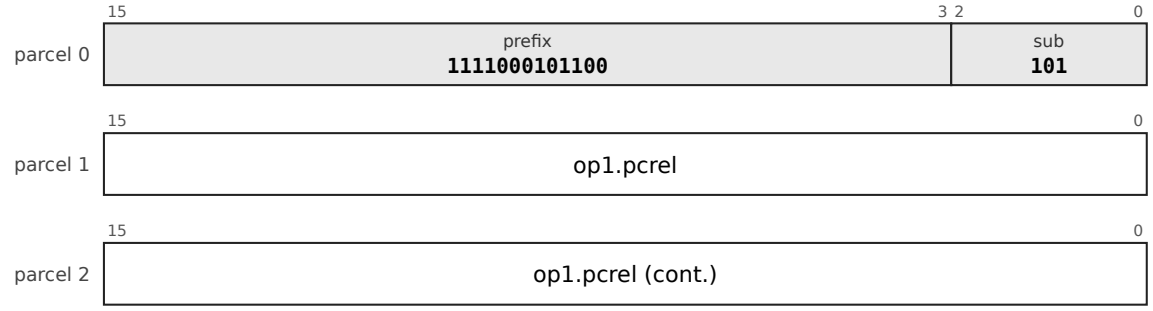

**BMI: BR2_16/c, prefix 1111000110000, sub-opcode 001; operands pc16-u2; 4 bytes**



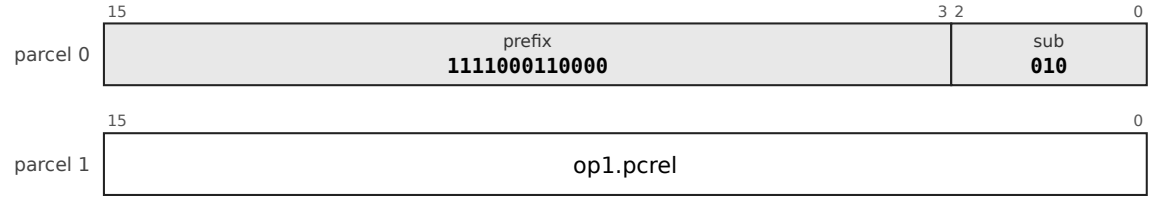

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BVC*

**BVC: BRF, prefix 1111000101100, sub-opcode 101; operands pc32-u2; 6 bytes**



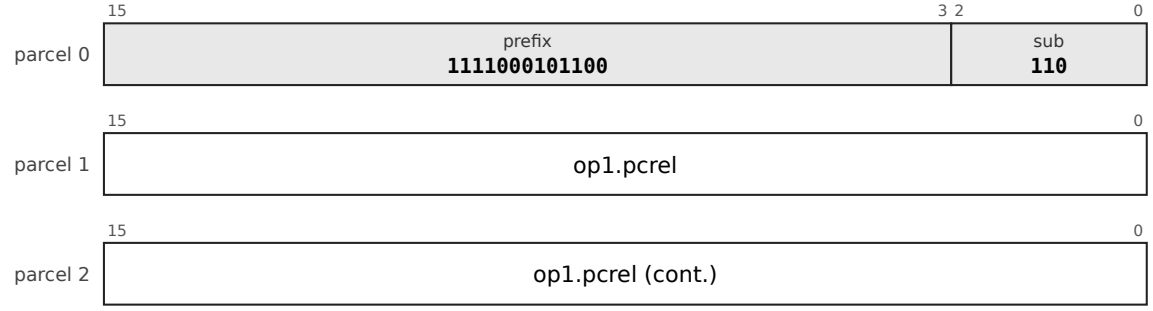

**BVC: BR2_16/c, prefix 1111000110000, sub-opcode 010; operands pc16-u2; 4 bytes**



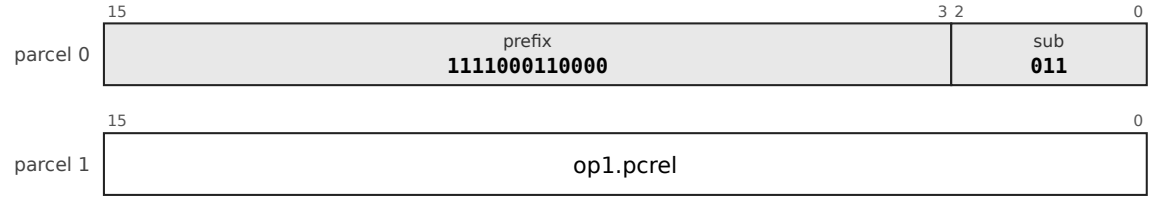

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BVS*

**BVS: BRF, prefix 1111000101100, sub-opcode 110; operands pc32-u2; 6 bytes**



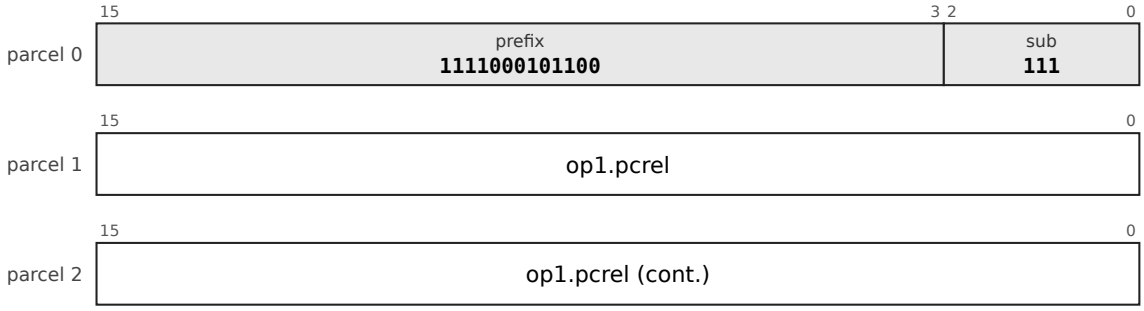

**BVS: BR2_16/c, prefix 1111000110000, sub-opcode 011; operands pc16-u2; 4 bytes**



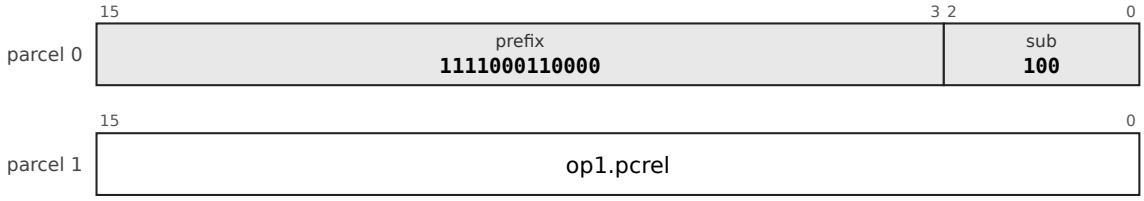

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BCC*

**BCC: BRF, prefix 1111000101100, sub-opcode 111; operands pc32-u2; 6 bytes**



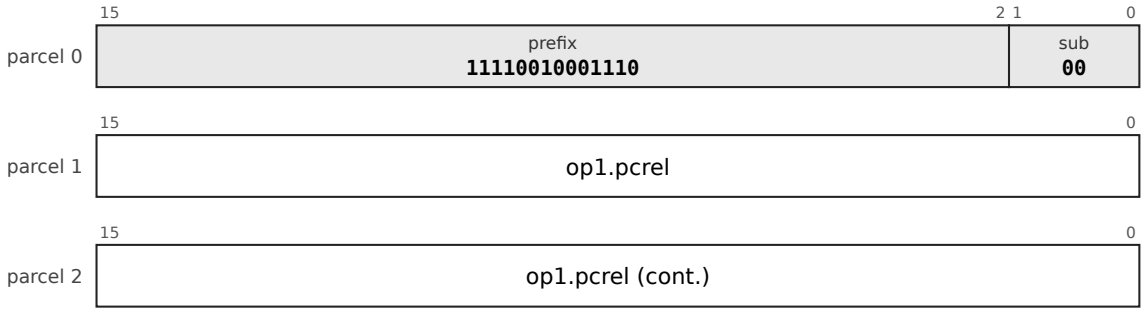

**BCC: BR2_16/c, prefix 1111000110000, sub-opcode 100; operands pc16-u2; 4 bytes**



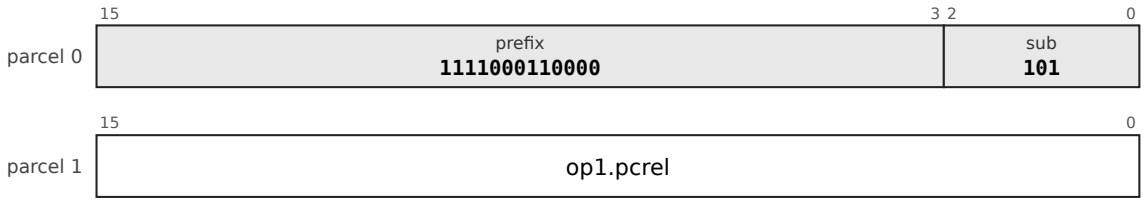

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BCS*

**BCS: BRF, prefix 11110010001110, sub-opcode 00; operands pc32-u2; 6 bytes**



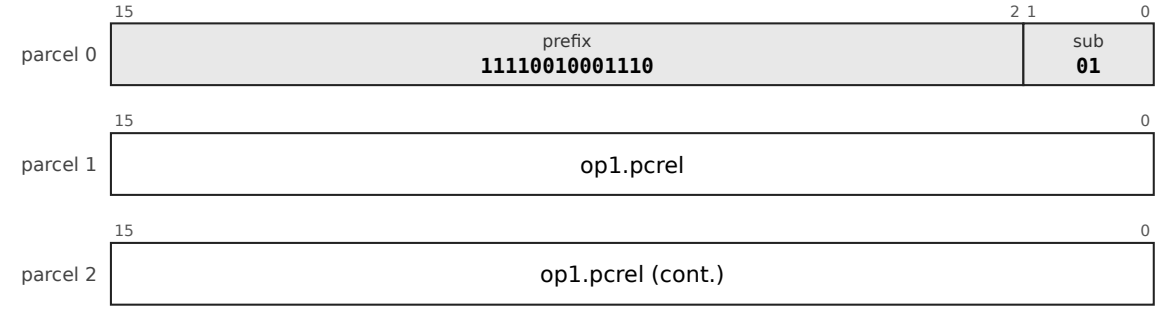

**BCS: BR2_16/c, prefix 1111000110000, sub-opcode 101; operands pc16-u2; 4 bytes**



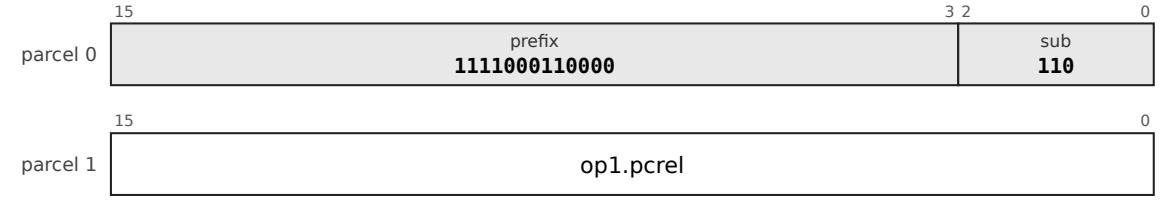

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BGE*

**BGE: BRF, prefix 11110010001110, sub-opcode 01; operands pc32-u2; 6 bytes**



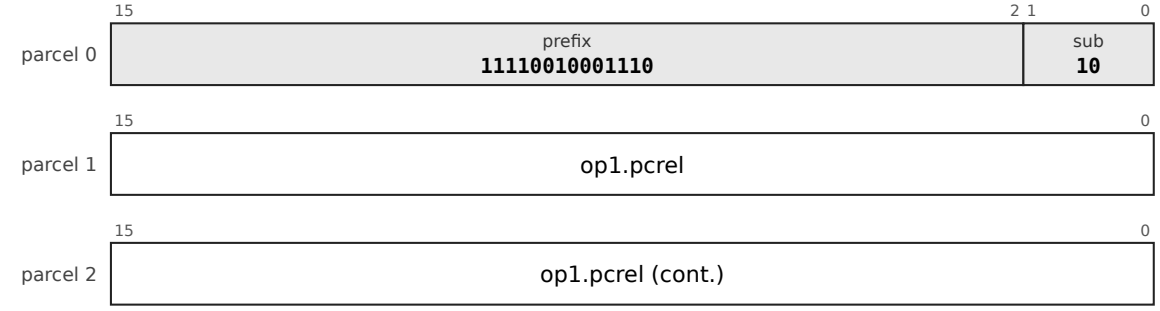

**BGE: BR2_16/c, prefix 1111000110000, sub-opcode 110; operands pc16-u2; 4 bytes**



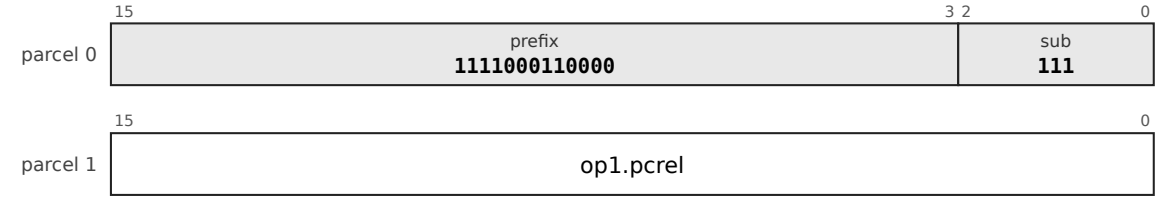

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BLT*

**BLT: BRF, prefix 11110010001110, sub-opcode 10; operands pc32-u2; 6 bytes**



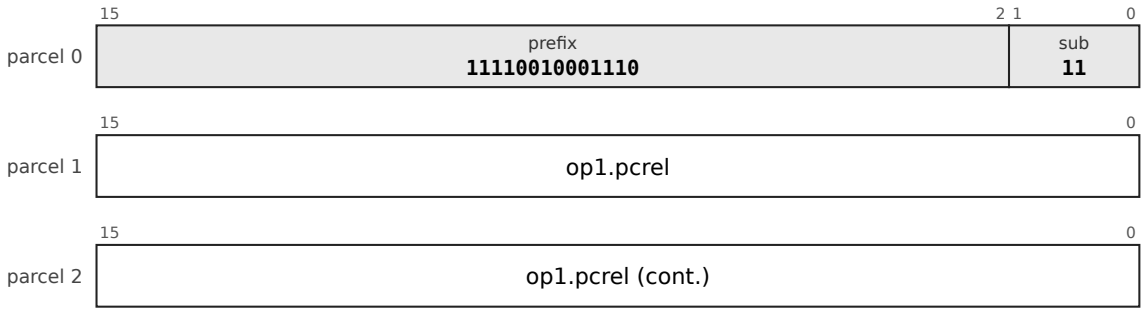

**BLT: BR2_16/c, prefix 1111000110000, sub-opcode 111; operands pc16-u2; 4 bytes**



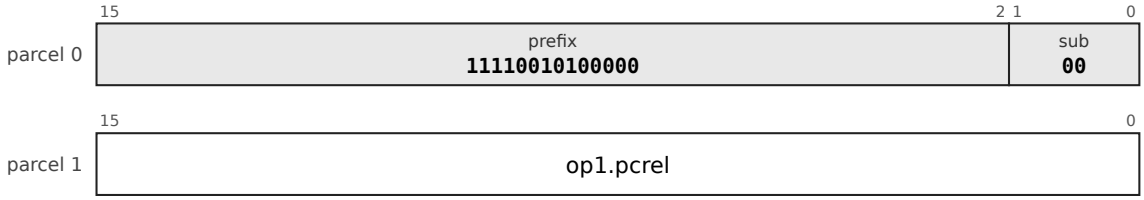

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BGT*

**BGT: BRF, prefix 11110010001110, sub-opcode 11; operands pc32-u2; 6 bytes**



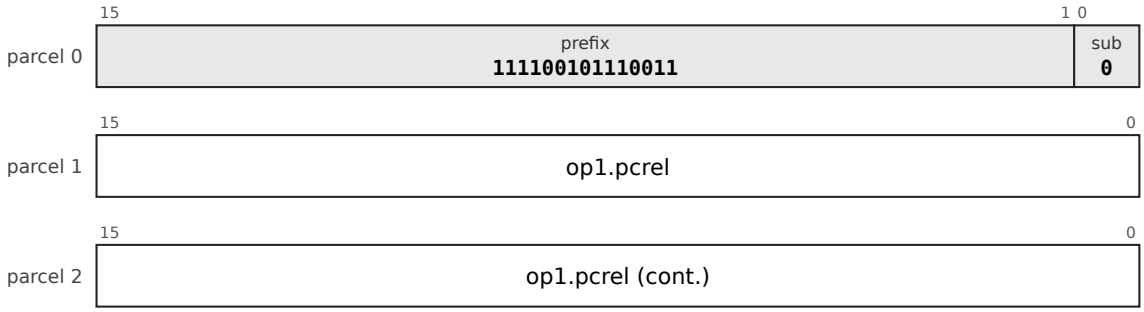

**BGT: BR2_16/c, prefix 11110010100000, sub-opcode 00; operands pc16-u2; 4 bytes**



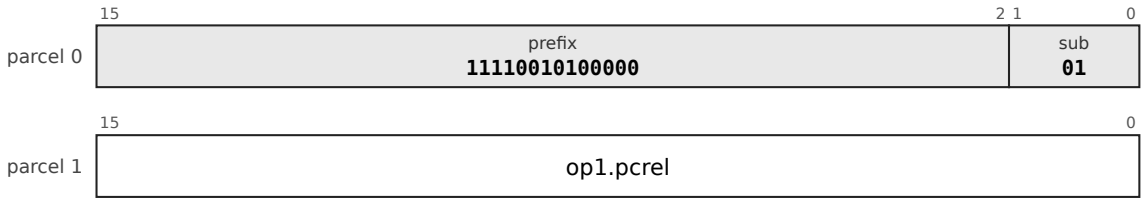

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BLE*

**BLE: BRF, prefix 111100101110011, sub-opcode 0; operands pc32-u2; 6 bytes**



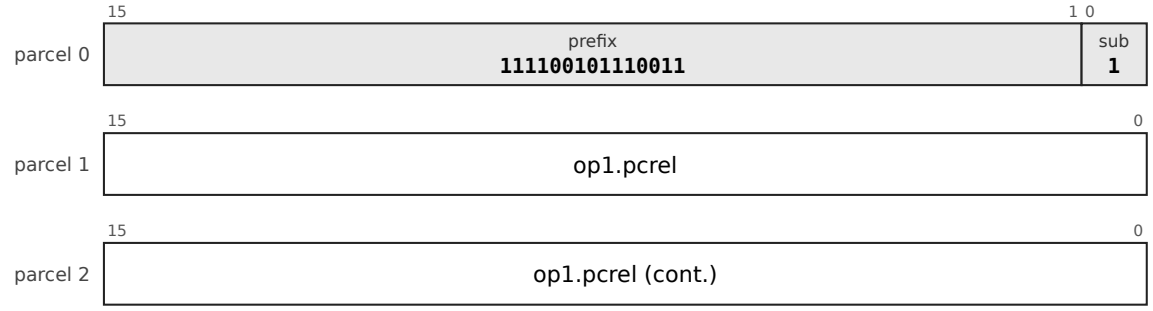

**BLE: BR2_16/c, prefix 11110010100000, sub-opcode 01; operands pc16-u2; 4 bytes**



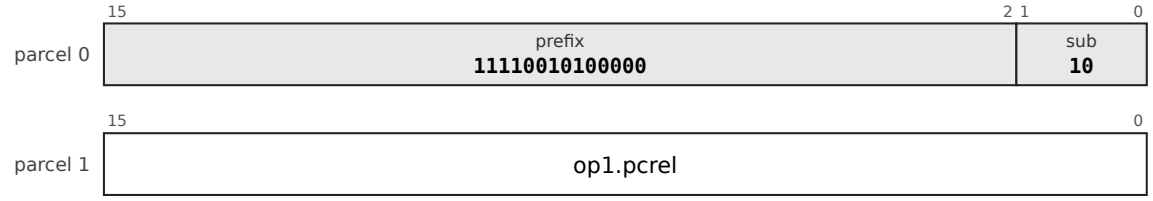

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BHI*

**BHI: BRF, prefix 111100101110011, sub-opcode 1; operands pc32-u2; 6 bytes**



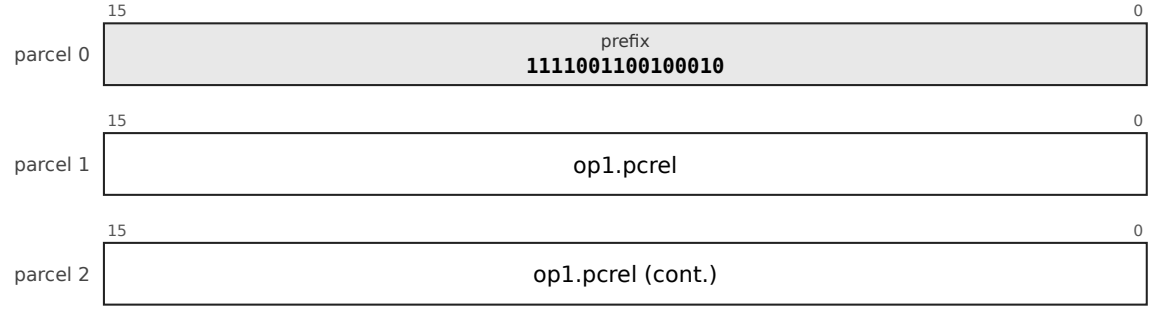

**BHI: BR2_16/c, prefix 11110010100000, sub-opcode 10; operands pc16-u2; 4 bytes**



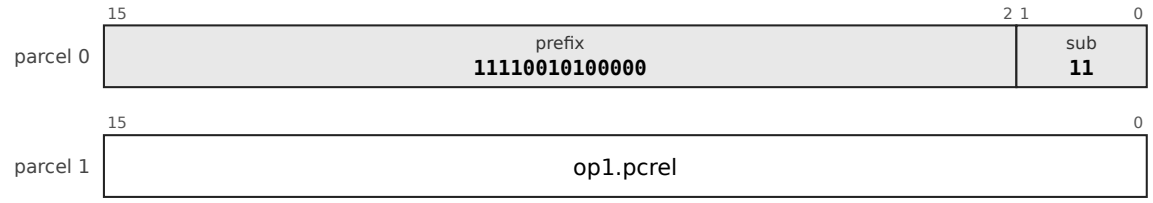

Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

### *BLOS*

**BLOS: BRF, prefix 1111001100100010; operands pc32-u2; 6 bytes**




**BLOS: BR2_16/c, prefix 11110010100000, sub-opcode 11; operands pc16-u2; 4 bytes**




Operation, flags and exceptions as for the entry BCC above; 2 layout forms, each a chart of its own (above).

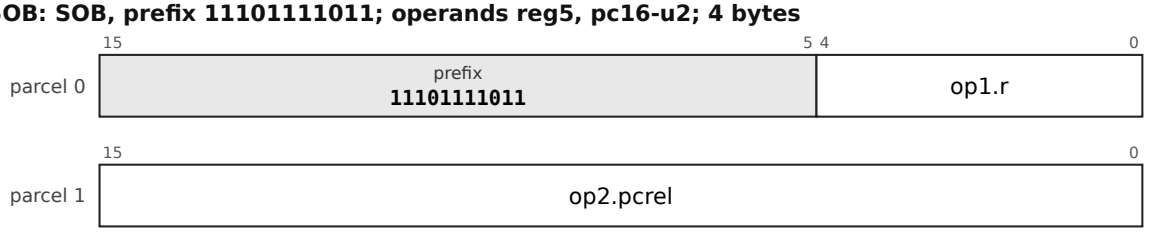
### *BHIS*

An alias of BCC: the assembler writes it as that word.

### *BLO*

An alias of BCS: the assembler writes it as that word.

## SOB: Subtract one and branch

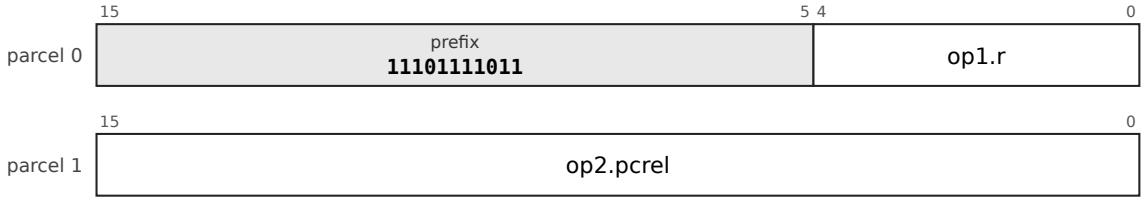



**Mnemonics:** `SOB`

### Syntax

```
SOB Rn, target
```

### Encoding




The chart is the leaf `SOB-1`, SOB, prefix `11101111011`. `op1` is `reg5`: register, 5 bits. `op2` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOB-1`.

### Operation

```
Rn ← Rn - 1
if Rn ≠ 0 then PC ← address of this instruction + 2 × disp
```

**Flags** None written.

**Exceptions** None.

From the PDP-11, where it could branch only backward; here the signed displacement goes either way. Four bytes at least.

**Why it is here.** Taken from the PDP-11. The PDP-11's could only branch backward (a 6-bit unsigned displacement); this one branches either way. Against the `cbnz` idiom of other architectures it does not save bytes (`DEC` plus `BNE` is also four in the original encoding) but it saves an instruction issue, a cycle per iteration on a machine that does not pipeline: same size, one instruction fewer (static `dec; bne` pairs exist).

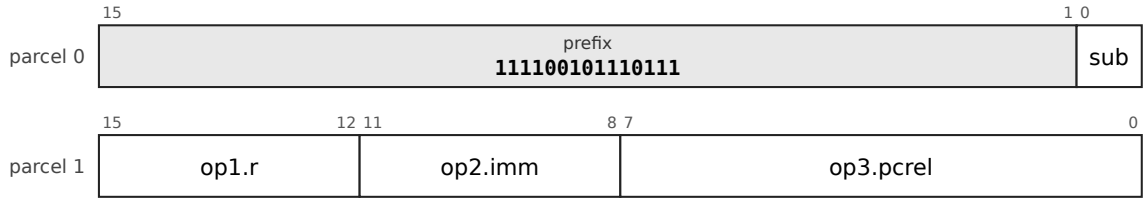
## CBCC: Compare and branch on condition

**Mnemonics:** `CBNE`, `CBEQ`, `CBPL`, `CBMI`, `CBVC`, `CBVS`, `CBCC`, `CBCS`, `CBGE`, `CBLT`, `CBGT`, `CBLE`, `CBHI`, `CBLOS` (aliases: `CBHIS`, `CBLO`)

### Syntax

```
CBNE Rd, #imm, target
CBEQ Ra, Rb, target ... and the rest
```

### Encoding




The chart is the leaf `CBI4-8H-a-1`, CBI4_8H/a, prefix `111100101110111`. `op1` is `reg4`: register, 4 bits (an index into the hot-register table). `op2` is `imm4s`: signed 4-bit immediate, -8 to 7. `op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes). `sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `CBI4-8H-a-1`, `CBR8H-a-1`, `CBZ2-12H-a-1`, `CBI5-8H-a-1`, `CBI3-8-a-1`, `CBI6-8H-a-1`, `CBI10-8H-a-1`, `CB3I8-20H-a-1`, `CB3R24H-a-1`, `CBI5-10H-a-1`, `CBR8-a-1`, `CB3I16-12-a-1`, `CBZ1-4-a-1`, `CBI6-10H-a-1`, `CBR10H-a-1`, `CBI8-10H-a-1`, `CBI3-12H-a-1`, `CBI3-10-a-1`, `CBZ2-14H-a-1`, `CBI4-8-a-1`, `CBI5-8-a-1`, `CBDI-1`, `CBDR-1`, `CBR8H-c-1`, `CB3I16-12H-c-1`, `CBI4-8H-c-1`, `CBR8-c-1`, `CB3R24H-c-1`, `CBI6-8H-c-1`, `CB3I12-16-c-1`, `CBR10H-c-1`, `CBI8-8H-c-1`, `CBI4-10H-c-1`, `CBI3-8-c-1`, `CB3I16-12H-b-1`, `CBR8H-b-1`, `CBI8-8H-b-1`, `CBZ2-12H-b-1`, `CB3I8-20-b-1`, `CBR8-b-1`, `CBI3-8-b-1`, `CBI4-10H-b-1`, `CB3R24H-b-1`, `CBI4-8-b-1`.

### Operation

```
tmp ← Rd - (imm or Rb)         ; as CMP, for the decision only
if cond(c) on that comparison then PC ← address of this instruction + 2 × disp
```

**Flags None written**: the instruction computes the comparison for its own decision and leaves `N Z V C` as they were.

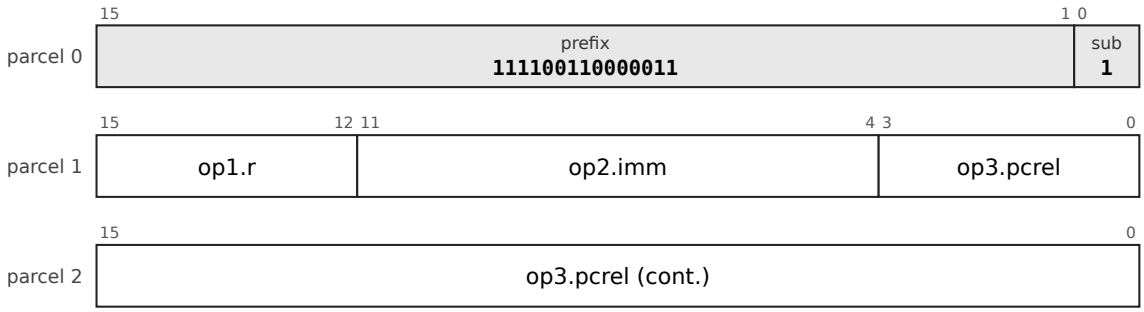
**Exceptions** None.

The family has several forms for each condition (the layouts of this entry): the second operand is a register or an immediate of some width, and the reach differs. A `cbCC` that no form reaches is assembled by `as64` as `CMP` and `bCC`, which **writes the flags** (errata 87, assumption 6; the integration measured 17 such sites in 34,524, errata 90).

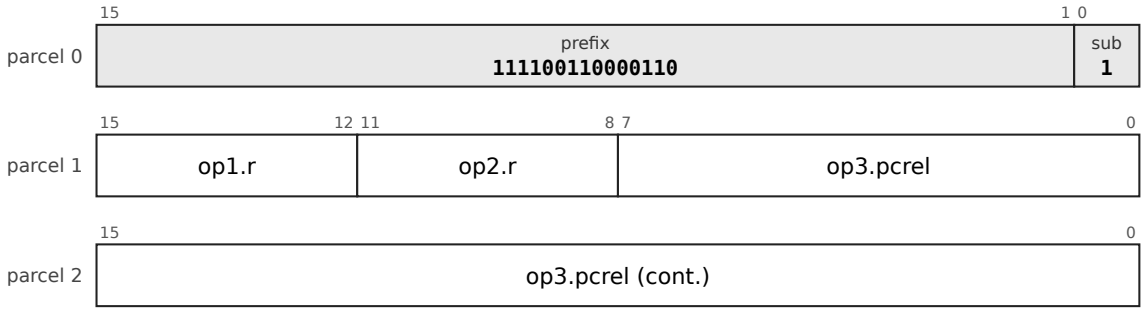
**Why it is here.** A fused compare-and-branch, estimated with `RORI32` at 22 KB (1.3%) of the image of the original encoding. There the wide-immediate form removes 11,735 instructions (3.4%) and 1,828 bytes, and the register-register compare-and-branch removes 4,631 instructions (1.4%) and 142 bytes. It is not a replacement for flags: LoongArch and RISC-V chose fused compare-and-branch instead of flags, and this architecture declines that (9.20) because it needs a comparator in the branch unit and a third read port; here the fused form is added beside the flags. `CBCC` writes no flags.

### *CBNE*

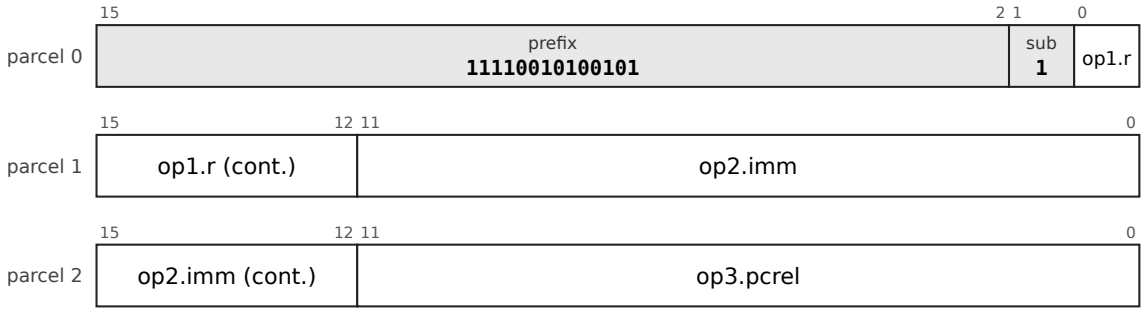
**CBNE: CB3I8_20H/a, prefix 111100110000011, sub-opcode 1; operands reg4, imm8s, pc20-u2**



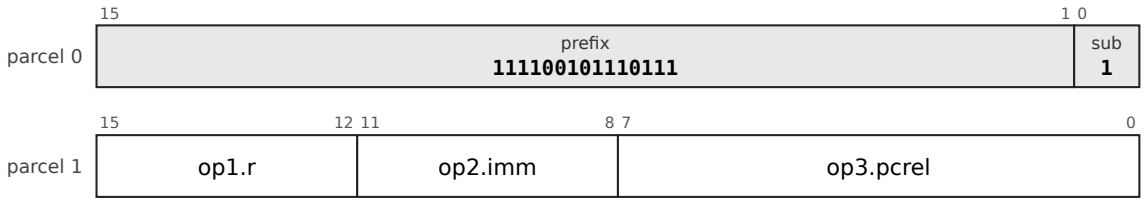

**CBNE: CB3R24H/a, prefix 111100110000110, sub-opcode 1; operands reg4, reg4, pc24-u2; 6 l**



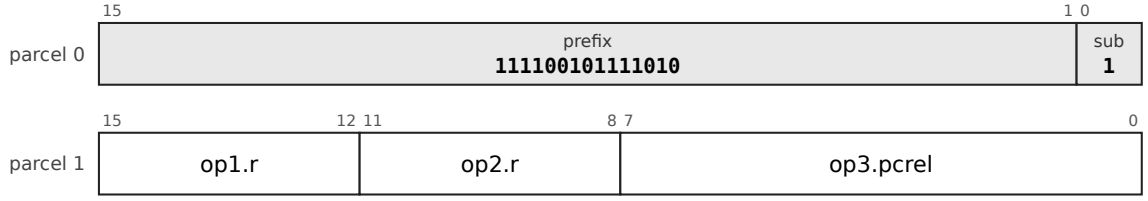

**CBNE: CB3I16_12/a, prefix 11110010100101, sub-opcode 1; operands reg5, imm16s, pc12-u2**



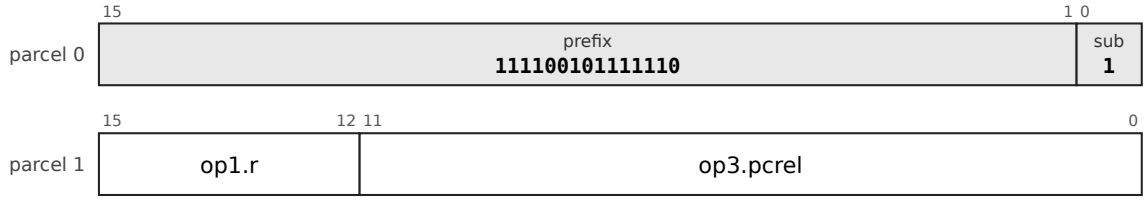

**CBNE: CBI4_8H/a, prefix 111100101110111, sub-opcode 1; operands reg4, imm4s, pc8-u2; 4 l**



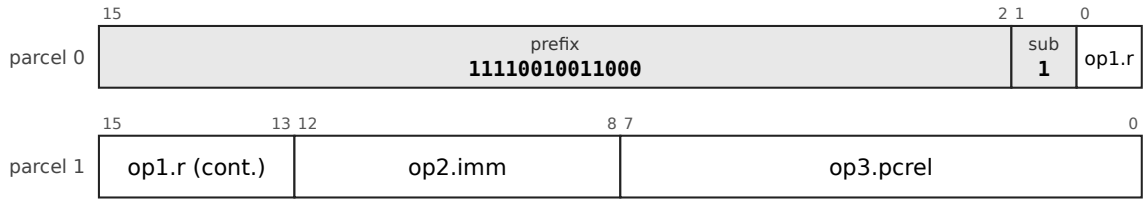
**CBNE: CBR8H/a, prefix 111100101111010, sub-opcode 1; operands reg4, reg4, pc8-u2; 4 byte**



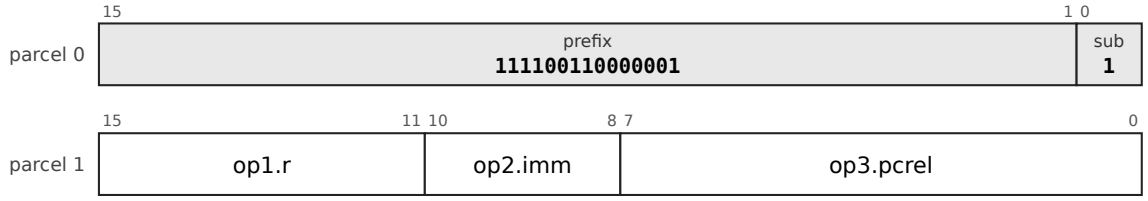

**CBNE: CBZ2_12H/a, prefix 111100101111110, sub-opcode 1; operands reg4, k-imm0, pc12-u2**



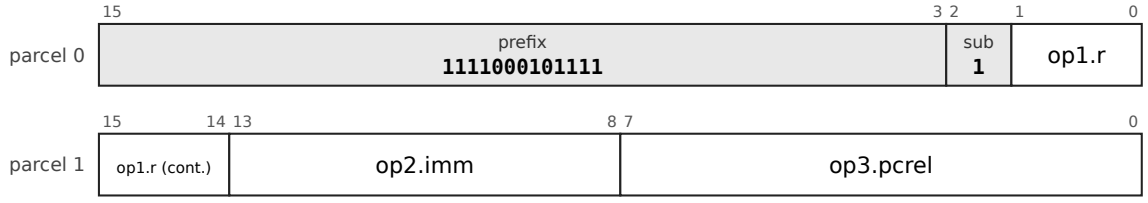

**CBNE: CBI5_8H/a, prefix 11110010011000, sub-opcode 1; operands reg4, imm5s, pc8-u2; 4 by**



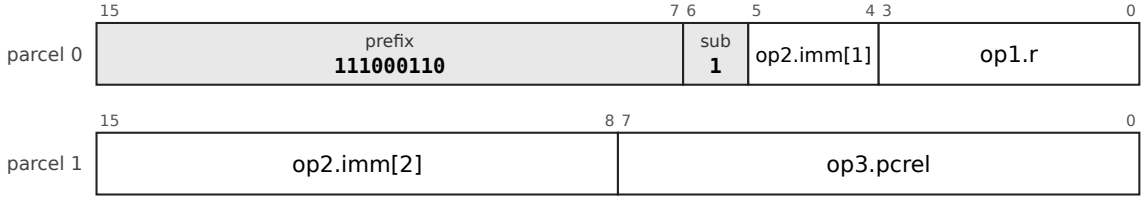

**CBNE: CBI3_8/a, prefix 111100110000001, sub-opcode 1; operands reg5, imm3s, pc8-u2; 4 by**



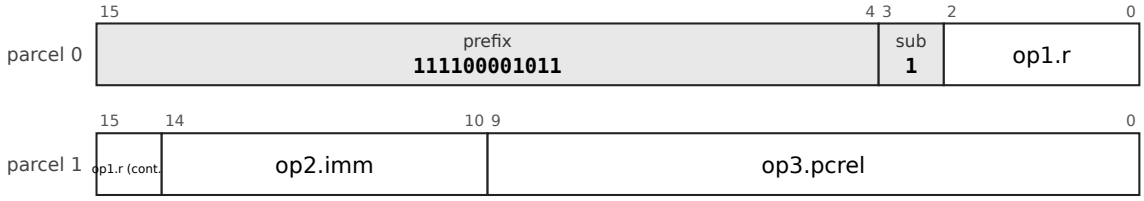

**CBNE: CBI6_8H/a, prefix 1111000101111, sub-opcode 1; operands reg4, imm6s, pc8-u2; 4 byt**



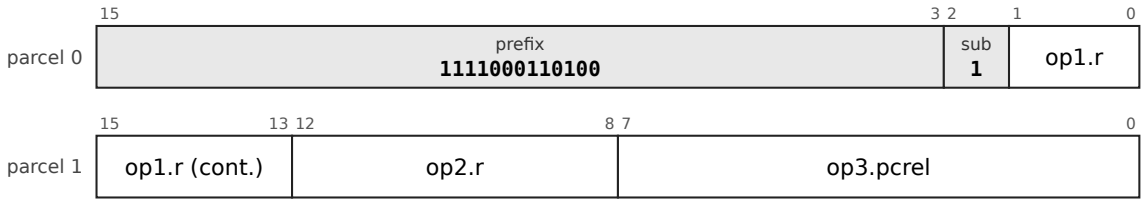
**CBNE: CBI10_8H/a, prefix 111000110, sub-opcode 1; operands reg4, imm10s, pc8-u2; 4 bytes**



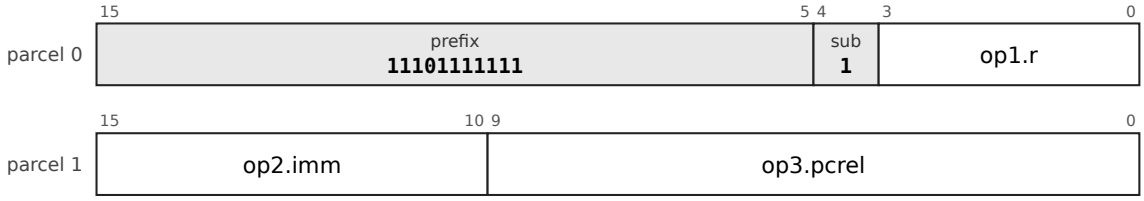

**CBNE: CBI5_10H/a, prefix 111100001011, sub-opcode 1; operands reg4, imm5s, pc10-u2; 4 by**



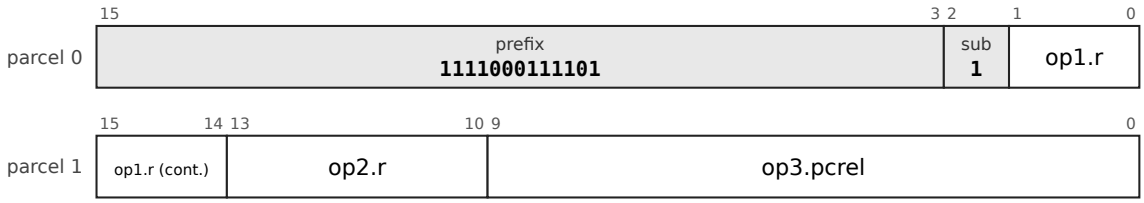

**CBNE: CBR8/a, prefix 1111000110100, sub-opcode 1; operands reg5, reg5, pc8-u2; 4 bytes**



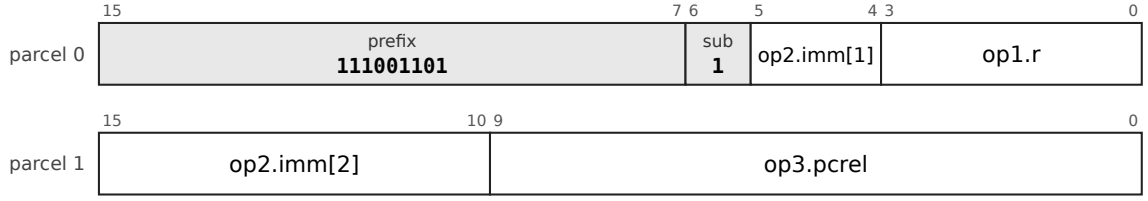

**CBNE: CBI6_10H/a, prefix 11101111111, sub-opcode 1; operands reg4, imm6s, pc10-u2; 4 byt**



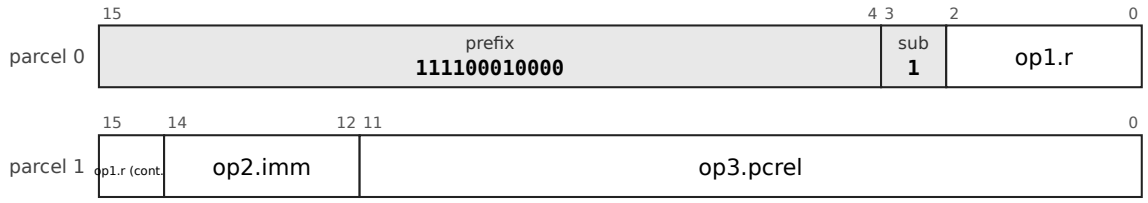

**CBNE: CBR10H/a, prefix 1111000111101, sub-opcode 1; operands reg4, reg4, pc10-u2; 4 byte**



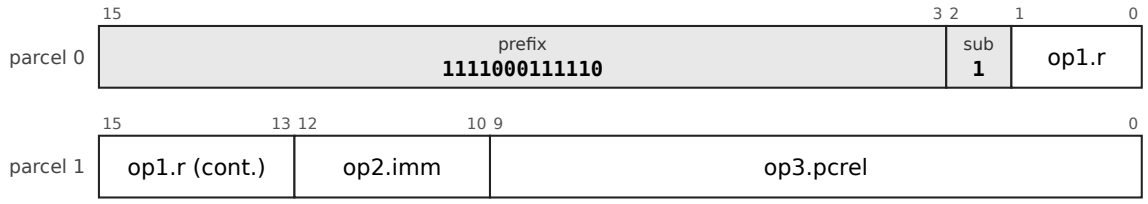
**CBNE: CBI8_10H/a, prefix 111001101, sub-opcode 1; operands reg4, imm8s, pc10-u2; 4 bytes**



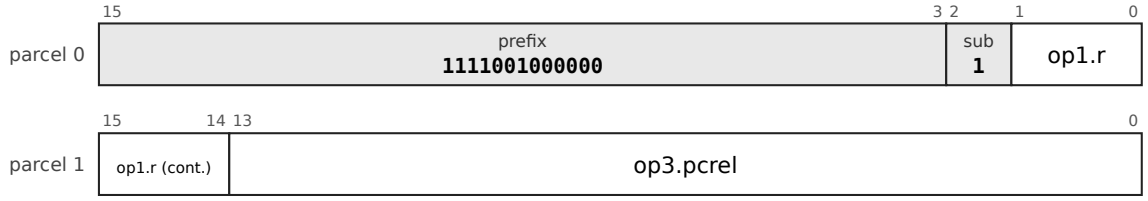

**CBNE: CBI3_12H/a, prefix 111100010000, sub-opcode 1; operands reg4, imm3s, pc12-u2; 4 by**



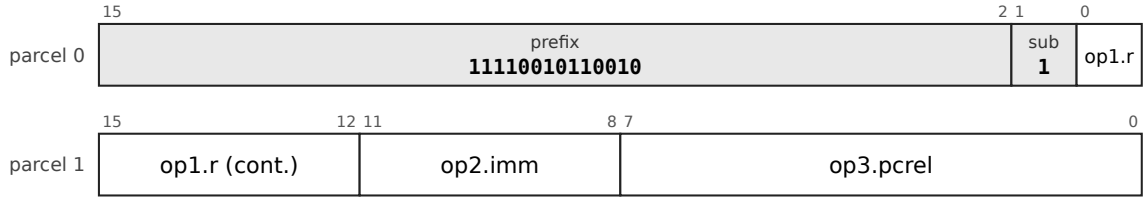

**CBNE: CBI3_10/a, prefix 1111000111110, sub-opcode 1; operands reg5, imm3s, pc10-u2; 4 by**



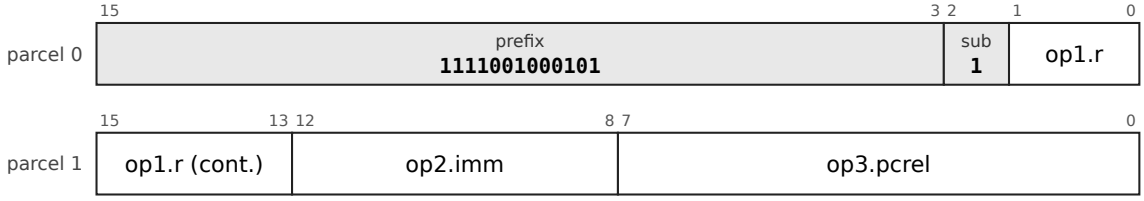

**CBNE: CBZ2_14H/a, prefix 1111001000000, sub-opcode 1; operands reg4, k-imm0, pc14-u2; 4**



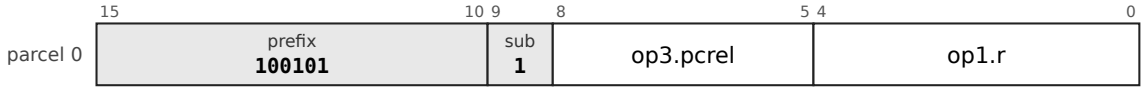

**CBNE: CBI4_8/a, prefix 11110010110010, sub-opcode 1; operands reg5, imm4s, pc8-u2; 4 byt**



**CBNE: CBI5_8/a, prefix 1111001000101, sub-opcode 1; operands reg5, imm5s, pc8-u2; 4 byte**




**CBNE: CBZ1_4/a, prefix 100101, sub-opcode 1; operands reg5, k-imm0, pc4-u2; 2 bytes**

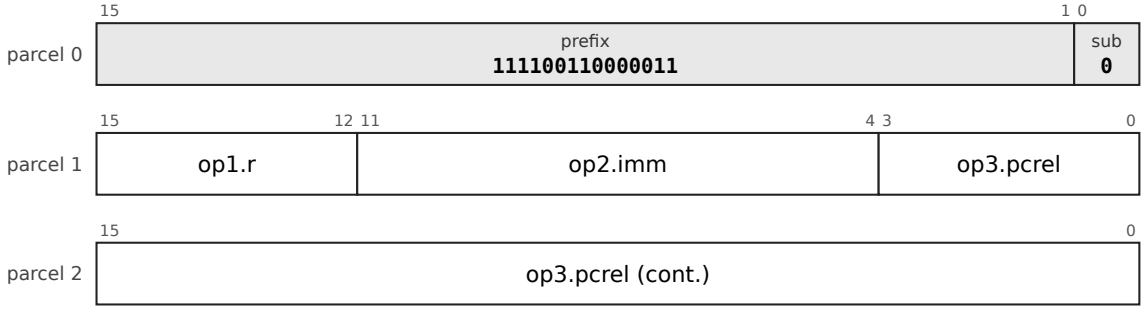



```
CBNE Rd, #imm, target
```

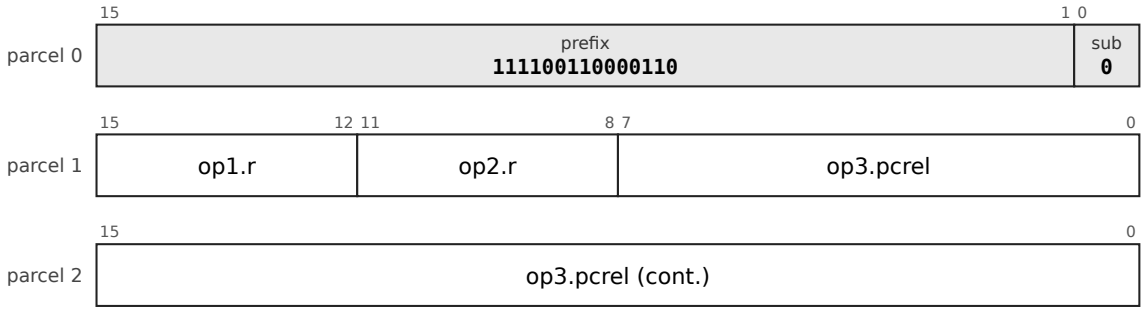
Operation, flags and exceptions as for the entry CBCC above; 21 layout forms, each a chart of its own (above).

### *CBEQ*

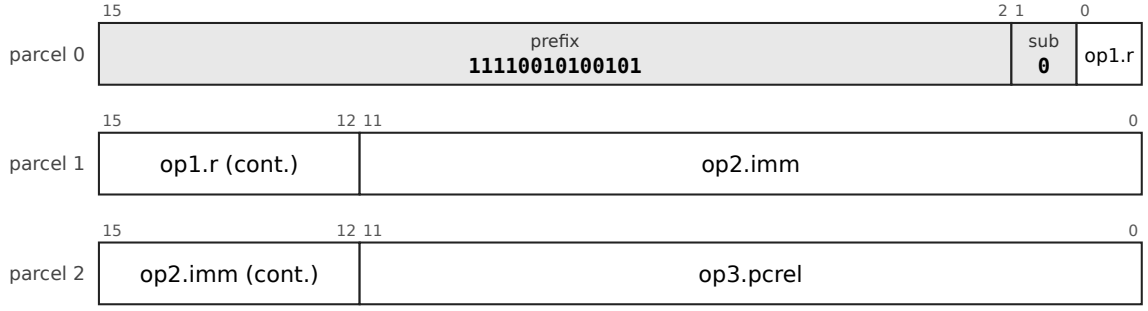
**CBEQ: CB3I8_20H/a, prefix 111100110000011, sub-opcode 0; operands reg4, imm8s, pc20-u2**



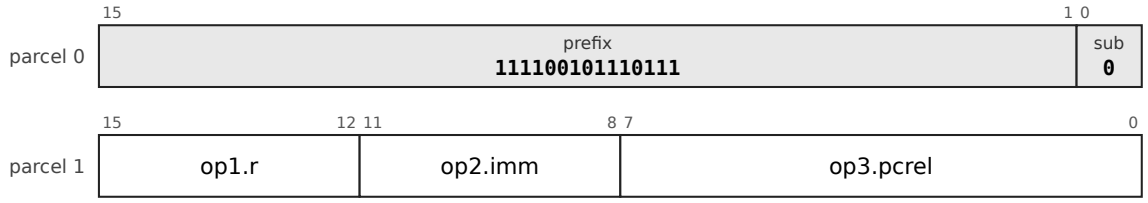

**CBEQ: CB3R24H/a, prefix 111100110000110, sub-opcode 0; operands reg4, reg4, pc24-u2; 6**



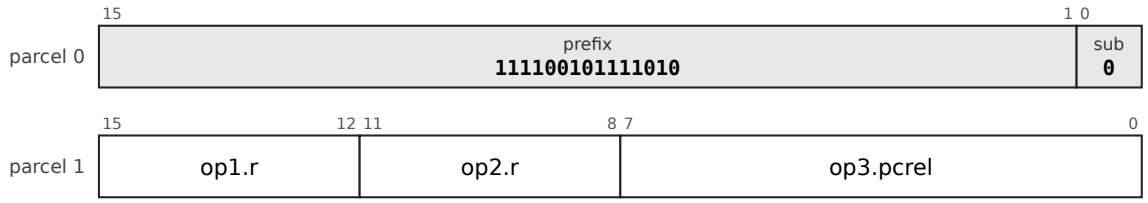
**CBEQ: CB3I16_12/a, prefix 11110010100101, sub-opcode 0; operands reg5, imm16s, pc12-u2**



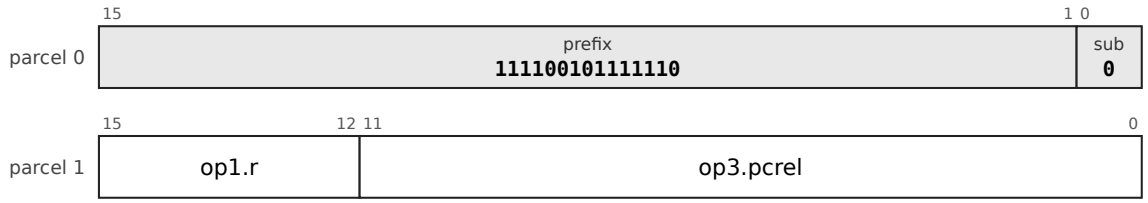

**CBEQ: CBI4_8H/a, prefix 111100101110111, sub-opcode 0; operands reg4, imm4s, pc8-u2; 4 I**



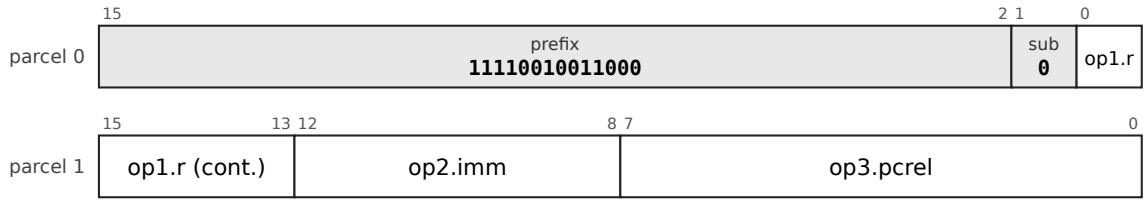

**CBEQ: CBR8H/a, prefix 111100101111010, sub-opcode 0; operands reg4, reg4, pc8-u2; 4 byte**



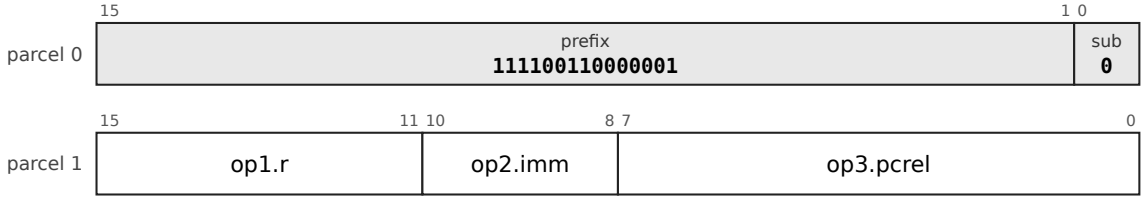

**CBEQ: CBZ2_12H/a, prefix 111100101111110, sub-opcode 0; operands reg4, k-imm0, pc12-u2**



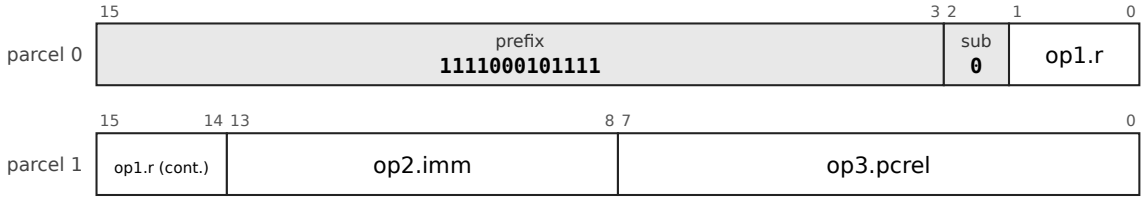

**CBEQ: CBI5_8H/a, prefix 11110010011000, sub-opcode 0; operands reg4, imm5s, pc8-u2; 4 by**



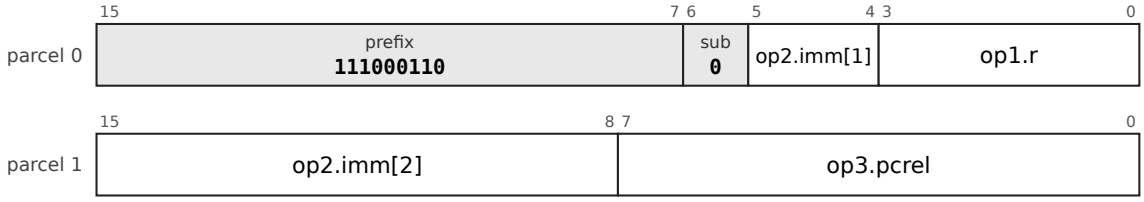
**CBEQ: CBI3_8/a, prefix 111100110000001, sub-opcode 0; operands reg5, imm3s, pc8-u2; 4 by**



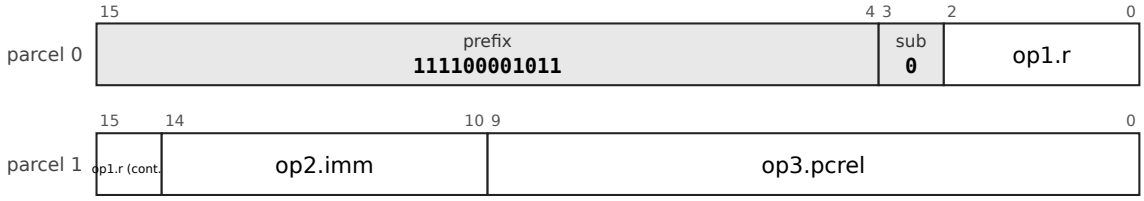

**CBEQ: CBI6_8H/a, prefix 1111000101111, sub-opcode 0; operands reg4, imm6s, pc8-u2; 4 byt**



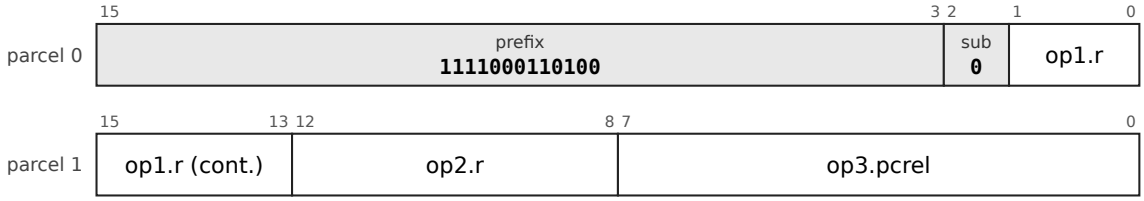

**CBEQ: CBI10_8H/a, prefix 111000110, sub-opcode 0; operands reg4, imm10s, pc8-u2; 4 bytes**



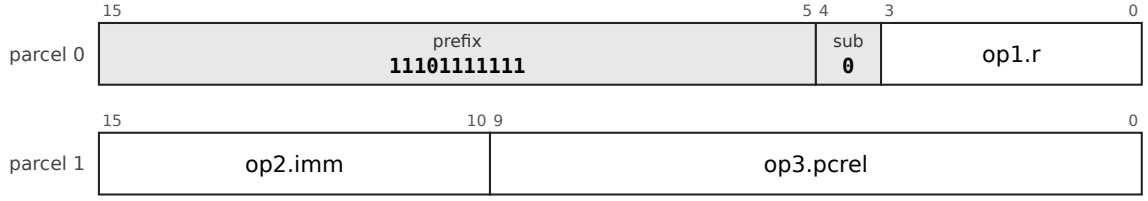

**CBEQ: CBI5_10H/a, prefix 111100001011, sub-opcode 0; operands reg4, imm5s, pc10-u2; 4 by**



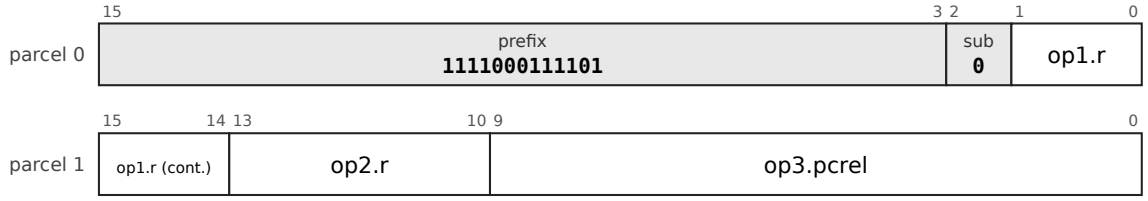

**CBEQ: CBR8/a, prefix 1111000110100, sub-opcode 0; operands reg5, reg5, pc8-u2; 4 bytes**



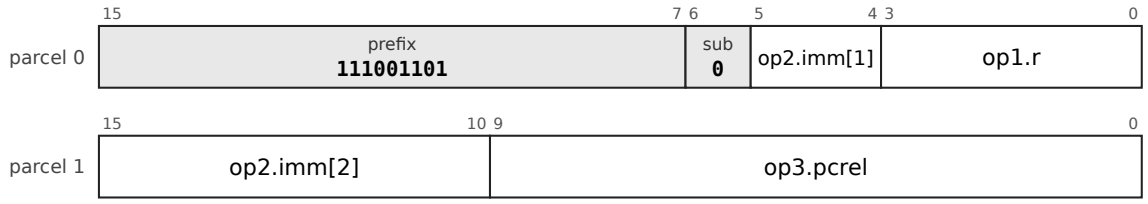
**CBEQ: CBI6_10H/a, prefix 11101111111, sub-opcode 0; operands reg4, imm6s, pc10-u2; 4 byt**



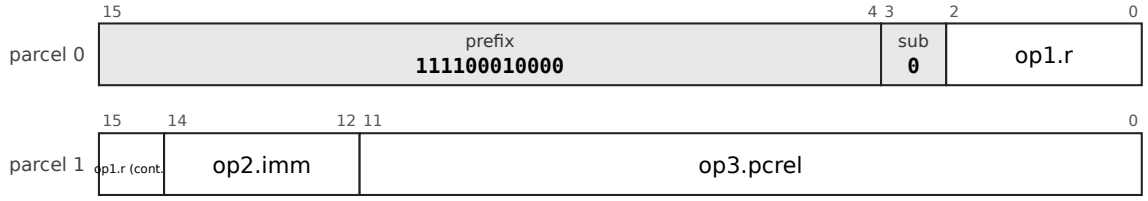

**CBEQ: CBR10H/a, prefix 1111000111101, sub-opcode 0; operands reg4, reg4, pc10-u2; 4 byte**



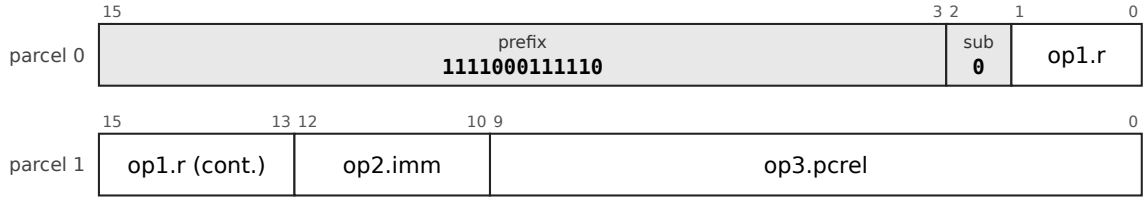

**CBEQ: CBI8_10H/a, prefix 111001101, sub-opcode 0; operands reg4, imm8s, pc10-u2; 4 bytes**



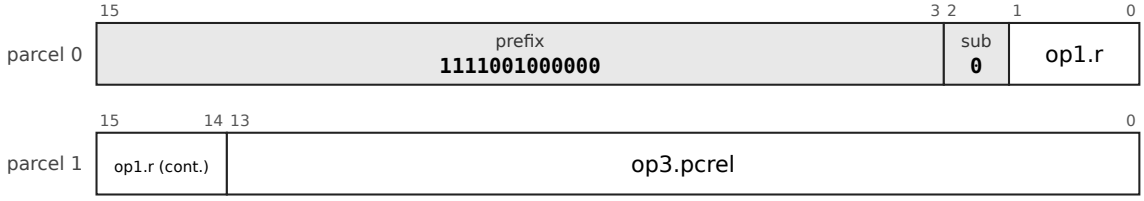

**CBEQ: CBI3_12H/a, prefix 111100010000, sub-opcode 0; operands reg4, imm3s, pc12-u2; 4 by**



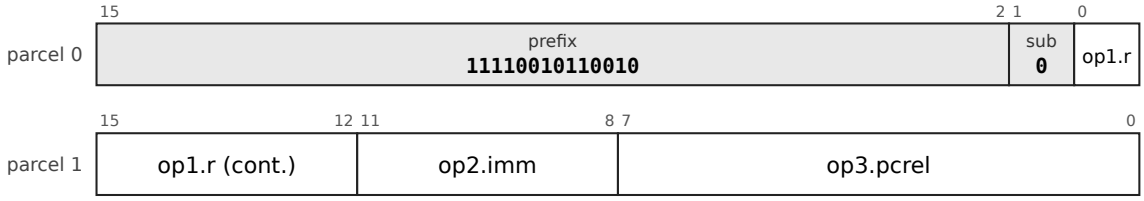

**CBEQ: CBI3_10/a, prefix 1111000111110, sub-opcode 0; operands reg5, imm3s, pc10-u2; 4 by**



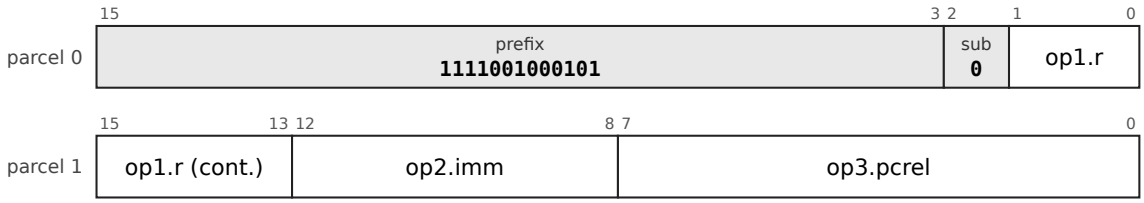
**CBEQ: CBZ2_14H/a, prefix 1111001000000, sub-opcode 0; operands reg4, k-imm0, pc14-u2; 4**



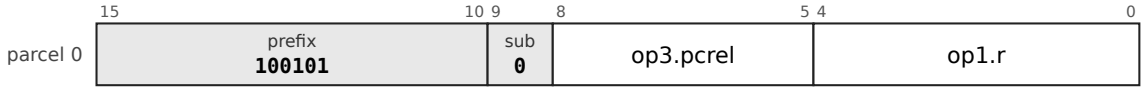

**CBEQ: CBI4_8/a, prefix 11110010110010, sub-opcode 0; operands reg5, imm4s, pc8-u2; 4 byt**




**CBEQ: CBI5_8/a, prefix 1111001000101, sub-opcode 0; operands reg5, imm5s, pc8-u2; 4 byte**




**CBEQ: CBZ1_4/a, prefix 100101, sub-opcode 0; operands reg5, k-imm0, pc4-u2; 2 bytes**

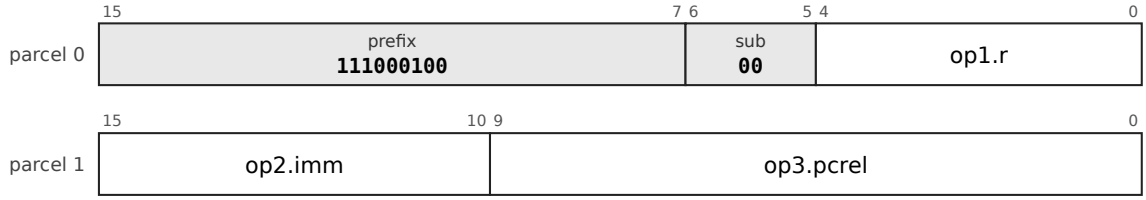



```
CBEQ Ra, Rb, target ... and the rest
```

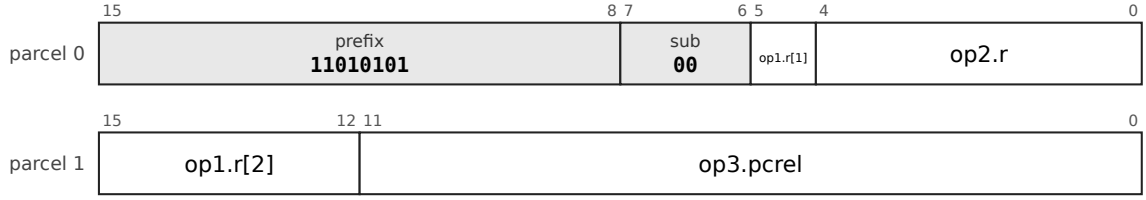
Operation, flags and exceptions as for the entry CBCC above; 21 layout forms, each a chart of its own (above).

### *CBPL*

**CBPL: CBDI, prefix 111000100, sub-opcode 00; operands reg5, imm6s, pc10-u2; 4 bytes**



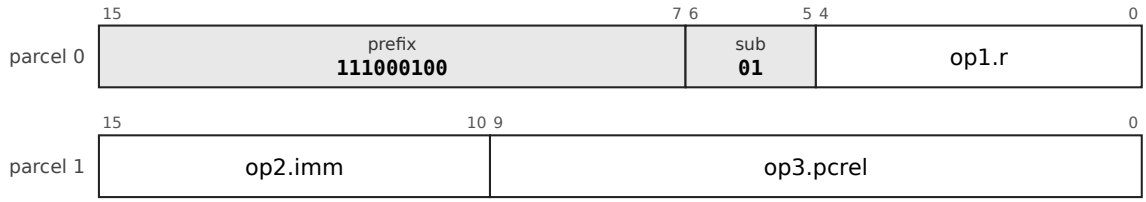

**CBPL: CBDR, prefix 11010101, sub-opcode 00; operands reg5, reg5, pc12-u2; 4 bytes**



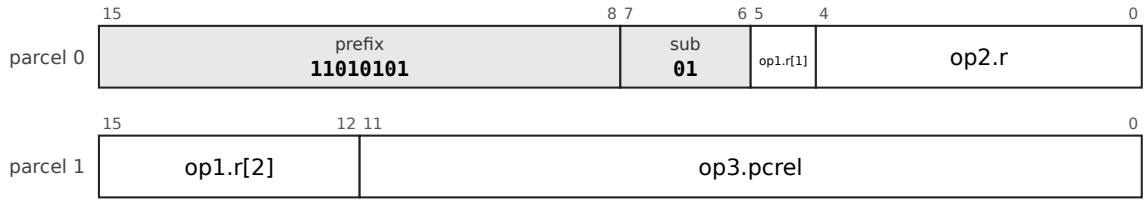

Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

### *CBMI*

**CBMI: CBDI, prefix 111000100, sub-opcode 01; operands reg5, imm6s, pc10-u2; 4 bytes**



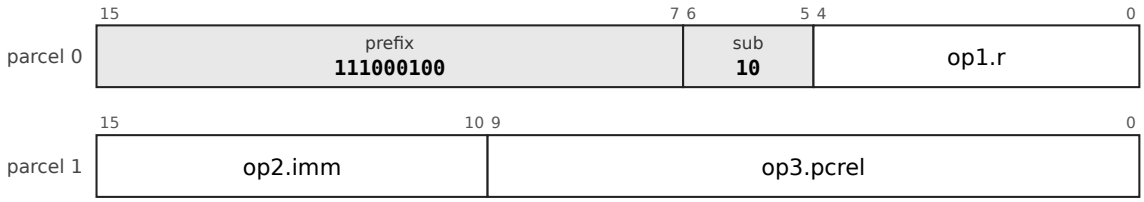

**CBMI: CBDR, prefix 11010101, sub-opcode 01; operands reg5, reg5, pc12-u2; 4 bytes**



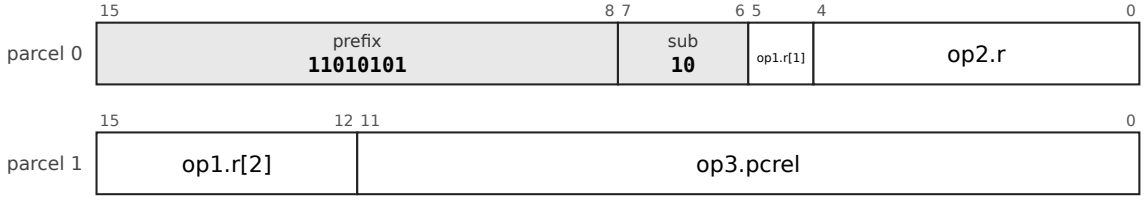

Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

### *CBVC*

**CBVC: CBDI, prefix 111000100, sub-opcode 10; operands reg5, imm6s, pc10-u2; 4 bytes**



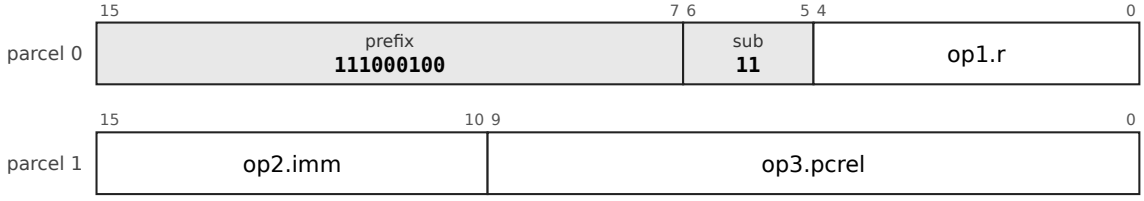

**CBVC: CBDR, prefix 11010101, sub-opcode 10; operands reg5, reg5, pc12-u2; 4 bytes**



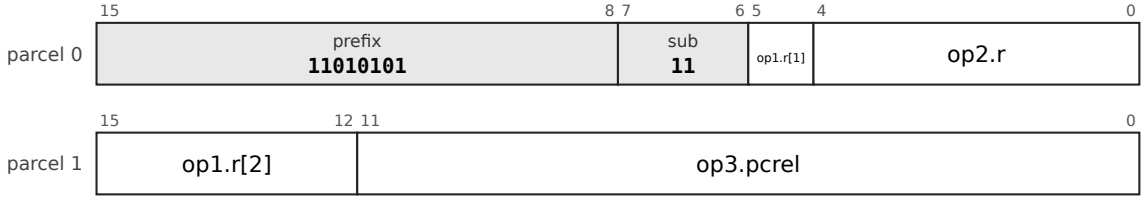

Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

### *CBVS*

**CBVS: CBDI, prefix 111000100, sub-opcode 11; operands reg5, imm6s, pc10-u2; 4 bytes**



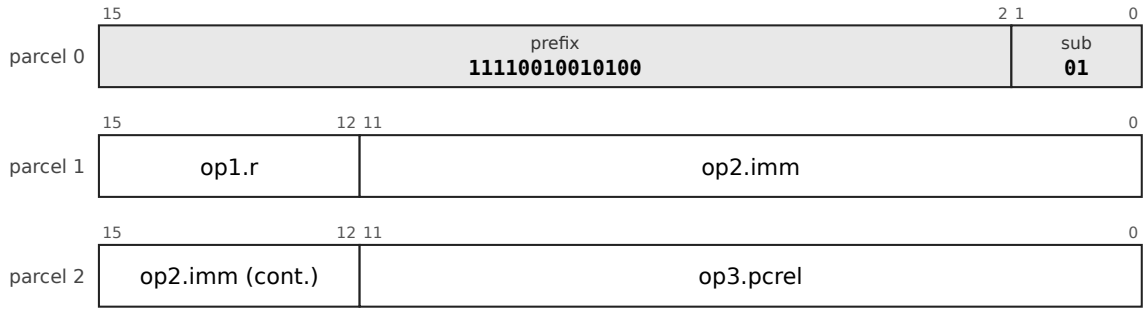

**CBVS: CBDR, prefix 11010101, sub-opcode 11; operands reg5, reg5, pc12-u2; 4 bytes**



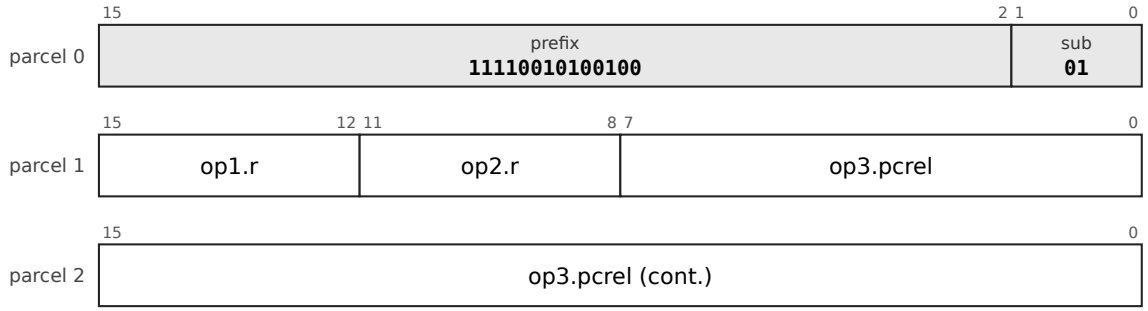

Operation, flags and exceptions as for the entry CBCC above; 2 layout forms, each a chart of its own (above).

### *CBCC*

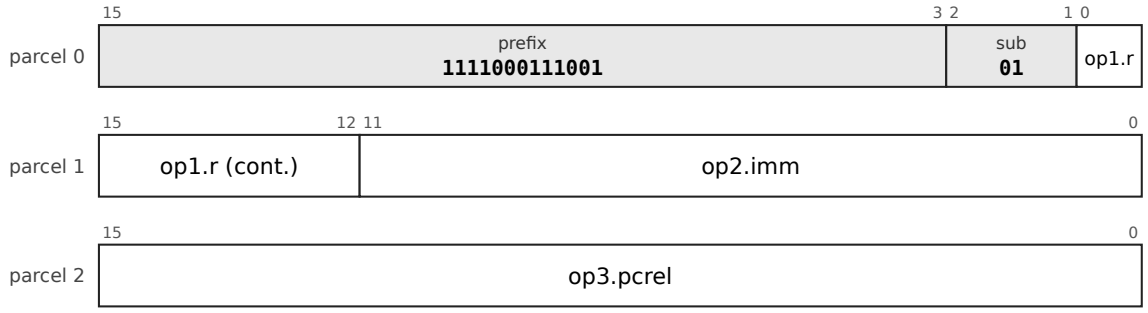
**CBCC: CB3I16_12H/c, prefix 11110010010100, sub-opcode 01; operands reg4, imm16s, pc12-**



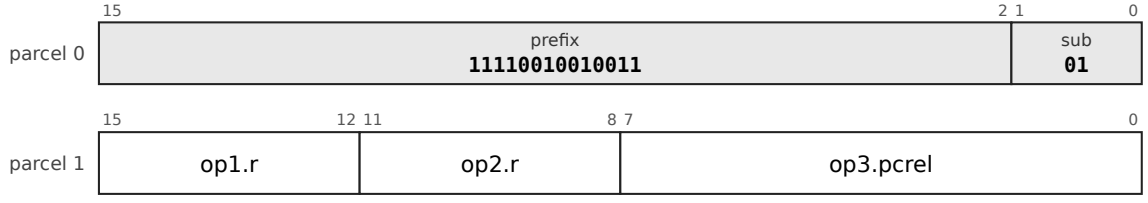

**CBCC: CB3R24H/c, prefix 11110010100100, sub-opcode 01; operands reg4, reg4, pc24-u2; 6 l**



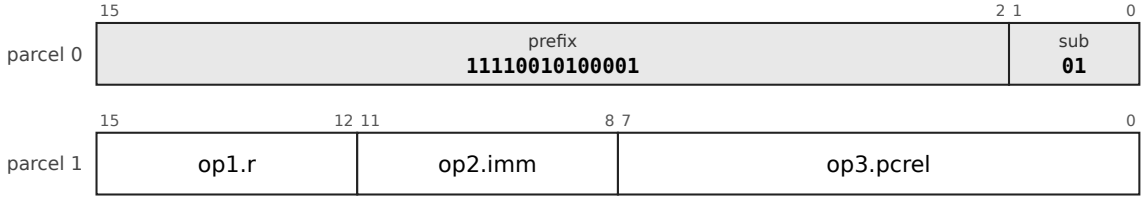

**CBCC: CB3I12_16/c, prefix 1111000111001, sub-opcode 01; operands reg5, imm12s, pc16-u2;**



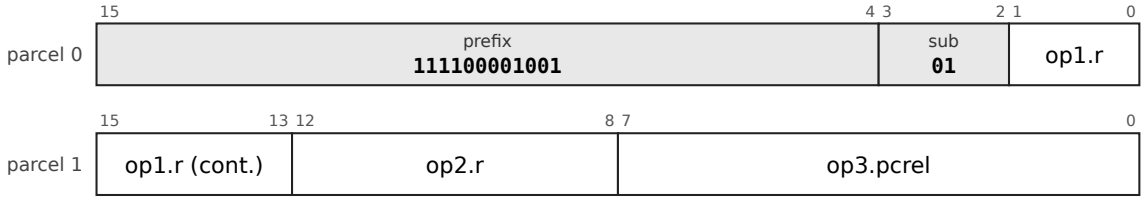

**CBCC: CBR8H/c, prefix 11110010010011, sub-opcode 01; operands reg4, reg4, pc8-u2; 4 byte**



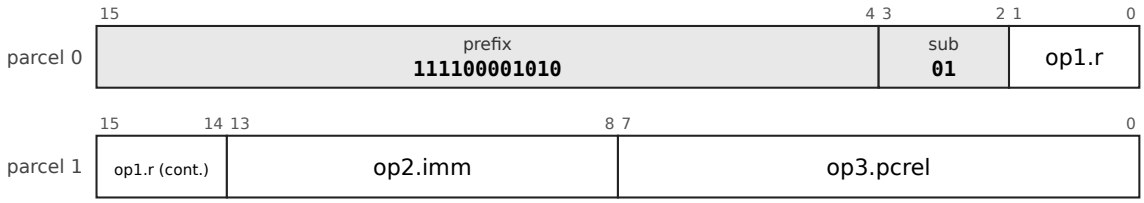
**CBCC: CBI4_8H/c, prefix 11110010100001, sub-opcode 01; operands reg4, imm4s, pc8-u2; 4 b**



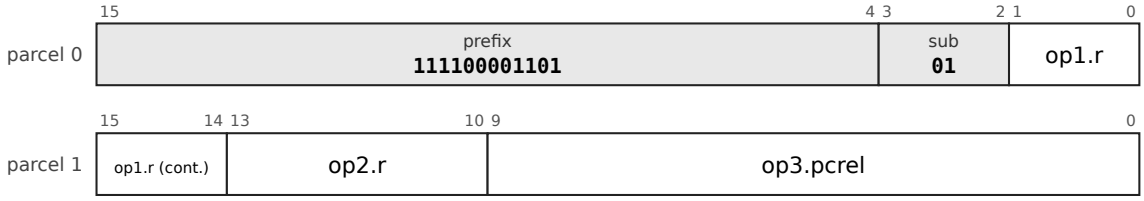

**CBCC: CBR8/c, prefix 111100001001, sub-opcode 01; operands reg5, reg5, pc8-u2; 4 bytes**



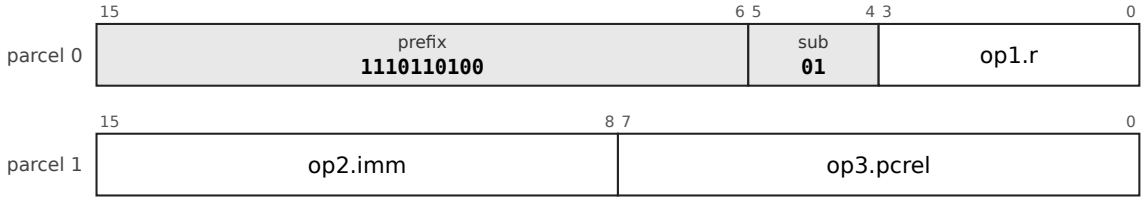

**CBCC: CBI6_8H/c, prefix 111100001010, sub-opcode 01; operands reg4, imm6s, pc8-u2; 4 byt**



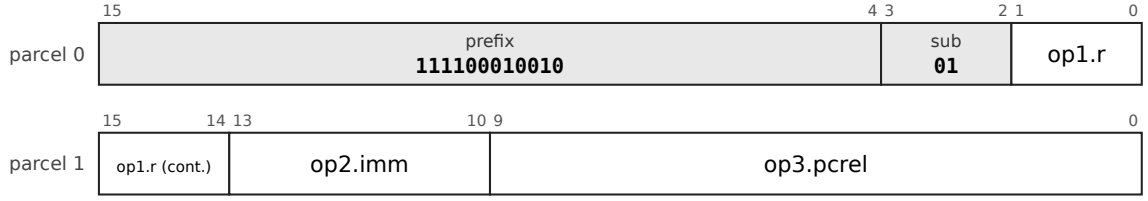

**CBCC: CBR10H/c, prefix 111100001101, sub-opcode 01; operands reg4, reg4, pc10-u2; 4 byte**



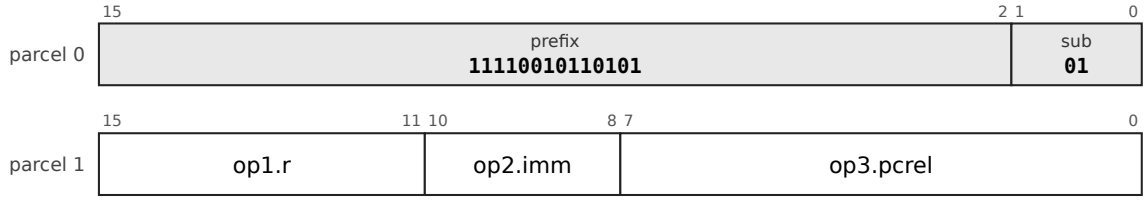

**CBCC: CBI8_8H/c, prefix 1110110100, sub-opcode 01; operands reg4, imm8s, pc8-u2; 4 bytes**



**CBCC: CBI4_10H/c, prefix 111100010010, sub-opcode 01; operands reg4, imm4s, pc10-u2; 4 b**



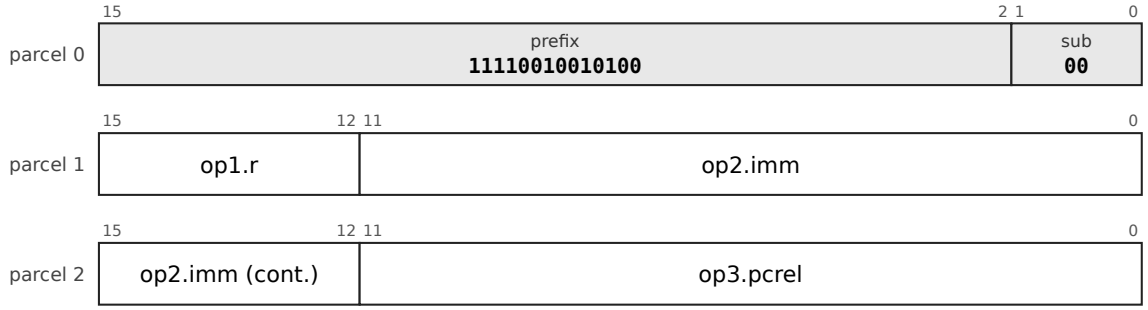

**CBCC: CBI3_8/c, prefix 11110010110101, sub-opcode 01; operands reg5, imm3s, pc8-u2; 4 by**



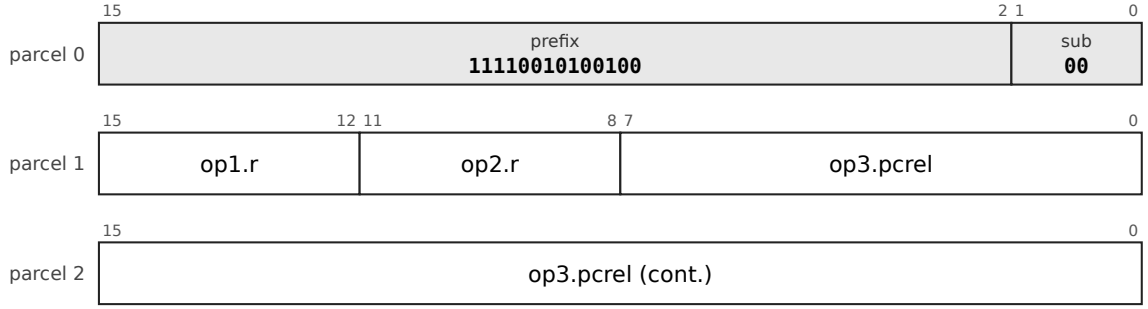

Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

***CBCS***

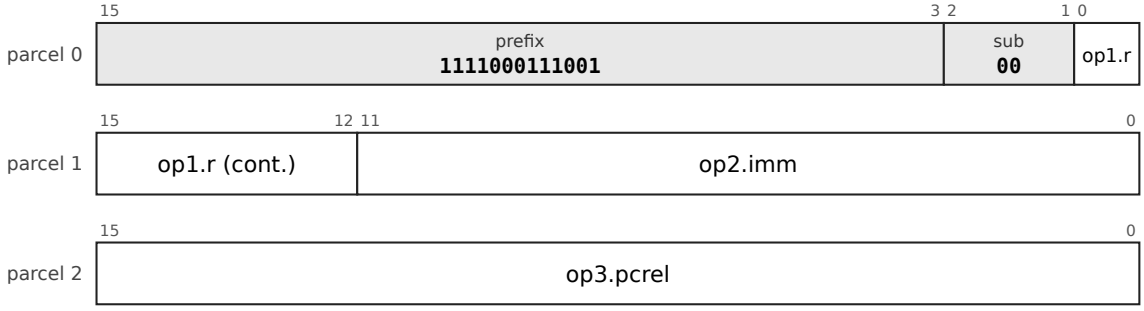
**CBCS: CB3I16_12H/c, prefix 11110010010100, sub-opcode 00; operands reg4, imm16s, pc12-**



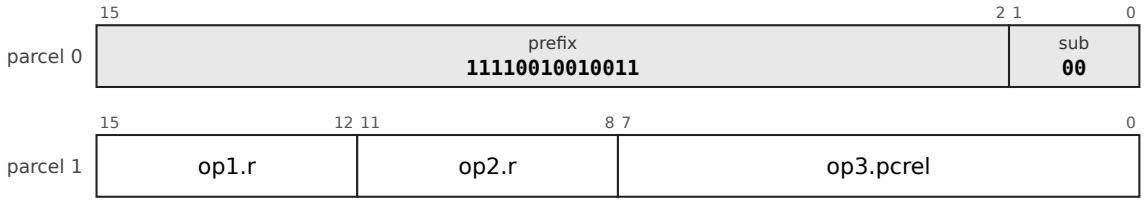

**CBCS: CB3R24H/c, prefix 11110010100100, sub-opcode 00; operands reg4, reg4, pc24-u2; 6 l**



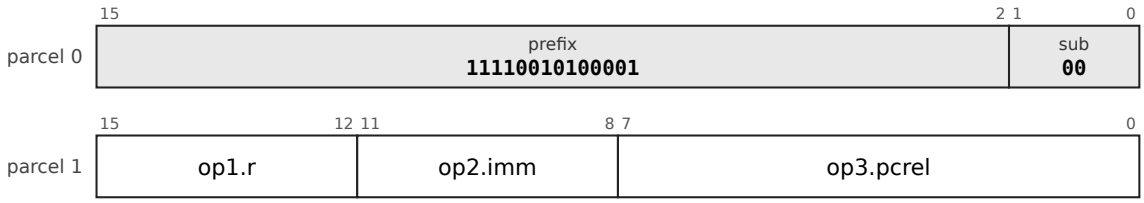
**CBCS: CB3I12_16/c, prefix 1111000111001, sub-opcode 00; operands reg5, imm12s, pc16-u2;**



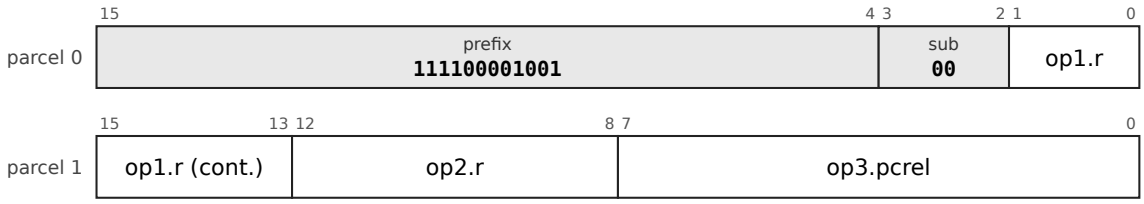

**CBCS: CBR8H/c, prefix 11110010010011, sub-opcode 00; operands reg4, reg4, pc8-u2; 4 byte**



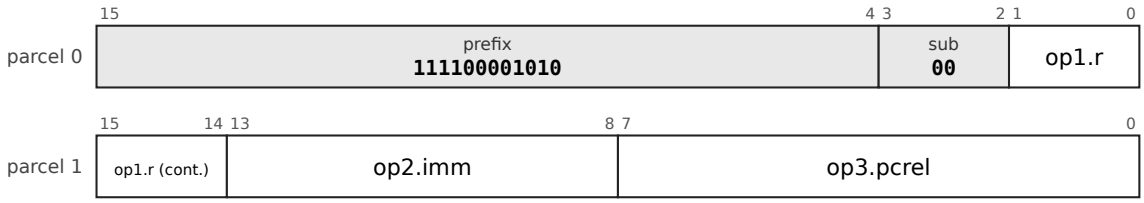

**CBCS: CBI4_8H/c, prefix 11110010100001, sub-opcode 00; operands reg4, imm4s, pc8-u2; 4 b**



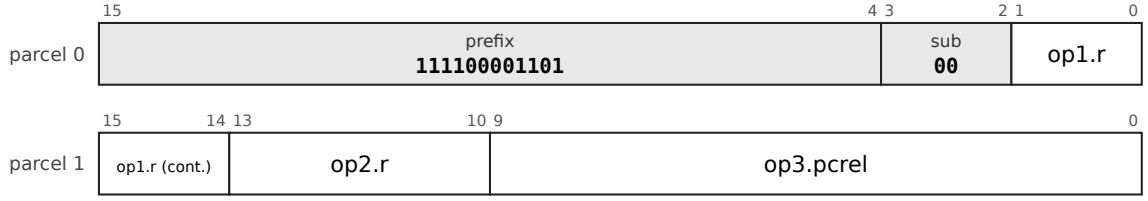

**CBCS: CBR8/c, prefix 111100001001, sub-opcode 00; operands reg5, reg5, pc8-u2; 4 bytes**



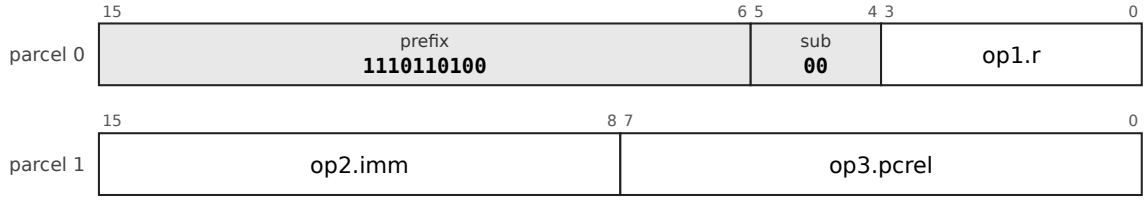

**CBCS: CBI6_8H/c, prefix 111100001010, sub-opcode 00; operands reg4, imm6s, pc8-u2; 4 byt**



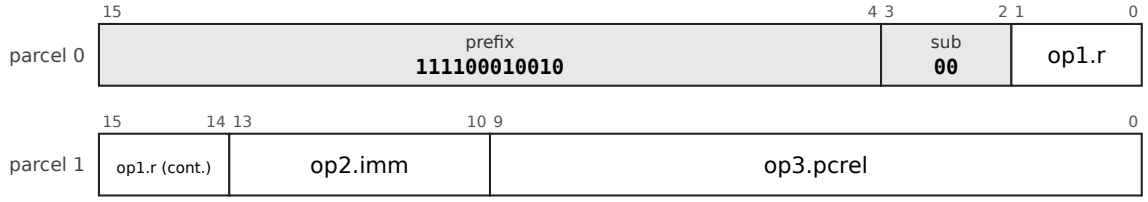
**CBCS: CBR10H/c, prefix 111100001101, sub-opcode 00; operands reg4, reg4, pc10-u2; 4 byte**



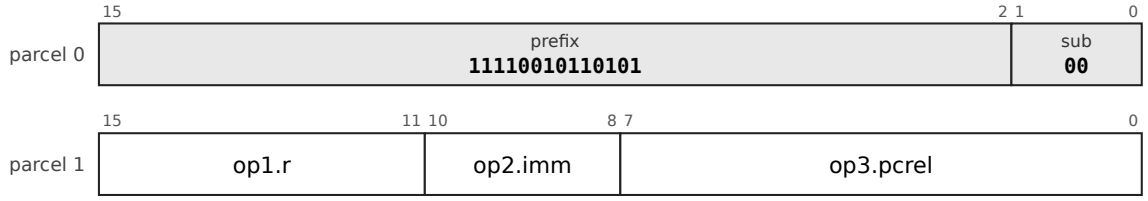

**CBCS: CBI8_8H/c, prefix 1110110100, sub-opcode 00; operands reg4, imm8s, pc8-u2; 4 bytes**




**CBCS: CBI4_10H/c, prefix 111100010010, sub-opcode 00; operands reg4, imm4s, pc10-u2; 4 b**



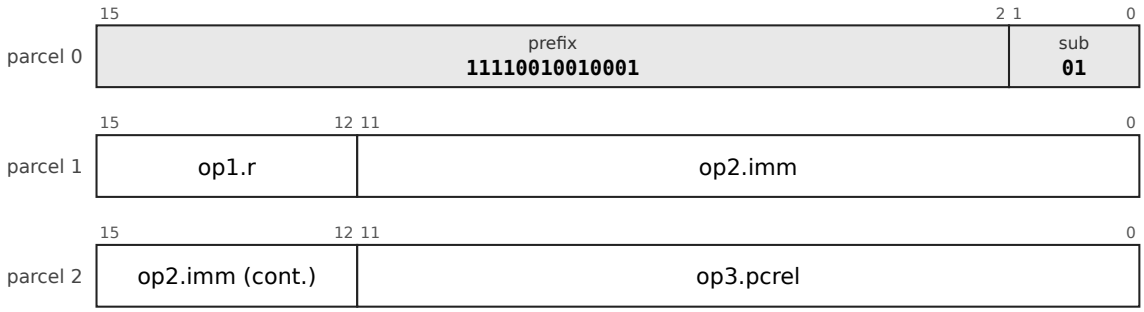

**CBCS: CBI3_8/c, prefix 11110010110101, sub-opcode 00; operands reg5, imm3s, pc8-u2; 4 by**



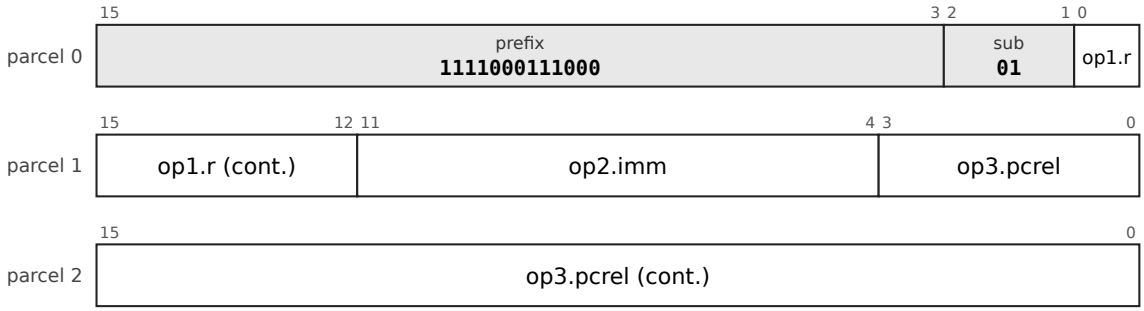

Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

### *CBGE*

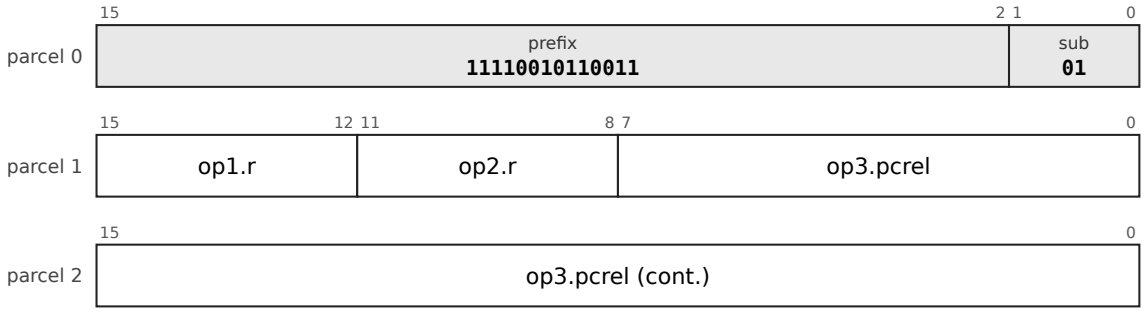
**CBGE: CB3I16_12H/b, prefix 11110010010001, sub-opcode 01; operands reg4, imm16s, pc12-**



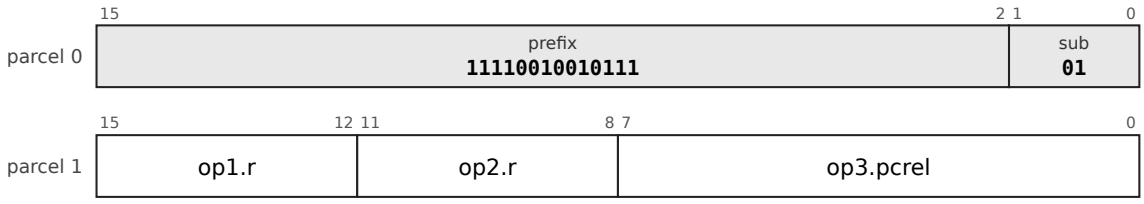

**CBGE: CB3I8_20/b, prefix 1111000111000, sub-opcode 01; operands reg5, imm8s, pc20-u2; 6**



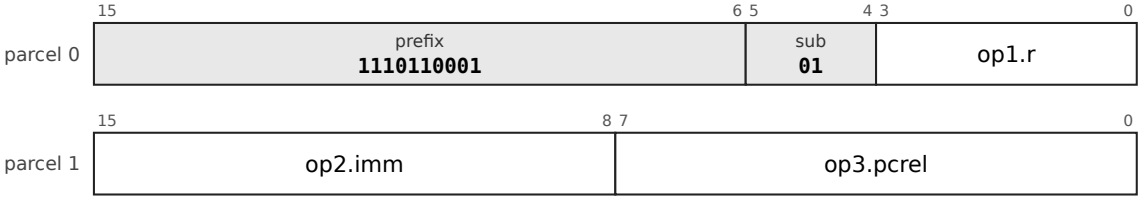

**CBGE: CB3R24H/b, prefix 11110010110011, sub-opcode 01; operands reg4, reg4, pc24-u2; 6**



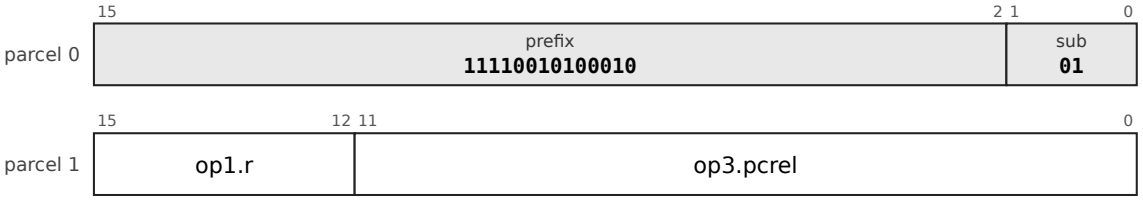

**CBGE: CBR8H/b, prefix 11110010010111, sub-opcode 01; operands reg4, reg4, pc8-u2; 4 byte**



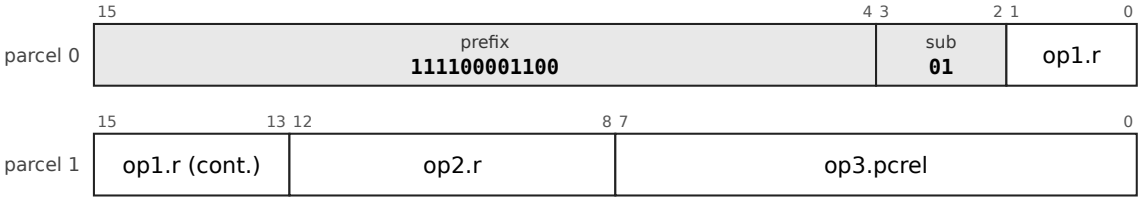
**CBGE: CBI8_8H/b, prefix 1110110001, sub-opcode 01; operands reg4, imm8s, pc8-u2; 4 bytes**



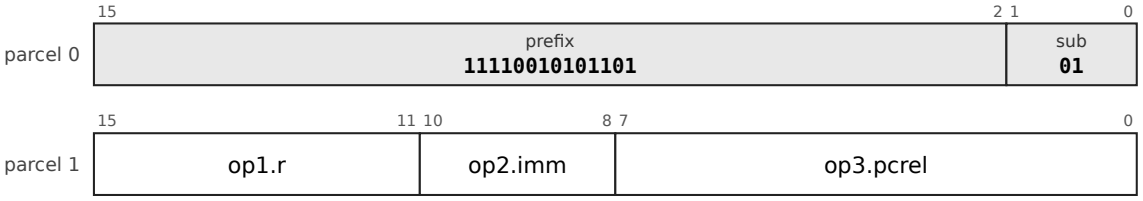

**CBGE: CBZ2_12H/b, prefix 11110010100010, sub-opcode 01; operands reg4, k-imm0, pc12-u2**



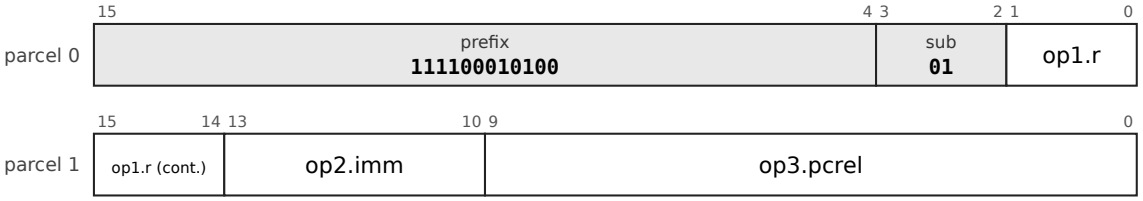

**CBGE: CBR8/b, prefix 111100001100, sub-opcode 01; operands reg5, reg5, pc8-u2; 4 bytes**



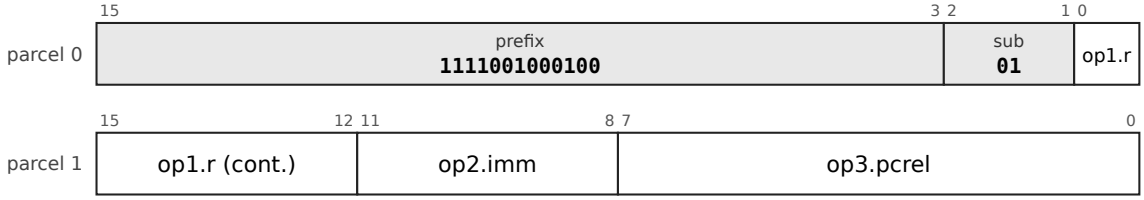

**CBGE: CBI3_8/b, prefix 11110010101101, sub-opcode 01; operands reg5, imm3s, pc8-u2; 4 by**




**CBGE: CBI4_10H/b, prefix 111100010100, sub-opcode 01; operands reg4, imm4s, pc10-u2; 4 l**



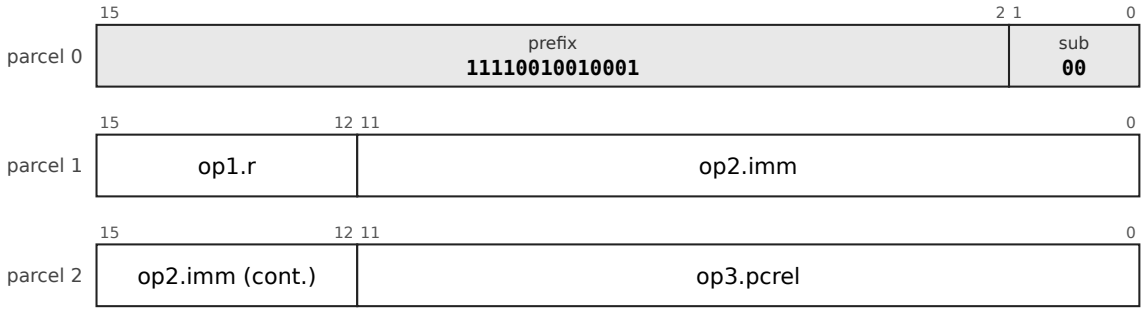
**CBGE: CBI4_8/b, prefix 1111001000100, sub-opcode 01; operands reg5, imm4s, pc8-u2; 4 byt**



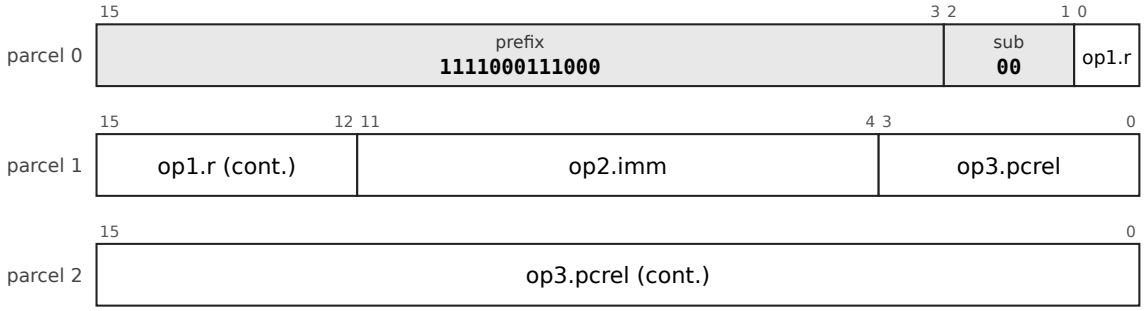

Operation, flags and exceptions as for the entry CBCC above; 10 layout forms, each a chart of its own (above).

***CBLT***

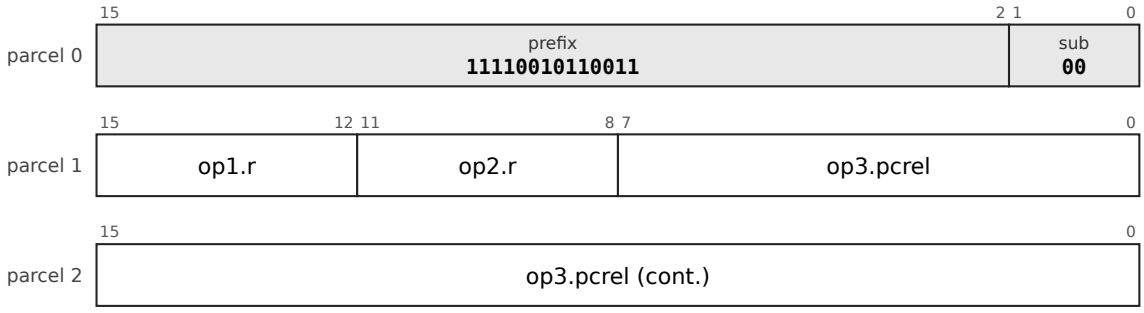
**CBLT: CB3I16_12H/b, prefix 11110010010001, sub-opcode 00; operands reg4, imm16s, pc12-u**



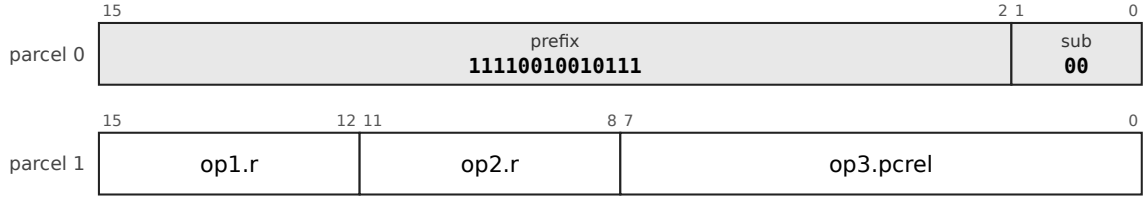

**CBLT: CB3I8_20/b, prefix 1111000111000, sub-opcode 00; operands reg5, imm8s, pc20-u2; 6 l**



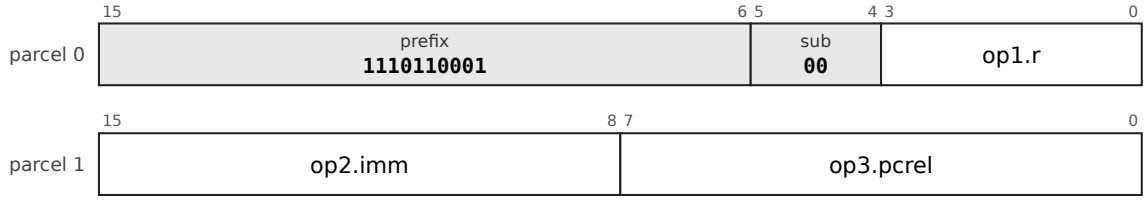

**CBLT: CB3R24H/b, prefix 11110010110011, sub-opcode 00; operands reg4, reg4, pc24-u2; 6 b**



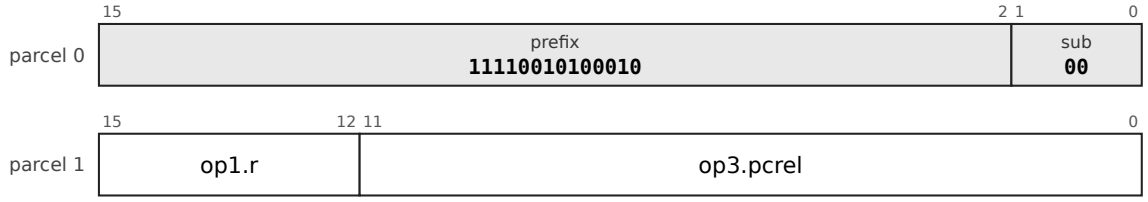
**CBLT: CBR8H/b, prefix 11110010010111, sub-opcode 00; operands reg4, reg4, pc8-u2; 4 byte**



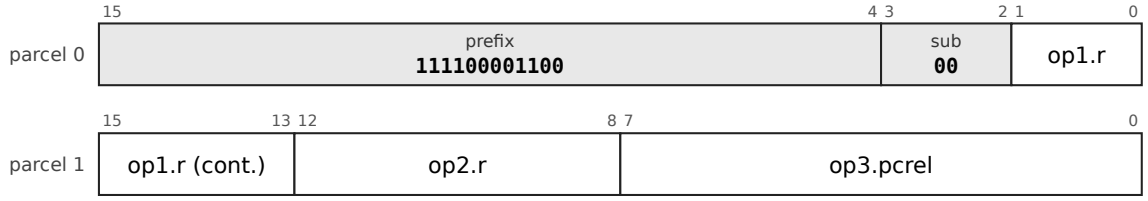

**CBLT: CBI8_8H/b, prefix 1110110001, sub-opcode 00; operands reg4, imm8s, pc8-u2; 4 bytes**



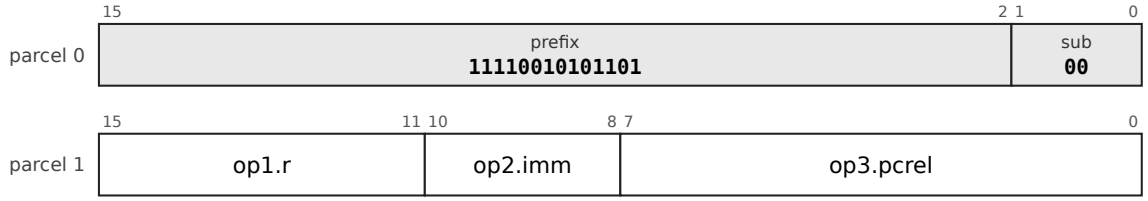

**CBLT: CBZ2_12H/b, prefix 11110010100010, sub-opcode 00; operands reg4, k-imm0, pc12-u2;**



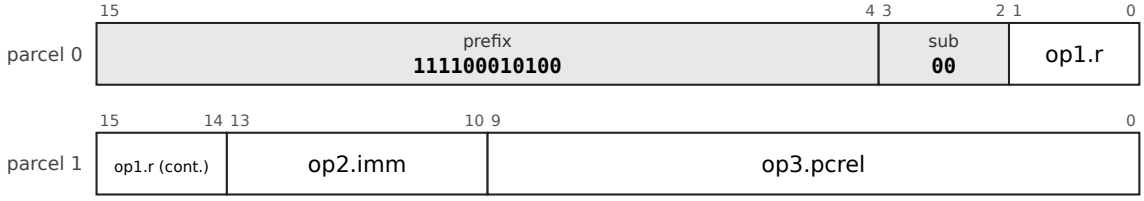

**CBLT: CBR8/b, prefix 111100001100, sub-opcode 00; operands reg5, reg5, pc8-u2; 4 bytes**



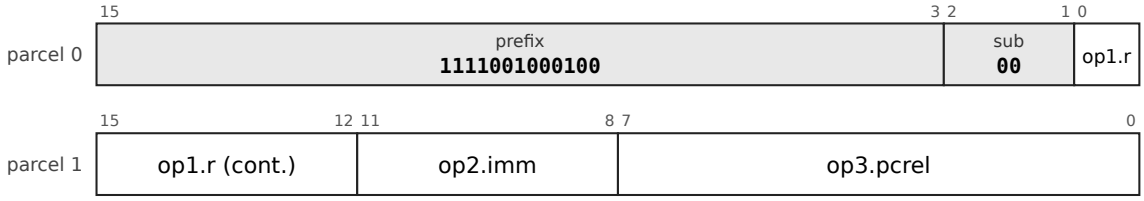

**CBLT: CBI3_8/b, prefix 11110010101101, sub-opcode 00; operands reg5, imm3s, pc8-u2; 4 by**



**CBLT: CBI4_10H/b, prefix 111100010100, sub-opcode 00; operands reg4, imm4s, pc10-u2; 4 b**



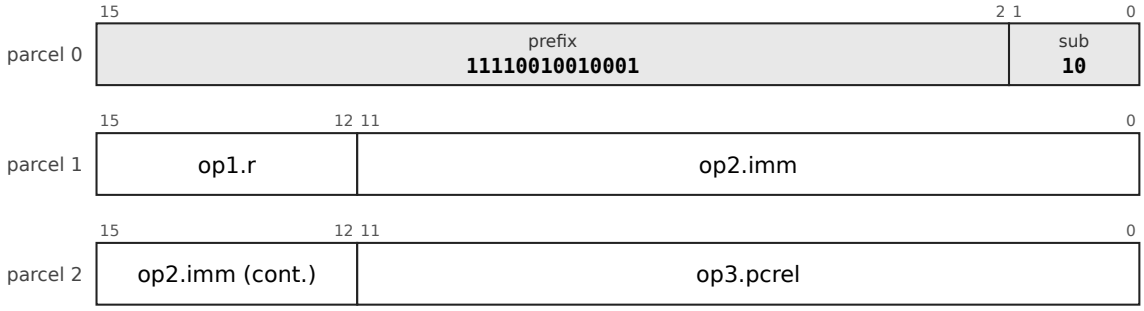

**CBLT: CBI4_8/b, prefix 1111001000100, sub-opcode 00; operands reg5, imm4s, pc8-u2; 4 byte**



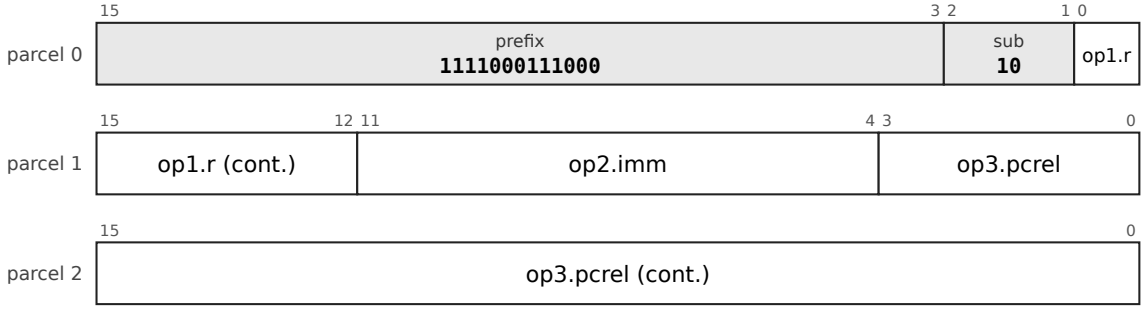

Operation, flags and exceptions as for the entry CBCC above; 10 layout forms, each a chart of its own (above).

### *CBGT*

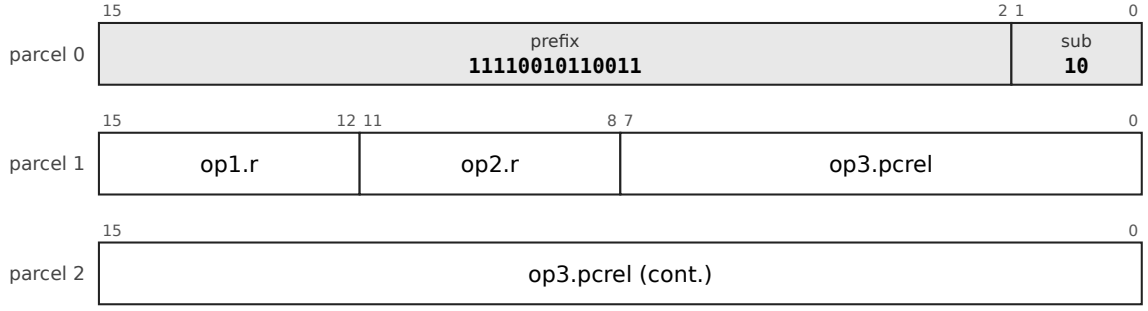
**CBGT: CB3I16_12H/b, prefix 11110010010001, sub-opcode 10; operands reg4, imm16s, pc12-**



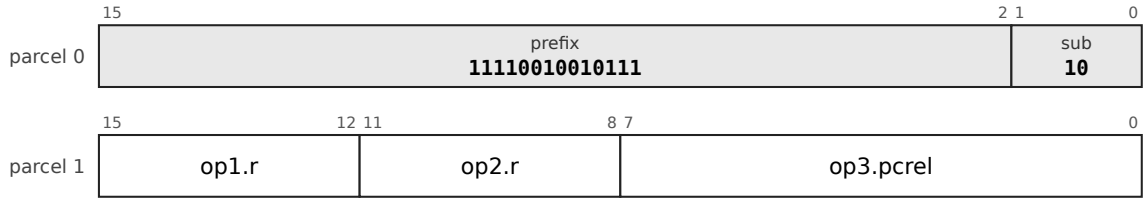

**CBGT: CB3I8_20/b, prefix 1111000111000, sub-opcode 10; operands reg5, imm8s, pc20-u2; 6**



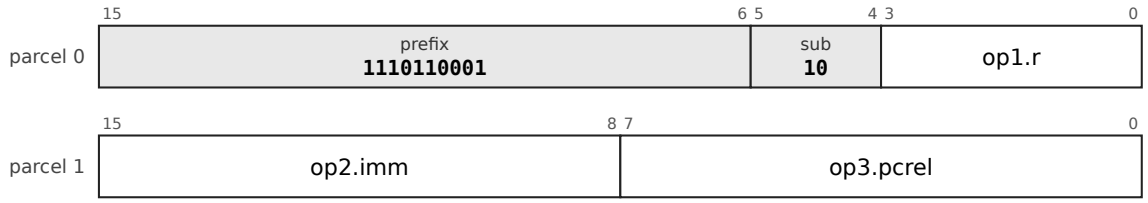
**CBGT: CB3R24H/b, prefix 11110010110011, sub-opcode 10; operands reg4, reg4, pc24-u2; 6 b**



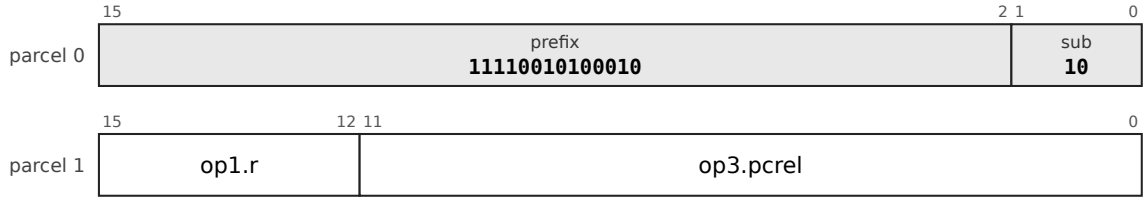

**CBGT: CBR8H/b, prefix 11110010010111, sub-opcode 10; operands reg4, reg4, pc8-u2; 4 byte**



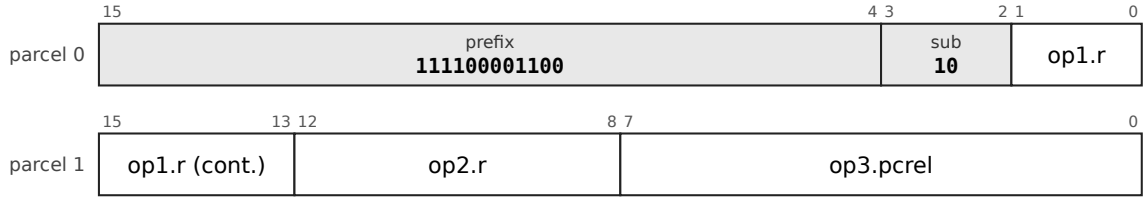

**CBGT: CBI8_8H/b, prefix 1110110001, sub-opcode 10; operands reg4, imm8s, pc8-u2; 4 bytes**



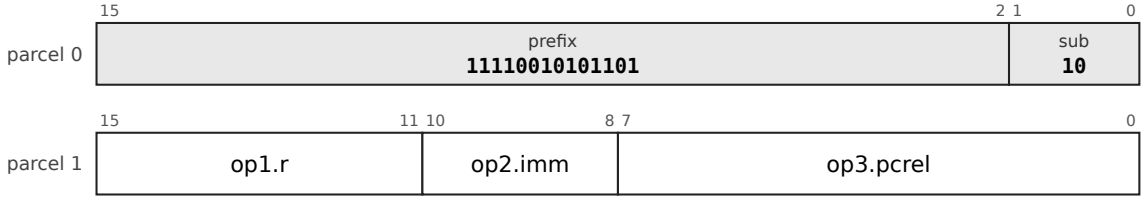

**CBGT: CBZ2_12H/b, prefix 11110010100010, sub-opcode 10; operands reg4, k-imm0, pc12-u2**



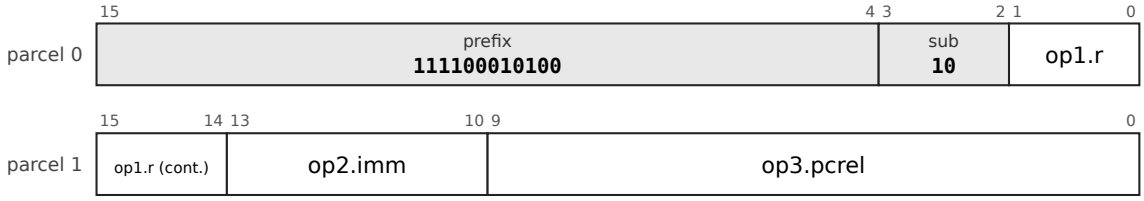

**CBGT: CBR8/b, prefix 111100001100, sub-opcode 10; operands reg5, reg5, pc8-u2; 4 bytes**



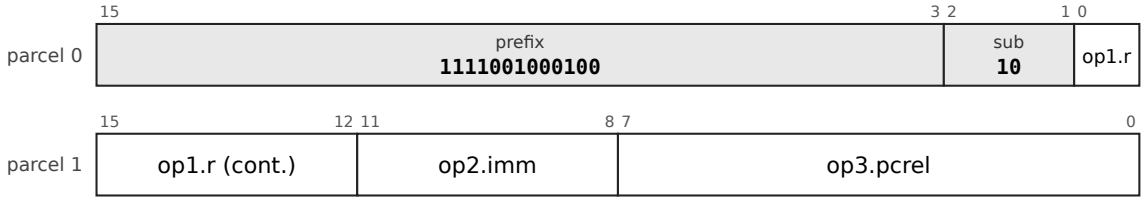
**CBGT: CBI3_8/b, prefix 11110010101101, sub-opcode 10; operands reg5, imm3s, pc8-u2; 4 by**




**CBGT: CBI4_10H/b, prefix 111100010100, sub-opcode 10; operands reg4, imm4s, pc10-u2; 4 b**



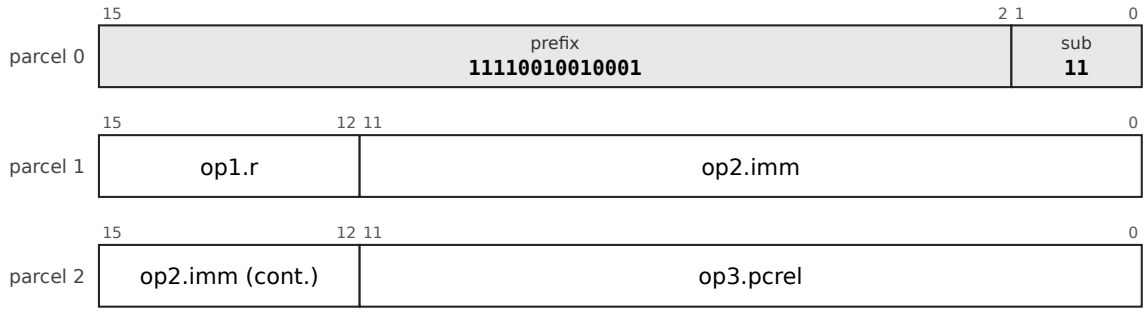

**CBGT: CBI4_8/b, prefix 1111001000100, sub-opcode 10; operands reg5, imm4s, pc8-u2; 4 byt**



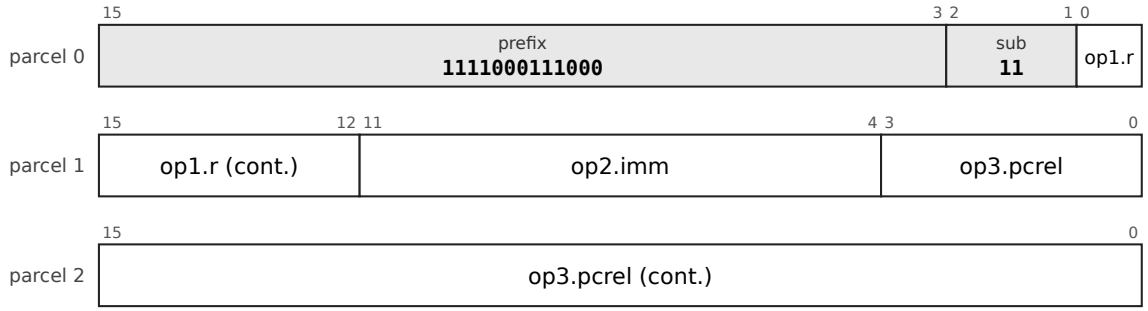

Operation, flags and exceptions as for the entry CBCC above; 10 layout forms, each a chart of its own (above).

### *CBLE*

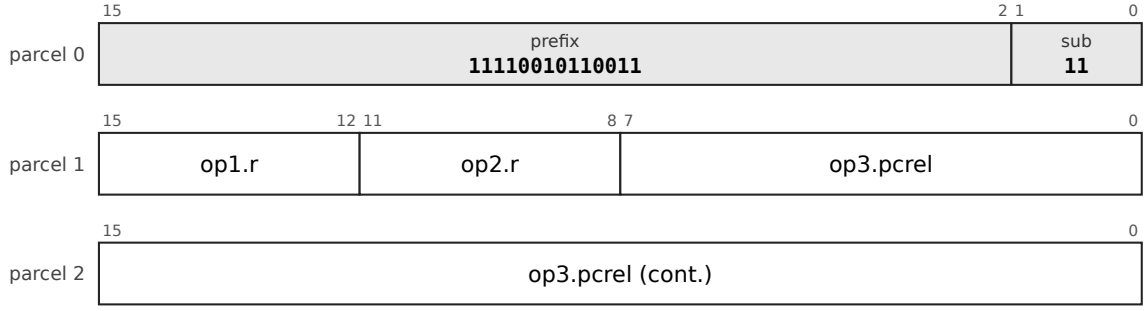
**CBLE: CB3I16_12H/b, prefix 11110010010001, sub-opcode 11; operands reg4, imm16s, pc12-**



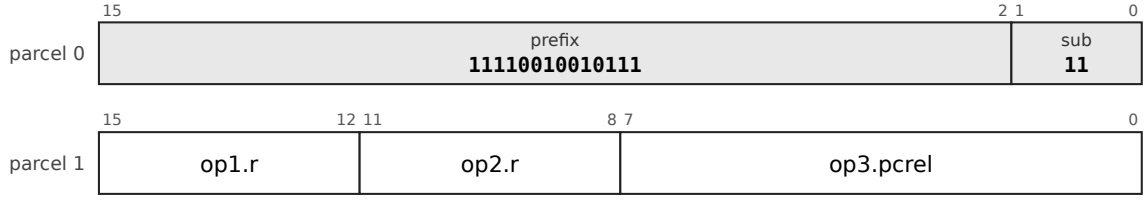

**CBLE: CB3I8_20/b, prefix 1111000111000, sub-opcode 11; operands reg5, imm8s, pc20-u2; 6**



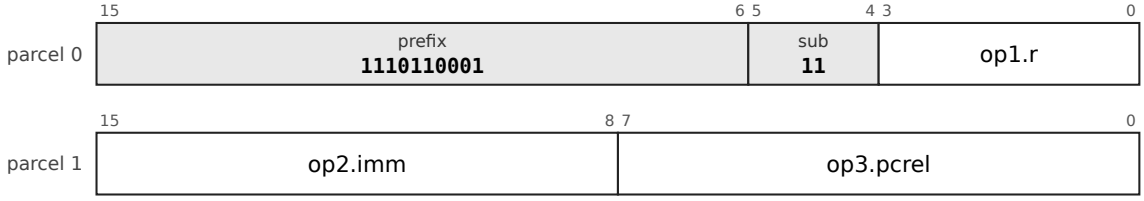

**CBLE: CB3R24H/b, prefix 11110010110011, sub-opcode 11; operands reg4, reg4, pc24-u2; 6 I**



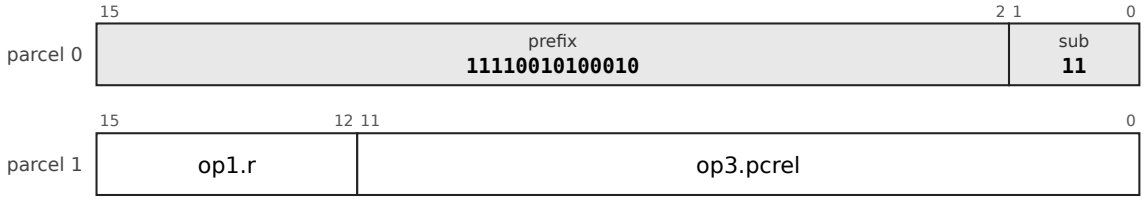

**CBLE: CBR8H/b, prefix 11110010010111, sub-opcode 11; operands reg4, reg4, pc8-u2; 4 byte**



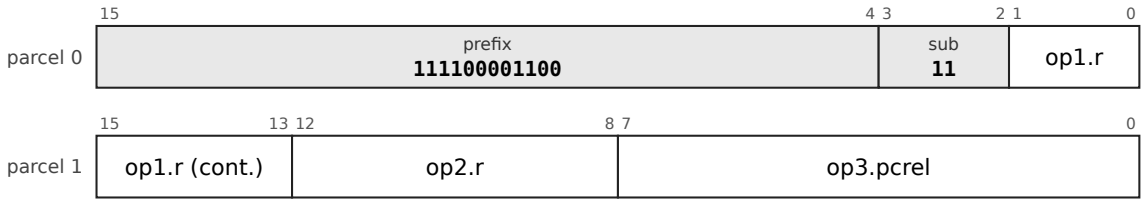
**CBLE: CBI8_8H/b, prefix 1110110001, sub-opcode 11; operands reg4, imm8s, pc8-u2; 4 bytes**



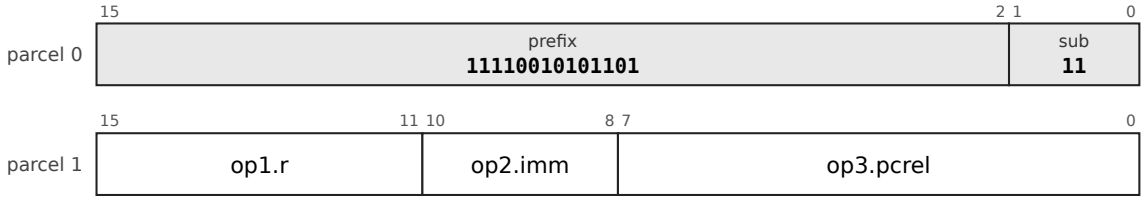

**CBLE: CBZ2_12H/b, prefix 11110010100010, sub-opcode 11; operands reg4, k-imm0, pc12-u2**



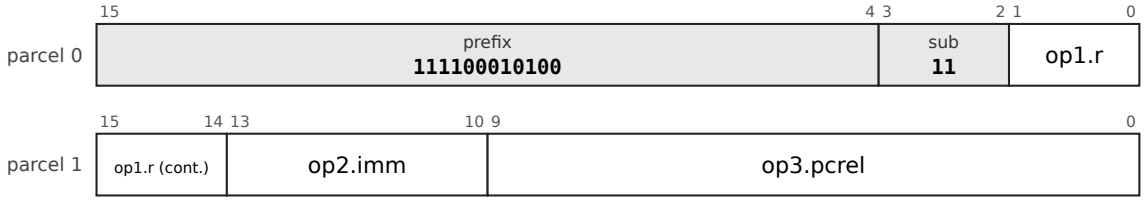

**CBLE: CBR8/b, prefix 111100001100, sub-opcode 11; operands reg5, reg5, pc8-u2; 4 bytes**



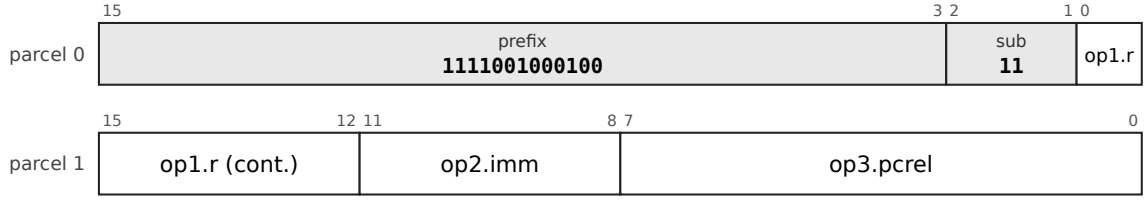

**CBLE: CBI3_8/b, prefix 11110010101101, sub-opcode 11; operands reg5, imm3s, pc8-u2; 4 by**




**CBLE: CBI4_10H/b, prefix 111100010100, sub-opcode 11; operands reg4, imm4s, pc10-u2; 4 b**



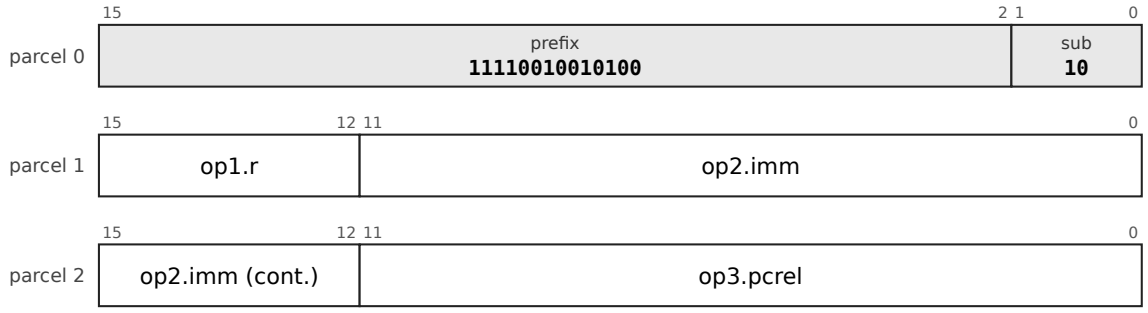
**CBLE: CBI4_8/b, prefix 1111001000100, sub-opcode 11; operands reg5, imm4s, pc8-u2; 4 byt**



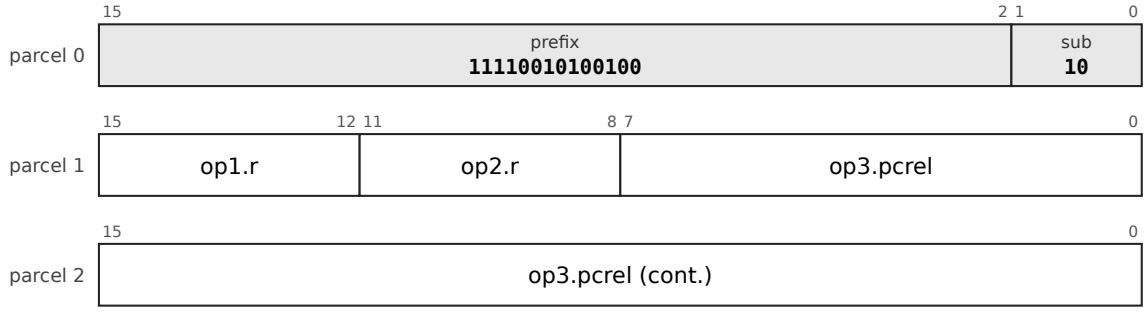

Operation, flags and exceptions as for the entry CBCC above; 10 layout forms, each a chart of its own (above).

***CBHI***

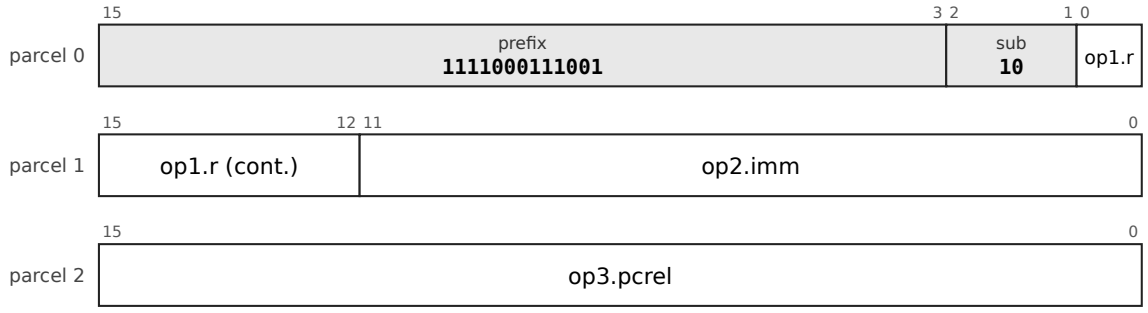
**CBHI: CB3I16_12H/c, prefix 11110010010100, sub-opcode 10; operands reg4, imm16s, pc12-u**



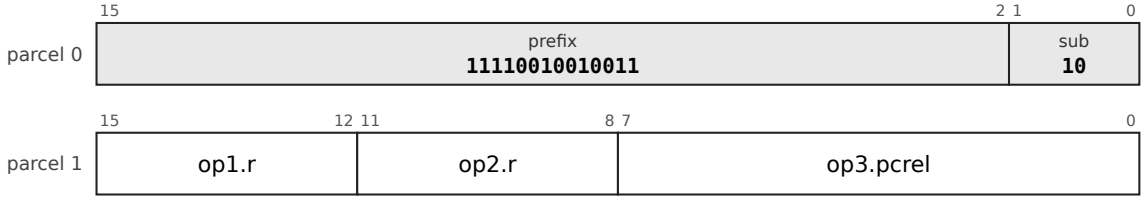

**CBHI: CB3R24H/c, prefix 11110010100100, sub-opcode 10; operands reg4, reg4, pc24-u2; 6 b**



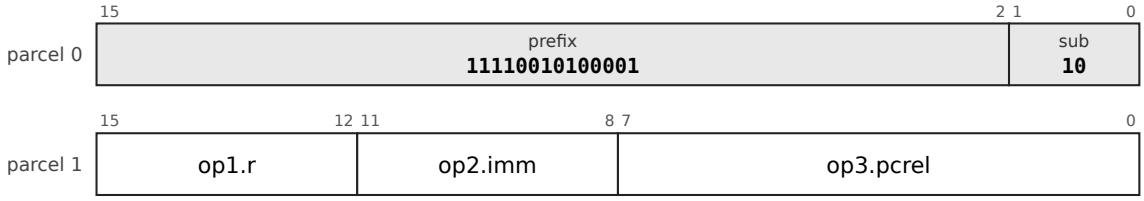

**CBHI: CB3I12_16/c, prefix 1111000111001, sub-opcode 10; operands reg5, imm12s, pc16-u2;**



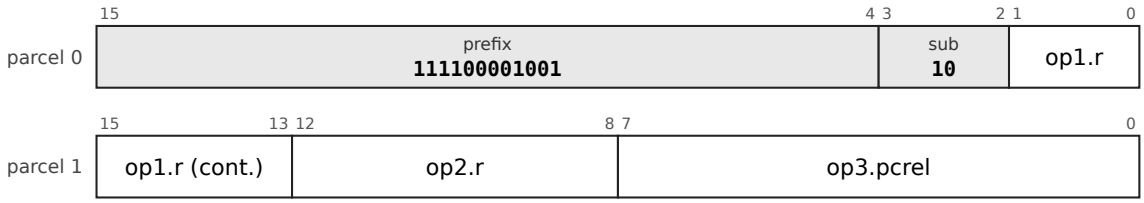
**CBHI: CBR8H/c, prefix 11110010010011, sub-opcode 10; operands reg4, reg4, pc8-u2; 4 bytes**



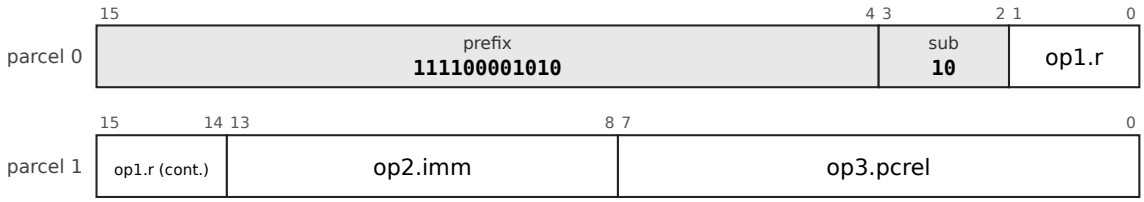

**CBHI: CBI4_8H/c, prefix 11110010100001, sub-opcode 10; operands reg4, imm4s, pc8-u2; 4 b**



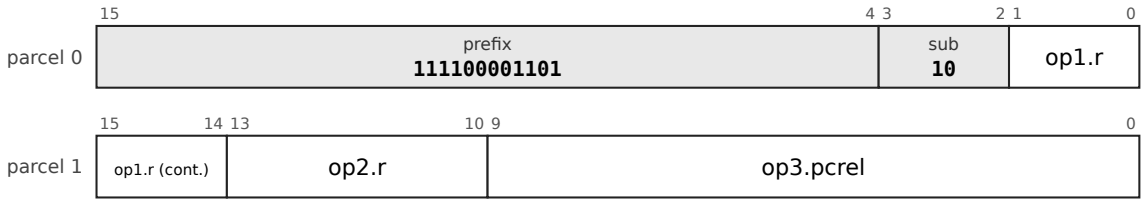

**CBHI: CBR8/c, prefix 111100001001, sub-opcode 10; operands reg5, reg5, pc8-u2; 4 bytes**



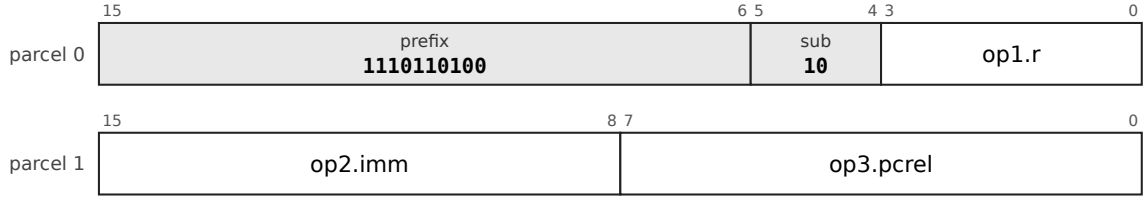

**CBHI: CBI6_8H/c, prefix 111100001010, sub-opcode 10; operands reg4, imm6s, pc8-u2; 4 byt**



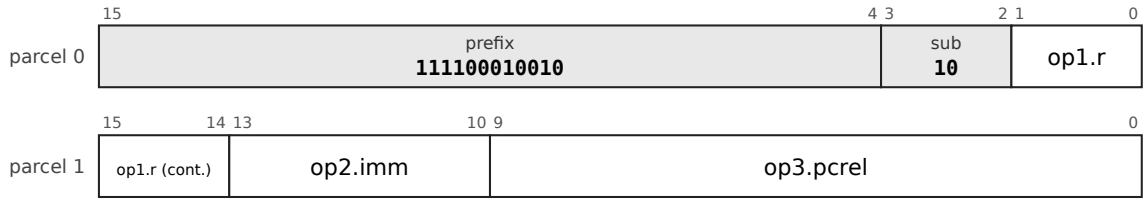

**CBHI: CBR10H/c, prefix 111100001101, sub-opcode 10; operands reg4, reg4, pc10-u2; 4 bytes**



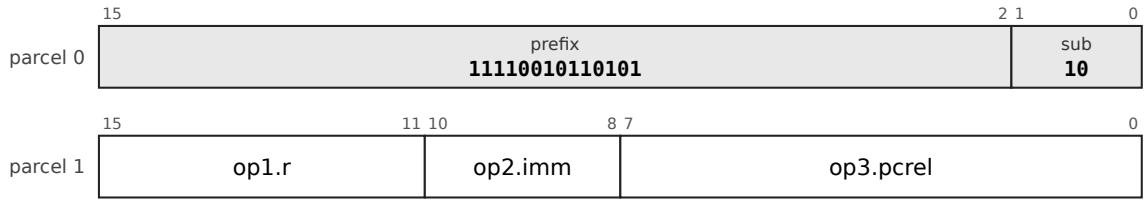
**CBHI: CBI8_8H/c, prefix 1110110100, sub-opcode 10; operands reg4, imm8s, pc8-u2; 4 bytes**




**CBHI: CBI4_10H/c, prefix 111100010010, sub-opcode 10; operands reg4, imm4s, pc10-u2; 4 b**



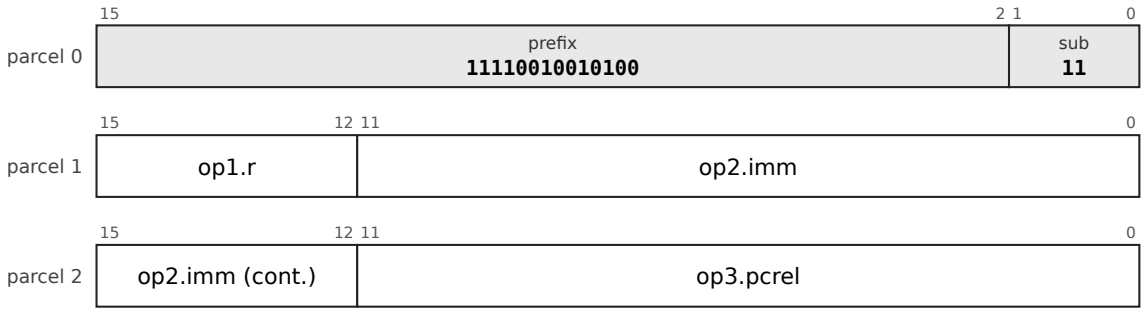

**CBHI: CBI3_8/c, prefix 11110010110101, sub-opcode 10; operands reg5, imm3s, pc8-u2; 4 by**



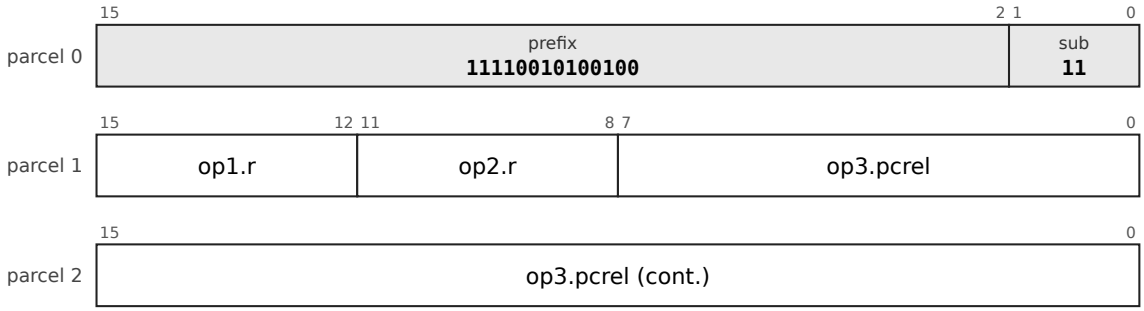

Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

### *CBLOS*

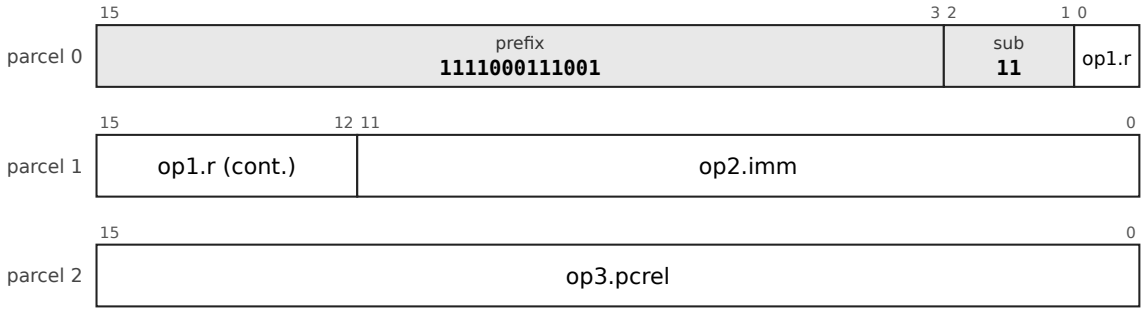
**CBLOS: CB3I16_12H/c, prefix 11110010010100, sub-opcode 11; operands reg4, imm16s, pc12**



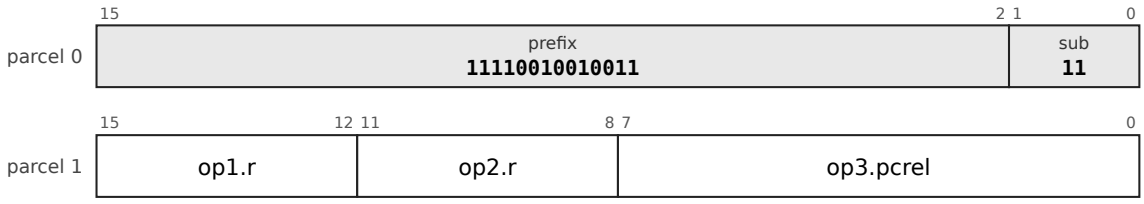

**CBLOS: CB3R24H/c, prefix 11110010100100, sub-opcode 11; operands reg4, reg4, pc24-u2; 6**



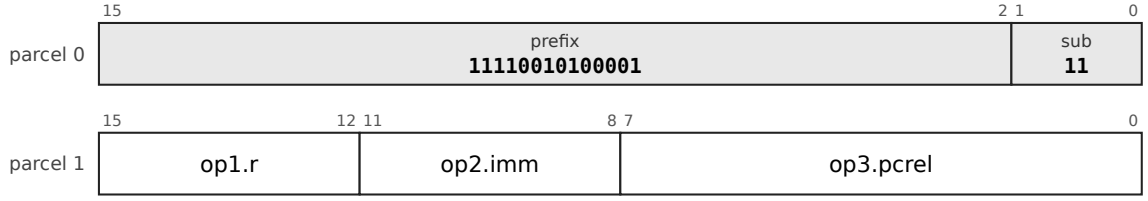

**CBLOS: CB3I12_16/c, prefix 1111000111001, sub-opcode 11; operands reg5, imm12s, pc16-u**



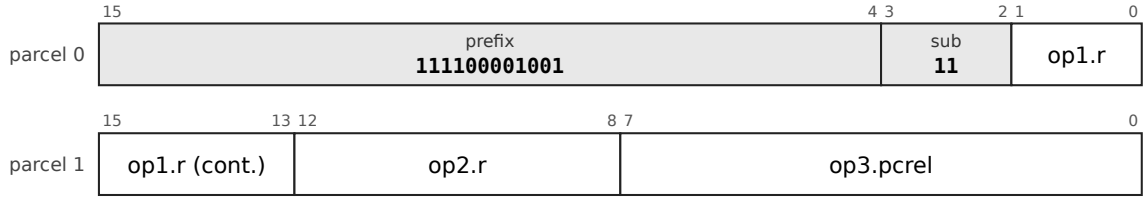

**CBLOS: CBR8H/c, prefix 11110010010011, sub-opcode 11; operands reg4, reg4, pc8-u2; 4 byt**



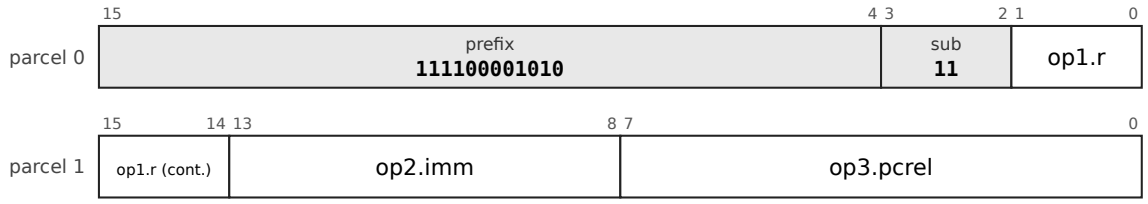
**CBLOS: CBI4_8H/c, prefix 11110010100001, sub-opcode 11; operands reg4, imm4s, pc8-u2; 4**



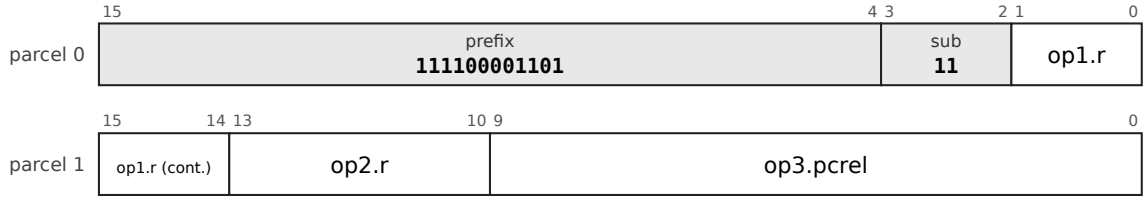

**CBLOS: CBR8/c, prefix 111100001001, sub-opcode 11; operands reg5, reg5, pc8-u2; 4 bytes**



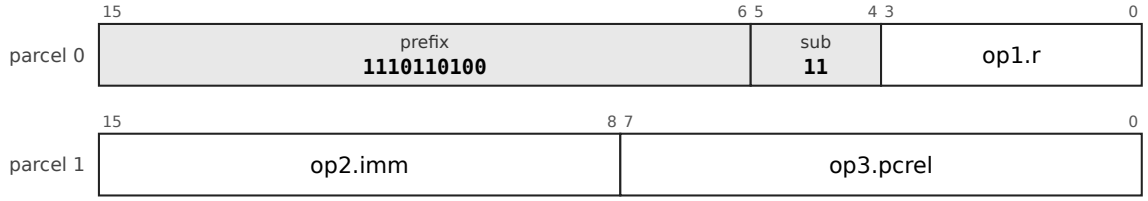

**CBLOS: CBI6_8H/c, prefix 111100001010, sub-opcode 11; operands reg4, imm6s, pc8-u2; 4 by**



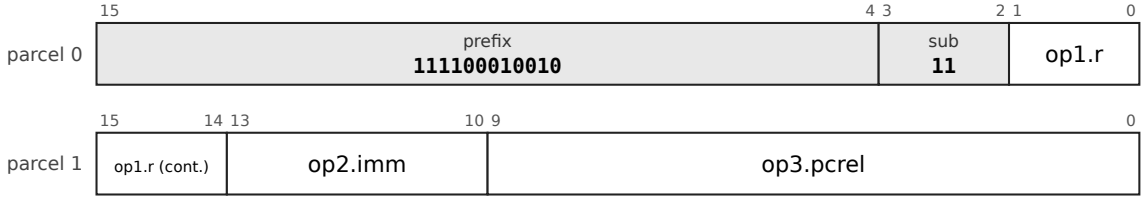

**CBLOS: CBR10H/c, prefix 111100001101, sub-opcode 11; operands reg4, reg4, pc10-u2; 4 byt**



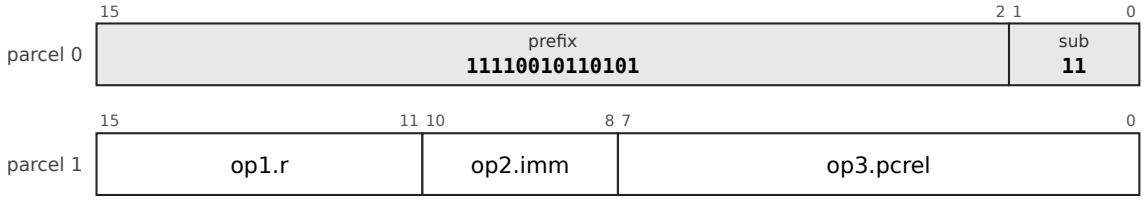

**CBLOS: CBI8_8H/c, prefix 1110110100, sub-opcode 11; operands reg4, imm8s, pc8-u2; 4 byte**



**CBLOS: CBI4_10H/c, prefix 111100010010, sub-opcode 11; operands reg4, imm4s, pc10-u2; 4**




**CBLOS: CBI3_8/c, prefix 11110010110101, sub-opcode 11; operands reg5, imm3s, pc8-u2; 4 b**




Operation, flags and exceptions as for the entry CBCC above; 11 layout forms, each a chart of its own (above).

***CBHIS***
An alias of `CBCC`: the assembler writes it as that word.

***CBLO***
An alias of `CBCS`: the assembler writes it as that word.

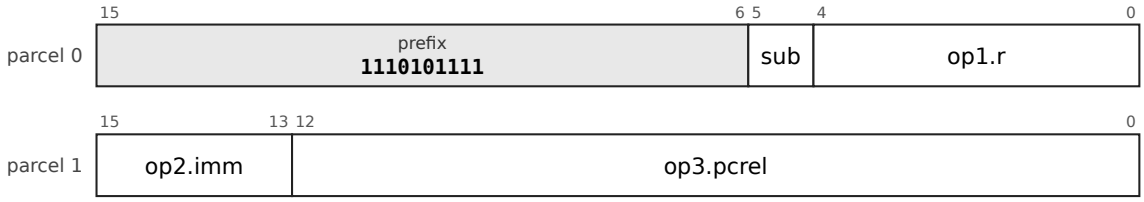
## BTAG: Branch on pointer tag

**Mnemonics:** `BTAG`, `BNTAG`

### Syntax

```
BTAG Rn, #tag, target
BNTAG Rn, #tag, target
```

### Encoding




The chart is the leaf `TAG-1`, TAG, prefix `1110101111`. `op1` is `reg5`: register, 5 bits. `op2` is `imm3u`: unsigned 3-bit immediate, 0 to 7. `op3` is `pc13-u2`: pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes). `sub` (1 bit) selects the mnemonic: `btag` 0, `bntag` 1.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `TAG-1`.

### Operation

```
BTAG : if (Rn & 111₂) = tag then PC ← address of this instruction + 2 × disp
BNTAG: if (Rn & 111₂) ≠ tag then PC ← address of this instruction + 2 × disp
```

**Flags** None written.

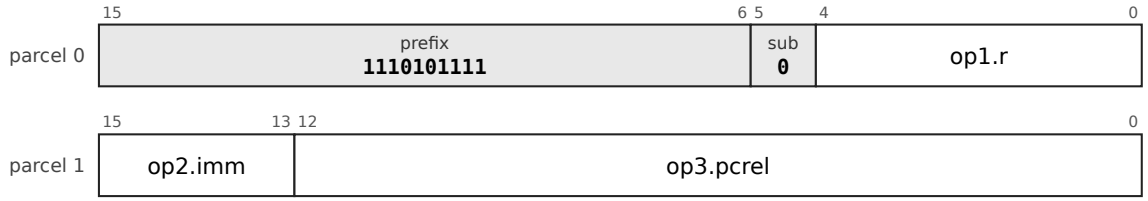
**Exceptions** None.

For runtimes that keep a 3-bit type tag in the low bits of a pointer (`lowtag`). The displacement is signed and counts parcels.

**Why it is here.** The tag branches BTAG and BNTAG and the table dispatch JTAG make tagged pointers cheap to use rather than merely possible. Because quad-aligned objects have zero in the low three bits, the tag is free, and an unaligned access is a fault. The compiler for C never emits them (the corpus shows none); they are for Lisp and other tagged runtimes.

### *BTAG*

**BTAG: TAG, prefix 1110101111, sub-opcode 0; operands reg5, imm3u, pc13-u2; 4 bytes**

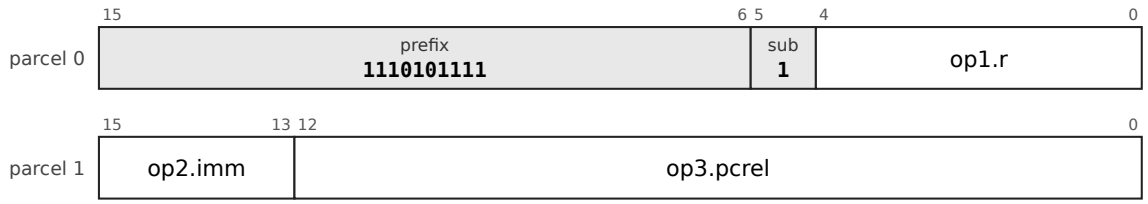



```
BTAG Rn, #tag, target
```

Operation, flags and exceptions as for the entry BTAG above; 1 layout form, each a chart of its own (above).

### *BNTAG*

**BNTAG: TAG, prefix 1110101111, sub-opcode 1; operands reg5, imm3u, pc13-u2; 4 bytes**

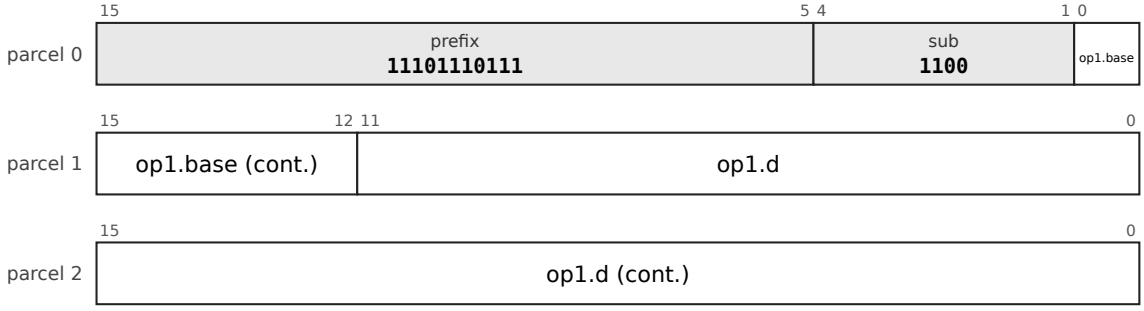



```
BNTAG Rn, #tag, target
```

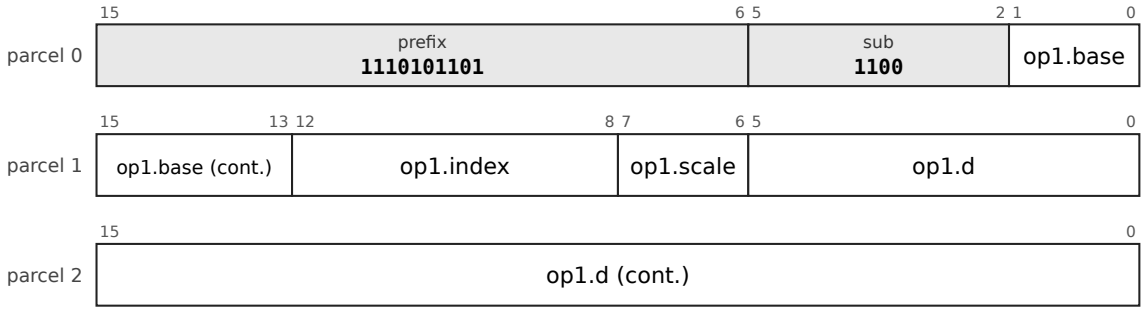
Operation, flags and exceptions as for the entry BTAG above; 1 layout form, each a chart of its own (above).

## JMP: Jump

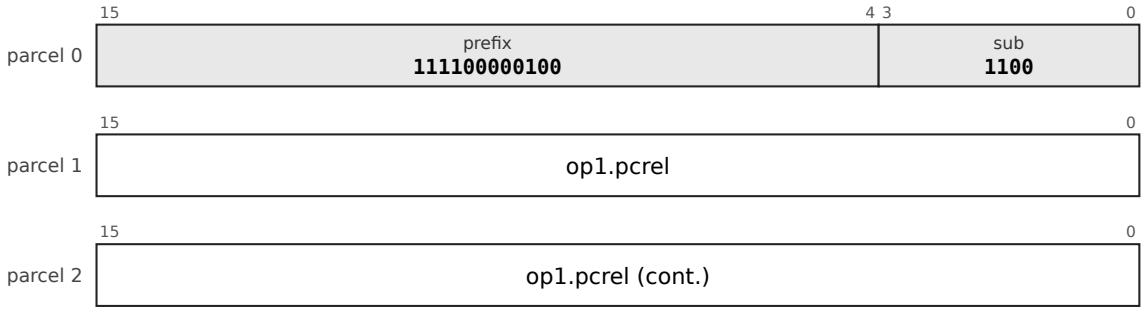
### JMP: SOF.B28, prefix 11101110111, sub-opcode 1100; operands b5d28s-x1; 6 bytes



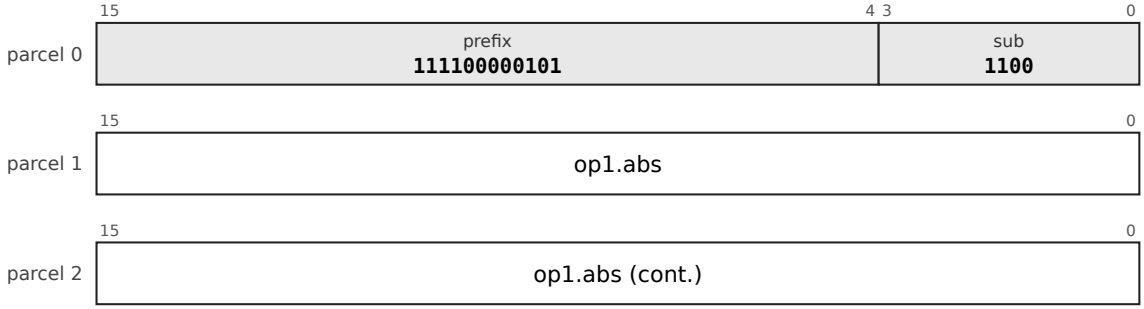

### JMP: SOF.X22, prefix 1110101101, sub-opcode 1100; operands x5-5d22s; 6 bytes



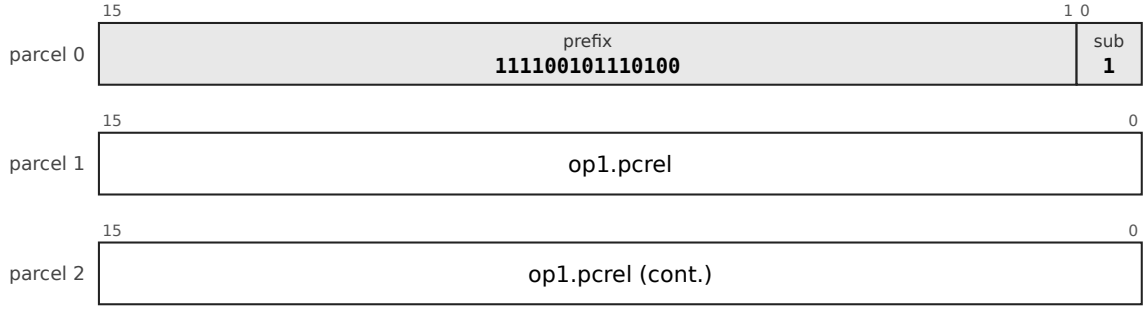

### JMP: SOF.PC32, prefix 111100000100, sub-opcode 1100; operands pc32-u1; 6 bytes



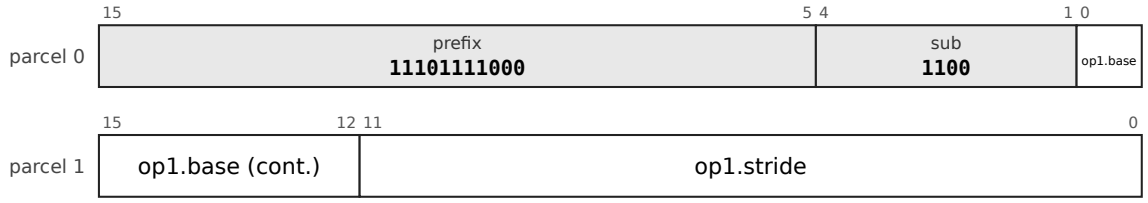

### JMP: SOF.A32, prefix 111100000101, sub-opcode 1100; operands abs32; 6 bytes



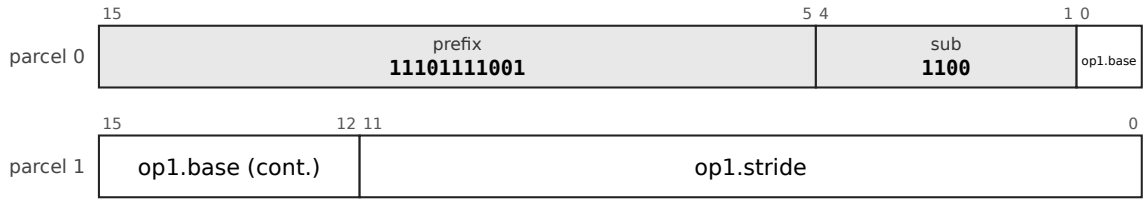
**JMP: CALLF, prefix 111100101110100, sub-opcode 1; operands pc32-u2; 6 bytes**



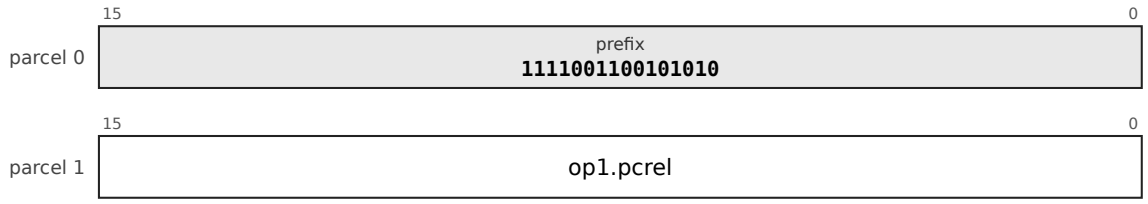

**JMP: SOF.PI12, prefix 11101111000, sub-opcode 1100; operands pi5-s12; 4 bytes**



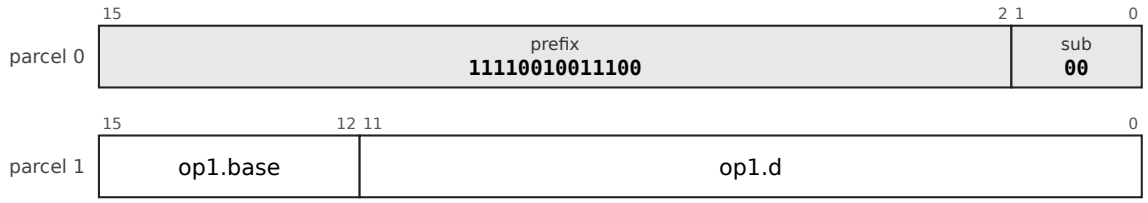

**JMP: SOF.PD12, prefix 11101111001, sub-opcode 1100; operands pd5-s12; 4 bytes**



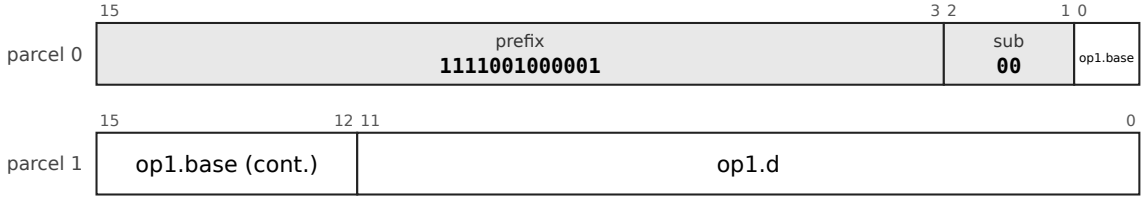

**JMP: CALL2_16/j, prefix 1111001100101010; operands pc16-u2; 4 bytes**



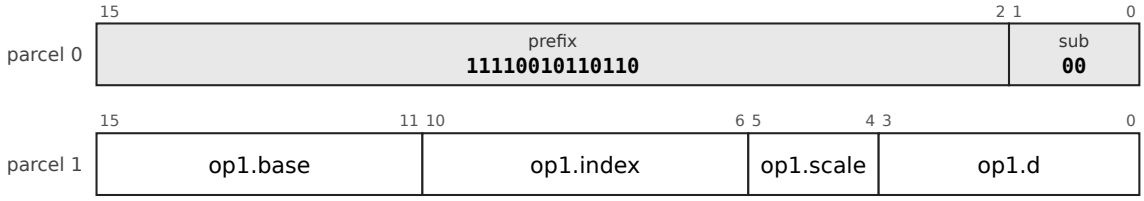

**JMP: SOBu12H/x, prefix 11110010011100, sub-opcode 00; operands b4d12u-xsz; 4 bytes**



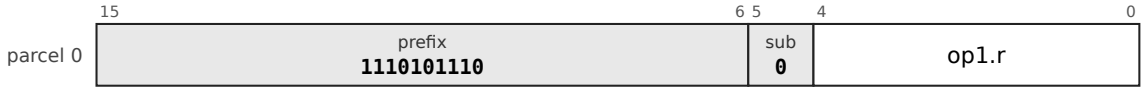
**JMP: SOBs12/x, prefix 1111001000001, sub-opcode 00; operands b5d12s-xsz; 4 bytes**




**JMP: SOX4/x, prefix 11110010110110, sub-opcode 00; operands x5-5d4s; 4 bytes**




**JMP: SOPR, prefix 1110101110, sub-opcode 0; operands reg5; 2 bytes**

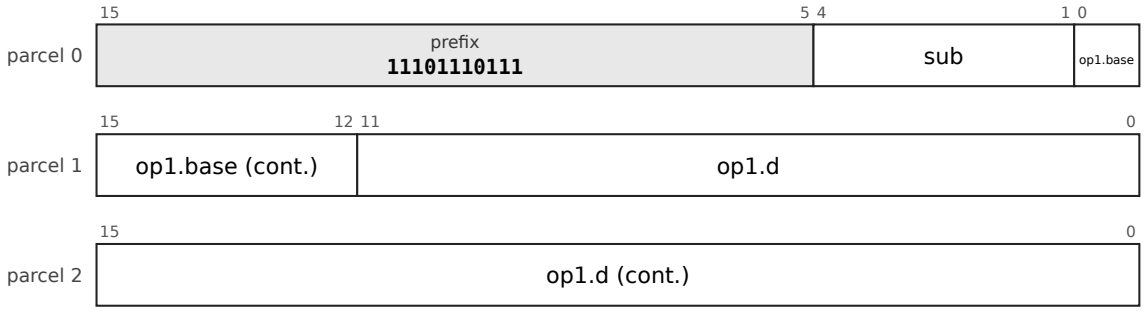



**Mnemonics:** `JMP`

### Syntax

```
JMP rm        ; a register, or an address by any memory mode
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `CALLF-1`, `SOPR-3`, `CALL2-16-j-1`, `SOBu12H-x-1`, `SOBs12-x-1`, `SOX4-x-1`.

### Operation

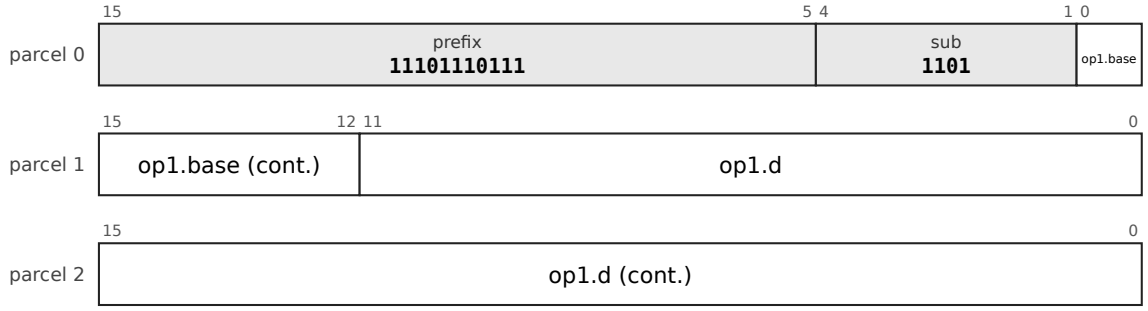
```
PC ← (register operand) ? Rm : EA(rm)
```

**Flags** None written.

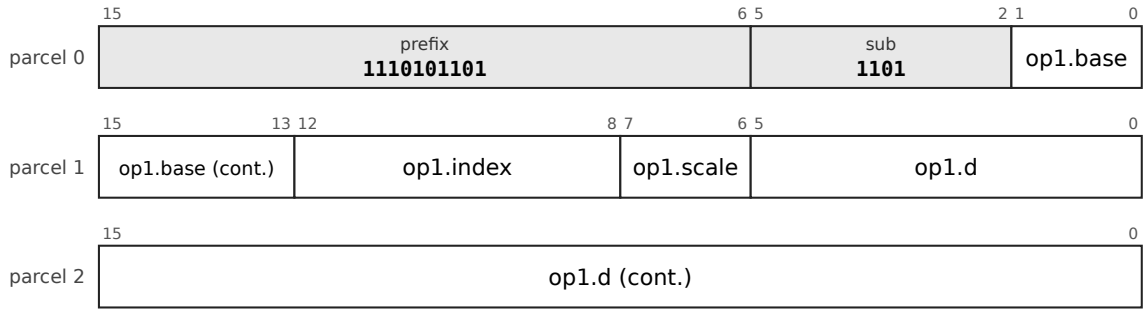
**Exceptions** A jump to an unmapped or non-executable address faults on the fetch of the target.
A switch is a `CMP` against the limit, a `BHIS` to the default and a `JMP` through a scaled index.

## CALL: Call

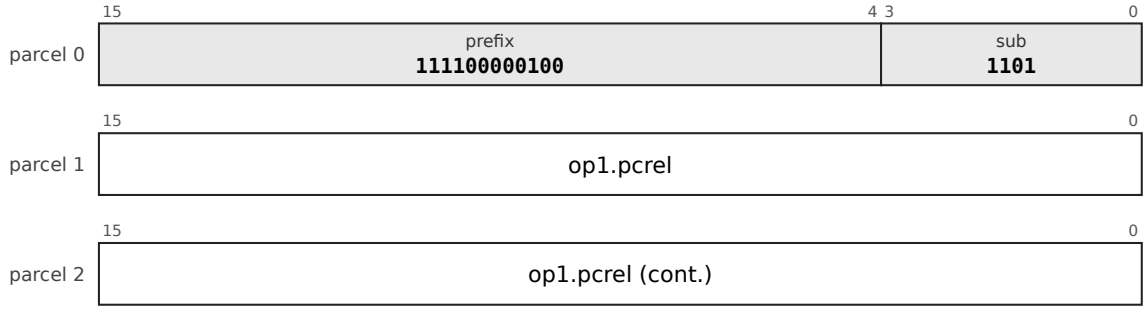
**CALL: SOF.B28, prefix 11101110111, sub-opcode 1101; operands b5d28s-x1; 6 bytes**



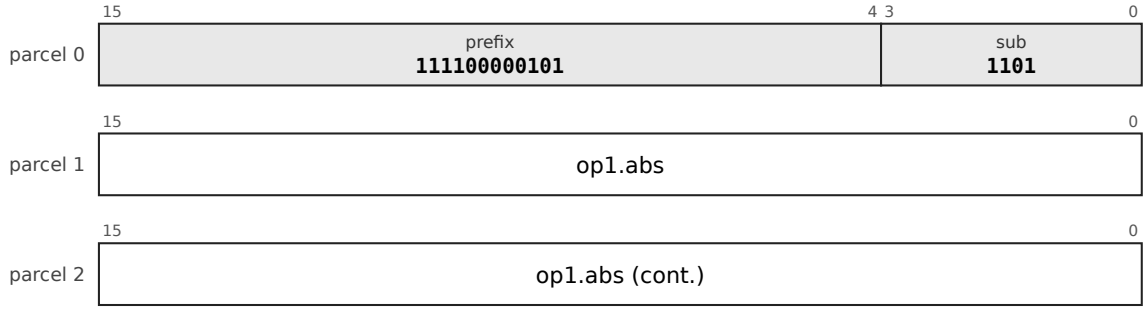

**CALL: SOF.X22, prefix 1110101101, sub-opcode 1101; operands x5-5d22s; 6 bytes**



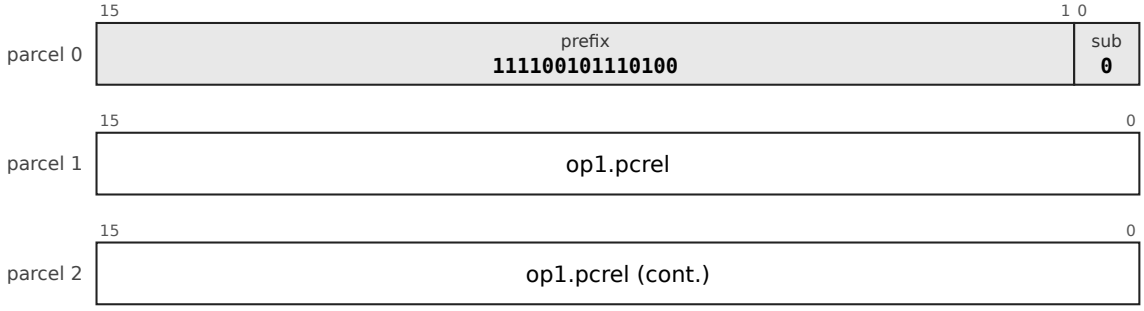

**CALL: SOF.PC32, prefix 111100000100, sub-opcode 1101; operands pc32-u1; 6 bytes**



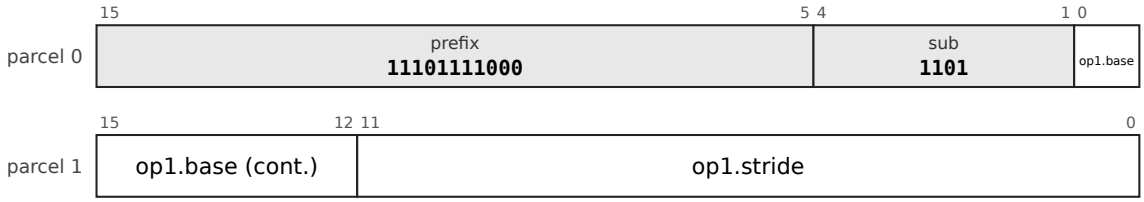

**CALL: SOF.A32, prefix 111100000101, sub-opcode 1101; operands abs32; 6 bytes**



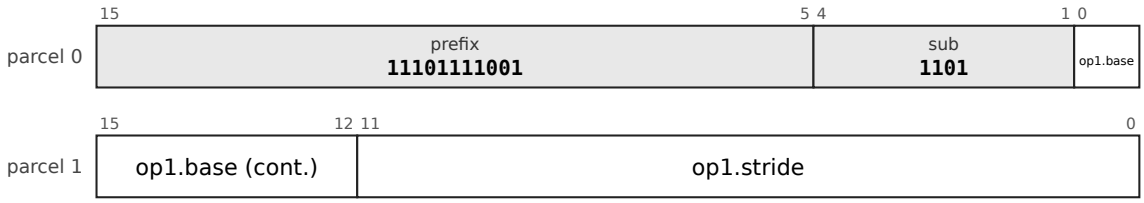
**CALL: CALLF, prefix 111100101110100, sub-opcode 0; operands pc32-u2; 6 bytes**



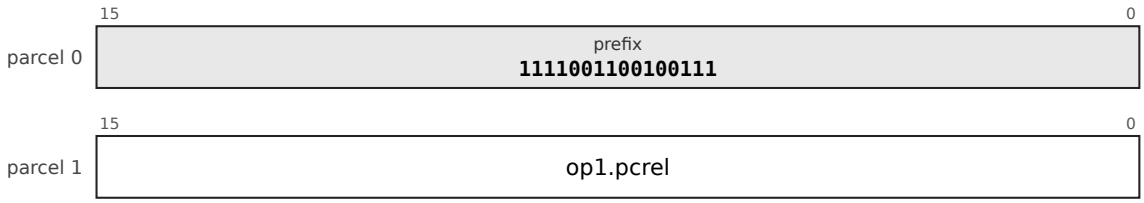

**CALL: SOF.PI12, prefix 11101111000, sub-opcode 1101; operands pi5-s12; 4 bytes**



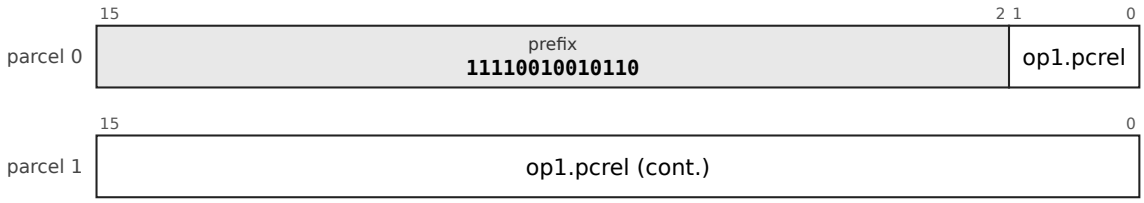

**CALL: SOF.PD12, prefix 11101111001, sub-opcode 1101; operands pd5-s12; 4 bytes**



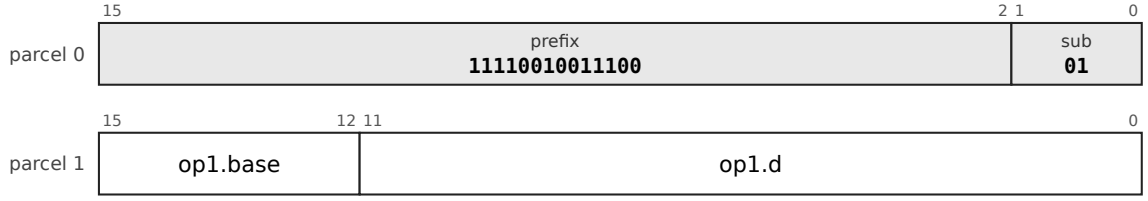

**CALL: CALL2_16/c, prefix 1111001100100111; operands pc16-u2; 4 bytes**



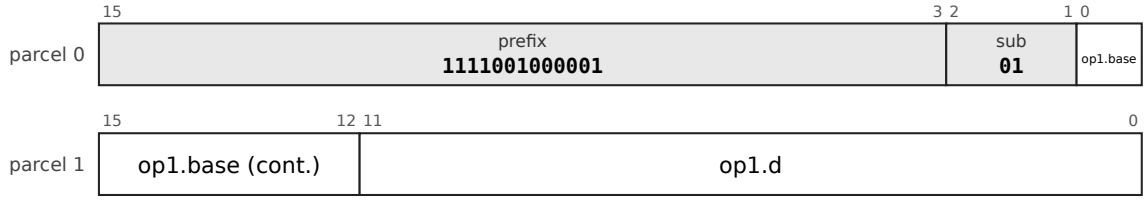

**CALL: CALL2_18/c, prefix 11110010010110; operands pc18-u2; 4 bytes**



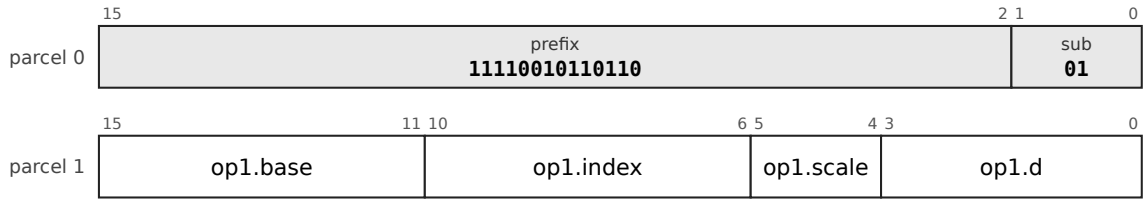
**CALL: SOBu12H/x, prefix 11110010011100, sub-opcode 01; operands b4d12u-xsz; 4 bytes**




**CALL: SOBs12/x, prefix 1111001000001, sub-opcode 01; operands b5d12s-xsz; 4 bytes**




**CALL: SOX4/x, prefix 11110010110110, sub-opcode 01; operands x5-5d4s; 4 bytes**




**CALL: SOPR, prefix 1110101110, sub-opcode 1; operands reg5; 2 bytes**

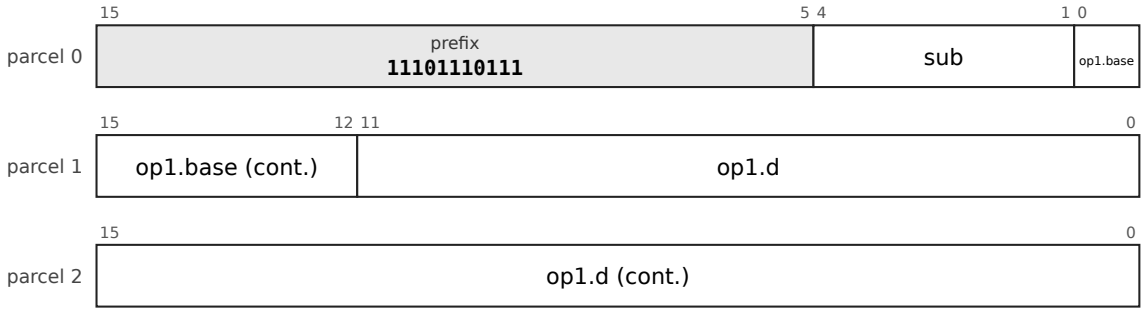



**Mnemonics:** `CALL`

## Syntax

```
CALL rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `CALLF-1`, `SOPR-3`, `CALL2-16-c-1`, `CALL2-18-c-1`, `SOBu12H-x-1`, `SOBs12-x-1`, `SOX4-x-1`.

### Operation

```
t ← (register operand) ? Rm : EA(rm)
SP ← SP - 8 ; M[SP] ← address of the next instruction
PC ← t
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The return address is on the data stack where every unwinder and debugger expects it. `RET` pops it.

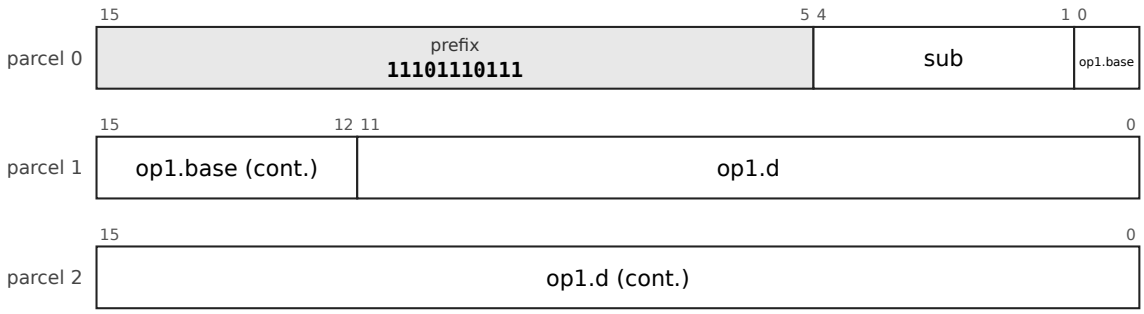
## JMPI: Jump and call through memory

**Mnemonics:** `JMPI`, `CALLI`

### Syntax

```
JMPI rm        ; PC ← M[EA]
CALLI rm       ; push the return address, then PC ← M[EA]
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOBu12H-x-1`, `SOBs12-x-1`, `SOX4-x-1`.

### Operation

```
JMPI : PC ← M[EA(rm)]
CALLI: t ← M[EA(rm)] ; SP ← SP - 8 ; M[SP] ← address of the next instruction ; PC ← t
```

**Flags** None written.

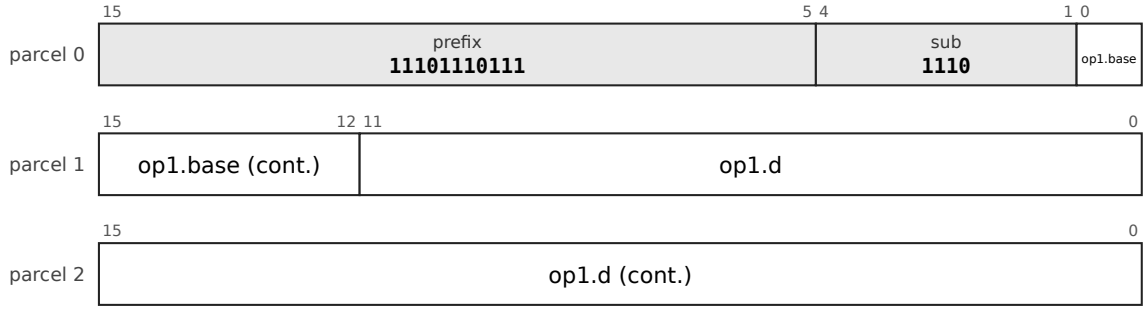
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. For `CALLI` the order is specified: load the target, then push, then transfer; a fault on the load leaves nothing changed.

The one deferred reference the architecture admits, for control transfer only: the PLT stub (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`). It needs a memory operand.

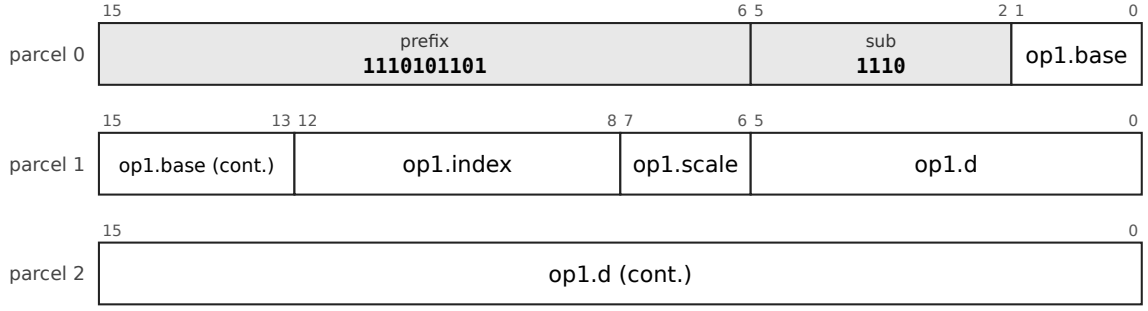
**Why it is here.** Exactly one deferred reference, for control transfer only: the PLT stub of every dynamically linked program (`JMPI got_slot(PC)`) and virtual-method dispatch (`CALLI vtbl_off(Robj)`), each two instructions and a scratch register without it. It is admitted because it breaks none of the reasons deferred modes are removed (no register is side-effected, the reference is in the branch path and not the operand path, a fault is clean by ordering). Jump tables through `JMPI` are not smaller than a compare chain, because a table entry is 8 bytes and a case in a compare chain is also 8.

### *JMPI*

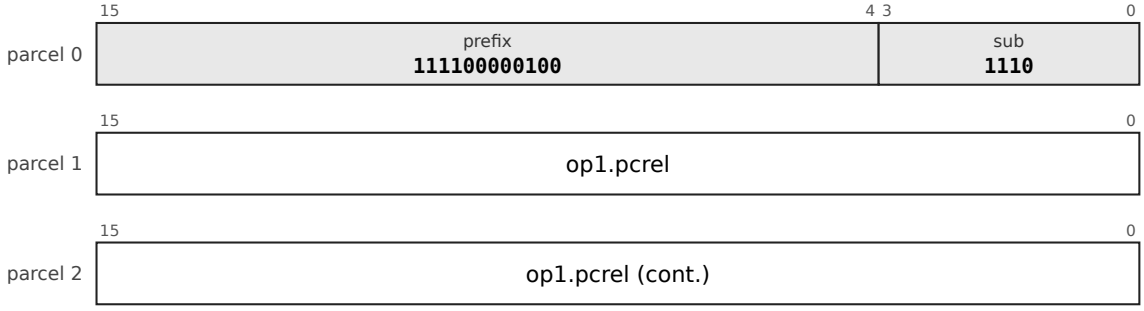
**JMPI: SOF.B28, prefix 11101110111, sub-opcode 1110; operands b5d28s-x1; 6 bytes**



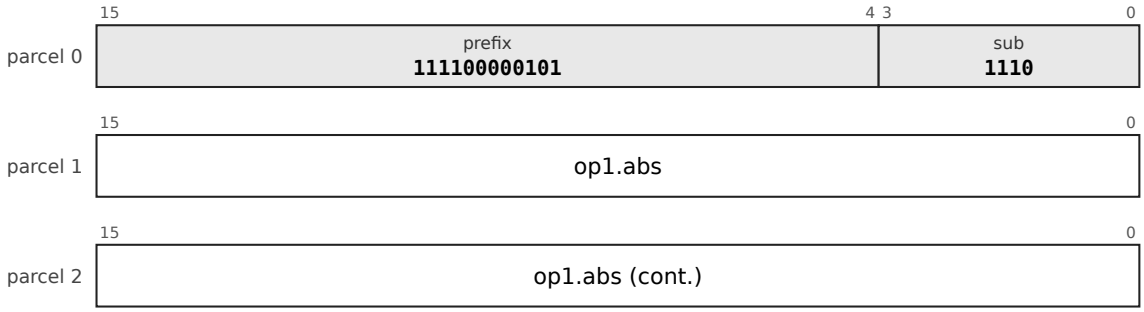

**JMPI: SOF.X22, prefix 1110101101, sub-opcode 1110; operands x5-5d22s; 6 bytes**



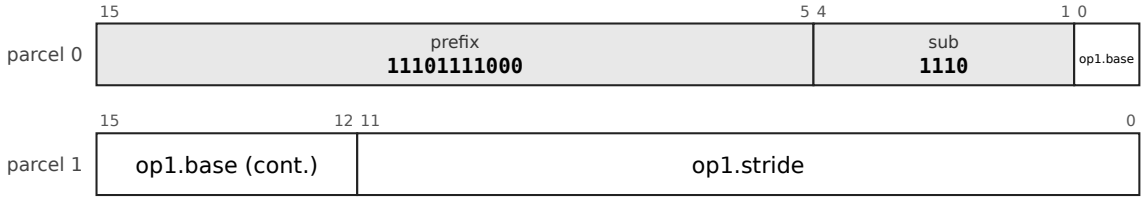
**JMPI: SOF.PC32, prefix 111100000100, sub-opcode 1110; operands pc32-u1; 6 bytes**



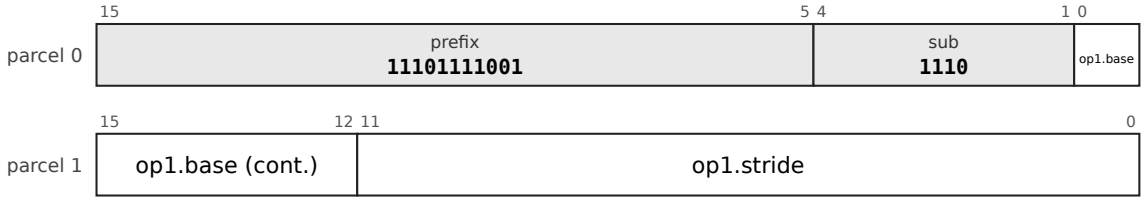

**JMPI: SOF.A32, prefix 111100000101, sub-opcode 1110; operands abs32; 6 bytes**



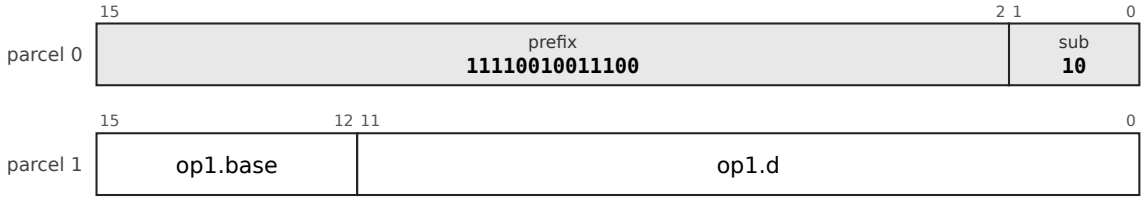

**JMPI: SOF.PI12, prefix 11101111000, sub-opcode 1110; operands pi5-s12; 4 bytes**



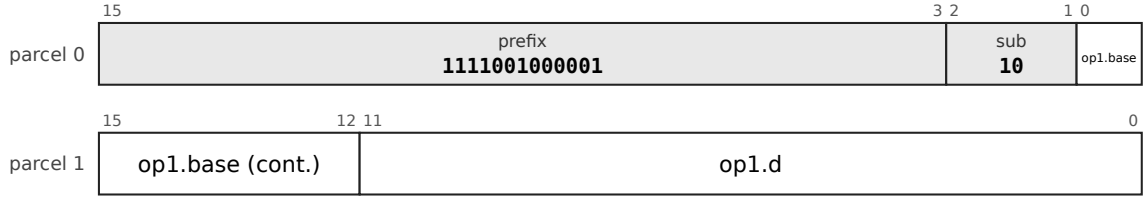

**JMPI: SOF.PD12, prefix 11101111001, sub-opcode 1110; operands pd5-s12; 4 bytes**



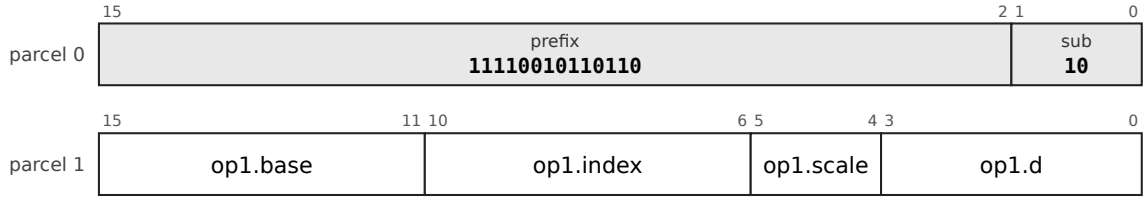

**JMPI: SOBu12H/x, prefix 11110010011100, sub-opcode 10; operands b4d12u-xsz; 4 bytes**



**JMPI: SOBs12/x, prefix 1111001000001, sub-opcode 10; operands b5d12s-xsz; 4 bytes**




**JMPI: SOX4/x, prefix 11110010110110, sub-opcode 10; operands x5-5d4s; 4 bytes**

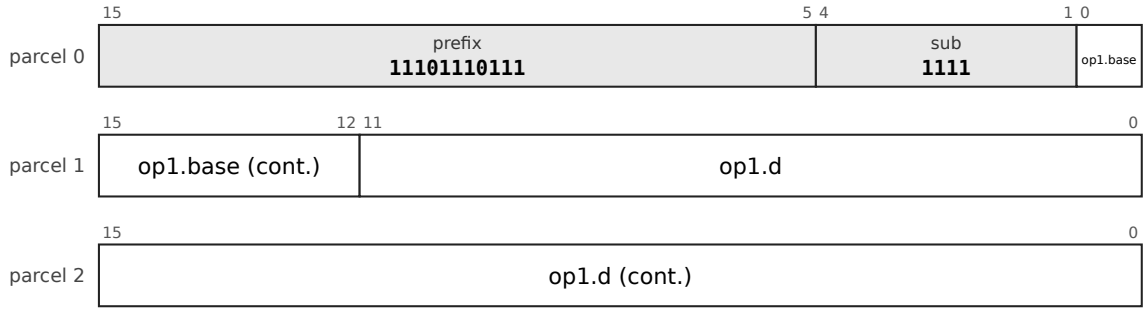



```
JMPI rm
```

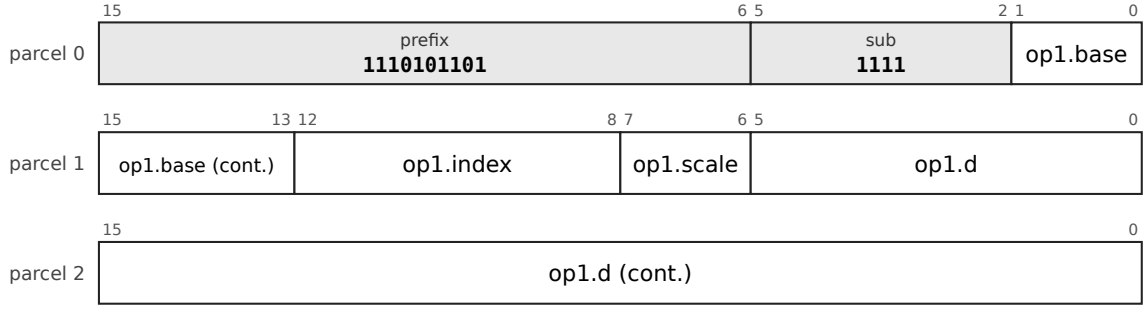
Operation, flags and exceptions as for the entry JMPI above; 9 layout forms, each a chart of its own (above).

### *CALLI*

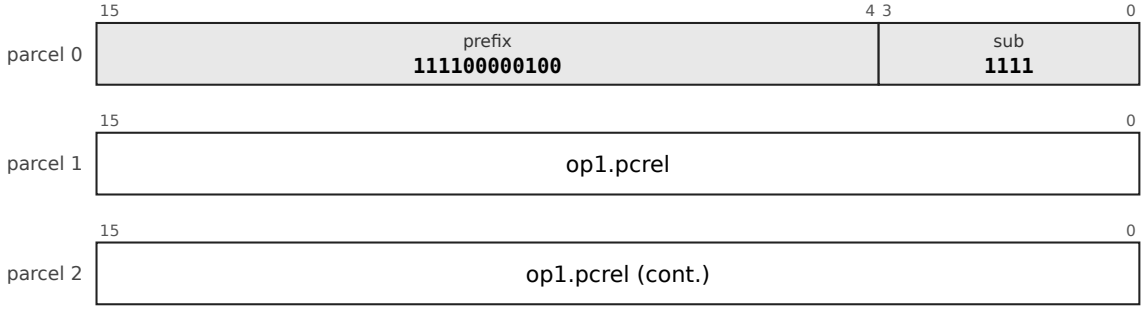
**CALLI: SOF.B28, prefix 11101110111, sub-opcode 1111; operands b5d28s-x1; 6 bytes**



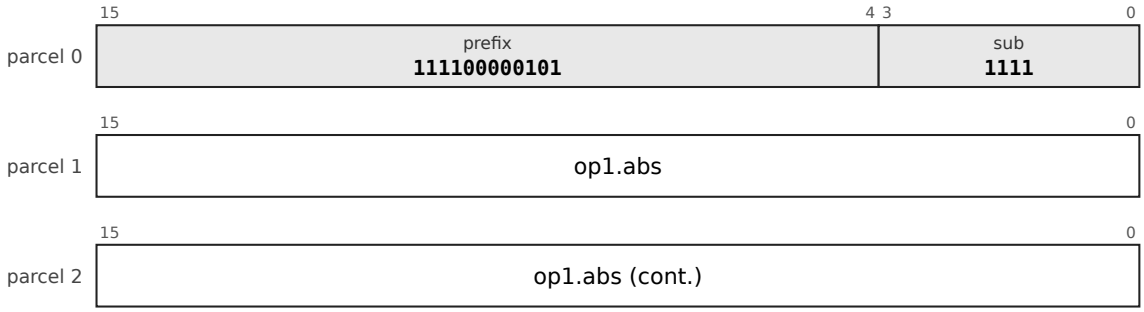

**CALLI: SOF.X22, prefix 1110101101, sub-opcode 1111; operands x5-5d22s; 6 bytes**



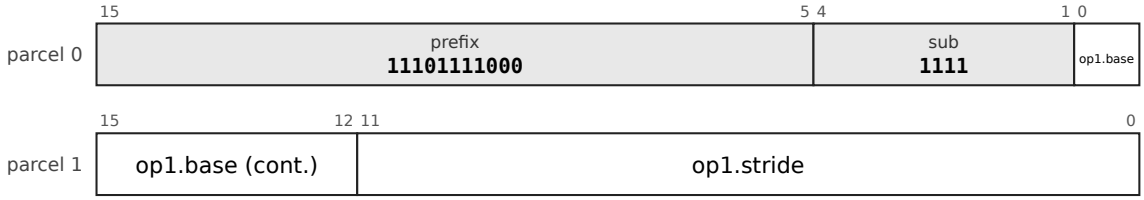
**CALLI: SOF.PC32, prefix 111100000100, sub-opcode 1111; operands pc32-u1; 6 bytes**



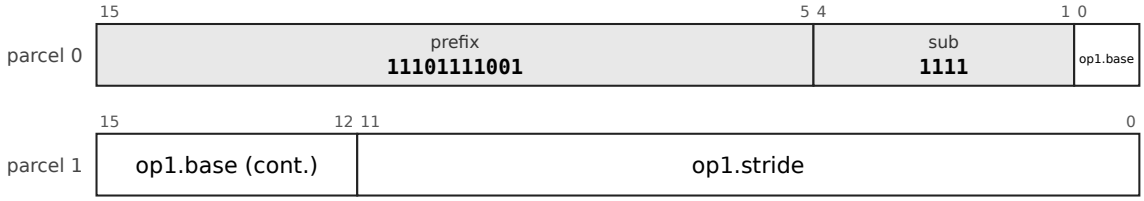

**CALLI: SOF.A32, prefix 111100000101, sub-opcode 1111; operands abs32; 6 bytes**



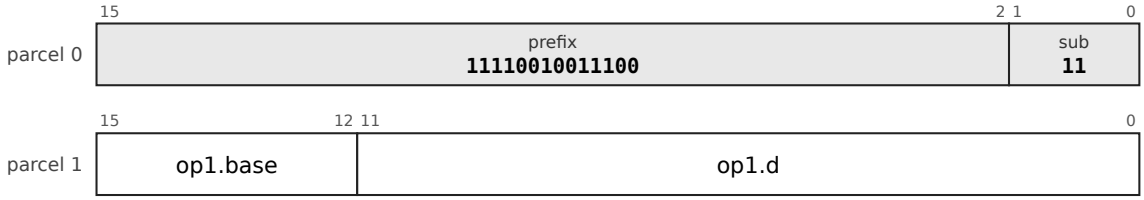

**CALLI: SOF.PI12, prefix 11101111000, sub-opcode 1111; operands pi5-s12; 4 bytes**



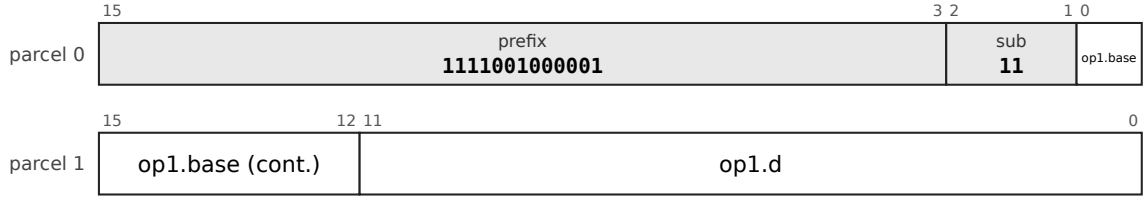

**CALLI: SOF.PD12, prefix 11101111001, sub-opcode 1111; operands pd5-s12; 4 bytes**



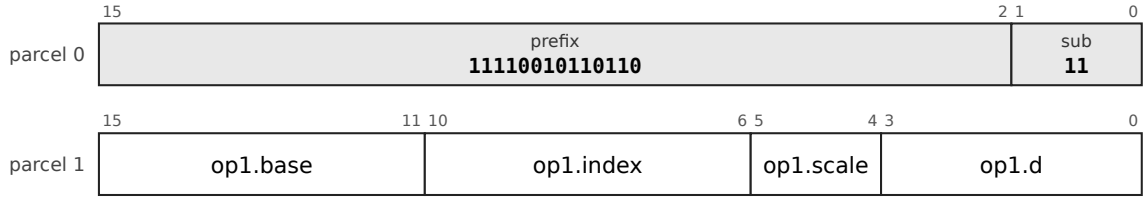

**CALLI: SOBu12H/x, prefix 11110010011100, sub-opcode 11; operands b4d12u-xsz; 4 bytes**



**CALLI: SOBs12/x, prefix 1111001000001, sub-opcode 11; operands b5d12s-xsz; 4 bytes**




**CALLI: SOX4/x, prefix 11110010110110, sub-opcode 11; operands x5-5d4s; 4 bytes**

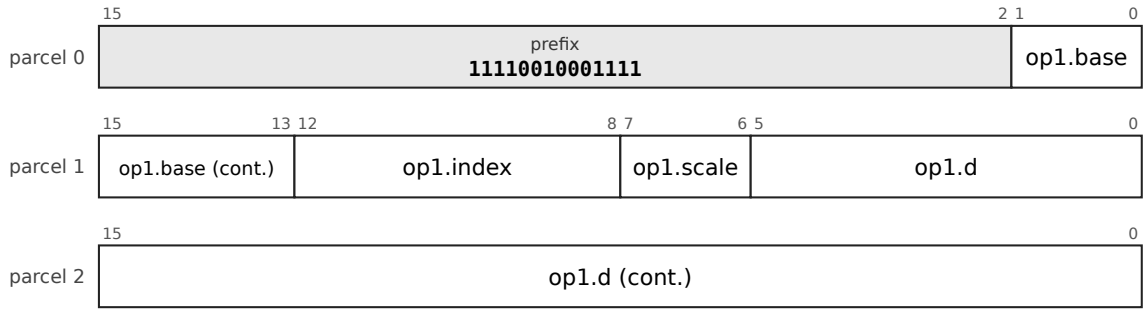



```
CALLI rm
```

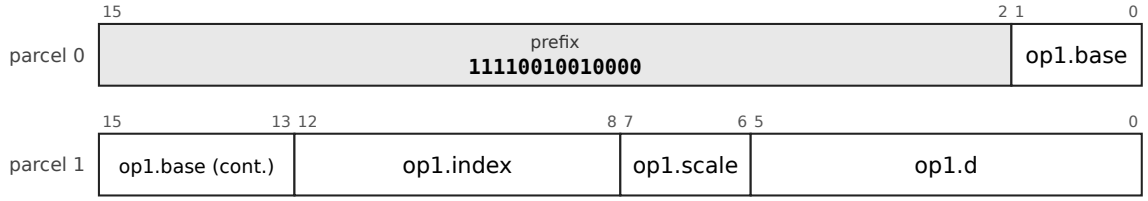
Operation, flags and exceptions as for the entry JMPI above; 9 layout forms, each a chart of its own (above).

## JTAG: Jump through a tag table

**JTAG: JTAG3, prefix 11110010001111; operands x5-5d22s; 6 bytes**




**JTAG: JTAG2, prefix 11110010010000; operands x5-5d6s; 4 bytes**

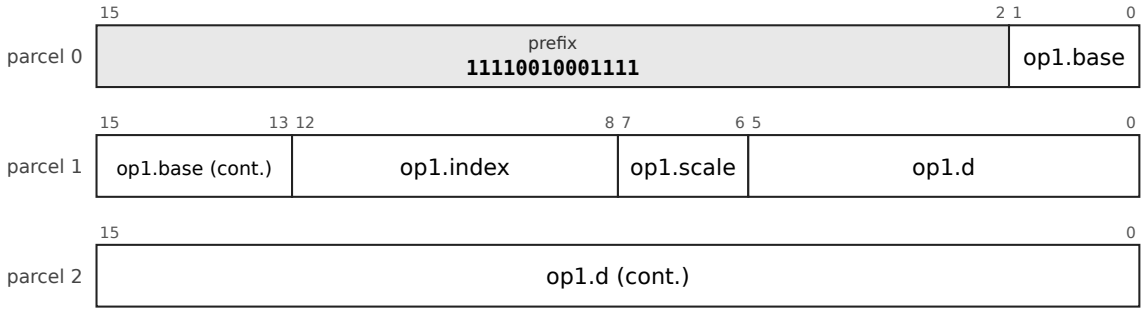



**Mnemonics:** `JTAG`

### Syntax

```
JTAG d(Rb)(Rx×s)
```

### Encoding




The chart is the leaf `JTAG3-1`, JTAG3, prefix `11110010001111`. `op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `JTAG3-1`, `JTAG2-1`.

### Operation

```
PC ← M[Rb + d + 8 × (Rx & 111₂)]
```

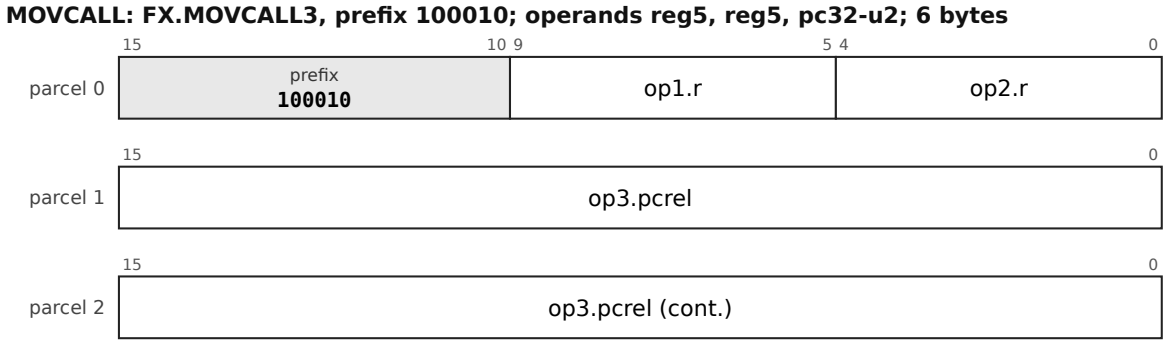
**Flags** None written.

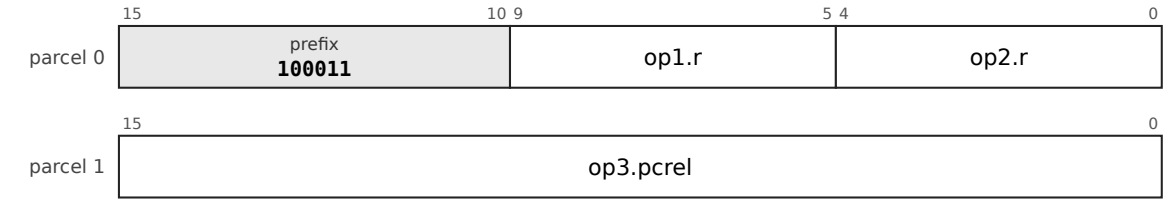
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

Legal only with the scaled-index mode (`0001`). Dispatches on the low three bits of a tagged pointer through a table of eight entries; the 3-bit tag cannot index out of range.

**Why it is here.** The table-dispatch partner of `BTAG`. It is legal only in the scaled-index mode, the one mode with a free register slot for the tagged value. It replaces the PDP-11 dispatch-table idiom `@(Rn)+` better, because it needs no pointer register and cannot index out of range.

## MOVCALL: Move a register and call



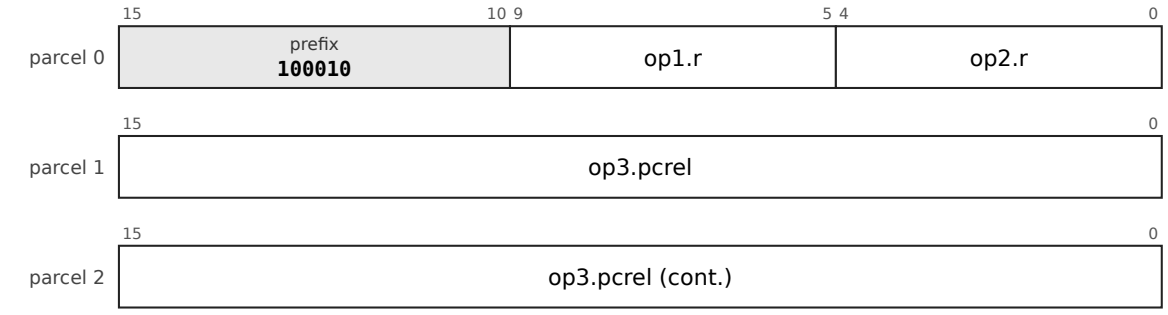




**Mnemonics:** `MOVCALL`

### Syntax

```
MOVCALL Rs, Rd, target
```

### Encoding




The chart is the leaf `FX-MOVCALL3-1`, FX.MOVCALL3, prefix `100010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `FX-MOVCALL3-1`, `FX-MOVCALL2-1`.

### Operation

```
Rd ← Rs
SP ← SP - 8 ; M[SP] ← address of the next instruction
PC ← target
```

**Flags** None written.
**Exceptions** The store of the return address can fault (the memory faults of `CALL`): the instruction restarts whole, because the first half reads a register that the second half does not change.

*(D)* One of the four fused pairs of errata 87: `MOVQ Rs, Rd` followed by `CALL target`, in one instruction. The target is a pc-relative field in parcels, measured from the first parcel of the fused instruction (errata 87, assumption 9); the instruction is two parcels (a 16-bit field) or three (a 32-bit field), and the linker grows it from its shortest form like a `CALL` (`docs/relaxation.md`).
**The assembler never fuses**: the compiler writes `MOVCALL` where it wants it. An instruction that writes two registers (`Rd` and the stack pointer) and stores to memory is a two-micro-operation instruction for an out-of-order core (errata 87).
**Why it is here.** The author decided the four fused pairs in errata 87 (`movcall`, `popmret`, `popret`, `mov2`) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of `docs/encoding-proposal.md` section 8 (static size of the training corpus, in the encoding): `MOVQ Rs, Rd` followed by `CALL` occurs 11,123 times, and fusing it alone shrinks the code by 2.39%, the largest of the four; all four together fuse 20,428 pairs and save 3.18% of the bytes and about 1.1% of the executed instructions. The call keeps its reach (four or six bytes). The cost, named in the encoding study: a new mnemonic, and one instruction that writes two registers (the destination and the stack pointer), which is two micro-operations for an out-of-order core. The record gives no earlier machine for this particular pair; the reasoning is the fourth priority of the definitive list (errata 83), fewer instructions executed, where coalescing is good.
**Cf.** 9.5 (the pc-relative operand and the relaxation), 6.4 (restart after a fault).

## POPRET: Pop a register and return

**POPRET: FX.POPRET, prefix 11101111100; operands reg5; 2 bytes**




**Mnemonics:** `POPRET`

### Syntax

```
POPRET Rd
```

### Encoding




The chart is the leaf `FX-POPRET-1`, FX.POPRET, prefix `11101111100`. `op1` is `reg5`: register, 5 bits.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `FX-POPRET-1`.

### Operation

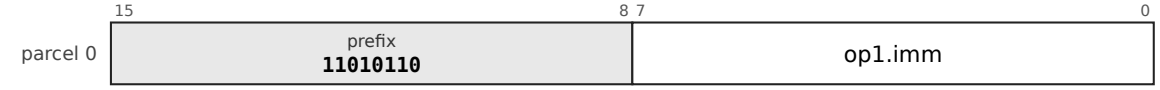
```
Rd ← M[SP] ; SP ← SP + 8
PC ← M[SP] ; SP ← SP + 8
```

**Flags** None written.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
*(D)* `POP Rd` followed by `RET`, fused (errata 87). The register is the one the function restores last, usually a callee-saved register or the frame pointer. One parcel of 16 bits, with the register in a 5-bit field.
**Why it is here.** The author decided the four fused pairs in errata 87 (`movcall`, `popmret`, `popret`, `mov2`) and left the fusing to the compiler: the assembler never fuses. The reason is the measurement of `docs/encoding-proposal.md` section 8 (static size of the training corpus, in the encoding): `POP Rd` followed by `RET` and `POPM` followed by `RET` (`POPMRET`) occur 4,862 times together and save 1.00% of the bytes; this is the epilogue of nearly every function. The fault of the second half restarts the whole instruction,

which is idempotent because the first half reads a register the second does not change and POPM is already specified restartable (decision 2 of the encoding study).

### POPMRET: Pop several registers and return

**POPMRET: FX.POPMRET, prefix 11010110; operands imm8u-x256; 2 bytes**

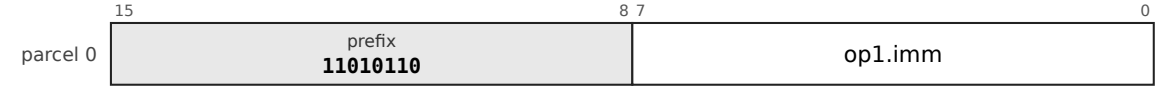



**Mnemonics:** POPMRET

**Syntax**

```
POPMRET #mask
```

**Encoding**




The chart is the leaf FX-POPMRET-1, FX.POPMRET, prefix 11010110. op1 is imm8u-x256: unsigned 8-bit immediate times 256, 0 to 65280.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): FX-POPMRET-1.

**Operation**

```
for each bit i set in mask, highest register first:
    Ri ← M[SP] ; SP ← SP + 8
PC ← M[SP] ; SP ← SP + 8
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A fault after some registers have been popped leaves them popped: the instruction restarts whole and pops them again, which is idempotent because the stack pointer is restored with the rest of the state at the start of the instruction.

*(D)* POPM #mask followed by RET, fused (errata 87): the epilogue of every function that saved registers with PUSHM. The mask is an 8-bit field scaled by 256, so the instruction names R8 to R15; any other set of registers is POPM and RET. One parcel.

**Why it is here.** See POPRET: the two are measured together (4,862 pairs, 1.00% of the bytes). The mask is an 8-bit field scaled by 256 because one parcel has no room for a wider one; the epilogue that restores R8 to R15 is the common case the corpus shows, and any other set stays POPM and RET.

## 9.11 The stack and block operations

*Cf. section 6.4 for restarting a block operation after a fault (progress is held in registers) and section 6.6 for the page faults it can take; section 7.5.8 for how block operations order their accesses (decided in part; the rest is open, J.7.7).*

**9.11.1** PUSH and POP move one register; PUSHM and POPM move a set named by a mask (a mask of 8 bits is two bytes). A mask containing R14 is reserved. *(D)*

**9.11.2** PUSHX and POPX save and restore the exception state EPSW and EPC in one instruction each; both words are checked before either is accessed, so an abort leaves nothing changed. *(D)*

**9.11.3** The block operations MCPY, MMOV (overlap-safe), MSET, MCMP, MSCN and MSCNN take a destination pointer, a source pointer or value and a count in registers; MVAL validates a buffer as UTF-8.

**9.11.4** A block operation keeps its progress in its registers: when it completes, the count is zero and the pointers are one past the end; when it stops early or is interrupted, they point where it stopped, and it restarts by running again. *(D)*

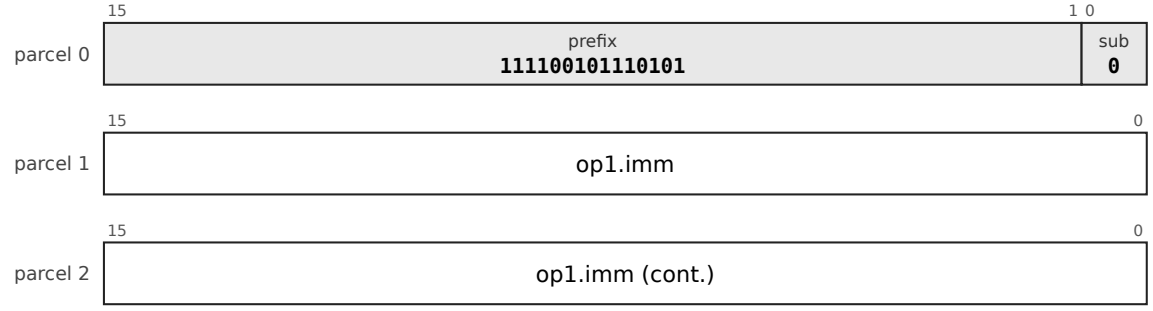
**9.11.5** An implementation may move more than a byte per step, provided the registers are consistent with byte-at-a-time progress wherever an exception can be taken. *(D)*

Open: what another core may see of a block operation in progress (J.7.7), the worst-case latency of a block operation (J.9.7) and whether the block operations have a form that does not fill the data cache (J.9.8).

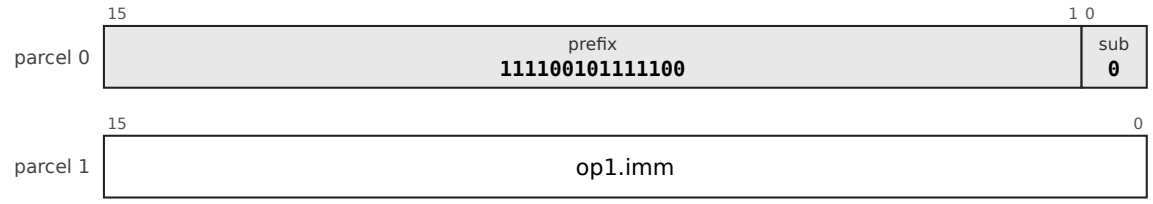
*Commentary.* **Precedent.** The VAX's `MOVC5` is the canonical argument against such instructions: its operands could be memory, its architectural definition needed the original values to survive, and so it required hidden state and a first-part-done bit in the status word to resume. The PDP-10 had a block transfer, and x86's `REP MOVS` keeps its progress in registers as the pdp64's do. **Trade-offs.** This is the one admitted exception to the one-memory-operand rule, accepted because its mechanism, hidden state, is absent: the state is the registers, which are already architectural. **Improvements.** ARM reached the same design in 2022 with `FEAT_MOPS`, whose specification says the registers are updated so that progress is not lost. **Benefits.** The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors. **Modern comparables.** x86 `REP MOVSB` and `REP STOSB` with fast-string microcode; AArch64 `CPYP`/`CPYM`/`CPYE` and `SETP`/`SETM`/`SETE`; s390 `MVC`. **Use.** `memcpy`, `memmove`, `memset`, `memcmp` and `memchr` in the C library; `MVAL` in parsers and kernels. **Impact.** *Kernel:* `PUSHM`, `POPM`, `PUSHX` and `POPX` shorten entry, exit and the context switch, and the block operations are `copyin`, `copyout`, page clearing and `memcpy`. *Compiler:* A prologue and an epilogue are one instruction each, and `memcpy` and `memset` lower to the block operations. *Hardware:* The block-move sequencer is three counters and a byte-lane selector, about 1,500 transistors, with no hidden state.

## PUSHM: Push registers by mask

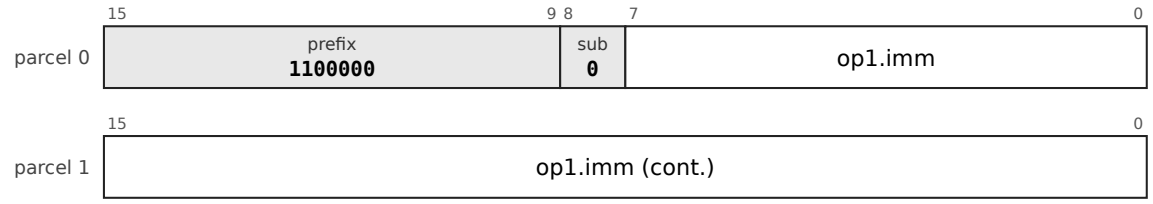
**PUSHM: PUSHMF, prefix 111100101110101, sub-opcode 0; operands imm32u; 6 bytes**



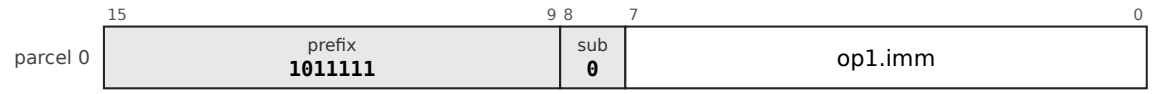

**PUSHM: PUSHM16b/a, prefix 111100101111100, sub-opcode 0; operands imm16u-x256; 4 by**




**PUSHM: PUSHM24/a, prefix 1100000, sub-opcode 0; operands imm24u-x256; 4 bytes**




**PUSHM: PUSHM8/a, prefix 1011111, sub-opcode 0; operands imm8u-x256; 2 bytes**

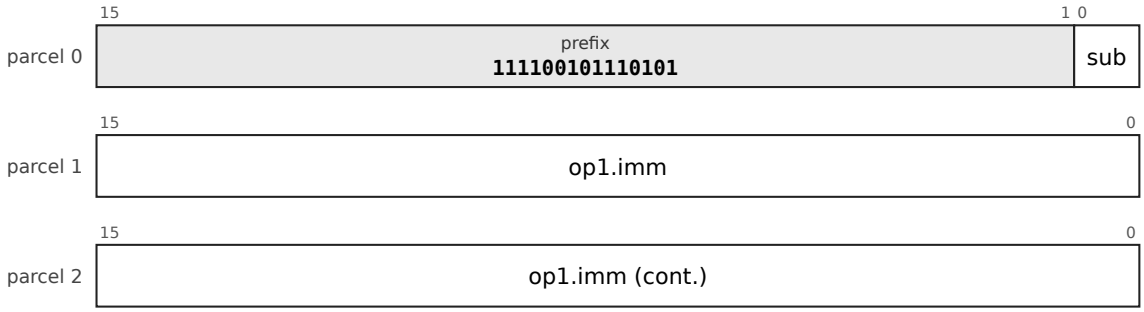



**Mnemonics:** `PUSHM`

### Syntax

```
PUSHM #mask        ; a 16-bit immediate, bit n is Rn
```

### Encoding




The chart is the leaf `PUSHMF-1`, PUSHMF, prefix `111100101110101`. `op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295. `sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `PUSHMF-1`, `PUSHM16b-a-1`, `PUSHM8-a-1`, `PUSHM24-a-1`.

### Operation

```
for n = 0 to 15, if mask bit n:
    SP ← SP - 8 ; M[SP] ← Rn
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. A mask that includes `R14` (`SP`) is reserved.

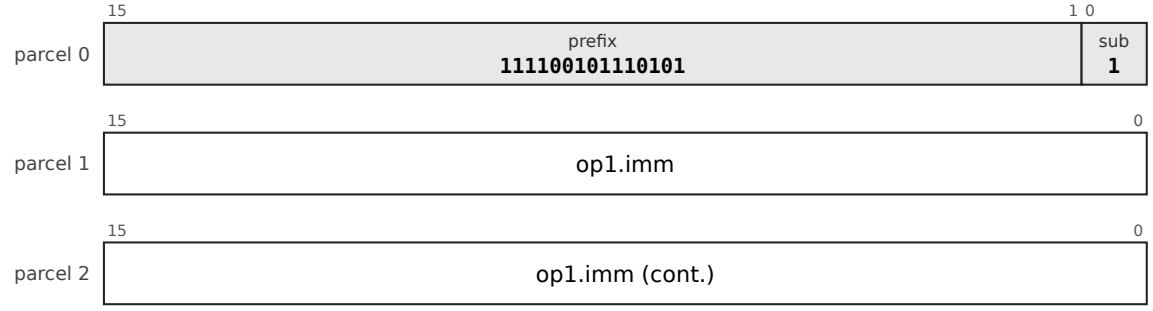
The registers are pushed in ascending order, so `POPM` with the same mask restores them. The mask is the low 32 bits of the number (errata 87, assumption 7); the one-parcel form takes an 8-bit mask scaled by 256, which names `R8` to `R15`.

**Why it is here.** The VAX procedure entry mask was the useful half of `CALLS`; `PUSHM` and `POPM` keep that half without the frame (hardware call frames ran slower than the sequence on later VAXes). The one-parcel forms take an 8-bit mask for `R8` to `R15`, because all 2,392 push sites and 2,392 pop sites in the measured corpus use a mask within `R8` to `R15`.

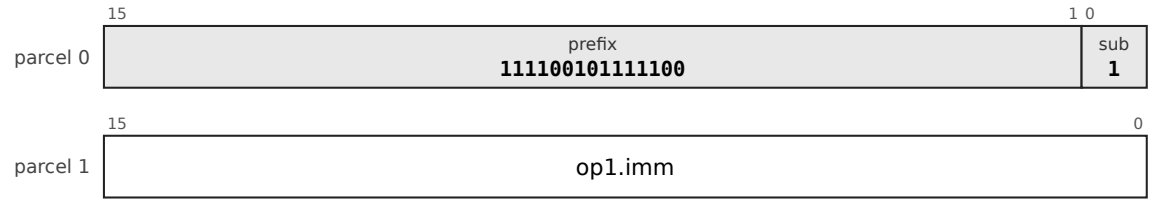
**Cf.** 6.4 for the restart rule.

## POPM: Pop registers by mask

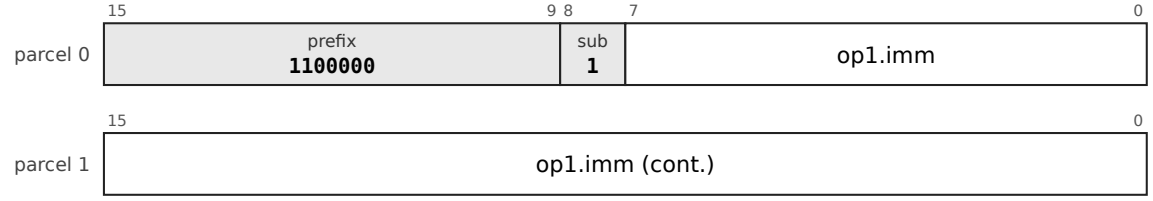
**POPM: PUSHMF, prefix 111100101110101, sub-opcode 1; operands imm32u; 6 bytes**



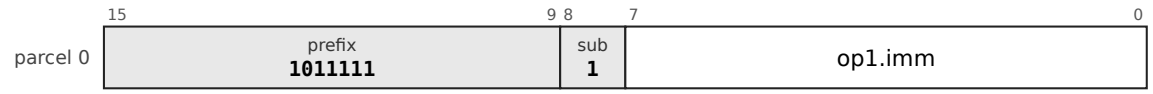

**POPM: PUSHM16b/a, prefix 111100101111100, sub-opcode 1; operands imm16u-x256; 4 byte**




**POPM: PUSHM24/a, prefix 1100000, sub-opcode 1; operands imm24u-x256; 4 bytes**




**POPM: PUSHM8/a, prefix 1011111, sub-opcode 1; operands imm8u-x256; 2 bytes**

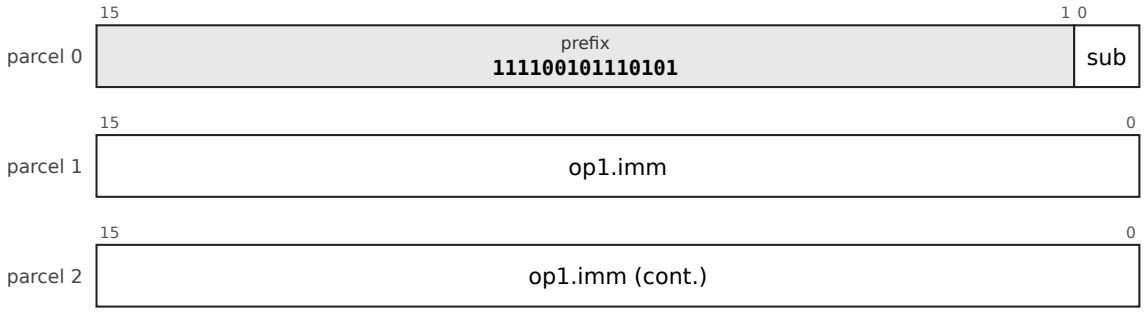



**Mnemonics:** `POPM`

### Syntax

```
POPM #mask
```

### Encoding




The chart is the leaf `PUSHMF-1`, PUSHMF, prefix `111100101110101`. `op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295. `sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `PUSHMF-1`, `PUSHM16b-a-1`, `PUSHM8-a-1`, `PUSHM24-a-1`.

### Operation

```
for n = 15 down to 0, if mask bit n:
    Rn ← M[SP] ; SP ← SP + 8
```

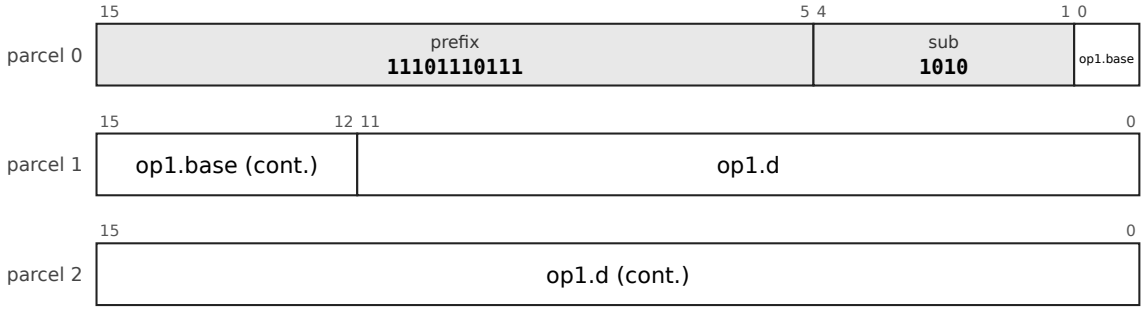
**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

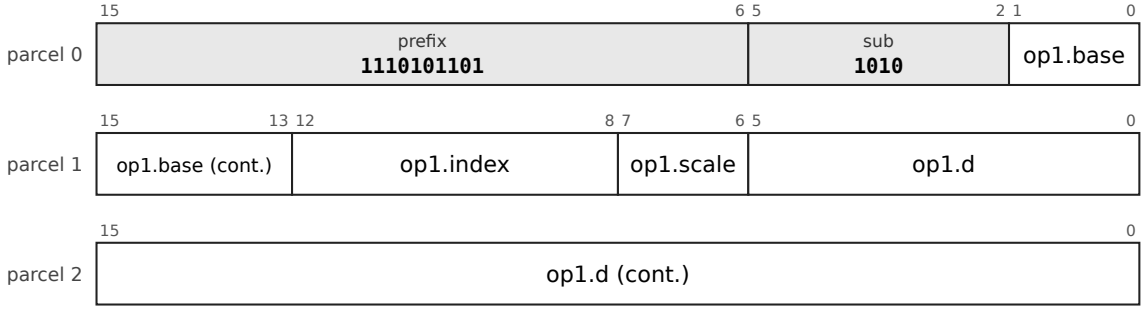
The mirror of `PUSHM`.

## PUSH: Push

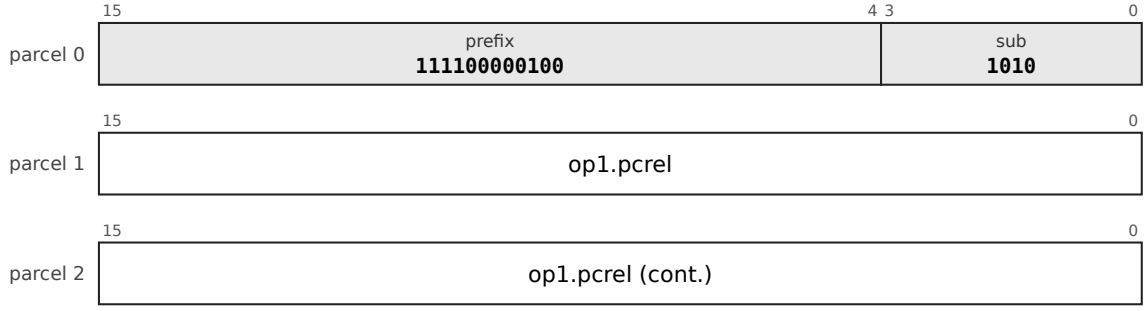
**PUSH: SOF.B28, prefix 11101110111, sub-opcode 1010; operands b5d28s-x1; 6 bytes**



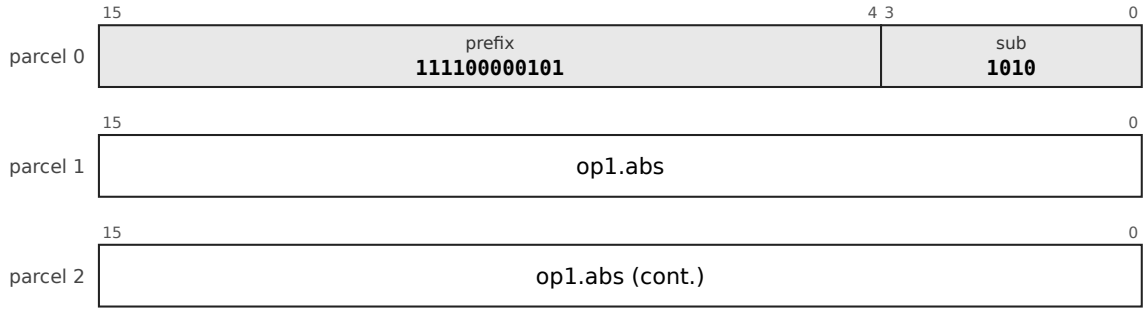

**PUSH: SOF.X22, prefix 1110101101, sub-opcode 1010; operands x5-5d22s; 6 bytes**



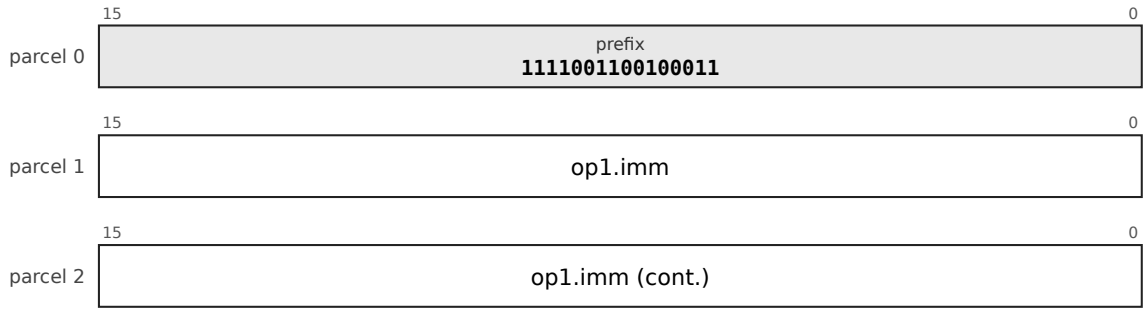
**PUSH: SOF.PC32, prefix 111100000100, sub-opcode 1010; operands pc32-u1; 6 bytes**



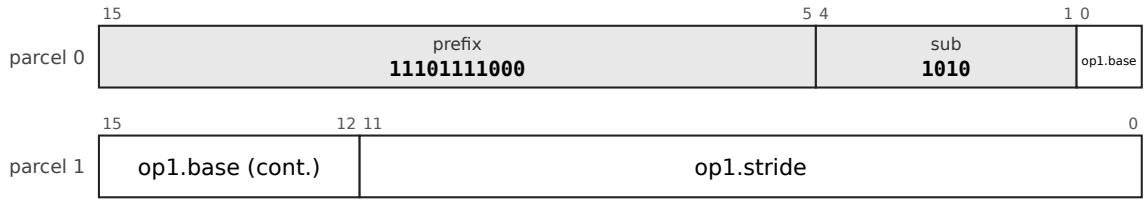

**PUSH: SOF.A32, prefix 111100000101, sub-opcode 1010; operands abs32; 6 bytes**



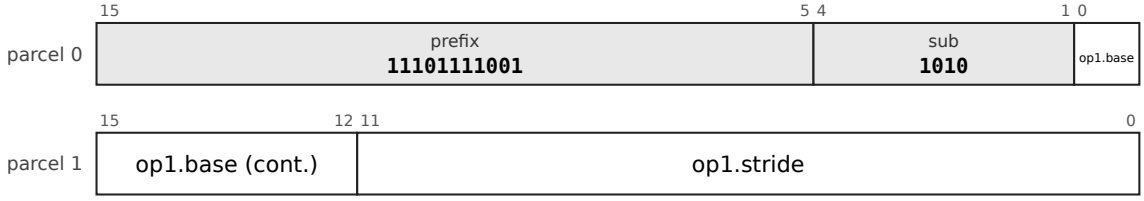

**PUSH: PUSHI3, prefix 1111001100100011; operands imm32s; 6 bytes**



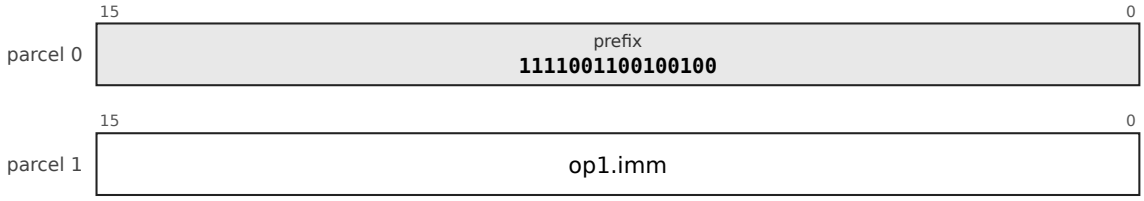

**PUSH: SOF.PI12, prefix 11101111000, sub-opcode 1010; operands pi5-s12; 4 bytes**



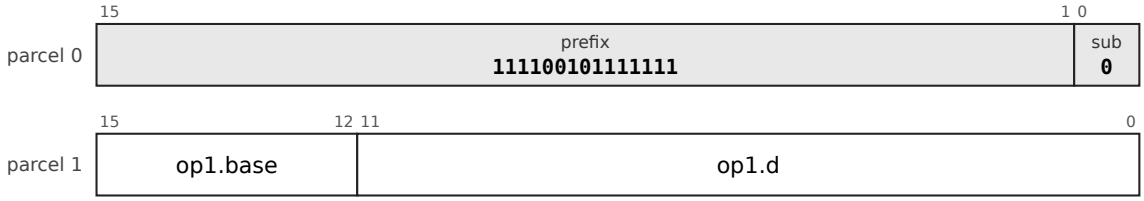
**PUSH: SOF.PD12, prefix 11101111001, sub-opcode 1010; operands pd5-s12; 4 bytes**



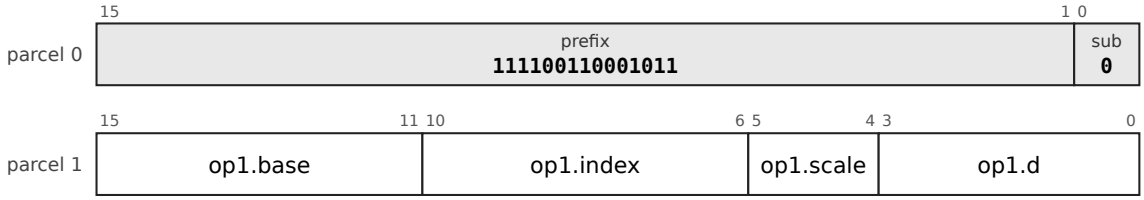

**PUSH: PUSHI2, prefix 1111001100100100; operands imm16s; 4 bytes**



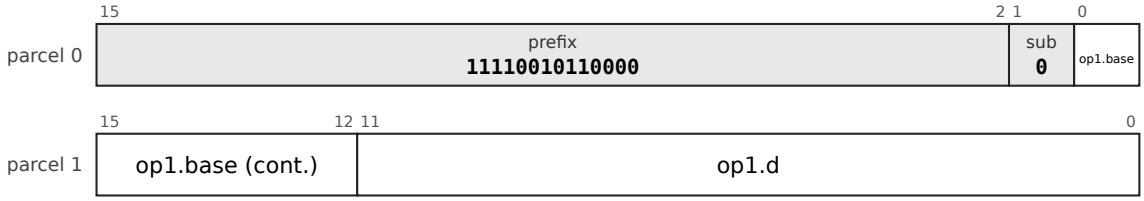

**PUSH: SOBu12H/a, prefix 111100101111111, sub-opcode 0; operands b4d12u-xsz; 4 bytes**




**PUSH: SOX4/a, prefix 111100110001011, sub-opcode 0; operands x5-5d4s; 4 bytes**




**PUSH: SOBs12/a, prefix 11110010110000, sub-opcode 0; operands b5d12s-xsz; 4 bytes**




**PUSH: SOPR, prefix 111000011, sub-opcode 10; operands reg5; 2 bytes**

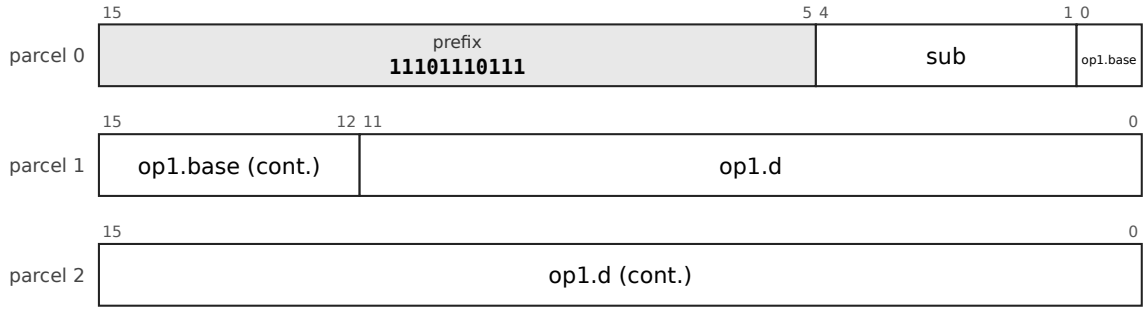



**Mnemonics:** PUSH

### Syntax

```
PUSH rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-2`, `PUSHI3-1`, `PUSHI2-1`, `SOBu12H-a-2`, `SOX4-a-2`, `SOBs12-a-2`.

### Operation

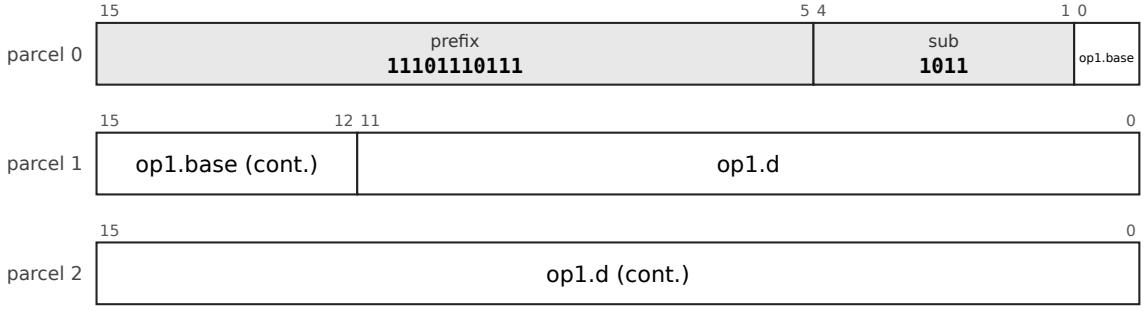
```
v ← rm ; SP ← SP - 8 ; M[SP] ← v
```

**Flags** None written.

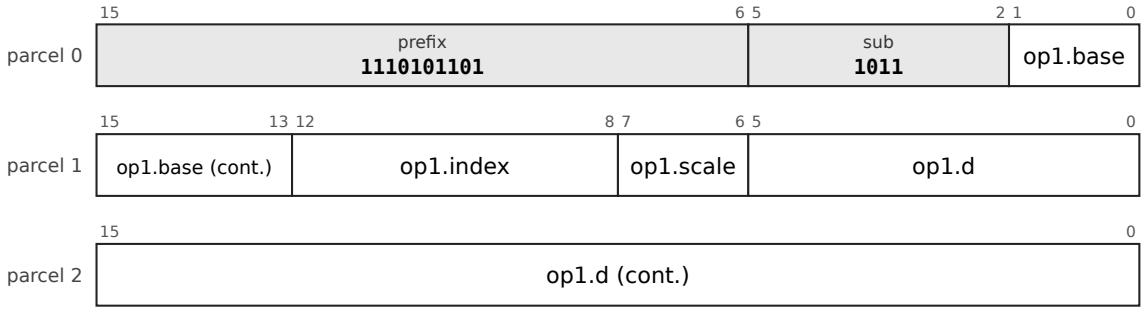
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

## POP: Pop

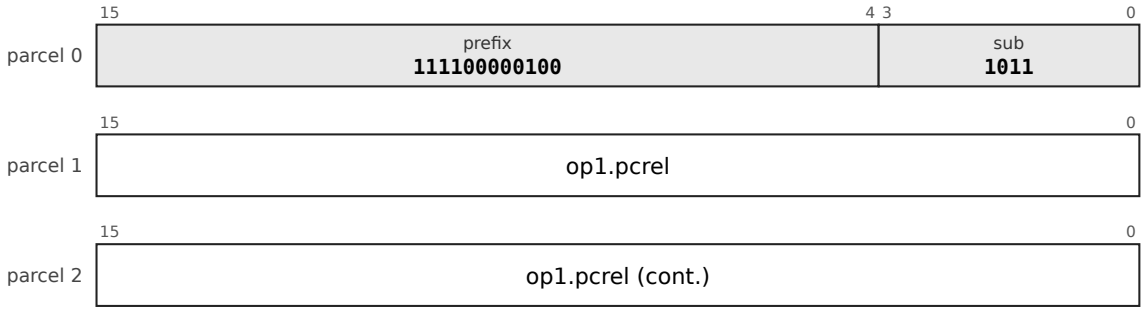
**POP: SOF.B28, prefix 11101110111, sub-opcode 1011; operands b5d28s-x1; 6 bytes**



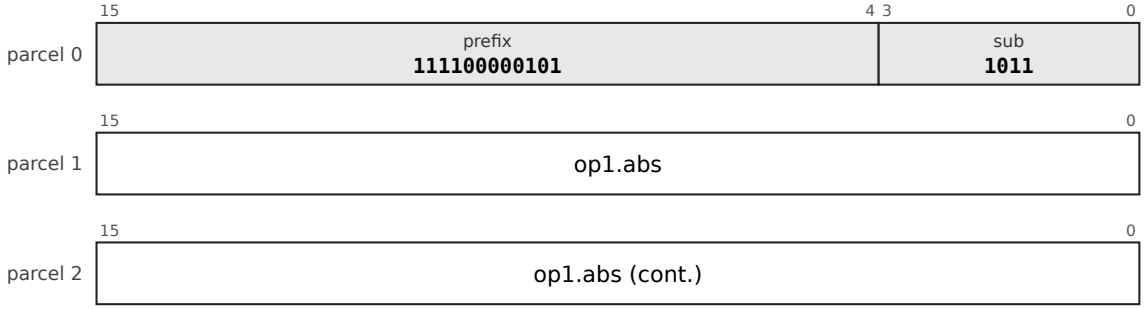

**POP: SOF.X22, prefix 1110101101, sub-opcode 1011; operands x5-5d22s; 6 bytes**



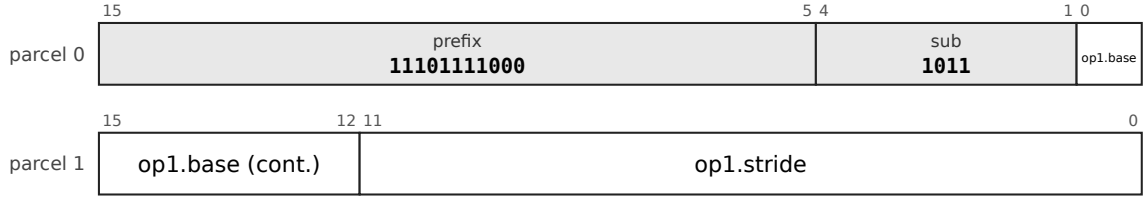

**POP: SOF.PC32, prefix 111100000100, sub-opcode 1011; operands pc32-u1; 6 bytes**



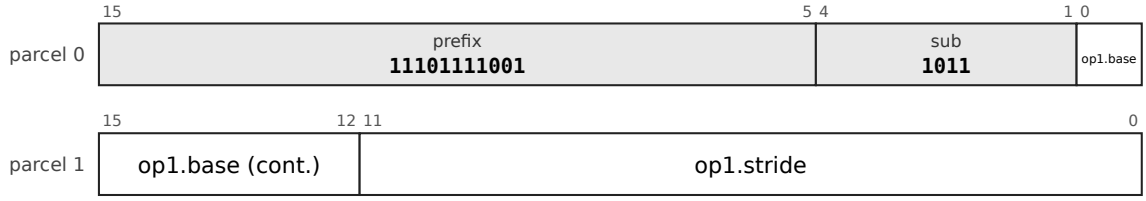

**POP: SOF.A32, prefix 111100000101, sub-opcode 1011; operands abs32; 6 bytes**



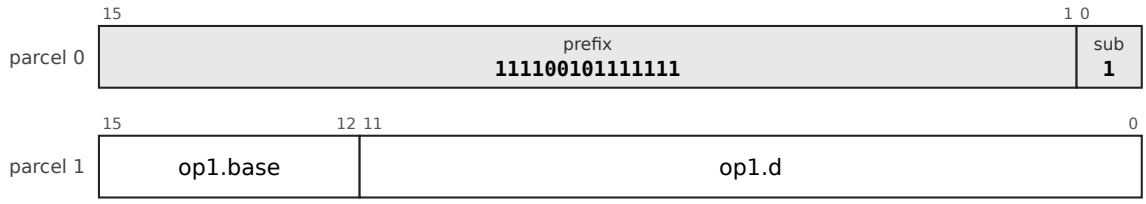
**POP: SOF.PI12, prefix 11101111000, sub-opcode 1011; operands pi5-s12; 4 bytes**



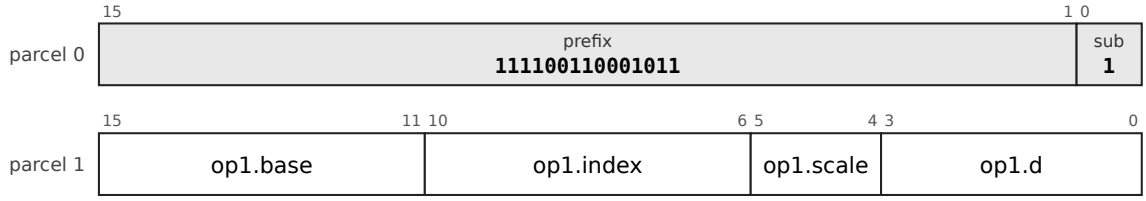

**POP: SOF.PD12, prefix 11101111001, sub-opcode 1011; operands pd5-s12; 4 bytes**



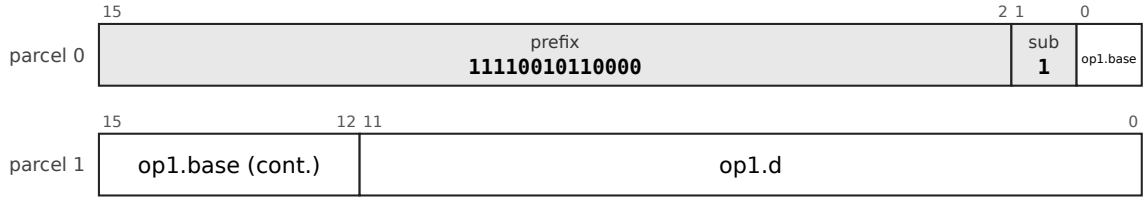

**POP: SOBu12H/a, prefix 111100101111111, sub-opcode 1; operands b4d12u-xsz; 4 bytes**




**POP: SOX4/a, prefix 111100110001011, sub-opcode 1; operands x5-5d4s; 4 bytes**




**POP: SOBs12/a, prefix 11110010110000, sub-opcode 1; operands b5d12s-xsz; 4 bytes**




**POP: SOPR, prefix 111000011, sub-opcode 11; operands reg5; 2 bytes**

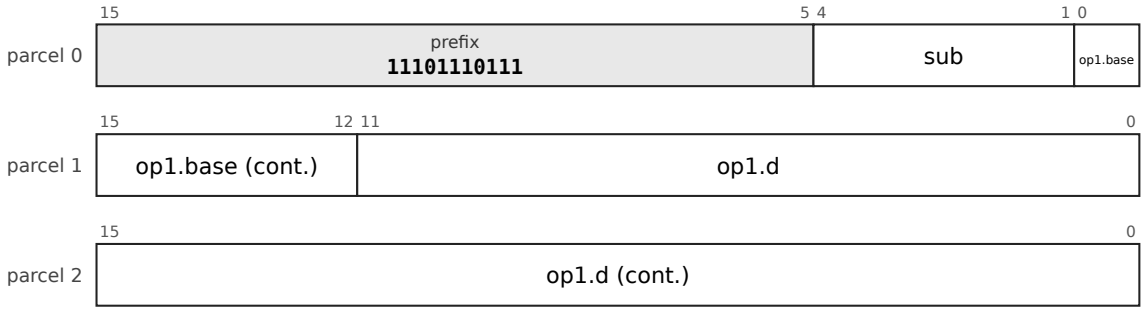



**Mnemonics:** POP

### Syntax

```
POP rm
```

### Encoding




The chart is the leaf `SOF-B28-1`, SOF.B28, prefix `11101110111`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SOF-B28-1`, `SOF-X22-1`, `SOF-PI12-1`, `SOF-PD12-1`, `SOF-PC32-1`, `SOF-A32-1`, `SOPR-2`, `SOBu12H-a-2`, `SOX4-a-2`, `SOBs12-a-2`.

### Operation

```
v ← M[SP] ; SP ← SP + 8 ; rm ← v
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

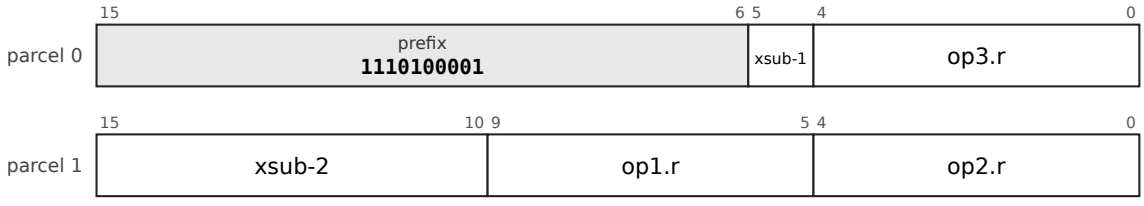
## MCPY: Copy a block of bytes

**Mnemonics:** `MCPY`, `MMOV`

### Syntax

```
MCPY Rd, Rs, Rn        ; destination pointer, source pointer, count
MMOV Rd, Rs, Rn        ; correct for overlap
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

**Operation**

```
MCPY:                           ; ascending; overlap is undefined
  while Rn ≠ 0:  M8[Rd] ← M8[Rs] ; Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
MMOV: as MCPY, but the direction is chosen from the pointers so that an overlapping copy is correct
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. The three registers are updated as work completes, so a fault leaves a consistent state and the instruction restarts from its registers.

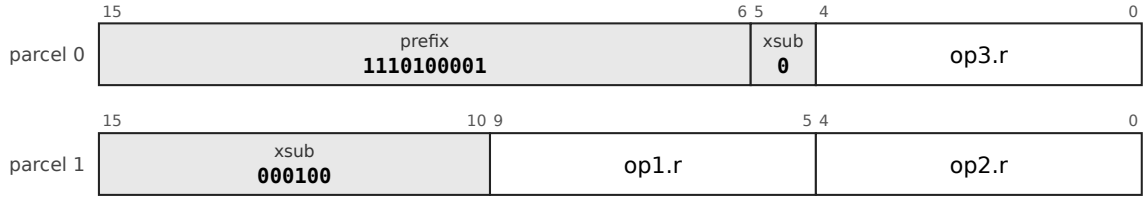
The one granted exception to the one-memory-operand rule. All three registers are architectural state: on completion Rn is 0 and Rd and Rs point one past the end. An implementation may move more than a byte per step, so long as the register state at every point where an exception can be taken is consistent with a byte-at-a-time copy. The accesses made inside one block operation are unordered with respect to each other.

**Why it is here.** The block operations are the one granted exception to the one-memory-operand rule (an exception to a principle is allowed where there is a clear benefit), made safe by keeping the progress in architectural registers: the VAX MOVC5 kept it in hidden microstate, which is why it is rejected, and ARM reached the same design in 2022 with FEAT_MOPS. The names MCPY, MMOV and MSET follow the memory operations of IBM System z; MCMP and MSCN complete the set. MCPY replaces the PDP-11 MOV (R1)+,(R2)+ loop, and the rep movsb of x86, with one restartable instruction.

**Cf.** 6.4 (restart, progress faults); 7.5 (ordering of the accesses).

***MCPY***

**MCPY: R3, prefix 1110100001, sub-opcode 0000100; operands reg5, reg5, reg5; 4 bytes**

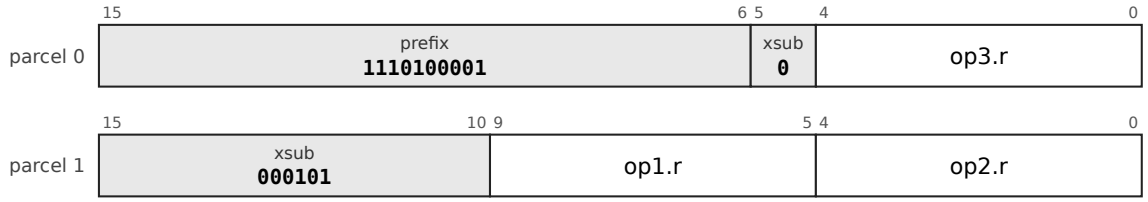



```
MCPY Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MCPY above; 1 layout form, each a chart of its own (above).

***MMOV***

**MMOV: R3, prefix 1110100001, sub-opcode 0000101; operands reg5, reg5, reg5; 4 bytes**

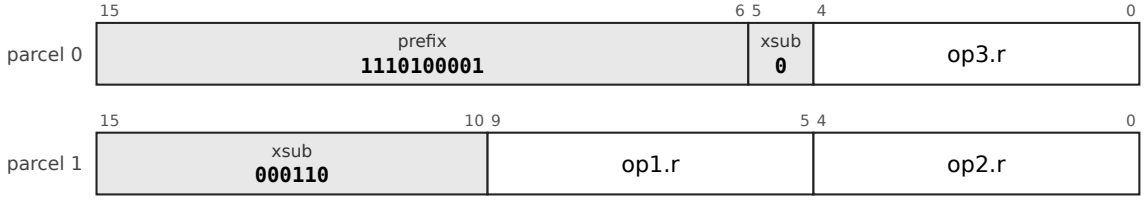



```
MMOV Rd, Rs, Rn
```

Operation, flags and exceptions as for the entry MCPY above; 1 layout form, each a chart of its own (above).

## MSET: Fill a block of bytes

**MSET: R3, prefix 1110100001, sub-opcode 0000110; operands reg5, reg5, reg5; 4 bytes**

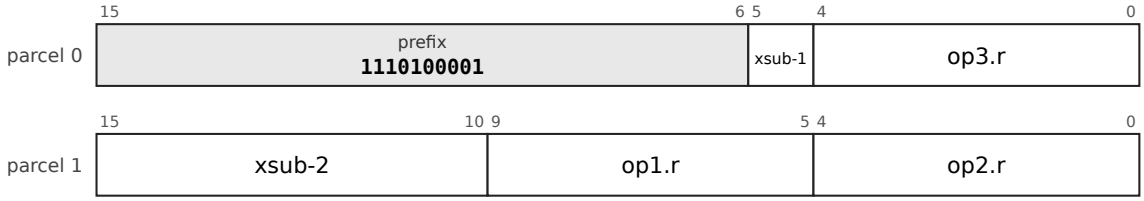



**Mnemonics:** `MSET`

### Syntax

```
MSET Rd, Rv, Rn
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
while Rn ≠ 0:  M8[Rd] ← Rv[7:0] ; Rd ← Rd + 1 ; Rn ← Rn - 1
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

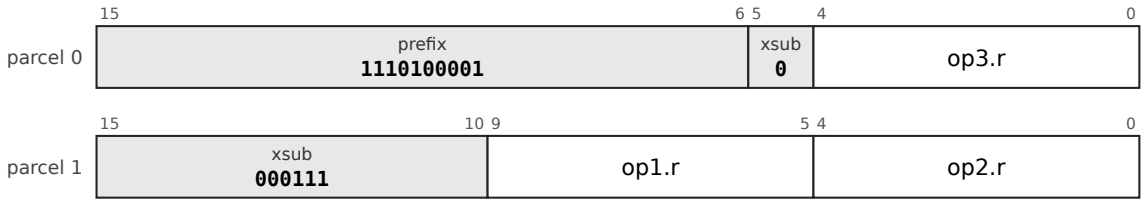
`MSET` with `Rv = 0` is the range-zeroing instruction: `bzero` and the clear of `calloc` are one instruction.

**Why it is here.** With a zero value it is the range-zeroing instruction, so `bzero` and the clearing of `calloc` need no separate opcode.

**Cf.** 6.4.

## MCMP: Compare two blocks of bytes

**MCMP: R3, prefix 1110100001, sub-opcode 0000111; operands reg5, reg5, reg5; 4 bytes**

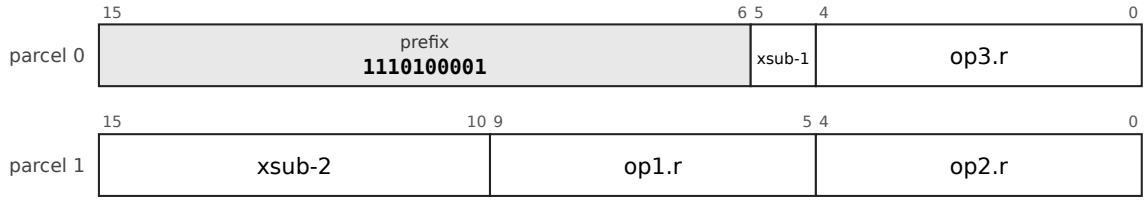



**Mnemonics:** `MCMP`

### Syntax

```
MCMP Rd, Rs, Rn
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
while Rn ≠ 0:
    x ← M8[Rd] ; y ← M8[Rs]
    if x ≠ y then  flags ← flags of (x - y) ; stop      ; pointers left AT the differing byte
    Rd ← Rd + 1 ; Rs ← Rs + 1 ; Rn ← Rn - 1
if equal:  N = 0, Z = 1, V = 0, C = 0
```

**Flags** From the first differing byte as for `CMP` of the two bytes (`C` set if `x < y` unsigned, `N` and `V` from the signed difference); for equal blocks `N` = 0, `Z` = 1, `V` = 0, `C` = 0.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

When the instruction stops at a difference, `Rd` and `Rs` point at the differing bytes and `Rn` is the count remaining, so the position of the difference is in the registers.

**Why it is here.** Seven of the VAX's ten character-string instructions are present (`MOVC3`, `CMPC3`, `LOCC` and `SKPC` among them); the rejected ones are `EDITPC` and packed decimal. `MCMP` is the `CMPC3` equivalent.

**Cf.** 6.4.

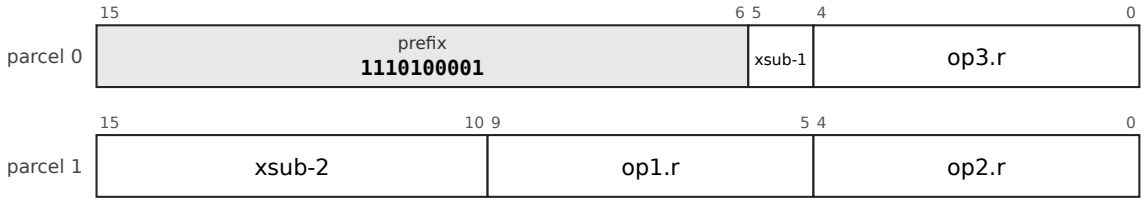
## MSCN: Scan a block for a byte, or for a byte not equal to it

**Mnemonics:** `MSCN`, `MSCNN`

### Syntax

```
MSCN Rd, Rv, Rn
MSCNN Rd, Rv, Rn
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
while Rn ≠ 0:
    hit ← (M8[Rd] = Rv[7:0])            ; MSCNN: hit ← (M8[Rd] ≠ Rv[7:0])
    if hit then  Z = 1 ; stop             ; Rd points AT the byte found
    Rd ← Rd + 1 ; Rn ← Rn - 1
not found: Z = 0
```

**Flags All four flags are written**: `Z` = 1 if the byte was found, else 0; `N`, `V` and `C` are cleared.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

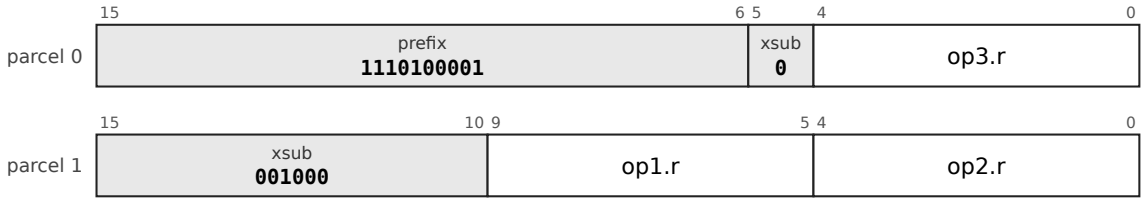
`MSCN` is `memchr`; with an explicit count it gives `strnlen`, and composing it with `MCPY` gives a bounded `strcpy` in two instructions. There are no NUL-terminated string instructions.

**Why it is here.** `MSCN` is `LOCC` and `memchr`; `MSCNN` fills the `SKPC` gap (the first byte that is *not* equal). Both cost almost no hardware. With an explicit count `MSCN` gives `strnlen`, and with `MCPY` a bounded `strcpy` in two instructions. NUL-terminated string instructions (the System/z `MVST`, `SRST`, `CLST`) are rejected, because they encode an unbounded read, the largest single source of memory-safety failures in C (9.17).

**Cf.** 6.4.

### *MSCN*

**MSCN: R3, prefix 1110100001, sub-opcode 0001000; operands reg5, reg5, reg5; 4 bytes**

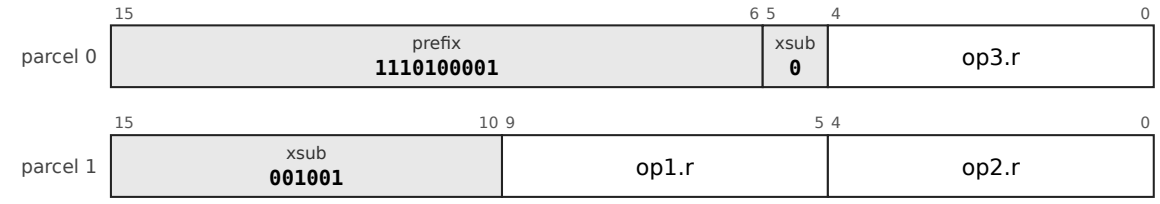



```
MSCN Rd, Rv, Rn
```

Operation, flags and exceptions as for the entry MSCN above; 1 layout form, each a chart of its own (above).

### *MSCNN*

**MSCNN: R3, prefix 1110100001, sub-opcode 0001001; operands reg5, reg5, reg5; 4 bytes**

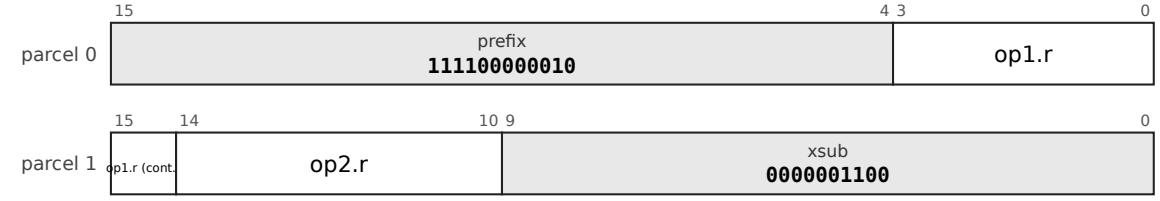



```
MSCNN Rd, Rv, Rn
```

Operation, flags and exceptions as for the entry MSCN above; 1 layout form, each a chart of its own (above).

## MVAL: Validate a block as UTF-8

**MVAL: R2, prefix 111100000010, sub-opcode 0000001100; operands reg5, reg5; 4 bytes**

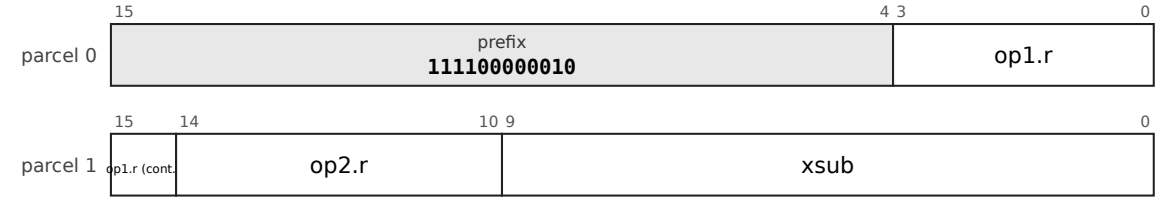



**Mnemonics:** `MVAL`

### Syntax

```
MVAL Rd, Rn
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
scan Rn bytes at Rd; stop at the first byte that cannot be part of a valid UTF-8 sequence (the rules of
UTF8DEC)
```

**Flags** On completion `Rn` is 0 and the buffer is valid; otherwise `Rd` points at the offending sequence and `Rn` counts the bytes remaining (details in the instruction-set text, section 4.7.1).
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
The `UTF8DEC` comparators wired to the block sequencer.

**Why it is here.** The UTF-8 validity comparators already built for `UTF8DEC`, wired to the streaming sequencer already built for `MCPY`, so it costs almost no hardware. The alternative, a SWAR loop (SIMD within a register), runs at about two to three cycles a byte and is respectable, which is why the justification is cost and not throughput.
**Cf.** 6.4.

## 9.12 Floating point

**9.12.1** Floating point uses the general registers: a `binary64` value fills one register, and the instruction, not the register, decides how its bits are read. *(D)*

**9.12.2** `FADD`, `FSUB`, `FMUL` and `FCMP` are two-byte instructions; `FDIV`, `FSQRT`, the fused multiply-add family, `FABS`, `FMIN`, `FMAX`, `FCLASS`, the roundings and the `binary32` forms are in the floating-point escape; `binary16` has seventeen operations and conversions. *(D)*

**9.12.3** The five IEEE flags (invalid, divide by zero, overflow, underflow, inexact) accumulate in `FPSR`; no floating-point operation traps; `FSFLAGS` reads and clears them. *(D)*

**9.12.4** Every NaN that an operation creates is the canonical quiet NaN. *(D)*

**9.12.5** There are no rounding-mode or precision mode bits: the precision is in the opcode, and so is the rounding mode wherever it is not nearest-even. *(D)*

**9.12.6** There are no transcendental instructions. *(D)*

Open: how a floating-point comparison sets flags that the signed branches can use (J.9.9), and directed rounding and the remaining half-precision operations (J.9.10).

*Commentary.* **Precedent.** The PDP-11's FP11 had `AC0` to `AC5` and mode bits for single and double precision; the VAX had F, D, G and H formats. **Trade-offs.** An integer operation applied to a register holding a `binary64` destroys it, with no trap and no tag; that is intended and the ABI states it. A separate register file would cost a second set of read ports and a floating-point context; on a small implementation the register file is among the largest blocks. **Improvements.** The fused multiply-add is the largest single addition to the datapath (about 9,000 transistors) and the least negotiable; the IEEE exception flags are recorded without traps, which is what C99 `<fenv.h>` needs. **Benefits.** No transfer instructions between files and a single calling convention. **Modern comparables.** RISC-V Zfinx and Zdinx use the integer register file for the same reason; AArch64 and x86 have separate vector and floating-point files. **Use.** `FCLASS` begins every library maths function; `FMA` is the building block of accurate polynomial evaluation. **Impact.** *Kernel:* There is no separate floating-point state to save at a context switch beyond the registers already saved, and the sticky flags go with the status word. *Compiler:* One calling convention for integers and floats, fused multiply-add contraction, and no rounding-mode state for the compiler to track. *Hardware:* No second register file (one of the largest blocks of a small floating-point unit); the floating-point unit with fused multiply-add is about 34,000 transistors.

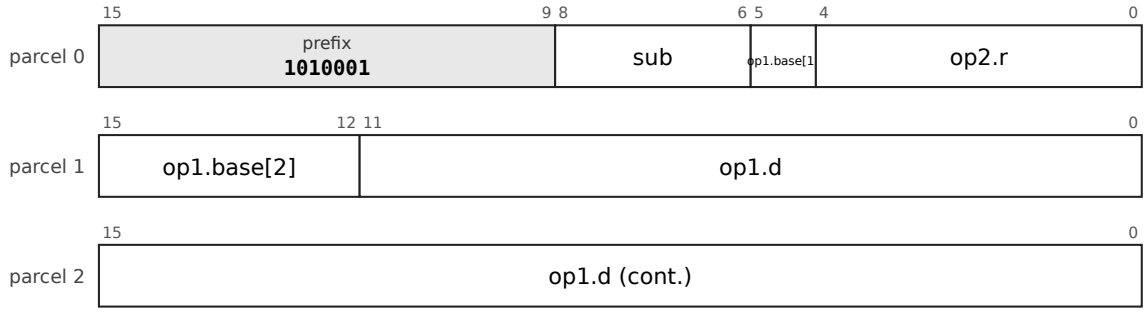
### FADD: Floating add, subtract and multiply, binary64

**Mnemonics:** `FADD`, `FSUB`, `FMUL` (aliases: `FADDD`, `FSUBD`, `FMULD`)

#### Syntax

```
FADD rm, Rd
FSUB rm, Rd
FMUL rm, Rd
```

### Encoding




The chart is the leaf `LDF-B28-2`, LDF.B28, prefix `1010001`. `op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `op2` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `LDF-B28-2`, `LDF-X22-2`, `LDF-PI12-2`, `LDF-PD12-2`, `LDF-PC32-2`, `LDF-A32-2`, `LDFI-I32-1`, `DPF-D28-2`, `RRF-2`, `RRF5-1`, `LDPC12H-d-1`, `LDPC12H-d-2`.

### Operation

```
Rd ← Rd + rm   (FADD)
Rd ← Rd - rm   (FSUB)
Rd ← Rd × rm   (FMUL)       ; IEEE 754 binary64, round to nearest even
```

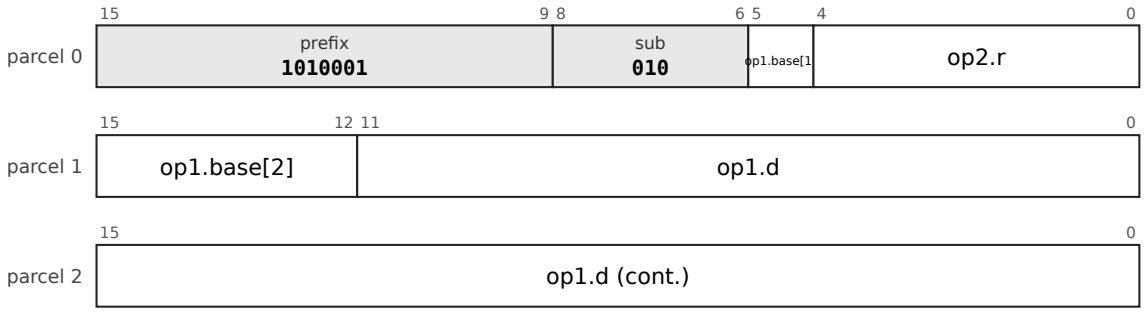
**Flags** None written. The five IEEE exception flags are ORed into `FPSR`; no instruction traps.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires. Floating-point exceptions never trap.

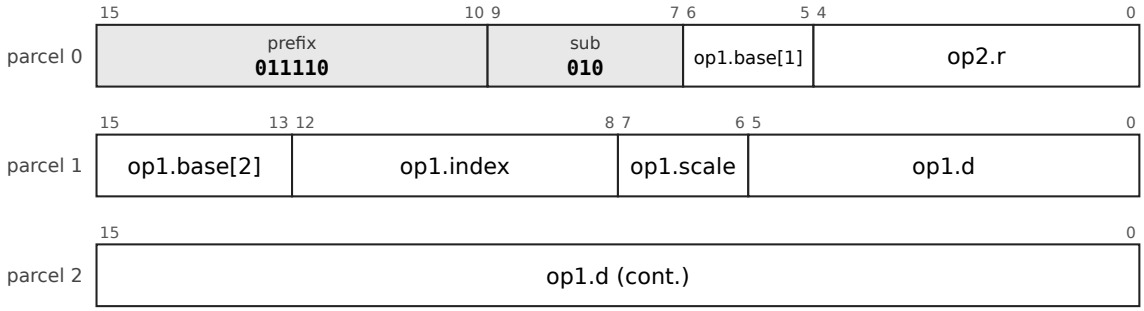
Floating-point values live in the general registers; there is no separate register file and no mode bit. A NaN result is the canonical quiet NaN `0x7FF8000000000000`. The alternative spellings `FADDD`, `FSUBD`, `FMULD` are aliases.

### *FADD*

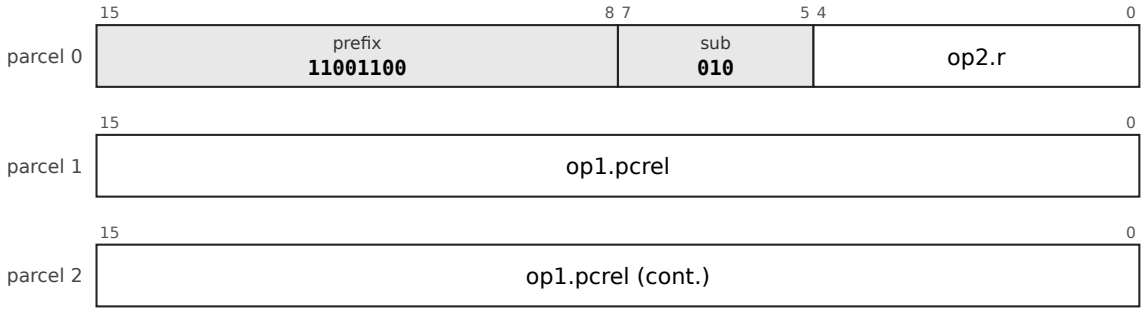
**FADD: LDF.B28, prefix 1010001, sub-opcode 010; operands b5d28s-x1, reg5; 6 bytes**



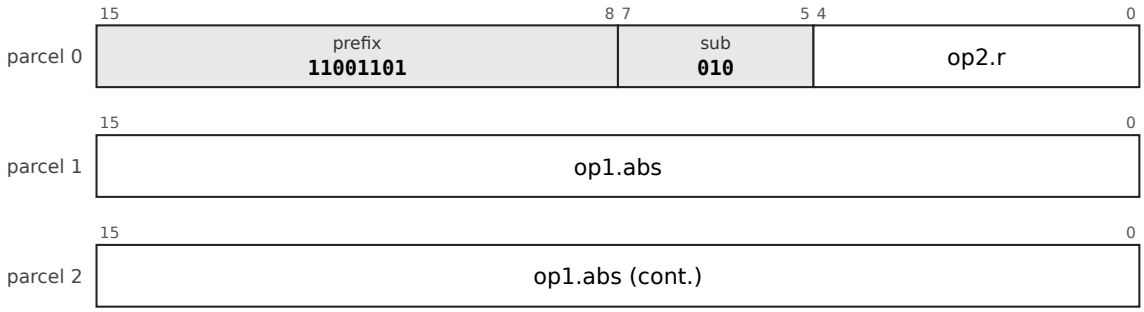

**FADD: LDF.X22, prefix 011110, sub-opcode 010; operands x5-5d22s, reg5; 6 bytes**



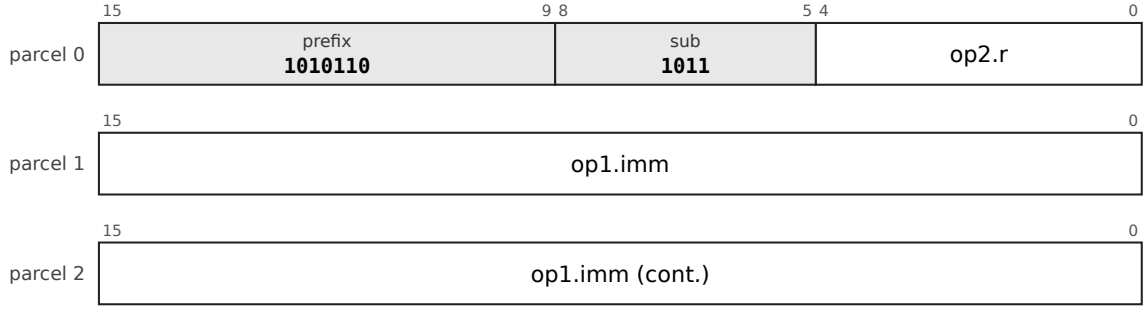

**FADD: LDF.PC32, prefix 11001100, sub-opcode 010; operands pc32-u1, reg5; 6 bytes**



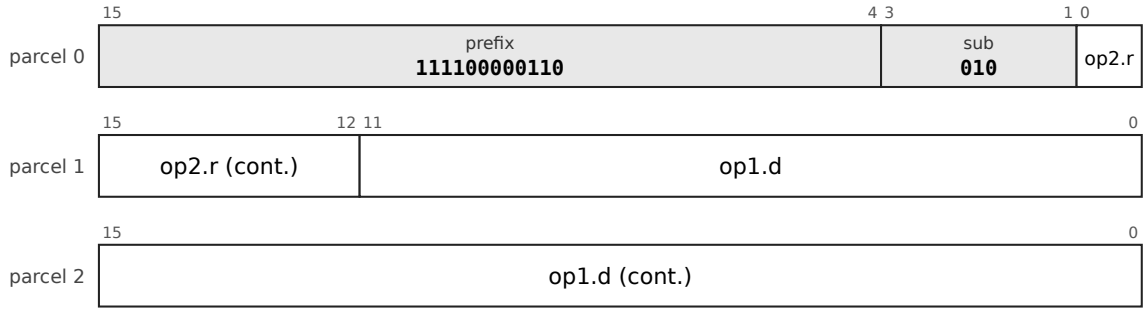

**FADD: LDF.A32, prefix 11001101, sub-opcode 010; operands abs32, reg5; 6 bytes**



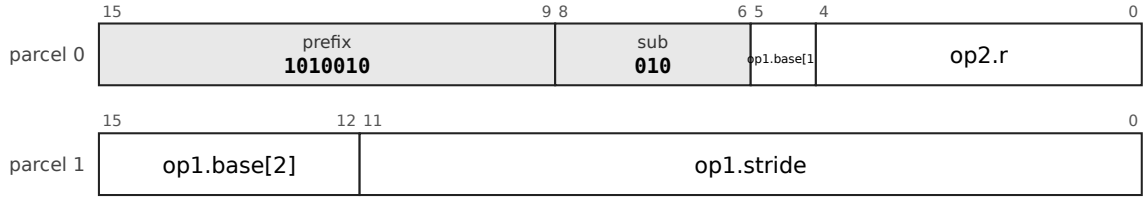
**FADD: LDFI.I32, prefix 1010110, sub-opcode 1011; operands imm32s, reg5; 6 bytes**



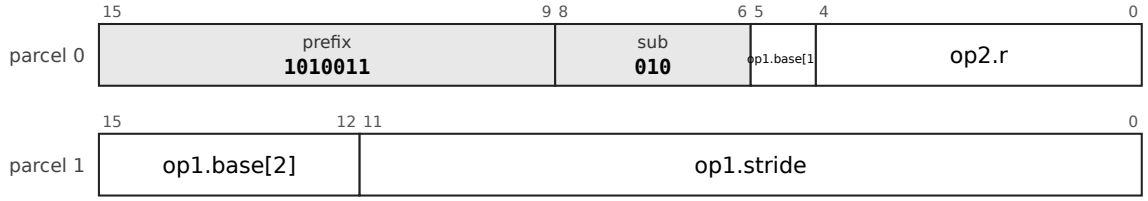

**FADD: DPF.D28, prefix 111100000110, sub-opcode 010; operands dpd28s-x1, reg5; 6 bytes**



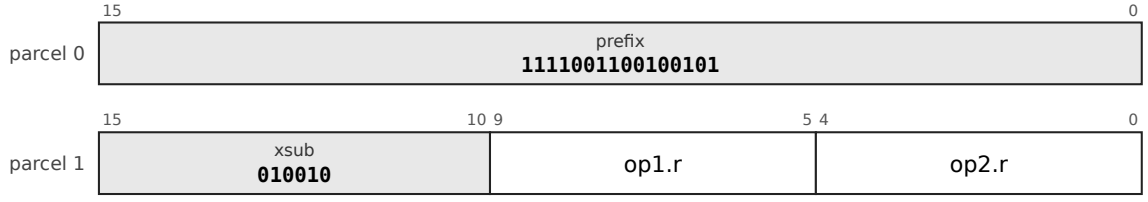

**FADD: LDF.PI12, prefix 1010010, sub-opcode 010; operands pi5-s12, reg5; 4 bytes**



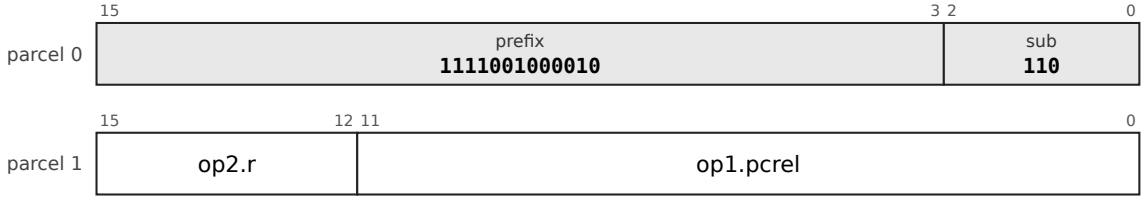

**FADD: LDF.PD12, prefix 1010011, sub-opcode 010; operands pd5-s12, reg5; 4 bytes**



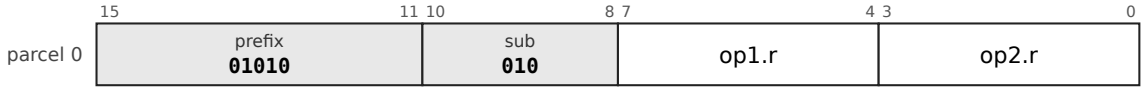

**FADD: RRF5, prefix 1111001100100101, sub-opcode 010010; operands reg5, reg5; 4 bytes**



**FADD: LDPC12H/d, prefix 1111001000010, sub-opcode 110; operands pc12-u1, reg4; 4 bytes**




**FADD: RRF, prefix 01010, sub-opcode 010; operands reg4, reg4; 2 bytes**

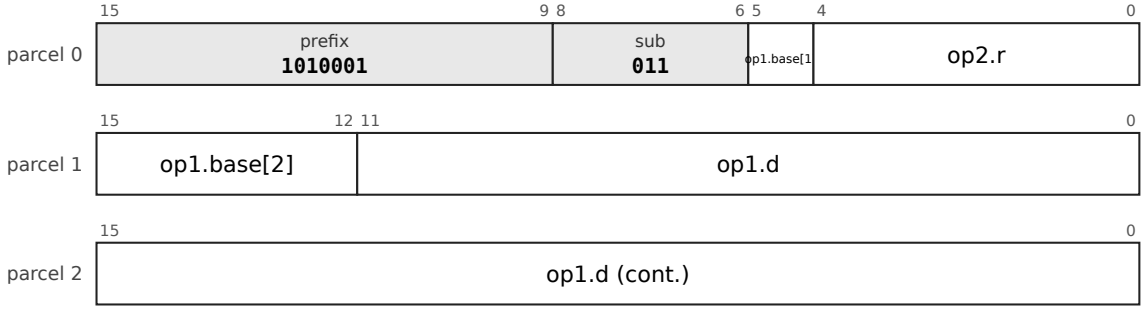



```
FADD rm, Rd
```

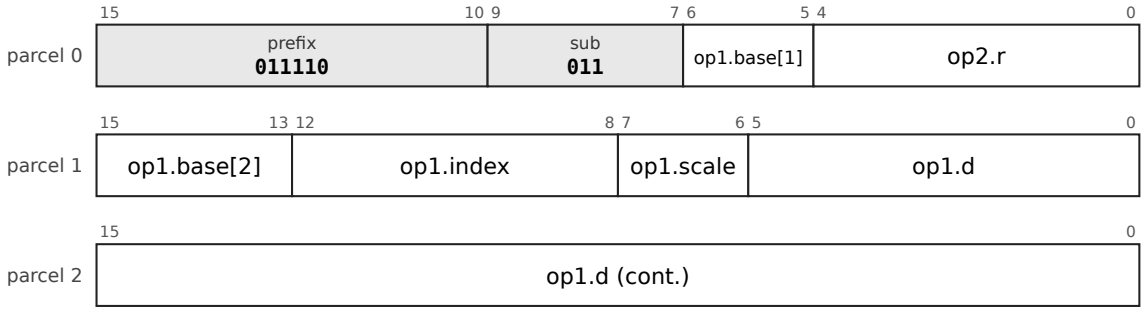
Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

***FSUB***

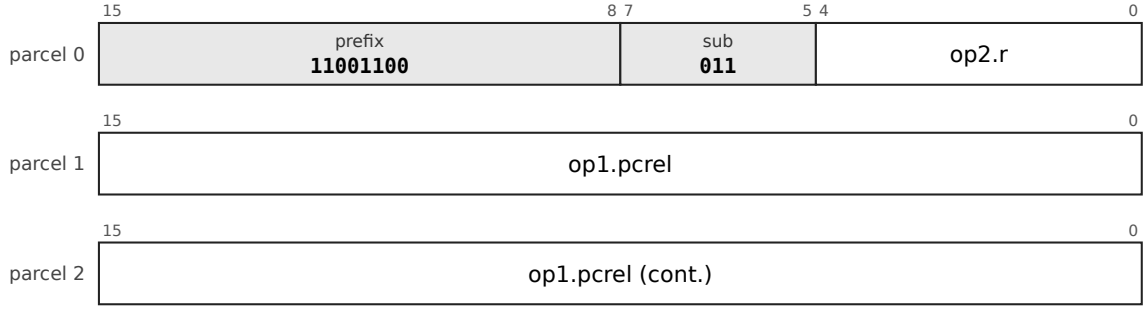
**FSUB: LDF.B28, prefix 1010001, sub-opcode 011; operands b5d28s-x1, reg5; 6 bytes**



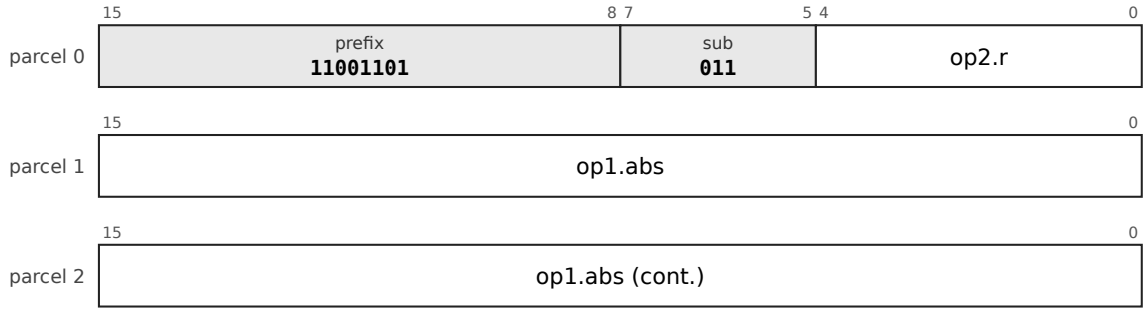

**FSUB: LDF.X22, prefix 011110, sub-opcode 011; operands x5-5d22s, reg5; 6 bytes**



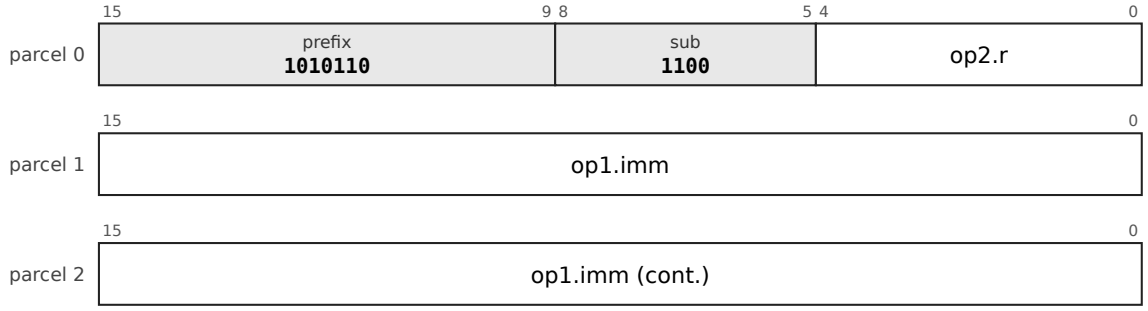
**FSUB: LDF.PC32, prefix 11001100, sub-opcode 011; operands pc32-u1, reg5; 6 bytes**



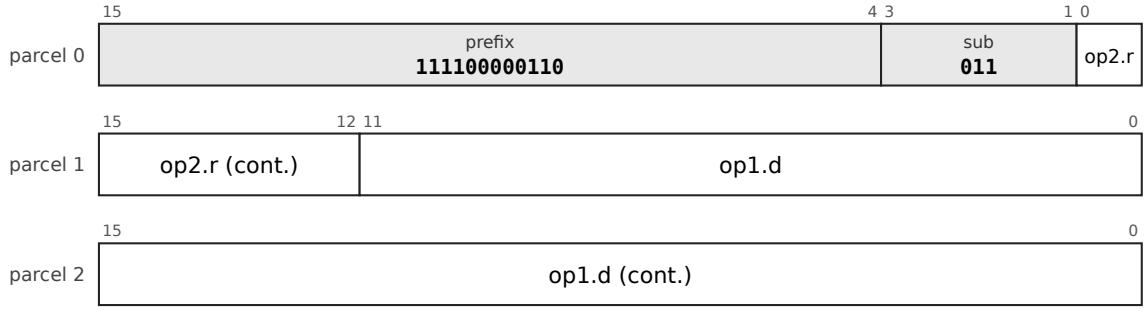

**FSUB: LDF.A32, prefix 11001101, sub-opcode 011; operands abs32, reg5; 6 bytes**



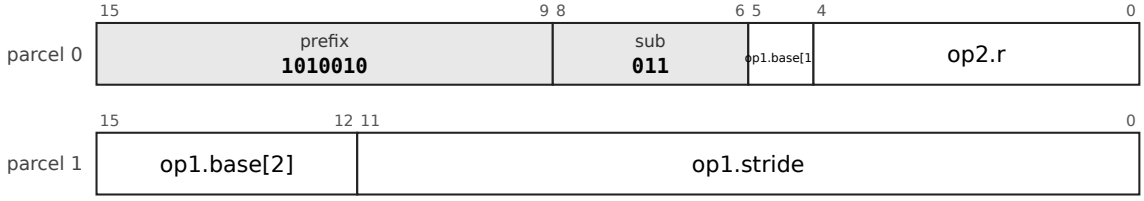

**FSUB: LDFI.I32, prefix 1010110, sub-opcode 1100; operands imm32s, reg5; 6 bytes**



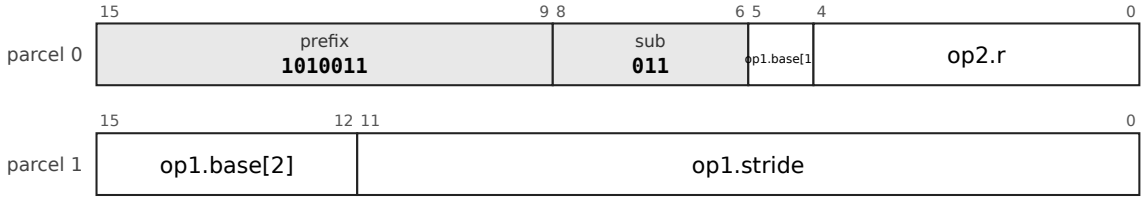

**FSUB: DPF.D28, prefix 111100000110, sub-opcode 011; operands dpd28s-x1, reg5; 6 bytes**



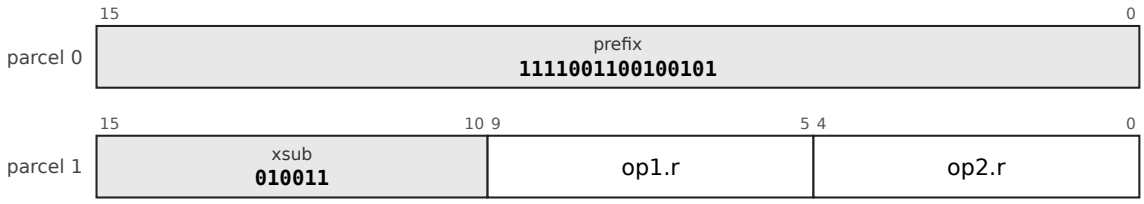
**FSUB: LDF.PI12, prefix 1010010, sub-opcode 011; operands pi5-s12, reg5; 4 bytes**



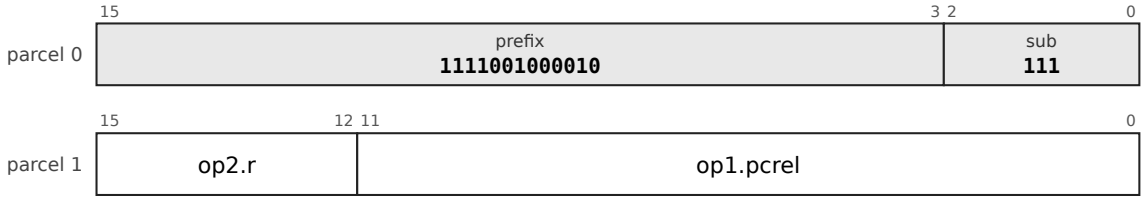

**FSUB: LDF.PD12, prefix 1010011, sub-opcode 011; operands pd5-s12, reg5; 4 bytes**



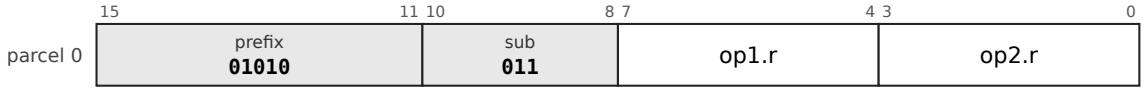

**FSUB: RRF5, prefix 1111001100100101, sub-opcode 010011; operands reg5, reg5; 4 bytes**




**FSUB: LDPC12H/d, prefix 1111001000010, sub-opcode 111; operands pc12-u1, reg4; 4 bytes**




**FSUB: RRF, prefix 01010, sub-opcode 011; operands reg4, reg4; 2 bytes**

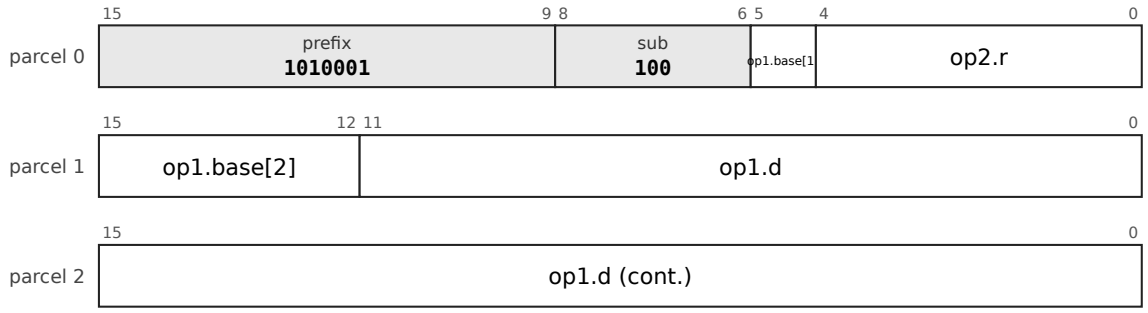



```
FSUB rm, Rd
```

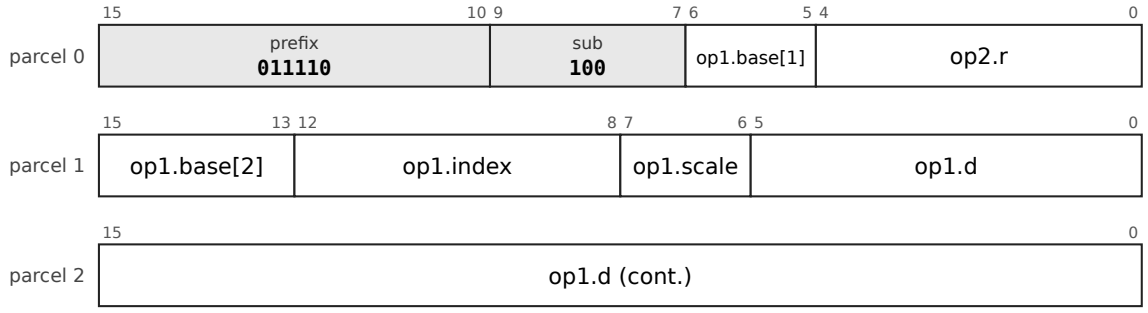
Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

### *FMUL*

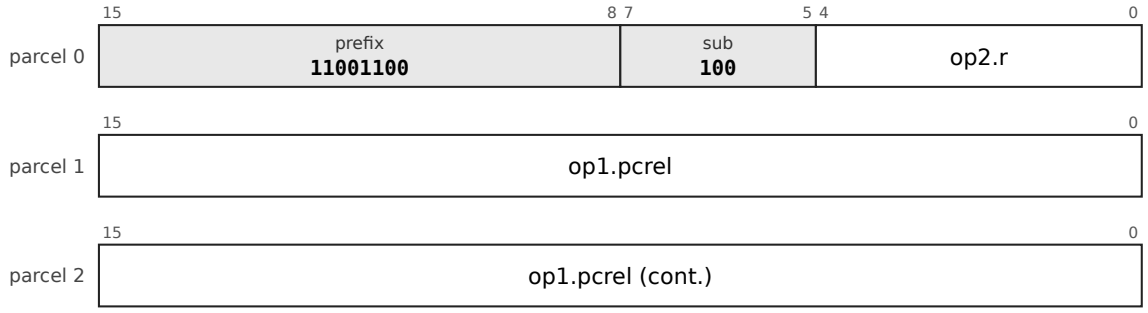
**FMUL: LDF.B28, prefix 1010001, sub-opcode 100; operands b5d28s-x1, reg5; 6 bytes**



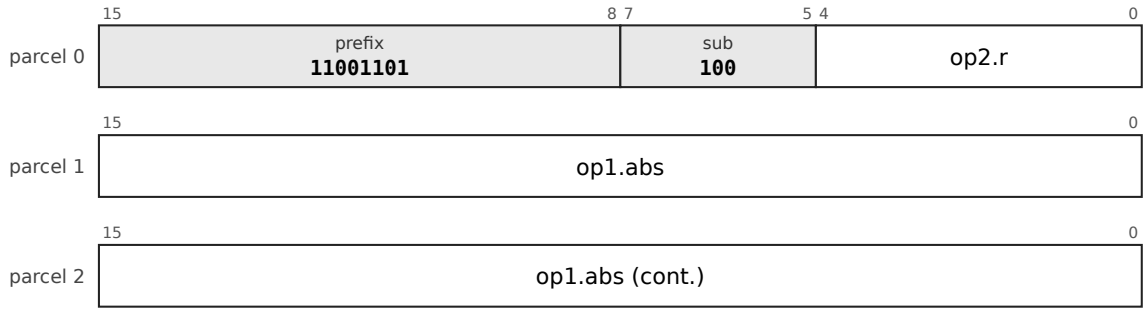

**FMUL: LDF.X22, prefix 011110, sub-opcode 100; operands x5-5d22s, reg5; 6 bytes**



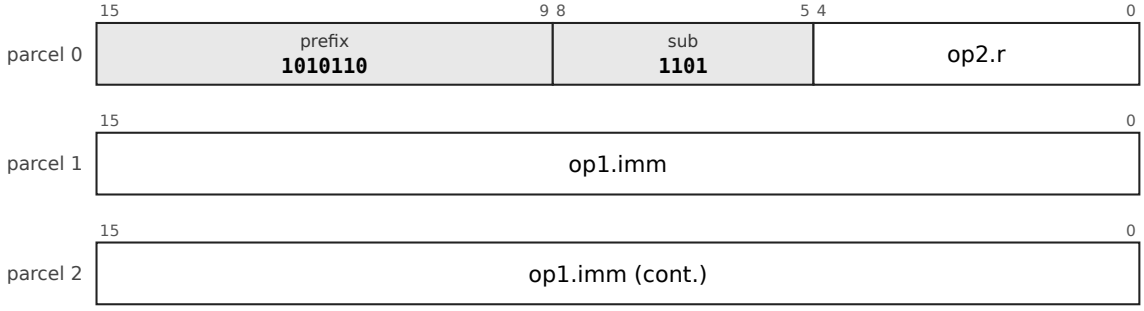

**FMUL: LDF.PC32, prefix 11001100, sub-opcode 100; operands pc32-u1, reg5; 6 bytes**



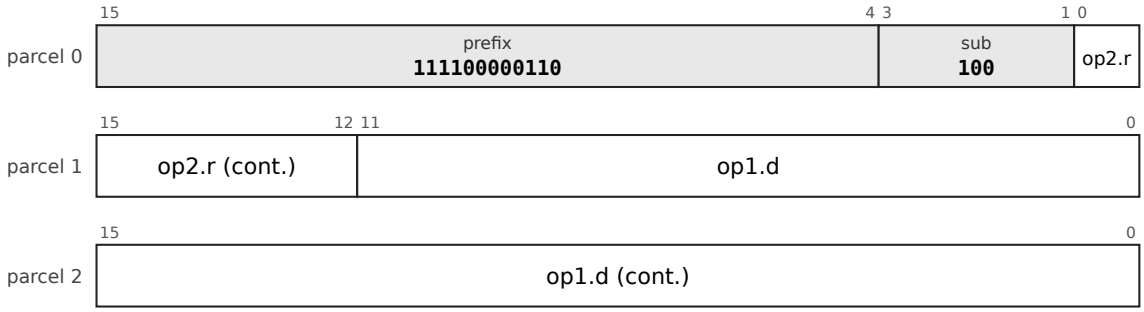

**FMUL: LDF.A32, prefix 11001101, sub-opcode 100; operands abs32, reg5; 6 bytes**



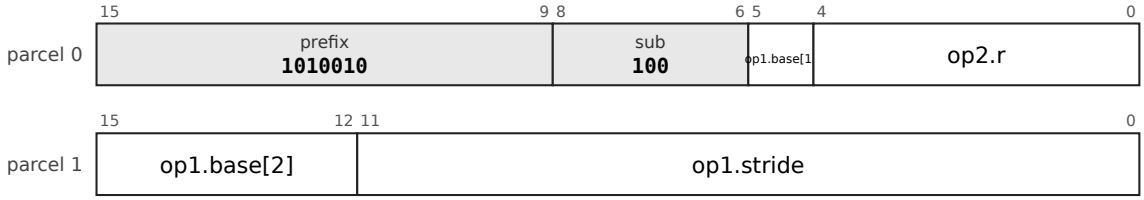
**FMUL: LDFI.I32, prefix 1010110, sub-opcode 1101; operands imm32s, reg5; 6 bytes**



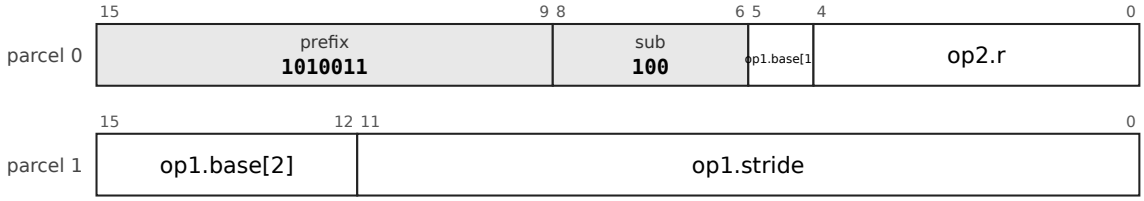

**FMUL: DPF.D28, prefix 111100000110, sub-opcode 100; operands dpd28s-x1, reg5; 6 bytes**



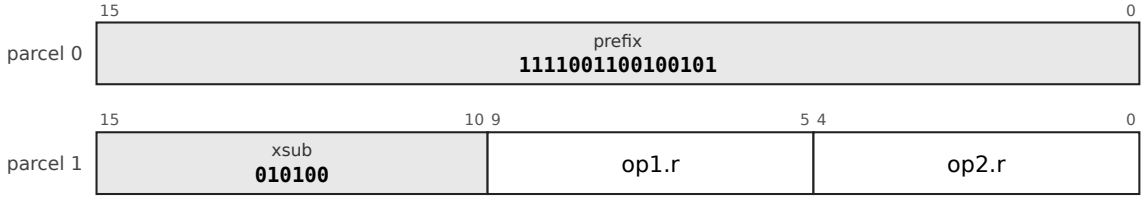

**FMUL: LDF.PI12, prefix 1010010, sub-opcode 100; operands pi5-s12, reg5; 4 bytes**



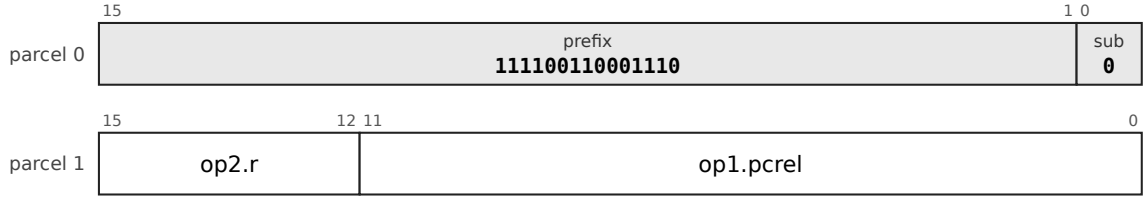

**FMUL: LDF.PD12, prefix 1010011, sub-opcode 100; operands pd5-s12, reg5; 4 bytes**



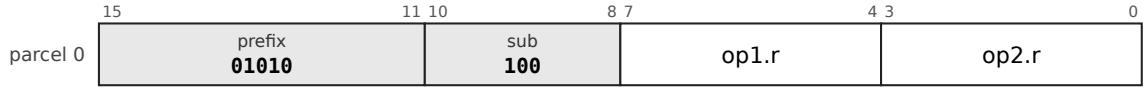

**FMUL: RRF5, prefix 1111001100100101, sub-opcode 010100; operands reg5, reg5; 4 bytes**



**FMUL: LDPC12H/d, prefix 111100110001110, sub-opcode 0; operands pc12-u1, reg4; 4 bytes**




**FMUL: RRF, prefix 01010, sub-opcode 100; operands reg4, reg4; 2 bytes**




```
FMUL rm, Rd
```

Operation, flags and exceptions as for the entry FADD above; 11 layout forms, each a chart of its own (above).

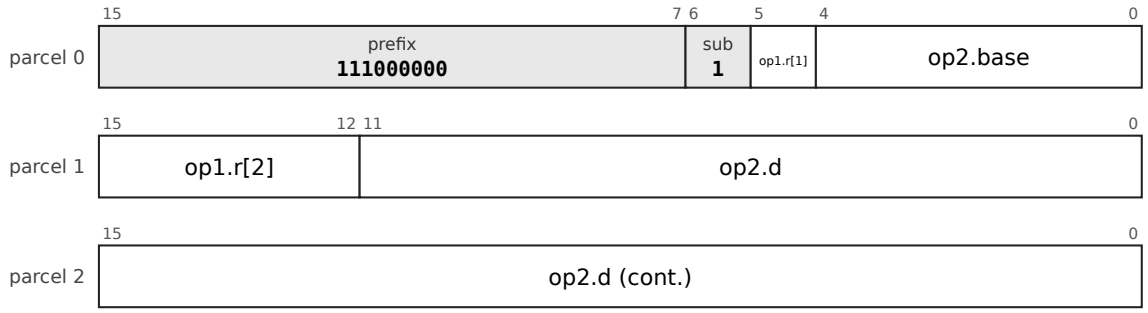
***FADDD***
An alias of FADD: the assembler writes it as that word.

***FSUBD***
An alias of FSUB: the assembler writes it as that word.

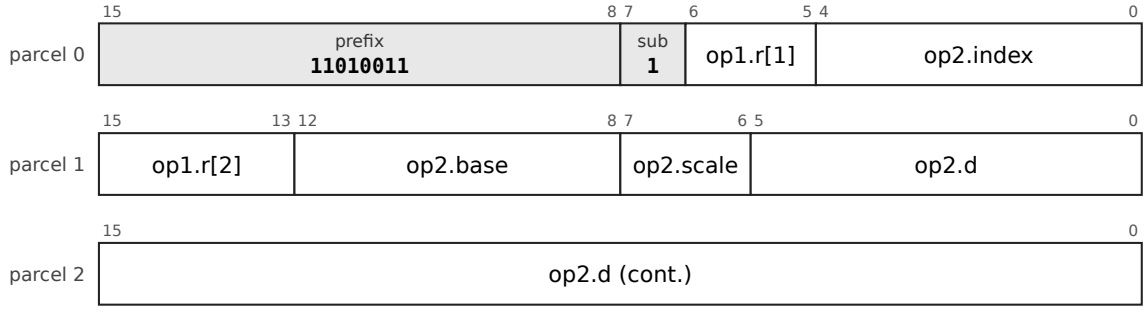
***FMULD***
An alias of FMUL: the assembler writes it as that word.

## FCMP: Floating compare, binary64

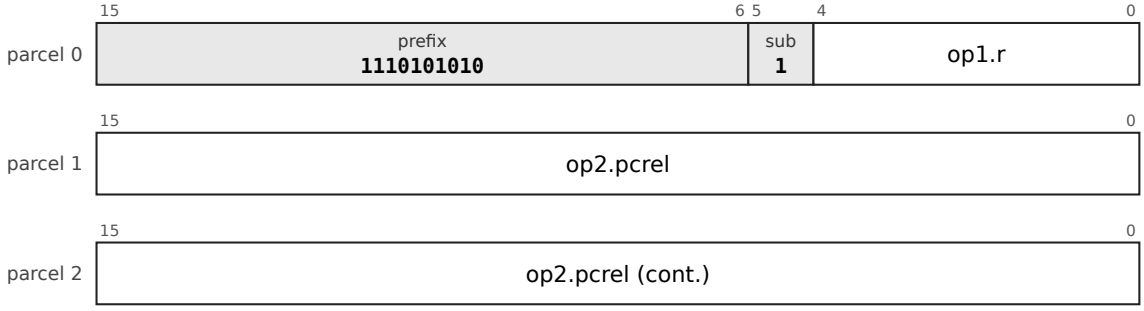
**FCMP: CMF.B28, prefix 111000000, sub-opcode 1; operands reg5, b5d28s-x1; 6 bytes**



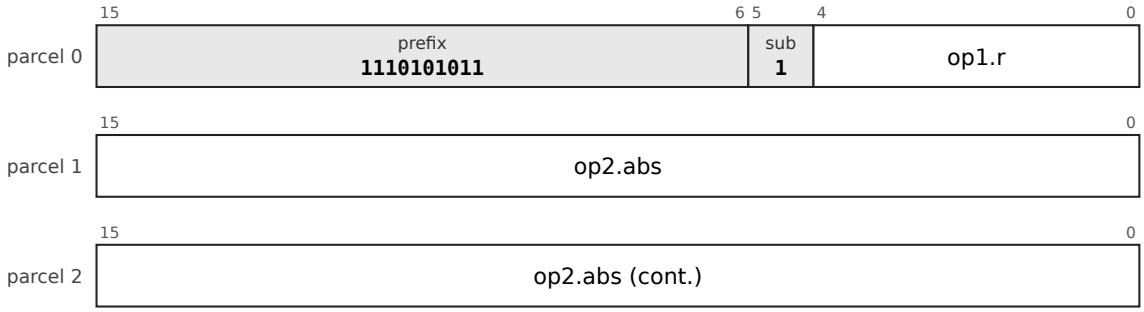

**FCMP: CMF.X22, prefix 11010011, sub-opcode 1; operands reg5, x5-5d22s; 6 bytes**



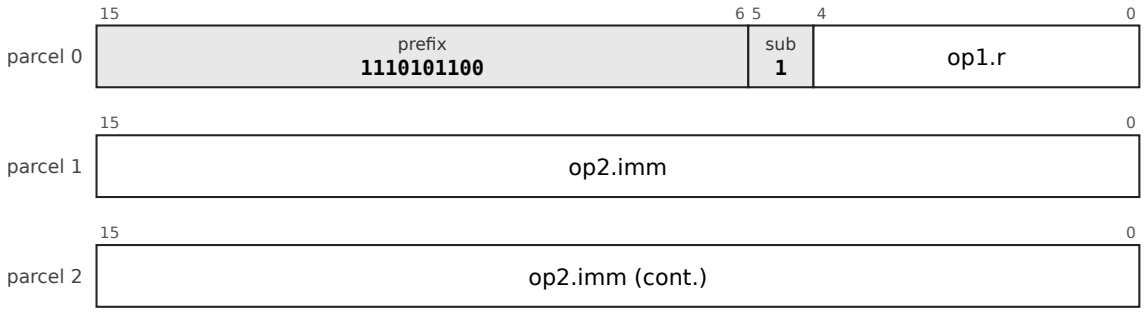
**FCMP: CMF.PC32, prefix 1110101010, sub-opcode 1; operands reg5, pc32-u1; 6 bytes**



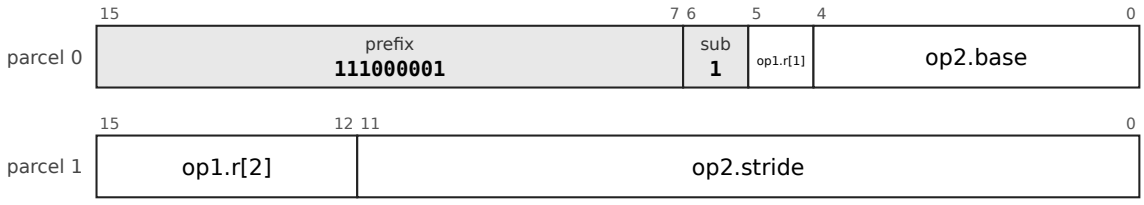

**FCMP: CMF.A32, prefix 1110101011, sub-opcode 1; operands reg5, abs32; 6 bytes**



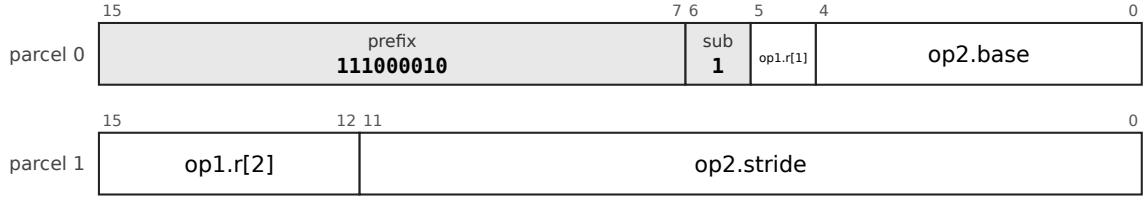

**FCMP: CMF.I32, prefix 1110101100, sub-opcode 1; operands reg5, imm32s; 6 bytes**



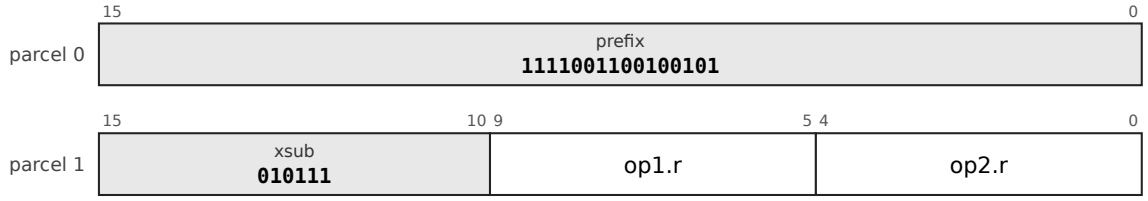

**FCMP: CMF.PI12, prefix 111000001, sub-opcode 1; operands reg5, pi5-s12; 4 bytes**



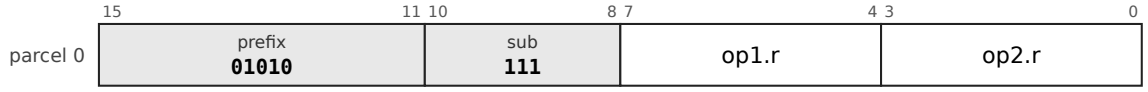
**FCMP: CMF.PD12, prefix 111000010, sub-opcode 1; operands reg5, pd5-s12; 4 bytes**




**FCMP: RRF5, prefix 1111001100100101, sub-opcode 010111; operands reg5, reg5; 4 bytes**




**FCMP: RRF, prefix 01010, sub-opcode 111; operands reg4, reg4; 2 bytes**

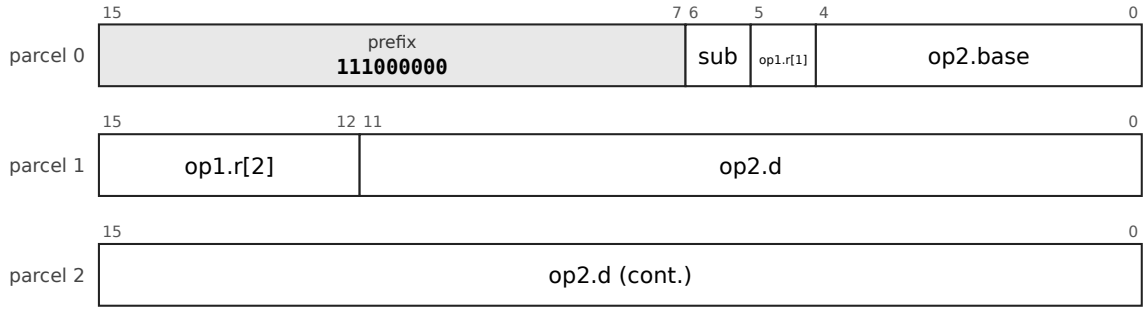



**Mnemonics:** `FCMP` (aliases: `FCMPD`)

### Syntax

```
FCMP Rd, rm        ; register first
```

### Encoding




The chart is the leaf `CMF-B28-1`, CMF.B28, prefix `111000000`. `op1` is `reg5`: register, 5 bits. `op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes. `sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `CMF-B28-1`, `CMF-X22-1`, `CMF-PI12-1`, `CMF-PD12-1`, `CMF-PC32-1`, `CMF-A32-1`, `CMF-I32-1`, `RRF-2`, `RRF5-1`.

### Operation

```
N ← Rd < rm ; Z ← Rd = rm ; V ← unordered ; C ← 0
```

**Flags** `N` set if `Rd < rm`, `Z` if equal, `V` if the operands are unordered (either is a NaN), `C` cleared. A signalling NaN raises `NV` in `FPSR`.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

The signed branches do not treat the unordered case cleanly (`BLT` after an unordered compare takes `N xor V` = 1). Open: how floating-point comparisons set flags that the signed branches can use (J.9.9).

**`FCMPD`** is an alias of `FCMP`: the assembler writes it as that word.

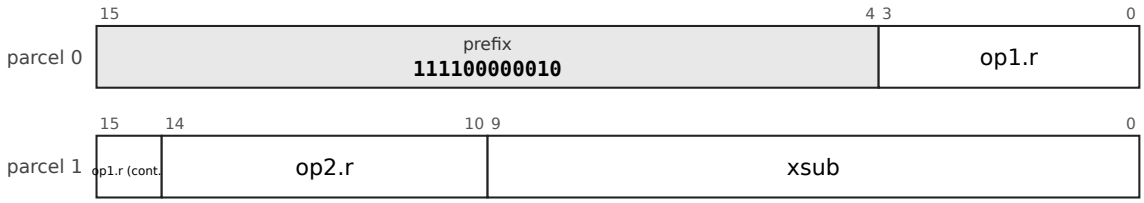
## FDIV: Floating divide and square root, binary64

**Mnemonics:** `FDIV`, `FSQRT` (aliases: `FDIVD`, `FSQRTD`)

### Syntax

```
FDIV Rs, Rd
FSQRT Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
FDIV : Rd ← Rd ÷ Rs
FSQRT: Rd ← sqrt(Rs)
```

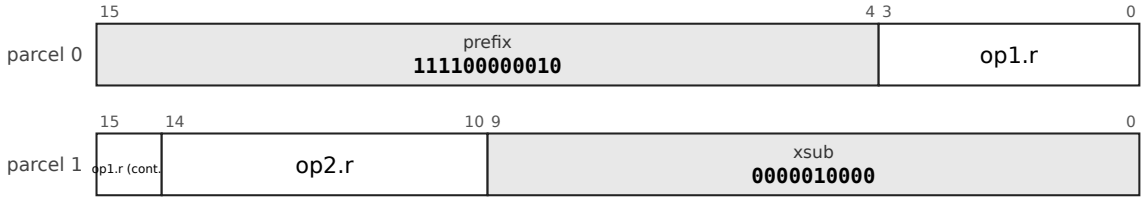
**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

**Exceptions** None.

`FDIV` and `FSQRT` are multi-cycle sequencer operations and rarely appear in a one-parcel form. The alternative spellings `FDIVD`, `FSQRTD` are aliases.

### *FDIV*

**FDIV: R2, prefix 111100000010, sub-opcode 0000010000; operands reg5, reg5; 4 bytes**

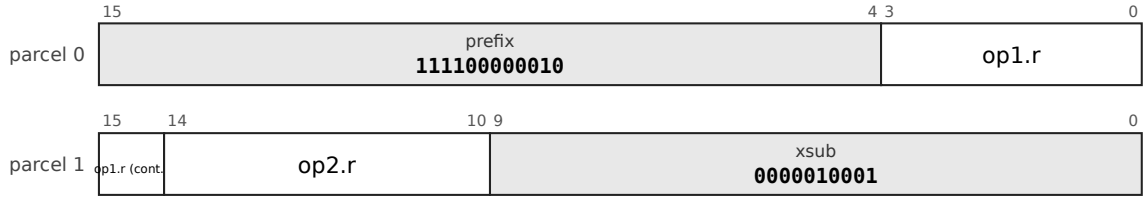



```
FDIV Rs, Rd
```

Operation, flags and exceptions as for the entry FDIV above; 1 layout form, each a chart of its own (above).

### *FSQRT*

**FSQRT: R2, prefix 111100000010, sub-opcode 0000010001; operands reg5, reg5; 4 bytes**




```
FSQRT Rs, Rd
```

Operation, flags and exceptions as for the entry FDIV above; 1 layout form, each a chart of its own (above).

### *FDIVD*

An alias of `FDIV`: the assembler writes it as that word.

### *FSQRTD*

An alias of `FSQRT`: the assembler writes it as that word.

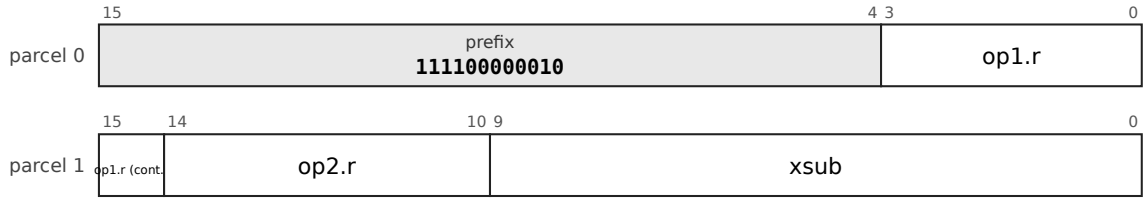
## FSINGLE: Floating arithmetic, binary32

**Mnemonics:** `FADD32`, `FSUB32`, `FMUL32`, `FDIV32`, `FSQRT32`, `FCMP32` (aliases: `FADDS`, `FSUBS`, `FMULS`, `FDIVS`, `FSQRTS`, `FCMPS`)

### Syntax

```
FADD32 Rs, Rd
FSUB32 Rs, Rd
FMUL32 Rs, Rd
FDIV32 Rs, Rd
FSQRT32 Rs, Rd
FCMP32 Rd, Rs
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
Rd ← Rd op Rs  (FADD32 FSUB32 FMUL32 FDIV32); Rd ← sqrt(Rs) (FSQRT32)
FCMP32: flags as FCMP, comparing Rd with Rs
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP32` writes `N Z V C` as `FCMP` does.

**Exceptions** None.

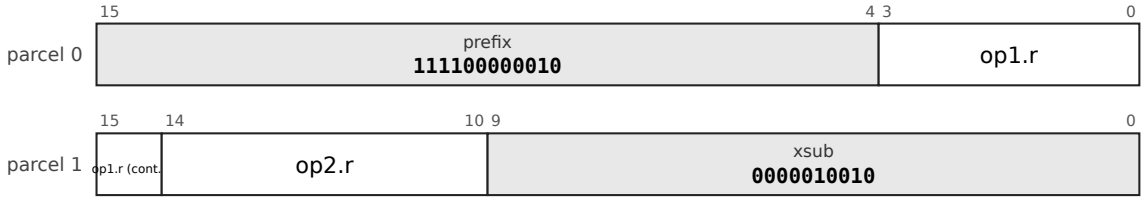
A `binary32` lives in the low 32 bits of a register, sign-extended to 64 like every 32-bit result. Results are correctly rounded to nearest even. The alternative spellings `FADDS`, `FSUBS`, ... are aliases.

**Why it is here.** `binary64` has first-class opcodes because on a 64-bit machine it is the common case; `binary32` has the same operations, held in the low 32 bits of a register and sign-extended like every other 32-bit value (see also the ABI). The naming

rule is an `F` in front of the integer mnemonic (`FADD`) and a size suffix for the other formats (`FADD16`); the older spellings (`FADDS` and so on) are aliases.

### *FADD32*

**FADD32: R2, prefix 111100000010, sub-opcode 0000010010; operands reg5, reg5; 4 bytes**

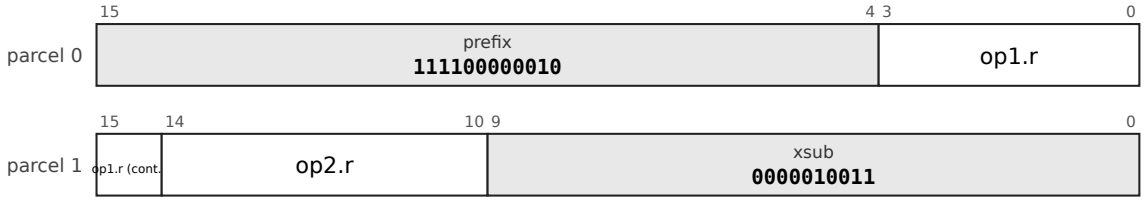



```
FADD32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

### *FSUB32*

**FSUB32: R2, prefix 111100000010, sub-opcode 0000010011; operands reg5, reg5; 4 bytes**

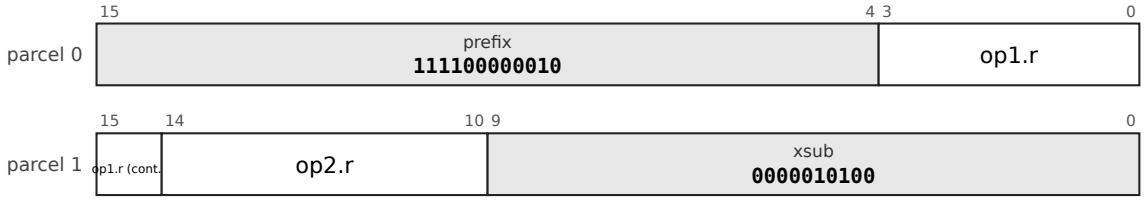



```
FSUB32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

### *FMUL32*

**FMUL32: R2, prefix 111100000010, sub-opcode 0000010100; operands reg5, reg5; 4 bytes**

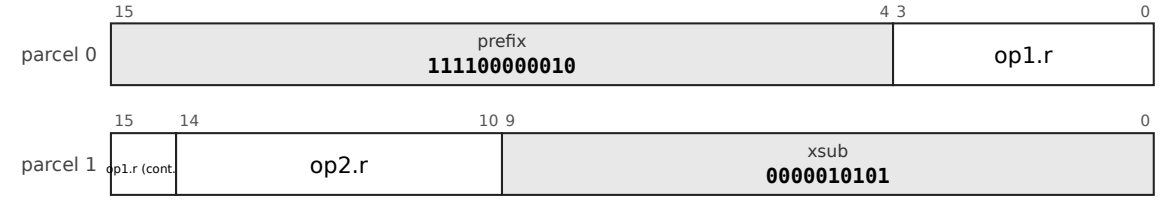



```
FMUL32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

***FDIV32***

**FDIV32: R2, prefix 111100000010, sub-opcode 0000010101; operands reg5, reg5; 4 bytes**

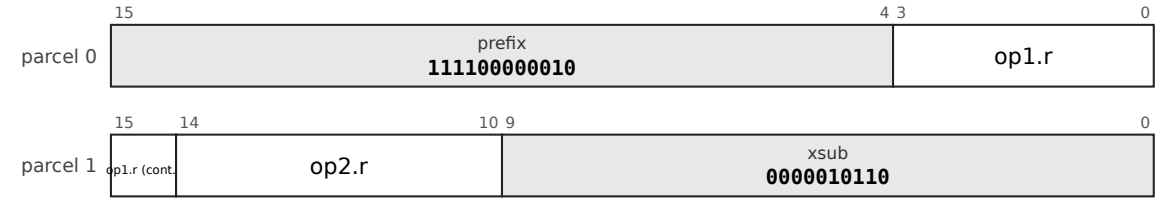



```
FDIV32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

***FSQRT32***

**FSQRT32: R2, prefix 111100000010, sub-opcode 0000010110; operands reg5, reg5; 4 bytes**

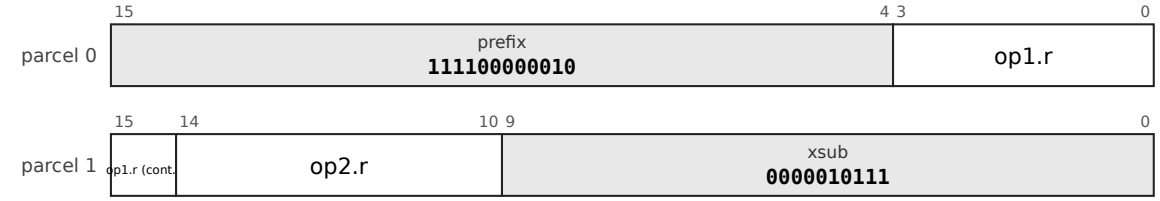



```
FSQRT32 Rs, Rd
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

***FCMP32***

**FCMP32: R2, prefix 111100000010, sub-opcode 0000010111; operands reg5, reg5; 4 bytes**




```
FCMP32 Rd, Rs
```

Operation, flags and exceptions as for the entry FSINGLE above; 1 layout form, each a chart of its own (above).

***FADDS***
An alias of `FADD32`: the assembler writes it as that word.

***FSUBS***
An alias of `FSUB32`: the assembler writes it as that word.

***FMULS***
An alias of `FMUL32`: the assembler writes it as that word.

***FDIVS***
An alias of `FDIV32`: the assembler writes it as that word.

***FSQRTS***
An alias of `FSQRT32`: the assembler writes it as that word.

***FCMPS***
An alias of `FCMP32`: the assembler writes it as that word.

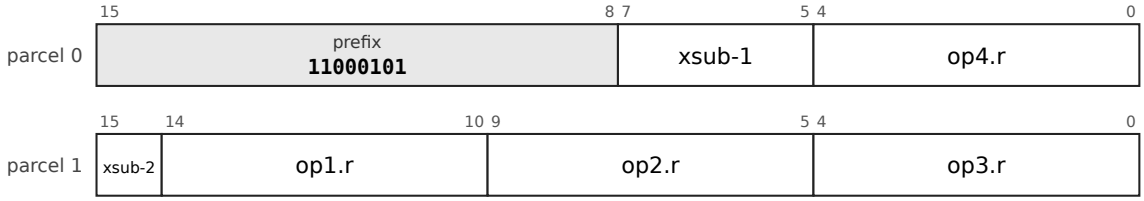
## FMADD: Fused multiply-add, binary64

**Mnemonics:** `FMADD`, `FMSUB`, `FNMADD`, `FNMSUB` (aliases: `FMADDD`, `FMSUBD`, `FNMADDD`, `FNMSUBD`)

### Syntax

```
FMADD Rs, Rt, Ru, Rd
FMSUB Rs, Rt, Ru, Rd
FNMADD Rs, Rt, Ru, Rd
FNMSUB Rs, Rt, Ru, Rd
```

### Encoding




The chart is the leaf `R4-1`, R4, prefix `11000101`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `op4` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R4-1`.

### Operation

```
FMADD : Rd ← Rs×Rt + Ru
FMSUB : Rd ← Rs×Rt - Ru
FNMADD: Rd ← -(Rs×Rt) - Ru
FNMSUB: Rd ← -(Rs×Rt) + Ru          ; each rounded once
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

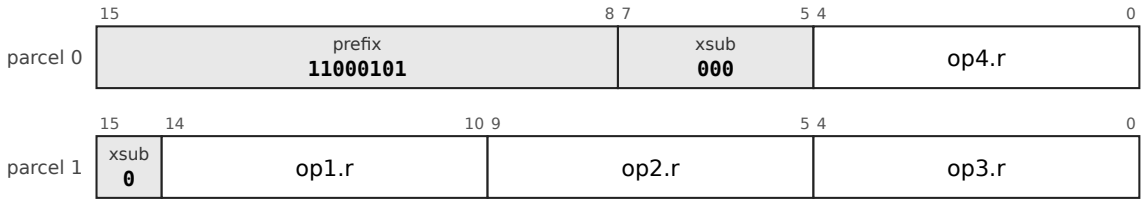
**Exceptions** None.

The alternative spellings `FMADDD`, ... are aliases.

**Why it is here.** The whole fused multiply-add family is in the instruction set because its absence would be the single worst gap in the floating-point set: one rounding instead of two, and the inner step of dot products, polynomial evaluation and every matrix kernel. It has four register operands. It replaces the VAX `POLY`, which was the best answer before fused multiply-add.

#### *FMADD*

**FMADD: R4, prefix 11000101, sub-opcode 0000; operands reg5, reg5, reg5, reg5; 4 bytes**

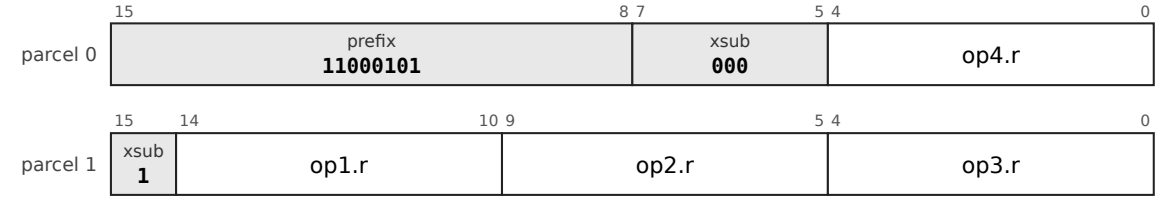



```
FMADD Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

***FMSUB***

**FMSUB: R4, prefix 11000101, sub-opcode 0001; operands reg5, reg5, reg5, reg5; 4 bytes**

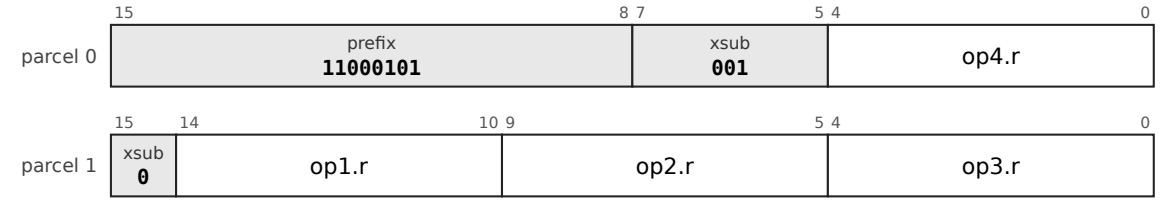



```
FMSUB Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

***FNMADD***

**FNMADD: R4, prefix 11000101, sub-opcode 0010; operands reg5, reg5, reg5, reg5; 4 bytes**

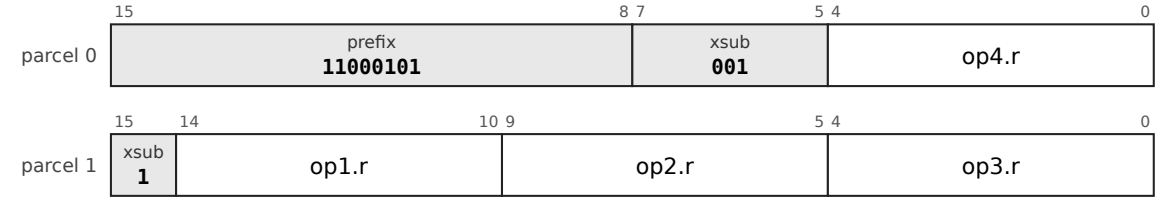



```
FNMADD Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

***FNMSUB***

**FNMSUB: R4, prefix 11000101, sub-opcode 0011; operands reg5, reg5, reg5, reg5; 4 bytes**




```
FNMSUB Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD above; 1 layout form, each a chart of its own (above).

***FMADDD***
An alias of `FMADD`: the assembler writes it as that word.

***FMSUBD***
An alias of `FMSUB`: the assembler writes it as that word.

***FNMADDD***
An alias of `FNMADD`: the assembler writes it as that word.

***FNMSUBD***
An alias of `FNMSUB`: the assembler writes it as that word.

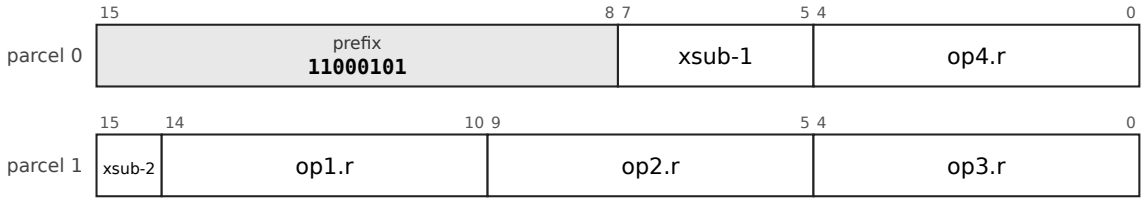
## FMADD32: Fused multiply-add, binary32

**Mnemonics:** `FMADD32`, `FMSUB32`, `FNMADD32`, `FNMSUB32` (aliases: `FMADDS`, `FMSUBS`, `FNMADDS`, `FNMSUBS`)

### Syntax

```
FMADD32 Rs, Rt, Ru, Rd
FMSUB32 ...
FNMADD32 ...
FNMSUB32 ...
```

### Encoding




The chart is the leaf `R4-1`, R4, prefix `11000101`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `op4` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R4-1`.

### Operation

```
As FMADD, in binary32, rounded once
```

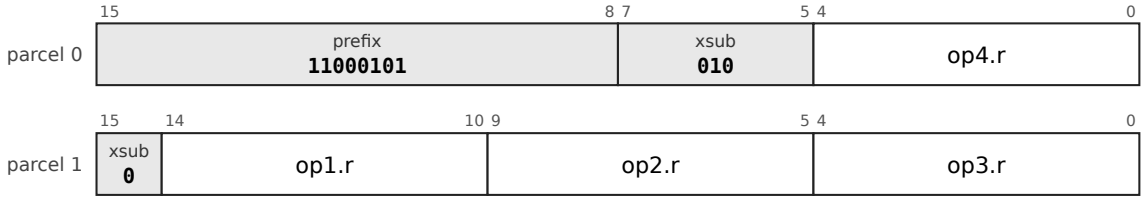
**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

**Exceptions** None.

**Why it is here.** The `binary32` forms of the fused multiply-add family.

***FMADD32***

**FMADD32: R4, prefix 11000101, sub-opcode 0100; operands reg5, reg5, reg5, reg5; 4 bytes**

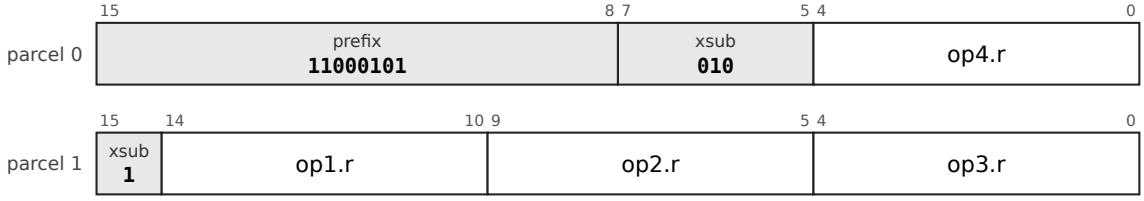



```
FMADD32 Rs, Rt, Ru, Rd
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

***FMSUB32***

**FMSUB32: R4, prefix 11000101, sub-opcode 0101; operands reg5, reg5, reg5, reg5; 4 bytes**

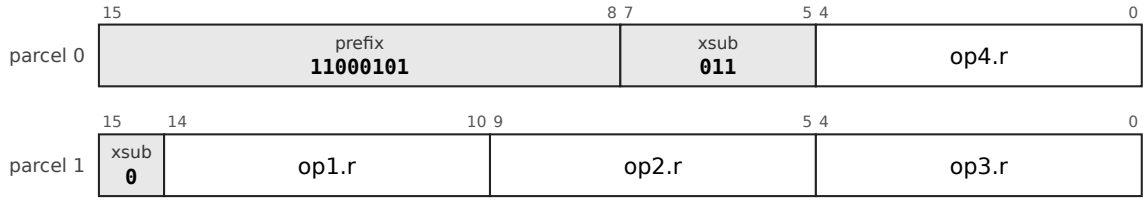



```
FMSUB32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

***FNMADD32***

**FNMADD32: R4, prefix 11000101, sub-opcode 0110; operands reg5, reg5, reg5, reg5; 4 bytes**

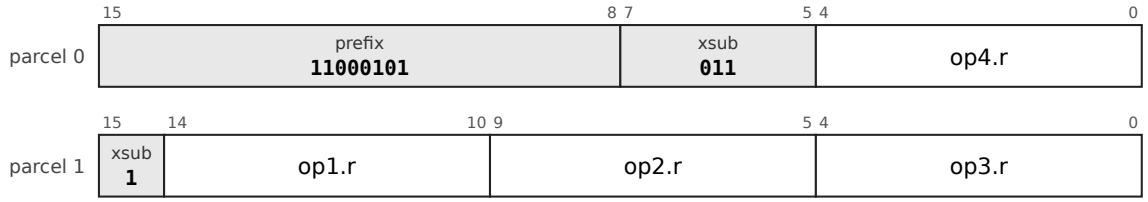



```
FNMADD32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

***FNMSUB32***

**FNMSUB32: R4, prefix 11000101, sub-opcode 0111; operands reg5, reg5, reg5, reg5; 4 bytes**




```
FNMSUB32 ...
```

Operation, flags and exceptions as for the entry FMADD32 above; 1 layout form, each a chart of its own (above).

***FMADDS***
An alias of `FMADD32`: the assembler writes it as that word.

***FMSUBS***
An alias of `FMSUB32`: the assembler writes it as that word.

***FNMADDS***
An alias of `FNMADD32`: the assembler writes it as that word.

***FNMSUBS***
An alias of `FNMSUB32`: the assembler writes it as that word.

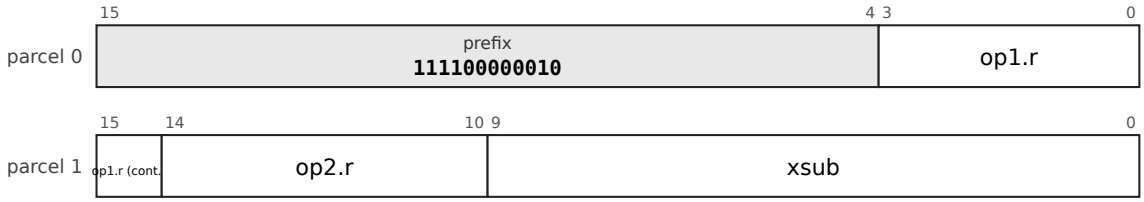
## FSIGN: Sign operations on the bits

**Mnemonics:** `FNEG`, `FABS`, `FCOPYSIGN`

### Syntax

```
FNEG Rs, Rd
FABS Rs, Rd
FCOPYSIGN Rs, Rt, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`, `R3-1`.

### Operation

```
FNEG     : Rd ← Rs with the sign bit flipped
FABS     : Rd ← Rs with the sign bit cleared
FCOPYSIGN: Rd ← magnitude of Rs, sign of Rt
```

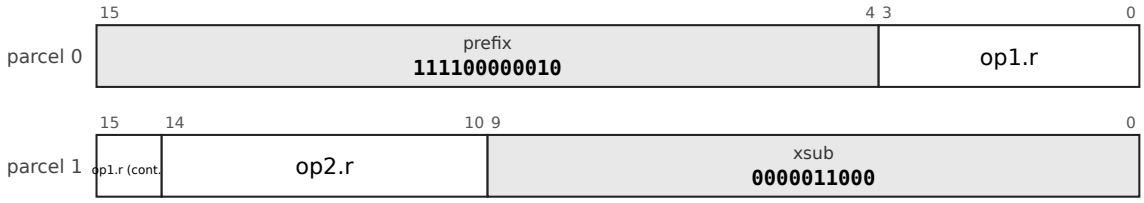
**Flags** None. These operate on the bits, do not canonicalise a NaN and raise no exception flag.

**Exceptions** None.

**Why it is here.** Sign and selection operations (negate, absolute value and the like) work on the bits, do not canonicalise a NaN and raise no flag, so they cost almost nothing. The PowerPC 601 floating-point set has the same operations (stated from memory, not checked against a manual). No separate reason is recorded for these three.

#### *FNEG*

**FNEG: R2, prefix 111100000010, sub-opcode 0000011000; operands reg5, reg5; 4 bytes**

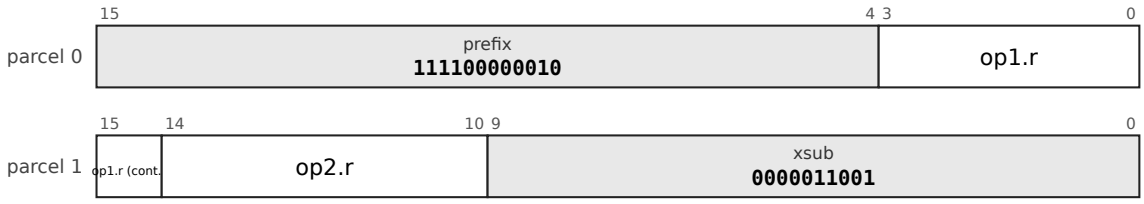



```
FNEG Rs, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

#### *FABS*

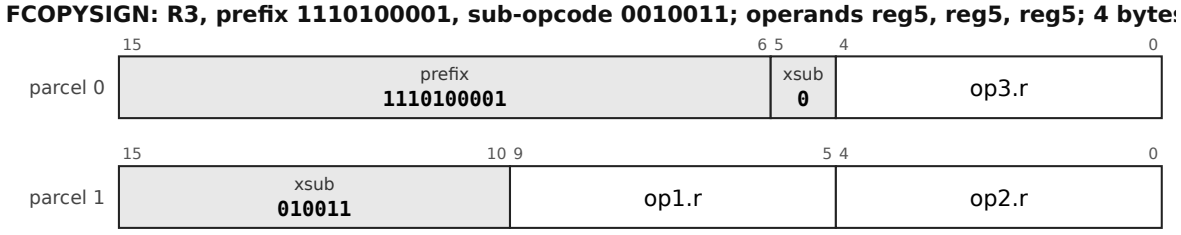
**FABS: R2, prefix 111100000010, sub-opcode 0000011001; operands reg5, reg5; 4 bytes**




```
FABS Rs, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

### *FCOPYSIGN*




```
FCOPYSIGN Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FSIGN above; 1 layout form, each a chart of its own (above).

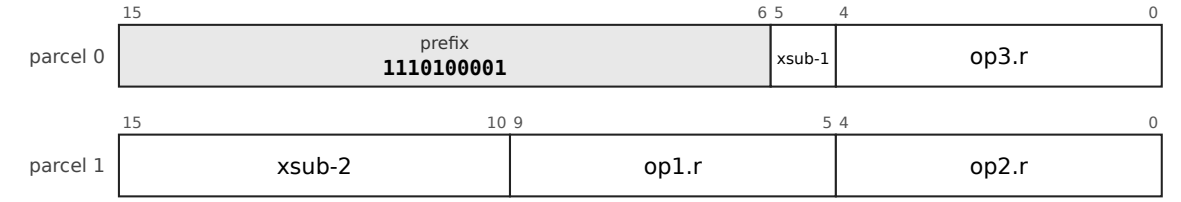
## FMIN: Floating minimum and maximum, binary64

**Mnemonics:** `FMIN`, `FMAX` (aliases: `FMIND`, `FMAXD`)

### Syntax

```
FMIN Rs, Rt, Rd
FMAX Rs, Rt, Rd
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
if Rs is a NaN then Rd ← Rt ; else if Rt is a NaN then Rd ← Rs ; else Rd ← smaller (FMIN) or larger (FMAX)
if both are NaNs the result is the canonical quiet NaN
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.

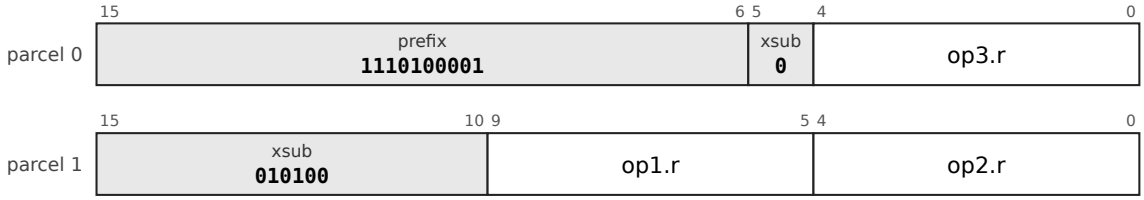
**Exceptions** None.

The alternative spellings `FMIND`, `FMAXD` are aliases.

**Why it is here.** The floating counterparts of the integer `MIN` and `MAX`. A NaN operand loses to a number and two NaNs give the canonical quiet NaN.

### *FMIN*

**FMIN: R3, prefix 1110100001, sub-opcode 0010100; operands reg5, reg5, reg5; 4 bytes**

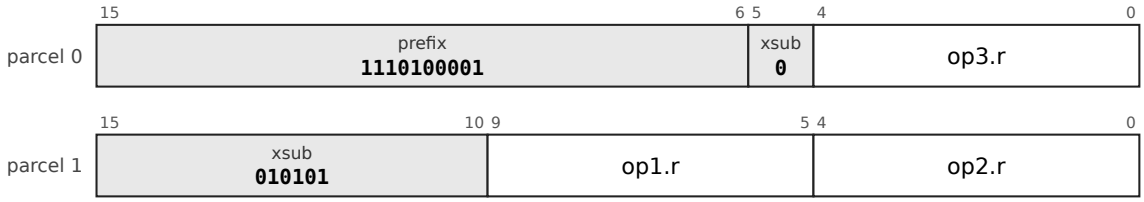



```
FMIN Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FMIN above; 1 layout form, each a chart of its own (above).

### *FMAX*

**FMAX: R3, prefix 1110100001, sub-opcode 0010101; operands reg5, reg5, reg5; 4 bytes**




```
FMAX Rs, Rt, Rd
```

Operation, flags and exceptions as for the entry FMIN above; 1 layout form, each a chart of its own (above).

### *FMIND*

An alias of `FMIN`: the assembler writes it as that word.

### *FMAXD*

An alias of `FMAX`: the assembler writes it as that word.

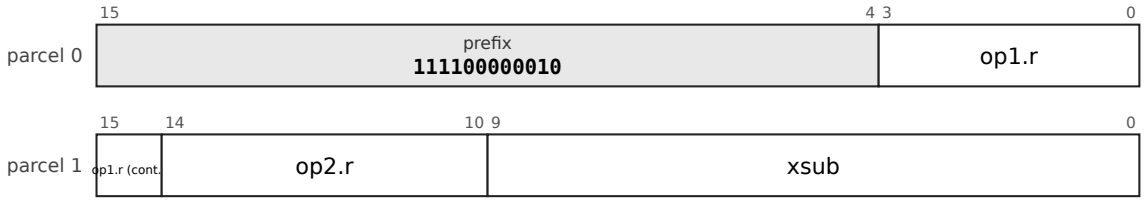
## FCLASS: Classify a floating-point value

**Mnemonics:** `FCLASS`, `FCLASS32`, `FCLASS16` (aliases: `FCLASSD`, `FCLASSS`)

### Syntax

```
FCLASS Rs, Rd
FCLASS32 Rs, Rd
FCLASS16 Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

**Operation**

```
Rd ← a bit vector with exactly one bit set:
bit 0 -infinity      bit 1 -normal      bit 2 -subnormal   bit 3 -zero
bit 4 +zero          bit 5 +subnormal   bit 6 +normal      bit 7 +infinity
bit 8 signalling NaN bit 9 quiet NaN
```

**Flags** None.

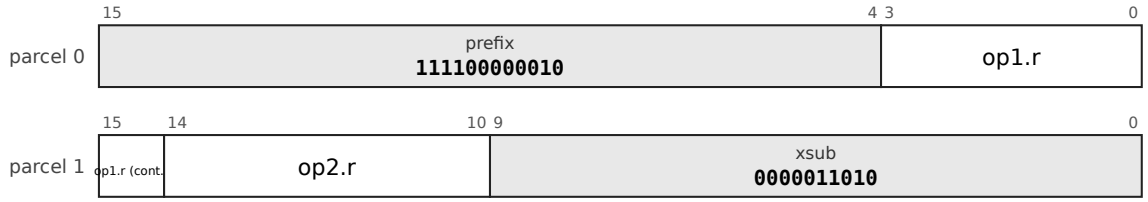
**Exceptions** None.

Every libm function begins with this dispatch. The alternative spellings `FCLASSD`, `FCLASSS` are aliases.

**Why it is here.** Every libm function begins with this dispatch: one instruction returns which of ten kinds a value is (signalling or quiet NaN, infinity, normal, subnormal, zero, each signed), where software needs several compares and masks. It is justified by a workload, library mathematics, and not by compiled C, as the tag branches are. Nothing in the corpus emits it, because the compiler does not use it.

### *FCLASS*

**FCLASS: R2, prefix 111100000010, sub-opcode 0000011010; operands reg5, reg5; 4 bytes**

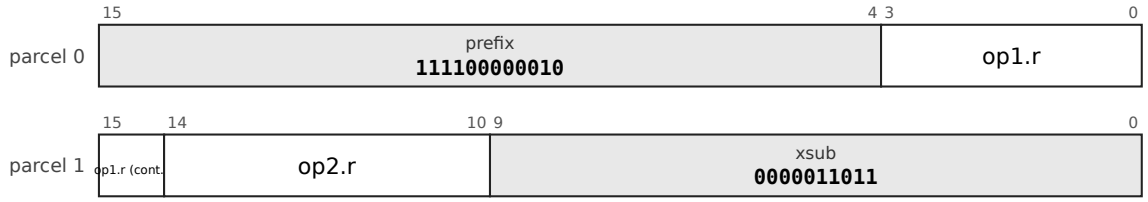



```
FCLASS Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

### *FCLASS32*

**FCLASS32: R2, prefix 111100000010, sub-opcode 0000011011; operands reg5, reg5; 4 bytes**

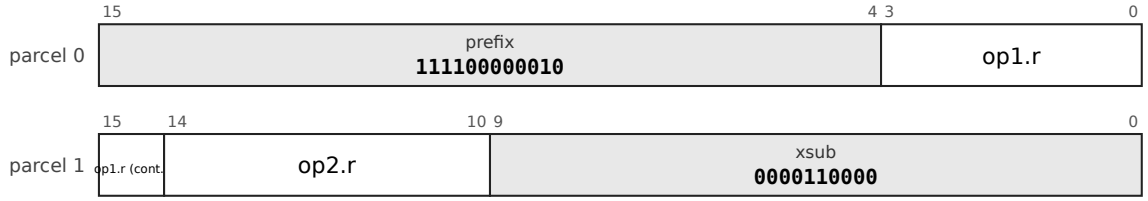



```
FCLASS32 Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

### *FCLASS16*

**FCLASS16: R2, prefix 111100000010, sub-opcode 0000110000; operands reg5, reg5; 4 bytes**




```
FCLASS16 Rs, Rd
```

Operation, flags and exceptions as for the entry FCLASS above; 1 layout form, each a chart of its own (above).

***FCLASSD***
An alias of `FCLASS`: the assembler writes it as that word.

***FCLASSS***
An alias of `FCLASS32`: the assembler writes it as that word.

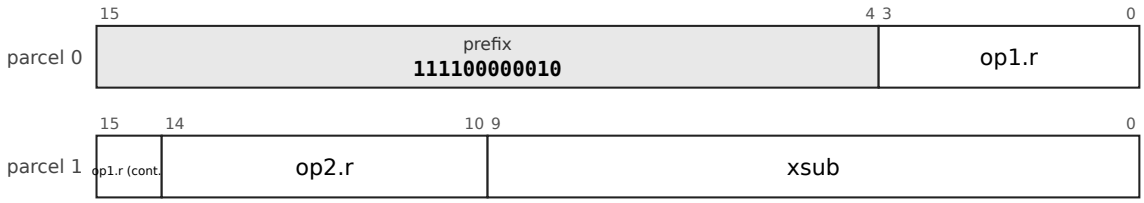
## FRINT: Round to an integral value, binary64

**Mnemonics:** `FRINT`, `FRINTZ`, `FRINTM`, `FRINTP`, `FRINTA` (aliases: `FRINTD`, `FRINTDZ`, `FRINTDM`, `FRINTDP`, `FRINTDA`)

### Syntax

```
FRINT Rs, Rd
FRINTZ Rs, Rd
FRINTM Rs, Rd
FRINTP Rs, Rd
FRINTA Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

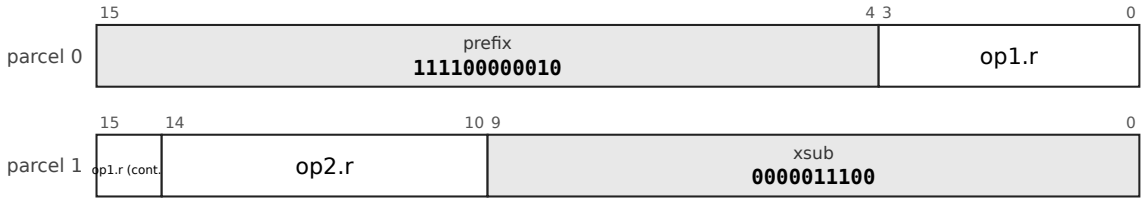
### Operation

```
FRINT : Rd ← Rs rounded to nearest, ties to even
FRINTZ: toward zero         FRINTM: toward -infinity
FRINTP: toward +infinity    FRINTA: to nearest, ties away from zero (C round())
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps.
**Exceptions** None.
The rounding mode is in the instruction, never in a mode bit. The alternative spellings `FRINTD`, `FRINTDZ`, ... are aliases.
**Why it is here.** Rounding to an integral value in each direction (nearest even, toward zero, down, up) is in the set, and `FRINTA`, ties away from zero, serves C's `round()`, which is common in libm; it is a qualifier of the round operation and so costs no new operation.

***FRINT***

**FRINT: R2, prefix 111100000010, sub-opcode 0000011100; operands reg5, reg5; 4 bytes**

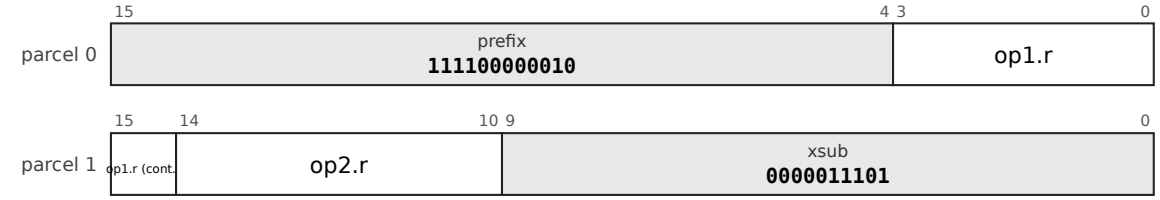



```
FRINT Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

### *FRINTZ*

**FRINTZ: R2, prefix 111100000010, sub-opcode 0000011101; operands reg5, reg5; 4 bytes**

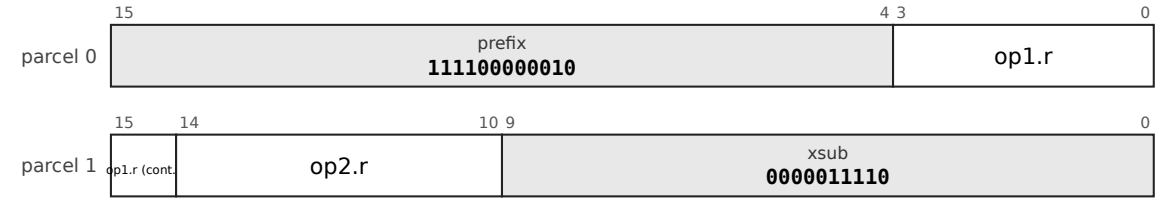



```
FRINTZ Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

### *FRINTM*

**FRINTM: R2, prefix 111100000010, sub-opcode 0000011110; operands reg5, reg5; 4 bytes**

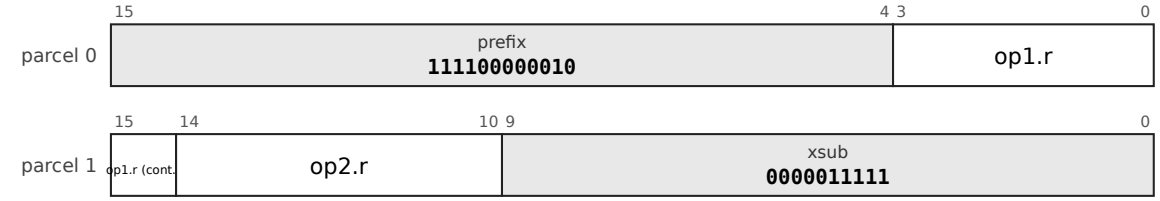



```
FRINTM Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

### *FRINTP*

**FRINTP: R2, prefix 111100000010, sub-opcode 0000011111; operands reg5, reg5; 4 bytes**

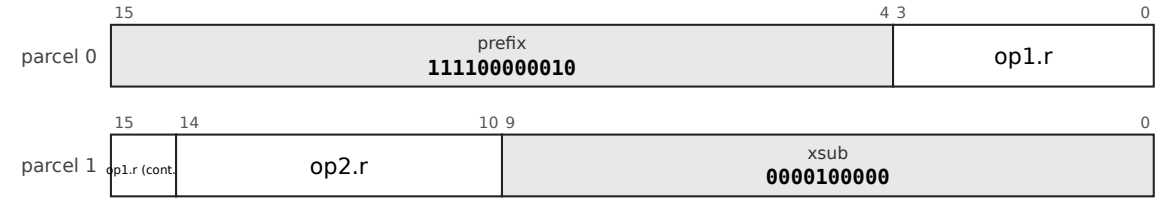



```
FRINTP Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

### *FRINTA*

**FRINTA: R2, prefix 111100000010, sub-opcode 0000100000; operands reg5, reg5; 4 bytes**




```
FRINTA Rs, Rd
```

Operation, flags and exceptions as for the entry FRINT above; 1 layout form, each a chart of its own (above).

***FRINTD***
An alias of FRINT: the assembler writes it as that word.

***FRINTDZ***
An alias of FRINTZ: the assembler writes it as that word.

***FRINTDM***
An alias of FRINTM: the assembler writes it as that word.

***FRINTDP***
An alias of FRINTP: the assembler writes it as that word.

***FRINTDA***
An alias of FRINTA: the assembler writes it as that word.

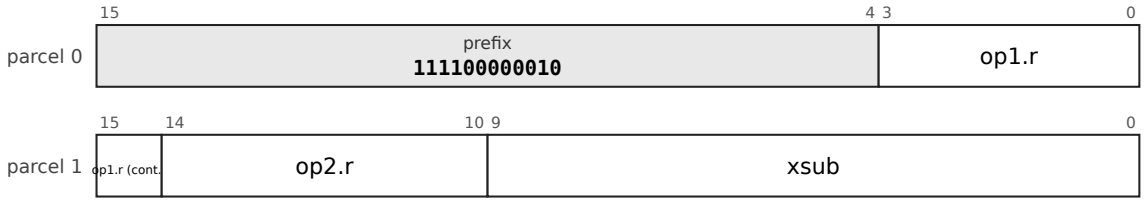
## FCVT: Floating conversions

**Mnemonics:** FCVTDQ, FCVTDQU, FCVTQD, FCVTQUD, FCVTDS, FCVTSD, FCVTSL, FCVTLS

### Syntax

```
FCVTDQ Rs, Rd        ; binary64 to signed quad
FCVTDQU Rs, Rd       ; binary64 to unsigned quad
FCVTQD Rs, Rd        ; signed quad to binary64
FCVTQUD Rs, Rd       ; unsigned quad to binary64
FCVTDS Rs, Rd        ; binary64 to binary32
FCVTSD Rs, Rd        ; binary32 to binary64
FCVTSL Rs, Rd        ; binary32 to signed 32-bit integer
FCVTLS Rs, Rd        ; signed 32-bit integer to binary32
```

### Encoding




The chart is the leaf R2-1, R2, prefix 111100000010. op1 is reg5: register, 5 bits. op2 is reg5: register, 5 bits. xsub selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): R2-1.

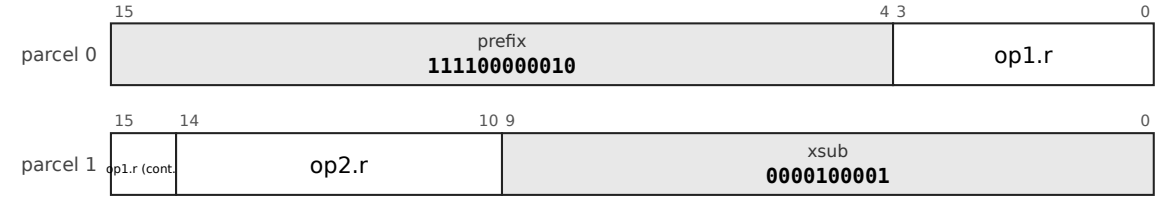
### Operation

```
float to integer: truncate toward zero, then saturate; a NaN gives the largest value
integer to float: round to nearest even
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps. A NaN or an out-of-range value raises NV and saturates.
**Exceptions** None.
Integer results are 64-bit (FCVTDQ, FCVTDQU) or 32-bit sign-extended (FCVTSL). Unsigned conversion of a negative value is invalid and gives 0.
**Why it is here.** FCVTDQU and FCVTQUD convert unsigned 64-bit integers to and from binary64. C needs both, and without them each takes a branch and several instructions.

### *FCVTDQ*

**FCVTDQ: R2, prefix 111100000010, sub-opcode 0000100001; operands reg5, reg5; 4 bytes**

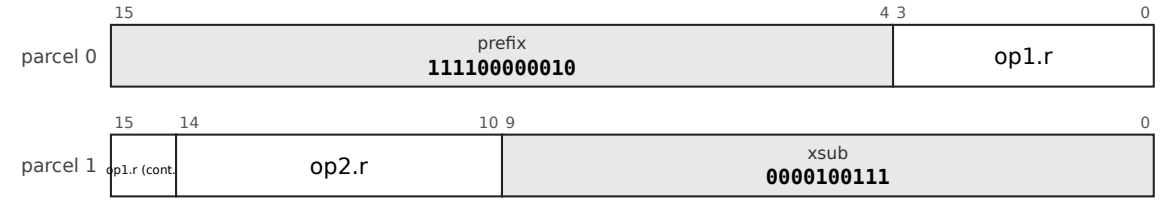



```
FCVTDQ Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTDQU*

**FCVTDQU: R2, prefix 111100000010, sub-opcode 0000100111; operands reg5, reg5; 4 bytes**

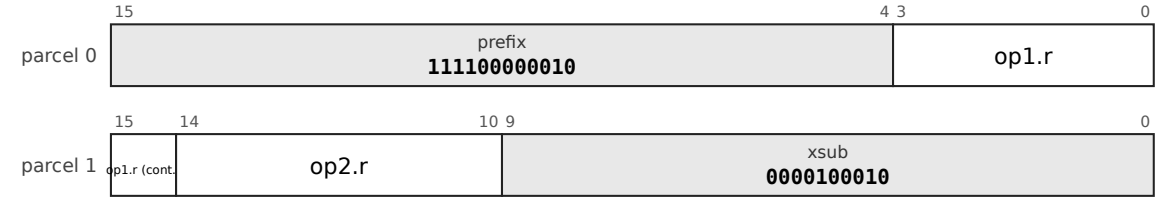



```
FCVTDQU Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTQD*

**FCVTQD: R2, prefix 111100000010, sub-opcode 0000100010; operands reg5, reg5; 4 bytes**

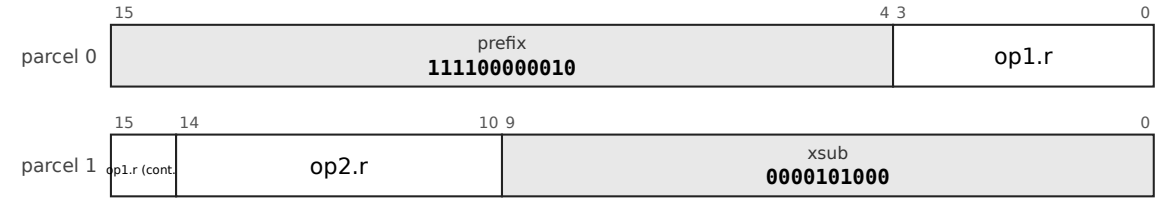



```
FCVTQD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTQUD*

**FCVTQUD: R2, prefix 111100000010, sub-opcode 0000101000; operands reg5, reg5; 4 bytes**

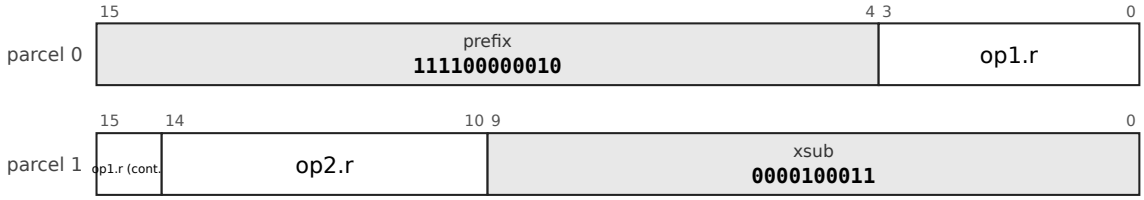



```
FCVTQUD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTDS*

**FCVTDS: R2, prefix 111100000010, sub-opcode 0000100011; operands reg5, reg5; 4 bytes**

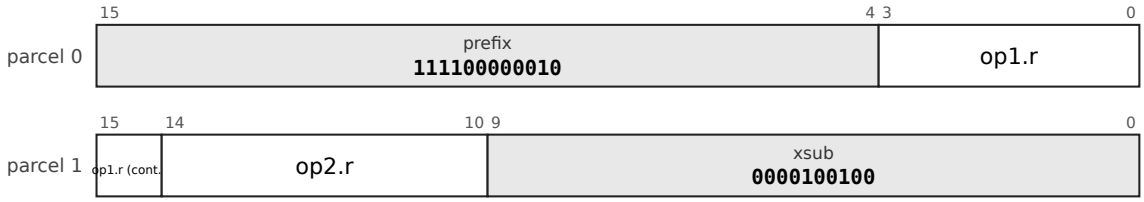



```
FCVTDS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTSD*

**FCVTSD: R2, prefix 111100000010, sub-opcode 0000100100; operands reg5, reg5; 4 bytes**

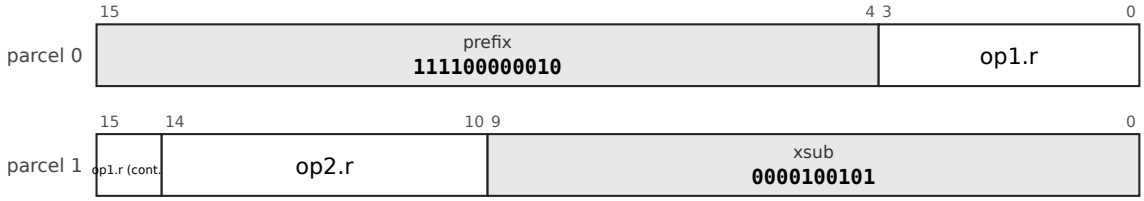



```
FCVTSD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTSL*

**FCVTSL: R2, prefix 111100000010, sub-opcode 0000100101; operands reg5, reg5; 4 bytes**

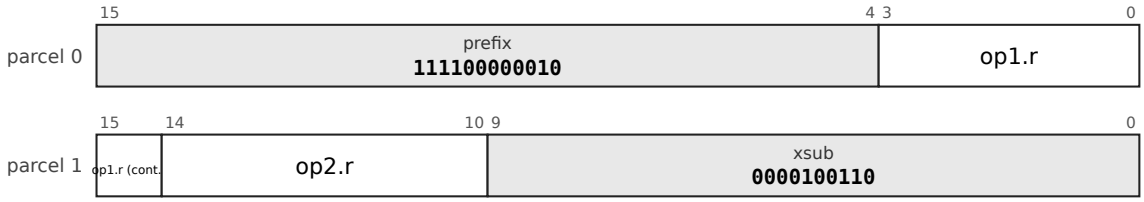



```
FCVTSL Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

### *FCVTLS*

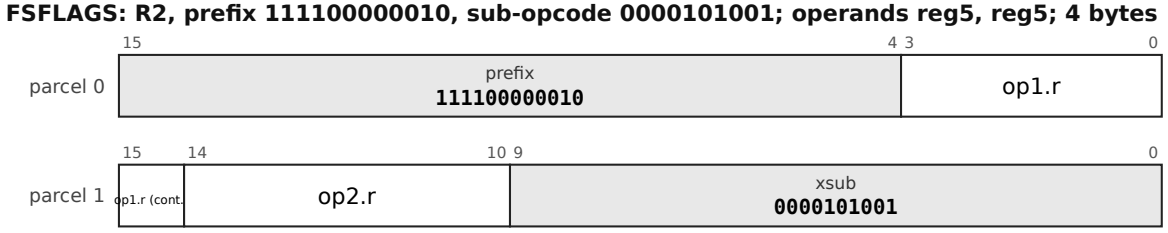
**FCVTLS: R2, prefix 111100000010, sub-opcode 0000100110; operands reg5, reg5; 4 bytes**




```
FCVTLS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVT above; 1 layout form, each a chart of its own (above).

## FSFLAGS: Swap the floating-point exception flags

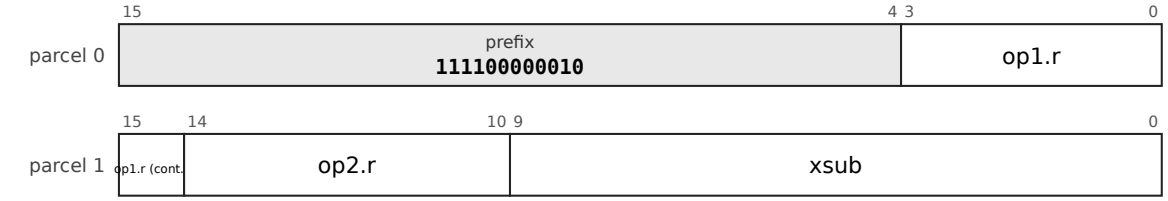



**Mnemonics:** `FSFLAGS`

### Syntax

```
FSFLAGS Rs, Rd
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

### Operation

```
old ← FPSR
FPSR ← Rs[4:0]
Rd ← old
```

**Flags** None.
**Exceptions** None.
`FPSR` is the sticky five-bit register (`NX` bit 0, `UF` 1, `OF` 2, `DZ` 3, `NV` 4). C99 `<fenv.h>` is built on this one instruction.
**Why it is here.** IEEE 754 requires five status flags, and C99 `<fenv.h>` needs software access to them. The architecture has a sticky five-bit `FPSR`, laid out as RISC-V's `fflags`, written by every operation that can raise a flag and never trapping, and one swap instruction to read and write it. It is a record of what happened, not a mode, so the rule against mode bits is untouched.

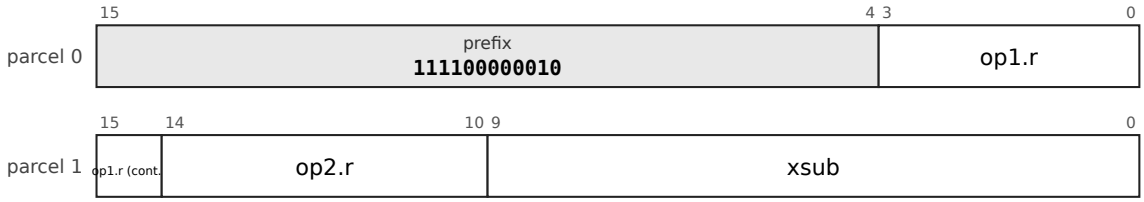
## FHALF: Floating arithmetic, binary16

**Mnemonics:** `FADD16`, `FSUB16`, `FMUL16`, `FDIV16`, `FSQRT16`, `FCMP16`, `FMADD16`, `FMSUB16`, `FNMADD16`, `FNMSUB16`

### Syntax

```
FADD16 Rs, Rd
FSUB16 Rs, Rd
FMUL16 Rs, Rd
FDIV16 Rs, Rd
FSQRT16 Rs, Rd
FCMP16 Rd, Rs
FMADD16 Rs, Rt, Ru, Rd ...
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`, `R4-1`.

### Operation

```
as the binary32 words, in binary16: Rd ← Rd op Rs ; the fused forms take Rs, Rt, Ru, Rd
a half lives in the low 16 bits of a register, sign-extended to 64
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into `FPSR` (`NX` inexact, `UF` underflow, `OF` overflow, `DZ` divide by zero, `NV` invalid); nothing traps. `FCMP16` writes `N Z V C` as `FCMP`.

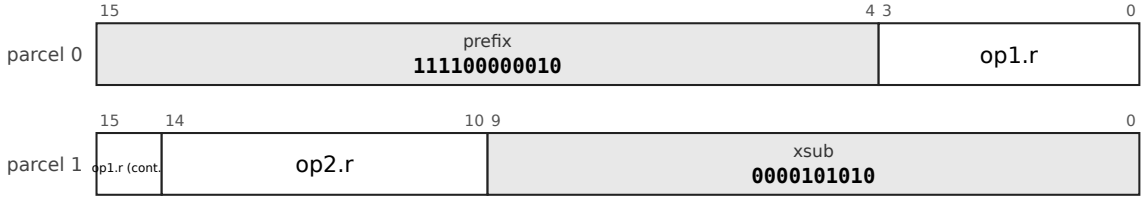
**Exceptions** None.

Results are correctly rounded to nearest even, tininess judged after rounding; a NaN result is the canonical `0x7E00`. Open: the remaining half-precision operations (J.9.10).

**Why it is here.** Half precision began as a GPU format (NVIDIA's `half`, 2002), spread to machine learning and image formats, and is in CPUs (x86 `F16C` and AVX-512 FP16, ARMv8.2, RISC-V `Zfh` and `Zfhmin`); this account is from memory and not checked against a manual. Two levels are possible: conversions only (about 500 to 1,000 lookup tables, no new operation by the folding rule) and full arithmetic (about 2,000 to 3,500 tables; estimates), and the existing double datapath can compute half correctly rounded. The architecture has both conversions and arithmetic. Open: the remaining half-precision operations (J.9.10). 16-bit *integer* arithmetic is not provided: the ALU has one width (9.2.9).

### *FADD16*

**FADD16: R2, prefix 111100000010, sub-opcode 0000101010; operands reg5, reg5; 4 bytes**

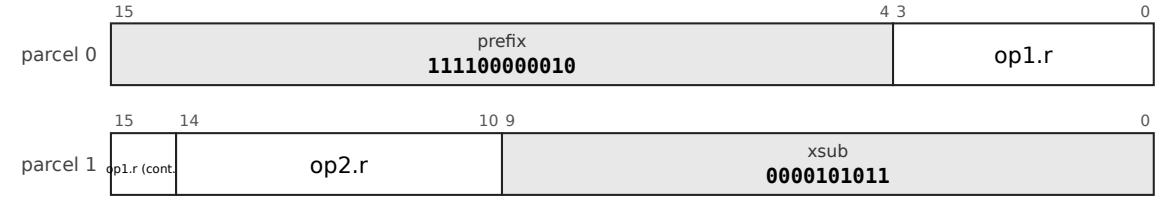



```
FADD16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FSUB16*

**FSUB16: R2, prefix 111100000010, sub-opcode 0000101011; operands reg5, reg5; 4 bytes**

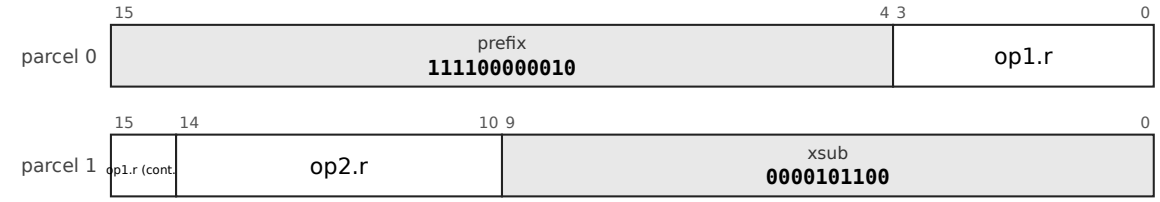



```
FSUB16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FMUL16*

**FMUL16: R2, prefix 111100000010, sub-opcode 0000101100; operands reg5, reg5; 4 bytes**

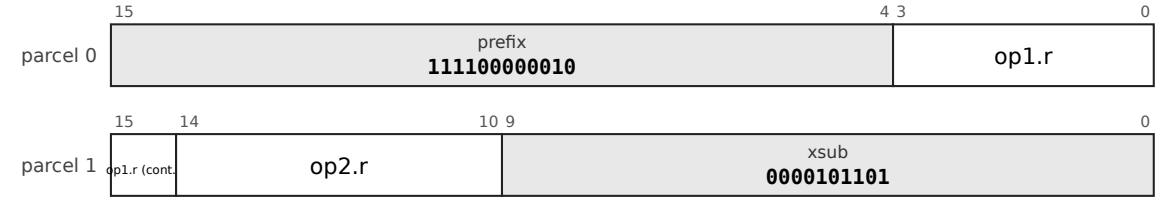



```
FMUL16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FDIV16*

**FDIV16: R2, prefix 111100000010, sub-opcode 0000101101; operands reg5, reg5; 4 bytes**

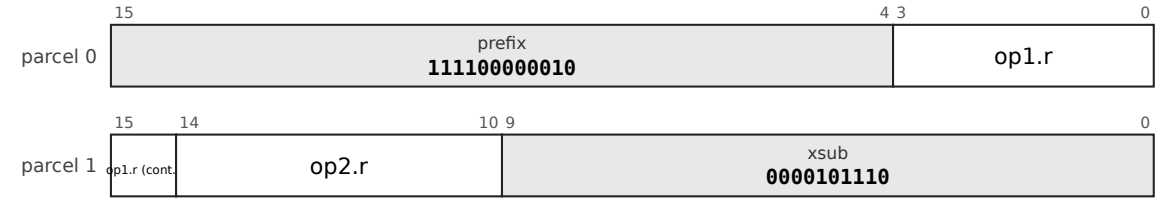



```
FDIV16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FSQRT16*

**FSQRT16: R2, prefix 111100000010, sub-opcode 0000101110; operands reg5, reg5; 4 bytes**

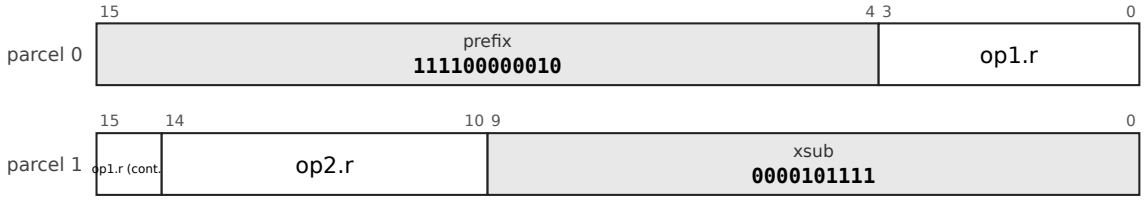



```
FSQRT16 Rs, Rd
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FCMP16*

**FCMP16: R2, prefix 111100000010, sub-opcode 0000101111; operands reg5, reg5; 4 bytes**

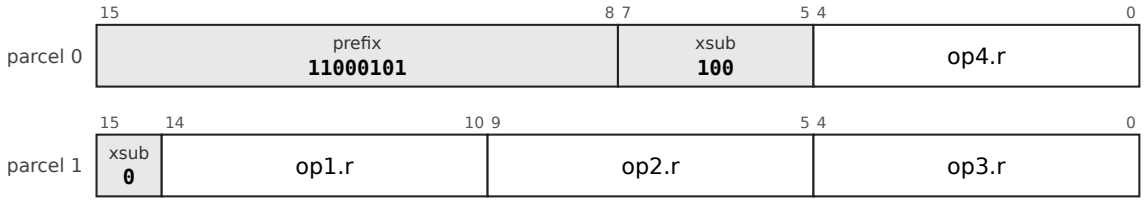



```
FCMP16 Rd, Rs
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FMADD16*

**FMADD16: R4, prefix 11000101, sub-opcode 1000; operands reg5, reg5, reg5, reg5; 4 bytes**

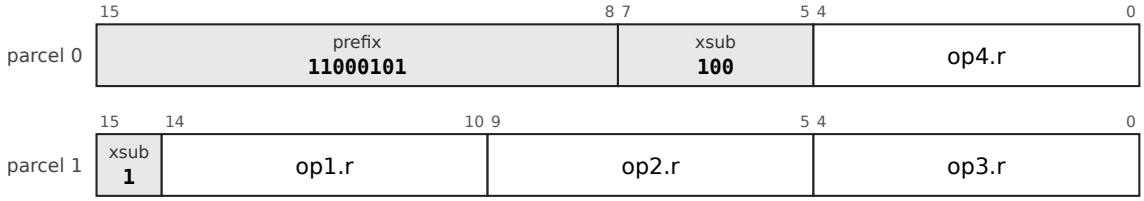



```
FMADD16 Rs, Rt, Ru, Rd ...
```

Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FMSUB16*

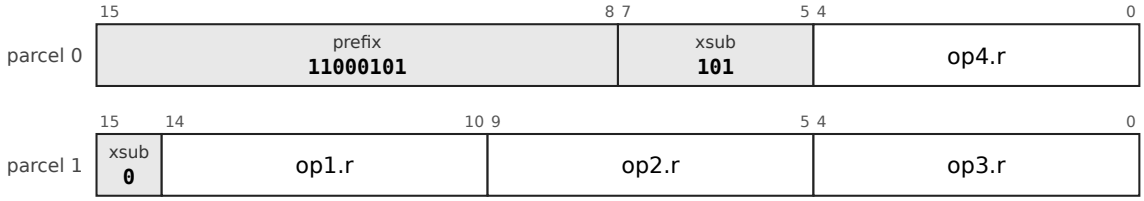
**FMSUB16: R4, prefix 11000101, sub-opcode 1001; operands reg5, reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FNMADD16*

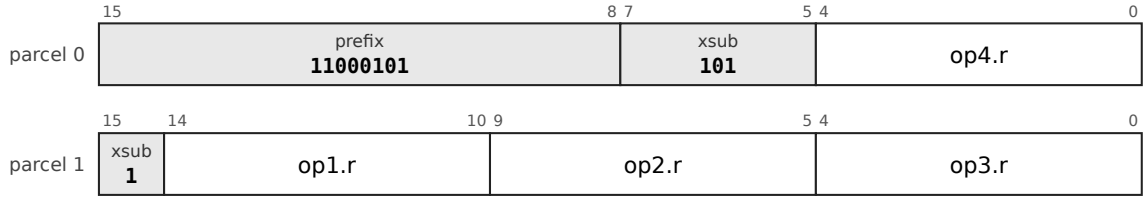
**FNMADD16: R4, prefix 11000101, sub-opcode 1010; operands reg5, reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

### *FNMSUB16*

**FNMSUB16: R4, prefix 11000101, sub-opcode 1011; operands reg5, reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry FHALF above; 1 layout form, each a chart of its own (above).

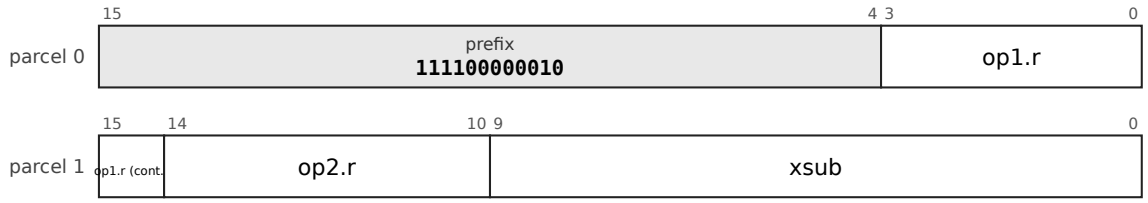
## FCVTH: Conversions to and from binary16

**Mnemonics:** FCVTSH, FCVTHS, FCVTDH, FCVTHD, FCVTHL, FCVTLH

### Syntax

```
FCVTSH Rs, Rd        ; binary32 to half
FCVTHS Rs, Rd        ; half to binary32
FCVTDH Rs, Rd        ; binary64 to half
FCVTHD Rs, Rd        ; half to binary64
FCVTHL Rs, Rd        ; half to signed 32-bit integer, toward zero
FCVTLH Rs, Rd        ; signed 32-bit integer to half
```

### Encoding




The chart is the leaf R2-1, R2, prefix 111100000010. op1 is reg5: register, 5 bits. op2 is reg5: register, 5 bits. xsub selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): R2-1.

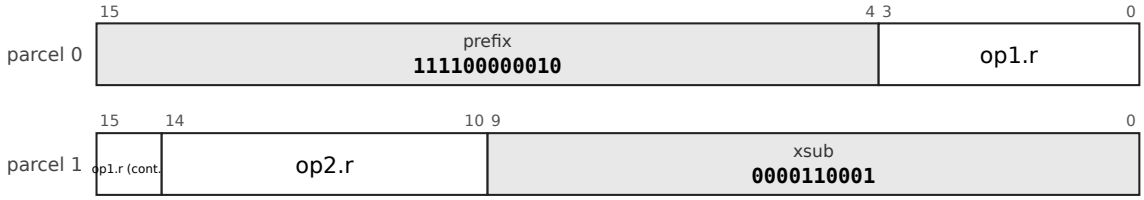
### Operation

```
narrowing: round to nearest even ; widening: exact
FCVTHL of a NaN or an infinity is invalid and saturates; truncating a fraction is inexact
```

**Flags** No integer flag is written. The IEEE exception flags raised are ORed into FPSR (NX inexact, UF underflow, OF overflow, DZ divide by zero, NV invalid); nothing traps.
**Exceptions** None.
**Why it is here.** The conversion half of the binary16 support (see FHALF). The conversions are the part machine-learning code needs most, because a half is normally loaded and stored with MOVW, converted, computed in single or double, and converted back.

### *FCVTSH*

**FCVTSH: R2, prefix 111100000010, sub-opcode 0000110001; operands reg5, reg5; 4 bytes**

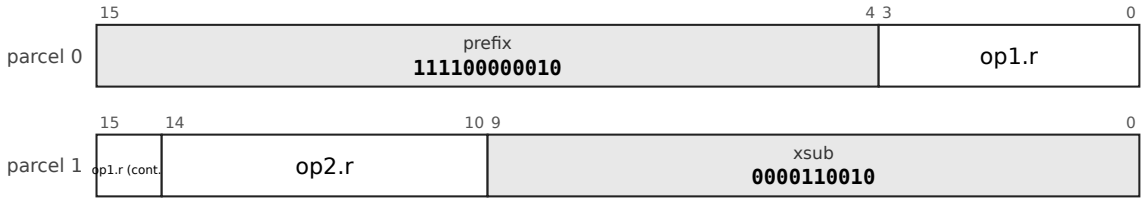



```
FCVTSH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

### *FCVTHS*

**FCVTHS: R2, prefix 111100000010, sub-opcode 0000110010; operands reg5, reg5; 4 bytes**

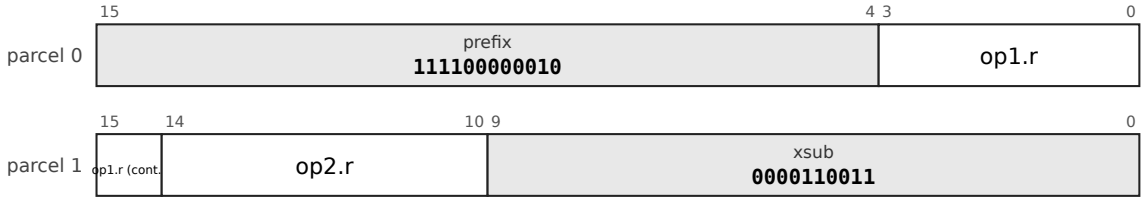



```
FCVTHS Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

### *FCVTDH*

**FCVTDH: R2, prefix 111100000010, sub-opcode 0000110011; operands reg5, reg5; 4 bytes**

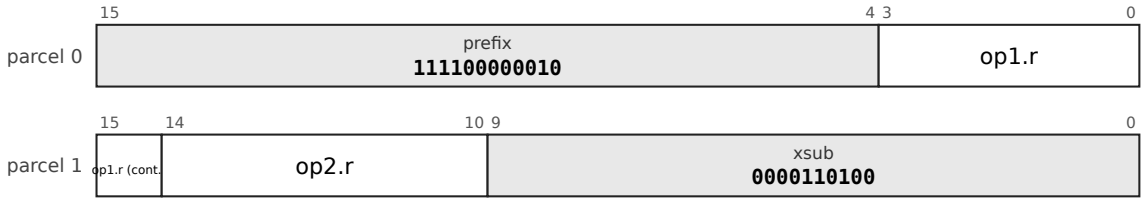



```
FCVTDH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

### *FCVTHD*

**FCVTHD: R2, prefix 111100000010, sub-opcode 0000110100; operands reg5, reg5; 4 bytes**

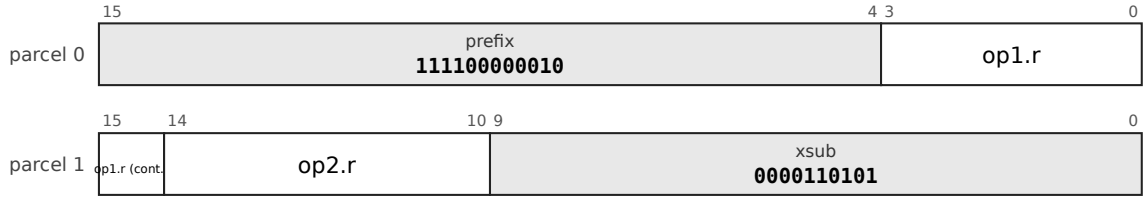



```
FCVTHD Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

***FCVTHL***

**FCVTHL: R2, prefix 111100000010, sub-opcode 0000110101; operands reg5, reg5; 4 bytes**

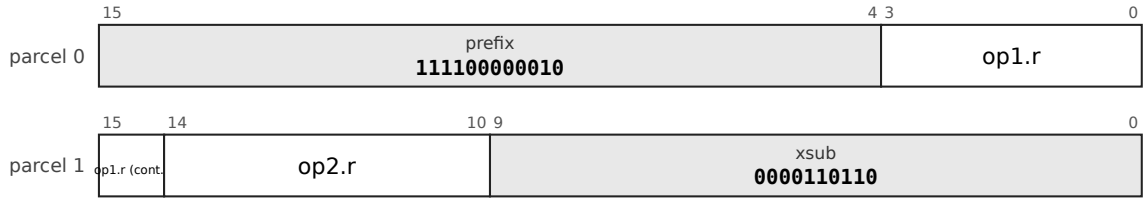



```
FCVTHL Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

***FCVTLH***

**FCVTLH: R2, prefix 111100000010, sub-opcode 0000110110; operands reg5, reg5; 4 bytes**




```
FCVTLH Rs, Rd
```

Operation, flags and exceptions as for the entry FCVTH above; 1 layout form, each a chart of its own (above).

## 9.13 Atomics and ordering

*Cf. chapter 7, section 7.4 (atomic operations), section 7.3 (the multiprocessor instructions at a glance) and section 7.5 (the memory model).*

**9.13.1** The atomic group ATOM holds CAS, SWAP, XADD, XAND, XOR_, XXOR, XMAX, XMIN, XMAXU and XMINU, each with a _DB variant that adds a barrier, and LL and SC; FENCE, WFE, SEV, ISYNC and HINT are in SYS. Their semantics, ordering rules and the reservation monitor are in chapter 7.

**9.13.2** CAS operates on 4 or 8 bytes; an 8-byte compare-and-swap may span two adjacent 4-byte fields, because the usual lock-and-count fast path of an operating system is exactly that (an 8-byte whole over two 4-byte members). *(D)* Open: the encoding of the CAS size and the placement of the operands of the atomics (J.9.3).

*Commentary.* The official entry of each instruction is below, in this section; chapter 7 gives the reasons, the memory model and the use. **Impact.** *Kernel:* the entries below are all a kernel needs to write its locks; chapter 7 explains how they compose. *Compiler:* a lock acquire is one _DB instruction. *Hardware:* a reservation monitor beside the data cache.

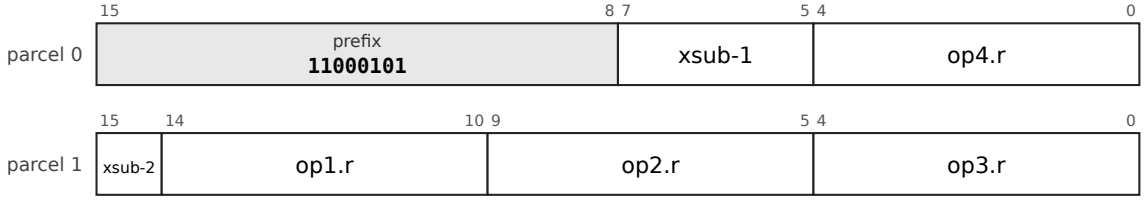
### CAS: Compare and swap

**Mnemonics:** CAS, CASDB

#### Syntax

```
CAS expected, new, (address)        ; size is 4 or 8 bytes; operand placement open (J.9.3)
CASDB ...                            ; with a full barrier
```

### Encoding




The chart is the leaf `R4-1`, R4, prefix `11000101`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `op4` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R4-1`.

### Operation

```
old ← M[address]                          ; 4 or 8 bytes, naturally aligned
if old = expected then M[address] ← new
result ← old          ; the program compares result with expected to learn whether the swap happened
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

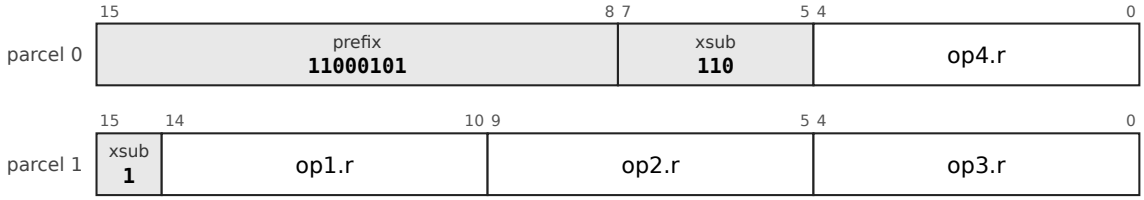
The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for `CAS` and `CASDB`, in the order written) is open (J.9.3).

**Why it is here.** The barrier-fused variants are taken from LoongArch's pragmatism: a lock acquire is one instruction rather than an atomic followed by a `FENCE`, and that pair is in every critical section of the system. With the other multiprocessor operations the atomics make the seventeen-operation SMP umbrella (7.3). They are defined on a reservation monitor and not on a bus lock (2.5).

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

#### *CAS*

**CAS: R4, prefix 11000101, sub-opcode 1101; operands reg5, reg5, reg5, reg5; 4 bytes**

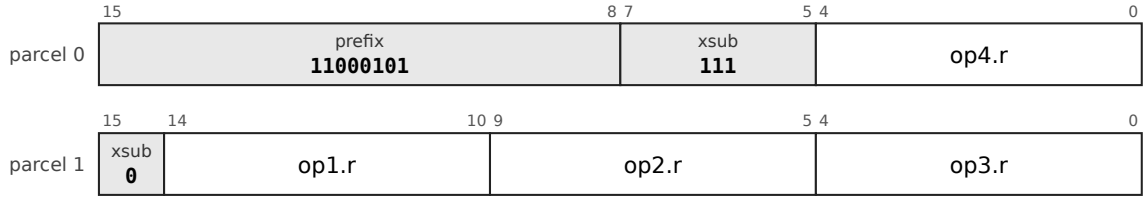



```
CAS expected, new, (address)
```

Operation, flags and exceptions as for the entry CAS above; 1 layout form, each a chart of its own (above).

### *CASDB*

**CASDB: R4, prefix 11000101, sub-opcode 1110; operands reg5, reg5, reg5, reg5; 4 bytes**




```
CASDB ...
```

Operation, flags and exceptions as for the entry CAS above; 1 layout form, each a chart of its own (above).

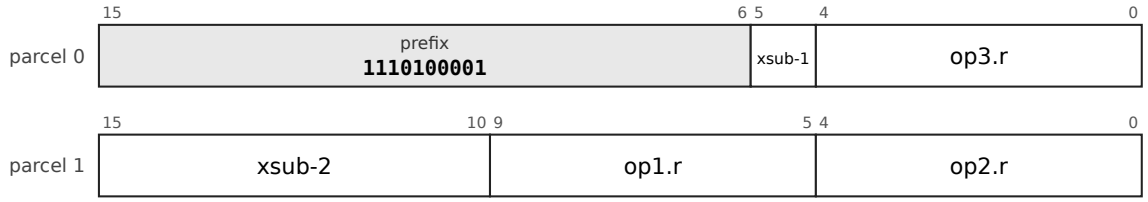
## SWAP: Atomic exchange

**Mnemonics:** `SWAP`, `SWAPDB`

### Syntax

```
SWAP value, (address)
SWAPDB ...
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
old ← M[address] ; M[address] ← value ; result ← old
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

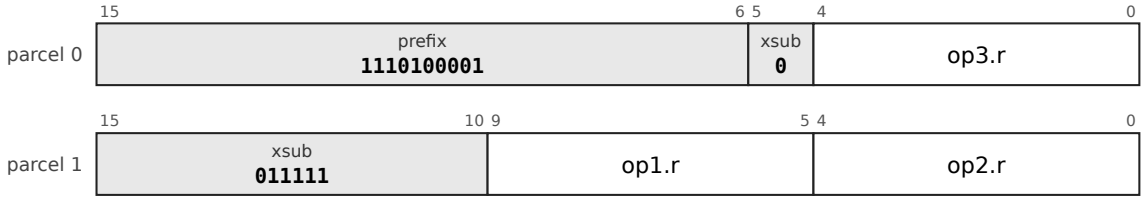
The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for `CAS` and `CASDB`, in the order written) is open (J.9.3).

**Why it is here.** One of the read-modify-write atomics (see `CAS`). No separate reason is recorded for it.

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

### *SWAP*

**SWAP: R3, prefix 1110100001, sub-opcode 0011111; operands reg5, reg5, reg5; 4 bytes**

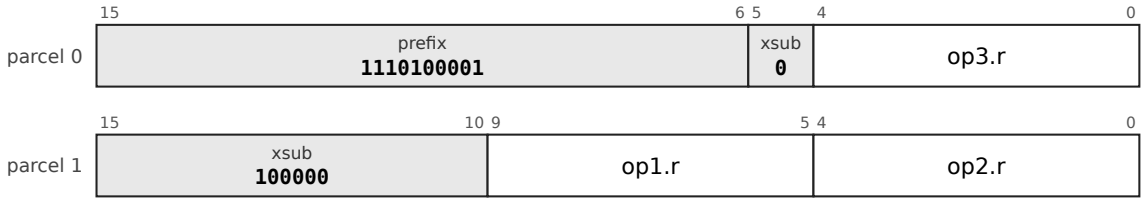



```
SWAP value, (address)
```

Operation, flags and exceptions as for the entry SWAP above; 1 layout form, each a chart of its own (above).

### *SWAPDB*

**SWAPDB: R3, prefix 1110100001, sub-opcode 0100000; operands reg5, reg5, reg5; 4 bytes**




```
SWAPDB ...
```

Operation, flags and exceptions as for the entry SWAP above; 1 layout form, each a chart of its own (above).

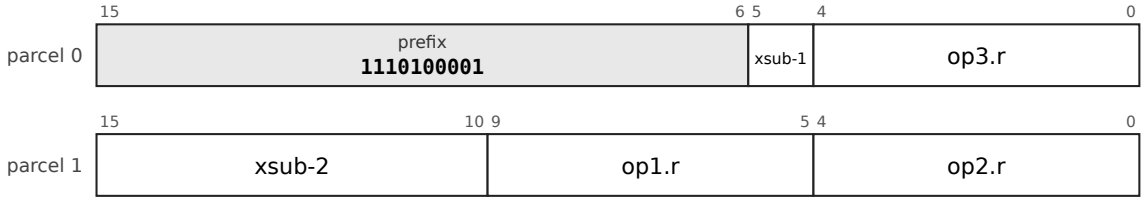
## XADD: Atomic fetch and add

**Mnemonics:** `XADD`, `XADDDB`

### Syntax

```
XADD value, (address)
XADDDB ...
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
old ← M[address] ; M[address] ← old + value ; result ← old
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

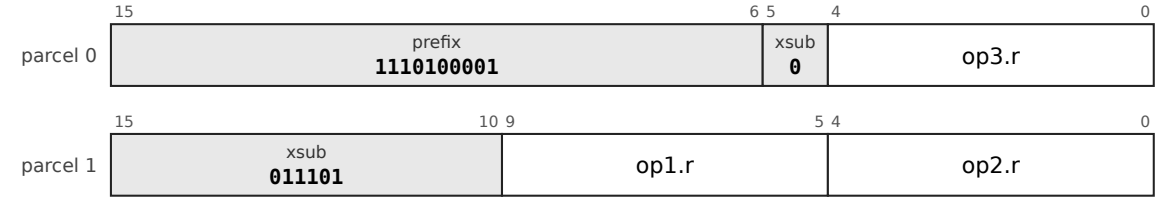
The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for CAS and CASDB, in the order written) is open (J.9.3).

**Why it is here.** One of the fetch-and-operate atomics (see CAS), in the place of the PDP-11's read-modify-write instructions, which were indivisible on the Unibus through the DATIP/DATO pair (7.1).

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

### *XADD*

**XADD: R3, prefix 1110100001, sub-opcode 0011101; operands reg5, reg5, reg5; 4 bytes**

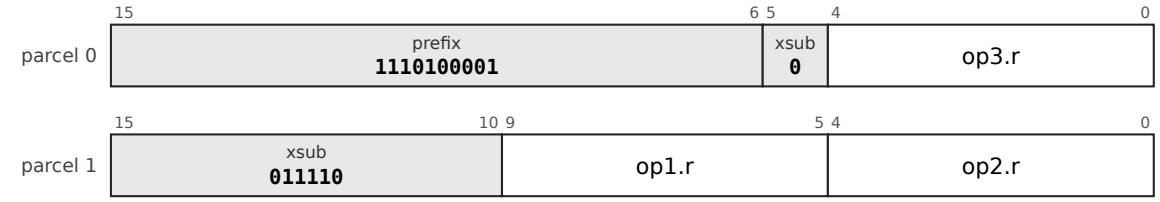



```
XADD value, (address)
```

Operation, flags and exceptions as for the entry XADD above; 1 layout form, each a chart of its own (above).

### *XADDDB*

**XADDDB: R3, prefix 1110100001, sub-opcode 0011110; operands reg5, reg5, reg5; 4 bytes**




```
XADDDB ...
```

Operation, flags and exceptions as for the entry XADD above; 1 layout form, each a chart of its own (above).

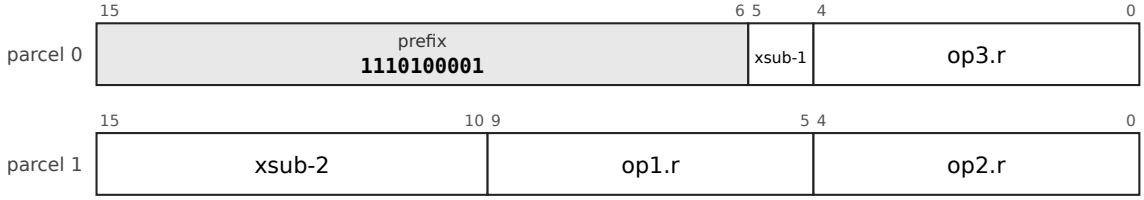
## XAND: Atomic fetch and and, or, xor

**Mnemonics:** XAND, XANDDB, XOR, XOR_DB, XXOR, XXORDB

### Syntax

```
XAND value, (address)
XOR_ value, (address)        ; the trailing underscore keeps the mnemonic distinct from XOR
XXOR value, (address)
each with a DB form
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
old ← M[address]
XAND: M[address] ← old AND value
XOR_: M[address] ← old OR value
XXOR: M[address] ← old XOR value
result ← old
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

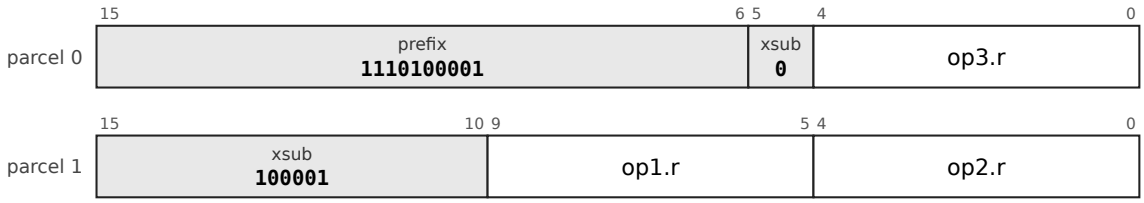
The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for `CAS` and `CASDB`, in the order written) is open (J.9.3).

**Why it is here.** The fetch-and-and, or and xor atomics (see `CAS`). No separate reason is recorded for them.

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

#### *XAND*

**XAND: R3, prefix 1110100001, sub-opcode 0100001; operands reg5, reg5, reg5; 4 bytes**

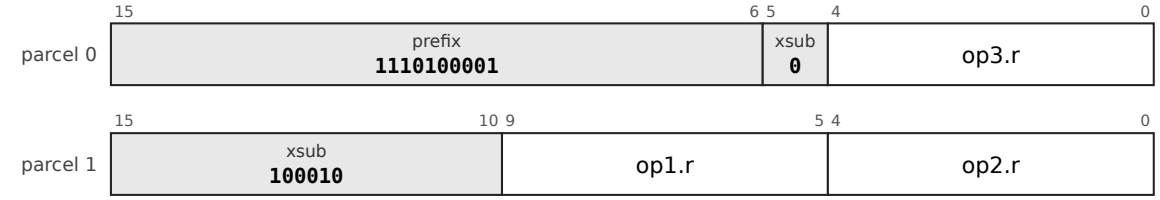



```
XAND value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

### *XANDDB*

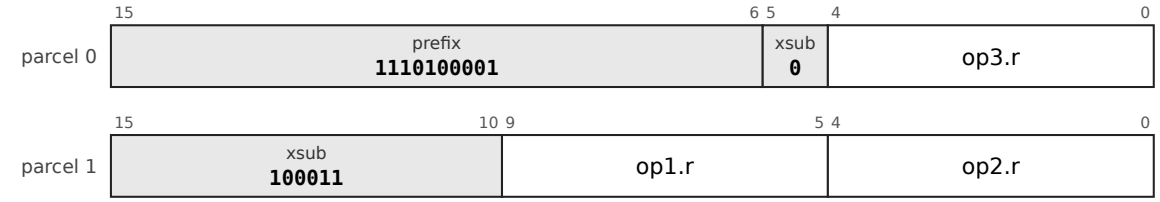
**XANDDB: R3, prefix 1110100001, sub-opcode 0100010; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

### *XOR_*

**XOR_: R3, prefix 1110100001, sub-opcode 0100011; operands reg5, reg5, reg5; 4 bytes**

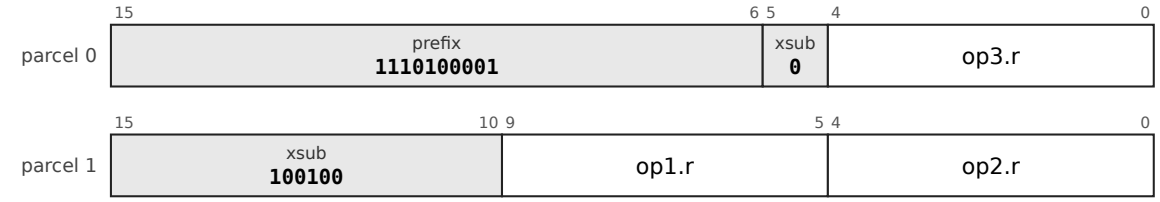



```
XOR_ value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

### *XOR_DB*

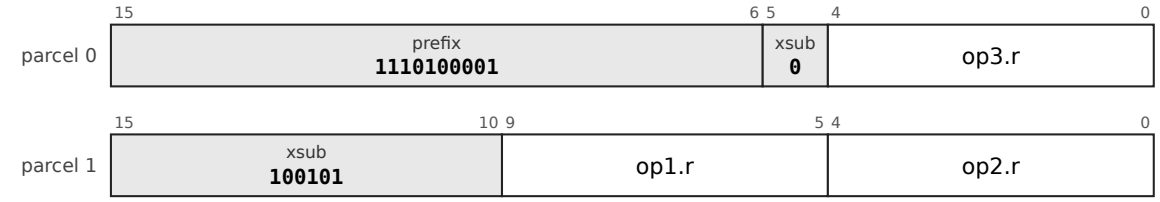
**XOR_DB: R3, prefix 1110100001, sub-opcode 0100100; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

### *XXOR*

**XXOR: R3, prefix 1110100001, sub-opcode 0100101; operands reg5, reg5, reg5; 4 bytes**

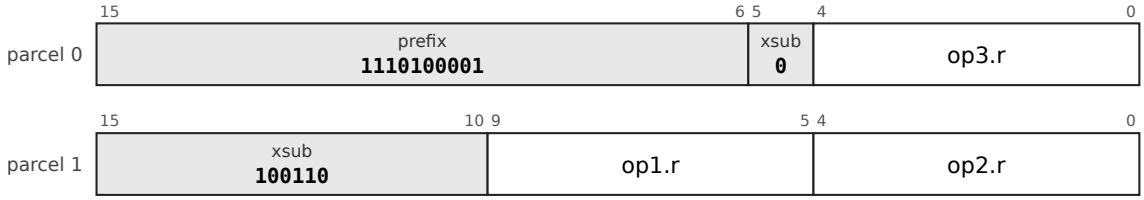



```
XXOR value, (address)
```

Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

### *XXORDB*

**XXORDB: R3, prefix 1110100001, sub-opcode 0100110; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XAND above; 1 layout form, each a chart of its own (above).

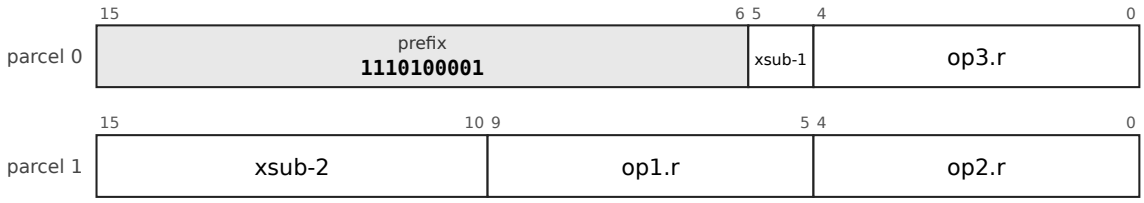
## XMAX: Atomic fetch and minimum or maximum

**Mnemonics:** `XMAX`, `XMAXDB`, `XMIN`, `XMINDB`, `XMAXU`, `XMAXUDB`, `XMINU`, `XMINUDB`

### Syntax

```
XMAX value, (address)
XMIN ..., XMAXU ..., XMINU ...
each with a DB form
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
old ← M[address] ; M[address] ← max(old, value) (XMAX, XMAXU) or min (XMIN, XMINU) ; result ← old
signed for XMAX and XMIN, unsigned for the U forms
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

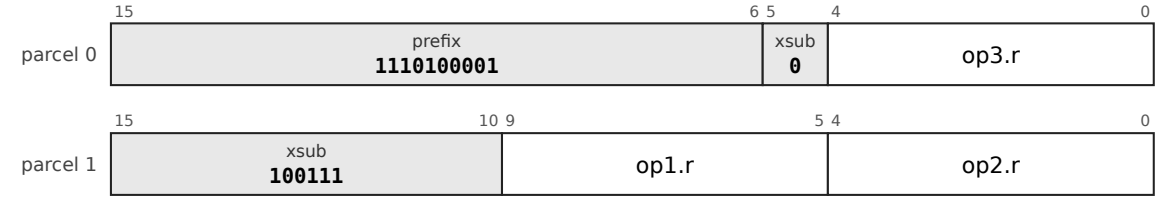
The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for `CAS` and `CASDB`, in the order written) is open (J.9.3).

**Why it is here.** The fetch-and-minimum and maximum atomics, borrowed from LoongArch's richer atomic set: each is a couple of hundred lookup tables and saves a compare-and-swap retry loop.

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

### *XMAX*

**XMAX: R3, prefix 1110100001, sub-opcode 0100111; operands reg5, reg5, reg5; 4 bytes**

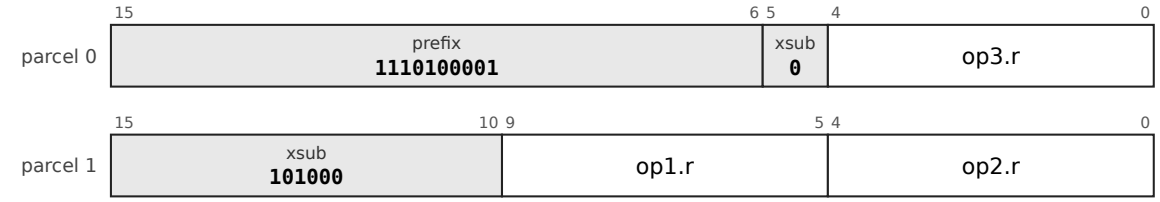



```
XMAX value, (address)
```

Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMAXDB*

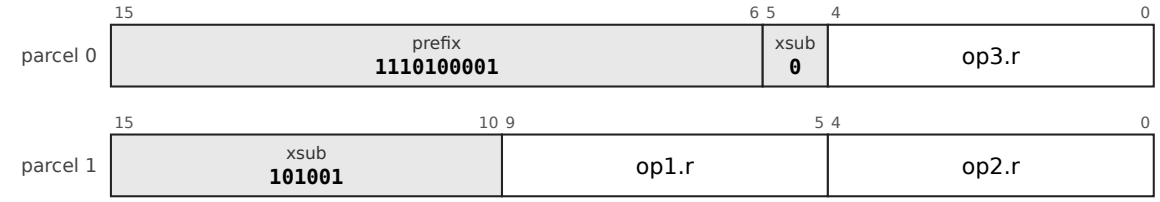
**XMAXDB: R3, prefix 1110100001, sub-opcode 0101000; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMIN*

**XMIN: R3, prefix 1110100001, sub-opcode 0101001; operands reg5, reg5, reg5; 4 bytes**

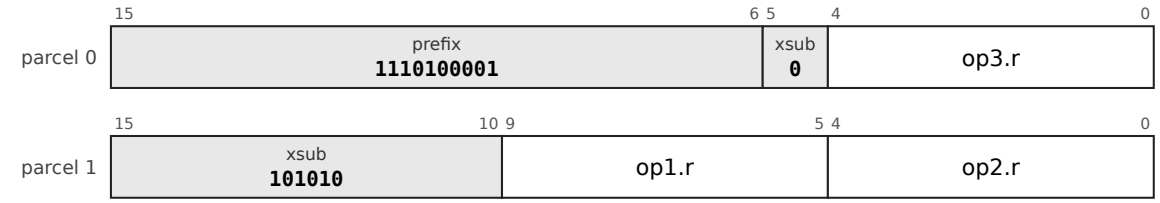



```
XMIN ..., XMAXU ..., XMINU ...
```

Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMINDB*

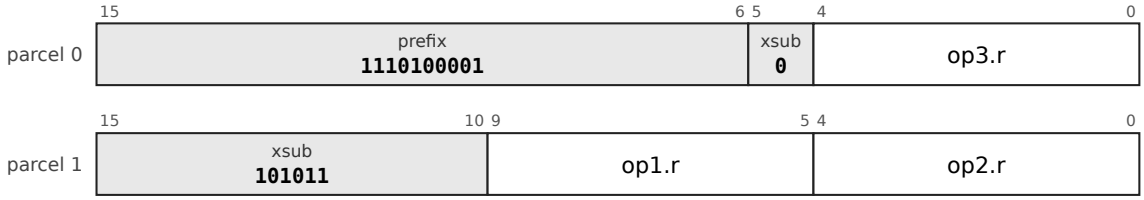
**XMINDB: R3, prefix 1110100001, sub-opcode 0101010; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMAXU*

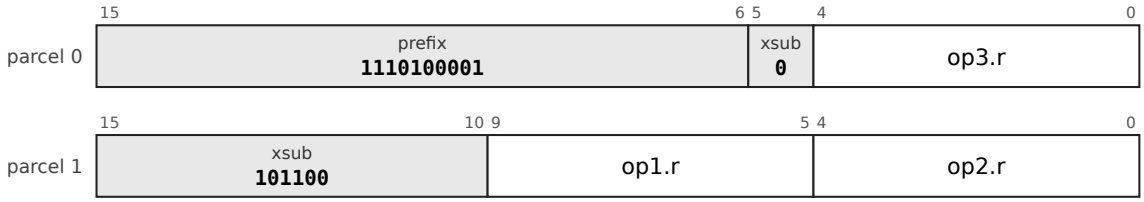
**XMAXU: R3, prefix 1110100001, sub-opcode 0101011; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMAXUDB*

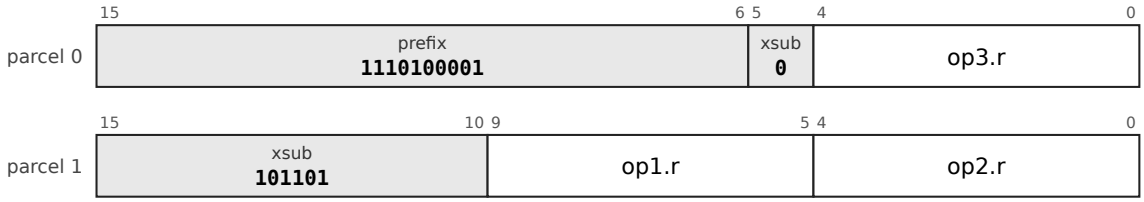
**XMAXUDB: R3, prefix 1110100001, sub-opcode 0101100; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMINU*

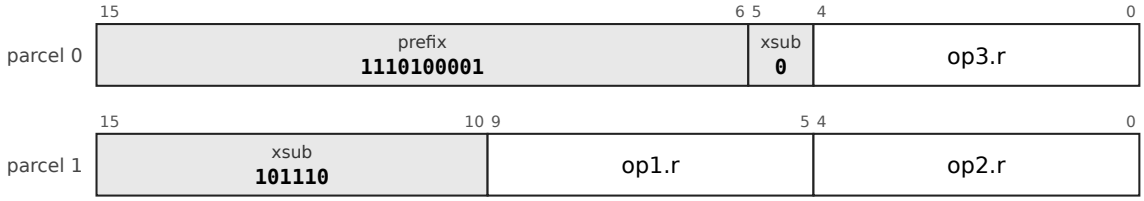
**XMINU: R3, prefix 1110100001, sub-opcode 0101101; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

### *XMINUDB*

**XMINUDB: R3, prefix 1110100001, sub-opcode 0101110; operands reg5, reg5, reg5; 4 bytes**




Operation, flags and exceptions as for the entry XMAX above; 1 layout form, each a chart of its own (above).

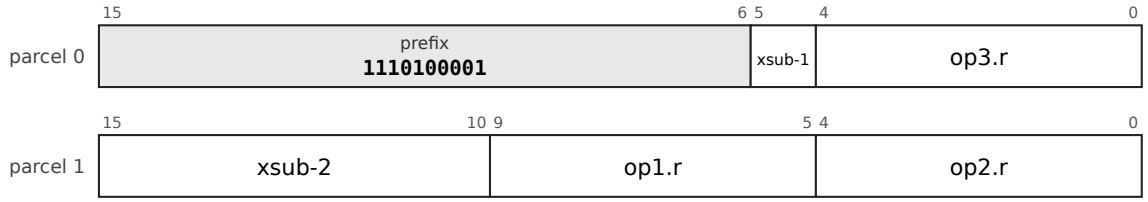
## LLSC: Load-linked and store-conditional

**Mnemonics:** LL, SC

### Syntax

```
LL (address), Rd
SC value, (address), result
```

### Encoding




The chart is the leaf `R3-1`, R3, prefix `1110100001`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `op3` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R3-1`.

### Operation

```
LL: Rd ← M[address] ; take a reservation on the granule containing address
SC: if the reservation is intact then M[address] ← value ; result ← 0 (success)
    else nothing is written ; result ← 1 (failure)
a write to the granule by any initiator, core or device, clears the reservation
```

**Flags** Not specified beyond "none for the read-modify-write forms"; `SC` reports success in its result register.

**Exceptions** Page fault, protection fault, alignment fault (the address must be naturally aligned) or bus error. The access is atomic with respect to every initiator, cores and devices alike.

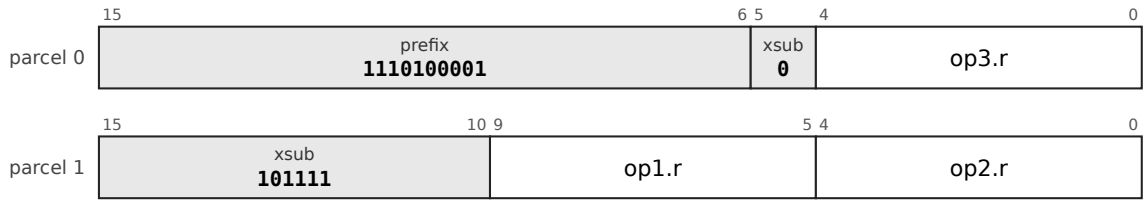
A short `LL`/`SC` sequence must eventually succeed if no other initiator keeps writing the granule (the RISC-V guarantee for constrained sequences). The sub-opcodes and the semantics of this group are specified. The forms below name the roles; the placement of the operands (three registers with the address in a register, four for `CAS` and `CASDB`, in the order written) is open (J.9.3). The result convention of `SC` is part of the open placement (J.9.3).

**Why it is here.** Load-linked and store-conditional, with the reservation a monitor, because a monitor maps onto the exclusive access of AXI and of FPGA and modern memory systems, where the Unibus's locked read-modify-write (`DATIP`/`DATO`) does not scale as cores multiply. A short sequence must eventually succeed if no other initiator keeps writing the granule, the RISC-V rule.

**Cf.** 7.4 (atomic operations: the reservation monitor), 7.3 (the multiprocessor instructions at a glance), 7.5 (the memory model).

***LL***

**LL: R3, prefix 1110100001, sub-opcode 0101111; operands reg5, reg5, reg5; 4 bytes**

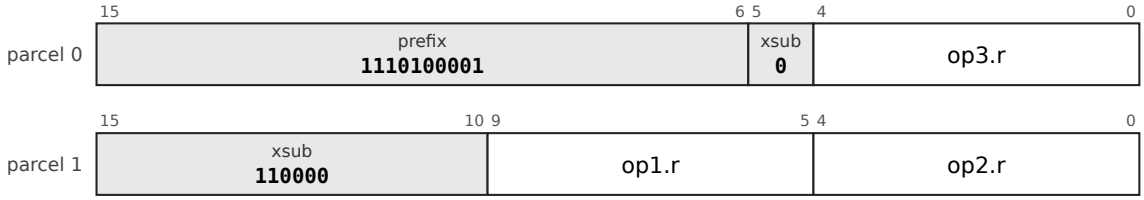



```
LL (address), Rd
```

Operation, flags and exceptions as for the entry LLSC above; 1 layout form, each a chart of its own (above).

### *SC*

**SC: R3, prefix 1110100001, sub-opcode 0110000; operands reg5, reg5, reg5; 4 bytes**




```
SC value, (address), result
```

Operation, flags and exceptions as for the entry LLSC above; 1 layout form, each a chart of its own (above).

## 9.14 System instructions, exceptions and interrupts

*Cf. chapter 6, section 6.13 (system instructions, exceptions and interrupts), section 6.3 (exceptions and interrupts) and section 6.5 (system registers).*

**9.14.1** The system group SYS holds the privileged and system instructions: TRAP, TRAPL, RTI, RTT, RET, HALT, BPT, MTPS, MFPS, SPL, MFSR, MTSR, MTDP, MFDP, MFCPUID, PROBER, PROBEW, PUSHX, POPX, and the translation and cache maintenance instructions (TLBI, TLBIA, TLBSYNC, MTASID, MFASID, ICINV, ICINVA). The user-memory guard instructions STUA and CLUA are described with them.

*Commentary.* The official entry of each instruction is below, in this section; the kernel chapter gives the modes, the entry sequence, the system registers and the reasons. **Impact.** *Kernel:* the entries below are the complete list of what the kernel may execute that user code may not, and of what both may; chapter 6 explains how they are used. *Compiler:* none, apart from TRAP in system-call stubs. *Hardware:* a class decode and one privilege check.

### NOP: No operation and landing pad

**Mnemonics:** NOP, LPAD

#### Syntax

```
NOP
LPAD
```

#### Encoding




The chart is the leaf SYS0-1, SYS0, prefix 11101110010. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

#### Operation

```
nothing
```

**Flags** None written.

**Exceptions** None.

LPAD marks a legal target of an indirect jump or call; no instruction enforces it in ABI version 1.

**Why it is here.** `NOP` is common and has no story; `LPAD` is the landing pad. Landing pads mark the legal targets of indirect jumps and calls so that jump-oriented attacks can be stopped at the transfer, as Intel's indirect-branch tracking and ARM's branch target identification do and OpenBSD enforces. ABI version 1 leaves them off (open: when they become mandatory, J.D.2), and the assembler provides `.lpad`; the kernel pain-points and mitigations reports record why deciding later is more expensive (appendix A, section A.1).
**Cf.** Appendix A, section A.1 (landing pads).

***NOP***

**NOP: SYS0, prefix 11101110010, sub-opcode 00000; operands no operand; 2 bytes**




```
NOP
```

Operation, flags and exceptions as for the entry NOP above; 1 layout form, each a chart of its own (above).

***LPAD***

**LPAD: SYS0, prefix 11101110010, sub-opcode 01001; operands no operand; 2 bytes**




```
LPAD
```

Operation, flags and exceptions as for the entry NOP above; 1 layout form, each a chart of its own (above).

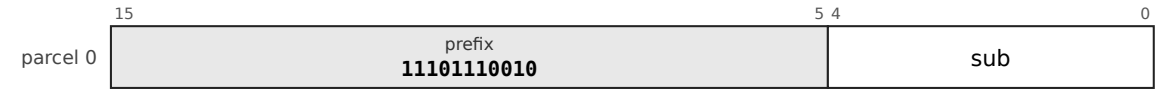
## RTI: Return from interrupt or exception

**Mnemonics:** `RTI`, `RTT`

### Syntax

```
RTI
RTT
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
PC ← EPC ; PSW ← EPSW          ; a PSW with the reserved mode 2 in either mode field aborts
RTT: and then set the trace bit T after the return
```

**Flags** Replaced by `EPSW`, including the flags.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

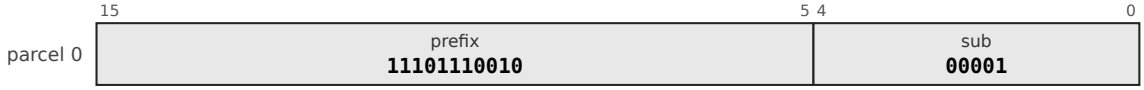
RTT sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

**Why it is here.** RTT looks redundant against RTI, and stays: it sets the trace bit *after* the return rather than before, which is what lets a debugger single-step out of its own trap handler.

**Cf.** 6.3 (exceptions and interrupts), 6.13.

### *RTI*

**RTI: SYS0, prefix 11101110010, sub-opcode 00001; operands no operand; 2 bytes**

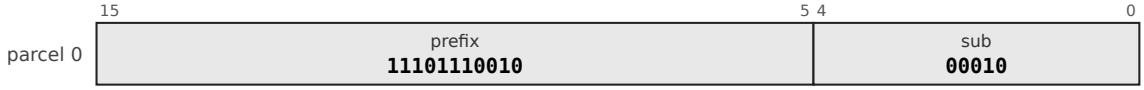



```
RTI
```

Operation, flags and exceptions as for the entry RTI above; 1 layout form, each a chart of its own (above).

### *RTT*

**RTT: SYS0, prefix 11101110010, sub-opcode 00010; operands no operand; 2 bytes**




```
RTT
```

Operation, flags and exceptions as for the entry RTI above; 1 layout form, each a chart of its own (above).

## RET: Return from subroutine

**RET: SYS0, prefix 11101110010, sub-opcode 00011; operands no operand; 2 bytes**




**Mnemonics:** RET

### Syntax

```
RET
```

### Encoding




The chart is the leaf SYS0-1, SYS0, prefix 11101110010. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

### Operation

```
PC ← M[SP] ; SP ← SP + 8
```

**Flags** None written.
**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause BUSFAULT). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.
Not privileged: it is the partner of CALL.

## HALT: Halt the processor

**HALT: SYS0, prefix 11101110010, sub-opcode 00100; operands no operand; 2 bytes**

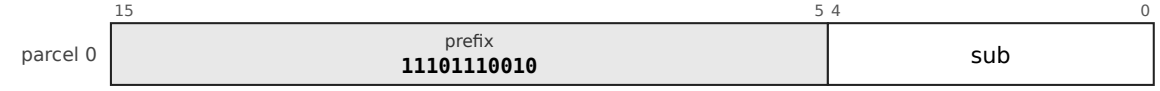



**Mnemonics:** HALT

### Syntax

```
HALT
```

### Encoding




The chart is the leaf SYS0-1, SYS0, prefix 11101110010. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

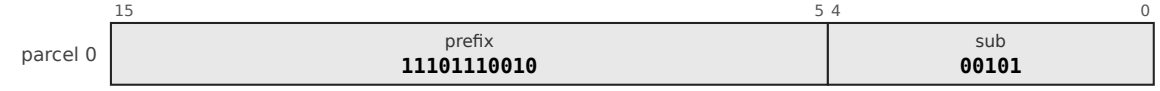
### Operation

```
stop the processor
```

**Flags** None written.
**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.
**Cf.** 6.12.

## BPT: Breakpoint

**BPT: SYS0, prefix 11101110010, sub-opcode 00101; operands no operand; 2 bytes**




**Mnemonics:** BPT

### Syntax

```
BPT
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
take the breakpoint vector with EPC = the address of the next instruction
```

**Flags** None written.

**Exceptions** Enters the kernel by the `BPT` vector.

A trap like any other, but kept as its own two-byte instruction because a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has.

**Why it is here.** `BPT` is a trap vector, but a debugger patches it over an existing instruction, so it must be the shortest instruction the machine has; that is why it is kept although `IOT` and `EMT` are not.

**Cf.** 6.3.

## FENCE: Memory fence

**FENCE: SYS0, prefix 11101110010, sub-opcode 00110; operands no operand; 2 bytes**

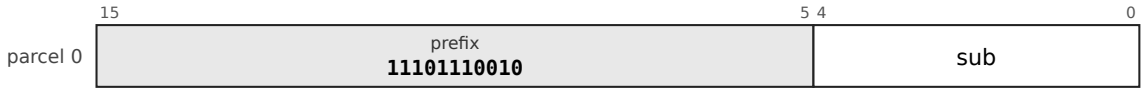



**Mnemonics:** `FENCE`

### Syntax

```
FENCE
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

### Operation

```
order the memory accesses before it against those after it, for every initiator that observes them
```

**Flags** None written.

**Exceptions** None.

Owed to devices (one FENCE separates writing a device's registers from setting its go bit) and to other cores.

**Why it is here.** The memory model is sequentially consistent, so FENCE is needed only for what is outside the model: devices and the framebuffer (one FENCE between writing a device's registers and setting its go bit, section 3.13) and instruction streams. The _DB barrier forms of the atomics exist so that a FENCE is not needed in a lock acquire.

**Cf.** 7.5 (the memory model), 2.2 (what the processor owes the bus).

## WFE: Wait for an event

**WFE: SYS0, prefix 11101110010, sub-opcode 00111; operands no operand; 2 bytes**




**Mnemonics:** WFE (aliases: WAIT)

### Syntax

```
WFE
```

### Encoding




The chart is the leaf SYS0-1, SYS0, prefix 11101110010. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

### Operation

```
sleep until an interrupt is pending or another core executes SEV
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

WAIT is an assembler alias.

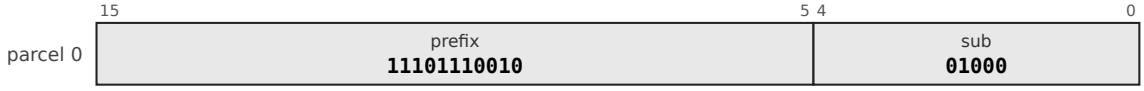
**Why it is here.** WAIT was a PDP-11 instruction (sleep until an interrupt); WFE subsumes it: it sleeps until an interrupt or another core's SEV, and on a uniprocessor it behaves exactly as WAIT did. WFE and SEV belong to the multiprocessor operations (7.10).

**Cf.** 7.10 (waiting and signalling).

**WAIT** is an alias of WFE: the assembler writes it as that word.

## SEV: Send an event

**SEV: SYS0, prefix 11101110010, sub-opcode 01000; operands no operand; 2 bytes**




**Mnemonics:** `SEV`

### Syntax

```
SEV
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
wake every core waiting in WFE
```

**Flags** None written.
**Exceptions** None.
**Why it is here.** The signalling half of `WFE`, so that a core waiting for a lock can sleep instead of spinning and be woken by the core that releases it.
**Cf.** 7.6.

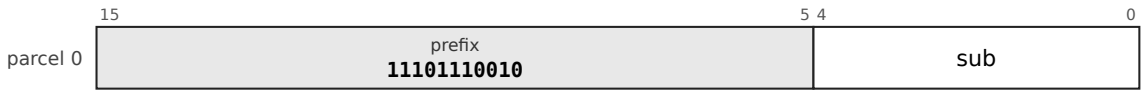
## TLBA: Invalidate all translations; wait for translation invalidations; invalidate the whole instruction cache

**Mnemonics:** `TLBIA`, `TLBSYNC`, `ICINVA`

### Syntax

```
TLBIA
TLBSYNC
ICINVA
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs`

10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
TLBIA   : invalidate every translation-buffer entry on this core and, by broadcast, on the others
TLBSYNC : wait until the invalidations this core started have completed on every core
ICINVA  : invalidate the whole instruction cache
```

**Flags** None written.

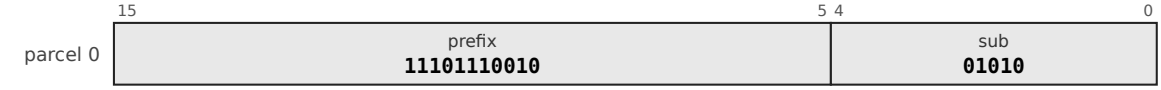
**Exceptions** Privileged (`TLBIA`, `TLBSYNC`)

**Why it is here.** `TLBI` and `TLBIA` broadcast, `TLBSYNC` waits for the other cores, and `ICINV` and `ICINVA` invalidate the instruction cache: a Lisp runtime that compiles a function cannot work without an instruction-cache invalidate.

**Cf.** 7.9 (translation and instruction-cache coherence).

#### *TLBIA*

**TLBIA: SYS0, prefix 11101110010, sub-opcode 01010; operands no operand; 2 bytes**

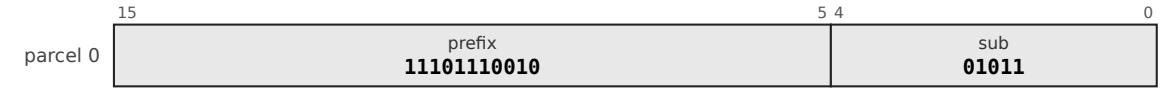



```
TLBIA
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

#### *TLBSYNC*

**TLBSYNC: SYS0, prefix 11101110010, sub-opcode 01011; operands no operand; 2 bytes**

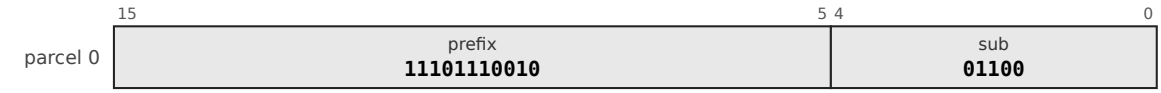



```
TLBSYNC
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

#### *ICINVA*

**ICINVA: SYS0, prefix 11101110010, sub-opcode 01100; operands no operand; 2 bytes**




```
ICINVA
```

Operation, flags and exceptions as for the entry TLBA above; 1 layout form, each a chart of its own (above).

## PUSHX: Push and pop the exception state

**Mnemonics:** `PUSHX`, `POPX`

### Syntax

```
PUSHX
POPX
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
PUSHX: SP ← SP - 16 ; M[SP] ← EPSW ; M[SP + 8] ← EPC
POPX : EPSW ← M[SP] ; EPC ← M[SP + 8] ; SP ← SP + 16
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. `SP` must be 8-byte aligned (a misaligned-access abort); both words are checked before either is touched.

For a handler that wants to be interrupted: it can save the exception state and lower the priority.

**Why it is here.** A handler that can be interrupted must save `EPC` and `EPSW` before it lowers the priority; with only `MFSR` that costs two moves and two stores (sixteen bytes) on every such kernel entry, and the same to restore. The instructions may not show up high in the instruction count, but they save bytes and cycles in a tight, very often used kernel loop.

**Cf.** 6.3, 6.13.

#### *PUSHX*

**PUSHX: SYS0, prefix 11101110010, sub-opcode 01101; operands no operand; 2 bytes**




```
PUSHX
```

Operation, flags and exceptions as for the entry PUSHX above; 1 layout form, each a chart of its own (above).

#### *POPX*

**POPX: SYS0, prefix 11101110010, sub-opcode 01110; operands no operand; 2 bytes**

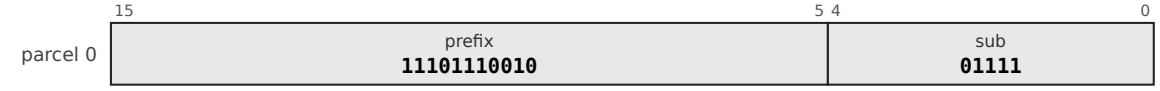



```
POPX
```

Operation, flags and exceptions as for the entry PUSHX above; 1 layout form, each a chart of its own (above).

## ISYNC: Instruction synchronisation

**ISYNC: SYS0, prefix 11101110010, sub-opcode 01111; operands no operand; 2 bytes**




**Mnemonics:** `ISYNC`

### Syntax

```
ISYNC
```

### Encoding




The chart is the leaf `SYS0-1`, SYS0, prefix `11101110010`. `sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYS0-1`.

### Operation

```
make earlier writes to PTBR, ASID, MMUCTL and VBR, and earlier ICINVs, take effect for the instructions that
follow
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A non-zero pad byte is an illegal instruction.

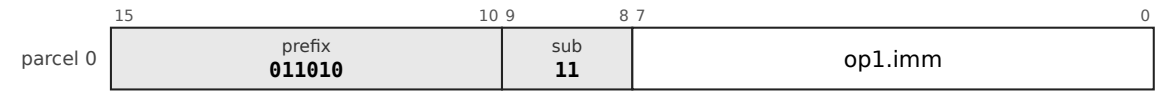
The architecture promises the effect only after the next `ISYNC`; an in-order implementation may make it immediate, and software must not rely on that.

**Why it is here.** Taken from the out-of-order readiness review: it makes earlier writes to `PTBR`, `ASID`, `MMUCTL` and `VBR` take effect for what follows, so an in-order machine may make them immediate and a speculative one has a place to serialise. It serves the aim that the architecture is efficient on both in-order and out-of-order cores (1.4.4).

**Cf.** 7.5, 6.5.

## HINT: Hint

**HINT: SYSI, prefix 011010, sub-opcode 11; operands imm8u; 2 bytes**

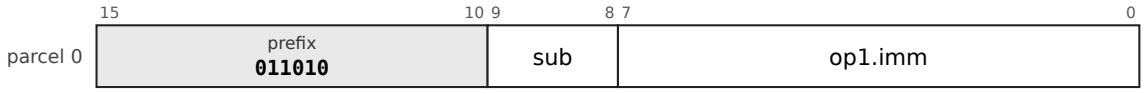



**Mnemonics:** `HINT`

### Syntax

```
HINT #n        ; n = 0 to 255
```

### Encoding




The chart is the leaf `SYSI-1`, SYSI, prefix `011010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSI-1`.

### Operation

```
no architectural effect
```

**Flags** None written.

**Exceptions** None.

Defined codes: 0 a plain no-op hint, 1 pause (spin-wait), 2 speculation barrier. Every other code is a reserved hint that executes as a no-op; software shall not use it. Not privileged.

**Why it is here.** Defined with `ISYNC`: a no-op with a byte of code, defined for `0` (plain), `1` (pause, for spin-wait loops) and `2` (speculation barrier), every other code a reserved hint that executes as a no-op so that a later machine can give it a meaning without breaking an old binary.

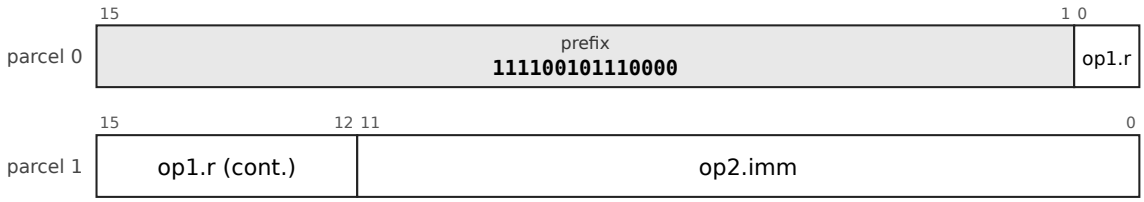
## MFSR: Move from and to a system register

**Mnemonics:** `MFSR`, `MTSR`

### Syntax

```
MFSR Rd, name
MTSR name, Rs
```

### Encoding




The chart is the leaf `MFSR-1`, MFSR, prefix `111100101110000`. `op1` is `reg5`: register, 5 bits. `op2` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `MFSR-1`, `MTSR-1`.

### Operation

```
MFSR: Rd ← system register `name`
MTSR: system register `name` ← Rs        ; read-only registers are an illegal instruction
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. An unassigned system register is an illegal instruction.

The register is chosen by name. A write to `PTBR`, `ASID` or `MMUCTL` flushes the translation buffer.

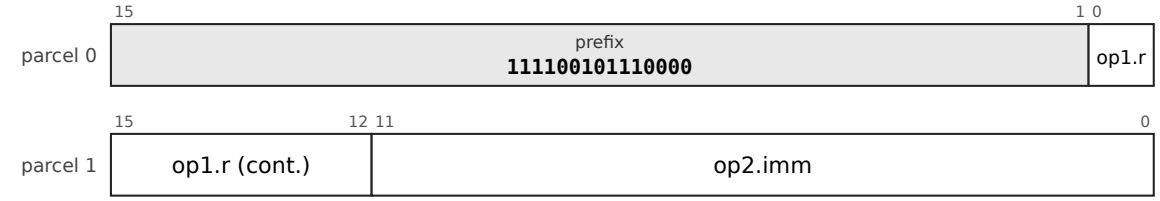
| Name | Number | Access |
|---|---|---|
| PTBR | 0 | read and write |
| EPC | 1 | read and write |
| EVA | 2 | read only |
| ECAUSE | 3 | read only |
| DMW0 | 4 | read and write |
| DMW1 | 5 | read and write |
| USP | 6 | read and write |
| SSP | 7 | read and write |
| EPSW | 8 | read and write |
| VBR | 9 | read and write |
| MMUCTL | 10 | read and write |
| ASID | 11 | read and write |
| CPUID | 12 | read only |

**Why it is here.** A paging operating system must install a page table and learn why it was entered, which needs instructions to write PTBR and read EPC, EVA and ECAUSE. One general pair reaches all of the system registers by number. The user and supervisor stack pointers are in the set because any kernel must read and write the user stack pointer to build a signal frame, copy a process or switch context.

**Cf.** 6.5 (system registers).

***MFSR***

**MFSR: MFSR, prefix 111100101110000; operands reg5, imm12u; 4 bytes**

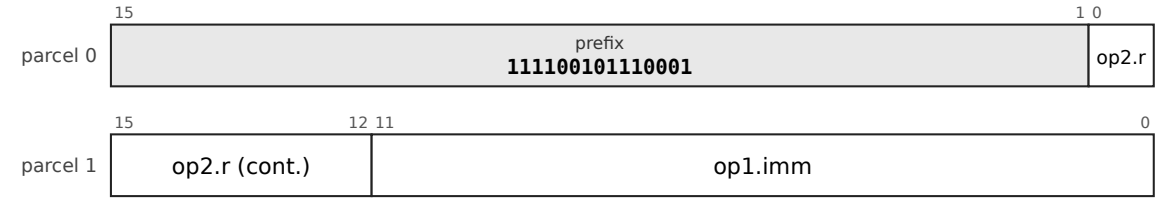



```
MFSR Rd, name
```

Operation, flags and exceptions as for the entry MFSR above; 1 layout form, each a chart of its own (above).

***MTSR***

**MTSR: MTSR, prefix 111100101110001; operands imm12u, reg5; 4 bytes**




```
MTSR name, Rs
```

Operation, flags and exceptions as for the entry MFSR above; 1 layout form, each a chart of its own (above).

## MTPS: Move to and from the processor status word

**Mnemonics:** MTPS, MFPS

### Syntax

```
MTPS Rs
MFPS Rd
```

### Encoding




The chart is the leaf `SYSR-1`, SYSR, prefix `11000100`. `op1` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-1`.

### Operation

```
MTPS: PSW ← Rs          ; in user mode only the flags N Z V C are written
MFPS: Rd ← PSW
```

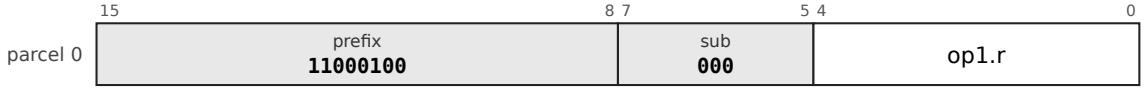
**Flags** `MTPS` writes the flags from `Rs`; `MFPS` writes none.

**Exceptions** A `MTPS` value with the reserved mode 2 in a mode field aborts.

**Cf.** 6.2 (modes and the status word).

#### *MTPS*

**MTPS: SYSR, prefix 11000100, sub-opcode 000; operands reg5; 2 bytes**




```
MTPS Rs
```

Operation, flags and exceptions as for the entry MTPS above; 1 layout form, each a chart of its own (above).

#### *MFPS*

**MFPS: SYSR, prefix 11000100, sub-opcode 001; operands reg5; 2 bytes**

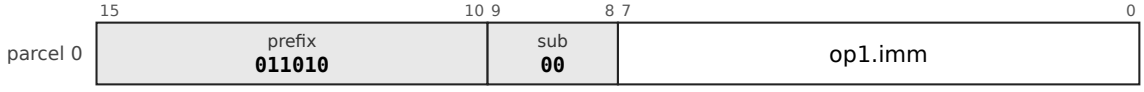



```
MFPS Rd
```

Operation, flags and exceptions as for the entry MTPS above; 1 layout form, each a chart of its own (above).

## SPL: Set the interrupt priority level

**SPL: SYSI, prefix 011010, sub-opcode 00; operands imm8u; 2 bytes**

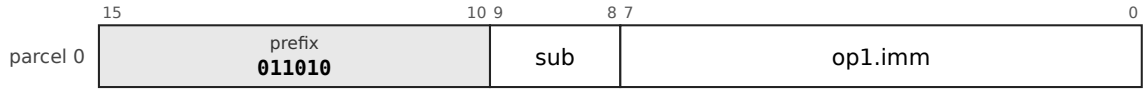



**Mnemonics:** `SPL`

### Syntax

```
SPL #n        ; n = 0 to 7
```

### Encoding




The chart is the leaf `SYSI-1`, SYSI, prefix `011010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSI-1`.

### Operation

```
PSW.priority ← n
```

**Flags** None written. The condition flags are untouched.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector. A priority above 7 is an illegal instruction.

Kept despite `MTPS` writing the whole word because it sets the priority alone: a kernel raising priority around a critical section must not clobber the flags.

**Why it is here.** `SPL` stays although `MTPS` writes the whole status word: it sets the priority alone, leaving the condition codes untouched, so a kernel raising priority around a critical section does not clobber flags. It is the PDP-11 instruction of the same name.

**Cf.** 6.3.

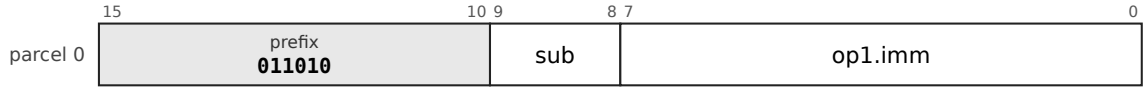
## TRAP: System call and trap

**Mnemonics:** `TRAP`, `TRAPL`

### Syntax

```
TRAP #n        ; n = 0 to 15, two bytes
TRAPL #n       ; n = 0 to 255, four bytes
```

### Encoding




The chart is the leaf `SYSI-1`, SYSI, prefix `011010`. `op1` is `imm8u`: unsigned 8-bit immediate, 0 to 255. `sub` (2 bits) selects the mnemonic: `spl` 00, `trap` 01, `trapl` 10, `hint` 11.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSI-1`.

### Operation

```
enter the kernel by vector (TRAP_BASE + n), with EPC = the address of the next instruction
```

**Flags** None written.

**Exceptions** Enters the kernel; not privileged.

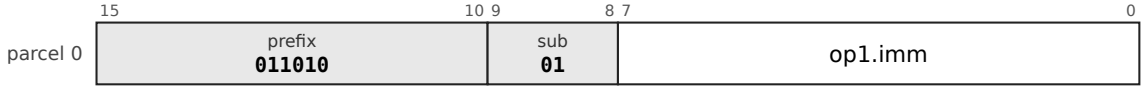
The replacement for the PDP-11 `IOT` and `EMT`. In these encodings `TRAP` and `TRAPL` are one word with an 8-bit vector number (`docs/encoding-proposal.md`).

**Why it is here.** In the original encoding `TRAPL` existed so that the vector range of 256 and the two-byte common case could both be kept (`TRAP #n` for `n` below 16 is two bytes, and the long form reaches 256); here one word carries an 8-bit vector number. `IOT` and `EMT`, which were `TRAP` at two particular vectors on the PDP-11, are not provided.

**Cf.** 6.3, 6.10 (system calls).

### *TRAP*

**TRAP: SYSI, prefix 011010, sub-opcode 01; operands imm8u; 2 bytes**

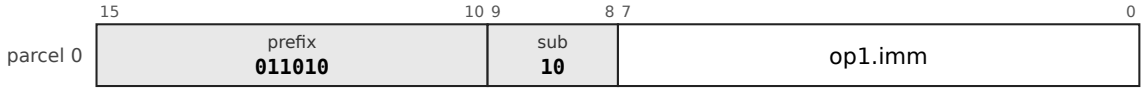



```
TRAP #n
```

Operation, flags and exceptions as for the entry TRAP above; 1 layout form, each a chart of its own (above).

### *TRAPL*

**TRAPL: SYSI, prefix 011010, sub-opcode 10; operands imm8u; 2 bytes**

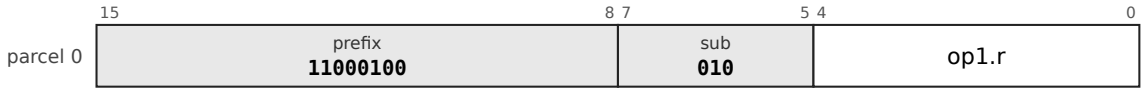



```
TRAPL #n
```

Operation, flags and exceptions as for the entry TRAP above; 1 layout form, each a chart of its own (above).

## TLBI: Invalidate a translation

**TLBI: SYSR, prefix 11000100, sub-opcode 010; operands reg5; 2 bytes**

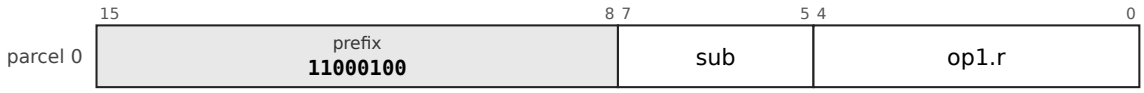



**Mnemonics:** `TLBI`

### Syntax

```
TLBI Rs
```

### Encoding




The chart is the leaf `SYSR-1`, SYSR, prefix `11000100`. `op1` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-1`.

### Operation

```
invalidate the translation-buffer entry for the page containing the address in Rs, on this core and by
broadcast on the others
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

**Why it is here.** It broadcasts: a translation changed by one core must be invalidated on all of them, and `TLBSYNC` waits for it.

**Cf.** 7.5, 6.8.

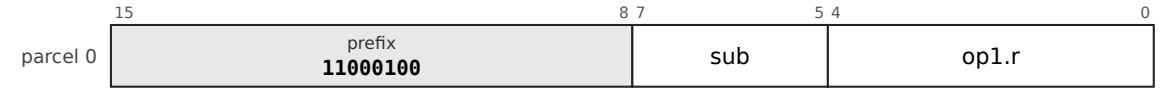
## MTASID: Move to and from the address-space identifier

**Mnemonics:** `MTASID`, `MFASID`

### Syntax

```
MTASID Rs
MFASID Rd
```

### Encoding




The chart is the leaf `SYSR-1`, SYSR, prefix `11000100`. `op1` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-1`.

### Operation

```
MTASID: ASID ← Rs[15:0]
MFASID: Rd ← ASID
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

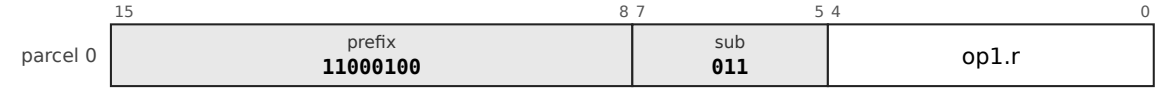
The `ASID` tags translation-buffer entries so a context switch is one store.

**Why it is here.** The `ASID` tags translation-buffer entries so a context switch is one store and no flush. No further reason is recorded.

**Cf.** 6.8.

#### *MTASID*

**MTASID: SYSR, prefix 11000100, sub-opcode 011; operands reg5; 2 bytes**

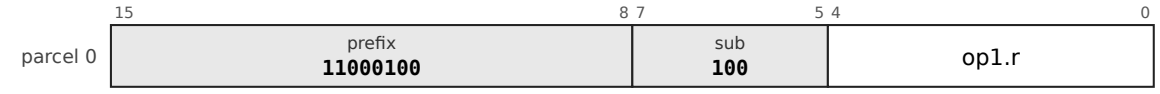



```
MTASID Rs
```

Operation, flags and exceptions as for the entry MTASID above; 1 layout form, each a chart of its own (above).

#### *MFASID*

**MFASID: SYSR, prefix 11000100, sub-opcode 100; operands reg5; 2 bytes**




```
MFASID Rd
```

Operation, flags and exceptions as for the entry MTASID above; 1 layout form, each a chart of its own (above).

## MTDP: Move to and from the direct pointer

**Mnemonics:** `MTDP`, `MFDP`

### Syntax

```
MTDP Rs
MFDP Rd
```

### Encoding




The chart is the leaf `SYSR-1`, SYSR, prefix `11000100`. `op1` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-1`.

### Operation

```
MTDP: DP ← Rs
MFDP: Rd ← DP
```

**Flags** None written.

**Exceptions** None.

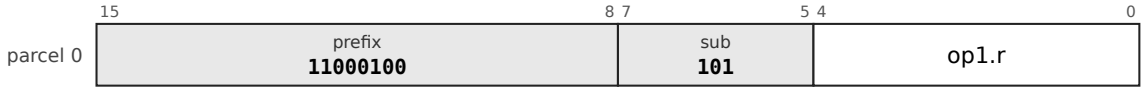
`DP` is the thread pointer in user mode and the per-CPU pointer in kernel mode; there is one copy per mode.

**Why it is here.** `DP` descends from the 6502's zero page and is kept for code density (a short displacement from `DP`); it is also the thread pointer in user mode and, in kernel mode, the per-CPU pointer that an SMP kernel wants (7.8). `MTDP` and `MFDP` read and write the current mode's copy.

**Cf.** 7.8 (per-CPU state).

#### *MTDP*

**MTDP: SYSR, prefix 11000100, sub-opcode 101; operands reg5; 2 bytes**

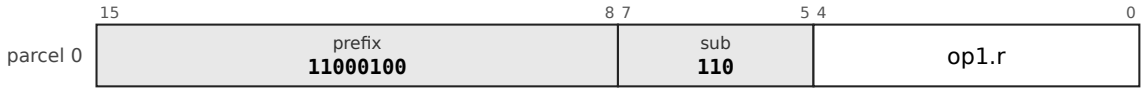



```
MTDP Rs
```

Operation, flags and exceptions as for the entry MTDP above; 1 layout form, each a chart of its own (above).

#### *MFDP*

**MFDP: SYSR, prefix 11000100, sub-opcode 110; operands reg5; 2 bytes**

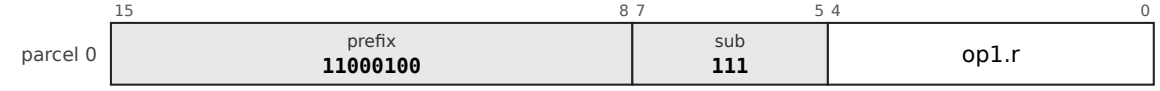



```
MFDP Rd
```

Operation, flags and exceptions as for the entry MTDP above; 1 layout form, each a chart of its own (above).

## MTDMW: Move to a direct-mapping window

**MTDMW: SYSR, prefix 11000100, sub-opcode 111; operands reg5; 2 bytes**

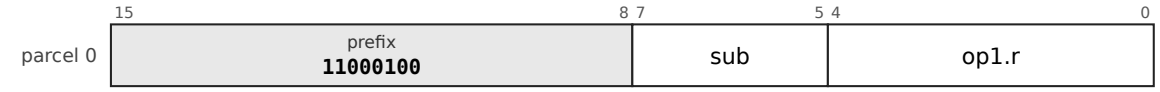



**Mnemonics:** `MTDMW`

### Syntax

```
MTDMW Rs
```

### Encoding




The chart is the leaf `SYSR-1`, SYSR, prefix `11000100`. `op1` is `reg5`: register, 5 bits. `sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-1`.

### Operation

```
DMW0 ← Rs          ; stored; it has no effect on translation
```

**Flags** None written.

**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.

The direct-mapping windows are stored and have no effect on translation.

**Why it is here.** Taken from LoongArch's direct-mapping windows, which stop the kernel taking translation-buffer misses on its own text. The instruction is storage only: the windows have no effect on translation.

**Cf.** 6.9.

## MFCPUID: Read the processor identifier

**MFCPUID: SYSR, prefix 1110100000, sub-opcode 0; operands reg5; 2 bytes**

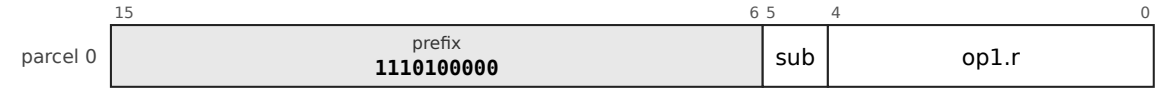



**Mnemonics:** `MFCPUID`

### Syntax

```
MFCPUID Rd
```

### Encoding




The chart is the leaf `SYSR-2`, SYSR, prefix `1110100000`. `op1` is `reg5`: register, 5 bits. `sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-2`.

### Operation

```
Rd ← CPUID
```

**Flags** None written.
**Exceptions** None.
The index of this core, for per-CPU data.
**Why it is here.** Part of the multiprocessor operations, because per-CPU data needs each core to know which one it is.
**Cf.** 7.8, 7.11.

## ICINV: Invalidate the instruction cache for an address

**ICINV: SYSR, prefix 1110100000, sub-opcode 1; operands reg5; 2 bytes**

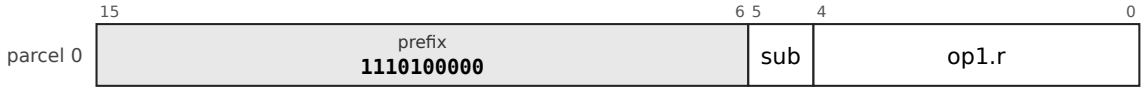



**Mnemonics:** `ICINV`

### Syntax

```
ICINV Rs
```

### Encoding




The chart is the leaf `SYSR-2`, SYSR, prefix `1110100000`. `op1` is `reg5`: register, 5 bits. `sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `SYSR-2`.

### Operation

```
invalidate the instruction-cache line containing the address in Rs
```

**Flags** None written.
**Exceptions** Privileged, as for the other cache-maintenance instructions.
**Why it is here.** Instruction-cache invalidation, by address (`ICINV`) and whole (`ICINVA`). The reason is the one given for `TLBA`: a runtime that compiles a function cannot work without it.
**Cf.** 7.5.

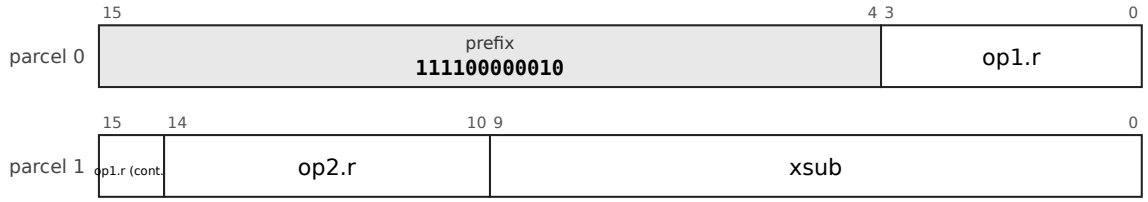
## PROBE: Probe an address for read or write access

**Mnemonics:** `PROBER`, `PROBEW`

### Syntax

```
PROBER Rd, Rs
PROBEW Rd, Rs
```

### Encoding




The chart is the leaf `R2-1`, R2, prefix `111100000010`. `op1` is `reg5`: register, 5 bits. `op2` is `reg5`: register, 5 bits. `xsub` selects the mnemonic.
**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): `R2-1`.

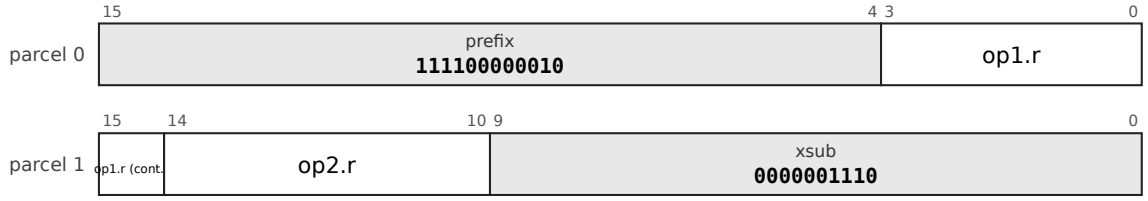
### Operation

```
Rd ← 1 if the address in Rs is readable (PROBER) or writable (PROBEW) in the previous mode, else 0
no fault is taken
```

**Flags** None written.
**Exceptions** Privileged: outside kernel mode the instruction aborts with the privileged-instruction vector.
Taken from the VAX `PROBER` and `PROBEW`: the kernel validates a user address before using it.
**Why it is here.** Taken from the VAX `PROBER` and `PROBEW`: the kernel validates a user address before using it, and it replaces the `nofault` global of the V7 kernel for the address-validation case.
**Cf.** 6.10 (the user-memory guard), 6.6.

#### *PROBER*

**PROBER: R2, prefix 111100000010, sub-opcode 0000001110; operands reg5, reg5; 4 bytes**

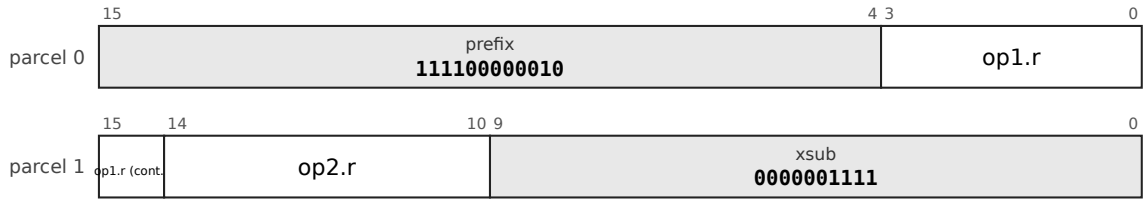



```
PROBER Rd, Rs
```

Operation, flags and exceptions as for the entry PROBE above; 1 layout form, each a chart of its own (above).

#### *PROBEW*

**PROBEW: R2, prefix 111100000010, sub-opcode 0000001111; operands reg5, reg5; 4 bytes**




```
PROBEW Rd, Rs
```

Operation, flags and exceptions as for the entry PROBE above; 1 layout form, each a chart of its own (above).

## TRAPCC: Trap on condition

**Mnemonics:** TRAPCC.NE, TRAPCC.EQ, TRAPCC.PL, TRAPCC.MI, TRAPCC.VC, TRAPCC.VS, TRAPCC.CC, TRAPCC.CS, TRAPCC.GE, TRAPCC.LT, TRAPCC.GT, TRAPCC.LE, TRAPCC.HI, TRAPCC.LOS

### Syntax

```
TRAPCC.cc          ; one word for each of the 14 conditions: TRAPCC.EQ, TRAPCC.VS, TRAPCC.HI ...
```

### Encoding




The chart is the leaf SYS0-1, SYS0, prefix 11101110010. sub (5 bits) selects the mnemonic: nop 00000, rti 00001, rtt 00010, ret 00011, halt 00100, bpt 00101, fence 00110, wfe 00111, sev 01000, lpad 01001, tlbia 01010, tlbsync 01011, icinva 01100, pushx 01101, popx 01110, isync 01111, trapcc.ne 10000, trapcc.eq 10001, trapcc.pl 10010, trapcc.mi 10011, trapcc.vc 10100, trapcc.vs 10101, trapcc.cc 10110, trapcc.cs 10111, trapcc.ge 11000, trapcc.lt 11001, trapcc.gt 11010, trapcc.le 11011, trapcc.hi 11100, trapcc.los 11101, syspg0 11110, syspg1 11111.

**All the layouts of this entry** (a chart for each, with its fields and the sub-opcode of each mnemonic, in the bit layouts of every opcode): SYS0-1.

### Operation

```
if condition cc holds on the flags then take the CHECK trap
    EPC ← address of the next instruction ; EVA ← address of this instruction
```

**Flags** None written.

**Exceptions** The CHECK trap, when the condition holds. A *trap*, not an abort: the instruction completes, EPC is the next instruction, and RTI resumes after the check.

*(D)* Errata 85 and 86: TRAPCC is a trap and not an abort, there is no compare-and-trap form, and the fused ADDV, SUBV and MULV carry the register-only cases. There is one word for each of the 14 conditions, in two bytes with no operand: the condition numbers are those of the branches (ne 1 ... los 14; al is not a word). After ADD the check for signed overflow is TRAPCC.VS (docs/study-proposals.md item 9 calls it ADD; TRAPV). EVA is the address of the TRAPCC itself, the one trap that defines it, so a handler need not decode backwards from EPC. Open: the number of the CHECK vector (J.9.4).

**Why it is here.** A check that costs two bytes and no branch. The comparison with the alternatives is in the study (docs/study-proposals.md item 9): ADD #1, 8(R2) and a check is four plus two bytes and the add keeps its immediate and memory forms, where ADD and a BVS to a shared stub is six bytes and two instructions; and the same instruction checks what is not overflow (a bounds check after CMP, unsigned overflow after ADD). The OpenBSD kernels of i386, hppa and sparc64 already deliver integer-overflow traps as SIGFPE (FPE_INTOVF), so the signal route exists. The author made it a trap and not an abort (errata 85 and 86), so a handler that promotes a fixnum to a bignum can repair the register and resume; the one case where a trap defines EVA.

**Cf.** 6.4 (traps and aborts), docs/study-proposals.md item 9.

#### *TRAPCC.NE*

**TRAPCC.NE: SYS0, prefix 11101110010, sub-opcode 10000; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.EQ*

**TRAPCC.EQ: SYS0, prefix 11101110010, sub-opcode 10001; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.PL*

**TRAPCC.PL: SYS0, prefix 11101110010, sub-opcode 10010; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.MI*

**TRAPCC.MI: SYS0, prefix 11101110010, sub-opcode 10011; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.VC*

**TRAPCC.VC: SYS0, prefix 11101110010, sub-opcode 10100; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.VS*

**TRAPCC.VS: SYS0, prefix 11101110010, sub-opcode 10101; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.CC*

**TRAPCC.CC: SYS0, prefix 11101110010, sub-opcode 10110; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.CS*

**TRAPCC.CS: SYS0, prefix 11101110010, sub-opcode 10111; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.GE*

**TRAPCC.GE: SYS0, prefix 11101110010, sub-opcode 11000; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.LT*

**TRAPCC.LT: SYS0, prefix 11101110010, sub-opcode 11001; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.GT*

**TRAPCC.GT: SYS0, prefix 11101110010, sub-opcode 11010; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.LE*

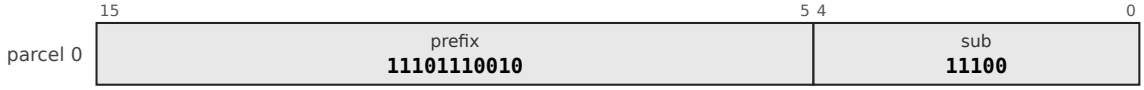
**TRAPCC.LE: SYS0, prefix 11101110010, sub-opcode 11011; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

### *TRAPCC.HI*

**TRAPCC.HI: SYS0, prefix 11101110010, sub-opcode 11100; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

***TRAPCC.LOS***

**TRAPCC.LOS: SYS0, prefix 11101110010, sub-opcode 11101; operands no operand; 2 bytes**




Operation, flags and exceptions as for the entry TRAPCC above; 1 layout form, each a chart of its own (above).

## 9.15 Faults, restart and memory management

*Cf. chapter 6, section 6.4 (faults and restart) and sections 6.6 to 6.9 (memory management).*

**9.15.1** A fault is a trap, an abort, a progress stop in a block operation or a machine check, and every instruction either completes or is annulled and restarted with no state to undo (`MCPY` and its kin keep their progress in registers); the page-table format, the memory types and the translation registers are in the kernel chapter. *(D)*

**9.15.2** A fused instruction (`MOVCALL`, `POPRET`, `POPMRET`, `MOV2`) is restarted whole after a fault: no part of it is visible as a separate instruction. *(D)*

*Commentary.* The instruction-set consequence is the restart rule: the register update of an auto-increment mode is held until the instruction retires. **Impact.** *Kernel:* see 6.4. *Compiler:* no scheduling restriction arises from faults. *Hardware:* a single-entry undo buffer and a read-only walker.

## 9.16 Position-independent code

**9.16.1** Position-independent code costs nothing extra on this machine: every access to code or data in the same module is PC-relative by an ordinary addressing mode, so there is no GOT register and no prologue to set one up. *(D)*

**9.16.2** A call to an imported function is `CALLI got_slot(PC)` and a PLT stub is the single instruction `JMPI got_slot(PC)`; an imported variable's address is one `MOVQ got_slot(PC), Rd`. *(D)*

**9.16.3** A position-independent access to a global is no longer than an absolute one: four bytes within 32 KB and six within 2 GB, against six for a 32-bit absolute address. *(D)*

**9.16.4** The thread pointer is `DP`, so the local-exec thread-local model is one six-byte `MOVQ tls_offset(DP), Rd`. *(D)*

**9.16.5** A six-byte branch reaches ±4 GiB, so no linker veneers are needed for any binary smaller than that; the assembler's and the linker's branch relaxation both reduce to choosing a length. *(D)*

**9.16.6** `R12` is the linker's scratch register. The ABI (appendix D) decides that imports are called without a PLT stub. *(D)* Open: lazy against eager binding (J.D.2).

*Commentary.* **Precedent.** On i386 a position-independent executable reserved `%ebx` (one of eight registers) for the GOT base and set it in every prologue, which cost a few percent of size and speed; x86-64 fixed that with RIP-relative addressing, and AArch64 needs linker veneers because its branches reach only 128 MB. **Trade-offs.** The two-byte PC-relative form is the shortest, so position-independent data access is slightly smaller than absolute access; the price is a GOT load per imported call site where a PLT stub would cost a load per function. **Improvements.** Position-independence is a side effect of three decisions made for other reasons: PC-relative as an ordinary mode (adopted because the PC left the register file), variable-length encoding (for density) and `DP` (a 6502 zero-page descendant kept for density), which is exactly the thread pointer. **Benefits.** OpenBSD builds everything position-independent by default, and here that costs nothing. **Modern comparables.** x86-64 RIP-relative addressing; AArch64 `ADRP` with a GOT load and veneers for far calls. **Impact.** *Kernel:* a loader applies only relocations for data pointers, and write-xor-execute and randomisation come with no code penalty. *Compiler:* `-fPIE` is the default and emits the same instructions as `-fno-pic` for local data; no reserved register, no per-function setup. *Hardware:* nothing; the PC-relative mode is in the address adder already.

## 9.17 What was rejected

**9.17.1** The architecture refuses, by design, every feature that adds hidden state, an unbounded instruction, a second operand path or a mode bit that changes what other instructions mean. *(D)*

**9.17.2** From the PDP-11 and VAX it refuses general memory-to-memory operands, deferred addressing as a general form (only `JMPI` and `CALLI` are deferred), unbounded instructions with hidden microstate (`MOVC5`, `EDITPC`), packed decimal, hardware

call frames (CALLS, CALLG), the PC as a general register, multiple floating-point formats, POLY, the IOT, EMT, WAIT and RESET instructions, a distinct low-half unsigned multiply, and the PDP-11's compare operand order. *(D)*

**9.17.3** From x86 it refuses partial-register writes that preserve upper bits, operand-size and REX prefixes, segmentation, flag writes that vary by instruction, implicit registers for multiply, divide and shift count, the x87 stack, transcendental instructions and byte-granular instruction fetch. *(D)*

**9.17.4** From the RISC family it refuses a register hardwired to zero (a thirty-second of a 32-register file), fused compare-and-branch in place of flags, branch delay slots, register windows and software-refilled translation buffers. *(D)*

**9.17.5** It also refuses features of its own invention: NUL-terminated string instructions (MSCN with an explicit maximum gives a bounded `strlen`), CRC32 words (carry-less multiply is the primitive under every CRC), reciprocal-estimate instructions, NaN payload propagation, a 32-bit profile, an eight-byte instruction alignment, MOVT and MOVK partial-immediate inserts, hardware tag checking, store-with-write-barrier, a self-synchronising encoding, and a scalar instruction for sub-1-bit inference (the accelerator of chapter 8 does it). *(D)*

*Commentary.* **Precedent.** The VAX's EDITPC is a bytecode interpreter in microcode, and DEC needed a separate control chip for the PDP-11's commercial instruction set and shipped none; the VAX's hardware call frames ran slower than the equivalent sequence on later models; ARM32's PC+8 and MIPS's delay slot expose one implementation's pipeline forever. **Trade-offs.** Each refusal gives up a real convenience: the lost `MOV (R1)+,(R2)+` copy loop, the byte-wide ADDB, the inline-argument JSR. Section 9.18 says what replaces each. **Improvements.** The common pattern is that a refused feature was either subsumed by a more general one (MCPY for the copy loop) or paid its cost in every implementation for a few users (EDITPC). **Benefits.** An instruction set a small in-order core can implement completely and a compiler can target without exceptions. **Modern comparables.** RISC-V and AArch64 also refuse memory-to-memory operands and delay slots; AArch64 keeps flags where RISC-V refuses them. **Impact.** *Kernel:* no instruction has a restart state except the three that keep their progress in architectural registers, so a fault is always restartable. *Compiler:* no special cases: every operand rule is uniform, one memory operand and the rest registers. *Hardware:* no microcode engine, no second operand path, no hidden-state latch, no alignment rotator in the fetch path.

## 9.18 Idioms: restored, lost and replaced

**9.18.1** Several idioms that the PDP-11 and VAX had and later RISC designs dropped are restored: subtract-one-and-branch (SOB, here in both directions), probe an address without faulting (PROBER, PROBEW), bit-field extract and insert, find-first-set (CTZ, CLZ), byte swap, block move, fill, compare and scan with their progress held in registers, the register-save mask (PUSHM, POPM, the useful half of CALLS), and carry-less multiply. *(D)*

**9.18.2** The VAX's counted-string instructions are largely present: MOVC3 is MCPY, CMPC3 is MCMP, LOCC is MSCN, SKPC is MSCNN, and MOVC5 is MCPY plus MSET. Table-translated move, table scans and substring search are absent because a per-byte table lookup stops a block operation from streaming. *(D)*

**9.18.3** Deliberately lost, with their replacements: memory-to-memory MOV (replaced by MCPY); autoincrement-deferred and the dispatch-table idiom (replaced by JTAG for tag dispatch and a scaled-index JMP for the rest); `JSR Rn` and `RTS Rn` with inline arguments and cheap coroutines (replaced by CALL and RET with the return address on the stack where every unwinder and debugger expects it); the VAX CASE and INDEX instructions (replaced by a compare, branch and scaled JMP); byte and word ALU operations (MOVZB, the operation, MOVB: three instructions where the PDP-11 had one); and the flag set and clear instructions. *(D)*

**9.18.4** Subsumed, so not losses: SXT (the conversion moves), SWAB (BSWAP), MARK (the convention it served is gone), ASH and ASHC (the shifts take a register count), MFPI and MTPI (PROBER with the previous-mode bits and the user-memory guard), MULU (identical to MUL in the low half), and ROL (an assembler alias for ROR). *(D)*

**9.18.5** Measurements decide what is included, and the compiler uses more of the set than the first compiler did: `SET cc, rd`, ABS, MIN and MAX (the last three assembler aliases, 9.7.6), the bit-field instructions, the strided auto-increment modes and the scaled index are all emitted, and the fused pairs are 19,786 instructions of the corpus. *(D)*

**9.18.6** The compiler fuses a move before a call (MOVCALL), a pop before a return (POPRET, POPMRET) and two register moves (MOV2); the pairs are matched by dynamic programming after register allocation, two bytes saved a pair, 39,674 bytes (4.16%) in the corpus when fusion was first added. The mask of a POPMRET names R8 to R15, so a save of R24 to R31 does not fuse.

**9.18.7** A dense `switch` is a jump table with eight-byte entries placed after the return (the analogue of the VAX `CASEL`); any other `switch` is a binary search, which costs 0.27% in static size and takes the opcode dispatch of Lua from 225.1 million to 148.7 million instructions executed. The jump tables are 3.4% to 5.0% fewer cycles on Lua in the timing model and 806 bytes smaller.

**9.18.8** Signed division by a power of two, and a pointer difference, are shifts; signed division and remainder of an `int` by a constant are a multiply and a shift (2 to 6 and 6 to 10 bytes longer than the plain code, 30% to 67% fewer cycles in micro-benchmarks). Unsigned and 64-bit division by a constant needs a `MULH`-based tree and is not done.

**9.18.9** An array index is scaled by the addressing mode and not by a shift and an add, including an index that is converted from a narrower type first (the inner loop of the `bs_ec` benchmark, 31% fewer instructions); a walk down an array is a strided post-increment; multiplication by 5 or 9 is a `LEA`.

*Commentary on 9.18.6 to 9.18.9.* **Precedent.** The VAX back end of the portable C compiler used `CASEL`, the autoincrement modes and `INDEX` for these jobs; this list is what survived of that table on this CPU. **Trade-offs.** Each pass was measured against its cost: the size-for-speed passes (division by constants, jump tables) lengthen the code a little and shorten the run a lot, and the static-single-assignment form was left off because it ran 3.2% to 5.1% slower in the timing model. **Impact.** *Kernel:* a switch in a driver or a system-call dispatch is a table. *Compiler:* every item is a switchable pass with its own flag, so a regression can be bisected. *Hardware:* none of these needs an instruction that the entries do not already describe.

*Commentary.* **Precedent.** The PDP-11's `JSR Rn` put the return address in a register of the caller's choosing, which made coroutines a pair of `JSR`s and inline arguments free; the VAX replaced it with `CALLS` and a frame, which was slower than the code it replaced. **Trade-offs.** The most frequently felt loss in hand-written code is the byte-wide read-modify-write; the compiler never notices it. **Improvements.** `SOB` branches either way, with four reaches; the block operations keep their progress in registers, which is what the VAX got wrong, so a fault restarts for free. **Benefits.** The code a compiler emits is the code the table above measures, and the extras are for the people who write assembler. **Modern comparables.** x86 keeps `LOOP` and the string instructions with `REP`; AArch64 has bit-field moves and no memory-to-memory forms; RISC-V has none of them in the base. **Impact.** *Kernel:* `PROBER` and `PROBEW` replace the `nofault` global in the user-copy routines. *Compiler:* idioms the compiler never emits are not built for it, so the compiler's instruction selection stays small. *Hardware:* each restored idiom is a few hundred gates of existing datapath, and each lost one is a decode that never existed.

## 9.19 Side by side: the PDP-11, the VAX and the pdp64

**9.19.1** The word is 64 bits against the PDP-11's 16 and the VAX's 32, the registers are 32, the virtual address is 39, 48 or 57 bits with 40 physical in the present implementation, and the instruction count is 304 instruction words in 268 leaves, against the VAX's 304. *(D)*

**9.19.2** The program counter is not a register, there is no argument pointer, and the frame pointer and the linker scratch are ordinary software conventions. *(D)*

**9.19.3** An instruction has one or two operands (three for the block operations) and at most one memory operand, against the PDP-11's two and the VAX's six. *(D)*

**9.19.4** The condition codes are the same four, `N`, `Z`, `V` and `C`; floating point is in the base and in the general registers, with three IEEE formats (half, single, double) and no mode bits that change the meaning of other instructions, where the PDP-11 had `FPS.FD`, `FL` and `FT` and the VAX had rounding bits in the status word. *(D)*

**9.19.5** An instruction is two, four or six bytes (one to three parcels of 16 bits), or ten with a 64-bit literal, against the PDP-11's two to six and the VAX's one to more than fifty. *(D)*

*Commentary.* **Precedent.** The PDP-11 and VAX instruction formats, with their encodings drawn side by side, are in the instruction-set text, section 13. **Trade-offs.** The pdp64 keeps what made the PDP-11 pleasant, orthogonal addressing and a small register set, and what made the VAX powerful, flags, a register-save mask and string operations, and drops what made both expensive: memory-to-memory operands and unbounded instructions. **Improvements.** The compiler's code averages 3.26 bytes an instruction on the 841-file corpus and 2.93 executed. **Benefits.** A processor that is smaller than either predecessor while running 64-bit code. **Modern comparables.** The closest family is AArch64 in the number of registers and addressing modes and RISC-V compressed in density; neither has flags and a variable length together. **Impact.** *Kernel:* the exception model is the PDP-11's with the VAX's per-mode stacks. *Compiler:* one memory operand makes instruction selection a register-allocation problem, as on a RISC. *Hardware:* a decoder that finds the length from a prefix of the first parcel, and no microcode.

## 9.20 What was taken from LoongArch, and what was not

**9.20.1** LoongArch64 is the most recent clean-room instruction set designed without an ideological commitment: it borrowed from MIPS, RISC-V and ARM as each had the better answer, which is the posture of the pdp64.

**9.20.2** Adopted from it: bit manipulation in the base (`CLO`, `CLZ`, `CTZ`, bit-field moves, byte reverse), atomics with the barrier folded in (the `_DB` variants), a richer atomic set (fetch-and-minimum and maximum), and memory type as a page attribute, and 32 registers, which the prefix-coded encoding has the bits for. *(D)*

**9.20.3** Declined, with reasons: `R0` hardwired to zero (a thirty-second of a 32-register file, and `CLR` and `NEG` are one parcel already); no condition codes (a third read port in the branch unit against a four-bit latch); `FCSR` rounding-mode bits (hidden state that changes other instructions); four PC-relative address-formation instructions (one `LEA` carries 32 bits); and separate vector register files (chapter 8). *(D)*

Open: whether a flag-reading conditional select is added (J.9.1) and how floating-point comparisons set flags (J.9.9).

**9.20.4** A branch-free conditional select for constant-time code, `MASKEQZ` and `MASKNEZ`, is adopted, and so is the bit select `BSEL` (9.9.6). *(D)*

*Commentary.* **Precedent.** LoongArch has a fixed 32-bit encoding with room for two register fields and a displacement, so it can afford compare-and-branch and needs four instructions to form an address; this machine's variable length reverses both trade-offs. **Improvements.** The same idea is right or wrong depending on the encoding around it, and each adoption is judged against this encoding. **Benefits.** Borrowing the tested parts saves design time and avoids their bugs. **Modern comparables.** LoongArch itself; the RISC-V bit-manipulation extensions (Zbb, Zbs) do much of the same. **Impact.** *Kernel:* atomics with barriers make a lock acquire one instruction. *Compiler:* bit operations in the base mean the compiler can use them without a feature test. *Hardware:* each adopted idea is a few hundred lookup tables and each declined one is a block not built.

## 9.21 What is still open in the instruction set

**9.21.1** Open, each with its discussion in appendix J: the displacement of the shortest scaled-index form and a store through `d(DP)` (J.9.5); the page-bit mechanisms and the reserved system words (J.9.6); the schedule of landing pads and lazy against eager binding in the ABI (J.D.2); the worst-case latency of the block operations (J.9.7); the block operations and the cache (J.9.8); the floating-point condition flags (J.9.9); directed rounding and the remaining half-precision operations (J.9.10); the conditional select and the expansion of `ABS`, `MIN` and `MAX` (J.9.1); constant-time execution (J.9.2); the operand placement of the atomics and the size of `CAS` (J.9.3); and the coherence protocol beyond four cores with the verification of the memory model (J.9.11).

**9.21.2** Settled: the instruction length (one to three parcels, with `MOVQL` the one exception), the prefix code in place of a reserved-opcode map, and UTF-8 scope (`MVAL` is committed). *(D)*

**9.21.3** Open in the encodings: the number of the `CHECK` vector and the register form of `ADDV`, `SUBV` and `MULV` (J.9.4); the poison bit, the specification of the user-qualified moves, and the semantics of the AES and SHA words (J.9.12).

*Commentary.* **Trade-offs.** None of the open items blocks the first implementation; the three that must be settled before silicon, the memory model verification, the coherence choice and the worst-case latency figure, are verification tasks, not design changes. **Impact.** *Kernel:* the open items touching the kernel are the page-bit mechanisms, the floating-point flags and the block-operation granularity. *Compiler:* the conditional select and directed rounding are the two open items that change instruction selection. *Hardware:* the coherence protocol must be chosen before the cache is built, not after.

# Chapter 10 — Conclusion: from FPGA to silicon

**The pdp64 is to be built first on an FPGA, with at least four cores and a clock aimed at 200 to 300 MHz, as an in-order core that an out-of-order design can follow; it aims to repeat none of the processor flaws of the last decade; and it reaches silicon only after an OpenBSD built by the project's own toolchain can be logged into over `ssh`.** The chapter is for a reader who has followed the PDP-11 from a cabinet of boards to a chip and asks what the pdp64's path to a chip is. It gives the first implementation and its estimated size (10.1, 10.2), the reasons the first cores are in order and how a faster one stays possible (10.3), the security aims (10.4), the finish line (10.5) and what remains (10.6). A figure tagged *(E)* is an estimate, not a measurement. A fact about another machine that could not be checked against a source on hand is marked *unverified*.

## 10.1 The first implementation: an FPGA

What you know: the PDP-11/70 was a processor of several boards, and DEC later put the PDP-11 on a few chips (the LSI-11, whose processor handbook is of 1975-76) and then on one (the T-11; all *unverified* here). A new architecture today starts one step earlier. An **FPGA** (field-programmable gate array: a chip whose wiring can be loaded again and again) holds a processor described in a hardware description language, a **soft core**. Its logic is built from **lookup tables** (LUTs: each a small memory that implements any function of six inputs), multiplier blocks and **block RAMs** (the RAM that the chip provides in fixed-size pieces).

**10.1.1** The first implementation is a soft core on an FPGA. *(D)* (R20 and R22, "modern fpga so insane high clock specs"; R24, "if we're implementing the 8502 in fpga")

**10.1.2** The clock the FPGA design aims at is 200 to 300 MHz. *(D)* (errata 99)

**10.1.3** The design supports at least four cores, and sixteen or more is the preferred ceiling that no decision may preclude; the initial device has four cores, hoped to fit the FPGA (R338: "for initial device 4 cores is enough hopefully that fits on the fpga"). *(D)* (errata 92)

**10.1.4** A complete core, with the floating-point unit, the memory-management unit, two 8 KB first-level caches and the bus bridge, is about 16,000 six-input lookup tables, 10 to 16 multiplier blocks and 11 block RAMs, with an error of about ±40%: the register file is 250 lookup tables of distributed RAM, the 64-bit ALU 600, the barrel shifter 450, the caches 300 and 500 plus 8 block RAMs, the bus bridge 600. The figures are for the original sixteen-register design; the decoder and the thirty-two-register file of the pdp64 have not been estimated separately. *(E)*

**10.1.5** Block RAM, not logic, is the scarce resource: the caches, the translation buffer, the framebuffer and the blitter's scratch memory compete for it. *(E)*

*Commentary.* **Chain.** PDP-11: a processor of boards, then of chips, each smaller than the last (10.2). VAX: the MicroVAX 78032 was the VAX reduced to a subset so that it fitted one chip (10.2). Soft cores: RISC-V's Rocket and VexRiscv are the usual FPGA reference points (*unverified*), and the plan ranks the compiler back end as the first schedule risk, as it is for any new architecture (`pdp64-fpga-plan.md`, section 11). The pdp64 is designed so that a first core fits an FPGA and the same description can go to silicon. **Trade-offs.** On an FPGA the 64-bit carry chain (the adder's chain of carries) is slow and sets the clock, the barrel shifter (the unit that shifts by any count in one step) is a large tree of multiplexers, and the flags serialise branch resolution behind the ALU. None of this argues for changing the instruction set. It says where pipeline stages go: a first-level cache that is indexed by the page offset and tagged with the physical address takes the translation buffer off the critical path, and a comparator for branch conditions that runs beside the ALU buys a stage (`pdp64-fpga-plan.md`, section 3; no core has been timed, so these are *unverified* expectations). The architecture helps in several places: one memory operand means one address generator, no deferred modes means address generation is one adder pass, the length comes from a prefix of the first parcel, there is no unaligned data path and no partial-register write (the same document, section 8). **Impact.** *Kernel:* none. *Compiler:* none. *Hardware:* the first device is a four-core design whose size is estimated in 10.1.4.

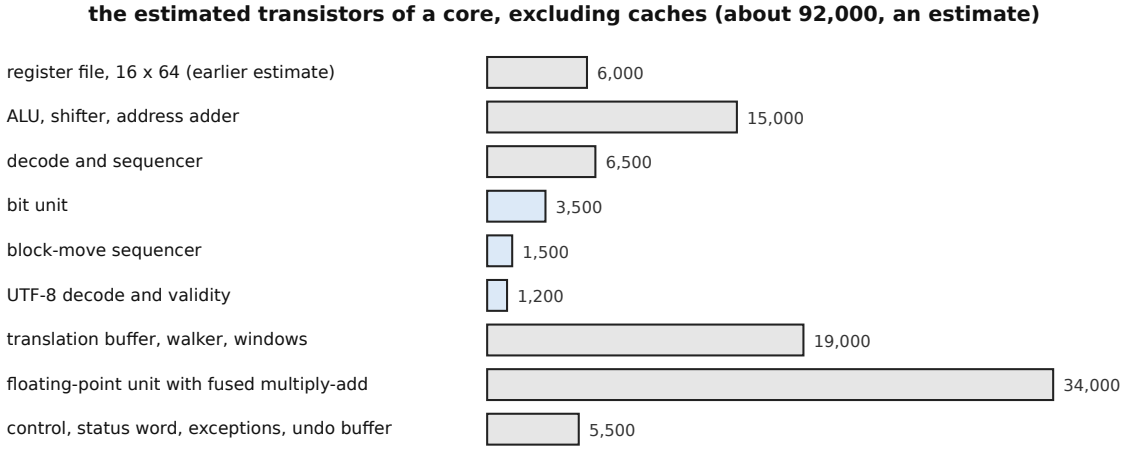
## 10.2 The chip: a transistor budget

What you know: how small a PDP-11 could be made. These figures are the project's historical table (`pdp64-isa.md`, section 10); the sources of the counts are not on this machine, so every one is *unverified*.

| Device | Year | Transistors | What it was |
|---|---|---|---|
| DEC T-11 | 1981 | about 12,000 | a complete PDP-11 integer core, no memory management, no floating point |
| ARM1 | 1985 | about 25,000 | 32-bit RISC, no memory management, no floating point |
| Motorola 68000 | 1979 | about 68,000 | 32-bit, orthogonal like the PDP-11; could not restart an instruction after a bus error |
| DEC J-11 | 1983 | about 120,000 (40,000 and 80,000 on two chips) | the full PDP-11/70: memory management, split instruction and data space, floating point |
| MicroVAX 78032 | 1984 | 125,000 | a subset of the VAX (175 of 304 instructions), no floating point |
| Intel 80386 | 1985 | about 275,000 | 32-bit, paging, segmentation |

**10.2.1** For a non-pipelined or short-pipeline implementation in a modern process, the whole core excluding caches is estimated at about 92,000 transistors. *(E)*

| Block | Transistors (E) |
|---|---|
| register file, 16 × 64, two read ports and one write | ~6,000 |
| ALU, shifter, address adder | ~15,000 |
| decode and sequencer | ~6,500 |
| bit unit (popcount tree, count zeros, bitfields) | ~3,500 |
| block-move sequencer | ~1,500 |
| UTF-8 decode and validity | ~1,200 |
| translation buffer (32 entries), walker, window compare | ~19,000 |
| floating-point unit with fused multiply-add, classify, min and max | ~34,000 |
| control, status word, exceptions, undo buffer | ~5,500 |
| **total, excluding caches** | **~92,000** |




**10.2.2** The bit unit, the block-move sequencer and the UTF-8 decoder together are about 6,200 of the 92,000 transistors, under 7% of the core; the fused multiply-add datapath is the largest single addition and the least negotiable. *(E)*

*Commentary.* **Chain.** The J-11 against the 78032 is the pair that shows what the instruction set costs: a complete PDP-11/70 with memory management and floating point in about 120,000 transistors against a VAX with its decimal, string-editing and H-float instructions removed and no floating point on the die in 125,000 (the project's reading of the table; *unverified*, and the two are not the same process, so only a rough comparison). The pdp64 sits below the 78032 with floating point and memory management included. **Trade-offs.** The estimate is not a measurement and excludes the caches, which dominate the area of a real chip. It was made for sixteen registers and the original encoding: a thirty-two-

register file costs about twice the first row (*inference*), which brings the total to about 98,000, and the decoder for the prefix code has not been estimated. An implementation of the translation buffer as a content-addressable memory (one that compares an address against every entry at once) is larger than the figure assumes; the buffer's size and organisation are the implementation's choice (6.8). **Impact.** *Kernel:* none. *Compiler:* none. *Hardware:* a core of this size fits in a small process node with room for the caches and the bus controller.

## 10.3 In order first, ready for out of order

What you know: the PDP-11 executed one instruction at a time, in the order written. **In order** still means that. An **out-of-order** core looks ahead in the program, runs whichever instructions have their inputs ready and keeps the results hidden until it is sure the program, run in order, would have produced them; to run ahead past a branch it **speculates** (guesses which way the branch goes and runs on).

**10.3.1** The first implementations, on the FPGA and in silicon, are small in-order cores without speculation; the architecture is kept ready for a large out-of-order design. *(D)* (R220: "i'm ok with the fpga/asic being in order, but want to be ready for the out of order large scale design")

**10.3.2** The condition flags were the structural obstacle: with the PDP-11 rule 68.2% of compiled instructions wrote a flag and 1.5% read one, so the flags form two groups that nothing writes in part, and moves, loads, stores and the compare-and-branch instructions write none. *(D)*

**10.3.3** In-order and out-of-order cores may share a die if software can tell which is which; the architectural memory model and the instruction semantics are the same on every core. *(D)* (errata 92)

*Commentary.* **Chain.** One instruction set has often spanned in-order and out-of-order cores: ARM's small cores beside its large ones, RISC-V's Rocket beside the BOOM, x86 from the Pentium to the P6, Alpha from the 21064 to the 21264 (all *unverified*). The VAX's long, orthogonal instructions with many side effects are generally blamed for making its later fast implementations so hard (*unverified*; 1.4). The pdp64's answer is restartable instructions with no hidden state (9.2.6) and flags in two independent groups that can be renamed separately (9.3.4). **Trade-offs.** Readiness has a cost, visible here as the compiled-code size paid for the flag rules (0.39%, measured) and as design effort spent on a core that may never be built. The 68.2% and 1.5% are counts on the first compiler's output on 827 files; after the flag rules 16.7% of instructions write a flag (`pdp64-ooo-readiness.md`). **Impact.** *Kernel:* flags are not state that survives half an instruction. *Compiler:* a compare-and-branch instead of a test after a move, 0.39% more code. *Hardware:* a property fixed in the specification while it is still being written is cheaper than one retrofitted after silicon.

## 10.4 Security: what the design must not repeat

What you know: the PDP-11's protection was its modes and its memory-management unit, which checked an access before it happened. The flaws of the last decade came from machines that ran ahead of the check or whose timing depended on a secret. **Speculative execution** attacks (Spectre, Meltdown) read data that a protection check would refuse, through a trace the guess leaves in a cache. A **timing channel** is any way a program learns a secret by how long something takes. **Rowhammer** flips bits in the DRAM chips by reading a row of memory hard enough to disturb its neighbours.

**10.4.1** The design does not repeat the processor flaws of the last decade that needed mitigations: no Rowhammer, no Spectre, no timing-channel attacks (R253: "no hammer-row, no spectre, no timing channel attacks"). The flaws are documented so that they are not repeated, and the rules that keep a faster core from repeating them are collected in the hardware-vulnerabilities report. *(D)* Details pending (J.10.1).

**10.4.2** An in-order, non-speculative core has none of the speculative side channels of the last decade, and an out-of-order core must keep that property. *(D)*

**10.4.3** At least one core can run in secure mode, with every timing channel removed; whether the mode is on every core or on one is decided by a rule of gates and slowdown (chapter 7, 7.11.3). *(D)*

**10.4.4** A bit `DIT` in the status word promises data-independent timing while it is set (9.3.7). *(D)* Open: the position of the bit and which instructions are constant-time while it is clear (J.9.2).

*Commentary.* **Chain.** The PDP-11 and the VAX had no speculation, and their protection was architectural, so the flaws of this section had no place to arise; they came with the speed-ups of the 1990s and 2000s, branch prediction and out-of-order execution (*unverified*). x86 and AArch64 answered the flaws with mitigations in software and microcode (page-table isolation, speculation barriers, flushes); the report lists them (appendix F). The pdp64's answer is to state, before a fast core

exists, the rules that it must obey: no speculation across protection domains, no predictor state shared across a privilege boundary without a flush, no data-dependent prefetcher, a named set of instructions with a timing independent of their operands. The report numbers them C1 to C16 (section 5). The report's status column of 2026-10-22 marked four of the sixteen as not met: the published constant-time set (C7), the cache and translation-buffer rule (C9), the memory controller's rule against Rowhammer (C11, a property of the memory chips and the controller) and the default-deny rule for bus masters (C12); the architecture text has since taken up parts of two (the IOMMU of 2.11, the `DIT` bit of 9.3.7). **Trade-offs.** A constant-time mode costs speed, so a core in secure mode carries the promise and a core in ordinary mode keeps its speed (the author's remark, R264: "for non-sensitive things, constant time isn't a priority"). **Impact.** *Kernel:* fewer mitigations to write in every entry path. *Compiler:* none. *Hardware:* each rule is a constraint on an out-of-order design, not a feature of the in-order one; the pain-points report finds the pdp64 avoids by design the largest classes of kernel pain: barriers (memory is sequentially consistent), floating-point state, a faulting entry, remote translation flushes and the speculative family (`kernel-pain-points.md`, the rows marked DESIGNED AWAY).

## 10.5 The finish line

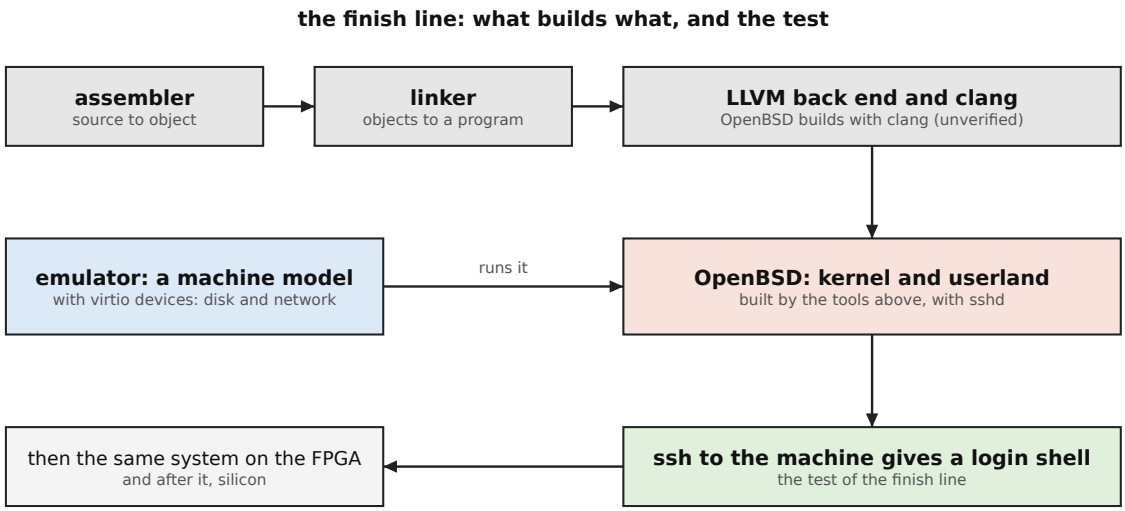
What you know: Unix went from the PDP-11 to the VAX as UNIX/32V (June 1979, a direct port of Seventh Edition Unix) and then, in 3BSD, gained virtual memory; the sources of all three are on this machine. The author's chain for the pdp64 (R216, R230) went from "if we are going straight to 3bsd why not openbsd" to "forget 3bsd we'll do 32v port if openbsd proves too hard, but i suspect it is doable".

**10.5.1** Before silicon the project delivers five things: an assembler, a linker, an LLVM compiler back end, an emulator, and a port of OpenBSD that can be logged into with `ssh` (R265: "this pdp64 project final goal (before we put it on actual silicon) is assembler linker compiler backend (llvm) emulator and port of openbsd we can ssh into"). *(D)*

**10.5.2** The architecture manual, which includes the instruction set but covers the whole machine, is the primary deliverable; the toolchain is advanced as needed to write and check it (R267, R266). *(D)*

**10.5.3** The path to OpenBSD goes through the LLVM back end and clang. If OpenBSD proves too hard, a port of Unix/32V is the fallback (R230), and it would not reach the goal of 10.5.1: the Unix/32V sources have no network code (no `inet`, `tcp` or socket files) and so no `ssh`. *(D)*

**10.5.4** The finish line is reached through the LLVM back end and clang, a machine model of the system for the operating system with virtio devices, a booting kernel and userland, and the network with `sshd`, until `ssh` to the machine gives a shell; the same system then goes on the FPGA. *(D)* Details to be determined (the order of the steps and the test for each).




If OpenBSD proves too hard, a port of Unix/32V is the fallback; it would give a shell on the emulator and no ssh (10.5.3).

*Commentary.* **Chain.** Unix reached the VAX as Unix/32V, and the VAX's virtual memory came in 3BSD; OpenBSD descends from that BSD line, and its newest port, riscv64, is the nearest measured precedent: 35,600 lines of machine-dependent code, of which 14,873 are the core, 11,771 platform drivers, 4,513 the loader and 4,443 headers; arm64 is 61,613 and amd64 79,167 (counted in the OpenBSD tree on this machine, `roadmap.md`). OpenBSD builds its base with clang on

arm64 and riscv64, and pcc cannot build its kernel or its userland (the kernel uses GNU C extensions and inline assembly); both are *unverified* here, and they are why the compiler is the long pole (the roadmap's assessment, an estimate). **Trade-offs.** The pdp64 port should be smaller than the riscv64 one in several places: no firmware tables to parse, one virtio interface for the devices instead of a platform's driver zoo, a trivial loader, no cache-maintenance code and no floating-point-state code; the core machine-dependent code of about 15,000 lines is the part that has to be written by hand *(E)*. The LLVM back end, estimated at three to six months for one engineer who has written one before, is the largest single item *(E)*; the figure has not been tested. **Benefits.** An OpenBSD that can be logged into shows that the architecture, the toolchain and the operating-system interface are complete in the way that matters to users, and it forces every piece (the compiler, the linker, the bus, the devices, the boot) to work together. **Impact.** *Kernel:* the port is the test of chapters 4 to 7. *Compiler:* it needs the atomics, thread-local storage and position-independent code of appendix D. *Hardware:* nothing beyond the architecture.

## 10.6 What remains

The questions the architecture has not yet answered are collected, chapter by chapter, with their options, in appendix J; the entries `J.2.n` to `J.10.n` belong to chapters 2 to 10. A decision whose details are still to be worked out carries the words *Details pending* or *Details to be determined* in its chapter. The proposals that nobody has decided are in appendix H, and what has been put off is in appendix I.

# Part II — The instruction reference tables

# The instruction reference tables

This is the reference part of the manual: **one entry for each instruction**, in the layout of a processor reference manual. Each entry gives its mnemonics, its syntax, its encoding (a chart of its layout, and the list of all its layouts), its operation in pseudocode, the flags it writes, the exceptions it can take, and where the manual explains it. The charts, the hot-register tables and the operand widths are generated from the grammars of the encoding (`spec/pdp64-enc.hbnf` and the grammar of the CPU), the one place where the encoding is written, so a layout here cannot disagree with the assembler or the simulators.

**Notation.** `Rd` is the destination register, `Rs` the first source, `Rt` the second, `Ru` the third; the assembler writes the destination last. `rm` is a general operand. `EA(rm)` is its effective address. `M[a]` is memory, `M8[a]` one byte. `sext` and `zext` sign- and zero-extend. `N Z V C` are the condition flags. Bit charts number the bits of a parcel from 15 at the left.

**Cf.** lines point to the chapter that explains the matter an instruction touches: the kernel (chapter 6), the multiprocessor model (chapter 7), the bus (chapter 2).

## Where the entries are

- Section 9.6, Moves, conversions and address formation: `MOVQ`, `MOVZL`, `LEA`, `MOVUB`, `MOV2`, `MOVQL`, `MOVQ.D`, `MOVQ.U`
- Section 9.7, Integer arithmetic: `ADD`, `SUB`, `CMP`, `MUL`, `MULH`, `ADDM`, `INC`, `DEC`, `NEG`, `ADC`, `SBC`, `DIV`, `MULHU`, `ABS`, `MIN`, `SET`, `MASKEQZ`, `ADDV`
- Section 9.8, Logic, shifts and rotates: `BIC`, `BIS`, `BIT`, `XOR`, `AND`, `SHL`, `CLR`, `COM`, `TST`, `ASL`, `ASR`
- Section 9.9, Bits, bytes and text: `CLZ`, `POPCNT`, `BSWAP`, `BITREV`, `ROR`, `RORI`, `RORI32`, `DSHR`, `EXTB`, `INSB`, `XNORPOP`, `CLMULL`, `UTF8LEN`, `UTF8DEC`, `UTF8ENC`, `BSEL`, `AESES`, `AESKS1`, `SHA256SIG0`, `SHA512SIG0`
- Section 9.10, Branches and control flow: `BR`, `SOB`, `CBNE`, `BTAG`, `JMP`, `CALL`, `JMPI`, `JTAG`, `MOVCALL`, `POPRET`, `POPMRET`
- Section 9.11, The stack and block operations: `PUSHM`, `POPM`, `PUSH`, `POP`, `MCPY`, `MSET`, `MCMP`, `MSCN`, `MVAL`
- Section 9.12, Floating point: `FADD`, `FCMP`, `FDIV`, `FADD32`, `FMADD`, `FMADD32`, `FNEG`, `FMIN`, `FCLASS`, `FRINT`, `FCVTDQ`, `FSFLAGS`, `FADD16`, `FCVTSH`
- Section 9.13, Atomics and ordering: `CAS`, `SWAP`, `XADD`, `XAND`, `XMAX`, `LL`
- Section 9.14, System instructions, exceptions and interrupts: `NOP`, `RTI`, `RET`, `HALT`, `BPT`, `FENCE`, `WFE`, `SEV`, `TLBIA`, `PUSHX`, `ISYNC`, `HINT`, `MFSR`, `MTPS`, `SPL`, `TRAP`, `TLBI`, `MTASID`, `MTDP`, `MTDMW`, `MFCPUID`, `ICINV`, `PROBER`, `TRAPCC.NE`

## The tables

- Assembler aliases and alternative spellings
- The architectural constants
- The mnemonics, alphabetically
- The bit layouts of every opcode: a chart for each layout, generated from the grammar

The specification of the pdp64 has 324 mnemonic words (288 instructions and 36 aliases), 54 of them added by the prefix-coded encoding. There are 121 instruction entries, 261 bit-layout charts of the layouts, and an entry for each of the 288 instructions with a chart of each of its 1150 forms.

*Generated by `tools/mkmanual.py`; do not edit.*

# Assembler aliases and old spellings

A mnemonic that assembles to another instruction. One mnemonic is one instruction (section 9.18); the disassembler never prints an alias.

*Generated by* `tools/mkmanual.py` *from* `spec/pdp64-mnemonics.hbnf`*; do not edit.*

| Written | Assembles to | Group |
|---|---|---|
| BHIS | BCC | branch |
| BLO | BCS | branch |
| CBHIS | CBCC | cbranch |
| CBLO | CBCS | cbranch |
| FADDD | FADD | alu |
| FADDS | FADD32 | fesc |
| FCLASSD | FCLASS | fesc |
| FCLASSS | FCLASS32 | fesc |
| FCMPD | FCMP | alu |
| FCMPS | FCMP32 | fesc |
| FDIVD | FDIV | fesc |
| FDIVS | FDIV32 | fesc |
| FMADDD | FMADD | fesc |
| FMADDS | FMADD32 | fesc |
| FMAXD | FMAX | fesc |
| FMIND | FMIN | fesc |
| FMSUBD | FMSUB | fesc |
| FMSUBS | FMSUB32 | fesc |
| FMULD | FMUL | alu |
| FMULS | FMUL32 | fesc |
| FNMADDD | FNMADD | fesc |
| FNMADDS | FNMADD32 | fesc |
| FNMSUBD | FNMSUB | fesc |
| FNMSUBS | FNMSUB32 | fesc |
| FRINTD | FRINT | fesc |
| FRINTDA | FRINTA | fesc |
| FRINTDM | FRINTM | fesc |
| FRINTDP | FRINTP | fesc |
| FRINTDZ | FRINTZ | fesc |
| FSQRTD | FSQRT | fesc |
| FSQRTS | FSQRT32 | fesc |
| FSUBD | FSUB | alu |
| FSUBS | FSUB32 | fesc |
| MULU | MUL | alu |
| ROL | RORI | besc |
| WAIT | WFE | sys-none |
| ROL rs, #n, rd | RORI rs, #(64-n), rd | assembler, immediate count only |

# The architectural constants

The numbers of the processor status word, the modes, the exception vectors, the page-table entry, the memory types, the translation geometry, the system registers, the addressing modes and the condition codes, each read from the machine-readable specification (`spec/pdp64-system.hbnf` and `spec/pdp64-mnemonics.hbnf`).
*Generated by `tools/mkmanual.py`; do not edit.*

## The processor status word (16 bits)

| Field | Bit | Width | Meaning |
|---|---|---|---|
| `C` | 0 | 1 | carry (borrow after a subtract) |
| `V` | 1 | 1 | signed overflow |
| `Z` | 2 | 1 | zero |
| `N` | 3 | 1 | negative |
| `T` | 4 | 1 | trace: take a trace trap after the next instruction |
| `PRI` | 5 | 3 | interrupt priority level, 0 to 7 |
| `MODE` | 8 | 2 | current mode |
| `PREV` | 10 | 2 | previous mode (the mode before the last exception) |

Bits 12 to 15 are reserved and read as zero. Bit 12 is the user-memory guard `UA`.

## The modes

| Mode | Value |
|---|---|
| kernel | 0 |
| supervisor | 1 |
| user | 3 |

Value 2 is reserved: loading it into either mode field of the status word is an illegal instruction.

## The exception vectors

The table has 512 entries of eight bytes at the physical address `VBR + 8 * vector`; `ECAUSE` equals the vector number.

| Vector | Number | Cause |
|---|---|---|
| `RESET` | 0 | processor reset: the first instruction fetch |
| `ILLEGAL` | 2 | a reserved or undefined opcode, a reserved encoding or an unassigned system register |
| `PRIVILEGED` | 3 | a system instruction executed outside kernel mode |
| `MISALIGNED` | 4 | a data access whose address is not a multiple of its size (the `MOVU` forms excepted) |
| `NONCANONICAL` | 5 | a virtual address whose bits 63 to 39 differ from bit 38 |
| `IFAULT` | 6 | instruction fetch from a page that is not mapped |
| `LFAULT` | 7 | load from a page that is not mapped |
| `SFAULT` | 8 | store to a page that is not mapped |
| `IPROT` | 9 | instruction fetch denied by the page permissions |
| `LPROT` | 10 | load denied by the page permissions |
| `SPROT` | 11 | store denied by the page permissions |
| `DIVZERO` | 12 | division by zero (`DIV`, `DIVU`, `MOD`, `MODU`) |
| `BUSFAULT` | 13 | a read of non-existent memory (a precise abort) |
| `LPAD` | 14 | an indirect transfer that did not land on `LPAD` (enforcement off in ABI version 1) |
| `BPT` | 15 | the `BPT` instruction |
| `TIMER` | 32 | the timer interrupt (level 6) |
| `COUNT` | 512 | the number of vectors; the table has this many eight-byte entries |

## The page-table entry (64 bits)

| Field | Bit | Width | Meaning |
|---|---|---|---|
| `V` | 0 | 1 | valid |
| `R` | 1 | 1 | readable |
| `W` | 2 | 1 | writable |
| `X` | 3 | 1 | executable |
| `U` | 4 | 1 | accessible from user mode |
| `G` | 5 | 1 | global: ignores the ASID |
| `A` | 6 | 1 | accessed (managed by software) |
| `D` | 7 | 1 | dirty (managed by software) |
| `MT` | 8 | 2 | memory type |
| `PPN` | 12 | 28 | physical page number: physical address bits 12 to 39, in place |

A non-leaf entry has `V` = 1 and `R` = `W` = `X` = 0 and points to the next table. Every bit not named is zero.

## The memory types (`MT`)

| Type | Value |
|---|---|
| cached | 0 |
| uncached | 1 |
| device | 2 |

## The translation geometry

| Quantity | Value |
|---|---|
| levels | 3 |

## The system registers (`MFSR`, `MTSR`)

| Name | Number | Access |
|---|---|---|
| `PTBR` | 0 | read and write |
| `EPC` | 1 | read and write |
| `EVA` | 2 | read only |
| `ECAUSE` | 3 | read only |
| `DMW0` | 4 | read and write |
| `DMW1` | 5 | read and write |
| `USP` | 6 | read and write |
| `SSP` | 7 | read and write |
| `EPSW` | 8 | read and write |
| `VBR` | 9 | read and write |
| `MMUCTL` | 10 | read and write |
| `ASID` | 11 | read and write |
| `CPUID` | 12 | read only |

## The condition codes

| Name | Number | Aliases |
|---|---|---|
| `al` | 0 | |
| `ne` | 1 | |
| `eq` | 2 | |
| `pl` | 3 | |
| `mi` | 4 | |
| `vc` | 5 | |
| `vs` | 6 | |
| `cc` | 7 | `hs` |
| `cs` | 8 | `lo` |
| `ge` | 9 | |
| `lt` | 10 | |

| Name | Number | Aliases |
|---|---|---|
| `gt` | 11 | |
| `le` | 12 | |
| `hi` | 13 | |
| `los` | 14 | |

# The mnemonics, alphabetically

*Generated by `tools/mkmanual.py`; do not edit.*

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| ABS | ABS (section 9.7) | 1, entry | Arithmetic (AESC) |
| ADC | ADC (section 9.7) | 7, entry | Single operand (SOP) |
| ADD | ADD (section 9.7) | 24, entry | Moves, address formation and the ALU |
| ADDM | ADDM (section 9.7) | 6, entry | Memory escape (MESC) |
| ADDV | OVF (section 9.7) | 1, entry | Added by the prefix-coded encoding |
| AESDS | AESR (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESDSM | AESR (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESES | AESR (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESESM | AESR (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESIM | AESK (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESKS1 | AESK (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AESKS2 | AESK (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| AND | AND (section 9.8) | 22, entry | Moves, address formation and the ALU |
| ASL | ASL (section 9.8) | 7, entry | Single operand (SOP) |
| ASR | ASR (section 9.8) | 7, entry | Single operand (SOP) |
| BCC | BCC (section 9.10) | 2, entry | Branches |
| BCS | BCC (section 9.10) | 2, entry | Branches |
| BEQ | BCC (section 9.10) | 3, entry | Branches |
| BGE | BCC (section 9.10) | 2, entry | Branches |
| BGT | BCC (section 9.10) | 2, entry | Branches |
| BHI | BCC (section 9.10) | 2, entry | Branches |
| BHIS (alias of BCC) | BCC (section 9.10) | 2, entry | Branches |
| BIC | BIC (section 9.8) | 13, entry | Moves, address formation and the ALU |
| BICM | ADDM (section 9.7) | 6, entry | Memory escape (MESC) |
| BIS | BIS (section 9.8) | 18, entry | Moves, address formation and the ALU |
| BISM | ADDM (section 9.7) | 6, entry | Memory escape (MESC) |
| BIT | BIT (section 9.8) | 12, entry | Moves, address formation and the ALU |
| BITREV | BITREV (section 9.9) | 1, entry | Bits and fields (BESC) |
| BLE | BCC (section 9.10) | 2, entry | Branches |
| BLO (alias of BCS) | BCC (section 9.10) | 2, entry | Branches |
| BLOS | BCC (section 9.10) | 2, entry | Branches |
| BLT | BCC (section 9.10) | 2, entry | Branches |
| BMI | BCC (section 9.10) | 2, entry | Branches |
| BNE | BCC (section 9.10) | 3, entry | Branches |
| BNTAG | BTAG (section 9.10) | 1, entry | Branches |
| BPL | BCC (section 9.10) | 2, entry | Branches |
| BPT | BPT (section 9.14) | 1, entry | System (SYS) |
| BR | BCC (section 9.10) | 6, entry | Branches |
| BSEL | BSEL (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| BSWAP | BSWAP (section 9.9) | 1, entry | Bits and fields (BESC) |
| BSWAPL | BSWAP (section 9.9) | 1, entry | Bits and fields (BESC) |
| BSWAPW | BSWAP (section 9.9) | 1, entry | Bits and fields (BESC) |
| BTAG | BTAG (section 9.10) | 1, entry | Branches |
| BVC | BCC (section 9.10) | 2, entry | Branches |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| BVS | BCC (section 9.10) | 2, entry | Branches |
| CALL | CALL (section 9.10) | 13, entry | Single operand (SOP) |
| CALLI | JMPI (section 9.10) | 9, entry | Indirect transfers (XFER) |
| CAS | CAS (section 9.13) | 1, entry | Atomics (ATOM) |
| CASDB | CAS (section 9.13) | 1, entry | Atomics (ATOM) |
| CBCC | CBCC (section 9.10) | 11, entry | Branches |
| CBCS | CBCC (section 9.10) | 11, entry | Branches |
| CBEQ | CBCC (section 9.10) | 21, entry | Branches |
| CBGE | CBCC (section 9.10) | 10, entry | Branches |
| CBGT | CBCC (section 9.10) | 10, entry | Branches |
| CBHI | CBCC (section 9.10) | 11, entry | Branches |
| CBHIS (alias of CBCC) | CBCC (section 9.10) | 11, entry | Branches |
| CBLE | CBCC (section 9.10) | 10, entry | Branches |
| CBLO (alias of CBCS) | CBCC (section 9.10) | 11, entry | Branches |
| CBLOS | CBCC (section 9.10) | 11, entry | Branches |
| CBLT | CBCC (section 9.10) | 10, entry | Branches |
| CBMI | CBCC (section 9.10) | 2, entry | Branches |
| CBNE | CBCC (section 9.10) | 21, entry | Branches |
| CBPL | CBCC (section 9.10) | 2, entry | Branches |
| CBVC | CBCC (section 9.10) | 2, entry | Branches |
| CBVS | CBCC (section 9.10) | 2, entry | Branches |
| CLMULH | CLMUL (section 9.9) | 1, entry | Bits and fields (BESC) |
| CLMULL | CLMUL (section 9.9) | 1, entry | Bits and fields (BESC) |
| CLO | COUNT (section 9.9) | 1, entry | Bits and fields (BESC) |
| CLR | CLR (section 9.8) | 10, entry | Single operand (SOP) |
| CLZ | COUNT (section 9.9) | 1, entry | Bits and fields (BESC) |
| CMP | CMP (section 9.7) | 19, entry | Moves, address formation and the ALU |
| COM | COM (section 9.8) | 7, entry | Single operand (SOP) |
| CTO | COUNT (section 9.9) | 1, entry | Bits and fields (BESC) |
| CTZ | COUNT (section 9.9) | 1, entry | Bits and fields (BESC) |
| DEC | DEC (section 9.7) | 10, entry | Single operand (SOP) |
| DIV | DIV (section 9.7) | 1, entry | Arithmetic (AESC) |
| DIVU | DIV (section 9.7) | 1, entry | Arithmetic (AESC) |
| DSHR | DSHR (section 9.9) | 1, entry | Bits and fields (BESC) |
| EXTB | EXTB (section 9.9) | 1, entry | Bits and fields (BESC) |
| EXTZB | EXTB (section 9.9) | 1, entry | Bits and fields (BESC) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| FABS | FSIGN (section 9.12) | 1, entry | Floating point (FESC) |
| FADD | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FADD16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FADD32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FADDD (alias of FADD) | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FADDS (alias of FADD32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FCLASS | FCLASS (section 9.12) | 1, entry | Floating point (FESC) |
| FCLASS16 | FCLASS (section 9.12) | 1, entry | Floating point (FESC) |
| FCLASS32 | FCLASS (section 9.12) | 1, entry | Floating point (FESC) |
| FCLASSD (alias of FCLASS) | FCLASS (section 9.12) | 1, entry | Floating point (FESC) |
| FCLASSS (alias of FCLASS32) | FCLASS (section 9.12) | 1, entry | Floating point (FESC) |
| FCMP | FCMP (section 9.12) | 9, entry | Moves, address formation and the ALU |
| FCMP16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FCMP32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FCMPD (alias of FCMP) | FCMP (section 9.12) | 9, entry | Moves, address formation and the ALU |
| FCMPS (alias of FCMP32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FCOPYSIGN | FSIGN (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTDH | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| FCVTDQ | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTDQU | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTDS | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTHD | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTHL | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTHS | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTLH | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTLS | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTQD | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTQUD | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTSD | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTSH | FCVTH (section 9.12) | 1, entry | Floating point (FESC) |
| FCVTSL | FCVT (section 9.12) | 1, entry | Floating point (FESC) |
| FDIV | FDIV (section 9.12) | 1, entry | Floating point (FESC) |
| FDIV16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FDIV32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FDIVD (alias of FDIV) | FDIV (section 9.12) | 1, entry | Floating point (FESC) |
| FDIVS (alias of FDIV32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FENCE | FENCE (section 9.14) | 1, entry | System (SYS) |
| FMADD | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FMADD16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FMADD32 | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| FMADDD (alias of FMADD) | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FMADDS (alias of FMADD32) | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FMAX | FMIN (section 9.12) | 1, entry | Floating point (FESC) |
| FMAXD (alias of FMAX) | FMIN (section 9.12) | 1, entry | Floating point (FESC) |
| FMIN | FMIN (section 9.12) | 1, entry | Floating point (FESC) |
| FMIND (alias of FMIN) | FMIN (section 9.12) | 1, entry | Floating point (FESC) |
| FMSUB | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FMSUB16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FMSUB32 | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FMSUBD (alias of FMSUB) | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FMSUBS (alias of FMSUB32) | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FMUL | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FMUL16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FMUL32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FMULD (alias of FMUL) | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FMULS (alias of FMUL32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FNEG | FSIGN (section 9.12) | 1, entry | Floating point (FESC) |
| FNMADD | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FNMADD16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FNMADD32 | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FNMADDD (alias of FNMADD) | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FNMADDS (alias of FNMADD32) | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FNMSUB | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FNMSUB16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FNMSUB32 | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FNMSUBD (alias of FNMSUB) | FMADD (section 9.12) | 1, entry | Floating point (FESC) |
| FNMSUBS (alias of FNMSUB32) | FMADD32 (section 9.12) | 1, entry | Floating point (FESC) |
| FRINT | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTA | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTD (alias of FRINT) | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTDA (alias of FRINTA) | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTDM (alias of FRINTM) | FRINT (section 9.12) | 1, entry | Floating point (FESC) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| FRINTDP (alias of FRINTP) | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTDZ (alias of FRINTZ) | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTM | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTP | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FRINTZ | FRINT (section 9.12) | 1, entry | Floating point (FESC) |
| FSFLAGS | FSFLAGS (section 9.12) | 1, entry | Floating point (FESC) |
| FSQRT | FDIV (section 9.12) | 1, entry | Floating point (FESC) |
| FSQRT16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| FSQRT32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FSQRTD (alias of FSQRT) | FDIV (section 9.12) | 1, entry | Floating point (FESC) |
| FSQRTS (alias of FSQRT32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FSUB | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FSUB16 | FHALF (section 9.12) | 1, entry | Floating point (FESC) |
| FSUB32 | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| FSUBD (alias of FSUB) | FADD (section 9.12) | 11, entry | Moves, address formation and the ALU |
| FSUBS (alias of FSUB32) | FSINGLE (section 9.12) | 1, entry | Floating point (FESC) |
| HALT | HALT (section 9.14) | 1, entry | System (SYS) |
| HINT | HINT (section 9.14) | 1, entry | System (SYS) |
| ICINV | ICINV (section 9.14) | 1, entry | System (SYS) |
| ICINVA | TLBA (section 9.14) | 1, entry | System (SYS) |
| INC | INC (section 9.7) | 10, entry | Single operand (SOP) |
| INSB | INSB (section 9.9) | 1, entry | Bits and fields (BESC) |
| ISYNC | ISYNC (section 9.14) | 1, entry | System (SYS) |
| JMP | JMP (section 9.10) | 12, entry | Single operand (SOP) |
| JMPI | JMPI (section 9.10) | 9, entry | Indirect transfers (XFER) |
| JTAG | JTAG (section 9.10) | 2, entry | Indirect transfers (XFER) |
| LEA | LEA (section 9.6) | 26, entry | Moves, address formation and the ALU |
| LL | LLSC (section 9.13) | 1, entry | Atomics (ATOM) |
| LPAD | NOP (section 9.14) | 1, entry | System (SYS) |
| MASKEQZ | MASK (section 9.7) | 1, entry | Added by the prefix-coded encoding |
| MASKNEZ | MASK (section 9.7) | 1, entry | Added by the prefix-coded encoding |
| MAX | MINMAX (section 9.7) | 1, entry | Arithmetic (AESC) |
| MAXU | MINMAX (section 9.7) | 1, entry | Arithmetic (AESC) |
| MCMP | MCMP (section 9.11) | 1, entry | Block memory (MBLK) |
| MCPY | MCPY (section 9.11) | 1, entry | Block memory (MBLK) |
| MFASID | MTASID (section 9.14) | 1, entry | System (SYS) |
| MFCPUID | MFCPUID (section 9.14) | 1, entry | System (SYS) |
| MFDP | MTDP (section 9.14) | 1, entry | System (SYS) |
| MFPS | MTPS (section 9.14) | 1, entry | System (SYS) |
| MFSR | MFSR (section 9.14) | 1, entry | System (SYS) |
| MIN | MINMAX (section 9.7) | 1, entry | Arithmetic (AESC) |
| MINU | MINMAX (section 9.7) | 1, entry | Arithmetic (AESC) |
| MMOV | MCPY (section 9.11) | 1, entry | Block memory (MBLK) |
| MOD | DIV (section 9.7) | 1, entry | Arithmetic (AESC) |
| MODU | DIV (section 9.7) | 1, entry | Arithmetic (AESC) |
| MOV2 | MOV2 (section 9.6) | 1, entry | Added by the prefix-coded encoding |
| MOVB | MOV (section 9.6) | 33, entry | Moves, address formation and the ALU |
| MOVB.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| MOVB.U | MOVUSER (section 9.6) | 6, entry | Added by the prefix-coded encoding |
| MOVCALL | MOVCALL (section 9.10) | 2, entry | Added by the prefix-coded encoding |
| MOVL | MOV (section 9.6) | 42, entry | Moves, address formation and the ALU |
| MOVL.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| MOVL.U | MOVUSER (section 9.6) | 6, entry | Added by the prefix-coded encoding |
| MOVQ | MOV (section 9.6) | 48, entry | Moves, address formation and the ALU |
| MOVQ.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVQ.U | MOVUSER (section 9.6) | 6, entry | Added by the prefix-coded encoding |
| MOVQL | MOVQL (section 9.6) | 1, entry | Added by the prefix-coded encoding |
| MOVUB | MOVU (section 9.6) | 13, entry | Memory escape (MESC) |
| MOVUL | MOVU (section 9.6) | 13, entry | Memory escape (MESC) |
| MOVUQ | MOVU (section 9.6) | 13, entry | Memory escape (MESC) |
| MOVUW | MOVU (section 9.6) | 13, entry | Memory escape (MESC) |
| MOVW | MOV (section 9.6) | 32, entry | Moves, address formation and the ALU |
| MOVW.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVW.U | MOVUSER (section 9.6) | 6, entry | Added by the prefix-coded encoding |
| MOVZB | MOVZ (section 9.6) | 25, entry | Moves, address formation and the ALU |
| MOVZB.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVZB.U | MOVUSER (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVZL | MOVZ (section 9.6) | 17, entry | Moves, address formation and the ALU |
| MOVZL.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVZL.U | MOVUSER (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVZW | MOVZ (section 9.6) | 17, entry | Moves, address formation and the ALU |
| MOVZW.D | MOVD (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MOVZW.U | MOVUSER (section 9.6) | 3, entry | Added by the prefix-coded encoding |
| MSCN | MSCN (section 9.11) | 1, entry | Block memory (MBLK) |
| MSCNN | MSCN (section 9.11) | 1, entry | Block memory (MBLK) |
| MSET | MSET (section 9.11) | 1, entry | Block memory (MBLK) |
| MTASID | MTASID (section 9.14) | 1, entry | System (SYS) |
| MTDMW | MTDMW (section 9.14) | 1, entry | System (SYS) |
| MTDP | MTDP (section 9.14) | 1, entry | System (SYS) |
| MTPS | MTPS (section 9.14) | 1, entry | System (SYS) |
| MTSR | MFSR (section 9.14) | 1, entry | System (SYS) |
| MUL | MUL (section 9.7) | 13, entry | Moves, address formation and the ALU |
| MULH | MULH (section 9.7) | 11, entry | Moves, address formation and the ALU |
| MULHU | MULHU (section 9.7) | 1, entry | Arithmetic (AESC) |
| MULU (alias of MUL) | MUL (section 9.7) | 13, entry | Moves, address formation and the ALU |
| MULV | OVF (section 9.7) | 1, entry | Added by the prefix-coded encoding |
| MVAL | MVAL (section 9.11) | 1, entry | Block memory (MBLK) |
| NEG | NEG (section 9.7) | 7, entry | Single operand (SOP) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| NOP | NOP (section 9.14) | 1, entry | System (SYS) |
| POP | POP (section 9.11) | 10, entry | Single operand (SOP) |
| POPCNT | POPCNT (section 9.9) | 1, entry | Bits and fields (BESC) |
| POPM | POPM (section 9.11) | 4, entry | Single operand (SOP) |
| POPMRET | POPMRET (section 9.10) | 1, entry | Added by the prefix-coded encoding |
| POPRET | POPRET (section 9.10) | 1, entry | Added by the prefix-coded encoding |
| POPX | PUSHX (section 9.14) | 1, entry | System (SYS) |
| PROBER | PROBE (section 9.14) | 1, entry | System (SYS) |
| PROBEW | PROBE (section 9.14) | 1, entry | System (SYS) |
| PUSH | PUSH (section 9.11) | 12, entry | Single operand (SOP) |
| PUSHM | PUSHM (section 9.11) | 4, entry | Single operand (SOP) |
| PUSHX | PUSHX (section 9.14) | 1, entry | System (SYS) |
| RET | RET (section 9.14) | 1, entry | System (SYS) |
| ROL (alias of RORI) | RORI (section 9.9) | 1, entry | Bits and fields (BESC) |
| ROR | ROR (section 9.9) | 1, entry | Bits and fields (BESC) |
| RORI | RORI (section 9.9) | 1, entry | Bits and fields (BESC) |
| RORI32 | RORI32 (section 9.9) | 1, entry | Bits and fields (BESC) |
| RTI | RTI (section 9.14) | 1, entry | System (SYS) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| RTT | RTI (section 9.14) | 1, entry | System (SYS) |
| SAR | SHIFT (section 9.8) | 14, entry | Moves, address formation and the ALU |
| SBC | SBC (section 9.7) | 7, entry | Single operand (SOP) |
| SC | LLSC (section 9.13) | 1, entry | Atomics (ATOM) |
| SET | SET (section 9.7) | 1, entry | Arithmetic (AESC) |
| SEV | SEV (section 9.14) | 1, entry | System (SYS) |
| SHA256SIG0 | SHA256 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA256SIG1 | SHA256 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA256SUM0 | SHA256 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA256SUM1 | SHA256 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA512SIG0 | SHA512 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA512SIG1 | SHA512 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA512SUM0 | SHA512 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHA512SUM1 | SHA512 (section 9.9) | 1, entry | Added by the prefix-coded encoding |
| SHL | SHIFT (section 9.8) | 15, entry | Moves, address formation and the ALU |
| SHR | SHIFT (section 9.8) | 14, entry | Moves, address formation and the ALU |
| SOB | SOB (section 9.10) | 1, entry | Branches |
| SPL | SPL (section 9.14) | 1, entry | System (SYS) |
| SUB | SUB (section 9.7) | 24, entry | Moves, address formation and the ALU |
| SUBM | ADDM (section 9.7) | 6, entry | Memory escape (MESC) |
| SUBV | OVF (section 9.7) | 1, entry | Added by the prefix-coded encoding |
| SWAP | SWAP (section 9.13) | 1, entry | Atomics (ATOM) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| SWAPDB | SWAP (section 9.13) | 1, entry | Atomics (ATOM) |
| TLBI | TLBI (section 9.14) | 1, entry | System (SYS) |
| TLBIA | TLBA (section 9.14) | 1, entry | System (SYS) |
| TLBSYNC | TLBA (section 9.14) | 1, entry | System (SYS) |
| TRAP | TRAP (section 9.14) | 1, entry | System (SYS) |
| TRAPCC.CC | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.CS | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.EQ | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.GE | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.GT | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.HI | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.LE | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.LOS | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.LT | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.MI | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.NE | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.PL | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.VC | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPCC.VS | TRAPCC (section 9.14) | 1, entry | Added by the prefix-coded encoding |
| TRAPL | TRAP (section 9.14) | 1, entry | System (SYS) |
| TST | TST (section 9.8) | 10, entry | Single operand (SOP) |
| UTF8DEC | UTF8DEC (section 9.9) | 1, entry | Bits and fields (BESC) |
| UTF8ENC | UTF8ENC (section 9.9) | 1, entry | Bits and fields (BESC) |
| UTF8LEN | UTF8LEN (section 9.9) | 1, entry | Bits and fields (BESC) |
| WAIT (alias of WFE) | WFE (section 9.14) | 1, entry | System (SYS) |
| WFE | WFE (section 9.14) | 1, entry | System (SYS) |
| XADD | XADD (section 9.13) | 1, entry | Atomics (ATOM) |
| XADDDB | XADD (section 9.13) | 1, entry | Atomics (ATOM) |
| XAND | XAND (section 9.13) | 1, entry | Atomics (ATOM) |
| XANDDB | XAND (section 9.13) | 1, entry | Atomics (ATOM) |
| XMAX | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMAXDB | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| XMAXU | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMAXUDB | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMIN | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMINDB | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMINU | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XMINUDB | XMAX (section 9.13) | 1, entry | Atomics (ATOM) |
| XNORPOP | XNORPOP (section 9.9) | 1, entry | Bits and fields (BESC) |
| XOR | XOR (section 9.8) | 18, entry | Moves, address formation and the ALU |

| Mnemonic | Entry | Layouts | Group |
|---|---|---|---|
| XOR_ | XAND (section 9.13) | 1, entry | Atomics (ATOM) |
| XOR_DB | XAND (section 9.13) | 1, entry | Atomics (ATOM) |
| XXOR | XAND (section 9.13) | 1, entry | Atomics (ATOM) |
| XXORDB | XAND (section 9.13) | 1, entry | Atomics (ATOM) |

# The bit layouts of every opcode

Every leaf of the encoding has a chart here. A leaf is a prefix of 3 to 16 bits of the first parcel, a sub-opcode that selects one of its mnemonics, and the pieces of its operands; there is no `len` field, because the prefix decides the length. The charts are generated by `tools/mkmanual.py` from the grammars `spec/pdp64-enc.hbnf` and `spec/pdp64/pdp64.hbnf`, read by `tools/encgram.py`; changing a field in a grammar changes the chart.

**How to read a chart.** Each row is one 16-bit parcel, bit 15 on the left, the bit numbers above the boxes. A shaded box has a fixed value, written in it; a white box is a field of the instruction. Fields are named `opN.part` for the part of the Nth operand of the assembly syntax (`op1.d` is the displacement of the first operand); a field the layout splits is `name[1]`, `name[2]` from the top. A register field of 3 or 4 bits holds an index into the hot-register table of the ABI, not a register number.

## The hot-register tables

The part of the ABI on which the encoding depends: a 3-bit register field holds, in order, `R0 R1 R8 R2 SP R3 R9 R10`; a 4-bit field holds `R0 R1 R8 R2 SP R3 R9 R10 R11 R4 R15 R5 R24 R25 R26 R6`. The fused instructions of the encoding are `movcall`, `popmret`, `popret`, `mov2`.

## The operand shapes

The widths and the reach of the immediates and displacements of each shape the forms use (the shapes are rules of the grammar):

| Shape | Meaning |
|---|---|
| `abs32` | absolute address, 32 bits |
| `b3d1u-xsz` | memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access |
| `b3d3u-xsz` | memory at base register, 3 bits (an index into the hot-register table) + unsigned 3-bit displacement times the size of the access |
| `b4d10u-xsz` | memory at base register, 4 bits (an index into the hot-register table) + unsigned 10-bit displacement times the size of the access |
| `b4d12u-xsz` | memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access |
| `b4d8s-xsz` | memory at base register, 4 bits (an index into the hot-register table) + signed 8-bit displacement times the size of the access |
| `b4d8u-xsz` | memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access |
| `b5d10s-xsz` | memory at base register, 5 bits + signed 10-bit displacement times the size of the access |
| `b5d12s-x1` | memory at base register, 5 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes |
| `b5d12s-xsz` | memory at base register, 5 bits + signed 12-bit displacement times the size of the access |
| `b5d28s-x1` | memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes |
| `b5d6s-xsz` | memory at base register, 5 bits + signed 6-bit displacement times the size of the access |
| `b5d6u-xsz` | memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access |
| `b5d8s-xsz` | memory at base register, 5 bits + signed 8-bit displacement times the size of the access |
| `b5d8u-xsz` | memory at base register, 5 bits + unsigned 8-bit displacement times the size of the access |
| `dpd28s-x1` | memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes |
| `imm10s` | signed 10-bit immediate, -512 to 511 |
| `imm12s` | signed 12-bit immediate, -2048 to 2047 |
| `imm12u` | unsigned 12-bit immediate, 0 to 4095 |
| `imm14s` | signed 14-bit immediate, -8192 to 8191 |
| `imm16s` | signed 16-bit immediate, -32768 to 32767 |
| `imm16u-x256` | unsigned 16-bit immediate times 256, 0 to 16776960 |
| `imm18s` | signed 18-bit immediate, -131072 to 131071 |
| `imm24u-x256` | unsigned 24-bit immediate times 256, 0 to 4294967040 |
| `imm32s` | signed 32-bit immediate, -2147483648 to 2147483647 |
| `imm32u` | unsigned 32-bit immediate, 0 to 4294967295 |
| `imm3s` | signed 3-bit immediate, -4 to 3 |
| `imm3u` | unsigned 3-bit immediate, 0 to 7 |
| `imm4s` | signed 4-bit immediate, -8 to 7 |

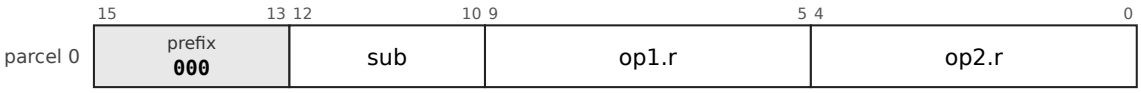
| Shape | Meaning |
|---|---|
| imm4u | unsigned 4-bit immediate, 0 to 15 |
| imm5s | signed 5-bit immediate, -16 to 15 |
| imm64s | signed 64-bit immediate, -9223372036854775808 to 9223372036854775807 |
| imm6s | signed 6-bit immediate, -32 to 31 |
| imm6u | unsigned 6-bit immediate, 0 to 63 |
| imm6u-x8 | unsigned 6-bit immediate times 8, 0 to 504 |
| imm7s | signed 7-bit immediate, -64 to 63 |
| imm8s | signed 8-bit immediate, -128 to 127 |
| imm8u | unsigned 8-bit immediate, 0 to 255 |
| imm8u-x256 | unsigned 8-bit immediate times 256, 0 to 65280 |
| k-imm0 | implicit constant 0 |

| Shape | Meaning |
|---|---|
| k-sp | implicit register r14 |
| pc10-u2 | pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes) |
| pc12-u1 | pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes) |
| pc12-u2 | pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes) |
| pc13-u2 | pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes) |
| pc14-u1 | pc-relative, 14 bits counted in 1-byte units (-8192 to 8191 bytes) |
| pc14-u2 | pc-relative, 14 bits counted in 2-byte units (-16384 to 16382 bytes) |
| pc16-u1 | pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes) |
| pc16-u2 | pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes) |
| pc18-u2 | pc-relative, 18 bits counted in 2-byte units (-262144 to 262142 bytes) |
| pc20-u2 | pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes) |
| pc24-u2 | pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes) |
| pc32-u1 | pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes) |
| pc32-u2 | pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes) |
| pc4-u2 | pc-relative, 4 bits counted in 2-byte units (-16 to 14 bytes) |
| pc6-u2 | pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes) |
| pc7-u2 | pc-relative, 7 bits counted in 2-byte units (-128 to 126 bytes) |
| pc8-u2 | pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes) |
| pc9-u2 | pc-relative, 9 bits counted in 2-byte units (-512 to 510 bytes) |
| pd5-s12 | pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access) |
| pi5-s12 | post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access) |
| reg3 | register, 3 bits (an index into the hot-register table) |
| reg4 | register, 4 bits (an index into the hot-register table) |
| reg5 | register, 5 bits |
| spd12s-x1 | memory at sp + signed 12-bit displacement times 1, -2048 to 2047 bytes |
| spd12u-x8 | memory at sp + unsigned 12-bit displacement times 8, 0 to 32760 bytes |
| spd12u-xsz | memory at sp + unsigned 12-bit displacement times the size of the access |
| spd14u-x8 | memory at sp + unsigned 14-bit displacement times 8, 0 to 131064 bytes |
| spd4u-x8 | memory at sp + unsigned 4-bit displacement times 8, 0 to 120 bytes |
| spd4u-xsz | memory at sp + unsigned 4-bit displacement times the size of the access |
| x4-4d4s | memory at base + (index << scale) + signed 4-bit displacement; base register, 4 bits (an index into the hot-register table), index register, 4 bits (an index into the hot-register table) |
| x4-4d6s | memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits (an index into the hot-register table), index register, 4 bits (an index into the hot-register table) |
| x5-5d0s | memory at base + (index << scale); base register, 5 bits, index register, 5 bits |
| x5-5d22s | memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits |
| x5-5d4s | memory at base + (index << scale) + signed 4-bit displacement; base register, 5 bits, index register, 5 bits |
| x5-5d6s | memory at base + (index << scale) + signed 6-bit displacement; base register, 5 bits, index register, 5 bits |

# The charts

## Instructions of 1 parcel (2 bytes)

### *RR5-1: RR5, prefix `000`*



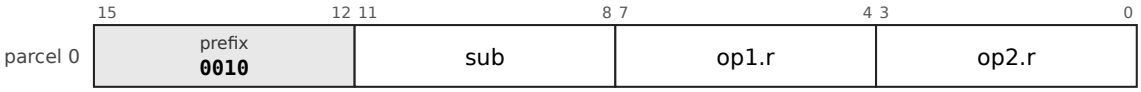

`op1` is `reg5`: register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `add` 010, `sub` 011, `xor` 100, `and` 101, `bis` 110, `shl` 111.

Fields: `sub=p0[12:10] op1.r=p0[9:5] op2.r=p0[4:0]`

### *RRF-1: RRF, prefix `0010`*



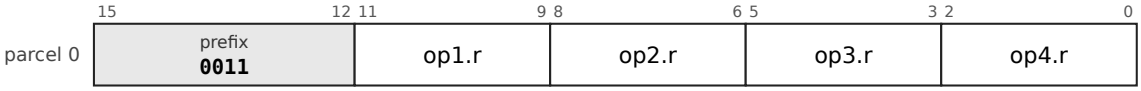

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[11:8] op1.r=p0[7:4] op2.r=p0[3:0]`

### *FX-MOV2-1: FX.MOV2, prefix `0011`*




`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

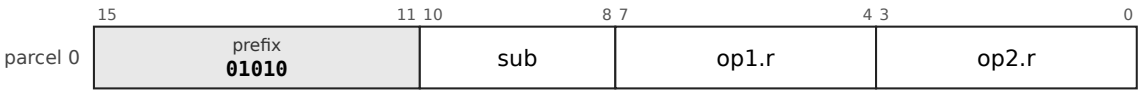
`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`op3` is `reg3`: register, 3 bits (an index into the hot-register table).

`op4` is `reg3`: register, 3 bits (an index into the hot-register table).

Fields: `op1.r=p0[11:9] op2.r=p0[8:6] op3.r=p0[5:3] op4.r=p0[2:0]`

### *RRF-2: RRF, prefix `01010`*



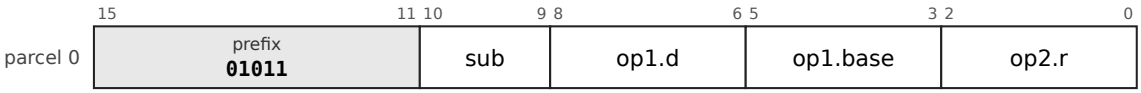

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `cmp` 110, `fcmp` 111.

Fields: `sub=p0[10:8] op1.r=p0[7:4] op2.r=p0[3:0]`

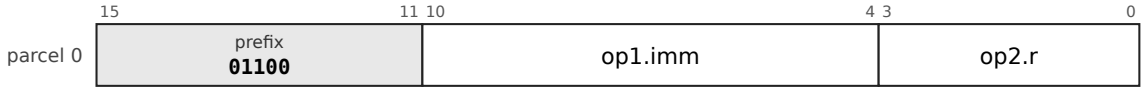
### *LDHHH3-a-1: LDHHH3/a, prefix `01011`*




`op1` is `b3d3u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 3-bit displacement times the size of the access.

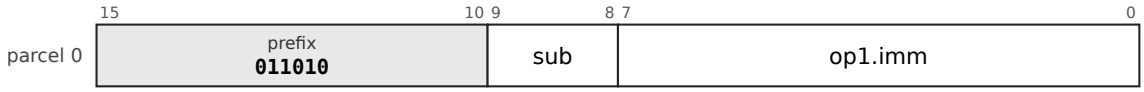
op2 is reg3: register, 3 bits (an index into the hot-register table).
sub (2 bits) selects the mnemonic: movq 00, movl 01, movzb 10, lea 11.
Fields: sub=p0[10:9] op1.d=p0[8:6] op1.base=p0[5:3] op2.r=p0[2:0]

### *IMMh47-a-1: IMMh47/a, prefix 01100*



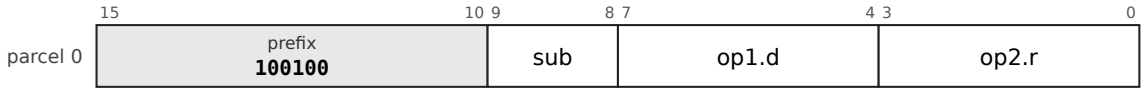

op1 is imm7s: signed 7-bit immediate, -64 to 63.
op2 is reg4: register, 4 bits (an index into the hot-register table).
Fields: op1.imm=p0[10:4] op2.r=p0[3:0]

### *SYSI-1: SYSI, prefix 011010*



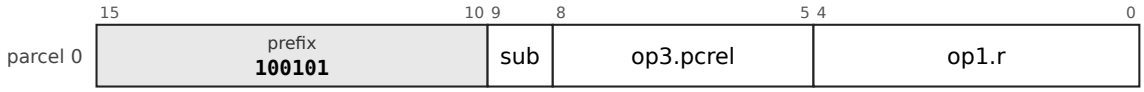

op1 is imm8u: unsigned 8-bit immediate, 0 to 255.
sub (2 bits) selects the mnemonic: spl 00, trap 01, trapl 10, hint 11.
Fields: sub=p0[9:8] op1.imm=p0[7:0]

### *LDSPh4-a-1: LDSPh4/a, prefix 100100*



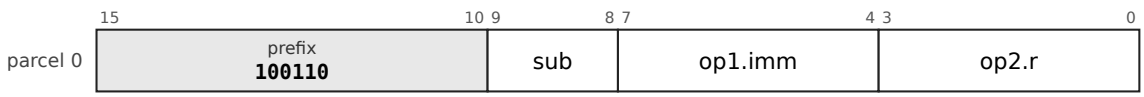

op1 is spd4u-xsz: memory at sp + unsigned 4-bit displacement times the size of the access.
op2 is reg4: register, 4 bits (an index into the hot-register table).
sub (2 bits) selects the mnemonic: movq 00, movl 01, movzb 10, lea 11.
Fields: sub=p0[9:8] op1.d=p0[7:4] op2.r=p0[3:0]

### *CBZ1-4-a-1: CBZ1_4/a, prefix 100101*



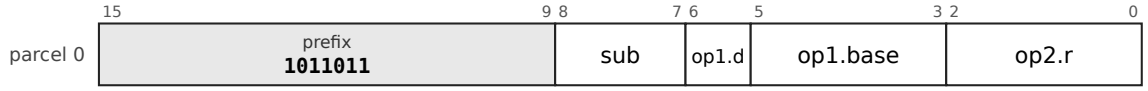

op1 is reg5: register, 5 bits.
op2 is k-imm0: implicit constant 0.
op3 is pc4-u2: pc-relative, 4 bits counted in 2-byte units (-16 to 14 bytes).
sub (1 bit) selects the mnemonic: cbeq 0, cbne 1.
Fields: sub=p0[9] op3.pcrel=p0[8:5] op1.r=p0[4:0]

### *SHIh44-a-1: SHIh44/a, prefix 100110*




op1 is imm4u: unsigned 4-bit immediate, 0 to 15.
op2 is reg4: register, 4 bits (an index into the hot-register table).
sub (2 bits) selects the mnemonic: shl 00, shr 01, sar 10, rol 11.
Fields: sub=p0[9:8] op1.imm=p0[7:4] op2.r=p0[3:0]

### *LDHHH1-a-1: LDHHH1/a, prefix `1011011`*



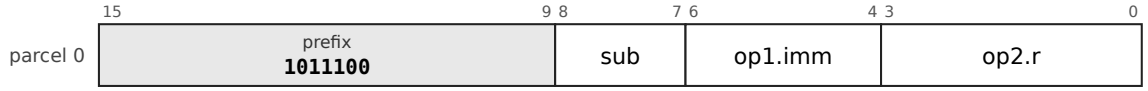

`op1` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`op2` is `reg3`: register, 3 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[8:7] op1.d=p0[6] op1.base=p0[5:3] op2.r=p0[2:0]`

### *SHIh43-a-1: SHIh43/a, prefix `1011100`*



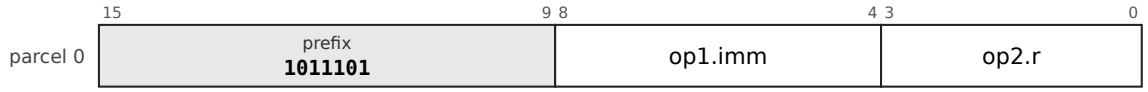

`op1` is `imm3u`: unsigned 3-bit immediate, 0 to 7.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `shl` 00, `shr` 01, `sar` 10, `rol` 11.

Fields: `sub=p0[8:7] op1.imm=p0[6:4] op2.r=p0[3:0]`

### *IMMh45-a-1: IMMh45/a, prefix `1011101`*

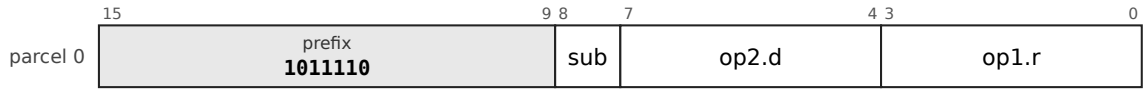



`op1` is `imm5s`: signed 5-bit immediate, -16 to 15.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op1.imm=p0[8:4] op2.r=p0[3:0]`

### *STSPh4-a-1: STSPh4/a, prefix `1011110`*



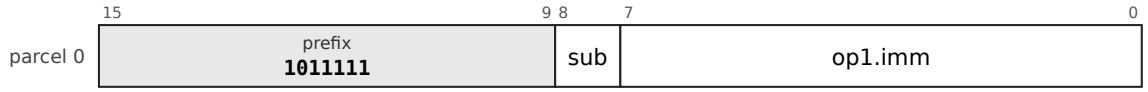

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `spd4u-xsz`: memory at sp + unsigned 4-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[8] op2.d=p0[7:4] op1.r=p0[3:0]`

### *PUSHM8-a-1: PUSHM8/a, prefix `1011111`*

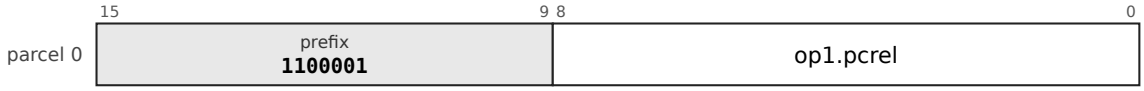



`op1` is `imm8u-x256`: unsigned 8-bit immediate times 256, 0 to 65280.

`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

Fields: `sub=p0[8] op1.imm=p0[7:0]`

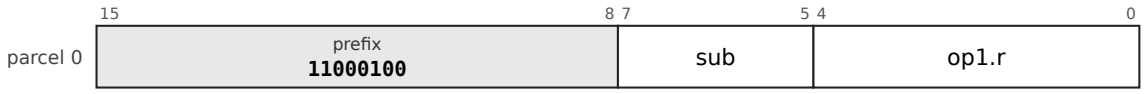
### *BR1-9-a-1: BR1_9/a, prefix 1100001*




`op1` is `pc9-u2`: pc-relative, 9 bits counted in 2-byte units (-512 to 510 bytes).

Fields: `op1.pcrel=p0[8:0]`

### *SYSR-1: SYSR, prefix 11000100*

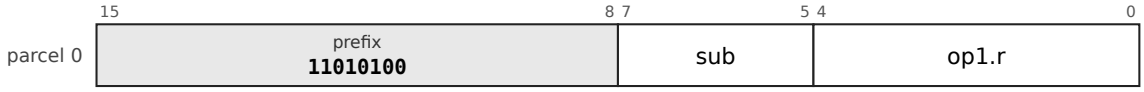



`op1` is `reg5`: register, 5 bits.

`sub` (3 bits) selects the mnemonic: `mtps` 000, `mfps` 001, `tlbi` 010, `mtasid` 011, `mfasid` 100, `mtdp` 101, `mfdp` 110, `mtdmw` 111.

Fields: `sub=p0[7:5] op1.r=p0[4:0]`

### *SOPR-1: SOPR, prefix 11010100*




`op1` is `reg5`: register, 5 bits.

`sub` (3 bits) selects the mnemonic: `clr` 000, `com` 001, `inc` 010, `dec` 011, `neg` 100, `tst` 101, `adc` 110, `sbc` 111.

Fields: `sub=p0[7:5] op1.r=p0[4:0]`

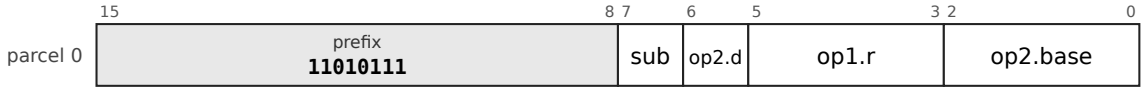
### *FX-POPMRET-1: FX.POPMRET, prefix 11010110*




`op1` is `imm8u-x256`: unsigned 8-bit immediate times 256, 0 to 65280.

Fields: `op1.imm=p0[7:0]`

### *STHHH1-a-1: STHHH1/a, prefix 11010111*




`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

`op2` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[7] op2.d=p0[6] op1.r=p0[5:3] op2.base=p0[2:0]`

### *LEASPh4-a-1: LEASPh4/a, prefix 11011000*

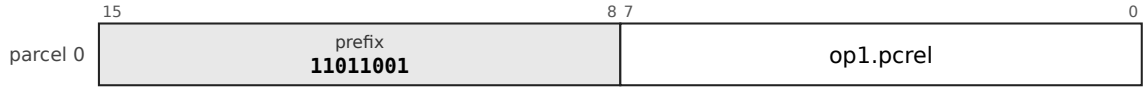



`op1` is `spd4u-x8`: memory at sp + unsigned 4-bit displacement times 8, 0 to 120 bytes.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op1.d=p0[7:4] op2.r=p0[3:0]`

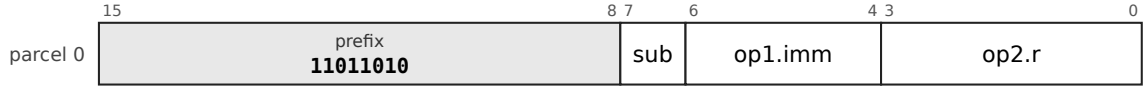
### *BR1-8-a-1: BR1_8/a, prefix `11011001`*




`op1` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

Fields: `op1.pcrel=p0[7:0]`

### *IMMh43-b-1: IMMh43/b, prefix `11011010`*



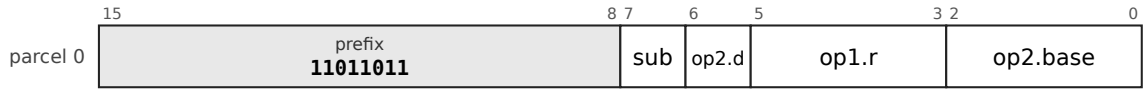

`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[7] op1.imm=p0[6:4] op2.r=p0[3:0]`

### *STHHH1-b-1: STHHH1/b, prefix `11011011`*



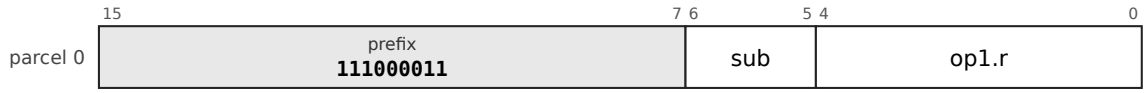

`op1` is `reg3`: register, 3 bits (an index into the hot-register table).

`op2` is `b3d1u-xsz`: memory at base register, 3 bits (an index into the hot-register table) + unsigned 1-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[7] op2.d=p0[6] op1.r=p0[5:3] op2.base=p0[2:0]`

### *SOPR-2: SOPR, prefix `111000011`*




`op1` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `asl` 00, `asr` 01, `push` 10, `pop` 11.

Fields: `sub=p0[6:5] op1.r=p0[4:0]`

### *IMMh43-a-1: IMMh43/a, prefix `111000101`*

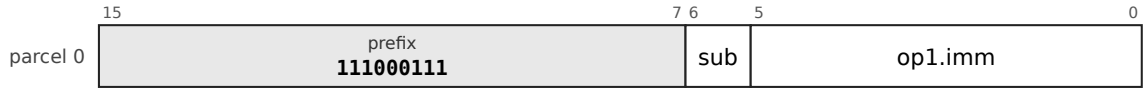



`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op1.imm=p0[6:4] op2.r=p0[3:0]`

### *SPADJ6-a-1: SPADJ6/a, prefix `111000111`*




`op1` is `imm6u-x8`: unsigned 6-bit immediate times 8, 0 to 504.

`op2` is `k-sp`: implicit register r14.
`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.
Fields: `sub=p0[6] op1.imm=p0[5:0]`

### *BR1-7-a-1: BR1_7/a, prefix `111001000`*




`op1` is `pc7-u2`: pc-relative, 7 bits counted in 2-byte units (-128 to 126 bytes).
Fields: `op1.pcrel=p0[6:0]`

### *BR1-6-b-1: BR1_6/b, prefix `111001001`*




`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).
`sub` (1 bit) selects the mnemonic: `beq` 0, `bne` 1.
Fields: `sub=p0[6] op1.pcrel=p0[5:0]`

### *IMMh43-b-2: IMMh43/b, prefix `111001011`*



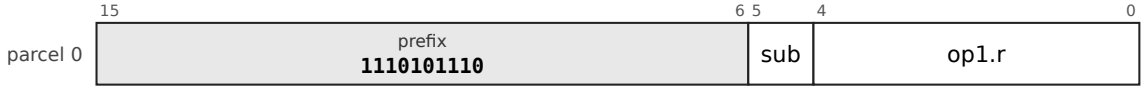

`op1` is `imm3s`: signed 3-bit immediate, -4 to 3.
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
Fields: `op1.imm=p0[6:4] op2.r=p0[3:0]`

### *SYSR-2: SYSR, prefix `1110100000`*




`op1` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `mfcpuid` 0, `icinv` 1.
Fields: `sub=p0[5] op1.r=p0[4:0]`

### *SOPR-3: SOPR, prefix `1110101110`*




`op1` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `jmp` 0, `call` 1.
Fields: `sub=p0[5] op1.r=p0[4:0]`

### *BR1-6-a-1: BR1_6/a, prefix `1110110000`*




`op1` is `pc6-u2`: pc-relative, 6 bits counted in 2-byte units (-64 to 62 bytes).

Fields: `op1.pcrel=p0[5:0]`

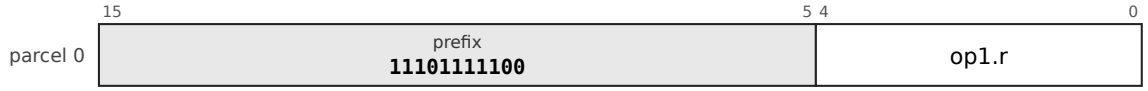
### *SYS0-1: SYS0, prefix `11101110010`*




`sub` (5 bits) selects the mnemonic: `nop` 00000, `rti` 00001, `rtt` 00010, `ret` 00011, `halt` 00100, `bpt` 00101, `fence` 00110, `wfe` 00111, `sev` 01000, `lpad` 01001, `tlbia` 01010, `tlbsync` 01011, `icinva` 01100, `pushx` 01101, `popx` 01110, `isync` 01111, `trapcc.ne` 10000, `trapcc.eq` 10001, `trapcc.pl` 10010, `trapcc.mi` 10011, `trapcc.vc` 10100, `trapcc.vs` 10101, `trapcc.cc` 10110, `trapcc.cs` 10111, `trapcc.ge` 11000, `trapcc.lt` 11001, `trapcc.gt` 11010, `trapcc.le` 11011, `trapcc.hi` 11100, `trapcc.los` 11101, `syspg0` 11110, `syspg1` 11111.

Fields: `sub=p0[4:0]`

### *FX-POPRET-1: FX.POPRET, prefix `11101111100`*

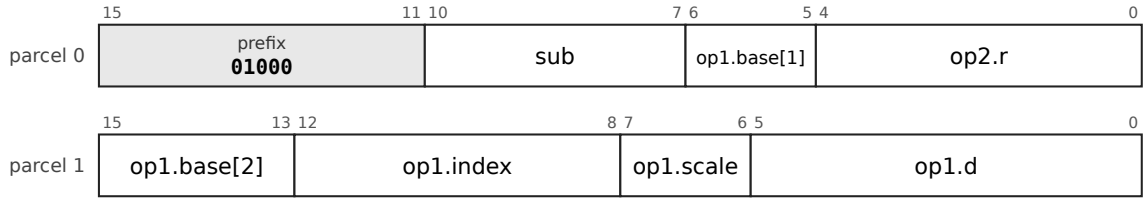



`op1` is `reg5`: register, 5 bits.

Fields: `op1.r=p0[4:0]`

## Instructions of 2 parcels (4 bytes)

### *LDU-X6-1: LDU.X6, prefix `01000`*



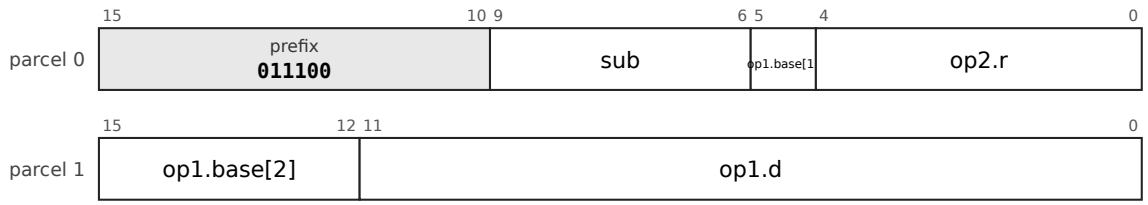

`op1` is `x5-5d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

Fields: `sub=p0[10:7]` `op1.base-1=p0[6:5]` `op2.r=p0[4:0]` `op1.base-2=p1[15:13]` `op1.index=p1[12:8]` `op1.scale=p1[7:6]` `op1.d=p1[5:0]`

### *LDU-B12-1: LDU.B12, prefix `011100`*



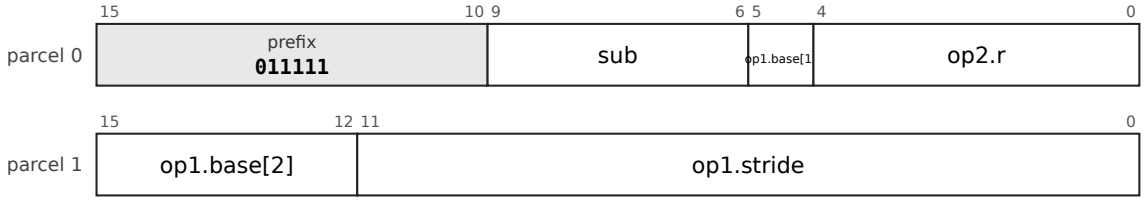

`op1` is `b5d12s-x1`: memory at base register, 5 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.

Fields: `sub=p0[9:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]`

### *LDF-PI12-1: LDF.PI12, prefix `011111`*



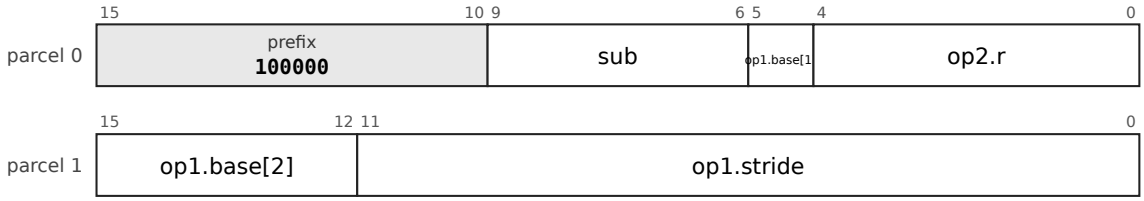

`op1` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[9:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *LDF-PD12-1: LDF.PD12, prefix `100000`*



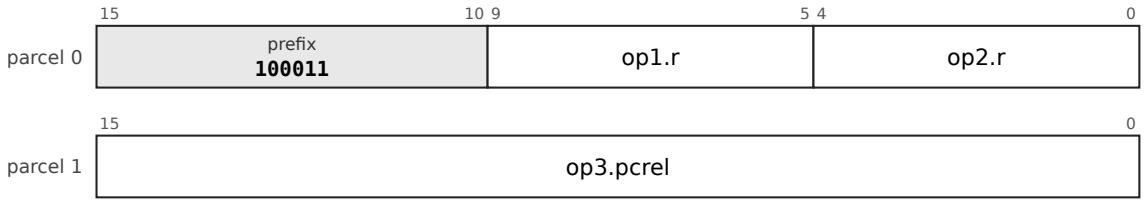

`op1` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[9:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *FX-MOVCALL2-1: FX.MOVCALL2, prefix `100011`*



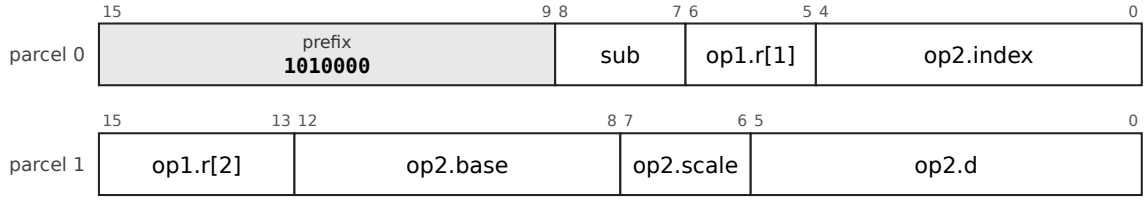

`op1` is `reg5`: register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`op3` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

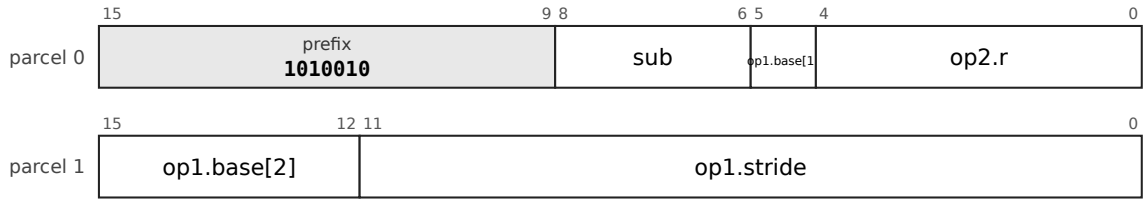
Fields: `op1.r=p0[9:5] op2.r=p0[4:0] op3.pcrel=p1[15:0]`

### *STU-X6-1: STU.X6, prefix `1010000`*



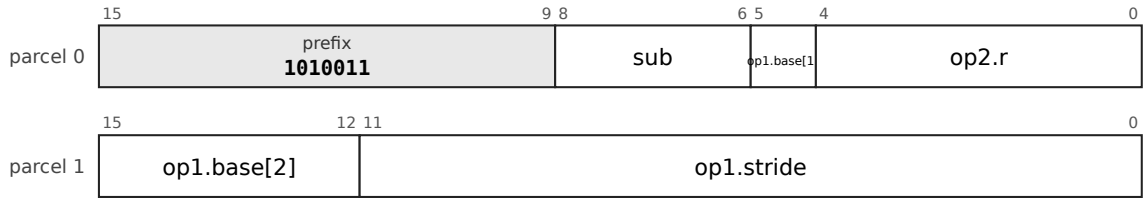

`op1` is `reg5`: register, 5 bits.
`op2` is `x5-5d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 5 bits, index register, 5 bits.
`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.
Fields: `sub=p0[8:7] op1.r-1=p0[6:5] op2.index=p0[4:0] op1.r-2=p1[15:13] op2.base=p1[12:8] op2.scale=p1[7:6] op2.d=p1[5:0]`

### *LDF-PI12-2: LDF.PI12, prefix `1010010`*



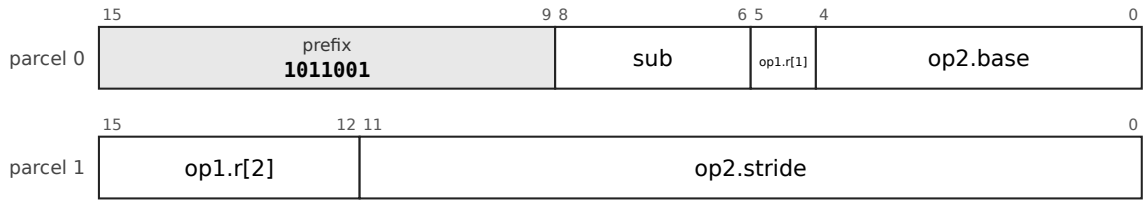

`op1` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[8:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *LDF-PD12-2: LDF.PD12, prefix `1010011`*



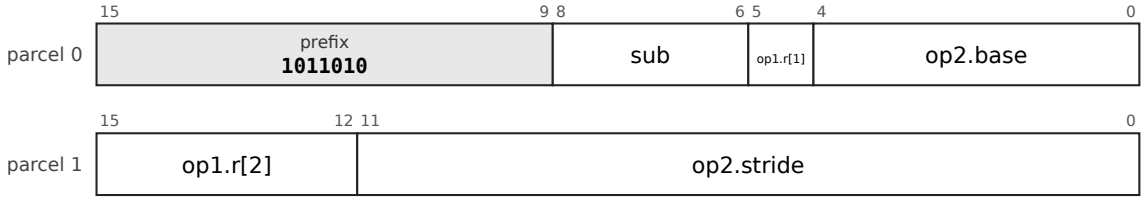

`op1` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[8:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *STF-PI12-1: STF.PI12, prefix `1011001`*



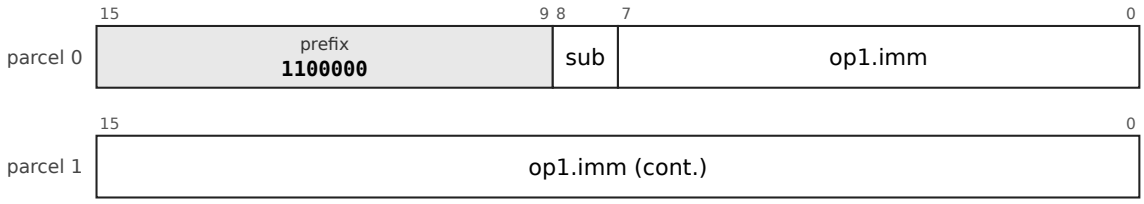

`op1` is `reg5`: register, 5 bits.
`op2` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[8:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *STF-PD12-1: STF.PD12, prefix `1011010`*



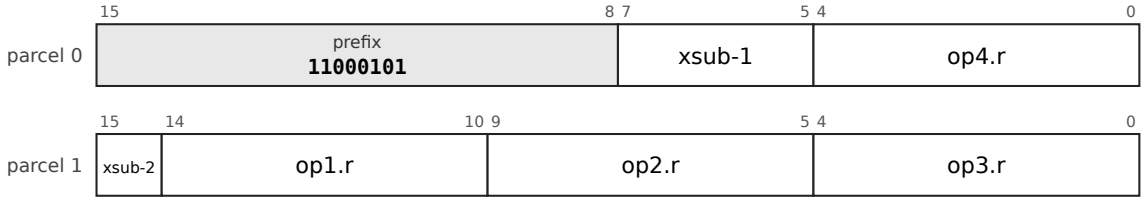

`op1` is `reg5`: register, 5 bits.
`op2` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[8:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *PUSHM24-a-1: PUSHM24/a, prefix `1100000`*



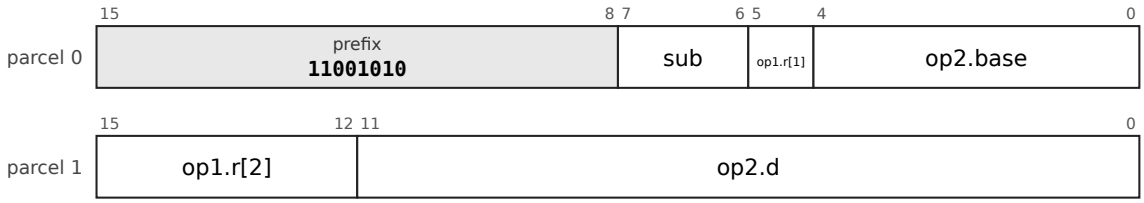

`op1` is `imm24u-x256`: unsigned 24-bit immediate times 256, 0 to 4294967040.
`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.
Fields: `sub=p0[8] op1.imm=p0[7:0]+p1[15:0]`

### *R4-1: R4, prefix `11000101`*




`op1` is `reg5`: register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`op3` is `reg5`: register, 5 bits.
`op4` is `reg5`: register, 5 bits.
`xsub` selects the mnemonic.
Fields: `xsub-1=p0[7:5] op4.r=p0[4:0] xsub-2=p1[15] op1.r=p1[14:10] op2.r=p1[9:5] op3.r=p1[4:0]`

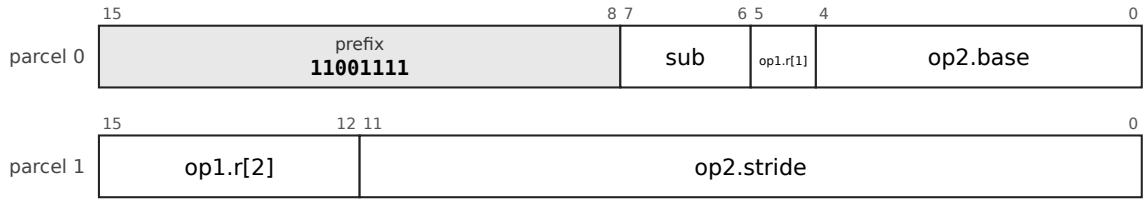
### *STU-B12-1: STU.B12, prefix `11001010`*




`op1` is `reg5`: register, 5 bits.
`op2` is `b5d12s-x1`: memory at base register, 5 bits + signed 12-bit displacement times 1, -2048 to 2047 bytes.
`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.

Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.d=p1[11:0]`

### *STF-PI12-2: STF.PI12, prefix `11001111`*



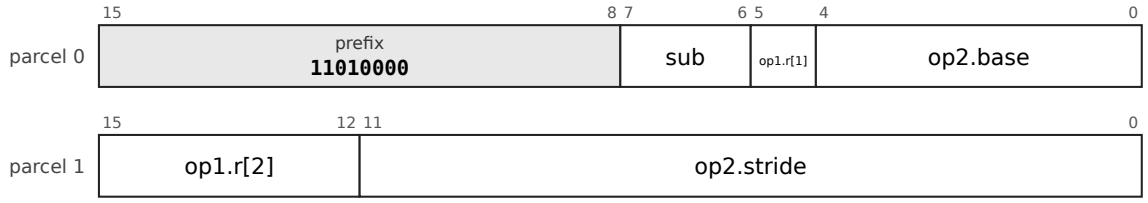

`op1` is `reg5`: register, 5 bits.

`op2` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *STF-PD12-2: STF.PD12, prefix `11010000`*



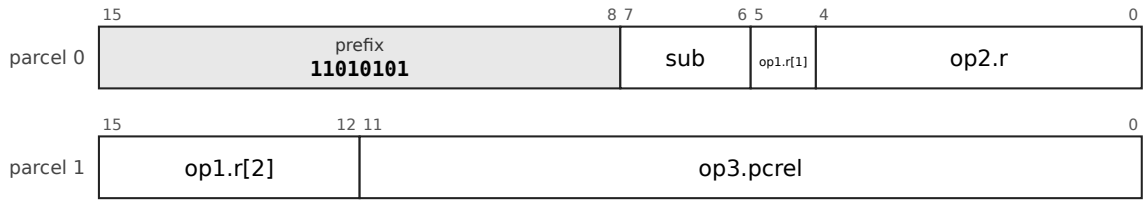

`op1` is `reg5`: register, 5 bits.

`op2` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *CBDR-1: CBDR, prefix `11010101`*




`op1` is `reg5`: register, 5 bits.

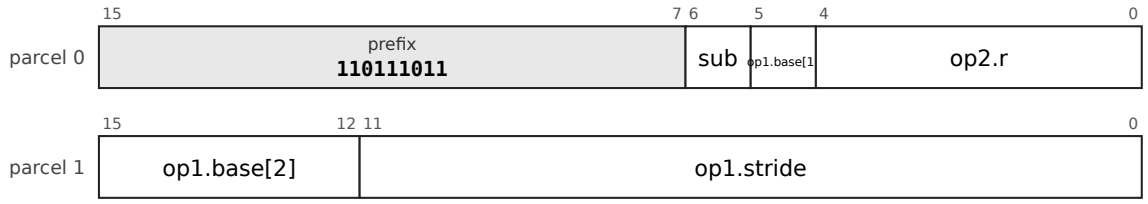
`op2` is `reg5`: register, 5 bits.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cbpl` 00, `cbmi` 01, `cbvc` 10, `cbvs` 11.

Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.r=p0[4:0] op1.r-2=p1[15:12] op3.pcrel=p1[11:0]`

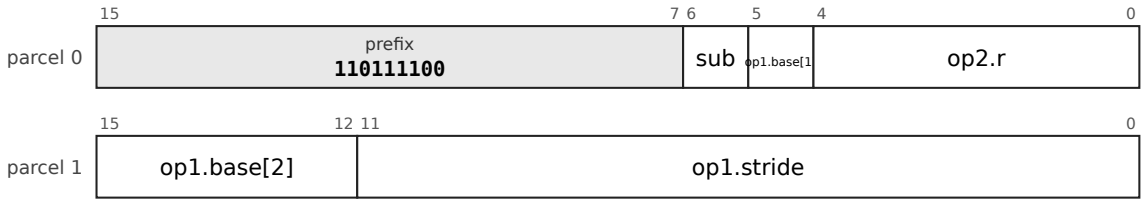
### *LDF-PI12-3: LDF.PI12, prefix `110111011`*




`op1` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

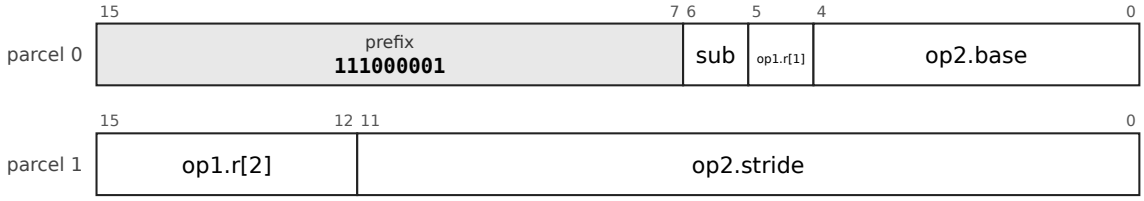
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *LDF-PD12-3: LDF.PD12, prefix `110111100`*



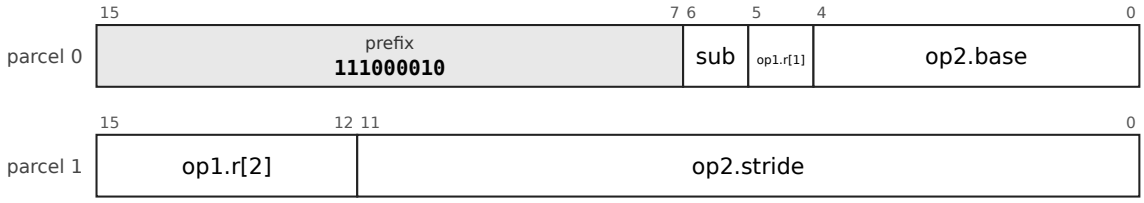

`op1` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *CMF-PI12-1: CMF.PI12, prefix `111000001`*



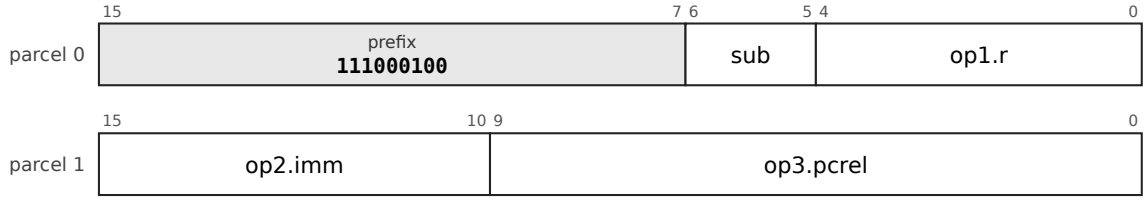

`op1` is `reg5`: register, 5 bits.
`op2` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *CMF-PD12-1: CMF.PD12, prefix `111000010`*




`op1` is `reg5`: register, 5 bits.
`op2` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.stride=p1[11:0]`

### *CBDI-1: CBDI, prefix `111000100`*




`op1` is `reg5`: register, 5 bits.

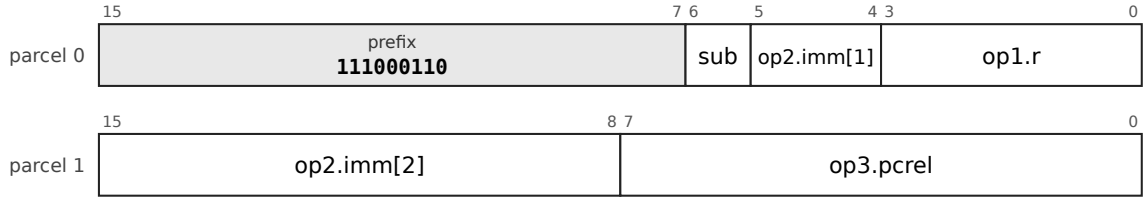
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (2 bits) selects the mnemonic: `cbpl` 00, `cbmi` 01, `cbvc` 10, `cbvs` 11.

Fields: `sub=p0[6:5] op1.r=p0[4:0] op2.imm=p1[15:10] op3.pcrel=p1[9:0]`

### *CBI10-8H-a-1: CBI10_8H/a, prefix `111000110`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

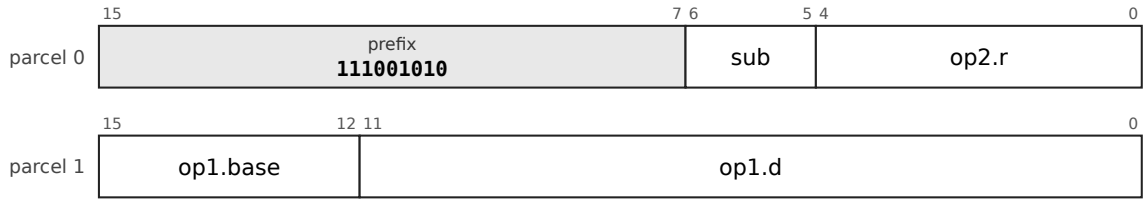
`op2` is `imm10s`: signed 10-bit immediate, -512 to 511.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:8] op3.pcrel=p1[7:0]`

### *LDBu12H-a-1: LDBu12H/a, prefix `111001010`*



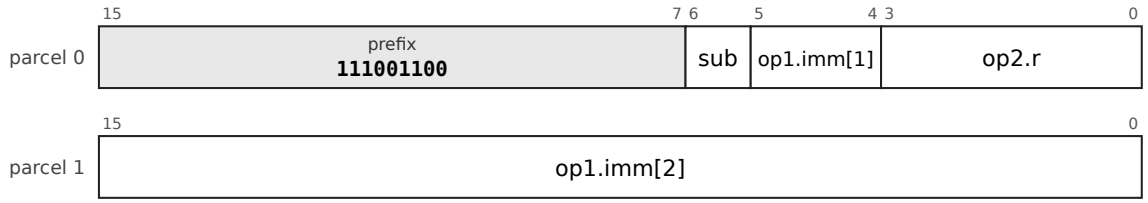

`op1` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[6:5] op2.r=p0[4:0] op1.base=p1[15:12] op1.d=p1[11:0]`

### *LDI18H-b-1: LDI18H/b, prefix `111001100`*



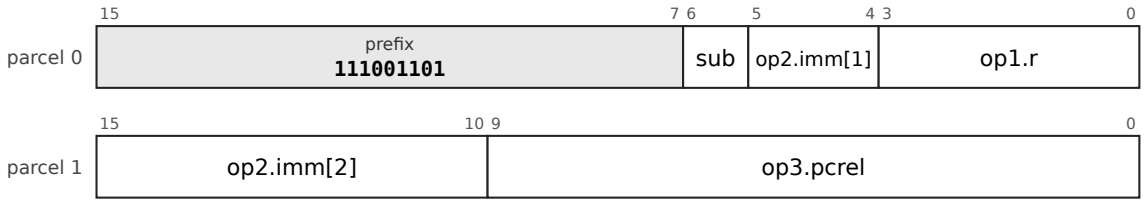

`op1` is `imm18s`: signed 18-bit immediate, -131072 to 131071.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[6] op1.imm-1=p0[5:4] op2.r=p0[3:0] op1.imm-2=p1[15:0]`

### *CBI8-10H-a-1: CBI8_10H/a, prefix `111001101`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

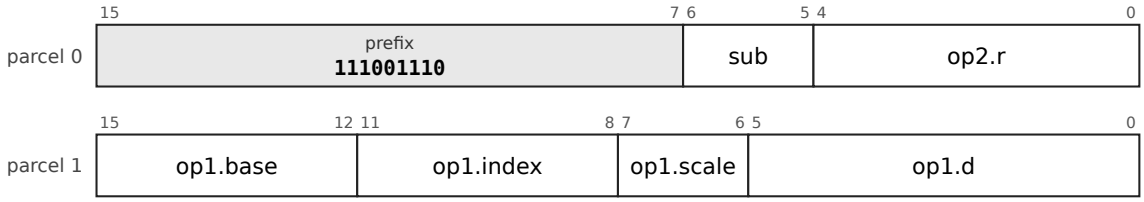
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[6] op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:10] op3.pcrel=p1[9:0]`

### *LDX6H-a-1: LDX6H/a, prefix `111001110`*



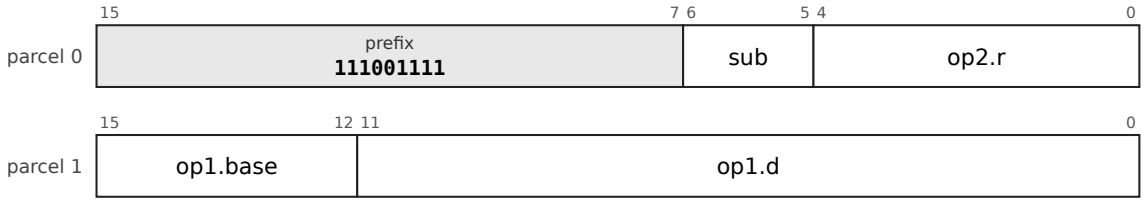

`op1` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits (an index into the hot-register table), index register, 4 bits (an index into the hot-register table).

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[6:5] op2.r=p0[4:0] op1.base=p1[15:12] op1.index=p1[11:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

### *LDBu12H-b-1: LDBu12H/b, prefix `111001111`*



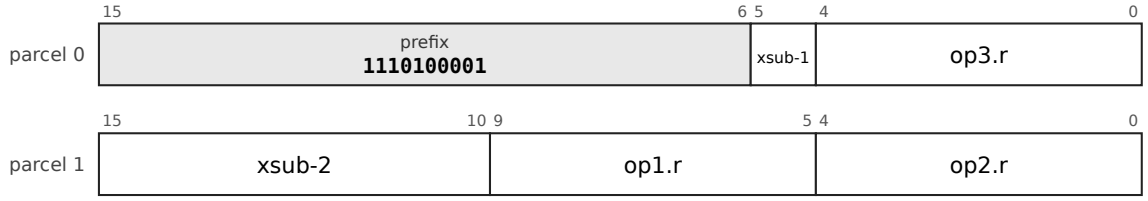

`op1` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[6:5] op2.r=p0[4:0] op1.base=p1[15:12] op1.d=p1[11:0]`

### *R3-1: R3, prefix `1110100001`*




`op1` is `reg5`: register, 5 bits.

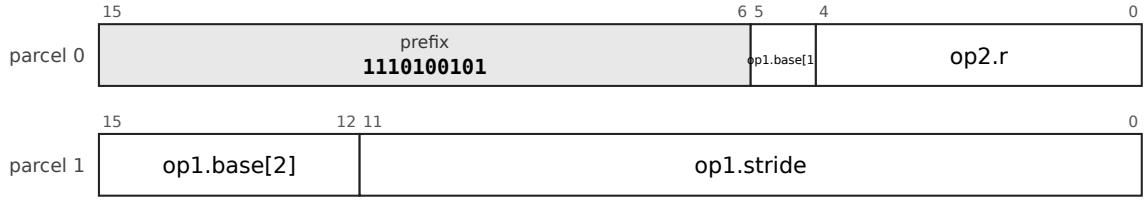
`op2` is `reg5`: register, 5 bits.

`op3` is `reg5`: register, 5 bits.

`xsub` selects the mnemonic.

Fields: `xsub-1=p0[5] op3.r=p0[4:0] xsub-2=p1[15:10] op1.r=p1[9:5] op2.r=p1[4:0]`

### *LDF-PI12-4: LDF.PI12, prefix `1110100101`*

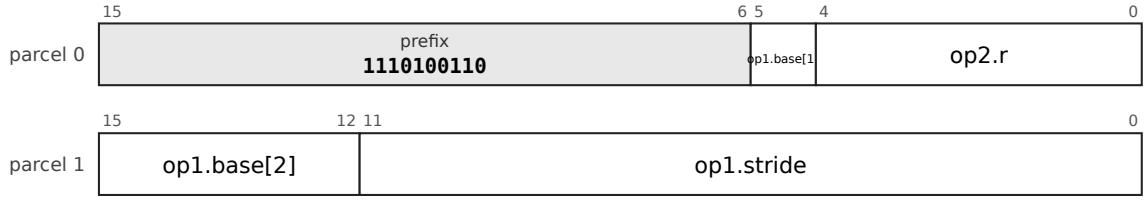



`op1` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *LDF-PD12-4: LDF.PD12, prefix `1110100110`*

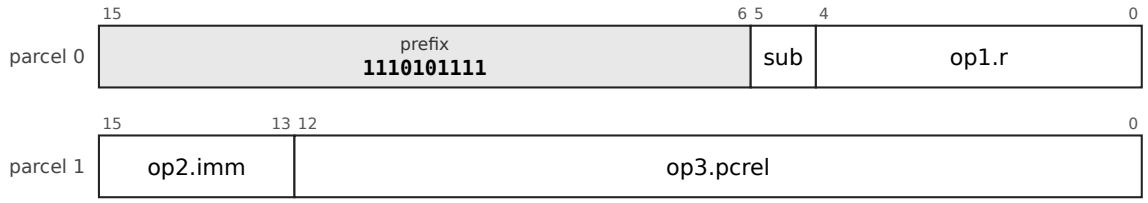



`op1` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.stride=p1[11:0]`

### *TAG-1: TAG, prefix `1110101111`*




`op1` is `reg5`: register, 5 bits.

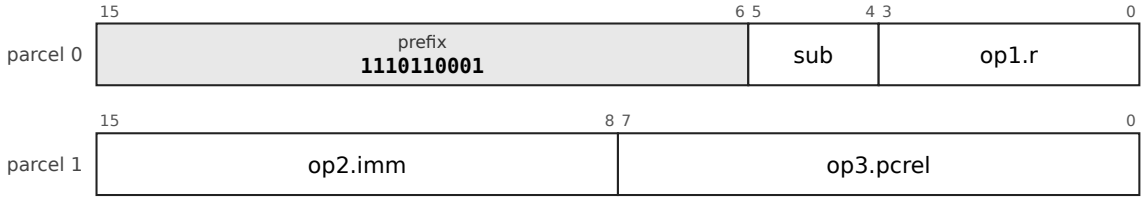
`op2` is `imm3u`: unsigned 3-bit immediate, 0 to 7.

`op3` is `pc13-u2`: pc-relative, 13 bits counted in 2-byte units (-8192 to 8190 bytes).

`sub` (1 bit) selects the mnemonic: `btag` 0, `bntag` 1.

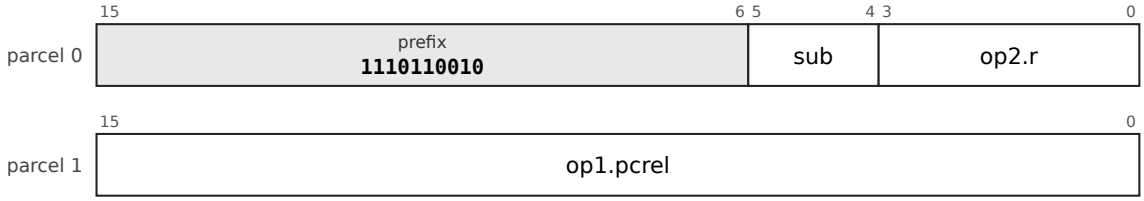
Fields: `sub=p0[5] op1.r=p0[4:0] op2.imm=p1[15:13] op3.pcrel=p1[12:0]`

### *CBI8-8H-b-1: CBI8_8H/b, prefix `1110110001`*



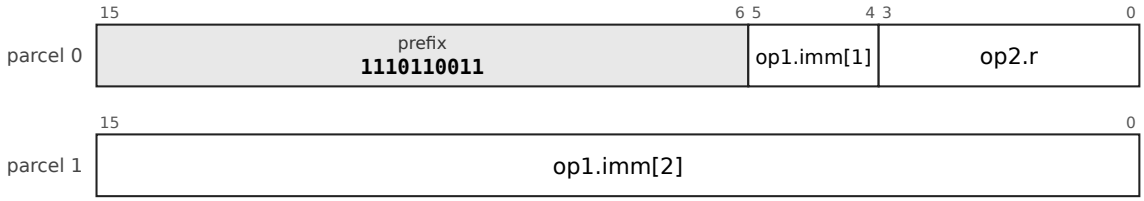

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:8] op3.pcrel=p1[7:0]`

### *LDPC16H-a-1: LDPC16H/a, prefix `1110110010`*



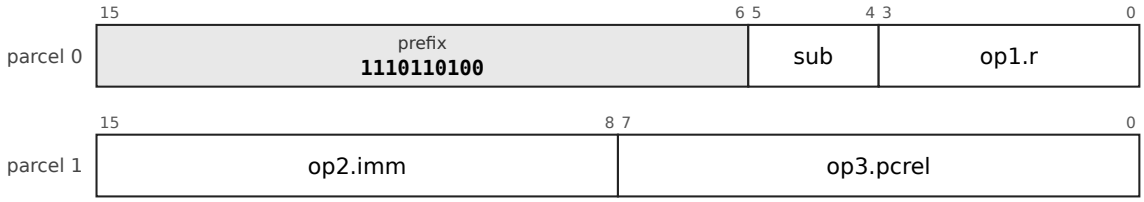

`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.
Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]`

### *LDI18H-b-2: LDI18H/b, prefix `1110110011`*




`op1` is `imm18s`: signed 18-bit immediate, -131072 to 131071.
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
Fields: `op1.imm-1=p0[5:4] op2.r=p0[3:0] op1.imm-2=p1[15:0]`

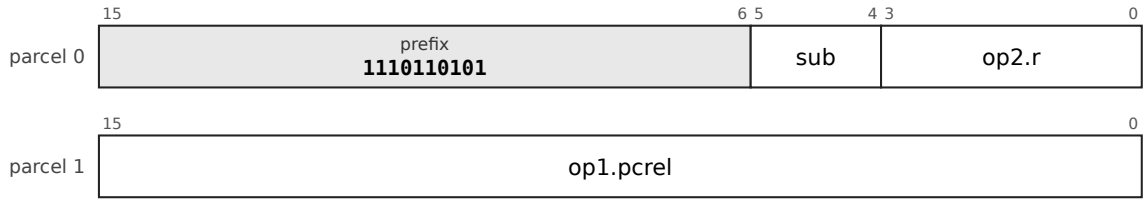
### *CBI8-8H-c-1: CBI8_8H/c, prefix `1110110100`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[5:4] op1.r=p0[3:0] op2.imm=p1[15:8] op3.pcrel=p1[7:0]`

### *LDPC16H-c-1: LDPC16H/c, prefix `1110110101`*



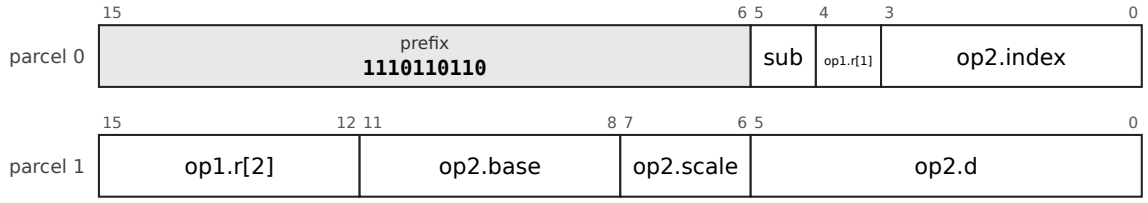

`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[5:4] op2.r=p0[3:0] op1.pcrel=p1[15:0]`

### *STX6H-a-1: STX6H/a, prefix `1110110110`*



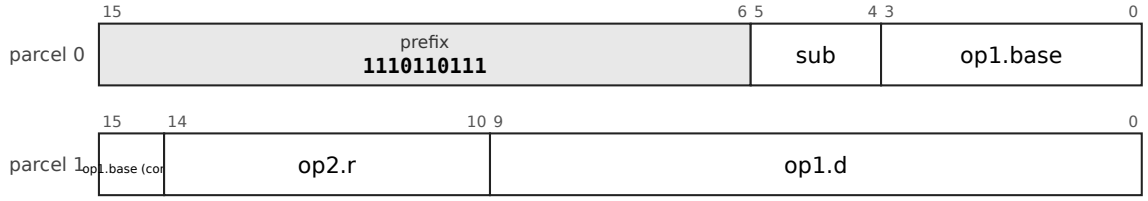

`op1` is `reg5`: register, 5 bits.

`op2` is `x4-4d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 4 bits (an index into the hot-register table), index register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[5] op1.r-1=p0[4] op2.index=p0[3:0] op1.r-2=p1[15:12] op2.base=p1[11:8] op2.scale=p1[7:6] op2.d=p1[5:0]`

### *LDBs10-a-1: LDBs10/a, prefix `1110110111`*



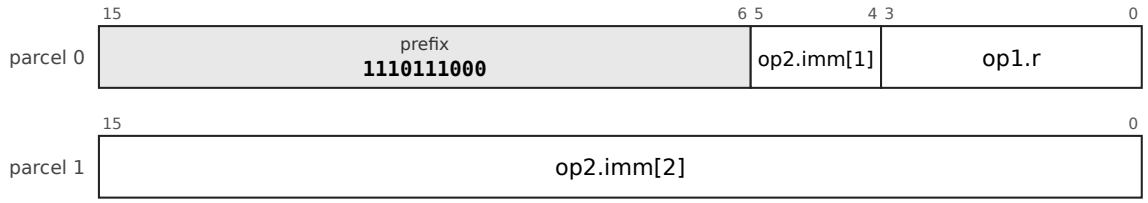

`op1` is `b5d10s-xsz`: memory at base register, 5 bits + signed 10-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[5:4] op1.base=p0[3:0]+p1[15] op2.r=p1[14:10] op1.d=p1[9:0]`

### *CMI18H-a-1: CMI18H/a, prefix `1110111000`*

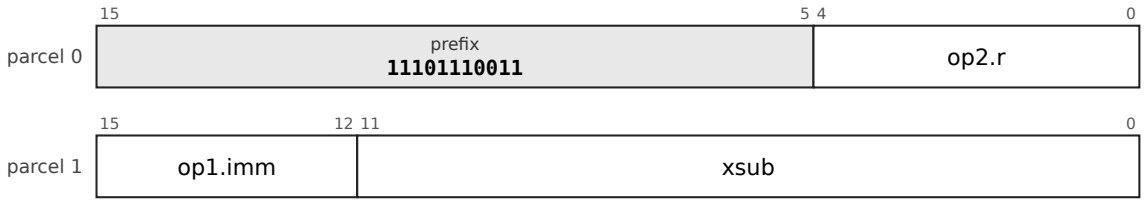



`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `imm18s`: signed 18-bit immediate, -131072 to 131071.

Fields: `op2.imm-1=p0[5:4] op1.r=p0[3:0] op2.imm-2=p1[15:0]`

### *SETF-1: SETF, prefix `11101110011`*



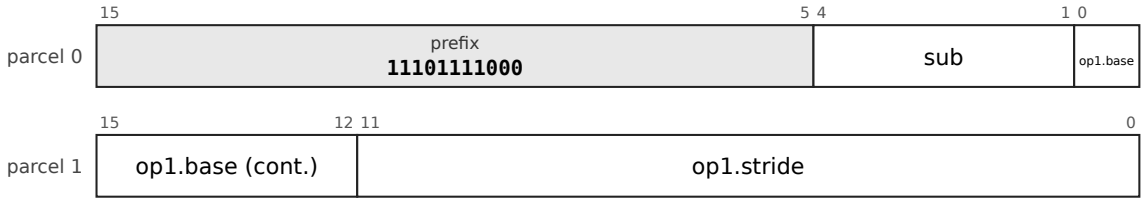

`op1` is `imm4u`: unsigned 4-bit immediate, 0 to 15.

`op2` is `reg5`: register, 5 bits.

`xsub` selects the mnemonic.

Fields: `op2.r=p0[4:0] op1.imm=p1[15:12] xsub=p1[11:0]`

### *SOF-PI12-1: SOF.PI12, prefix `11101111000`*

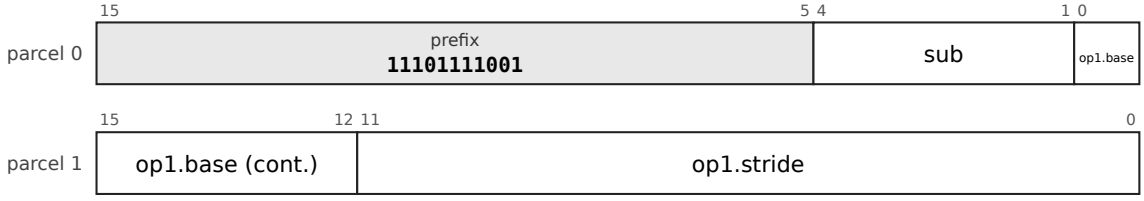



`op1` is `pi5-s12`: post-increment of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[4:1] op1.base=p0[0]+p1[15:12] op1.stride=p1[11:0]`

### *SOF-PD12-1: SOF.PD12, prefix `11101111001`*

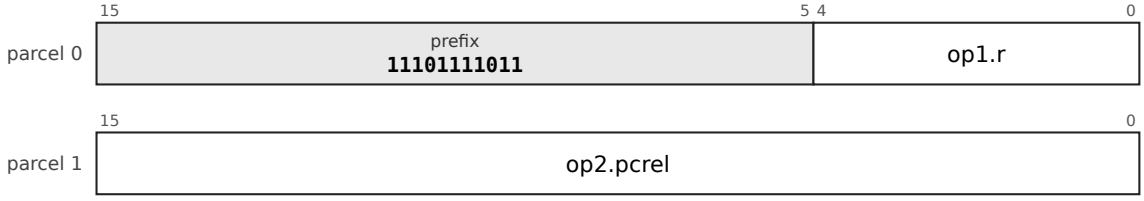



`op1` is `pd5-s12`: pre-decrement of a base register (register, 5 bits), 12-bit stride (0: the size of the access).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[4:1] op1.base=p0[0]+p1[15:12] op1.stride=p1[11:0]`

### *SOB-1: SOB, prefix `11101111011`*

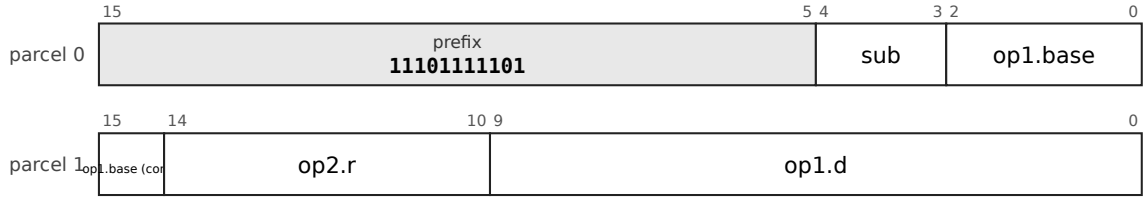



`op1` is `reg5`: register, 5 bits.

`op2` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.r=p0[4:0] op2.pcrel=p1[15:0]`

### *LDBu10H-a-1: LDBu10H/a, prefix `11101111101`*



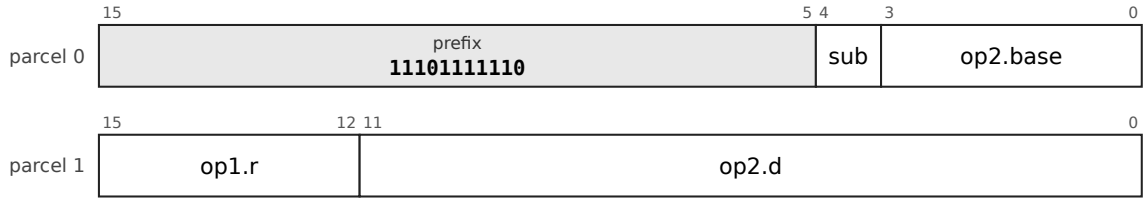

`op1` is `b4d10u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 10-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[4:3] op1.base=p0[2:0]+p1[15] op2.r=p1[14:10] op1.d=p1[9:0]`

### *STBu12H-a-1: STBu12H/a, prefix `11101111110`*



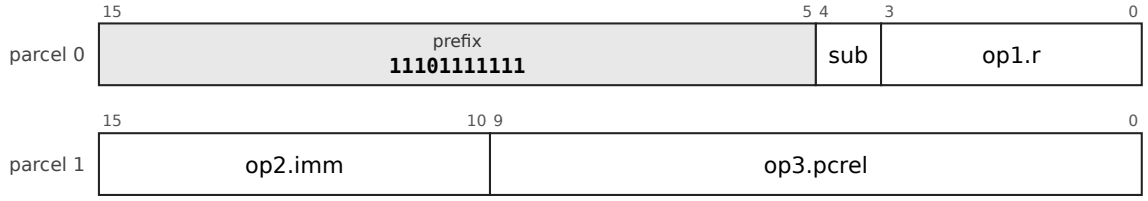

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]`

### *CBI6-10H-a-1: CBI6_10H/a, prefix `11101111111`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

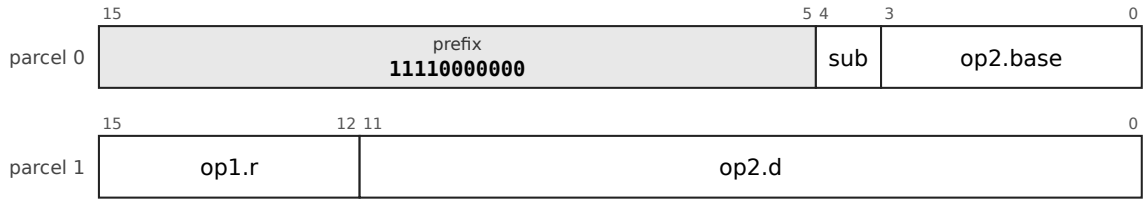
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[4] op1.r=p0[3:0] op2.imm=p1[15:10] op3.pcrel=p1[9:0]`

### *STBu12H-b-1: STBu12H/b, prefix `11110000000`*



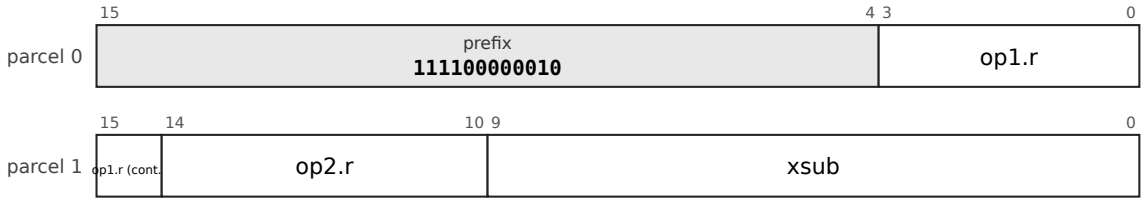

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

op2 is b4d12u-xsz: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

sub (1 bit) selects the mnemonic: movw 0, movb 1.

Fields: sub=p0[4] op2.base=p0[3:0] op1.r=p1[15:12] op2.d=p1[11:0]

### *R2-1: R2, prefix 111100000010*



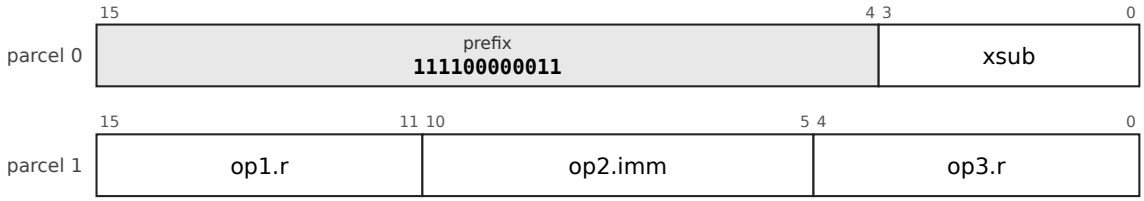

op1 is reg5: register, 5 bits.

op2 is reg5: register, 5 bits.

xsub selects the mnemonic.

Fields: op1.r=p0[3:0]+p1[15] op2.r=p1[14:10] xsub=p1[9:0]

### *RRI-1: RRI, prefix 111100000011*




op1 is reg5: register, 5 bits.

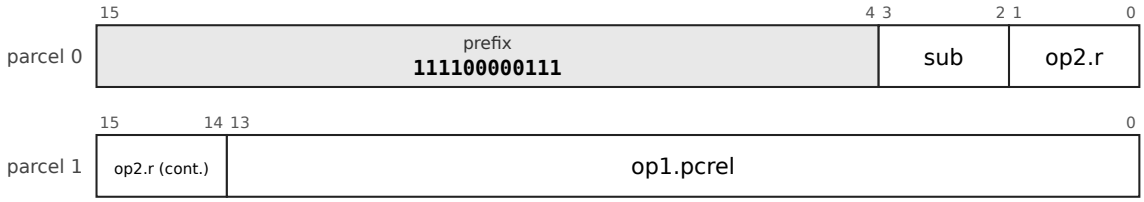
op2 is imm6u: unsigned 6-bit immediate, 0 to 63.

op3 is reg5: register, 5 bits.

xsub selects the mnemonic.

Fields: xsub=p0[3:0] op1.r=p1[15:11] op2.imm=p1[10:5] op3.r=p1[4:0]

### *LDPC14H-a-1: LDPC14H/a, prefix 111100000111*



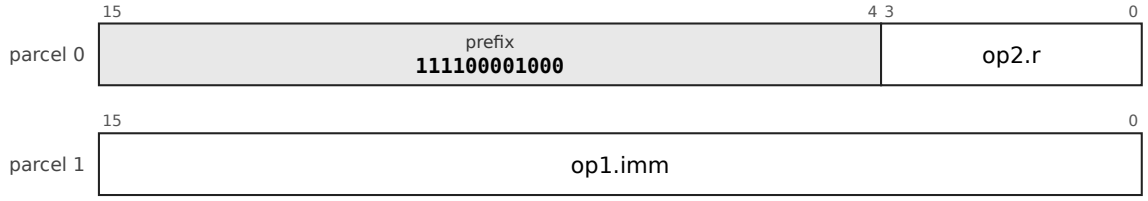

op1 is pc14-u1: pc-relative, 14 bits counted in 1-byte units (-8192 to 8191 bytes).

op2 is reg4: register, 4 bits (an index into the hot-register table).

sub (2 bits) selects the mnemonic: movq 00, movl 01, movzb 10, lea 11.

Fields: sub=p0[3:2] op2.r=p0[1:0]+p1[15:14] op1.pcrel=p1[13:0]

### *LDI16H-a-1: LDI16H/a, prefix `111100001000`*

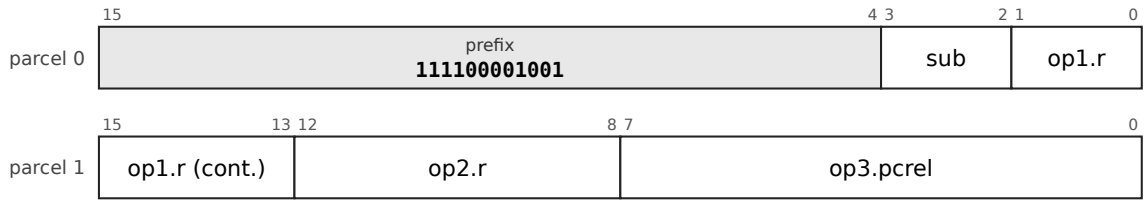



`op1` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p0[3:0] op1.imm=p1[15:0]`

### *CBR8-c-1: CBR8/c, prefix `111100001001`*




`op1` is `reg5`: register, 5 bits.

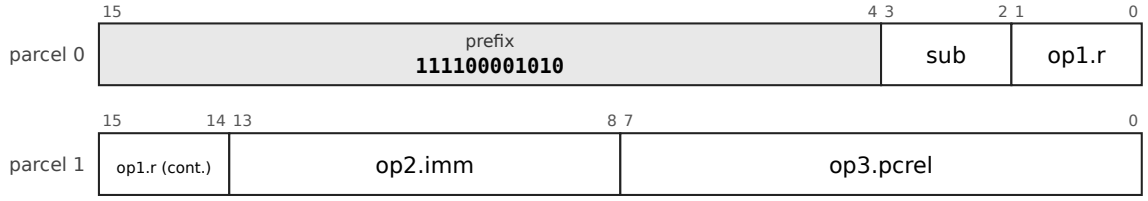
`op2` is `reg5`: register, 5 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:13] op2.r=p1[12:8] op3.pcrel=p1[7:0]`

### *CBI6-8H-c-1: CBI6_8H/c, prefix `111100001010`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

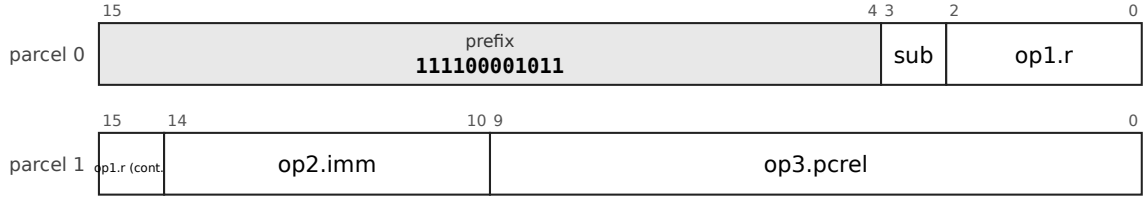
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:8] op3.pcrel=p1[7:0]`

### *CBI5-10H-a-1: CBI5_10H/a, prefix `111100001011`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

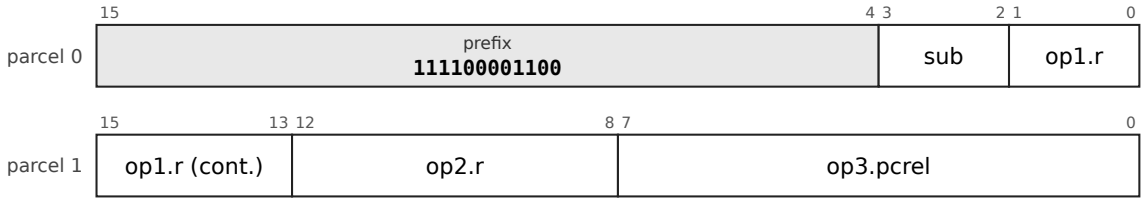
`op2` is `imm5s`: signed 5-bit immediate, -16 to 15.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[3] op1.r=p0[2:0]+p1[15] op2.imm=p1[14:10] op3.pcrel=p1[9:0]`

### *CBR8-b-1: CBR8/b, prefix `111100001100`*




`op1` is `reg5`: register, 5 bits.

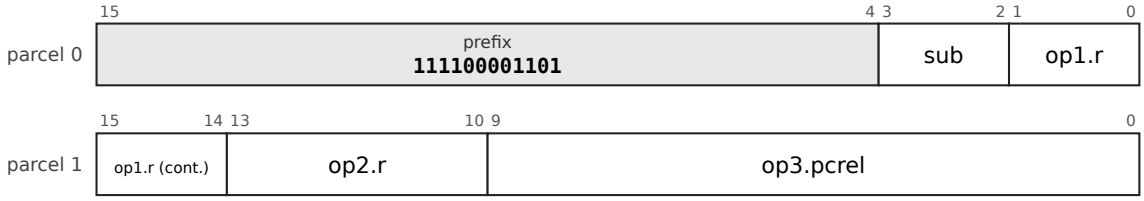
`op2` is `reg5`: register, 5 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:13] op2.r=p1[12:8] op3.pcrel=p1[7:0]`

### *CBR10H-c-1: CBR10H/c, prefix `111100001101`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

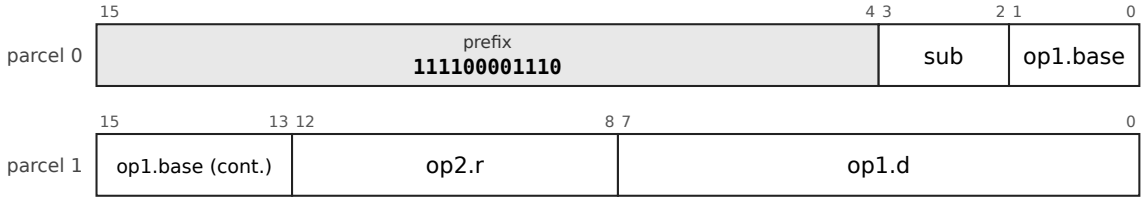
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.r=p1[13:10] op3.pcrel=p1[9:0]`

### *LDBu8-a-1: LDBu8/a, prefix `111100001110`*



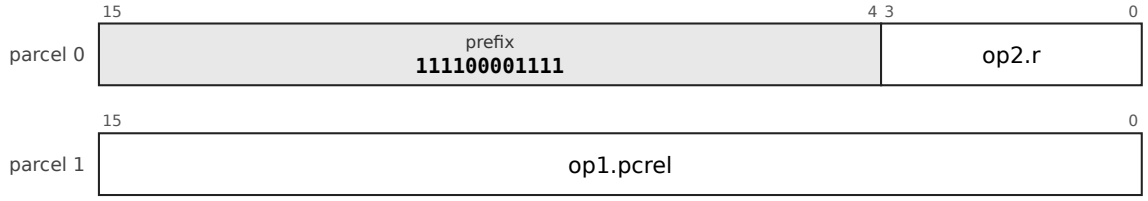

`op1` is `b5d8u-xsz`: memory at base register, 5 bits + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

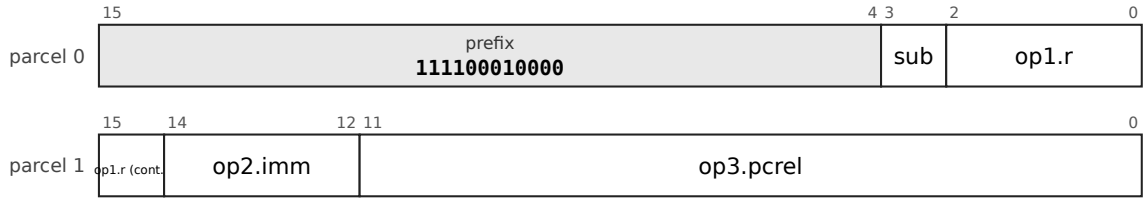
Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *LDPC16H-c-2: LDPC16H/c, prefix `111100001111`*



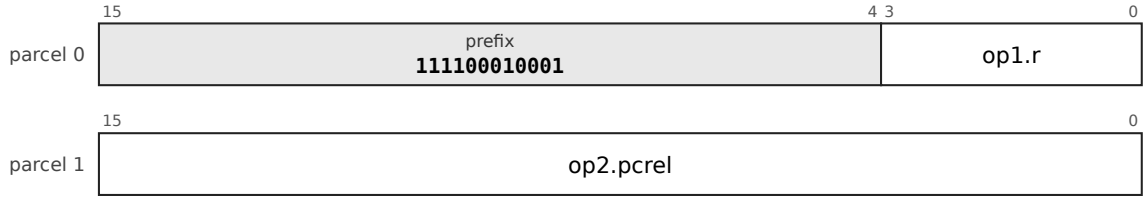

`op1` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
Fields: `op2.r=p0[3:0] op1.pcrel=p1[15:0]`

### *CBI3-12H-a-1: CBI3_12H/a, prefix `111100010000`*



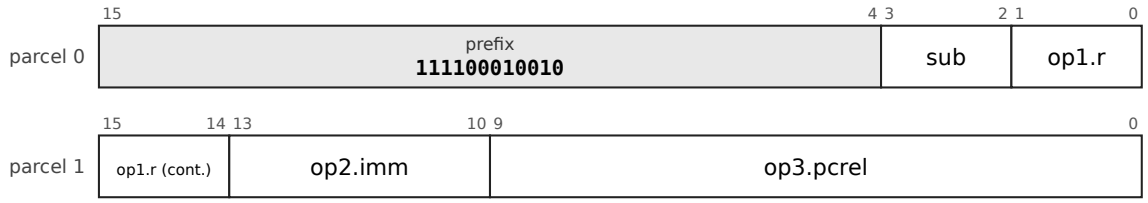

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `imm3s`: signed 3-bit immediate, -4 to 3.
`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[3] op1.r=p0[2:0]+p1[15] op2.imm=p1[14:12] op3.pcrel=p1[11:0]`

### *CMPC16H-a-1: CMPC16H/a, prefix `111100010001`*



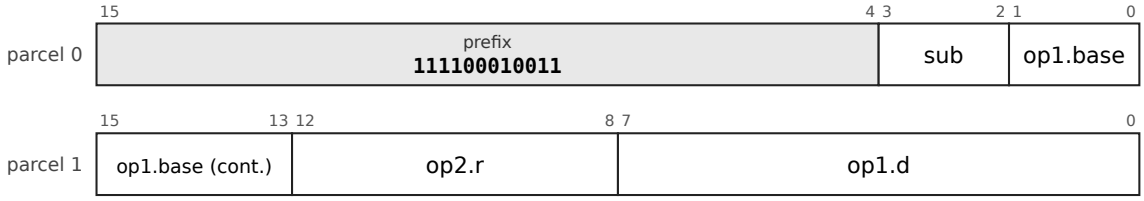

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `pc16-u1`: pc-relative, 16 bits counted in 1-byte units (-32768 to 32767 bytes).
Fields: `op1.r=p0[3:0] op2.pcrel=p1[15:0]`

### *CBI4-10H-c-1: CBI4_10H/c, prefix `111100010010`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.
`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:10] op3.pcrel=p1[9:0]`

### *LDBu8-c-1: LDBu8/c, prefix `111100010011`*



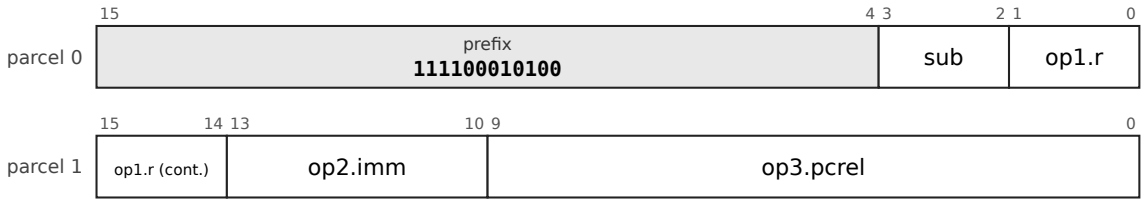

`op1` is `b5d8u-xsz`: memory at base register, 5 bits + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[3:2] op1.base=p0[1:0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *CBI4-10H-b-1: CBI4_10H/b, prefix `111100010100`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

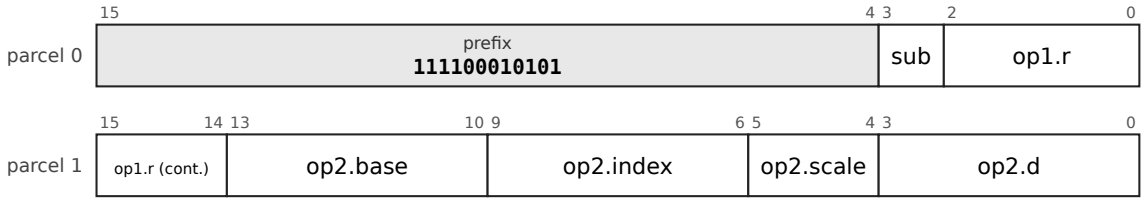
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[3:2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:10] op3.pcrel=p1[9:0]`

### *STX4H-b-1: STX4H/b, prefix `111100010101`*



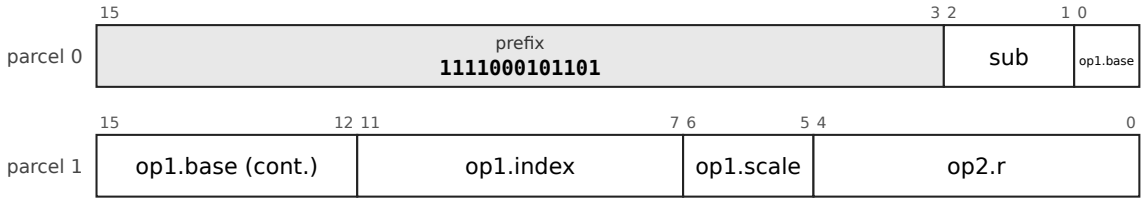

`op1` is `reg5`: register, 5 bits.

`op2` is `x4-4d4s`: memory at base + (index << scale) + signed 4-bit displacement; base register, 4 bits (an index into the hot-register table), index register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[3] op1.r=p0[2:0]+p1[15:14] op2.base=p1[13:10] op2.index=p1[9:6] op2.scale=p1[5:4] op2.d=p1[3:0]`

### *LDX0-a-1: LDX0/a, prefix `1111000101101`*



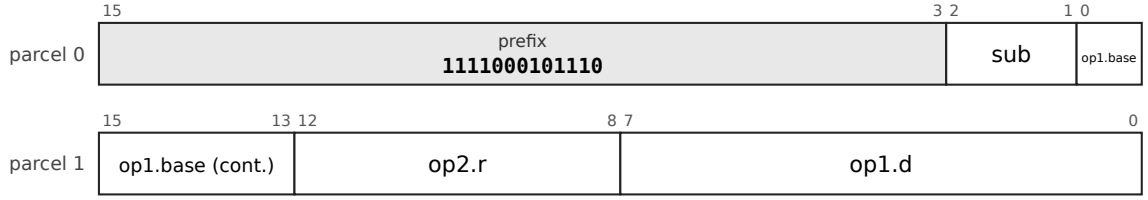

`op1` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:12] op1.index=p1[11:7] op1.scale=p1[6:5] op2.r=p1[4:0]`

### *LDBu8H-a-1: LDBu8H/a, prefix `1111000101110`*



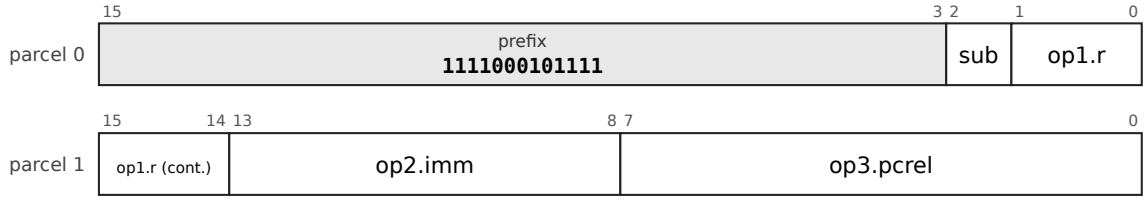

`op1` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *CBI6-8H-a-1: CBI6_8H/a, prefix `1111000101111`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

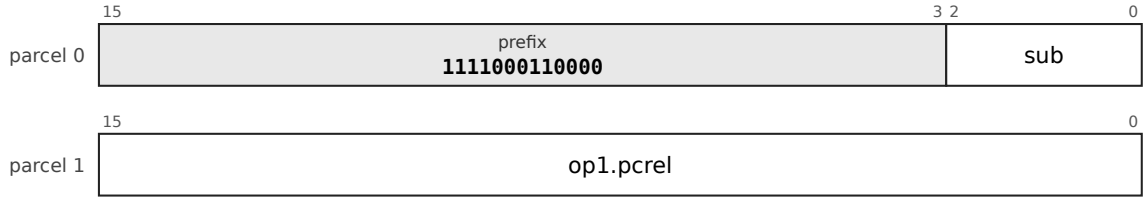
`op2` is `imm6s`: signed 6-bit immediate, -32 to 31.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.imm=p1[13:8] op3.pcrel=p1[7:0]`

### *BR2-16-c-1: BR2_16/c, prefix `1111000110000`*

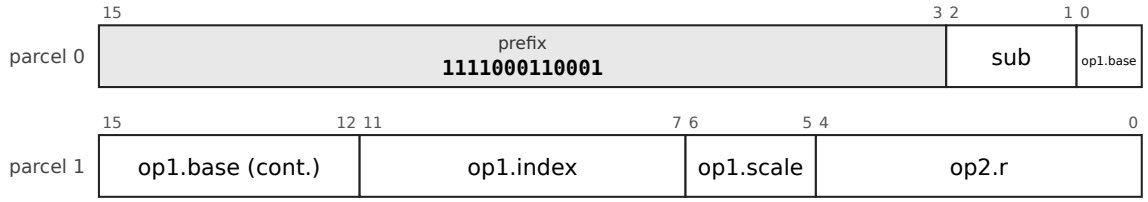



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

`sub` (3 bits) selects the mnemonic: `bpl` 000, `bmi` 001, `bvc` 010, `bvs` 011, `bcc` 100, `bcs` 101, `bge` 110, `blt` 111.

Fields: `sub=p0[2:0] op1.pcrel=p1[15:0]`

### *LDX0-b-1: LDX0/b, prefix `1111000110001`*



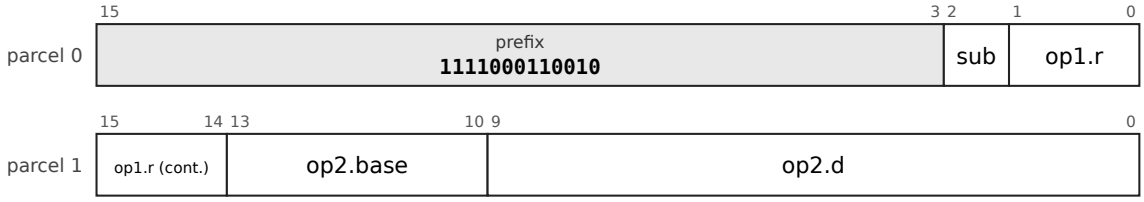

`op1` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:12] op1.index=p1[11:7] op1.scale=p1[6:5] op2.r=p1[4:0]`

### *STBu10H-a-1: STBu10H/a, prefix `1111000110010`*



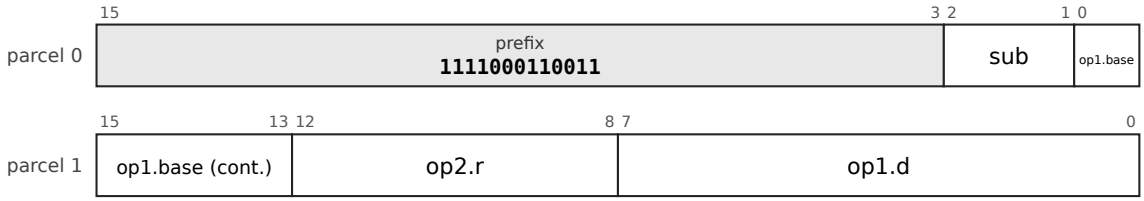

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d10u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 10-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

### *LDBu8H-c-1: LDBu8H/c, prefix `1111000110011`*



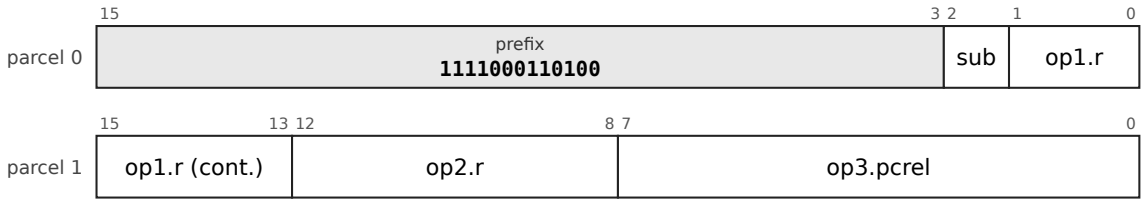

`op1` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *CBR8-a-1: CBR8/a, prefix `1111000110100`*




`op1` is `reg5`: register, 5 bits.

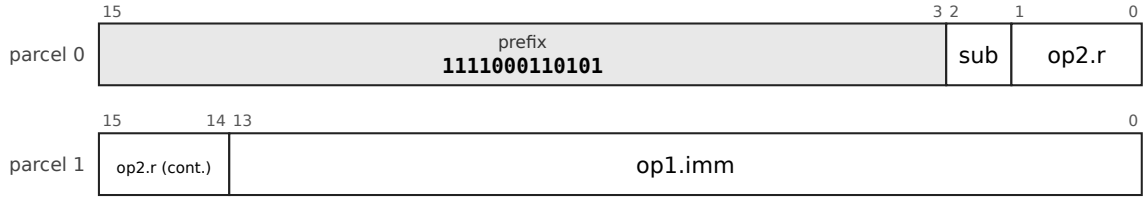
`op2` is `reg5`: register, 5 bits.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:13] op2.r=p1[12:8] op3.pcrel=p1[7:0]`

### *LDI14H-b-1: LDI14H/b, prefix `1111000110101`*



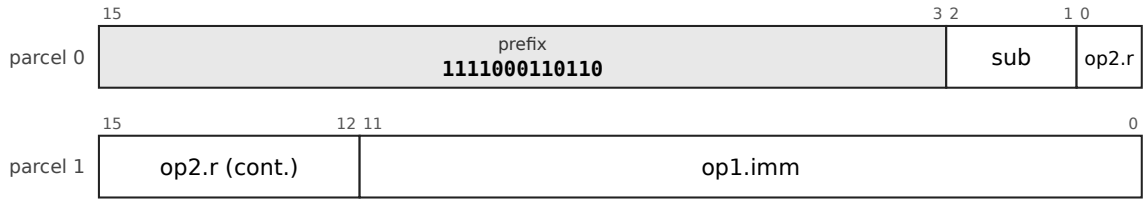

`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[2] op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

### *LDI12-d-1: LDI12/d, prefix `1111000110110`*



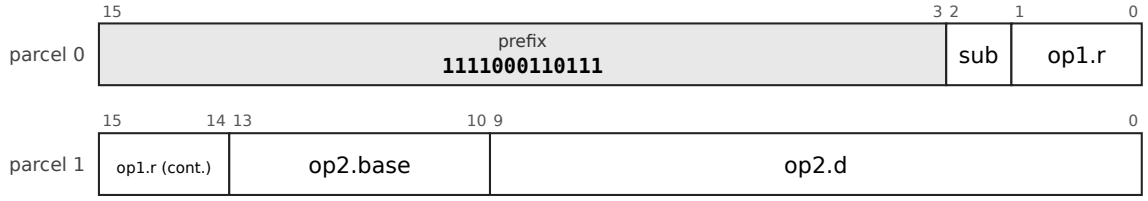

`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `shl` 00, `shr` 01, `sar` 10, `mul` 11.

Fields: `sub=p0[2:1] op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *STBu10H-b-1: STBu10H/b, prefix `1111000110111`*



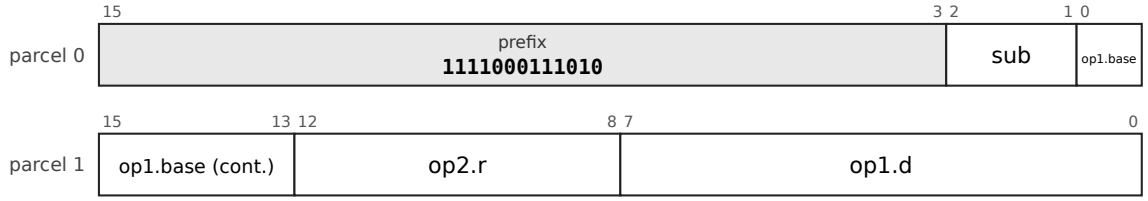

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d10u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 10-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

### *LDBu8H-b-1: LDBu8H/b, prefix `1111000111010`*



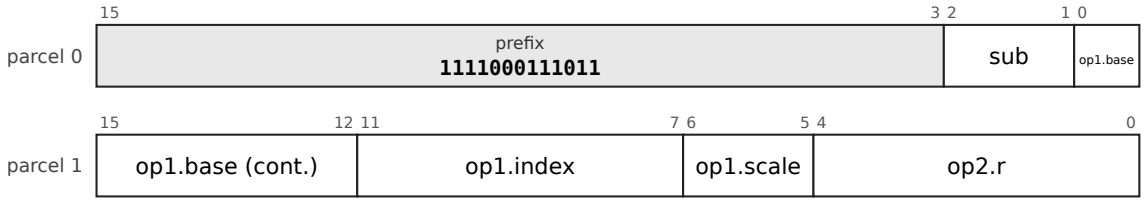

`op1` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *LDX0-c-1: LDX0/c, prefix `1111000111011`*



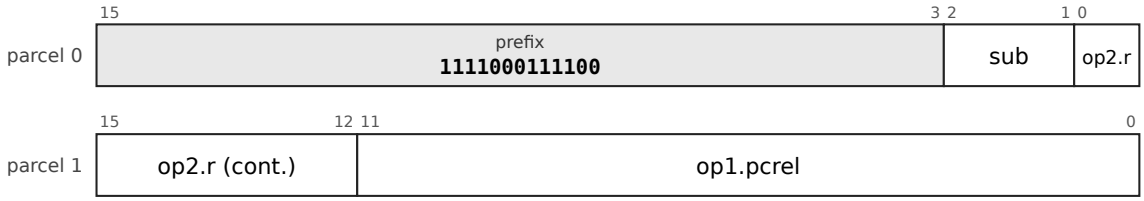

`op1` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:12] op1.index=p1[11:7] op1.scale=p1[6:5] op2.r=p1[4:0]`

### *LDPC12-a-1: LDPC12/a, prefix `1111000111100`*



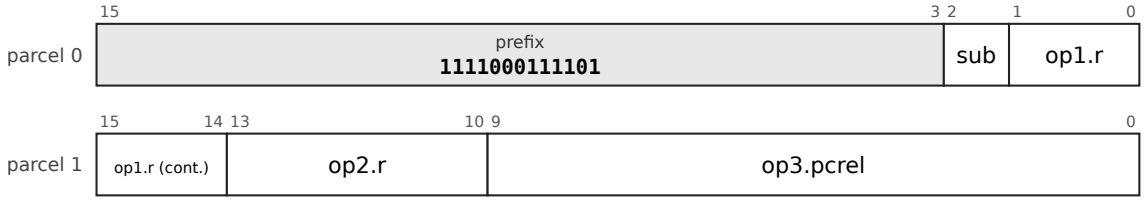

`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[2:1] op2.r=p0[0]+p1[15:12] op1.pcrel=p1[11:0]`

### *CBR10H-a-1: CBR10H/a, prefix `1111000111101`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

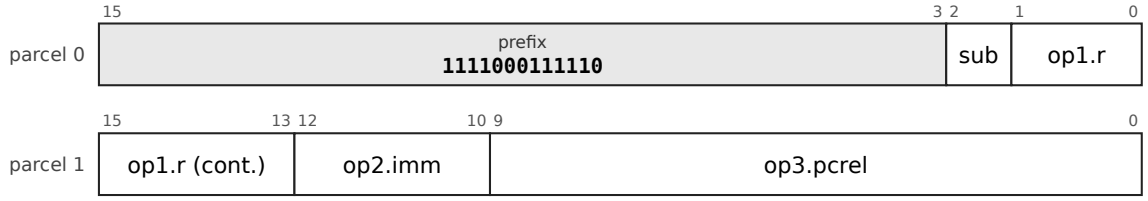
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op2.r=p1[13:10] op3.pcrel=p1[9:0]`

### *CBI3-10-a-1: CBI3_10/a, prefix `1111000111110`*




`op1` is `reg5`: register, 5 bits.

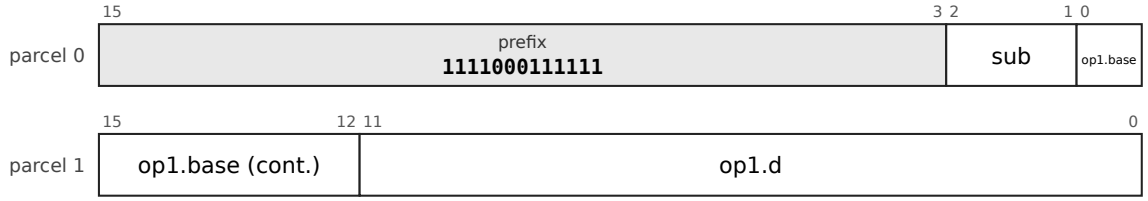
`op2` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op3` is `pc10-u2`: pc-relative, 10 bits counted in 2-byte units (-1024 to 1022 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:13] op2.imm=p1[12:10] op3.pcrel=p1[9:0]`

### *SOBs12-a-1: SOBs12/a, prefix `1111000111111`*

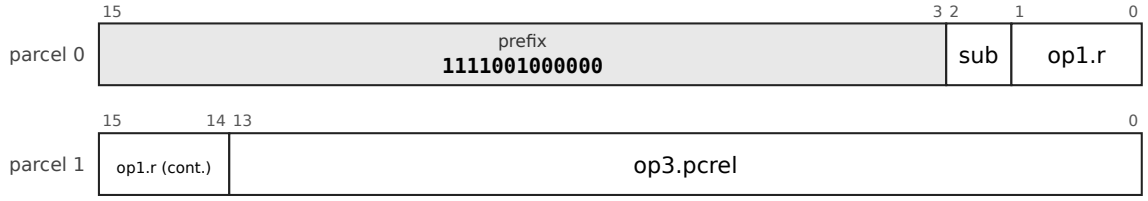



`op1` is `b5d12s-xsz`: memory at base register, 5 bits + signed 12-bit displacement times the size of the access.

`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.

Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:12] op1.d=p1[11:0]`

### *CBZ2-14H-a-1: CBZ2_14H/a, prefix `1111001000000`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

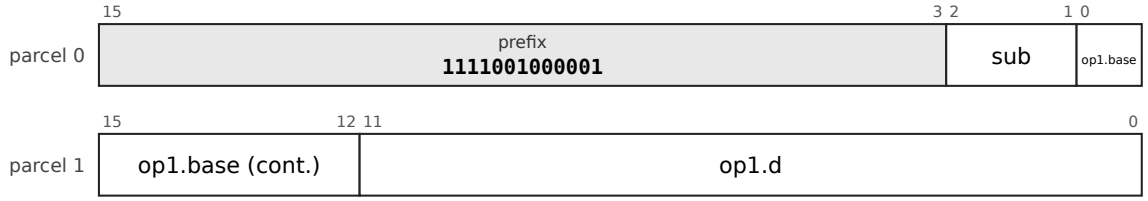
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc14-u2`: pc-relative, 14 bits counted in 2-byte units (-16384 to 16382 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:14] op3.pcrel=p1[13:0]`

### *SOBs12-x-1: SOBs12/x, prefix `1111001000001`*

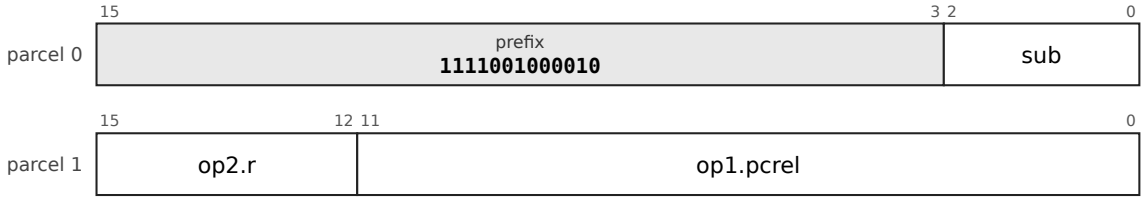



`op1` is `b5d12s-xsz`: memory at base register, 5 bits + signed 12-bit displacement times the size of the access.

`sub` (2 bits) selects the mnemonic: `jmp` 00, `call` 01, `jmpi` 10, `calli` 11.

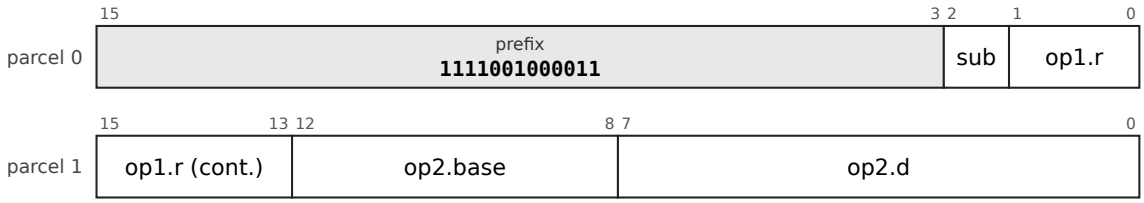
Fields: `sub=p0[2:1] op1.base=p0[0]+p1[15:12] op1.d=p1[11:0]`

### *LDPC12H-d-1: LDPC12H/d, prefix `1111001000010`*



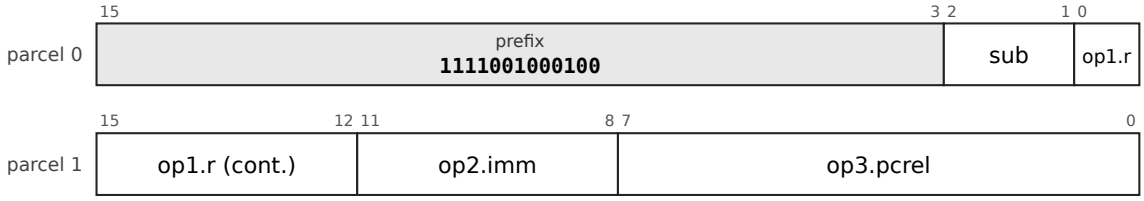

`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
`sub` (3 bits) selects the mnemonic: `bic` 000, `mul` 001, `mulh` 010, `shl` 011, `shr` 100, `sar` 101, `fadd` 110, `fsub` 111.
Fields: `sub=p0[2:0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

### *STBs8-a-1: STBs8/a, prefix `1111001000011`*



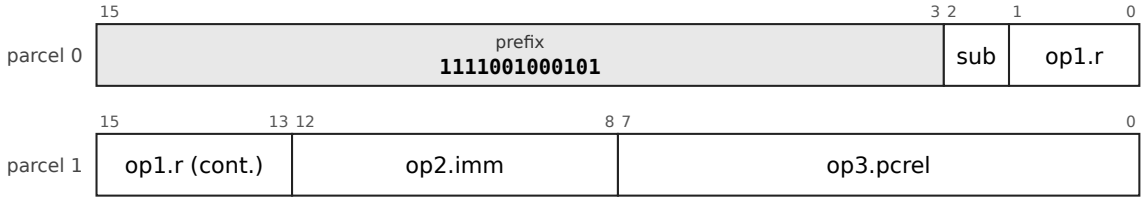

`op1` is `reg5`: register, 5 bits.
`op2` is `b5d8s-xsz`: memory at base register, 5 bits + signed 8-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.
Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:13] op2.base=p1[12:8] op2.d=p1[7:0]`

### *CBI4-8-b-1: CBI4_8/b, prefix `1111001000100`*




`op1` is `reg5`: register, 5 bits.
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[2:1] op1.r=p0[0]+p1[15:12] op2.imm=p1[11:8] op3.pcrel=p1[7:0]`

### *CBI5-8-a-1: CBI5_8/a, prefix `1111001000101`*

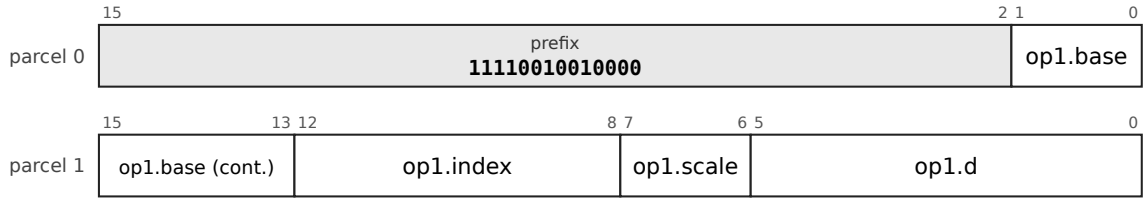



`op1` is `reg5`: register, 5 bits.
`op2` is `imm5s`: signed 5-bit immediate, -16 to 15.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[2] op1.r=p0[1:0]+p1[15:13] op2.imm=p1[12:8] op3.pcrel=p1[7:0]`

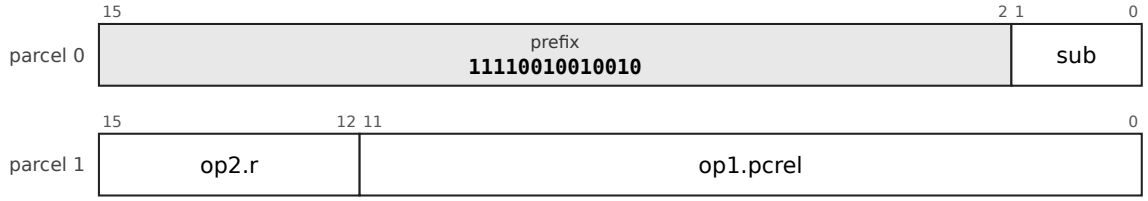
### *JTAG2-1: JTAG2, prefix `11110010010000`*




`op1` is `x5-5d6s`: memory at base + (index << scale) + signed 6-bit displacement; base register, 5 bits, index register, 5 bits.

Fields: `op1.base=p0[1:0]+p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]`

### *LDPC12H-a-1: LDPC12H/a, prefix `11110010010010`*



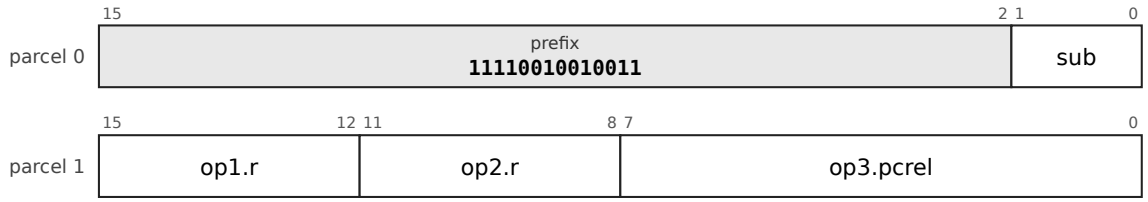

`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

### *CBR8H-c-1: CBR8H/c, prefix `11110010010011`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

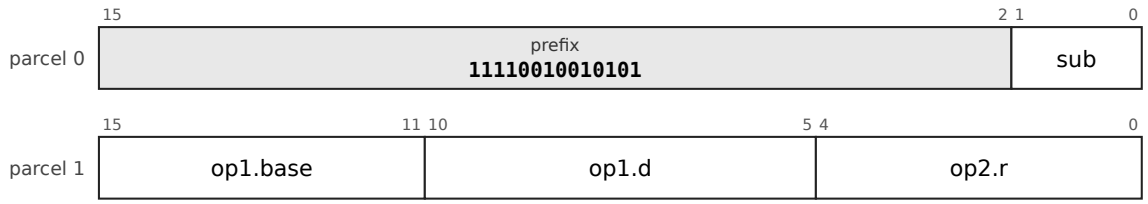
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

### *LDBu6-b-1: LDBu6/b, prefix `11110010010101`*



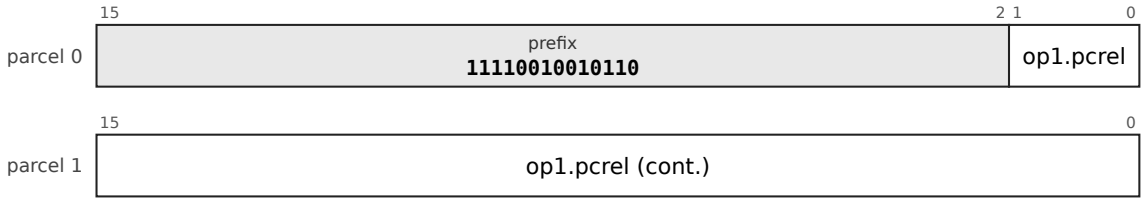

`op1` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.d=p1[10:5] op2.r=p1[4:0]`

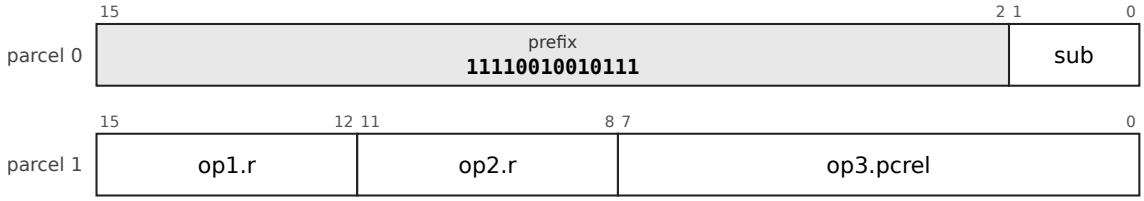
### *CALL2-18-c-1: CALL2_18/c, prefix `11110010010110`*




`op1` is `pc18-u2`: pc-relative, 18 bits counted in 2-byte units (-262144 to 262142 bytes).

Fields: `op1.pcrel=p0[1:0]+p1[15:0]`

### *CBR8H-b-1: CBR8H/b, prefix `11110010010111`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

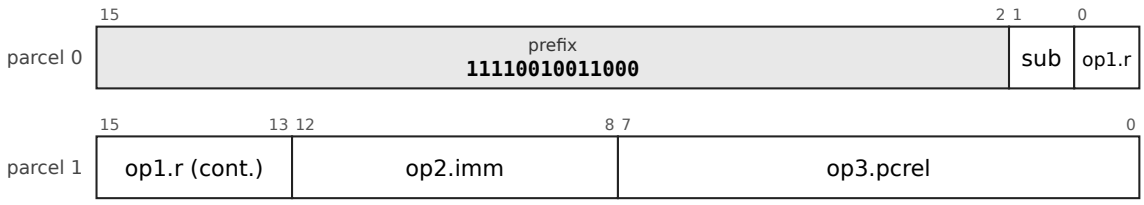
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

### *CBI5-8H-a-1: CBI5_8H/a, prefix `11110010011000`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

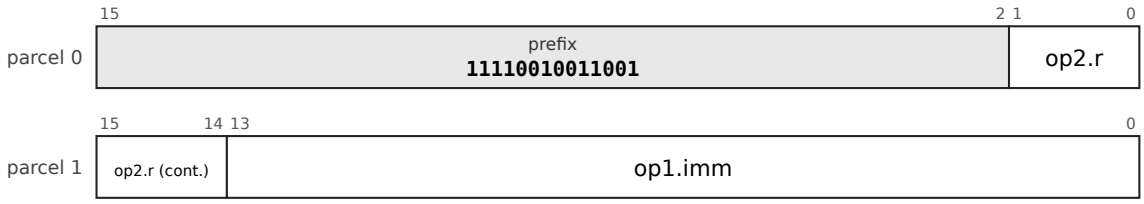
`op2` is `imm5s`: signed 5-bit immediate, -16 to 15.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[1] op1.r=p0[0]+p1[15:13] op2.imm=p1[12:8] op3.pcrel=p1[7:0]`

### *LDI14H-a-1: LDI14H/a, prefix `11110010011001`*

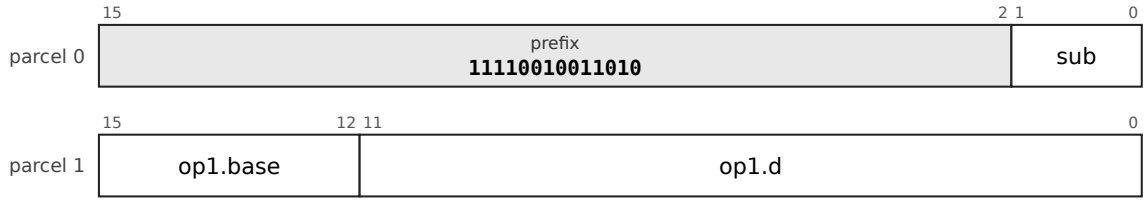



`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

### *SOBu12H-a-1: SOBu12H/a, prefix `11110010011010`*

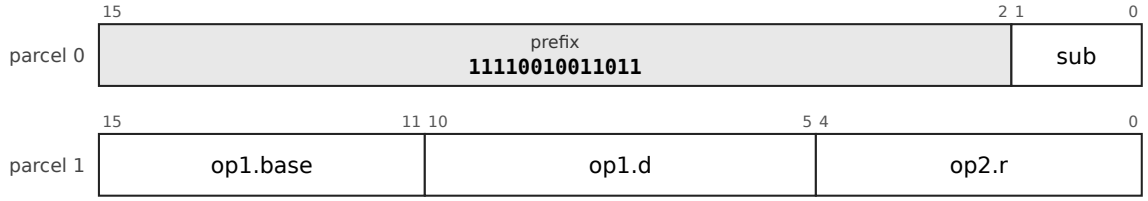



`op1` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.d=p1[11:0]`

### *LDBu6-c-1: LDBu6/c, prefix `11110010011011`*



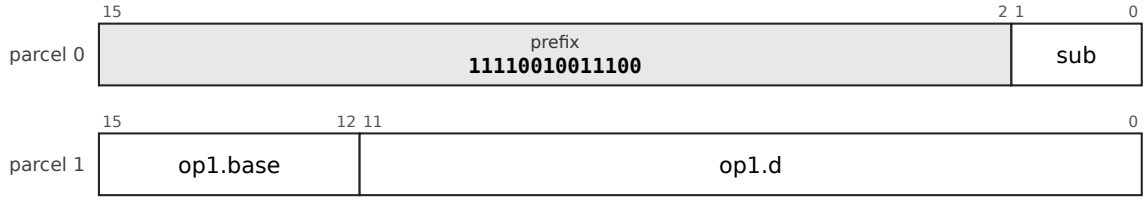

`op1` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.d=p1[10:5] op2.r=p1[4:0]`

### *SOBu12H-x-1: SOBu12H/x, prefix `11110010011100`*

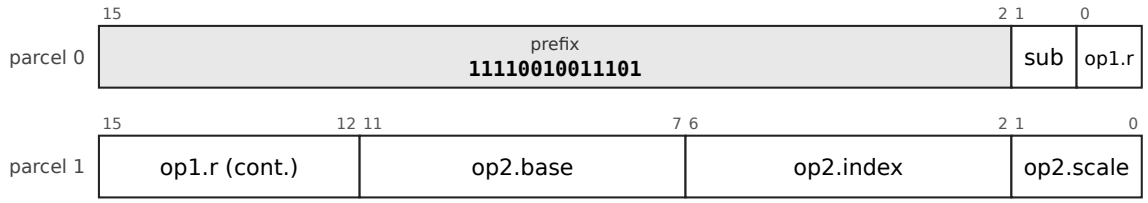



`op1` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`sub` (2 bits) selects the mnemonic: `jmp` 00, `call` 01, `jmpi` 10, `calli` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:12] op1.d=p1[11:0]`

### *STX0-b-1: STX0/b, prefix `11110010011101`*



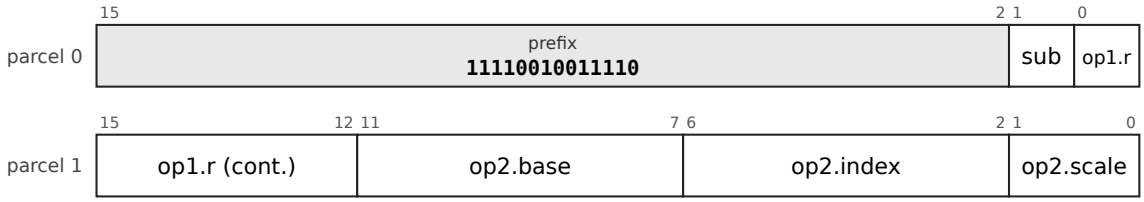

`op1` is `reg5`: register, 5 bits.

`op2` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[1] op1.r=p0[0]+p1[15:12] op2.base=p1[11:7] op2.index=p1[6:2] op2.scale=p1[1:0]`

### *STX0-a-1: STX0/a, prefix `11110010011110`*



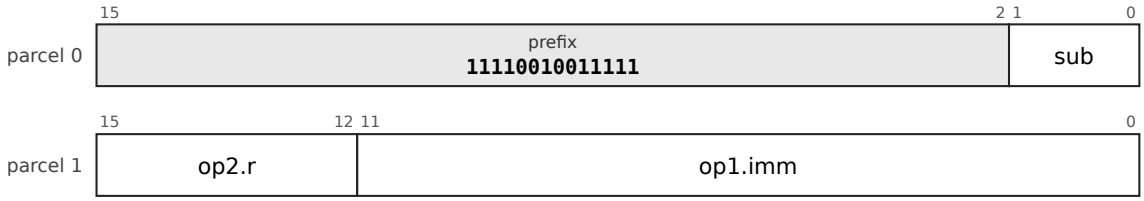

`op1` is `reg5`: register, 5 bits.

`op2` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[1] op1.r=p0[0]+p1[15:12] op2.base=p1[11:7] op2.index=p1[6:2] op2.scale=p1[1:0]`

### *LDI12H-c-1: LDI12H/c, prefix `11110010011111`*



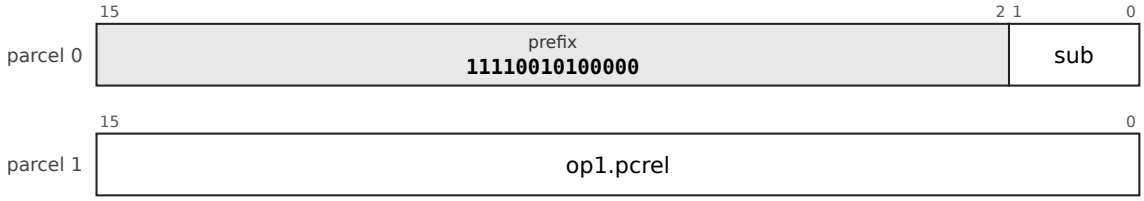

`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `bis` 00, `xor` 01, `mul` 10, `bic` 11.

Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.imm=p1[11:0]`

### *BR2-16-c-2: BR2_16/c, prefix `11110010100000`*

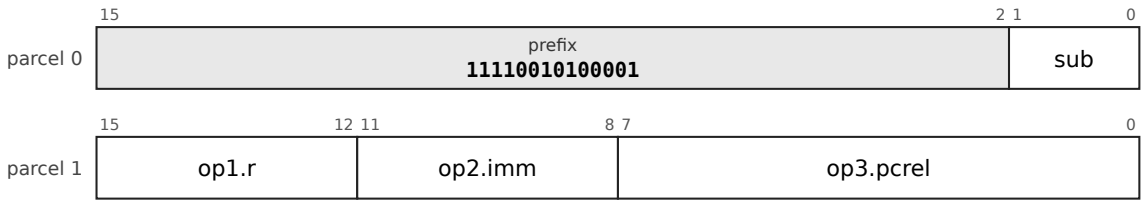



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

`sub` (2 bits) selects the mnemonic: `bgt` 00, `ble` 01, `bhi` 10, `blos` 11.

Fields: `sub=p0[1:0] op1.pcrel=p1[15:0]`

### *CBI4-8H-c-1: CBI4_8H/c, prefix `11110010100001`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

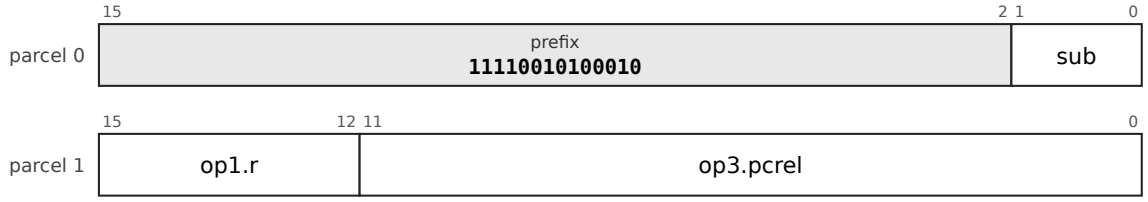
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:8] op3.pcrel=p1[7:0]`

### *CBZ2-12H-b-1: CBZ2_12H/b, prefix `1111001010010`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

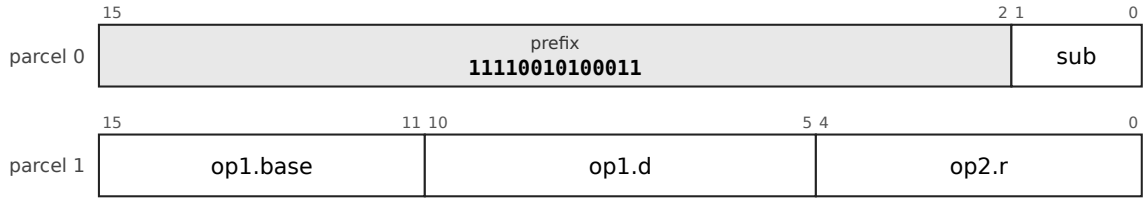
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op3.pcrel=p1[11:0]`

### *LDBs6-a-1: LDBs6/a, prefix `1111001010011`*



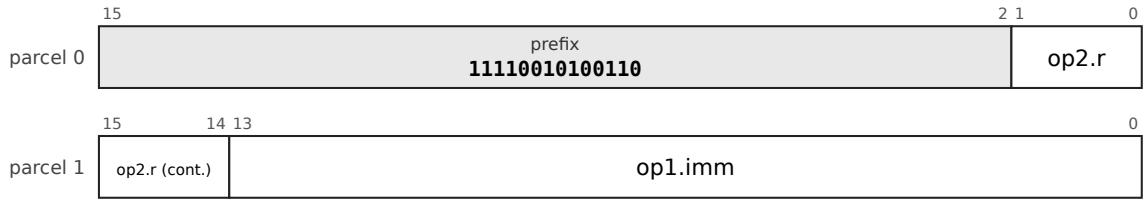

`op1` is `b5d6s-xsz`: memory at base register, 5 bits + signed 6-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movq` 00, `movl` 01, `movzb` 10, `lea` 11.

Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.d=p1[10:5] op2.r=p1[4:0]`

### *LDI14H-b-2: LDI14H/b, prefix `1111001010011`0*

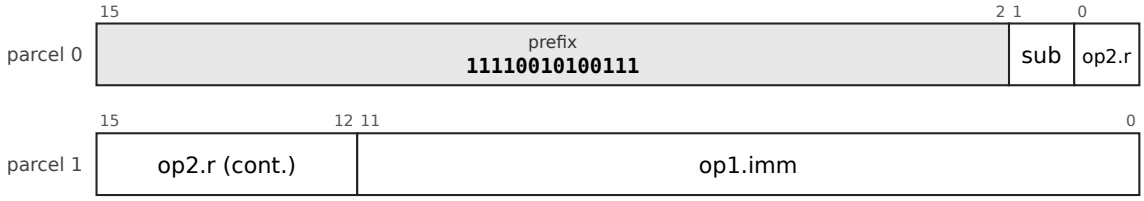



`op1` is `imm14s`: signed 14-bit immediate, -8192 to 8191.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p0[1:0]+p1[15:14] op1.imm=p1[13:0]`

### *LDI12-d-2: LDI12/d, prefix `11110010100111`*



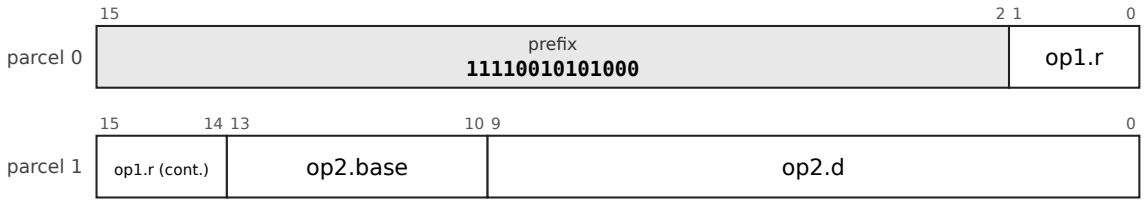

`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg5`: register, 5 bits.

`sub` (1 bit) selects the mnemonic: `bic` 0, `movb` 1.

Fields: `sub=p0[1] op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *CMBu10H-a-1: CMBu10H/a, prefix `11110010101000`*

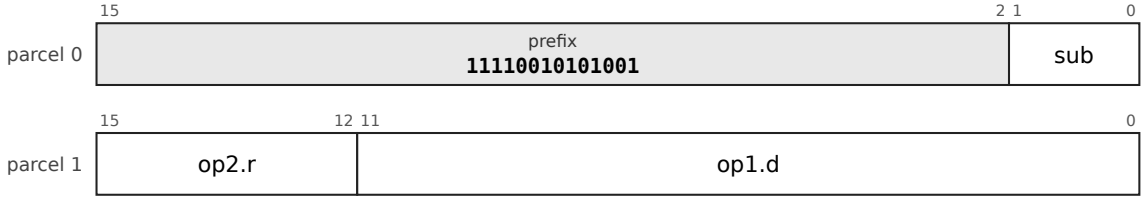



`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d10u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 10-bit displacement times the size of the access.

Fields: `op1.r=p0[1:0]+p1[15:14] op2.base=p1[13:10] op2.d=p1[9:0]`

### *LDS12H-c-1: LDS12H/c, prefix `11110010101001`*



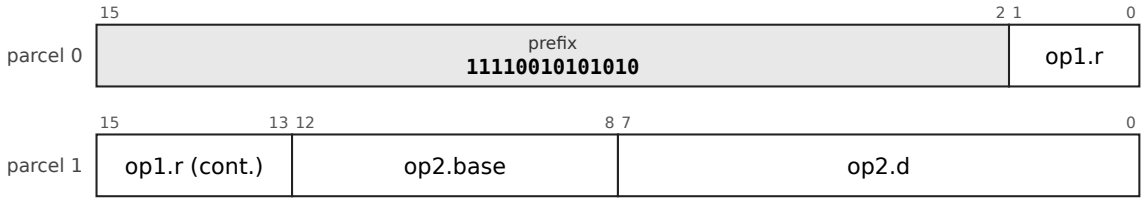

`op1` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (2 bits) selects the mnemonic: `add` 00, `sub` 01, `and` 10, `bis` 11.

Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.d=p1[11:0]`

### *CMBu8-a-1: CMBu8/a, prefix `11110010101010`*

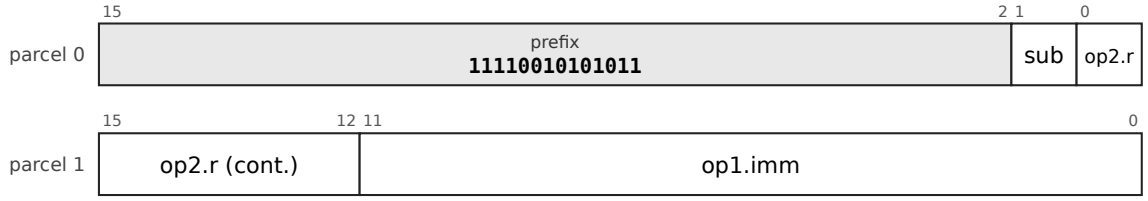



`op1` is `reg5`: register, 5 bits.

`op2` is `b5d8u-xsz`: memory at base register, 5 bits + unsigned 8-bit displacement times the size of the access.

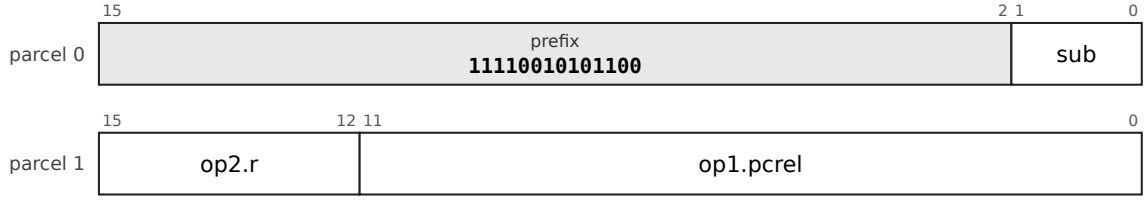
Fields: `op1.r=p0[1:0]+p1[15:13] op2.base=p1[12:8] op2.d=p1[7:0]`

### *LDI12-b-1: LDI12/b, prefix `11110010101011`*



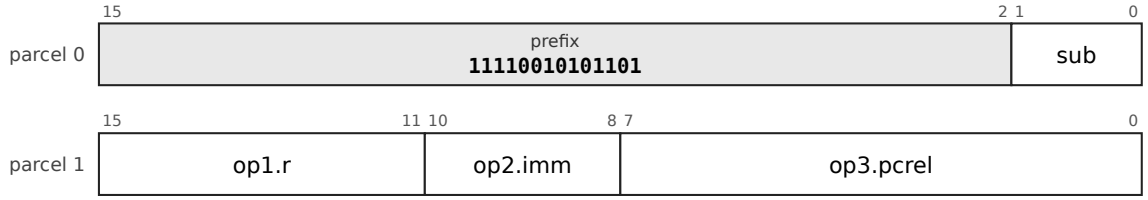

`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.
Fields: `sub=p0[1] op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *LDPC12H-b-1: LDPC12H/b, prefix `11110010101100`*



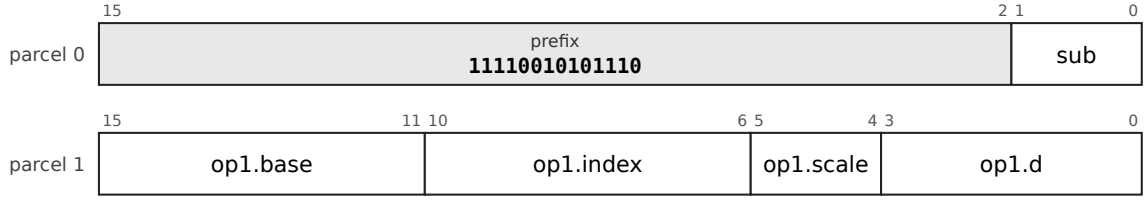

`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

### *CBI3-8-b-1: CBI3_8/b, prefix `11110010101101`*



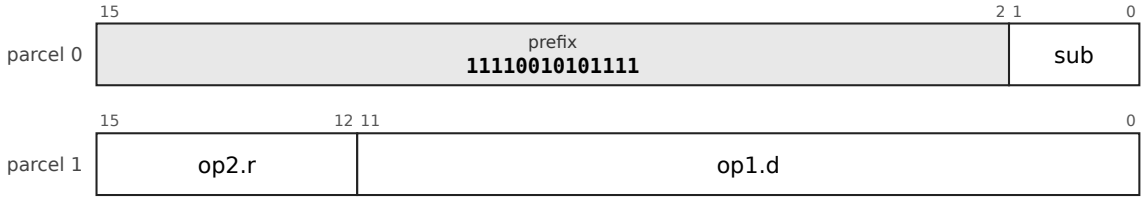

`op1` is `reg5`: register, 5 bits.
`op2` is `imm3s`: signed 3-bit immediate, -4 to 3.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[1:0] op1.r=p1[15:11] op2.imm=p1[10:8] op3.pcrel=p1[7:0]`

### *SOX4-a-1: SOX4/a, prefix `11110010101110`*



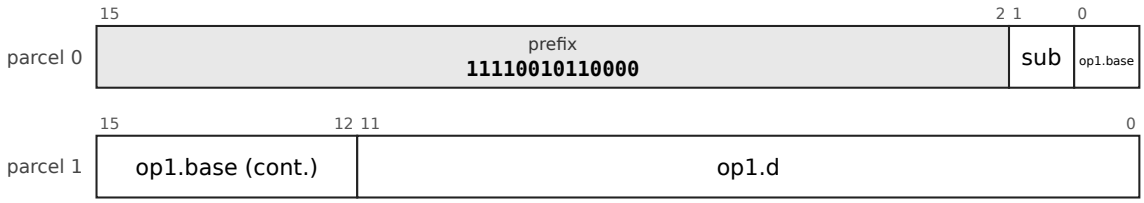

`op1` is `x5-5d4s`: memory at base + (index << scale) + signed 4-bit displacement; base register, 5 bits, index register, 5 bits.
`sub` (2 bits) selects the mnemonic: `clr` 00, `inc` 01, `dec` 10, `tst` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.index=p1[10:6] op1.scale=p1[5:4] op1.d=p1[3:0]`

### *LDS12H-b-1: LDS12H/b, prefix `11110010101111`*



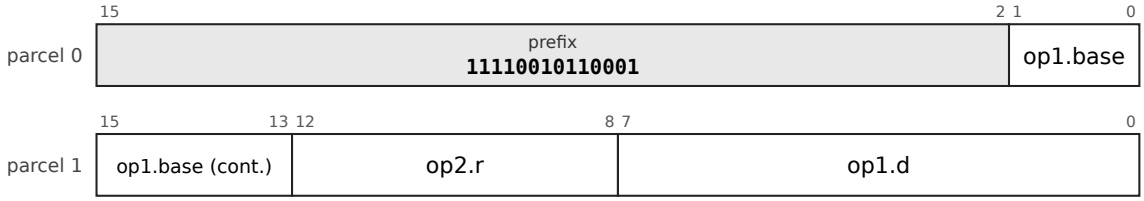

`op1` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op2.r=p1[15:12] op1.d=p1[11:0]`

### *SOBs12-a-2: SOBs12/a, prefix `11110010110000`*



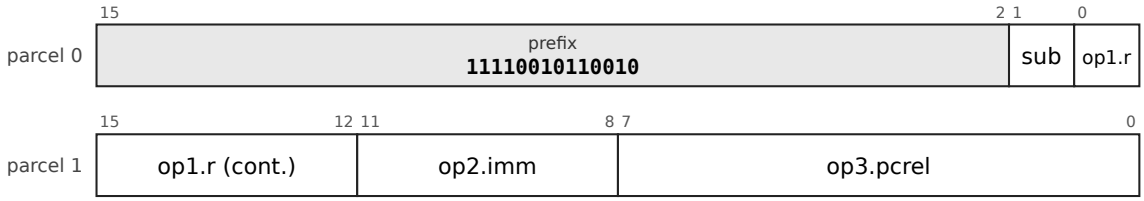

`op1` is `b5d12s-xsz`: memory at base register, 5 bits + signed 12-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.
Fields: `sub=p0[1] op1.base=p0[0]+p1[15:12] op1.d=p1[11:0]`

### *LDBu8-c-2: LDBu8/c, prefix `11110010110001`*



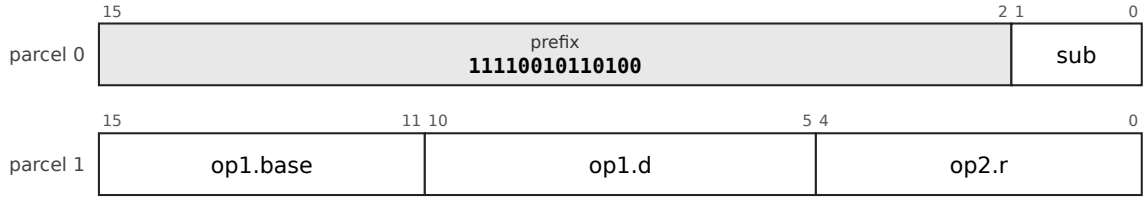

`op1` is `b5d8u-xsz`: memory at base register, 5 bits + unsigned 8-bit displacement times the size of the access.
`op2` is `reg5`: register, 5 bits.
Fields: `op1.base=p0[1:0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *CBI4-8-a-1: CBI4_8/a, prefix `11110010110010`*



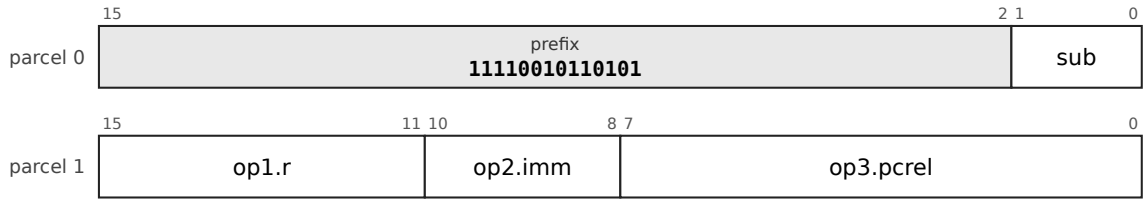

`op1` is `reg5`: register, 5 bits.
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[1] op1.r=p0[0]+p1[15:12] op2.imm=p1[11:8] op3.pcrel=p1[7:0]`

### *LDBs6-b-1: LDBs6/b, prefix `11110010110100`*



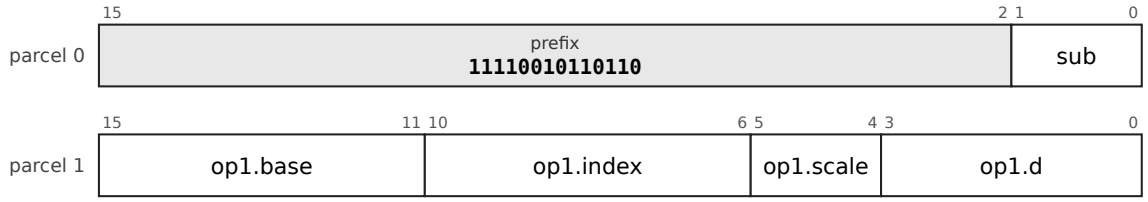

`op1` is `b5d6s-xsz`: memory at base register, 5 bits + signed 6-bit displacement times the size of the access.
`op2` is `reg5`: register, 5 bits.
`sub` (2 bits) selects the mnemonic: `movw` 00, `movb` 01, `movzl` 10, `movzw` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.d=p1[10:5] op2.r=p1[4:0]`

### *CBI3-8-c-1: CBI3_8/c, prefix `11110010110101`*



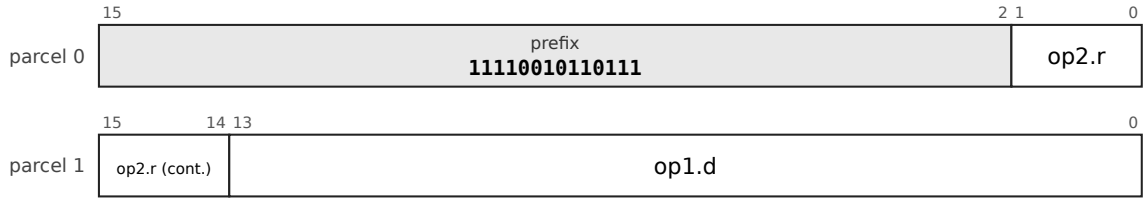

`op1` is `reg5`: register, 5 bits.
`op2` is `imm3s`: signed 3-bit immediate, -4 to 3.
`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).
`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.
Fields: `sub=p0[1:0] op1.r=p1[15:11] op2.imm=p1[10:8] op3.pcrel=p1[7:0]`

### *SOX4-x-1: SOX4/x, prefix `11110010110110`*



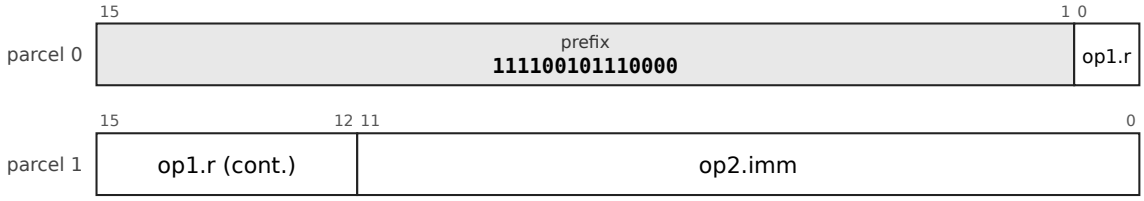

`op1` is `x5-5d4s`: memory at base + (index << scale) + signed 4-bit displacement; base register, 5 bits, index register, 5 bits.
`sub` (2 bits) selects the mnemonic: `jmp` 00, `call` 01, `jmpi` 10, `calli` 11.
Fields: `sub=p0[1:0] op1.base=p1[15:11] op1.index=p1[10:6] op1.scale=p1[5:4] op1.d=p1[3:0]`

### *LEAS14H-a-1: LEAS14H/a, prefix `11110010110111`*




`op1` is `spd14u-x8`: memory at sp + unsigned 14-bit displacement times 8, 0 to 131064 bytes.
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
Fields: `op2.r=p0[1:0]+p1[15:14] op1.d=p1[13:0]`

### *MFSR-1: MFSR, prefix `111100101110000`*

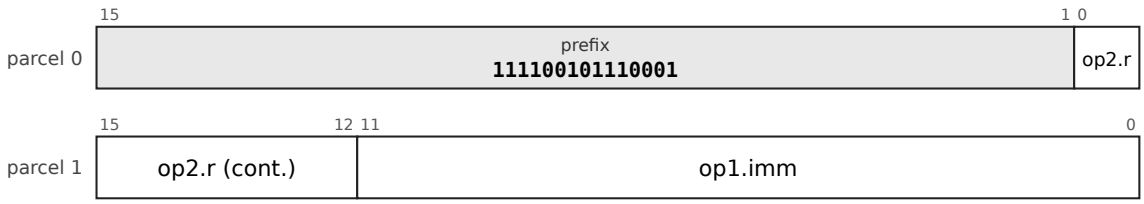



`op1` is `reg5`: register, 5 bits.

`op2` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.

Fields: `op1.r=p0[0]+p1[15:12] op2.imm=p1[11:0]`

### *MTSR-1: MTSR, prefix `111100101110001`*

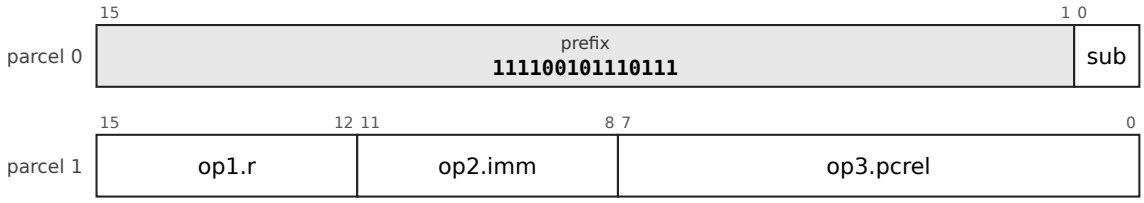



`op1` is `imm12u`: unsigned 12-bit immediate, 0 to 4095.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *CBI4-8H-a-1: CBI4_8H/a, prefix `111100101110111`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

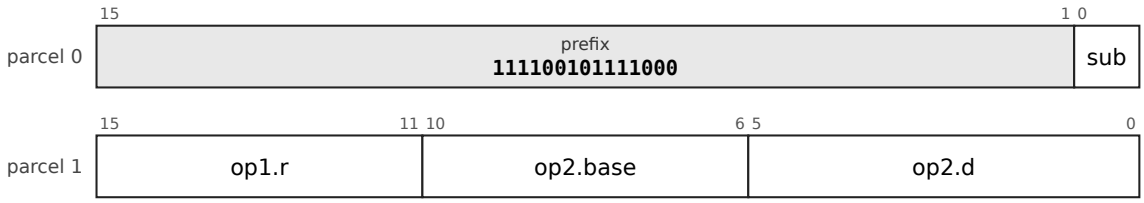
`op2` is `imm4s`: signed 4-bit immediate, -8 to 7.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.imm=p1[11:8] op3.pcrel=p1[7:0]`

### *STBu6-a-1: STBu6/a, prefix `111100101111000`*



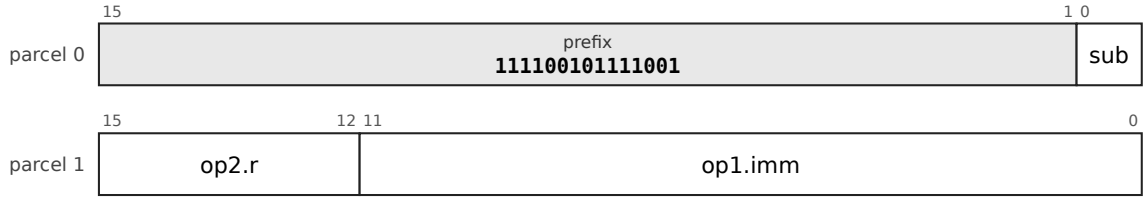

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[0] op1.r=p1[15:11] op2.base=p1[10:6] op2.d=p1[5:0]`

### *LDI12H-b-1: LDI12H/b, prefix `111100101111001`*



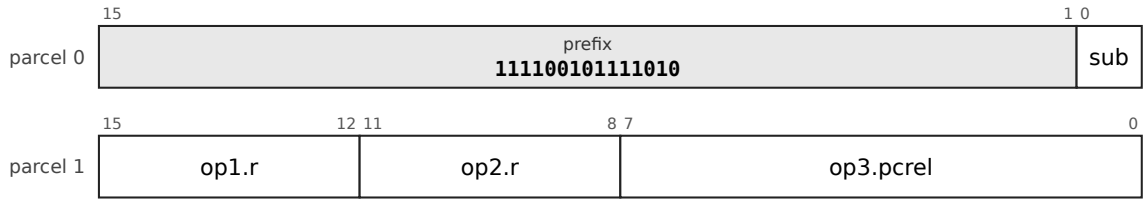

`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[0] op2.r=p1[15:12] op1.imm=p1[11:0]`

### *CBR8H-a-1: CBR8H/a, prefix `111100101111010`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

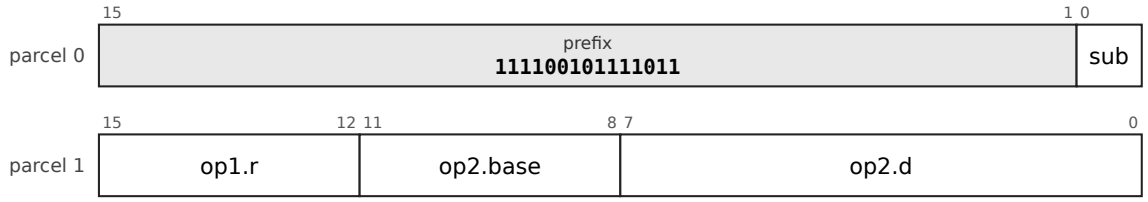
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]`

### *STBu8H-b-1: STBu8H/b, prefix `111100101111011`*



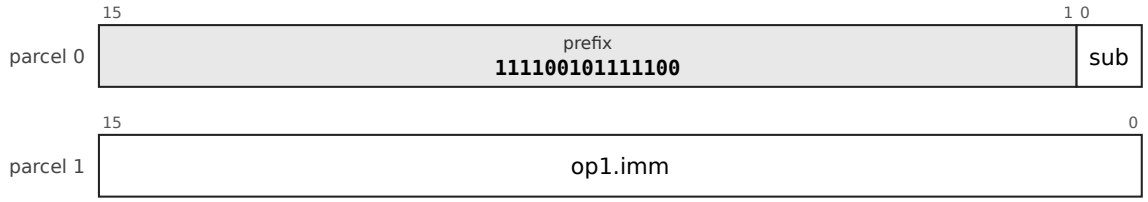

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

### *PUSHM16b-a-1: PUSHM16b/a, prefix `111100101111100`*

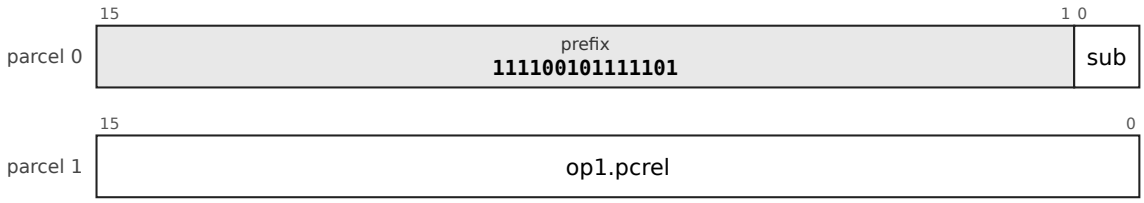



`op1` is `imm16u-x256`: unsigned 16-bit immediate times 256, 0 to 16776960.

`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

Fields: `sub=p0[0]` `op1.imm=p1[15:0]`

***BR2-16-b-1: BR2_16/b, prefix `111100101111101`***

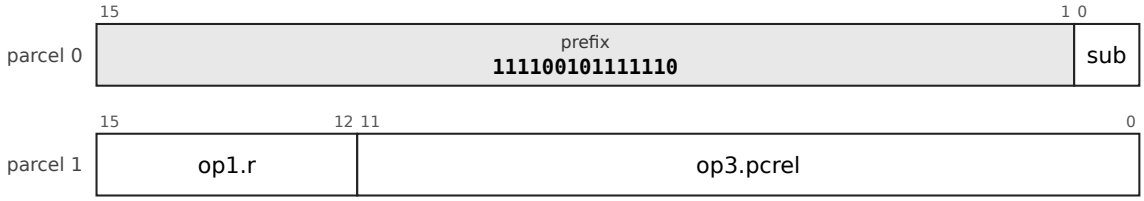



`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

`sub` (1 bit) selects the mnemonic: `beq` 0, `bne` 1.

Fields: `sub=p0[0]` `op1.pcrel=p1[15:0]`

***CBZ2-12H-a-1: CBZ2_12H/a, prefix `111100101111110`***




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

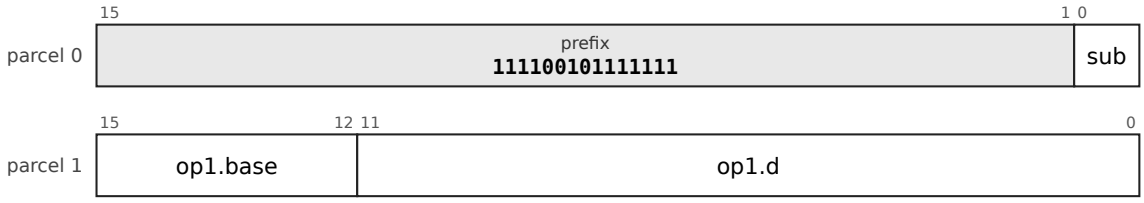
`op2` is `k-imm0`: implicit constant 0.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0]` `op1.r=p1[15:12]` `op3.pcrel=p1[11:0]`

***SOBu12H-a-2: SOBu12H/a, prefix `111100101111111`***

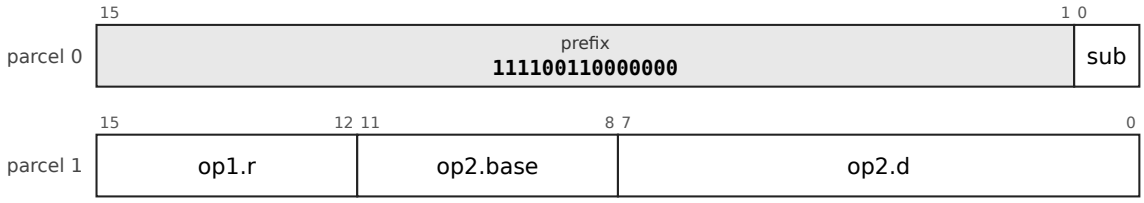



`op1` is `b4d12u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 12-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.

Fields: `sub=p0[0]` `op1.base=p1[15:12]` `op1.d=p1[11:0]`

***STBs8H-a-1: STBs8H/a, prefix `111100110000000`***



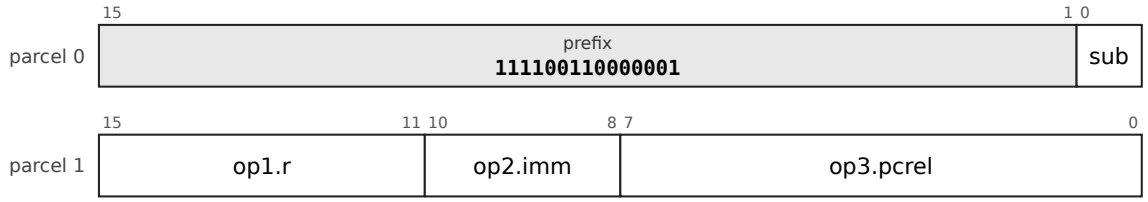

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d8s-xsz`: memory at base register, 4 bits (an index into the hot-register table) + signed 8-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

### *CBI3-8-a-1: CBI3_8/a, prefix `111100110000001`*




`op1` is `reg5`: register, 5 bits.

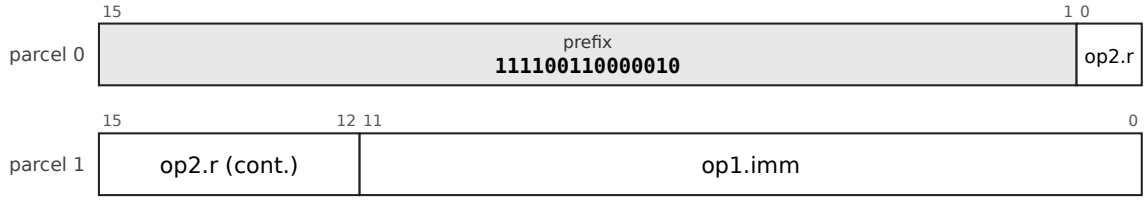
`op2` is `imm3s`: signed 3-bit immediate, -4 to 3.

`op3` is `pc8-u2`: pc-relative, 8 bits counted in 2-byte units (-256 to 254 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[0] op1.r=p1[15:11] op2.imm=p1[10:8] op3.pcrel=p1[7:0]`

### *LDI12-a-1: LDI12/a, prefix `111100110000010`*

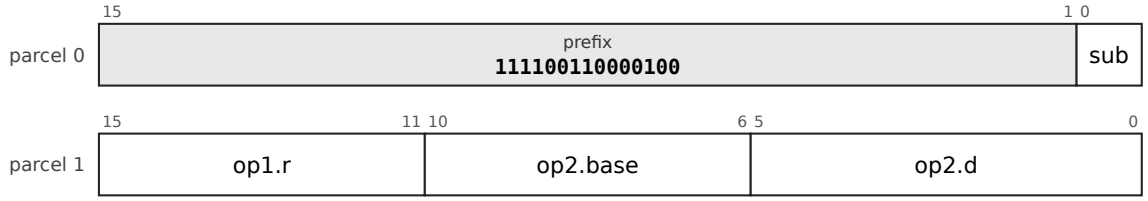



`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *STBs6-b-1: STBs6/b, prefix `111100110000100`*



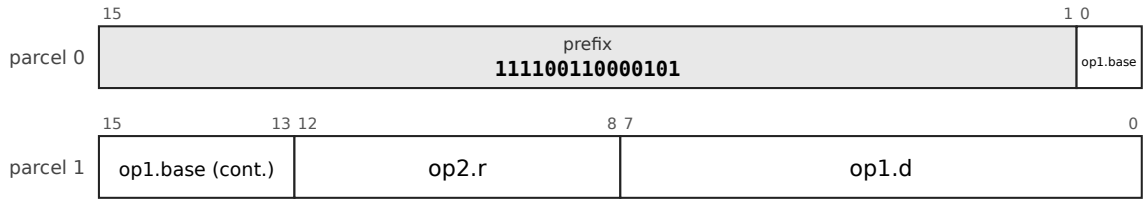

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d6s-xsz`: memory at base register, 5 bits + signed 6-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[0] op1.r=p1[15:11] op2.base=p1[10:6] op2.d=p1[5:0]`

### *LDBu8H-c-2: LDBu8H/c, prefix `111100110000101`*

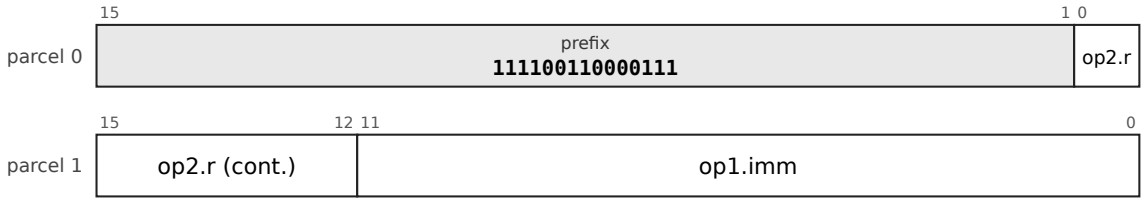



`op1` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base=p0[0]+p1[15:13] op2.r=p1[12:8] op1.d=p1[7:0]`

### *LDI12-d-3: LDI12/d, prefix `111100110000111`*

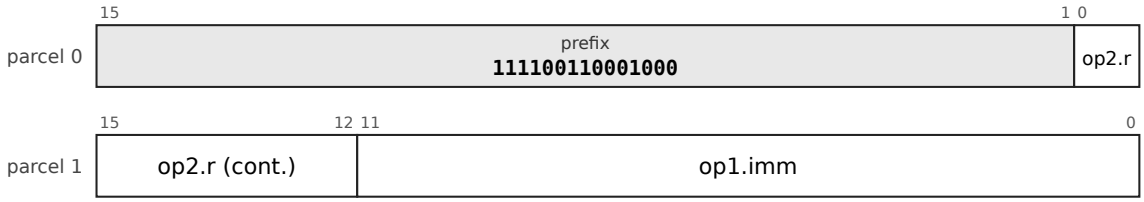



`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *LDI12-b-2: LDI12/b, prefix `111100110001000`*

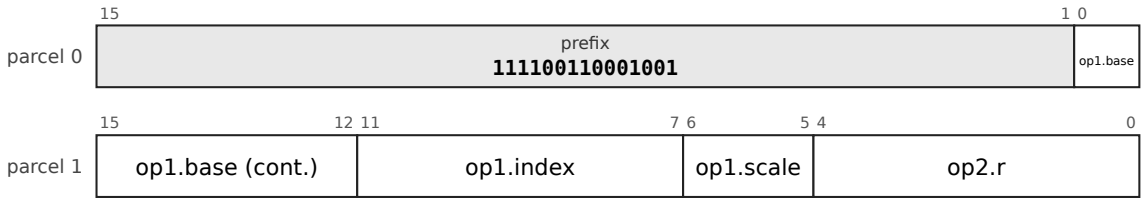



`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[0]+p1[15:12] op1.imm=p1[11:0]`

### *LDX0-c-2: LDX0/c, prefix `111100110001001`*

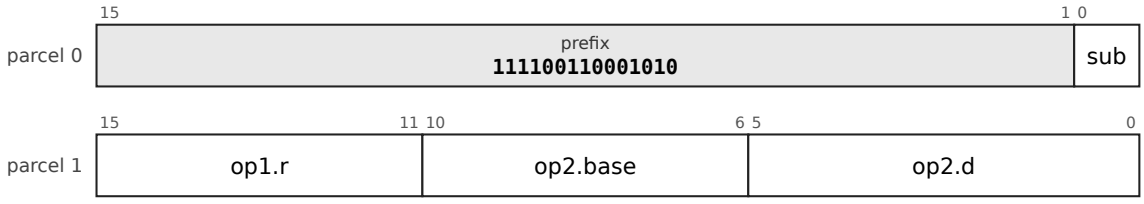



`op1` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base=p0[0]+p1[15:12] op1.index=p1[11:7] op1.scale=p1[6:5] op2.r=p1[4:0]`

### *STBs6-a-1: STBs6/a, prefix `111100110001010`*



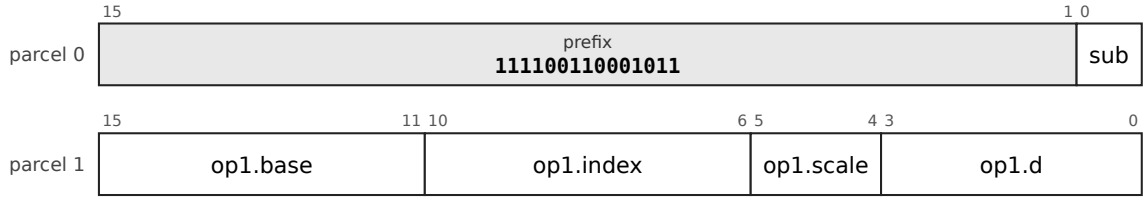

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d6s-xsz`: memory at base register, 5 bits + signed 6-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movq` 0, `movl` 1.

Fields: `sub=p0[0] op1.r=p1[15:11] op2.base=p1[10:6] op2.d=p1[5:0]`

### *SOX4-a-2: SOX4/a, prefix `111100110001011`*

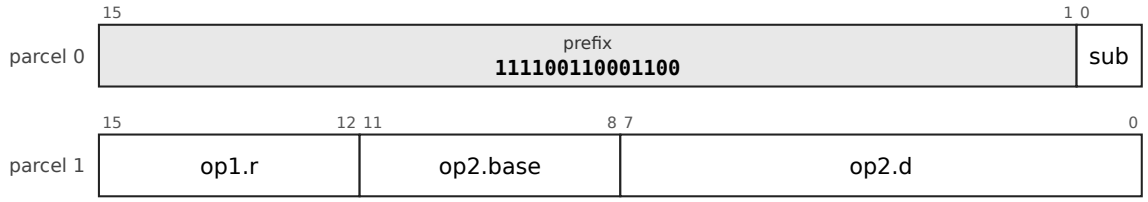



`op1` is `x5-5d4s`: memory at base + (index << scale) + signed 4-bit displacement; base register, 5 bits, index register, 5 bits.

`sub` (1 bit) selects the mnemonic: `push` 0, `pop` 1.

Fields: `sub=p0[0] op1.base=p1[15:11] op1.index=p1[10:6] op1.scale=p1[5:4] op1.d=p1[3:0]`

### *STBs8H-b-1: STBs8H/b, prefix `111100110001100`*



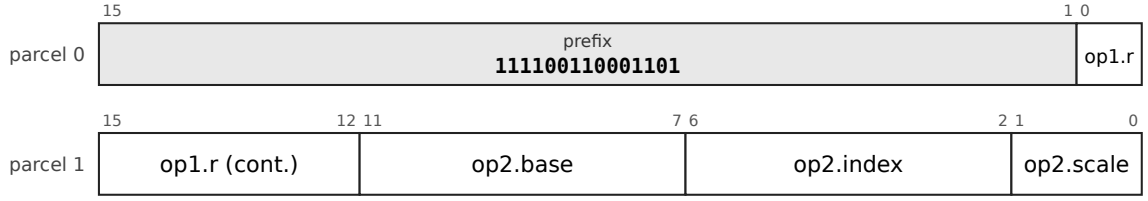

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d8s-xsz`: memory at base register, 4 bits (an index into the hot-register table) + signed 8-bit displacement times the size of the access.

`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.

Fields: `sub=p0[0] op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

### *CMX0-a-1: CMX0/a, prefix `111100110001101`*

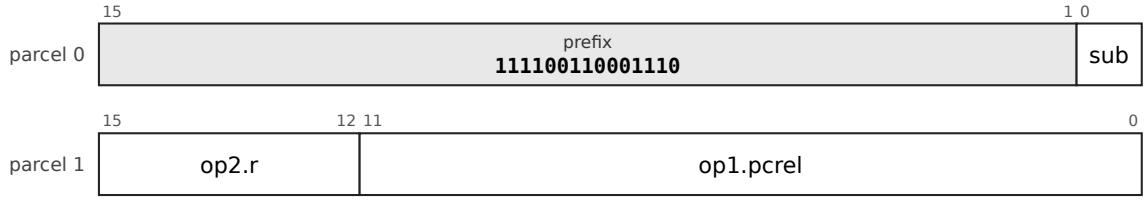



`op1` is `reg5`: register, 5 bits.

`op2` is `x5-5d0s`: memory at base + (index << scale); base register, 5 bits, index register, 5 bits.

Fields: `op1.r=p0[0]+p1[15:12] op2.base=p1[11:7] op2.index=p1[6:2] op2.scale=p1[1:0]`

### *LDPC12H-d-2: LDPC12H/d, prefix `111100110001110`*



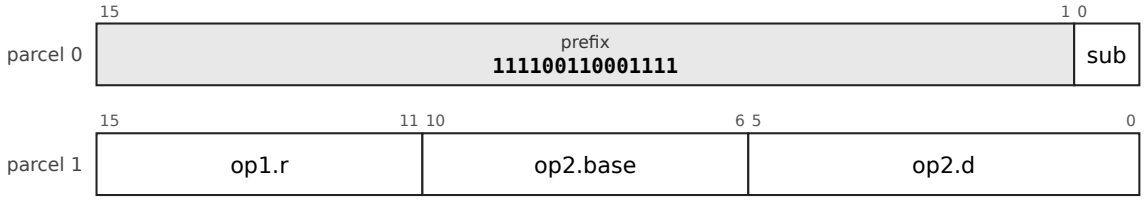

`op1` is `pc12-u1`: pc-relative, 12 bits counted in 1-byte units (-2048 to 2047 bytes).

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`sub` (1 bit) selects the mnemonic: `fmul` 0, `bit` 1.

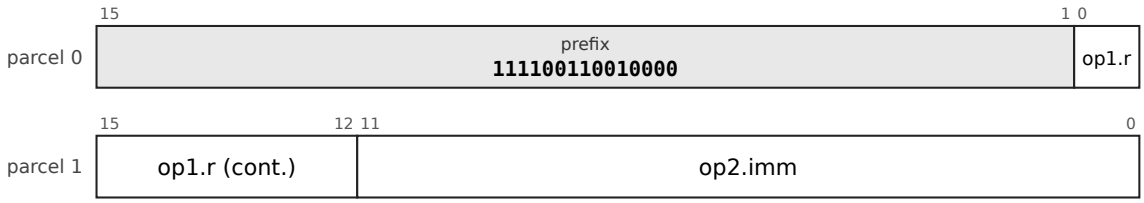
Fields: `sub=p0[0] op2.r=p1[15:12] op1.pcrel=p1[11:0]`

### *STBu6-b-1: STBu6/b, prefix `111100110001111`*



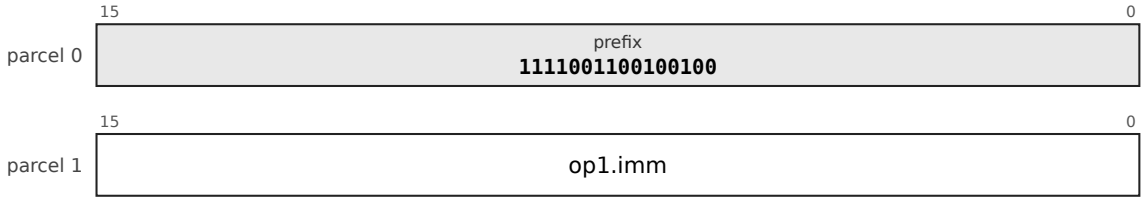

`op1` is `reg5`: register, 5 bits.
`op2` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.
`sub` (1 bit) selects the mnemonic: `movw` 0, `movb` 1.
Fields: `sub=p0[0] op1.r=p1[15:11] op2.base=p1[10:6] op2.d=p1[5:0]`

### *CMI12-a-1: CMI12/a, prefix `111100110010000`*



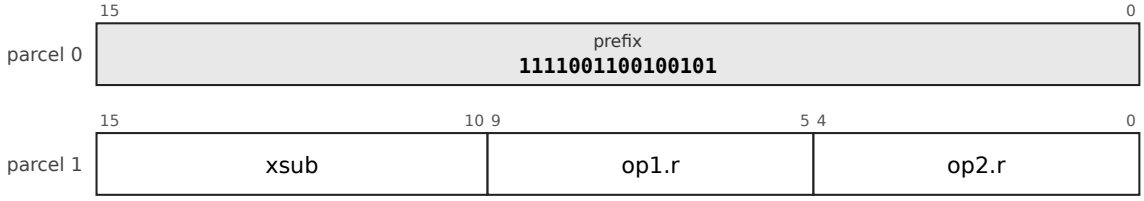

`op1` is `reg5`: register, 5 bits.
`op2` is `imm12s`: signed 12-bit immediate, -2048 to 2047.
Fields: `op1.r=p0[0]+p1[15:12] op2.imm=p1[11:0]`

### *PUSHI2-1: PUSHI2, prefix `1111001100100100`*



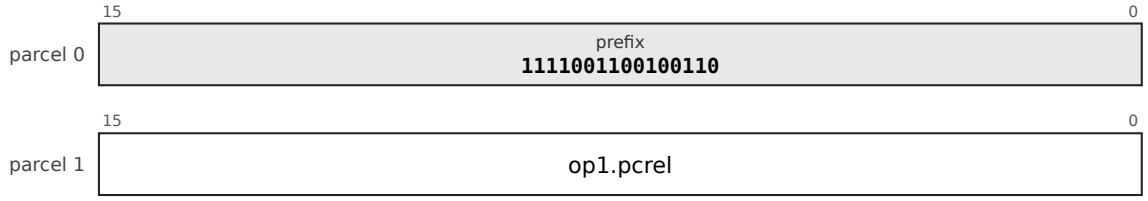

`op1` is `imm16s`: signed 16-bit immediate, -32768 to 32767.
Fields: `op1.imm=p1[15:0]`

### *RRF5-1: RRF5, prefix `1111001100100101`*




`op1` is `reg5`: register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`xsub` selects the mnemonic.
Fields: `xsub=p1[15:10] op1.r=p1[9:5] op2.r=p1[4:0]`

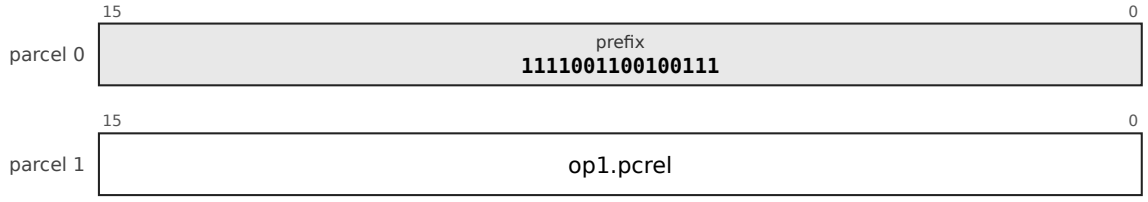
### *BR2-16-a-1: BR2_16/a, prefix `1111001100100110`*




`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

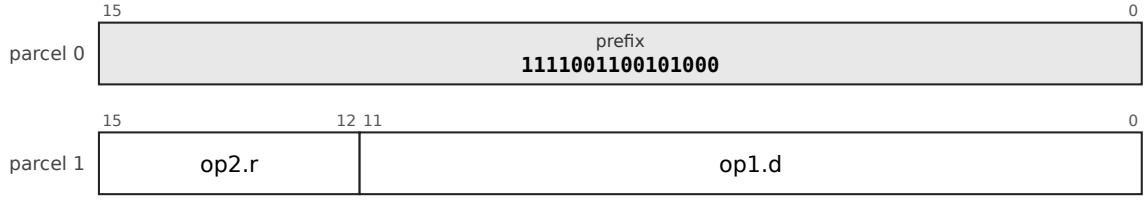
### *CALL2-16-c-1: CALL2_16/c, prefix `1111001100100111`*




`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

### *LEASx12H-a-1: LEASx12H/a, prefix `1111001100101000`*

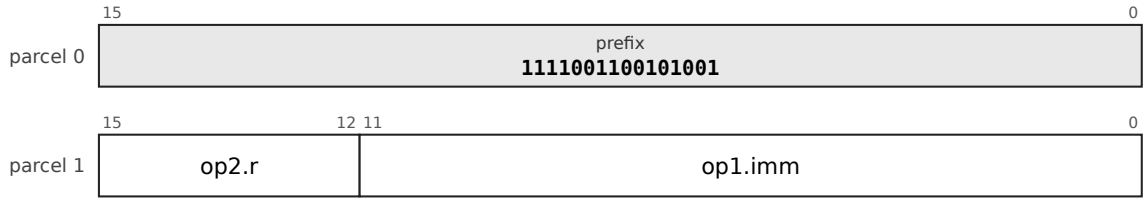



`op1` is `spd12s-x1`: memory at sp + signed 12-bit displacement times 1, -2048 to 2047 bytes.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p1[15:12]` `op1.d=p1[11:0]`

### *LDI12H-b-2: LDI12H/b, prefix `1111001100101001`*

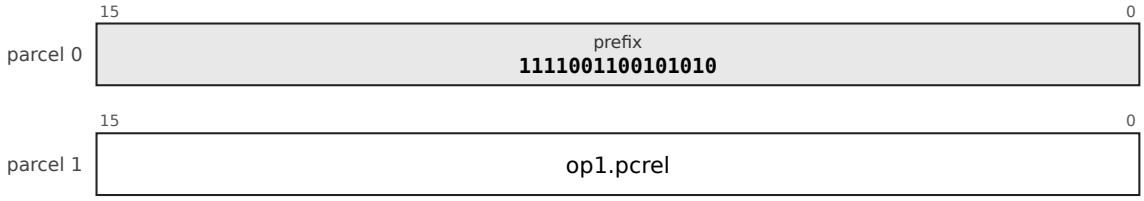



`op1` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p1[15:12]` `op1.imm=p1[11:0]`

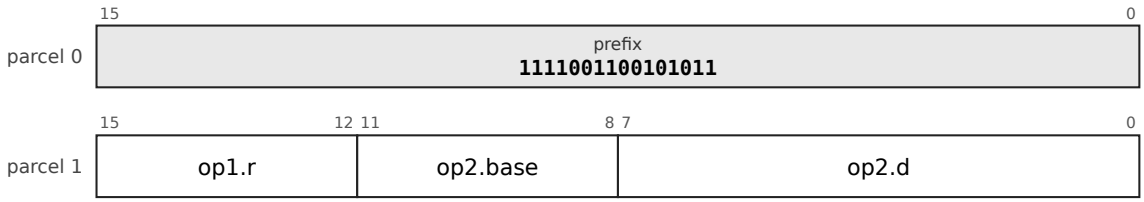
### *CALL2-16-j-1: CALL2_16/j, prefix `1111001100101010`*




`op1` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

Fields: `op1.pcrel=p1[15:0]`

### *CMBu8H-a-1: CMBu8H/a, prefix `1111001100101011`*

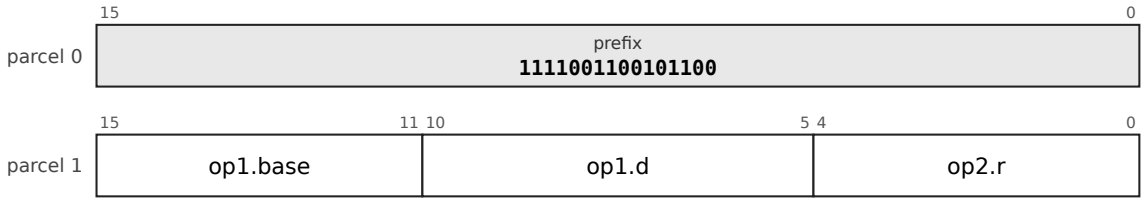



`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `b4d8u-xsz`: memory at base register, 4 bits (an index into the hot-register table) + unsigned 8-bit displacement times the size of the access.

Fields: `op1.r=p1[15:12] op2.base=p1[11:8] op2.d=p1[7:0]`

### *LDBu6-c-2: LDBu6/c, prefix `1111001100101100`*

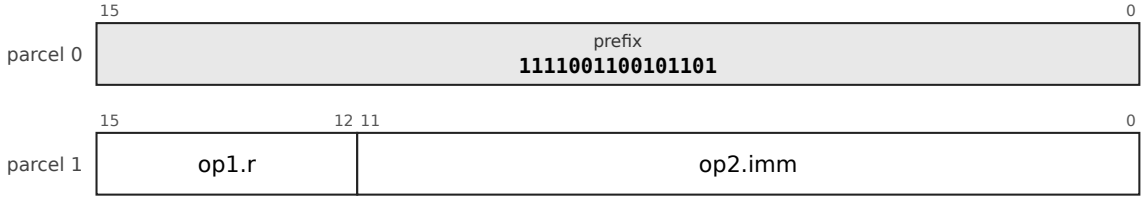



`op1` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base=p1[15:11] op1.d=p1[10:5] op2.r=p1[4:0]`

### *CMI12H-a-1: CMI12H/a, prefix `1111001100101101`*

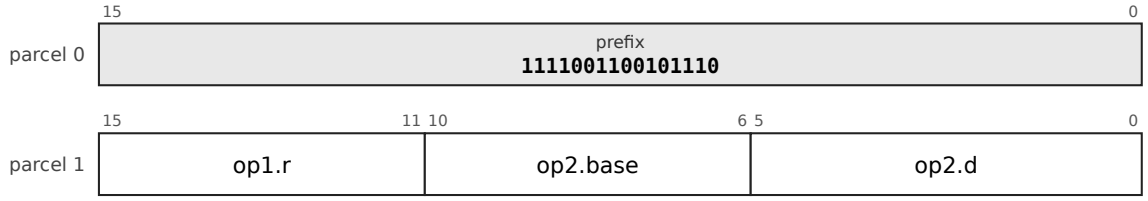



`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

Fields: `op1.r=p1[15:12] op2.imm=p1[11:0]`

### *CMBu6-a-1: CMBu6/a, prefix `1111001100101110`*

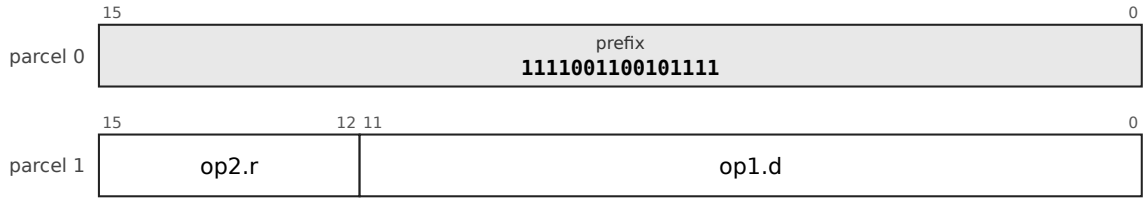



`op1` is `reg5`: register, 5 bits.

`op2` is `b5d6u-xsz`: memory at base register, 5 bits + unsigned 6-bit displacement times the size of the access.

Fields: `op1.r=p1[15:11] op2.base=p1[10:6] op2.d=p1[5:0]`

### *LEAS12H-a-1: LEAS12H/a, prefix `1111001100101111`*

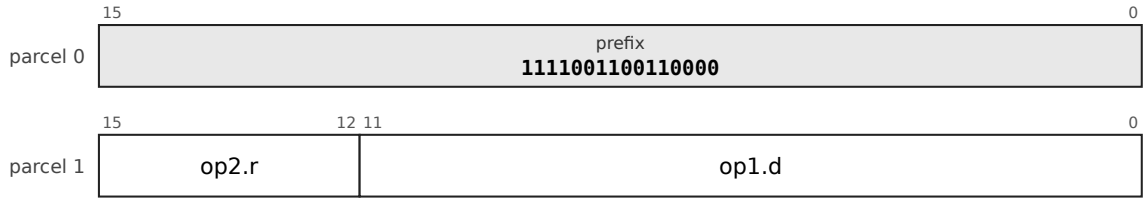



`op1` is `spd12u-x8`: memory at sp + unsigned 12-bit displacement times 8, 0 to 32760 bytes.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p1[15:12] op1.d=p1[11:0]`

### *LDS12H-c-2: LDS12H/c, prefix `1111001100110000`*

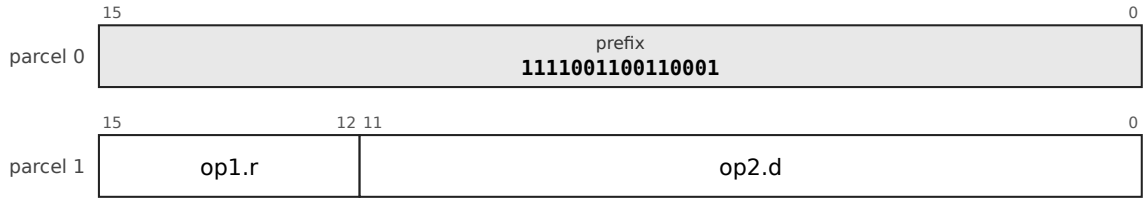



`op1` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.

`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

Fields: `op2.r=p1[15:12] op1.d=p1[11:0]`

### *CMS12H-a-1: CMS12H/a, prefix `1111001100110001`*



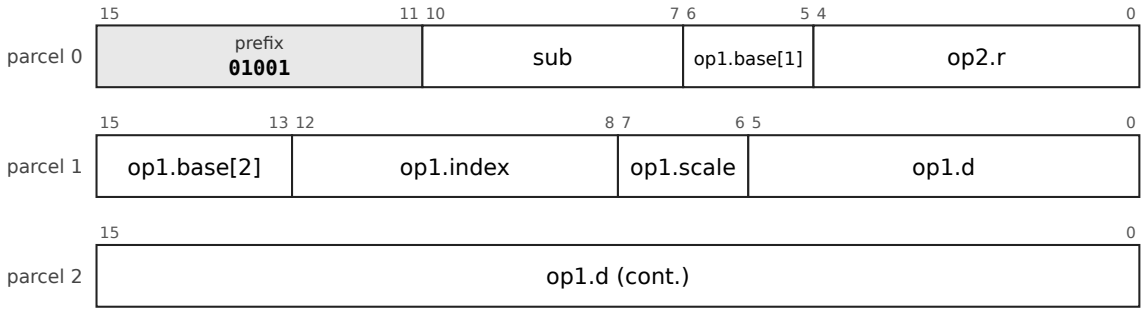

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

`op2` is `spd12u-xsz`: memory at sp + unsigned 12-bit displacement times the size of the access.

Fields: `op1.r=p1[15:12] op2.d=p1[11:0]`

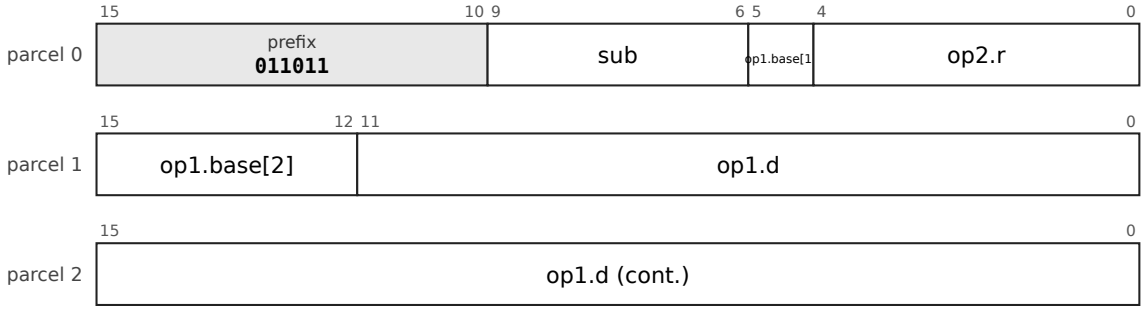
## Instructions of 3 parcels (6 bytes)

### *LDF-X22-1: LDF.X22, prefix `01001`*



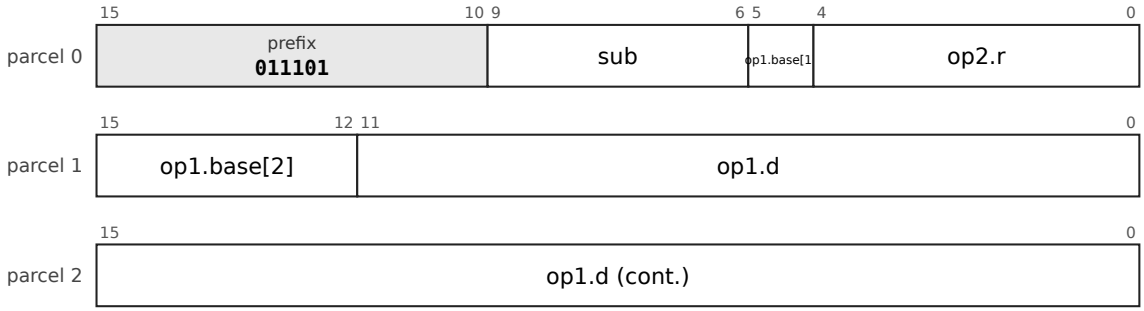

`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.
Fields: `sub=p0[10:7] op1.base-1=p0[6:5] op2.r=p0[4:0] op1.base-2=p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *LDU-B28-1: LDU.B28, prefix `011011`*




`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg5`: register, 5 bits.
`sub` (4 bits) selects the mnemonic: `movq.u` 0000, `movl.u` 0001, `movw.u` 0010, `movb.u` 0011, `movzl.u` 0100, `movzw.u` 0101, `movzb.u` 0110, `movq.d` 0111, `movl.d` 1000, `movw.d` 1001, `movb.d` 1010, `movzl.d` 1011, `movzw.d` 1100, `movzb.d` 1101, `movq.ud` 1110, `movl.ud` 1111.
Fields: `sub=p0[9:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

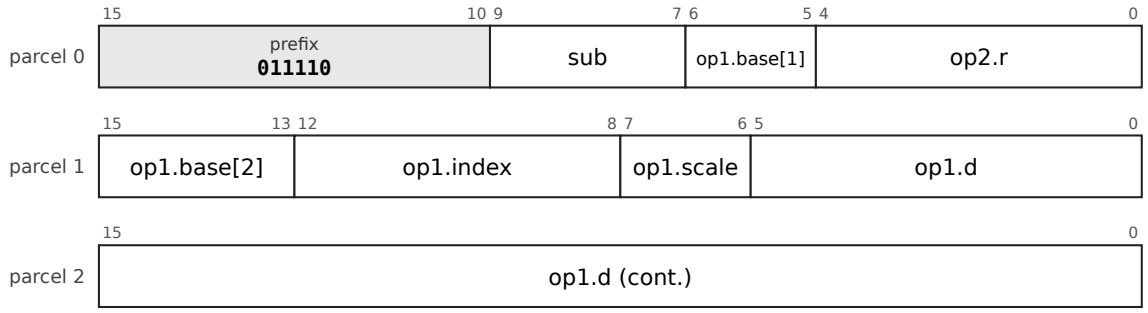
### *LDF-B28-1: LDF.B28, prefix `011101`*




`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg5`: register, 5 bits.
`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

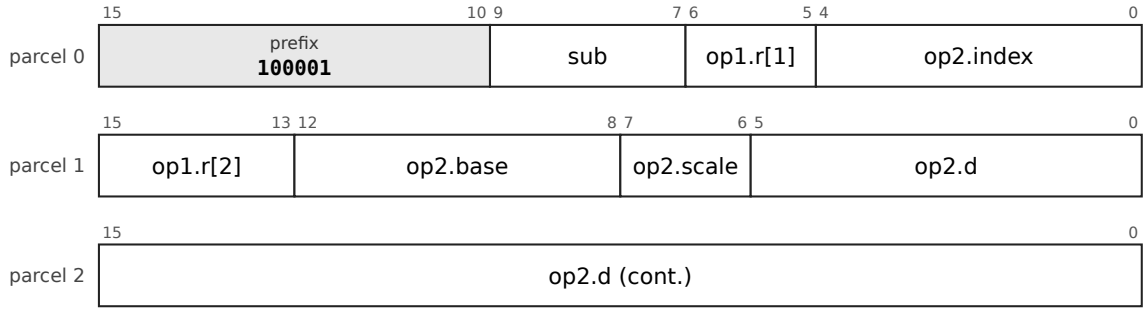
Fields: `sub=p0[9:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *LDF-X22-2: LDF.X22, prefix `011110`*



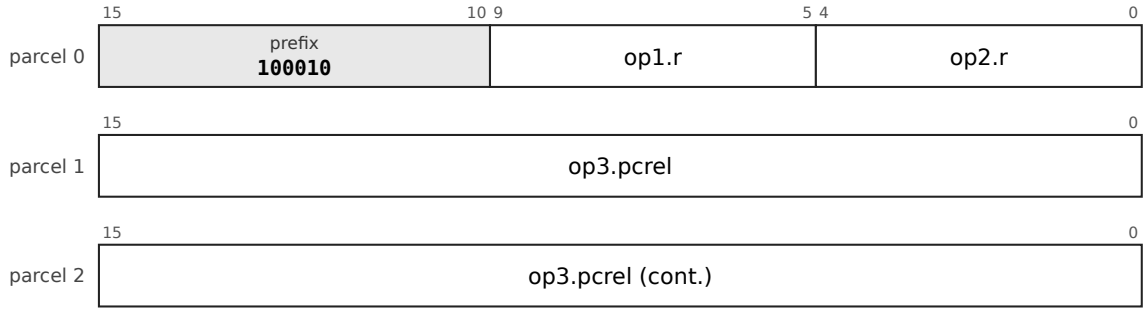

`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[9:7] op1.base-1=p0[6:5] op2.r=p0[4:0] op1.base-2=p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *STF-X22-1: STF.X22, prefix `100001`*



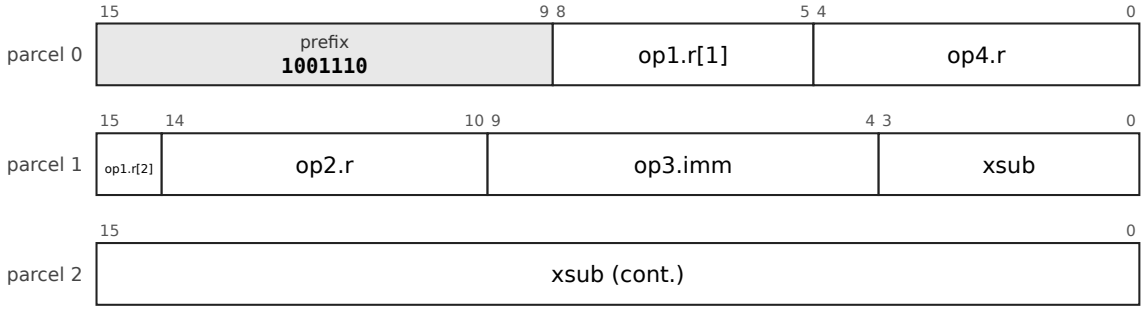

`op1` is `reg5`: register, 5 bits.
`op2` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.
`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.
Fields: `sub=p0[9:7] op1.r-1=p0[6:5] op2.index=p0[4:0] op1.r-2=p1[15:13] op2.base=p1[12:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

### *FX-MOVCALL3-1: FX.MOVCALL3, prefix `100010`*




`op1` is `reg5`: register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`op3` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
Fields: `op1.r=p0[9:5] op2.r=p0[4:0] op3.pcrel=p1[15:0]+p2[15:0]`

### *BFx-1: BFx, prefix `1001110`*




`op1` is `reg5`: register, 5 bits.

`op2` is `reg5`: register, 5 bits.

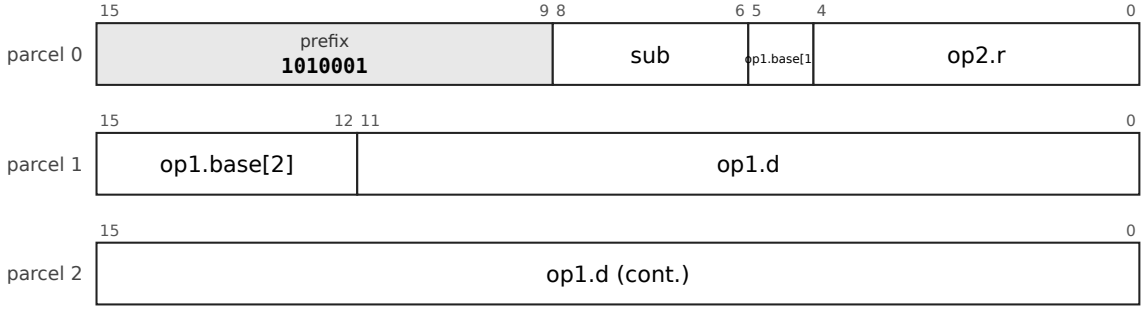
`op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63.

`op4` is `reg5`: register, 5 bits.

`xsub` selects the mnemonic.

Fields: `op1.r-1=p0[8:5] op4.r=p0[4:0] op1.r-2=p1[15] op2.r=p1[14:10] op3.imm=p1[9:4] xsub=p1[3:0]+p2[15:0]`

### *LDF-B28-2: LDF.B28, prefix `1010001`*



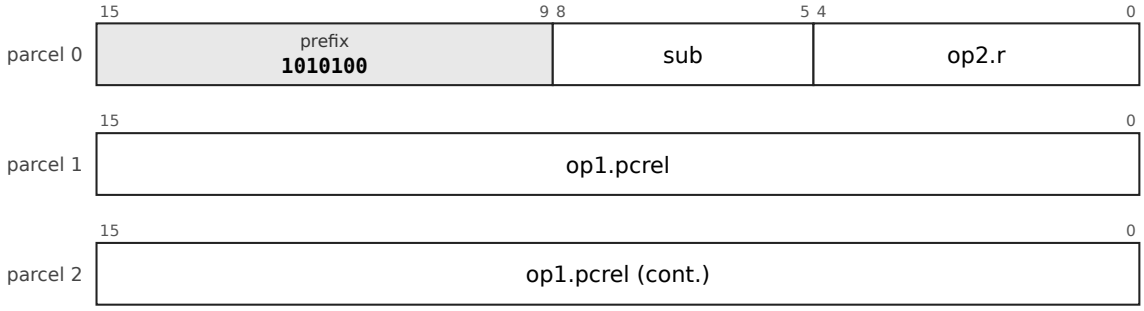

`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg5`: register, 5 bits.

`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.

Fields: `sub=p0[8:6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *LDF-PC32-1: LDF.PC32, prefix `1010100`*



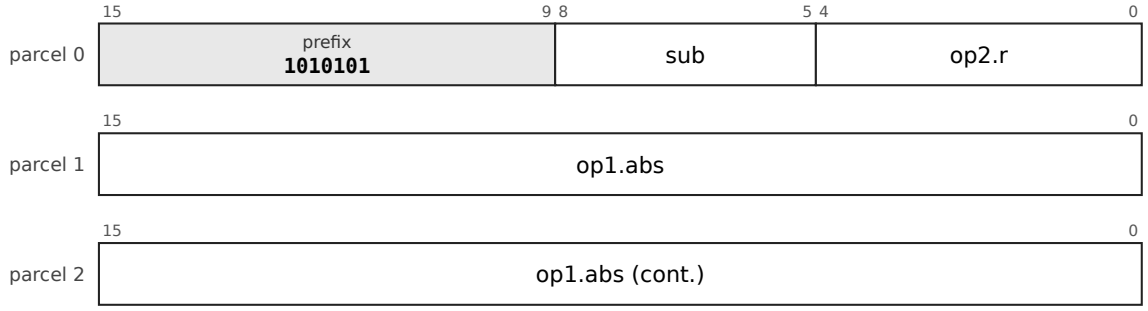

`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[8:5] op2.r=p0[4:0] op1.pcrel=p1[15:0]+p2[15:0]`

### *LDF-A32-1: LDF.A32, prefix `1010101`*



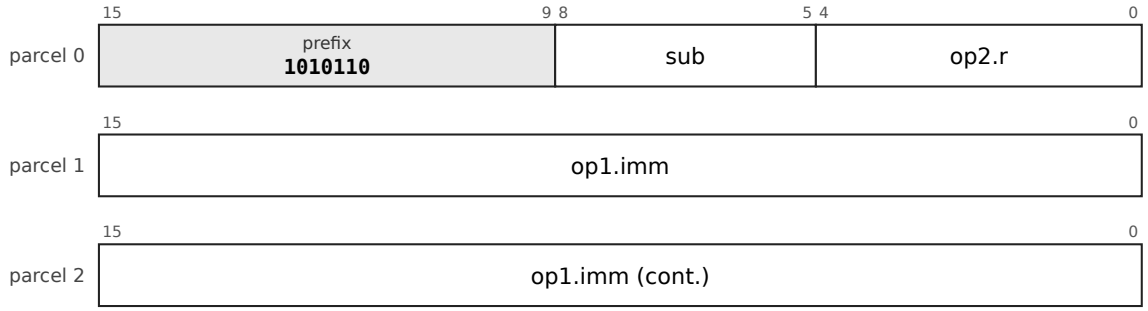

`op1` is `abs32`: absolute address, 32 bits.

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[8:5] op2.r=p0[4:0] op1.abs=p1[15:0]+p2[15:0]`

### *LDFI-I32-1: LDFI.I32, prefix `1010110`*



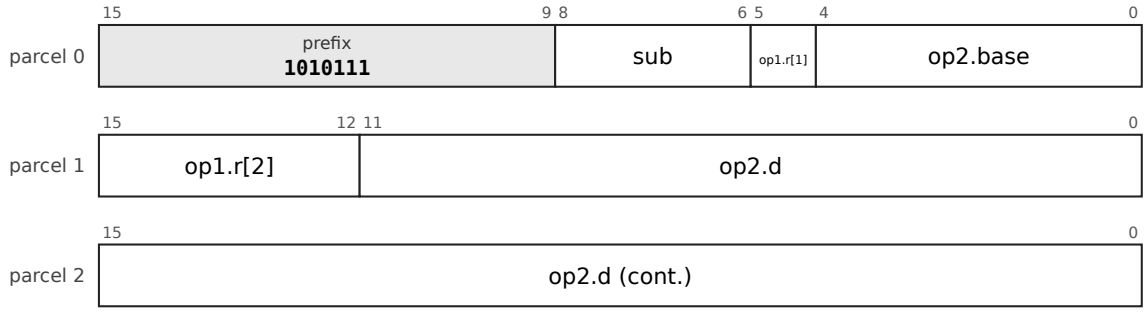

`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `add` 0000, `sub` 0001, `bic` 0010, `bis` 0011, `xor` 0100, `and` 0101, `mul` 0110, `mulh` 0111, `shl` 1000, `shr` 1001, `sar` 1010, `fadd` 1011, `fsub` 1100, `fmul` 1101, `bit` 1110, `movq` 1111.

Fields: `sub=p0[8:5] op2.r=p0[4:0] op1.imm=p1[15:0]+p2[15:0]`

### *STF-B28-1: STF.B28, prefix `1010111`*



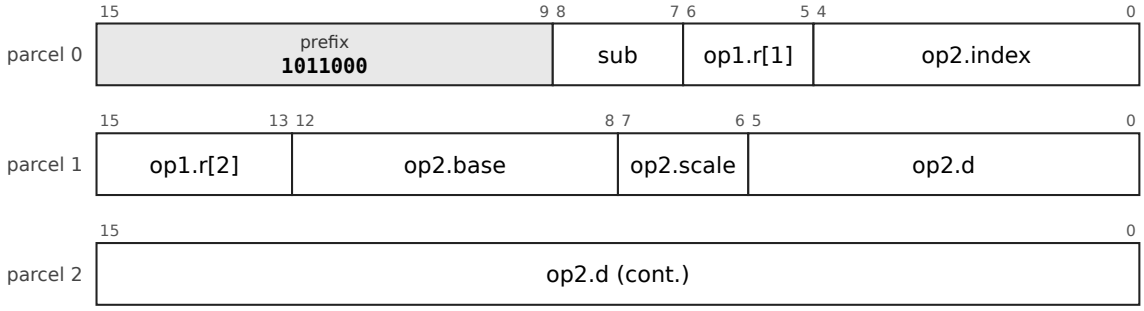

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

Fields: `sub=p0[8:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

### *STF-X22-2: STF.X22, prefix `1011000`*



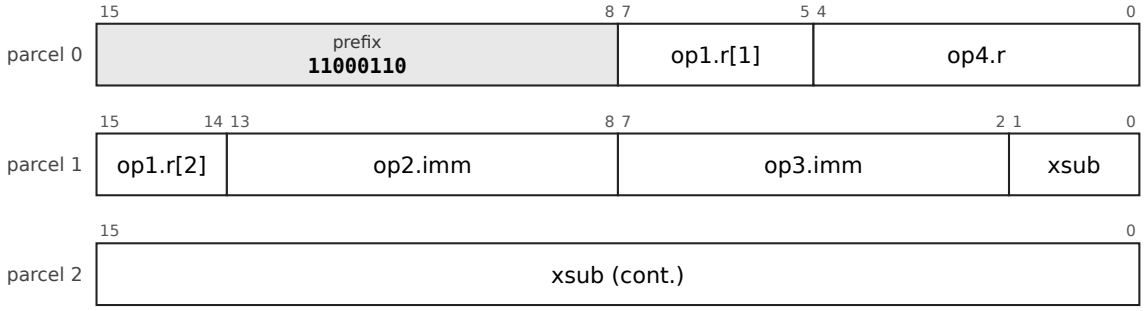

`op1` is `reg5`: register, 5 bits.

`op2` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[8:7] op1.r-1=p0[6:5] op2.index=p0[4:0] op1.r-2=p1[15:13] op2.base=p1[12:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

### *BFy-1: BFy, prefix `11000110`*




`op1` is `reg5`: register, 5 bits.

`op2` is `imm6u`: unsigned 6-bit immediate, 0 to 63.

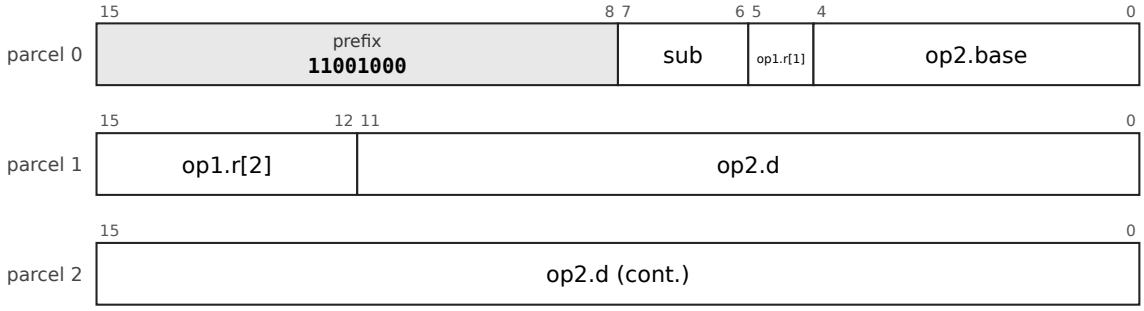
`op3` is `imm6u`: unsigned 6-bit immediate, 0 to 63.

`op4` is `reg5`: register, 5 bits.

`xsub` selects the mnemonic.

Fields: `op1.r-1=p0[7:5] op4.r=p0[4:0] op1.r-2=p1[15:14] op2.imm=p1[13:8] op3.imm=p1[7:2] xsub=p1[1:0]+p2[15:0]`

### *STU-B28-1: STU.B28, prefix `11001000`*



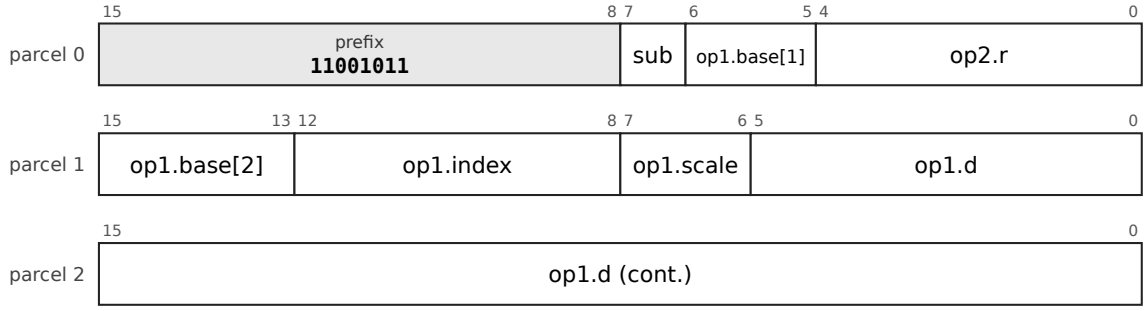

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (2 bits) selects the mnemonic: `movq.u` 00, `movl.u` 01, `movw.u` 10, `movb.u` 11.

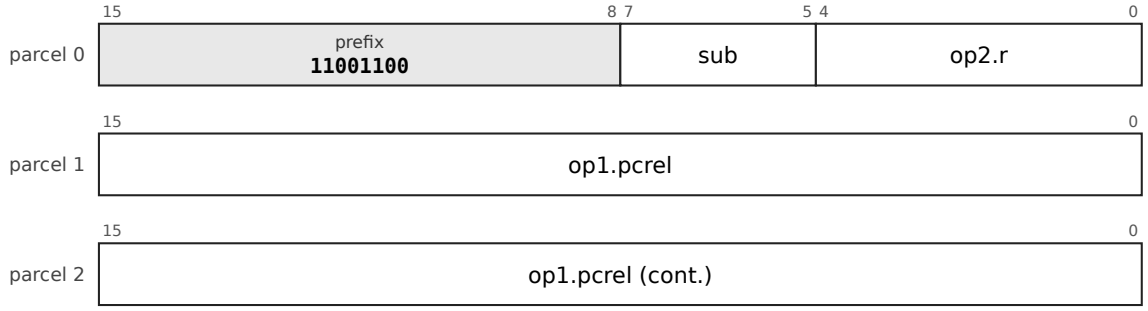
Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

### *LDF-X22-3: LDF.X22, prefix `11001011`*



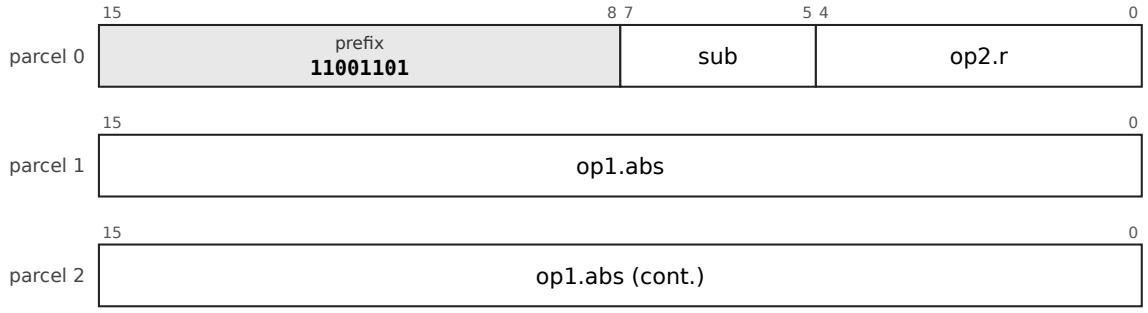

`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[7] op1.base-1=p0[6:5] op2.r=p0[4:0] op1.base-2=p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *LDF-PC32-2: LDF.PC32, prefix `11001100`*



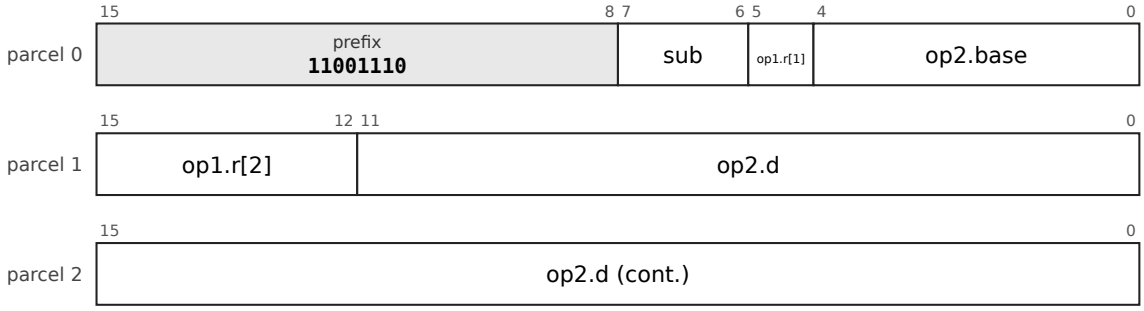

`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[7:5] op2.r=p0[4:0] op1.pcrel=p1[15:0]+p2[15:0]`

### *LDF-A32-2: LDF.A32, prefix `11001101`*




`op1` is `abs32`: absolute address, 32 bits.
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[7:5] op2.r=p0[4:0] op1.abs=p1[15:0]+p2[15:0]`

### *STF-B28-2: STF.B28, prefix `11001110`*



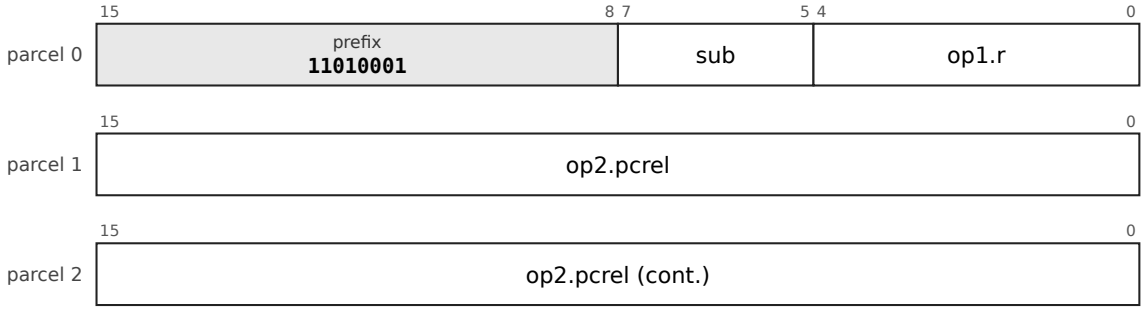

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[7:6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

### *STF-PC32-1: STF.PC32, prefix `11010001`*



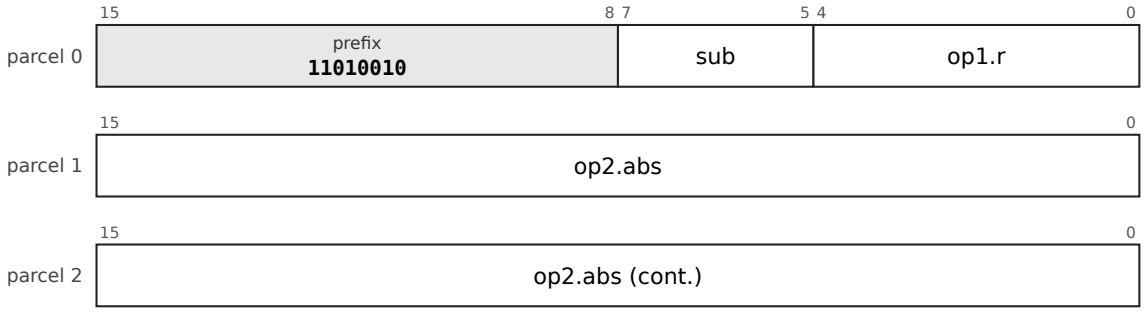

`op1` is `reg5`: register, 5 bits.

`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

Fields: `sub=p0[7:5] op1.r=p0[4:0] op2.pcrel=p1[15:0]+p2[15:0]`

### *STF-A32-1: STF.A32, prefix `11010010`*



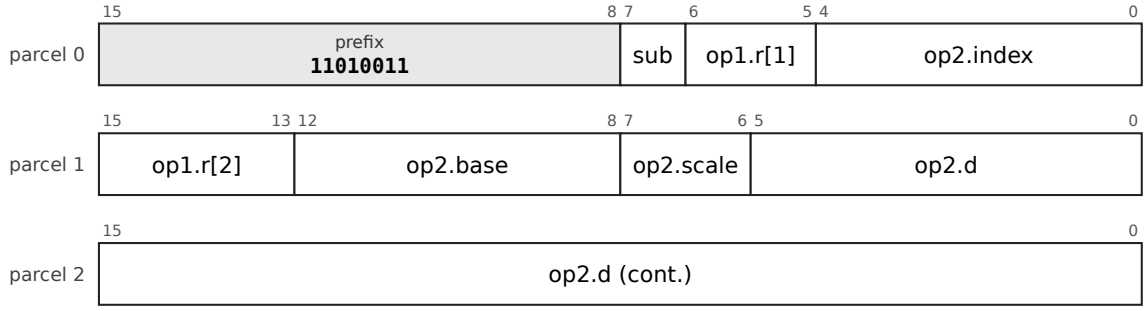

`op1` is `reg5`: register, 5 bits.

`op2` is `abs32`: absolute address, 32 bits.

`sub` (3 bits) selects the mnemonic: `movq` 000, `movl` 001, `movw` 010, `movb` 011, `movub` 100, `movuw` 101, `movul` 110, `movuq` 111.

Fields: `sub=p0[7:5] op1.r=p0[4:0] op2.abs=p1[15:0]+p2[15:0]`

### *CMF-X22-1: CMF.X22, prefix `11010011`*



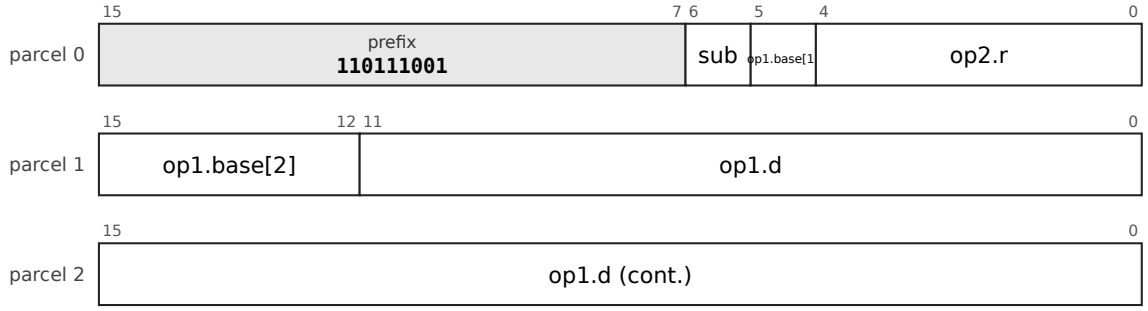

`op1` is `reg5`: register, 5 bits.

`op2` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[7] op1.r-1=p0[6:5] op2.index=p0[4:0] op1.r-2=p1[15:13] op2.base=p1[12:8] op2.scale=p1[7:6] op2.d=p1[5:0]+p2[15:0]`

### *LDF-B28-3: LDF.B28, prefix `110111001`*



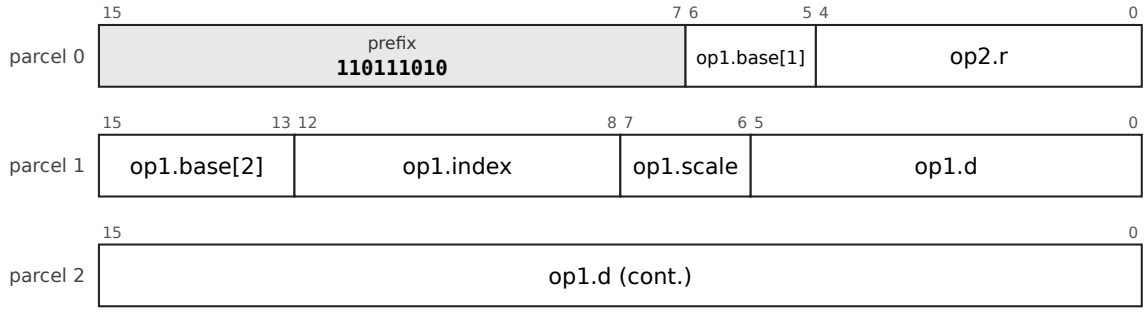

`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg5`: register, 5 bits.

`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.

Fields: `sub=p0[6] op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *LDF-X22-4: LDF.X22, prefix `110111010`*

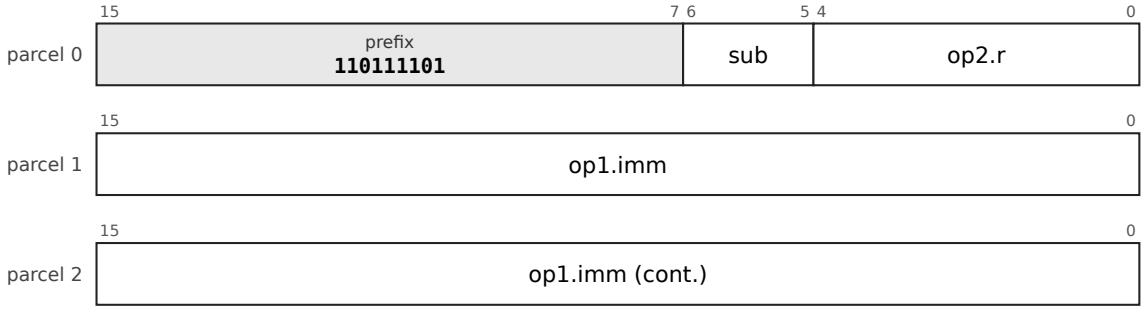



`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base-1=p0[6:5] op2.r=p0[4:0] op1.base-2=p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *LDFI-I32-2: LDFI.I32, prefix `110111101`*



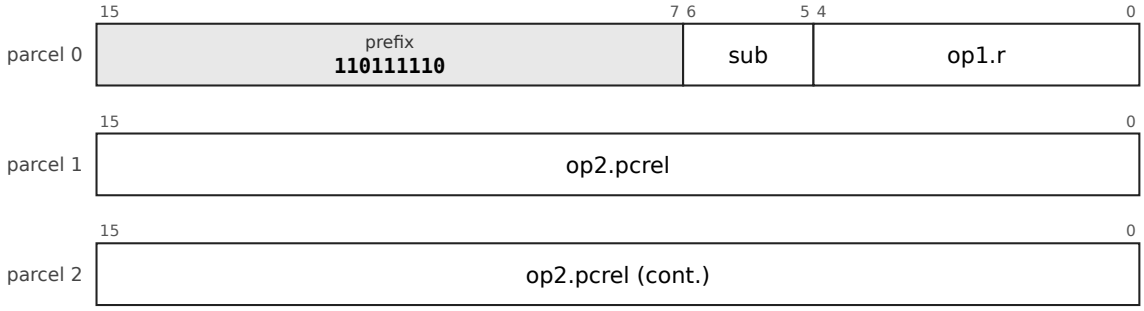

`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

`op2` is `reg5`: register, 5 bits.

`sub` (2 bits) selects the mnemonic: `movl` 00, `movw` 01, `movb` 10, `movzl` 11.

Fields: `sub=p0[6:5] op2.r=p0[4:0] op1.imm=p1[15:0]+p2[15:0]`

### *STF-PC32-2: STF.PC32, prefix `110111110`*



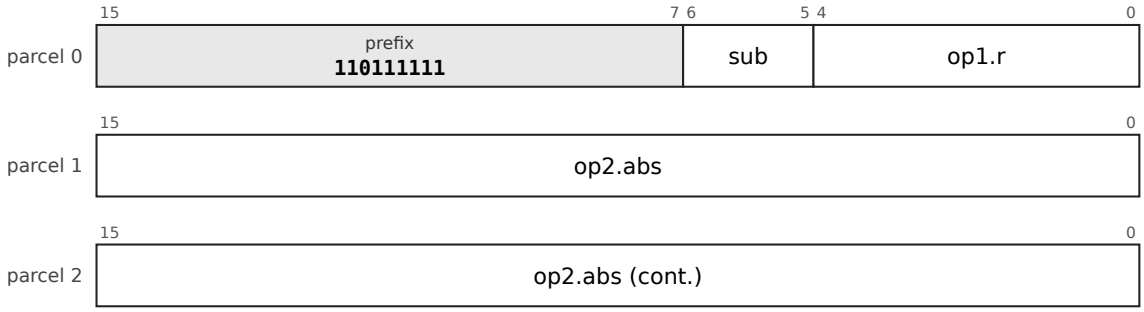

`op1` is `reg5`: register, 5 bits.

`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[6:5] op1.r=p0[4:0] op2.pcrel=p1[15:0]+p2[15:0]`

### *STF-A32-2: STF.A32, prefix `110111111`*



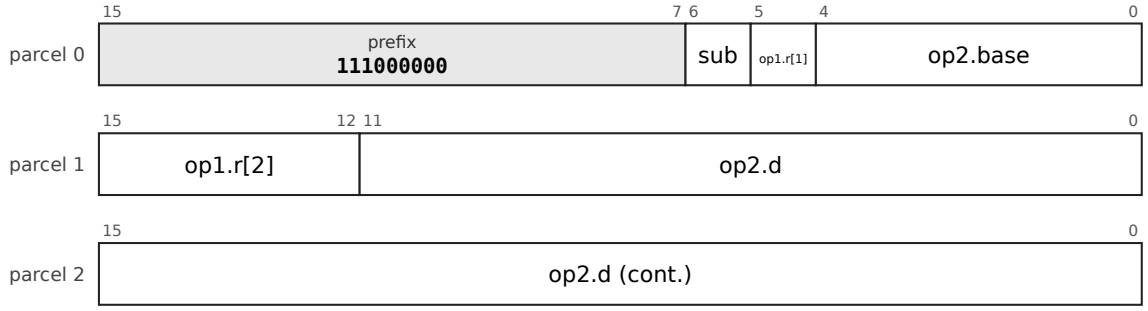

`op1` is `reg5`: register, 5 bits.

`op2` is `abs32`: absolute address, 32 bits.

`sub` (2 bits) selects the mnemonic: `addm` 00, `subm` 01, `bicm` 10, `bism` 11.

Fields: `sub=p0[6:5] op1.r=p0[4:0] op2.abs=p1[15:0]+p2[15:0]`

### *CMF-B28-1: CMF.B28, prefix `111000000`*



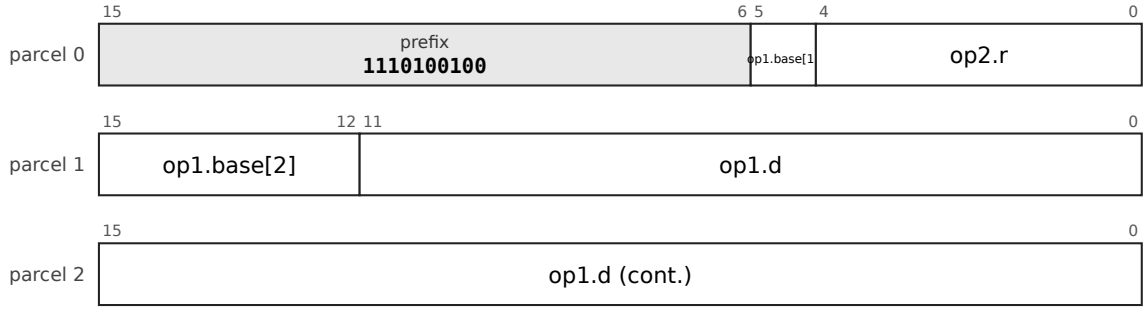

`op1` is `reg5`: register, 5 bits.

`op2` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[6] op1.r-1=p0[5] op2.base=p0[4:0] op1.r-2=p1[15:12] op2.d=p1[11:0]+p2[15:0]`

### *LDF-B28-4: LDF.B28, prefix `1110100100`*

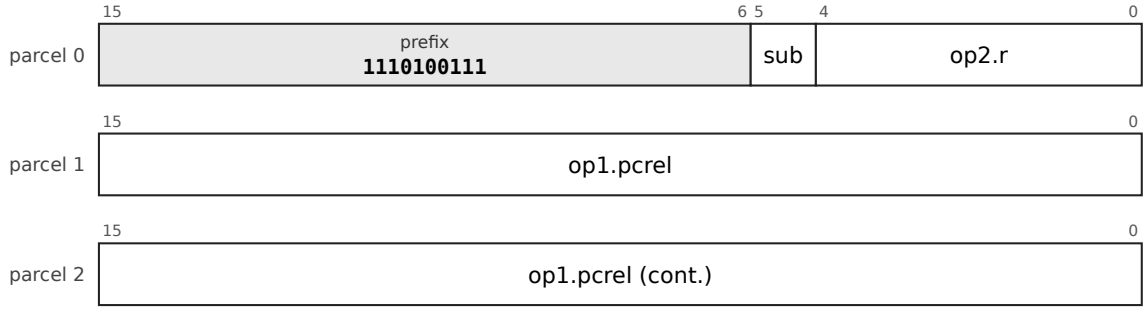



`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg5`: register, 5 bits.

Fields: `op1.base-1=p0[5] op2.r=p0[4:0] op1.base-2=p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *LDF-PC32-3: LDF.PC32, prefix `1110100111`*



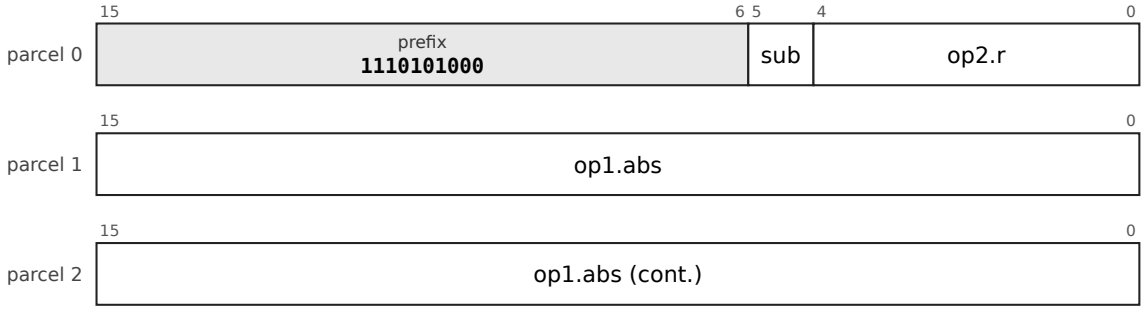

`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`op2` is `reg5`: register, 5 bits.

`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.

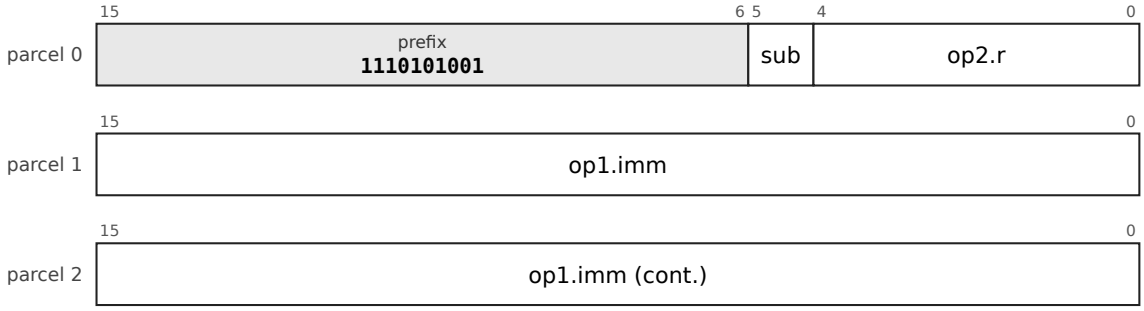
Fields: `sub=p0[5] op2.r=p0[4:0] op1.pcrel=p1[15:0]+p2[15:0]`

### *LDF-A32-3: LDF.A32, prefix `1110101000`*



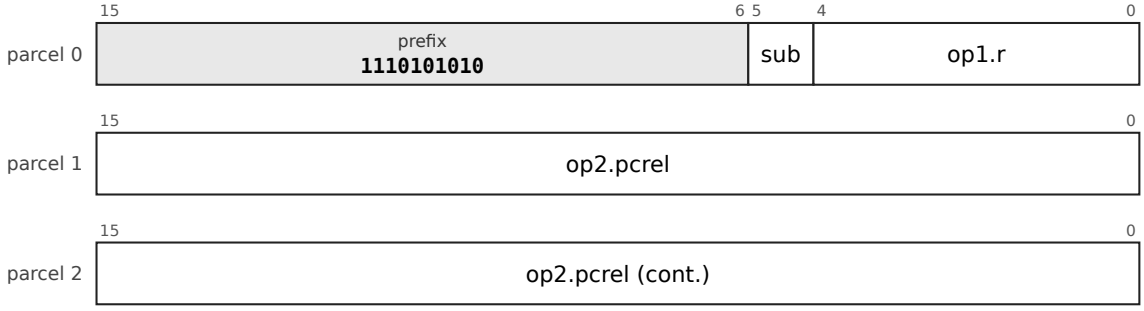

`op1` is `abs32`: absolute address, 32 bits.
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.
Fields: `sub=p0[5] op2.r=p0[4:0] op1.abs=p1[15:0]+p2[15:0]`

### *LDFI-I32-3: LDFI.I32, prefix `1110101001`*



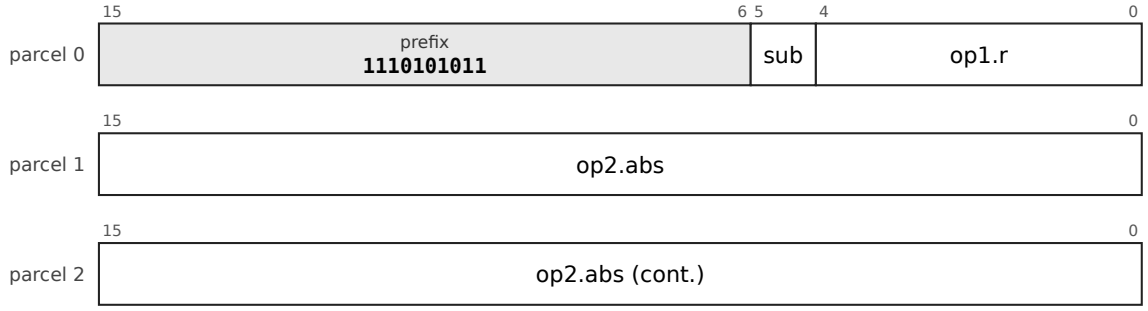

`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
`op2` is `reg5`: register, 5 bits.
`sub` (1 bit) selects the mnemonic: `movzw` 0, `movzb` 1.
Fields: `sub=p0[5] op2.r=p0[4:0] op1.imm=p1[15:0]+p2[15:0]`

### *CMF-PC32-1: CMF.PC32, prefix `1110101010`*




`op1` is `reg5`: register, 5 bits.
`op2` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).
`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.
Fields: `sub=p0[5] op1.r=p0[4:0] op2.pcrel=p1[15:0]+p2[15:0]`

### *CMF-A32-1: CMF.A32, prefix `1110101011`*



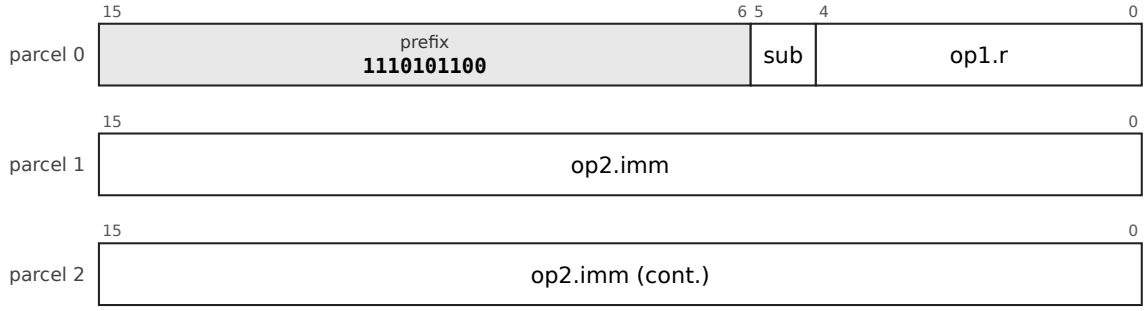

`op1` is `reg5`: register, 5 bits.

`op2` is `abs32`: absolute address, 32 bits.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[5] op1.r=p0[4:0] op2.abs=p1[15:0]+p2[15:0]`

### *CMF-I32-1: CMF.I32, prefix `1110101100`*



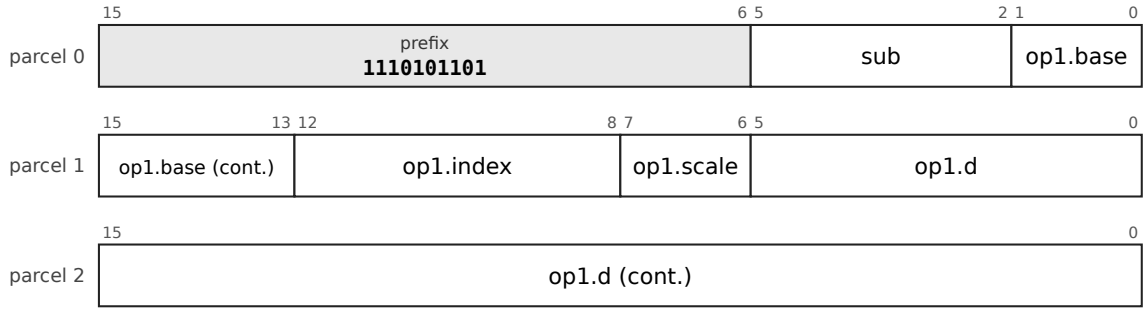

`op1` is `reg5`: register, 5 bits.

`op2` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

`sub` (1 bit) selects the mnemonic: `cmp` 0, `fcmp` 1.

Fields: `sub=p0[5] op1.r=p0[4:0] op2.imm=p1[15:0]+p2[15:0]`

### *SOF-X22-1: SOF.X22, prefix `1110101101`*

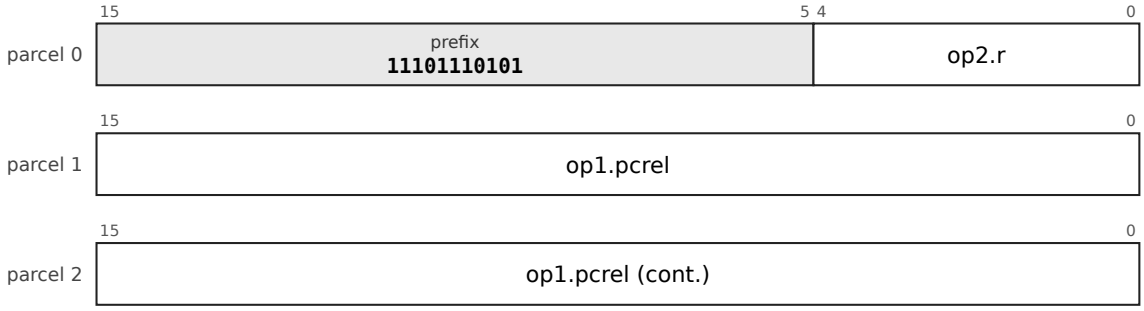



`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[5:2] op1.base=p0[1:0]+p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *LDF-PC32-4: LDF.PC32, prefix `11101110101`*

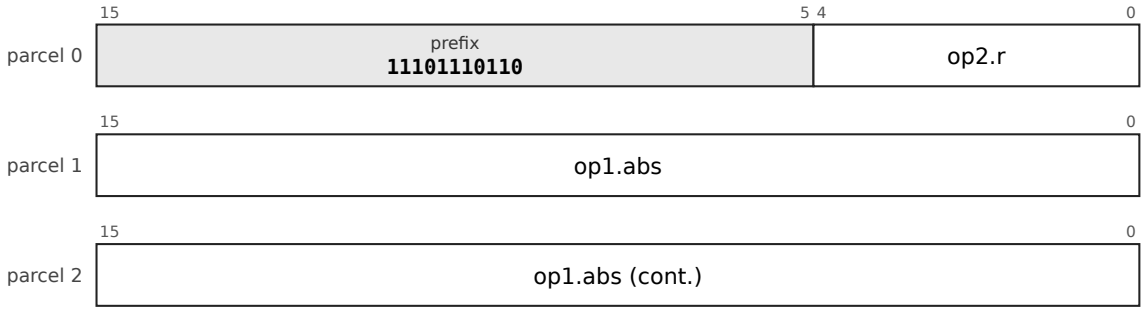



`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[4:0]` `op1.pcrel=p1[15:0]+p2[15:0]`

### *LDF-A32-4: LDF.A32, prefix `11101110110`*

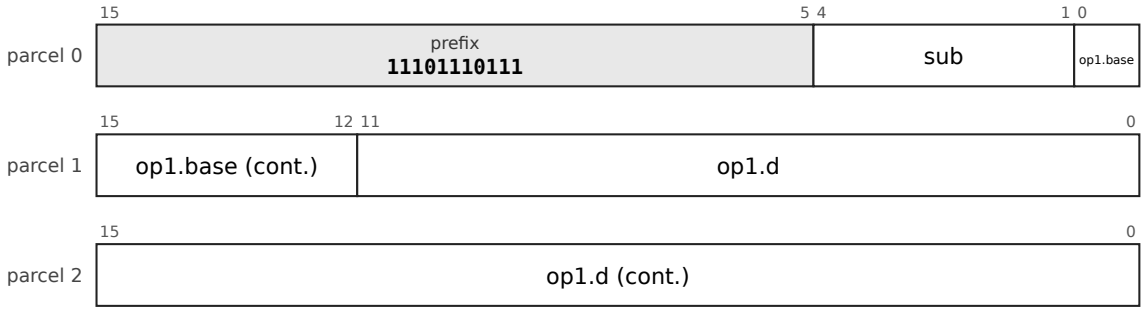



`op1` is `abs32`: absolute address, 32 bits.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[4:0]` `op1.abs=p1[15:0]+p2[15:0]`

### *SOF-B28-1: SOF.B28, prefix `11101110111`*

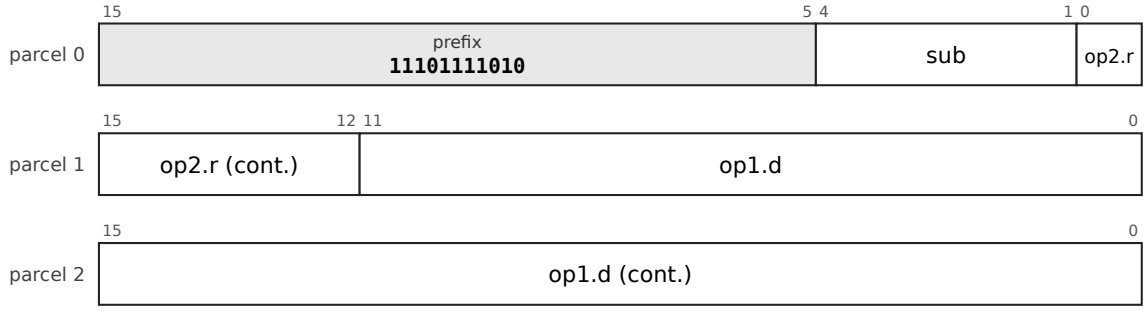



`op1` is `b5d28s-x1`: memory at base register, 5 bits + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[4:1]` `op1.base=p0[0]+p1[15:12]` `op1.d=p1[11:0]+p2[15:0]`

### *DPF-D28-1: DPF.D28, prefix `11101111010`*



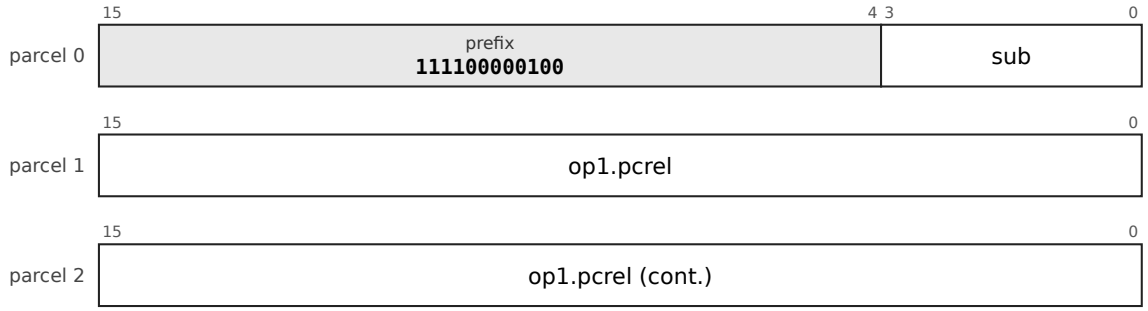

`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg5`: register, 5 bits.

`sub` (4 bits) selects the mnemonic: `movq` 0000, `movl` 0001, `movw` 0010, `movb` 0011, `movzl` 0100, `movzw` 0101, `movzb` 0110, `add` 0111, `sub` 1000, `bic` 1001, `bis` 1010, `xor` 1011, `and` 1100, `mul` 1101, `mulh` 1110, `shl` 1111.

Fields: `sub=p0[4:1] op2.r=p0[0]+p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *SOF-PC32-1: SOF.PC32, prefix `111100000100`*

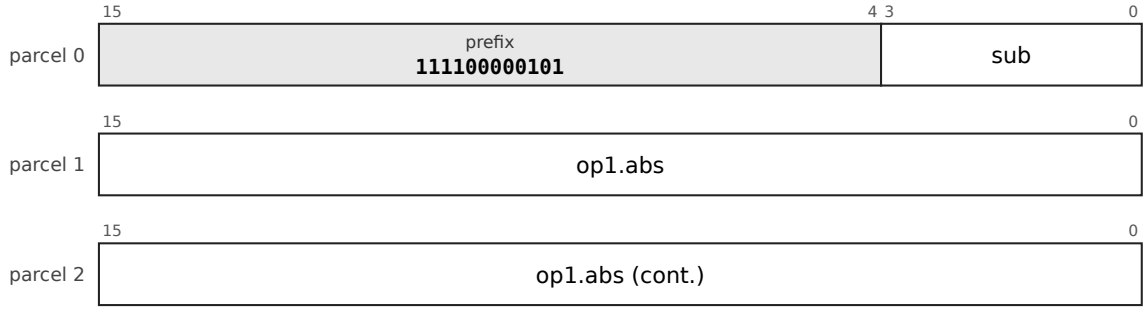



`op1` is `pc32-u1`: pc-relative, 32 bits counted in 1-byte units (-2147483648 to 2147483647 bytes).

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

Fields: `sub=p0[3:0] op1.pcrel=p1[15:0]+p2[15:0]`

### *SOF-A32-1: SOF.A32, prefix `111100000101`*

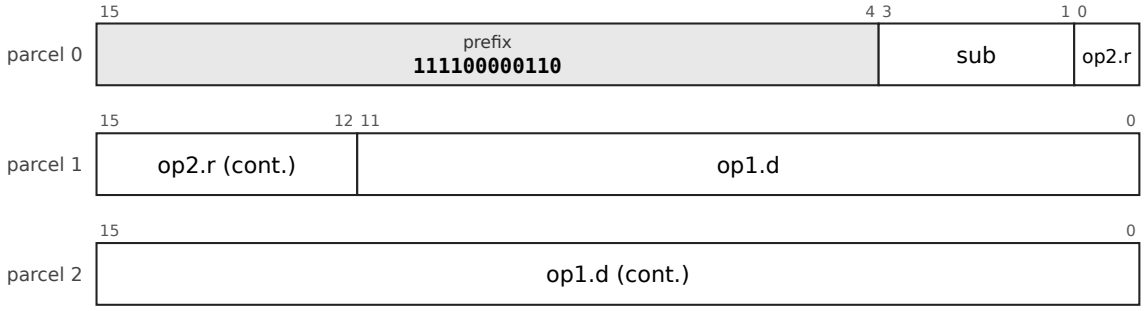



`op1` is `abs32`: absolute address, 32 bits.

`sub` (4 bits) selects the mnemonic: `clr` 0000, `com` 0001, `inc` 0010, `dec` 0011, `neg` 0100, `tst` 0101, `adc` 0110, `sbc` 0111, `asl` 1000, `asr` 1001, `push` 1010, `pop` 1011, `jmp` 1100, `call` 1101, `jmpi` 1110, `calli` 1111.

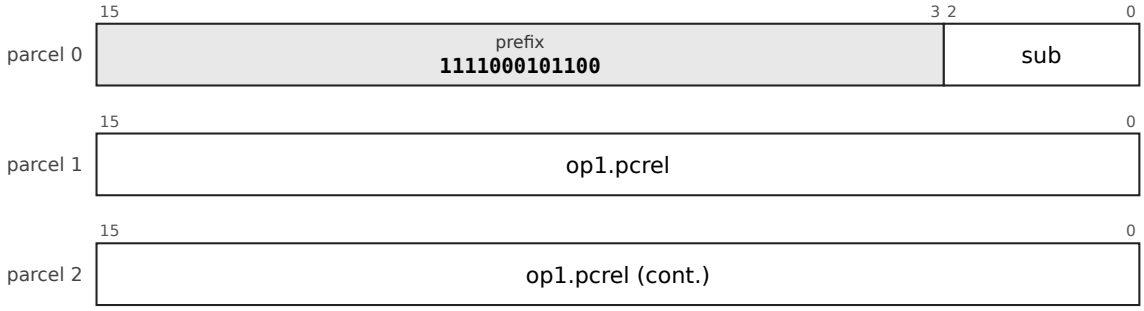
Fields: `sub=p0[3:0] op1.abs=p1[15:0]+p2[15:0]`

### *DPF-D28-2: DPF.D28, prefix `111100000110`*



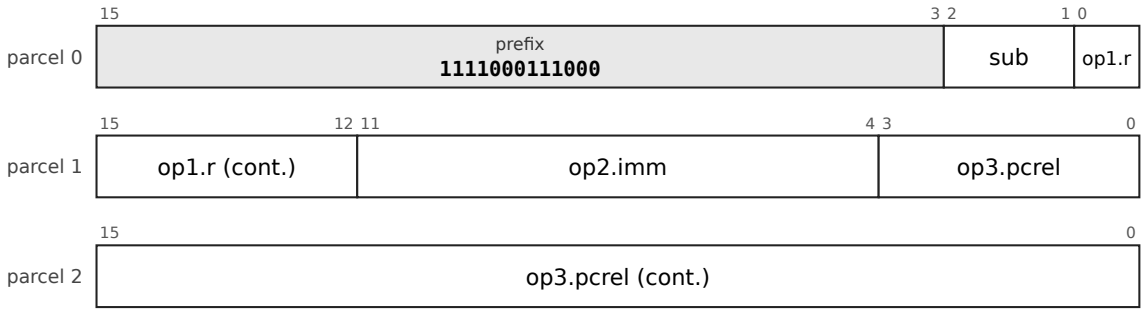

`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg5`: register, 5 bits.
`sub` (3 bits) selects the mnemonic: `shr` 000, `sar` 001, `fadd` 010, `fsub` 011, `fmul` 100, `bit` 101, `lea` 110, `movub` 111.
Fields: `sub=p0[3:1] op2.r=p0[0]+p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *BRF-1: BRF, prefix `1111000101100`*



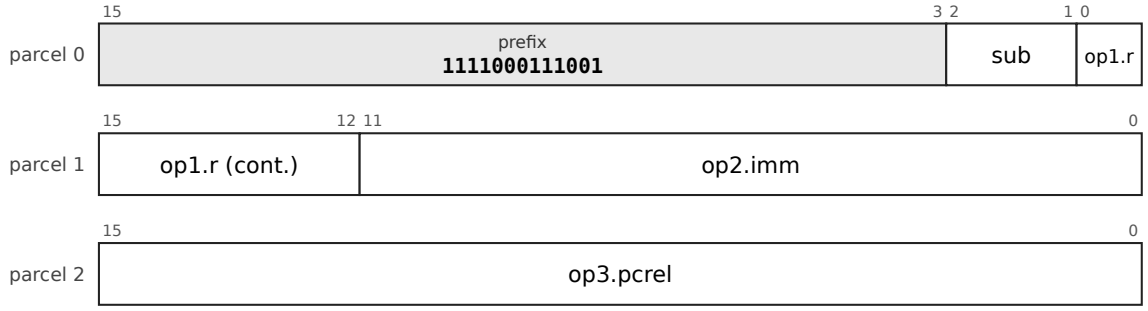

`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (3 bits) selects the mnemonic: `br` 000, `bne` 001, `beq` 010, `bpl` 011, `bmi` 100, `bvc` 101, `bvs` 110, `bcc` 111.
Fields: `sub=p0[2:0] op1.pcrel=p1[15:0]+p2[15:0]`

### *CB3I8-20-b-1: CB3I8_20/b, prefix `1111000111000`*




`op1` is `reg5`: register, 5 bits.
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.
`op3` is `pc20-u2`: pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes).
`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.
Fields: `sub=p0[2:1] op1.r=p0[0]+p1[15:12] op2.imm=p1[11:4] op3.pcrel=p1[3:0]+p2[15:0]`

### *CB3I12-16-c-1: CB3I12_16/c, prefix `1111000111001`*




`op1` is `reg5`: register, 5 bits.

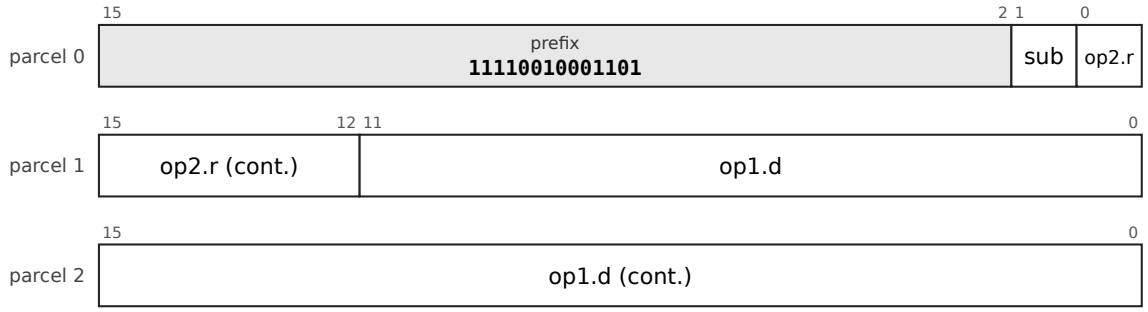
`op2` is `imm12s`: signed 12-bit immediate, -2048 to 2047.

`op3` is `pc16-u2`: pc-relative, 16 bits counted in 2-byte units (-65536 to 65534 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[2:1] op1.r=p0[0]+p1[15:12] op2.imm=p1[11:0] op3.pcrel=p2[15:0]`

### *DPF-D28-3: DPF.D28, prefix `11110010001101`*



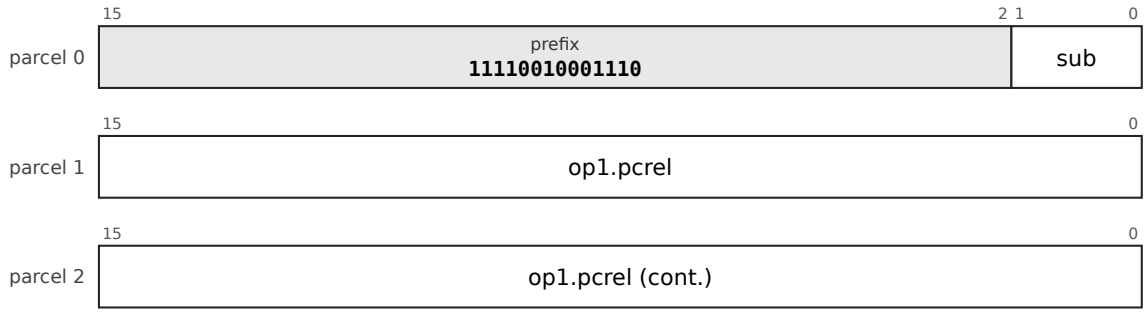

`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.

`op2` is `reg5`: register, 5 bits.

`sub` (1 bit) selects the mnemonic: `movuw` 0, `movul` 1.

Fields: `sub=p0[1] op2.r=p0[0]+p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *BRF-2: BRF, prefix `11110010001110`*

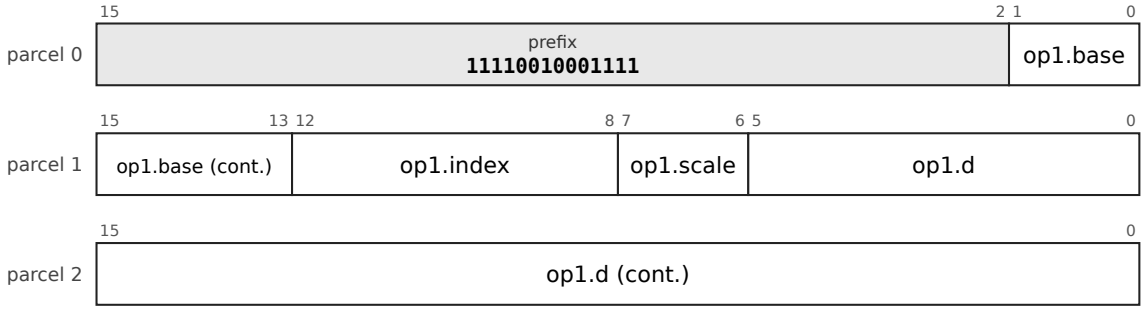



`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).

`sub` (2 bits) selects the mnemonic: `bcs` 00, `bge` 01, `blt` 10, `bgt` 11.

Fields: `sub=p0[1:0] op1.pcrel=p1[15:0]+p2[15:0]`

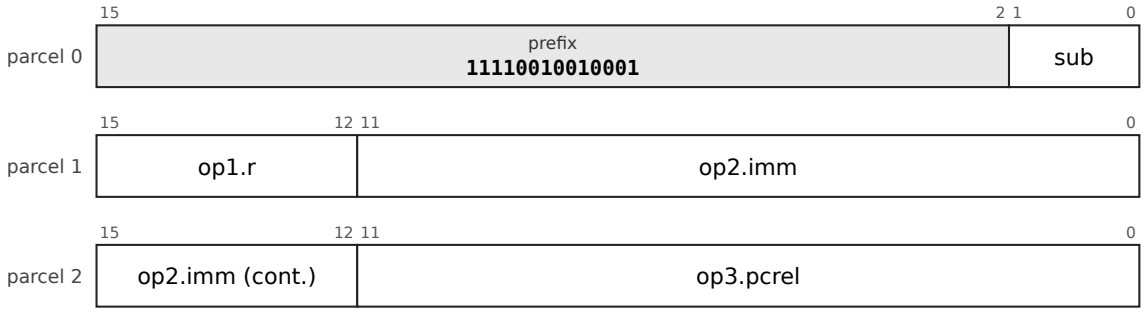
### *JTAG3-1: JTAG3, prefix `11110010001111`*




`op1` is `x5-5d22s`: memory at base + (index << scale) + signed 22-bit displacement; base register, 5 bits, index register, 5 bits.

Fields: `op1.base=p0[1:0]+p1[15:13] op1.index=p1[12:8] op1.scale=p1[7:6] op1.d=p1[5:0]+p2[15:0]`

### *CB3I16-12H-b-1: CB3I16_12H/b, prefix `11110010010001`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

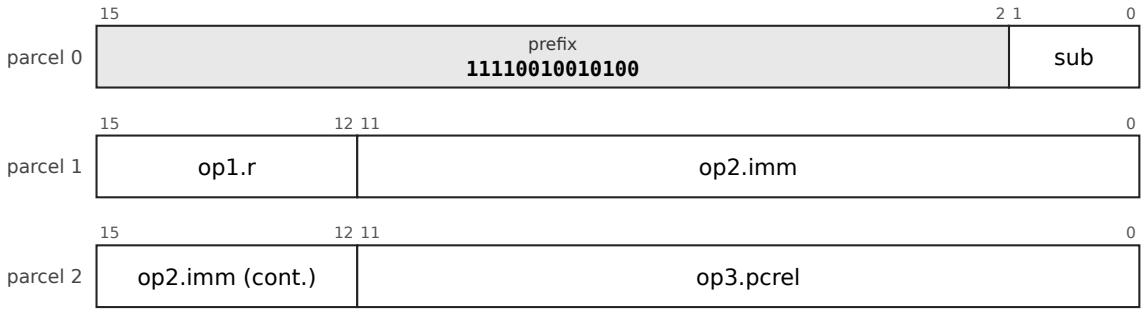
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

### *CB3I16-12H-c-1: CB3I16_12H/c, prefix `11110010010100`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

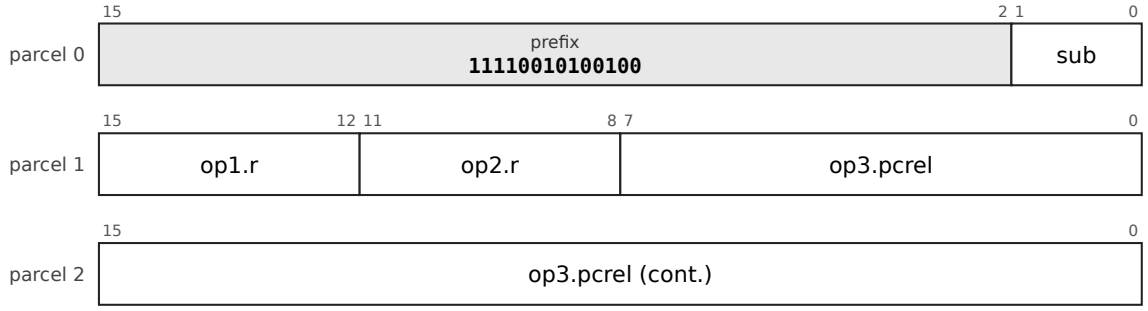
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

### *CB3R24H-c-1: CB3R24H/c, prefix `11110010100100`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

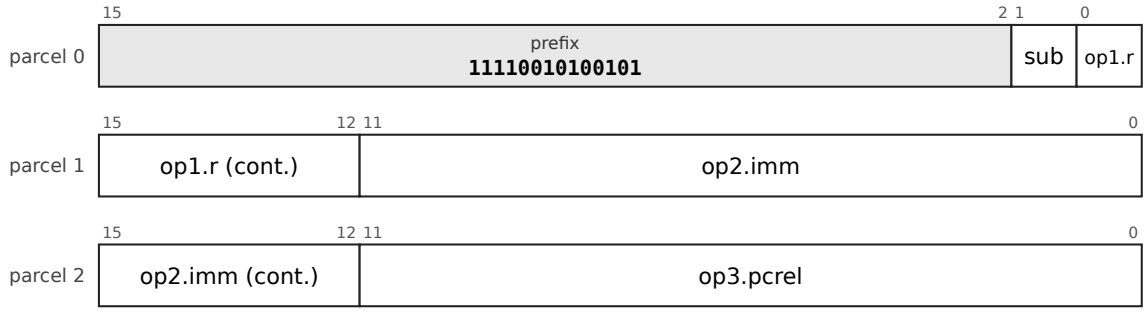
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).

`sub` (2 bits) selects the mnemonic: `cbcs` 00, `cbcc` 01, `cbhi` 10, `cblos` 11.

Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

### *CB3I16-12-a-1: CB3I16_12/a, prefix `11110010100101`*




`op1` is `reg5`: register, 5 bits.

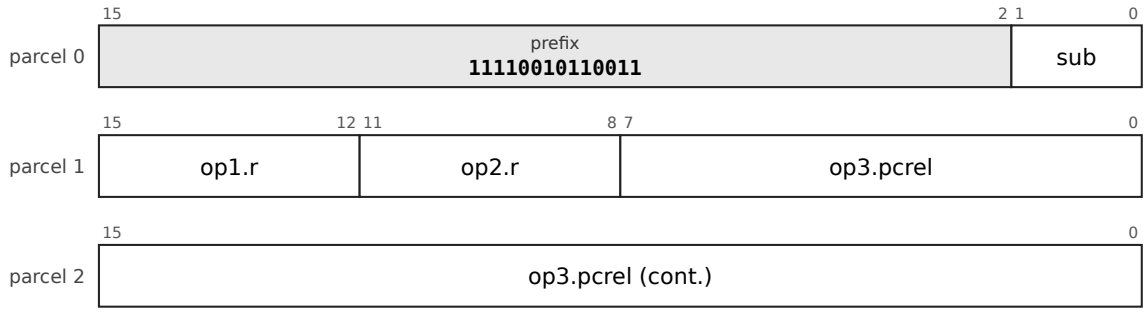
`op2` is `imm16s`: signed 16-bit immediate, -32768 to 32767.

`op3` is `pc12-u2`: pc-relative, 12 bits counted in 2-byte units (-4096 to 4094 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

Fields: `sub=p0[1] op1.r=p0[0]+p1[15:12] op2.imm=p1[11:0]+p2[15:12] op3.pcrel=p2[11:0]`

### *CB3R24H-b-1: CB3R24H/b, prefix `11110010110011`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

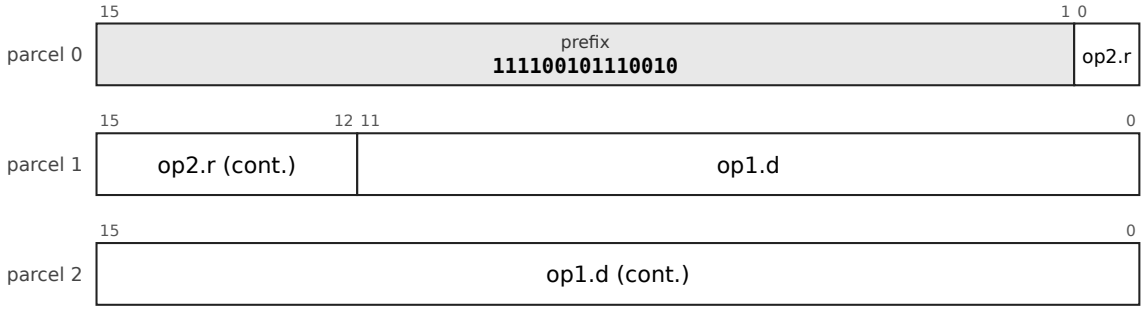
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).

`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).

`sub` (2 bits) selects the mnemonic: `cblt` 00, `cbge` 01, `cbgt` 10, `cble` 11.

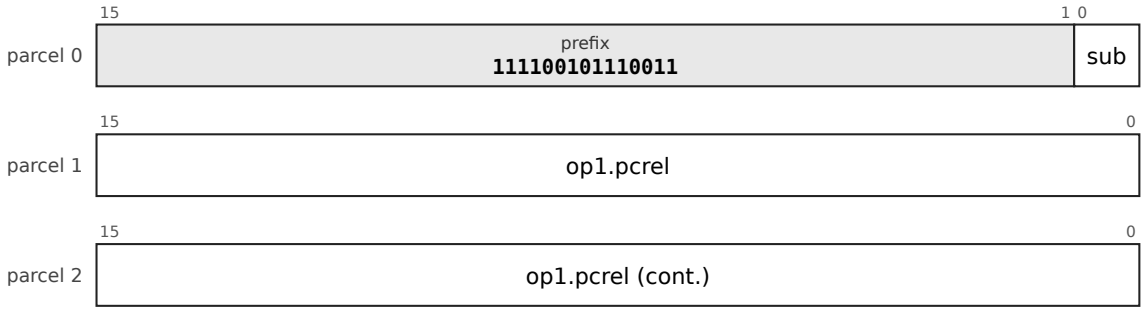
Fields: `sub=p0[1:0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

### *DPF-D28-4: DPF.D28, prefix `111100101110010`*



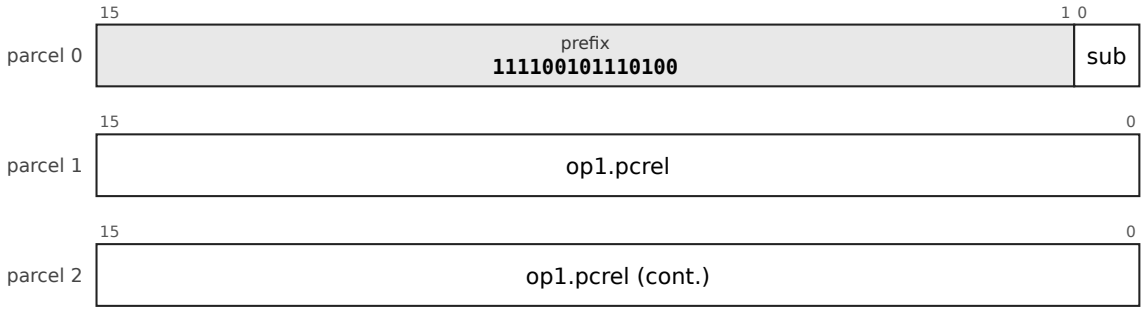

`op1` is `dpd28s-x1`: memory at dp + signed 28-bit displacement times 1, -134217728 to 134217727 bytes.
`op2` is `reg5`: register, 5 bits.
Fields: `op2.r=p0[0]+p1[15:12] op1.d=p1[11:0]+p2[15:0]`

### *BRF-3: BRF, prefix `111100101110011`*



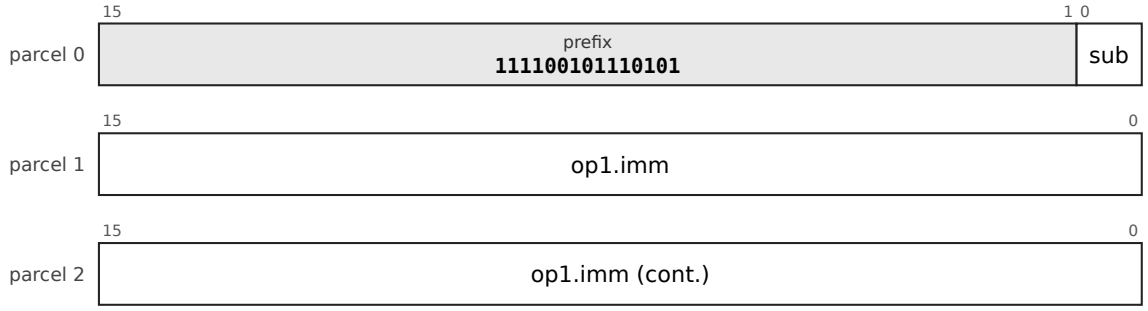

`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (1 bit) selects the mnemonic: `ble` 0, `bhi` 1.
Fields: `sub=p0[0] op1.pcrel=p1[15:0]+p2[15:0]`

### *CALLF-1: CALLF, prefix `111100101110100`*




`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
`sub` (1 bit) selects the mnemonic: `call` 0, `jmp` 1.
Fields: `sub=p0[0] op1.pcrel=p1[15:0]+p2[15:0]`

### *PUSHMF-1: PUSHMF, prefix `111100101110101`*

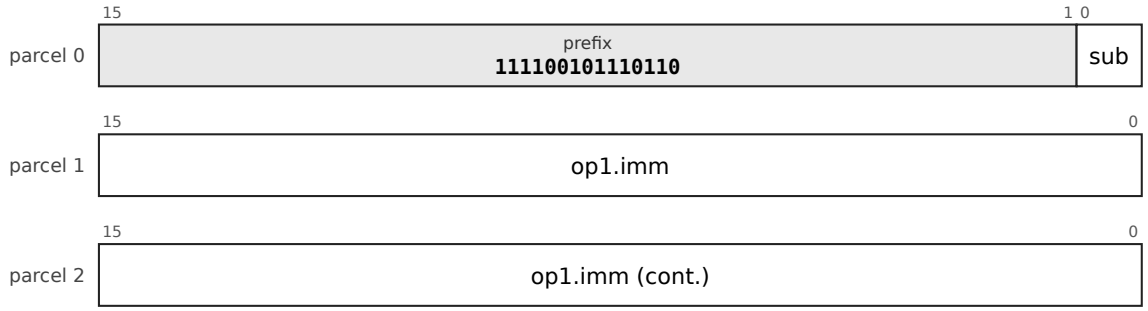



`op1` is `imm32u`: unsigned 32-bit immediate, 0 to 4294967295.

`sub` (1 bit) selects the mnemonic: `pushm` 0, `popm` 1.

Fields: `sub=p0[0] op1.imm=p1[15:0]+p2[15:0]`

### *SPADJF-1: SPADJF, prefix `111100101110110`*



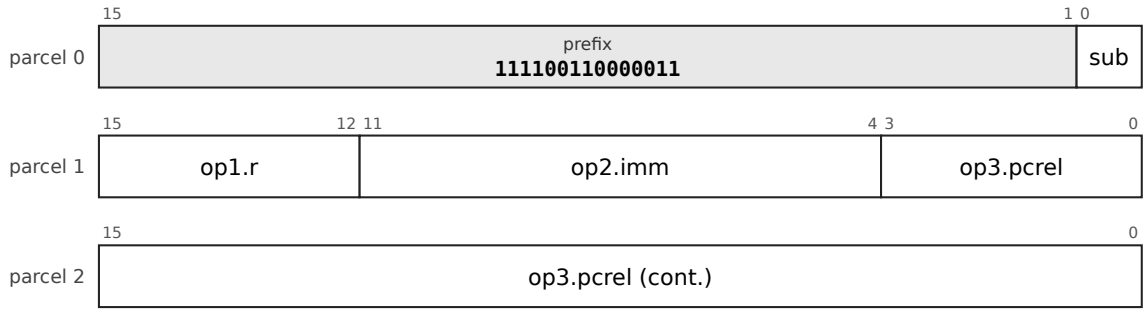

`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.

`op2` is `k-sp`: implicit register r14.

`sub` (1 bit) selects the mnemonic: `add` 0, `sub` 1.

Fields: `sub=p0[0] op1.imm=p1[15:0]+p2[15:0]`

### *CB3I8-20H-a-1: CB3I8_20H/a, prefix `111100110000011`*




`op1` is `reg4`: register, 4 bits (an index into the hot-register table).

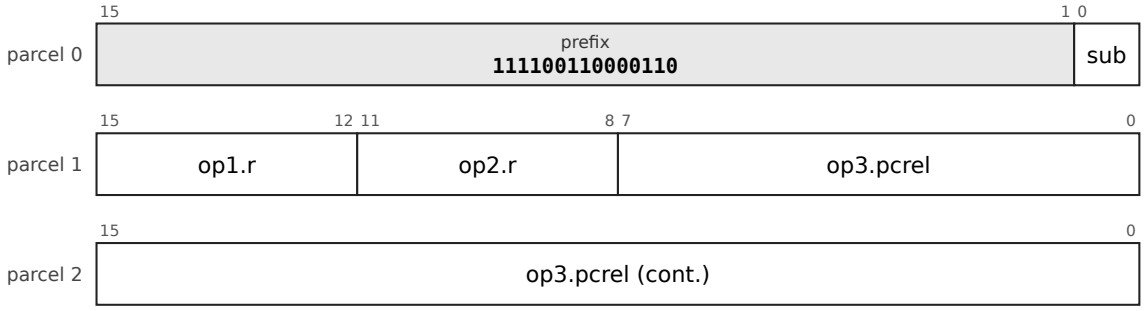
`op2` is `imm8s`: signed 8-bit immediate, -128 to 127.

`op3` is `pc20-u2`: pc-relative, 20 bits counted in 2-byte units (-1048576 to 1048574 bytes).

`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.

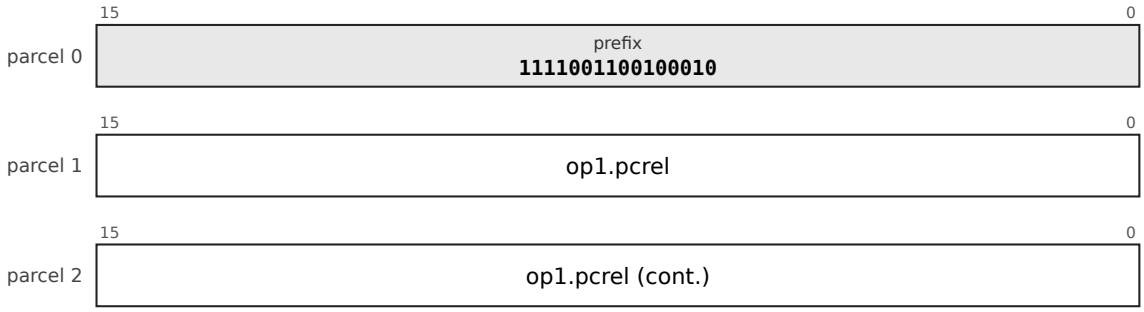
Fields: `sub=p0[0] op1.r=p1[15:12] op2.imm=p1[11:4] op3.pcrel=p1[3:0]+p2[15:0]`

### *CB3R24H-a-1: CB3R24H/a, prefix `111100110000110`*



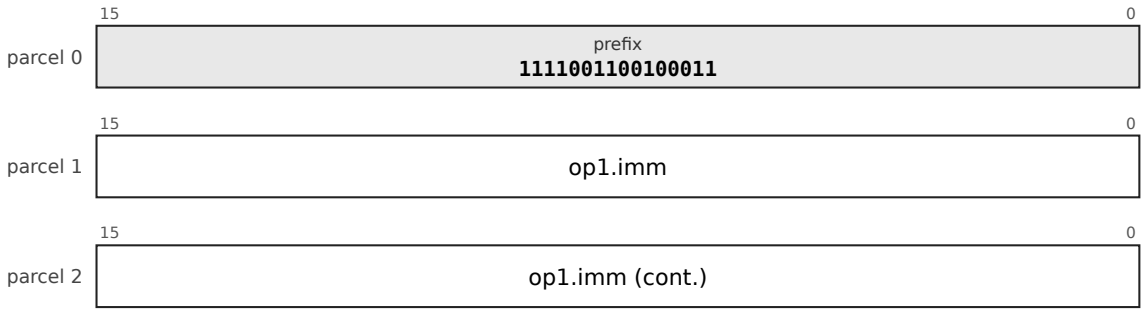

`op1` is `reg4`: register, 4 bits (an index into the hot-register table).
`op2` is `reg4`: register, 4 bits (an index into the hot-register table).
`op3` is `pc24-u2`: pc-relative, 24 bits counted in 2-byte units (-16777216 to 16777214 bytes).
`sub` (1 bit) selects the mnemonic: `cbeq` 0, `cbne` 1.
Fields: `sub=p0[0] op1.r=p1[15:12] op2.r=p1[11:8] op3.pcrel=p1[7:0]+p2[15:0]`

### *BRF-4: BRF, prefix `1111001100100010`*




`op1` is `pc32-u2`: pc-relative, 32 bits counted in 2-byte units (-4294967296 to 4294967294 bytes).
Fields: `op1.pcrel=p1[15:0]+p2[15:0]`

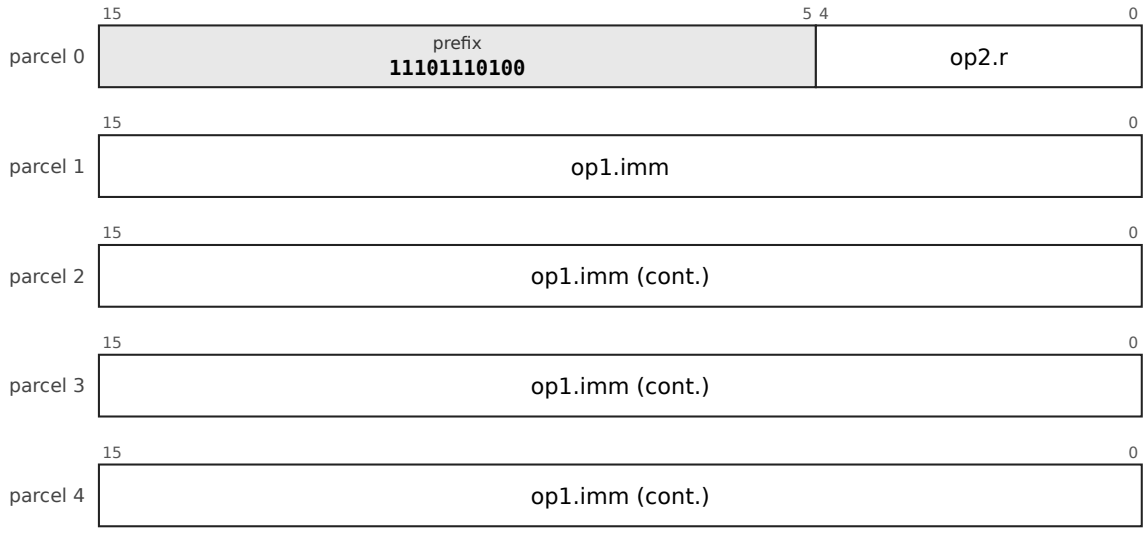
### *PUSHI3-1: PUSHI3, prefix `1111001100100011`*




`op1` is `imm32s`: signed 32-bit immediate, -2147483648 to 2147483647.
Fields: `op1.imm=p1[15:0]+p2[15:0]`

### Instructions of 5 parcels (10 bytes)

***MOVQL-1: MOVQL, prefix `11101110100`***




`op1` is `imm64s`: signed 64-bit immediate, -9223372036854775808 to 9223372036854775807.

`op2` is `reg5`: register, 5 bits.

Fields: `op2.r=p0[4:0]` `op1.imm=p1[15:0]+p2[15:0]+p3[15:0]+p4[15:0]`

## The index of the mnemonics

For every mnemonic, the charts in which it is a sub-opcode or the whole leaf.

- `ABS`: R2-1
- `ADC`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1
- `ADD`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, SPADJF-1, RR5-1, LDI12H-b-1, LDBu6-c-1, SPADJ6-a-1, LDBu8H-c-1, LDI14H-b-1, LDS12H-c-1, IMMh43-b-1, LDI18H-b-1, LDI12-b-1, LDX0-c-1, LDPC16H-c-1, LDBu8-c-1
- `ADDM`: STF-B28-2, STF-X22-2, STF-PI12-2, STF-PD12-2, STF-PC32-2, STF-A32-2
- `ADDV`: R2-1
- `AESDS`: R3-1
- `AESDSM`: R3-1
- `AESES`: R3-1
- `AESESM`: R3-1
- `AESIM`: R2-1
- `AESKS1`: RRI-1
- `AESKS2`: R3-1
- `AND`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, LDI12H-b-2, LDBu6-c-1, LDBu8H-c-1, LDI14H-b-2, LDS12H-c-1, IMMh43-b-2, LDI18H-b-2, LDI12-b-2, LDX0-c-1, LDPC16H-c-1, LDBu8-c-1
- `ASL`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-2
- `ASR`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-2
- `BCC`: BRF-1, BR2-16-c-1
- `BCS`: BRF-2, BR2-16-c-1
- `BEQ`: BRF-1, BR2-16-b-1, BR1-6-b-1
- `BGE`: BRF-2, BR2-16-c-1
- `BGT`: BRF-2, BR2-16-c-2
- `BHI`: BRF-3, BR2-16-c-2

- BIC: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, LDI12H-c-1, LDI12-d-2, LDPC12H-d-1
- BICM: STF-B28-2, STF-X22-2, STF-PI12-2, STF-PD12-2, STF-PC32-2, STF-A32-2
- BIS: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, LDBu6-c-1, LDI12H-c-1, LDBu8H-c-1, LDS12H-c-1, LDX0-c-1, LDPC16H-c-1, LDBu8-c-1
- BISM: STF-B28-2, STF-X22-2, STF-PI12-2, STF-PD12-2, STF-PC32-2, STF-A32-2
- BIT: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDI12-d-3, LDPC12H-d-2
- BITREV: R2-1
- BLE: BRF-3, BR2-16-c-2
- BLOS: BRF-4, BR2-16-c-2
- BLT: BRF-2, BR2-16-c-1
- BMI: BRF-1, BR2-16-c-1
- BNE: BRF-1, BR2-16-b-1, BR1-6-b-1
- BNTAG: TAG-1
- BPL: BRF-1, BR2-16-c-1
- BPT: SYS0-1
- BR: BRF-1, BR2-16-a-1, BR1-6-a-1, BR1-7-a-1, BR1-8-a-1, BR1-9-a-1
- BSEL: R4-1
- BSWAP: R2-1
- BSWAPL: R2-1
- BSWAPW: R2-1
- BTAG: TAG-1
- BVC: BRF-1, BR2-16-c-1
- BVS: BRF-1, BR2-16-c-1
- CALL: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, CALLF-1, SOPR-3, CALL2-16-c-1, CALL2-18-c-1, SOBu12H-x-1, SOBs12-x-1, SOX4-x-1
- CALLI: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOBu12H-x-1, SOBs12-x-1, SOX4-x-1
- CAS: R4-1
- CASDB: R4-1
- CBCC: CBR8H-c-1, CB3I16-12H-c-1, CBI4-8H-c-1, CBR8-c-1, CB3R24H-c-1, CBI6-8H-c-1, CB3I12-16-c-1, CBR10H-c-1, CBI8-8H-c-1, CBI4-10H-c-1, CBI3-8-c-1
- CBCS: CBR8H-c-1, CB3I16-12H-c-1, CBI4-8H-c-1, CBR8-c-1, CB3R24H-c-1, CBI6-8H-c-1, CB3I12-16-c-1, CBR10H-c-1, CBI8-8H-c-1, CBI4-10H-c-1, CBI3-8-c-1
- CBEQ: CBI4-8H-a-1, CBR8H-a-1, CBZ2-12H-a-1, CBI5-8H-a-1, CBI3-8-a-1, CBI6-8H-a-1, CBI10-8H-a-1, CB3I8-20H-a-1, CB3R24H-a-1, CBI5-10H-a-1, CBR8-a-1, CB3I16-12-a-1, CBZ1-4-a-1, CBI6-10H-a-1, CBR10H-a-1, CBI8-10H-a-1, CBI3-12H-a-1, CBI3-10-a-1, CBZ2-14H-a-1, CBI4-8-a-1, CBI5-8-a-1
- CBGE: CB3I16-12H-b-1, CBR8H-b-1, CBI8-8H-b-1, CBZ2-12H-b-1, CB3I8-20-b-1, CBR8-b-1, CBI3-8-b-1, CBI4-10H-b-1, CB3R24H-b-1, CBI4-8-b-1
- CBGT: CB3I16-12H-b-1, CBR8H-b-1, CBI8-8H-b-1, CBZ2-12H-b-1, CB3I8-20-b-1, CBR8-b-1, CBI3-8-b-1, CBI4-10H-b-1, CB3R24H-b-1, CBI4-8-b-1
- CBHI: CBR8H-c-1, CB3I16-12H-c-1, CBI4-8H-c-1, CBR8-c-1, CB3R24H-c-1, CBI6-8H-c-1, CB3I12-16-c-1, CBR10H-c-1, CBI8-8H-c-1, CBI4-10H-c-1, CBI3-8-c-1
- CBLE: CB3I16-12H-b-1, CBR8H-b-1, CBI8-8H-b-1, CBZ2-12H-b-1, CB3I8-20-b-1, CBR8-b-1, CBI3-8-b-1, CBI4-10H-b-1, CB3R24H-b-1, CBI4-8-b-1
- CBLOS: CBR8H-c-1, CB3I16-12H-c-1, CBI4-8H-c-1, CBR8-c-1, CB3R24H-c-1, CBI6-8H-c-1, CB3I12-16-c-1, CBR10H-c-1, CBI8-8H-c-1, CBI4-10H-c-1, CBI3-8-c-1

- `CBLT`: CB3I16-12H-b-1, CBR8H-b-1, CBI8-8H-b-1, CBZ2-12H-b-1, CB3I8-20-b-1, CBR8-b-1, CBI3-8-b-1, CBI4-10H-b-1, CB3R24H-b-1, CBI4-8-b-1
- `CBMI`: CBDI-1, CBDR-1
- `CBNE`: CBI4-8H-a-1, CBR8H-a-1, CBZ2-12H-a-1, CBI5-8H-a-1, CBI3-8-a-1, CBI6-8H-a-1, CBI10-8H-a-1, CB3I8-20H-a-1, CB3R24H-a-1, CBI5-10H-a-1, CBR8-a-1, CB3I16-12-a-1, CBZ1-4-a-1, CBI6-10H-a-1, CBR10H-a-1, CBI8-10H-a-1, CBI3-12H-a-1, CBI3-10-a-1, CBZ2-14H-a-1, CBI4-8-a-1, CBI5-8-a-1
- `CBPL`: CBDI-1, CBDR-1
- `CBVC`: CBDI-1, CBDR-1
- `CBVS`: CBDI-1, CBDR-1
- `CLMULH`: R3-1
- `CLMULL`: R3-1
- `CLO`: R2-1
- `CLR`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1, SOBu12H-a-1, SOX4-a-1, SOBs12-a-1
- `CLZ`: R2-1
- `CMP`: CMF-B28-1, CMF-X22-1, CMF-PI12-1, CMF-PD12-1, CMF-PC32-1, CMF-A32-1, CMF-I32-1, RRF-2, RRF5-1, CMBu8H-a-1, CMI12H-a-1, CMBu10H-a-1, CMBu6-a-1, CMBu8-a-1, CMPC16H-a-1, CMI18H-a-1, CMS12H-a-1, CMX0-a-1, CMI12-a-1
- `COM`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1
- `CTO`: R2-1
- `CTZ`: R2-1
- `DEC`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1, SOBu12H-a-1, SOX4-a-1, SOBs12-a-1
- `DIV`: R3-1
- `DIVU`: R3-1
- `DSHR`: BFx-1
- `EXTB`: BFy-1
- `EXTZB`: BFy-1
- `FABS`: R2-1
- `FADD`: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDPC12H-d-1
- `FADD16`: R2-1
- `FADD32`: R2-1
- `FCLASS`: R2-1
- `FCLASS16`: R2-1
- `FCLASS32`: R2-1
- `FCMP`: CMF-B28-1, CMF-X22-1, CMF-PI12-1, CMF-PD12-1, CMF-PC32-1, CMF-A32-1, CMF-I32-1, RRF-2, RRF5-1
- `FCMP16`: R2-1
- `FCMP32`: R2-1
- `FCOPYSIGN`: R3-1
- `FCVTDH`: R2-1
- `FCVTDQ`: R2-1
- `FCVTDQU`: R2-1
- `FCVTDS`: R2-1
- `FCVTHD`: R2-1
- `FCVTHL`: R2-1
- `FCVTHS`: R2-1
- `FCVTLH`: R2-1
- `FCVTLS`: R2-1

- FCVTQD: R2-1
- FCVTQUD: R2-1
- FCVTSD: R2-1
- FCVTSH: R2-1
- FCVTSL: R2-1
- FDIV: R2-1
- FDIV16: R2-1
- FDIV32: R2-1
- FENCE: SYS0-1
- FMADD: R4-1
- FMADD16: R4-1
- FMADD32: R4-1
- FMAX: R3-1
- FMIN: R3-1
- FMSUB: R4-1
- FMSUB16: R4-1
- FMSUB32: R4-1
- FMUL: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDPC12H-d-2
- FMUL16: R2-1
- FMUL32: R2-1
- FNEG: R2-1
- FNMADD: R4-1
- FNMADD16: R4-1
- FNMADD32: R4-1
- FNMSUB: R4-1
- FNMSUB16: R4-1
- FNMSUB32: R4-1
- FRINT: R2-1
- FRINTA: R2-1
- FRINTM: R2-1
- FRINTP: R2-1
- FRINTZ: R2-1
- FSFLAGS: R2-1
- FSQRT: R2-1
- FSQRT16: R2-1
- FSQRT32: R2-1
- FSUB: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDPC12H-d-1
- FSUB16: R2-1
- FSUB32: R2-1
- HALT: SYS0-1
- HINT: SYSI-1
- ICINV: SYSR-2
- ICINVA: SYS0-1
- INC: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1, SOBu12H-a-1, SOX4-a-1, SOBs12-a-1
- INSB: BFy-1
- ISYNC: SYS0-1

- `JMP`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, CALLF-1, SOPR-3, CALL2-16-j-1, SOBu12H-x-1, SOBs12-x-1, SOX4-x-1
- `JMPI`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOBu12H-x-1, SOBs12-x-1, SOX4-x-1
- `JTAG`: JTAG3-1, JTAG2-1
- `LEA`: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, DPF-D28-2, LEASx12H-a-1, LDX0-a-1, LDPC12H-a-1, LDPC14H-a-1, LDBu8H-a-1, LDPC16H-a-1, LDBu10H-a-1, LDHHH1-a-1, LEASPh4-a-1, LDBs6-a-1, LEAS12H-a-1, LDSPh4-a-1, LDBu12H-a-1, LDHHH3-a-1, LDPC12-a-1, LDBu8-a-1, LDX6H-a-1, LDBs10-a-1, LEAS14H-a-1
- `LL`: R3-1
- `LPAD`: SYS0-1
- `MASKEQZ`: R3-1
- `MASKNEZ`: R3-1
- `MAX`: R3-1
- `MAXU`: R3-1
- `MCMP`: R3-1
- `MCPY`: R3-1
- `MFASID`: SYSR-1
- `MFCPUID`: SYSR-2
- `MFDP`: SYSR-1
- `MFPS`: SYSR-1
- `MFSR`: MFSR-1
- `MIN`: R3-1
- `MINU`: R3-1
- `MMOV`: R3-1
- `MOD`: R3-1
- `MODU`: R3-1
- `MOV2`: FX-MOV2-1
- `MOVB`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-2, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-1, RRF-1, RRF5-1, STBu8H-b-1, LDBu6-b-1, STX0-b-1, STBs6-b-1, LDX0-b-1, LDI12-d-2, STBu10H-b-1, LDBu8H-b-1, STHHH1-b-1, STBu12H-b-1, LDPC12H-b-1, STBs8H-b-1, LDS12H-b-1, STX4H-b-1, LDBu12H-b-1, LDBs6-b-1, STBu6-b-1
- `MOVB.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVB.U`: LDU-B28-1, STU-B28-1, LDU-B12-1, STU-B12-1, LDU-X6-1, STU-X6-1
- `MOVCALL`: FX-MOVCALL3-1, FX-MOVCALL2-1
- `MOVL`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-2, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, STBu6-a-1, LDX0-a-1, LDPC12H-a-1, LDPC14H-a-1, STBs8H-a-1, LDBu8H-a-1, STX0-a-1, LDPC16H-a-1, STBu10H-a-1, LDBu10H-a-1, LDHHH1-a-1, STHHH1-a-1, LDBs6-a-1, LDSPh4-a-1, STSPh4-a-1, LDBu12H-a-1, STBu12H-a-1, LDHHH3-a-1, LDPC12-a-1, LDBu8-a-1, STBs6-a-1, LDX6H-a-1, STX6H-a-1, LDBs10-a-1, STBs8-a-1
- `MOVL.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVL.U`: LDU-B28-1, STU-B28-1, LDU-B12-1, STU-B12-1, LDU-X6-1, STU-X6-1
- `MOVQ`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, STBu6-a-1, LDX0-a-1, LDPC12H-a-1, LDI14H-a-1, LDPC14H-a-1, STBs8H-a-1, LDBu8H-a-1, LDI16H-a-1, STX0-a-1, IMMh43-a-1, LDI12-a-1, LDPC16H-a-1, STBu10H-a-1, LDBu10H-a-1, LDHHH1-a-1, STHHH1-a-1, LDBs6-a-1, LDSPh4-a-1, IMMh45-a-1, STSPh4-a-1, LDBu12H-a-1, STBu12H-a-1, LDHHH3-a-1, LDPC12-a-1, IMMh47-a-1, LDBu8-a-1, STBs6-a-1, LDX6H-a-1, STX6H-a-1, LDBs10-a-1, STBs8-a-1
- `MOVQ.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVQ.U`: LDU-B28-1, STU-B28-1, LDU-B12-1, STU-B12-1, LDU-X6-1, STU-X6-1

- `MOVQL`: MOVQL-1
- `MOVUB`: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-2
- `MOVUL`: LDF-B28-3, LDF-X22-3, LDF-PI12-3, LDF-PD12-3, LDF-PC32-3, LDF-A32-3, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-3
- `MOVUQ`: LDF-B28-4, LDF-X22-4, LDF-PI12-4, LDF-PD12-4, LDF-PC32-4, LDF-A32-4, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-4
- `MOVUW`: LDF-B28-3, LDF-X22-3, LDF-PI12-3, LDF-PD12-3, LDF-PC32-3, LDF-A32-3, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-3
- `MOVW`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-2, STF-B28-1, STF-X22-1, STF-PI12-1, STF-PD12-1, STF-PC32-1, STF-A32-1, DPF-D28-1, RRF-1, RRF5-1, STBu8H-b-1, LDBu6-b-1, STX0-b-1, STBs6-b-1, LDX0-b-1, STBu10H-b-1, LDBu8H-b-1, STHHH1-b-1, STBu12H-b-1, LDPC12H-b-1, STBs8H-b-1, LDS12H-b-1, STX4H-b-1, LDBu12H-b-1, LDBs6-b-1, STBu6-b-1
- `MOVW.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVW.U`: LDU-B28-1, STU-B28-1, LDU-B12-1, STU-B12-1, LDU-X6-1, STU-X6-1
- `MOVZB`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-3, DPF-D28-1, RRF-1, RRF5-1, LDX0-a-1, LDPC12H-a-1, LDPC14H-a-1, LDBu8H-a-1, LDPC16H-a-1, LDBu10H-a-1, LDHHH1-a-1, LDBs6-a-1, LDSPh4-a-1, LDBu12H-a-1, LDHHH3-a-1, LDPC12-a-1, LDBu8-a-1, LDX6H-a-1, LDBs10-a-1
- `MOVZB.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVZB.U`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVZL`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-2, DPF-D28-1, RRF-1, RRF5-1, LDBu6-b-1, LDX0-b-1, LDBu8H-b-1, LDPC12H-b-1, LDS12H-b-1, LDBu12H-b-1, LDBs6-b-1
- `MOVZL.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVZL.U`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVZW`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-3, DPF-D28-1, RRF-1, RRF5-1, LDBu6-b-1, LDX0-b-1, LDBu8H-b-1, LDPC12H-b-1, LDS12H-b-1, LDBu12H-b-1, LDBs6-b-1
- `MOVZW.D`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MOVZW.U`: LDU-B28-1, LDU-B12-1, LDU-X6-1
- `MSCN`: R3-1
- `MSCNN`: R3-1
- `MSET`: R3-1
- `MTASID`: SYSR-1
- `MTDMW`: SYSR-1
- `MTDP`: SYSR-1
- `MTPS`: SYSR-1
- `MTSR`: MTSR-1
- `MUL`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, LDI12H-c-1, LDI12-d-1, LDPC12H-d-1
- `MULH`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, LDPC12H-d-1
- `MULHU`: R3-1
- `MULV`: R2-1
- `MVAL`: R2-1
- `NEG`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1
- `NOP`: SYS0-1
- `POP`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-2, SOBu12H-a-2, SOX4-a-2, SOBs12-a-2
- `POPCNT`: R2-1
- `POPM`: PUSHMF-1, PUSHM16b-a-1, PUSHM8-a-1, PUSHM24-a-1

- `POPMRET`: FX-POPMRET-1
- `POPRET`: FX-POPRET-1
- `POPX`: SYS0-1
- `PROBER`: R2-1
- `PROBEW`: R2-1
- `PUSH`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-2, PUSHI3-1, PUSHI2-1, SOBu12H-a-2, SOX4-a-2, SOBs12-a-2
- `PUSHM`: PUSHMF-1, PUSHM16b-a-1, PUSHM8-a-1, PUSHM24-a-1
- `PUSHX`: SYS0-1
- `RET`: SYS0-1
- `ROR`: R3-1
- `RORI`: RRI-1
- `RORI32`: RRI-1
- `RTI`: SYS0-1
- `RTT`: SYS0-1
- `SAR`: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDI12-d-1, SHIh43-a-1, SHIh44-a-1, LDPC12H-d-1
- `SBC`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1
- `SC`: R3-1
- `SET`: SETF-1
- `SEV`: SYS0-1
- `SHA256SIG0`: R2-1
- `SHA256SIG1`: R2-1
- `SHA256SUM0`: R2-1
- `SHA256SUM1`: R2-1
- `SHA512SIG0`: R2-1
- `SHA512SIG1`: R2-1
- `SHA512SUM0`: R2-1
- `SHA512SUM1`: R2-1
- `SHL`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, LDI12-d-1, SHIh43-a-1, SHIh44-a-1, LDPC12H-d-1
- `SHR`: LDF-B28-2, LDF-X22-2, LDF-PI12-2, LDF-PD12-2, LDF-PC32-2, LDF-A32-2, LDFI-I32-1, DPF-D28-2, RRF-2, RRF5-1, LDI12-d-1, SHIh43-a-1, SHIh44-a-1, LDPC12H-d-1
- `SOB`: SOB-1
- `SPL`: SYSI-1
- `SUB`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, SPADJF-1, RR5-1, LDI12H-b-1, LDBu6-c-1, SPADJ6-a-1, LDBu8H-c-1, LDI14H-b-1, LDS12H-c-1, IMMh43-b-1, LDI18H-b-1, LDI12-b-1, LDX0-c-1, LDPC16H-c-1, LDBu8-c-1
- `SUBM`: STF-B28-2, STF-X22-2, STF-PI12-2, STF-PD12-2, STF-PC32-2, STF-A32-2
- `SUBV`: R2-1
- `SWAP`: R3-1
- `SWAPDB`: R3-1
- `TLBI`: SYSR-1
- `TLBIA`: SYS0-1
- `TLBSYNC`: SYS0-1
- `TRAP`: SYSI-1
- `TRAPCC.CC`: SYS0-1
- `TRAPCC.CS`: SYS0-1
- `TRAPCC.EQ`: SYS0-1
- `TRAPCC.GE`: SYS0-1

- `TRAPCC.GT`: SYS0-1
- `TRAPCC.HI`: SYS0-1
- `TRAPCC.LE`: SYS0-1
- `TRAPCC.LOS`: SYS0-1
- `TRAPCC.LT`: SYS0-1
- `TRAPCC.MI`: SYS0-1
- `TRAPCC.NE`: SYS0-1
- `TRAPCC.PL`: SYS0-1
- `TRAPCC.VC`: SYS0-1
- `TRAPCC.VS`: SYS0-1
- `TRAPL`: SYSI-1
- `TST`: SOF-B28-1, SOF-X22-1, SOF-PI12-1, SOF-PD12-1, SOF-PC32-1, SOF-A32-1, SOPR-1, SOBu12H-a-1, SOX4-a-1, SOBs12-a-1
- `UTF8DEC`: R2-1
- `UTF8ENC`: R2-1
- `UTF8LEN`: R2-1
- `WFE`: SYS0-1
- `XADD`: R3-1
- `XADDDB`: R3-1
- `XAND`: R3-1
- `XANDDB`: R3-1
- `XMAX`: R3-1
- `XMAXDB`: R3-1
- `XMAXU`: R3-1
- `XMAXUDB`: R3-1
- `XMIN`: R3-1
- `XMINDB`: R3-1
- `XMINU`: R3-1
- `XMINUDB`: R3-1
- `XNORPOP`: R3-1
- `XOR`: LDF-B28-1, LDF-X22-1, LDF-PI12-1, LDF-PD12-1, LDF-PC32-1, LDF-A32-1, LDFI-I32-1, DPF-D28-1, RRF-1, RRF5-1, RR5-1, LDBu6-c-2, LDI12H-c-1, LDBu8H-c-2, LDS12H-c-2, LDX0-c-2, LDPC16H-c-2, LDBu8-c-2
- `XOR_`: R3-1
- `XOR_DB`: R3-1
- `XXOR`: R3-1
- `XXORDB`: R3-1

*Generated by `tools/mkmanual.py`; do not edit.*

# Part III — Appendices

# Appendix A — Improvements beyond the PDP-11

**These are improvements that the author has approved, each with its benefit, its cost and what other machines do.** They come from the reports on kernel pain points, hardware vulnerabilities and OpenBSD's mitigations. Most are approved "to be specified": the decision is made and the specification is not written, and what is not yet specified is listed in appendix J. The same ordering principle as the rest of the manual applies: code size, then easy recoverability, then speed and efficiency through fewer gates and tighter hot loops. Ideas that nobody has decided are in appendix H, and what has been put off is in appendix I.

## A.1 A page bit that enforces landing pads

**A.1.1** The `LPAD` instruction marks a legal target of an indirect jump or call; a page bit makes the hardware fault when an indirect transfer lands anywhere else, with the fault attributed to the branch.

**A.1.2** A page bit that gates landing pads is approved, to be specified (errata 84, item 5). *(D)* Open: how the bit is specified and what the six reserved system words become (J.9.6), and when enforcement is turned on in the ABI (J.D.2).

*Commentary.* **Precedent.** `LPAD` already exists, and the ABI leaves landing pads off in version 1, with no switch to enforce them. **Trade-offs.** The bit costs one comparator and two bytes per address-taken function. **Benefits.** It blocks jump-oriented attacks at the transfer, with no recovery code beyond the fault handler that already exists. **Modern comparables.** Intel's indirect-branch tracking and ARM's branch target identification, both of which OpenBSD enforces by default where the hardware has them. **Impact.** *Kernel:* A landing-pad violation is a fault the handler already handles; the kernel sets the page bit at exec time. *Compiler:* The compiler emits `LPAD` at every address-taken function, two bytes each. *Hardware:* One page bit and one comparator on an indirect-transfer target.

## A.2 A page bit that says a system call may be made from here

**A.2.1** A page-table bit lets the hardware refuse a system call from any page not so marked, so that the kernel's check of where a call came from moves into the instruction.

**A.2.2** A page bit for system-call entry (the `SYS` page bit) is approved, to be specified (errata 84, item 8). *(D)* Open: its specification (J.9.6).

*Commentary.* **Precedent.** OpenBSD's `msyscall` and `pinsyscalls` restrict where system calls may be issued from. **Trade-offs.** One bit and one gate; the kernel no longer needs to check the origin on every call. **Benefits.** About four instructions removed from every system call, and region pinning with no hot-path check. **Modern comparables.** No architecture surveyed has this bit; it is done by the kernel in software. **Impact.** *Kernel:* The kernel no longer checks where a system call came from on every call, which removes about four instructions from each. *Compiler:* None. *Hardware:* One page bit and one gate at the `TRAP` decode.

## A.3 A check of the stack pointer at system-call entry

**A.3.1** The hardware checks that the stack pointer lies in a region registered as a stack when a system call or a fault is taken.

**A.3.2** A stack-limit register is approved, to be specified (errata 84, item 4). *(D)* Open: its specification (J.9.6).

*Commentary.* **Precedent.** OpenBSD checks the stack pointer against a `MAP_STACK` region on system calls and faults. **Trade-offs.** It adds a comparison to entry, which is the path the design most wants short. **Impact.** *Kernel:* The kernel's own check moves into the hardware. *Compiler:* None. *Hardware:* A comparator and a region register on the entry path.

## A.4 Protecting the return address

**A.4.1** The return address lives on the data stack, because `CALL` pushes it and there is no link register.

**A.4.2** The mirror shadow stack is approved with its recommendations (errata 85, study item 6). *(D)*

*Commentary.* **Precedent.** x86 has the same exposure; Intel's control-flow enforcement has a shadow stack, and OpenBSD's clang has `retguard`. **Benefits.** Detection of a changed return address with little or no recovery code. **Impact.** *Kernel:* A shadow stack is a second stack for the kernel to manage and unwind. *Compiler:* none beyond the setjmp path. *Hardware:* A second stack structure and its control.

## A.5 Tag or key bits in pointers

**A.5.1** The 25 spare bits of a 64-bit pointer (a 39-bit virtual address, the three-level form, uses the rest; 48 and 57 bits leave fewer) can carry a colour, as a memory-tagging scheme does.

**A.5.2** Full 128-bit CHERI is rejected, and a valid-pointer tag per register and per 8-byte word (stage 1) is kept (errata 86, study item 3); granule colouring and reserved colours for uninitialised and freed memory are approved, to be specified (errata 84, items 1 and 2). *(D)* Open: they spend pointer bits that the canonical-address rule refuses to software, so that rule must be restated (J.A.1).

*Commentary.* **Precedent.** ARM's top-byte-ignore, pointer authentication and memory tagging. **Trade-offs.** The cost is canonical-address rules: pointers must be masked before use, and the hardware must define what a tagged pointer does. **Impact.** *Kernel:* The kernel defines what a tagged pointer means across the user and kernel boundary. *Compiler:* Every pointer use needs masking, and pointer arithmetic must preserve tags. *Hardware:* Gates and state in the address path.

## A.6 A physical address wider than 40 bits

**A.6.1** The physical address is 40 to 64 bits (errata 102, *(D)*), which lets petabytes of memory and device windows be mapped; the present page-table entry holds the physical page number in bits 12 to 39 and keeps bits 40 to 63 zero, so the entry format has to grow its page number into them or change. Open: the entry format (J.6.2).

*Commentary.* **Trade-offs.** The author's reason is that mapping petabytes needs those bits (errata 97). The cost is a wider page number in the entry, in every translation-buffer entry and in every bus address; the benefit is room for devices and memory in one address space without a decoding bit. **Modern comparables.** x86-64 and AArch64 define physical addresses of up to 52 bits and RISC-V up to 56 (stated from memory, not confirmed in the sources read). **Impact.** *Kernel:* a wider entry and a physical-address width to discover. *Compiler:* none. *Hardware:* more bits in the walker, the translation buffer and the bus.

## A.7 A second page-table base register

**A.7.1** A second page-table base, `KTBR`, translates the upper half of the address space, so that a kernel and a user program run with different tables without flushing (6.9.1; the author accepted the split, errata 71). *(D)*

*Commentary.* **Precedent.** The cost of separating kernel and user page tables on x86 after Meltdown was about 5% typical and about 30% in the worst case in the developers' own estimates, and the tagged translation buffers (PCID) made it tolerable. **Benefits.** The isolation is in the hardware from the start. **Modern comparables.** AArch64 has two translation-table base registers, one for each half of the address space. **Impact.** *Kernel:* The kernel switches one base register at the privilege change instead of managing a shared table. *Compiler:* None. *Hardware:* A second base register and a rule choosing which half uses which.

## A.8 The user-memory guard instructions

**A.8.1** `PSW.UA`, `STUA` and `CLUA` stay in the architecture until the OpenBSD port confirms the user-qualified moves (6.10.4, 9.6.7); the author accepted this with the five leanings of the study of user access (errata 86, item 18). *(D)* Details to be determined.

## A.9 Streaming block operations

Open: whether the block operations have a form that does not fill the data cache (J.9.8).

## A.10 Watchpoints, and what was rejected

**A.10.1** Precise hardware watchpoints are approved, to be specified (errata 84, item 16). *(D)* Open: the debug registers and how a watchpoint is reported (J.6.11).

**A.10.2** Three other retire-stage items are approved, to be specified (errata 84): predictor and cache-way isolation by address-space number, both parts (item 13: "smp is immediate, so don't defer"); the residue check of the results of the arithmetic

unit (item 14); and precise reporting of ECC and parity errors (item 15), with the restart contract of item 17. *(D)* Details to be determined.

**A.10.3** A user-visible transactional region is rejected (errata 84, item 19), and so are a sealed (immutable) page bit and a hardware mask for `pledge`, which have no benefit over the kernel doing the checks. *(D)*

# Appendix B — Decisions pending

**This appendix is a list of pointers.** Each decision listed here was a question with one proposed answer: the question and its answer, with the precedent, the trade-offs and the benefits, are in appendix H under "Appendix B" (the identifiers P, R, S, N and B are those of the working document `decisions-pending.md`, and the boot choices are worked through in `boot.md`). A question that the architecture must answer, with each option weighed, is an open item in appendix J. What has been decided is in the chapters, and what has been put off in appendix I. Questions of this list that the author answered are no longer listed: the wake rule of `WFE` (7.10.2), priority 7 on interrupt entry as well as exception entry (6.3.2), constant-time execution by the secure core and the `DIT` bit (7.11.3, 9.3.7; what stays open is J.9.2), and, in the entries that remain, the parts the author decided (the random-number source, the device route for vector work, enumeration, the all-ones read of an absent function and the block-0 boot) are stated as decided. Each question was judged by the three principles in order: code size, easy recoverability (less recovery code), and speed and efficiency through fewer gates and tighter hot loops.

**B.1 The instruction set and the system**

- B.1.2 An unprivileged time counter (P3)
- B.1.3 Feature and identification registers (P4)
- B.1.4 TLB invalidation by address-space tag (P5)
- B.1.5 What a multi-register instruction leaves behind when it faults (P12)
- B.1.7 A prefetch instruction (P15)
- B.1.8 Performance counters (P16)
- B.1.10 The literal pool and execute-only code (P18)
- B.1.11 Vector and SIMD math (S1 to S6)

**B.2 The bus, boot and devices**

- B.2.1 Data-cache maintenance by address (P6)
- B.2.2 Firmware calls (P7)
- B.2.3 Double-width compare-and-swap and a random-number instruction (P8, P9)
- B.2.4 The interrupt controller and routing to cores (R1)
- B.2.5 Inter-processor interrupts and starting other cores (R2, B2)
- B.2.6 How coherence scales (R3) See also the open entry J.9.11.
- B.2.8 The device layout and the descriptor (R5)
- B.2.10 Errors raised by devices and the bus (R7)
- B.2.11 Virtualisation (R8)
- B.2.12 Enumeration and hot-plug (N1 to N3)
- B.2.13 The boot decisions (B1, B3, B4, B5, B8) The UEFI decision is in chapter 4.
- B.2.14 How USB attaches, and virtio first

# Appendix C — Glossary

**Terms the manual uses, collected in one place.** Each entry gives the plain meaning and, in brackets, the chapter or section where the manual deals with it. Terms for the instruction groups (the "escape" opcodes) are listed by name and opcode; the full encodings are in the instruction-set text `pdp64-isa.md`.

## C.1 Architecture and encoding

| Term | Meaning |
|---|---|
| **pdp64** | the 64-bit extension of the PDP-11 this manual describes, as the PDP-11 extended the PDP-10's ideas down and the VAX extended the PDP-11 up (chapter 1) |
| **earlier iteration** | pdp64/r20 (the original encoding) or pdp64/16 (a 16-register prefix code): a design built and measured before the pdp64 was chosen, superseded by it, and no longer a CPU (1.8) |
| **parcel** | a 16-bit unit of instruction encoding; an instruction is one to three parcels, two to six bytes, or five parcels for `MOVQL` (9.4) |
| **leaf** | a group of mnemonics with one form, selected by a prefix of the first parcel; a sub-opcode picks the mnemonic (9.1) |
| **prefix** | the top 3 to 16 bits of the first parcel; the prefixes form a prefix code (no prefix begins another), so the first parcel matches at most one leaf and fixes the length of the instruction (9.1) |
| **form** | the list of operand shapes of a leaf: how each operand is written in assembly and how many bits each part has (9.1) |
| **sub-opcode** | the field after the prefix that selects one mnemonic of the leaf (9.1) |
| **hot register** | one of the eight (3-bit field) or sixteen (4-bit field) most-used registers, which a register field of a one-parcel form names by an index into a table; the tables are part of the ABI (9.4, appendix D) |
| **fused instruction** | `MOVCALL`, `POPRET`, `POPMRET` or `MOV2`: one instruction for a pair that the compiler finds adjacent (9.10) |
| **relaxation** | the linker's choice, after the sections are laid out, of the shortest form that reaches for a call or an address; on by default (9.5) |
| `CBCC` | compare-immediate-and-branch on condition, the one fused compare and branch (9.10) |
| `RORI32` | rotate right by an immediate, 32-bit, sign-extended result |
| `SOB` | subtract one and branch, either direction (9.10) |
| `MCPY, MMOV, MSET, MCMP, MSCN, MSCNN` | the block operations: copy, move (overlap-safe), fill, compare, scan for a byte, scan for the first byte not equal; progress is held in registers so a fault restarts for free (9.11) |
| `MVAL` | block scan of valid UTF-8, one of the string instructions of the same family (9.9) |
| `PUSHM, POPM` | push and pop a register set named by a mask; the useful half of the VAX's `CALLS` without the frame (9.11) |
| `JMPI, CALLI` | jump and call through a memory word; the only deferred forms, used for PLT stubs and imports (9.10, 9.16) |
| `JTAG, TAGOP` | multiway dispatch on a pointer's low tag bits, and the tag-branch group (9.10) |
| `LPAD` | marks a legal target of an indirect jump or call; optional in ABI version 1 (appendix A.1) |
| `CLMUL` | carry-less multiply, the primitive under every CRC and Galois-field operation (9.9) |
| `XNORPOP, POPCNT` | bit-population instructions used by binary operations and bitsets (8.5) |
| `SET cc, rd` | set a register to 0 or 1 from a condition (9.7) |
| `DP` | the direct pointer: in user mode it is the thread pointer, in kernel mode the per-CPU pointer; a descendant of the 6502's zero page (9.3, 9.16) |
| `PSW` | the processor status word: flags, mode, priority and the guard bit (6.2) |
| `N Z V C` | the four condition flags: negative, zero, overflow, carry (9.3) |
| **canonical address** | a 64-bit address whose bits above the top bit of the virtual address (bit 38, 47 or 56 with three, four or five levels) equal that bit; any other value faults, so software cannot hide tags in the high bits (6.6) |
| `lowtag` | a type tag stored in the low bits of a pointer, free because objects are quad-aligned (instruction-set text, section 6.2; not otherwise used in this manual) |
| **SWAR** | SIMD within a register: using one wide scalar register as several narrow lanes through masking and carry-aware arithmetic (instruction-set text, glossary; the manual's bit and byte instructions of 9.9 make it effective) |
| `FPSR` | the sticky floating-point exception flag register, read and written by `FSFLAGS` (9.12) |

## C.2 Memory, protection and the kernel

| Term | Meaning |
|---|---|
| `MT` | memory type, a two-bit page attribute: normal cached, normal uncached, device strongly ordered (2.3, 6.7) |
| `ASID` | address space identifier: tags translation-buffer entries so a context switch flushes nothing (6.8) |
| `G` **bit** | global: a translation-buffer entry that ignores the `ASID` (6.8) |
| **TLB** | translation lookaside buffer: the cache of page-table entries; `TLBI` invalidates one, `TLBIA` all, `TLBSYNC` waits for other cores (7.9) |
| `DMW` | direct mapping window: a region translated by a fixed offset with no page table; stored without effect, the format is open (6.9) |
| `MMUCTL` | the register that enables translation and sets the table base (6.5, 6.8) |
| **virtual address, physical address** | the number a program uses, and the number the memory answers to; the page tables translate one into the other (6.6) |
| **page, frame, page table** | a 4 KB piece of virtual space, a 4 KB piece of physical memory, and the tree of tables of 512 entries that says which frame holds which page (6.6) |
| **PTE** | page-table entry: one 64-bit word with the valid, permission, accessed, dirty, global and memory-type bits and the physical page number (6.7) |
| **leaf, pointer** | the two kinds of page-table entry: one that maps a page (`R`, `W` or `X` set) and one that points to the next table (6.7) |
| `KT11`, `PAR`, `PDR` | the PDP-11's memory-management unit, with eight pairs of page address and page descriptor registers for each mode (6.1, 6.6) |
| `EPC`, `EPSW`, `EVA`, `ECAUSE` | the registers that exception entry writes: where to resume, the old status word, the faulting address, the vector number (6.3) |
| `VBR` | the vector base register: where the table of 512 eight-byte entries begins (6.3) |
| `ISYNC` | makes earlier writes to system registers take effect: wait for earlier instructions, discard later ones, fetch again (6.5) |
| `MFSR`, `MTSR` | move from and to a system register, by number (6.5) |
| **software-managed** `A`, `D` | accessed and dirty bits set by the kernel's page-fault handler, never by the walker (6.8) |
| **shootdown** | removing a stale translation from the buffers of other cores; on the pdp64 `TLBI` broadcasts and `TLBSYNC` waits (6.8) |
| **walker** | the hardware that reads page-table entries on a translation-buffer miss (6.6) |
| `STUA` / `CLUA` | set and clear the user-access guard bit, which lets kernel code touch user memory only inside the copy routines (6.10) |
| `PROBER` / `PROBEW` | test whether an address is readable or writable in a given mode without faulting (6.10, 6.13) |
| **PIE** | position-independent executable: loadable at any address; costs nothing here (9.16) |
| **GOT, PLT** | global offset table and procedure linkage table: the indirection that lets position-independent code reach imported symbols (9.16) |
| **ABI** | application binary interface: calling convention, register use, data layout; version 1 is given in appendix D |
| **trap, exception, interrupt** | a deliberate entry (`TRAP #n`), a fault raised by an instruction, and a signal from a device; all enter through the vector table (6.3) |
| **precise fault** | a fault after which every earlier instruction has completed and no later one has changed state (6.4) |
| `SPL` | set the interrupt priority level (6.3) |
| `WFE`, `SEV` | wait for an event and send an event: the idle and wake-up pair (7.10) |
| `FENCE` | orders memory accesses; owed to devices and to other cores (2.8, 7.5) |
| `ATOM`, **monitor** | the atomic operations, built on a reservation monitor as in AXI and ARM, not a locked bus cycle (7.4) |
| **SMP** | symmetric multiprocessing: several identical cores sharing memory (chapter 7) |
| **core class** | the read-only register of each core that gives its class and capacity, so that software can tell an in-order core from an out-of-order one (7.11.1) |
| **secure mode** | the mode of a core in which every timing channel is removed (every instruction and cache fetch takes a time independent of its data), with private cache lines and no periodic tick; on every core or on one secure core, by the rule of 7.11.3 (J.7.8) |
| **coherence, snooping MSI** | the protocol that keeps each core's cache consistent with the others; snooping means each cache watches the bus; MSI is its three states (modified, shared, invalid) (7.6) |
| **sequential consistency** | the memory model in which all cores see all accesses in one order; the pdp64 declares it (7.5) |

## C.3 The bus, devices and boot

| Term | Meaning |
|---|---|
| **Unibus, Q-bus** | the PDP-11's buses; PCI Express keeps their programming model, with devices in the address space (2.3) |

| Term | Meaning |
|---|---|
| **DATIP / DATO** | Unibus and Q-bus transaction types forming a locked read-modify-write (2.10) |
| **window** | a region of the physical address space that a device's base-address register describes, a whole number of pages, found by enumeration (2.4, 5.3) |
| **root complex** | the root of the PCI Express bus: it enumerates the functions, assigns their windows, and reports arrivals and departures (2.4) |
| **bridge** | a function that is itself a bus: it presents the functions behind it, so arbitrary buses (PCI Express, USB) can be connected to the bus and enumerated to any depth (2.4.3) |
| **MMIO** | memory-mapped input/output: device registers reached by ordinary loads and stores (5.3) |
| **DMA** | direct memory access: a device moving data to or from memory without the processor (5.10) |
| **virtqueue, descriptor, available ring, used ring, doorbell** | the common device protocol: a table of 16-byte descriptors in memory, a ring in which the driver offers chains, a ring in which the device returns them, and a write that tells the device to look (5.4) |
| **device_status** | the common-configuration field through which the driver and the device handshake; writing zero resets the device (5.4, 5.5) |
| **virtio** | a standard family of devices of the same queue-and-descriptor shape, the pdp64's device interface (5.4) |
| **virtio-blk** | the standard disk: a virtqueue of read, write and flush requests (5.6.1, 5.6.2) |
| **NVMe, SCSI, ATA, SATA** | storage interfaces, each a command queue in memory and a doorbell, which is the same device pattern (5.4); an NVMe device may be attached as any PCI Express device may, but the standard disk is virtio-blk |
| **driver, strategy routine, device switch, probe, attach** | the kernel code for one kind of device; the routine that queues one block transfer; the table of such routines indexed by the major device number (`bdevsw`, `cdevsw`); the routines that test whether a device is present and set it up (5.1, 5.5) |
| **INIT and READY** | the PDP-11 CSR's reset command and done bit; on the pdp64 the `device_status` handshake (5.5) |
| **softc, `cfattach`** | OpenBSD's per-device structure and the table of a driver's `match` and `attach` functions (5.13) |
| **HDMI transmitter** | the chip that takes a pixel stream and does the link protocol; the pdp64 side is a pixel generator and registers (3.10) |
| **Blit** | Rob Pike's bit-mapped windowing terminal: an 8 MHz Motorola 68000 machine that received downloaded code, characters and editing messages from its host over a serial line and drew every pixel itself (its description as a precursor of the X11 protocol and of NeWS is stated by no source read *(S)*; the lineage the sources confirm runs from its `mux` to 8½ and `rio`); a framebuffer in the address space and a bit-block-transfer operation, done in software, that moves rectangles of pixels; with the C64, one of the two machines the pdp64's terminal is based on (3.1) |
| **blitter, raster operation** | a device that copies and combines rectangles of pixels; the raster operation says how source and destination combine (3.8) |
| **scanout, tearing, double buffering** | the display reading the picture out of memory row by row at a fixed rate; the seam where a picture changed during the reading shows old above and new below; drawing the next frame in a hidden second picture and swapping at the vertical blank (3.15) |
| **glyph map, glyph atlas** | a table of slots each holding the picture of one glyph, filled by a TrueType engine (3.9); the GPU form is one texture holding every glyph the screen uses (3.7) |
| **rio, NeWS** | Plan 9's window system, in which each window is a private screen served as files, a descendant of the Blit's `mux` through 8½; and Sun's window server that programs downloaded from the host could extend (3.18) |
| **mixer, ring buffer, underrun** | the program that sums the sample streams of several programs into one after converting their rates and formats; the circular buffer the sound device reads at the converter's clock; the reader reaching samples not yet written, audio's tear (3.17) |
| **GPU, shader** | a graphics processing unit, thousands of simple lanes; a shader is a small program it runs for every vertex or every pixel (3.6) |
| **bootstrap, boot ROM, block 0** | the program that loads programs; the ROM in the controller chip that reads the first block of the boot device into memory and jumps to it; that first 512-byte block (4.1, 4.4) |
| **BIOS** | the PC's firmware: bring-up and self-test, the first sector, a library of services called by software interrupt, and setup; the pdp64's "BIOS" is its boot ROM and monitor, with no resident services; open source (4.2, 4.3) |

| Term | Meaning |
|---|---|
| **first stage, second stage, `installboot`** | the program in block 0, which carries a list of where the next program lies; the C program that reads the label and the file system and loads `/bsd`; the tool that writes the first stage and its list (4.3, 4.5, 4.6) |
| **partition, MBR, protective MBR** | a named region of a disk; the PC's first sector, with 446 bytes of boot code, a four-entry table and `0xAA55`; the MBR of a GPT disk, with one entry of type `0xEE` covering the disk (4.2, 4.5) |
| **GPT, disk label** | the GUID partition table wrapped around the disk's own label so other systems see the disk is in use; the BSD label that names the root, swap and other partitions inside one partition (4.2, 4.5) |
| **boot-information block** | the structure whose address the second stage passes in `r1`: memory ranges, cores, the root complex, the boot device, the console, the screen, the command line (4.6) |
| **UEFI, PE/COFF, shim** | the PC's later firmware standard, which runs a loader file of the PE/COFF format and offers it boot services; not used by the pdp64; a shim is the small program that would offer those services over virtio to an existing loader (4.2, 4.9) |
| **`ODT`, monitor** | the PDP-11's console debugger; the ROM's console program for examine, deposit, load and start (4.1) |
| **INIT, START-UP, spin table, hart** | x86's signals that start another core; ARM's table the waiting cores poll; RISC-V's word for a core; the pdp64 writes a start register instead (4.7) |
| **APL, K** | array languages whose primitives work on whole arrays; one primitive maps to one descriptor (8.5) |

## C.4 Process and sources

| Term | Meaning |
|---|---|
| **errata** | the numbered decision log of the project (`../errata.md`, appendix F) |
| **fair corpus** | the 827 or 841 C files that every back end compiles, used for all size comparisons (1.7) |
| **`pcc`** | the portable C compiler the project extends to emit pdp64 code (1.7) |
| **`sim64`, QEMU** | the instruction-set simulator and the full-system emulator the tests run on (1.8, 10.5; not otherwise described in this manual) |
| **(D)** | the tag on a statement that the author decided (index) |
| **details pending** | the mark on a decided statement whose details are not yet worked out; it names the open item that discusses them (appendix J), or says that the details are to be determined (index) |
| **deferred** | a matter that the author has decided to put off; it is listed in appendix I and is not open (index) |
| **open item** | a question that the architecture must answer and has not; each has an entry in appendix J with the issue, the options and what each costs, and the body points to it in a few words (index) |
| **proposed** | an idea or design that nobody has decided; it is listed in appendix H and is not part of the architecture (index) |
| **Impact** | the line ending each decision's commentary: effect on the kernel, on the compiler and on the hardware layout (index) |

# Appendix D — The ABI, version 1

**The application binary interface fixes how compiled code, the linker and the operating system agree: the sizes of types, which registers carry arguments and results, who saves what, how the stack looks, and how position-independent code reaches its data.** These are decided choices (section 9.16 states what the instruction set makes cheap). The source is `abi.md`; the compiler, the assembler and the linker follow it, and the version number allows a later change without breaking old binaries; in version 1 `long double` is `double`.

## D.1 The conventions

| | |
|---|---|
| Data model | LP64: `char` 8, `short` 16, `int` 32, `long` 64, `long long` 64, pointers 64. Little-endian. `char` is signed. `long double` is `double`. |
| Alignment | natural; the stack pointer is a multiple of **8** at all times (data alignment is architectural, ISA 7.4: a quad access to an address not a multiple of 8 aborts; the 2-byte parcel alignment is for instructions only). No 16-byte alignment: no type or register is wider than a quad. Open: whether 8 stays (J.D.1). |
| Registers | `R0-R5` arguments, `R0` (and `R1`) results, `R0-R7` and `R12` caller-saved; `R8-R11`, `R15` callee-saved; `R12` linker scratch; `R13` frame pointer (callee-saved, optional); `R14` stack pointer. |
| Floating point | `binary64` in a general register, `binary32` in the low 32 bits sign-extended (ISA 2.3); passed and returned exactly like integers. |
| Arguments | in order in `R0-R5` (a struct of up to 16 bytes takes one or two consecutive, as many as needed); the rest pushed right to left, each in an 8-byte slot (a larger struct in as many slots as it needs, by value). |
| Results | `R0`; `R0:R1` for a 128-bit pair or a struct up to 16 bytes; a larger struct through a hidden pointer passed in `R0` (the arguments then start at `R1`), and returned in `R0`. |
| Calls | `CALL`/`CALLI` push the return address; `RET` pops it. The caller removes stack arguments. |
| Varargs | the callee stores `R0-R5` in a save area directly below the stack arguments; `va_list` is a pointer into the area, then on into the stack arguments. One area suffices because floats use the same registers. |
| Red zone | none (version 1). |
| Symbols | no leading underscore. |
| Landing pads | off in version 1; the assembler provides `.lpad`. Open: when they become mandatory (J.D.2). |
| PIC | globals through `d(PC)`; imported symbols through `MOVQ got(PC),Rd`; calls to imports through `CALLI got(PC)`; TLS local-exec `MOVQ off(DP),Rd`. |

## D.2 Variadic calls

In a call to a function declared with …, every argument goes in an 8-byte slot, in order: slots 0-5 are `r0-r5`, later slots are on the stack. A structure of any size is passed by value in consecutive slots and may split between `r5` and the stack. A variadic callee leaves its named parameters in their slots (`fp+16+8k`) and `va_start` is the address after the last named slot; `va_list` is a `char *` and `va_copy` is assignment. The prologue pops the return address, pushes `r5`..`r0` and pushes the return address again, so the register save area is contiguous with the stack arguments.

## D.3 The hot-register tables

**D.3.1** In a one-parcel form a 3-bit register field is an index into a table of the eight hottest registers, and a 4-bit field is an index into a table of the sixteen hottest. The tables are part of the ABI, because the encoding depends on them and the compiler's register allocator prefers the hot registers. *(D)*

| 3-bit table (index 0 to 7) | 4-bit table (index 0 to 15) |
|---|---|
| `R0 R1 R8 R2 SP R3 R9 R10` | `R0 R1 R8 R2 SP R3 R9 R10 R11 R4 R15 R5 R24 R25 R26 R6` |

**D.3.2** The 3-bit table covers 90.8% of the register uses of the compiled corpus. `MOV2` takes only registers of the 3-bit table. A different allocation changes the sizes and not the correctness; the measured effect of preferring the hot registers in the allocator is at most 0.03%. *(E)*

**D.3.3** A register index beyond a table is an illegal instruction. *(D)*

## D.4 The register-save masks and the registers above R15

**D.4.1** `PUSHM`, `POPM` and `POPMRET` name registers by bit: bit n is `Rn`. The mask is 32 bits, carried in 8, 16, 24 or 32 bits by the length of the form. The 8-bit mask of `POPMRET` is scaled by 256 and names `R8` to `R15`, so a save of `R24` to `R31` is not fused with its return. *(D)* **D.4.2** `R16` to `R31` are ordinary registers to the hardware; `R24`, `R25` and `R26` are among the sixteen hot registers. *(D)*

## D.5 What the ABI leaves open

Open: the stack alignment, 8 or 16 (J.D.1); and mandatory landing pads, lazy against eager binding and whether imports are always called without a PLT stub beyond version 1 (J.D.2).

# Appendix E — The assembly language and the object format

**The assembler's source layout is the V7 assembler's, as; its lexical rules are C's.** This appendix is the syntax of a source file: lines, labels, numbers, strings, operators, directives, registers, the operand modes, the mnemonic conventions and the object format. The grammar itself is `spec/pdp64-asm.hbnf`; the source of this appendix is `asm-syntax.md`. The mnemonics are in the instruction dictionary.

The grammar is `spec/pdp64-asm.hbnf` (hbnf text rules).

- **Lines.** One statement to a line. `;` starts a comment that runs to the end of the line.
- **Labels.** `name:` defines a label; `1:` defines a numeric label, referenced as `1b` (the nearest one before) or `1f` (the nearest after). `sym = expr` assigns. `.` is the location counter. Names are letters, digits, `_`, `.` and `$` and any non-ASCII code point (`café`, `ルート`); a name may start with a digit as long as the whole bareword is not a number (`3dfx:` is a label; `3`, `0x3d`, `0b101`, `0o17` are numbers; `1f` and `1b` are local references), and a bareword of digits only is a numeric local label. Names are case-sensitive and never truncated. ELF symbol names are any bytes, so a name with other characters is written in double quotes (`"odd name!"`). A mnemonic or register name cannot be a label, and neither can `r` followed only by digits: `r16` is an error, not a name (`r2d2` is a name).
- **UTF-8.** A source file is UTF-8, and the parser refuses anything that is not (a stray byte, an overlong form, a surrogate, a truncated sequence), in names, strings, characters and comments alike; a NUL byte is refused too. In a string, UTF-8 text is copied as bytes and `\u00e9`, `\U0001F600` are written as UTF-8; `'é'` is the code point 0xE9, as is `'\u00e9'`.
- **Numbers and characters** are C's: decimal by default (`010` is ten), `0x`, `0b` or `0o` when asked for; `'a'`, `'\n'`, `'\x41'`, `'\101'`.
- **Strings** are `"text\n"` with C's escapes (`\n \t \r \a \b \f \v \0 \\ \' \" \xHH \ooo`), in `.byte` and `.ascii`.
- **Operators** are C's: `* / % & << >>` and `+ - | ^`, unary `- ~ !`, and parentheses. `/` is division. `*`, `/`, `%`, `&` and the shifts need constants; `+` and `-` accept one relocatable symbol, and the difference of two symbols of one section is a constant.
- **Directives.** `.text .data .bss .globl .comm .byte .word .long .quad .ascii .even .align .space .lpad .pool`. `.word` is 16 bits, `.long` 32, `.quad` 64 (the ISA's PDP-11 terminology). `.align n` aligns to n bytes (a power of two).
- **Registers.** `r0`..`r31`; `sp` is `r14`, `fp` is `r13`. `pc` and `dp` appear only as a base: `d(pc)`, `d(dp)`.
- **Operand modes** (ISA section 5): `rN`; `#expr`; `@expr`; `expr(rN)` and `(rN)`; `expr(rN)(rX*s)` with `s` 1, 2, 4 or 8; `expr(pc)`; `expr(dp)`; `(rN)+` and `(rN)+d`; `-(rN)` and `(rN)-d`; a bare expression is a PC-relative reference.
- **Mnemonics** are lowercase (`movq`, `add`, `bne`). A multi-register instruction writes its destination last: `add src, rd`, `xnorpop ra, rb, rd`. `cmp` is the exception, `cmp rd, src`, because the encoding computes `rd - src`.
- **Compare and branch.** `cbCC rd, #imm, label` compares `rd` with an immediate and branches on the condition: `cbeq r3, #5, label`; `cbCC ra, rb, label` compares two registers, and `cbeq` and `cbne` against zero have one-parcel forms. The mnemonics are `cbne cbeq cbpl cbmi cbvc cbvs cbcc cbcs cbge cblt cbgt cble cbhi cblos`. The assembler chooses among the one-, two- and three-parcel forms by the size of the immediate and the distance (the widths are in 9.10.4); when no form reaches, a label in another object or an operand pair that no form holds, it writes `cmp` followed by `bCC`, which writes the flags. `rori32 rs, #n, rd` rotates the low 32 bits right and sign-extends the result.
- **Literal pool: `=expr`.** An operand written with a leading `=` is syntactic sugar for a pooled constant and has no other meaning: `movq =0x123456789ABC, r1` puts the 64-bit value in the literal pool and loads it with one PC-relative `movq`. It is accepted where a quad memory source is (`movq`, `push`, the ALU group with the register first for `cmp`: `cmp rd, =K`, and the floating-point ALU ops), and refused for sized moves, shifts, stores, `lea` and a relocatable symbol. `=` always pools, even a small value; use `#imm` for a constant that fits the signed 32-bit field. A bare `#K` that does not fit that field is an error ("use =K to load it from the literal pool"): the assembler never pools silently. Entries are 8 bytes, 8-byte aligned, one per distinct value per pool, in order of first use, emitted into `.text` at the end of the section or at the directive `.pool` (flush now; code must jump over a mid-section pool). The `=` marks the sugar on the page, which is why it is allowed.
- **Length.** The assembler chooses the shortest form: the first form in the table's order, shorter first and then the order of the catalogue in the grammar, whose fields can hold the operands. Lengths only grow from one pass to the next,

and every distance is measured on the layout of the previous pass. The one-parcel forms have no mnemonics of their own, so `movq (r3), r1` assembles to the one-parcel form when it fits and to a longer form when it does not. An unresolved symbol needs a form with a field of 32 bits or more and a relocation; a `movq #imm, rd` whose constant fits no `movq` form is assembled as `movql`.

- **Invocation.** `as64 [--no-relax] [--stats] [--dump-lens file] [-o obj] file.s`; `ld64 [-M] [-o out] [-e entry] [-Ttext addr] [--no-relax] [--stats] [--dump-map file] objects...`; `sim64 [options] program` runs the executable. The object and the executable record the instruction set in the low four bits of `e_flags` (the value 2), and `ld64` and `sim64` refuse a file that names another. There is one target, so there is no option that selects an encoding; `as64` and `sim64` accept `-m pdp64` and nothing else after `-m`, for the driver convention of the C compiler (`ccom -m pdp64`). `--stats` prints the instruction counts and bytes, `--dump-lens file` writes the length chosen for every instruction, and `--no-relax` (for `as64` and for `ld64`) gives the plain object and links an object as the assembler laid it out.
- **Words of the pdp64.** The qualified moves `movq.u`, `movq.d` and the others of 9.6.7; `trapcc.<cc>`, one parcel with no operand; `movql #imm64, rd`; the fused `movcall rs, rd, label`, `popret rd`, `popmret #mask` and `mov2 ra, rb, rc, rd`, which the assembler never chooses for itself; `pushm`, `popm` and `popmret` take a mask. `(r)+` and `-(r)` without a stride write stride 0, which means the size of the access, and `(r)+8` writes 8. `extb`, `extzb` and `insb` have a 6-bit position and width, so a width of 64 is refused. The 22 meaningless shapes the first assembler accepted (`clr #imm`, `jmp #imm`, a store to an immediate, `lea #imm` and the others) are refused.
- **Relaxation objects.** By default the assembler marks an object (`e_flags` 0x10, `EF_PDP64_RELAX`) and gives every instruction that holds the address of a symbol a relocation, local symbols included, and every padding of `.text` an `R_PDP64_ALIGN` (type 6; addend = padding bytes << 32 | alignment); the linker then gives each such instruction the shortest form that reaches. An object whose `.text` holds a distance that is not a relocation (a difference of two labels used as a number, a `d(pc)` written as a number) is marked `EF_PDP64_NOADJ_TEXT` (0x20) and its `.text` is left alone. A `cbCC` that the assembler expanded to `cmp` and `bCC` stays expanded. `--no-relax` opts out.
- **Object format.** ELF64, little-endian, `e_machine` 0xF064 (`EM_PDP64`); relocations `R_PDP64_D64`, `R_PDP64_D32`, `R_PDP64_INSN_ABS`, `R_PDP64_INSN_PC`, `R_PDP64_INSN_BR` (`as64/relocs.h`; LLVM: `ELFRelocs/PDP64.def`).

What it leaves out of V7 `as`: octal numbers by default, `<string>` and the `'\e` escape, `\/ \< \>` and `[ ]` in expressions, `.if`, `.stabs`, symbols cut to eight characters.

# Appendix F — The evidence behind the manual

**The manual states decisions; these are the documents that hold the evidence, the surveys and the long arguments they were drawn from.** Each is a document of the repository, not a chapter of this manual, and each says in its own opening what it is based on and what it did not check. They are listed with what they hold and their status; the *Why it is here* paragraphs of the instruction entries and the *Impact* lines cite them.

## F.1 The architecture, in full

| Document | What it holds |
|---|---|
| pdp64-isa.md | the full instruction-set text with every encoding, the rejected features, the idioms lost and restored, the comparison with the PDP-11 and the VAX |
| pdp64-system-architecture.md | privilege, exceptions, interrupts, memory management and boot as an operating system sees them, with what is undefined stated plainly |
| pdp64-system-design.md | System v1: the options, how others do it, and how the choices serve the project's goals |
| boot.md | boot and bring-up: how the main systems start themselves, whether UEFI is worth having, and the proposal |
| pdp64-in-order-and-out-of-order.md and pdp64-ooo-readiness.md | what the in-order design point costs and buys, and what the architecture must decide to stay ready for an out-of-order core |
| pdp64-fpga-plan.md and pdp64-lb64-accelerator.md | the FPGA build plan and the inference accelerator |
| abi.md and asm-syntax.md | the ABI and the assembly language (appendices D and E are drawn from them) |
| roadmap.md | the order of work from the present toolchain to an OpenBSD that can be logged into |

## F.2 The record of decisions

| Document | What it holds |
|---|---|
| errata.md | every decision, numbered, with the reasons |
| decisions-pending.md | every open decision, with the need, five options, the survey of what other systems do and what the PDP-11 and VAX did (appendix B is drawn from it) |
| fact-check-2026-10-22.md | the comparison of statements about the outside world with a source where one could be read; the claims it could not check are marked "not confirmed" in the manual |

## F.3 Measurements and surveys

| Document | What it holds |
|---|---|
| codegen-analysis.md | what compiled C says about the instruction set: instruction use, operand forms, the instructions never emitted |
| kernel-asm-survey.md | what Linux, FreeBSD, NetBSD and OpenBSD write in assembly, and what pdp64 can learn from it |
| kernel-pain-points.md | what hurts kernel developers: tight loops, pain points and workarounds in the current kernels, from the source trees and the mailing lists, compared with pdp64 |
| hardware-vulnerabilities.md | the processor, memory and system flaws of the last decade that needed mitigations, and the rules that keep pdp64 from repeating them, including the evidence on constant-time modes |
| openbsd-mitigations.md | OpenBSD's memory-safety and exploit mitigations, what pdp64 gives for free and what would make the rest soft and easy |
| sim64.md | the reference emulator |
| smp-decisions.md | the open symmetric-multiprocessing decisions, proposals awaiting the author's annotations (tag P) |
| crypto-secure-core.md, security-proposals.md, study-proposals.md, retire-stage-candidates.md | the proposals on the secure core, the security mechanisms, the studied instructions and the retire-stage mechanisms, with the author's replies in errata 84 to 86 |

| Document | What it holds |
|---|---|
| priority-lists.md | the lists of priorities that errata 83 settled |

The data behind the tables is in docs/codegen-data/, the generators are in tools/, and the sources of the design discussions are chat01.txt and the archive of the working session.

## F.4 The encoding: the measurements

**F.4.1** The encoding, the compiler, the assembler, the linker, the simulator and the timing model were built and measured together; the sources of the numbers below are encoding-proposal.md (the study, sections 6 and 12) and encoding-work/integration.md (section 4 is the final table, made on the integrated tree), with the reports of each phase in encoding-work/. The baseline is the first encoding (the earlier iteration of 1.8) with the compiler as it was before the work. There is no baseline of the first encoding with the new compiler: several of the compiler's passes use instructions only the prefix-coded encodings have. Relaxation is on by default; the model of timing is an in-order toy with ideal memory (128 exact checks of its own pipeline), not hardware.

**F.4.2** The static size on the 841-file corpus (instruction bytes from as64 --stats):

| | first encoding, old compiler | pdp64, new compiler |
|---|---|---|
| instruction bytes | 1,083,870 | 964,158 (-11.04%) |
| without memcpy-a2.c (27% of the bytes) | 785,836 | 706,042 (-10.15%) |
| instructions | 338,313 | 296,070 |
| fused pairs (MOVCALL, POPMRET, POPRET, MOV2) | 0 | 11,531, 1,608, 2,586, 4,061 = 19,786 |
| CBCC replaced by CMP and BCC | 0 | 13 |
| text of the objects (840 files, before linking) | 1,305,878 | 1,197,018 |
| the same linked with relaxation | not applicable | 1,152,010 (-3.76% against unrelaxed) |

Relaxation shortens the text of the objects by 45,008 bytes. The corpus stub places the symbols right behind each file, so the saving is an upper bound for the reach of the 4-byte call.

**F.4.3** Six large programs (lz4, zlib, Lua and three BearSSL kernels, linked with a stand-in C library, run on the simulator; all print exactly the host's output on all three encodings), totals:

| | first encoding, old compiler | pdp64, new compiler |
|---|---|---|
| instructions executed | 755,816,022 | 572,801,956 (-24.21%) |
| bytes fetched | 2,365,226,284 | 1,680,662,586 (-28.94%) |
| bytes per instruction | 3.129 | 2.934 |
| linked text (relaxed) | 226,606 | 203,404 (-10.24%) |
| fused instructions executed | 0 | 10,381,414 |
| frame loads and stores executed | 68,936,548 | 24,276,195 (-64.78%) |

Linking the same objects with --no-relax instead of the default makes the linked text 1.65% larger and changes the bytes executed by 0.4%.

**F.4.4** The timing model (in order, ideal memory, no branch predictor; issue width 1 or 2, fetch window W bytes a cycle; the prefix-coded encodings pay one cycle of length-decode latency on a taken branch and a refill), all six programs summed:

| model | first encoding, old compiler | pdp64, new compiler |
|---|---|---|
| issue 1, W = 4: cycles | 1,133,787,653 | 842,065,005 (-25.73%) |
| issue 2, W = 4: cycles | 1,001,691,461 | 735,251,238 (-26.60%) |
| issue 1, W = 2: cycles | 1,305,661,982 | 1,002,176,356 (-23.24%) |
| issue 1, W = 4: fetch-wait, execute-wait (share of cycles) | 1.7%, 31.6% | 7.2%, 24.8% |

At four bytes a cycle and above the programs are bound by execution and not by fetch (widening the fetch from 4 to 16 bytes a cycle takes 1.0% off the cycles at issue width 1); the saving in cycles is the saving in instructions, and the saving in bytes buys nothing at those widths. By program (issue 1, W = 4, against the first encoding): lz4 -7.9%, zlib -18.2%, Lua -49.1%, bs_hash -2.4%, bs_sym -8.5%, bs_ec -24.9%. The gain on Lua is the switch dispatch of the new compiler, not the encoding.

**F.4.5** The predictions of the study against the final measurement: the model of the study put the corpus at 1,040,280 bytes and, with the four fused pairs, 1,007,172; the measurement is 964,158, for the same reasons. The study put relaxation at 9% of the corpus text and a first measurement at 3.5%; the final measurement is 3.76%.

**F.4.6** What these measurements do not establish: the timing is one sweep of an in-order model with ideal memory; the cache variants were not re-run on the final compiler; the 3-parcel `CBCC` and the other form choices were trained on the corpus and not on the six programs, so a profile-tuned variant was not tried; one corpus file does not link in the stub measurement (a 64-bit constant used as a displacement); the compiler does not use `MOVUW`, `MOVUL`, `MOVUQ` (the unaligned-access idiom is 10% of the instructions of lz4), `MULH`-based division by 64-bit constants, loop-invariant motion or live-range splitting; QEMU does not yet execute this encoding; the atomics, a number of cryptographic words, `MVAL`, `UTF8DEC` and `UTF8ENC` are not implemented in the reference executor.

# Appendix G — The evolution of the design

**The design began as a small question and grew by steps.** In the author's words of 2026-11-10: "we started off as a pdp16 modernized a tiny bit, then got more and more ambitious", and "the initial idea was, could a pdp16 run effectively today with a blit terminal and off-loaded 8-register nic", with "the blit being a mashup with the c64 for audio". This appendix lists, in point form, how it grew and which influences, inspirations and precedents entered, each with the date and time (UTC) of the message in which it first appeared.

## G.1 How it grew

- 2026-08-23 05:50 — the founding question, in his first message about it: an Ethernet card "that held the entire tcp ip udp icmp stack inside itself ... so that a pdp11 could plug it in and use it", and whether the PDP-11 could use the Blit terminal (R376)
- 2026-08-23 05:58 — wider addresses: "was there a 22 bit pdp11?" (R377); at 06:19: "What if all addresses were expanded to 22 bit? ... an update to the pdp11 arch" (R378)
- 2026-08-23 06:21 — "Is there pdp11 fpga?" (R379); at 06:46: "Could I have dozens of pdp11 on a postage stamp? How about the blit?" with 24-bit colour and 1024x768 (R380)
- 2026-08-23 19:54 — "combine 8 pdp11 systems plus one blit system and a smart nic ethernet card all together on one silicon die" (R381)
- 2026-08-28 19:31 — "if one implemented pdp11 without the peripherals, what is involved?" (R382)
- 2026-08-29 00:29 — the first design brief: "assume mmu and fp units"; in the same message the C64 graphics and audio, a modern USB keyboard and mouse and an NVMe disk are asked for (R20)
- 2026-08-29 00:29 — the Blit as the terminal, mashed up with the C64: "could we revamp the blit to be c64 adjacent", since "c64 is superior to blit terminal in many ways" (R20)
- 2026-08-29 00:49 — the 8502 chosen over the plain C64 as the partner: "they seem much closer to each other"; the same message fixes little-endian and a 200 to 300 MHz FPGA clock (R22)
- 2026-08-30 09:13 — the 32-bit step skipped: "skip 32 bit version and go straight to 64bit" (R29)
- 2026-08-30 09:15 — and the widening stated: "basically upgrade pdp-11 straight to 64 bit" (R30)
- 2026-08-30 09:30 — a constraint on the widening: "only 2 byte memory alignment; don't put 8 byte memory alignment constraints" (R32)
- 2026-10-20 19:59 — devices become standard hardware on the bus: "we can put standard gpu on the bus too?" (R135)
- 2026-10-20 22:44 — multiprocessing enters: what the design needs "to allow smp, symmetric mutiprocessing" (R137)
- 2026-10-22 16:16 — the ambition of a large machine: "ready for the out of order large scale design" (R220)
- 2026-10-22 20:35 — the early rule of no mode bits becomes "a very soft rule" (R241)
- 2026-10-22 20:48 — the priorities written down: "code size, easy recoverability, and speed/efficiency through fewer gates and tighter hot loops" (R243)
- 2026-10-22 23:01 — security as a design goal: "there should be constant time flag ... so instructions always take an equal amount of time" (R253)
- 2026-10-23 00:49 — the terminal takes the C64's audio: a Blitter terminal "based on a modernized and updated c64 with sid chip and also modern 48 bit audio and 48 bit pixels" (R278)
- 2026-10-23 00:53 — the VAX idea of a device space in the address map is added to the book (R281)
- 2026-11-04 00:07 — the bus becomes PCI Express with NVMe and USB, with the security fixes of the pain-points document
- 2026-11-04 00:57 — cores dedicated to devices: "also one core for audio processing, doing midi, dsp, etc." (R313)
- 2026-11-04 08:07 — a new instruction encoding: "an encoding scheme that minimizes code size and fits them all in with 32 reg and with 16 reg" (R328)

- 2026-11-04 09:57 — "the pdp64 now has children, the pdp64/32 and pdp64/16" (R331)
- 2026-11-04 19:21 — the first device: "4 cores is enough hopefully that fits on the fpga" (R338)
- 2026-11-08 02:37 — the link between Commodore and DEC found: "I found the evidence, there IS a link between C64 and DEC!" (R383). The source, a Commodore history of the industrial designer Ira Velinsky, shows one designer who worked for both companies (appendix H, 1.1.5). The author's own description, 2026-11-10 06:47: "the smoking gun that there was indeed intercourse between dec and commodore" (R393)
- 2026-11-09 07:02 — virtio across the board for the devices, approved for the bus chapter (R360)
- 2026-11-10 05:30 — "pdp64/32 is the decided upon pdp64"; r20 and pdp64/16 are no longer part of it (R370)
- 2026-11-10 — "r20 and 64/16 are gone as cpus with manuals etc; they are previous iterations that were superseded by 64/32 which is now the official pdp64"; one manual of the pdp64, and the two iterations are history (errata 122)

## G.2 The influences, inspirations and precedents, in the order they were introduced

- Each line: date and time of the first message that named it · the referent · who named it · its stance when introduced (**chosen**: the choice follows it; **rejected**: turned down; **considered**: compared or cited, not decided) · the part of the design it bears on · the source in parentheses (archive thread and message number, or `j` for a message found only in the session files).
- "Later" gives the first date and time at which its stance changed, where it did.
- Only what the messages say: a referent proposed by the assistant is marked so, and counts as an influence only where its line says it was accepted.
- 2025-07-23 22:43 · MS-DOS console font · named by the author · considered — C64 console (c64x), the earlier SDL console: font "reminiscent of the old Commodore 64 font, or the old MSDOS console font" (R1)
- 2026-06-18 00:31 · kitty · named by the author · considered — C64 console font and 9p interface: c64x is much the same "but with peek/poke instead of byte stream" (R8)
- 2026-06-18 00:31 · xterm-256color subset shim · named by the author · chosen — C64 console font and 9p interface: "we'll have a shim that implements xterm-256color subset" (R8)
- 2026-06-18 00:48 · 9P · named by the author · chosen — C64 console font and 9p interface: "9p all the way"; also a 9p server for window info [R8] (R9)
- 2026-06-18 00:48 · vsock · proposed by the assistant, not taken up · considered — C64 console font and 9p interface: he "didn't understand" the vsock question (R9)
- 2026-06-18 02:54 · Commodore 64 · named by the author · chosen — C64 console (c64x), the earlier SDL console: "as sweet and usable as the C64" (R11); later rejected 2026-06-25 05:41
- 2026-06-18 02:54 · newLISP · named by the author · chosen — C64 console (c64x), the earlier SDL console: as driver of the console (R11)
- 2026-06-25 05:41 · RPi 500 keyboard · named by the author · chosen — C64 in spirit, not the hardware: "adequate" (R15)
- 2026-06-25 05:57 · original Lisp · named by the author · chosen — newlisp environment: bare function name at beginning of a line (R16)
- 2026-06-25 05:57 · Tcl · named by the author · chosen — newlisp environment: some idioms incorporated (R16)
- 2026-06-25 05:57 · Smalltalk · named by the author · chosen — newlisp environment: idioms and code browser, but showing all contexts (R16)
- 2026-06-25 05:57 · vim · named by the author · chosen — newlisp environment: "incorporate some of vim" into the environment (R16)
- 2026-08-23 05:51 · Blit · named by the author · considered — asked how much memory it needs and whether the PDP-11 could use it (R376); later chosen 2026-10-23 18:57
- 2026-08-23 06:10 · Plan 9 · named by the author · considered — asked how Plan 9 did demand paging (R384)
- 2026-08-23 06:28 · VAX ISA and DEC Alpha · named by the author · considered — asked how the VAX instruction set compares with the Alpha and the PDP-11 (R385)
- 2026-08-23 06:36 · DEUNA and DELUA network cards · named by the author · considered — asked about their history and how much of the sockets API and TCP stack they provided (R386)

- 2026-08-23 06:54 · SIMH · named by the author · considered — asked what it supports (R387); later chosen 2026-10-13 17:05
- 2026-08-23 06:55 · Qbus and Unibus against PCI and SCSI · named by the author · considered — asked how they differ from modern buses (R388)
- 2026-08-23 19:45 · Linux and Plan 9 virtual memory · named by the author · considered — asked which is better for the 32V virtual-memory retrofit (R389)
- 2026-08-29 00:28 · PDP-11 · named by the author · chosen — Mashup of PDP-11 with C64 / M8502: “use pdp-11 as the base” (R19); later rejected 2026-08-29 00:49, considered 2026-08-30 08:56
- 2026-08-29 00:29 · SID · named by the author · chosen — FPGA target and clock: fold in the good parts of c64 (R20); later considered 2026-10-24 23:11
- 2026-08-29 00:29 · USB · named by the author · chosen — Modern I/O hardware (USB, NVMe): modern usb keyboard, mouse and hard drive (R20)
- 2026-08-29 00:29 · NVMe · named by the author · chosen — Modern I/O hardware (USB, NVMe): “nvme hard drive” (R20)
- 2026-08-29 00:29 · Raspberry Pi SD-card storage · named by the author · considered — Modern I/O hardware (USB, NVMe): asked how those little SanDisk drives interface with rpi (R20)
- 2026-08-29 00:40 · QNX/QUNIX · named by the author · considered — Mashup of PDP-11 with C64 / M8502: fold design decisions into V7 for realtime guarantees (R21)
- 2026-08-29 00:40 · Commodore 128 · named by the author · considered — Blit and C64 as the terminal model: asked what advances it had over the c64 (R21); later chosen 2026-11-04 00:58
- 2026-08-29 00:40 · V7 Unix · named by the author · chosen — Realtime V7 with QNX ideas: “assume v7 source code as the base” (R21); later considered 2026-10-22 07:46
- 2026-08-29 00:49 · middle-endian longword order · named by the author · rejected — Endianness: “none of this middle endian stuff, that gets fixed right now” (R22)
- 2026-08-29 00:49 · 6502 · named by the author · considered — Endianness: 8-bit, so word order did not matter there (R22)
- 2026-08-29 00:49 · IEEE 754 floating point · named by the author · chosen — Floating-point format: new FP unit compatible with the modern representation (R22)
- 2026-08-29 00:49 · m8502 · named by the author · chosen — FPGA target and clock: mash it up with the PDP-11 instead of the c64, they seem closer (R22)
- 2026-08-29 01:38 · 8502/6502 compatibility in FPGA · named by the author · considered — Spec revisions and write-up: all registers readable by holding the rdy pin (R24)
- 2026-08-29 01:38 · 8502 RDY pin hold · named by the author · chosen — C64 island: 8502 registers readable from outside: freeze the CPU to read and write registers (R24)
- 2026-08-29 07:52 · HDMI · named by the author · chosen — Video output (HDMI): drive it however we want (R25)
- 2026-08-29 07:52 · PAL/NTSC · named by the author · considered — Video output (HDMI): 50 fps only for legacy code, backwards compat (R25)
- 2026-08-30 08:23 · x86 · named by the author · chosen — Backward compatibility: adopt what is needed from it to keep transistor count down (R26)
- 2026-08-30 08:23 · Strecker VAX-11/780 paper · proposed by the assistant, not taken up · considered — Backward compatibility: DEC considered extending PDP-11; opcode-space exhaustion was the reason (R26)
- 2026-08-30 08:23 · M8502 and x86 · named by the author · chosen — MMU and FP units included: adopt what is needed from them, low transistor count (R26)
- 2026-08-30 08:23 · VAX-11 · proposed by the assistant, not taken up · considered — Widening the PDP-11 to 64 bits: widened PDP-11 pleasant to write and target (R26); later chosen 2026-10-20 22:44
- 2026-08-30 09:08 · x86 ah/al · named by the author · considered — Register width: asked for 64-bit registers addressable as two 32-bit halves, sort of like ah/al (R28)
- 2026-08-30 09:13 · UTF-8 · named by the author · chosen — Word size: showed how the length-first encoding can be done (R29)

- 2026-08-30 09:13 · ARM32 LDRD / MIPS32 FP pairs · proposed by the assistant, not taken up · considered — Word size: register-pair cost of 32-bit FP, in the case against a 32-bit version (R29)
- 2026-08-30 09:30 · AArch64 · proposed by the assistant, not taken up · considered — Spec revisions and write-up: holds both rules at once; cited for partial writes (R32)
- 2026-08-30 09:50 · Lisp machine · named by the author · chosen — Tagged pointers: tagged pointers; hardware tag checking on dereference not added (R33)
- 2026-10-12 13:17 · dmr cc · named by the author · chosen — dmr optimizations: goal and scope: make the modern pcc pdp11 backend emit identical code, including optimizer (R35); later considered 2026-10-13 18:37
- 2026-10-12 13:42 · modern pcc -xssa · named by the author · chosen — dmr optimizations: goal and scope: run in parallel with dmr optimizations (R36)
- 2026-10-12 13:42 · modern pcc SSA optimizations · named by the author · considered — SSA and pcc optimizations alongside dmr: allowed alongside dmr; dropped meanwhile for getnum bug (R36)
- 2026-10-12 13:46 · dmr optimization pass · named by the author · chosen — SSA and pcc optimizations alongside dmr: other optimizations should run after it (R37)
- 2026-10-13 15:50 · VAX and x86 platforms · named by the author · chosen — dmr optimizations on other targets: final step after the pdp-11 backend is complete (R41)
- 2026-10-13 17:22 · pcc · named by the author · chosen — Upstream PortableCC: keep tree up to date with it; keep upstream's fix (R47); later considered 2026-10-14 00:43
- 2026-10-13 17:54 · VAX backend · named by the author · chosen — mips backend: prioritised over mips (R49)
- 2026-10-13 17:54 · nova second pass in pcc · named by the author · considered — mips backend: if pcc ships it, bring the dmr c2 pass in too (R49)
- 2026-10-13 18:10 · V7 disk image · named by the author · chosen — simh testing: used for the simh tests (R50)
- 2026-10-13 18:36 · 32V assembler · named by the author · chosen — VAX output for the 32V assembler: flag so the vax backend emits what 32v as accepts (R51)
- 2026-10-13 19:46 · 32V cc · named by the author · chosen — dmr optimizations on other targets: original cc, checked for optimizations beyond dmr package (R55)
- 2026-10-13 19:57 · dmr and VAX tricks · named by the author · chosen — x86 backend changes: document how they worked and bring them to x86 and other backends (R57)
- 2026-10-13 19:57 · VAX pcc as good as dmr on PDP-11 · named by the author · considered — x86 backend changes: remarked it sounds like VAX pcc was about as good (R57)
- 2026-10-13 19:57 · VAX pcc / 32V · named by the author · considered — Optimization tricks document and optimization docs: "about as good as dmr was on pdp11" (R57)
- 2026-10-13 19:57 · x86, then VAX · named by the author · chosen — Optimization tricks document and optimization docs: "do the x86 ones first, then vax" (R57)
- 2026-10-17 09:28 · optim2.c · named by the author · chosen — Optimizer placement (optim2.c): put whatever possible there (R66)
- 2026-10-17 10:25 · original Unix code (dmr) · named by the author · considered — Documenting bugs and defects: "note any dmr errors in the docs" (R68)
- 2026-10-17 10:59 · pdp11/vax/x86 backends only · named by the author · chosen — Optimization tricks document and optimization docs: other backends left alone for now (R69)
- 2026-10-17 21:39 · V7 and 32V old K&R forms · named by the author · considered — Old forms and long long libc documentation: forms pcc doesn't support; cost of supporting them (R76)
- 2026-10-17 21:39 · C99 · named by the author · considered — Old forms and long long libc documentation: regressions from C99 if old forms were supported (R76)
- 2026-10-18 06:30 · V7/32V libc side · named by the author · rejected — Long long support and libc helpers: asked to do longlong helpers on pcc side instead (R81)
- 2026-10-18 07:33 · PDP-11 and VAX-11 · named by the author · considered — Documenting bugs and defects: asks whether pre-existing bugs are only pdp11 or vax too (R84); later chosen 2026-10-22 06:52
- 2026-10-18 16:03 · clang -Os · named by the author · considered — aarch64 against clang -Os: asked if -xdmr beats clang -Os on aarch64 (R89); later chosen 2026-10-22 07:18

- 2026-10-18 16:36 · Lua and SQLite · named by the author · chosen — Benchmark report and corpus: fetch them as test corpora, use for each arch (R93)
- 2026-10-18 16:57 · clang · named by the author · considered — Benchmark report and corpus: asked which backends pcc and clang share; test all (R94)
- 2026-10-18 18:03 · yacc · named by the author · chosen — as32v in the v7unix-toolchain repo and tool naming: use it for the old-yacc grammar (R103)
- 2026-10-18 18:06 · v7as / v7cc prefix naming · named by the author · rejected — as32v in the v7unix-toolchain repo and tool naming: change policy to asv7, ccv7, cppv7 suffixes (R104)
- 2026-10-18 22:43 · 32V pcc · named by the author · considered — Benchmark and size reports: name to use for the 32V cc (R111)
- 2026-10-18 23:52 · modern C inline semantics · named by the author · considered — Benchmark freeze and fixes against stock: inline is a hint, not a command (R113)
- 2026-10-19 01:56 · GCC 4.7 or older (plain C) · proposed by the assistant, not taken up · considered — gcc cross-build and gcc comparison: would avoid C++ host build; judged not worth the trouble (R116)
- 2026-10-19 04:13 · upstream git history · named by the author · chosen — Upstream PortableCC: source for the pattern of what ragge accepts (R118)
- 2026-10-19 04:41 · PR #47 aarch64 backend · named by the author · chosen — aarch64 PR 47 reconciliation: compare ours with it and fold in anything good (R119)
- 2026-10-19 07:16 · Debian multiarch include path · proposed by the assistant, accepted · chosen — aarch64 PR 47 reconciliation: /usr/include/aarch64-linux-gnu (R123)
- 2026-10-19 07:16 · Apple Mach-O variadic convention · proposed by the assistant, accepted · chosen — aarch64 PR 47 reconciliation: fix variadic calls for Mach-O (R123)
- 2026-10-19 07:57 · libgcc __addtf3 family · proposed by the assistant, accepted · chosen — aarch64 binary128 support: stage 3: arithmetic and conversions through library calls (R124)
- 2026-10-19 09:22 · GNU libgcc runtime · proposed by the assistant, not taken up · considered — aarch64 long double default (A64QUAD): option 1; depends on GCC being installed (R126)
- 2026-10-19 09:22 · own runtime tools/a64quad/tf.c · proposed by the assistant, not taken up · considered — aarch64 long double default (A64QUAD): option 2; assistant's recommendation, not chosen (R126)
- 2026-10-19 09:27 · gcc cross-compilers for pdp11 and vax · named by the author · chosen — gcc cross-build and gcc comparison: built and measured as comparison reference (R127)
- 2026-10-19 10:33 · gcc -Os · proposed by the assistant, accepted · considered — Benchmark and size reports: size gap explained as GCC deleting undefined-behaviour code (R130); later chosen 2026-10-21 08:03
- 2026-10-19 20:58 · gcc · named by the author · chosen — Fair corpus and compiler comparison: files gcc cannot compile excluded from the fair corpus (R131)
- 2026-10-19 22:05 · sjpcc · named by the author · chosen — Fair corpus and compiler comparison: include in the evaluation on vax and pdp11 (R132)
- 2026-10-20 19:59 · LoongArch64 / AArch64 · named by the author · considered — R7 as a view of the PC: asked how they handle the PC (R135)
- 2026-10-20 19:59 · LoongArch64 · named by the author · considered — ISA comparison table: asked how they handle the PC (R135)
- 2026-10-20 20:28 · GPU · named by the author · considered — GPU and accelerator meaning, helper chips: meant as "a helper/accelerator that uses the same ram, does dma" (R136)
- 2026-10-20 20:28 · SIMD/AVX · named by the author · considered — GPU and accelerator meaning, helper chips: helps like a GPU's, "the simd/avax stuff" (R136)
- 2026-10-20 20:28 · Samsung LittleBit-2 · named by the author · considered — GPU and accelerator meaning, helper chips: a GPU optimized purely for its encoding (R136)
- 2026-10-20 23:14 · ror over rol · named by the author · chosen — Mnemonic and width spelling: assistant made the case; assembler could map rol to ror (R138)
- 2026-10-21 13:17 · Ockham ("ockam") · named by the author · chosen — Efficiency order for PIE: entity-multiplication rule (R145)

- 2026-10-21 15:20 · asv7 and ldv7 · named by the author · chosen — pdp64 toolchain elements and build plan: port them to assemble and link pdp64 (R148)
- 2026-10-21 15:21 · ELF · named by the author · chosen — pdp64 toolchain elements and build plan: "actually.... elf linker is ok" (R149)
- 2026-10-21 15:21 · QEMU · named by the author · chosen — QEMU and LLVM targets: wants a plan for a qemu backend for the pdp64 (R151)
- 2026-10-21 15:29 · V7 a.out with 64-bit variant · proposed by the assistant, not taken up · considered — pdp64 toolchain elements and build plan: assistant's alternative to ELF64 output (R152)
- 2026-10-21 15:29 · GNU ld / lld · proposed by the assistant, not taken up · considered — pdp64 toolchain elements and build plan: no back end for this ISA; judged more work (R152)
- 2026-10-21 15:29 · ELF64 output (widen ld.c) · proposed by the assistant, accepted · chosen — Spec revisions and write-up: said "yes to all" to the assistant's proposal (R152)
- 2026-10-21 15:29 · V7 a.out 64-bit variant · proposed by the assistant, not taken up · considered — Spec revisions and write-up: offered as the smaller change (R152)
- 2026-10-21 15:45 · PDP-11 and x86 signed char · proposed by the assistant, accepted · chosen — pdp64 ABI and long double: signed char matches their tradition (R154)
- 2026-10-21 15:45 · modern ELF symbol names (no underscore) · proposed by the assistant, accepted · chosen — pdp64 ABI and long double: instead of V7 leading underscore (R154)
- 2026-10-21 15:45 · software 128-bit long double · proposed by the assistant, not taken up · considered — pdp64 ABI and long double: alternative to same-as-double (R154)
- 2026-10-21 20:37 · asv7 (V7 as) · named by the author · rejected — Assembler syntax and names: not to copy its warts or anachronisms (R157)
- 2026-10-21 20:38 · V7 octal-default numbers · named by the author · rejected — Assembler syntax and names: numbers default to decimal (R158)
- 2026-10-21 20:46 · C syntax for numbers and strings · named by the author · chosen — Assembler syntax and names: instead of V7 <...> strings and comment slash (R159)
- 2026-10-21 20:48 · modern ELF symbol names · named by the author · chosen — Assembler syntax and names: names and labels follow it (R160)
- 2026-10-21 22:09 · PowerPC 601 · named by the author · considered — Opcode usage, instruction count and pruning: cited as the measure pdp64 already beats; against cutting to about 70 (R172); later chosen 2026-10-23 00:15
- 2026-10-21 22:13 · C round() / libm · proposed by the assistant, accepted · chosen — FP rounding variants: ties-away-from-zero rounding added as a qualifier (R173)
- 2026-10-21 22:16 · carry-less multiply · named by the author · chosen — CRC instructions: replaces both CRC variants (R175)
- 2026-10-21 22:27 · cmov/csel/fsel · named by the author · considered — Integer abs/min/max and conditional select: wants to know more; abs/min/max as assembler aliases on top (R177)
- 2026-10-21 22:36 · ARM csel family · proposed by the assistant, not taken up · considered — Integer abs/min/max and conditional select: select variants nearly free once the first exists (R178)
- 2026-10-22 06:21 · manpage-style SEE ALSO section · named by the author · chosen — Links and navigation between docs: wants "a manpage style section at the bottom" listing repos and docs (R188)
- 2026-10-22 07:46 · LLVM · named by the author · considered — Undefined items: asks how hard one would be (R203); later chosen 2026-10-23 00:04
- 2026-10-22 07:46 · Linux port (GCC or LLVM back end) · proposed by the assistant, not taken up · considered — Undefined items: a BSD or V7-style Unix is closer to the toolchain (R203)
- 2026-10-22 08:09 · gcc -Os x86-64 / clang -Os · proposed by the assistant, not taken up · considered — One-parcel forms: pdp64 averages 3.36 bytes per instruction against 3.66 and 3.78 (R207)
- 2026-10-22 15:37 · OpenBSD · named by the author · considered — QEMU and LLVM targets: "how does openbsd bring up a new architecture?" vs going straight to 3bsd (R216); later chosen 2026-10-22 18:52
- 2026-10-22 15:37 · 3BSD · named by the author · considered — QEMU and LLVM targets: premise of his question: going straight to 3bsd (R216); later rejected 2026-10-22 18:52

- 2026-10-22 15:38 · LLVM target sources under llvm/ in pdp64 repo · proposed by the assistant, not taken up · considered — QEMU and LLVM targets: assistant proposal, not a GitHub fork; he answered only about machine (R217)
- 2026-10-22 15:47 · major operating systems and architectures · named by the author · considered — Comparison with other architectures: asked how the options compared against them (R218)
- 2026-10-22 15:47 · gcc cross-compilers on other arches · named by the author · considered — System doc: asks what cross-compile infrastructure exists (R218)
- 2026-10-22 16:00 · out-of-order design · named by the author · considered — System doc: asks what supporting it needs and what refusing it costs (R219)
- 2026-10-22 16:16 · ARM32 SRS/RFE · proposed by the assistant, not taken up · considered — PUSHX and POPX: same idea; assistant said from memory, unchecked (R220)
- 2026-10-22 16:28 · Linux and BSD kernel assembly · named by the author · considered — Kernel assembly survey: survey for instructions like popx/pushx that compilers never emit (R222); later chosen 2026-10-22 16:28
- 2026-10-22 16:28 · clang backends · named by the author · considered — Kernel assembly survey: compile the corpus with each to find instructions to adopt (R222)
- 2026-10-22 16:28 · each clang backend that we have · named by the author · chosen — Comparison with other architectures: compile the fair corpus with each to find instructions to adopt (R222)
- 2026-10-22 16:29 · other backends with -Os · named by the author · considered — pdp64 measurement against clang and gcc: asked how pdp64 compares with them (R223)
- 2026-10-22 16:37 · VAX svpctx/ldpctx · proposed by the assistant, not taken up · considered — Kernel assembly survey document: hardware context switch sidebar (R225)
- 2026-10-22 18:52 · Unix/32V · named by the author · chosen — First operating system port: fallback "if openbsd proves too hard"; has demand paging (R230)
- 2026-10-22 18:56 · AArch64 ldtr/sttr and m68k moves · proposed by the assistant, not taken up · considered — Decisions document: option B: separate access-as-user load and store forms (R232)
- 2026-10-22 18:56 · x86, ARM and RISC-V kernels (incl. OpenBSD) · proposed by the assistant, not taken up · considered — Decisions document: assistant's reason for recommending option A (R232)
- 2026-10-22 19:03 · unix on PDP-11 and VAX-11 · named by the author · considered — Decisions document: asked to compare decisions with what they used to do, and tradeoffs (R233)
- 2026-10-22 19:22 · VAX/PDP · named by the author · considered — SMP and its decisions document: survey "what all major modern os and cpu do, plus vax/pdp" (R235)
- 2026-10-22 20:07 · TUHS source · named by the author · chosen — Fact-checking: already on the box, no download needed (R237)
- 2026-10-22 20:23 · kernel source code · named by the author · considered — Fact-checking: suggested instead of hard-to-access manuals (R239)
- 2026-10-22 20:35 · PDP-11 SR0 / VAX MAPEN · proposed by the assistant, not taken up · considered — Mode bits and the "no mode bits" rule: translation on/off switch, called the heritage choice by the assistant (R241)
- 2026-10-22 20:56 · Linux and BSD current kernel source · named by the author · chosen — Pain-points research: current version only, not full history (R246)
- 2026-10-22 21:01 · mailing lists · named by the author · chosen — Pain-points research: expected pain points from mailing lists, not only source (R247)
- 2026-10-22 21:11 · lwn.net · named by the author · rejected — Browser use, politeness and site bans: avoided after it triggered spammer code; later ban removed [R290] (R250); later considered 2026-10-22 21:12
- 2026-10-22 23:01 · Spectre · named by the author · rejected — Constant-time execution: design must not repeat it (R253)
- 2026-10-22 23:01 · Rowhammer · named by the author · rejected — Constant-time execution: written as "hammer-row"; design must not repeat it (R253)
- 2026-10-22 23:01 · timing channel attacks · named by the author · rejected — CPU vulnerability avoidance: wants a constant-time flag so instructions take equal time (R253)
- 2026-10-22 23:01 · OpenBSD mitigations · named by the author · chosen — Memory safety from OpenBSD mitigations: make memory-safety mitigations "soft and easy"; improvements "theo would love" (R254)

- 2026-10-22 23:05 · Linux security · named by the author · chosen — Pain-points research: same list from its people (R255)
- 2026-10-22 23:08 · UEFI · named by the author · considered — Boot process, BIOS and open firmware: asked to look into it (R256)
- 2026-10-22 23:15 · OpenBSD disk partitioning · named by the author · considered — Boot process, BIOS and open firmware: asked where it originated and how it differs (R257)
- 2026-10-22 23:15 · GPT · named by the author · considered — Boot process, BIOS and open firmware: asked how it plays into boot (R257); later chosen 2026-10-22 23:23
- 2026-10-22 23:23 · majority boot block offset · named by the author · chosen — Disk label and boot block: "follow the majority as far as boot block offset" (R261)
- 2026-10-23 00:12 · PDP-10, PDP-11 and VAX-11 · named by the author · chosen — Manual: deliverable and chapter order: manual to cover what was so great about programming them (R268)
- 2026-10-23 00:14 · old Sanskrit books (aphorism plus commentary) · named by the author · chosen — Manual purpose, style and notation: style: aphorism followed by commentary (R269)
- 2026-10-23 00:14 · Mishnah and Gemara · named by the author · chosen — Manual purpose, style and notation: same aphorism-then-commentary structure (R269)
- 2026-10-23 00:17 · VAX and PDP-11 (minimality) · named by the author · considered — Design philosophy and optimization layers: pdp64 is "even more minimal" in hindsight (R272)
- 2026-10-23 00:41 · ARM-style memory-mapped device · named by the author · chosen — SIMD and vector work: "mmap device arm style route" for SIMD/AVX2 vector work (R277)
- 2026-10-23 00:41 · AVX2 · named by the author · considered — SIMD and vector work: the SIMD work the chapter covers (R277)
- 2026-10-23 00:41 · APL/K · named by the author · chosen — SIMD and vector work: mention as making vector languages easier to support (R277)
- 2026-10-23 00:49 · HDMI, SATA, USB · named by the author · considered — Standard GPU, NIC and audio interfaces: "standard kit", our interface goes through mmio (R278)
- 2026-10-23 00:53 · VAX I/O space · proposed by the assistant, accepted · considered — Address space (VAX I/O space): memory low, I/O above; to be noted in the book (R281)
- 2026-10-23 01:06 · SCSI/ATA/SATA/NVMe · named by the author · considered — Physical address bit that selects device space: all follow the same device pattern, not limited to 8-register mmio (R282)
- 2026-10-23 01:47 · mkbook.py · named by the author · rejected — Book toolchain, build image and aports: reimplementing what buildabook does (R285)
- 2026-10-23 08:12 · alpine:edge docker with ada-on-alpine · named by the author · chosen — Book toolchain, build image and aports: build and install all the aports on it (R288)
- 2026-10-23 08:57 · buildabook · named by the author · chosen — Book toolchain, build image and aports: replace mkbook.py and push buildabook (R289)
- 2026-10-23 19:01 · USB enumerated over PCIe · named by the author · chosen — Bus enumeration and attached buses: model for the bus enumerating arbitrary attached buses (R292)
- 2026-10-23 20:06 · Plan 9 draw(3)/devdraw · proposed by the assistant, not taken up · considered — Architecture recommendations and open proposals: compared with ring-based command encoding for the native device (R294)
- 2026-11-04 00:07 · PCI Express · named by the author · chosen — The bus: DMA design, document and PCI Express base: pcie for the bus, with pain points solved or mitigated (R311)
- 2026-11-04 00:07 · IOMMU mapping of C64/C128 · named by the author · considered — The bus: DMA design, document and PCI Express base: "i think we should, but that is extraneous" (R311)
- 2026-11-04 15:28 · Steve Johnson pcc VAX optimizations · named by the author · chosen — Dual-encoding toolchain, mnemonics and integration: fold into the -xdmr flag (R333)
- 2026-11-04 15:30 · hbnf · named by the author · chosen — Grammar as the single source of truth: source of truth for the tools (R335)
- 2026-11-04 15:30 · Python pivot · named by the author · rejected — Grammar as the single source of truth: "undo your pivot to python" (R335)

- 2026-11-05 06:58 · hbnf C, Ada, Rust and Zig backends · proposed by the assistant, not taken up · considered — hbnf layout grammar and templates: chart backend would sit alongside them (R346)
- 2026-11-05 06:58 · Python chart generator · proposed by the assistant, not taken up · considered — hbnf layout grammar and templates: interim, behind one layout model (R346)
- 2026-11-05 07:01 · Mustache · named by the author · rejected — hbnf layout grammar and templates: "the mustache templates are going away" once rewrite rules exist (R347)
- 2026-11-05 07:39 · hbnf %emit{} %action{} with prologue/epilogue · named by the author · considered — hbnf layout grammar and templates: asks if C code in the grammar can do the job until rewrite rules land (R350)
- 2026-11-08 05:33 · Velinsky (DEC Rainbow and Commodore designer) · proposed by the assistant, accepted · considered — Velinsky link between C64 and DEC: common designer as link between DEC and Commodore (R355)
- 2026-11-08 05:34 · C64 and PDP-11 similarities · named by the author · considered — Velinsky link between C64 and DEC: commonalities found, cited as evidence (R356)
- 2026-11-09 06:52 · PCIe/NVMe/USB · named by the author · chosen — Device register model: native 8-register devices to virtio: standardizing on the latest buses (R359)
- 2026-11-09 06:52 · 12 register + dma · named by the author · considered — Device register model: native 8-register devices to virtio: compared with virtio devices (R359)
- 2026-11-09 07:02 · virtio · named by the author · chosen — The bus: DMA design, document and PCI Express base: "virtio across the board" (R360); later considered 2026-11-09 07:14
- 2026-11-09 07:02 · native 8-register devices · named by the author · rejected — The bus: DMA design, document and PCI Express base: "forget the native 8 register devices. gone." (R360)
- 2026-11-09 07:02 · Loongson 5-level TLB · named by the author · considered — Physical address bit that selects device space: asked what else is affected if pdp64 had it (R360)
- 2026-11-09 09:10 · RISC-V · named by the author · considered — Prefix-coded encodings, 16-register and 32-register grammars: faulted for no comparison of opcode counts or bit layouts with pdp64 (R365)

# Appendix H — Proposed

**This appendix holds designs that nobody has decided: statements tagged *(P)*, sketches and lists of options.** The body of the manual describes the architecture as it is to be; what is only proposed is kept here, where a reader can see that it is not settled and an author can decide it later. Each section is one chapter's share, headed with the chapter's number and title. A statement that moved here from the body keeps its number as a label (`**2.3.3** (moved from chapter 2)`), so that a citation of that number still leads to a place. The tag *(P)* stays on a statement that carries it. Questions that the architecture must answer and has not are in appendix J; what has been decided to be put off is in appendix I.

## H.1 Chapter 1: Introduction

**1.1.5** (moved from chapter 1, from the commentary of 1.1) *(P)* **A thread between the two lineages.** The pdp64 joins a DEC machine's instruction set to a Commodore machine's devices, and the two companies are linked by at least one person: Ira Velinsky, the industrial designer credited with modifying the Commodore 64's case, had earlier worked at Digital Equipment Corporation (he is credited there with the Rainbow 100 of 1981), and a Commodore history credits his DEC work with drawing the attention of Commodore's management. The author predicted such a link before finding it. The hypothesis he draws from it is that personnel passed between the two companies and that the similarities between the C64 and the PDP-11 are too many to be chance *(P)*; the source itself claims only the person, and no influence of DEC designs on Commodore's, and the similarities are not yet listed with sources (the record is in `docs/c64-devices-from-chat.md`, section 8).

## H.2 Chapter 2: The bus

*Alternative noted in the commentary on 2.5 (moved).* A cache-maintenance instruction would remove the cost of reading received data from uncached memory for a non-coherent device that may come later.

## H.3 Chapter 3: Graphics: the terminal and its devices

### 3.8 The pdp64 terminal: the blitter (moved from chapter 3)

**3.8.2** (the graphics core, moved from chapter 3; the number is that of the statement in the chapter before the move, and is not the 3.8.2 that this fragment cites for the pixel format) One core is dedicated to the graphics device, the Blit's model of a terminal with a processor of its own (errata 76, the author's recommendation; the core classes are in chapter 7, 7.11). *(P)* Decided on 2026-11-04 and shrunk by the decision of 2026-11-10 (errata 132): what remains of dedicated roles is open (J.7.5).

**3.8.4** (moved from chapter 3) The blitter, the Blit's bit-block-transfer operation as a device, copies and combines rectangles of pixels with raster operations: registers for the source and destination base addresses, the strides, the width and height, a raster operation (the sixteen binary operations of source against destination and the common ternary ones), a mask or transparent colour, and a control register with direction, go, busy and interrupt on done. The device chat chose a three-operand `draw` instead (`decisions-pending.md` E3). The register list is the assistant's sketch from the ISA chat; the author has not discussed it. *(P)*

**3.8.5** (moved from chapter 3) Each of the three is a device in a slot: it is enumerated by the bus, mapped into its own 4 KB page, and found by its identification register like any other (chapters 2 and 5). The device chat put the C64 devices in one 8 KB window per process (`decisions-pending.md` E5). The slot and the page rest on the eight-register convention that errata 95 withdrew for the standard devices. *(P)*

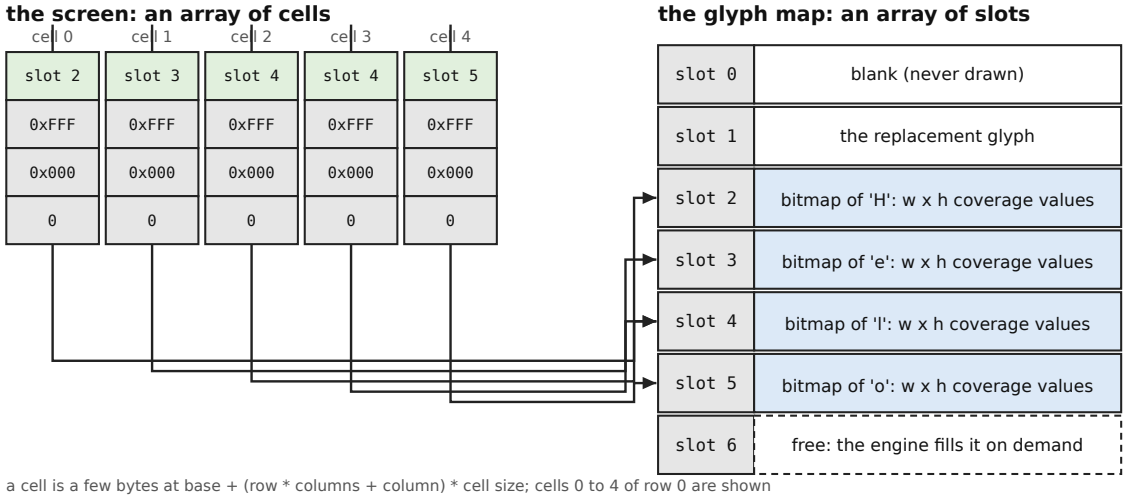
**3.8.6** (moved from chapter 3) The framebuffer stays plain memory, so that software can draw without the blitter; the blitter is an assist that can be bypassed, not the only way to draw (the Blit authors' advice in 1985 was "transparent, so it can be bypassed"). *(P)*

**3.8.7** (moved from chapter 3) Four raster operations are required of the blitter: store, OR, clear and exclusive-OR (these four sufficed on the Blit, and exclusive-OR dominated its use); the other twelve binary operations and the ternary ones are optional. The requirement is the assistant's, from the Blit papers. *(P)*

## 3.9 The terminal as the author describes it (moved from chapter 3)

The terminal and its glyph map are decided in principle (3.9.1 in the body). What follows is the assistant's elaboration of the cell and the slot.

**What the author's earlier terminal chat adds** (2026-06-18, `docs/c64-devices-from-chat.md` section 6; a separate project, not a pdp64 decision): a cell holds a 32-bit foreground colour, a 32-bit background colour, a UTF-8 character and about eight bits of attributes (bold, italic, underline, blink); "the number of cells take up so little memory, the 64 bits devoted to color is trivial"; "each cell holds its actual data, not a reference to the data"; sprites are a separate plane that composites over the cells; existing tty programs must run through a shim and native programs use the terminal fully, "peek and poke is the internal interface". That design fits the cell array of the figure below and differs from the author's new words in one place: **the chat-era cell holds the character code itself, while the glyph map of the new words implies a slot** (3.14, pair 3, candidate (a) against (b)).




a cell is a few bytes at base + (row * columns + column) * cell size; cells 0 to 4 of row 0 are shown

The same slot 4 serves both letters l: a glyph is drawn once and used by every cell that names it.
A TrueType engine rasterises each glyph into its slot (3.9.3). Field names and sizes are placeholders, not a specification.
Slot 6 shows an empty slot, to be filled when a cell first needs it.

As an arithmetic example, with the chat-era cell (about 13 bytes: 4 + 4 + 4 for a character + 1 for attributes) and a 1920 by 1080 screen of 8 by 16 pixel cells, there are 240 columns and 67 rows, 16,080 cells and about 209 KB of cells; with slots of 16 by 32 coverage bytes (512 bytes) and 4,096 slots, the glyph map is 2 MB. Both are small against a 12.4 MB frame at 48 bits (3.11.2), which is the author's observation of 2026-06-18: the cells are small, and text is cheap to store. These are examples, not decisions.

## 3.10 A peek/poke program for the terminal (moved from chapter 3)

This is what the author's description would let a program do. **No register of the terminal is specified** (3.11), so the capitalised names are placeholders and any number is an example.

```
# placeholders: SCREEN is the base of the cell array, COLS its width in cells,
#               GLYPHS the base of the glyph map, GLYPH_SIZE the size of one slot

put_glyph(row, col, slot, fg, bg):                # one cell: four stores
    cell = SCREEN + (row * COLS + col) * CELL_SIZE
    poke32(cell + GLYPH, slot)                    # which glyph (or the character code, 3.9)
    poke32(cell + FG, fg)                         # colours, modern depth: the pixel format of 3.8.2
    poke32(cell + BG, bg)
    poke8 (cell + ATTR, 0)

define_glyph(slot, font, codepoint):              # a glyph the program has just thought of
```

```
    bitmap = ttf_rasterise(font, codepoint, CELL_W, CELL_H)   # software: the TrueType engine
    copy(GLYPHS + slot * GLYPH_SIZE, bitmap, GLYPH_SIZE)      # an ordinary memory copy

put_glyph(0, 0, slot_of('A'), RED, BLACK)          # an A, red on black, top left
define_glyph(200, my_font, 0x03BB)                 # a lambda goes into slot 200
put_glyph(0, 1, 200, WHITE, BLACK)                 # and is used by naming slot 200
```

Graphics beside text, in the C64 manner, are another array and one more register set:

```
# the graphics plane: a framebuffer of FB_W by FB_H pixels; this sketch uses 32-bit pixels, as the standard
device does
# (the 48-bit pixel of 3.8.2 would be 6 bytes: a different multiplier and three 16-bit stores)
put_pixel(x, y, colour):
    poke32(FB_BASE + y * FB_STRIDE + x * 4, colour)         # FB_STRIDE counts bytes, as in 3.5
# sprites, if the design has them (3.9.4: not decided):
poke(SPRITE_N + X, 100); poke(SPRITE_N + Y, 80); poke(SPRITE_N + SHAPE, shape_slot)
```

Each line is one store, so the program is as small as the C64's was. Everything is memory, so `memcpy` fills a glyph slot as it fills any buffer. Compared with the list of 3.14 there is no mode setting, no format conversion, no flush (the memory type of 3.11.1 takes its place), no atlas to pack, no shader and no command buffer. The terminal does no three-dimensional graphics; for that the standard device, virtio-gpu, remains (3.12, 3.13).

## 3.12 How the terminal drives HDMI and the other ports (moved from chapter 3)

**3.12.2** (moved from chapter 3) The video device's output side is therefore a pixel stream with horizontal and vertical sync, a clock and a data-enable, which is what every such transmitter accepts; the device's registers set the mode (width, height, blanking, pixel clock, the pixel format), the base address of the framebuffer and the display on and off. *(P)*

(3.10.2, that other standard outputs are the same shape, is in the body. That each port is one more slot or one more output of the video device is the assistant's.)

**3.12.4** (moved from chapter 3) The sound device's output is likewise a standard digital audio stream (I2S or the HDMI audio channel) into a converter or the transmitter. The processor never sees the HDMI packets, but whoever sets the mode must also program the audio clock relation: the transmitter needs N (and CTS, or a hardware CTS calculation) for each pixel clock and sample rate (an ADV7511 driver computes both; general knowledge, *unverified*). *(P)*

## 3.13 The framebuffer (moved from chapter 3)

## 3.14 virtio-gpu (moved from chapter 3)

**3.14.3** (moved from chapter 3) The framebuffer of virtio-gpu is the backing memory that the driver allocates; on the pdp64 it is ordinary memory mapped uncached (3.11.1), and the transfer command is the hand-off that tells the device to read it (the five steps of 2.5.3). *(P)*

## 3.15 A standard graphics card (moved from chapter 3)

**3.15.3** (moved from chapter 3) On the other side PCI Express is a standard enumerated bus, which suits the bus-as-a-device design (each function a slot, as each USB device is), and OpenBSD carries graphics drivers for several vendors' cards ported from Linux (*unverified*). *(P)*

## 3.16 What made graphics programming annoying: the pdp64 side (moved from chapter 3)

The goal (3.14.1) and the measure (3.14.2) are the author's and are in the body. The sketches of the pdp64's side below are the assistant's candidates for the means.

**Pair 1: getting a picture on the screen at all** (the sketch of the pdp64 side).

The pdp64 could offer **a device that already shows something**:

```
# the pdp64 way, open (a sketch)
# the device powers up with a sane default mode, in the monitor's preferred mode, with a canonical pixel
format:
fb = peek(DISPLAY_BASE)           # where the picture is, always mapped
w  = peek(DISPLAY_WIDTH); h = peek(DISPLAY_HEIGHT)
poke(fb + (y * w + x) * 4, 0xFFFFFFFF)     # a white pixel: done
```

Options, with trade-offs. (a) The device **reports its mode** (a register holds the width, height and refresh rate of the mode chosen at start-up from the monitor's data; virtio-gpu's `GET_DISPLAY_INFO` already returns the width and height of each output in a `virtio_gpu_resp_display_info`); nothing to set. *Cost:* the device reads the EDID itself (a small I2C engine, which 3.10.1 already puts at the transmitter chip) and applies a policy; a program that wants another mode still has the registers. (b) A **fixed canonical pixel format** with the device converting at scanout (the repository's 48-bit RGBA pixel, errata 73, would be that format on the native device; virtio-gpu offers only 32-bit formats with eight bits per channel: `VIRTIO_GPU_FORMAT_B8G8R8A8_UNORM` and its siblings, in `virtio_gpu.h`, so the 48-bit pixel does not exist on the standard device; 3.23 lists the inconsistency). *Cost:* a conversion stage in the scanout, and memory a program wastes if it wants fewer bits. (c) **Stride equal to width** unless the program asks for padding, and the framebuffer window **always mapped**: the program has no `mmap`, no bank register, no `map_physical`; the window is in its address space from the start, a decision of the page tables and the bus (2.3.3 and 3.13.4). *Cost:* a fixed place in the address space, and a rule for what a program does that wants two framebuffers.

**Pair 2: the cache and the flush** (the open part).

A write-combining memory type for the framebuffer is a candidate improvement that the four memory types of 2.3.3 do not name; the cache rule itself is decided *(D)* and the speed is open *(P)*.

**Pair 3: drawing text** (the sketch of the pdp64 side).

The pdp64's terminal would do it as the C64 did, with one store:

```
# the pdp64 way, open (a sketch; names are placeholders)
poke(CELL(row, col).glyph, SLOT_OF('A'))     # or: poke(CELL(row, col).char, 'A')
poke(CELL(row, col).fg, RED)
# a new glyph: one library call fills a slot from the TrueType engine
font_render_into_slot(font, 'λ', slot)       # rasterises the outline and writes the bitmap into the slot
```

The author's glyph slots do the work of the atlas, and one library call replaces the first half of the boilerplate (3.9). Two candidates: (a) the cell holds the **character code**, and the device or a table maps the code to a slot, so the program never names slots (the author's c64x terminal of 2026-06-18, `c64stuff.txt`, line 5015: "each cell holds its actual data, not a reference to the data"); or (b) the cell holds the **slot number** directly, which is the simplest hardware and makes the program responsible for assigning slots. The choice is open (3.23): (a) is easier for the program and costs a lookup table in the scanout; (b) is nearly free and puts a small allocator in a library.

### *The pain points with the sugar*

The table of 3.14 with its two further columns: the sugar that would remove each pain point, and where it appears in the sketches of 3.21 (S1 layers, S2 layers plus blitter, S3 command queue, S4 plain virtio-gpu with a library).

| Pain point | Typical boilerplate (illustrative) | The sugar that would remove it | Where (3.21) |
|---|---|---|---|
| Mode setting and discovering what the display supports (EDID, modes) | 15 lines (VESA) to 40 (DRM) | the device reads the monitor and starts in its preferred mode; a register reports it | S1 S2 S3 native; S4: `GET_DISPLAY_INFO` gives the size, a library the rest |
| Pixel formats | one format code to get right, and conversion code when it is not | one canonical format, the device converts at scanout | S1 S2 S3; S4 has 32-bit formats only |
| Stride and pitch | one multiply in every pixel address | stride equal to width unless asked; the address of pixel (x, y) is a register-reported formula | S1 S2 S3 |
| Endianness and colour order (RGB or BGR, alpha first or last) | a swap or a table, in every program | one defined order, little-endian (decided for the machine, errata 72); the colour field order named once | all |
| Bank switching and mapping the framebuffer | 5 to 20 lines | a window always mapped, no banks | all (3.13.4 is the rule for the type) |
| Double buffering and tearing | 10 to 30 lines, plus a frame loop | a latching base register, a fence, per-layer flip, versioned buffers (3.15) | S1 S2 per layer; S3 queue; S4 flush |
| Synchronising with the display | polling or a vblank interrupt handler | raster interrupt and a fence (3.15); decided: vblank interrupt at a fixed line (3.8.9) | all |
| Cache coherence between CPU writes and the device | 3 lines per update, forgotten once per project | decided: uncached memory type, no flush exists (2.5.2, 3.11.1); open: write-combining for speed | all |

| Pain point | Typical boilerplate (illustrative) | The sugar that would remove it | Where (3.21) |
|---|---|---|---|
| Device initialisation and context creation | 50 to 200 lines (Vulkan), 25 (virtio-gpu) | the device works at power-up; a single `enable` bit, a default | S1 S2 S3; S4 via a library |
| Buffer and memory management (allocate, map, upload, barriers, fences) | 20 to 100 lines | all video memory is a plain array in a window (S1 S2); no upload, nothing to map; or the device allocates (S3) | S1 S2 plain; S3 managed |
| Shader compilation and pipeline state | 50 to 300 lines | no shaders at all in S1 S2; a few fixed programs in S3, chosen by a field | S1 S2 none; S3 fixed |
| Drawing text (glyph atlases, kerning, layout) | 100 to 300 lines | a one-store put-character; glyph slots filled by a library call (3.9) | all native; S4 in a library |
| Clipping and windows | 10 to 30 lines per drawing call | a clip rectangle register per layer; a window is a layer | S2 S3 |
| Sprites and cursors | 20 lines to save, draw and restore | hardware sprites and a hardware cursor, two registers each | S1 S2 S3; S4 has the cursor queue only |
| Colour spaces and gamma | a table and a conversion, or wrong colours | one defined space (linear or sRGB, to be decided) with the device applying the transfer curve | open: S1 S2 S3 |
| Resolution changes and hot-plug | 20 to 60 lines of event handling | the bus reports a plug as an event (decided in chapter 2, 2.4); the device re-reads the monitor and signals "mode changed" | all |
| Fonts and scaling (HiDPI: screens with several pixels to a logical dot) | a scale factor threaded through every program | cells and glyph sizes defined in physical units by the device; the TrueType engine renders for the real pixel size | S1 S2 S3 |
| Input of palettes | 256 writes through ports (VGA) | a palette is an array in memory, loaded by `copy`; or none (true colour) | optional |
| Sound and video sync | a timestamp and a resampler | one clock for both devices (the audio clock relation of 3.12.4, decided as a duty of the mode setting) and a frame counter that audio can read | all |

Every item costs gates or flexibility, and the sugar column is a list of things to decide, not a recommendation. The repository has already decided the coherence rule *(D)*, the vertical-blank latch *(D)*, the uncached framebuffer *(D)* and little-endian *(D)*; every other cell in the sugar column is a candidate.

## 3.17 Anti-tearing sugar (moved from chapter 3)

The sugar is wanted (3.15.1 in the body, the author's). "So that the common program needs no more than a store" is the assistant's aim. The candidates, with what each costs, are proposals:

- **A scanout base register that latches at the vertical blank** (a page flip: the register holds the address of the picture to show and takes the new value only in the blank), with a completion **interrupt** or a **fence** (a counter that the device advances when the flip has happened, which the program can read or wait on, so it knows that the old picture is free). *Gives:* no tearing, no polling of the raster; the program writes one register and goes on. *Costs:* two pictures of memory for the program that uses it, and a rule for what the register shows if it is written twice in one frame (the first is dropped, or the second waits). *Already in 3.8.9, except the fence.*
- **Per-layer latching**, in a design with layers (3.21, sketches 1 and 2): each layer has its own base register and flips independently, so a cursor or a sprite layer can move every frame while the text layer flips only when its text changes. *Gives:* the cheapest case for text, a moving cursor with no redraw. *Costs:* one set of registers per layer.
- **An atomic commit of several registers**: several registers are staged and applied together at the blank (this is what `ARM` of 3.8.9 is for the single set; Linux calls the same idea "atomic modesetting" and offers `DRM_MODE_ATOMIC_TEST_ONLY` to ask "would this work?" and `DRM_MODE_ATOMIC_NONBLOCK` to ask for it without waiting, both in `drm_mode.h` on this machine). *Gives:* a change of mode, picture and cursor that cannot be seen half-done. *Costs:* the shadow copies of the registers, which a small device pays in flip-flops; an optional test-before-commit is more.

- **A raster-position register and a raster interrupt** (the C64 way): the program can know the row being read and be interrupted at a chosen row. *Gives:* mid-frame changes (the C64's trick of a different colour or sprite position per band) and a way to time drawing behind the beam. *Costs:* one register and an interrupt source; encourages programs that are tied to the display's timing.
- **Variable refresh** (the monitor waits for the frame instead of reading at a fixed rate: so a frame that is late is not a missed blank but a short wait; the standards are called adaptive sync, FreeSync, G-Sync and, for HDMI, VRR, variable refresh rate; unverified general knowledge). *Gives:* a late frame costs only its own delay, not a halving of the rate. *Costs:* the output must support it (the HDMI transmitter chip of 3.10.1 and the monitor), and the device's mode registers must say so.
- **Damage rectangles** (the program says which part of the picture changed and only that part is sent on): this is what virtio-gpu's `TRANSFER_TO_HOST_2D` and `RESOURCE_FLUSH` take as a `virtio_gpu_rect` (3.12). *Gives:* less memory traffic when little changes (a terminal). *Costs:* the program must track what it changed.
- **A hardware cursor and sprites**: objects that the scanout overlays at a position without touching the picture under them, so moving one never needs a redraw and never tears. *Gives:* the cursor, and in a game its moving objects, for the price of two registers each. *Costs:* gates in the scanout path and a limit on how many. (virtio-gpu has a cursor in its protocol: a separate cursor queue, `UPDATE_CURSOR` and `MOVE_CURSOR`, `virtio_gpu.h`.)
- **Versioned or copy-on-write buffers**: the device keeps the picture the scanout is reading and the program writes to a fresh version that replaces it atomically when the program says it is done, so the program can never see or show a half-drawn frame and never has to name two buffers. *Gives:* the sugar nearest to "nothing to do": a program draws and says `commit`. *Costs:* the memory and copying are the device's; a page-granular copy-on-write needs the memory-management machinery of chapter 6 or a device-side equivalent; a whole-frame version is twice the memory with a rule of who allocates it.
- **A fence or sequence counter**: a register that the device increments when each commit has been applied, which the program can read (poll) or wait for on an interrupt, to know when a buffer is free. *Gives:* synchronisation with no guess. *Costs:* a counter and a rule for wraparound.

And the same loop, in the style the pdp64 could offer, written with three of the candidates (a latching base register, a fence, and the interrupt) in the form that the program meets:

```
# 3. the flip at the blank, with the sugar: no polling of the raster, no waiting for the flip to be safe
poke(SCANOUT_BASE, front)
loop:
    draw_scene(back)
    poke(SCANOUT_BASE, back); poke(ARM, 1)    # staged now, latched by the device at the next blank (3.8.9)
    wait_until(peek(FLIP_COUNT) >= expected)  # a fence: the old picture is free (a candidate, not defined)
    swap(front, back)
```

Section 3.21 says which sketch gives which of these cheaply: the layered sketches give per-layer latching and a hardware cursor almost for free, the command-queue sketch gives the fence and the atomic commit, and the virtio-gpu route gives damage rectangles and fences and nothing about the raster. `FLIP_COUNT` and `ARM` above are placeholders; the repository defines `ARM` only as a name in 3.8.9.

### 3.18 A c64 mode: a small computer on the die that acts as the terminal (moved from chapter 3)

The c64 mode is the author's (3.16.1 and 3.16.2 in the body). This section is the assistant's explanation of how it could work, for the two readers, and states the questions it leaves open.

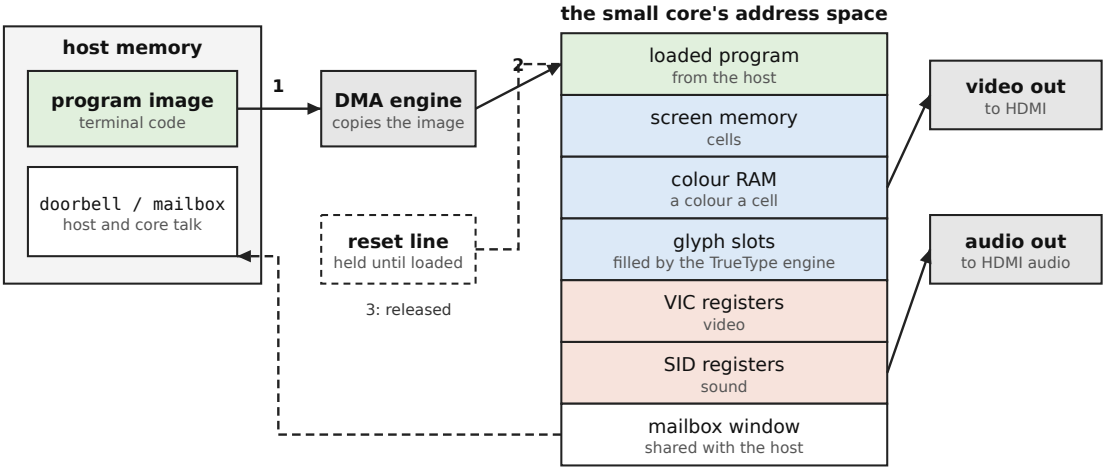
**What the Blit did.** The Blit (3.3) was a terminal with a computer of its own, and the host sent it **programs** instead of escape codes to interpret. The records in `docs/kernel-pain-points.md` (section 14I, from the Blit papers) say that the terminal received downloaded code over its serial line, that `mpxterm`, the terminal's program, "is 1627 lines of C" plus "204 lines of assembler" and multiplexed several host programs over the one line, and that the authors held that "an intelligent terminal is not an idiot savant, it is one that can be educated". A **terminal with its own small computer** is therefore a display, keyboard and mouse driven by a processor on the terminal's side of the cable, running a program the host loaded; the host sends it messages and the program draws. How the download worked (the protocol, the program format) is not in the records and is *unverified*. The VT52 and VT100 were the opposite: a fixed circuit obeying a fixed set of codes (3.2).

**A small core with the C64's chips is a terminal on a chip.** Take a small computer, give it the C64 family's circuits (the **VIC**, the video chip; the **SID**, the sound chip), **screen memory**, **colour RAM**, **glyph memory** (3.9) and a window towards the host, and let the host load its program. The program is the terminal's firmware: it receives bytes or messages from the host (by a mailbox), puts characters into screen memory, plays a sound, moves a sprite. The host sees a device that takes characters, as it saw a teletype, and the cleverness sits in a program the host can replace, as in the Blit.

**What "loaded by PCI Express DMA" means, mechanically.** The host puts the program image (the terminal's code and initial data) in its own memory and tells the device's **DMA engine** (the part that copies memory without the processor's help, chapter 2) the image's address, its length and the address in the small core's memory where it goes. The engine reads host memory over the PCI Express link (the bus-master read of 2.5) and writes the bytes into the core's memory. The core has been held in **reset** (a line that keeps a processor from running until it is released) since power-on, so it never executes incomplete memory; when the copy is done the host releases the reset and the core starts at its fixed start address. With placeholder names (no register of any such device exists):

```
load_terminal(image, length):                    # on the host, as a driver would
    poke(CORE_CTRL, HOLD_RESET)                  # the core is stopped
    poke(DMA_SRC,  physical_address_of(image))   # where the image is, in host memory
    poke(DMA_DST,  CORE_LOAD_ADDRESS)            # where it must go, in the core's address space
    poke(DMA_LEN,  length)
    poke(DMA_GO,   1)
    wait_until(peek(DMA_DONE) == 1)              # or an interrupt
    poke(CORE_CTRL, RELEASE_RESET)               # the core starts executing the loaded code
    # the host now talks to it through the mailbox:
    mailbox_send(PUT_TEXT, row, col, "hello")    # a message the loaded program understands
```

**c64 mode: the host loads a program into a small core on the die, and the core is the terminal**




1 the host places the image in its own memory; the device copies it into the core by DMA (over PCI Express).
2 the program lands at the core's load address; 3 the core is let out of reset and runs it as the terminal.

Which core, what the mailbox is, and how the image is checked are open (3.16); sizes and order are an example.

The core's address space is the C64's (3.4) with a mailbox added. Everything the loaded program needs is in one flat space, as in the 1982 machine: its own code, screen and colour memory, glyph slots, the video and sound registers, and a window the host also sees. Video goes to the display connector and audio to its audio channel (3.10); audio over HDMI brings the audio clock relation of 3.12.4 with it, which the core's program must honour or the transmitter's own calculation must supply.

The questions that the author's words leave open, and the options for the core, are in J.3.11 (the statement 3.18.3, moved from chapter 3, is that list).

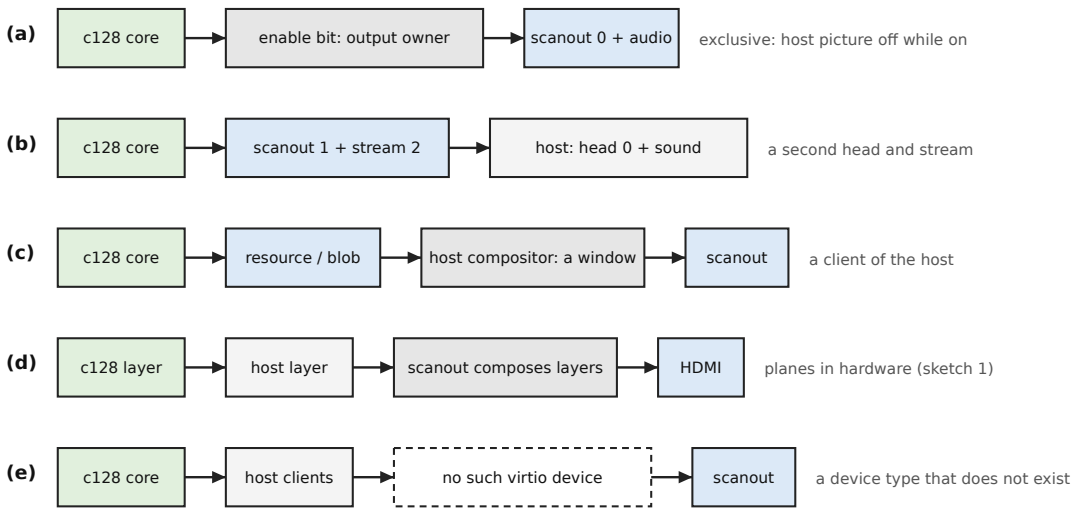
### *The compositor question*

The author asked, 2026-11-10: "not sure what that would look like. is there virtio-compositor?"

**What a compositor is, at more length.** A **compositor** (defined in 3.16.2) combines the pictures of several windows or sources into the one picture sent to the display, which is then scanned out as in 3.15. In a desktop window system each program draws into its own picture (its **surface**) and the compositor places the surfaces, one over another, into the frame the scanout reads. The PC's windowing systems did this in software (3.5); in the layered sketches of 3.21 the scanout hardware does it, as the C64's chip did with sprites. The PDP-11 has no antecedent; its terminal was a single stream.

**Why "takes over" is the simple case.** If the c128 owns the output outright when enabled, there is nothing to compose: the machine has one scanout and one audio path, and the only question is who drives them. The multiplexer is a single enable bit (path (a) below). The compositor case differs in one way: the core is one client among others, so its picture and sound must be handed over in a form that the host's compositor, or the display controller's layers, can place.

**five ways the c64-mode core and the host could share the display and the audio**




Green: the on-die core. Blue: output. Grey: the arbiter. Dashed: a thing that would have to be invented.
In (a) the multiplexer is one enable bit; in (b) to (e) the core becomes a client of something else.

The hand-over of any of these should happen in a vertical blank, with the audio muted by a short ramp (3.16).

**What exists, checked.** No virtio device is called a compositor: no entry of `virtio_ids.h` on this machine names one. The display-related types are GPU (16), input (18), sound (25), video encoder and decoder (30, 31), DMA buffer (37) and audio policy (39). The nearest parts in `virtio_gpu.h` are up to 16 scanouts (`VIRTIO_GPU_MAX_SCANOUTS`); `SET_SCANOUT_BLOB`, which sets a scanout from a blob resource (a memory object shared with the host without copying); a **cursor queue** separate from the control queue (`UPDATE_CURSOR`, `MOVE_CURSOR`); and capability sets (`VIRTIO_GPU_CAPSET_VIRGL`, `VIRGL2`, `GFXSTREAM_VULKAN`, `VENUS`, `CROSS_DOMAIN`, `DRM`) selected with `context_init`, so that several rendering contexts, each with its own ordering (the "timelines" of the header comment), share the device. QEMU's `docs/system/devices/virtio/virtio-gpu.rst` describes `virtio-gpu-rutabaga`, which provides "gfxstream rendering and Wayland display passthrough", with a `cross-domain` capset for Linux guests and experimental `x-gfxstream-composer` for Android guests; it says no more about what the cross-domain path carries. That the crosvm project uses the same mechanism to show a guest's windows as windows of the host's Wayland compositor (Wayland is the Linux display protocol in which clients hand surfaces to a compositor) is general knowledge and *unverified*. **virtio-snd** and **virtio-input** exist for audio and for keyboard and mouse. Paths (b) and (c) can therefore be built from parts that exist, and path (e) is a new device type; how a type gets its number is *unverified* here.

The five compositor paths, with their gains and costs, are in J.3.12.

The author's path (c), 3.16.3 in the body, makes the core a client like any other, the way rio treats every window (3.18: a window is a buffer and a rectangle), and its parts exist (a virtio-gpu resource or blob as the surface, 3.12). The details under it are in J.3.12; the audio form of the same choice is in 3.17.

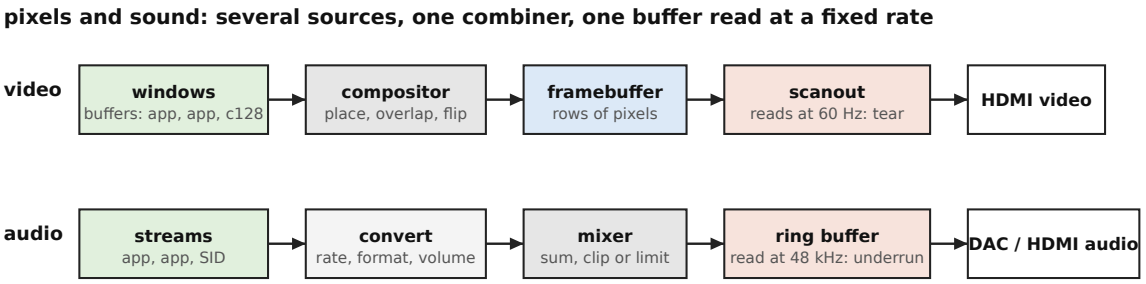
**How the switch avoids tearing and pops.** Enabling or disabling the mode changes who feeds the scanout and the audio path. A change in mid-frame tears the picture (3.15), and a sudden change of the waveform's value is heard as a pop. The

remedy has the shape of 3.15: hand over at a **vertical blank**, as the latch of 3.8.9 does for registers (the enable bit is one more shadowed register that ARM latches), and **mute with a short ramp** (a fade of a few milliseconds down before and up after, not a cut). In path (a) the enable bit is the shadowed register and the mute belongs to the audio path; in (b) to (e) the compositor does the hand-over and the core only stops or starts its stream. The audio clock relation of 3.12.4 must stay unchanged across the switch, or the transmitter loses its lock and the sound drops.

*Commentary.* **Chain.** The PDP-11 had only the fixed terminals of 3.2. The Blit loaded its terminal program from the host. The C128 put several complete computers in one box (general knowledge, *unverified*), and the MiSTer framework runs C64 and C128 cores in programmable logic. The pdp64 proposal puts a small core on the die and loads it by DMA. **Trade-offs.** A computer of its own makes the terminal programmable and independent of the host's drawing, and makes it a second machine to load, secure and keep consistent with the first. **Impact.** *Kernel:* a driver for the loader and the mailbox. *Compiler:* none. *Hardware:* a core, memory, DMA and the C64 family's circuits, or nothing in option (d).

## 3.19 The audio of the c128 core (moved from chapter 3)

The options for the audio form of the c64-mode core are in J.3.13.




A late frame tears; a late buffer underruns: a click. Both are cured by writing ahead of the reader and by letting the device clock, not the program clock, decide when the combiner runs.

The SID is the C64's exception: it makes the sound itself from three voices, so it has no buffer and no mixer in memory.

Stages after the model of general knowledge of sound servers (unverified, 3.19); virtio-snd streams as in virtio_snd.h.

## 3.20 Windows: the goal (moved from chapter 3)

The author, 2026-11-10: "it should mention blit terminal was inspiration for the rio and NeWS windowing systems, and we'd like to bring over what we can of that simplicity also."

The goal (3.18.1) is in the body; the means are in J.3.14.

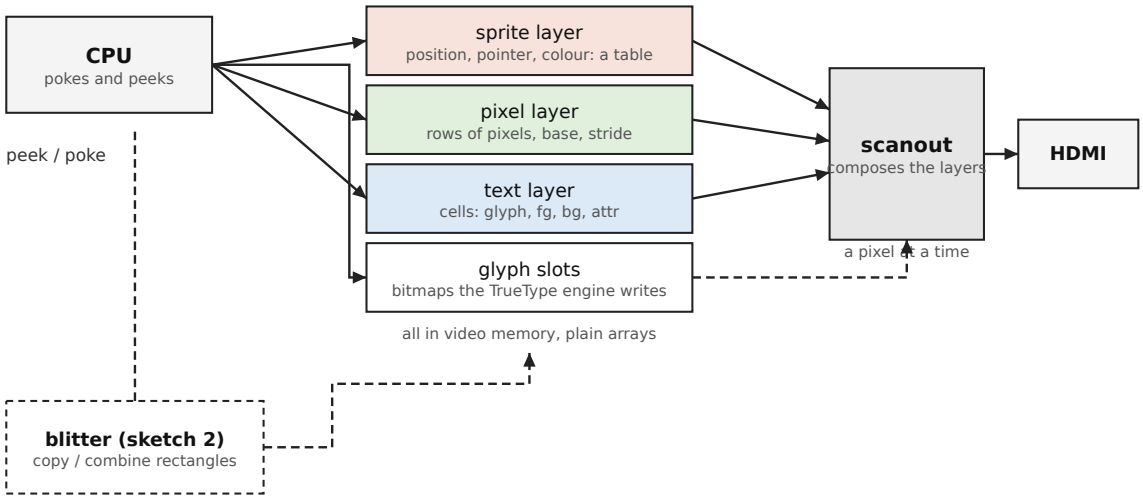
## 3.21 A design still being worked out: what a mashup of GPU, video memory, C64 and Blit could look like (moved from chapter 3)

The author, 2026-11-10: "conclude the graphics chapter: we're still figuring it out, what would a mashup of gpu, vram, c64 and blit look like." **Nothing in this section is decided.** Every statement is tagged *(P)* or *(S)*; the sketches are shapes of ideas put side by side so that the questions at the end can be answered, and the repository recommends none. The chain behind them: the PDP-11 terminal was a character stream (3.1), the Blit and the C64 put memory-mapped bitmaps, text, sprites and sound behind a processor or a few chips (3.3, 3.4), the PC split them (3.5), and the GPU turned drawing into queued commands and shaders (3.6); the sketches mix those inheritances.

**Two terms first. VRAM** (video memory) is memory that belongs to the display device: it sits on the device, behind a window on the bus, and the scanout reads it without crossing the bus; the processor reaches it through the window (slowly: every access is a bus transaction). The alternative is **system memory** that the device reads by DMA as a bus master, as the video device of 3.11.2 does; the framebuffer is then ordinary memory and the scanout competes with the processor for it. A GPU card has VRAM; the first board of the pdp64 has none to speak of (the block RAM is 5.5 megabits and a frame is 8.3 MB, 3.11.2), so on it every layer lives in system memory, and "VRAM" in the sketches below means "memory the device owns the use of", whichever kind it is. **Compositing** is combining several pictures into the one that is shown; the C64's sprites over text over a background were compositing in hardware, as the scanout read them out.

The device in these sketches is not specified beyond the 48-bit pixel and sample formats (errata 73 and 74). The properties the author has asked for, which each sketch is measured against: **(T)** simple peek/poke text and graphics at once; **(C)** modern colour depth and screen size; **(G)** glyph slots filled by a TrueType engine; **(S)** sprites, colour and sound in the C64's spirit; **(B)** the Blit's bit-blit and windows; **(U)** the GPU's command queue, parallel lanes and shaders. And the other two wishes of the day: anti-tearing sugar (3.15) and the boilerplate goal (3.14).

**sketches 1 and 2: layers in video memory, composed by the display hardware as it reads them out**




Sketch 1: the CPU writes the arrays; the display hardware does the composing; there is no command queue.
Sketch 2: the same, plus a few registers that move rectangles inside video memory (scrolling, window moves, glyph copies).
Nothing here is decided (3.21); the boxes are the shape of the idea.

***Sketch 1: layered scanout, no command queue***

**3.21.1** (moved from chapter 3) Three **layers** (a text-cell layer, a pixel layer and a sprite layer) are plain arrays in video memory that the processor pokes; the display hardware composes them as it reads them out, one pixel at a time, and there is no command queue. *(P)*

*What the programmer sees.* Three arrays, each at a fixed place, each with a small set of registers (base, size, stride, enable). The text array is the cell array of 3.9, with its glyph slots in another array; the pixel array is a framebuffer; the sprite layer is a table of entries (x, y, shape, colour, enable). A program pokes whichever it wants, in any order. *What is in video memory:* the cells, the glyph slots, the pixels, the sprite table and the sprite shapes. *At scanout:* for each pixel the hardware asks the layers from the top down (sprites, then text, then pixels), looks the cell up to its glyph, takes the glyph's coverage and the colours, and gives the first layer that is not transparent; the sprite layer needs to know only a few entries per row. The composing is done in a **line buffer** (a small memory that holds one row), as 3.11.2 already provides for. *Costs:* a compositor of the order of the 1,500 lookup tables the repository estimates for the blitter, plus a line buffer per layer (an estimate of mine, unmeasured); scanout traffic is the pixel layer's 498 MB/s at 1920 by 1080, 4 bytes, 60 Hz (746 MB/s at 6 bytes), plus about 13 MB/s for the cells and the glyph reads (arithmetic from the examples of 3.9). *Gives up:* anything beyond what the layers can say: no scaling or rotation, no effect that is not in the compositor; every change a program wants to make to the pixel layer is the processor writing pixels. *Properties kept:* T yes (this is the C64 and the Blit's memory-mapped screen); C yes; G yes (the text layer's glyphs); S yes (sprites, and colour per cell; sound is a separate device, 3.8.3); B no (no blitter, windows are the program's job); U no. *Text and graphics split:* healed, the C64's way: a program has text and pixels at once in one frame, with no mode switch. *A game:* sprites and a scrolling text or pixel layer, moved with a few pokes a frame; the C64 game, with a modern screen. *A terminal emulator:* pokes cells (a scroll is a few stores, or, with a base register for the layer, one store); the glyph slots are filled by the TrueType engine. *Sugar:* per-layer flip and a hardware cursor for free (3.15); the fence is the vertical-blank counter. *With virtio across the board:* needs a virtio device type of its own (reading 1 of 3.9); the layers are a device's contents, not the virtio-gpu protocol.

### *Sketch 2: the same, plus a small blitter and a glyph atlas*

**3.21.2** (moved from chapter 3) Sketch 1 plus a **blitter**, a few registers that copy and combine rectangles inside video memory (the Blit's RasterOp, the Amiga's blitter; the register sketch of 3.8.4), and a glyph atlas that the TrueType engine fills and the text layer indexes. *(P)*

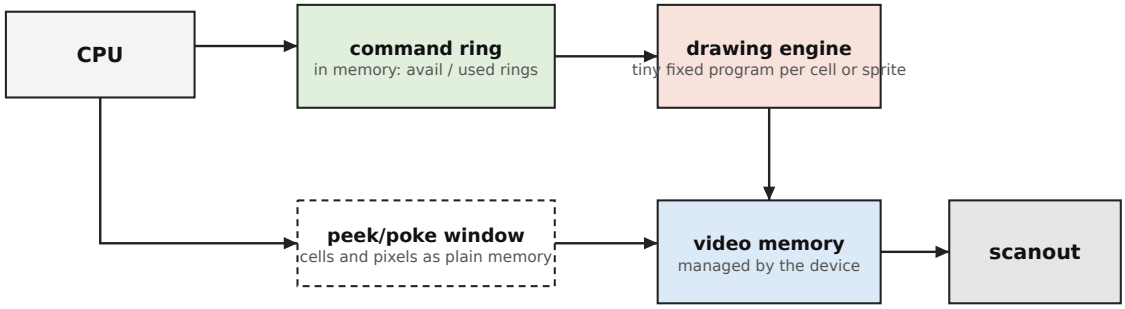
*What the programmer sees.* Sketch 1, and a handful of blitter registers: source, destination, strides, width, height, raster operation, go. The atlas is the glyph map seen as one picture (3.7): a glyph is drawn from it by the blitter into the pixel layer when a program wants a glyph on the pixel layer at an arbitrary position, and the text layer's cells point at it directly. *In video memory:* sketch 1's contents and the atlas. *At scanout:* as sketch 1. The blitter runs beside it, in the blanking time or contending for the memory. *Costs:* the blitter and its arbitration with scanout; the repository's own estimate is about 1,500 lookup tables and a line buffer, "nearly all address arithmetic" (3.8 commentary), and a bus-master port on video memory. *Gives up:* the same as sketch 1, except that drawing, scrolling and window moves in the pixel layer no longer cost the processor the pixels. *Properties kept:* T, C, G, S as sketch 1; **B yes** (bit-blit with the raster operations of 3.8.7, and windows as layers or as rectangles in the pixel layer); U no. *Split:* healed as sketch 1, and the pixel layer gains the Blit's operations, so that text on it (a window of a program that draws its own glyphs) is cheap. *A game:* sprites for the actors and the blitter for the background tiles. *A terminal emulator:* the text layer as in sketch 1; the blitter scrolls and moves windows drawn on the pixel layer. *Sugar:* as sketch 1, plus a blit that starts at the vertical blank (the "start-at-vblank" of `architecture-recommendations.md` R-9), which stops a copy from tearing. *With virtio:* as sketch 1, the blitter a feature of the device; it would be a feature bit and a queue if its commands are descriptors rather than registers (the chat chose a three-operand `draw` command, `decisions-pending.md` E3).

```
# sketch 2: scroll the pixel layer up by one text line, then draw a glyph from the atlas (placeholders)
poke(BLT_SRC, FB + LINE_H * STRIDE);  poke(BLT_DST, FB)
poke(BLT_STRIDE, STRIDE);  poke(BLT_W, W);  poke(BLT_H, H - LINE_H)
poke(BLT_OP, STORE);  poke(BLT_GO, 1);  wait_until(peek(BLT_BUSY) == 0)
```

### *Sketch 3: a command-queue device with a peek/poke window*

**3.21.3** (moved from chapter 3) A device whose main interface is a **ring of commands** in the manner of virtio-gpu, with a tiny shader-like fixed program for drawing glyphs and sprites and video memory managed by the device, and a peek/poke compatibility window on top for the programs that want to poke cells and pixels. *(P)*

**sketch 3: commands in a ring, a small drawing engine, and a peek/poke window on top**




The program mostly writes commands (put this text, move this sprite); the window is for the cases where a poke is simpler.
The window and the engine both reach video memory, so the device must say which one wins when they touch the same cell.

Nothing here is decided (3.21).

*What the programmer sees.* Mostly commands: "put this text at this cell", "move this sprite", "fill this rectangle", "present"; and, as a convenience, the window: cells and pixels as plain memory that mirrors the device's. The device allocates its own video memory, so a program does not choose addresses. *In video memory:* what the device decides; the program sees handles. *At scanout:* the device renders the commands into whatever it scans out, at its own pace, and flips at the blank by itself (the sugar is free: a commit is a command). *Costs:* a command processor (a queue engine, as the virtio devices of chapter 2 already are), a drawing engine of fixed programs (the small piece of the GPU that the terminal needs: a textured quad with a glyph

and two colours, a sprite), a memory manager for video memory and a consistency rule between the window and the queue; more gates than sketches 1 and 2 by a clear margin (a judgement of mine, unmeasured), and more software in the driver. *Gives up:* the plainness of sketch 1; the program must know the device's model, and two ways to reach one cell must be reconciled. *Properties kept:* T only through the window (and only as well as the window is made); C yes; G yes (the engine reads glyph slots); S yes; B no blit, but "fill" and "copy" commands do its job; **U yes**, a command queue and parallel work, but a *fixed* program, not the general shaders of 3.6. *Split:* healed by the engine, which draws text and pixels from one queue. *A game:* a command per sprite or per tile, one "present" per frame, with little processor work. *A terminal emulator:* one command per changed run of cells; the glyph atlas as in 3.7, but made by the device. *Sugar:* the fence and atomic commit are the natural form (each command completes in order; a commit is a command); versioned buffers are the device's business. *With virtio:* the natural fit, a device with queues as in 2.7, and the closest in spirit to virtio-gpu, of which it could be a profile (reading 2 of 3.9).

***Sketch 4: virtio-gpu 2D, the sugar in a library***

**3.21.4** (moved from chapter 3) The standard virtio-gpu 2D route of 3.12, with the C64 and Blit conveniences done in software by a library (cells, glyphs, sprites and cursor, blits, flips), and no special hardware. *(P)*

*What the programmer sees.* A library that offers the peek/poke look: a cell array in memory, `put_char`, sprites as library objects, and a `present` call that does the transfer and the flush for the damaged rectangles. *In video memory:* the host's resources; on the pdp64 the program's memory is the backing (3.14.3). *At scanout:* the host reads its resource when it likes; the library cannot flip at a blank because virtio-gpu has none (3.12); tearing is the host's concern and, on an emulator, usually invisible. *Costs:* nothing in gates; the processor draws everything and a flush copies; bus traffic is the damage rectangles. The glyph slots are a library table, filled from the TrueType engine on the processor. *Gives up:* everything hardware: the hardware cursor is the cursor queue only, there are no sprites except the cursor, no blits, no layers; the C64's "the chip draws it while you do other things" is the library's loop. *Properties kept:* T only through the library (the program pokes memory the library owns, and the library presents it); C only to 32 bits (3.14.4); G yes, in software; S no (the cursor excepted); B in software; U no. *Split:* healed in the library's picture, but each screen is still one pixel framebuffer. *A game:* draws into a back buffer and flushes it, in the PC way of 3.5 (slow, but running now). *A terminal emulator:* the library draws glyphs from a cache and flushes the damaged rows. *Sugar:* damage rectangles and the fence only (3.15, 3.12). *With virtio:* it is the standard (errata 79, 95); sketches 1 to 3 would be added to it as another device or as a profile.

***Sketch 5: a c64-mode core as the terminal (a variant)***

**3.21.5** (moved from chapter 3) A small core with the C64 family's chips, into which the host loads the terminal program by DMA (3.16), as a variant or addition to any of the sketches above: it is the terminal, and the other sketches say what the display behind it does. *(P)*

*What the programmer sees.* The host programmer sees a device that takes messages (the mailbox); the programmer of the small core sees the C64: peek, poke, a screen at a fixed place, colour RAM, glyph slots, sprites and the SID, which is the author's "as simple and pleasurable as the c64 was" at its most literal. *In video memory:* the core's screen memory, colour RAM, glyph slots, sprite shapes. *At scanout:* the VIC (or its extension) composes text, bitmap and sprites as sketch 1 does; the flip is a register of the core's own. *Costs:* a core, its memory, a DMA loader and the chips (3.16, question 7); a second program to write, load and keep secure. *Gives up:* a single, standard programming model on the host's side; the host cannot poke a cell, only send a message. *Properties kept:* T yes, for the core's programmer; C if the chips are widened as the native devices were (3.8); G yes; S yes, with sound; B only if the chip family has a blitter; U no. *Split:* healed, in the core. *A game:* a C64 game, or one written for the core. *A terminal emulator:* the core *is* the terminal; the host sends text. *Sugar:* the VIC's latches and raster interrupt, which the core's program uses; nothing is hidden from it. *With virtio:* the open tension of 3.16, question 6.

***The four main sketches side by side (sketch 5 is a variant of any of them)***

| | S1 layers | S2 layers + blitter | S3 command queue + window | S4 virtio-gpu + library |
|---|---|---|---|---|
| What the program does | pokes arrays | pokes arrays, starts blits | writes commands (or pokes the window) | calls a library, which sends the commands of 3.12 |
| Who composes | display hardware, at scanout | display hardware, at scanout | the device's engine | the host, or the program |

| | S1 layers | S2 layers + blitter | S3 command queue + window | S4 virtio-gpu + library |
|---|---|---|---|---|
| Where glyphs live | slot array in video memory | atlas in video memory | the device's cache of slots | the library's table, in the program's memory |
| Sprites | hardware layer | hardware layer | engine primitive | no (cursor only) |
| Blitter | no | yes | fill/copy commands | in software |
| Commands or pokes | pokes | pokes and a few registers | commands, pokes as a window | commands under a library |
| Anti-tearing sugar | per-layer latch, cursor, raster | the same plus blit at blank | commit, fence, atomic | damage and fence only |
| Cost in gates (my estimate, unmeasured) | small: a compositor and line buffers | small to medium: add a blitter | larger: a queue engine, a drawing engine, a memory manager | none |
| Bus traffic | scanout, plus the pokes | the same, less processor traffic for blits | scanout, plus commands (small) | scanout is the host's; the program writes every pixel |
| Fits virtio everywhere (errata 95) | needs its own device type | needs its own device type | a profile of virtio-gpu, or its own | is virtio-gpu |
| Properties T C G S B U | T C G S, no B U | T C G S B, no U | C G S U; T by the window; B by commands | T by library; C only 32-bit; G in software |

## H.4 Chapter 4: Boot, the BIOS, the disk label and the GPT

### 4.3 What takes the place of a BIOS (moved from chapter 4)

**4.3.1b** (moved from chapter 4) The ROM brings up the machine, loads the first block, and offers a console monitor when nothing boots. It has no resident services and nothing is called after boot. *(P)* (The first sentence of 4.3.1 is in the body, decided by errata 47.)

*Commentary (moved).* **Trade-offs.** The pdp64 returns to the PDP-11 split and drops job 3: every device speaks virtio through one transport (chapter 2), so the loader and the kernel share one driver design and the loader needs no firmware to read a disk. The cost is a driver for the boot disk in the second stage, small because a virtio device is a handful of structures. **Benefits.** There is no firmware API to specify, no compatibility layer, no resident management code and no closed blob at the bottom of the trust chain (x86 firmware also keeps resident system-management code that the operating system cannot see, unverified; arm64 and RISC-V servers need trusted firmware and usually UEFI beside it, unverified, though OpenBSD's arm64 and riscv64 ports do boot through `efiboot`, read); the failure cases reduce to one, the first block did not load, and the monitor is the recovery. **Impact.** *Kernel:* no firmware services to call or to fix. *Compiler:* builds the ROM, loader and tools. *Hardware:* a boot ROM of a few hundred bytes to a few kilobytes, with a driver per boot device (not specified more closely).

### 4.4 The reset state and the first block (moved from chapter 4)

**The ROM's details (moved; its job as pseudocode, and 4.4.1, are in the body).** The ROM does five things in order: find a boot device, read block 0 into low memory, check that it looks like a boot block (the `0x55AA` signature), set core 0's registers, and start core 0 at the block. The assistant's fuller pseudocode, with the register state:

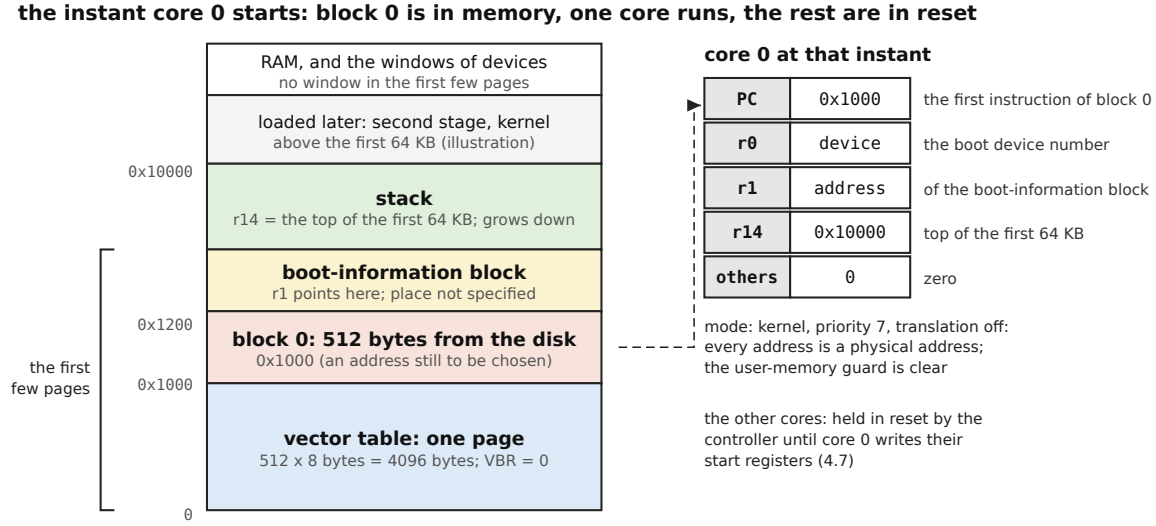
```
quick_memory_check()                    # a short test of the first pages (repo: `boot.md`, job 1)
for dev in boot_order:                  # disk or flash, then SD card, then the serial line (4.4.1)
    ...
    if byte[LOAD + 510] != 0x55 or byte[LOAD + 511] != 0xAA: continue
    core0.mode = KERNEL; core0.priority = 7; core0.translation = off; core0.VBR = 0
    core0.guard = clear                 # the user-memory guard
    core0.r0 = number_of(dev); core0.r1 = info; core0.r14 = 0x10000
    core0.all_other_registers = 0
```

read_block (the standard disk: one virtio-blk request, 5.6.2): build a three-descriptor chain (request header, a 512-byte buffer at `address`, a one-byte status); FENCE; write the queue's doorbell; poll the used ring until the chain returns.

The ROM polls (checks in a loop) rather than using interrupts. Inference, not stated: the ROM runs before any software has programmed the interrupt controller (2.6.3: how the interrupt controller is programmed is not specified), so there is nothing to interrupt it; polling needs no set-up, and 2.7.8 shows the polled form. It needs a driver for each kind of boot device; for the standard disk that is the virtio-blk reader of chapter 5: find the function in the bus's configuration space, set

up one queue, read one block. Which device comes first, and how the ROM names a device in `r0`, are not specified (F10 of `decisions-pending.md` leaves the order after the disk open).

**The reset state of core 0 and the load address (moved from the paragraph "The first few pages of memory").** Block 0 is loaded **above** the table, at 0x1000 in this chapter's example (the address is "to be chosen"; `docs/boot.md`), so that the first stage runs without overwriting the vectors. The stack pointer `r14` starts at the top of the first 64 KB, so the first stage has a stack before it has run an instruction. `r1` points at the **boot-information block** (4.6). The author's rule of errata 102 is that no device window is mapped in **the first few pages**, where "the boot stuff is loaded"; how many pages that is, is not decided.




0x1000 and the 64 KB stack top follow the proposal of 4.4 and the figure of chapter 6; how many pages are reserved is not decided (errata 102).
Not to scale. The ROM itself is not in this memory: it sits in the controller chip, outside the address space the kernel uses.
On the PDP-11 the vectors sat at 0 to 0377 and the bootstrap was loaded elsewhere, often high in memory (4.1).

**4.4.1** (moved from chapter 4) At reset the controller brings up memory, holds every core in reset, and looks for a boot device in a fixed order: disk or flash, then SD card, then the serial line. *(P)*

**4.4.2b** (moved from chapter 4) The ROM reads block 0 of the first device that answers (for the standard disk, a virtio-blk read request), checks the `0x55AA` signature in the last two bytes, and does not parse a partition table. *(P)* (That the ROM reads block 0 and starts core 0 is in the body.)

**4.4.3** (moved from chapter 4) Core 0 starts in kernel mode at priority 7 with translation off (every address physical), `VBR` zero and the user-memory guard clear; `r0` holds the boot device number, `r1` the address of the boot-information block, `r14` the top of the first 64 KB of memory, and the other registers are zero. *(P)* The first few pages of the physical address space are reserved for the boot code and boot data, and no device window is mapped there (2.3.5, *(D)*); how many pages is not decided. *(P)*

**4.4.4** (moved from chapter 4) If a key is held at power-up, the monitor runs on the serial console. *(P)* (That the monitor runs when nothing boots is 4.10.1 in the body.)

*Commentary (moved).* **Trade-offs.** A boot ROM with a driver per boot device is more code in the controller than a "jump to a fixed flash address" scheme and much less than firmware; the first block must fit in one sector. **Benefits.** The first stage is the same on every board, and the first kernel code makes no guesses about the reset state, which earlier machines left to the console (inference: the VAX console and the PC's BIOS each defined their own). **Impact.** *Compiler:* the first block is 446 bytes of assembler. *Hardware:* the ROM reads one block and starts one core.

## 4.5 The disk: a GPT around the BSD label (moved from chapter 4)

**The first stage lives in the protective MBR.** Block 0 of a GPT disk is the protective MBR, 512 bytes: 446 bytes of code, the table (one entry, type `0xEE`), then `0xAA55` (the bytes `0x55`, `0xAA`: the `0x55AA` of 4.4.1). The first stage has 446 bytes. It cannot understand a GPT or a file system in that space and does not try: it carries a **block list**, three numbers per piece of the second stage (first block, number of blocks, load address), written into its code area by the **`installboot`** tool at installation; it

reads those blocks and jumps to the first. The BSDs already do this. OpenBSD 4.6's `installboot(8)` page for the sparc says the first-stage program "loads the second stage program from a set of filesystem block numbers that have been hard-coded into it by the installboot program"; on the PC `installboot` patches `biosboot` with the location of `/boot`, and `biosboot(8)` says to re-run it whenever `boot(8)` is changed, "as its inode may change" (read). Because the boot-code bytes are the only place the pdp64 keeps block-list data, a damaged GPT cannot stop the machine reaching the second stage, which can repair it.

```
first_stage() with the block list (the assistant's sketch; the body has the general form):   # at LOAD
(0x1000): 446 bytes of code plus the block list
    # entering state: kernel mode, priority 7, r0 = boot device, r1 = boot-information block, r14 = a stack
    for entry in block_list:                 # entries written by installboot: (start, count, address)
        read_blocks(r0, entry.start, entry.count, entry.address)     # (see the open question below)
        if failed: goto monitor_or_halt      # what to do on failure is not specified
    jump to block_list[0].address            # the second stage, with r0 and r1 unchanged
```

The pseudocode hides a question the repository does not answer: **how the first stage reads blocks.** 4.3.1 says the ROM offers no services after boot, and a virtio-blk read needs a queue, three descriptors, a doorbell and a poll (4.4). Whether that fits in 446 bytes beside the list is not established. The PDP-11 primary bootstrap carried its own disk driver (the `hpcs1` and `hpcs2` writes of 4.1) and the PC's called the BIOS. Three ways out, none decided: the first stage holds a small polled virtio-blk reader of its own (the PDP-11 way, consistent with 4.3.1); or the ROM leaves a read routine and tells the first stage where it is (a firmware call in miniature, against 4.3.1); or the ROM loads more than block 0, the whole 8 KB boot area of 4.5.3 for example, so the first stage has room for the reader. The question is in 4.13.

**4.5.2b** (moved from chapter 4) The first stage carries a short block list (start block, count, load address) to the second stage; neither the ROM nor the first stage parses the GPT. *(P)* (Decision B8, a proposal. That the first stage lives in the 446 bytes of the protective MBR is 4.5.2 in the body.)

From the paragraph on the superblock: The second stage can live in the boot area, in a small boot partition, or as a file in the root file system; whether a dedicated boot partition is needed is open (4.13), since the block list can point at any of them.

From the commentary: the benefit "The first stage does not depend on a GPT that may be damaged."

## 4.6 The second stage and the hand-off (moved from chapter 4)

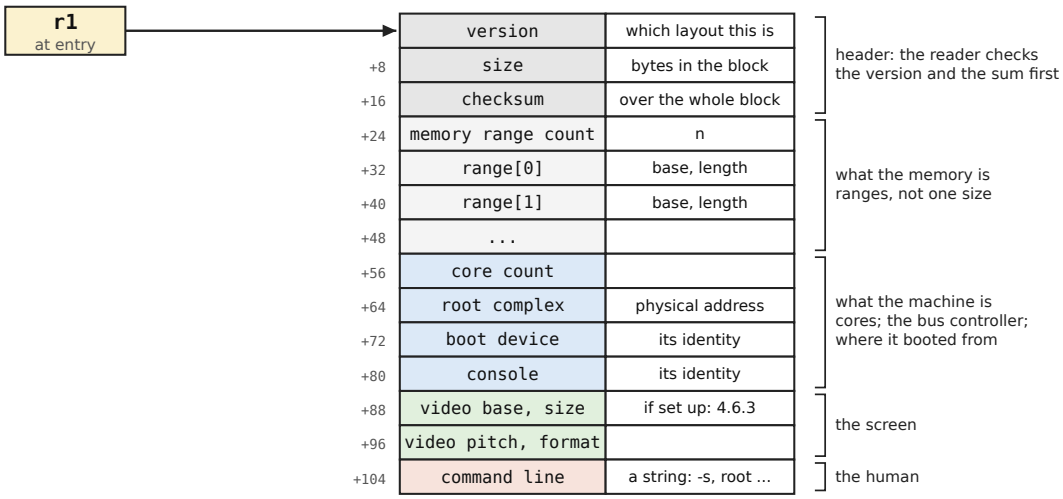
**What the second stage is for (the fuller sketch; the general form is in the body).** The first stage put a bigger program in memory. The second stage does what the PDP-11's `boot` did, with more to read: it finds the disk, reads the partition table, reads the label, finds the root file system, finds the kernel by name in it, loads the kernel and enters it. Because it is a C program loaded from disk it can be as large as it needs and be replaced by copying a file. It has its own drivers (the virtio-blk reader, now complete) and no firmware to call. As a sketch:

```
second_stage():                              # loaded by the first stage; r0 and r1 as at reset
    dev = open_boot_device(r0)               # the same virtio-blk driver the ROM used, now complete
    gpt = read_gpt(dev)                      # header at block 1: check "EFI PART" and the CRC32;
                                             # if it is bad, use the backup at the end of the disk
    part = first entry whose type GUID is the OpenBSD type
    label = read(dev, part.start + 1)        # sector 1 of the partition (4.5.3)
    check label.magic == 0x82564557 and the label's checksum
    root = the label's root partition ('a'); mount it read-only
    cmdline = prompt("boot> ") or the default   # as the V7 ':' prompt: which kernel, which options
    elf = root.open("/bsd")                  # an ELF file, the toolchain's own object format
    for seg in elf.loadable_segments:
        copy seg.file_bytes to physical address seg.address; zero the rest of seg
    info = r1                                # the block the ROM began; add what only we know:
    info.cmdline = cmdline
    info.video = framebuffer if we set one up (4.6.3)
    info.checksum = checksum(info)
    jump elf.entry with r0 = boot device, r1 = info     # still kernel mode, priority 7, translation off
```

**The pdp64's hand-off (moved from the hand-off paragraph).** A kernel that is to run on any pdp64 has to be told: how much memory there is and where (ranges, since memory is not always one block), how many cores, where the bus's root complex is (so that it can **enumerate**, walk the bus and list what is attached, 2.4), which device it was loaded from, where the console is, and the command line. The block is addressed by `r1`. The pdp64's hand-off is small because the bus describes

its own devices: the block names memory, cores, the root complex, the boot device, the console and the screen if one was set up, and the kernel finds everything else by enumeration.

**the boot-information block: a struct the second stage fills in and r1 points at**




Each row is one 64-bit word in this drawing (the offsets are bytes). The contents are statements 4.6.2 and 4.6.3; the offsets, the widths and the order drawn are an example, not specified.

Defined once in a grammar from which the headers are generated, so the loader and the kernel cannot disagree (4.6.3). The devices are found by enumerating the bus (2.4), so the block stays short.

The VAX and Alpha passed similar blocks, a restart parameter block each (from the boot review, unverified, 4.12).

**4.6.1b** (moved from chapter 4) The second stage is a program in C that understands the root file system, reads the GPT and the label, loads `/bsd` (an ELF file) and jumps to it with `r0` and `r1` as before and a command line in the boot-information block. *(P)* (That the second stage carries its own drivers, finds the kernel and enters it is 4.6.1 in the body.)

**4.6.2b** (moved from chapter 4) The boot-information block names the memory ranges, the core count, the address of the root complex and the identity of the boot device and the console; the kernel enumerates every other device by PCI Express enumeration (section 2.4). *(P)*

**4.6.3** (moved from chapter 4) The block's layout is defined once, in a grammar from which the headers are generated, and carries a checksum and a version. *(P)*

*Commentary (moved).* **Trade-offs.** A fixed-layout block with a checksum has two failure cases (bad checksum, unknown version), against the parser a device tree or ACPI needs. OpenBSD's arm64 and riscv64 ports already parse a device tree and attach drivers from it, which could save kernel code if the port reuses many of their drivers (4.13). **Benefits.** Because the bus describes its own devices, the block shrinks to memory, cores and the bus controller's address, and the generated header cannot disagree with the loader. **Impact.** *Kernel:* reads a structure. *Compiler:* generates the header from one grammar. *Hardware:* none.

## 4.7 Starting the other cores (moved from chapter 4)

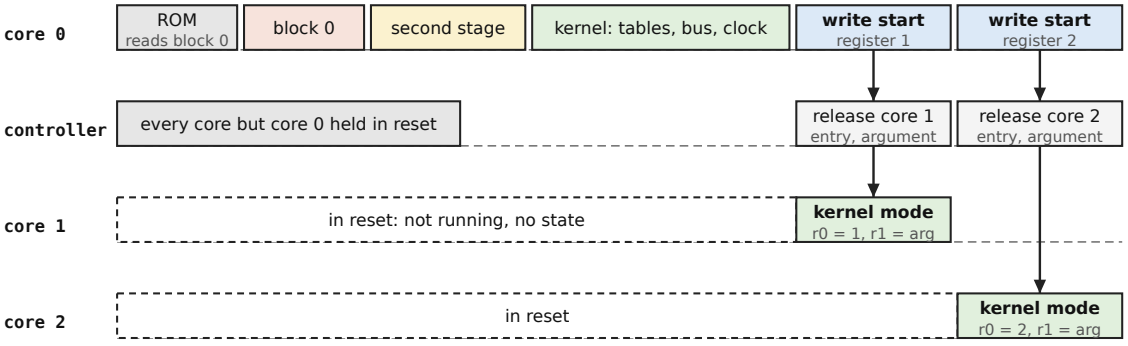
**The pdp64's rule: one register write.** A pdp64 core has no mode ladder, so it can start in its normal mode at any address. The controller holds every core but core 0 in reset; to start core *n*, the kernel writes, to a register block in the controller that belongs to core *n*, the address where the core should begin and one argument, and the controller releases the core into **kernel mode at priority 7** with its core number in `r0` and the argument in `r1`. By the kernel's convention the argument points to a structure that belongs to that core (its stack, its per-CPU data). A core that sticks is reset by a write to a core-reset register and started again.

```
start_secondary_cores():                    # core 0, in the kernel, after its tables are built
    for c in 1 .. core_count - 1:           # core_count comes from the boot-information block
        percpu[c] = allocate(percpu_size); percpu[c].stack = allocate(KSTACK)
        percpu[c].online = 0
        write(start[c].entry, secondary_entry)   # the exact registers are not specified (4.7.1)
        write(start[c].argument, percpu[c])
        write(start[c].go, 1)                    # the controller releases core c into kernel mode
```

```
        wait up to a timeout until percpu[c].online == 1
        if not online:
            write(core_reset[c], 1)             # 4.7.2: stop it, and try once more
            ... start it again, or mark core c failed and go on without it

secondary_entry():                          # core c: kernel mode, priority 7; r0 = c, r1 = percpu[c]
    set VBR and the page-table base to the kernel's, turn translation on, ISYNC    # chapter 6 (6.12)
    set the per-CPU pointer DP from r1; set R14 = percpu.stack top                 # chapter 7 (7.8.2)
    percpu.online = 1                       # core 0 is polling this word
    enable the clock; enter the scheduler's idle loop (WFE until there is work)    # chapter 6, 7.10
```

**starting the other cores: one register write each, then the core is a core**




time runs to the right (not to scale); priority 7 on entry. The entry address is where the kernel's per-core code starts; the argument is a pointer to that core's own structure. A core that sticks is reset by a write to a core-reset register of the controller and started again (4.7.2).

x86, for comparison (general knowledge and Linux smpboot.c, 4.7): the boot core sends INIT and START-UP signals through the interrupt controller; the other core wakes in 16-bit real mode at an address below 1 MB, and a trampoline must walk it through protected mode into 64-bit mode.

The pdp64 has no such ladder: a core starts in its normal mode at any address. What the start registers look like is proposed, not decided (4.7.1); the register block's address is open (4.13); the SMP proposal (H.7, 7.8.1) names r0 only, where 4.7.1 loads r1 too.

The cost is one register block in the controller, with a start register and a reset hold per core. The started core has no translation or stack of its own; the kernel's entry code sets them up in its first instructions, as the first kernel does (6.12). Whether a started core begins with translation off, and what `MMUCTL` it sees, are not specified.

**4.7.1b** (moved from chapter 4) (4.7.1 of the body says only that the controller chip provides the way to start the other cores.) The controller holds every core except core 0 in reset; a write to a core's start register in the controller gives it an entry address and an argument and releases it into kernel mode at priority 7, with its core number in `r0` and the argument in `r1`. *(P)*

**4.7.2** (moved from chapter 4) A stuck core is reset by a write to a core-reset register. *(P)*

*Other chapters.* This is option 1 of boot decision B2 and of R2 in `decisions-pending.md`, which recommend it and leave it proposed (the other options were the x86 signals, a firmware call, a spin table and a table that all cores start from). Chapter 7 stated the same rule (H.7, 7.8.1) and loaded `r0` only, where 4.7.1 loads `r1` too; the difference is to settle (4.13). Waiting uses `WFE` and `SEV` (7.10); an **inter-processor interrupt**, a signal from one core to another, is a write to a per-core register of the interrupt controller (H.7, 7.8.2).

*Commentary.* **Chain.** PDP-11: one processor, nothing to start. VAX: the console processor started the others (unverified). x86: INIT and START-UP signals and a trampoline through three modes. arm64 and RISC-V: a firmware call or a spin loop. pdp64: a register write. **Trade-offs.** With no mode ladder the whole protocol is one write, at the cost of a small register block in the controller. **Benefits.** No trampoline, no firmware call, fewer lines than any system counted, and one recovery path, a core reset. **Impact.** *Kernel:* starting a core is a register write. *Compiler:* none. *Hardware:* a start register and a reset hold per core.

## 4.8 Starting the devices (moved from chapter 4)

**The INIT bit (the sentences moved).** Statement 4.8.1 keeps `INIT` and `READY`. At boot the ROM runs the device start for the boot disk (4.4), the second stage again for the devices it uses, and each kernel driver at attach.

**Firmware for a device.** Some devices contain a small processor and need a program, **firmware**, loaded into them before they work; a network or wireless card is the usual case, a graphics device another. The PDP-11 had no such device and no way to give it one. On a PC the driver loads the blob from a file at attach time (Linux's `request_firmware`, declared in `include/linux/firmware.h`; OpenBSD's `loadfirmware(9)`, which reads the file from `/etc/firmware`; NetBSD's `firmload(9)`: all read), and the files come in a separate package. The pdp64 proposal is that such a device is given its firmware through a descriptor operation (a request on the device's queue, or a descriptor giving the address and length of an image in memory, with a small header the device checks: identifier, version, length, checksum) **once the root file system is available**, and that a device the boot depends on needs none: the boot disk, the console and the interrupt controller work from their built-in logic alone, so that no device waits for firmware that is on the disk it controls.

```
start_device(dev):                          # at boot or at attach
    init_device(dev)                        # the handshake of 2.7.3; reset first
    if dev.needs_firmware:                  # a device the boot depends on never does
        image = read_file(root_fs, firmware_name(dev))      # the file system must be up (4.6)
        check image.header: device id, version, length, checksum
        submit a descriptor chain giving dev the address and length of image
        wait for the device to answer; if it refuses, mark the device failed
```

**4.8.1b** (moved from chapter 4) A device that wants firmware is given it through a descriptor operation once the root file system is available, and a device the boot depends on needs no firmware. *(P)* (That a device is initialised by a register write behind the bus, and may be given values or a firmware image, is 4.8.1 in the body.)

*How the statement reads under virtio.* 4.8.1 is the PDP-11 form, one `INIT` bit and one `READY` bit (errata 29); under virtio (errata 95) it is the `device_status` handshake above. How a virtio device is handed firmware is not specified: virtio has no firmware operation of its own, so a request type or a separate device would have to be defined (4.13).

*Commentary.* **Chain.** PDP-11: the CSR's `INIT` bit, nothing more. PC: PCI Express devices initialised through configuration space and per-device driver code, firmware blobs loaded from files. VAX: the console and some adapters loaded microcode from storage (unverified). pdp64: the virtio handshake, firmware by descriptor. **Trade-offs.** A reset bit and a status register make the simplest device, with one failure case, a timeout. A firmware window lets a device be cheap (no flash) and makes an update a file replacement, at the cost of a header format and a device that rejects a wrong image safely; boot-critical devices are excluded because a device whose firmware is on a disk it must itself read cannot start. NVMe has a firmware-download command that needs a working queue first (unverified; not found in the QEMU headers read). **Impact.** *Kernel:* a device start is the 2.7.3 handshake. *Compiler:* none. *Hardware:* a status register per device.

## 4.9 Why UEFI is not used (moved from chapter 4: the shim's details)

Statements 4.9.1 and 4.9.2 stay in the chapter. The details of a shim are proposed.

**What a shim is, and what it is not.** The author asked, "not sure why firmware and coff are needed for uefi, with virtio native that shuld be minimal, no?" and then, "yes, state the shim's real scope." Virtio removes most of the driver mass that a full firmware carries (it ships drivers for every disk, network card and screen it might meet; the pdp64 has one device interface), and the shim needs only the drivers the second stage has. Two things stay, and they are the shim's real scope:

1. **PE/COFF.** The UEFI specification requires a UEFI application to be a PE32+ image (unverified: the specification is not here); that is a property of the **loader programs**, not of the firmware. To run an existing loader such as OpenBSD's `efiboot` it must be built as a PE/COFF image *for the pdp64*, so the toolchain has to produce one, and a **machine-type code** (a 16-bit number in the image's header that names the processor; OpenBSD's arm64 loader uses `IMAGE_FILE_MACHINE_ARM64 0xaa64`, read) would have to be assigned by the owners of the specification (unverified). It need not be a linker feature: OpenBSD's arm64 loader is linked as an ordinary ELF file and wrapped as PE32+ by a header written by hand in assembler (`start.S`, 176 lines) plus a self-relocator (`self_reloc.c`, 124 lines), with `objcopy` doing the rest (read, counted). The loader is itself a program that must be compiled for the pdp64.
2. **A small services layer.** The shim provides the few services loaders use: memory allocation and the **memory map**; reading a file from the boot partition; the console and the **graphics output protocol** (a framebuffer to draw on); **exit boot services**; a runtime clock and reset; all over virtio. It also hands the kernel a device description in the form such a loader expects, a device tree or tables.

**How many services, counted.** The three UEFI loaders in `<work>/kernel-src/openbsd/sys/arch/*/stand/efiboot` call these (counted as `BS->`, `RS->` and console calls in the `.c` files; the number is the count of call sites in the amd64 loader):

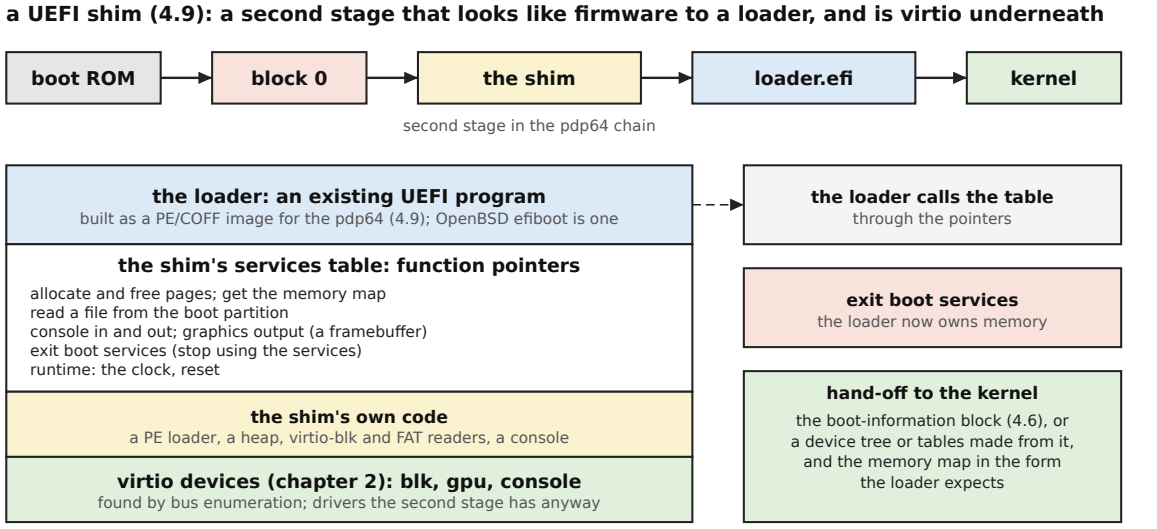
| Boot services (`BS->`) | Runtime services (`RS->`) | Console and file protocols (through a handle) |
|---|---|---|
| `HandleProtocol` 13, `AllocatePages` 7, `LocateHandle` 5, `LocateProtocol` 2, `LocateHandleBuffer` 2, `GetMemoryMap` 2, `FreePages` 2, `WaitForEvent`, `SetWatchdogTimer`, `FreePool`, `AllocatePool`, `Exit`, `ExitBootServices` (1 each); the arm64 and riscv64 loaders add `SetTimer`, `CreateEvent`, `CloseEvent` and lack the two pool calls | `ResetSystem` 3, `GetVariable` 3, `SetVariable` 2, `GetTime` 1 (called as `ST->RuntimeServices->GetTime`) | the text console (`OutputString`, `ReadKeyStroke`, `SetMode`, `QueryMode`, ...), the disk (`ReadBlocks`, `WriteBlocks`), files (`Open`, `Read`, `Write`, `Close`, `SetPosition`, `GetInfo`), the network (`Mtftp`, `Transmit`, `Receive`), the graphics output protocol, the random-number protocol |

That is 17 distinct boot and runtime calls in the amd64 loader (13 and 4) and 18 in the arm64 and riscv64 ones (14 and 4); errata 109 and `boot.md` say "about fifteen to seventeen", which this count corrects. It counts what **one** loader uses and is not a specification of what a shim must offer: another loader uses others, and whoever writes the shim reads the loader.

**A sketch of the shim (an illustration).** The shim is the second stage, loaded by block 0; it builds the services table, loads the loader's PE image, calls it, and answers its calls.

```
shim_entry():                                # the second stage of a pdp64 that wants UEFI loaders
    info = r1; heap = free_ranges_from(info.memory_ranges)
    table = new system_table {
        boot_services = {
            allocate_pages = shim_allocate_pages,   # out of the heap; the loader asks for a page count
            free_pages     = shim_free_pages,
            get_memory_map = shim_get_memory_map,   # the ranges of `info`, plus what the heap handed out
            locate_protocol = shim_locate_protocol, # the console, the graphics output, a file volume
            exit_boot_services = shim_exit_boot_services,
            ... }                                    # the others a given loader uses; unused ones return
"unsupported"
        runtime_services = { get_time = clock_register, reset_system = reset_register, ... }
        configuration_tables = [ (DEVICE_TREE_GUID, make_device_tree(info)) ]   # made from the boot-
information block
    }
    image = read_file(boot_partition, loader_path)  # the path convention is not specified
    entry = pe_load(image)                          # headers, sections, relocations
    status = call entry(image_handle, &table)       # the loader runs, and calls back through the table

shim_exit_boot_services(handle, map_key):
    if map_key != current_map_key: return INVALID_PARAMETER    # the map changed since the loader read it
    stop the shim's console and timer; free the memory of the boot-services data
    keep only the runtime services: the clock and reset are register writes
    return SUCCESS                                  # the loader owns the memory and enters the kernel
```




The loader needs the services table, a PE/COFF image of its own and a machine-type number for the pdp64 that the owners of the UEFI specification would have to assign (unverified, 4.12). The shim is a few services over drivers that already exist; it is not a firmware stack: no PEI, DXE or driver model, no variable store, no setup screen. This illustrates the size of the job (4.9).

The services named are those OpenBSD's efiboot loaders call (measured, 4.12): 17 (amd64) or 18 (arm64, riscv64) boot and runtime services, count
Which of them a given loader needs is for the person who writes the shim to find by reading that loader.

**The costs.** A shim is not a UEFI firmware stack. The boot review's figure of "hundreds of thousands to millions of lines" for a UEFI firmware (`docs/boot.md`, "Is UEFI worth it?") was never confirmed against a source, describes a full stack such as EDK II, and is not used as the cost of a shim. **No count of the shim's lines is given, because none can be measured here:** no UEFI implementation source is on this machine (the `edk2` and `u-boot` directories under the QEMU tree's `roms/` are empty). Measured are the loaders that would run on it (OpenBSD's `efiboot`: 4,593 lines for amd64, 7,195 for arm64, 4,513 for riscv64) and the 176 lines of header and 124 of relocator above. The costs that remain:

- **A PE/COFF build and a number.** The toolchain emits a PE/COFF image, and a machine-type number must be obtained (unverified).
- **A loader to port** to the pdp64 as any program is, the larger part of the work.
- **A services layer** written, tested and kept in step with the loaders it serves.
- **Code to trust before the kernel.** The shim is open source (4.3.2), runs only at boot and is not resident afterwards except for the clock and reset.
- **A platform description** (a device tree or tables) built from the boot-information block.

It does not cost a closed firmware, a variable store, a setup screen or a second boot path in the machine itself: the machine's own boot stays block 0.

*Commentary (moved).* The shim sentence: "A shim gives the first benefit without the rest, at the price of a services layer and a PE/COFF build." The impact lines: *Kernel:* a shim would translate the boot-information block. *Compiler:* no PE/COFF unless a shim is built.

## 4.10 Recovery (moved from chapter 4: what remains proposed)

**What to do when nothing boots.** A PDP-11 with a dead disk was not a dead machine: `ODT` let you examine memory and key a bootstrap in (4.1), and V7 shipped `/boot.bu`, a backup of the second stage. The pdp64 keeps the idea and gives it a place. The ROM that reads block 0 (4.4) also contains a **monitor**, a small program that runs on the serial console when no boot device answers or when a key is held at power-up and offers a few commands: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. It needs a working serial port and nothing else, so it also brings up a new board before any boot device works.

```
monitor():                                  # the ROM's console program; no interrupts, no memory but the
first pages
    loop:
```

```
        print("pdp64> ")
        line = read_line(serial)
        case first word of line:
            "x" addr [n]    : print n words at addr                 # examine (the PDP-11's `addr/`)
            "d" addr value  : store value at addr                   # deposit
            "r" [name value]: print or set a register of the stopped core
            "l"             : read a program from the serial line into memory   # load
            "g" addr        : start core 0 at addr (r0, r1 as in 4.4.3)
            "b" device      : try to boot from this device
            "c" n           : write core n's reset register (4.7.2)
            "z"             : write the root complex's reset register (2.9.4; what else it resets is not
specified)
            otherwise       : print("?")
```

The command letters and the prompt are an illustration. A **watchdog** is a hardware timer that counts down and restarts the machine unless software writes to it ("kicks" it) in time; a hung kernel stops kicking and is restarted. The PDP-11 had none. PC chipsets carry watchdog timers (QEMU models the ICH9 chipset's TCO timer in `hw/acpi/ich9_tco.c` and the Intel 6300ESB watchdog in `hw/watchdog/wdt_i6300esb.c`: read); whether PC firmware arms one is unverified. It needs about ten lines in the kernel. `decisions-pending.md` P7 asks for a bus-controller reset register and a watchdog **in place of** firmware calls, and the monitor's reset command writes that register.

**4.10.1b** (moved from chapter 4) The monitor is reached on the serial console and offers: examine and deposit memory and registers, load a program over the serial line, start at an address or from a named device, reset a core, reset the machine. *(P)* (That the controller's ROM includes a monitor is 4.10.1 in the body.)

**4.10.2** (moved from chapter 4) A hardware watchdog restarts the machine if the kernel hangs. *(P)*

*Commentary.* **Chain.** PDP-11: `ODT`, the front panel and `/boot.bu`. VAX: the console processor (unverified). PC and UEFI: a setup screen and rescue media (unverified). ARM and RISC-V boards: the console of U-Boot and the debug console of OpenSBI (unverified). pdp64: a monitor in ROM and a watchdog. **Trade-offs.** A monitor is a few hundred bytes of ROM (an estimate) and needs a serial port. **Benefits.** It is the recovery path that needs the least code. **Impact.** *Kernel:* the way in when nothing boots. *Compiler:* none. *Hardware:* a serial port and the ROM bytes.

### 4.13 Still open (moved from chapter 4: what is not an open item of the architecture)

The open items (a) to (i) of the former list are entries J.4.1 to J.4.8 of appendix J. The rest of the list:

**(j)** the monitor's commands, the serial speed and what is kept in non-volatile memory are now J.4.10; **(k)** the shim: the path of the loader it runs, the services it offers, the machine-type number (4.9), none to be settled until someone wants one.

**Facts still to check against their own documents:** the PDP-11 power-up behaviour and the M9312's ROM contents (only its contract with block 0 was read); the VAX console, VMB and the Alpha console's boot protocol (only the restart block was read); the C64 boot; the x86 reset vector and START-UP rules; PSCI, the SBI hart calls and the spin table as specified; what the UEFI specification requires of a loader and of firmware, including the machine-type registry; the size of a UEFI services layer, which only an implementation could give.

## H.5 Chapter 5: The device driver model

**5.3.2** (moved from chapter 5) A device's windows start on page boundaries and are whole pages. *(P)*

**5.10.2** (moved from chapter 5) A device addresses physical memory directly and the driver gives it physical addresses; until an address-translation unit for devices exists, every device must be able to address all of memory. *(P)*

**5.13.1** (moved from chapter 5) Because a device's windows are whole pages, a driver can run in user space with only those pages and its buffers mapped. *(P)*

**5.13.2** (moved from chapter 5) Until devices have an address-translation unit, a user-space driver can instruct its device to write anywhere in memory, so it is as trusted as the device. *(P)*

*In user space* (moved from 5.13). Microkernels and exokernels run drivers as ordinary programs where the hardware lets one device be mapped to one process (*unverified*); the PDP-11's Unibus gave every device a fixed address and no protection. Windows of whole pages make that possible here (5.13.1), and the IOMMU is what would make it safe: until it exists, a user-space driver is as trusted as its device (5.13.2). Linux `VFIO` and `UIO` and the DPDK libraries are the same idea and rely on an I/O translation unit (*unverified*). **Trade-offs.** A driver in user space can be tested, restarted and replaced without taking

the kernel with it; its cost is a crossing for each request and the trust above. The whole-page rule also costs address space and no memory, and lets the kernel give one device to one protection domain.

**5.14** (moved from chapter 5) A virtio driver could be exercised before the machine has them by compiling its logic for the host and running it against a software model of a virtqueue. *(P)*

**5.5.4** (moved from chapter 5, the second half) A device that sets its needs-reset bit, or that stops answering, is reset and its queues reprogrammed, and the driver resubmits the chains that are not in the used ring. *(P for the resubmission rule)*

**5.11.2** (moved from chapter 5) A device that is reset leaves its queues in memory untouched; the driver reprograms the queue registers and resubmits the chains that are not in the used ring. *(P)*

**5.11.4** (moved from chapter 5) Requests are repeatable where possible (reads, writes of the same data, sends, buffer posts), so that resending an unfinished chain is safe; a request that must not run twice is tracked by its class driver, which can tell "never started" from "started, outcome unknown" only by the request's own status. *(P)*

## 5.7 Illustrations of a console and a timer (moved from chapter 5)

These illustrate the fixed devices of the open entry J.5.1; they decide nothing.

A complete tiny console driver for a KL11-style console at a fixed address, an illustration (a virtio-console version would replace the three register writes with a transmit-queue post and keep the shape):

```
# fixed address in the present machine (0x8000001000)
CON = MAP_CONSOLE
RXCS = 0;  RXDB = 8;  TXCS = 16;  TXDB = 24  # four 64-bit registers
READY = 1 << 7;  IE = 1 << 6                 # as the KL11's bits
ring txq, rxq                                # circular byte buffers: the clist's job

# vectors 33, 34: con_rxint, con_txint (6.3.3)
con_attach():
    # receive interrupts on; transmit off for now
    mmio_write64(CON + RXCS, IE)

# the boot code and a panic: no queue, no interrupt
con_putc_polled(c):
    while (mmio_read64(CON + TXCS) & READY) == 0: spin
    mmio_write64(CON + TXDB, c)

con_write(buf, n):                           # from the tty layer, as cnstart's caller
    s = spl4()                               # keep con_txint out while txq changes
    for c in buf[0..n): put(txq, c)
    # interrupts while IE is set: starts output
    mmio_write64(CON + TXCS, IE)
    splx(s)

con_txint():                                 # vector 34, level 4
    if empty(txq): mmio_write64(CON + TXCS, 0)       # nothing left: interrupt off
    else:          mmio_write64(CON + TXDB, get(txq))

con_rxint():                                 # vector 33, level 4
    tty_input(mmio_read64(CON + RXDB))       # reading RXDB clears READY
```

A timer tick handler for a fixed counter-and-compare timer: the interrupt stays asserted while the counter is at or above the compare register, so the handler moves the compare register forward by one period. An illustration:

```
timer_attach():
    mmio_write64(TIMER + COMPARE, mmio_read64(TIMER + COUNTER) + PERIOD)
    mmio_write64(TIMER + CONTROL, ENABLE)    # bit 0

timer_intr():                                # vector 32, level 6
    # drops the request
    mmio_write64(TIMER + COMPARE, mmio_read64(TIMER + COMPARE) + PERIOD)
    hardclock()                              # ticks++, due callouts, accounting
```

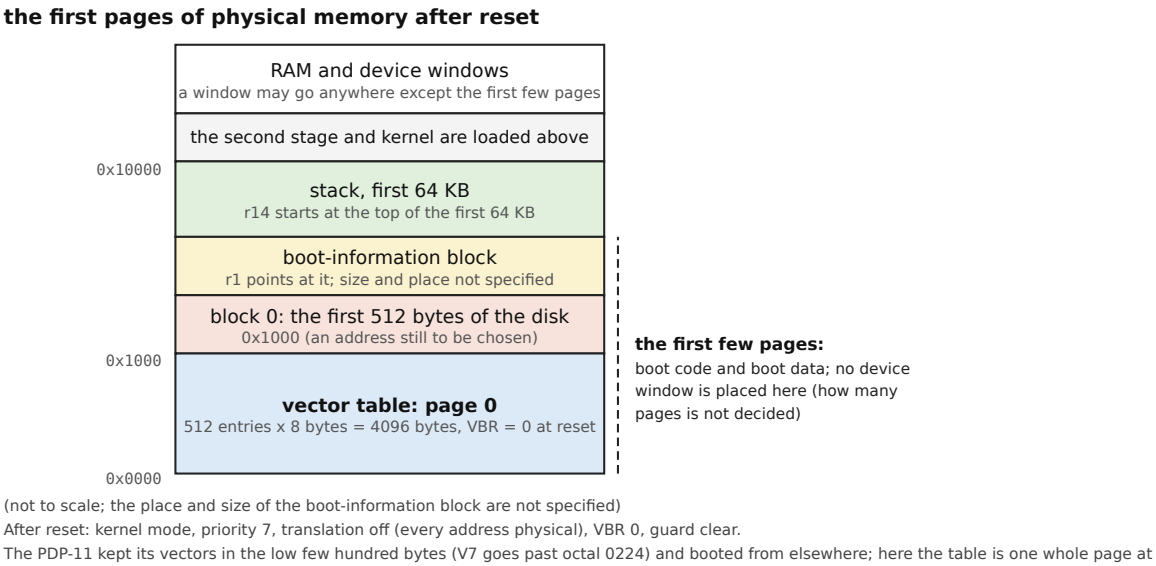
*Commentary.* **Chain.** PDP-11: the KL11 console has a receive and a transmit CSR with READY and IE bits and interrupts at BR4 on vectors 060 and 064; the KW11-L line clock interrupts at line frequency at BR6 on vector 0100 and `hardclock`

runs on each tick; both are at fixed addresses and need no probe (`cons.c`, `scb.s`, read). VAX: an interval timer in processor registers and a console processor (*unverified*). The PC: a UART at a legacy port and a programmable timer, then per-core timers; the ARM and RISC-V systems of today describe their UART and timer to the kernel in a device tree (*unverified*; OpenBSD has the `sys/dev/fdt` bus, read, and a virtio-over-memory-mapped-registers transport `virtio_mmio.c` for machines that have a device tree and no PCI). **Trade-offs.** A fixed console works before the bus does and costs a second device model; a virtio console is one model and cannot print before the queue is set up, unless the emergency write serves; a timer in the core removes a device and a lock and adds system registers, a counter distribution and an architectural definition. **Impact.** *Kernel:* the console must work before enumeration for the boot ROM and the panic path; the timer must work before any driver. *Compiler:* none for the console and entropy; a timer in system registers adds `MTSR` and `MFSR` numbers. *Hardware:* a fixed UART or timer is a few registers; a virtio console is a queue engine; an entropy source is a physical circuit whose quality the standard would promise.

## H.6 Chapter 6: The kernel: modes, exceptions and memory management

**6.8 (moved from chapter 6, the lineage and impact of the accessed and dirty bits)** Hardware-managed accessed and dirty bits, advertised in a feature register, are a possible extension of the software-managed bits that the body decides (6.8.1): an implementation would set the bits itself by writing the page-table entry, and a feature register would tell the kernel which form the machine has. The reasoning for the decided form, which is the assistant's and *unverified* (no document in the repository names a flaw in hardware-managed accessed and dirty updates): a walker that never writes cannot damage a table, and it stays a small read-only state machine. *(P)*

**6.12 (moved from chapter 6, the first pages of physical memory)** There is no kernel yet at power-on. A boot ROM (chapter 4) reads one block into low memory and starts core 0. The assistant's sketch of the first pages of physical memory (the body states only that they hold the boot code and boot data, with the vector table; errata 102) has the vector table, the block, the information block and the stack:




(not to scale; the place and size of the boot-information block are not specified)
After reset: kernel mode, priority 7, translation off (every address physical), VBR 0, guard clear.
The PDP-11 kept its vectors in the low few hundred bytes (V7 goes past octal 0224) and booted from elsewhere; here the table is one whole page at

The vector table is at address 0 because `VBR` is zero at reset. The pdp64's block 0, the 512 bytes the ROM reads from the disk, goes to **0x1000**, an address still marked "to be chosen" (`docs/boot.md`), so the first stage runs above the table. The **boot-information block** (memory ranges, core count, the address of the root complex, the boot device, the console, a checksum and a version, 4.6.2) is pointed to by `r1`; its size and place are not specified. **`r14`**, the stack pointer, is the top of the first 64 KB of memory. The 64 KB of `r14` is only the stack's place. *(P)*

## H.7 Chapter 7: Multiple processors

The numbers in this fragment are labels: they are those of chapter 7 before its rewrite (errata 127 gives the map), not those of the body, and a citation of one of them reads "H.7, 7.8.1".

### 7.8 Starting and signalling the other cores (moved from chapter 7)

**7.8.1** (moved from chapter 7) The controller holds every core but core 0 in reset; a write to a per-core start register gives a core an entry address and an argument and releases it into kernel mode at priority 7, with its number in `r0`. *(P)* The same proposal is statement 4.7.1 of the boot chapter (its fragment is the one to read for the start protocol, the pseudocode and the figure); it is entered here as well because chapter 7 stated it independently.

**7.8.2** (the mechanism, moved from chapter 7) An inter-processor interrupt is a write to a per-core register in the interrupt controller; no APIC-like machinery is needed. *(P)* (That one core can interrupt another is decided, 7.10.4 of the body; how interrupts reach a core is open: J.6.3.)

**7.8.3** (moved from chapter 7) The routing of device interrupts to cores, and a shared second-level cache or a coherent interconnect, are device-side and unspecified. *(P)* (The routing of interrupts is the open item J.6.3; the cache and the interconnect are implementation choices that the architecture does not fix.)

*Commentary on 7.8.1 to 7.8.3 (moved from chapter 7).* **Precedent.** An x86 core wakes in 16-bit real mode and has to be walked through three modes by a trampoline (1,528 lines of start-up code and 1,258 of real-mode code in Linux); arm64 uses a firmware call in 124 lines or a spin table in 127; RISC-V 108 or 58; the VAX's console processor started the others. **Benefits.** A core can start in its normal mode at any address, so the whole protocol is one register write. **Modern comparables.** x86's INIT and START-UP interrupts, ARM's PSCI, RISC-V's hart-start call. **Impact.** *Kernel:* Starting a core is one register write, and a stuck core is reset by another. *Compiler:* None. *Hardware:* A start register and a reset-hold per core in the controller.

### 7.9 A kernel's view (moved from chapter 7)

**7.9.1** (the first clause, moved from chapter 7) The kernel starts a core by writing its start register. *(P)*

**7.9.4** (the last clause, moved from chapter 7) It wakes an idle core with an inter-processor interrupt. *(P)*

### 7.2 and 7.11 Commentary (moved from chapter 7)

*Total store order as the retreat.* If a large out-of-order core is ever built, total store order is the least painful retreat from sequential consistency (from the commentary of 7.2). The decision is reversible in the direction that matters: total store order can be the retreat for a large core (from the commentary of 7.11). *(P)* The smp-decisions record leans to writing the retreat down now as a one-way, announced switch for the whole machine, with no per-core or per-page weaker mode.

### 7.11 The dedicated cores (moved from chapter 7)

**7.11.6** (moved from chapter 7) The machine has one core dedicated to the graphics device, one dedicated to the network interface and running the full TCP/IP stack, one dedicated to audio (MIDI, digital signal processing and the like), and a number of other cores for general processing. *(P)* Decided on 2026-11-04 (errata 76: "we recommend one core dedicated to the gpu/graphics device, one core dedicated to the nic, running the full tcp/ip stack, then a bunch of other cores for general processing"; "one core for audio processing, doing midi, dsp, etc."); shrunk by the decision of 2026-11-10 (errata 132): what remains of dedicated roles is open (J.7.5). The graphics core was the Blit's model (chapter 3); the network core drives a standard PCI Express card, so no network hardware beyond that is implied.

## H.8 Chapter 8: Vector work, AI and array languages: the device route

This fragment holds what the assistant proposed for the vector and accelerator work and the author has not accepted. The reader is the same as the chapter's: a PDP-11 programmer or a first-year student, who knows from 8.1 to 8.4 what a vector unit, a SIMD unit, a descriptor and a device on the memory bus are. The numbers are labels: a statement keeps the number it had in chapter 8 before the rewrite, and the pointer to its new place in the chapter is given where it has one. Since the author's decision that the vector and accelerator engine is a standard part of the architecture (8.4.1, errata 131), the proposals about the engine's operations, number formats and identification block are carried by the open items J.8.1 to J.8.5, and are no longer repeated here; what remains is the LB64 design, the scalable-vector remark, the interpreter threshold and the driver sequence.

### 8.2 Why not SIMD in the processor (moved from chapter 8)

The decision of the chapter (8.3.1) says that the processor has no vector register file and no SIMD instructions. Two further statements were attached to it as reasons, and they are the assistant's.

**8.2.3** (moved from chapter 8) A scalable design (ARM SVE, RISC-V V) avoids the fixed-width problem of 8.3.3, and is the most complex to specify and to implement; it is the only in-processor form that would suit this project's range, from a small FPGA core to a possible large one. *(P)* (It is option 3 of J.8.5.)

A **scalable** vector design is a vector machine (8.2) in which the register width is not part of the architecture: the hardware has registers of whatever length it can build, and a program asks it how many elements it can handle at a time and loops until the array is done (strip mining, with the length chosen by the machine and not by the programmer). The same binary runs on a small and a large implementation, which a fixed-width SIMD set cannot do. The cost is the largest specification in the architecture and a compiler model that must handle a length it does not know.

**8.2.4** (moved from chapter 8) The compiled corpus shows little vectorisable code at the size-optimising setting; whether it shows more at higher optimisation is a measurement to make before any in-processor form is reconsidered. *(P)*

*Commentary (moved from chapter 8).* **Chain.** PDP-11: no vector unit. VAX: a fixed 64-element vector length with a length register for partial vectors *(unverified here: from the repository's notes on the VAX 6000 guide)*. ARM and RISC-V: a scalable form after fixed-width NEON and the x86 history. **Trade-offs.** A scalable vector extension would let one binary run on a small and a large implementation, at the price named above. **Modern comparables.** SVE and the RISC-V vector extension. (Packed lanes in the integer registers, the other in-processor form, are the open item J.8.3.)

## 8.3 AI and large-language-model inference: the LB64 accelerator (moved from chapter 8)

The author decided that inference is the target (8.5.1 of the chapter). The design of one accelerator for it, LB64, is the assistant's, written in `docs/pdp64-lb64-accelerator.md` after the author asked "what would a gpu look like that is optimized purely for samsung's littlebit-2 encoding" (register row R136). No decision of the author names the design, and it was written for the register model of the first device family, which errata 95 later set aside for virtio. It is kept here as a proposal; the statements below, which the chapter had tagged decided, are entered as proposed for that reason. LB64 is option 1 of J.8.4, and, if the engine is made standard with optional sets (J.8.1, option 6), it would be one of those sets; that reading is the assistant's, and no decision of the author's says it.

LB64 is for one kind of model, in which the weights are stored as one sign bit each, with a few 16-bit scale factors per row and column. With weights of plus or minus one, a multiplication of an activation by a weight is the activation itself or its negative, so a layer needs adders and a sign flip and no multipliers: the accumulator adds or subtracts each activation according to its weight bit (**sign-flip accumulation**). The device streams the weights from RAM at the memory's speed.

**8.3.1** (moved from chapter 8) The inference accelerator, LB64, evaluates the layers of a binary-weight language model as sign-flip accumulates with no multipliers, taking a chain of descriptors and writing a status back on completion. *(D in the chapter; judged proposed because no decision of the author names the device)*

**8.3.2** (moved from chapter 8) A layer is two descriptors, the second accumulating into the first's result for the residual path, and a transformer block is a chain; the processor writes the chain once and is then idle. *(D in the chapter; judged proposed, as 8.3.1)*

**8.3.3** (moved from chapter 8) The work is partitioned between small engines, each doing one thing with its own local memory: LB64 for the projections and the feed-forward layers, a floating-point multiply-accumulate engine for the attention products, a vector engine for normalisation, softmax, activation and rotary position encoding, and the processor for the descriptor chains, sampling and the tokeniser. *(P)*

**8.3.4** (moved from chapter 8) The accelerator has no instruction set: its computation has exactly one shape, so a programmable sequencer would spend its area on flexibility the work does not use. *(D in the chapter; judged proposed, as 8.3.1)*

**8.3.5** (the last clause of the old statement, moved from chapter 8) `POPCNT` and `XNORPOP` are not what the accelerator uses: its activations are 16-bit floating-point numbers, and the processor's bit instructions (8.5.2 of the chapter) serve the binary-activation case.

*Commentary (moved from chapter 8).* **Chain.** PDP-11: heavy numerical work went to an attached array processor (8.1). Neural-network engines of today: a ring of commands and data by DMA (8.2). A ring of descriptors in memory, each with an ownership bit, is how the DEUNA Ethernet controller of 4.3BSD worked (`if_de.c`, chapter 5), and the partition into small single-purpose engines fed by chains is the idea of the 144-processor GA144 chip applied to a workload that suits it *(unverified)*. **Trade-offs.** The accelerator is fast at what it does and does nothing else: attention is not helped, the element-wise work needs another engine, and a training accelerator is a different design again. The accelerator's descriptor is larger than the 16-byte virtio descriptor of chapter 5 (5.4.2), so the two must be reconciled; how the two are reconciled is the open

item J.8.2, whose options include a device-defined work block that the ring points to and a command header in the first buffer of the chain, as virtio-blk and virtio-gpu do. **Benefits.** The lane count is set by the memory bandwidth and not by the amount of logic, so a small board can run a model the processor alone could not. **Modern comparables.** ARM's Ethos engines, Google's TPU, and the inference accelerators on small boards *(unverified)*. **Impact.** *Kernel:* a driver builds a chain and starts it; the kernel owns no model state and the only scheduling is of the device. *Compiler:* none; inference frameworks emit descriptors. *Hardware:* adders and sign-flip logic and no multipliers in LB64; the multiply-accumulate engine needs multiplier blocks; the memory interface sets the size.

### 8.4 Array languages: APL and K (moved from chapter 8)

**8.4.3** (moved from chapter 8; the length-in-the-descriptor part stayed as 8.5.4 of the chapter) The interpreter stays on the processor and sends a primitive to the device when the array is long enough to repay the setup; short arrays run on the processor. *(P)*

*Commentary (moved from chapter 8).* **Trade-offs.** Offloading a primitive has a fixed cost, so the benefit is for arrays above a size threshold that has not been measured; reductions and scans carry a dependence along the array that the device must support directly; sorting and gather are limited by memory bandwidth. **Benefits.** The same device that serves the array languages serves the element-wise work of inference and the lane-wise work of graphics. **Impact.** *Hardware:* a streaming engine with a small set of element-wise, reduce, scan and gather units behind the ring engine, sized to the memory bandwidth. (The operation set of the engine is the open item J.8.1.)

### 8.6 Driving a vector device (moved from chapter 8)

The chapter (8.4) shows the driver of an accelerator in pseudocode as an illustration. The statements below were its prose form, and the order of events they fix is a proposal until the descriptor is settled.

**8.6.1** (moved from chapter 8) A program fills a descriptor naming the operation and the arrays, executes `FENCE`, and writes the doorbell; the device processes the descriptor, writes the results and a status, and clears the ownership bit; the program polls or takes the interrupt. *(P)*

**8.6.2** (moved from chapter 8) A chain of descriptors runs without the processor, and a work block that is a chain is how a transformer layer or a sequence of array primitives is expressed. *(P)*

**8.6.3** (moved from chapter 8) The buffers are uncached, as for every device, so the processor's reads of results are slow; a large result is left in the device's memory or read back by the next descriptor. *(P)*

*Commentary (moved from chapter 8).* This is the same driver skeleton as for a disk (chapter 5), with an operation field that means "add these arrays". **Trade-offs.** The slow uncached readback is the cost of the explicit hand-off; an interpreter that needs many small results back on the processor should leave them in device memory. **Impact.** *Kernel:* one driver skeleton. *Compiler:* none. *Hardware:* one ring engine shared with every device.

## H.9 Chapter 9: The instruction set

**9.3.9** (moved from chapter 9) `R16` to `R31` in the ABI: the compiler saves `R24` to `R31` in the callee and treats `R16` to `R23` as clobbered by a call. The ABI documents call this split a draft; it is built into the compiler and the runtime of the six large programs. *(P)*

**9.20.4** (moved from chapter 9) Under consideration: separate floating-point condition flags, software-visible translation-buffer manipulation (to pin an entry for a real-time kernel). *(P)* The floating-point condition flags and the conditional select are open items now (J.9.9, J.9.1); the translation-buffer manipulation is only a proposal: an instruction that pins one translation so that a real-time kernel's interrupt path never takes a miss.

**9.2 commentary** (moved from chapter 9) Whether the extension instruction after some narrow operations costs enough to justify 32-bit forms of add and subtract is a proposal that the compiled corpus can answer; it is the proposal of appendix A.9. *(P)*

**9.18 commentary** (moved from chapter 9) A bounds-checked `INDEX` instruction for Pascal and Ada is a possible addition. *(P)*

**9.21.3** (moved from chapter 9) Not decided in the encodings: the combination `.ud` of the two load qualifiers, the stage 2 whole-buffer hash and cipher words (`MSHA256`, `MSHA512`, `MAESCTR`), and a profile-tuned variant of the encoding (not tried). In the compiler: the use of `MOVUW`, `MOVUL` and `MOVUQ`, division by 64-bit constants, loop-invariant motion and live-range splitting. *(P)*

## The instruction entries held back from chapter 9 (moved from the dictionary of chapter 9)

The entries below were in the instruction chapter, in the section named at the end of each entry. They describe instructions that the grammar of the encoding carries and that the author has not decided: each is tagged *(P)*. (The words the author did decide, with their details pending, are back in the chapter: `BSEL`, the deferred-fault and user-qualified moves, and the AES and SHA words.) The bit layouts are in the grammars `spec/pdp64-enc.hbnf` and `spec/pdp64-32/pdp64-32.hbnf`; the charts are not reproduced.

### *MOVUSERD: User-qualified deferred-fault load*

**Mnemonics:** `MOVQ.UD`, `MOVL.UD`, `MOVW.UD`, `MOVB.UD`, `MOVZL.UD`, `MOVZW.UD`, `MOVZB.UD` (section 9.6 of the instruction chapter)

#### Syntax

```
MOVQ.UD d(Rn), Rd
MOVL.UD ..., MOVW.UD ..., MOVB.UD ...
MOVZL.UD ..., MOVZW.UD ..., MOVZB.UD ...
```

#### Operation

```
Rd <- M[EA], the access being both user-qualified (MOVU) and deferred (MOVD)
```

**Flags** None written.

**Exceptions** The faults of `MOVU` and `MOVD` together.

*(P)* The combination of the two qualifiers. The repository defines each qualifier alone (items 10 and 18 of `docs/study-proposals.md`) and gives the combination no text of its own: a kernel that reads a user word without wanting to fail on it (a probe of a user pointer) is the case the combination serves, and its meaning is the composition. Loads only, base and indexed operands.

**Why it is here.** See `MOVD` and `MOVUSER`. The record gives no separate reason for the combination; it is listed because both qualifiers exist and a probe of a user pointer wants both.

### *MSHA: Hash whole blocks of memory*

**Mnemonics:** `MSHA256`, `MSHA512` (section 9.11 of the instruction chapter)

#### Syntax

```
MSHA256 Rd, Rs, Rn
MSHA512 Rd, Rs, Rn
```

#### Operation

```
hash the Rn bytes (whole blocks) at (Rs) into the state at (Rd)
progress is held in Rd, Rs and Rn; an interrupt or a fault restarts the instruction where it stopped
```

**Flags** None written.

**Exceptions** Memory operand: page fault, protection fault, alignment fault or bus error (a read of non-existent memory is a precise abort with cause `BUSFAULT`). Every such fault is precise and the instruction restarts after it, because the register update of an auto-increment or auto-decrement mode is held until the instruction retires.

*(P)* Stage 2 of the crypto proposal (`docs/crypto-secure-core.md` section 3.4): the whole-buffer form, in the manner of the s390 CPACF instructions, to be built "only after measurement"; the grammars reserve its encoding. The unit would need about 2,000 look-up tables and 1,000 flip-flops (the proposal's estimate).

**Why it is here.** Stage 2 of the crypto proposal, in the manner of the s390 CPACF instructions (`KIMD` and kin): one instruction hashes a buffer, the padding and chaining are done in hardware and so cannot be got wrong, and progress is held in registers, so a fault or an interrupt restarts it, as for `MCPY`. The estimate is 64 cycles for a 64-byte SHA-256 block against about 1,200 instructions, roughly 18 times; the cost is gates in proportion to the algorithms frozen into opcodes. The proposal says to build `MSHA256` first and the others only after measurement, and the author has not decided it.

### *MAESCTR: AES in counter mode over a buffer*

**Mnemonics:** `MAESCTR`

**Syntax**

```
MAESCTR Rd, Rs, Rn, Rp
```

**Operation**

```
AES counter mode over the Rn bytes at (Rs), written at (Rd); the key, the counter and the number of rounds
are in the parameter block at (Rp)
progress is held in the registers and the parameter block; an interrupt or a fault restarts the instruction
where it stopped
```

**Flags** None written.

**Exceptions** As `MSHA`.

*(P)* Stage 2, as `MSHA`; the layout of the parameter block is not specified.

**Why it is here.** Stage 2, as `MSHA`; whole-buffer AES gains less (about 4 times: 0.6 against 2.6 instructions per byte), because the round instructions are already good, and the proposal puts it after `MSHA256` and `MSHA512`.

## H.10 Chapter 10: Conclusion: from FPGA to silicon

**10.1.2** (moved from chapter 10) The first board is the Terasic DE10-Nano (Cyclone V SE: about 110,000 logic elements, dual Cortex-A9, 1 GB of memory on the hard processor, HDMI, and the MiSTer ecosystem); the Digilent Zybo Z7-20 or the AMD Kria KV260 are the move when timing becomes the limit; the open-toolchain ULX3S is kept in view but loses the hard-core harness. The device chat staged devices first on a Zynq UltraScale+ (`decisions-pending.md` E6). *(P)*

**10.1.8** (moved from chapter 10) On the paper estimates of the SMP decisions, four cores with the floating-point unit fit only a board of the Kria KV260 class, and sixteen cores fit none of the boards of the plan; no core has been synthesised. A board must fit the four cores of the first device, about 4 x 16,000 = 64,000 six-input lookup tables by the per-core estimate of 10.1.4 (an estimate with an error of about 40%), before the coherence logic. *(E)*

**10.3.4** (moved from chapter 10) Sequential consistency is nearly free on a small in-order core and is the most expensive memory model for a large speculative one; if a large core is built it may be necessary to move to total store order, which is the least painful retreat. *(P)*

**10.1.1** (moved from chapter 10) The first implementation is a soft core on a system-on-chip FPGA whose hard ARM cores serve as the bring-up harness: they load programs, run the tests and provide the console while the pdp64 core is still being debugged, and run an instruction-set simulator beside the core to compare the architectural state after every instruction (`pdp64-fpga-plan.md`, section 6). The text of the chapter tagged this *(D)*; no decision of the author was found for the harness (the register's R24, "since the soc will have hdmi", assumes a system-on-chip and nothing more), so it is proposed. *(P)*

**10.1.4** (moved from chapter 10) The estimated clock a simple single-issue core reaches is 50 to 80 MHz on the Cyclone V, 90 to 130 MHz on a Zynq 7020 and 180 to 250 MHz on an UltraScale+; every one of those estimates is below the 200 to 300 MHz target except the upper part of the UltraScale+ range, and reaching the target means pipeline stages in the datapath. A correct machine at a lower clock comes before a fast broken one. *(E)*

**10.1.5** (moved from chapter 10) The work is staged in phases, each ending in a demonstration and a regression suite: specification and tools; an integer core; exceptions; the memory-management unit; the memory system; floating point; a booting kernel; peripherals; and, last, the other cores. The text of the chapter tagged this *(D)*; the phase plan is the assistant's (`pdp64-fpga-plan.md`, section 5) and the author has not spoken to it (`decisions-pending.md`, E6: "No erratum backs either"), so it is proposed. The decided part, the number of cores, stays in the chapter (10.1.3). *(P)*

**10.1.6** (moved from chapter 10) The second phase, the fault model, is not optional and does not move; the block operations are built with it, because their whole specification is about what state is visible when an exception arrives. *(P)*

**10.1.7** (moved from chapter 10, from 10.1.3) On the first board the block RAM is about 5.5 megabits, and the caches, the translation buffer, the framebuffer and the blitter's scratch memory compete for it. *(E)*

**10.1.9** (moved from the commentary of 10.1) A single engineer working part-time can reach a booting system with floating point in roughly 10 to 19 months, by the plan's wide-error-bar estimate (phase by phase: 5 to 9 weeks for specification and tools, 9 to 14 for the integer core, 6 to 11 for exceptions, 7 to 11 for the memory-management unit, 4 to 8 for the memory system, 4 to 7 for floating point, 8 to 16 for the kernel), with phases 2 and 3 the ones that historically double. *(E)*

**10.1.10** (moved from the commentary of 10.1) The register-transfer description is kept vendor-neutral, with every vendor primitive in one thin directory that holds the register file, block RAM and DSP wrappers and should stay under 500 lines, so that moving between boards costs that directory and nothing else; the datapath is written in SystemVerilog and the floating-point unit is bought, not written (Berkeley HardFloat). *(P)*

**10.1.11** (moved from the commentary of 10.1) A multiplier built from two to four multiplier blocks over four to eight cycles closes timing where a full combinational one will not. *(E)*

## H.A Appendix A: Improvements beyond the PDP-11

### A.4 Protecting the return address (moved from appendix A)

**A.4.1** There are four ways to protect the return address: software canaries and OpenBSD's return-address protection in the compiler, a hardware return stack, a shadow stack, and nothing. *(P)* The mirror shadow stack is approved (errata 85, study item 6); the others are the alternatives that were weighed.

*Commentary.* **Trade-offs.** A shadow stack is the strongest and brings the most recovery code (a second stack to overflow, switch and unwind), so the analysis did not recommend it by default; software detection costs a few instructions per function and no hardware. **Verdict** (the assistant's, before the decision). Start with the compiler; consider a hardware mechanism only if code size allows. **Benefits.** Detection with little or no recovery code. **Impact.** *Compiler:* The compiler adds a canary or a return-address check per function. *Hardware:* Software costs nothing; a hardware return stack costs a second stack structure and its control.

### A.9 Thirty-two-bit forms of add, subtract and shift (moved from appendix A)

**A.9.1** If the extension instructions the compiler emits after narrow arithmetic are more than a few percent of the code, 32-bit forms of the common operations in one-parcel leaves would remove them.

*Commentary.* **Precedent.** AArch64's `w` forms and RISC-V's `ADDW` family. **Trade-offs.** A second set of leaves in the table against extension instructions in the compiled code; the overhead has never been measured over the whole corpus. **Verdict.** Measure first; it is one script over the compiled corpus. **Impact.** *Kernel:* None. *Compiler:* The compiler stops emitting an extension after common 32-bit operations. *Hardware:* A second set of opcodes in the page-2 map and a little ALU decode.

### A.10 Packed-lane instructions in the integer registers (moved from appendix A)

**A.10.1** A small set of lane operations (add, subtract, saturating add, compare, minimum, maximum, average, sum of absolute differences, shuffle on 8, 16 and 32-bit lanes) in the existing registers could speed up string, codec and image loops.

*Commentary.* **Precedent.** Alpha's motion-video instructions and PA-RISC's multimedia instructions did exactly this, in integer registers. **Trade-offs.** It adds no state and little hardware, but it is 64 bits wide so the gain per instruction is modest, and compilers use it poorly. **Verdict.** Collect the evidence first: the compiler corpus was built at a size setting that hides how much vectorisable code there is (appendix B, S1). **Impact.** *Kernel:* String and checksum routines get faster. *Compiler:* Vectorisation into integer lanes is weak, so the library routines would be hand-written. *Hardware:* A lane-aware ALU: little extra hardware and no new state.

### A.12 Smaller possibilities (moved from appendix A)

- `MULHSU` (the high half of a signed-by-unsigned product, for a signed top limb meeting an unsigned one) is reserved in the arithmetic escape; it is one correction step in software and has one caller, a bignum library's innermost loop.
- **Hardware page zeroing**, and **guard-page states in the page-table entry that do not consume translation-buffer entries**, would make the allocator hardening of OpenBSD's `malloc` cheaper.
- **A mixed leaf size for the kernel's text** is needed if the kernel is to be write-xor-execute with one-gigabyte leaf pages.

*Commentary.* **Trade-offs.** None of these is ranked above the decisions in appendix B; the first four sections of this appendix (landing pads, the system-call page bit, the stack check and return-address protection) are the ones OpenBSD's developers have shown the most interest in. (Hardware watchpoints are approved and the sealed page bit and `pledge` mask are rejected: A.10.)

# H.B Appendix B: Decisions pending

## B.1 The instruction set and the system (moved from appendix B)

### *B.1.2 An unprivileged time counter (P3)*

**B.1.2.1** The proposal is `RDTIME rd`, an unprivileged read of a free-running, constant-rate, monotonic counter. *(P)*

*Commentary.* **Precedent.** The PDP-11 and VAX kept the time in memory updated by a line clock and read it with a system call. **Trade-offs.** A user-readable counter is a timing side channel, so the kernel needs an enable bit; the counter's width and rate become architecture. **Benefits.** A clock read without a trap, which is what a fast `clock_gettime` in a shared page needs. **Modern comparables.** x86 `RDTSC`, RISC-V `rdtime`, PowerPC `mftb`, s390 `STCKF`, MIPS `RDHWR`, LoongArch `rdtime`; the x86 counter's history of unreliability is the argument for fixing monotonicity and constant rate at the start. **Use.** Timestamps, timeouts, and the fast path of every library's clock call. **Impact.** *Kernel:* A fast `clock_gettime` with no trap. *Compiler:* An intrinsic for the counter read. *Hardware:* One counter, a user-read enable bit and a `SYSX` decode.

### *B.1.3 Feature and identification registers (P4)*

**B.1.3.1** The proposal is a block of read-only system registers (numbers 13 and up) giving the version, the optional features, the cache geometry, the page sizes and the address width. *(P)*

*Commentary.* **Precedent.** The VAX had a system-identification register the kernel switched on; x86 has `CPUID`, which has grown hundreds of leaves and traps under virtualisation. **Trade-offs.** Each register's layout must be written once in a grammar so it cannot drift. **Benefits.** One binary adapts to the machine, and a kernel need not patch itself per model (OpenBSD carries 75 references to its boot-time patching). **Modern comparables.** AArch64's identification registers, RISC-V's `misa`. **Impact.** *Kernel:* The kernel need not patch itself per model. *Compiler:* Runtime selection of routines by feature bits. *Hardware:* A small block of read-only registers.

### *B.1.4 TLB invalidation by address-space tag (P5)*

**B.1.4.1** The proposal is one new instruction, `TLBIASID`, that invalidates every translation of one address space, broadcast like `TLBI`. *(P)*

*Commentary.* **Precedent.** The VAX flushed process translations on every context switch and kept system ones; x86 added tags (PCID) and `INVPCID` much later. **Trade-offs.** A flush-all works until the first rollover of the tag numbers, after which every process loses its translations. **Benefits.** Process exit and tag reuse become cheap. **Modern comparables.** AArch64 `TLBI ASIDE1IS`, RISC-V `sfence.vma` with an address-space operand, x86 `INVPCID`. **Impact.** *Kernel:* Process exit and tag reuse become one broadcast. *Compiler:* None. *Hardware:* A tag compare in the translation buffer's invalidate path.

### *B.1.5 What a multi-register instruction leaves behind when it faults (P12)*

**B.1.5.1** The proposal is a rule and a test: every multi-access instruction (`PUSHM`, `POPM`, `PUSHX`, `POPX`, the block operations) is either all-or-nothing or safe to repeat from the start, and an emulator test interrupts each at every access and checks that a restart reaches the same state. *(P)*

*Commentary.* **Precedent.** The PDP-11 needed a status register (`MMR1`) and, on the 11/40, a kernel routine that decoded the faulting instruction to undo it; the VAX needed a first-part-done bit; ARM's load-multiple restarts with the base register restored. **Trade-offs.** The area below the stack pointer is dead memory, so a partial push is invisible; the risk is another observer (a device or core) seeing a half-written area. **Benefits.** The fault handler restarts blindly, with no instruction-specific cases, which is what recoverability means. **Modern comparables.** RISC-V has no multi-register instruction; ARM and the VAX restart as above. **Impact.** *Kernel:* The fault handler restarts every multi-access instruction blindly. *Compiler:* None. *Hardware:* Each multi-access instruction checks every address before acting, or is repeatable.

### *B.1.7 A prefetch instruction (P15)*

**B.1.7.1** The proposal is none, with the shape of a non-faulting `PREFETCH` with a memory operand reserved on a free sub-opcode. *(P)*

*Commentary.* **Precedent.** Neither the PDP-11 nor the VAX had one; caches there were transparent. **Trade-offs.** The kernel survey counts over a hundred prefetch instructions across the four kernels, mostly in tuned list and copy code; an in-order core with no data cache gains nothing. **Modern comparables.** AArch64 `PRFM`, ARM `PLD`, PowerPC `DCBT`, x86 `PREFETCHT0`. **Impact.** *Kernel:* List walks and copy loops can warm the cache. *Compiler:* `PREFETCH` would need compiler heuristics. *Hardware:* A non-faulting load path with no destination; nothing on a core with no data cache.

### *B.1.8 Performance counters (P16)*

**B.1.8.1** The proposal is two free-running counters, cycles and instructions retired, readable in user mode, added after the clock (P3) with the same enable bit. *(P)*

*Commentary.* **Precedent.** Unix profiled by sampling the program counter at clock ticks. **Trade-offs.** A user-readable cycle counter is a timing channel; a full counter block with event selection costs a large block and interrupt machinery. **Modern comparables.** x86 `RDPMC`, the AArch64 PMU, RISC-V's `hpm` counters. **Impact.** *Kernel:* A profiler reads counters instead of sampling at ticks. *Compiler:* Intrinsics. *Hardware:* Two counters and an enable bit.

### *B.1.10 The literal pool and execute-only code (P18)*

**B.1.10.1** The proposal is an ABI rule: literal pools move out of `.text` into a read-only section beside it, so code pages hold only instructions and can be execute-only. *(P)*

*Commentary.* **Precedent.** The PDP-11's immediates sit in the instruction stream, and with separate instruction and data spaces a program could not read its own instructions as data. **Trade-offs.** The PC-relative reach limits how far the pool may be, and the linker must keep the two sections near each other. **Benefits.** Zero hardware; a memory-disclosure bug no longer reveals code. **Modern comparables.** OpenBSD makes code execute-only on amd64 and arm64; ARM compilers offer a flag to keep pools out of code. **Impact.** *Kernel:* None. *Compiler:* Pools are emitted into a read-only section beside the text. *Hardware:* None; the linker keeps the sections within PC-relative reach.

### *B.1.11 Vector and SIMD math (S1 to S6)*

**B.1.11.2** Decided (errata 59, chapter 8): data-parallel work lives in memory-mapped devices, and the processor has no vector register file and no SIMD instruction set. What remains is the number formats and operations of the devices (half precision, bfloat16, 8-bit integer dot products, 8-bit floating point), the reconciliation of the device descriptors with the standard descriptor, and whether packed integer lanes are added to the integer registers (A.10). *(P)*

*Commentary.* **Precedent.** The PDP-11 sent heavy numerical work to attached array processors; the VAX had a vector extension on its 6000 series, with 16 vector registers of 64 elements, supported by VMS; neither Unix used it much. **Trade-offs.** A fixed-width vector file brings a second register file and its context-switch cost, opcodes, and a new width for every generation (the history of SSE, AVX and AVX-512); a scalable design runs one binary on a small and a large implementation at the price of the most complex specification. Lazy saving of vector state was a security hole on x86, and Linux went back to eager restore. **Benefits.** The coprocessor needs no opcodes and no processor state. **Modern comparables.** ARM SVE and RISC-V V (scalable), ARM NEON, x86 AVX, PowerPC VSX, Alpha MVI. **Use.** Neural-network inference (the LB64 accelerator), graphics, signal processing and the library's string and memory routines. The evidence needed first: compile the corpus at higher optimisation and count vectorised loops, measure the share of time in memory and string routines, and decide the target workloads. **Impact.** *Kernel:* No vector state to save. *Compiler:* No vectoriser for the processor; the runtime lowers array primitives to descriptors. *Hardware:* No register file; the devices carry the width.

## B.2 The bus, boot and devices (moved from appendix B)

### *B.2.1 Data-cache maintenance by address (P6)*

**B.2.1.1** The proposal is to reserve nothing: DMA buffers are uncached, so no cache instruction is needed until a non-coherent device appears. *(P)*

*Commentary.* **Precedent.** The 11/70's cache watched DMA writes; the VAX's adapter had data paths to purge. **Trade-offs.** Uncached buffers are slow to read; cache maintenance would speed them and needs three reserved sub-opcodes. **Modern comparables.** ARM, MIPS and embedded PowerPC with non-coherent DMA wrap cache maintenance in a DMA-mapping layer; x86 DMA is coherent. **Impact.** *Kernel:* Drivers need no cache code. *Compiler:* None. *Hardware:* No cache-maintenance operations.

### *B.2.2 Firmware calls (P7)*

**B.2.2.1** The proposal is none: everything goes through devices, with a required reset register and a watchdog in the bus controller. The kernel survey disagrees and would specify a firmware-call convention on reserved trap vectors. *(P)*

*Commentary.* **Precedent.** The PDP-11 kernel never called firmware; the VAX had console registers the kernel read and wrote. **Trade-offs.** No firmware means no firmware to trust and no extra privilege level, and every platform must say where its console and reset registers are, which the others get from firmware. **Modern comparables.** RISC-V's supervisor binary interface, ARM's PSCI and secure monitor calls, Alpha's PALcode, SPARC's Open Firmware. **Impact.** *Kernel:* Reboot

and power-off through a device register and a watchdog. *Compiler:* None. *Hardware:* A reset register and a watchdog timer in the controller.

### *B.2.3 Double-width compare-and-swap and a random-number instruction (P8, P9)*

**B.2.3.1** The proposal is to add no double-width compare-and-swap: the evidence for it is one Linux use (software can tag a pointer with the 25 spare bits instead). Random numbers come from an entropy device on the bus rather than an instruction: that is decided (errata 85, 5.7.2). *(P)*

*Commentary.* **Precedent.** The VAX solved the lock-free queue with whole-queue interlocked instructions; no machine of the period had a random-number instruction. **Modern comparables.** x86 `CMPXCHG16B`, AArch64 `CASP`, RISC-V `AMOCAS.Q`; x86 `RDRAND` and `RDSEED`, PowerPC `DARN`, s390 `PRNO`, AArch64 `RNDR`. **Impact.** *Kernel:* Entropy from a device at boot. *Compiler:* Tagged pointers in software if a double-width compare-and-swap is not wanted. *Hardware:* An entropy source device; no new opcode.

### *B.2.4 The interrupt controller and routing to cores (R1)*

**B.2.4.1** The proposal is wired interrupt lines to core 0 with a per-line target register, keeping the eight priority levels, with message-signalled interrupts as an extension. *(P)*

*Commentary.* **Precedent.** The PDP-11 had no controller; a device asserted one of four request lines and supplied its vector. **Trade-offs.** The smallest design adds no new failure cases and cannot give a fast multi-queue device its own interrupt. **Modern comparables.** x86's APIC, ARM's GIC, RISC-V's PLIC and AIA. **Impact.** *Kernel:* A handler sees one interrupt model. *Compiler:* None. *Hardware:* Wired lines to one core with a target register.

### *B.2.5 Inter-processor interrupts and starting other cores (R2, B2)*

**B.2.5.1** The proposal is a per-core doorbell register and a per-core start register in the bus controller (4.7), with `SEV` and `WFE` for waiting. *(P)*

*Commentary.* **Precedent.** The VAX's console processor started the other processors. **Modern comparables.** x86's INIT and START-UP sequence, ARM's PSCI and spin table, RISC-V's hart-start call. **Impact.** *Kernel:* One register write per core and one per doorbell. *Compiler:* None. *Hardware:* A start register and a doorbell per core.

### *B.2.6 How coherence scales (R3)*

**B.2.6.1** The proposal is to keep snooping for the four cores of the first device; the ceiling is now sixteen or more (errata 92), and the protocol for that many is open (7.6.4). *(P)*

*Commentary.* **Trade-offs.** Directory protocols scale and are large; software coherence is cheap and fragile. **Modern comparables.** x86's snoop filters and directories across sockets, ARM's coherent interconnects. **Impact.** *Kernel:* Cache coherence is invisible to the kernel. *Compiler:* None. *Hardware:* Snooping on two to four cores.

### *B.2.8 The device layout and the descriptor (R5)*

**B.2.8.1** The feature subset of each device is open. *(S)*

*Commentary.* **Precedent.** The RK11 and the DEUNA, as in chapter 5. **Trade-offs.** A standard interface gives up the freedom to shape the registers and gains every existing driver. **Modern comparables.** `virtio`, NVMe, SATA's command lists, Intel Ethernet's done bit. **Impact.** *Kernel:* One transport driver and a thin driver per class. *Compiler:* The descriptor is a generated C structure. *Hardware:* Standard devices.

### *B.2.10 Errors raised by devices and the bus (R7)*

**B.2.10.1** The proposal is the precise and imprecise paths the ISA already has plus a small register block in the bus controller for the cause. *(P)*

*Commentary.* **Precedent.** A PDP-11 bus timeout trapped through vector 4 and usually ended in a panic; the VAX had a machine-check frame. **Modern comparables.** x86's machine-check banks and PCI Express error reporting, ARM's RAS extension. **Impact.** *Kernel:* One recovery path for device errors. *Compiler:* None. *Hardware:* A small error register block in the controller.

### *B.2.11 Virtualisation (R8)*

**B.2.11.1** The proposal is to declare it out of scope for System v1 and fix five constraints: privileged state stays privileged or controllable, the page-table walker could be wrapped by a second level, interrupts are injectable, system-register access can trap, and the translation-enable bit and software accessed and dirty bits are reviewed for what a hypervisor would need. *(P)*

*Commentary.* **Precedent.** The PDP-11 was not designed for virtualisation and has sensitive instructions that do not trap in user mode; the VAX was not cleanly virtualisable either, and a 1991 paper described the changes needed. **Trade-offs.** Adding virtualisation late cost the VAX instruction changes; settling the constraints early costs a paragraph. **Modern comparables.** x86 VMX with nested paging, ARM exception level 2, the RISC-V hypervisor extension, IBM's trap-and-emulate machines. **Impact.** *Kernel:* A hypervisor could be added without changing a guest. *Compiler:* None. *Hardware:* Constraints on privileged state, the walker and interrupt injection.

### *B.2.12 Enumeration and hot-plug (N1 to N3)*

**B.2.12.1** Enumeration by PCI Express configuration space is decided (N1, errata 80, 2.4), and so is the read of a function that is absent, which returns all ones (errata 104, 2.4.7). The proposals that remain are the placement of the windows in the physical address space, an interrupt and a rescan on a hot-plug event (N2), and the queued work left in memory when a device departs (N3). *(P)*

*Commentary.* See section 2.4 and its commentary: the VAX's nexus scan, PCI's configuration space and all-ones reads, USB's status-change endpoint. **Impact.** *Kernel:* Enumeration and hot-plug are one loop and one rescan. *Compiler:* None. *Hardware:* A standard root complex and the precise fault for an absent device.

### *B.2.13 The boot decisions (B1, B3, B4, B5, B8)*

**B.2.13.1** The boot ROM that reads block 0 (B1) is decided (errata 109, 4.4). The proposals that remain are the contents of the boot-information block: memory, cores, the root complex's address and the boot device and console (B3), an `INIT` bit with an optional firmware descriptor (B4) and a block list in the protective MBR's boot code (B8). *(P)*

*Commentary.* See chapter 4 and `boot.md`: the PDP-11's block 0 and `ODT`, the VAX's boot block and restart parameter block, the BIOS and UEFI (4.9 for the UEFI decision and the shim). **Impact.** *Kernel:* A boot-information struct and a core start register. *Compiler:* The loader and first block are built by the project's compiler. *Hardware:* A boot ROM, a block-0 reader and per-core start registers.

### *B.2.14 How USB attaches, and virtio first*

**B.2.14.1** Whether USB is a conventional ring-based host controller driven by an existing operating-system USB stack, the proposal of chapter 5, or a controller that does the protocol itself.

*Commentary.* **Trade-offs.** A controller that does the protocol moves it into the chip and takes the stack out of the operating system; the conventional controller uses code that already exists. `virtio` reuses OpenBSD's existing drivers in the emulator and in silicon. **Modern comparables.** xHCI for the conventional controller; `virtio-mmio` on every emulated RISC-V and ARM board. **Impact.** *Kernel:* A disk driver serves a USB disk; virtio reuses the existing drivers. *Compiler:* None. *Hardware:* USB protocol in the controller costs gates; virtio costs nothing for the emulator.

## H.D Appendix D: The ABI, version 1

**D.1** (moved from appendix D, the Registers row) `R16` to `R23` are caller-saved and `R24` to `R31` callee-saved. *(P)*

**D.4.2** (moved from appendix D) The compiler treats `R16` to `R23` as clobbered by a call and `R24` to `R31` as saved by the callee, and the stand-in runtime of the six large programs saves `R24` to `R31` in `setjmp` and `longjmp`. `docs/abi.md` calls this split a draft. *(P)*

# Appendix I — Deferred

**This appendix holds what the author has decided to put off: the answer is "later", and the work waits.** The body of the manual describes the architecture as it is to be and does not mention these things. Each section is one chapter's share, headed with the chapter's number and title. A statement that moved here from the body keeps its number as a label (`**2.3.3** (moved from chapter 2)`), so that a citation of that number still leads to a place. What is proposed but not decided is in appendix H; the questions the architecture must still answer are in appendix J.

## I.3 Chapter 3: Graphics: the terminal and its devices

The author deferred (errata 78, 2026-11-03) a native video device, a native sound path and a bit-block-transfer assist of the pdp64's own, and then dropped the register model on which they were defined (errata 95, 2026-11-09: "virtio across the board"). The standard video and audio devices are virtio-gpu (2D) and virtio-snd (errata 79). The decisions below stand as properties of the deferred devices: the 48-bit pixel (errata 73) and the 48-bit sound sample (errata 74). What to do with the devices is an open item (J.3.1). The statements keep their numbers and tags; wording that assumes the abolished register model is the wording of the time.

### 3.8 The pdp64 terminal: the native devices (moved from chapter 3)

**3.8.2** (moved from chapter 3) A native video device is the Commodore 64's video chip (the VIC-II) extended: a framebuffer in the address space with 48-bit pixels, scanned out through a line buffer. A pixel is RGBA with 12 bits per channel (errata 73, the author's decision of 2026-10-24); the pixel format is what is decided. The register formats and the other pixel formats are unspecified. *(D)*

**3.8.3** (moved from chapter 3) A native sound path is the C64's sound chip (the SID) extended: voices with filters, and also a general sound path that is not limited to the SID's voices; audio samples are 48 bits (errata 74, the author's decision of 2026-10-24); the sample width is what is decided. The register formats and the layout of a 48-bit sample are unspecified. *(D)*

**3.8.9** (moved from chapter 3) Every register of the native video device that takes effect at a frame boundary is shadowed: one write to its `ARM` register latches them all together at the start of the next vertical blank, and a status bit says whether the arm was taken in this frame or missed it. A sticky underrun bit and an underrun counter report a scan-out that ran dry. The vertical-blank interrupt occurs at a fixed line of the frame, whose number a register holds. The sound path's audio clock relation (3.12.4) is part of the same mode setting. *(D)* (Errata 71.)

*Commentary (moved from chapter 3).* **Chain.** The PDP-11 reached its terminal through a serial register pair (3.1). The Blit (1980s) put the bitmap in the address space and had a framebuffer in the address space and did the bit-block-transfer in software ("no pixel-moving hardware", Pike, Locanthi and Reiser, 1985); hardware blitters came later, on the Amiga and other machines of the 1980s (general knowledge, *unverified*). The Blit received downloaded code, characters and editing messages over a serial line and drew every pixel itself. The C64 (1982) put video and sound in chips whose registers a program pokes and whose screen memory the video chip reads by itself. No source read says that it preceded the X11 protocol; the lineage the sources confirm runs from the Blit's `mux` through 8½ to `rio` (`docs/kernel-pain-points.md`, section 14I). The pdp64 needs no display protocol between host and terminal, because the display is a set of devices on the bus; the nearest thing is the descriptor ring of chapter 5. The native design takes the framebuffer and the operation from the Blit, adds a hardware blitter, and takes the video and sound chips from the C64. Plan 9 moved its window system into files (`/dev/bitblt`, later `/dev/draw`), a stream of small commands written to a device; that has the shape of a descriptor-ring entry (the comparison is an inference, not a source's statement).

**Trade-offs.** The blitter covers scrolling a terminal, moving a window, drawing a glyph, filling a rectangle and exclusive-oring a cursor; it does no three-dimensional graphics. `MCPY` is no substitute: it is one-dimensional and occupies the processor, where a blit has a stride and a raster operation and runs concurrently. The 48-bit pixel (four 12-bit channels, errata 73) probably exceeds what a display connector carries (unverified), so the display device converts to the connector's depth on

the way out; likewise the 48-bit audio sample exceeds what a converter takes, so the sound device keeps the precision for mixing and filtering and reduces it at the output.

**Improvements.** The C64's video and sound registers were scattered and irregular; the pdp64 versions use the uniform register face, so one driver skeleton serves them (errata 95 removed that face for the standard devices, 3.9). **Benefits.** The blitter is about 1,500 lookup tables and a line buffer, nearly all address arithmetic (`docs/pdp64-fpga-plan.md`, line 316). **Modern comparables.** Display controllers in embedded systems commonly pair a scan-out engine with a blitter, and sound chips sit behind an I2S interface (general knowledge, *unverified*). **Impact.** *Kernel:* a framebuffer, a sound and a blit driver are each a mapped page and a few register writes; the uncached framebuffer needs no cache maintenance. *Compiler:* none. *Hardware:* a scan-out engine, a line buffer, an address-arithmetic unit for the blitter and a mixer for the sound device, each independent of the processor.

### 3.11 The stub: the C64 and Blit documentation (moved from chapter 3)

**3.11.1** (moved from chapter 3) The register-level definitions of the terminal's video, sound and blitter devices (register names, offsets, bit fields, the 48-bit pixel and sample formats, the sprite, raster-interrupt and voice models inherited from the C64, and the bit-block-transfer operations and windowing model inherited from the Blit) are a stub: only the blitter registers listed in 3.8.4 are listed, and that list is the assistant's sketch, which the author has not discussed. The author's earlier device chat decided much of the rest (register conventions, the 8502 island, the display rules); it is extracted in `docs/c64-devices-from-chat.md` and awaits adoption (`decisions-pending.md` E1 to E11). *(S)*

**3.11.2** (moved from chapter 3) What is fixed: the devices exist, they follow the eight-register convention in the terminal's slots (errata 95 later withdrew that convention for the standard devices), and the interrupt, `FENCE` and uncached-framebuffer rules of section 3.11 apply. *(S)*

*Commentary.* This section is the placeholder for the C64- and Blit-derived specification. The Blit side can draw on the papers recorded in section 14I of `docs/kernel-pain-points.md`: the four primitives `bitblt`, `texture`, `segment` and `point`, the `Bitmap`, `Rectangle`, `Point` and `Texture` types, half-open rectangles, and overlapping windows with obscured parts kept off screen. **Impact.** *Kernel:* none until defined. *Compiler:* none. *Hardware:* the block RAM budget (3.11.3) is settled before the video device is built.

### 3.17 Scanout: what the native video device decides (moved from chapter 3)

**What the repository already decides.** The video device (3.8) shadows every register that takes effect at a frame boundary: one write to `ARM` latches them all at the start of the next vertical blank, a status bit says whether the arm was taken or missed, a sticky underrun bit reports a scan-out that ran dry, and the vertical-blank interrupt occurs at a fixed line (3.8.9, errata 71, from `kernel-pain-points.md`). A base address in that register set therefore flips at the blank without the program arranging anything. The author's device chat of 2026-08 said the same ("multi-register updates are atomic at a named raster line or vblank", `docs/c64-devices-from-chat.md` section 2) and, for the eight-way machine, the assistant's proposal that the author accepted: "Flip on vsync unconditionally with an overrun interrupt instead of a barrier" (`c64stuff.txt` line 5647; the capitalisation is mine). No text decides anything beyond one shadowed register set: independent layers, damage, variable rate, fences, or a default that needs no program setup.

**3.17.2** (moved from chapter 3) The mechanism that is decided is the shadowed registers and the one `ARM` write of 3.8.9; everything else in this section is a candidate. *(D)*

## I.6 Chapter 6: The kernel: modes, exceptions and memory management

**6.9.2** (moved from chapter 6, the direct-mapping windows) The author decided that the format of the direct-mapping windows is deferred (errata 44, from the principle of errata 42): `DMW0` and `DMW1` (system registers 4 and 5) stay stored-only registers in System v1 (errata 36), and their format is defined when a kernel needs a window, not before. The body keeps the decided half: the registers are stored and have no effect on translation. What follows is the sketch that the text carried, and it is not decided.

A **direct-mapping window** (DMW: a range of virtual addresses that the hardware maps to a range of physical addresses by one addition, with no page table and no translation-buffer entry) lets the kernel reach all of physical memory (to write a page it has just allocated, a DMA descriptor or a page table) and the memory of devices, where walking tables is slow and uses buffer entries. The instruction-set text sketches two windows, `DMW0` and `DMW1`, each with a virtual base, a size and a memory type, kernel only, translating as `PA = VA - VirtualBase + PhysicalBase`. It gives three reasons: the kernel stops taking translation misses on its own text and data; physical memory is reachable without mapping it; and early boot becomes simple (`docs/pdp64-`

isa.md section 8.3). The idea is LoongArch's DMW registers, a tidier descendant of MIPS's fixed KSEG0 region (whose behaviour on the real machine is *unverified* here). The author's remark (2026-11-09) was "good thing we already had dmw in the design!": the kernel need not walk anything to reach petabytes of physical memory or a deep table.

The architecture document says that the machine comes up with DMW0 covering physical memory. Errata 44 supersedes that sentence for System v1: MMUCTL is zero at reset and every address is physical.

*Lineage (moved from 6.9).* **Between:** MIPS had the fixed KSEG0 region (*unverified*); LoongArch added the DMW registers (the author's referent). **pdp64:** MIPS's fixed region generalised to two windows; a kernel that wants more regions than two windows maps them with large pages. The cost is one comparator pair and one adder per window.

*Impact (moved from 6.9).* The kernel stops taking translation misses on its own text and data, physical memory and a petabyte-scale device are reachable with no page table, and the first page table is easy to build, because the windows reach the memory it is built in. *Kernel:* physical memory is reachable by addition; the windows serve the copy of a page, a DMA buffer, a page-table page and early boot. *Hardware:* one comparator pair and one adder per window.

The wide page-table entry (entry J.6.2) and the fetch after the write that enables translation (entry J.6.6) are two questions that a window would answer, and the deferral therefore bears on them.

## I.9 Chapter 9: The instruction set

**9.20 commentary** (moved from chapter 9) The direct-mapping windows that LoongArch uses to avoid translation-buffer misses on kernel text are deferred: DMW0 and DMW1 are stored and have no effect on translation, and their format is defined when a kernel needs a window (6.9.2, and the deferred matter of chapter 6 in appendix I).

## I.A Appendix A: Improvements beyond the PDP-11

**A.4.2** (moved from appendix A) Return-address authentication is deferred (errata 84, item 7). *(D)*

**A.5.2** (moved from appendix A) Bounds for the valid-pointer tag are deferred (errata 86, study item 3): the tag per register and per 8-byte word is kept and the bounds of a capability are not part of it.

### A.8 Two-byte forms of the user-memory guard instructions (moved from appendix A)

**A.8.1** STUA and CLUA are not in the grammar of this CPU; whether they are added as one-parcel system words is decided when the kernel exists, and the user-access moves (section 6.10.4) may make them unnecessary (they are kept until the OpenBSD port confirms them, A.8.1).

*Commentary.* **Trade-offs.** It is not worth deciding until a kernel exists to count the copy routines. **Benefits.** Smaller and faster user copies. **Verdict.** Decide when the kernel exists. **Impact.** *Kernel:* Each user copy routine is four bytes smaller. *Compiler:* None. *Hardware:* One more system word.

# Appendix J — Open items

**This appendix discusses each question that the architecture must answer and has not.** In the body of the manual an open item is one short sentence with a pointer to its entry here (`Open: how many pages the first few are (J.3.2).`). The entry states the issue, why it matters, every option with its advantages and disadvantages, the precedents, the author's own words on it where he has given them, and the places in the body that refer to it. Where the assistant that wrote the entry has a leaning, it is labelled "The assistant's leaning" and is not a decision. Each section is one chapter's share, headed with the chapter's number and title (`J.3` for chapter 3, `J.D` for appendix D); the items in it are numbered `J.3.1`, `J.3.2` and so on, and a number is not reused once it has been cited. Statements that moved here from the body keep their old number as a label. What is proposed but not decided is in appendix H; what has been put off is in appendix I.

## J.2 Chapter 2: The bus

### J.2.1 Locating devices without a selecting bit

**Issue.** On the Unibus, an address whose top bits were all ones meant "a device register", so every driver knew where its device was. The pdp64 has no such bit (errata 97, 102): a device window is an ordinary range of physical addresses, and the physical address is at least 40 and at most 64 bits wide. Software must therefore learn from somewhere where each window is, and something must choose where it goes. There are three ways to do it: **enumeration** (the processor reads a small self-description from every device and assigns each one a window), a **table handed over at boot** (the first stage tells the kernel what is where), or **fixed addresses** written into the architecture. The manual's text assumes enumeration through the root complex (the chip at the top of the bus tree, which turns the processor's accesses into bus transactions) and says only that the address of the root complex arrives in the boot-information block.

**Why it matters.** The kernel cannot attach a driver before it knows a base address. With no address bit to mark a device page, the memory type of a mapping is the only marker of what a page is. The root complex must decode exactly the windows that were assigned, and must refuse a device's DMA (direct memory access: a device reading or writing memory by itself) into another device's window; the IOMMU (a unit that translates the addresses a device uses, so that a device can only reach the memory it was given) needs the same map to do that. The statement that says where the configuration space and the windows sit is unspecified.

**Options.**

1. *PCI Express enumeration, with the root complex address and a window pool in the boot-information block.* The kernel assigns each function's base-address registers (the registers by which a device states the size of a window it wants and is told where it sits) from a range of addresses that the block gives it. For: one mechanism for everything on the bus; only two numbers are passed (the base of the configuration space and the window pool); the platform may move them freely, so the architecture fixes nothing; drivers learn their base from the device. Against: the kernel must trust the pool, and a pool that overlaps RAM can be detected only by comparing ranges; the console needs a separate base before enumeration has run.
2. *A device table in the boot-information block* (kind, base, size, interrupt for each device), with no enumeration for the platform's own devices. For: suits devices that are not on PCI Express at all; needs no walk of the bus. Against: duplicates what enumeration finds, so there are two mechanisms; a device that arrives later is not in the table.
3. *A fixed platform map written in the architecture:* the root complex and a window range at fixed addresses. For: no table to trust and no problem early in boot; very simple for a small board. Against: it fixes an address below $2^{40}$, the smallest width, and so brings back the habit of the upper half of the address space that the author dropped; a wider machine cannot put its windows high; the VAX shows that a fixed map is changed from model to model.
4. *Top-down placement:* the root-complex window ends at the top of the implemented physical address space, so it moves with the width the processor reports. For: never collides with RAM, which grows up from zero; one rule for 40 and

for 64 bits. Against: depends on the width being discoverable (an open question of its own); a program that assumes a width computes a wrong address; memory sizes near the top collide with it.

5. *An address-map list handed over at boot,* each range typed as RAM, device or reserved, with the memory type of a page as the only runtime marker. For: one place says what every range is; a window can sit anywhere; the kernel can refuse to map RAM as a device and the reverse. Against: the list can be wrong; a wrong memory type on a device page is no longer caught by hardware, as the old address bit caught it; more kernel code.
6. *A self-describing window-map register block at one fixed low address,* just above the first pages, through which everything else is found. For: only one address is architectural, and it is in the reserved boot area; no block is needed. Against: a new device and a new fixed address; the reservation of the first pages must cover its page.

**Precedents.** The RISC-V virt board of QEMU gives the configuration space at 0x30000000 and the PCI Express windows at 0x40000000 and describes them to the kernel *(verified: QEMU hw/riscv/virt.c).* The x86 ACPI table that names the configuration-space base *(unverified: general knowledge).* The VAX had the Interconnect (the bus of the VAX-11/780) and a table that its adapter slots filled; its I/O space began at 0x20000000 with one 8 KB window per adapter slot, changing from model to model *(verified: the 4.3BSD* `nexus.h` *read for chapter 2).* The PDP-11 I/O page at the top of the 16-bit space, and the extended spaces of the later models *(verified for the 16-bit page; the 18- and 22-bit extensions unverified).* The VAX RPB and the Alpha HWRPB (restart and hardware parameter blocks that hand a device table to the operating system) and device trees on ARM and RISC-V *(unverified: general knowledge).* The x86 PCI hole just under 4 GB and the BIOS E820 and UEFI memory maps *(unverified: general knowledge).* The VAX nexus status registers describe each slot *(verified: the 4.3BSD* `probenexi` *read for chapter 2).*

**The author's referents.** USB, enumerated over a PCI Express bus ("arbitrary buses ... can be connected to it and enumerated"); virtio "across the board"; the VAX's I/O space ("it should be noted in the book how the vax does this io space idea"); "mmio allows larger windows as needed"; and, of the old address bit, "if we are going to mmap petabytes, we need those bits".

**The assistant's leaning.** Option 1, with the memory type of the page as the safety check and a kernel rule that a device mapping is made only on an enumerated window. It is the text the manual already has, it adds no table and it works from 40 to 64 bits. It would change if the author wants platform devices without PCI Express (a timer, a console, an interrupt controller); then Option 2's table is added for those alone.

**Where the body refers to it.** Sections 2.2 and 2.4.6.

### J.2.2 The bus controller

**Issue.** An early decision (errata 49) made the bus a device in its own right, with registers by which the processor could detect what is on it, ask it to map a device to a given address range, and be alerted when a device enters or leaves. The register interface was later dropped (errata 95), but the three jobs were kept: the bus **enumerates** (lists what is on it), **maps** (assigns each device a window) and **alerts** (tells the processor of arrivals and departures). Under PCI Express the three jobs already have standard mechanisms: enumeration by reading each function's configuration space through the root complex; mapping by sizing and assigning the base-address registers; alerts by the hot-plug events of the slot and the root complex's own interrupt. The question is which jobs standard PCI Express and virtio already do, which need a piece of the pdp64's own, and what, if anything, the "bus controller" now is.

**Why it matters.** The kernel needs one recursive walk of the bus (section 2.4.3) and one routine to recover from a departure; the mature PCI code of OpenBSD can be reused only if the machine presents a standard PCI Express bus. For the hardware, a root complex is a large block, and the first boards (the Cyclone V and the Zynq 7020) have no hard one, while a Zynq UltraScale+ has; a native controller would be a small custom block. The alert is decided (2.4.4, errata 49); the queued work of a departed device staying in memory is decided (2.4.5, the author's suggestion of 2026-10-22), its mechanism is not, and the precise fault for a departed device (2.4.5, appendix H.2) is a proposal, and where the configuration space and the windows sit (2.4.8) is open (J.2.1).

**Options.**

1. *Pure PCI Express.* The root complex is the bus controller; there is no pdp64 device; enumeration, window assignment, hot-plug and error reporting are the standard ones (configuration space, slot status, message-signalled interrupts, advanced error reporting); the pdp64's own rules live in how the processor treats the root complex. For: the least new design; compatible with every card; OpenBSD already enumerates PCI Express. Against: needs a root complex the cheap

boards lack; the notorious configuration difficulties of PCI (about thirty boot options in the BSD and Linux kernels for bus numbers and window assignment; guessed window sizes for hot-plugged devices) come with it; a removed device reads as all ones, the opposite of the precise fault of 2.4.5, which would need extra circuitry anyway.

2. *PCI Express plus a small pdp64 root-complex function* (the shape of the proposals of appendix H.2). Enumeration and hot-plug are standard; the root complex adds a few processor-side registers and rules: a bus reset write, the departure alert, the precise bus fault and drop-and-count for a departed window, and a reset that produces no removal events. These stay invisible to cards. For: it keeps the three verbs of errata 49 and gives them to standard mechanisms; departure is detected and recoverable. Against: still needs the root complex; the extras are proposals whose detail is not specified; the precise-fault rule needs presence tracked per function, which standard PCI Express provides only for hot-plug slots.
3. *virtio-mmio devices behind a native bus controller* (errata 49 as written, with virtio devices). The controller is a small memory-mapped device with a table of present devices (type, base, size, interrupt) and an alert register; each device is a virtio-mmio transport (virtio's form for machines without PCI) in a window the controller assigned. For: no root complex, so the first boards work; it gives errata 49 exactly. Against: gives up compatibility with PCI Express and the plan to attach USB as a PCI Express function; a real card (a network adapter, a graphics card, NVMe, a USB controller) cannot be attached; the drivers carry two transports; it reverses decisions of 2026-11-03.
4. *A soft host-bridge facade.* The processor always sees a configuration space and virtio-PCI functions; on the FPGA, a small block in the fabric presents simple devices as PCI functions with no PCI Express link; on silicon, a real root complex. For: software sees one bus on every board; no hard block is needed on the first boards. Against: custom logic that must emulate enough configuration space for the kernel's PCI code to accept it, so a fault in it looks like a driver fault; no real card on that board; one more block to test.
5. *A static description for the fixed machine; hot-plug only for USB.* The boot-information block or a small device tree lists the root complex and every fixed device; the kernel enumerates once at boot; only the USB host controller reports arrivals and departures; PCI Express slot hot-plug is software only (the fault and event rules, no slot hardware) until a board has slots. For: fewest moving parts; the bus controller becomes boot data. Against: no live add or remove of anything but USB; a device tree is another artefact the boot ROM must carry and keep in step with the hardware; statement 2.4.4 would shrink to USB. It is compatible with Option 1 or 2 underneath.

**Precedents.** On x86, ARM and RISC-V servers the configuration space is reached through a memory-mapped window (ECAM, the enhanced configuration access mechanism) and hot-plug uses the slot capability and its interrupt *(unverified: general knowledge)*. PCI sizes a window by writing all ones to the base register and reading back *(not confirmed in the project's own sources)*. The VAX scanned its interconnect through an adapter on the nexus: `probenexi()` reads each slot's status register, and the UNIBUS adapter's own registers hold the map registers *(verified: the 4.3BSD sources* `vaxuba/ubareg.h` *and* `vax/autoconf.c`*, read for chapter 2)*. Amiga AutoConfig, in which each board announces itself and is told where to live *(unverified: general knowledge)*. virtio-mmio is what QEMU's virt and microVM machines use where PCI is absent *(unverified: general knowledge)*. ARM and RISC-V boards describe fixed devices in a device tree and enumerate only PCI below the root port *(unverified: general knowledge)*. A PCI-compatible host bridge in an FPGA with no transceiver is a configuration-space emulator of some hundreds of registers *(unverified: no specific design found)*.

**The author's referents.** The first idea, in his words: "the bus itself is another device with its own set of mmap registers so we can detect what is on it and request it to map a device to a particular address range, we do need some way for the bus to alert us when a device enters or leaves the bus"; the eight registers were later rejected ("forget the native 8 register devices. gone."). USB enumerated over a PCI Express bus; "we're going with pcie and nvme and usb"; and "didn't we talk about hot swap?".

**The assistant's leaning.** Option 2 as written, with Option 5's limit on hot-plug (USB live; PCI Express in software only), and two things put to the author: where the configuration space and the windows sit (for example a fixed base outside the first few pages), and a short table of the controller's registers (reset, departure alert, presence). It delivers the three verbs with standard mechanisms and costs a few registers beyond PCI Express. It would change if the first board has no PCI Express block and no budget for a soft core (then Option 4 or 3), or if the departure fault turns out not to be buildable on root-port hardware (then a presence register per function).

**Where the body refers to it.** Section 2.4.4.

### J.2.3 The IOMMU: table format, place, faults and virtualization

**Issue.** The author has decided that the pdp64 has an IOMMU (a unit that translates the addresses a device uses, with a page table of its own for each device) and accepted five rules for it (errata 75, 85): translation on by default, one domain for each device, no kernel pointers in descriptors, permanent mappings for the rings, bounce buffers for small data. The leaf entry shares bits 0 to 39 with the processor's page-table entry and bits 40 to 63 are free (errata 85). Four things are not decided: the format of the table above the leaf (how many levels, how a device is found from the identity the bus gives it), where the unit sits (in the root complex, in each bridge, or beside the memory controller), how a refused access is reported (to the device, to the kernel, or both), and what happens when a guest operating system runs its own devices (virtualization, R8 of appendix B).

**Why it matters.** The kernel's mapping routines, the driver interface (a descriptor holds a device address, not a physical address) and the interrupt controller (J.6.3, which may use the same unit to check where a device's interrupt write goes) all depend on it. A unit placed at the root complex sees every device but cannot tell a device behind a bridge from the bridge; a refused write that is silently dropped is hard to debug, one that stops the machine is easy to attack.

**Options.**

1. *A device table indexed by the bus-device-function number, one page table for each entry.* For: the shape PCI Express already identifies devices with; each device its own domain falls out. Against: a large sparse table unless it is indexed in levels; functions behind a bridge may share one identity.
2. *A small table of domains, with the device-to-domain map in a register file in the root complex.* For: cheap in hardware; fits the first FPGA boards. Against: a fixed number of devices; a hot-plugged device may find the table full.
3. *Same, with a second-level table, so a guest's mapping is composed with the host's* (nested translation). For: virtualization costs no emulation. Against: two walks for every access; a larger unit.
4. *Report a fault only to the kernel, as an interrupt with a log entry* (device, address, kind), the device's access being refused or dropped. For: one path. Against: a device waiting for a read completion hangs unless the unit completes it with an error.
5. *Report a fault to the device as a failed completion and also log it.* For: the device's driver sees an error at once. Against: not every device handles it; the log can overflow under attack.

**Precedents.** x86 has the IOMMU of Intel (VT-d) and of AMD, indexed by bus, device and function, with a fault-recording ring and an interrupt *(unverified: general knowledge)*. ARM has the SMMU with stream identifiers and stage 1 and stage 2 translation *(unverified)*. The VAX Unibus adapter's map registers were per adapter, not per device *(verified: 4.3BSD `ubareg.h`)*.

**The author's referents.** "iommu will allow us to map in c64/c128" (errata 75, a later use); the five accepted recommendations (errata 85).

**The assistant's leaning.** Option 1 with a two-level device table, and Option 5 (an error completion and a log entry), because both keep the same code for boot and for a hot-added device. It would change if virtualization is wanted early (then Option 3).

**Where the body refers to it.** Section 2.11.

### J.2.4 Which optional virtio features the pdp64 uses

**Issue.** virtio lets each side agree optional features by bit. Three concern the machine and not one device. `VIRTIO_RING_F_EVENT_IDX` (bit 29) replaces the two on-off flags with an index at which a side wants to be woken. `VIRTIO_F_ACCESS_PLATFORM` (bit 33) says that a device's addresses go through the platform's translation (the IOMMU). `VIRTIO_F_ORDER_PLATFORM` (bit 36) says that the device obeys the platform's ordering rules and not weaker ones. The exact feature subset of each standard device is a fourth, per-device item, set when each device is written.

**Why it matters.** A driver on a machine with an IOMMU that does not agree `ACCESS_PLATFORM` leaves the device on physical addresses and defeats the unit; a device that is not told to order its writes may let an interrupt overtake data (2.5.4). `EVENT_IDX` changes the notification test in every driver.

**Options.**

1. *Require `ACCESS_PLATFORM` and `ORDER_PLATFORM` of every device and refuse the device without them.* For: the IOMMU and the ordering rule hold for every device. Against: some emulators and old devices do not offer them.

2. *Require them on the physical machine, accept their absence under an emulator with a warning.* For: runs on QEMU as is. Against: two behaviours; the secure case is not the tested one.
3. *Do not use* `EVENT_IDX`*: the two flags only.* For: smaller driver, nothing to get wrong. Against: more doorbells and interrupts under load.
4. *Use* `EVENT_IDX` *when the device offers it.* For: fewer notifications. Against: more code and a corner case at index wrap; the polled mode needs no help from it.
5. *Declare nothing beyond* `VERSION_1`*, and treat translation and ordering as properties of the platform, not negotiated.* For: fewest bits. Against: a driver cannot tell that a device honours them.

**Precedents.** Linux negotiates `ACCESS_PLATFORM` and refuses translation-less operation on a platform that needs it, and uses `EVENT_IDX` when offered *(unverified: general knowledge)*. The bit numbers are in the Linux headers `virtio_config.h` and `virtio_ring.h` *(verified)*.

**The author's referents.** "virtio across the board" (errata 95).

**The assistant's leaning.** Option 1 for the first two, and Option 3 until a measurement shows a need for `EVENT_IDX`.

**Where the body refers to it.** Section 2.7.

### J.2.5 The root complex's own registers and the pdp64's additions

**Issue.** Three decided rules name a register of the root complex and give no place for it: a bus reset is a write to a register (2.9.4), a read of an empty configuration position is recorded in a sticky, write-one-to-clear bit of its status register (2.4.8, errata 104), and the circuitry that the pdp64 adds to PCI Express for convenience and security is mentioned without detail (2.3.4). A fourth, where the root complex itself sits, is J.2.1. The author asked that the miss be recorded in a status register and mode flags that already exist and not in new state, and agreed to the root complex's own bit; the processor status word was ruled out because it belongs to the running thread.

**Why it matters.** The kernel cannot reset the bus, or tell a dead link from an empty slot, without an address; and a block of registers is the natural home for the later additions.

**Options.**

1. *A register block at a fixed offset from the configuration-space base.* For: one address (the base) is passed; nothing new to discover. Against: shares the address range of the functions, and an enumeration walk must skip it.
2. *A PCI Express function on bus 0 whose windows hold the registers.* For: found by enumeration like everything else; no fixed address. Against: the function needs a driver, and the reset is then a write to a device that the reset itself disturbs.
3. *A block at an address in the boot-information block.* For: free placement. Against: one more field to hand over.
4. *Bits in a system register* (the numbers from 13 up are reserved). For: no address. Against: the register is per core, so the root complex's single flag lives in a per-core place; the author did not choose it.
5. *A block in the interrupt controller's page,* if J.6.3 gives that unit its own range. For: the registers of the machine's central units together. Against: ties two decisions together.

**Precedents.** The x86 root complex registers (root control, root status) are in the configuration space of the root port *(unverified: general knowledge)*. The VAX adapter's registers (configuration and status) sit in its nexus window *(verified: 4.3BSD* `ubareg.h`*)*.

**The author's referents.** "don't we already have a status register and mode flags, can't you use a bit in one of those or add a bit?"; "agree root complex bit" (errata 104).

**The assistant's leaning.** Option 1 or 3: pass the base once in the boot-information block and place the block at a fixed offset from it.

**Where the body refers to it.** Sections 2.3.4, 2.4.8 and 2.9.4.

## J.3 Chapter 3: Graphics: the terminal and its devices

### J.3.1 The deferred native devices

**Issue.** The first design of the graphics and sound hardware was a family of devices of the pdp64's own: a video device derived from the Commodore 64's video chip, with 48-bit pixels (12 bits for each of red, green, blue and alpha; errata 73); a sound device derived from the C64's sound chip, with voices and a general sound path of 48-bit samples (errata 74); and a blitter (an engine that copies and combines rectangles of pixels, in the manner of the Blit terminal), all three controlled through

a small fixed set of registers. They were deferred (errata 78) and the register model on which they were defined was then abolished (errata 95: "virtio across the board"). The standard interfaces are now virtio-gpu (2D) and virtio-snd. What is unanswered is what the architecture should say about the native devices now: keep them as a marked future part, drop them, redefine them over virtio, or something else. Whatever the answer, the scan-out hardware (the circuit that reads the frame from memory and feeds the display) still has to be built, and the standard video device does not remove it, since virtio-gpu is a protocol, not a scan-out engine.

**Why it matters.** Each native device is one more driver for the OpenBSD port, which is exactly what errata 78 wanted to avoid. Nothing in the instruction set depends on them; the 48-bit pixel and sample matter only to software that renders or mixes for them. The author's wishes (the Blit's model, the C64's graphics and sound, 48-bit samples) have no home until this is answered. The same author also wants the standard audio and video interface to be the one that every operating system already supports.

**Options.**

1. *Keep as a marked future part.* Keep the native-device material in the deferred appendix (appendix I, chapter 3), with every statement and tag, rewrite the wording that assumes the abolished register model, and state that nothing in the standard depends on it. For: preserves the author's work and the 48-bit decisions at the least cost; the author said "planned for the future", not "dropped". Against: a body of statements that nobody implements; wording written for the old model becomes false and must be caveated; the old text may be read as current.
2. *Remove it.* Delete the material and keep the decisions only in the record; the graphics chapter then says virtio-gpu, virtio-snd, the display output and the framebuffer. For: nothing dead is specified; one source of contradiction fewer. Against: loses the Blit and C64 design record from the manual; contradicts "planned for the future"; the 48-bit pixel decision loses its home.
3. *Redefine as virtio.* Treat virtio-gpu and virtio-snd as the whole interface and carry the extras as feature bits or formats on those devices (a 48-bit pixel format, a 48-bit sample format), implemented in every model. For: one transport driver serves everything; a format is a negotiation, not a device. Against: stock hosts and drivers would reject a 48-bit format (virtio-gpu's 2D formats are, as far as is known, 8 bits for each channel, and virtio-snd's stop at 32- and 64-bit integers and floats; the lists are to be checked), so a fork is needed; none of the sprites, raster interrupts or voices survives.
4. *An optional PCI Express card outside the standard.* Publish the video, sound and blitter designs as an optional card with its own driver, defined from scratch with configuration space and message-signalled interrupts; the standard stays virtio. For: keeps the designs as real hardware someone may build, in the manner of the optional C128 of errata 77; available at any time, since any PCI Express device may be attached. Against: a full register specification must finally be written; the first boards have no PCI Express block; drivers exist only if someone writes them.
5. *Keep the Blit model as software.* Drop the three devices as hardware; keep the Blit's ideas (the framebuffer in the address space, the four primitives of bit-block transfer, texture, segment and point, half-open rectangles, layers) as a library run by a core dedicated to graphics, drawing into a virtio-gpu 2D resource and flushing rectangles. For: the Blit itself had no pixel-moving hardware and did everything in software; reuses cores that exist; no new device. Against: one core is slow at 1920 by 1080 at 48 bits (12.4 MB a frame) unless it is wide or has vector instructions; depends on the dedicated core, which is still a proposal; the C64 and sound personalities vanish.
6. *A pdp64 blit and sprite extension as its own virtio device type,* or as a feature of virtio-gpu, whose queue carries blit commands (source, destination, strides, rectangle, raster operation) and C64-style sprite and raster-interrupt services, so that the idea of registers plus DMA becomes a command queue. For: the virtio way to carry what registers carried; keeps the author's blit and sprite design on the standard transport. Against: no stock host implements a new device, so the device model, the hardware and the driver must all be written, which is the "bunch of device drivers" of errata 78; a new device number in the shared virtio registry needs care.

**Precedents.** The repository keeps the alternatives of errata 94 as a documented comparison and keeps the C128 as an optional device outside the standard *(verified: errata 94, 77)*. The Blit terminal (a Motorola 68000 at 8 MHz that drew every pixel itself) *(verified: chapter 3 commentary)*; Plan 9's draw device and libdraw keep the same primitives in software *(unverified: general knowledge)*. VGA-compatible PCI cards that expose a legacy register set beside a native one *(unverified: general knowledge)*; the Amiga's fixed custom chips *(unverified)*. virtio-gpu carries all its commands as structures in a control queue; whether its 2D set has a resource-to-resource blit is not known *(unverified: to check)*. NVMe and xHCI also

command through queues in memory *(verified: chapter 5)*. OpenBSD has a driver for virtio-gpu *(verified from a fetched directory listing)*; whether it has one for virtio-snd is open *(errata 79)*.

**The author's referents.** The Blit ("i like the blit model", "we are basing the design not only on the c64, but also on the blit terminal"); "all the c64 graphics and audio goodies" and "48 bit sound as well, not just the sid"; "our 12 register nic and sound card and graphics card are planned for the future"; "a standard audio/video interface so there is no bullshit with supporting a bunch of device drivers, although as a pcie bus any device MAY be attached"; the C128 as "just another device on the bus; not part of the pdp64 standard"; "modern 48 bit audio and 48 bit pixels"; HDMI "like sata and usb, is standard kit".

**The assistant's leaning.** Option 1 now, with Option 5 as the intended direction: the material is kept in the deferred appendix, and the live part of the chapter is the display output (HDMI through a transmitter chip fed a pixel stream), the framebuffer and its memory type, now about virtio-gpu and virtio-snd. Removal would contradict the author's words; Options 3 and 6 need a format or device that no stock host provides; Option 4 stays available later. It would change if the author says that the 48-bit pixel and sample are required in the first iteration (Option 3 or 6), that he will not build them (Option 2), or if a measurement shows that virtio-gpu scan-out cannot reach 1080p on the first board (a native device for the FPGA alone, Option 4).

**Where the body refers to it.** Nowhere: the matter is deferred (appendix I, chapter 3: the sections 3.8 and 3.11 moved from the body), and the body does not mention the devices.

### J.3.2 The form of the terminal device and its name

**Issue.** On 2026-11-10 the author described "our virtio-console device" as "an attempt to bring the nice properties of the blit terminal and c64 for doing simple and easy peek poke style display of text and also doing graphics, but with modern color depths and screen sizes" (errata 105). That there is such a device, with peek and poke text and graphics and glyph slots, is decided (3.9.1); what is open is its form and its name. No document defines a device of that name: not its identification, its queues or its registers. The real virtio-console is a serial text device (device type 3; a configuration structure of `cols`, `rows`, `max_nr_ports` and `emerg_wr`; the feature bits `SIZE`, `MULTIPORT` and `EMERG_WRITE`; control messages for ports; one receive and one transmit queue for each port). Its data are byte streams, with no cells, colours, glyphs or pixels, like the teletype of 3.1. The device the author describes has cells, colours, glyph slots and graphics, which neither the real virtio-console nor virtio-gpu 2D has. The standard video device is virtio-gpu in 2D mode (errata 79, 95).

**Why it matters.** The kernel needs one driver for the text display and one for graphics, and the author wants no more drivers than the standard demands (errata 78). Until the form is chosen, the terminal cannot be specified, the boot console cannot be named and the glyph-slot question (J.3.4) has no device to belong to.

**Options.**

1. *A new virtio device type of the pdp64's own,* with the cell array and the glyph slots as the device's memory. For: one device that matches the author's words; virtio's transport, queues and discovery are reused. Against: no stock host implements it, so the device model, the hardware and the driver must all be written; a number in the shared virtio registry needs care.
2. *virtio-gpu extended by a text layer and glyph slots.* For: one display device, not two; the host's existing virtio-gpu driver still works for the parts it knows. Against: the extension is a fork of a shared specification; stock hosts ignore the new commands, so text would fall back to pixels.
3. *The real virtio-console as the text stream for existing tty programs (3.2), beside a separate display device.* For: nothing to invent; every existing tty program works; it is the form the boot console already assumes. Against: it is a stream, so it gives none of the peek and poke the author wants; two devices must be coordinated.
4. *The native device family, renamed* (the deferred devices of J.3.1). For: the closest match to the author's description; the C64 and Blit designs are kept. Against: errata 95 abolished the register model they used; each one is one more driver.
5. *A library over virtio-gpu* that offers the peek and poke look in the program's own memory (the fourth sketch of J.3.3), as a first realisation of the device. For: no new hardware or driver; runs now. Against: not yet the "device" the author names; the chip does not draw text while the program does other things.

**Precedents.** The Linux header `virtio_console.h` and the device-type list in `virtio_ids.h` (types 3 for console, 16 for GPU, 18 for input, 25 for sound) *(verified: the headers on this machine)*. The repository mentions virtio-console only as a candidate console for the boot ROM and kernel (5.7.1). Plan 9's draw device, a stream of small commands written to a file *(unverified: general knowledge)*.

**The author's referents.** The Blit terminal and the C64; "peek poke style"; "glyph maps ... a ttf engine generate glyphs and they will go into their memory slots"; the author's earlier terminal chat of 2026-06-18, a separate project in which a cell holds colours, a character and attributes and existing tty programs run through a shim.

**The assistant's leaning.** Option 3 for the console stream and Option 5 as the first display, with Option 1 or 2 as the device the author describes.

**Where the body refers to it.** Section 3.9.1.

### J.3.3 The mashup: how the display composes text, pixels and sprites

**Issue.** The author asked on 2026-11-10 "what would a mashup of gpu, vram, c64 and blit look like" (errata 105). Five shapes were drawn, all proposed (appendix H, 3.21): (S1) layers composed at scanout, with no command queue; (S2) the same plus a small blitter and a glyph atlas; (S3) a device whose main interface is a ring of commands, with a peek and poke window; (S4) virtio-gpu 2D with the conveniences in a library; (S5) a small core with the C64's chips, as a variant of any of them. **VRAM** is memory the display device owns, behind a window on the bus; on the first board there is none to speak of (the block RAM is 5.5 megabits and a frame is 8.3 MB), so every layer lives in system memory. The properties the author asked for: (T) simple peek and poke text and graphics at once; (C) modern colour depth and screen size; (G) glyph slots filled by a TrueType engine; (S) sprites, colour and sound in the C64's spirit; (B) the Blit's bit-blit and windows; (U) the GPU's command queue, parallel lanes and shaders.

**Why it matters.** It decides whether the display needs hardware beyond the standard virtio-gpu, how many drivers the kernel carries, and how much of the author's "as simple and pleasurable as the c64 was" is delivered by the machine and how much by a library.

**Options.**

1. *S1, layered scanout.* A text layer, a pixel layer and a sprite layer in memory, composed by the display hardware as it reads them out. For: keeps T, C, G and S; the C64's model with a modern screen; per-layer flip and a cursor come free. Against: no blitter, no scaling or rotation; needs a device type of its own; scanout traffic is the pixel layer's 498 MB/s at 1920 by 1080 (4 bytes, 60 Hz), plus the cells.
2. *S2, S1 plus a blitter and an atlas.* For: adds B; scrolling and window moves cost the processor no pixels; the blitter is about 1,500 lookup tables (`docs/pdp64-fpga-plan.md`). Against: a bus-master port on video memory and its arbitration with scanout; its own device type.
3. *S3, a command-queue device with a peek and poke window.* For: the natural virtio shape (queues of 2.7); fence and atomic commit are the form of a commit; U in a fixed form. Against: more gates and a memory manager; two ways to reach one cell must be reconciled; T only through the window.
4. *S4, virtio-gpu 2D with a library.* For: it is the standard (errata 79, 95); no gates. Against: no sprites, no blits, no layers; C only to 32 bits; the processor draws every pixel; the host reads the resource when it likes.
5. *S5, a small core with the C64's chips as the terminal.* For: the author's phrase at its most literal; the host sends messages only. Against: a second program to write, load and secure; see J.3.11.
6. *A combination,* for example S4 now and S2 or S3 as a second device later. For: nothing is wasted while the author decides. Against: two display models to keep consistent.

**Precedents.** The C64's text, bitmap and sprite compositing at scanout *(unverified: general knowledge)*; the Amiga's blitter *(unverified)*; the Blit, which did every pixel in software on a 68000 *(verified: the Blit papers as recorded in* `docs/kernel-pain-points.md` *section 14I)*; a GPU's fixed-function scanout planes *(unverified)*; virtio-gpu 2D as the first iteration *(verified: errata 79)*.

**The author's referents.** The Blit, the C64, "gpu, vram, c64 and blit"; "graphics programming should be as simple and pleasurable as the c64 was".

**The assistant's leaning.** S4 is what the standard already is. Of the hardware shapes S2 matches his words most closely, and S1 is S2 without the blitter, which can be added later because it only reads and writes video memory; S3 is nearer if he wants commands rather than pokes as the main interface.

**The questions to answer in order to choose.** Who composes the layers (the hardware, the device's engine, or software); where glyphs live; whether sprites exist; whether there is a blitter, as registers or as commands; whether commands or pokes are the main interface; which virtio shape; VRAM or system memory; what pixel format (J.3.6).

**Where the body refers to it.** Section 3.9.1.

### J.3.4 Glyph slots against character codes

**Issue.** The author decided a **glyph map** (3.9.1): a table of slots, each holding the picture of one glyph, filled by a TrueType engine (software that draws a glyph from the font's outline at the display's pixel size) and used by every cell that names it. What is open, the details, is what a cell holds, a **character code** that a table maps to a slot, or the **slot number** itself; the layout of a slot (width, height, pixel format, whether a slot may span several cells for a wide glyph); and the number of slots. The author's earlier terminal chat held that "each cell holds its actual data, not a reference to the data", which suggests the code. As arithmetic only: a 1920 by 1080 screen of 8 by 16 pixel cells is 240 by 67 cells, 16,080 cells; with a cell of about 13 bytes that is 209 KB, and 4,096 slots of 16 by 32 coverage bytes are 2 MB, small against a 12.4 MB frame at 48 bits or 8.3 MB at 32.

**Why it matters.** The choice sets the program's work (naming slots against naming characters), whether the scanout needs a lookup table, and how Unicode, wide glyphs and fallback fonts are handled.

**Options.**

1. *The cell holds the character code;* a table maps code to slot. For: easy for the program, which never names slots; matches the chat-era cell. Against: a lookup table in the scanout; a policy for codes that have no slot.
2. *The cell holds the slot number.* For: nearly free in hardware. Against: the program or a library must allocate slots and keep a map from characters to slots.
3. *Both:* a cell has a flag that says which. For: ASCII by code and special glyphs by slot. Against: two paths in the scanout and the library.
4. *A glyph atlas, as one picture,* from which the cells or a blitter copy (the glyph map seen as the atlas of the GPU text path of 3.7). For: the same model as the GPU's; the blitter can draw a glyph anywhere. Against: needs the blitter of J.3.3 for the pixel layer.
5. *No slots:* the library draws glyphs into the pixel layer, as on virtio-gpu (S4). For: no hardware. Against: gives up the hardware text layer.

**Precedents.** The C64's character ROM of 8 bytes a glyph, replaceable by RAM *(verified: chapter 3, unverified numbers)*; VGA text mode's font in the adapter's memory *(unverified)*; the GPU glyph atlas of Alacritty, kitty and WezTerm *(unverified)*.

**The author's referents.** "with glyph maps we can even have a ttf engine generate glyphs and they will go into their memory slots for use"; "each cell holds its actual data, not a reference to the data" (the chat of 2026-06-18, `c64stuff.txt` line 5015).

**The assistant's leaning.** Option 1 for the program's sake, if the display is hardware; if it is the library of S4 the question disappears.

**Where the body refers to it.** Section 3.9.1.

### J.3.5 Sprites and the hardware cursor

**Issue.** A **sprite** is a small picture the display hardware overlays at a position the program gives, without touching the screen memory beneath. The C64 had eight. The Blit drew its cursor in software, in exclusive-OR, at 60 Hz. The author's description of 2026-11-10 does not mention sprites; the earlier terminal chat put them on a separate plane. virtio-gpu has a cursor in its protocol (the cursor queue, `UPDATE_CURSOR`, `MOVE_CURSOR`), and no sprites.

**Why it matters.** A moving cursor or game object either costs a redraw of the picture beneath it, which tears, or costs gates and a limit on the number.

**Options.**

1. *None:* the program draws the cursor in software. For: no hardware. Against: save, draw and restore, about 20 lines a program; may tear.
2. *The virtio-gpu cursor queue only.* For: standard; nothing to build. Against: one cursor; no sprites; the host decides the timing.
3. *A hardware cursor, two registers.* For: cheap; never tears. Against: a native device.
4. *A sprite layer of several entries.* For: the C64's model; games and moving objects without redraw. Against: gates in the scanout path; a limit on the number.
5. *Sprites as library objects over the cursor queue or a layer.* For: a program sees sprites. Against: not hardware; the speed is the library's.

**Precedents.** The C64's eight sprites *(unverified: general knowledge)*; the Blit's software cursor *(verified:* `kernel-pain-points.md 14I`*)*; the virtio-gpu cursor queue *(verified:* `virtio_gpu.h`*)*.

**The author's referents.** The C64 ("the c64 did games, so it had sound and color and sprites as well"); the Blit.

**Where the body refers to it.** Section 3.9.1.

### J.3.6 The 48-bit pixel against the formats of virtio-gpu

**Issue.** The author decided a 48-bit pixel, RGBA with 12 bits for each channel (errata 73), and a 48-bit sound sample (errata 74), as properties of devices that are deferred (appendix I). The standard video device, virtio-gpu 2D, has eight 32-bit formats with 8 bits for each channel (`B8G8R8A8_UNORM`, `B8G8R8X8_UNORM`, `A8R8G8B8_UNORM`, `X8R8G8B8_UNORM`, `R8G8B8A8_UNORM`, `X8B8G8R8_UNORM`, `A8B8G8R8_UNORM`, `R8G8B8X8_UNORM`), and virtio-snd's formats stop at 32-bit integers and 64-bit floats. A 48-bit pixel probably exceeds what a display connector carries (unverified), so the device that scans out would convert on the way out.

**Why it matters.** Software that renders at 48 bits has no standard device to show it on; the author's wish for 48-bit pixels and samples has no home until this is answered.

**Options.**

1. *The 48-bit formats belong only to a deferred native device.* The standard device is 32 bits. For: nothing is forked. Against: the decision of errata 73 has no standard effect.
2. *A pdp64 format added to virtio-gpu* (a 48-bit, or a 64-bit with 16 bits a channel, resource format) and likewise for virtio-snd. For: one transport; a format is a negotiation. Against: stock hosts and drivers reject it, so a fork is needed; the format lists are to be checked.
3. *Render at 48 bits in software, convert to 32 for the flush.* For: no change to the device; programs keep the precision. Against: twice the memory and a conversion on every frame.
4. *A 64-bit-per-pixel format,* 16 bits for each channel, as a power-of-two size. For: aligned; keeps all 12 bits. Against: not the author's 48; wastes 16 bits.
5. *The pixel width is a property of the scan-out hardware,* converted at the edge to what the transmitter takes. For: the standard protocol stays; the hardware keeps the precision. Against: the program cannot see it.

**Precedents.** virtio-gpu's formats, from the header *(verified:* `virtio_gpu.h`*)*; virtio-snd's formats *(verified:* `virtio_snd.h`*)*; 10- and 12-bit-per-channel deep colour over HDMI *(unverified: general knowledge)*.

**The author's referents.** "modern 48 bit audio and 48 bit pixels"; "the 48 bit is a new decision of mine, rgba format, 12 bits per color/channel"; "i want 48 bit sound as well, not just the sid".

**Where the body refers to it.** Section 3.12.1.

### J.3.7 The virtio feature subset, hot-plug and the OpenBSD drivers

**Issue.** virtio-gpu has optional parts: the EDID feature (the display reports the modes it supports), the fence flag, the cursor queue, blob resources and the 3D features of the header. virtio-snd has controls such as volume and flags for shared memory, polling and period events. The pdp64 must say which of these every implementation provides. Whether OpenBSD has a virtio-snd driver is not known (errata 79: "if not, one is written"); it has one for virtio-gpu (a directory listing). The scope of hot-plug (a monitor or a device arriving while running) is also open (`decisions-pending.md` F23).

**Why it matters.** An application can use a feature only if every machine has it; the three models (QEMU, `sim64`, the FPGA) must agree.

**Options.**

1. *The minimum:* 2D, fence, display information, no cursor, no EDID. For: least to implement. Against: no cursor, no real mode list.
2. *Minimum plus the cursor queue and EDID.* For: a hardware-like cursor; a monitor's real modes. Against: more to implement.
3. *All of the 2D features, none of 3D or blob.* For: everything a 2D driver uses. Against: the blob feature for a core's surface (J.3.12) is excluded.
4. *Everything the header offers.* For: no later restriction. Against: 3D and capability sets have no use on a core with no shader hardware.
5. *Whatever the first driver (QEMU's) needs,* settled by experience. For: honest. Against: the architecture states no requirement.

**Precedents.** QEMU's `virtio-gpu.c` rejects out-of-range rectangles, and has a `refresh_rate` field *(verified: the QEMU sources)*; OpenBSD `viogpu` *(verified: a fetched listing)*.

**The author's referents.** "yeah, virtio is fine"; "the simplest to program while still being efficient".

**Where the body refers to it.** Section 3.12.1.

### J.3.8 Anti-tearing conveniences

**Issue.** A **tear** is a seam in the picture when a program changes it while the scanout reads it. The author asked for "nice architectural sugar to avoid 'tearing' and all those other issues graphics programmers had to use double buffering to fix" (errata 105) and asked whether tearing is a concern with HDMI at all. It is not a property of HDMI: it happens in the display controller. A program that writes straight into the framebuffer with no operating system can still tear. Variable refresh (the monitor waits for the frame) reduces the need to flip at the blank but does not remove it. virtio-gpu defines no vertical blank and no page flip: a flush says "show this now". That the architecture offers such sugar is decided (3.15.1); which sugar, and whether a tear-free flip is worth hardware on a variable-refresh display, is the author's to say.

**Why it matters.** Every game and every compositor depends on how a flip is timed.

**Options.**

1. *A scanout base register that latches at the vertical blank,* with an interrupt or a fence. For: no tearing, no raster polling. Against: two pictures of memory; a rule for two writes in a frame.
2. *Per-layer latching.* For: a cursor moves every frame while text flips only when it changes. Against: one register set per layer; needs layers (J.3.3).
3. *Atomic commit of several registers at the blank* (Linux's "atomic modesetting"). For: a change of mode, picture and cursor cannot be seen half-done. Against: shadow copies cost flip-flops.
4. *A raster-position register and a raster interrupt.* For: the C64's mid-frame tricks. Against: ties programs to the display's timing.
5. *Versioned buffers:* the device keeps what is being read and the program writes a fresh version that replaces it atomically. For: nearest to "nothing to do". Against: memory and copying; page-granular copy-on-write needs the machinery of chapter 6.
6. *Damage rectangles and a fence only,* as virtio-gpu has. For: standard; nothing to build. Against: no timing at the blank; the host decides.

**Precedents.** The C64's raster register *(unverified)*; the VGA retrace *(unverified)*; Linux page-flip and atomic ioctls with `DRM_MODE_PAGE_FLIP_ASYNC` *(verified: `drm_mode.h`)*; "vblank evasion" in a Linux graphics driver *(verified: `kernel-pain-points.md` 0.6 #13)*; HDMI VRR and adaptive sync *(unverified)*.

**The author's referents.** "with modern hdmi is tearing even a concern?"; the C64's raster interrupt; "Flip on vsync unconditionally with an overrun interrupt instead of a barrier" (`c64stuff.txt` line 5647).

**The assistant's leaning.** Option 6 for the standard device, and Options 1 and 3 together if a native scanout engine is ever built.

**Where the body refers to it.** Section 3.15.1.

### J.3.9 The graphics boilerplate sugar

**Issue.** The author's goal: "basically, anything that made graphics programming annoying or cumbersome or boiler platey, we want to sugar away to almost nothing" and "graphics programming should be as simple and pleasurable as the c64 was" (errata 105). They are goals, not designs. The candidates are in the rows of the table of 3.14: a device that powers up in a sane default mode, in the monitor's preferred mode, which a register reports; a fixed canonical pixel format; stride equal to width; a framebuffer window always mapped; one colour space with the transfer curve applied by the device; cell and glyph sizes in physical units for HiDPI; a palette as an array in memory, or none; and one clock for sound and video.

**Why it matters.** Each one costs gates or flexibility, and the standard virtio-gpu device keeps its own protocol (five commands to put a picture up).

**Options.**

1. *Nothing in the architecture;* a library over virtio-gpu sugars it. For: no hardware. Against: every system writes its own.
2. *A default mode and a mode-report register,* the device reads the monitor itself. For: nothing to set. Against: a small I2C engine and a policy.

3. *A canonical pixel format with conversion at scanout.* For: no format to get wrong. Against: a conversion stage; wasted memory for fewer bits.
4. *Stride equal to width and the framebuffer window always mapped.* For: no `mmap`, no bank register. Against: a fixed place in the address space; a rule for two framebuffers.
5. *One defined colour space (linear or sRGB).* For: no gamma code. Against: the choice is open.
6. *Physical units for cells and fonts.* For: HiDPI without a scale factor in every program. Against: the TrueType engine must know the screen.

**Precedents.** VESA mode lists; Linux DRM/KMS; Vulkan *(unverified for the line counts, which are illustrative)*; virtio-gpu's `GET_DISPLAY_INFO` *(verified)*.

**The author's referents.** The Abrash books ("800 page michael abrash books"); the C64.

**Where the body refers to it.** Section 3.14.2.

### J.3.10 A write-combining memory type for the framebuffer

**Issue.** The framebuffer is mapped uncached (3.11.1). An uncached single write is slow, and a long run is no slower per byte. A **write-combining** memory type (writes are gathered and sent as one burst) would make processor drawing faster. The four memory types of 2.3.3 do not name it.

**Why it matters.** It decides how fast pixel-by-pixel drawing from the processor is, and whether a fifth memory type is added.

**Options.**

1. *Uncached only.* For: simple; no stale data. Against: slow single writes.
2. *Add a write-combining type to the page-table entry.* For: fast runs. Against: a fifth type; ordering rules for a `FENCE`.
3. *Let the display's buffer be device memory with its own merge rules.* For: the device controls it. Against: depends on the device.
4. *Draw into cached memory and copy by the device.* For: fast drawing. Against: the cache must be cleaned, which chapter 2 removes.

**Precedents.** Write-combining types on x86 and ARM *(unverified: general knowledge)*.

**The author's referents.** None.

**Where the body refers to it.** Section 3.11.3.

### J.3.11 The C128 core and the c64 mode

**Issue.** The c64 mode is decided (3.16.1, 3.16.2); what follows are its details. The author, 2026-11-10: "we will have a c64 mode, where a c128 is built onto the die with sid etc, so a bit of code loaded into its address space by pcie dma will let it act as a terminal the way the blit and vt55 did"; "the mode is just enabling/disabling the c128 as a pcie device so it takes over the graphics and audio output" (errata 105). The C128 is an optional PCI Express device, not part of the standard (errata 77). Open: which core (the C128 had an 8502, a Z80, the VIC-IIe, the 8563 VDC and a SID, with a memory-management chip); whether "c64 mode" runs unmodified C64 software or only terminal firmware; how video memory is shared with the host and with virtio-gpu; how the core and the host talk (a mailbox, a virtio device, a window of memory); how the loaded image is authenticated and contained (the IOMMU of errata 75 and 85 answers this for devices that master the bus; whether the core's DMA goes through it, whether the image is signed and who holds the key); how it relates to the deferred native devices (errata 78) and to "virtio across the board" (errata 95); its cost in gates (a 6502-class core is a few thousand gates, unverified); and how its sound reaches HDMI audio.

**Why it matters.** It decides whether the die holds a second machine, and whether the terminal is a program the host can replace, as in the Blit.

**Options.**

1. *A hard core on the die,* a real 6502-family core with the chips. For: fastest and smallest per core. Against: permanent area; fixes the idea in silicon before the questions are answered.
2. *A soft core in the FPGA phase only.* For: no silicon cost; the MiSTer framework does it. Against: gone from a later chip unless made hard.
3. *Software emulation of the C64 on a pdp64 core.* For: no hardware; easy to change. Against: uses a core; loses the C64's exact timing.

4. *None: the terminal is a virtio device served by a host process.* For: the standard route of errata 95. Against: departs from the decision that the C128 is built onto the die (3.16.1); the terminal is no longer a computer of its own, which the author values in the Blit.
5. *A core of the pdp64's own instruction set with the C64's registers.* For: one instruction set. Against: no C64 software runs.

**Precedents.** The Blit loaded its terminal program from the host *(verified:* `kernel-pain-points.md` *14I; the protocol is unverified)*; the C128 held several computers in one box and MiSTer runs C64 and C128 cores in programmable logic *(unverified: general knowledge)*; the earlier device chat's "8502 compatibility island" (`docs/c64-devices-from-chat.md` section 4, nothing decided).

**The author's referents.** The Blit and the VT-series terminals; "a bit of code loaded ... by pcie dma"; the C128.

**Where the body refers to it.** Section 3.16.1.

### J.3.12 The compositor path

**Issue.** The author prefers path 3 below, the core drawing into a buffer that the host's compositor treats as a surface (3.16.3); the other paths show what it was preferred over, and the open part is the details of path 3. A **compositor** is the program or hardware that combines the pictures of several windows or sources into the one picture that is scanned out. The author wrote of the c128 that it "takes over the graphics and audio output... or sends them to a compositor. not sure what that would look like. is there virtio-compositor?", and later "c128 drawing into a buffer makes most sense" (errata 105, 106: a stated preference, not a decision). No virtio device is a compositor: `virtio_ids.h` lists GPU (16), input (18), sound (25), video encoder and decoder (30, 31), DMA buffer (37) and audio policy (39). The nearest parts are up to 16 scanouts, `SET_SCANOUT_BLOB`, a separate cursor queue and capability sets. Open under the preferred path: how the compositor learns that the buffer is the core's, who owns the copy or shared memory, and how the core's frame timing meets the compositor's; and how the switch avoids a tear and a pop (a hand-over at a vertical blank and a short mute ramp).

**Why it matters.** It decides whether the core is a screen of its own or a window of the host's.

**Options.**

1. *Exclusive takeover.* The core owns scanout 0 and the audio path. For: an enable bit and a mute. Against: the host shows nothing meanwhile.
2. *One more virtio-gpu scanout and virtio-snd stream.* For: nothing new; the shape of errata 95. Against: the core must speak the protocols.
3. *A surface for the host's compositor* (the author's preference). The core renders into a resource or blob. For: a window with the host's window management. Against: the compositor must know the surface is the core's; a copy or a shared blob.
4. *Layers at scanout.* For: no copy; per-layer latch. Against: needs the layered display of J.3.3.
5. *A new virtio device type for compositing.* For: one model for every client. Against: a specification to write and have accepted, and a driver for every host; it repeats what virtio-gpu says.

**Precedents.** QEMU's `virtio-gpu-rutabaga`, "gfxstream rendering and Wayland display passthrough" *(verified:* `docs/system/devices/virtio/virtio-gpu.rst`*)*; crosvm shows a guest's windows as windows of the host's Wayland compositor *(unverified: general knowledge)*.

**The author's referents.** "c128 drawing into a buffer makes most sense" (errata 106); rio's windows as buffers.

**Where the body refers to it.** Sections 3.16.2 and 3.16.3.

### J.3.13 Audio mixing

**Issue.** A **mixer** combines the sound of several programs into the one stream a converter takes: it converts each stream to a common rate and format (resampling), applies a volume, sums and limits, balances latency against underrun and absorbs clock drift. virtio-snd defines one stream at a time, as one card, and says nothing about who mixes; QEMU mixes on the host. The pdp64 is a machine where the mixing may have no host. The author asked "what do those mixers do to combine multiple programs audio?" (errata 106). For the audio of a c64-mode core (J.3.11) the options parallel those for pixels: exclusive takeover; the core's voices as a virtio-snd stream that the host mixes (needs the 48-bit sample path of errata 74); a fixed-gain mix in the device; and a fade over a few milliseconds at hand-over so that no pop is heard.

**Why it matters.** It decides who resamples and whose clock rules, and whether two programs can play at once on the first board.

**Options.**

1. *The kernel mixes.* For: one place; programs see one stream. Against: kernel code and floating-point or fixed-point work in the kernel.
2. *A library mixes in the program* (as a sound server does). For: policy outside the kernel. Against: only one program may hold the device; a server is needed.
3. *The device mixes.* For: no CPU time. Against: gates for rate converters; a fixed policy.
4. *A mixing process with the device's clock driving it,* as PipeWire and PulseAudio do. For: a proven design. Against: a user-level server.
5. *One stream only,* no mixing at all. For: nothing to decide. Against: no two programs at once.

**Precedents.** PulseAudio, PipeWire, ALSA `dmix`, CoreAudio and WASAPI shared mode *(unverified: general knowledge; the programs `pulseaudio` and `libasound` are present on this machine)*; QEMU's mixing engine `audio/mixeng.c` *(verified: the QEMU sources)*; virtio-snd as one stream *(verified: `virtio_snd.h`)*; the SID mixes its voices inside itself *(unverified).*

**The author's referents.** "what is the audio equivalent, what do those mixers do to combine multiple programs audio?"; the SID.

**Where the body refers to it.** Section 3.17.1.

### J.3.14 Windows

**Issue.** The author: "it should mention blit terminal was inspiration for the rio and NeWS windowing systems, and we'd like to bring over what we can of that simplicity also" (errata 106). The Blit's `mux`, Plan 9's 8½ and rio, and NeWS are in 3.18. That the pdp64 brings over what it can of their simplicity is decided (3.18.1); which of their simplicities, and in what form, is open.

**Why it matters.** It decides what a program calls to put a window on the screen, and what the kernel provides.

**Options.**

1. *A window is a plain buffer and a rectangle.* For: drawing is writes to memory. Against: a buffer a window (memory); a rule for who may write whose.
2. *Drawing is writes to that buffer,* with no drawing protocol in the base system. For: no round trips. Against: every program draws for itself; a library must exist.
3. *The server composes the buffers* (a small program or the display hardware). For: one place for overlap, flip and tear. Against: a copy or a layer per window; trust.
4. *Downloadable server-side code,* as the Blit and NeWS had, here into the core of J.3.11. For: new abilities without changing the host. Against: a second program to secure; the Blit's "two halves" problem.
5. *No toolkit in the base system.* For: the size rio had. Against: every program has its own look.
6. *Windows as files, or as virtio queues.* For: the uniformity of rio and 8½. Against: a naming and permission model (chapter 6) or a queue per window.

**Precedents.** The Blit's `mux`; 8½ and rio *(verified: `kernel-pain-points.md` 14I)*; NeWS as an extensible window server *(verified as quoted from the UNIX-HATERS Handbook, `kernel-pain-points.md` 14G; that it was Sun's, in PostScript, by Gosling is unverified).*

**The author's referents.** The Blit, rio and NeWS; "that simplicity".

**Where the body refers to it.** Section 3.18.1.

## J.4 Chapter 4: Boot, the BIOS, the disk label and the GPT

### J.4.1 The first few pages

**Issue.** The author's rule (errata 102) is that device windows can go anywhere in physical space "except the first few pages where the boot stuff is loaded". How many pages are "the first few", and what exactly lives in them, is not fixed. The rule is a **reservation**: a range of physical addresses that no device window may overlap, so that the boot ROM and the first stage can assume plain memory there. What the documents already put in that range: the vector table (the table through which an exception or an interrupt finds its handler: 512 entries of 8 bytes, exactly one page of 4096 bytes at address 0, because the register `VBR` is zero at reset); block 0 (512 bytes), read to an address "to be chosen" (0x1000 is the example, which is why the load address is part of this entry); the boot-information block, whose size is not specified and which lists the memory

ranges, boot device, console, core count, device table, command line, ABI version and a checksum; and the top of the first 64 KB, which register r14 holds as the first stack.

**Why it matters.** Before translation is on, the kernel needs a known place for the vector table, the boot-information block and the early stack. The root complex must refuse to decode a window below the limit, and the boot ROM's load address and the reset value of VBR are tied to it. Section 4.4 and the boot sequence cannot quote an address until it is chosen.

**Options.**

1. *Two pages (8 KB).* Page 0 is the vector table; page 1 holds block 0 at 0x1000 with the boot-information block after it. For: the smallest reservation that holds what is already named; a window can start at 0x2000; the decoder compares against one constant. Against: block 0, a block of a few hundred bytes and a stack can overflow 8 KB when the device table grows; moving the limit later breaks every board that placed a window at 0x2000.
2. *64 KB (sixteen pages), matching the rule for r14.* Everything below the top of the first 64 KB is boot territory. For: reuses a number the reset state already defines, so the reservation, the initial stack and the load area agree; room for the vectors, block 0, the boot-information block, a first stack and the second-stage loader; costs nothing at 40 bits or more. Against: wasted if a board's RAM does not start at 0.
3. *1 MiB (256 pages), the low megabyte of the PC.* For: room for a second-stage loader, the first segment of a kernel image, a hand-off area for a frame buffer and the pages where other cores start. Against: wastes the low megabyte of a small FPGA board with tens of megabytes; invites sloppy use; invents a number.
4. *Nothing fixed by the architecture.* The platform states the limit in the boot-information block or the root complex; the architecture says only that page 0 is never a device window. For: every board picks; one page is enough for the vector table at VBR 0. Against: a loader and a kernel can assume nothing about low memory and must parse the block first, though r1 itself points into the reserved area; the documents lose a number to quote.
5. *Move the boot area to the top of physical memory,* or of the implemented space. For: address 0 becomes an unmapped page, so a null pointer used as a physical address is caught; the reservation moves with the width, so no device meets low RAM. Against: VBR zero at reset must change; every sentence of the reset state (r14, the load address of block 0) is rewritten; the top depends on the width, which is itself open.
6. *Reserve by type, not by count.* The root complex refuses windows only over ranges that the memory map in the boot-information block marks as RAM or reserved; there is no "first few pages" rule. For: "a window anywhere" becomes literally true. Against: whoever enumerates the bus needs the map before placing any window, and the same code that reads the block makes the first decisions; more code than a constant.

**Precedents.** The PDP-11 kept its trap vectors in the first octal 0 to 0377 bytes of memory, with the boot ROMs elsewhere *(unverified: general knowledge)*. The RISC-V virt board of QEMU has a debug region at 0x0, the boot ROM at 0x1000 and devices from 0x2000000, and 0x1000 is the default reset vector *(verified: QEMU hw/riscv/virt.c and target/riscv/cpu_bits.h)*. The PC reserves the first megabyte of real mode for vectors, BIOS data and the boot sector loaded at 0x7C00 *(unverified: general knowledge)*. The VAX console passes a boot block and a restart parameter block that lives in low memory *(unverified: general knowledge)*. The PDP-11 I/O page sat at the very top of its address space *(verified in the project's chapter 2 commentary of the V7 drivers)*, and x86 resets at 0xFFFFFFF0, at the top of 4 GB *(unverified: general knowledge)*. The PC BIOS E820 map and the UEFI memory map type each range *(unverified: general knowledge)*.

**The author's referents.** The PDP-11 boot ("load 512 byte block 0 then jmp 0"; "not precisely true, but directionally true"); "follow the majority as far as boot block offset"; the VAX's I/O space (memory low, I/O above; "it should be noted in the book how the vax does this io space idea"); UEFI ("is it worth it at all?", since decided against); the GPT as the disk label.

**The assistant's leaning.** Option 2, 64 KB, because the reset state already defines the first 64 KB through r14, so no new number is invented; the vector page and block 0 fit with room; and the cost is nothing at 40 bits. It would change if a board's RAM does not start at 0, or if the boot-information block grows beyond 8 KB (then Option 3 or 4). To decide it, the author's size for the boot-information block and for the second-stage loader would help, and whether the vector table stays at 0 after boot (VBR can be moved).

**Where the body refers to it.** Sections 2.3.5, 4.4.1 and 6.12.

### J.4.2 How the first stage reads blocks

**Issue.** The first stage is the program in block 0, one 512-byte block, and its job is to read the next stage from the disk. The ROM has done the same job once, with a virtio-blk request: a **virtqueue** (the shared ring through which a driver hands requests to a virtio device), three descriptors (a request header, a 512-byte buffer and a one-byte status), a write to the doorbell register, and a poll until the device returns the chain. The PDP-11's first stage carried its own disk driver; the PC's called the BIOS. Which of those the pdp64's first stage does is not decided. If the block is a protective MBR, only its first 446 bytes are code, which is also the space for whatever the first stage needs to know about where the second stage lies.

**Why it matters.** The size of the first stage, what the ROM must leave behind for it, and whether the ROM offers anything after boot all follow from the answer. The ROM and the first stage cannot be written until it is settled.

**Options.**

1. *A small polled virtio-blk reader inside the first stage.* The PDP-11 way. For: no dependence on the ROM; the ROM keeps no service after it has run. Against: the reader, the list of what to read and a stack-and-queue set-up may not fit in 446 bytes; it duplicates the ROM's reader.
2. *The ROM leaves a read routine in memory and tells the first stage where it is.* For: the first stage is a few instructions. Against: it is a firmware call in miniature; the routine's address and calling convention become part of the architecture and the ROM's code can then never change.
3. *The ROM loads more than block 0*, for example the whole 8 KB boot area. For: room for a reader and for more than a list. Against: the ROM must learn a size and a place, so it is no longer "read block 0 and jump"; it conflicts with a block 0 that is the protective MBR, since the rest of the 8 KB belongs to the GPT.
4. *The ROM reads the second stage itself,* from a list in block 0 that the ROM parses. For: there is no first-stage reader at all. Against: the ROM is larger and the list format is frozen in ROM; block 0 becomes data and not a program.
5. *A simple programmed-I/O read register on the boot device or the controller,* beside virtio. For: a few instructions, no queue. Against: a second device interface that every board must implement.

**Precedents.** The V7 primary bootstrap for the RM02/3 writes the disk's registers itself *(verified: `sys/mdec/hpuboot.s`)*. OpenBSD's PC first stage calls `INT 0x13` *(verified: `biosboot.S`)*. OpenBSD's `installboot` hard-codes the block numbers of the second stage into the first stage *(verified: `installboot(8)` for the sparc, and `biosboot(8)` for the PC)*. The 4.3BSD-Tahoe VAX `raboot` is documented as working with the 750's boot ROM and reading 7.5 KB *(verified for the header; how the ROM helps it is unverified)*.

**The author's referents.** The PDP-11 boot ("load 512 byte block 0 then jmp 0"); "follow the majority as far as boot block offset"; the GPT wrapper of errata 50.

**The assistant's leaning.** Option 1 if a measured reader fits, because it needs nothing from the ROM; Option 3 if it does not. A measurement of a polled virtio-blk read in assembler for the pdp64 would decide it.

**Where the body refers to it.** Section 4.5.2.

### J.4.3 How the ROM finds and maps the boot device

**Issue.** Section 2.4 says that the processor assigns the windows of the bus's devices. The ROM is the first software to run, and to read block 0 it must reach the boot device's registers and queues, which lie in a window that nobody has assigned yet. It is not stated whether the ROM assigns every window or only the boot device's, where it places them, and whether it leaves them for the kernel to keep or to change. The author's rule (errata 102) keeps windows out of the first few pages (J.4.1).

**Why it matters.** The kernel's own enumeration starts from the state the ROM leaves. The root complex's reset state, the ROM's size and the boot-information block's description of the root complex all depend on it.

**Options.**

1. *The ROM enumerates the whole bus and assigns every window,* as PC firmware does. For: the kernel finds a mapped machine. Against: a long program in the ROM, the largest of the options.
2. *The ROM assigns only the boot device's window,* at a fixed address, and the kernel enumerates afresh. For: a few lines; the ROM needs one function. Against: the kernel must keep that window or move it, and the second stage must be told which.
3. *The boot device's window is decoded from reset* at an address fixed by the board, so that the ROM assigns nothing. For: no ROM code for it. Against: it makes the boot device a special case beside the rule that the processor assigns windows; the address is per board.

4. *A table in the ROM lists where the boot devices are,* per board. For: no scan. Against: a port to a new board is a new ROM; the table duplicates what the bus reports.
5. *The ROM maps, reads, then unmaps everything it assigned,* and the second stage and the kernel begin clean. For: one rule for every later stage. Against: the second stage must map the device again to read the next block, so the code is written twice.

**Precedents.** PC firmware assigns the base address registers of the devices before boot *(unverified: general knowledge)*. QEMU's RISC-V virt board places its devices at fixed addresses, and 0x1000 is its reset vector *(verified:* `hw/riscv/virt.c`*)*. The PDP-11's I/O page was fixed by the hardware at the top of the address space *(verified in chapter 2's commentary of the V7 drivers)*.

**The author's referents.** PCI Express enumeration (2.4); the PDP-11 and VAX I/O space.

**Where the body refers to it.** Section 4.4.1.1.

### J.4.4 The boot-information block: fields, who fills them, layout

**Issue.** The kernel must be told about the machine it is on: memory ranges, the number of cores, the address of the root complex, the boot device, the console, a command line, and, when the loader has set one up, the screen (decided: 4.6.3). That the kernel is told through a boot-information block, and that the bus lists the devices (so the block need not), is decided (4.6.2, errata 109 and 49). It has been assumed that the ROM fills what it knows (memory, cores, the device) and the second stage adds the command line and the screen, and that the address of the block is passed in `r1`; none of that is decided. The details that are open: the fields, and the layout (order, field widths, a checksum, a version, a header), whether the block is generated from a grammar like the rest of the specification, and whether it is replaced or joined by a device tree are open.

**Why it matters.** The ROM, the second stage and the kernel are written against it; the generated headers and the ABI appendix cannot carry it until it is fixed.

**Options.**

1. *A fixed-layout record, with a version and a checksum, generated from a grammar.* The ROM fills the fixed part; the second stage adds the rest. For: simple to read in a few instructions; the header cannot disagree with the loader. Against: a new field means a new version.
2. *A list of tagged records (tag, length, value).* For: extends without breaking old readers; a loader adds its own. Against: a parser in the kernel, though a short one.
3. *A flattened device tree built by the ROM or the second stage.* For: OpenBSD's arm64 and riscv64 drivers attach from a device tree, so reusing them would save kernel code. Against: a parser and a second description of the devices beside bus enumeration, which already lists them.
4. *A record plus a device tree* for the devices that enumeration cannot find. For: both worlds. Against: two things to build and keep consistent.
5. *Registers only,* as on the PDP-11 and VAX. For: no structure. Against: a kernel for any pdp64 needs the memory ranges, which a few registers cannot hold.
6. *A memory map alone;* everything else by enumeration. For: the smallest. Against: the console, the core count and the command line have no place.

**Precedents.** The VAX restart parameter block and the Alpha hardware restart parameter block are fixed-layout records of the machine *(verified: 4.3BSD* `vax/rpb.h`*, OpenBSD* `alpha/include/rpb.h`*)*. UEFI passes a memory map and configuration tables for ACPI or a device tree, and OpenBSD's loaders look for both *(verified:* `efiboot`*)*. OpenBSD's arm64 and riscv64 kernels parse a device tree *(verified through* `efiboot`*'s* `FDT_TABLE_GUID`*)*.

**The author's referents.** UEFI ("is it worth it at all?"), since decided against; "load 512 byte block 0 then jmp 0"; errata 109: the hand-off "kept simple enough that a UEFI shim could be written".

**The assistant's leaning.** Option 1, with a version field: the bus describes its own devices, so the block only has to carry what enumeration cannot (memory, cores, root complex, boot device, console, command line, screen). It changes if the OpenBSD device-tree drivers are to be reused, which would favour Option 4.

**Where the body refers to it.** Section 4.6.2.

### J.4.5 Firmware for a virtio device

**Issue.** The author decided (4.8.1; R256) that device initialisation happens behind the bus, a register write telling the device to initialise itself, and that a device may also be given values or a binary firmware image to load. What is open is how. Some devices contain a small processor and need a program, **firmware**, loaded into them before they work; a network or wireless card is the usual case. The PDP-11 had no such device. Virtio has no firmware operation of its own. A device the boot depends on (the disk, the console, the interrupt controller) cannot wait for firmware that lies on the disk it controls, so it must work from built-in logic.

**Why it matters.** Without a mechanism, such a device cannot be used, or each vendor invents one.

**Options.**

1. *A firmware request type on the device's own queue.* For: one queue, the mechanism the device already has. Against: virtio would need a new request type for each device class.
2. *A descriptor that gives the address and length of an image in memory, with a small header* (identifier, version, length, checksum) that the device checks. For: the device pulls the image by DMA; the header lets it refuse a wrong image. Against: a header format to specify.
3. *A separate small virtio device for firmware loading* that other devices consult. For: one mechanism for every device. Against: a second device, and a dependency between devices.
4. *A firmware window in the device's register space,* written by the driver. For: needs no queue. Against: programmed I/O of a whole image.
5. *No loaded firmware at all:* every device carries its program in its own flash. For: nothing to specify. Against: the device is dearer, and an update is a hardware operation.

**Precedents.** Linux `request_firmware`, OpenBSD `loadfirmware(9)` (reads from `/etc/firmware`) and NetBSD `firmload(9)` load a blob from a file at attach time *(verified: read in the trees)*. NVMe has a firmware-download command *(unverified; not found in the QEMU headers read)*. The VAX console and some adapters loaded microcode from storage *(unverified)*.

**The author's referents.** "the init protocol should mostly happen behind the bus, poke a register to tell the device 'initialize yourself' and possibly have another register populated with values or a binary firmware blob for the device to load" (R256); the `INIT` and `READY` bits of the Unibus device (errata 29); virtio's `device_status` handshake (errata 95).

**Where the body refers to it.** Section 4.8.1.

### J.4.6 Starting the other cores

**Issue.** At reset only core 0 runs, and the controller chip provides the way the others are started (4.7.1; the author, R256: "we'll need a way for the controller chip to 'boot' the other chips"). The open part is the mechanism. The kernel must start the others, each with its own stack and its own identity. The mechanism is not decided: how a core is given its entry address and an argument, what registers and mode it starts with (and so whether it starts with translation off, and what `MMUCTL` it sees), which registers carry its core number and the argument (chapter 7 states one rule, `r0` only, and the first design of this chapter a second, `r0` and `r1`), and how a core that sticks is reset.

**Why it matters.** The kernel's secondary-start code, the controller's register block and the boot-information block's core count are written against it.

**Options.**

1. *A register block in the controller: write an entry address and an argument to the core's start register, and the controller releases the core into kernel mode at priority 7.* A core-reset register restarts a core that sticks. For: no trampoline and no firmware call; one write. Against: a register block with a start and a reset register for every core.
2. *The x86 way: INIT and START-UP signals through the interrupt controller.* For: a known protocol. Against: the signal carries only an address below a limit; the x86 needs a trampoline because the woken core starts in a mode ladder, which the pdp64 does not have.
3. *A firmware call* (PSCI on arm64, the SBI hart start on RISC-V). For: a standard form on two architectures. Against: it needs resident firmware after boot, which the project does not otherwise have.
4. *A spin table:* every core runs from reset and waits on a word in memory until core 0 writes an address there. For: no register block. Against: the cores run before the kernel has set up memory, and they must be told where the table is.
5. *All cores start at the same address and choose a role from their number.* For: no start protocol. Against: the first stage and the ROM's hand-off must handle every core at once.

6. *An inter-processor interrupt* from core 0 carrying the entry address. For: uses the interrupt controller already needed. Against: the interrupt controller must be programmed on the started core first, a circle.

**Precedents.** Linux counted: `smpboot.c` 1,528 lines and `arch/x86/realmode` 1,258 for x86; PSCI 124 and the spin table 127 on arm64; the SBI start 108 and the spin wait 58 on RISC-V *(verified, re-counted in Linux `7be4bc52`)*. The VAX console processor started the other processors *(unverified)*.

**The author's referents.** The PDP-11, which had one processor; "we'll need a way for the controller chip to 'boot' the other chips; do a thorough review of how all the main smp systems do it, which is best/easiest" (R256). He has not chosen among the options.

**The assistant's leaning.** Option 1: it removes the trampoline and the firmware call and recovers a stuck core by one write.

**Where the body refers to it.** Sections 4.2 and 4.7.1.

### J.4.7 The order of the boot media

**Issue.** The ROM must choose a boot device when more than one is present: a disk or flash, an SD card, a serial line. Which comes first, whether a key at power-up changes it, how the chosen device is named in a register for the later stages, and whether the order is kept in non-volatile memory is not decided (F10 of `decisions-pending.md` leaves the order after the virtio-blk disk open).

**Why it matters.** A board with two disks or a removable card cannot boot predictably without it, and the ROM's device list and its use of non-volatile memory follow from it.

**Options.**

1. *A fixed order wired into the ROM:* disk or flash, then SD card, then the serial line. For: no state. Against: no way to boot from the card when a disk is present, short of removing the disk.
2. *A fixed order, changed by a key held at power-up.* For: a rescue path without storage. Against: a key input every board must offer.
3. *An order kept in non-volatile memory.* For: the user chooses. Against: the memory must exist, and a bad setting leaves a machine that does not boot, so a reset to defaults is needed.
4. *The order of the bus: the first device in enumeration order that has a valid block 0.* For: no table at all. Against: the order depends on the wiring, not on the user's intent.
5. *One boot device per board,* fixed by the board's description. For: the simplest. Against: no choice and no recovery medium.

**Precedents.** The PC BIOS keeps a boot-device order in its setup memory *(unverified)*. UEFI keeps boot variables in non-volatile memory, which its loaders read and write *(verified: `GetVariable` and `SetVariable` calls in `efiboot`)*. The DEC boot ROM cards chose the device by a console command or switch *(M9312 comment verified; the command unverified)*.

**The author's referents.** The PDP-11 ROM; no statement of his on the order was found.

**Where the body refers to it.** Section 4.4.

### J.4.8 Where the second stage lies, and a dedicated boot partition

**Issue.** The first stage loads the second stage from somewhere on the disk. Three places are candidates: the 8 KB boot area at the start of the BSD partition (4.5.3), a file in the root file system, and a small partition of its own with a partition type chosen for it. The second stage holds a virtio-blk driver, a GPT reader, a label reader and a file-system reader; OpenBSD's amd64 second stage is 189 lines beside a `libsa` of 6,319 lines *(counted)*, and whether such a program fits the 7.5 KB left in the boot area is not measured.

**Why it matters.** It decides whether the disk layout needs another partition type, what the installation tool writes and where, and whether the second stage has a size limit.

**Options.**

1. *The 8 KB boot area of the BSD partition.* For: the BSD tradition (sector 0 the primary bootstrap, sectors 1 to 15 the label and a secondary bootstrap); no new partition. Against: a hard size limit of 7.5 KB, and the label occupies sector 1.
2. *A file in the root file system,* its blocks written into the first stage by `installboot`. For: no size limit. Against: the list must be rewritten whenever the file moves (`biosboot(8)` says to re-run `installboot` "as its inode may change"), and a damaged root file system stops the boot before the second stage.
3. *A dedicated boot partition with its own type.* For: a place that no file system moves; room for a larger loader. Against: another partition to create, to size and to keep, and a type to assign.

4. *The EFI system partition (FAT).* For: the same disk serves a UEFI route (4.9). Against: a FAT reader in the second stage, and a partition that the pdp64's own boot does not need.
5. *The gap between the GPT and the first partition.* For: no partition entry. Against: a tool that repartitions the disk may overwrite it.

**Precedents.** 4.3BSD-Tahoe and 2.11BSD reserve 15 sectors after block 0 for the bootstraps *(verified: `disklabel.h`)*. OpenBSD on the PC patches `biosboot` with the location of `/boot` *(verified: `biosboot(8)`)*. OpenBSD defines both the OpenBSD-type and the EFI-system partition GUIDs *(verified: `disklabel.h`)*. Other systems keep a "BIOS boot partition" for a loader *(unverified)*.

**The author's referents.** The GPT wrapper (errata 50); "follow the majority as far as boot block offset" (errata 51); the 2026-11-10 choice of block 0 with a UEFI route kept possible (errata 109).

**Where the body refers to it.** Sections 4.5.2 and 4.5.4.

### J.4.9 The state core 0 starts in

**Issue.** The ROM starts core 0 at block 0 (4.4.1, decided). What core 0 holds when it starts is not decided: its mode and priority, whether translation is on, `VBR`, the user-memory guard, which registers carry the boot device and the boot-information block (4.1 says the pdp64 passes `r0` and `r1`, after the PDP-11 form), and the stack.

**Why it matters.** The first stage is written against it; it must not guess.

**Options.**

1. *Kernel mode, priority 7, translation off, `VBR` zero, guard clear; `r0` the device, `r1` the boot-information block, `r14` the top of the first 64 KB, the rest zero.* For: the first stage has a stack and every fact before it runs an instruction; the PDP-11 handed on registers the same way. Against: the top of 64 KB is a number to fix with J.4.1.
2. *As option 1 but the stack is left to the first stage.* For: no stack number. Against: 446 bytes spend instructions on it.
3. *Only the boot-information block's address in one register; everything else a documented reset state of the core.* For: fewest registers. Against: the device number moves into the block.

**Precedents.** 2.11BSD passes `r0`, `r1`, `r3` and a check word in `r2` *(verified: `pdpstand/srt0.s`, `boot.c`)*; the VAX boot passes `r1` to `r5` *(verified: `raboot.c`)*.

**The author's referents.** "load 512 byte block 0 then jmp 0".

**The assistant's leaning.** Option 1.

**Where the body refers to it.** Section 4.4.1.

### J.4.10 The monitor: commands, how it is reached, its console

**Issue.** The controller's ROM includes a monitor (4.10.1, decided). Open: its command set, how it is reached (when nothing boots, a key at power-up, or both), its console and speed, whether it loads a program over the serial line, and what is kept in non-volatile memory (boot order, serial speed). Whether a hardware watchdog, which restarts the machine when the kernel hangs, belongs beside it is the assistant's proposal and is in appendix H.

**Why it matters.** The monitor is the way in when nothing boots, so its ROM size and its dependence on a serial port decide how a new board is brought up.

**Options.**

1. *Examine, deposit, load over serial, start at an address, start from a device, reset a core, reset the machine.* For: the PDP-11's `ODT` plus a loader. Against: a few hundred bytes of ROM (an estimate) and a working serial port.
2. *Examine, deposit and start only, the `ODT` set.* For: the smallest. Against: no program can be loaded when the disk is dead.
3. *A boot menu (choose a device) only.* For: simple. Against: no inspection of a dead machine.

**Precedents.** `ODT` on the PDP-11 *(2.11BSD `cons(4)` read; the commands unverified)*; U-Boot and OpenSBI consoles *(unverified)*.

**The author's referents.** Errata 47: "the boot ROM and its monitor".

**The assistant's leaning.** Option 1.

**Where the body refers to it.** Section 4.10.1.

## J.5 Chapter 5: The device driver model

### J.5.1 The console, the timer and the entropy source

**Issue.** What is decided is in 5.7.1 and 5.7.2: the main and secure cores take no periodic tick, and the entropy pool is one shared pool that assists the operating system's generator. What is open is the form of the console, of the timer that serves a tickless machine, and of the way the pool is reached and fed. Every standard device of the pdp64 is a virtio device, found by PCI Express enumeration, and three things the machine cannot do without are still unassigned: the **console** (the channel for text in and out, for boot messages and for the last words of a crashing kernel), the **timer** (a source of ticks or one-shot interrupts, and a counter that tells the time) and a **source of random numbers**. Virtio defines a console device and an entropy device (a device that hands out random bytes). It defines no standard timer; a real-time-clock device has been proposed (unverified). A timer can instead live in the processor, as a counter and a compare register in the system registers (the ARM generic timer and the RISC-V `time` register with its compare extension are built this way), or in a small memory-mapped block (a counter and a compare register at a fixed address, as the RISC-V machine timer and the PC's HPET are). On entropy the author has said that he is "not sure random number in hardware is a good idea", and the decided shape of the pool (errata 85, now statement 5.7.2) is one pool shared by all cores, with no per-process pool, no version register and no updatable firmware: it assists the operating system's own generator, which feeds it and draws from it.

**Why it matters.** The console must work before enumeration: the boot ROM, a kernel panic and a machine whose bus is broken all need it. The timer must work before any driver has run. A virtio console needs a queue that the boot code has set up; a fixed UART (universal asynchronous receiver-transmitter: a simple serial port with a data register and a status register) needs two registers. OpenBSD's random-number generator must work from its first call. A timer in the system registers adds registers, and possibly a `RDTIME` instruction, to the instruction set and ABI; a memory-mapped timer adds nothing to either. A fixed UART or timer costs tens of gates, a virtio console costs a queue engine, an entropy source is a physical circuit (ring oscillators and a conditioner) whose quality the standard would be promising, and per-core comparators grow with the core count. Section 5.7, the vector table of 6.3.6 and the boot hand-off of chapter 4 wait on the answer.

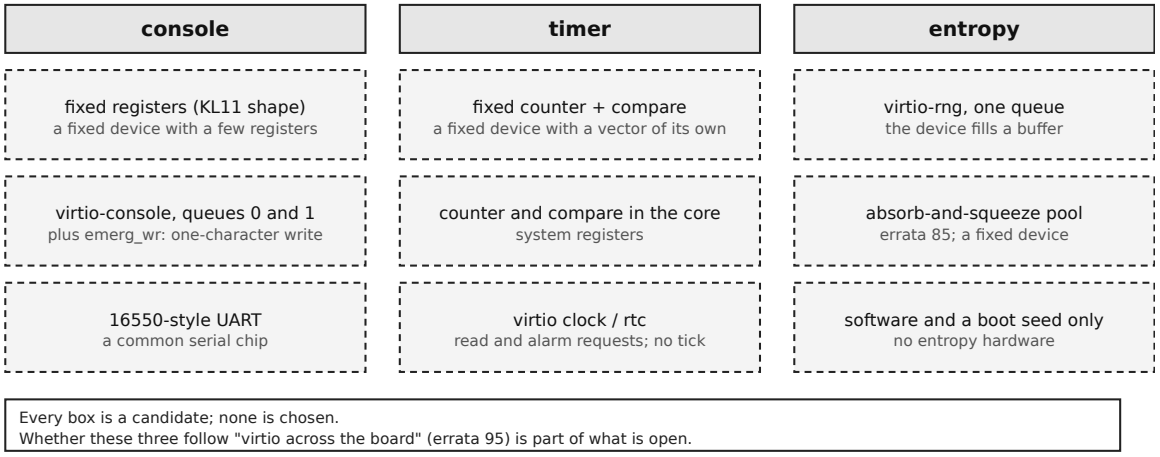
**Options.**

1. *Fixed console and timer, virtio-rng for entropy.* A UART-style console and a counter-and-compare timer sit at fixed addresses, found by a small device tree or by the boot-information block; entropy comes from virtio-rng in front of a physical source. For: both fixed devices work without enumeration or a queue, so the boot ROM and the panic path stay tiny; only entropy needs a virtio driver. Against: the manual then has two device models, and "virtio across the board" (the author's words) may not have meant that; one timer shared by all cores forces a lock or per-core windows; wired interrupt lines from the fixed devices need the converter of J.6.3.
2. *Timer in the core, fixed UART, virtio-rng.* A shared constant-rate counter and a per-core one-shot compare register, written with `MTSR` and read with `RDTIME`; a minimal fixed UART for boot and panic; entropy by virtio-rng. For: the timer cannot fire on the wrong core, tickless operation is one write, the secure core simply never writes its compare, and the kernel interface is what OpenBSD's clock-interrupt code needs (read the counter, set a one-shot compare, take the interrupt). Against: it adds system registers and probably `RDTIME` to the instruction set (with the side-channel switch that goes with it); the counter must reach many cores with a stated bound on skew; the architecture, not only the platform, now defines the timer; `RDTIME` needs an ordering rule for out-of-order cores; the UART is still a non-virtio device.
3. *Everything enumerated.* The console is virtio-console, found by enumeration (the boot ROM carries a small polled virtio driver, or a console private to the ROM); entropy is virtio-rng; the timer is in the core as in option 2. For: the strictest reading of "virtio across the board"; one transport driver and one enumeration walk serve console, entropy, disk, network, graphics and audio; OpenBSD already has drivers for both devices. Against: the console cannot work before the bus does, so when the queue, the IOMMU mapping or the root complex is wrong nothing prints, and the earliest boot is hardest to debug where debugging is most needed; a panic path that first needs a descriptor in uncached memory and a doorbell is longer than a store to a data register; a queue engine in hardware just for text.
4. *The absorb-and-squeeze pool as a fixed device.* A write mixes into a pool of 256 or 512 bytes and a read returns fresh randomness; one device shared by all cores, no version register, no firmware; console and timer as in option 1 or 2. For: it is the author's own proposal, accepted in errata 85 ("a read with a known and wanted effect is not a strange side effect"); it feeds the operating system's generator without replacing it; the hardware is a small pool and a mixer. Against: it would break if OpenBSD changed its randomness algorithm (the author's own remark); a read with an effect on shared state is

a known hazard (the shared-buffer leaks of 2020 in x86 processors); the device is neither virtio nor standard and needs a PCI Express function or a fixed window; the author's doubt argues against putting more in silicon than a physical source.

5. *No entropy hardware.* OpenBSD's generator is used as it is: it gathers timing from interrupts, disk, network and clock, and starts from a seed file written at the last shutdown, with a flag that says whether the seed was good; no virtio-rng; console and timer as in option 1 or 2. For: it answers the author's doubt with no hardware to trust, costs no gates and is the smallest design; OpenBSD's design assumes weak seeding and copes with it. Against: the first boot with no good seed is weak, and so is a clone of a virtual disk (the seed repeats); main cores are tickless by the author's rule, so fewer timer interrupts feed the generator; address-space randomisation spends entropy, and the address space is small.
6. *Virtio-console, with its emergency write for boot and panic.* The console is virtio-console, found by enumeration; the boot ROM and the panic path use the feature `EMERG_WRITE` (feature bit 2), which lets a driver print one character by storing it in the `emerg_wr` field of the device configuration (offset 8), with no queue, no descriptor, no memory buffer and no IOMMU entry; the ordinary tty driver uses the queues. For: it keeps the console a virtio device and the panic path a store; the field is in the standard; OpenBSD's `viocon` and QEMU already carry the device. Against: the store is to a mapped window found by enumeration and not to a fixed address, so the boot ROM still needs the bus walk and the capability search before it can print; OpenBSD's `viocon.c` does not use `EMERG_WRITE`, so the panic path would be new kernel code; a device or emulator that does not offer the feature leaves no way to print.

**the console, timer and entropy source: the options (OPEN, 5.7.1; nothing decided)**




A virtio-console needs the bus and a queue before it can print; its emergency write needs the bus and no queue. A fixed console needs neither.

Sources: spec/pdp64-system.hbnf, qemu/hw-pdp64, virtio_console.h, virtio_rtc.h, QEMU virtio-rng.c (read)

**Precedents.** QEMU's `virt` machines for ARM and RISC-V place a UART at a fixed address, and on RISC-V a core-local timer, both described to the kernel by a device tree (a data structure the firmware passes to say what the machine contains), with virtio devices beside them *(unverified)*. The RISC-V `stimecmp` register (the supervisor timer compare register, with the Sstc extension) is written directly where present, and by a firmware call where it is not *(verified: OpenBSD riscv64 `clock.c`, lines 63 to 70)*; OpenBSD's whole riscv64 clock backend is 175 lines *(verified)*. The ARM generic timer has a shared system counter and a per-core compare *(unverified)*. x86's TSC-deadline mode (a per-core one-shot compare against the time-stamp counter) depends on an invariant counter, and the Linux maintainers' own remark on it is that "the stupid hardware is not reliable" *(verified: quoted in the repository)*. Firecracker and QEMU's microvm use virtio for the console where no UART is wanted *(unverified)*. OpenBSD has `viocon` and `viornd` drivers; `viornd` asks for 16 bytes at boot and then every 15 seconds *(verified by a fetched listing only)*. OpenBSD's generator absorbs events into a pool and draws ChaCha20 output from it *(verified)*; on machines with no hardware generator it runs the software path with a seed file read by the boot loader *(verified)*. Intel `rdrand` and `rdseed` and the AArch64 `RNDR` register are hardware sources that OpenBSD reads and mixes *(verified: source files of the local tree)*; RISC-V's Zkr extension provides a `seed` register of raw bits for the operating system to condition *(unverified)*. What virtio provides *(verified: the Linux headers, QEMU and OpenBSD sources read)*: console is device type 3, port 0 has a receive and a transmit queue (queues 0 and 1 in QEMU's `virtio-serial-bus.c`, with further control queues when

MULTIPORT is negotiated), the configuration holds cols, rows, max_nr_ports and emerg_wr, and QEMU passes the low byte written to emerg_wr to the port; OpenBSD's viocon.c (617 lines) drives the queues through the tty layer and asks only for SIZE. Entropy is device type 4 with one queue: the driver posts a device-writable buffer and the device fills it with random bytes (QEMU virtio-rng.c, 8 entries); OpenBSD's viornd.c (201 lines) passes each word to enqueue_randomness, so the device feeds the kernel's generator, the shape of errata 85. Timer: the identifier list has VIRTIO_ID_CLOCK (17) and virtio_rtc.h defines a clock-reading request, a cross-timestamp request and an alarm (VIRTIO_RTC_F_ALARM, SET_ALARM), and QEMU has a vhost-user-rtc device; whether a periodic tick or a one-shot compare could be served by it is not stated by any file read. The PDP-11 had a KL11 console (a receive and a transmit register, BR4, vectors 060 and 064) and a KW11-L line clock (BR6, vector 0100) at fixed addresses with no probe *(verified: 2.11BSD cons.c, scb.s)*.

**The author's referents.** "forget the native 8 register devices. gone. virtio across the board." (chosen); for the boot console, the PDP-11's "load 512 byte block 0 then jmp 0" (considered). He named no console, timer or entropy system from elsewhere.

**The assistant's leaning.** Entropy: option 5 with virtio-rng as an optional device, no instruction and no register, and the pool of option 4 left for the author to revive only if virtio-rng proves too slow. Timer: option 2, a counter and compare in the core. Console: a minimal fixed UART kept for the boot ROM and the panic path, with virtio-console optional for ordinary use. The reason is that the earliest boot and the panic path are the author's first priority (errors must be easy to detect), and a data register that the boot ROM can write in one store works before any queue exists; the timer in the core removes a device and a lock and matches the tickless rule; the entropy doubt is respected by keeping any physical source behind virtio-rng and mixing it, never using it alone. A decision that no non-virtio device is allowed would favour option 3, at the price of a harder first boot; a need for a hardware key-generation source in the secure core would add a seed register of raw bits.

**Where the body refers to it.** Sections 5.7.1 and 5.7.2.

### J.5.2 The features that the pdp64 accepts

**Issue.** Before any queue runs, a virtio driver reads the features the device offers and writes back the subset it accepts, and the device confirms it (5.4.4). The version-1 feature is required. Everything else is a choice that the architecture has not made: which features a driver accepts for the disk (flush, block size, segment limits, read-only), the network card, the display and the sound device, whether the platform declares that device addresses go through the IOMMU (VIRTIO_F_ACCESS_PLATFORM, feature 33) and whether it declares that the platform orders memory accesses by its own rules (the platform-ordering feature, VIRTIO_F_ORDER_PLATFORM, feature 36, *unverified*).

**Why it matters.** The features fix what a device may do to memory and in what order: with ACCESS_PLATFORM the descriptor holds a device address that the IOMMU of 2.11 translates, and without it the device is told to use physical addresses and the IOMMU has nothing to do. The ordering feature decides whether the FENCE of 5.3.5 and the device-side order of 5.10.3 are enough. The choice also decides how much of each virtio device the first operating system's drivers must handle.

**Options.**

1. *The minimum.* Version 1 and the features a device class cannot work without; nothing else. For: the smallest driver, the least to test, the fewest paths in hardware. Against: no flush on the disk means a write that is not known to be on the medium; no segment limit means a driver that must discover failure by an error.
2. *The set that OpenBSD's drivers already ask for.* vioblk asks for RO, NOTIFY_ON_EMPTY, SIZE_MAX, SEG_MAX and FLUSH *(verified: vioblk.c)*, and the other class drivers ask for their own. For: the first operating system needs no change; the devices of QEMU already offer them. Against: the set is another system's habit and not a pdp64 decision; it changes with that system.
3. *The minimum plus the platform features.* Declare ACCESS_PLATFORM always, so that every device address is translated, and the ordering feature, so that the order is the platform's. For: matches the IOMMU on by default (errata 85) and the one fence rule; a device that cannot work this way is refused at the handshake. Against: a device or emulator that does not offer the features cannot be used; the IOMMU must exist before the first driver runs.
4. *Option 2 plus the throughput features.* Indirect descriptors (one descriptor pointing at a table of descriptors), the event index (the device and driver tell each other when to interrupt or notify) and the packed ring (the newer queue layout of version 1.1, *unverified*). For: fewer doorbells and interrupts, longer chains. Against: more states for the driver and for the hardware queue engine to model; the packed ring is a different queue format, so all descriptors, rings and the ordering rules of chapter 2 would have two forms.

**Precedents.** Linux and OpenBSD accept a subset and ignore the rest; OpenBSD's `virtio_negotiate_features` masks what the driver does not know *(verified in `virtio.c`: the source read; the exact masking was not checked).* `ACCESS_PLATFORM` is how a guest with an IOMMU or a secure enclave tells the device to use the platform's address translation *(unverified).*

**The author's referents.** "forget the native 8 register devices. gone. virtio across the board" (errata 95); "we will support all the virtio devices in silicon (the ones it makes sense to support, that is)" (R359).

**The assistant's leaning.** Option 3 for the platform features, and the set of option 2 for each class (they are what the first operating system needs anyway), with indirect descriptors left out until a measurement asks for them.

**Where the body refers to it.** Sections 5.4.4 and 5.10.3.

### J.5.3 Firmware for a virtio device

**Issue.** A device that contains a small processor may need a program, firmware, loaded into it before it works. A PDP-11 peripheral needed none. On a PC the driver reads a file at attach time (OpenBSD's `loadfirmware` reads `/etc/firmware/<name>`; 19 files under `sys/dev/pci` call it, counted). Virtio has no firmware operation: a search of all 29 `virtio_*.h` files and of QEMU's `hw/virtio` finds none, and each device class defines its own set of requests, so a firmware download would be a request that no class has. The pdp64 has set one precedent: the entropy pool has "no updatable firmware" (errata 85).

**Why it matters.** A network card or a GPU built as a virtio function may need its program before the first request, and the standard gives no way to send it. Whatever is chosen must leave a device the boot depends on (the disk, and the console if it is a device) free of it, since the root file system is not yet readable when the boot device is started.

**Options.**

1. *A firmware request in each class.* Each device class defines a request type that carries an image through a queue. For: uses the transport that exists; the image is checked by the device. Against: not in the standard, so every device and every class driver differs; a QEMU model needs it too.
2. *A firmware-loader device.* A separate device type, enumerated beside the function it serves, that takes the image. For: one mechanism for all classes; a device that needs nothing is not affected. Against: a new device type outside the standard; two functions to pair.
3. *A window in the device's own space.* The driver writes the image into a window of one of the function's base-address registers, as PCI devices do today. For: independent of virtio; any PCI Express function can use it. Against: every device defines its own window and command sequence; the "same driver for every device" aim of virtio is lost for those devices.
4. *No downloadable firmware.* The pdp64's standard devices carry their own program in ROM or flash; a device that needs a download is not a standard device and has its own driver. For: no mechanism to define; matches the entropy pool's rule. Against: a device that is cheap because it has no flash cannot be a standard device; an update means new hardware.

**Precedents.** PC practice: a file per device, a loader in the kernel (`loadfirmware(9)` in OpenBSD, `firmload(9)` in NetBSD) and a separate package of files *(verified for OpenBSD: `dev/firmload.c`)*; Linux's `request_firmware` is the same idea *(unverified).* The PDP-11 had none *(verified: 4.1).*

**The author's referents.** None named for firmware; the entropy pool "no updatable firmware" (errata 85).

**The assistant's leaning.** Option 4 for the standard devices, with option 3 left to the drivers of devices that are not standard.

**Where the body refers to it.** Section 5.12.

### J.5.4 Failure, reset and departure: what happens to the queued work

**Issue.** A virtio device can fail in three ways that matter to its driver: it reports a failed request through the status byte, it sets its needs-reset bit or stops answering, or it leaves the bus (hot-plug is decided; see 2.4). In the last two the chains in the queue were handed to a device that will not finish them. What is decided is that a departed device's work is left in memory, "so it can be read, saved, resent" (R262; 2.4.5 and 5.5.5). What is not worked out is the rule: when the driver resubmits the chains that are not in the used ring, when it fails them, and whether another core takes them over. A request that must not run twice (a write to a device that took it and died, a send) cannot simply be resent.

**Why it matters.** Every class driver has a recovery routine, and the disk driver's behaviour decides whether a file system sees an error or a delay. The kernel's rule decides whether a driver has a detach routine at all (the OpenBSD `vioblk` has none) and who reads a departed device's rings.

**Options.**

1. *Fail every queued request.* After a reset or a departure the driver completes each outstanding request with an error and starts afresh. For: simple; no request is run twice; it is what OpenBSD's `vioblk_reset` does. Against: a transient hiccup becomes an I/O error for the file system; the author's wish that the work be resent is not met.
2. *Resubmit the chains that are not in the used ring.* The driver compares the available ring with the used ring, reprograms the queue and posts the unfinished chains again. For: the author's stated purpose; no error for a stall. Against: a request that already ran, or ran partly, runs twice; the driver of OpenBSD zeroes the ring memory on reinitialisation, so the chains would have to be copied out first.
3. *Resubmit only repeatable requests.* Reads, writes of the same data, sends and buffer posts are resent; any other request is failed. The class driver knows which is which. For: safe by construction; each class decides. Against: a class driver can tell "never started" from "started, outcome unknown" only by the request's own status, so some requests must fail even when they never ran.
4. *Hand the queue to another core.* The core that handles the departure event may not be the one that owns the queue; the unfinished work is handed over with the chains and their buffers. For: fits a machine of many cores and a secure core; the author's reference to a hand-over. Against: more kernel machinery; a rule is needed for what the old core may still touch.
5. *Leave it to each class driver.* The kernel gives a reset and a departure event; each driver chooses one of the above. For: a disk and a network card have different needs (a lost frame is harmless, a lost write is not). Against: no common routine, as many recoveries as classes.

**Precedents.** OpenBSD's `vioblk_reset` stops the device, completes what finished and fails every other request; its re-initialisation zeroes the ring memory and restarts both indices at 0, and `vioblk` has `{ sizeof(softc), vioblk_match, vioblk_attach, NULL }` as its `cfattach`, so it has no detach routine *(verified: `vioblk.c`, `virtio.c`)*. The VAX's `ubareset` resets the adapter and calls every driver's reset routine, so that each re-issues what was pending: resending, driver by driver *(verified: 4.3BSD `vaxuba/uba.c`)*. NVMe and virtio drivers in other systems requeue outstanding requests after a reset *(unverified)*. Linux's driver model has `probe` and `remove` *(unverified)*. Whether the virtio specification requires the ring indices to restart after a reset *(unverified: the specification is not on this computer)*.

**The author's referents.** "so it can be read, saved, resent" (R262); the precise bus fault for a departed device (2.4.9, 2.9.1).

**The assistant's leaning.** Option 3 as the kernel's common routine, with a class driver free to be stricter (option 5 above it): the file system sees no error for a repeatable request, and no request runs twice.

**Where the body refers to it.** Sections 5.5.4, 5.5.5 and 5.11.2.

### J.5.5 The fence in C

**Issue.** A driver must execute `FENCE` between the last write that the device will read and the write that notifies the device (5.3.5, 5.4.3). The instruction exists. Nothing in the C language names it, and the compiler has no intrinsic (a built-in function that the compiler turns into an instruction) for it. A driver can call a function written in assembly, but then the fence is a call, with the call's cost and the compiler's assumption that registers may be changed.

**Why it matters.** Every driver of the chapter, and every lock and queue of the kernel that shares memory with a device, executes this fence on its fast path. The answer is part of the compiler and of the ABI (appendix D), and the OpenBSD port needs a spelling for its own memory-barrier macros.

**Options.**

1. *An intrinsic.* A built-in such as `__builtin_pdp64_fence()` that the compiler turns into the instruction and treats as a barrier against reordering. For: one instruction, no call, the compiler knows it is a barrier. Against: a new name that only this compiler knows; the LLVM back end needs it as well.
2. *Inline assembly.* The driver writes `asm volatile("fence" ::: "memory")`. For: needs nothing new in the compiler. Against: every driver repeats it; the compiler that has no inline assembly cannot build it.
3. *A function in assembly.* A leaf function in the runtime library. For: works with any compiler. Against: a call on every doorbell; the compiler must assume everything changed.
4. *The C11 fences.* `atomic_thread_fence` and the release and acquire orderings, mapped by the compiler to `FENCE` as needed. For: standard C; portable drivers. Against: the C11 orderings describe threads and not a device, and a mapping that is too weak or too strong is a bug that no test shows.
5. *The platform's barrier macros.* The operating system defines `membar_producer`-style macros in a header, built on one of the above. For: what OpenBSD code expects. Against: it answers only the spelling, and not how the compiler treats the fence.

**Precedents.** Linux defines `mb()`, `wmb()` and `rmb()` per architecture, and OpenBSD `membar_producer()` and `membar_consumer()` *(unverified).* The compiler's name table (`arch/pdp64/pdp64-names.h` of the pcc tree) knows the mnemonic and nothing else *(verified: read).*

**The author's referents.** None named; the one fence rule of the bus chapter (2.8).

**The assistant's leaning.** Option 1, with option 5 on top of it for the operating system: it is the only spelling that costs nothing at run time and that the compiler can reason about.

**Where the body refers to it.** Section 5.13.1.

### J.5.6 Which virtio devices are built in silicon, and syntactic sugar for devices

**Issue.** On 2026-11-09 the author wrote that "we will support all the virtio devices in silicon (the ones it makes sense to support, that is)", and asked what "syntactic sugar" (notation that makes a thing easier to write, not new opcodes) could make devices easier to use "like the 12 register + dma thing" (R359). Both are decided in principle (statement 5.2.1). Two details are not. The standard devices are fixed (block, network, graphics, audio: errata 79, 101); virtio has others (console, entropy, balloon, SCSI, file system, socket, input) and it is not settled which count as "it makes sense". And it is not settled what the sugar is.

**Why it matters.** The list decides what the hardware contains and what OpenBSD drivers are used. The sugar decides how short a driver is, and whether the instruction set or only the assembler and the compiler change.

**Options.**

1. *Only the four standard devices.* Block, network, graphics and audio; everything else is an ordinary PCI Express device with its own driver. For: the least silicon. Against: the console and the entropy source of J.5.1 would have to be in this list or outside virtio.
2. *The four plus the console and the entropy source.* For: the machine can boot and run with virtio alone; both have OpenBSD drivers (`viocon`, `viornd`). Against: the console's form is itself open (J.5.1).
3. *Every device that QEMU and OpenBSD both have.* That adds the balloon (`viomb`) and SCSI (`vioscsi`). For: free drivers. Against: silicon for devices that few uses need.
4. *Sugar: assembler macros.* The post, fence and notify sequence of 5.4.3 as one macro, and the reap loop as another. For: no change to the instruction set, no change to the compiler. Against: only assembly programmers benefit.
5. *Sugar: a C header of inline functions.* `post` and `reap` of chapter 2 as inline functions over the intrinsic of J.5.5. For: C drivers become short; no hardware. Against: depends on J.5.5.
6. *Sugar in the device: helper registers.* The bus chapter allows "helper registers" for convenience (2.3.4); for example a doorbell that also performs the fence. For: the driver's sequence shortens. Against: not in the standard, so a driver that uses it works on the pdp64 only.

**Precedents.** The RK11's register set (a command register and a go bit) was the "12 register + dma" style the author refers to *(verified: 5.1).* virtio drivers share `virtio_enqueue` and `virtio_alloc_vq` helpers in OpenBSD *(verified: `virtio.c`).*

**The author's referents.** "like the 12 register + dma thing" (R359).

**The assistant's leaning.** Option 2 for the list, and options 4 and 5 together for the sugar.

**Where the body refers to it.** Section 5.2.1.

### J.5.7 USB: the controller interface and the class drivers

**Issue.** USB is a standard device of the pdp64 (statement 5.8.3: the author, 2026-08-29 and 2026-11-03, "modern usb keyboard, mouse, and nvme hard drive", "pcie and nvme and usb"): a host controller that is a PCI Express function, with the operating system's USB stack enumerating what is plugged in. Two details are not worked out. The first is the controller interface, the register and memory layout that the driver programs. The one that PCI Express USB controllers have is xHCI (extensible host controller interface): the controller runs the protocol in hardware, the driver gives it lists of transfer descriptors in memory and reads an event ring, the same pattern as a virtqueue with formats of its own. The second is which class drivers are needed: the author's uses are a keyboard and a mouse (the human interface device class) and a hard drive (mass storage); that a USB-to-Ethernet adapter appears to the host as a network class device is *unverified*.

**Why it matters.** The controller interface decides the one USB driver that the kernel carries and what the hardware and the emulator model must provide; the class drivers decide whether the machine has a keyboard and a mouse at first boot.

**Options.**

1. *xHCI only.* One controller interface, a small compliance-tested subset of it. For: what real USB 3 controllers on PCI Express present; one driver. Against: a large and complex interface (many registers, rings and contexts); no older USB 1 or 2 controller works.
2. *xHCI plus a simpler controller interface for the first board.* For: a smaller first driver. Against: two drivers; a second model for the emulator.
3. *A pdp64 controller that is a virtio device.* For: "virtio across the board". Against: the virtio device types read here list no USB host controller *(unverified: not searched for)*; a bridge to real USB hardware is still needed.

**Precedents.** NVMe and xHCI both command through queues in memory *(unverified: general knowledge)*. OpenBSD drives PCI Express USB controllers through its `xhci` driver *(unverified here)*.

**The author's referents.** USB enumerated over PCI Express as the model of a bus attached to a bus ("just as usb is unuerated over a pcie bus", R292); "modern usb keyboard, mouse, and nvme hard drive. usb hard drive too" (R20). He named no controller interface.

**The assistant's leaning.** Option 1: xHCI only, as the repository's earlier lean (decisions-pending F9) said.

**Where the body refers to it.** Section 5.8.3.

## J.6 Chapter 6: The kernel: modes, exceptions and memory management

### J.6.1 Discovering the implemented widths

**Issue.** An implementation chooses its physical address width (from 40 to 64 bits) and its number of page-table levels (3, 4 or 5, giving virtual addresses of 39, 48 or 57 bits). The author's requirement is that this be "discoverable easily at runtime", so that one kernel binary adapts to the machine without a build option. How software learns the two numbers is not specified: no register or field reports them. System registers 0 to 12 are defined and 13 upward are reserved. A feature register is a read-only register whose fields describe the hardware; a probe is a write of a value followed by a read of what stuck.

**Why it matters.** The kernel must choose the walk depth and the width of every page number it writes. User code that tags pointers with spare high bits needs to know which bits are free: the top bit of the virtual address, which decides the canonical test, is bit 38, 47 or 56 according to the levels (6.9.1). The hardware cost is a few flip-flops. The body of 6.6 and statement 2.3.5 each carry a pointer to this entry. The same question covers the feature and identification registers that the numbers from 13 up would hold.

**Options.**

1. *A read-only feature register read by* `MFSR` (number 13), with fields for the physical bits, the maximum levels and the set of levels supported. For: one instruction in one place, no side effect and no mode change, room for later feature bits, and a read is always safe. Against: one register number and a few gates; privileged only, so user code that needs the canonical bit must ask the kernel (or see option 5); the layout is fixed forever once code depends on it.
2. *Probe by write and read back* (the register accepts any write and holds only a legal value). The kernel writes the level count it wants into a field of `MMUCTL` and reads back; an unsupported value leaves the old one. The physical width is found by writing all ones to a field that holds a page number and reading what stuck. For: no new register, no new instruction. Against: a write with a side effect (it changes the translation mode, needs `ISYNC`, can crash if translation is on); the probe code is longer than one `MFSR`; the width probe needs a writable field of the right width, which may not exist; a probe is a place for error.
3. *Both: a read-only register for what exists, and a writable field to choose.* The register reports the maximum and the set supported, as a bit mask with one bit per level count; a field in `MMUCTL` selects the number in use, legal only if its bit is set. For: one read to learn, one write to choose; the kernel may run three levels on a five-level machine for compatibility or to save walk cost; a bad write is detectable. Against: a register and a field, the most state of any option; whether a machine must support all lower level counts becomes a rule (three levels on every machine is what the author's "System v1" form implies, errata 96, so the mask would say 3 always, 4 and 5 optional).
4. *The boot ROM states the widths in the boot-information block and the processor reports nothing.* For: the block already lists the memory ranges and the root-complex address, so the early kernel reads it with the rest; the processor is unchanged. Against: the facts do not come from the hardware itself, and a stale or wrong block gives wrong widths that cannot be detected; useless to a second-stage loader run by hand or a bare-metal program that has no block.

5. *An unprivileged read-only register or instruction* (a counterpart of x86's CPUID, readable in user mode). For: a JIT compiler or a pointer-tagging runtime learns the canonical bit without a system call; the information is not secret. Against: a new user-visible instruction, or an exception to the rule that system registers are privileged (6.2.5); a virtualiser or sandbox cannot hide it; it lets a program fingerprint the machine.
6. *A profile number instead of fields.* One read-only identification register says "profile 1 is 3 levels and 40 bits, profile 2 is 4 levels and 48 bits", and the manual defines each profile. For: a few well-tested combinations in place of the 3 by 25 that would have to be verified. Against: every new width forces a new profile; the freedom "40 to 64 anywhere" (errata 102) is lost; the kernel needs a table.

**Precedents.** x86 reports physical bits in bits 7:0 and virtual bits in bits 15:8 of CPUID leaf 0x80000008, and a 57-bit virtual address by the LA57 bit of leaf 7 *(verified: QEMU `target/i386/cpu.c`)*; CPUID is readable in user mode *(unverified)*. AArch64 reports the physical range in the PARange field of `ID_AA64MMFR0`, bits 3:0 *(verified: QEMU `target/arm/cpu-features.h`)*. LoongArch reports PALEN and VALEN in fields of `CPUCFG1` *(verified: QEMU `target/loongarch/cpu.h`)*. RISC-V's `satp` register ignores a write of an unsupported mode *(verified: QEMU `target/riscv/tcg/csr.c`)*, and QEMU keeps a bit mask of supported modes and checks the write against it *(verified: `target/riscv/cpu.c`)*; x86 turns on five-level paging by setting `CR4.LA57` only where CPUID reports it *(the CPUID bit verified, the control bit unverified)*. Linux on RISC-V and ARM learns the memory mode from the device tree the loader passes *(unverified)*; the VAX restart parameter block and the Alpha hardware restart parameter block carry machine parameters *(named in the boot chapter's sources)*. RISC-V profiles (RVA20, RVA22, RVA23) define sets of mandatory extensions *(unverified)*.

**The author's referents.** "as long as this info is discoverable easily at runtime" (chosen); "good thing we already had dmw in the design!" (chosen); of LoongArch, "looking at loongarch64, they are pragmatic; what can we learn from them and adopt" and "what other parts of the design are effected if we have 5 level tlb like loongso[n]" (both considered); "what is the largest physical ram supported right now by any cpu? how many table levels?" (considered).

**The assistant's leaning.** Option 1 first: a read-only `MMUCFG` at system register 13 with the physical bits, a mask of supported level counts and the virtual bits. It needs no probe, so a bad value cannot arise, and the number 13 is already reserved. The writable selector of option 3 is added only if the author wants a kernel to run fewer levels than the maximum. Option 2 would be the choice if the author wants no new register, and option 5 as a user-readable copy if user code needs the canonical bit.

**Where the body refers to it.** Sections 2.3.5 and 6.6.1.

### J.6.2 The page-table entry for wide physical addresses

**Issue.** The page-table entry is 64 bits, with the physical page number in bits 12 to 39 (28 bits, a 40-bit physical address) and bits 10, 11 and 40 to 63 reserved and zero. The author has decided that the physical address may be anywhere from 40 to 64 bits (errata 102) and that the old rule of using bit 39 to tell device space from memory is gone, because "if we are going to mmap petabytes, we need those bits". A 64-bit physical address needs a 52-bit page number, bits 12 to 63, and that leaves no bit above bit 11 for any flag. The question is where the page number sits, how wide it may be, and where spare flag bits go. $2^{52}$ bytes is 4 PiB and $2^{64}$ bytes is 16 EiB.

**Why it matters.** Every `fork`, `mmap` and fault handler writes entries, so the position of the page number is shift-and-mask code on the hot path. In hardware, the width of each translation-buffer entry and of the physical tag of every cache grows with the page number: 52 bits in place of 28 is 24 more bits per translation-buffer entry, and a 32 KB cache with 64-byte lines (512 lines) would carry 24 more tag bits each, about 12,000 flip-flops (an estimate, not a measurement). Statements 6.7.1, 6.6.2 and 6.7.5 wait on the answer, and the memory type has only two bits (8 and 9) with nowhere to grow.

**Options.**

1. *The page number in place, bits 12 to 63.* For: no change for the 40-bit machine (bits 12 to 39 stay), the present text is a subset, and the hardware extracts the same field; cheapest in hardware. Against: no spare bit remains for any later flag (a no-execute extension, a wider memory type, a protection key, software bits), so any later flag forces a new format.
2. *Cap the page number at 40 bits (a physical address of 52 bits), in place at bits 12 to 51, with bits 52 to 63 for flags and software.* For: twelve spare bits and 4 PiB of reach, which already covers "petabytes"; room for protection. Against: it contradicts "up to full 64 bit" (errata 102), which becomes "40 to 52 in page tables"; a device mapped above $2^{52}$ cannot go through a page table (see option 5).

3. *Move the page number down to bit 10, as RISC-V does.* The page number occupies bits 10 to 61 (52 bits) and bits 62 and 63 are free. For: a 64-bit address fits and two bits remain at the top. Against: the physical address is `(pte >> 10) << 12`, one more step in the walker and the kernel; every existing entry format is rewritten; only two spare bits, and a 64-bit device address still uses all of them; no gain in reach over option 2 if option 2's cap is accepted.
4. *Keep the 64-bit entry, but the page-number width equals the implemented physical width.* Bits above the implemented width are reserved and must be zero, and the spare bits are defined only as "the bits above the physical width". For: a 40-bit machine keeps bits 40 to 63 for future flags, a 64-bit machine has none; one format with varying free space; a reserved-bit fault is a good detector. Against: the position of a flag depends on the implementation, so code that uses a spare bit works on 40-bit machines and not on 64-bit ones; the detection works only above the implemented width.
5. *Page tables reach 52 bits (option 2), and everything above is reached by the direct-mapping windows* `DMW0` and `DMW1`, whose physical base is a full 64-bit field. A direct-mapping window (appendix I, 6.9.2) maps a range of virtual addresses to a range of physical addresses with no table walk. For: "good thing we already had dmw in the design!"; one window maps a petabyte-scale device or a very large memory without any entry, and the entries keep their spare bits. Against: only two windows; user processes cannot map above $2^{52}$, because a window is kernel-only; the window format, which the author has deferred (appendix I, 6.9.2), would have to be designed now, and the registers are at present stored without effect.
6. *A 16-byte entry (128 bits).* For: no cramping; a full 64-bit address, memory type, protection key and 40 free bits. Against: 256 entries per 4 KB table, so 8 bits per level, and three levels cover only 36 bits and five levels 60; the whole 39, 48 and 57-bit design of errata 96 changes; twice the memory per walk step; larger translation-buffer entries.

**Precedents.** x86-64 addresses bits 51 to 12 in place, with the no-execute bit at 63 *(the mask verified in QEMU `target/i386/cpu.h`, the rest unverified)*, and puts protection keys in the upper bits *(unverified)*; reserved bits above the implemented physical width must be zero and fault *(unverified)*. AArch64's large-physical-address extension stops at 52 bits *(verified: QEMU `target/arm/ptw.c`, the table of sizes ends at 52)*, and output-address bits above the implemented range are reserved *(unverified)*. RISC-V puts the page number at bit 10, with up to 44 bits at 10 to 53, flag fields at 63 to 61 and bits 60 to 54 reserved *(verified: QEMU `target/riscv/cpu_bits.h`)*. LoongArch direct-mapping windows are named in the ISA text, and MIPS's KSEG0 is the older example *(MIPS behaviour unverified)*. IBM's z/Architecture has region and segment tables of another shape *(unverified)*; no common processor uses a 128-bit entry.

**The author's referents.** "if we are going to mmap petabytes, we need those bits" (chosen); "good thing we already had dmw in the design!", and his question "what are dmw registers? part of pcie?" (chosen); "what other parts of the design are effected if we have 5 level tlb like loongso[n]" (considered).

**The assistant's leaning.** Option 2 for page tables with option 5 for the rest: a 40-bit page number (a 52-bit physical address through page tables, 4 PiB) and the windows for anything above. A 64-bit page number consumes the whole entry, and every later need (protection, memory type, a no-execute bit separate from `x`) would force a new format, whereas $2^{52}$ already covers "petabytes". If the author insists that user processes map above $2^{52}$, option 3 (page number shifted to bit 10, two spare bits) is the alternative.

**Where the body refers to it.** Sections 6.7.1 and A.6.1.

### J.6.3 Interrupt delivery and vector numbers

**Issue.** How does a device's interrupt reach one of the cores, how many vectors are there, how are they numbered and prioritised, and how is an interrupt moved from one core to another? The old bus made this trivial: each slot had a number and the vector came from it. PCI Express devices have no slot in that sense, and the author retired slot-derived vectors (errata 29, 49 and 75). The PCI Express answer is **MSI-X** (message-signalled interrupts: the device performs an ordinary memory write whose address and data say which core and which vector, and the write itself is the interrupt, with no wire). Every virtio device carries one MSI-X vector for a change of its configuration and one for each queue (2.6 and 2.7.4). What is missing is the unit on the processor side that receives the write and turns it into "core N takes vector V at level L". A second fact bounds the answer: the vector table has 512 entries, entries 32 to 255 are available to interrupts, 256 to 511 are `TRAP #n`, so only 224 vector numbers (32 to 255) can be used by interrupts.

**Why it matters.** OpenBSD needs an interrupt-controller driver and a number of vectors. Linux's complaint, on x86, is that 256 vectors per core are shared by every device class; a device with many queues (a network card, an NVMe disk) wants one vector per queue per core. In hardware, the controller needs per-core pending state, a path to receive writes and perhaps a remapping table; the rule that an interrupt entry sets priority 7 (errata 40) and the wake-up of `WFE` (7.10.2) must agree with

it. The interrupt write must not overtake the data writes before it, and a moved target must be masked and read back or changed atomically (errata 71). Statements 2.6.3, 5.9, 7.10.4 and 7.10.5 and the vector table of 6.3.6 wait on the answer; the eight priority levels (6.3.7) are decided.

**Options.**

1. *A per-core interrupt file, messages only.* Each core has a 4 KB page in the controller at `base + 4096 * core`. An MSI-X write of a vector number sets that vector's pending bit in that core's file; each vector has a 3-bit level and an enable bit; the core takes the highest pending enabled vector above the priority in the status word and puts its number in ECAUSE. Vector *n* of the file maps to a table entry in the interrupt range (so the file has 224 vectors, 32 to 255, or entries are remapped). For: scales to any core count; the same store is the inter-processor interrupt; no claim read (one memory fetch fewer than RISC-V's PLIC or ARM's GIC); the status-word level stays the only threshold; the pending file sits outside the pipeline, which suits an out-of-order core. Against: about 1,300 bits of state per core by the estimate of the multiprocessor record (a few hundred look-up tables in an FPGA, or a small RAM); a wired legacy source needs a converter; on its own it gives no protection, because a hostile device's write can reach the wrong core (that is option 2).
2. *Option 1 plus an IOMMU interrupt-remapping table.* The device's write carries an index, not a core number; the IOMMU (the unit that translates and checks the addresses of devices) maps the index to a core, a vector and a level in a 64-bit entry; retargeting is one atomic 64-bit store; a hostile device reaches only the entries the kernel gave it. The kernel owns the vector numbers of each core, and devices see only indices. For: gives many vectors, atomic retargeting and masking during a change by construction; forged interrupts cannot arise; invisible to cards (errata 81); an IOMMU already exists for DMA (errata 85). Against: a table walk or entry cache on every interrupt write, and a structure to size (about 1,024 entries of 8 bytes per device range for the first iteration); the IOMMU becomes a dependency of interrupts, so a broken IOMMU silences them; the author must decide whether remapping belongs to the first iteration.
3. *A GIC-style distributor with an ITS.* A central block holds all state, a distributor forwards to per-core interfaces and a translation service (the ITS) turns messages into interrupts; 16 software-generated interrupts per core serve as inter-processor interrupts. For: mature and well tested; it would match ARM software; full affinity control from one place. Against: the largest block and by far the largest driver (2,166 lines in OpenBSD against 375 plus 228 for RISC-V's IMSIC and APLIC); no benefit over option 2 in a design with no legacy; more state to get wrong.
4. *Wired lines to core 0 with a per-line target register.* One wire per device into a small controller whose per-line register names a core; messages may be added later; the vector can be 32 plus the line number. For: the smallest (PDP-11 style: bus-request lines BR4 to BR7, with the device supplying its own vector); enough for a four-core machine with a UART, a timer and a few virtio devices. Against: virtio devices raise messages, not wires, so an adapter is needed anyway; every interrupt of sixteen cores goes through one front door; a per-queue vector cannot have its own target; shared level lines have the "nobody cared" interrupt storms recorded in the Linux sources.
5. *Interrupts as events in a per-core memory ring, with one doorbell per core.* The controller appends (device, queue, level) to a ring in the core's memory and raises one wake signal; the handler reads the ring; the vector is the single doorbell number. For: consistent with the choice for virtio that progress lives in memory (5.2.4); one hardware vector per core, so the 224-entry problem disappears; priority is eight rings or a field; the handler reads memory it was going to read anyway. Against: the controller becomes a bus master that writes memory on every interrupt, so it needs the IOMMU treatment; the ring can overflow, and the overflow rule is new; hardware priority arbitration is replaced by software ordering of rings; no OpenBSD driver exists to copy.
6. *Polled devices on dedicated cores, interrupts only for inter-processor signals, the timer and errors.* The graphics, network and audio cores poll their queues (a driver may mask interrupts and poll, 2.6.3 and 5.9.2, decided); general cores take only inter-processor interrupts, the timer and device-error interrupts, with a small fixed vector set (for example 32 timer, 33 inter-processor, 34 error, 35 hot-plug alert). For: removes most interrupt work from the cores that run the kernel; vector numbers stay a handful, so numbering is trivial; compatible with options 1 or 2 as a front end. Against: needs dedicated cores, which is still only a proposal; a polling core burns power that WFE and interrupts would save; the disk and console on a general core still need some interrupt; it hides the delivery question for any device a general core drives.

**Precedents.** RISC-V's AIA has an interrupt file per hart with enable, pending and threshold registers (the IMSIC); OpenBSD's driver is 375 lines and keeps a threshold table per priority level, and Linux has the same driver in files of 308, 390 and 977 lines plus a header *(verified: line counts and sources of the local tree)*. Intel's VT-d interrupt remapping puts a table between the device's message and its destination *(unverified)*; ARM's GICv3 ITS translates a device identifier and

an event identifier to an interrupt through tables *(file existence verified, 5,903 lines in Linux; behaviour unverified)*. The ARM GIC with GICv3 redistributors and the ITS has OpenBSD's `agintc.c` of 2,166 lines *(verified)*. The PDP-11 had bus-request lines and device-supplied vectors *(verified in the repository's sources)*; RISC-V's PLIC has wired sources and a priority and threshold per context, in a 737-line OpenBSD driver *(verified)*; RISC-V's APLIC converts wired lines to messages in 228 lines *(verified)*. POWER's XIVE uses event queues with per-queue notification *(names verified, behaviour unverified)*; NVMe completion queues and virtio used rings have the same shape for completions *(verified in the repository)*. Polled-mode network and storage stacks (DPDK, SPDK) and Linux's NAPI, which turns interrupts off under load, follow the model of option 6 *(unverified)*.

**The author's referents.** "go with priority 7 on entry and document its throwback to pdp-11 design" (chosen); "what if all interrupts were suspended/defered for one instruction giving time for that spl #7" (considered); "iommu will allow us to map in c64/c128" (considered); "survey of what all major modern os and cpu do, plus vax/pdp" (considered); "virtio across the board" (chosen).

**The assistant's leaning.** Option 2 as the end state, in two steps: option 1 in the first iteration, since a four-core machine with virtio devices gains little from remapping, and the remapping table before sixteen cores or untrusted devices. For the numbering, the file's vector number is 8 bits and is delivered in ECAUSE at a fixed offset (entry 32 + *n* for *n* from 0 to 223, entries 224 to 255 of the file unused), or the TRAP range moves; the author has not yet been shown the 224-entry fact and should decide this. The fixed devices (timer and UART, J.5.1) enter through a small wired-to-message converter, as RISC-V's APLIC does. A first iteration that never exceeds a dozen vectors would favour option 6 as a stand-in; a wish for ARM-compatible software favours option 3; a finding by the OpenBSD port that a per-core threshold register is needed would add an IMSIC-style threshold to option 1.

**Where the body refers to it.** Sections 2.6.1, 5.9.1, 5.9.4, 6.3.7, 7.10.4 and 7.10.5.

### J.6.4 The register number and the layout of the kernel's root table (KTBR)

**Issue.** The author accepted the split of the address space by its top bit (errata 71): the lower half is walked from PTBR, one table for each address space, and the upper half from a second register, KTBR, one table for the whole system. The system-register table (6.5.1) has no number for KTBR: numbers 0 to 12 are taken and 13 and up are reserved. Nothing says how the top-level tables of the two roots are laid out. With the split by the top bit, each root has 256 entries in use at the top level (the top level has 512 entries and the top bit picks the half), so a root could be a full page with half of it unused, or a half page. Nor is it settled whether the user half is the lower half in every case.

**Why it matters.** The kernel writes KTBR once and the walker reads it on every upper-half miss, so the number is fixed in the instruction stream of every kernel (MTSR takes an 8-bit constant), and the layout decides how a kernel builds its tables and whether a page can hold both roots. It also decides what a user process with two halves (a guest kernel, a library with its own privilege) could do.

**Options.**

1. *Number 13, the first free register.* For: no gap, and the numbers 0 to 12 stay as they are. Against: it takes the number that J.6.1 would use for a feature register, so one of them moves.
2. *A number in a block kept for translation (for example 16 and up), leaving 13 to 15 for identification.* For: translation registers stay together, and identification keeps its own run. Against: a gap in the numbering, and the decision about the block is itself new.
3. *Each root is a full page; KTBR holds the address of a 4 KB table whose upper 256 entries are used.* For: the same table shape as every level, so one walker routine serves both roots. Against: half a page of each root is unused; with one kernel root for the whole system the loss is small, with a root for each address space it is not.
4. *PTBR and KTBR each point at a half-page of 256 entries.* For: no unused entries. Against: a table that is not page-sized is an exception to the rule that every table is one page (6.6), and a page cannot be given back whole.
5. *One shared table with two roots indexed by the top bit* (what x86 kernels do when they share a table and split it by position in the top level). For: a single register, so the number question disappears. Against: the user half would see the kernel entries in the same table, which is what the split was made to avoid (the speculative read of kernel translations, 6.9.1); this is the arrangement the author's decision replaced.
6. *The upper half is selected by a field instead of the top bit* (a base and a size register). For: the kernel could choose where the halves meet. Against: it defeats the canonical test that decides the half by one bit and puts a comparator in the walker.

**Precedents.** AArch64 has two translation base registers, `TTBR0` and `TTBR1`, and selects between them by the top bits of the address *(unverified: not in the QEMU source checked here)*. RISC-V has one `satp` and no second root *(verified in the QEMU source)*. x86 kernels shared the user's tables until page-table isolation of January 2018 and have since switched the table on every entry *(repository text: `docs/kernel-pain-points.md`, row 1)*. The VAX had a base register for the process space and another for the system space *(unverified)*.

**The author's referents.** His acceptance of "all 8" of the edits of research round 7 (errata 71), of which the split is one.

**The assistant's leaning.** Option 1 with option 3: `KTBR` as number 13 if J.6.1 puts its feature register elsewhere, otherwise the first number after the feature block, and each root a full page. It costs the least new rule; the unused half is small because there is one kernel root.

**Where the body refers to it.** Section 6.9.1.

### J.6.5 The system-call register convention

**Issue.** A system call is `TRAP #n` through vector 256 + n (6.10.2), and the call number and arguments "travel in registers by the ABI". Appendix D, the ABI, does not say which registers. In V7 the call number was fetched back from the user's `sys n` instruction and the arguments were read from the user's stack with `fuword`.

**Why it matters.** The first operating system (OpenBSD, 6.12.2) and its C library each hard-code the convention, and so does a system-call stub. The kernel's entry code reads the registers before it knows anything else about the process. A convention that overlaps the registers the callee-saved rule preserves costs a move on every call.

**Options.**

1. *The number in the `TRAP` vector alone, and arguments in the first argument registers of the ordinary calling convention.* The vector number is the system-call number, so the kernel dispatches in hardware. For: no register carries the number, and the dispatch is a table lookup. Against: 256 numbers (`TRAPL` reaches them), which is a limit, and an operating system with more calls than that needs a second level.
2. *The number in one register (the first argument register or a dedicated one), arguments in the following ones, and one vector (`TRAP #0`).* For: the form of Linux and of the BSDs on every architecture *(unverified)*; any number of calls; one entry point. Against: one register is spent on the number; the dispatcher reads it before using the table.
3. *The number in the `TRAP` operand and arguments in the argument registers, results in the first result register, an error in the carry flag.* For: PDP-11 Unix reported an error in the carry bit after `sys` *(unverified)* and the BSDs on x86 kept it; the result and error need no second register. Against: ties the convention to the flags, which the entry clears (6.3.2) and the exit must set.
4. *The number in a register and the arguments in memory at an address held in a register.* For: any number of arguments without a limit from the register file. Against: the kernel must read user memory with the guard open (6.10) before it can dispatch, which is the V7 cost the register form was meant to remove.
5. *Follow the platform's ordinary calling convention exactly, so the stub is a function with one `TRAP`.* For: the compiler needs no special case; a wrapper of the C library is trivial. Against: the callee-saved registers and the argument registers are the ABI's to choose first (appendix D), so this option only moves the question.

**Precedents.** V7 read the number from the user's instruction and the arguments from the user's stack *(verified: `trap.c` on this machine)*. Linux and the BSDs on x86-64 and AArch64 put the number in a register and the arguments in the first registers *(unverified)*; RISC-V Linux uses `a7` for the number and `a0` to `a5` for arguments *(unverified)*.

**The author's referents.** None recorded for the convention; the decisions are that `TRAP #n` has its own vector (6.10.2) and that the ABI names the registers.

**The assistant's leaning.** Option 2 with the error and result in registers, following the ABI's argument registers, because it matches what the first operating system's other ports do. The author or the ABI work decides it, and the decision belongs in appendix D.

**Where the body refers to it.** Section 6.10.1.

### J.6.6 Enabling translation safely

**Issue.** Translation is off at reset and `MMUCTL` bit 0 turns it on (6.8.2); the write takes effect after the next `ISYNC` (6.5.2). The instruction after the `ISYNC` is fetched through the new tables. The running code was fetched at a physical address, so it

continues only if the new tables map it at the same address, or if something else (a window, a trampoline, a second mapping) covers it. The architecture does not say which.

**Why it matters.** Every kernel's first page of boot code has to do this. If the architecture does not say how it is safe, each port invents a trick, and a trick that depends on the implementation (the length of an instruction queue, say) is a bug.

**Options.**

1. *The kernel maps its own boot code at the address it is running at.* The kernel builds the tables so that the virtual address of the next instruction equals its physical address (an identity mapping of the boot code), enables translation, and jumps to the real address. For: no hardware help; it is what the first OS ports to RISC-V do. Against: the kernel must build an identity mapping before it can enable anything, and tear it down after.
2. *A direct-mapping window covers physical memory at reset or at the write.* For: the kernel need not build anything first. Against: the format of the windows is deferred (appendix I, 6.9.2), and the ISA text's old rule that `DMW0` covers physical memory at reset was superseded by errata 44, so this option needs the deferred decision made.
3. *Translation enables in two steps: the write sets a bit that takes effect at the next* `RTI`. The kernel arranges `EPC` and `EPSW` so that the `RTI` jumps to the mapped address. For: one clean transition, in the same way the kernel already leaves for user mode. Against: a new rule for one bit, and `RTI` now has a second job.
4. *The write takes effect immediately and the instruction after it is fetched physical, once.* For: no mapping needed for that single instruction, and the kernel puts a jump there. Against: it breaks the rule that `ISYNC` is the point at which a change takes effect (6.5.2), and an out-of-order core cannot promise it.
5. *A bare mode: when translation is on, an address in the first page range is still taken as physical.* For: the early kernel keeps running where it is. Against: it recreates a fixed window the architecture otherwise refuses (errata 97: no address bit means "device"), and the range is another number to choose (J.4.1).
6. *The boot ROM enables translation with a ready-made identity table and hands over with translation on.* For: the kernel starts in a mapped world. Against: the reset state is then different from the decided one (translation off, 6.8.2), and the ROM carries a page table.

**Precedents.** RISC-V's `satp` has a bare mode and kernels turn paging on after an identity mapping or a jump to a mapped trampoline *(satp modes verified in QEMU; the kernel practice unverified)*. AArch64 has `SCTLR.M`, and a kernel that turns the MMU on runs from an identity-mapped page *(unverified)*. The PDP-11 enabled the memory manager by a write to `SSR0` and the V7 boot code ran with the map loaded to identity (`mov $1,SSR0` in `mch_i.s`, verified; the identity map unverified). The VAX turned on mapping with `MAPEN` and a jump to a mapped address *(unverified)*.

**The author's referents.** "good thing we already had dmw in the design!" (chosen; deferred by errata 44).

**The assistant's leaning.** Option 1, since it needs no new rule and is what a kernel already does. If a window is later specified, option 2 makes the early kernel simpler and can be added without changing option 1.

**Where the body refers to it.** Section 6.12.2.

### J.6.7 Saving and restoring a thread's user `DP`

**Issue.** `DP` is the direct-page register, banked by mode (6.2.4). A user program's copy points to its thread's private data. At a thread switch the kernel must save the old thread's user `DP` and restore the new one's. `MTDP` and `MFDP` name only the copy of the current mode, and the system-register table (6.5.1) has no number for the user's `DP`, whereas `USP` and `SSP` have (numbers 6 and 7).

**Why it matters.** Every thread switch and every signal frame needs the thread pointer. Without a way to read and write the user copy from kernel mode, the kernel cannot give threads their own thread-local storage, which every C library with threads needs.

**Options.**

1. *A system register for the user's* `DP` (for example `UDP`, beside `USP`). For: the same idiom as `USP`; one `MFSR` and one `MTSR`. Against: one more register number, and the supervisor copy would want its own for symmetry.
2. *An instruction that reads or writes the previous mode's* `DP` (as `MFPI` and `MTPI` reach the previous mode's space on the PDP-11). For: no number; the previous mode field already says which. Against: a new opcode, and it works only on the way in from user mode, not for a thread other than the one that trapped.
3. *The kernel keeps the thread pointer in its own thread structure and writes it back into the trap frame.* The user's `DP` is saved on entry like any other register. For: no new hardware; `DP` becomes an ordinary register for exception handling. Against:

the entry code must save a banked register that the hardware did not save for it, so the claim that entry costs one PUSHX and a few stores no longer holds for it.

4. *The thread pointer is an ordinary general register by the ABI, as RISC-V's tp is, and DP stays the per-CPU pointer in kernel mode only.* For: no banked user copy; the register is saved in the trap frame with the rest. Against: the user DP is a decided feature (6.2.4, the thread pointer is one `MOVQ d(DP), Rd`), and removing the user copy is a change of the decided design.
5. *MTDP and MFDP in kernel mode with a mode operand.* For: one pair of instructions. Against: it changes decided instructions.

**Precedents.** RISC-V keeps the thread pointer in the general register tp and the kernel saves it in the trap frame *(unverified)*. x86-64 reads the user's FS base with `rdfsbase` or writes it with `wrfsbase`, and the kernel swaps GS with `swapgs` *(unverified)*. AArch64 has `TPIDR_EL0` for the user thread pointer, reachable by MRS and MSR from the kernel *(unverified)*. The PDP-11 reached the other space by MFPI and MTPI *(verified: V7 source)*.

**The author's referents.** None recorded; the banked DP is the author's decision (6.2.4).

**The assistant's leaning.** Option 1, in the same register table as USP, because it repeats a pattern that is already decided.

**Where the body refers to it.** Section 6.11.

### J.6.8 What a write to the translation registers flushes, and invalidating by address-space number

**Issue.** Two texts of the specification disagree. Section 6.8 (and the purpose of the address-space tags) says that a context switch is a write to PTBR and to ASID and an ISYNC, with the old process's entries staying in the buffer. The entry of MFSR in chapter 9 says that a write to PTBR, ASID or MMUCTL flushes the whole translation buffer. The reference executor does the latter (`docs/implementation-status.md`). A related gap: the architecture has TLBI (by address) and TLBIA (everything) but no invalidate by ASID, so a kernel that reuses a number, or that wraps the 16-bit counter, can only flush everything; the policy for a wrap is the kernel's.

**Why it matters.** If a write flushes, tags save nothing and every context switch pays a refill of the buffer, which is what the tags were added to avoid. If it does not flush, the hardware must keep the stale entries of a retired address space until they are replaced, and an ASID reused by a new process must first be invalidated, or the old entries would answer for it.

**Options.**

1. *A write to PTBR, ASID or MMUCTL flushes nothing; the kernel invalidates when it reuses a number.* For: the tags work, and a switch costs two register writes. Against: a reused ASID needs a whole-buffer flush (TLBIA) or an invalidate-by-ASID instruction, and a mistake in the kernel's allocator shows up as a wrong translation.
2. *A write to MMUCTL flushes everything, and a write to PTBR or ASID flushes nothing.* For: turning translation on or off is rare, so a flush there is cheap, and it removes stale state at the one moment when it matters. Against: one more rule to state, and an extra flush on a write that sets only a bit.
3. *Every write flushes the entries of the old tag only.* For: the old process's entries cannot answer under a reused number. Against: it is a hardware search for the tag on a write, as costly as an invalidate by ASID; the entries that survive are the ones of other tags.
4. *Add TLBIASID rN, an invalidate by address-space number, to option 1.* For: the cheap way to reuse one number without flushing the others; a wrap is then a loop over numbers or a TLBIA. Against: a new broadcast instruction (each core must search the tag), and the kernel needs the bookkeeping either way.
5. *Flush all on every write to PTBR or ASID* (the text of chapter 9 today). For: the simplest hardware, with no tag matching in the invalidate path; the buffer is always consistent with the registers. Against: the tags are then dead weight (16 bits in every buffer entry), and every switch is a full refill.
6. *The tag has the generation in it* (the high bits count the wraps). For: a wrap needs no flush because an old number never equals a new one. Against: a wider tag, and the generation is the kernel's policy, not the hardware's, so this is option 1 with the policy written in.

**Precedents.** RISC-V's `satp` carries a 16-bit address-space number and the `sfence.vma` instruction can invalidate by it *(SATP64_ASID verified in the QEMU source; the instruction form unverified)*. x86 has a process-context tag (`CR4.PCIDE`, verified in the QEMU source) and an instruction `invpcid` that invalidates by it *(unverified)*. AArch64 has TLBI ASIDE1 *(unverified)*. The PDP-11 loaded all page registers at each switch and had no buffer to flush.

**The author's referents.** The ISA text takes the ASID and the G bit from the Commodore 128's memory-management unit (`docs/pdp64-isa.md` section 8.1); the purpose of the tag, a switch without a flush, is stated in 6.8.

**The assistant's leaning.** Option 1, and the entry of `MFSR` in chapter 9 corrected to match; the invalidate by number of option 4 only if the first kernel finds that `TLBIA` at each wrap is too expensive.

**Where the body refers to it.** Section 6.8.3. chapter 9 (the entry of `MFSR`).

### J.6.9 Whether supervisor mode has a use

**Issue.** The status word has a mode field of two bits with kernel 0, supervisor 1 and user 3, and the value 2 reserved (6.2.2). The stack pointer and `DP` are banked for supervisor mode as for the other two (6.2.4), and the system registers include `SSP`. No code uses the mode. The mode is decided; whether it has a use is not. 2.11BSD runs its network code in supervisor mode on the PDP-11 (`trap()` tests `SUPVMODE(ps)`, verified).

**Why it matters.** A mode that nothing uses costs a third copy of two registers and a third set of mode checks (`docs/pdp64-system-design.md`, "an unresolved question, not a decision"). If it is to be used, the kernel's entry code, the page-table permission bits (`U` has no supervisor form) and the privileged-instruction set (6.2.5) would need rules for it. If it is not, the question is only whether to keep or remove it.

**What the earlier text said (restored at the author's request, errata 117).** "Three modes cost a third copy of two registers and no logic the kernel uses; supervisor mode has no user, and is included because adding a mode would change the status word and every saved copy of it." That is the assistant's inference, not the author's statement, and no document gives a reason; `docs/pdp64-system-architecture.md` (section 1) says the mode is carried from the PDP-11, whose Unix did not use it. Checked against the status word: the mode field is two bits wide, so kernel 0, supervisor 1 and user 3 fill it with the value 2 reserved. Dropping supervisor mode would not free a bit, because kernel 0 and user 3 still need both bits, so the third mode costs nothing in the word. The earlier wording is weaker than it looked: a fourth mode (the value 2) would also fit without changing the layout, so "adding a mode would change the status word" holds only for a fifth mode or a change of encoding. What the PDP-11 encoding does buy is that the values need no translation for anyone who knows the PDP-11.

**Options.**

1. *Keep it unused.* For: nothing to build or specify; a later use needs no change of the status word. Against: a third bank of two registers and a mode comparison that nothing uses.
2. *Use it for a library with its own privilege* (a user-level subsystem protected from the program that calls it). For: finer protection between the kernel and a program. Against: needs a page permission that supervisor but not user may use, a new `U`-like bit, and a way to enter and leave the mode cheaply.
3. *Use it for a hypervisor's guest kernel* (the guest runs in supervisor mode and the host in kernel mode). For: a guest kernel needs a privilege between its host and its user programs. Against: a hypervisor needs more than a mode (a second translation, virtual-machine control), which is not specified.
4. *Use it for the network stack, as 2.11BSD did.* For: the one historical use; a bug in the stack cannot reach the kernel's own data. Against: 2.11BSD gave it up in later systems *(unverified)*; the cost of the mode change on each packet.
5. *Remove the mode and reserve the value 3 for user and 2 and 1 as illegal.* For: three copies become two and the decode is simpler. Against: the status-word layout does not change, so no bit is saved (see above); loses the PDP-11 encoding's meaning; reverses a decided statement (6.2.1).
6. *Make the mode field a width that holds four modes, reserve the fourth for a hypervisor.* For: room to grow. Against: it already holds four values; the value 2 is free now.

**Precedents.** x86 has four rings of which two are used *(unverified)*; AArch64 has exception levels 0 to 3, of which a hypervisor uses level 2 *(unverified)*; RISC-V has user, supervisor and machine modes, and the hypervisor extension adds virtual versions *(unverified)*. 2.11BSD uses the PDP-11's supervisor mode for the network *(verified: `SUPVMODE` in `trap.c`)*; V7 used kernel and user only *(verified)*.

**The author's referents.** The PDP-11 encoding (kernel 0, supervisor 1, user 3), which the author kept (6.2.1).

**The assistant's leaning.** Option 1: keep the mode, since removing it saves no bit, and decide a use when the hypervisor question (option 3) is asked.

**Where the body refers to it.** Section 6.2.6.

### J.6.10 The vector of `TRAPCC`

**Issue.** `TRAPCC.cc` (the word that enters the kernel when a condition holds) is a decided trap: it completes and `EPC` is the next instruction (6.13.8), and the body gives it the check vector, 16, which the study the author approved (errata 85, 86)

names. What is not worked out is the number itself, which the author did not discuss (errata 88: "the vector number is the proposal's"), and what EVA holds (the study says the address of the TRAPCC). The options below are the alternatives to the study's choice.

**Why it matters.** The kernel's vector table needs the entry, a debugger and a signal layer need to know what the trap means, and a compiler that emits TRAPCC for a runtime check (an array bound, an arithmetic overflow) must name what the program sees.

**Options.**

1. *Vector 16, a vector of its own, with EVA the address of the TRAPCC.* For: the kernel tells a check from an illegal instruction at once; the vectors 0 to 31 are the processor's, and 16 is free. Against: a vector of the processor's block used up by an optional word of two members.
2. *The vector is 256 + n of a TRAP number named by the instruction.* For: no processor vector is spent, and the operating system names the check. Against: TRAPCC has no operand (one parcel), so it cannot carry n.
3. *The illegal-instruction vector (2).* For: nothing to add. Against: a check is not an error in the instruction; the kernel must decode to tell them apart.
4. *The breakpoint vector (15).* For: the same shape (a deliberate trap by a program). Against: the debugger and a check share a vector.
5. *One vector for each condition (fourteen vectors).* For: the handler learns the condition from the vector. Against: it uses fourteen of the thirty-two entries of the processor block, which leaves no room.
6. *The vector chosen by a system register* (a TRAPCC vector register). For: the operating system picks. Against: one more register and a hidden dependence between an instruction and the register.

**Precedents.** MIPS has TEQ, TNE and the other trap-on-condition instructions, which raise the trap exception with the condition in a code field *(unverified)*. PowerPC has TW and TD (trap word and doubleword on a condition) *(unverified)*. x86 has INTO (trap on overflow, its own vector 4) *(unverified)*; AArch64 has BRK with an immediate *(unverified)*; RISC-V has no trap-on-condition instruction *(unverified)*.

**The author's referents.** Errata 85 (item 9: TRAPCC with the fused ADDV, SUBV and MULV forms) and errata 86 (TRAPCC is a trap and not an abort); none recorded for the vector number or for EVA.

**The assistant's leaning.** Option 1 as the study had it, and as the body states it; a decision of the author fixes the number.

**Where the body refers to it.** Sections 6.3 and 6.13.8.

### J.6.11 Hardware breakpoints and watchdogs, performance counters, machine-check reporting

**Issue.** Three groups of kernel-visible facilities are not specified: debug registers (hardware breakpoints and watchpoints), performance counters, and what a machine check reports beyond the halt of 6.4.3. A watchdog (a timer that resets the machine if the kernel does not touch it) is the fourth. None has a system-register number: 13 and up are reserved.

**Why it matters.** A kernel debugger without hardware breakpoints cannot stop on a data access, and a profiler without counters cannot count cache misses. A machine check that reports nothing leaves a kernel nothing to log. None is needed to boot; each would have to be present before the system is used for serious work.

**Options.**

1. *None in System v1.* For: smallest. Against: each is requested by a debugging and operations workflow later, and a software replacement is slow.
2. *A fixed small set* (two breakpoint registers, one cycle counter, one machine-check status register). For: enough to start a debugger and a profiler. Against: the set is then fixed forever.
3. *Adopt RISC-V's sets* (the debug trigger module, the hardware performance monitor counters and the `mcause` convention). For: ready-made specifications and software. Against: much larger than a first iteration needs.
4. *Leave them to the platform, not the architecture.* For: each implementation chooses. Against: a kernel then has no portable way to use them.
5. *The memory-mapped debug unit of the bus (PCI Express's own capabilities).* For: no new registers. Against: the processor's state is then not reachable in the processor's own terms.

**Precedents.** x86 has debug registers DR0 to DR7, performance counters selected by MSRs and machine-check banks *(unverified)*; AArch64 has the debug, PMU and RAS extensions *(unverified)*; RISC-V has a trigger module, `hpmcounter` registers and a `mcause` *(unverified)*.

**The author's referents.** None recorded.

**Where the body refers to it.** Section A.10.1.

### J.6.12 Recovering from a failed user copy

**Issue.** A kernel copy routine opens the guard, copies and closes it (6.10.3). Before the copy PROBER checks that the page is readable, but the copy may still fault (a file mapped into the process may be truncated under it, or a page may have to be brought in). The page-fault handler can bring a missing page in and restart the instruction (6.4), so a fault that the kernel can repair is not a problem. A fault that it cannot repair (an unmapped address) has to turn into an EFAULT return from the copy routine. The author accepted the fixup table with the .U qualifier (errata 86, item 18; the body, 6.10.4), so how the handler finds the place to return to is decided for the .U moves. What is pending is the rest: the form for the atomics (once the operand placement of ATOM is fixed), the full specification of the fixup table, and the recovery of a copy that does not use a .U move (for example in a member without the qualifier, or a block copy).

**Why it matters.** Every system call that takes a user pointer depends on it, and a kernel that cannot recover panics on a bad pointer from user mode, which is the attack that the guard exists to stop.

**Options.**

1. *A global `nofault` return address, as V6 and V7 had.* The routine stores a recovery address, the handler on a fault in kernel mode at a user address jumps there. For: already described in 6.4 and tested in V7; no new instruction. Against: a global (or per-thread) variable that the routine must reset on every path out; a fault in other kernel code at the same time would be taken for it.
2. *Address ranges of the copy routines.* The handler compares EPC with the start and end of the copy routines and jumps to one recovery address. For: no variable to set, no instruction change. Against: every copy routine must lie in the range, and any new copy loop must be added to it.
3. *A fixup table: the qualifier .U marks the instructions that touch user memory and the handler looks up EPC in a table of recovery addresses.* This is the decided form of 6.10.4. For: exact for each instruction, no variable, no range, and the move accesses user memory with user privilege so the guard is not needed. Against: a new qualifier on seven words, a table to build, and the specification is incomplete (the atomics).
4. *Check every page first with PROBER and PROBEW, never fault.* For: no recovery path. Against: a page can disappear between the check and the copy (a file truncated by another thread), so it is not safe for a mapped file.
5. *Run the copy with a fault handler stack: `setjmp`-style context saved at the system-call entry.* For: any depth of nesting returns to the system-call entry with an error. Against: the context save is memory traffic on every system call; it needs a new rule that unwinds kernel state.
6. *Exception tables in the manner of Linux, in a section of the kernel image.* For: the same as option 3 without a new qualifier: any instruction can be listed in the table. Against: the table is a section that the assembler and linker must support, and each use is a macro.

**Precedents.** V6 and V7 set a global `nofault` and the trap handler jumped to it *(verified: `trap.c` and `mch_i.s` on this machine)*. Linux keeps an exception table of (instruction, recovery) pairs for user access *(unverified)*; OpenBSD on x86 and ARM keeps a `pcb_onfault` address in the process control block, which the fault handler reads *(unverified)*. x86 reads user memory with SMAP open by STAC and CLAC and Linux uses the exception table *(unverified)*.

**The author's referents.** "pdp64 specifically designs the instructions for minimal uops, so we have minimal restart/recovery code needed, no?" (2026-11-09, on restart); the five leanings of the study that the author accepted (the fixup table, the .U spelling, keeping UA, STUA and CLUA until the OpenBSD port confirms them, bit 15 of ECAUSE, the atomics taking the user form).

**The assistant's leaning.** Option 3 is decided for the .U moves; option 1 for the first kernel's other copies, which is what the OpenBSD port will expect (`pcb_onfault`).

**Where the body refers to it.** Sections 6.10 and 6.10.4.

## J.7 Chapter 7: Multiple processors

### J.7.1 The wake rule of WFE and the idle loop

**Issue.** WFE (wait for event) puts a core to sleep until another core executes SEV (send event) or an interrupt arrives (7.10.1). The idle loop of a kernel checks the run queue and then sleeps; if an interrupt arrives between the check and the sleep, it queues work, returns, and the core sleeps with work waiting (a *lost wake-up*, an event that arrives in the gap between the test

and the sleep and is never seen). The PDP-11's V7 kernel had the same gap and a clock tick repaired it within one tick. A core that takes no periodic tick has no repair. The question is what ends a `WFE` sleep. The author decided (2026-10-22: "Yes, wake on pending") that `WFE` ends its sleep when an interrupt is pending even if the priority masks it, so that the idle loop is `SPL #7`, check for work, `WFE`, `SPL #0`, with no gap (7.10.2). What is open is the rest of the wake rule: whether `SEV` is sticky for one wait, and whether other events also end the sleep (options 2 to 5).

**Why it matters.** The idle loop of every kernel, and the port's `cpu_idle_cycle`, depend on it. A lock waiter that could sleep instead of spinning would save memory bandwidth on 16 cores, and the answer decides whether an unlock must execute `SEV`. A wrong rule is a hang that occurs once a week.

**Options.**

1. *The decided rule alone, with `SEV` sticky for one wait.* The loop is `SPL #7`, test, `WFE`, `SPL #0`. For: closes the gap with no new instruction; two rules in the core. Against: none beyond the decided rule: a core wakes for an interrupt it will not take until the level drops (it then takes it, as intended).
2. *Option 1, and `WFE` also ends when the reservation of a preceding `LL` is lost.* A spin wait becomes `LL r, lock; test; WFE`; the holder's ordinary release store wakes the waiter, and the unlocker executes no `SEV`. For: unlock stays one store; a waiter costs no bus bandwidth. Against: the reservation must stay armed while the core sleeps; a spurious wake is allowed, so the loop re-reads; the reservation rules (7.4.1) must say that the reservation survives the sleep.
3. *`WFE` takes a priority operand.* One instruction lowers the level and sleeps. For: no `SPL` around it. Against: a new operand form to encode; no gain where the loop is already two instructions.
4. *A timeout form with a deadline.* For: a waiter cannot sleep forever if the wake source breaks; the secure core's bounded waits. Against: an operand; a deadline needs the timer.
5. *No `WFE`; poll with the `HINT` pause.* For: nothing to specify. Against: burns power and bandwidth, and on an FPGA a spinning core takes the memory bandwidth the other cores want.

**Precedents.** ARM's `WFE` ends on `SEV`, on an interrupt and on the clearing of the exclusive monitor, which is how a lock waiter sleeps until the holder's store *(unverified: general knowledge)*. RISC-V's `wfi` ends when an interrupt is pending and enabled in its own register whatever the global enable says, and may be a no-op; `Zawrs` adds a wait on a reservation *(verified in the repository's notes)*. x86 has `HLT` after `STI` (the instruction after `STI` cannot be interrupted, which makes the pair safe) and `MONITOR`/`MWAIT` on a cache line *(unverified)*. V7's idle routine lowered the priority and executed `wait` in a loop *(verified in the repository's notes)*. OpenBSD's riscv64 idle is `intr_enable(); wfi` with a comment that doubts it *(verified)*.

**The author's referents.** "main cores tickless"; no system named for the idle loop.

**The assistant's leaning.** Option 2, which contains option 1: a lock waiter sleeps without an `SEV`.

**Where the body refers to it.** Sections 6.11 and 7.10.2.

## J.7.2 Coherence beyond four cores

**Issue.** The first device has four cores that snoop (every cache watches the others' writes) with three states, modified, shared and invalid (7.6.1), and the architecture supports at least four cores, with sixteen or more as the preferred ceiling (7.6.3, errata 92). The protocol that serves more than four coherent cores has not been chosen. The architecture promises that memory is coherent and sequentially consistent (7.5.1) and says nothing of the protocol. The statement as first written: the protocol for more than four coherent cores is open; the leaning of the SMP decisions is MESI with a snoop filter inside clusters of four and a directory between clusters, with dedicated cores as non-coherent partners that use uncached rings. *(P)*

**Why it matters.** The number of general cores sets the size of the cache controller, the FPGA that can hold the design (about 16,000 look-up tables for a core before coherence, by a paper estimate at plus or minus 40 percent; four cores fit a Kria-class board and none of the planned boards holds sixteen), and what the kernel's scheduler sees as a group. Also the line size (64 bytes), the reservation granule and the invalidate granularity hang from it.

**Options.**

1. *Keep three-state snooping for two to four cores and say so.* For: smallest; fits the first device. Against: breaks the target of sixteen or more cores; snooping traffic grows with the cores.
2. *MESI within clusters of up to four cores, a snoop filter at a shared second-level cache, and a directory between clusters.* MESI adds an exclusive state, so a write to a clean line that nobody else holds needs no bus transaction. For: four cores are one cluster and nothing is wasted; the ceiling is reached by adding the inter-cluster layer. Against: two protocols; the directory is where protocol deadlocks hide.

3. *A flat directory.* One home place per line and a bit-vector of sharers. For: simplest to reason about; scales to 64. Against: two hops for the common case, which is more than a four-core machine needs.
4. *No coherence between clusters; explicit cache operations.* For: cheapest to scale. Against: every missing flush is a race that reproduces once a week; it weakens sequential consistency to "within a cluster".
5. *Dedicated cores as non-coherent partners of coherent general cores.* The graphics, network and audio cores use uncached rings and `fence`, as a DMA device does. For: the coherent count stays at the general cores. Against: the data of a partner is not coherent with the kernel's buffers without a copy or a flush; it depends on the dedicated cores of 7.11.6, whose details are pending (J.7.5).

**Precedents.** PDP-11/70 caches were kept coherent by write-through and invalidation on DMA writes *(unverified: from memory)*. Bus-snooping multiprocessors (Sequent, Sun Enterprise) stopped near 16 to 64 processors *(unverified)*; directory machines (Stanford DASH, SGI Origin) scale by messaging the sharers *(unverified)*; x86 uses snoop filters inside a die and directories across sockets; ARM's coherent interconnects use filters or directories *(unverified)*.

**The author's referents.** "4 cores minimum, 16 or more preferred"; "for initial device 4 cores is enough hopefully that fits on the fpga".

**The assistant's leaning.** Option 2 as the direction, built in two steps: one cluster of four cores first, with the shared cache designed so a directory can be added; the architecture text says "coherent, sequentially consistent, line size 64 bytes" and nothing of the protocol, so more cores are an implementation and not a change.

**Where the body refers to it.** Section 7.6.4.

### J.7.3 The operands of the atomic instructions

**Issue.** The atomic words (7.3.1) take three register fields in the grammar (four for `cas`), with the address in a register and the size in the sub-opcode. The roles of the registers, the size encoding of `cas` and any displacement are not fixed in the text. The statement as first written: the operands of the atomic words sit in three register fields (four for `cas`), the roles are not fixed, and the study leans to fixing three register fields now, with the fourth reserved zero. *(P)*

**Why it matters.** The placement decides how many instructions an atomic costs, how many bytes it takes, and what the lock and reference-count code of the kernel compiles to. `cas` in the four-field form fills the last four-register slots of the encoding.

**Options.**

1. *Register only, address in a register, three fields.* `rd` receives the old value, `rs` is the operand, `ra` holds the address; `cas` has the expected value in and the old value out in `rd`; the fourth field is reserved zero. For: densest; the simplest address path; a misaligned address is detected at one place. Against: one `lea` (load effective address) when the address is a structure field at a nonzero offset.
2. *Option 1 and a small scaled displacement in the spare field.* The address is `ra` plus `d` times the size, `d` from 0 to 15. For: removes the `lea` for the first sixteen words of a structure, where lock owners and reference counts usually sit. Against: uses the spare field; one small adder in the atomic unit.
3. *Full memory operand modes as for loads and stores.* For: indexed atomics. Against: a general address generator in the atomic unit and side effects (auto-increment) that complicate restart after a fault.
4. *`cas` with fixed registers, as x86.* For: frees two register fields. Against: a fixed register is a permanent constraint on the compiler in a 16-register machine.
5. *Double-width `cas` as a register pair.* For: lock-free lists with a tag. Against: not needed while a pointer has spare bits to hold a version count.

**Precedents.** RISC-V's `amo` words take three registers, address in `rs1` with no displacement *(unverified: from memory)*; ARM's large-system atomics take registers; x86 takes any memory operand with `lock` *(unverified)*. OpenBSD's mutex is one compare-and-swap on the owner word and its ticket lock one add *(verified in the local source)*.

**The author's referents.** None named beyond the PDP-11 and VAX atomics (`datip`/`dato`, interlocked queue instructions).

**The assistant's leaning.** Option 1 now, with the spare field reserved, and a displacement taken only when the OpenBSD port is compiled and the `lea` count in front of atomics is measured.

**Where the body refers to it.** Section 7.4.6.

### J.7.4 Atomics on uncached memory and on device memory

**Issue.** The read-modify-write atomics are atomic with respect to every initiator, cores and devices alike (7.4.3). That is easy on cached memory, where the cache line is held exclusively. On a page of uncached memory or a device page the access goes to the bus, and a PCI Express transaction has atomic operations (`AtomicOp`) only on devices that choose to implement them. The mapping of the machine's atomics onto the bus is therefore not stated.

**Why it matters.** The rings of every virtio device and the locks of dedicated cores sit in memory that may be uncached; a program that issues `CAS` on such a word needs either a defined result or a defined fault. The software cannot be silent about it, and a silent non-atomic update is the worst outcome.

**Options.**

1. *Abort.* An atomic to a device or uncached page raises an access fault. For: easy to detect and cheap to build. Against: a driver cannot use an atomic on a shared uncached ring.
2. *Performed at the memory controller (far atomics).* The operation travels to the memory and the result returns. For: constant cost under contention; works for uncached normal memory. Against: the memory side needs an arithmetic unit and a defined form for every word, and a clear rule for the interaction with reservations.
3. *Forwarded as PCI Express atomic operations where the device advertises them, abort otherwise.* For: uses what the bus offers. Against: only compare-and-swap, swap and fetch-and-add exist on the bus; the other words would still abort.
4. *Defined only for cached normal memory, undefined elsewhere.* For: nothing to build. Against: undefined behaviour in a place where a fault would be better.

**Precedents.** PCI Express defines `AtomicOp` for fetch-and-add, swap and compare-and-swap *(unverified)*. AXI5 has atomic transactions performed near the memory *(unverified)*. The PDP-11's `DATIP`/`DATO` pair locked the Unibus for the read-modify-write *(verified in the repository's sources)*.

**The author's referents.** None named.

**The assistant's leaning.** Option 1 for device pages and option 2 or an abort for uncached normal memory, decided with the transport of the bus (errata 80); a fault that is easy to detect is better than a silent non-atomic update.

**Where the body refers to it.** Section 7.4.5.

### J.7.5 What remains of the dedicated cores

**Issue.** On 2026-11-04 the author recommended (errata 76: "we recommend one core dedicated to the gpu/graphics device, one core dedicated to the nic, running the full tcp/ip stack, then a bunch of other cores for general processing", and "one core for audio processing, doing midi, dsp, etc.") dedicated cores for graphics, network and audio. On 2026-11-10 he wrote, of the decision on the secure mode (J.7.8): "the idea of dedicated roles now shrinks" (errata 132). The statements that named the roles are in Appendix H (7.11 and 3.8.2). What remains of the idea, if anything, is not decided; the earlier open questions (how a dedicated core shares memory with the others, what runs the TCP/IP stack, the IOMMU domain of each) apply only to the options that keep a dedicated core. The first device has four cores in all (7.6.3).

**Why it matters.** With four cores, each core given a role leaves fewer for general work; the coherence of J.7.2, the interrupt delivery of J.6.3 (a dedicated core can poll its virtio queues instead) and the secure mode of J.7.8 depend on the answer, and so does chapter 3's account of who drives the terminal.

**Options.**

1. *None: roles are a software matter.* The kernel may pin a driver to a core; the architecture names no role. For: nothing to build; the four cores stay general. Against: no core class marks a core as suited to a device; the Blit's model survives only as a driver structure.
2. *The graphics core only.* One core for the terminal, as the Blit had a processor of its own. For: the Blit is the model the author named for the terminal (chapter 3), and a polled terminal needs no interrupt. Against: a quarter of the first device; the terminal's devices are virtio devices that a general core can serve.
3. *The network core only.* One core runs the TCP/IP stack and drives a standard PCI Express card. For: the network is the heaviest interrupt source; smart network cards run the stack on an on-card core. Against: a quarter of the first device; the kernel's network stack is built for several cores.
4. *Roles as scheduling policy on ordinary cores.* The kernel dedicates a core to a role while the load asks for it and gives it back. For: nothing is lost when a device is idle. Against: no isolation, since the core is an ordinary one.

5. *One core class per role, as before.* Dedicated cores as non-coherent partners with uncached rings and `FENCE`, each in its own IOMMU domain, or coherent like the others. For: a fault in one does not reach the others (non-coherent form); shared buffers need no flushes (coherent form). Against: a copy or flush for data shared with the kernel's buffers, or a coherence protocol (J.7.2) that must reach them; the cores in all must exceed four to leave general work.

**Precedents.** The Blit's terminal had a processor of its own (Pike, Locanthi and Reiser); the VAX console processor was a second processor; smart network cards run the stack on an on-card core. All three are from general knowledge and *unverified* here (the Blit and the VAX console processor are as 3 and 7.11 state them).

**The author's referents.** The Blit model; a standard PCI Express network card (errata 75); errata 76; the words of 2026-11-10 above.

**The assistant's leaning.** Option 1 for the first device, with option 2 or 5 reconsidered when the machine has sixteen cores; the secure mode, not the roles, now carries the question of unlike cores. This is the assistant's view, not a decision.

**Where the body refers to it.** Section 7.11 (the last paragraph) and Appendix H (3.8.2 and 7.11.6).

### J.7.6 The reservation granule and how software learns its size

**Issue.** `LL` reserves a granule: a naturally aligned piece of memory of a size the implementation chooses, not smaller than 8 bytes (errata 45; 7.4.1). The text leaves the size to the implementation and says that a feature register is to report it (errata 45 lists "granule discovery in the feature registers" as still to write). Nothing says which register, nor what software may assume before it reads it. The size matters to software because two lock words in one granule make an `SC` fail whenever the neighbour is written (false sharing of the reservation), and because a constrained `LL`/`SC` sequence (7.4.2) must not touch the granule between the two instructions.

**Why it matters.** The kernel lays out its lock words and per-CPU structures with the granule in mind (padding a hot lock to a granule costs memory and saves failed `SC`s). The compiler's `LL`/`SC` loops and the C library's allocator need a safe assumption before any feature register is read. The cache controller's reservation tag, one per core, is sized by it.

**Options.**

1. *The granule is the cache line, fixed by the architecture.* One number for every implementation (the line size is itself open, J.7.2). For: software needs no discovery; one rule for locks and for padding. Against: an implementation with another line size cannot conform; it fixes the line size as a side effect.
2. *An implementation choice, reported in a feature register, with a stated minimum of 8 bytes.* This is the text as it stands. For: the implementation is free. Against: portable code must read the register or assume 8 bytes, the worst case for padding.
3. *An implementation choice between 8 bytes and a stated maximum, for example 128 bytes, reported in a register.* For: software can pad to the maximum with no register read; the implementation keeps a range. Against: a maximum is another number to commit to.
4. *A reservation on the exact word, held in a monitor at the memory controller.* For: no false sharing, whatever the lines are. Against: a monitor with an address comparator for every core, and a rule for cached lines that move between cores while a reservation is held.

**Precedents.** RISC-V lets the implementation choose the size of the reservation set and requires no minimum beyond the word *(unverified: from memory of the specification)*. Arm exposes the exclusives reservation granule size in a system register and gives a range of 8 bytes to 2 KB *(unverified)*. x86's `lock` prefix works on a cache line and has no granule to report *(unverified)*.

**The author's referents.** None named; the spec is the author's (errata 45: "a naturally aligned granule of the implementation's choice (not smaller than 8 bytes)").

**The assistant's leaning.** Option 3, with the maximum equal to the line size once J.7.2 fixes it.

**Where the body refers to it.** Section 7.4.6.

### J.7.7 How block operations order their accesses

**Issue.** The block operations are `MCPY`, `MMOV` and `MSET`, which write memory, and `MCMP`, `MSCN`, `MSCNN` and `MVAL`, which only read it (9.11.3). Each keeps its progress in its registers (9.11.4) and an implementation may move more than a byte per step (9.11.5). One part of the ordering is decided: the accesses inside one block operation are unordered with respect to one another, and only the final contents of memory and the register progress are defined (7.5.8, errata 39 item 7; the text was never written into the instruction-set document, and the decision list of the ledger does not carry it). Nothing is decided about the rest:

(1) the order of the elements, and the meaning of overlap for MMOV; (2) how far an access may tear, per byte, per aligned word or per cache line; (3) whether another core may observe an intermediate state, and which; (4) what a program must execute to publish the result, given that FENCE orders only device accesses (7.5.7); (5) what a fault or an interrupt leaves in memory beyond the progress in the registers (the progress category of 6.4); (6) the effect on a reservation (7.4.1 clears it on any write) and on atomics that touch the same granule; (7) uncached and device memory, where the number and size of accesses are visible. The ledger lists the block instructions themselves as in the air (no standing decision beyond the progress in registers, R362: "good" to a single move with no resume opcode).

**Why it matters.** The flag idiom (copy a buffer, then store a flag that another core polls) is the commonest use of a copy between cores, and under sequential consistency (7.5.1) it needs no barrier for ordinary stores. Whether it works for a block copy decides whether a kernel's `copyout` of a message and the C library's `memcpy` into a shared buffer need code that the rest of the machine avoids. A copy into a device queue is the other case. The out-of-order readiness review found that this is the one place where the text of the instruction set (a byte at a time, "stated as a loop") and the promise of 7.5.1 disagree.

**Options.** Options 1, 2, 4 and 5 differ in what another core sees; option 6 is about device memory and combines with 1, 2 or 4.

1. *Unordered inside, ordered as a whole.* Every access of the instruction is after all earlier accesses of the core and before all later ones, as the other cores see them; inside, any order, any intermediate state. A plain store of a flag after the instruction is safe. For: the flag idiom works with no fence; the decided rule of 7.5.8 holds. Against: the engine must finish before the next access issues, so a large copy cannot overlap later work; an interrupted copy has published part of its data.
2. *Unordered inside and with respect to later accesses; a fence after the instruction publishes.* The engine runs in the background as a DMA device does, and FENCE is widened to all memory. For: the cheapest hardware and the most overlap. Against: breaks the rule of 7.5.1 for the code after a copy and brings back the barrier a kernel writer must remember; FENCE changes meaning (7.5.7).
3. *Ordered as the loop of the instruction-set text, byte by byte.* For: nothing new to learn; the model is the PDP-11 loop. Against: contradicts the decided 7.5.8; a wide engine must hold back stores to keep the order seen by others.
4. *Option 1, and every aligned 8-byte piece is single-copy atomic.* The head and the tail that are not aligned are byte accesses. For: a reader of copied pointers or counters never sees a torn word (7.5.3 and 7.5.6 already promise this for ordinary accesses); the engine moves aligned quads. Against: an engine that moves lines must still be able to split them; MSET of one byte into a word needs a byte access.
5. *The block operation is atomic against other cores.* For: nothing to reason about. Against: an unbounded hold on memory against interrupts and the progress rule, and a latency no real-time claim survives (J.9.7).
6. *Device and uncached pages are accessed in element order, one access of the program's size per element, never combined or repeated.* Ordinary pages follow option 1, 2 or 4. For: a block write to a device register ring means what a loop would mean. Against: the behaviour depends on the memory type of the page, and the same instruction runs at two speeds.

**Precedents.** x86 REP MOVS keeps its progress in registers and, in the fast-string forms, is allowed to order its stores differently from ordinary ones; Linux's `memcpy` uses `rep movsb` where the processor has the fast form (`arch/x86/lib/memcpy_64.S`, read; the rules in the Intel manual are unverified). ARM64 FEAT_MOPS splits a copy and a set into prologue, main and epilogue instructions (`cpyp`, `setp` in Linux's `arch/arm64/lib/memcpy.S` and `memset.S`, read), updates the registers as it goes and leaves the order and size of the accesses to the implementation (the architectural wording is unverified). The VAX MOVC3 and MOVC5 were interruptible, with the registers as state and a first-part-done bit in the status word (9.11 and J.9.7; the NetBSD VAX copy routines use `movc3`, `subr.S`, read; the memory ordering is moot on a machine that was one processor in most models). The PDP-11 had no block move: a loop of `MOV (R0)+,(R1)+` makes every word its own access in program order (inference). RISC-V's Zicboz zeroes a whole aligned block with one instruction (ZICBOZ in Linux's `hwcap.h`, read; its ordering rule is unverified) and proposals for memcpy and memset instructions exist (unverified). A DMA engine, a virtio device, moves its data in any order and publishes it by writing the used ring last, with the driver's barrier between the two (the rule of the virtio specification, unverified here).

**The author's referents.** "rep movsb r1,r2,10;" and "how about mmov mcpy mset ? like ibm system z?" (the author, 2026-10-20); "doesn't mcmp/mmov count?"; VAX MOVC5 as the argument against hidden state (9.11); errata 39 item 7.

**The assistant's leaning.** Option 4 with option 6: ordered as a whole against the other accesses of the core, aligned quads atomic, and element order on device pages. The leaning keeps the flag idiom free of fences, which is what the decision for sequential consistency was bought for. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 7.5.8, 7.5.9 and 9.11.

### J.7.8 Secure mode on every core, or one secure core

**Issue.** The author decided on 2026-11-10 (errata 132): "at least one core that can be in 'secure' mode; if it doesn't take too many gates, let that be a mode for all the cores, but if it slows things down too much then return to a single secure core" (7.11.3). Two things are not decided: what counts as "too many gates" and "too much slowdown", and how the two are measured. Secure mode here is the whole of 7.11.3 to 7.11.5: constant-time instructions and cache fetches, private cache lines, no periodic tick, stop and preemption by an inter-processor interrupt, constant-time AES and SHA instructions, the private fixed-latency window, the locked translation buffer, no speculation, a fixed clock, and the clear at a domain switch. The status-word bit `DIT` (9.3.7) is already decided for every core; it makes the data-dependent timing of instructions, of the clock and of prefetch disappear while set, and leaves out the private window, the locked buffer, the absence of speculation and the clear.

**Why it matters.** A mode of every core means every core carries the extra logic and a kernel may place a secret-handling process anywhere; one secure core means a smaller, simpler design for the rest and a scheduling rule that sends secret work to one core. On four cores, one secure core is a quarter of the first device. The choice sets the class register's fields (7.11.1), the instruction set of 9.9.7 on every core or one, the kernel's affinity rule, and what the first device's FPGA must hold (10.1.3).

**The measurement plan.** *Gates:* synthesise one core twice for the FPGA of the plan, with and without the mode, and compare the lookup-table counts (and block RAMs) of the logic the mode adds: the fixed-latency window, the locked translation buffer, the clear engine, the clock hold, the speculation disable and the AES and SHA datapaths. *Slowdown:* with the mode's circuitry present but unused, measure the instructions per cycle of ordinary mode on the benchmark set and the critical path (the clock the core reaches), the extra multiplexers in the memory path and the cache arrangement the private lines need; separately measure the performance while in secure mode, which is allowed to be slow. *Threshold:* a number for each, set by the author (for example a percentage of the core's lookup tables, a percentage of the clock); nothing is measured yet, and no core has been synthesised (10.1.8).

**Options.**

1. *One dedicated secure core.* The design of `crypto-secure-core.md`: a small in-order core with the window, the clear engine and the bus slot, and no mode on the other cores. For: the other cores pay nothing; the guarantee is by construction (no speculation, no sharing); the design is already approved (errata 86). Against: a core that general work cannot use; secret work must move to it, and a process that needs more than the window does not fit.
2. *A secure mode on every core, entered by a privileged instruction or by a status-word bit.* The mode relates to `DIT` (9.3.7): `DIT` makes ordinary instructions constant-time, and secure mode adds the private window, the locked buffer, a fixed clock and no speculation. For: any core can run a secure domain; no core is set aside. Against: every core carries the logic; an out-of-order core must turn speculation off and drain its predictors on entry (a cost in time); the critical path may lengthen.
3. *Secure mode on a subset of cores (a core class).* The class register of 7.11.1 says which. For: a middle way: more than one secure core without the cost on all. Against: a scheduling rule that depends on the machine; the subset size is another number to choose.
4. *Secure mode as a core class chosen at build time.* A configuration parameter of the die, a number of secure cores from none to all. For: one design serves every product; the first device can have one and a larger die more. Against: the kernel must read the class and plan for any count; more variants to test.
5. *A time-shared secure domain on any core, with the clear-on-switch rule only.* The kernel clears the core's state when it enters and leaves a secure domain, and the core has no other mode logic. For: the least hardware. Against: a shared cache, translation buffer and predictors cannot all be cleared at a cost that hides the leak; it removes no channel that a clear does not reach.
6. *No secure mode, only `DIT`.* For: nothing added; `DIT` (9.3.7) covers constant-time instructions. Against: leaves table lookups indexed by secrets exposed to the cache and the translation buffer, which the private window and locked buffer exist to close; contradicts the decided 7.11.3.

**Precedents.** Arm TrustZone gives every core a secure state (*unverified*; Linux's `PSR_DIT_BIT` in `arch/arm64/include/uapi/asm/ptrace.h` and `PSTATE_DIT` in `sysreg.h` show the data-independent-timing bit on Arm, *read*). Intel SGX and TDX are modes of every core (*unverified* as designs; Linux has an SGX driver, `arch/x86/kernel/cpu/sgx`, and TDX configuration in `arch/x86/Kconfig`, *read*); the repository records that enclaves on a shared core suffer every microarchitectural channel (`hardware-vulnerabilities.md`, the SGX and TEE attacks row, and rule C15 against enclave claims without partitioned hardware, *read*). The RISC-V physical memory protection registers (Linux `arch/riscv/include/asm/csr.h`, `PMP_R`, *read*) and the security extensions are modes of every

core (*unverified* beyond the register names). Arm CCA realms are a state of every core (*unverified*). Apple's Secure Enclave and AMD's platform security processor are separate cores (*unverified*); OpenTitan is a separate security chip (*unverified*). The repository's own design (`crypto-secure-core.md`) is a separate core class, and its section 1.4 makes `DIT` constant on it (*read*).

**The author's referents.** "i like your design. approved." (errata 86); "constant-time AES and SHA instructions" (errata 85); the decision of 2026-11-10 above; `DIT` (9.3.7).

**The assistant's leaning.** Option 4: build the mode as a class that a die parameter sets, measure the gates and the slowdown on the first device, and let the author's threshold decide between one and all. The gate figures should come before the leaning is trusted. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 7.11 and 7.11.3, and 10.4.3.

## J.8 Chapter 8: Vector work, AI and array languages: the device route

### J.8.1 The operations and the number formats of the engine

**Issue.** The engine is a standard part of the architecture (8.4.1), so the architecture has to say what it does: its operations, its number formats, and how software learns what a given engine implements (8.4.2). The processor has `binary64`, `binary32` and `binary16` as scalar types (8.6.1) and nothing else; the engine is where `bfloat16`, 8-bit floating point, integers of 8 and 4 bits and one-bit weights would live, if they live anywhere. Three families of work make different demands.

- *Array languages* (APL and K, 8.5) want element-wise arithmetic and comparison, reductions (sum, minimum, maximum), scans (running sums), and index operations: gather, scatter (read or write through an array of indices) and compress (keep the elements for which a condition holds), over integers, bits and floating-point numbers.
- *Numerical libraries* are written on the BLAS (basic linear algebra subprograms), three levels of routines that date from 1979, 1988 and 1990 *(unverified)*: level 1 is vector with vector (`axpy`, `dot`), level 2 is matrix with vector (`gemv`), level 3 is matrix with matrix (`gemm`).
- *Machine learning* wants matrix multiply and convolution, normalisation, softmax, activation functions and gather.

What decides which operations repay an engine is how many arithmetic operations each byte moved supports (the **arithmetic intensity**). The figures below are computed for `binary64` and 8-bit data.

| Operation | Operations per byte moved | Why |
|---|---|---|
| `binary64` add of two arrays | 1 per 24 bytes, 0.042 | two reads and one write of 8 bytes for each add |
| matrix times vector (`gemv`, batch of one), 8-bit weights | 1 multiply-accumulate per weight byte | every weight is read once and used once |
| matrix times matrix (`gemm`) of 4096 by 4096, all 8-bit | about 1,365 multiply-accumulates per byte | 4096 cubed is 68,719,476,736 multiply-accumulates over 3 x 16,777,216 bytes |

An element-wise operation is limited by memory bandwidth whoever executes it: at the 2 GB/s that the LB64 proposal assumes (H.8), a `binary64` add stream moves 83 million elements a second (2,000,000,000 divided by 24 bytes) on the processor or on the engine alike; the loop of 8.1 at four instructions an element and one instruction per cycle at 200 MHz (errata 99) manages 50 million. A single element-wise pass is therefore not what the engine is for. It pays on fused chains of operations, on operations with reuse (matrix multiply), and on offloading the processor.

**The number formats.** The sizes are for one matrix of 4096 by 4096 elements (16,777,216 elements), computed.

| Format | Bits | What it is | Largest finite value | Bytes for the matrix |
|---|---|---|---|---|
| `binary64` | 64 | IEEE double | about 1.8e308 | 134,217,728 |
| `binary32` | 32 | IEEE single | about 3.4e38 | 67,108,864 |
| `binary16` | 16 | IEEE half: 5 exponent bits, 10 fraction bits | 65,504 | 33,554,432 |
| `bfloat16` | 16 | 8 exponent bits, 7 fraction bits: the range of `binary32` with less precision | about 3.39e38 | 33,554,432 |
| FP8 E4M3 | 8 | 4 exponent bits, 3 fraction bits | 448 | 16,777,216 |
| FP8 E5M2 | 8 | 5 exponent bits, 2 fraction bits | 57,344 | 16,777,216 |
| INT8 | 8 | signed integer, usually with a scale | 127 | 16,777,216 |
| INT4 | 4 | signed integer, usually with a scale | 7 | 8,388,608 |
| ternary | about 1.585 | the values -1, 0 and 1 (the base-2 logarithm of 3 is 1.585; five of them fit a byte, so 1.6 bits) | 1 | 3,323,907 (3,355,443 packed five to a byte) |

| Format | Bits | What it is | Largest finite value | Bytes for the matrix |
|---|---|---|---|---|
| binary | 1 | the sign alone: -1 or +1 | 1 | 2,097,152 |
| binary, 0.1 bit a weight | 0.1 | the LittleBit-2 form: low-rank sign factors with 16-bit scales (the LB64 document: 209 KB for the layer, signs and scales together) | | about 209,715 |

The largest values of the two 8-bit formats follow from their definitions: E5M2 follows the IEEE rules and tops out at 1.75 x 2 to the 15 = 57,344; E4M3 as the machine-learning formats define it tops out at 1.75 x 2 to the 8 = 448 *(the definitions are unverified here)*. Two more families have no entry above. **Posits** (a floating-point format with a variable split between exponent and fraction, giving more precision near one and less at the extremes) *(unverified)*. **Block floating point** and its standardised form, the **MX formats** (a block of 32 elements of 8, 6 or 4 bits that share one 8-bit exponent, which costs 8.25 bits an element for 8-bit elements and 4.25 for 4-bit ones; for the matrix 17,301,504 and 8,912,896 bytes, computed): the names `e2m1`, `e2m3`, `e3m2`, `e4m3`, `e5m2` and `ue8m0` appear in LLVM's NVIDIA intrinsic definitions (read), which is the evidence that the family is in hardware.

**Accumulate widths.** A dot product adds many products, so the sum needs more bits than the factors. Two 8-bit signed integers multiply to at most 128 x 128 = 16,384 (2 to the 14); a sum of n such products fits a signed 32-bit integer for any n up to 131,071 (131,072 x 16,384 is exactly 2 to the 31), so an 8-bit dot product accumulates in 32 bits without overflow over any realistic layer. Floating-point products of 8-bit and 16-bit formats are summed in `binary32` (24 significant bits), and a sum of thousands of terms in `binary16` loses its low-order bits. A count of agreeing bits (the binary case, 9.9.3) needs 13 bits for 4,096 terms.

**Why it matters.** The operation set decides what an array-language interpreter can hand to the engine and what stays on the processor, and so whether the engine repays its setup. The formats decide whether a model published in one of them runs. Because the engine is standard, a library and a program are written against the operation set and the list of formats as against `FADD`, and a later change costs a revision of the architecture and not of one device. A library must also be able to ask what a given engine supports.

**Options.**

1. *The array-language set: element-wise, reduce, scan, index.* About 50 operations in the assistant's count of the primitives of APL and K: 16 binary element-wise (add, subtract, multiply, divide, remainder, minimum, maximum, and, or, exclusive-or, the shifts, power), 12 unary (negate, absolute value, not, square root, reciprocal, floor, convert), 6 comparisons, 7 reductions and 7 scans (sum, product, minimum, maximum, and, or, exclusive-or), gather, scatter, compress, index generation and select; over integers of 8 to 64 bits, `binary16`, `binary32`, `binary64` and bit arrays. *Effect:* the processor is unchanged; the engine is streaming units behind the ring engine; the driver builds one kind of request. For: it is what the author named (APL and K, R277), it is small, and each operation maps to one descriptor (8.5.3). Against: no matrix operation, so inference gets only its element-wise parts; by the figures above a single pass is memory-bound, so the benefit is fusing a short expression into one descriptor, which this option does not specify.
2. *BLAS levels 1 to 3 over the floating-point formats.* About a dozen routines of levels 1 and 2 (`axpy`, `dot`, `scal`, `gemv`, `ger`, ...) and `gemm` (and its relatives `syrk` and `trsm`) at `binary16`, `binary32` and `binary64`, accumulating in the wider format. *Effect:* processor unchanged; the engine needs floating-point multiply-accumulate units, `binary64` ones being the largest; the driver passes matrix shapes, strides and scalars (`alpha`, `beta`). For: the software of numerical computing is written against the BLAS (`LAPACK` is built on it, *unverified*), so it ports; its numerical behaviour is well understood. Against: no integer, bit or index operations (nothing for APL's `where`), no convolution, softmax or normalisation; levels 1 and 2 are memory-bound like option 1.
3. *The machine-learning operator set, whole-tensor, at the level of TOSA.* The MLIR dialect that implements TOSA (the Tensor Operator Set Architecture) defines 71 operations (read, in the LLVM sources): 10 tensor operators (matrix multiply, three convolutions, depthwise convolution, two pooling operators, two Fourier transforms, argmax), 4 activations, 17 element-wise binary operators, 13 unary, 4 selection and comparison, 6 reductions, 7 layout operators (concatenate, pad, reshape, reverse, slice, tile, transpose), gather and scatter, resize, cast and rescale, and five helpers (constant, identity, custom, `if`, `while`). The dialect describes the set as meant "to enable a variety of implementations running on a diverse range of processors, with the results at the TOSA level consistent across those implementations", and defines profiles (supported subsets) and integer types down to 4 bits. ONNX's operator set is larger, about two

hundred operators *(unverified). Effect:* processor unchanged; the engine implements tensor operators with shape, stride, dilation and padding attributes; the descriptor carries those. For: a ready-made, reviewed operator set that frameworks already lower to, with a conformance idea (profiles). Against: 71 operators each with shape rules is a large specification to verify, and a small engine cannot do convolution and `while`; the control operators (`if`, `while`) are beyond a streaming unit.

4. *One matrix multiply-accumulate on tiles, plus a few element-wise helpers.* This is the tensor-core family. NVIDIA's matrix operations take fragments in `binary16`, `bfloat16`, TF32, 4-bit integers and single bits (with `xor.popc` and `and.popc`, the binary dot product that `XNORPOP` is), and the 8-bit floating-point and MX element types (read in LLVM's NVVM intrinsics). Intel AMX has 8 tile registers of 1 KiB (read: 8,192 bits each in LLVM, so 8 KiB in all) and instructions for 8-bit integer (four sign combinations), `bfloat16` and `binary16` products (29 definitions in `X86InstrAMX.td`, pseudo-instructions included). ARM's SME has outer-product accumulates, including a bitwise one (`BMOPA`) and `BFMOPA` for `bfloat16` (read). SiFive's `Xsfvqmaccdod` and `Xsfmm` are 8-bit matrix products for RISC-V (read). Google's TPU is a 256 by 256 array of 8-bit multiply-accumulate units with a command set of about a dozen instructions, and Apple has a matrix coprocessor (AMX) and a neural engine (ANE) *(unverified). Effect:* processor unchanged; the engine is a multiply-accumulate array; the tile shape is part of the operation. For: it covers nearly all the arithmetic of inference and training. Against: nothing for the array languages; element-wise work needs another unit; the tile geometry must be fixed or reported, and a matrix unit starved by memory does not help batch-one inference (J.8.4 computes this).
5. *A narrow binary or ternary kernel: sign-flip accumulate and population-count dot product.* The five steps of a LittleBit layer (the LB64 document: scale the activations, accumulate or subtract them by weight bit, scale, accumulate again, scale) on one-bit weights and 16-bit activations. *Effect:* processor unchanged; the engine is adders and sign flips with no multipliers; the driver builds a chain of descriptors per layer. For: the author's referent (LittleBit-2, thread 533); 160 times less weight traffic than `binary16` for a 4096 by 4096 layer (33,554,432 bytes against about 209,715, computed). Against: a standard part that serves one family of models is stranded if the field moves, as it has moved before (ternary models, *unverified*); it does nothing for the array languages.
6. *A small mandatory core and named optional sets, found by asking.* The core is option 1 plus `gemm`, at the integer formats, `binary16`, `binary32` and `binary64`; optional sets are named in a capability block that the program reads: further formats (`bfloat16`, FP8, 4-bit integers, binary, MX), further operators (convolution, softmax, normalisation). *Effect:* processor unchanged; every engine implements the core, which a small system may implement in firmware (J.8.4, option 5); the driver reads the capability block with the virtio feature words (5.4.4) and the device-specific configuration. For: standard (the core is always present, 8.4.2) and still extensible; the sets can follow the field. Against: a library must handle engines of different capability; conformance tests grow with the sets.

**Precedents.** *Verified in sources on this machine.* LLVM's NVVM intrinsics name the formats `f16`, `bf16`, `tf32`, `s4`, `u4`, `b1`, `e4m3`, `e5m2`, `e2m1`, `e2m3`, `e3m2` and `ue8m0`. LLVM's x86 target defines AMX tiles (8 of 8,192 bits) and the `VPDPBUSD` dot products; its AArch64 target defines `SDOT` and `UDOT` (92 definitions with their variants), `BFMLALB`, `BMOPA` and `BFMOPA`. The QEMU decoder for the RISC-V vector extension has the `bfloat16` forms (`vfwmaccbf16`, `vfncvtbf16`, `vfwcvtbf16`) and ordered and unordered floating-point sum reductions (`vfredosum`, `vfredusum`: the order of a floating-point sum changes the result, so the architecture names both). LLVM's RISC-V target has `Zvqdotq` (quad widening dot product), `Xsfvqmaccdod` and `Xsfmm`. LLVM's Hexagon target defines the HVX vector unit: 32 registers of 64 or 128 bytes and 992 definitions named `V6_`. MLIR defines TOSA (71 operations, profiles). The Linux accelerator documentation (`Documentation/accel`) sorts compute accelerators into edge inference, data-centre inference and data-centre training, and describes the AMD NPU, the Qualcomm Cloud AI accelerators and the Rockchip NPU, each with its own driver and its own user-space stack. *Unverified.* The TPU, Apple's AMX and ANE, ARM's Ethos engines, Tenstorrent (small RISC-V cores beside matrix and vector units), the RISC-V matrix extension group (IME), the Cray-1 (eight vector registers of 64 elements) and the VAX vector facility (sixteen registers of 64 elements; the repository's notes cite the VAX 6000 programmer's guide), the BLAS dates, the ONNX operator count, posits and the MX standard. The PDP-11's attached array processors took library calls with a fixed function list *(one brochure, unverified here).*

**The author's referents.** "mmap device arm style route" (R277); APL and K "easier to support and more efficient" (R277); "just optimize inference whether in cpu or gpu whatever makes most sense" (R139); "the simd/avax stuff" (R136); the LittleBit-2 encoding (thread 533, R136); and, 2026-11-10 (errata 131): "I want all the best options with rationale and precedents for each of those items. even if it doesn't involve specific opcodes 'in the cpu' still it must be built in as a standard part of the architecture, just as the fpu mmu and iommu are."

**The assistant's leaning.** Option 6, with the core of options 1 and 2 (the element-wise, reduce, scan and index set, and `gemm`, with 8-bit integers accumulating in 32 bits) and the field's moving formats as named sets. It makes the engine standard without freezing a format list that has changed every few years, and it lets the smallest system implement the core in software.

**Where the body refers to it.** Statements 8.4.1, 8.4.2, 8.5.5 and 8.6.2.

### J.8.2 The descriptor of the engine and the virtio descriptor

**Issue.** Chapter 5 puts every standard device on virtio queues (5.4.2): a descriptor is 16 bytes (a 64-bit address, a 32-bit length, flags, and a `next` index), and descriptors chain with `NEXT`. The engine needs a request that names an operation, several arrays with lengths and strides, formats and a place for the status. Two sizes follow by computation (an illustration, not a layout). An element-wise request fits 64 bytes: operation (2), format (2), flags (4), length (8), three addresses (24), completion address (8), three strides (12), cookie (4). A matrix request fits 128: header (8), three dimensions (24), three addresses (24), three leading strides (12), scale and bias addresses (16), completion address (8), four formats (8), cookie (4), with 24 bytes to spare. The architecture must say how such a request travels on a virtio queue, or say that the engine does not use one. The sizes of the requests of other devices:

| System | Entry | Bytes | Source |
|---|---|---|---|
| virtio split ring | descriptor (address, length, flags, next) | 16 | `linux/virtio_ring.h`, read |
| virtio packed ring | descriptor (address, length, id, flags) | 16 | the same header, read |
| virtio-blk | request header (type, priority, sector), then data buffers, then one status byte | 16 | `linux/virtio_blk.h`, read |
| virtio-net | header before each frame | 10 (12 with the buffer count) | `linux/virtio_net.h`, read |
| virtio-gpu | control header (type, flags, fence, context, ring), then the command body; `SUBMIT_3D` is a 32-byte header followed by a buffer of commands | 24 | `linux/virtio_gpu.h`, read |
| NVMe | submission queue entry (opcode, flags, command id, namespace, two pointers, six command words) | 64 | `include/block/nvme.h`, QEMU, read |
| io_uring | submission queue entry, with 65 operations | 64 | `linux/io_uring.h`, read; size from the compiler |
| Intel DSA | hardware descriptor, with 17 opcodes (no-op, batch, drain, move, fill, compare, compare with pattern, create and apply delta, dual cast, CRC, copy with CRC, four data-integrity operations, cache flush) and a 32-byte completion record | 64 | `linux/idxd.h`, read |
| Arm SMMU | command queue entry, with at least 25 commands | 16 | `hw/arm/smmuv3-internal.h`, QEMU, read |
| e1000 | transmit descriptor (address, length, command, status) | 16 | `hw/net/e1000_regs.h`, QEMU, read |
| DEUNA, 4.3BSD | ring entry (length, address low and high, flags, errors, request id) | 10 | `sys/vaxif/if_dereg.h`, read |
| RK11, PDP-11 | no entry: the bus address, word count and disk address are registers (`RKBA`, `RKWC`, `RKDA`) | 0 | chapter 5, read |

The DSA batch descriptor is the nearest relative of a chain of operations: its address field points to a list of descriptors and its count says how many (the fields `desc_list_addr` and `desc_count` in `idxd.h`). Virtio itself already carries commands in its first buffer: the virtio-blk request header and the virtio-gpu control header are device-defined blocks that the 16-byte descriptors point to.

**Why it matters.** If the engine does not fit the virtio queue it needs a ring engine, a driver skeleton and an IOMMU story of its own, which is the cost that errata 95 removed. A chain, one operation following another with no processor between, is how a layer of a network or an array expression is expressed. Each pointer in a request is an address the device will use, so the rule of 2.11 (descriptors hold device addresses, no kernel pointers) has to reach every pointer in the request, wherever it sits.

**Options.** For every option the processor is unchanged (`FENCE` and a doorbell, 8.4.6), apart from the instruction named in option 3.

1. *The virtio form: the command header is the first buffer, the operands are the next buffers of the chain, the status is the last.* This is the shape of virtio-blk and virtio-gpu. An element-wise request is five 16-byte descriptors (header, two inputs, one output, status) plus a header block of, say, 32 bytes: about 112 bytes of fetches against 64 for a fixed descriptor

(computed); a virtio indirect descriptor (VRING_DESC_F_INDIRECT, read) puts the five in one ring slot. *Device and driver:* the engine is a virtio device with one queue engine shared with the disk; the driver is the generic transport skeleton plus a request builder. For: nothing new in the transport; the IOMMU validates every operand because each is in a descriptor; the IN_ORDER feature (35, read) lets a sequence of requests complete in order. Against: five fetches for one operation; "the next operation uses this one's result" is two requests and a buffer in RAM between them.

2. *One descriptor points at a device-defined work block that holds the operands and can chain.* This was the previous leaning of this entry. The ring carries one 16-byte descriptor to a block of 64 or 128 bytes with a `next` field, as the DSA batch does. *Device and driver:* one ring, a block format in the engine's specification; the driver fills blocks. For: one fetch for the block, any size, a chain in the block. Against: the operand addresses are inside the block, outside the descriptors the transport and its tools understand; the IOMMU rule has to be written for the block's contents; the status of a chain is the block's business.
3. *A fixed 64-byte descriptor in a ring of the engine's own.* This is the NVMe, io_uring and DSA form. *Device and driver:* a second ring structure beside virtio; its own driver skeleton and its own IOMMU hooks; optionally one processor instruction that stores 64 bytes to the doorbell in one atomic write (x86 has ENQCMD and MOVDIR64B, which LLVM defines; their exact semantics *unverified*). For: one fetch per operation, a size that suits one cache line, the simplest engine. Against: it leaves virtio (errata 95 said every device is virtio); a 128-byte matrix request needs two entries; a second contract to keep correct.
4. *A 16-byte head followed by extension descriptors in the same chain.* The head says which operation and how many descriptors follow; some of those carry values (operation, format, stride) in place of an address, marked by a flag. *Device and driver:* the engine is virtio-like; a new descriptor kind is a change to what virtio means by a descriptor. For: all in 16-byte units on the existing ring, with the IOMMU applying to the pointer descriptors. Against: the engine must walk the chain to find its parameters; a "value descriptor" is not virtio.
5. *Commands that name objects held by the device: handles.* This is the virtio-gpu pattern: create a resource, attach its backing memory (once), then send small commands that name resources by number; the SMMU command queue is the same idea with 16-byte commands. Engine commands would be ALLOC, ATTACH, `EXEC(op, handle, handle, handle)` and FREE; results stay on the device between operations. *Device and driver:* the engine has memory it manages (allocation, lifetime, exhaustion); the driver tracks handles. For: weights are attached once and each layer is a small command; the IOMMU is consulted at ATTACH, not at every operation; no uncached readback between layers (8.4). Against: the device carries an allocator, which is operating-system work inside the device; on an engine that uses only host RAM the handles add a layer for nothing.
6. *Registers, as the RK11 had.* A few device registers hold the operation and the addresses, one request at a time. *Device and driver:* the native register model that errata 95 removed. For: the simplest hardware. Against: one operation at a time, an uncached store for each field, no queue and no chain. It is listed as the ancestor and not as a candidate.

**Precedents.** Verified: the table above. Virtio's own use of the first buffer for a device-defined header (blk, net, gpu) and its indirect descriptors are in the Linux headers. The Linux headers define no virtio device type for compute: the identifiers in `virtio_ids.h` run from VIRTIO_ID_NET (1) to VIRTIO_ID_SPI (45), with a GPU (16) whose SUBMIT_3D command carries an opaque command stream in a buffer, and none for a neural-network or vector engine; the engine would need an identifier of its own *(the allocation procedure is unverified)*. Unverified: the PDP-11 attached array processors took library calls and device registers.

**The author's referents.** "virtio across the board" (errata 95); "descriptors" (R277); the PDP-11 RK11 and the DEUNA ring as the ancestors in chapter 5.

**The assistant's leaning.** Option 1, refined: the engine is a virtio device whose requests have a command header in the first buffer, as virtio-blk and virtio-gpu do, and that header may itself hold a short chain of operations (option 2's list) so that fused expressions are one request; handles (option 5) only if the engine has memory of its own. No change to virtio is needed.

**Where the body refers to it.** Statements 8.4.2 and 8.4.7.

### J.8.3 Packed integer lanes in the ordinary registers

**Issue.** The processor has no vector register file (8.3.1). A smaller question remains: whether instructions should treat a 64-bit integer register as eight bytes, four halves or two words (a **lane** is one of those slots), adding, comparing or taking the

minimum in every lane at once. The technique without such instructions is **SIMD within a register** (SWAR): ordinary instructions with masks that stop a carry crossing a lane boundary. Adding eight bytes at a time, with `H` the constant `0x8080808080808080`:

```
sum = ((a & ~H) + (b & ~H)) ^ ((a ^ b) & H)      # six operations, one add: each byte added modulo 256
```

Clearing the top bit of every byte leaves room for the carry; the exclusive-or puts the top bits back. The test for a zero byte in a word, `((x - 0x0101010101010101) & ~x & H) != 0`, is four operations. Both were checked against a byte-at-a-time loop on random 64-bit values. So a packed instruction saves a factor of about six on an add, and more on saturating forms. What the pdp64 already has bears on how much a packed set would add: `POPCNT`, `XNORPOP`, `CLZ`, `CTZ`, `BSWAP` and `BSEL` (9.9), and the block operations `MSCN`, `MSCNN`, `MCMP` and `MVAL` (9.11, 9.18.2), which already do the loops of `strlen`, `memchr`, `memcmp` and UTF-8 validation with no packed instruction.

**Why it matters.** The opcode map has room for few words; the compiler would need to recognise the loops; and the gain is a measurement that nobody has made. The cases that remain after the block operations are hashing, checksums, pixel arithmetic (a 48-bit pixel, errata 73, is three 16-bit lanes), base-64 and similar byte-parallel work, and, for inference on the processor, the 8-bit dot product.

**Options.**

1. *None. Processor:* no change. For: no opcodes; the processor stays small; the block operations already serve the string cases. Against: byte-parallel loops run one byte at a time or in SWAR sequences of four to six operations.
2. *SWAR helpers only: a zero-byte marker and its relatives.* `ORCB` (each byte becomes `0xFF` if nonzero and `0x00` if zero; RISC-V's `Zbb` extension has it as `orc.b`, read in LLVM) makes the zero-byte test two operations, and a few lane-wise selects are of similar size. *Processor:* 1 to 3 instructions. For: the smallest set that speeds hand-written scanners and is near `POPCNT` in cost. Against: little over what `MSCN` does; no arithmetic.
3. *A full packed integer set.* Add, subtract, saturating add and subtract (signed and unsigned), minimum and maximum (signed and unsigned), average, compare for equality and for greater-than (signed and unsigned) and shifts, on 8, 16 and 32 bits: about 14 operations times 3 widths = 42 mnemonics, which is one leaf with a 6-bit sub-opcode (9.1.2; the encoding is being re-optimised, so the cost is measured again then). *Processor:* an instruction-set extension, with lane-masking in the adder. For: serves every byte-parallel loop and pixel operation. Against: 42 opcodes, assembler and compiler support, and a width fixed at 64 bits; the vectoriser must find the loops. Precedents are mixed: ARM's A32 instruction set has packed operations in the general registers (`SADD8`, `UADD8`, `QADD8`, `USAD8`, `SEL` are defined in LLVM's ARM target, read) and its A64 set dropped them for NEON (no such definition in LLVM's AArch64 target, read); Alpha's MVI is 13 instructions (read in QEMU: `minub8`, `maxsb8`, `perr`, `pkwb` and others); PA-RISC's MAX-2 has 17 halfword patterns (`hadd`, `havg`, `hshl`, `mixh`, `permh`, ..., read in QEMU); SPARC's VIS has 39 helper functions in QEMU (`fpmerge`, `pdist`, `fmul8x16`, `fpack16`, the lane compares `fcmpgt16`, array addressing); the RISC-V `P` extension is in LLVM as an experimental extension at version 0.14 with a partial definition (read). MMX began as packed integers in the floating-point registers, MDMX is MIPS's, and TI's C6x has packed halfword and byte operations *(unverified)*.
4. *Only the integer dot product.* `DOT8S`, `DOT8U`, `DOT8SU` and `DOT16S`: multiply eight bytes of one register by eight of another and add the eight products into a 64-bit accumulator (four mnemonics, a 2-bit sub-opcode). *Processor:* 4 instructions. For: it is the packed operation that inference needs, the 8-bit counterpart of `XNORPOP`, which does 64 one-bit multiply-accumulates per instruction. A 4096 by 4096 layer of 8-bit weights is 16,777,216 multiply-accumulates; with a scalar load, multiply and add at about 3 instructions each, 50.3 million instructions, 252 ms at 200 MHz and one instruction a cycle; with `DOT8S` and two loads per 8 products, 6.3 million instructions, 31 ms; memory at 2 GB/s needs 8.4 ms (all computed under those assumptions), so the processor would be within a factor of four of memory-bound. Against: 8 multiply-accumulates an instruction is still two orders below a matrix engine; the accumulator and the sign variants add four opcodes for one workload. Precedent: x86 `VPDPBUSD` (VNNI) and ARM `SDOT` and `UDOT` are in LLVM's targets (read); RISC-V `Zvqdotq` (read).
5. *Packed lanes on the engine only. Processor:* no change. For: nothing in the processor. Against: the setup cost (a descriptor, a doorbell, uncached readback) is far above the cost of a short string or a pixel row (8.4).
6. *A lane prefix: one prefix over the existing ALU operations.* The prefix names the lane width and the next instruction (`ADD`, `SUB`, `MIN`, `MAX`, the compares, the shifts) runs once per lane; this is the closest thing to "normal operations with a SIMD bit" (J.8.5). *Processor:* one prefix leaf in the prefix code (9.1.3) and lane-masking in the ALU. For: the smallest opcode cost; the operations keep their names. Against: every ALU operation must define its behaviour per lane (carry, overflow,

saturation, and what a flag-writing instruction does when there are eight results); the decoder pairs a prefix with the next instruction. x86's VEX and EVEX prefixes are the large precedent (unverified in detail).

**Precedents.** Verified in the sources named above: ARM A32 against A64; Alpha MVI (13), PA-RISC MAX-2 (17), SPARC VIS (39 helpers), RISC-V `P` (experimental), `orc.b`, `SDOT`, `VPDPBUSD`, `Zvqdotq`. Unverified: MMX, MDMX, C6x. The PDP-11 had none.

**The author's referents.** "the simd/avax stuff" as the thing to settle by the device route (R136); the bit instructions that exist (9.9) were his.

**The assistant's leaning.** Decide by measurement, as before: compile the corpus at higher optimisation and count the loops that would vectorise, measure the share of time in byte-parallel loops after the block operations, and estimate the cost in block RAM before any opcode is spent. Of the options, 4 is the one with an inference argument behind it (R139, inference "in cpu or gpu whatever makes most sense"); 2 and 3 add little over `MSCN` and `MCMP`.

**Where the body refers to it.** Statement 8.3.4.

## J.8.4 Which engine is built first

**Issue.** The engine is standard (8.4.1) and the architecture says what it must do (J.8.1, J.8.2); what remains is what the first implementation is. The author's words point two ways. On 2026-10-20 (R136) he asked what "a gpu ... optimized purely for samsung's littlebit-2 encoding" would look like, which points to an engine for one family of binary-weight models. On 2026-10-23 (R278) he wrote "if we can plug a regular card into the bus, why not", and on 2026-11-04 (R315) "for now we will have a standard pcie set of audio/video devices", which point to cards from the field. On 2026-11-10 (errata 131) he said the engine is a standard part of the architecture like the FPU, the MMU and the IOMMU. A first engine needs a transport (J.8.2), operations and formats (J.8.1) and a driver. The numbers that separate the candidates are the time to stream the weights of one layer of 4096 by 4096 weights at the 2 GB/s that the LB64 proposal assumes (computed: bytes divided by 2 x 10 to the 9):

| Weights | Bytes | Time at 2 GB/s |
|---|---|---|
| `binary16` | 33,554,432 | 16.8 ms |
| 8-bit | 16,777,216 | 8.4 ms |
| 4-bit | 8,388,608 | 4.2 ms |
| ternary | 3,323,907 | 1.7 ms |
| binary | 2,097,152 | 1.05 ms |
| LittleBit, 0.1 bit a weight | about 209,715 | 105 microseconds |

A matrix engine of 16 by 16 multiply-accumulate units at 200 MHz does 256 x 200,000,000 = 51.2 billion multiply-accumulates a second, which at batch one (one multiply-accumulate per weight byte) needs 51.2 GB/s of weights; at 2 GB/s it is used 3.9% of the time, and it reaches full use only when each weight byte serves about 26 products (the batch or the reuse in a convolution). An element-wise engine is memory-bound already (J.8.1): one adder at 100 MHz exceeds the 83 million elements a second that 2 GB/s allows.

**Why it matters.** The first engine sets the first descriptor format, which every later one copies, and the first operation set that libraries are written against. An engine built for one model family is fast on it and idle on the rest. Until one is chosen, "inference on the engine" (8.5.1) has nothing to run on.

**Options.**

1. *The LittleBit-style binary inference engine (LB64).* Sign-flip accumulate with no multipliers, driven by chains of descriptors, one chain per transformer block (H.8). The LB64 document computes 1,409,024 sign-flip accumulates for one 4096 by 4096 layer at 0.1 bit a weight: 11,008 cycles with 128 lanes, 110 microseconds at 100 MHz, in step with the 105 microseconds of streaming. *Processor:* no change. *Device and driver:* a small device (adders and sign flips), a chain-building driver. For: the strongest fit to the workload the author named; the best use of memory bandwidth by a factor of 80 against 8-bit weights (8.4 ms against 105 microseconds); small logic. Against: one model family; no help for the array languages or for attention (activations against activations: the LB64 document says so); a standard part that serves one family is a poor standard.
2. *A general matrix engine.* A tile or systolic array with accumulate, over 8-bit and 16-bit formats (J.8.1, option 4). *Processor:* no change. *Device and driver:* a multiply-accumulate array with local tile memory; tile-shape-aware descriptors. For: serves all dense inference and training; the form of the TPU, tensor cores, AMX and SME. Against: by the arithmetic

above it is starved at batch one on a 2 GB/s memory; multipliers cost the most in an FPGA; nothing for the array languages.

3. *A general element-wise vector engine.* The array-language set (J.8.1, option 1) with fused short expressions in one descriptor. *Processor:* no change. *Device and driver:* streaming units behind the ring engine, one request kind. For: serves APL and K, graphics work and the element-wise parts of inference; the cheapest hardware; it is the engine the author's R277 describes. Against: a single pass is memory-bound, so the processor loop is within a factor of two of it (50 against 83 million elements a second, J.8.1), and the benefit depends on fusion and on offloading the processor, not on speed.
4. *A standard PCI Express GPU or NPU card driven through virtio, and no engine of the pdp64's own.* The accelerator is whatever card has a driver (R278). *Processor:* no change. *Device and driver:* the card's own command formats. For: no hardware to design. Against: it contradicts 8.4.1, because a card is optional and its command set is its maker's, not the architecture's, unless the card implements the architecture's engine interface (a possibility only if it speaks the descriptor of J.8.2); the Linux accelerator subsystem shows each such card with its own driver and its own user-space stack (AMD, Qualcomm and Rockchip accelerators are documented in `Documentation/accel`, read), and the virtio headers define no compute device (J.8.2), so the card is not driven through virtio today.
5. *No silicon yet: the architecture specifies the engine, and the first implementation is software.* A core (J.7.5) or the processor itself runs the operation set behind the same descriptors and the same virtio queue; hardware engines follow when a measurement justifies one. *Processor:* no change. *Device and driver:* a virtio device implemented in software; the driver is the same as for hardware. For: the interface is settled and exercised before any silicon is committed; it satisfies "every system has the engine" at once; QEMU already implements devices of other classes as software behind virtio (read). Against: no accelerator to measure the decision against; a software engine is no faster than the loop it replaces and may be slower.

**Precedents.** Verified: the Linux accelerator documentation and its three categories; the LB64 numbers (the proposal's own); QEMU's software virtio devices. Unverified: ARM systems attach Ethos and Mali engines as memory-mapped devices; Google's TPU is a PCI Express card; the PDP-11's attached array processors took library calls and a fixed function list.

**The author's referents.** "what would a gpu look like that is optimized purely for samsung's littlebit-2 encoding" (R136, thread 533 message 51); "if we can plug a regular card into the bus, why not?" (R278); "for now we will have a standard pcie set of audio/video devices" (R315); "just optimize inference whether in cpu or gpu whatever makes most sense" (R139); and the 2026-11-10 words on the standard part (errata 131).

**The assistant's leaning.** Option 5 first, then option 3: specify the engine and implement it in software, so that the descriptor and the operation set are exercised by programs, then build the element-wise engine with fusion as the first hardware, with LB64 (option 1) as an optional set of J.8.1's option 6 and the matrix engine when a batch workload exists to feed it.

**Where the body refers to it.** Statement 8.4.8.

### J.8.5 Operations on regions of memory in the instruction set

**How SIMD works.** *The idea.* A **SIMD** instruction (single instruction, multiple data) applies one operation to several numbers at once. The numbers sit side by side in one wide register; each slot is a **lane**. A 128-bit register holds four 32-bit lanes, and one `paddd` instruction of x86 adds four pairs, cutting the carry at every lane boundary:

```
reg a:    | 7 | 250 | 3 | 100 |      four 32-bit lanes (shown as small numbers)
reg b:    | 1 |   5 | 3 |  28 |
a + b:    | 8 | 255 | 6 | 128 |      one instruction, four additions, no carry between lanes
```

*The parts.* A SIMD design has a **register file** of wide registers (SSE: 16 of 16 bytes; AVX-512: 32 of 64 bytes, with 8 mask registers, read in LLVM); **element widths** (the same register read as 16 bytes, 8 halves, 4 words or 2 doublewords, chosen by the instruction or by a state register); **masks** or **predicates** (a bit for each lane saying whether it takes part: AVX-512's `k` registers, ARM SVE's predicate registers, the vector mask `v0` of RISC-V); **shuffles** (a lane moved to another position: `vrgather` of RISC-V, `TBL` of ARM, `vpermd` of x86); **widening and narrowing** forms (`vwadd` adds 8-bit lanes into 16-bit results, `vnsrl` goes the other way); **saturating** forms (clamp at the largest value instead of wrapping); **reductions** (add all the lanes into one number); and loads and stores that are contiguous, strided or indexed (`vle`, `vlse` and `vlxei` in the RISC-V decoder).

*Why the families differ.* The question each answers is where the register width lives. **SSE, AVX and AVX-512** (x86), **NEON** (ARM) and **AltiVec** (PowerPC) put the width in the instruction set: 128, 256 or 512 bits, so a wider generation is new instructions, new state and new kernel code, and an old binary cannot use the new width (8.2). **ARM SVE** and the **RISC-V vector extension** make the width the machine's choice and the program's question: the program asks how many elements it can do this round and loops (**vector-length agnostic**, VLA); SVE's registers are multiples of 128 bits (Linux `sve.rst`, read), RISC-V's width is `VLEN`, fixed by the implementation. **Cray-1 and the VAX vector facility** had a fixed 64 elements and a length register *(unverified; the repository's notes cite the VAX 6000 programmer's guide)*.

*How many instructions there are.* Each operation, times each element width, times signed and unsigned, times saturating or wrapping, times register forms (vector with vector, with scalar, with immediate, masked or not) multiplies. The counts below were made from sources on this machine; the unit differs between sources, and each is stated.

| Instruction set | Count | Unit and source |
|---|---|---|
| RISC-V vector extension | 407 patterns, 235 distinct operations; 362 patterns and 205 operations without the crypto, bit-manipulation and `bfloat16` additions | patterns in QEMU 11.1.2 `insn32.decode` whose names begin with `v`; for comparison, 370 patterns in the same file for everything else (integer, multiply, atomics, floating point, bit manipulation, crypto, scalar) |
| ARM SVE | 790 distinct pattern names (903 patterns) | QEMU `sve.decode` |
| ARM SME | 349 distinct pattern names (610 patterns) | QEMU `sme.decode` |
| ARM A64, all of it (integer, floating point, Advanced SIMD) | 702 distinct pattern names; 1,546 distinct mnemonics and 9,002 instruction definitions in LLVM | QEMU `a64.decode`; LLVM `AArch64.td` read with `llvm-tblgen` (a definition is one form of an instruction, pseudo-instructions included) |
| x86, all of it | 1,800 distinct mnemonics; 22,371 instruction definitions | LLVM `X86.td`, read the same way |
| x86, AVX family (VEX and EVEX forms) | 925 mnemonics; of the 1,800, about 270 are SSE and MMX by name pattern (approximate) | the same file |
| Hexagon HVX | 992 definitions | LLVM `HexagonDepInstrInfo.td`, names beginning `V6_` |
| Intel AMX | 8 tile registers of 1 KiB; 29 definitions | LLVM `X86InstrAMX.td` |
| Alpha MVI | 13 | QEMU `translate.c` |
| PA-RISC MAX-2 | 17 patterns | QEMU `hppa/insns.decode` |
| SPARC VIS | 39 helper functions | QEMU `sparc/vis_helper.c` |
| MMX, MDMX, TI C6x, Cray-1, VAX vector facility, CDC STAR-100 | unverified | no source on this machine; the order of magnitude of the early sets is tens |

The explosion is shown by one operation, the integer add of vectors. LLVM's x86 target has 194 definitions whose names begin `VPADD`: eight mnemonics (`vpaddb`, `vpaddw`, `vpaddd`, `vpaddq` and the four saturating forms `vpaddsb`, `vpaddsw`, `vpaddusb`, `vpaddusw`), each at 128, 256 and 512 bits, each from registers or from memory, and each plain, merge-masked or zero-masked, with a broadcast form for the 32- and 64-bit widths. The RISC-V decoder has three patterns for the same add: `vadd_vv`, `vadd_vx`, `vadd_vi` (vector with vector, with a scalar register, with a 5-bit immediate); the element width is in the state register `vtype` and the mask is one bit of the instruction. LLVM's AArch64 target has 8 `ADDv` definitions for the NEON add (arrangements of 8, 16, 32 and 64 bits) and 18 `ADD_Z…` definitions for the SVE add.

*Is SIMD "the normal operations with a SIMD bit"?* Not in any real design, with one partial exception; each design differs in precisely this respect.

- **RISC-V V** has its own major opcode (`1010111`, the same for all, read in the decoder) and its own operation fields, with names that echo the scalar ones (`vadd`, `vand`, `vsll`). Three forms (`vv`, `vx`, `vi`) and one mask bit. The element width and the vector length are not in the instruction: they are state set by `vsetvli`. So it is separate vector instructions that share the scalar names, with width as state.
- **AVX and AVX-512** are the nearest to "ordinary operations plus modifier bits": a prefix (VEX or EVEX) in front of the legacy opcode (`paddd` becomes `vpaddd`) carrying the register width, the mask register, a zeroing bit and a broadcast bit *(the encoding is unverified here; the names are read in LLVM)*. The operation is the legacy opcode and the vector shape is the prefix.
- **ARM SVE** has the element size in the instruction (`ADD_ZPmZ_B`, `_H`, `_S`, `_D`), a predicate register named by the instruction, and the vector length unnamed (hardware). **NEON** has the arrangement in the instruction (`ADDv8i8`, `ADDv16i8`, …).
- **Cray-1 and the VAX vector facility** have separate vector opcodes, a length register and a mask register *(unverified)*.

So the answer to the author's question is: partly, in x86, where a prefix reuses the opcode map; elsewhere the vector operations are a separate group with their own opcodes. A lane prefix is possible in the pdp64's prefix code (J.8.3, option 6).

**Issue.** The author asked (2026-11-10, errata 131): "how does simd work, how many instructions are there, is simd just normal ops fused with a 'simd' bit or something like that? would it make sense to have simd ops that work on regions of memory instead of register files, since we'd be load/storing register files anyways? I guess then we'd be invisibly load/storing into a specialized register file of a size which user code doesn't need to worry about." Already decided: the processor has no vector register file and no SIMD instruction set (errata 59, 8.3.1), and the wide work goes to the standard engine (8.4.1). This item asks whether to add, besides, instructions that operate on regions of memory with no register named. The precedent for the idea is old (memory-to-memory vector machines) and the pdp64 already has the mechanism for it: the block operations `MCPY`, `MMOV` and `MSET` take a destination, a source and a count in registers, keep their progress in those registers, and restart after a fault (9.11, 6.4: the **progress** category). A region operation would be a block operation with arithmetic: `c[i] = a[i] + b[i]` for a count in a register.

**Why it matters.** The answer decides whether the instruction set gains a group of instructions (opcodes, restartability, a rule for hidden state, an addition to the progress category of 6.4.1) or the engine stays the only path for wide work, and whether short arrays, which the engine's setup cost makes slow (8.4), have a fast path on the processor. One number frames it: a chain of k binary operations over `binary64` arrays moves 8 x (k + 2) bytes an element when the intermediates stay in registers (k + 1 inputs read, one result written) and 24 x k bytes when every operation goes memory to memory (two reads and one write each). For k = 1 they are equal (24 and 24); for k = 2, 32 against 48; for k = 4, 48 against 96; for k = 8, 80 against 192 (computed). The register file exists to keep intermediates; a region operation gives that up unless it fuses a chain inside one instruction.

**Options.** The five options of the author's brief, with what user code sees and what each costs. "Opcodes" counts mnemonics in the pdp64's terms: a leaf with a sub-opcode field of n bits holds up to 2 to the n of them (9.1.2), and the encoding is being re-optimised, so each count is measured again then.

1. *(a) Memory-to-memory vector instructions with a hidden internal buffer, restartable, with the progress in registers.* One instruction takes destination, source and count in registers, for example `VADD rd, rs, rt, rn`; the processor streams the data through an internal buffer that no program can name and that lives only inside the instruction. *User code sees:* a block operation that does arithmetic; no vector register, no state across instructions. *Context switch and faults:* nothing to save beyond the registers, since the buffer is dead between instructions; a page fault stops it as `MCPY` stops, with the pointers and the count showing the position (progress, 6.4.1, 9.11.4), and an interrupt does the same. A reduction (a sum) keeps its partial result in a general register; floating-point sums depend on the order, so the architecture must name the order, as RISC-V names ordered and unordered sums (`vfredosum`, `vfredusum`, read). *Precise exceptions:* as for the block operations, consistent with element-at-a-time progress wherever an exception is taken (9.11.5). *Memory ordering with other cores (chapter 7):* each element access is an ordinary access in the single order (7.5.1); an implementation may read ahead only if it replays when a store from another core touches the data (7.7.3), and the rule for overlapping source and destination must say which element goes first (`MMOV` is overlap-safe; the forward-only rule is the cheaper). *Caches and IOMMU:* the accesses go through the processor's data cache and MMU, so they are coherent, and the IOMMU is not involved; a streaming operation pollutes the cache, which is the matter of J.9.8. *Cost in opcodes:* with the operation and format in the mnemonic, about 24 operations times 7 formats (four integer widths, `binary16`, `binary32`, `binary64`) is 168 mnemonics, an 8-bit sub-opcode; with the format in a 3-bit field, 24 mnemonics and a 5-bit sub-opcode; with one instruction `VEXEC rd` that executes a descriptor in memory (the engine's descriptor, J.8.2), 1 or 2 mnemonics; Intel's `LDTILECFG` also loads a block of configuration from memory *(its size is unverified)*. *Compiler:* a library or an intrinsic lowers to it, as `memcpy` lowers to `MCPY`; a loop-recognising vectoriser is easier than for SIMD because the count is a register (no tail loop, no alignment, no width). *Processor:* an instruction-set extension plus a sequencer and a buffer. *Device and driver:* none. For: no architectural state; short arrays have a fast path with no setup; uses the progress mechanism already decided. Against: the byte traffic above for chains; a long-running instruction (the question of J.9.7, the worst-case latency of a block operation, applies); the progress category grows.
2. *(b) The same operations executed by the standard engine through the descriptor route, with an instruction that submits and waits.* The operations of J.8.1 run on the engine (decided direction, 8.4.4); the processor gets at most two conveniences: an atomic submit (`ENQCMD`-like: one 64-byte store to the doorbell with a success flag) and a wait (the `WFE` of 7.10 already waits for an event). *User code sees:* a library call; or an instruction `VSUBMIT` and the existing wait. *Context switch and faults:* no

processor state; the engine cannot take a page fault, so the buffers are pinned and mapped for the device (5.10, 2.11) and a process's pending requests need the kernel's attention at exit. *Precise exceptions:* none involved; completion is by status. *Memory ordering:* by `FENCE` and the doorbell (7.5.7); the buffers are uncached (5.10.1), so data the processor produced in cached memory is not what the engine reads unless it was written to uncached memory, and a result is read back slowly. *Caches and IOMMU:* device addresses through the IOMMU (2.11), each device its own domain. *Cost in opcodes:* 0, or 1 for the atomic submit. *Compiler:* none; runtime and library. *Processor:* no change, or one instruction. *Device and driver:* the engine of J.8.4. For: no change to the processor; it is the decided path; nothing hidden. Against: the setup and the uncached hand-off make short arrays slower than a scalar loop; the processor cannot use it from a signal handler or the kernel without the driver.

3. *(c) A vector register file of implementation-defined size, behind an instruction set that never names its length.* This is what the author's last sentence describes: loads and stores into a hidden register file whose size the program need not know, the VLA idea applied to memory regions. Instructions `VLOAD` (region into the hidden file), `VOP` (operate in the file), `VSTORE`, and a length query. *User code sees:* vectors of unknown length, a loop that asks how many elements it may do (strip mining, 8.2); the registers themselves are still architectural state, though unnamed in length. *Context switch and faults:* the hidden file is live across instructions, so it must be saved, which needs an instruction to save and restore it and a way to ask its size. The precedents show the price: ARM SVE's registers go up to 256 bytes each (32 of them, 8,192 bytes plus predicates; QEMU's limit, read); SME's `ZA` array is up to 256 by 256 bytes, 64 KiB (read); Intel AMX's tiles are 8 KiB, and Linux does not enable them by default: the first use traps, because the 2 KB minimum signal stack that old programs compile in is too small for the state (`xstate.rst`, read); RISC-V made vector use an opt-in per thread (`prctl`, `vector.rst`, read); for comparison the FP11's state is 50 bytes (8.1) and a 32-register file of 16 bytes is 512. Interrupts must save or defer the file; a page fault in `VLOAD` or `VSTORE` must restart without corrupting it. *Precise exceptions:* the file is a visible state after every instruction, so each instruction must be precise; a long `VOP` needs progress too. *Memory ordering:* as option 1 for each load and store. *Caches and IOMMU:* as option 1. *Cost in opcodes:* RISC-V's vector set is 362 patterns without extensions (read); a restricted design is 40 to 60 (loads, stores, 20 operations, reductions, shuffles, the length and the save). *Compiler:* LLVM already vectorises for RISC-V V and SVE (their targets are in the LLVM sources, read); the ABI needs a rule for the hidden file in calls and signals. *Processor:* a second register file in hardware, the thing 8.3.2 declined. For: the standard shape of modern vector machines; keeps intermediates in registers. Against: it is the decision of errata 59 reversed in all but name (a register file the program cannot size but must save), with the kernel and ABI work that decision avoided. Whether a file that is "invisible" counts as a vector register file in the sense of errata 59 is the author's to say.
4. *(d) Fixed-width SIMD.* 128-bit or 256-bit lanes in a register file, as SSE, AVX2 and NEON. *Cost in opcodes:* 925 in the AVX family of x86 and 1,546 mnemonics in all of A64 (LLVM, read); a width in every instruction. *Context switch:* 256 bytes (SSE) to 2,112 bytes (AVX-512: 32 registers of 64 bytes and 8 mask registers). Rejected by the author's decision (errata 59, 8.3.3); listed for completeness. For: no setup, the largest installed base of software. Against: all of 8.3.2 and 8.3.3.
5. *(e) None: scalar loops and the engine only. User code sees:* loops, `MCPY` and its kin, the bit instructions (9.9), and the engine for wide work. *Cost in opcodes:* 0. For: nothing to build; the decision stands as it is. Against: short arrays are scalar loops, and every wide operation pays the engine's setup.

**Precedents.** *Verified in sources on this machine.* Memory-to-memory data movement by descriptor: Intel DSA's 17 operations (`idxd.h`). The pdp64's own block operations and x86 `REP MOVS` with fast-string microcode, and ARM's `CPYP`, `CPYM` and `CPYE` (LLVM AArch64 defines the `CPYE` family; chapter 9 cites `FEAT_MOPS`). Register groups: RISC-V `LMUL` (the whole-register forms for 1, 2, 4 and 8 registers: `vl1re8`, `vl2re8`, `vl4re8`, `vl8re8`, read). Hidden state and its price: AMX tiles (8 KiB, `xstate.rst`), SME's `ZA` (up to 64 KiB, QEMU), SVE's registers, RISC-V's opt-in, the lazy-switch leak of x86 recorded in 8.2. *Unverified.* The CDC STAR-100 (1974) and Cyber 205 were memory-to-memory vector machines, with vectors up to tens of thousands of elements and a start-up cost per instruction that made short vectors slower than scalar code; the TI ASC was another (the repository's notes mention the STAR-100 and the ASC as memory-to-memory, "not confirmed in the sources read"). The Cray-1 chose vector registers and was faster on short vectors, which is the usual account of why memory-to-memory vector machines gave way.

**The author's referents.** "how does simd work, how many instructions are there ... would it make sense to have simd ops that work on regions of memory instead of register files" (errata 131); "the simd/avax stuff" (R136); "mmap device arm style route" (R277). The block operations and their progress rule are the decided statements 9.11.3 to 9.11.5.

**The assistant's leaning.** Option 2 (b) stays the path for wide work, as decided. Region operations are a sound idea and a real design (the STAR-100 was one, and the pdp64's block operations are the same mechanism without arithmetic), but their weakness is the 24 bytes an element and operation, which fusion cures only by turning the instruction into a small program, and at that point it is the engine's descriptor executed by the processor. If the author wants a fast path for short arrays without the engine's setup, the form to consider is `VEXEC` of option 1 (one instruction, a descriptor in memory, progress in registers), measured against the loops of J.8.3's corpus count first. Options 3 and 4 are the register file that errata 59 declined.

**Where the body refers to it.** Statement 8.3.5.

## J.9 Chapter 9: The instruction set

### J.9.1 The conditional select and the expansions of ABS, MIN, MINU, MAX and MAXU

**Issue.** A conditional select is an instruction that picks one of two values by a condition, with no branch (x86 `CMOV`, ARM64 `CSEL`). The pdp64 has none, except the pair `MASKEQZ` and `MASKNEZ` (each tests one register against zero and gives either zero or the other operand: `rd = rc == 0 ? 0 : ra`, and the opposite), and `SET cc, rd`, which turns a flag condition into 0 or 1. The author has decided that `ABS`, `MIN`, `MINU`, `MAX` and `MAXU` are assembler aliases (words the assembler expands into other instructions), not native instructions, and that the manual entry of each shows the alias (errata 98). He said he was happy for them to be aliases on top of a select ("`cmov`, `csel`, `fsel`"), but nothing yet says which select, or what each alias expands to. The flags the five words promise are fixed: `ABS` sets `N` and `Z` from the result, sets `V` when the operand is the most negative number, and leaves `C`; `MIN`, `MINU`, `MAX` and `MAXU` set `N` and `Z` from the result, clear `V` and leave `C`. Any expansion that starts with `CMP` or `SUB` overwrites `C`, and `TST` on the result restores `N` and `Z` and clears `V` but not `C`; so the real choice is to change the flags and say so, or to spend an extra instruction.

**Why it matters.** Until the expansion is fixed the entries of the five words cannot show it (section 9.7.6). Kernel code uses clamps and minimum and maximum in size computations, and a constant-time select is wanted for checking message-authentication codes and handling keys. A compiler must know whether a comparison used as a value may become a select, and an optimizer that is free to turn a select back into a branch would break the constant-time promise (J.9.2), so the ABI needs a rule. In hardware, a select that reads flags needs the flag file read by a data instruction (in an out-of-order core, renamed flags read at the select); `MASKEQZ` reads only registers. A 64-bit select is estimated at about 100 LUTs for the mask pair, and a native `ABS`, `MIN` and `MAX` at 150 to 250 LUTs. How often the words matter is measured on the compiled corpus (amd64, optimized for size): 18 `abs` sites, 5 unsigned, 171 `min` and 52 `max` in 95 files, under 0.1% of code.

**Counting convention.** `CMP`, `MOVQ`, `BIS` and `NEG` between registers are 2 bytes; `SET cc`, `MASKEQZ`, `MASKNEZ`, `BSEL` and any new select are 4 bytes. A native `MIN` is 1 instruction of 4 bytes. These are estimates to be checked against the assembler.

**Options.**

1. *A flag-reading select,* `CSEL cc, ra, rb, rd`*.* One new 4-byte instruction that reads `N`, `Z`, `C` and `V` and picks `ra` or `rb`. `MIN` is `CMP a,b ; CSEL lt,a,b,d` (2 instructions, 6 bytes); `ABS` is `NEG a,t ; CSEL mi,a,t,d` (2 or 3 instructions). One select also serves floating-point values, which live in the general registers. For: the natural select for a machine with flags; the shortest alias of the real options; 15 condition words fit in the 7-bit sub-opcode space of the three-register group (51 of 128 used). Against: after `CMP` the flags describe the comparison, not the result, so `N` and `Z` are wrong and `V` and `C` are the comparison's: the flag rule above breaks unless a `TST` is added (3 instructions, 8 bytes); needs a flag read on the select unit; a second select beside `MASKEQZ` multiplies entities (Ockham, the author's rule: do not multiply entities without benefit); effort in the assembler, simulator, compilers and both manuals; 250 LUTs or less (estimate).
2. `SET` *plus* `MASKEQZ` *and* `MASKNEZ`*, no new instruction.* The aliases expand over what the author already adopted (errata 85). `MIN a,b,d` is `CMP a,b ; SET lt,c ; MASKEQZ a,c,t ; MASKNEZ b,c,d ; BIS t,d` (5 instructions, 16 bytes, two scratch registers). `ABS` needs no select: `ASR a,#63,m ; XOR m,a,d ; SUB m,d` (3 instructions, about 6 bytes, `V` not set at the minimum). For: no new opcode or flag reader; one select for everything; constant time by construction; no gates. Against: 4 more instructions and 12 more bytes per `MIN` or `MAX` site (on the corpus about 890 instructions and 2.7 KB, about 0.3% of instructions, an estimate from amd64 counts); `C` is changed; the assembler needs scratch registers and nothing is reserved except `R12`, which is the linker's, so an alias that silently uses a register is a strange side effect; five hidden instructions behind one word.
3. *A register-conditioned select,* `SELECT rc, ra, rb, rd`*.* `rd = rc != 0 ? ra : rb`, three source registers in one instruction (the four-register 4-byte form exists: `FMADD` has four register fields). `MIN` is `CMP a,b ; SET lt,c ; SELECT c,a,b,d` (3 instructions, 10 bytes);

`MASKEQZ(a,c)` is `SELECT c,a,zero`, so it replaces the two mask instructions or sits beside them. For: a constant-time select in 1 instruction instead of 3; no flag read. Against: it reopens the adoption of `MASKEQZ` (errata 85) or adds a third entity; `MIN` still costs 3 instructions and the flags of the sequence are the comparison's.

4. *Bit select, `BSEL rm, ra, rb, rd`, as the only select.* `BSEL` is decided (9.9.6, 9.20.4); this option is to use it also as the select for the aliases and drop the mask pair's role in them. `rd = (rm & ra) | (~rm & rb)`, with the mask made from the condition. `MIN` is `CMP a,b ; SET lt,c ; NEG c,m ; BSEL m,a,b,d` (4 instructions, about 14 bytes). For: nothing new, since the instruction is adopted (it is the Ch and Maj functions of SHA-2 and the constant-time swap); selects any width and bit pattern. Against: the scalar select is long, with a comparison, a `SET` and a negate before it; a four-register form.
5. *A two-operand conditional move, `CMOVcc rs, rd`.* 2 bytes, destructive, reads flags (the x86 shape). `MIN a,b,d` is `MOVQ a,d ; CMP d,b ; CMOVgt b,d` (3 instructions, 6 bytes, no scratch register). For: the smallest code of any select; the shape compilers know. Against: needs a 2-byte slot, and the last spare slot in the arithmetic group would be spent (a 2-byte form writes flags); a read-modify-write destination makes the instruction depend on the old `rd` even when it moves nothing, which out-of-order machines dislike; the flags after it are the comparison's.
6. *No select: a compare and a short branch over a move.* `MOVQ a,d ; CMP d,b ; Bgt over ; MOVQ b,d`. A core may fuse a forward branch over one instruction into a single micro-operation, so the sequence costs nothing on cores that fuse and is promised nothing on cores that do not. For: nothing new in the architecture; 3 or 4 instructions. Against: it branches on data, so `MIN`, `MAX` and `ABS` cannot stay on the list of always-constant-time instructions, which contradicts the secure core of errata 85 and the author's "no timing channel attacks"; a data-dependent branch mispredicts at a rate that depends on the data; the flags are the comparison's.

**Precedents.**

- ARM64 `CSEL`, `CSINC`, `CSINV`, `CSNEG` select by flags, and Arm's data-independent-timing guarantee lists conditional select among the instructions covered *(verified)*; the architecture's own reserved slot for a flag-based `CSEL` in the secure-core proposal *(verified)*.
- x86 `CMOVcc`, SPARC V9 `MOVcc`, s390x `LOCR`, Alpha `CMOVxx` *(unverified: general knowledge)*.
- LoongArch `maskeqz` and `masknez` and RISC-V Zicond `czero.eqz` and `czero.nez` are exactly the mask pair; RISC-V's base set has no select *(unverified: general knowledge)*. RISC-V Zbb has native `min`, `minu`, `max`, `maxu` and no `abs` *(unverified)*.
- MIPS `MOVN` and `MOVZ`, and Alpha `CMOVxx` (which tests a register against zero), are register-conditioned selects *(unverified)*.
- ARM NEON `BSL`, `BIT`, `BIF` and the x86 ternary-logic instruction `VPTERNLOG` are bit selects *(unverified)*; `BSEL` in the secure-core design *(verified)*.
- The PDP-11 and the VAX have no select and compilers emit a branch; a few high-performance RISC-V cores fuse a short forward branch with the instruction after it *(unverified: general knowledge)*.

**The author's referents.** `cmov`, `csel`, `fsel` (no source machine named): "I am interested in the cmov/csel/fsel ... am happy for abs/min/max to be aliases on top of it in the assembler". On `ABS`, `MIN` and `MAX` as opcodes: "add them. and if we have to teach the front end about abs/min/max nodes that is fine ... surely other cpus have these opcodes", and later "why do them as single opcodes instead of as an opcode that takes a flag?" Ockham's rule: "don't multiply entities without benefit" and "no weird side effects, no bumping other opcodes in a way that makes code size increase". The PDP-11 `ROL` as an alias of `ROR` ("the assembler could take rol and map it to ror"). Floats live in the general registers. "No timing channel attacks."

**The assistant's leaning.** Option 2 for the architecture: `MASKEQZ` and `MASKNEZ` stay the one select (errata 85), `ABS` expands without a select (`ASR`, `XOR`, `SUB`), and the manual says plainly that the five words change `c` and that the assembler needs a reserved temporary register. If `MIN` and `MAX` in hot loops prove to matter, Option 1 can be added later in the same sub-opcode block and the same aliases then shrink from 5 instructions to 2 without anyone rewriting source. This is the assistant's view, not a decision. It would change if a compile showed `MIN` or `MAX` inside hot loops, or if the author wants the alias as short as the native form (then Option 1 now).

**Where the body refers to it.** Sections 9.7.6, 9.20.3 and 9.21.1; and the dictionary entries of `ABS`, `MIN`, `MINU`, `MAX`, `MAXU` and `MASKEQZ`.

### J.9.2 Constant-time execution: what holds while `DIT` is clear, and where `DIT` lives

**Issue.** A timing channel is a leak of secret data through how long an instruction takes, when the time depends on the operand values. "Constant-time" means that the architecture promises that an instruction takes the same time whatever the data (data-independent timing, the property that cryptographic code needs). It is necessary but not sufficient: a secret-dependent branch, address or table index still leaks through the branch predictor and the cache. The author wants "a

constant time flag or something so instructions always take an equal amount of time no matter the input"; on 2026-11-04 he decided that add and divide need not take the same time as each other, "only they are timing independent of their data", and that "for non-sensitive things, constant time isn't a priority". Two things are decided. Section 9.3.7 states a bit DIT in the status word, set by the program, that promises data-independent timing while it is set (the author took the contract of the architecture recommendations, errata 71; a mode bit that is saved with the status word is what he called a freebie, errata 83). And errata 85 decided that at least one secure core has every timing channel removed. Two things are open. **First, what is promised while DIT is clear:** nothing, a published list of instructions that never vary, or every integer instruction. **Second, the position of the bit:** bit 12 of the 16-bit status word is UA, bits 13 to 15 are free and read as zero, and the saved status word of an exception frame carries whichever bit is chosen.

**Why it matters.** The fast paths (an early exit from divide, from the block scan MSCN, from a multiplier) are gates and speed, and a fixed-latency multiplier is smaller than an early-out one. A core must be able to disable each fast path while DIT is set. The compiler must not reorder, replace or turn into branches the instructions that are promised to be constant-time, and the manual must list them; no instruction timing is stated in the instruction set today. Measured on 827 compiled files (370,115 instructions): divide 149 (0.040%), block copy 154, floating-point divide 10, multiply 2,204 (0.595%), MSCN none. A fixed worst-case divide is estimated to slow a whole program by about 1% (assuming 64 cycles worst and 30 average; an estimate, not run time data). The position of the bit fixes the format of the saved status word that the kernel, the debugger and the signal frame share.

**Options** (for what holds while DIT is clear).

1. *Nothing: DIT is the only promise.* Every instruction may have fast paths, and code that needs constant time sets the bit. For: nothing more to specify; fast paths stay for normal code. Against: software forgets to set it, and a default of "insecure" is the one that a 2026 kernel post calls unacceptable; an out-of-order core has to hold the promise through the whole pipeline and not only in the arithmetic unit.
2. *A published core list that is always constant-time, with DIT for the rest.* The architecture lists the instructions that always take data-independent time (add, subtract, logic, shifts and rotates, MUL, MULH, MULHU, CLMUL, POPCNT, CLZ, BSWAP, BITREV, SET, MASKEQZ, MASKNEZ, and the expansions of ABS, MIN and MAX once J.9.1 fixes them); DIV, MOD, floating point, MCMP, MSCN, MSCNN and MVAL need the bit. For: the common case is free, the rare instructions are covered when asked, and software may rely on the list on every core; a fixed multiplier needs fewer gates. Against: two things to specify and test; the list must be kept as instructions are added; any expansion in J.9.1 that branches on data is off it.
3. *Every integer instruction always constant-time, a prohibition list, DIT for floating point and the scanners only.* DIV always takes its worst-case time. For: the simplest to specify and verify; fewest gates in the fast-path logic. Against: an average divide is slower (about 1% over a program, and loops that divide on every iteration weigh many times more); MSCN loses its early exit; it contradicts "not a priority" for non-sensitive code.
4. *The promise on a class of core.* The strong promise lives on the secure class of errata 85 (identified by a class register; DIT reads 1 and is fixed on it; every instruction fixed-latency, caches private, speculation absent). On general cores DIT is a promise of the instructions in the list of option 2, and a core says in a feature register what it cannot hold. For: the strongest guarantee where it is needed, covering the cache and the bus that a mode bit cannot reach; the general cores stay fast. Against: a second core design to build and verify; general cores that run secrets have no promise beyond the list.

**The position.** Bit 13, the next free bit above UA, keeps the used bits together; bit 15 leaves 13 and 14 for later. For bit 13: no gap in the status word, and the save and restore is one mask. Against: a later mode bit that the kernel wants next to UA will not have the place.

**Precedents.**

- Arm's DIT bit in the processor state, which Linux on arm64 enables in the kernel "to be cautious" *(verified)*.
- Intel's DOITM, with the vendor saying it does not recommend enabling it globally and kernel posts (July 2026) saying it is best left on *(verified)*.
- RISC-V scalar crypto extension Zkt: an implementer declares a subset of instructions with latency independent of the data values, a promise and not a mode *(verified)*.
- x86 before DOITM: Intel published a guidance list and later found that AES-NI and ADD and XOR varied on later parts, so the drift is the precedent against a list that is only published *(verified)*.
- The repository's own secure class, with DIT reading 1 and fixed on it, a class register, a reserved bus slot and a private window *(verified)*.

- Small cryptographic microcontrollers with a fixed-cycle multiplier and divider, such as the Cortex-M0+ with its single-cycle multiplier option *(unverified: general knowledge)*.

**The author's referents.** The failures to avoid: "no hammer-row, no spectre, no timing channel attacks; there should be constant time flag or something so instructions always take an equal amount of time no matter the input". On cost: "I wonder how much that would slow a processor down ... for non-sensitive things, constant time isn't a priority". "Mode bits are case by case, but in general should be visible, not hidden state. putting modes in bits that are saved anyway sounds like a freebie." Cryptography on separate devices on the PCI bus, "so they can pay the price of constant time computation".

**The assistant's leaning.** Option 2 with the secure class of option 4 as the strong form, the published list written into the manual now, `DIT` saved with the status word and the compiler treating the list as untouchable; and bit 13. This is the assistant's view, not a decision. It would change if a cycle measurement showed divide in hot loops (option 2 stays, option 3 is dropped), or if hardware cannot cheaply disable a fast path (the instruction goes off the list).

**Where the body refers to it.** Sections 9.3.7 and 9.21.1.

### J.9.3 The atomic group: where the operands go, and the size of `CAS`

**Issue.** The atomic group holds `CAS`, `SWAP`, `XADD`, `XAND`, `XOR_`, `XXOR`, `XMAX`, `XMIN`, `XMAXU`, `XMINU`, `LL` and `SC`, each with a `_DB` variant that adds a barrier (9.13.1). Their semantics, ordering and the reservation monitor are decided (chapter 7). Not decided is the shape of the instruction: each names an address, an operand and a place for the old value, and `CAS` also an expected value. The grammar of the encoding takes three registers for all but `CAS` and `CASDB` (four), with the address in a register, in the order written (errata 87, assumption 4), and the text has not confirmed that. `SC` has to say where its success flag goes. And `CAS` works on 4 or 8 bytes (9.13.2): how a 4-byte and an 8-byte form are told apart is not fixed. The rule that an instruction has at most one memory operand (9.2.2) and the refusal of implicit registers (9.17.3) are decided.

**Why it matters.** The assembler syntax, the compiler's lock and reference-count sequences, the operating system's lock code and every disassembler depend on the order of the operands, and a mistake is a lock that corrupts memory and not a fault.

**Options** (the placement).

1. *Address in a register, three registers, a separate destination; four for* `CAS`. `XADD Ra, Rb, Rd`: the old word at `(Ra)` goes to `Rd`, and `Rb` is added. `CAS` names expected, new, address and the destination of the old value. For: no operand is destroyed; one leaf shape for the group; the assumption of the grammars. Against: `CAS` needs a four-register leaf, which is six bytes.
2. *Three registers throughout; the expected value is also the destination of* `CAS`. `CAS Rd, Rb, (Ra)`: `Rd` holds the expected value and receives the old value. For: three registers for every member of the group; the form of RISC-V's `AMOCAS` and of ARM's `CAS`. Against: the expected value is destroyed, so a retry loop reloads it; one more rule to remember.
3. *Two registers, destructive.* `XADD (Ra), Rd`: `Rd` is the operand and receives the old value. For: the shortest form, two or four bytes; the shape of x86's `XADD`. Against: `CAS` has no room for four values without an implicit register (refused, 9.17.3); the operand is destroyed.
4. *The address as the ordinary memory operand* `rm`. For: reuses the addressing forms of the rest of the instruction set; the same shape as `ADDM`. Against: allows modes with side effects (auto-increment) that an atomic must refuse, and the forms are longer.

**The size of `CAS`** (4 or 8 bytes). (a) *Two words,* `CASL` *and* `CASQ`, spelled like `MOVL` and `MOVQ`: For: the size is in the mnemonic, as for the moves; Against: two sub-opcodes, and the `_DB` variants double again. (b) *A size bit in the sub-opcode or leaf*, one mnemonic with a suffix: For: one entry; Against: the assembler must carry the suffix, which no other instruction of the group does. (c) `CAS` *is always 8 bytes, and a 4-byte compare-and-swap is a* `CAS` *of an aligned pair that includes the neighbour*: For: nothing to encode; Against: it makes a 4-byte lock word a 8-byte critical value, and the span-two-fields case of 9.13.2 would have to be written differently.

**Precedents.** RISC-V `AMO` instructions take `rd, rs2, (rs1)` and `AMOCAS` takes the expected value in `rd` *(unverified: general knowledge)*; ARM64 `LDADD` and `CAS` take the compare value in a register that is overwritten with the old value *(unverified)*; x86 `LOCK XADD` is destructive and `CMPXCHG` has the accumulator as an implicit operand *(unverified)*; the VAX interlocked instructions (`BBSSI`, `ADAWI`, `INSQHI`) use memory operands *(unverified)*.

**The author's referents.** "Only one memory operand per instruction" and no implicit registers as rules of the instruction set (9.2.2, 9.17.3); the author has not spoken on the order of the operands.

**The assistant's leaning.** Option 1, which is what the grammars already assume, with the size as two words (a). This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.13.2 and 9.21.1; the atomic entries of section 9.13.

### J.9.4 The checked arithmetic: the number of the `CHECK` vector and the register form

**Issue.** `ADDV`, `SUBV` and `MULV` abort at the instruction, with `EPC` at it and the destination unwritten, when the signed result does not fit in 64 bits; `TRAPCC` takes the same `CHECK` vector as a trap (errata 85). Two things are not fixed. The vector table has no vector of that name: numbers 16 to 31 are free between `bpt` (15) and the interrupt base (32). And the grammars give the two-register form (`Rd <- Rd op Rs`, like `ADD`), where the reason for the fused forms was one non-destructive instruction in place of `ADD` and `TRAPCC`, which a three-register form (`Rd <- Ra op Rb`) serves.

**Why it matters.** The vector number is the index of the handler in the table that the kernel writes; software that handles an overflow (a language runtime that promotes an integer to a bignum) names it. The register form decides how long the instruction is and whether a copy is needed before it.

**Options** (the vector).

1. *Vector 16, the first free number.* For: keeps the fault group together; matches the number the study used. Against: a later fault of the same group has no number before the interrupt base without a gap rule.
2. *A number at the top of the free range (31).* For: leaves 16 to 30 for faults not yet defined. Against: an arbitrary number that the first reader will ask about.
3. *No new vector: a `TRAP`-range number.* The overflow enters as `TRAP #n` would. For: no hardware vector to define. Against: the hardware has to synthesise a system call for a fault, and a handler cannot tell an overflow from a real `TRAP`.

**Options** (the form).

4. *Two registers, destructive (the grammars).* For: the shape of `ADD`; two parcels. Against: the operand is destroyed, so a check that needs the old value copies it first.
5. *Three registers.* For: non-destructive; the reason the fused forms exist. Against: a longer leaf (three register fields), where `ADD` and `TRAPCC` is four plus two bytes.
6. *Both.* For: the compiler chooses. Against: two leaves for one idea.

**Precedents.** x86 `INTO` traps on the overflow flag and is not in 64-bit mode; MIPS `ADD` and `ADDI` trap on signed overflow while `ADDU` does not; the VAX has an integer-overflow trap enabled by a bit in the status word *(all unverified: general knowledge).*

**The author's referents.** Errata 85: `TRAPCC` is a trap and not an abort, and the fused `ADDV`, `SUBV` and `MULV` carry the register-only cases; errata 84 item 9, "overflow-trapping arithmetic (leaning yes)".

**The assistant's leaning.** Vector 16 and the three-register form, because the fused forms were accepted as the non-destructive case. This is the assistant's view, not a decision.

**Where the body refers to it.** Section 9.21.3; the entries of `OVF` and `TRAPCC` in section 9.7 and 9.14.

### J.9.5 Addressing-mode gaps: the scaled-index displacement and a store through `d(DP)`

**Issue.** The scaled-index mode `d(Rn)(Rx*s)` exists for the array of structures. Its shortest form (two parcels) has a displacement of 6 or 8 bits, so ±32 to ±128 bytes before it takes three parcels and a 22-bit displacement (9.5.1). The mode `d(DP)`, the direct page that serves the thread pointer and the per-CPU pointer, has a load form of six bytes and no store form in the grammars (9.3.5).

**Why it matters.** Code that walks an array of records touches fields beyond 32 bytes and pays two more bytes for each; a thread-local store needs a separate address computation.

**Options.**

1. *Leave both as they are.* For: nothing changes; the three-parcel forms exist. Against: the cost is paid in the hot loop of the array of structures and in every thread-local store.
2. *Reduce the scale to one bit (x1 or x8) and give the bit to the displacement.* For: one more bit of displacement. Against: the scales 2 and 4, which index 16- and 32-bit arrays, are lost; the compiler's best case (`bs_ec`'s inner loop, 31% fewer instructions) uses the scaled index.
3. *Add a two-parcel scaled-index leaf with hot registers for the base and the index.* The register fields shrink, so the displacement grows. For: serves the common case with the registers the allocator prefers. Against: a prefix spent on one leaf; a compiler rule for which registers qualify.

4. *Add a store form through* `d(DP)`*, six bytes, one word for each size.* For: symmetrical with the load; one instruction for a thread-local store. Against: spends prefix space on a form that the compiler may use rarely.
5. *Add two-parcel* `d(DP)` *forms with a 12-bit displacement for loads and stores.* For: most thread-local and per-CPU blocks are small; four bytes instead of six. Against: the 28-bit load form still stands beside it.

**Precedents.** x86 scaled index with a 32-bit displacement; AArch64 scaled register offset with no displacement and an unsigned scaled 12-bit offset; the VAX index mode that follows any other mode *(all unverified: general knowledge)*. The 6502's zero page is the ancestor of `DP` *(stated in chapter 9)*.

**The author's referents.** The PDP-11 as the base ("update and refresh of the pdp-11 instruction set and architecture"); Ockham's rule: "don't multiply entities without benefit".

**The assistant's leaning.** Option 1 until a compiled corpus shows how many array accesses lie beyond 32 bytes. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.5.3 and 9.21.1.

### J.9.6 The page-bit mechanisms and the six reserved system words

**Issue.** The author approved three mechanisms "to be specified" (errata 84): a page bit that gates landing pads, so that an indirect transfer that does not land on `LPAD` faults, with the fault attributed to the branch (item 5); a page bit that says a system call may be made from this page (item 8); and a stack-limit register checked when a system call or a fault is taken (item 4). The grammar of the encoding reserves six system words, `SYSPG0` to `SYSPG5`, for these mechanisms, and the retire-stage study found that they need no opcode. What the words do, or whether they are released, is not said; the page-table entry has no bit for either page bit (the entry is full below bit 40 and the format for wide physical addresses is open, J.6.2).

**Why it matters.** The bits and registers must be in the page-table entry and the system-register list before the kernel and the linker are written, and software must be told not to use the six words until they mean something.

**Options.**

1. *No opcodes: enable bits in a system register and enforcement bits in the page-table entry.* The words stay reserved and abort as illegal. For: nothing to execute; matches the retire-stage study. Against: needs two page-table bits, which compete with the wide-physical-address format.
2. *One system word for each mechanism.* `SYSPG0` and the others set or clear the mechanism for the current address space. For: explicit, and a privileged instruction is easy to trap. Against: a mode that an instruction changes (the rule of 9.2.5 is that modes are visible state).
3. *A page-attribute instruction that sets bits of the entry for an address.* For: the kernel need not walk the table to change an attribute. Against: an instruction with a translation-buffer side effect, and the format of the entry must have room.
4. *Release the six words.* For: the opcode space is freed. Against: the mechanisms approved by the author are still to be specified.

**Precedents.** Intel's indirect-branch tracking is enabled by a bit in a model-specific register and ends a branch on a missing `ENDBR`; ARM's branch-target identification is a page attribute in the translation-table entry *(both unverified: general knowledge)*; OpenBSD's `msyscall` and `pinsyscalls` check where a system call comes from in software, and OpenBSD enforces landing pads by default where the hardware has them (appendix A, from the reports).

**The author's referents.** "Do it, no delay" for the colours of memory; landing pads gated by a page bit, "with the fault attributed to the branch"; "minimize recovery code".

**The assistant's leaning.** Option 1, since the study found no opcode is needed, and the words released only when the mechanisms are specified. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.21.1, A.1.2, A.2.2 and A.3.2. The schedule of landing pads in the ABI is J.D.2.

### J.9.7 The worst-case latency of the block operations

**Issue.** An ordinary instruction is bounded: at most six bytes of fetch across two pages and one memory operand across two pages. A block operation (`MCPY`, `MSET`, `MCMP`, `MSCN` and the rest, 9.11.3 is not bounded by its definition; it keeps its progress in registers and may be interrupted between steps, and an implementation may move more than a byte per step provided the registers are consistent where an exception can be taken (9.11.5). So an interrupt-latency figure depends on the step.

**Why it matters.** A claim of real-time response rests on a bound. Until the step is stated, the claim is an argument and not a number.

**Options.**

1. *No architectural bound: each implementation states its granularity.* For: freedom to move a line or a page at a time. Against: software cannot rely on a latency.
2. *An architectural maximum between interrupt checks (one cache line, 64 bytes).* For: a bound that holds on every core. Against: ties a fast engine to a small step.
3. *An architectural maximum of one page between checks (4 KiB).* For: the largest unit with a single translation; bulk engines work at full speed. Against: tens of microseconds of latency on a slow memory.
4. *A feature register that reports the granularity.* For: software reads the figure. Against: it is information and not a promise.

**Precedents.** x86 `REP MOVS` is interruptible between iterations and its progress is in registers; ARM's `FEAT_MOPS` leaves the progress in registers; the VAX `MOVC` kept a first-part-done bit in the status word *(all unverified: general knowledge)*.

**The author's referents.** The VAX `MOVC5` as the argument against hidden state (9.11); the registers as the only progress.

**The assistant's leaning.** Options 1 and 4 together: no bound in the architecture, the granularity reported in a register. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.11.5 and 9.21.1.

### J.9.8 The block operations and the data cache

**Issue.** A one-megabyte `MCPY` evicts the whole data cache for data that is never read again; it shows at once on a cache of 8 KiB. Every real `memcpy` on a machine with non-temporal stores uses them for this reason. An inference workload that moves tens of megabytes through a cache of a few kilobytes is the extreme case.

**Why it matters.** The kernel's working set is what a large copy destroys; and the fix is an opcode, a memory type or a rule inside the core, and only the first two are visible to software.

**Options.**

1. *Do nothing: leave the cache alone.* For: no new opcode or memory-type value. Against: the eviction is real.
2. *A non-temporal variant of the block operations.* For: the program says that the data will not be read again. Against: a new sub-opcode and a rule that the data may not be cached.
3. *A streaming value of the memory-type field of the page-table entry.* For: no opcode; a buffer is declared streaming once, and every access to it, by the block operations or not, follows. Against: the kernel must set it per mapping; a wrong type slows ordinary access.
4. *The implementation decides by length.* For: invisible to software. Against: a performance cliff with no architectural visibility, the worst of the options.

**Precedents.** x86 non-temporal stores (`MOVNT*`) and fast-string moves that avoid the cache above a threshold; ARM non-temporal load and store pairs *(all unverified: general knowledge)*.

**The author's referents.** The memory type as a page attribute (adopted from LoongArch, 9.20.2).

**The assistant's leaning.** Option 3, because the streaming workload is known by buffer, not by instruction. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.11.5, 9.21.1 and A.9.

### J.9.9 The floating-point condition flags

**Issue.** `FCMP` writes the integer flags: `N` if the first operand is less than the second, `Z` if they are equal, `V` if they are unordered (either is a NaN), `C` cleared. The signed branches do not treat the unordered case as false: `BLT` after an unordered compare takes `N xor V`, which is 1. The compiler must therefore avoid `BLT` and `BLE` for floating point. A reading of the flag rules by the assistant, not checked against the compiler: `BGT` (`Z` clear and `N` equal to `V`) and `BGE` (`N` equal to `V`) are false for an unordered pair, so `a < b` can be tested as `b > a` with the operands swapped, `a <= b` as `b >= a`, `a == b` with `BEQ` and `a != b` with `BNE`; the swap needs the second operand in a register. A second problem is that `FCMP` writes the one set of integer flags, so two independent floating-point comparisons depend on the same resource and floating-point compares serialise against integer work.

**Why it matters.** The compiler's rules for `<` and `<=` on floating point; the number of flag copies an out-of-order core must rename; the status word that is saved and restored.

**Options.**

1. *Keep the integer flags and the swap rule.* For: no change; the existing branches suffice. Against: the swap needs a register for the second operand; the shared flags stay a dependence.

2. *Two floating-point condition bits in the status word.* For: separates floating-point compares from integer flags. Against: two more bits to save and restore, and a second set of branches or a condition mapping.
3. *Eight one-bit condition registers (LoongArch `fcc`).* For: many independent comparisons. Against: new state to save at a context switch.
4. *A compare that writes 0 or 1 into a general register (RISC-V `feq`, `flt`, `fle`).* For: no flags at all; the result feeds the one-parcel compare-and-branch against zero; fits floating point in the general registers. Against: a compare and a branch are two instructions, as `FCMP` and a branch are now.
5. *A fused floating-point compare-and-branch.* For: one instruction; no flags. Against: a leaf with two registers and a displacement, six bytes in the common case.

**Precedents.** LoongArch has eight `FCC` registers *(stated in the instruction-set text; unverified here)*; RISC-V's floating-point comparisons write an integer register *(unverified: general knowledge)*; ARM64 sets `NZCV` from `FCMP` so that unordered makes the ordered conditions false *(unverified: general knowledge)*.

**The author's referents.** "Floating point is done in the general registers" (9.2.7); the rule that no instruction writes part of `{N, Z, V}` (9.3.4).

**The assistant's leaning.** Option 1 now, as it costs nothing, with option 4 the one to take if the flag dependence is measured to matter. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.12.6, 9.20.3 and 9.21.1; the entry of `FCMP`.

### J.9.10 Directed rounding and the remaining half-precision operations

**Issue.** *Directed rounding.* IEEE 754 requires rounding toward positive infinity, toward negative infinity and toward zero, as well as to nearest, for add, subtract, multiply, divide, square root, fused multiply-add and the conversions. The manual decides that there is no rounding-mode register or bit and that the rounding is in the opcode wherever it is not nearest-even (9.12.5); but no arithmetic opcode takes a rounding mode, only the `FRINT` family does. A conforming C library keeps the dynamic mode of `fesetround` in thread-local storage and dispatches on it. *Half precision.* `binary16` has seventeen operations and conversions; the maximum and minimum, the roundings, negation and absolute value of half, conversions to and from 64-bit integers, and `bfloat16` are not defined. The costs are unmeasured (conversions 500 to 1,000 lookup tables, full arithmetic 2,000 to 3,500).

**Why it matters.** Interval arithmetic and the C99 `<fenv.h>` conformance depend on directed rounding; machine-learning code on `bfloat16`.

**Options** (directed rounding).

1. *None: software emulates the modes.* For: nothing to add; the library's dispatch is the present answer. Against: it is slow, and the letter of the C standard is not met by hardware.
2. *A two-bit static rounding field in the instruction.* The `FESC` two-register forms have a free nibble; the four top-level operations have none. For: the mode is in the opcode as 9.12.5 says. Against: only the escape forms can carry it.
3. *Extra opcodes for the six core operations* (add, subtract, multiply, divide, square root, fused multiply-add) *in each of the three directions.* For: no field. Against: eighteen opcodes.

**Options** (half precision).

4. *Nothing more: the existing conversions and arithmetic.* For: no cost. Against: the library does the rest through `binary32`.
5. *The missing half operations except `bfloat16`.* For: a complete `binary16` set. Against: tables for a format little used outside machine learning.
6. *Add `bfloat16` conversions, or the arithmetic as well.* For: the machine-learning format. Against: another format, and the accelerator of chapter 8 does the volume work.

**Precedents.** RISC-V's floating-point instructions carry a static rounding-mode field with a dynamic value; ARM has a rounding-mode field in `FPCR`; x86 `AVX-512` has embedded rounding control; ARM and x86 have `bfloat16` conversions and dot products *(all unverified: general knowledge)*.

**The author's referents.** "No rounding-mode or precision mode bits" (9.12.5), the rule that a mode bit is by default visible state set by an instruction that names it (9.2.5).

**The assistant's leaning.** Option 1 for directed rounding until a libm port asks, and option 4 for half precision. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.12.6 and 9.21.1; the entry of the half-precision conversions.

### J.9.11 The coherence protocol beyond four cores, and the verification of the memory model

**Issue.** The memory model is sequentially consistent (9.2.8, decided), and the machine has at least four cores with sixteen or more as the preferred ceiling that no decision may preclude (errata 92). The protocol chosen for two to four cores is snooping (invalidation by watching a shared bus, the simplest protocol); at eight or more cores a snooping bus is the wrong protocol, and the one for sixteen is not chosen. Separately, a declaration of sequential consistency is not an implementation: it is easy to violate by a store buffer that forwards wrongly or a cache that reorders fills, and the litmus tests of the published suites must be run before a second core exists.

**Why it matters.** The cache and its protocol must be chosen before the cache is built, not retrofitted; the litmus testing is a verification task that belongs in the plan before silicon.

**Options** (the protocol).

1. *Snooping up to four cores, and the ceiling is four.* For: the smallest design. Against: contradicts the sixteen of errata 92.
2. *Snoop filters or a directory per cache slice.* For: scales to sixteen and beyond; the usual answer. Against: a large structure and a protocol with many states.
3. *Clusters of four that snoop, joined by a directory.* For: small clusters stay simple. Against: two protocols to verify.
4. *Software coherence between clusters.* For: cheap in hardware. Against: fragile, and it breaks the sequentially consistent model across clusters.

**Precedents.** x86 snoop filters and directories across sockets, ARM coherent interconnects *(unverified: general knowledge)*.

**The author's referents.** "yeah let's have 4 cores minimum, 16 or more preferred" (errata 92).

**The assistant's leaning.** Option 3 or option 2, decided before the cache is designed; the litmus tests run on the model first. This is the assistant's view, not a decision.

**Where the body refers to it.** Section 9.21.1.

### J.9.12 The poison bit, the user-qualified moves and the cryptographic words: what is not yet specified

**Issue.** The author decided these instructions and left their details: the deferred-fault and user-qualified loads and stores (`.d` and `.u`: the three leanings of item 10 and the five of item 18 of `docs/study-proposals.md`, accepted in errata 86), the AES and SHA words of the secure core (errata 85: "or whichever opcodes would speed up and enable aes and sha"; the design approved in errata 86), and `bsel`. What is not written down: where the poison of a deferred load lives and how it travels (the author leans to a poison bit per register, errata 84 item 10, and the study does not say how it is saved and restored, or how it moves through a store); the full text of the user-qualified moves (the operand placement of the atomics that take the `.u` form, errata 86, waits on that of `atom`, J.9.3); whether `stua` and `clua` remain once the port confirms the qualified moves (the author kept them until then); whether the result of `sha256sig0` and its kin is zero-extended, as the crypto study has it, or sign-extended like every other 32-bit result of the machine; and the check of the AES and SHA definitions against FIPS 197 and FIPS 180-4.

**Why it matters.** The kernel's copy routines, the dispatcher's test of argument registers for poison, the compiler's hoisting of loads and the crypto library each depend on the exact semantics; an instruction in the grammar whose behaviour differs between the manual, the assembler and the hardware is a security bug of the kind the priority list puts first.

**Options** (the result of the 32-bit SHA words).

1. *Zero-extended (the study).* For: the value is a hash word, not a signed number, and a following 64-bit add of two such words does not need a mask. Against: it breaks the rule that every 32-bit result is sign-extended, so a compiler special-cases the words.
2. *Sign-extended, like `rori32`.* For: one rule for every 32-bit result. Against: a 32-bit hash word that is later used as a 64-bit unsigned value needs `movzl` first.

**Options** (the poison of a deferred load).

3. *A poison bit beside each register, saved with it.* For: precise and deterministic, as the author asked of the taint item ("no false positives"); the register file is small. Against: a bit for each register in the context switch and the exception frame; a store of a poisoned register must fault or carry it.
4. *A reserved value in the register.* For: no new state. Against: the value cannot be a data value any more; a data-dependent condition that the author refused (a strange side effect).

**Precedents.** RISC-V's scalar cryptography extension (`aes64es`, `sha256sig0`, the zero-extended 32-bit result) and the Itanium deferred-exception token (`NaT`) with `ld.s`, which is exactly a poison bit per register *(unverified: general knowledge)*; Arm's `LDTR` and `STTR` for the user-qualified access *(unverified)*.

**The author's referents.** "Constant-time AES and SHA instructions" for the secure core (errata 85); "i like your design. approved." and, of the select instructions, "looks good" (errata 86); of item 10, the three leanings accepted; of item 18, the five leanings accepted; of the secure core, "no timing channel attacks".

**The assistant's leaning.** Option 1, because the study chose it and the words feed 32-bit additions in any case, and option 3, because the author already leans to it (errata 84, item 10). This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.6.7, 9.9.7 and 9.21.3; the dictionary entries of `MOVD`, `MOVUSER`, `AESR`, `AESK` and `SHA256`.

## J.10 Chapter 10: Conclusion: from FPGA to silicon

### J.10.1 Which rules keep a faster core from the speculative and timing flaws

**Issue.** The author decided (errata 85; 2026-10-22: "no hammer-row, no spectre, no timing channel attacks") that the architecture does not repeat the flaws of the last decade, and that a secure core has every timing channel removed (10.4.1, 10.4.3). What is not yet chosen is the exact rule set for a future out-of-order core, and the mechanism that makes execution time independent of data.

**Why it matters.** A rule fixed while the specification is being written costs little; one retrofitted after silicon costs a respin.

**Options.**

1. *Rules in the report only.* The rules (for example: no speculation across protection domains; no predictor state shared across a privilege boundary without a flush) stay advice in the hardware-vulnerabilities report. For: nothing to implement now. Against: nothing makes an implementer follow them.
2. *Rules in the architecture text.* They are stated as requirements of the specification, with a declared set of instructions whose timing is independent of operand values. For: testable. Against: the declared set and a data-independent-timing control (a mode bit or a core class) are not chosen (proposal P17).

**Where the body refers to it.** Section 10.4.1.

## J.A Appendix A: Improvements beyond the PDP-11

### J.A.1 The canonical-address rule and the pointer bits that colours spend

**Issue.** A virtual address is canonical when its bits above the translated width repeat the top translated bit (39, 48 or 57 bits of address, 9.2.10); a non-canonical address aborts. A 64-bit pointer therefore has 25 spare bits with three levels of page table, fewer with four or five, and the rule today refuses software any use of them. The author approved granule colouring and reserved colours for uninitialised and freed memory (errata 84, items 1 and 2), and "to be specified": a colour travels in the pointer, so the hardware, not software, must define and check those bits.

**Why it matters.** The rule is stated in the architecture, in the kernel's address-space layout, in the compiler's pointer arithmetic and in the debugger; a colour that software may set changes all four. The tag trick of the instruction set (a pointer tag in the low three bits, checked by the quad alignment abort) is a separate use and is not affected.

**Options.**

1. *Keep the canonical rule; colours live outside the pointer.* The colour of a granule is a table in memory, indexed by address, and a check compares the table entry against a colour register. For: the rule stands; no pointer bits spent. Against: a table lookup on every access that is checked; more memory traffic.
2. *The colour travels in a fixed field of the spare bits, ignored by translation and checked by the hardware.* For: no table; the check is a compare. Against: every pointer use must mask (the hardware, as on ARM, can ignore the field in the address path); the canonical rule is restated; the field shrinks with each level of page table added.
3. *The colour field depends on the translation width.* Three levels leave the 25-bit field, five levels fewer, and the width is read from a register (J.6.1). For: uses what each configuration has. Against: software that depends on the field must read the width first.
4. *Choose the field with the page-table level, once: the smallest field that every configuration has.* For: one rule. Against: wastes the extra bits of the narrow configurations.

**Precedents.** ARM's top-byte-ignore, pointer authentication and the memory tagging extension *(unverified: general knowledge)*; SPARC's application data integrity colours the pointer and the memory *(unverified)*; the study of item 3 of the retire-stage candidates, which considered the same bits for a capability bit *(verified: docs/retire-stage-candidates.md, cited in errata 84)*.

**The author's referents.** "Do it, no delay" for granule colouring; the capability bit "asked whether it can be done with 64-bit pointers and flags outside the 64 usable bits" (errata 84, item 3).

**The assistant's leaning.** Option 3, because the width is read anyway for the address space (J.6.1). This is the assistant's view, not a decision.

**Where the body refers to it.** Section A.5.2.

## J.D Appendix D: The ABI, version 1

### J.D.1 Stack alignment

**Issue.** The stack pointer `R14` must be kept at some multiple, so that pushes and locals land at addresses a quad (8-byte) access can use. Version 1 of the ABI says the stack pointer is a multiple of 8 at all times (D.1). The author asked: "why not 2 bytes instead of 8?" Two is the right number only when the machine pushes 16-bit words, as the PDP-11 does. Here a push, a return address and an argument slot are each 8 bytes (a call pushes the return address), and a quad access to an address that is not a multiple of 8 aborts with `ALIGN` (section 7.4 of the instruction set: natural alignment is required and "may not be relaxed"); a stack pointer at 2 modulo 8 would make the first push abort. Two would work only if quads were pushed with the explicit unaligned forms `MOVUQ`, which cost two memory accesses. The open question is which number the ABI states, and whether any function may ask for more.

**Why it matters.** In hardware, the 8-byte rule is checked by the existing `ALIGN` abort and costs no gates; a stricter rule (16) could be checked when `R14` is written or left to the first access. In the compiler, frame layout, `alloca`, variable-length arrays and spill slots must keep the alignment, and the compiler either assumes it or realigns in a prologue for types that need more. In the kernel, the exception and signal frame start on a stack of the same alignment, a context switch saves `R14` unchanged, and `setjmp` and `longjmp` restore `R14` whole and so preserve whatever alignment it has. For C, `max_align_t` is 8 here (`long double` is `double` and no type is wider than a quad), and `malloc` alignment follows it; a larger stack alignment is wanted only for a type with larger alignment (a 128-bit type or a vector, none planned). The variadic convention (D.2) pops the return address, pushes `r5` to `r0` and pushes the return address again, so the six-slot register save area (48 bytes) lies directly below the stack arguments and `va_list` is a pointer that runs on into them; a rule of 16 would break that contiguity.

**Options.**

1. *8 at all times, checked by the quad `ALIGN` abort (the present ABI).* For: the smallest number that every push and spill obeys; no padding beyond rounding frames to 8; a violation is caught by the first quad access. Against: a future 128-bit type or vector needs a declared realignment (option 6) or an ABI version change.
2. *2 (what the author asked).* The stack pointer is a multiple of 2, so a byte-sized local is not rounded to 8. For: saves on average 4 bytes per frame on a machine with little memory (the PDP-11's reason); matches the 2-byte parcel of instructions. Against: pushes of quads would land at misaligned addresses and abort, so each push becomes two accesses (`MOVUQ`) or the compiler rounds every frame anyway and nothing is saved; `PUSHM`, `CALL` and exception entry would be unaligned stores; the pointer-tag trick of the instruction set depends on quad alignment; any frame with a quad local needs rounding.
3. *4 (long alignment).* For: a halfway number that matches `int`; no padding for a frame of 32-bit data only. Against: the same problem as option 2 for every quad push; the save area stays 8-aligned only by luck; any quad local needs 8, so there is no gain.
4. *16 at all times.* The usual hardware-friendly number. For: room for a 128-bit pair (`R0:R1`) and future vectors without an ABI change; the same as the three most used ABIs, so ported code and compilers assume it. Against: on average 8 bytes of padding per frame and about one extra instruction in some prologues; the variadic save area plus the return address needs 8 padding bytes to keep `va_list` contiguous, so D.2 would change; every exception frame must round; a stronger constraint than the one the author called horrible.
5. *8 at all times, 16 at calls only.* The caller guarantees 16 when it executes the call; the callee may move the stack pointer by pushes. For: only functions that use a 16-aligned object pay; leaf frames keep 8-byte steps. Against: a callee cannot

know the alignment at an arbitrary point unless it tracks the parity of its own pushes; interrupt and signal handlers have no guarantee; hand-written assembly is harder; it adds a rule hidden in the parity of the stack pointer.

6. *8 is the ABI, and a function that needs more declares it.* Its prologue realigns the stack pointer and keeps the old one in the frame pointer. For: pay only where needed; no ABI version change when a wider type arrives; `longjmp` is still correct because it restores `R14` whole. Against: makes `R13`, optional today, mandatory in those functions; breaks `alloca` and unwinding unless the unwinder knows; a compiler feature used only once something wider exists.

**Precedents.**

- The ABI of version 1 itself: 8 at all times *(verified)*.
- The PDP-11 kept the stack pointer even (a word) and trapped on an odd one *(unverified: general knowledge)*; the VAX stack pointer is longword-aligned by convention *(unverified)*.
- 32-bit ARM before its EABI and early i386 ABIs used 4 *(unverified)*.
- x86-64 System V: 16 at the call instruction, 8 after the return address is pushed; AArch64 AAPCS64: 16, enforced by hardware at stack-pointer-based accesses if enabled; RISC-V psABI: 16 *(all unverified: general knowledge)*.
- gcc `-mstackrealign` and `__attribute__((force_align_arg_pointer))`, with the `AND SP,#-16` prologue *(unverified)*.

**The author's referents.** The PDP-11 as the base ("update and refresh of the pdp-11 instruction set and architecture", "upgrade it to 32/64 bit keeping nice properties"). On alignment: "only 2 byte memory alignment; don't put 8 byte memory alignment constraints that would be horrible", and later "align 64 bit values the natural way; as long as we have circuitry to access individual bytes i'm ok with loading in 64 bits". "Stack alignment why not 2 bytes instead of 8?" The VAX-11 register roles ("SP is R14 and FP is R13 to match it"). Ockham: "don't multiply entities without benefit".

**The assistant's leaning.** Option 1, keeping 8, and answering the author's question in the ABI text: 2 would be right only if a push were a 16-bit word, and every push here is a quad while a misaligned quad access aborts. Option 6 is reserved for the first type that needs more. This is the assistant's view, not a decision. It would change if a 128-bit type, a vector or a 128-bit `long double` enters the language ABI, or if unmodified third-party code that assumes 16 is to run.

**Where the body refers to it.** Sections D.1 and D.4.2.

## J.D.2 Binding, landing pads and the PLT stub after version 1

**Issue.** Three matters of the ABI are version 1 decisions with a question behind them. *Landing pads:* `LPAD` marks a legal target of an indirect jump or call; version 1 leaves enforcement off and the assembler provides `.lpad`; the author approved a page bit that enforces them (errata 84, item 5). When enforcement is mandatory, and whether it is optional first, is not decided: a tagged function pointer reached by `JMP -k(Rn)` must still land on an `LPAD`, and `JTAG`, which jumps through a table, would need a pad at each entry. *Binding:* imports are called through `CALLI got_slot(PC)` with no PLT stub (decided, 9.16.2). Lazy binding (resolving a symbol on its first call) needs something that runs on the first call and knows which symbol it is; eager binding resolves every import when the program loads. Which one is open. *The PLT stub beyond version 1:* whether imports are always called without a stub.

**Why it matters.** Every binary built before landing pads become mandatory lacks the pads, so the later the decision the more binaries break; a linker, a loader and a compiler must agree on the binding rule; the page-bit mechanism of J.9.6 enforces whatever is chosen.

**Options** (landing pads).

1. *Mandatory from version 1.* For: no binary ever lacks the pads. Against: two bytes at every address-taken function and indirect call target, now, before the hardware enforces anything.
2. *Optional in version 1, mandatory in version 2, enforced per page by the page bit.* For: the ABI version number exists for this; a binary opts in. Against: binaries built in the meantime must be rebuilt.
3. *Optional for ever, enforced only for the pages that carry the bit.* For: no flag day. Against: a binary that does not opt in is unprotected, and the attack surface is the part that does not.

**Options** (binding).

4. *Eager binding only.* All imports resolved at load, the table made read-only afterwards. For: no resolver in the program; the table is read-only after load. Against: the load time grows with the number of imports.
5. *Lazy binding by a stub per import.* The table entry points at a stub that identifies the symbol and calls the resolver. For: fast start. Against: reintroduces the stub that the one-instruction call avoids, and a writable table.

6. *Lazy binding through a resolver that finds the symbol from the return address.* For: no per-import stub. Against: the resolver must decode the call site; the table stays writable.

**Precedents.** Intel's indirect-branch tracking and ARM's branch target identification are enforced by OpenBSD where the hardware has them (appendix A); `ld.so` with `LD_BIND_NOW` and `-z now` against lazy binding, and RELRO, which makes the table read-only after load *(unverified: general knowledge)*; the i386 position-independent executable that kept a register for the table base (9.16 commentary).

**The author's referents.** Landing pads gated by a page bit, "with the fault attributed to the branch" (errata 84); "minimize recovery code".

**The assistant's leaning.** Option 2 for landing pads, and eager binding with a read-only table (option 4), because the call is then one instruction with no stub and there is no writable table to attack. This is the assistant's view, not a decision.

**Where the body refers to it.** Sections 9.16.6, 9.21.1, A.1.2, D.1 and D.4.2.